深入理解 LTE-A

基于 3GPP RELEASE-10 协议



温金辉 著

Table of Contents

第 1 章 LTE 概述 10

1.1 空口传输概述 10

1.1.1 下行传输简介 10

1.1.2 上行传输简介 11

1.2 LTE 协议栈结构 13

1.2.1 用户面 14

1.2.2 控制面 21

1.3 LTE 下行峰值速率计算 23

第 2 章 LTE 时域、频域和空间域资源 25

2.1 时域 25

2.2 频域 32

2.3 空间域 36

第 3 章 下行参考信号 37

3.1 小区特定的参考信号（Cell-specific Reference Signal） 37

3.2 UE 特定的参考信号（UE-specific Reference Signal） 43

3.2.1 Rel-8 中的 UE 特定的参考信号 43

3.2.2 Rel-9/ Rel-10 中的 UE 特定的参考信号 46

3.3 CSI 参考信号（CSI-RS） 52

3.3.1 空间域 54

3.3.2 时域 55

3.3.3 频域 56

3.3.4 Muted CSI-RS 60

3.4 MBSFN 参考信号（MBSFN-RS） 64

3.5 定位参考信号（Positioning Reference Signal） 64

第 4 章 下行 L1/L2 控制信道 66

4.1 REG 66

4.2 PCFICH 69

4.2.1 PCFICH 介绍 69
4.2.2 PCFICH 物理层处理 70
4.2.3 载波聚合对 PCFICH 的影响 72
4.2.4 如何选择 CFI 以及 PCFICH 对 PCI 规划的影响 73
4.3 PHICH 75
4.3.1 PHICH 资源介绍 75
4.3.2 PHICH 物理层处理 79
4.3.3 UE 如何确定其使用的 PHICH 资源 80
4.3.4 载波聚合对 PHICH 的影响 82
4.4 PDCCH 83
4.4.1 搜索空间 84
4.4.2 下行子帧可用的 CCE 数 86
4.4.3 PDCCH 盲检 88
4.4.4 载波聚合对 PDCCH 的影响 91
4.4.5 DCI format 92
第 5 章 PDSCH 108
5.1 下行资源分配类型 108
5.1.1 RBG 介绍 109
5.1.2 资源分配类型 0（Resource allocation type 0） 110
5.1.3 资源分配类型 1（Resource allocation type 1） 111
5.1.4 资源分配类型 2（Resource allocation type 2） 116
5.2 VRB 到 PRB 的映射 120
5.2.1 集中式 VRB 121
5.2.2 分布式 VRB 122
5.3 如何确定调制阶数和 TB size 129

5.3.1 **确定调制阶数** 129
5.3.2 **确定 TB size** 130
第 6 章 传输模式（Transmission Mode） 136
6.1 TM 模式简介 136
6.2 TB/codeword/layer/precoding/port 140
6.2.1 TB 141
6.2.2 codeword 142
6.2.3 layer 142
6.2.4 precoding 144
6.2.5 antenna port 144
6.3 基于码本的预编码和基于非码本的预编码 145
6.3.1 **基于码本的预编码** 145
6.3.2 **基于非码本的预编码** 147
6.4 SU-MIMO 和 MU-MIMO 149
6.5 TM 模式 151
6.5.1 TM 1 151
6.5.2 TM 2 152
6.5.3 TM 3 156
6.5.4 TM 4 160
6.5.5 TM 5 163
6.5.6 TM 6 167
6.5.7 TM 7 173
6.5.8 TM 8 175
6.5.9 TM 9 182
第 7 章 上行参考信号 192
7.1 DMRS 192
7.1.1 **上行参考信号序列** 192
7.1.2 DMRS：Demodulation Reference Signal 198
7.2 SRS 206

7.2.1 SRS 子帧配置 208
7.2.2 周期性 SRS 210
7.2.3 非周期性 SRS 213
7.2.4 SRS 频域资源 215
7.2.5 SRS 跳频 222
7.2.6 Cyclic Shift 223
7.2.7 SRS 的冲突处理 223
第 8 章 PUCCH 226
8.1 PUCCH 简介 226
8.2 PUCCH 中的码分复用 229
8.3 PUCCH format 1/1a/1b 232
8.4 PUCCH format 2/2a/2b 240
8.5 PUCCH format 1/1a/1b 与 PUCCH format 2/2a/2b 混合的 PUCCH RB 247
8.6 PUCCH format 3 250
8.7 SORTD 258
8.8 PUCCH 资源分配 260
8.8.1 PUCCH 2 资源分配 262
8.8.2 混合 PUCCH 资源分配 264
8.8.3 PUCCH 1 资源分配 264
8.8.4 PUCCH 3 资源分配 266
8.8.5 PUCCH 资源特殊配置 267
第 9 章 PUSCH 271
9.1 上行资源分配类型 271
9.1.1 上行资源分配类型 0（uplink resource allocation type 0） 272
9.1.2 上行资源分配类型 1（uplink resource allocation type 1） 272
9.2 上行跳频 275
9.2.1 PUSCH 跳频类型 1 280

9.2.2 PUSCH 跳频类型 2 283
9.3 如何确定调制阶数、冗余版本和 TB size 288
9.3.1 确定调制阶数 288
9.3.2 确定冗余版本 RV 290
9.3.3 确定 TB size 290
9.4 上行 TM 模式 293
第 10 章 CSI 297
10.1 RI 299
10.2 PMI 299
10.2.1 UE 上报 PMI 300
10.2.2 eNodeB 选择预编码矩阵 304
10.3 CQI 305
10.3.1 CSI 参考资源（CSI reference resource） 306
10.3.2 CSI 子帧集合 308
10.4 非周期性 CSI 上报（Aperiodic CSI Reporting） 311
10.4.1 触发方式 312
10.4.2 上报模式 316
10.4.3 上行控制信息映射到 PUSCH 的方式 326
10.5 周期性 CSI 上报（Periodic CSI Reporting） 329
10.5.1 PUCCH reporting type 331
10.5.2 如何配置周期 341
10.5.3 PUCCH 2 的容量（capacity）计算 347
10.5.4 周期性 CSI 上报的几种模式 349
10.5.5 “8 天线端口”的 TM9 的周期性 CSI 上报 359
10.6 小结 361
第 11 章 HARQ 介绍 363

11.1 带软合并的 HARQ（HARQ with soft combining） 363
11.2 HARQ process 367
11.3 HARQ 相关概念 368
11.4 同步/异步、自适应/非自适应 371
11.5 BLER 372
第 12 章 上行 HARQ 374
12.1 上行 HARQ 介绍 374
12.2 自适应/非自适应、同步 375
12.2.1 判断新传/重传 376
12.2.2 TB 使能/去使能以及与 codeword 的对应关系 378
12.2.3 上行 HARQ 自适应/非自适应/延迟重传处理 378
12.2.4 上行 HARQ 同步处理 381
第 13 章 下行 HARQ 391
13.1 下行 HARQ 介绍 392
13.1.1 下行 DCI 中与 HARQ 相关的字段 393
13.1.2 TB 使能/去使能以及与 codeword 的对应关系 395
13.2 自适应、异步 396
13.2.1 判断新传/重传 396
13.2.2 下行 HARQ 的 timing 关系 398
13.3 DAI 400
13.4 HARQ bundling 和 HARQ multiplexing 406
13.4.1 HARQ bundling 406
13.4.2 HARQ multiplexing 415
13.5 载波聚合下的下行 HARQ 423
13.5.1 PUCCH format 1b with channel selection 424
13.5.2 PUCCH format 3 434
13.6 下行 HARQ 的冲突处理 444
13.6.1 ACK/NACK 与 SR 的冲突处理 445
13.6.2 ACK/NACK 与 CSI 的冲突处理 447
13.7 HARQ-ACK Repetition 449

13.8 广播 HARQ 452
第 14 章 SR 454
第 15 章 BSR 459
15.1 BSR MAC Control Element 460
15.2 BSR 的触发方式 461
第 16 章 DRX 465
16.1 DRX 介绍 465
16.2 载波聚合（CA）对 DRX 的影响 471
第 17 章 MAC 复用与逻辑信道优先级 472
17.1 逻辑信道优先级与 MAC 复用 473
17.2 MAC PDU 478
17.2.1 用于 DL-SCH 和 UL-SCH 的 MAC PDU 478
17.2.2 Transparent MAC PDU 481
17.2.3 RAR MAC PDU 481
第 18 章 上行同步 484
第 19 章 小区搜索过程（Cell Search Procedure） 490
第 20 章 系统信息（System Information） 498
20.1 MIB 500
20.2 SIB 504
20.3 系统信息有效性和变更通知 509
20.4 载波聚合对系统信息的影响 511
第 21 章 随机接入过程（Random Access Procedure） 513
21.1 preamble 513
21.2 PRACH 资源 515
21.3 随机接入过程 522
21.3.1 **步骤一**：UE 发送 preamble 524
21.3.2 **步骤二**：eNodeB 发送 Random Access Response 528
21.3.3 **步骤三**：UE 发送 Msg3 533
21.3.4 **步骤四**：eNodeB 发送 contention resolution 534

21.4 各类触发事件下的随机接入过程 536
21.5 preamble 传输达到最大传输次数的处理 545
第 22 章 寻呼（Paging） 548
第 23 章 载波聚合（Carrier Aggregation） 557
23.1 PCell / SCell / Serving Cell 558
23.2 添加/修改/删除 SCell 562
23.2.1 添加/修改 SCell：（见 36.331 的 5.3.10.3b 节） 563
23.2.2 删除 SCell：（见 36.331 的 5.3.10.3a） 563
23.3 SCell 激活/去激活（Activation/Deactivation） 563
23.4 CA 的部署场景 565
23.5 异构网络（heterogeneous network） 568
23.6 CA 对物理层的影响 569
23.6.1 PCFICH 569
23.6.2 PHICH 569
23.6.3 PDCCH 570
23.6.4 对上行 L1/L2 Control Signaling 的影响 570
23.7 CA 对 MAC 层的影响（见 36.300 的 6.4 节） 570
23.7.1 **对 HARQ 的影响（见 36.300 的 11 章）** 570
23.7.2 **对调度的影响（见 36.300 的 11 章）** 572
23.7.3 **跨载波调度（cross-carrier scheduling）** 572
23.8 CA 对 RLC/PDCP 层的影响 577
23.9 CA 对 RRC 层的影响 577
23.10 CA 对 random access 过程的影响（见 36.300 的 10.1.5 节） 578
23.11 CA 对 DRX 的影响 579
23.12 CA 对 BSR 的影响 579
第 24 章 SPS 580
24.1 下行 SPS 585
24.2 上行 SPS 586
第 25 章 TTI bundling 591

第 26 章 RLC 599
26.1 TM 模式 600
26.2 UM 模式 602
26.2.1 发送端 603
26.2.2 接收端 610
26.3 AM 模式 620
26.3.1 发送端 621
26.3.2 接收端 634
26.3.3 ARQ 流程 644

第1章　LTE 概述

本章的主要目的是为了让大家对 LTE 的数据传输形成一个宏观上的认识。在本章中，我们将介绍 LTE 空口上的上下行传输、LTE 协议栈涉及的各层协议，以及如何计算 LTE 的下行峰值速率。

在介绍上下行传输时，我们主要随着横向的时间顺序来简单阐述相关流程。而在介绍 LTE 协议栈时，我们会根据协议栈的层次结构逐层进行描述，同时还会简单介绍 UE 如何与 eNodeB/核心网进行控制面/数据面的通信。在最后一节，我们会介绍如何计算 LTE 的下行峰值速率。

1.1　空口传输概述

在 LTE 中，下行（downlink）传输可以简单地理解为由 eNodeB 发送，而 UE 负责接收的传输；上行（uplink）传输可以简单地理解为由 UE 发送，而 eNodeB 负责接收的传输。

注：本书描述的“eNodeB 与 UE 之间”的传输，确切地说，是指“UE 与其所接入的小区之间”的传输。但通常这 2 种描述并不会引起歧义，所以本书在不同的地方，可能会使用不同的描述，但都是同一个意思。

首先，我们从横向的时间顺序来介绍空口（eNodeB 与 UE 之间）上的上下行传输流程。

1.1.1　下行传输简介

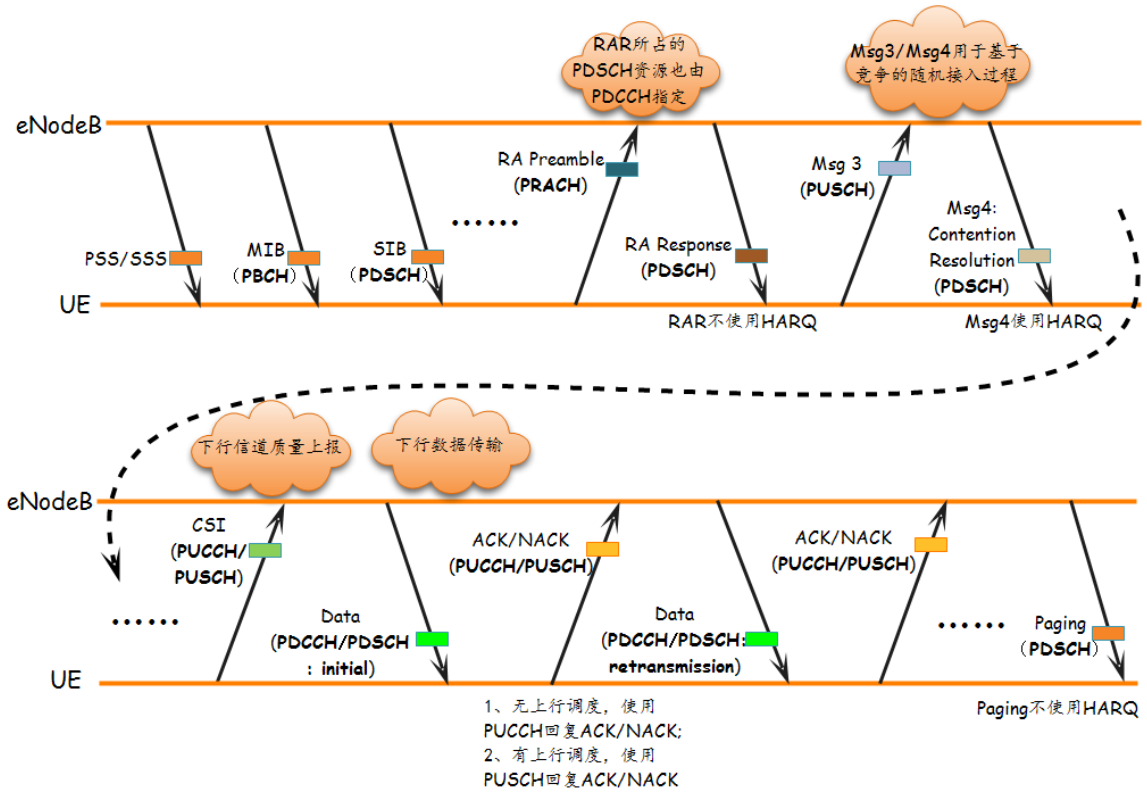


图 1-1：下行流程

我们先来简单介绍在空口上，与**下行**相关的流程。

UE 在开机前并不知道小区（cell）是否存在，也不知道小区是如何工作的。UE 要与某个小区进行通信，首先要选择一个运营商（如移动、联通、电信），即选择 PLMN。选择完 PLMN 后，

1. UE 会进行小区搜索，选择一个它认为最好的小区进行驻留。这是根据 eNodeB（小区）每隔 5 ms 发送一次的同步信号 PSS/SSS 来决定的。通过 PSS/SSS，UE 能够与小区获得时间和频率上的同步（但没有获得上行时间同步），以及得到小区的 PCI 等。（见第 19 章）

2. UE 确定了要进行通信的小区后，需要获取该小区的信息，以便获知如何在该小区上正确地工作。小区会不停地发送与该小区相关的系统信息（MIB/SIB），UE 会在需要的时候去获取这些信息。（见第 20 章）

3. 获取了小区信息之后，UE 就知道了该如何接入该小区，此时 UE 会发起随机接入过程以便与小区建立连接。（见第 21 章）

4. UE 与 eNodeB 建立起连接以后，UE 可能需要与 eNodeB 进行数据传输。eNodeB 会通过 PDSCH 来承载它所发给 UE 的数据，并通过 PDCCH 告诉 UE 对应的 PDSCH 在哪些无线资源上传输以及如何传输。而 UE 需要使用 ACK/NACK 来告诉 eNodeB 它是否成功接收到了数据。此时 ACK/NACK 是通过 PUCCH 或 PUSCH 来发给 eNodeB 的。如果 UE 没有成功接收到下行数据，eNodeB 需要重传数据。（见第 4、5、6、11 章等）

5. 无线信道条件可能是不断变化的，UE 需要将其看到的下行无线信道条件通过 CSI（上行传输）反馈给 eNodeB，以便 eNodeB 在下行调度时将信道质量考虑在内。（见第 10 章）

6. 当 UE 与 eNodeB 之间没有数据传输时，UE 并不需要一直保持连接（RRC_CONNECTED）状态，UE 可以处于 RRC_IDLE 态，并每隔一段时间“醒来”一次，去接收 Paging 消息，以确定是否有呼叫请求。eNodeB 还可以通过 Paging 来告诉 UE，系统信息发生了变化。（见第 22 章）

1.1.2 上行传输简介

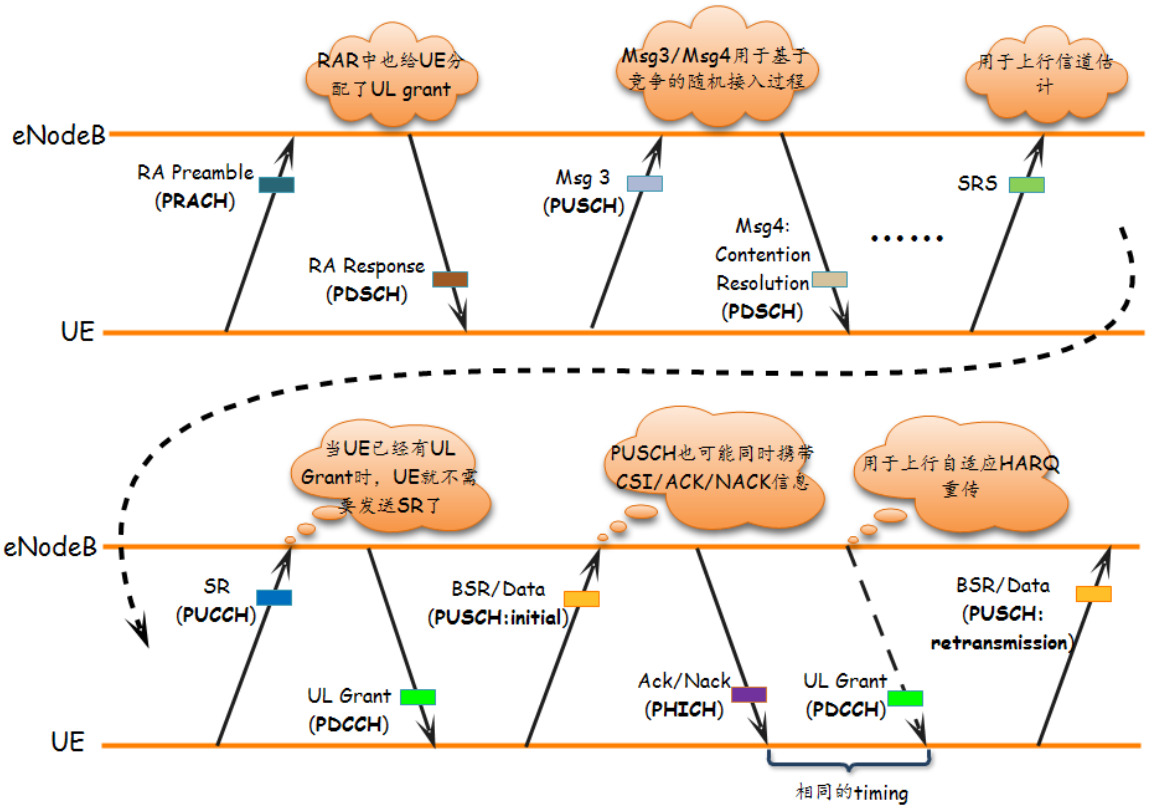


图 1-2：上行流程

接下来，我们将简单介绍在空口上，与**上行**相关的流程。

1. 随机接入过程中，UE 在上行可能需要发送 preamble 和 Msg3 以便与小区建立起连接。（见第 21 章）

2. 与下行类似，eNodeB 在进行上行调度时，需要进行上行信道估计，这是通过对 UE 发送的 SRS 进行测量得到的。（见第 7 章）

3. 对于上行传输，只有当 eNodeB 通过 UL grant（PDCCH）给 UE 分配了上行 PUSCH 资源时，UE 才能够使用对应的资源进行上行传输。当 UE 没有被分配上行 PUSCH 资源，但又有上行数据要发送时，UE 会通过发送 SR（Scheduling Request）告诉 eNodeB 有数据要发送，并请求 eNodeB 分配上行 PUSCH 资源。（关于 SR，见第 14 章的介绍）

4. UE 需要告诉 eNodeB 自己有多少数据要发送，以便 eNodeB 决定给 UE 分配多少资源。UE 通过 SR 只是告诉了 eNodeB 其有上行数据要发，而没有告诉 eNodeB 有多少数据要发，因此 UE 需要通过 BSR（Buffer Status Report）来告诉 eNodeB 自己有多少数据需要发送。（见第 15 章）

5. UE 与 eNodeB 建立起连接以后，UE 可能需要与 eNodeB 进行数据传输。UE 会通过 PUSCH 来承载它所发给 eNodeB 的数据。而 eNodeB 需要使用 ACK/NACK 来告诉 UE 它是否成功接收到了数据。此时 ACK/NACK 是通过 PHICH 发送给 UE 的。如果 eNodeB 没有成功接收到上行数据，UE 需要重传数据。（见 4.3 节、第 12 章）

6. 无线信道条件可能是不断变化的，UE 需要将其看到的下行无线信道条件通过 CSI 反馈给 eNodeB，以便 eNodeB 在下行调度时将信道质量考虑在内。CSI 是通过 PUCCH 或 PUSCH 传输的。（见第 10 章）

1.2 LTE 协议栈结构

本节的主要目的是为了介绍 eNodeB 与 UE 之间如何进行用户面和控制面的数据传输的，以便大家对 UE 如何与核心网进行通信有个大概的了解。

图 1-3 和图 1-4 分别从下行和上行描述了 LTE 无线协议的整体架构，以及无线承载、逻辑信道、传输信道和物理信道的对应关系。

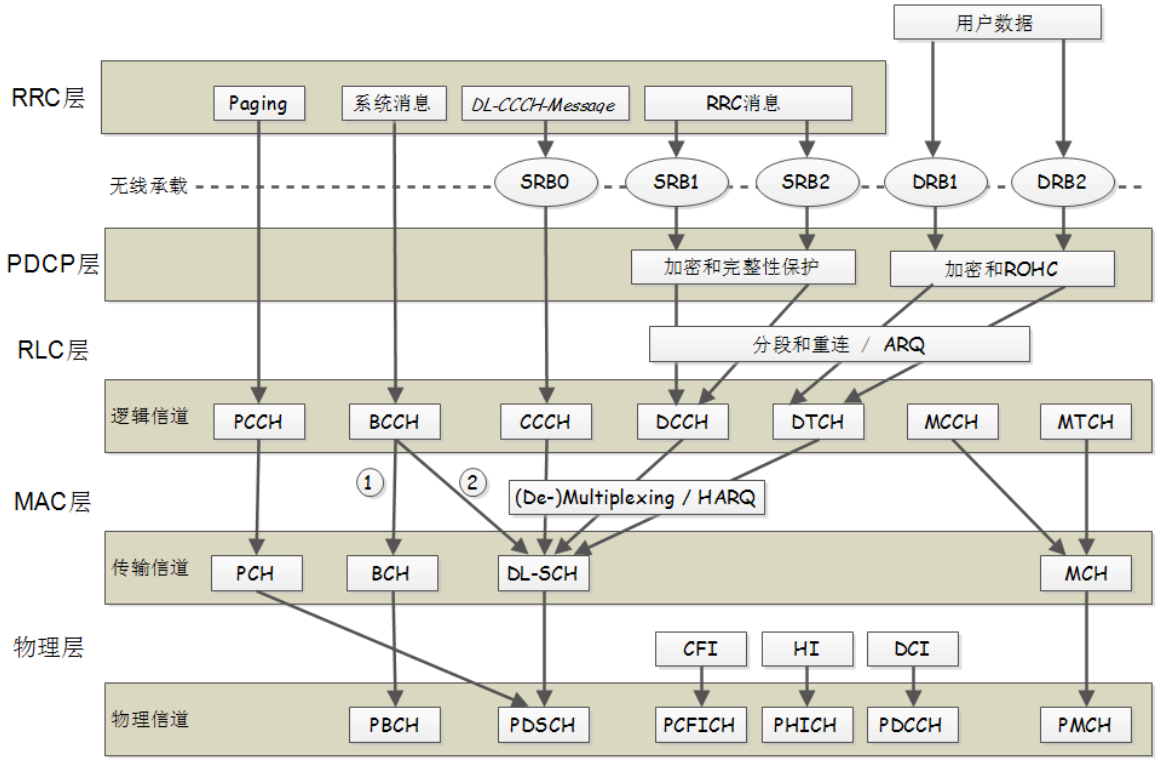


图 1-3：下行无线协议架构

注：系统消息包括 MIB 和 SIB，二者都映射到逻辑信道 BCCH，但 MIB 会在传输信道 BCH 上传输，而 SIB 会在传输信道 DL-SCH 上传输。

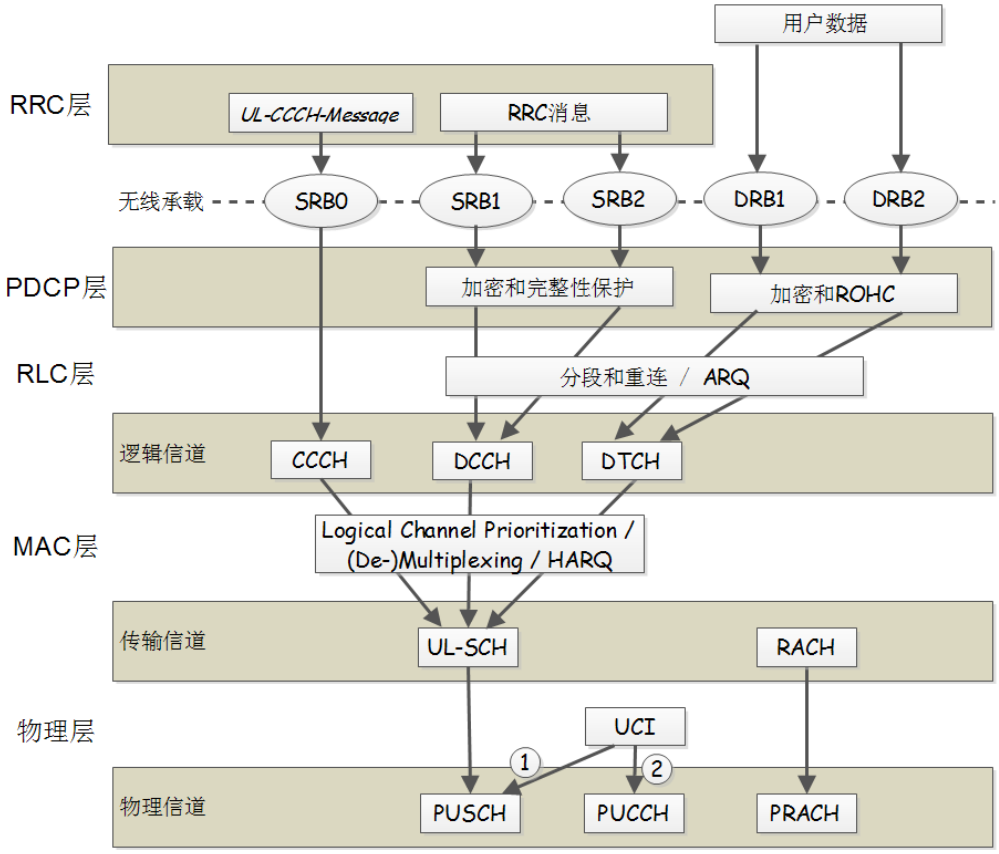


图 1-4：上行无线协议架构

注：当 UE 被分配了上行 PUSCH 资源时，UCI（上行控制信息，包括 CSI 和 ACK/NACK）会在 PUSCH 上传输，否则 UCI 在 PUCCH 上传输。

LTE 的空口协议栈包括用户面（user plane）协议和控制面（control plane）协议。

1.2.1 用户面

用户面的协议栈如 36.300 的 Figure 4.3.1-1 所示。

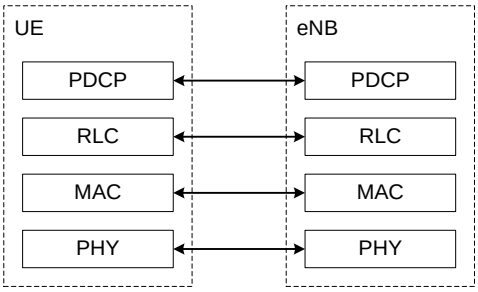


Figure 4.3.1-1: User-plane protocol stack

PDCP 层主要对来自控制面的 RRC 消息和来自数据面的 IP 包进行处理，其功能包括（见 36.323）：

- 头部压缩和解压缩：只使用 ROHC，并只用于用户面数据。
- 加密/解密（Ciphering and deciphering）：用于用户面和控制面数据。
- 完整性保护（Integrity Protection）：只用于控制面数据。
- 传输用户数据和控制面数据。
- 切换（handover）时的重排序和重传处理。
- 由于超时而丢弃用户面数据。

RLC 层主要负责（见 36.322）：

- 分段/串联和重组 RLC SDU（concatenation/segmentation/reassembly，只适用于 UM 和 AM 模式）：由于 RLC PDU 的大小是由 MAC 层指定的，其大小通常并不等于 RLC SDU 的大小，所以在发送端需要分段/串联 RLC SDU 以便其适合 MAC 层指定的大小。相应地，在接收端需要对之前分段的 RLC SDU 进行重组，以便恢复出原来的 RLC SDU 并发往上层。
- 通过 ARQ 来进行纠错（只适用于 AM 模式）：MAC 层的 HARQ 机制的目标在于实现非常快速的重传，其反馈出错率大概在 1%左右。对于某些业务，如 TCP 传输（要求丢包率小于10^{-5}），HARQ 反馈的出错率就显得过高了。对于这类业务， RLC 层的重传处理能够进一步降低反馈出错率。
- 对 RLC 数据 PDU 进行重排序（reordering，只适用于 UM 和 AM 模式）：MAC 层的 HARQ 可能导致到达 RLC 层的报文是乱序的，所以需要 RLC 层对数据进行重排序。

- 重复包检测（duplicate detection，只适用于 UM 和 AM 模式）：出现重复包的最大可能性为发送端反馈了 HARQ ACK，但接收端错误地将其解释为 NACK，从而导致了不必要的 MAC PDU 重传。
- 对 RLC 数据 PDU 进行重分段（resegmentation，只适用于 AM 模式）：当 RLC 数据 PDU（注意：这里不是 SDU）需要重传时，也可能需要进行重分段。例如：MAC 层指定的大小比原始的 RLC 数据 PDU 还要小时，就需要进行重分段。

MAC 层主要负责（见 36.321）：

- 匹配逻辑信道和传输信道。见图 1-3 和图 1-4。
- 将属于一个或不同的逻辑信道（无线承载）的多个 MAC SDU 复用到同一个 MAC PDU（Transport Block）上，并发往物理层。反之为解复用。（见第 17 章）
- 通过 HARQ 来进行纠错。（见第 4 篇）
- 调度处理。
- 逻辑信道优先级处理。（见第 17 章）
- 调度信息上报（只存在于 UE 侧和上行）。（见第 5 篇）
- 随机接入过程处理。（见第 21 章）

接下来，我们来简单介绍一下 UE 是如何与核心网进行用户面的数据通信的。

从 36.300 的 Figure 13.1-1 可以看出，UE 为了与对端实体进行通信，会与对端实体建立起一个端到端的服务（end-to-end service）。核心网（EPC）会通过外部承载（external bearer）与因特网建立连接，并通过 EPS bearer 与 UE 建立连接。也就是说，核心网与 UE 之间的数据通信是承载在 EPS bearer 上的。

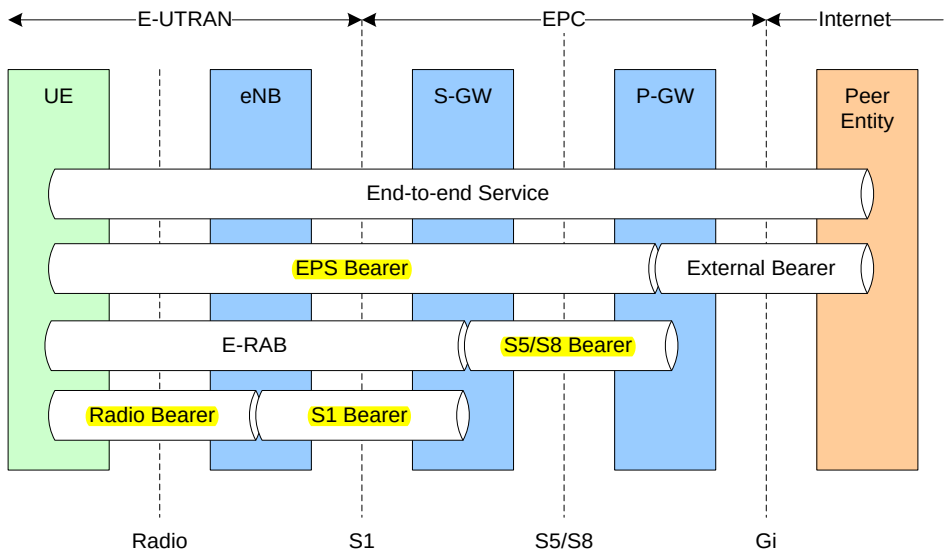


Figure 13.1-1: EPS Bearer Service Architecture

一个 EPS bearer 跨越多个接口（interface）：

- P-GW 和 S-GW 之间的 S5/S8 接口；
- S-GW 和 eNodeB 之间的 S1 接口；
- eNodeB 和 UE 之间的无线接口（即 Uu 接口）。

一个 EPS bearer 由一个 S5/S8 bearer 和一个 E-RAB 组成。一个 E-RAB 由一个 S1 bearer 和一个 DRB 组成。

一个 E-RAB 在 UE 和 EPC 之间传输一个 EPS bearer 的数据。如果存在一个 E-RAB，则该 E-RAB 与一个 EPS bearer 存在一一映射关系。

一个 DRB 在 UE 和 eNodeB 之间（Uu 接口）传输一个 EPS bearer 的数据。如果存在一个 DRB，则该 DRB 与一个 EPS bearer/E-RAB 存在一一映射关系。

一个 S1 bearer 在 eNodeB 和 S-GW 之间（S1 接口）传输一个 EPS bearer 的数据。

一个 S5/S8 bearer 在 S-GW 和 P-GW 之间（S5/S8 接口）传输一个 EPS bearer 的数据。

在上行，UE 使用 UL TFT（Traffic Flow Template）将一个 SDF（Service Data Flow，即上行用户数据）与一个 EPS bearer（在 UE 侧体现为 DRB）绑定在一起。通过在 UL TFT 中包含多个上行包过滤（uplink packet filter），可以将多个 SDF 复用到相同的 EPS bearer 上。

在下行，P-GW 使用 DL TFT 将一个 SDF（Service Data Flow，即下行用户数据）与一个 EPS bearer（在 P-GW 侧体现为 S5/S8 bearer）绑定在一起。通过在 DL TFT 中包含多个下行包过滤（downlink packet filter），可以将多个 SDF 复用到相同的 EPS bearer 上。

映射到相同 EPS bearer 的多个 SDF 会得到相同的 bearer 级别的包转发处理，如调度策略、队列管理策略、速率整形策略、RLC 配置等。

UE 需要保存上行包过滤（uplink packet filter）和 DRB 的对应关系，以便在上行建立起 SDF 和 DRB 之间的绑定关系（如图 1-5 中，UE 保存的 UL-TFT 与 RB ID 的映射关系）。

eNodeB 需要保存 DRB 和 S1 bearer 之间的一一映射关系，以便在上行和下行建立起 DRB 和 S1 bearer 之间的绑定关系（如图 1-5 中，eNodeB 保存的 RB ID 与 S1-TEID 的映射关系）。

S-GW 需要保存 S1 bearer 和 S5/S8 bearer 之间的一一映射关系，以便在上行和下行建立起 S1 bearer 和 S5/S8 bearer 之间的绑定关系（如图 1-5 中，S-GW 保存的 S1-TEID 与 S5/S8-TEID 的映射关系）。

P-GW 需要保存下行包过滤（downlink packet filter）和 S5/S8 bearer 的对应关系，以便在下行建立起 SDF 和 S5/S8 bearer 之间的绑定关系（如图 1-5 中，P-GW 保存的 DL-TFT 与 S5/S8-TEID 的映射关系）。

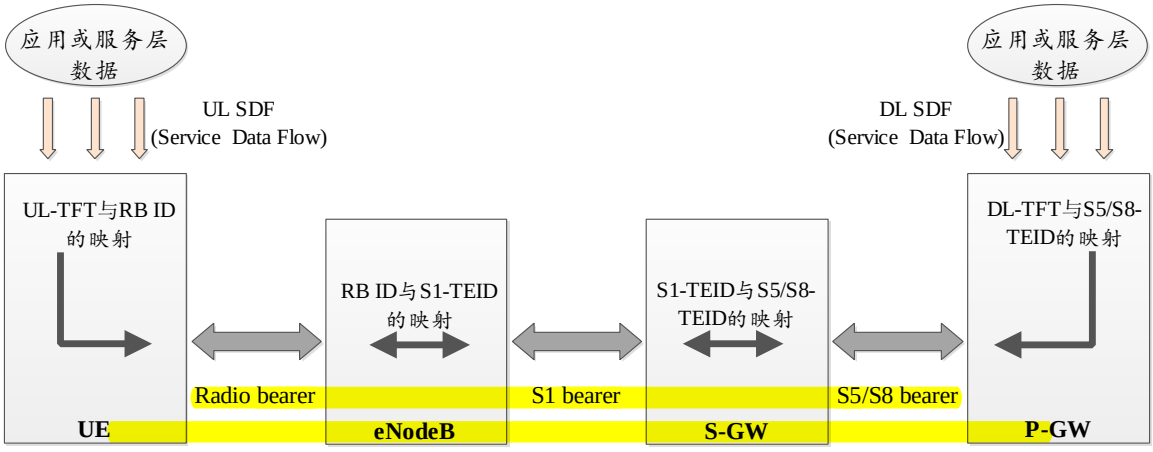


图 1-5：跨越不同接口的 LTE/SAE bearer

这里我们只关心 UE 与 eNodeB，以及 eNodeB 与 S-GW 之间的承载建立过程，36.300 的 19.2.2.4.1 节介绍了 E-RAB 建立的信令流程。

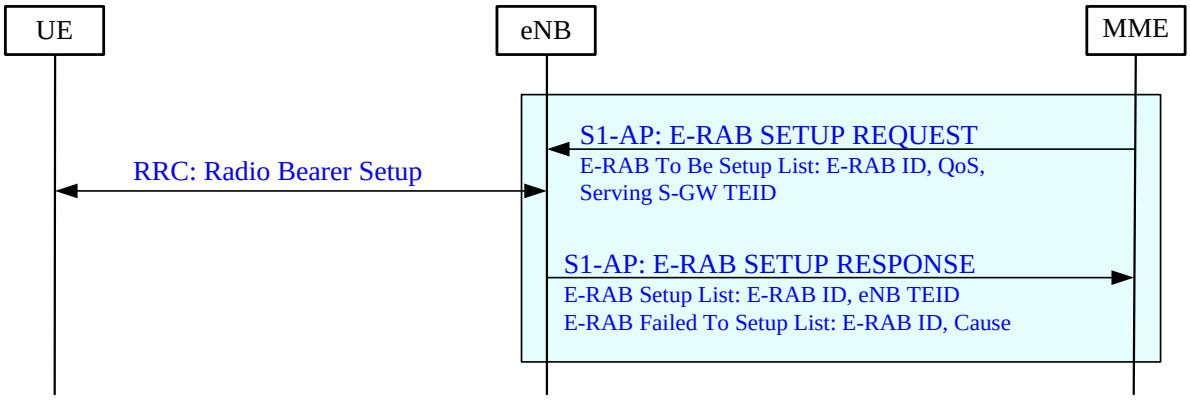


Figure 19.2.2.4.1-1: E-RAB Setup procedure

从 36.413 的 9.1.3.1 节可以看出，由 MME 发往 eNodeB 的 E-RAB SETUP REQUEST 消息的"*E-RAB To Be Setup Item IEs*"中包含了 E-RAB ID 和 GTP TEID 等信息，eNodeB 收到这条消息后，就会通过 RRC 消息 *RRCConnectionReconfiguration* 来发起 DRB 的建立过程（由 *DRB-ToAddMod* 来指定某个特定 DRB 的建立，见 36.331 的 5.3.10.3 节）。

```
DRB-ToAddMod ::=    SEQUENCE {
    eps-BearerIdentity                  INTEGER (0..15)         OPTIONAL,       -- Cond
DRB-Setup
    drb-Identity                        DRB-Identity,
    pdcp-Config                         PDCP-Config             OPTIONAL,       -- Cond
PDCP
    rlc-Config                          RLC-Config              OPTIONAL,       -- Cond
Setup
    logicalChannelIdentity              INTEGER (3..10)         OPTIONAL,       -- Cond
DRB-Setup
    logicalChannelConfig                LogicalChannelConfig    OPTIONAL,       -- Cond
Setup
    ...
}
```

E-RAB（E-UTRAN Radio Access Bearer）唯一地标记 S1 Bearer 和对应的 DRB 的串联。用在 S1 和 X2 接口上并用于指定一个 E-RAB 的 E-RAB ID 值，与用在 Uu 接口上并用于指定相关联的 EPS bearer 的 EPS bearer ID 值是相同的。也就是说，E-RAB SETUP REQUEST 消息中的 E-RAB ID 值与 *DRB-ToAddMod* 中的 *eps-BearerIdentity* 值是相同的。

这样，通过 E-RAB SETUP REQUEST 和 *DRB-ToAddMod*，我们就建立起了：（1）E-RAB 与 DRB 之间的一一映射关系；（2）eNodeB 会保存 GTP TEID（即前面介绍的 S1-TEID）与 DRB ID（*drb-Identity*）的映射关系；（3）*DRB-ToAddMod* 中的 *pdcp-Config* 和 *rlc-Config* 指定了该 DRB 的数据在 PDCP 层和 RLC 层如何处理；（4）*DRB-ToAddMod* 中的 *logicalChannelIdentity* 和 *logicalChannelConfig* 用于指定 MAC 层如何进行复用处理；（5）物理层的处理基于 MAC PDU 来进行的，而不是基于单个承载进行的，所以 *DRB-ToAddMod* 并不指定物理层的处理。

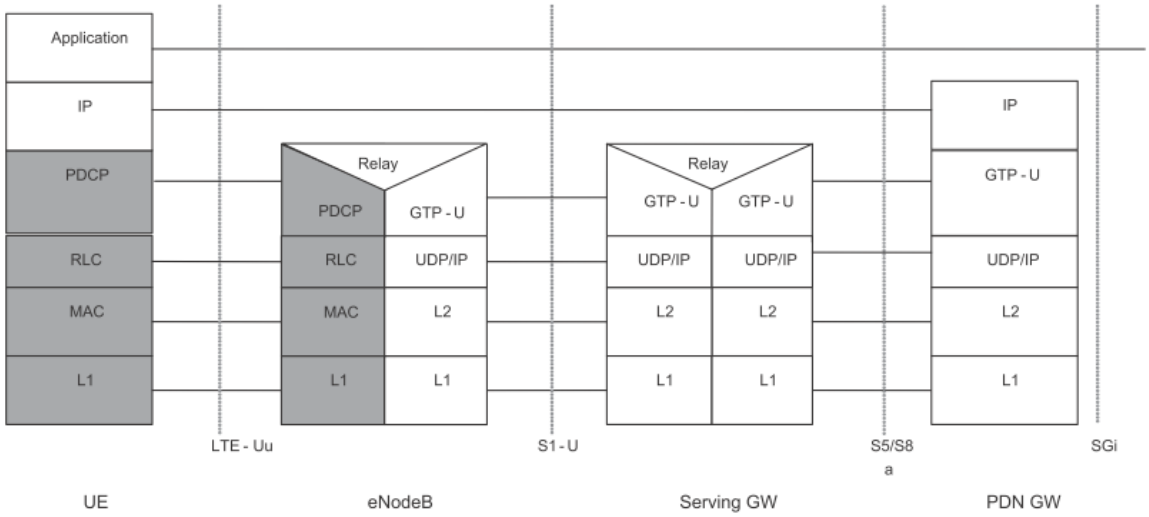


图 1-6：E-UTRAN 用户面协议栈

图 1-6 是 E-UTRAN 用户面协议栈，可以看出，eNodeB 从核心网（S1-U）接收到的用户数据是 GTP-U 报文。GTP-U 头包含了一个 TEID 字段，通过该字段，eNodeB 会根据在承载建立时保存的 S1-TEID 与 RB ID 的对应关系，将报文送到对应的 DRB 来处理。

对于上行而言，UL-TFT 是由 PCRF 建立的，并在上下文建立消息中发给 UE（DL TFT 是在 P-GW 中处理的，也是由 PCRF 建立的）。24.301 的 9.9.4.15 节简要地介绍了 TFT，而 24.008 的

10.5.6.12 节详细地介绍了真实过滤的内容。简单地说，就是将满足一定条件的报文送到对应的 DRB 上处理，而不满足该条件的报文则不会由该 DRB 处理。eNodeB 在接收并处理完某个 DRB 的数据后，根据 RB ID 与 S1-TEID 的对应关系，把处理后的报文封装成 GTPU 报文，再送往核心网。

关于用户面协议的介绍，还可参见参考资料[4]。

1.2.2 控制面

控制面的协议栈如 36.300 的 Figure 4.3.2-1 所示。

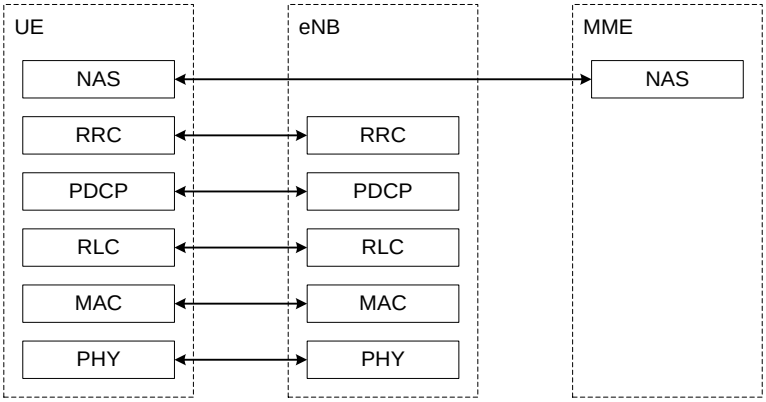


Figure 4.3.2-1: Control-plane protocol stack

RRC 层主要负责（见 36.331）：

- 广播系统消息。（见第 20 章）
- RRC 连接控制。
- Inter-RAT 移动性。
- 测量配置和上报。
- 其它功能：如发动专用的 NAS 消息、传输 UE 接入能力信息等。

RRC 消息（以及 NAS 消息）使用 SRB（Signaling Radio Bearer）来传输。LTE 中共定义了 3 种 SRB：SRB0、SRB1 和 SRB2。

SRB0 是默认建立的无线承载。使用 SRB0 的 RRC 消息都使用 CCCH 来传输，而 CCCH 的配置（包括逻辑信道配置）是固定的，可参见 36.331 的 9.1.1.2 节。

SRB0 并没有一个对应的 PDCP 实体，使用 SRB0 的 RRC 消息在 PDCP 层是透传的（既没有完整性保护，也没有加解密处理）。

在 36.331 的第 6 章搜索 SRB0，就可以知道哪些 RRC 消息使用 SRB0 进行传输（SRB1 和 SRB2 也类似）。同时可以看到，这些消息使用的都是 RLC TM 模式，说明使用 SRB0 的 RRC 消息在 RLC 层是透传的。

SRB1 用于发送 RRC 消息（该消息可能携带 NAS 信息）；并在 SRB2 建立之前，用于发送 NAS 消息，这些消息都使用 DCCH 来发送。

SRB2 用于发送包含记录的测量信息在内的 RRC 消息，并用于发送 NAS 消息，这些消息都使用 DCCH 来发送。SRB2 的优先级要低于 SRB1，并且 SRB2 总是在安全激活之后配置的。

其实大家没有必要去记住哪些 RRC 消息（或 NAS 消息）使用哪种 SRB 来传输。正如前面所说，在 36.331 的第 6 章中搜索 SRB0、SRB1 或 SRB2，就可以知道对应的消息使用哪种 SRB 来进行传输。

SRB1 和 SRB2 的建立是通过 IE：*SRB-ToAddModList* 来配置的。

```
SRB-ToAddModList ::=                SEQUENCE (SIZE (1..2)) OF SRB-ToAddMod

SRB-ToAddMod ::=    SEQUENCE {
    srb-Identity                        INTEGER (1..2), ------1对应SRB1，2对应SRB2
    rlc-Config                          CHOICE {        ------RLC层的配置
        explicitValue                       RLC-Config, ------显式配置SRB1或SRB2
        defaultValue                        NULL        ------使用36.331的9.2.1节的默认值来配置
SRB1或SRB2
    }       OPTIONAL,                                                       --
Cond Setup
    logicalChannelConfig                CHOICE {        ------MAC层的配置
        explicitValue                       LogicalChannelConfig, ------显式配置SRB1或SRB2
        defaultValue                        NULL        ------使用36.331的9.2.1节的默认值来配置
SRB1或SRB2

    }       OPTIONAL,                                                       --
Cond Setup
    ...
}
```

rlc-Config 和 *logicalChannelConfig* 都是一个 CHOICE，eNodeB 可以选择显式地配置 SRB1 或 SRB2，也可以选择使用默认配置。SRB1 和 SRB2 的默认配置见 36.331 的 9.2.1 节。SRB1 和 SRB2 在 RLC 层只能使用 AM 模式。

除 SRB0 外，每个 RB（包括 SRB1、SRB2 和 DRB）都有一个对应的 PDCP 实体（进行完整性保护和加解密）。而每个 PDCP 实体都有 1 个或 2 个 RLC 实体（每个方向各一个），这取决于 RB 的属性（单向还是双向）和 RLC 模式。

SRB1 是在 RRC 连接建立时创建的（见 36.331 的 5.3.1.1 节）。当安全激活（初始安全激活流程见 5.3.4 节）后，所有使用 SRB1 和 SRB2 的 RRC 消息将进行完整性保护和加密处理。

SRB 建立起来后，PDCP/RLC/MAC 层的配置也就确定了（物理层的处理基于 MAC PDU 来进行的，而不是基于单个承载进行的，所以 *SRB-ToAddMod* 并不指定物理层的处理），对应的 RRC 消息就可以在上面传输了。

【参考资料】

[1] TS 36.300 的 13.1 节 Bearer service architecture

[2] TS 36.300 的 19.2.2.4 节 E-RAB signalling procedures

[3] TS 36.413 的 8.2.1 节 E-RAB Setup

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的第 4 章和 2.4 节

[5] http://lte-gtp.blogspot.com/2012/02/tft-filtering-rule-in-lte-network.html

1.3 LTE 下行峰值速率计算

LTE 的下行峰值速率（peak data rate）可定义为满足以下条件时的最大吞吐量（throughput）：

- 整个带宽均分配给一个 UE；
- 使用最高阶的 MCS；
- 使用可支持的最大天线数。

在实际中，需要考虑典型的无线信道开销，如控制信道、参考信号、保护间隔等。

对于 **FDD** 而言，峰值速率的计算方法如下：

1 slot = 0.5 ms（一个系统帧 system frame 为 10 ms，每个子帧 subframe 为 1 ms，每个子帧包含 2 个 slot）；

1 slot = 7 modulation symbols（使用正常的循环前缀）；

1 modulation symbol = 6 bits（使用 64QAM 调制）；

单个子载波下的峰值速率 = 每个 slot 的 symbol 数 * 每个 symbol 的比特数 / 每个 slot 所占的时间 = 7 * 6 / 0.5 ms = 84kbps。（1s = 1000ms）

对于 20M 带宽而言，100 个 RB 用于数据传输，每个 RB 包含 12 个子载波，共有 1200 个子载波，则单天线下峰值速率为 1200 * 84kbps = 100.8Mbps。

如果是 4*4 MIMO，则峰值速率为单天线时的 4 倍，即 403.2Mbps。如果使用 3/4 的信道编码，则速率降低为 302.4Mbps。

注：（1）UE 看到的实际速率取决于即时的信道条件以及共享无线资源的用户数。例如：如果由于信道质量较差，调制从 64QAM 降低到 QPSK，则速率从 302.4Mbps 降到 100.8Mbps。如果把码率从 3/4 降到 1/3，则速率进一步降低到 44.8 Mbps。

（2）前面介绍的并未把 PDCCH、参考信号、PBCH、PSS/SSS 以及编码的开销考虑进去。假设这些开销总共为 25%，非空分复用情况下，真正可用于传输用户数据的最大速率为 100.8Mbps * 75% = 75.6Mbps。

（3）也可以先计算 RE 总数，再乘以每个 symbol 的比特数 6，得到峰值速率。

对于 TDD 而言，由于一个 10 ms 的系统帧内既存在下行子帧，又存在上行子帧，以及特殊帧的存在，因此同等条件下，其峰值速率小于 FDD 的峰值速率。（根据下行子帧在一个系统帧中所占的比例，乘以相应的系数）

在 5.3.2 节中，我们会从 TB size 的角度来介绍协议规定的能够支持的下行峰值速率。这个速率是将其它开销排除在外后，计算出的最大速率。

而在 9.3.3 节中，我们会从 TB size 的角度来介绍协议规定的能够支持的上行峰值速率。

【参考资料】

[1] 《How to calculate peak data rate in LTE?》by Hongyan

[2] 《LTE Data Rate Calculation》

第2章　LTE 时域、频域和空间域资源

对于 LTE 中的上下行传输，我们首先要确定空口上有哪些资源可用来发送这些数据。本章将分别从时域、频域和空间域来介绍 LTE 中的空口资源。如果没有特别说明，则相关内容同时适用于 FDD 和 TDD、上行和下行。

LTE 在下行使用 OFDM（Orthogonal Frequency Division Multiplexing），上行使用 SC-FDMA（Single-Carrier Frequency Division Multiple Access）。

2.1　时域

为了提供一致而精确的时间定义，LTE 定义了一个基本的时间单位 $T_s = 1/(15000 * 2048) = 1/30720000$ 秒。该时间单位可以看作是基于 FFT（Fast Fourier Transform，快速傅里叶变换）且 FFT 大小等于 2048 的发射机/接收机所使用的采样时间。

在时域上，上下行传输都被组织成 10 ms（$T_f = 307200 \times T_s = 10\ \text{ms}$）的系统帧（system frame，或称为无线帧：radio frame）。LTE 支持 2 种系统帧结构：用于 FDD 的类型 1 和用于 TDD 的类型 2。

1、帧结构类型 1（FDD：上下行数据在不同的频率内传输，使用成对频谱，支持全双工和半双工）

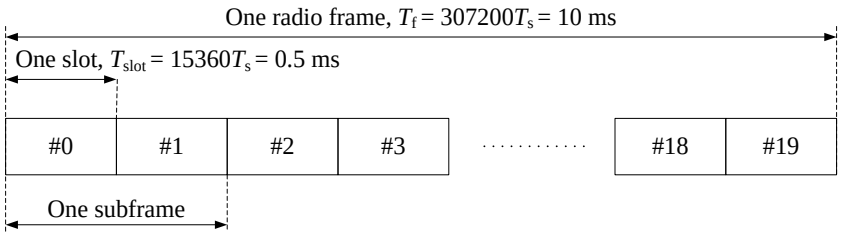


图 2-1：帧结构类型 1（FDD）

FDD 下，每个系统帧长达 10 ms，由 10 个子帧（subframe）组成。每个子帧长达 1 ms，由 2 个连续的 slot 组成。每个 slot 长达 0.5 ms（$T_{slot} = 15360 \cdot T_s = 0.5\ \text{ms}$）。

对于 FDD 而言，上下行传输是通过频域区分开的。在每一个 10 ms 内，各有 10 个子帧可用于上行传输和下行传输。

2、帧结构类型 2（TDD：上下行数据在同一频率内传输，使用非成对频谱）

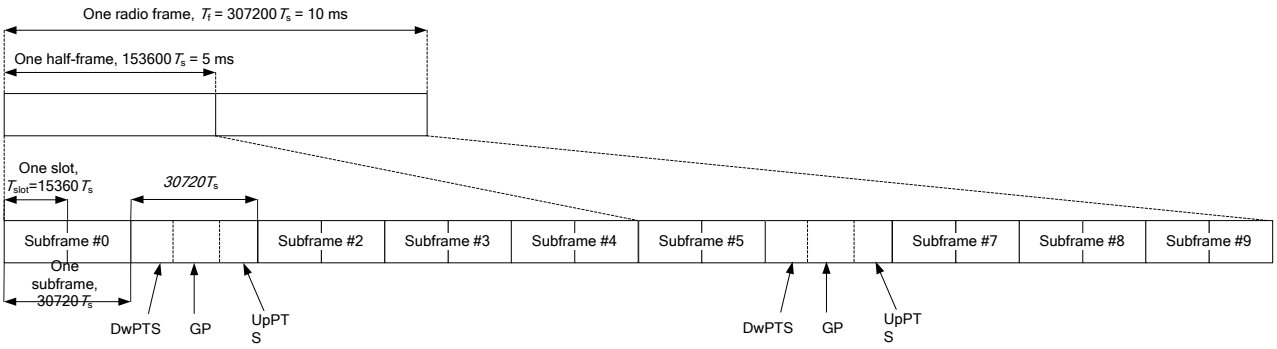


图 2-2：帧结构类型 2（TDD，5 ms 切换周期）

TDD 下，每个系统帧长达 10 ms，由 2 个长达 5 ms 的半帧（half-frame）组成。每个半帧由 5 个长达 1 ms 的子帧组成。TDD 中的子帧包括正常子帧和特殊子帧。

对于 TDD 而言，上下行传输是通过时域区分开的。

TDD 支持 7 种不同的上下行配置（uplink-downlink configuration），对应不同的上下行配比，具体见 36.211 的 Table 4.2-2。其中“D”对应一个下行子帧，“U”对应一个上行子帧，“S”对应一个特殊子帧。与特殊子帧相对应，我们将“D”和“U”对应的子帧称为正常子帧。

TDD 上下行配置是通过 RRC 消息中的 *TDD-Config -> subframeAssignment* 字段来设置的。

Table 4.2-2: Uplink-downlink configurations.

Uplink-downlink configuration	Downlink-to-Uplink Switch-point periodicity	Subframe number									
		0	1	2	3	4	5	6	7	8	9
0	5 ms	D	S	U	U	U	D	S	U	U	U
1	5 ms	D	S	U	U	D	D	S	U	U	D
2	5 ms	D	S	U	D	D	D	S	U	D	D
3	10 ms	D	S	U	U	U	D	D	D	D	D
4	10 ms	D	S	U	U	D	D	D	D	D	D
5	10 ms	D	S	U	D	D	D	D	D	D	D
6	5 ms	D	S	U	U	U	D	S	U	U	D

从 36.211 的 Table 4.2-2 可以看出，TDD 上下行配置支持 5 ms 和 10 ms 的下行到上行的切换周期。在 5 ms 的切换周期中，在 2 个半帧都存在特殊子帧；在 10 ms 的切换周期中，只有第一个半帧存在特殊子帧。

注意：在本书中，为了描述的方便，我们将 TDD 的 7 种不同的上下行配置简称为 TDD 0~6。例如 TDD 1 表示 TDD Uplink-downlink configuration 1。

TDD 下的正常子帧结构与 FDD 下的子帧结构是相同。

特殊子帧包含 3 个域：DwPTS、GP 和 UpPTS，这 3 个域的时长相加等于 1 ms。特殊子帧有 9 种不同的配置，对应不同的 DwPTS 和 UpPTS 长度，见 36.211 的 Table 4.2-1。特殊子帧配置是通过 RRC 消息中的 *TDD-Config -> specialSubframePatterns* 字段来设置的。

Table 4.2-1: Configuration of special subframe (lengths of DwPTS/GP/UpPTS). （36.211）

Special subframe configuration	Normal cyclic prefix in downlink			Extended cyclic prefix in downlink		
	DwPTS	UpPTS		DwPTS	UpPTS	
		Normal cyclic prefix in uplink	Extended cyclic prefix in uplink		Normal cyclic prefix in uplink	Extended cyclic prefix in uplink
0	$6592 \cdot T_s$	$2192 \cdot T_s$	$2560 \cdot T_s$	$7680 \cdot T_s$	$2192 \cdot T_s$	$2560 \cdot T_s$
1	$19760 \cdot T_s$			$20480 \cdot T_s$		
2	$21952 \cdot T_s$			$23040 \cdot T_s$		
3	$24144 \cdot T_s$			$25600 \cdot T_s$		
4	$26336 \cdot T_s$			$7680 \cdot T_s$	$4384 \cdot T_s$	$5120 \cdot T_s$
5	$6592 \cdot T_s$	$4384 \cdot T_s$	$5120 \cdot T_s$	$20480 \cdot T_s$		
6	$19760 \cdot T_s$			$23040 \cdot T_s$		
7	$21952 \cdot T_s$			-	-	-
8	$24144 \cdot T_s$			-	-	-

与 FDD 类似，TDD 下的每个子帧（包括特殊子帧）长达 1 ms，由 2 个连续的 slot 组成，每个 slot 长达 0.5 ms（ $T_{slot} = 15360 \cdot T_s = 0.5\,\text{ms}$ ）。

子帧 0、5 以及 DwPTS 总是用于下行传输；UpPTS 以及紧随特殊子帧之后的子帧总是用于上行传输。

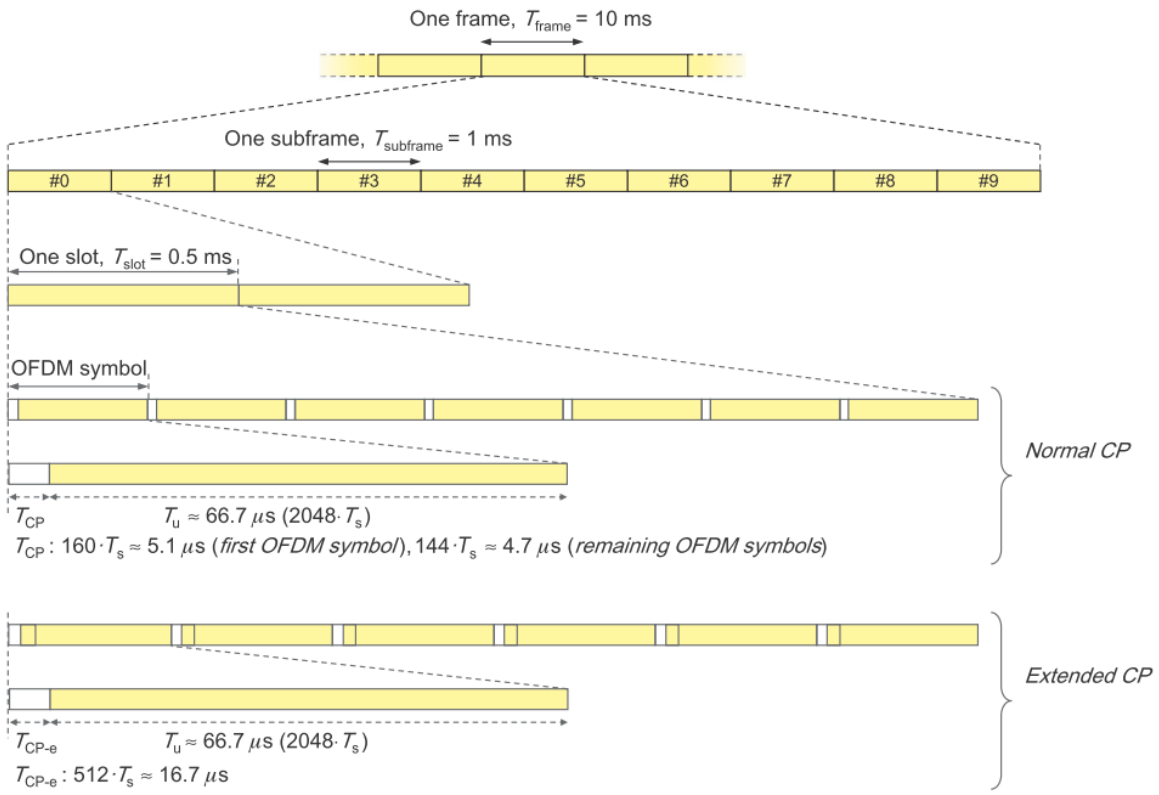


图 2-3：LTE 时域结构

对于 FDD 和 TDD 而言，系统帧的编号范围为 0 ~ 1023；一个系统帧内的子帧编号范围为 0 ~ 9；一个系统帧内的 slot 编号范围为 0 ~ 19，即子帧 i 包含 slot $2i$ 和 $2i+1$。

协议中系统帧号（System Frame Number，SFN）使用 n_f 表示，一个系统帧内 slot 号使用 n_s 表示，一个系统帧内的子帧号使用 $\lfloor n_S / 2 \rfloor$ 表示。

一个 slot 由多个符号（symbol）组成，每个符号（用 l 表示）由循环前缀（Cyclic Prefix，简称 CP）和可用的符号时间组成。上行使用 SC-FDMA 符号（SC-FDMA symbol），下行使用 OFDM 符号（OFDM symbol）。

一个 slot 包含的符号数（下行：$N_{\text{symb}}^{\text{DL}}$；上行：$N_{\text{symb}}^{\text{UL}}$）取决于循环前缀的长度和子载波的间距（子载波的介绍见下一节），如下表所示（下行见 36.211 的 Table 6.2.3-1；上行见 36.211 的 Table 5.2.3-1）。

Table 6.2.3-1: Physical resource blocks parameters.

Configuration		N_{sc}^{RB}	N_{symb}^{DL}
Normal cyclic prefix	$\Delta f = 15$ kHz	12	7
Extended cyclic prefix	$\Delta f = 15$ kHz		6
	$\Delta f = 7.5$ kHz	24	3

Table 5.2.3-1: Resource block parameters.

Configuration	N_{sc}^{RB}	N_{symb}^{UL}
Normal cyclic prefix	12	7
Extended cyclic prefix	12	6

从上表可以看出，除了 TDD 中的特殊子帧，当正常子帧（包括 FDD 和 TDD）使用正常的循环前缀（normal cyclic prefix，简称为 normal CP）时，每个 slot 由 7 个符号（$l = 0 \sim 6$）组成；当正常子帧使用扩展的循环前缀（extended cyclic prefix，简称为 extended CP）时，每个 slot 由 6 个符号（$l = 0 \sim 5$）组成。

OFDM 系统中可以插入一段空白符号作为保护间隔，这样做虽然能消除 ISI（Inter Symbol Interference，符号间干扰），但不能消除 ICI（Inter Carrier Interference，子载波间干扰）。循环前缀是将 OFDM 信号的最后一定长度内的部分提取出来放在 OFDM 信号的头部，将加入了循环前缀后变长了的 OFDM 信号作为新的 OFDM 信号，这样就可以完全消除 ISI 和 ICI。

循环前缀有正常的循环前缀和扩展的循环前缀之分，其区别在于长度的不同。如下表（下行见 36.211 的 Table 6.12-1；上行见 36.211 的 Table 5.6-1）所示。

Table 6.12-1: OFDM parameters.

Configuration		Cyclic prefix length $N_{CP,l}$
Normal cyclic prefix	$\Delta f = 15$ kHz	160 for $l = 0$ 144 for $l = 1,2,...,6$
Extended cyclic prefix	$\Delta f = 15$ kHz	512 for $l = 0,1,...,5$
	$\Delta f = 7.5$ kHz	1024 for $l = 0,1,2$

Table 5.6-1: SC-FDMA parameters.

Configuration	Cyclic prefix length $N_{CP,l}$
Normal cyclic prefix	160 for $l = 0$ 144 for $l = 1,2,...,6$
Extended cyclic prefix	512 for $l = 0,1,...,5$

UE 是在小区搜索过程中确定下行的循环前缀长度的，并通过接收 RRC 消息中的 *UL-CyclicPrefixLength* 字段确定上行的循环前缀长度。

从前面的 2 张表可以看出，对于正常的循环前缀，每个 slot 的第一个符号的循环前缀长度比其它的符号要大。这是因为 0.5 ms 的 slot 长度（对应 15360 个 T_s ），是不能被 7 整除的，所以第一个符号的循环前缀要长一些。对于扩展的循环前缀，就不存在这个问题（15360 能被 6 整除）。

每个符号中，可用的符号时间为 2048 个 T_s 。

可以看出，对于一个 slot，其长度等于所有符号（symbol）的长度相加。对于正常的循环前缀，有 $0.5ms = 15360 * T_s = ((160 + 2048) + (144 + 2048) * 6) * T_s$ 。对于扩展的循环前缀，有 $0.5ms = 15360 * T_s = ((512 + 2048) * 6) * T_s$ 。对于 TDD 下的特殊子帧，DwPTS、GP 和 UpPTS 这 3 个域的时长相加等于 1 ms，即 $30720 * T_s$ ，对于 DwPTS 和 UpPTS 的长度见 36.211 的 Table 4.2-1，剩下的长度为 GP 的长度（注意：虽然特殊子帧的结构不同，但我们可以认为其符号数同正常子帧的符号数是一样的）。

对照 36.211 的 Table 4.2-1，可得到正常的循环前缀下，特殊子帧的 DwPTS/UpPTS/GP 所占的符号数，见表 2-1。

表 2-1：正常的循环前缀下，特殊子帧的 DwPTS/UpPTS/GP 所占的符号数（共 14 个符号）

特殊子帧配置	DwPTS	UpPTS	GP
0	3	1	10
1	9	1	4
2	10	1	3
3	11	1	2
4	12	1	1
5	3	2	9
6	9	2	3
7	10	2	2
8	11	2	1

DwPTS 可被当作正常的下行子帧来看待，即可用于 PDSCH 传输。但由于 DwPTS 的长度（所占的符号数）小于正常的下行子帧，所以其携带的数据量要更少（从 36.213 的 7.1.7 节可以看出，DwPTS 携带的数据量约为正常下行子帧的 0.75 倍）。

注：对于正常循环前缀下的特殊子帧配置 0 和 5，或扩展的循环前缀下的特殊子帧配置 0 和 4 来说，其 DwPTS 只占 3 个符号，长度太短，所以不能用于 PDSCH 传输。

UpPTS 由于长度太短，不能用于 PUSCH 传输，而只能用于 SRS 或 PRACH 传输。

定义 2 个循环前缀长度的原因在于：（1）一个更长的循环前缀，虽然从循环前缀的开销角度来说效率更低，但在某些带有广泛的传输延迟的特定场景下，如非常大的小区，可能会带来好处。但需要注意的是，一个更长的循环前缀并不一定会对大型的小区带来好处，即使这种情况下延迟传播是非常广泛的。如果在大型小区中，链路的性能主要受限于噪声，而不是受限于循环前缀不能覆盖剩余的时间弥散所导致的信号损坏，此时使用更长的循环前缀所带来的额外的无线信道时间弥散的可靠性，可能无法证明使用更长的循环前缀所需要的额外能量开销是合理的；（2）在基于 MBSFN 的多播/广播传输中，需要使用扩展的循环前缀。

需要注意的是，同一个系统帧内的不同子帧可能使用不同的循环前缀长度。例如：基于 MBSFN 的多播/广播传输只会配置在有限的子帧上，在这些子帧上会使用扩展的循环前缀。而系统帧内的其它子帧可能使用正常的循环前缀。

7.5 kHz 的子载波间隔（ $\Delta f = 7.5\,\text{kHz}$ ）对应 2 倍长的 OFDM 符号时间，这种子载波间隔配置专门针对基于 MBSFN 的多播/广播传输（只存在于下行传输，上行传输是没有的）。LTE 规范并未完全实现 7.5 kHz 的子载波间隔，至少到 Rel-10 版本，LTE 只完整地支持 15 kHz 的子载波间隔。所以后面的介绍中，除非特殊说明，都假定使用的是 15 kHz 的子载波间隔。

从前面的介绍可以看出，LTE 中时域资源主要包括系统帧、子帧、slot、symbol 等。

表 2-2：LTE 中的时域资源

$T_s = 1/(15000 * 2048) = 1/30720000$ 秒
1 个系统帧的长度：$T_f = 307200 \cdot T_s = 10\,\text{ms}$
1 个子帧的长度：$T_{subframe} = 30720 \cdot T_s = 1\,\text{ms}$
1 个 slot 的长度：$T_{slot} = 15360 \cdot T_s = 0.5\,\text{ms}$
1 个 slot 包含 7 个 symbol（正常的循环前缀）或 6 个 symbol（扩展的循环前缀）

2.2 频域

LTE 中，频域上的基本单位为一个子载波（subcarrier）。上行和下行的子载波间距均为 15 kHz（即 $\Delta f = 15\,\text{kHz}$，这里不考虑 $\Delta f = 7.5\,\text{kHz}$ 的情况）。15 kHz 的子载波间隔对应每个符号（symbol）内可用的符号时间为 $2048 * T_s$，近似为 66.7 us。

LTE 中，频域上可用的所有资源称之为系统带宽（与信道带宽的对应关系见表 2-3），下行系统带宽使用 $N_{\text{RB}}^{\text{DL}}$ 表示，上行系统带宽使用 $N_{\text{RB}}^{\text{UL}}$ 表示，其单位是 RB（Resource Block）。每个 RB 包含 12 个子载波。

表 2-3：信道带宽与传输带宽之间的对应关系

信道带宽（MHz）	1.4	3	5	10	15	20
传输带宽（N_{RB}，单位为 RB）	6	15	25	50	75	100

LTE 中可用的下行系统带宽 $N_{\text{RB}}^{\text{DL}}$ 如表 2-3 所示，eNodeB 通过 MIB（对应 RRC 消息 *MasterInformationBlock* 的 *dl-Bandwidth* 字段）告知 UE 小区所使用的下行系统带宽。

LTE 中可用的上行系统带宽 $N_{\text{RB}}^{\text{UL}}$ 如表 2-3 所示，eNodeB 通过 SIB2（对应 RRC 消息 *SystemInformationBlockType2* 的 *ul-Bandwidth* 字段）告知 UE 小区所使用的上行系统带宽。对于 TDD 而言，eNodeB 并不需要发送 *ul-Bandwidth* 字段，因为其上下行的系统带宽是相同的。

一个 slot 上传输的信号可以描述成一个或多个频域上包含 $N_{\text{RB}}^{*} N_{\text{sc}}^{\text{RB}}$ 个子载波，时域上包含 N_{symb}^{*} 个符号（symbol）的时频资源网格（resource grid）。图 2-4 对应下行的时频资源网格，对于上行的时频资源网格，将图中的 DL 改为 UL 即可。（下行也可见 36.211 的 Figure 6.2.2-1，上行也可见 36.211 的 Figure 5.2.1-1）

N_{RB}^{*} 表示系统带宽，对于下行，其值等于 $N_{\text{RB}}^{\text{DL}}$；对于上行，其值等于 $N_{\text{RB}}^{\text{UL}}$。$N_{\text{symb}}^{*}$ 表示每个 slot 包含的符号数（见上一节介绍），对于下行，其值等于 $N_{\text{symb}}^{\text{DL}}$；对于上行，其值等于 $N_{\text{symb}}^{\text{UL}}$。$N_{\text{sc}}^{\text{RB}}$ 表示每个 RB 包含的子载波数。对于下行，$N_{\text{symb}}^{\text{DL}}$ 和 $N_{\text{sc}}^{\text{RB}}$ 的取值可见 36.211 的 Table 6.2.3-1；

对于上行，$N_{\text{symb}}^{\text{UL}}$和$N_{\text{sc}}^{\text{RB}}$的取值可见 36.211 的 Table 5.2.3-1。（由于只考虑$\Delta f = 15\,\text{kHz}$的情况，可以认为$N_{\text{sc}}^{\text{RB}}$的值固定为 12）

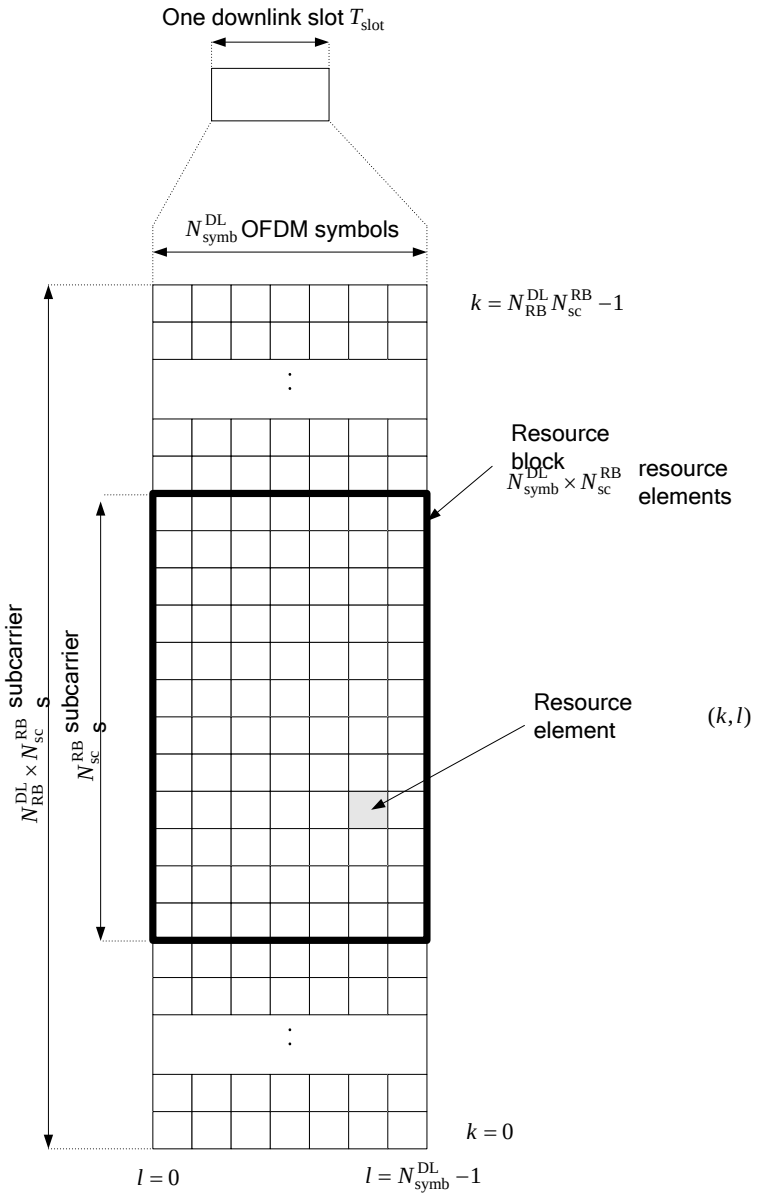


图 2-4：下行时频资源网格

资源网格上的每个元素称为一个 RE（Resource Element），并且通过(k, l)唯一指定的，其中$k = 0,...,N_{\text{RB}}^{*} N_{\text{sc}}^{\text{RB}} - 1$（对应频域上的每个子载波的索引）且$l = 0,...,N_{\text{symb}}^{*} - 1$（对应 slot 内的

每个符号的索引）。天线端口 p 上索引为 (k,l) 的 RE 对应的值使用 $a_{k,l}^{(p)}$ 表示。如果不会引起误解，或是没有指定特定的天线端口，则 p 可以省略，即可以使用 $a_{k,l}$ 表示。

RE 是 LTE 中的最小物理资源。一个 RE 可存放一个调制符号（modulation symbol），该调制符号可使用 QPSK（对应一个 RE 存放 2 比特数据）、16QAM（对应一个 RE 存放 4 比特数据）或 64QAM（对应一个 RE 存放 6 比特数据）调制。

注意：调制符号（modulation symbol，有时也简称为符号 symbol）强调的是放在一个 RE 上的数据，而前一节介绍的符号（symbol）强调的是时域上的概念，而非数据。

一个 RB（Resource Block）在时域上包含 N_{symb}^{*}（其值为 6 或 7）个连续的符号，在频域上包含 $N_{\text{sc}}^{\text{RB}}=12$ 个连续的子载波。因此一个 RB 由 $N_{\text{symb}}^{*}\times N_{\text{sc}}^{\text{RB}}$ 个 RE 组成，对应时域上的 1 个 slot 和频域上 12 个连续的子载波（180 kHz）。可以看出，对于正常的循环前缀，每个 RB 包含 7 * 12 = 84 个 RE；对于扩展的循环前缀，每个 RB 包含 6 * 12 = 72 个 RE。

RB 的编号范围为 0 ~ $N_{\text{RB}}^{*}-1$。频域上的 RB 编号 n_{PRB} 与一个 slot 上索引为(k, l)的 RE 的对应关系为

$$n_{\text{PRB}}=\left\lfloor \frac{k}{N_{\text{sc}}^{\text{RB}}} \right\rfloor$$

注意：这里介绍的 RB 与系统带宽中使用的 RB 是有区别的：这里的 RB 既包含了时域上的信息（1 个 slot）又包含了频域上的信息（12 个子载波），1 个 RB 由多个 RE 组成；而系统带宽中的 RB 专指频域上的信息，即 1 个 RB 包含 12 个子载波。

虽然 RB 是基于一个 slot（0.5 ms）定义的，但 LTE 中调度的基本时间单位是一个子帧（1 ms，对应 2 个 slot），称为一个 TTI。一个 TTI 内的调度（调度 PDSCH 和 PUSCH 资源）的最小单位实际上由同一子帧上时间上相连的 2 个 RB（每个 slot 对应一个 RB）组成，并被称为 RB pair。

在一个 slot 上定义 RB 的原因在于“下行分布式传输”和“上行 PUSCH 跳频”是基于一个 slot 或者说是一个 RB 定义的（即同一 RB pair 在 2 个 slot 间进行“分布式”或“跳频”传输）。

从上面的介绍可以看出：LTE 中频域资源包括整个系统带宽、RB、子载波等。如表 2-4 所示。

表 2-4：LTE 中的频域资源

LTE 中的频域资源
1 个子载波：$\Delta f=15\,\text{kHz}$

1 个 RB（频域资源）：包含 12 个子载波
系统带宽：包含 N_{RB}^{DL}（下行）或 N_{RB}^{UL}（上行）个 RB

一个 RE 既包含了时域信息（所在的 symbol），又包含了频域信息（所在的子载波）。如表 2-5 所示。

表 2-5：LTE 中的时频资源

LTE 中的时频资源
调度的基本单位：1 TTI = 1 ms = 1 subframe = 2 slots 注：对于 1 个 UE，最小分配 2 个 RB（1 个 RB pair）
1 个 RB（时频资源）= $N_{symb}^{*} \times N_{sc}^{RB}$ = 84 REs（正常的循环前缀） 或 72 REs（扩展的循环前缀）
1 个 RE = 1 subcarrier * 1 symbol

对于下行而言，在载波中心频点的位置有一个未使用的 DC 子载波（DC-subcarrier）。DC 子载波未被下行传输使用的原因在于该子载波可能由于本地晶振泄露等原因而遭受不成比例的高干扰。

而对于上行来说，载波中心频率上并不存在 DC 子载波。这是因为中心频点处的未使用 DC 子载波会导致无法在将整个上行系统带宽分配给一个单一 UE 的同时，将其映射到 OFDM 调制器的连续输入上，而这对于保持上行数据传输的 DFTS-OFDM 调制的低立方度量特性是必须的。

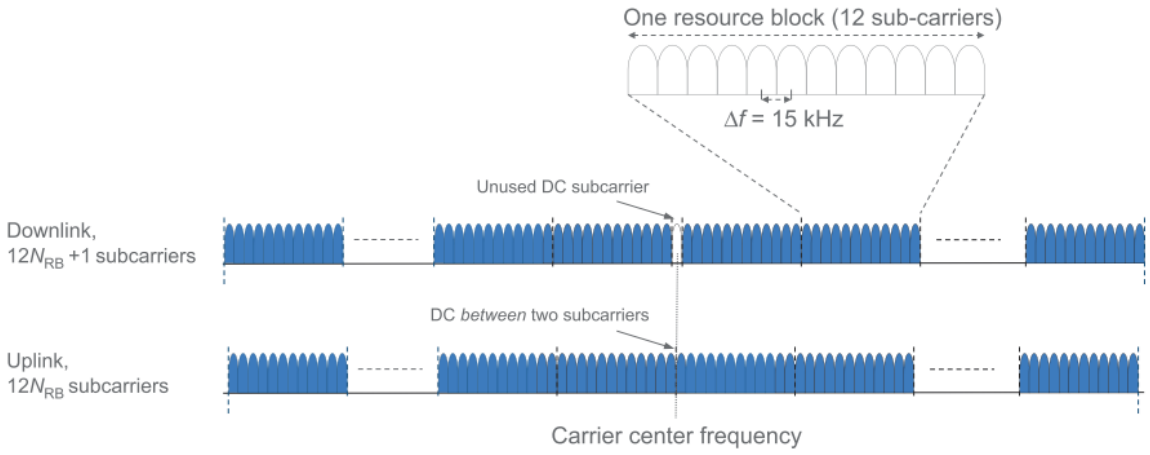


图 2-5：LTE 的频域结构

2.3 空间域

LTE 中，空间维度是以“层（layer）”来度量的，并使用多天线传输和多天线接收技术来实现。每层对应一条有效的数据流，并会映射到逻辑上的“天线端口（antenna port）”上。每个天线端口对应一个时频资源网格，并有一个对应的参考信号（Reference Signal），以便接收端进行信道估计和相干解调等。

通俗点讲，如果不进行空间维度的复用，即不使用空分复用，则在相同的时频资源上，只能存在单层，同时只能传输一条数据流。而如果进行空间维度的复用，即使用空分复用，则在相同的时频资源上，可同时存在多层，同时传输多条数据流，这样频谱的利用率更高，吞吐量也越高。

关于空分复用的具体应用，会在后续介绍上下行传输时予以说明。

【参考资料】

[1] TS 36.211 的 5.2 节 Slot structure and physical resources (uplink)

[2] TS 36.211 的 6.2 节 Slot structure and physical resource elements (downlink)

[3] 《4G LTE/LTE-Advanced for Mobile Broadband》的第 9 章

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 6.2 节和 14.2 节

第3章　下行参考信号

下行参考信号是预先定义好的，并占用时频资源网格中的某些特定 RE。LTE 中定义了多种下行参考信息，每种参考信号有各自不同的应用场景。本章将详细介绍除 MBSFN 参考信号和定位参考信号外的其它下行参考信号。

每个天线端口（antenna port）对应一个时频资源网格，并对应一个参考信号（Reference Signal，简称为 RS）。对于下行，一个小区支持的天线端口集合依赖于该小区的参考信号配置。下行参考信号在时频资源网格上占据特定的 RE。下行参考信号包括以下几种类型：

- 小区特定的参考信号（Cell-specific reference signal，简称为 CRS）：支持 1、2、4 个天线端口配置，并在天线端口 $p=0$、$p\in\{0,1\}$ 和 $p\in\{0,1,2,3\}$ 上传输；
- MBSFN 参考信号：在天线端口 $p=4$ 上传输；
- UE 特定的参考信号（UE-specific reference signal，也称为 DeModulation Reference Signal，简称为 DMRS）：在天线端口 $p=5$、$p=7$、$p=8$，或 $p\in\{7,8,9,10,11,12,13,14\}$ 中的一个或几个中传输；
- 定位参考信号（Positioning reference signal）：在天线端口 $p=6$ 上传输；
- CSI 参考信号（CSI-RS）：支持 1、2、4、8 个天线端口配置，并在天线端口 $p=15$、$p=15,16$、$p=15,...,18$ 和 $p=15,...,22$ 上传输。

3.1　小区特定的参考信号（Cell-specific Reference Signal）

小区特定的参考信号对小区内的所有 UE 都有效，其作用包括：（1）可被 UE 用于除 PMCH 和 TM 7/8/9（对应基于非码本的预编码）下的 PDSCH 传输之外的其它任何下行物理信道的信道估计；（2）可被 UE 用来获取 CSI；（3）基于小区特定的参考信号的终端测量可用作决定小区选择和切换的基础。

注：TM 7 使用天线端口 5，TM 8 使用天线端口 7 和 8，TM 9 使用天线端口 7~14，都使用 UE 特定的参考信号来进行相干解调，这些 TM 模式都不使用小区特定的参考信号来进行解调。

小区可以使用 1 个、2 个或 4 个小区特定的参考信号，分别对应使用 1 个、2 个或 4 个天线端口。小区特定的参考信号只在天线端口 0~3 中的一个或几个中传输。

小区特定的参考信号在每个下行子帧，整个下行带宽内的每个 RB 上都会发送。

在 MBSFN 子帧上，小区特定的参考信号只能在该子帧的非 MBSFN 区域（non-MBSFN region）上传输。

小区特定的参考信号只支持 $\Delta f = 15\text{ kHz}$。

小区特定的参考信号序列由一系列的参考符号组成，每个参考符号占一个 RE。图 3-1 给出了小区特定的参考信号序列的生成方式。

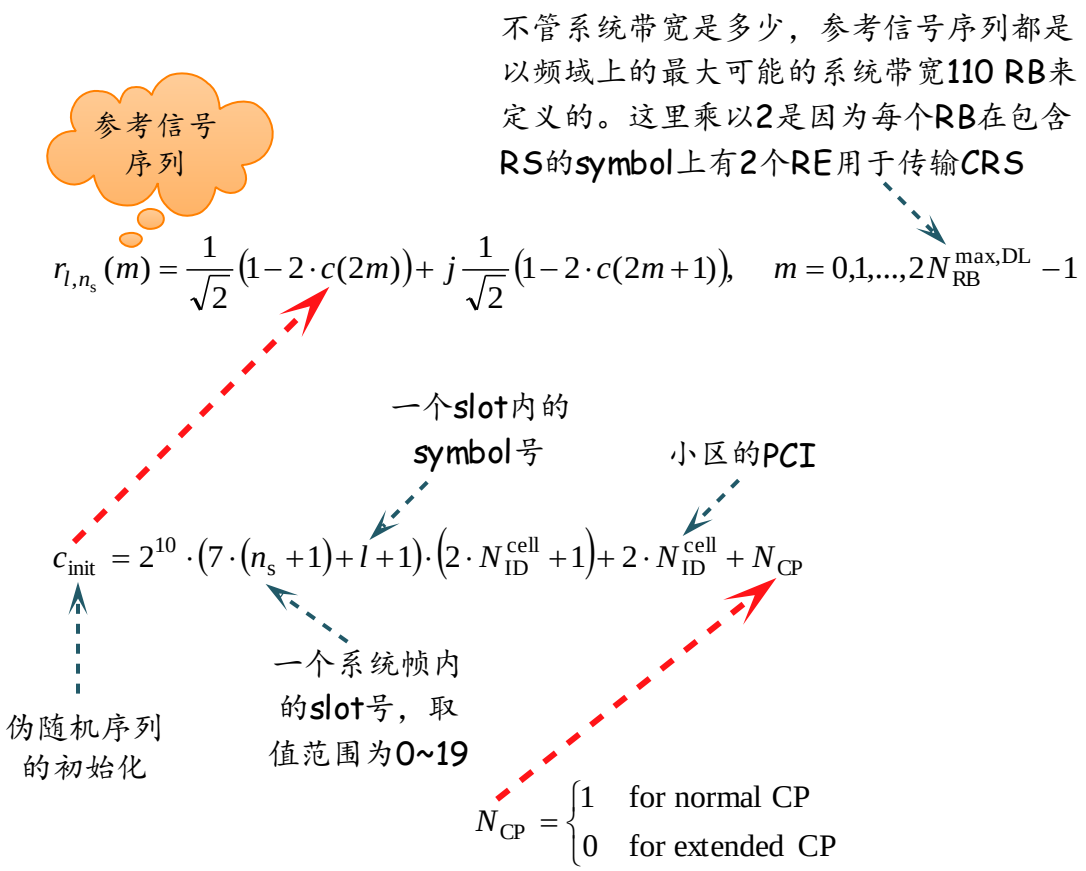


图 3-1：小区特定的参考信号序列的生成

从图 3-1 可以看出，小区特定的参考序列的生成与小区的 PCI（Physical Cell ID）、一个系统帧内的 slot 号、一个 slot 内的 symbol 号以及循环前缀的类型相关。这个序列的变化周期为 10 ms，即一个系统帧。小区特定的参考信号由带预定义值（主要由 PCI 决定）的一系列参考符号（reference symbol）组成。

不管下行系统带宽是多少，参考信号序列都是以频域上的最大可能的系统带宽 110 RB 来定义的。

接下来，我们来介绍如何将小区特定的参考符号映射到 RE 上。

首先我们来介绍只使用 1 个小区特定的参考信号（对应单天线端口 0）的情况。

对于只使用 1 个小区特定的参考信号的情况，参考符号会插入到每个 slot 的第 1 个和倒数第 3 个 OFDM 符号中，同一 OFDM 符号内相邻的 2 个参考符号在频域上间隔 6 个子载波。同时，倒数第 3 个 OFDM 符号中的参考符号和第一个 OFDM 符号中的参考符号在频域上间隔 3 个子载波。此时一个 RB pair 内有 8 个 RE 用于发送小区特定的参考信号。如图 3-2 所示。

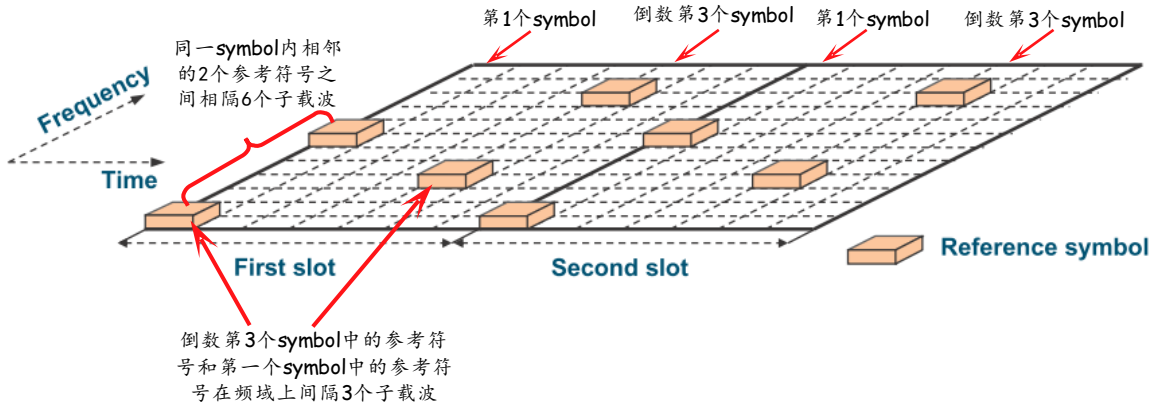


图 3-2：使用 1 个小区特定的参考信号时，一个 RB pair 内的的结构（正常的循环前缀）

参考符号在每个 RB 内的起始位置还和小区特定的频率偏移（frequency shift）相关。LTE 定义了 6 个频率偏移，该偏移与小区的 PCI 相关，其值为 PCI mod 6（同样适用于 2 个和 4 个小区特定的参考信号的情况），如图 3-3 所示。这个频率偏移可以避免至多 6 个相邻小区的小区特定的参考信号之间的时频资源冲突。

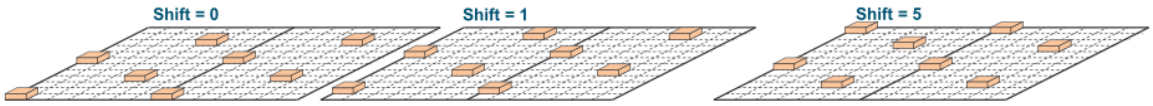


图 3-3：频率偏移

通过给不同的小区分配合适的 PCI，可使邻居小区之间拥有不同的频率移位。为了有效地提高参考信号的 SIR，可以让参考信号比其他的 RE 拥有更高的能量（相比周围的数据符号，参考符号可以有至多 6 dB 的功率提升），即“参考信号功率提升”。但如果相邻小区的参考信号使用相同的时频资源，则某个小区上提升了功率的参考信号将会被所有相邻小区同样提升了功率的参考信号所干

扰，这样就抵消了功率提升带来的参考信号 SIR 的增益。如果此时相邻小区的参考信号之间使用不同的频率偏移的话，则某个小区的参考信号只会被邻居小区的非参考信号的符号（没有功率提升）所干扰，这样参考信号功率提升就能够提高参考信号的 SIR。这也是为什么在规划 PCI 时，需要避免 “模 6 干扰”的原因。

接下来，我们将小区特定的参考信号扩展到使用 2 或 4 个天线端口的情况。

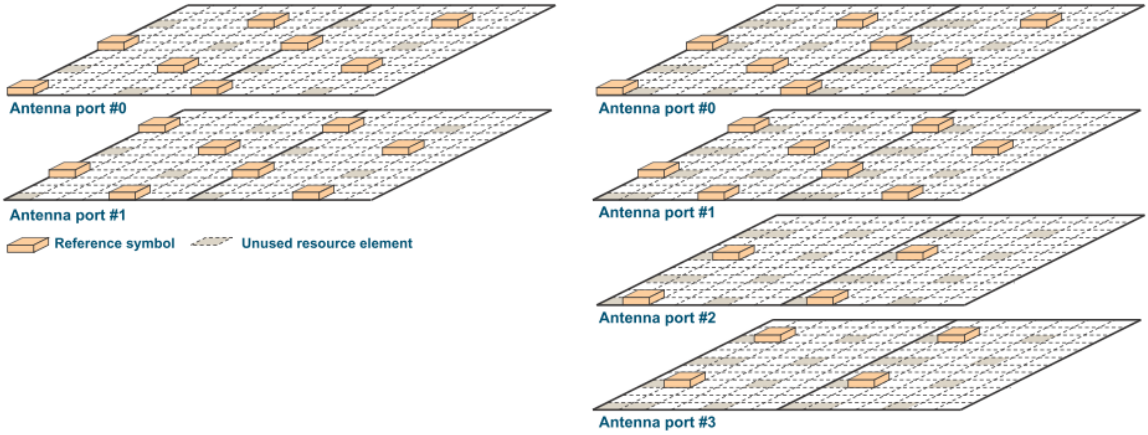


图 3-4：使用 2 个或 4 个小区特定的参考信号时，一个 RB pair 内的的结构（正常的循环前缀）

图 3-4 是一个小区使用 2 个和 4 个小区特定的参考信号时的结构。

- 当小区使用 2 个参考信号时（对应图 3-4 的左边部分），第 1 个天线端口上（对应 port 0）的参考信号与第 2 个天线端口上（对应 port 1）的参考信号在频域上复用，二者在频域上偏移了 3 个子载波。
- 当小区使用 4 个参考信号时（对应图 3-4 的右边部分），第 3 个天线端口上（对应 port 2）的参考信号和第 4 个天线端口上（对应 port 3）的参考信号在频域上复用，二者在频域上偏移了 3 个子载波。且第 3 个和第 4 个天线端口上的参考信号在每个 slot 的第 2 个 OFDM 符号上传输，这样就与第 1 个和第 2 个天线端口上的参考信号在时域上进行了复用。（注：图 3-4 中，port #2 和 port #3 上的参考信号在频域上的位置是错误的，正确的位置请参考 36.211 的 Figure 6.10.1.2-1）

为了降低参考信号的开销，第 3 和第 4 个天线端口上的参考符号密度是第 1 和第 2 个天线端口的一半。

每个天线端口上，被小区特定的参考信号占据的 RE 都不能用于其它传输。与此同时，如果某个天线端口上的某个 RE 被用于发送参考信号，则其它天线端口上时频位置相同的 RE 上不能传输任何东西。这是为了避免小区特定的参考信号被其它天线端口上的传输所干扰，例如空分复用这样的多天线传输，很大程度上依赖于良好的信道估计以在接收端抑制不同层间的干扰。但信道估计本身并没有这样的抑制，因此降低对一个天线端口上的参考信号的干扰，对于获得好的信道估计以及在接收端获得较好的干扰抑制是很重要的。

图 3-5 从协议角度详细地描述了如何将小区特定的参考信号序列映射到 RE 上。

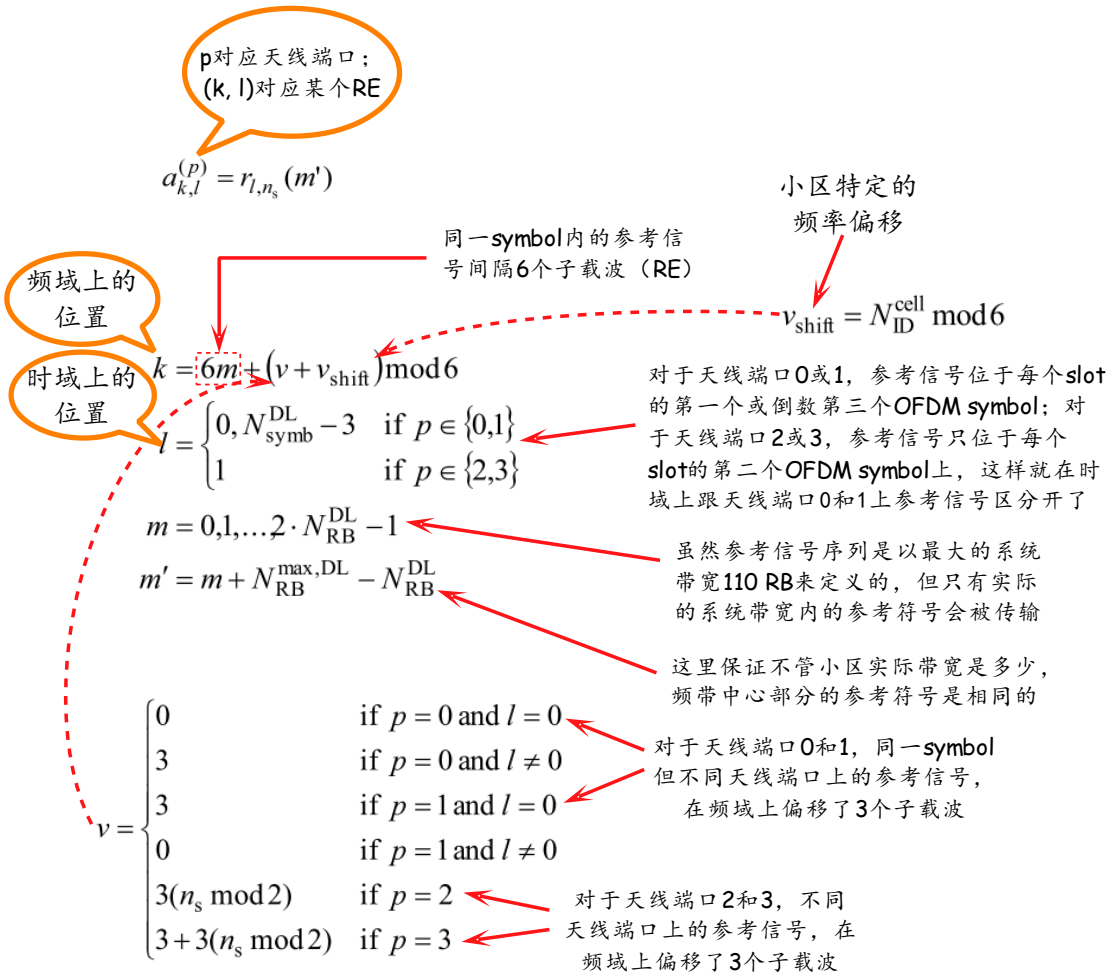


图 3-5：小区特定的参考信号如何映射到 RE

36.211 的 Figure 6.10.1.2-1 给出了正常的循环前缀下，小区特定的参考信号映射到 RE 的一个例子（PCI mod 6 = 0）。其中 R_p 表示用于传输参考信号的 RE 位于天线端口 p 上。

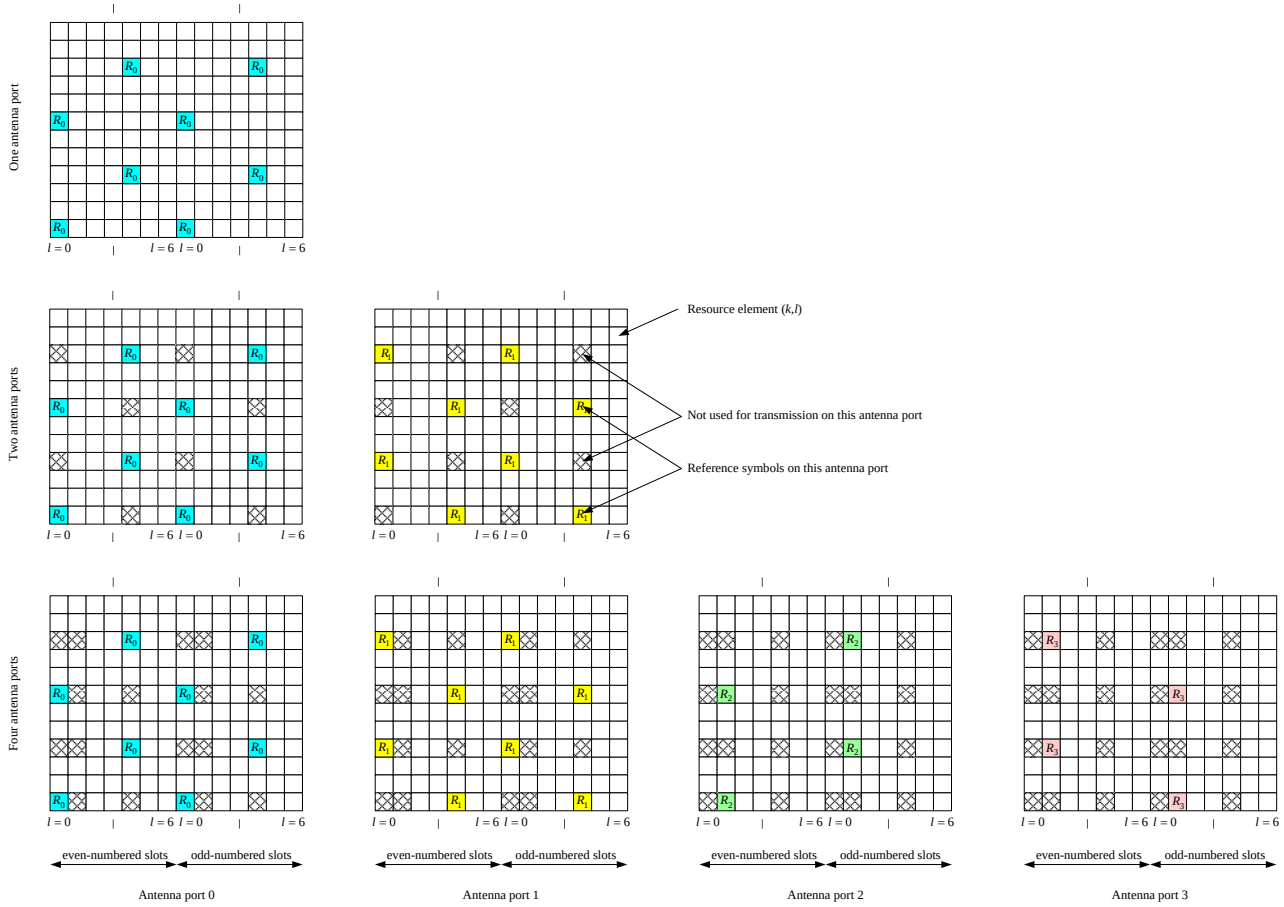


Figure 6.10.1.2-1. Mapping of downlink reference signals (normal cyclic prefix).

在第 19 章介绍“小区搜索过程”时我们会看到，通过小区搜索过程，UE 可以得到小区的 PCI、10 ms timing（也就知道子帧 0 所在的位置，进而也就知道了每个 slot 号所在的位置）、循环前缀配置（也就知道了使用的是正常的循环前缀还是扩展的循环前缀，以及每个 symbol 号所在的位置），由于小区特定的参考信号及其时频位置与 PCI 是一一对应的，因此在小区搜索过程之后，UE 也就知道了该小区的小区特定的参考信号序列及其时频位置。

在 20.1 节介绍“MIB” 时我们会看到，通过接收 MIB，UE 又知道了小区特定的天线端口数（1，2 或 4），从而知道了小区使用多少个小区特定的参考信号，也即小区特定的参考信号会在哪几个天线端口上传输。

需要强调的是，本节介绍的参考信号和参考符号都是针对小区特定的参考信号（符号）而言的。

【参考资料】

[1] TS 36.211 的 6.10.1 节 Cell-specific reference signals

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.2.1 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 8.2 节

3.2 UE 特定的参考信号（UE-specific Reference Signal）

UE 特定的参考信号又称为解调参考信号（DeModulation Reference Signal，简称 DMRS），该参考信号只在分配给某个特定 UE 的 RB 上传输，并只对该 UE 有效。UE 特定的参考信号只用于配置了 TM 7/8/9 的 UE 的 PDSCH 信道估计，即应用于基于非码本的预编码的场景。

“UE 特定”的意思是每个 UE 特定的参考信号只能用于单个 UE，不同 UE 之间是不同的（或者说是可区分的）。

UE 特定的参考信号与 PDSCH 传输使用相同的预编码，因此 eNodeB 不需要告诉 UE 所使用的预编码信息。

在 Rel-8 中，UE 特定的参考信号只能用于单层传输（TM 7），即不支持空分复用，每个 UE 至多只有一个 UE 特定的参考信号。在 Rel-9 中，基于 UE 特定的参考信号的传输扩展到支持至多 2 层传输（TM 8），对应同时传输至多 2 个参考信号，每层各一个。在 Rel-10 中，基于 UE 特定的参考信号的传输扩展到支持至多 8 层传输（TM 9），对应同时传输至多 8 个参考信号。

在 Rel-9 中，UE 特定的参考信号的结构并不是直接扩展自 Rel-8 中定义的 UE 特定的参考信号，而是重新定义了一个结构，以支持单层和双层传输。而在 Rel-10 中，直接扩展自 Rel-9 中定义的结构，就可支持至多 8 层的传输。

UE 特定的参考信号与小区特定的参考信号在不同的天线端口上传输，并在时频位置上使用不同的 RE，因此 UE 特定的参考信号不会影响小区特定的参考信号相关的特性。

接下来，我们会分别介绍 Rel-8 和 Rel-9/Rel-10 中定义的 UE 特定的参考信号。

3.2.1 Rel-8 中的 UE 特定的参考信号

Rel-8 中定义的 UE 特定的参考信号只用于 TM 7，并只在天线端口 5（port 5）上传输。同一小区的不同 UE 对应的 UE 特定的参考信号位于不同的频域资源上，彼此之间互不干扰，处理起来也比较简单。

Rel-8 中定义的 UE 特定的参考信号序列由一系列的参考符号组成，每个参考符号占一个 RE。图 3-6 给出了 UE 特定的参考信号序列的生成方式。

Rel-8 中定义的 UE 特定的参考信号的初始化与 UE 的 RNTI 相关，并且会在每个子帧开始时进行重新初始化。

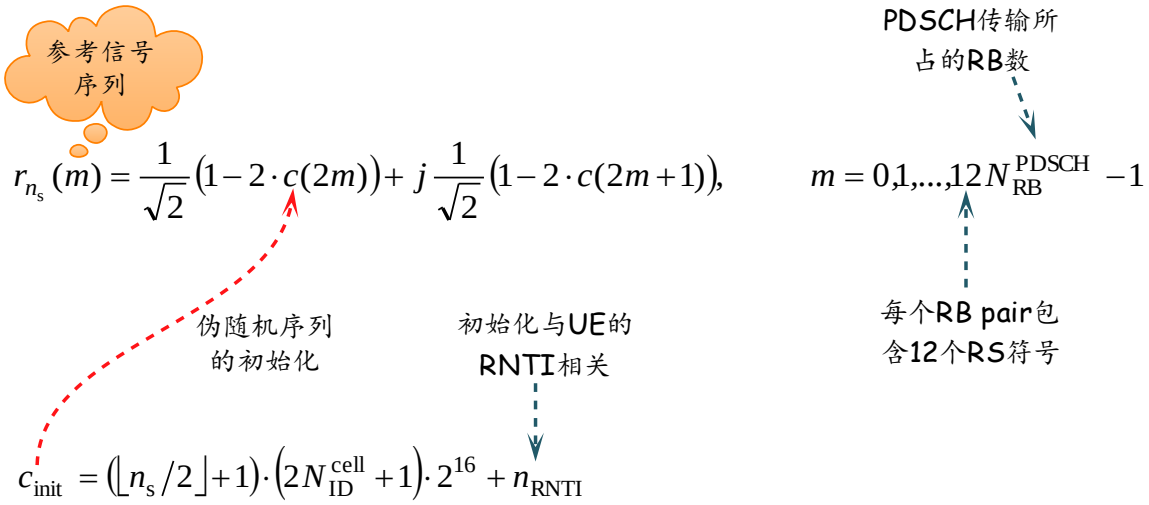


图 3-6：UE 特定的参考信号序列的生成（port 5）

Rel-8 中定义的 UE 特定的参考信号映射到 RE 的方式如图 3-7 所示（正常的循环前缀）。与小区特定的参考信号类似，UE 特定的参考信号在每个 RB 内的起始位置也和小区特定的频率偏移相关。该偏移与小区的 PCI 相关，其值为 PCI mod 3。

扩展的循环前缀下，Rel-8 中定义的 UE 特定的参考信号映射到 RE 的方式可参见 36.211 的 6.10.3 节，这里就不予以介绍了。

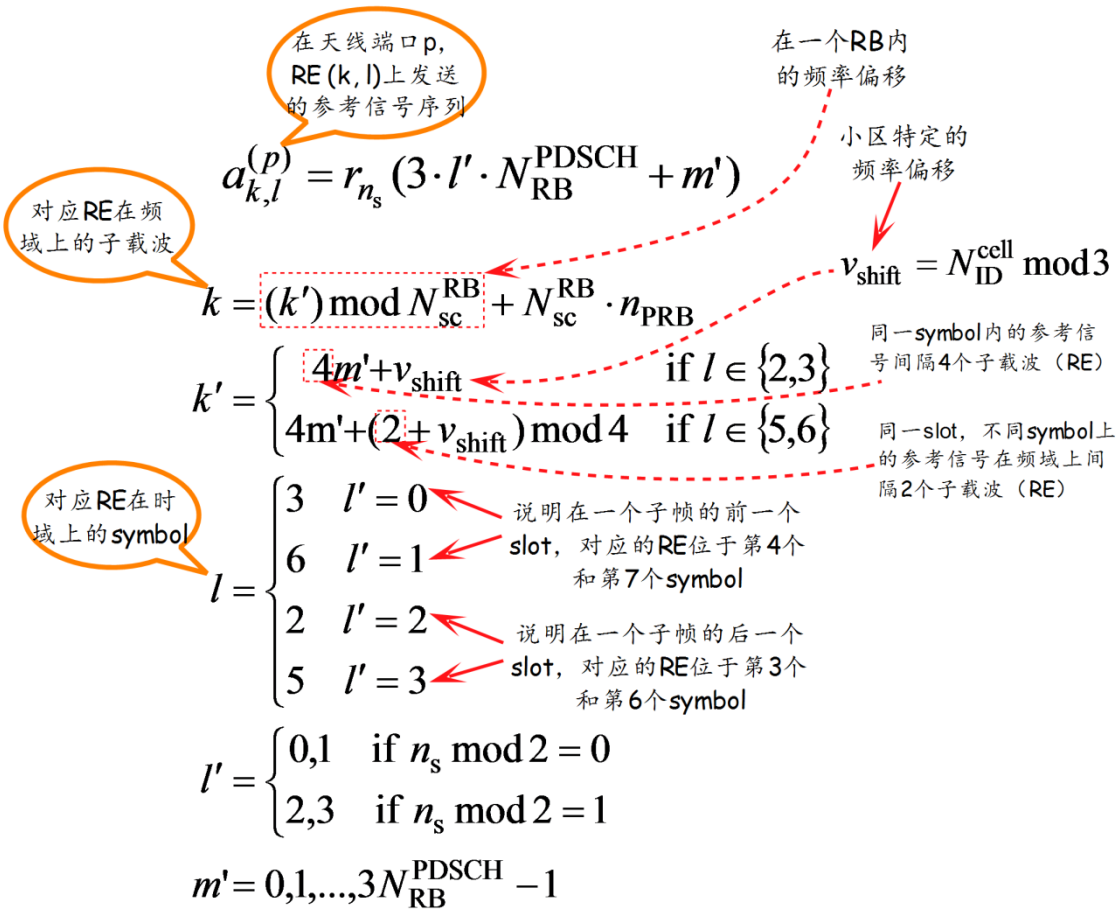


图 3-7：UE 特定的参考信号如何映射到 RE（port 5，正常的循环前缀）

36.211 的 Figure 6.10.3.2-1 给出了正常的循环移位下，UE 特定的参考信号（port 5）映射到 RE 的一个例子（PCI mod 3 = 0）。其中 R_p 表示用于传输参考信号的 RE 位于天线端口 p 上。（扩展的循环前缀对应的例子可见 36.211 的 Figure 6.10.3.2-2，其 PCI mod 3 = 0）

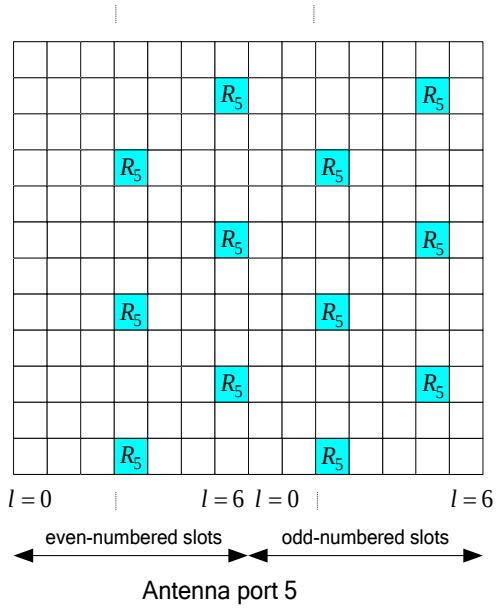


Figure 6.10.3.2-1: Mapping of UE-specific reference signals, antenna port 5 (normal cyclic prefix).

3.2.2 Rel-9/ Rel-10 中的 UE 特定的参考信号

Rel-9/Rel-10 中定义的 UE 特定的参考信号序列用于 TM 8/TM 9 中，并只在 port 7~14 上传输。

前面已经介绍过，Rel-9/Rel-10 中定义的 UE 特定的参考信号序列与 Rel-8 中不同，其序列的生成公式如图 3-8 所示：

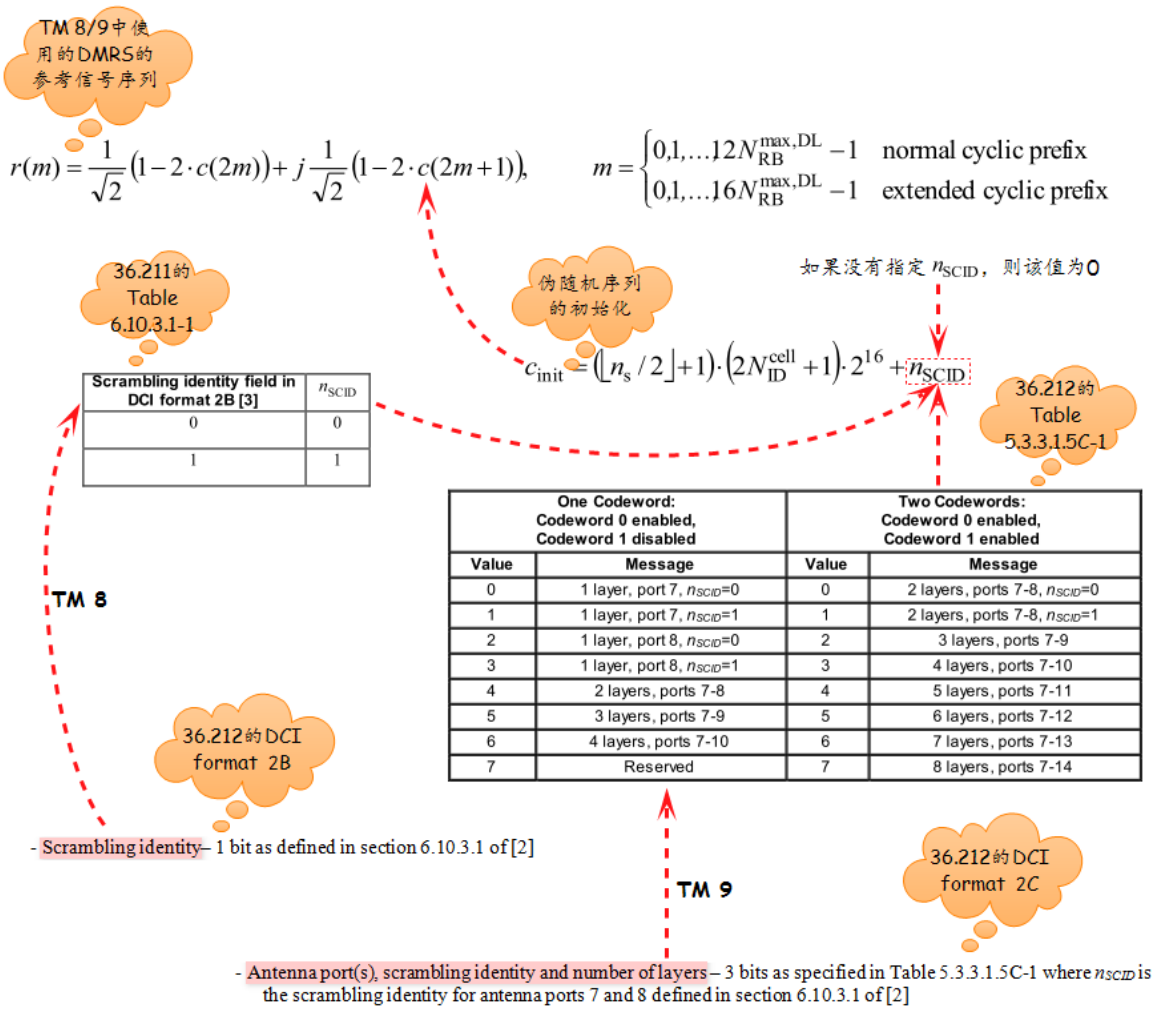


图 3-8：TM 8/9 中，UE 特定的参考信号序列的生成

与 Rel-8 的 UE 特定的参考信号相比，Rel-9/ Rel-10 中并没有基于 PCI 的频率偏移。

与 Rel-8 的 UE 特定的参考信号相比，Rel-9/ Rel-10 中伪随机序列的初始化与 UE 的 RNTI 无关，这是为了使正交码分复用的多个 UE 特定的参考信号能被用在 MU-MIMO 上。

Rel-9 中定义的 TM 8 支持至多 2 层传输，并使用天线端口 7~ 8。其 DCI format 2B 中有一个 1 比特的“Scrambling identity”字段用于指定只用于 port 7 和 port 8 的加扰 ID： n_{SCID} （见图 3-8）。如果 2 个 TB 都使能，则使用的层数为 2（即 port 7 和 8 都被用于空分复用）；如果只使能了 1 个 TB，则使用的层数为 1（只能使用 port 7 或 8 中的一个），此时 UE 使用哪个天线端口与去使能的那个 TB 对应的“New data indicator”字段相关，如表 3-1 所示。

表 3-1：TM 8 中，单层传输所使用的天线端口与去使能的 TB 对应的 New data indicator 字段的关系

去使能的 TB 对应的 New data indicator 字段	天线端口
0	7
1	8

Rel-10 中定义的 TM 9 支持至多 8 层传输，并使用天线端口 7~14。其 DCI format 2C 中有一个 3 比特的“Antenna port(s), scrambling identity and number of layers”字段，eNodeB 通过该字段告诉 UE 该子帧的 PDSCH 传输使用的天线端口和层数，以及只用于 port 7 和 port 8 的加扰 ID：n_{SCID}（对于其它天线端口，$n_{SCID}=0$），见图 3-9（对应 36.212 的 Table 5.3.3.1.5C-1）。如果只使能了一个 codeword，则 Value = 4、5、6 只应用于对应 TB 的重传，且该 TB 的前一次传输使用 2、3 或 4 层传输。

- Antenna port(s), scrambling identity and number of layers – 3 bits as specified in Table 5.3.3.1.5C-1 where n_{SCID} is the scrambling identity for antenna ports 7 and 8 defined in section 6.10.3.1 of [2]

Table 5.3.3.1.5C-1: Antenna port(s), scrambling identity and number of layers indication

One Codeword: Codeword 0 enabled, Codeword 1 disabled		Two Codewords: Codeword 0 enabled, Codeword 1 enabled	
Value	Message	Value	Message
0	1 layer, port 7, n_{SCID}=0	0	2 layers, ports 7-8, n_{SCID}=0
1	1 layer, port 7, n_{SCID}=1	1	2 layers, ports 7-8, n_{SCID}=1
2	1 layer, port 8, n_{SCID}=0	2	3 layers, ports 7-9
3	1 layer, port 8, n_{SCID}=1	3	4 layers, ports 7-10
4	2 layers, ports 7-8	4	5 layers, ports 7-11
5	3 layers, ports 7-9	5	6 layers, ports 7-12
6	4 layers, ports 7-10	6	7 layers, ports 7-13
7	Reserved	7	8 layers, ports 7-14

如果只使能了一个codeword，则Value = 4、5、6只应用于对应TB的重传，且该TB的前一次传输使用2或3或4层传输。

知道了所使用的层数、天线端口以及加扰ID，也就知道了UE特定的参考信号的结构

图 3-9：TM 9 中， DCI format 2C 中的 Antenna port(s), scrambling identity and number of layers 字段

Rel-10（TM 9）中支持至多 8 层传输的 UE 特定的参考信号序列与 Rel-9（TM 8）中新定义的支持至多 2 层传输的 UE 特定的参考信号序列相同，但天线端口 9~14 只有 1 种初始化序列，因为其 $n_{SCID}=0$。

可以看出，TM 8/9 中，一个小区只定义了 2 个 UE 特定的参考信号序列，并通过 n_{SCID} 进行区分（对于一个小区，PCI 固定，同一子帧的子帧号相同）。n_{SCID} 使得非正交的 UE 特定的参考信号的复用成为可能，例如，每个 UE 至多分配 2 层传输的 MU-MIMO。

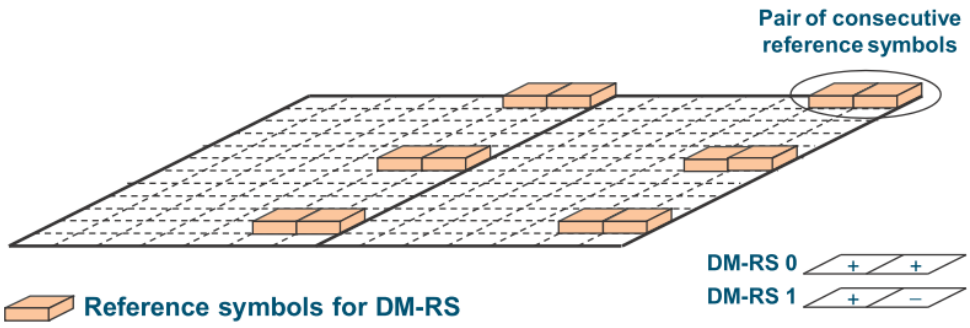


FIGURE 10.11

Structure of demodulation reference signals (DM-RS) for the case of one or two reference signals, including orthogonal cover codes to separate the two reference signals.

图 3-10：Rel-9（TM 8）中，1 或 2 个 UE 特定的参考信号到 RE 的映射

图 3-10 表示了 Rel-9 中支持至多 2 层传输（TM 8，使用 port 7 和 port 8）的 UE 特定的参考信号在一个 RB pair 内的结构。可以看出，每个 RB pair 中包含了 12 个参考符号。在小区特定的参考信号中，如果某个天线端口上某个 RE 被用于参考符号，则在另一个天线端口上，该 RE 不能被使用。而 UE 特定的参考信号则不同，12 个参考符号在 2 个天线端口上使用相同的 RE 来传输，并通过正交覆盖码（Orthogonal Cover Code，OCC）进行区分（见图 3-10 的右下角），从而抑制了不同天线端口上传输的 UE 特定的参考信号之间的干扰。TM 8 中每个天线端口使用的 OCC 见 36.211 的 Table 6.10.3.2-1（对应天线端口 7 和 8 的项）。

Table 6.10.3.2-1: The sequence $\overline{w}_p(i)$ for normal cyclic prefix.

Antenna port p	$\left[\overline{w}_p(0) \quad \overline{w}_p(1) \quad \overline{w}_p(2) \quad \overline{w}_p(3)\right]$
7	$\left[+1 \quad +1 \quad +1 \quad +1\right]$
8	$\left[+1 \quad -1 \quad +1 \quad -1\right]$
9	$\left[+1 \quad +1 \quad +1 \quad +1\right]$
10	$\left[+1 \quad -1 \quad +1 \quad -1\right]$
11	$\left[+1 \quad +1 \quad -1 \quad -1\right]$
12	$\left[-1 \quad -1 \quad +1 \quad +1\right]$
13	$\left[+1 \quad -1 \quad -1 \quad +1\right]$
14	$\left[-1 \quad +1 \quad +1 \quad -1\right]$

除了 OCC，还可以将一个伪随机序列应用到参考信号上，对于同一个 UE 的 2 个参考信号来说，这个序列是相同的（该伪随机序列与 n_{SCID} 相关），因此不影响正交性。但是如果使用 MU-MIMO，不同的 UE 可以使用不同的伪随机序列（分配不同的 n_{SCID} ），以区分不同 UE 在同一 RB 上的传输。

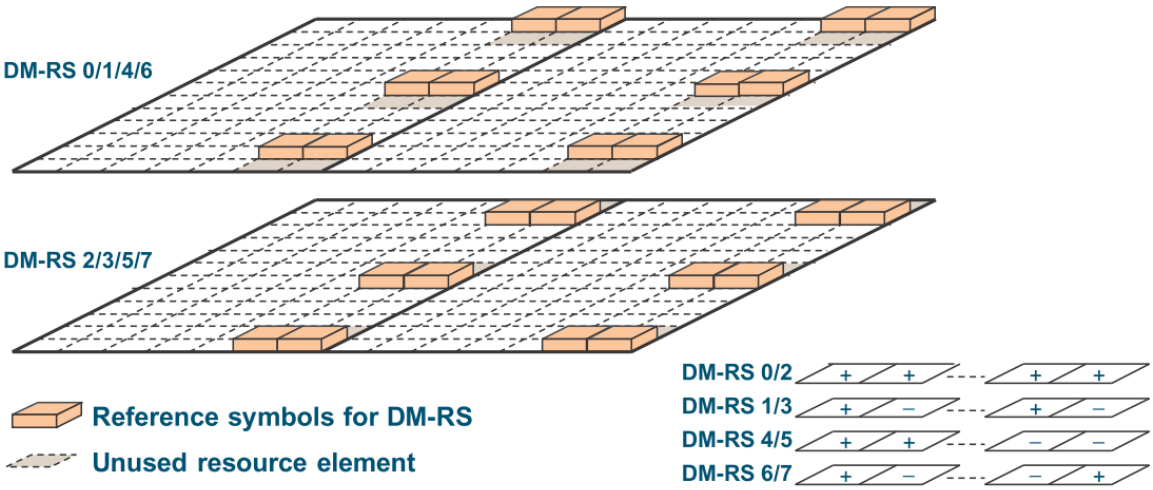


FIGURE 10.12

Demodulation-reference-signal structure for the case of more than two reference signals.

图 3-11：TM 9 中，至多 8 个 UE 特定的参考信号到 RE 的映射

图 3-11 表示了 Rel-10 中支持至多 8 层传输（TM 9，对应 8 个天线端口，port 7~15）的 UE 特定的参考信号的结构，该结构扩展自 Rel-9 中定义的 UE 特定的参考信号。可以看出，每个 RB pair 中包含了 24 个参考符号。根据频域位置的不同，8 个参考信号可以分成 2 组，每组包含 4 个参考信

号，如图 3-11 所示，DMRS 0/1/4/6（对应天线端口 7/8/11/13）为一组，DMRS 2/3/5/7（对应天线端口 9/10/12/14）为另一组。同一组内的参考信号占用相同的 RE 资源，彼此之间是通过不同的正交覆盖码 OCC 来区分的（见 36.211 的 Table 6.10.3.2-1）。正交覆盖码 OCC 应用于同一子帧上的频域位置相同（使用相同的子载波）但时域位置不同（不同的 OFDM 符号）的 4 个 RE 上。不同组的参考信号占用不同的 RE 资源，因此与另一组参考信号互不干扰。

为了保持这 8 个参考信号之间的正交性，要求限制信道的变化，因此超过 4 层的传输通常只用于低移动性的场景。

从前面的介绍可以看出，对于使用 TM 7（对应天线端口 5）的多个 UE，不同 UE 对应的 DMRS 是通过不同的频域资源（不同 UE 分配了不同的 RB）进行区分的。

对于使用 TM 8（对应天线端口 7 和 8）的多个 UE，如果使用 SU-MIMO，则不同 UE 对应的 DMRS 是通过不同的频域资源（不同 UE 分配了不同的 RB）进行区分的，而同一 UE 的不同天线端口上的 2 个 DMRS 是通过使用不同的 OCC 进行区分的；如果使用 MU-MIMO，则 2 个 UE 使用相同的时频资源，不同的 UE 对应的 DMRS 是通过使用不同的 OCC 和 n_{SCID} 组合进行区分的。

对于使用 TM 9（对应天线端口 7~14）的多个 UE，如果使用 SU-MIMO，则不同 UE 对应的 DMRS 是通过不同的频域资源（不同 UE 分配了不同的 RB）进行区分的，而同一 UE 的不同天线端口间的多个 DMRS 是通过不同的频域资源（不同组的天线端口使用不同的子载波）或不同的 OCC（同组的天线端口使用不同的 OCC）进行区分的；如果使用 MU-MIMO（此时只能使用天线端口 7 和 8），则 2 个 UE 使用相同的时频资源，不同的 UE 对应的 DMRS 是通过不同的 OCC 和 n_{SCID} 组合进行区分的。

注：关于 Rel-9/Rel-10 中定义的 UE 特定的参考信号到 RE 的映射过程，可参见 36.211 的 6.10.3 节，计算出的结果如图 3-10 和图 3-11 所示，这里就不再画图说明了。

在 TDD 中，由于特殊子帧中的 DwPTS 的时间跨度比正常子帧要短，因此其 UE 特定的参考信号有一些变化，这可以参见 36.211 的 6.10.3 节。同时，本节也没有介绍扩展的循环前缀下的 UE 特定的参考信号的结构，感兴趣的可以去阅读 36.211 的 6.10.3 节。

UE 特定的参考信号所占的 RE 是不能用于传输 PDSCH 的，即在 PDSCH 符号映射到对应的 RB 时，那些被参考信号所占的 RE 是不能被在**任意**天线端口上传输的 PDSCH 所使用的。但之前版本定义的 UE 并不能识别后续版本定义的 UE 特定的参考信号，也不知道哪些 RE 被后续版本定义的 UE 特定的参考信号所占用，例如 Rel-8 的 UE 并不能识别 Rel-9/ Rel-10 中定义的 UE 特定的参考信号。因此 UE 特定的参考信号只能在那些支持同一类型的参考信号的 UE 所分配的 RB 上传输。例如，Rel-8 的 UE 不能被调度在传输了 Rel-9/ Rel-10 中定义的 UE 特定的参考信号的那些 RB 上，也即 Rel-8 的 UE 不能跟 Rel-9/ Rel-10 的 UE 一起配对做 MU-MIMO。

传输层数是可以动态变化的，传输的 UE 特定的参考信号的数目也会随之变化，因此 UE 特定的参考信号的结构会动态地发生变化。eNodeB 需要在对应的 DCI 中将传输层数通知给 UE，以便 UE 知道当前子帧使用的 UE 特定的参考信号的结构，以及 PDSCH 如何映射到 RE。

【参考资料】

[1] TS 36.211 的 6.10.3 节 UE-specific reference signals

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.2.2 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的第 8 章和 29.1.1 节

3.3 CSI 参考信号（CSI-RS）

CSI-RS 是在 Rel-10 才引入的。CSI-RS 只应用于 TM 9（确切地说，要求配置了 TM9 的同时，配置了 PMI/RI 上报）。在 TM 9 中，CSI-RS 被 UE 用于获取 CSI，而 UE 特定的参考信号被用于信道估计。与小区特定的参考信号（也可用于获取 CSI）相比，CSI-RS 的时频密度更低，因此开销更小。

CSI-RS 的主要目的在于支持至多 8 个天线端口的信道质量反馈。前面已经介绍过，小区特定的参考信号也能用于获取 CSI，但其至多只有 4 个，只支持至多 4 个天线端口，如果扩展至 8 个天线端口，会导致其开销过大，所以 Rel-10 新增了 CSI-RS 来支持配置了 TM 9 的 UE 的 CSI 测量。

CSI-RS 还可以使得 UE 能够估计多个小区的 CSI，而不仅仅是其所在的 serving cell 的 CSI，以支持多小区合作传输模式（CoMP）。

CSI-RS 有正常的 CSI-RS（normal CSI-RS）和 muted CSI-RS 之分。正常的 CSI-RS 在其占用的 RE 上发送真正的 CSI-RS，即在对应 RE 上是有发射功率的。Muted CSI-RS 在其占用的 RE 上并不发送任何东西。Muted CSI-SR 可以看做是零发射功率的正常 CSI-RS。

Rel-10 中支持在 1、2、4 或 8 个天线端口上传输 CSI-RS。CSI-RS 只在天线端口 15~22 中的一个或几个中传输。

CSI-RS 只支持 $\Delta f = 15\,\text{kHz}$。

CSI-RS 参考信号序列由一系列的参考符号组成，每个参考符号占一个 RE。CSI-RS 的参考信号序列的生成方式如图 3-12 所示：

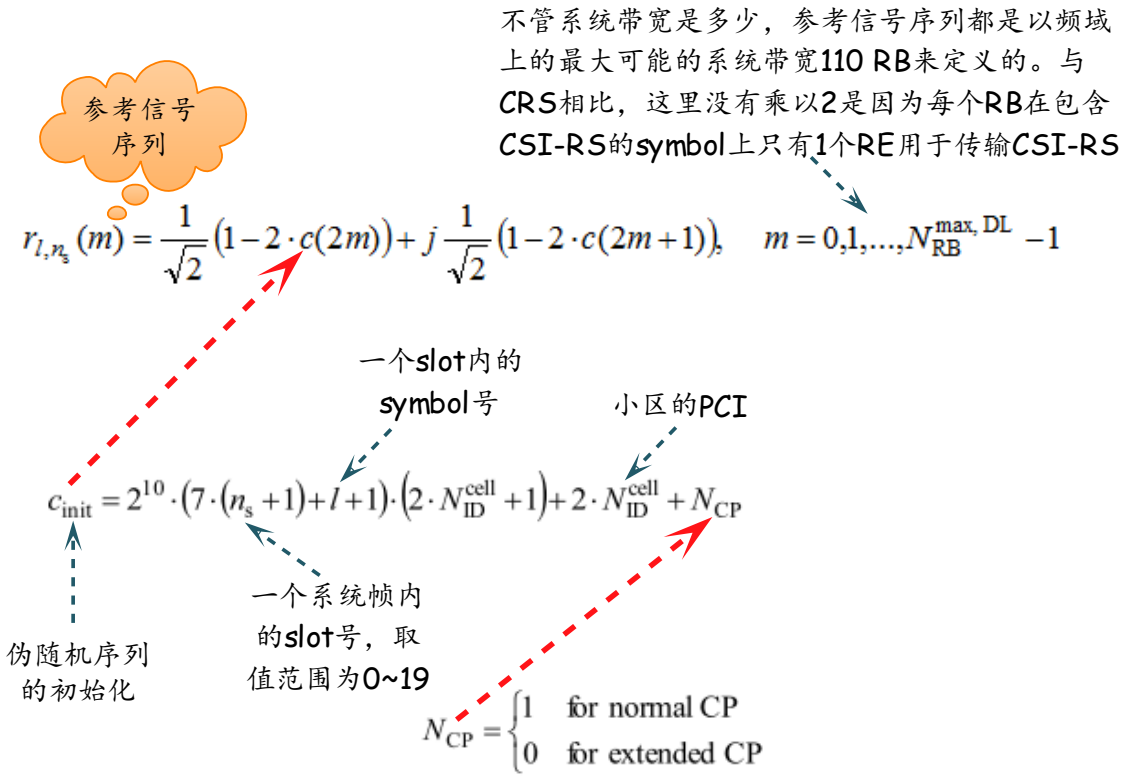


图 3-12：CSI-RS 的参考信号序列生成

从图 3-12 可以看出，与小区特定的参考序列类似，CSI-RS 的生成与小区的 PCI、一个系统帧内的 slot 号、一个 slot 内的 symbol 号以及循环前缀的类型相关。这个序列的变化周期为 10 ms，即一个系统帧。CSI-RS 由带预定义值（主要由 PCI 决定）的一系列参考符号（reference symbol）组成。

不管下行系统带宽是多少，CSI-RS 的参考信号序列都是以频域上的最大可能的系统带宽 110 RB 来定义的。但每个天线端口的每个 RB 在包含 CSI-RS 的每个 OFDM 符号上只有 1 个 RE 能用于传输 CSI-RS。

eNodeB 通过 *CSI-RS-Config-r10* 来配置 UE 的 CSI-RS，该配置是 UE 级别的配置。

```
CSI-RS-Config-r10 ::=       SEQUENCE {------UE级别的配置
    csi-RS-r10                  CHOICE {------用于非零发射功率CSI-RS（normal CSI-RS）
        release                     NULL,
        setup                       SEQUENCE {
            antennaPortsCount-r10           ENUMERATED {an1, an2, an4, an8},------用于传输
CSI-RS的天线端口数
```

```
            resourceConfig-r10              INTEGER (0..31),------CSI-RS配置，见36.211的Table
6.10.5.2-1和Table 6.10.5.2-2

            subframeConfig-r10              INTEGER (0..154), ------ I_CSI–RS ，见36.211的Table
6.10.5.3-1

            p-C-r10                         INTEGER (-8..15)------ P_c ，见36.213的7.2.5节
        }
    }                                                                   OPTIONAL,           -
- Need ON
    zeroTxPowerCSI-RS-r10       CHOICE {------用于零发射功率CSI-RS（muted CSI-RS）
        release                     NULL,
        setup                       SEQUENCE {
            zeroTxPowerResourceConfigList-r10   BIT STRING (SIZE (16)), ------见36.211的
6.10.5.2节中介绍的ZeroPowerCSI-RS

            zeroTxPowerSubframeConfig-r10       INTEGER (0..154) ------ I_CSI–RS ，见36.211的
Table 6.10.5.3-1

        }
    }                                                                   OPTIONAL            -
- Need ON
}
```

接下来，我们从时域、频域和空间域这几个维度来说明 eNodeB 会周期性地在哪个子帧的哪些 RE 资源以及哪些天线端口上给对应的 UE 发送 CSI-RS，以便 UE 进行 CSI 的测量。

3.3.1 空间域

antennaPortsCount-r10 指定了用于传输 CSI-RS 的天线端口数。确定了天线端口数，也就确定了用于传输 CSI-RS 的天线端口。如表 3-2 所示。

表 3-2：用于 CSI-RS 的天线端口数与使用的天线端口的对应关系

天线端口数	使用的天线端口（port）
1	port 15
2	port 15~16
4	port 15~18
8	port 15~22

3.3.2 时域

在时域上，CSI-RS 是周期性发送，并通过 *subframeConfig-r10*（用于 normal CSI-RS，即非零发射功率 CSI-RS）和 *zeroTxPowerSubframeConfig-r10*（用于 muted CSI-RS，即零发射功率 CSI-RS）进行配置。该值对应 36.211 的 Table 6.10.5.3-1 的 $I_{\text{CSI-RS}}$，$I_{\text{CSI-RS}}$ 指定 CSI-RS 的发送周期 $T_{\text{CSI-RS}}$ 和在周期内的偏移 $\Delta_{\text{CSI-RS}}$，二者共同决定了 CSI-RS 会周期性地在哪个子帧上发送。包含 CSI-RS 的子帧必须满足：$(10n_f + \lfloor n_s/2 \rfloor - \Delta_{\text{CSI-RS}}) \bmod T_{\text{CSI-RS}} = 0$。其中 n_f 为系统帧号，n_s 为一个系统帧内的 slot 号，取值范围为 0~19，$\lfloor n_s/2 \rfloor$ 为子帧号。

从上面的公式可以看出，$\bmod T_{\text{CSI-RS}}$ 保证了每个 UE 对应的 CSI-RS 在每 $T_{\text{CSI-RS}}$ 个子帧只出现一次。$-\Delta_{\text{CSI-RS}}$ 指定了每个 UE 对应的 CSI-RS 在其周期内的第几个子帧发送。

Table 6.10.5.3-1: CSI reference signal subframe configuration.

CSI-RS-SubframeConfig $I_{\text{CSI-RS}}$	CSI-RS periodicity $T_{\text{CSI-RS}}$ (subframes)	CSI-RS subframe offset $\Delta_{\text{CSI-RS}}$ (subframes)
0 – 4	5	$I_{\text{CSI-RS}}$
5 – 14	10	$I_{\text{CSI-RS}} - 5$
15 – 34	20	$I_{\text{CSI-RS}} - 15$
35 – 74	40	$I_{\text{CSI-RS}} - 35$
75 – 154	80	$I_{\text{CSI-RS}} - 75$

图 3-13 是 CSI-RS 周期配置的一个例子，3 个 UE 的周期都为 10ms，但在周期内的子帧偏移各不相同。

注意：图 3-13 只是一个举例，目的是为了说明 CSI-RS 的时域配置。实际部署中，同一个小区内的所有 UE 通常使用相同的 $I_{\text{CSI-RS}}$ 值。

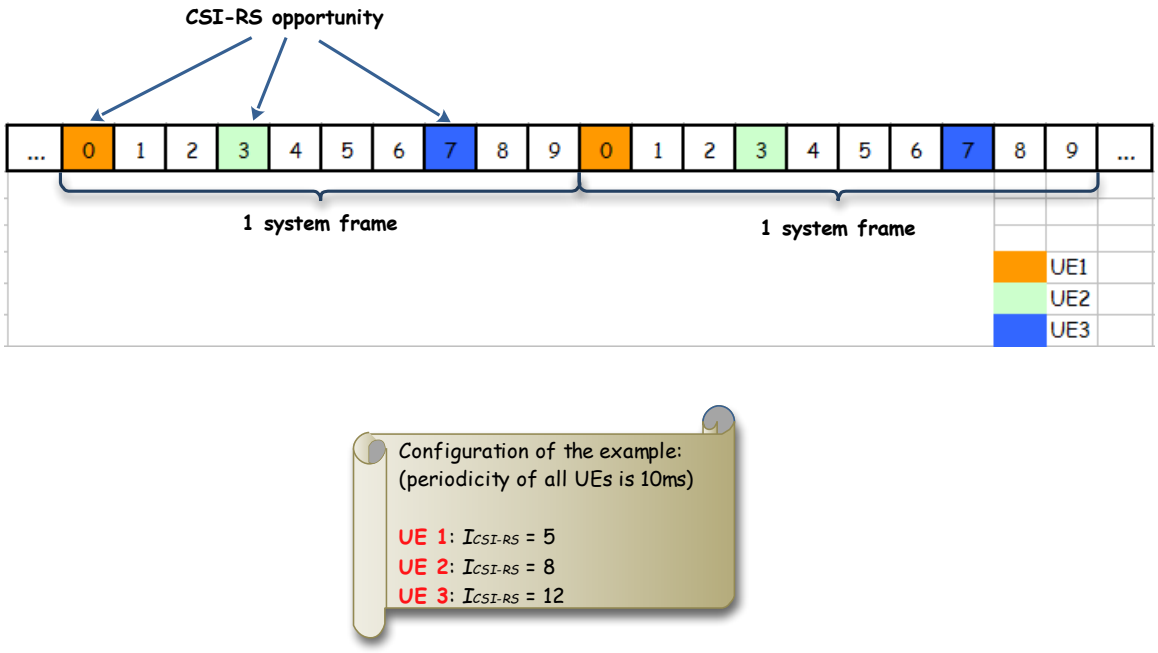


图 3-13：CSI-RS 的周期以及周期内的偏移举例

从 36.211 的 Table 6.10.5.3-1 可以看出，CSI-RS 的发送周期可以配置成 5 ms 到 80 ms。周期越长，意味着 CSI-RS 的开销越小。

CSI-RS 在时域上的哪个 slot 的哪些 symbol 上发送，见下一节介绍。

3.3.3 频域

这里说是频域，实际上是计算 CSI-RS 所占的 RE，即计算 CSI-RS 所占 RE 的 (k,l) 。具体公式见图 3-14：

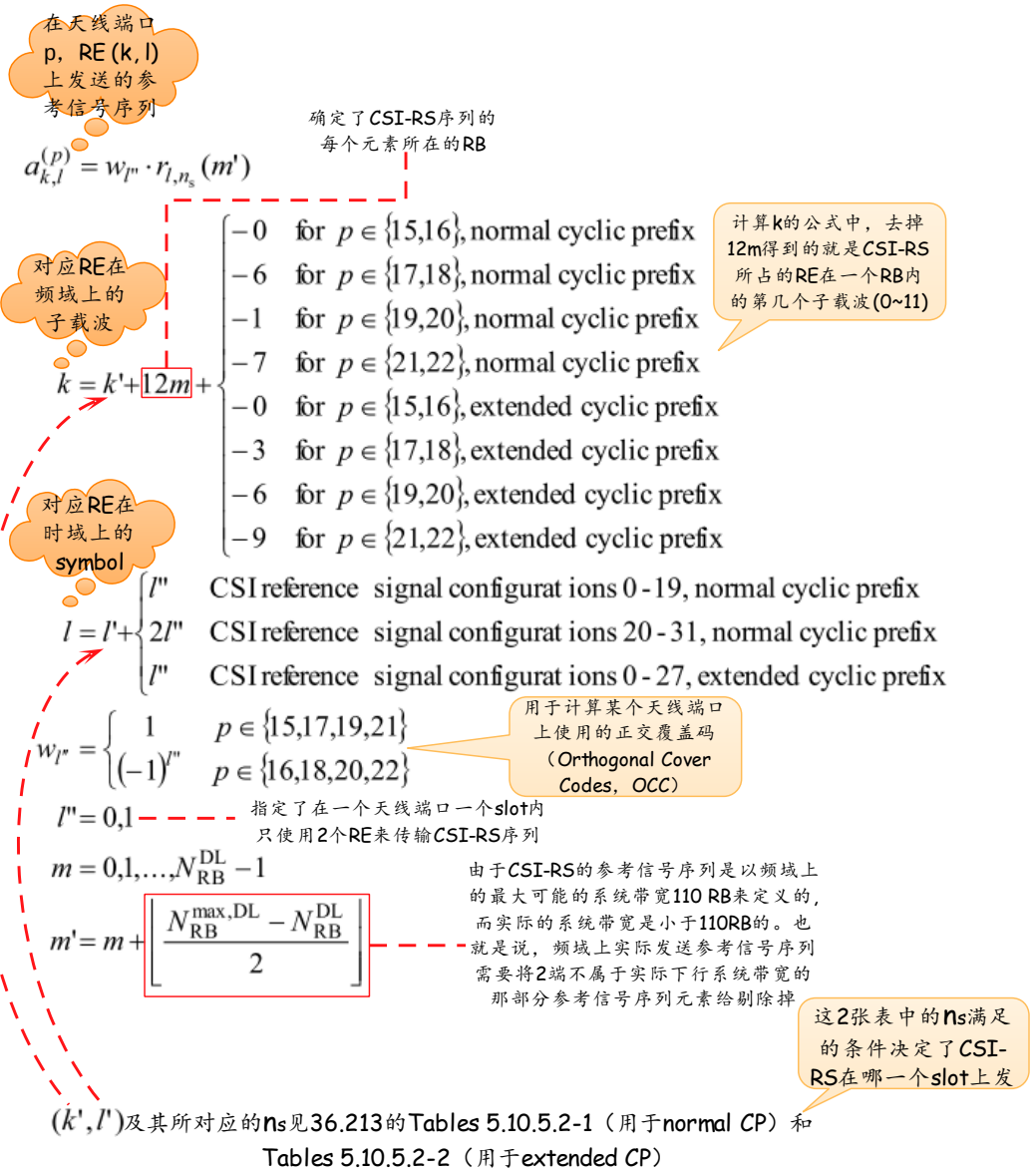


图 3-14：CSI-RS 所占 RE 的映射公式

图 3-14 实际上计算了几部分内容：CSI-RS 所占 RE 的 (k,l)，CSI-RS 在一个子帧内的哪一个 slot 上发送，以及在某个天线端口上使用的正交覆盖码（OCC）。

从公式中可以看出，CSI-RS 在频域上会在每一个 RB 上传输，即 CSI-RS 的传输会覆盖整个下行系统带宽。

从 (k,l) 的计算公式可以看出，CSI-RS 在同一组天线端口“15 和 16”、“17 和 18”、“19 和 20”、“21 和 22”上占用的 RE 是相同的，并且这 4 组天线端口之间是通过占用不同的 RE 集合来区分的。

但同一组天线端口（例如：15 和 16）的 CSI-RS 占用的 RE 集合是相同的，彼此之间要使用正交覆盖码（Orthogonal Cover Codes，OCC）来区分。$w_{l''}$ 就是用来计算正交覆盖码的，通过公式可以看出，正交覆盖码的长度为 2，正好对应一个天线端口在一个 RB 内只使用 2 个 RE 来传输 CSI-RS 序列。最终的结果是在天线端口 $\{15,17,19,21\}$ 上使用的正交覆盖码为 $\{1,1\}$，在天线端口 $\{16,18,20,22\}$ 上使用的正交覆盖码为 $\{1,-1\}$。

(k',l') 和一个子帧上使用的 slot 需要满足的条件是通过 36.211 的 Tables 6.10.5.2-1（用于正常的循环前缀）和 Tables 6.10.5.2-2（用于扩展的循环前缀）的“CSI reference signal configuration”列（这里我将其称为“CIS-RS 配置”）来决定的。“CSI reference signal configuration”又是 eNodeB 通过 *resourceConfig-r10*（用于 normal CSI-RS）和 *zeroTxPowerResourceConfigList-r10*（用于 muted CSI-RS）来配置的。

36.211 的 Figure 6.10.5.2-1 和 Figure 6.10.5.2-2 是“CSI reference signalconfiguration”等于 0 时，CSI-RS 在不同的天线端口上所占的 RE 的一个例子。参照该例子，大家把具体的值代入图 3-14 中的公式，能够对这些公式有更深刻的理解。

在图 3-15 中，我将 4 天线端口 CSI-RS 在各种“CSI reference signal configuration”配置下，在每个 PRB pair 所占的 RE 通过表格的形式都给罗列了出来。

CSI-RS configuration for 4-port (normal CP)

	0	1	2	3	4	5	6	0	1	2	3	4	5	6
11										1	1			
10										6	6			
9						0	0			2	2		4	4
8						5	5			7	7		9	9
7										3	3			
6										8	8			
5										1	1			
4										6	6			
3						0	0			2	2		4	4
2						5	5			7	7		9	9
1										3	3			
0										8	8			

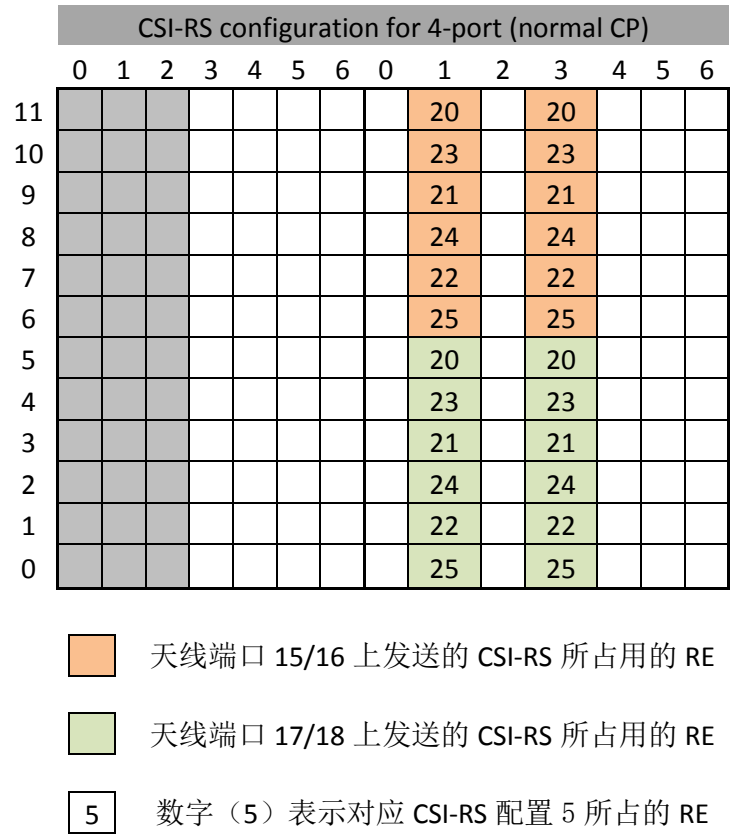


图 3-15：4 天线端口下，每种 CSI-RS 配置下，CSI-RS 所占的 RE

通过前面的介绍可以看出，CSI-RS 的发送在时域上需要满足以下条件：（1）所在的 slot：$n_s \bmod 2$ 必须满足 36.211 的 Tables 6.10.5.2-1（用于正常的循环前缀）和 Tables 6.10.5.2-2（用于扩展的循环前缀）给定的条件；（2）所在的子帧：必须满足 36.211 的 6.10.5.3 节给定的条件（也即 3.3.2 节介绍的时域配置）。

CSI-RS 在以下条件不能够发送：（1）CSI-RS 不能在 TDD 的特殊子帧上发送；（2）如果在某个子帧上，既要发送 CSI-RS，又要发送同步信号（PSS/SSS）、PBCH（MIB）、SIB1 或 paging 消息时，则 eNodeB 不会在该子帧发送 CSI-RS。

在集合 S 指定的天线端口上发送的 CSI-RS 所占 RE 的 (k,l) 不能用于任一天线端口在相同 slot 上的 PDSCH 传输，其中 $S=\{15\}$，$S=\{15,16\}$，$S=\{17,18\}$，$S=\{19,20\}$ 或 $S=\{21,22\}$。也即为了避免干扰，CSI-RS（包括 normal CSI-RS 和 muted CSI-RS）所占的 RE 不能用于任一天线端口上的 PDSCH 传输。当然在相同 slot 上，这些 RE 也不能用于不属于该集合 S 的其它天线端口的 CSI-RS 传输（公式已经保证了这一点）。

与 DMRS 类似，Rel-10 中定义的 CSI-RS 也不能被 Rel-8/ Rel-9 中定义的 UE 所识别。与 DMRS 只在分配给对应 UE 的那些 RB 上传输不同，CSI-RS 是覆盖整个下行系统带宽的，PDSCH 在映射时应该显式地避开那些被 CSI-RS 所占用的 RE。在那些包含 CSI-RS 的子帧里，Rel-10 的 UE 的 PDSCH 传输在速率匹配时，将假定 CSI-RS 所占的 RE 并不被 PDSCH 数据所占用，这是因为 Rel-10 的 UE 知道这些 RE 要被 CSI-RS 使用而不能被 PDSCH 所使用。但由于 CSI-RS 是 Rel-10 引入的， Rel-8/Rel-9 的 UE 并不知道 CSI-RS 的存在，所以这种映射方式不能被 Rel-8/ Rel-9 的 UE 所识别。因此 Rel-8/ Rel-9 的 UE 的 PDSCH 传输在速率匹配时，是按照 Rel-8 中介绍的方式进行的，即 PDSCH 传输在映射时并不避开那些被 CSI-RS 所占用的 RE。但实际上，eNodeB 并不会在那些 RE 上真正传输 PDSCH（即那些 RE 上的传输功率为 0），以避免对 CSI-RS 的传输产生干扰。也就是说，此时 PDSCH 传输将被 CSI-RS 所打孔（punctured），一些 PDSCH 调制符号会被破坏，并可能会对 PDSCH 的解调性能产生一些不利的影响。但实际上，不被打孔的那些剩余的 PDSCH 调制符号并不受影响，PDSCH 还是可解码的，但 CSI-RS 所选择的密度和周期还是应该保证打孔对性能的影响被限制在一个可接受的范围内。（注意：这里的介绍既适用于正常的 CSI-RS，也适用于 muted CSI-RS）

3.3.4 Muted CSI-RS

Muted CSI-RS 与正常的（non-muted）CSI-RS 有相同的结构，但 Muted CSI-RS 在对应的 RE（称为 muted RE）上并不发送任何东西。Muted CSI-SR 可以看做是零发射功率的正常 CSI-RS。

使用 Muted CSI-RS 是为了在其它相邻的小区上真正发送 CSI-RS 的 RE 上创建一个“传输空洞”。这有 2 个目的：

1、使得 UE 能够接收来自邻居小区的 CSI-RS，而该 CSI-RS 不会受到本小区内的传输的强干扰。对邻居小区的 CSI 测量对于支持多小区传输技术，如 CoMP，是有好处的。

2、减少对邻居小区的 CSI-RS 传输的干扰。这尤其适用于异构网络（heterogeneous network）部署中，此时不同功率的小区在同一频谱上重叠在一起，形成重叠的小区层。

简单地说，就是邻居小区在哪些 RE 上发送了 CSI-RS，本小区就将对应的 RE 设置成 muted 的，以避免干扰。

与不使用 muted CSI-RS，或使用 CRS 相比，使用 muted CSI-RS 能够提高小区间（inter-cell）的信道测量性能并提高 CoMP 的 UE 的吞吐量（throughput）。但由于 muted CSI-RS 所占的 RE 并不能被 PDSCH 传输所使用，因此会导致更多的 RE 被丢弃，这会对 Rel-8/Rel-9 的 UE 产生负面的影响。

一个小区里可以有 1 个或多个 muted CSI-RS 的集合。如果只是为了避免对邻居小区的干扰（对应前面的第 2 点），则配置一个 muted CSI-RS 集合通常就足够了，因为此时只需要避免对一个单一的低功率小区层产生干扰。如果 muted CSI-RS 是为了移除本小区的干扰，以便能够更好地接收

来自邻居小区的 CSI-RS，则通常需要配置多个 muted CSI-RS 集合，因为不同的邻居小区通常使用不同并且互斥的 CSI-RS 配置。

Muted CSI-RS 的集合是通过 *zeroTxPowerCSI-RS-r10* 来配置的，其中 *zeroTxPowerSubframeConfig-r10* 决定了 muted CSI-RS 会周期性地在哪些子帧上出现（见 3.3.2 节），*zeroTxPowerResourceConfigList-r10* 决定了在这些子帧上，哪些 RE 会被用于 muted CSI-RS。

zeroTxPowerResourceConfigList-r10 是一个 16 比特的 bitmap，每个比特对应一个 **4-port** 的 CSI-RS 配置（见 36.211 的 Tables 6.10.5.2-1 和 6.10.5.2-2 中 Number of CSI reference signals configured 为 4 的列。对应正常的循环前缀，正好是 16 个可选项；对应扩展的循环前缀，为 8 个可选项）。其中最高比特位对应最小的 CSI-RS 配置索引，而接下来的比特以递增的顺序对应 CSI-RS 配置索引。正常的循环前缀下，*zeroTxPowerResourceConfigList-r10* 与“CSI reference signal Configuration”的对应关系见表 3-3：

表 3-3：*zeroTxPowerResourceConfigList-r10* 与“CSI reference signal Configuration”的对应关系

Bitmap 中每个比特的位置	15（最高比特）	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Table 6.10.5.2-1 中的“CSI reference signal Configuration”	0	1	2	3	4	5	6	7	8	9	20	21	22	23	24	25

如果某个比特被设置为 1，则除非本小区的 normal CSI-RS 的配置与该比特对应的 CSI-RS 配置相同，否则该比特对应的 CSI-RS 配置计算出的所有 RE 为 muted 的。

图 3-16 是一个例子：*zeroTxPowerResourceConfigList-r10* 的配置为“1010001000000000”时，被 muted CSI-RS 所占用的 RE。

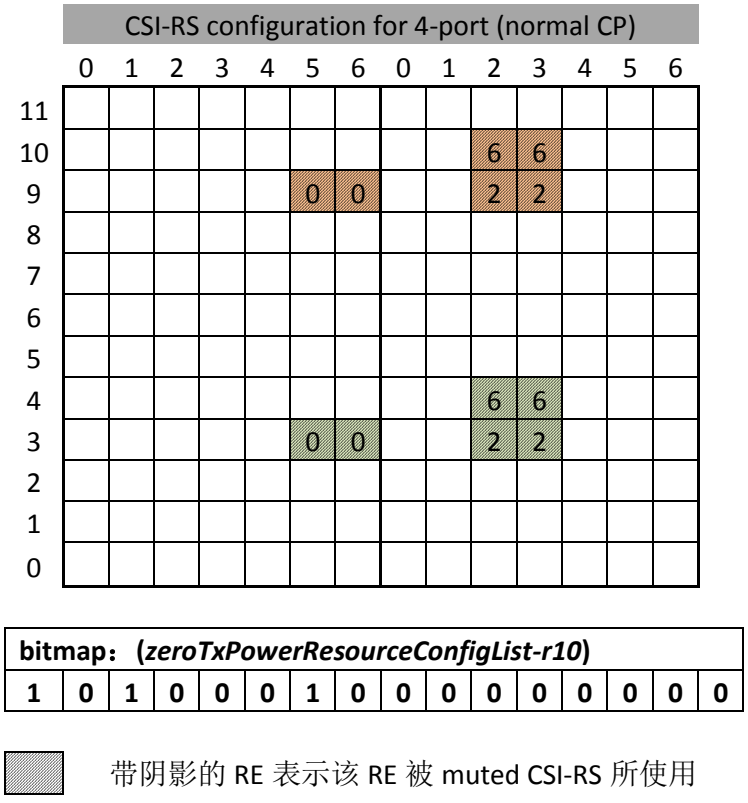


图 3-16：muted CSI-RS 配置举例（正常 CSI-RS 与 muted CSI-RS 无冲突）

当 muted CSI-RS 的配置不变，但正常 CSI-RS 的配置 *CSI-RS-Config-r10 -> resourceConfig-r10* 为 2 时，此时 CSI-RS 配置 2 所占的 RE 会变成 non-muted 的 RE，因为这些 RE 要被本小区正常的 CSI-RS 所占用。如图 3-17 所示。

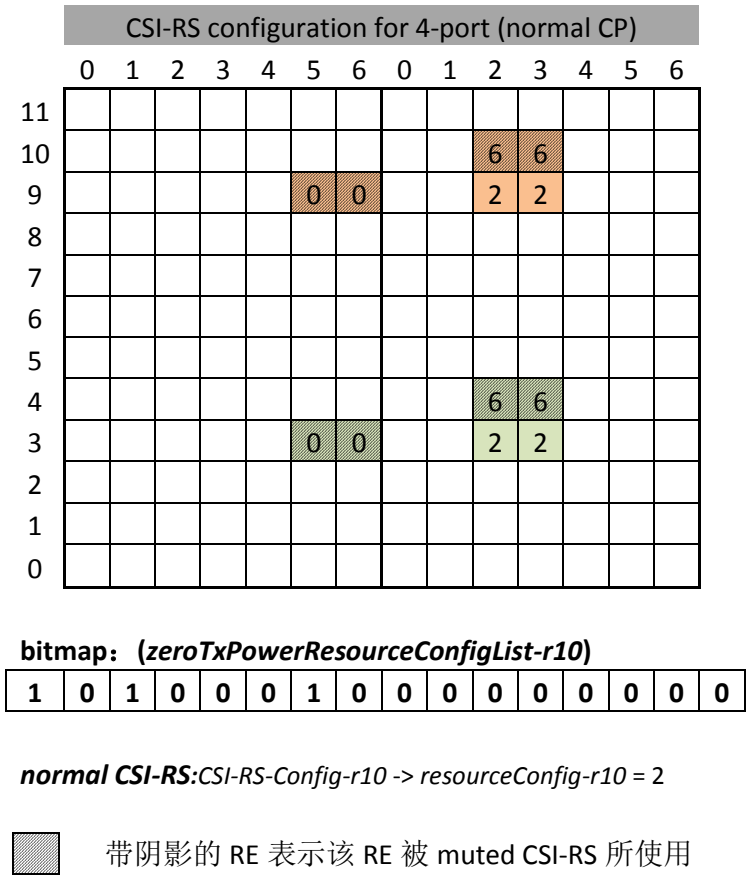


图 3-17：muted CSI-RS 配置举例（正常 CSI-RS 与 muted CSI-RS 有冲突）

由于 eNodeB 在 muted RE 上并没有发射功率，因此这部分多出来的功率可以用于正常的 CSI-RS 的传输，以提高其发射功率（power boost），这是通过设置 *p-C-r10* 字段来实现的（有或无 muted CSI-RS 时，可以设置不同的值，具体见 36.213 的 7.2.5 节）。但从 UE 的角度上看，它只关心 *p-C-r10* 的值，且认为在每个子帧上，正常的 CSI-RS 的传输功率是固定的，并独立于 muted CSI-RS 的配置。除非 UE 重新接收到不同的 *p-C-r10* 值，否则它认为功率是不变的。

同一小区内的所有使用了 CSI-RS 的 UE，应该使用相同的正常 CSI-RS 和 muted CS-RS 配置。

而不同小区间（inter-cell）的 CSI-RS 应该使用 FDM（频分）、CDM（码分）或二者的组合，而不是使用 TDM（时分）来区分。也就是说，多个相邻的小区应该在同一子帧上发送 CSI-RS，彼此之间通过 FDM/CDM（优先使用 FDM，即优先使用频分而不是码分）区分。具体原因可参见参考资料[5]和[6]中的介绍。

Muted CSI-RS 至少需要支持能够与正常的 CSI-RS 配置相同的周期和周期内的偏移（即 muted CSI-RS 通常与正常的 CSI-RS 在相同的子帧上发送，但反过来却未必）。在异构网络中，通常不同的小区拥有不同的周期配置。与 macro-cell 相比，pico-cell 通常有更长的 CSI-RS 周期，因为 macro-cell 需要支持更高速的 UE。因此从 macro-cell 的角度上看，muting CSI-RS 比正常的 CSI-RS 有更长的周期（此场景下，pico-cell 通常无需配置 muting CSI-RS，因为其发射功率较低，对 macro-cell 的影响较小）。

eNodeB 是否开启 muting CSI-RS 与网络部署、流量、UE 的能力等相关，可参见参考资料[5]和[6]，里面还介绍了关于如何配置相关参数的介绍，以及一些仿真结果。

【参考资料】

[1] TS 36.211 的 6.10.5 节 CSI reference signals

[2] TS 36.213 的 7.2.5 节 Channel-State Information – Reference Signal (CSI-RS) definition

[3] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.2.3 节

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 29.1.2 节

[5] "R1-105132, Proposal for specification of PDSCH Muting", Huawei, LG Electronics, Samsung, Panasonic, Intel, HiSilicon, New Postcom, CATR, Potevio, CMCC.

[6] "R1-094648, Inter-cell CSI-RS design and performance", Nokia, Nokia Siemens Networks.

3.4 MBSFN 参考信号（MBSFN-RS）

MBSFN-RS 专门用于使用 MBSFN 方式的 MCH 进行相关解调的信道估计。由于本书不会介绍 MBMS 的相关内容，所以这里就不再做叙述了。

3.5 定位参考信号（Positioning Reference Signal）

定位参考信号在 Rel-9 引入，主要用于增强 LTE 的定位功能，更具体地说，是支持多个 LTE 小区进行终端测量，以估计终端的地理位置。某个特定小区的定位参考信号在邻居小区可以配置成空闲 RE，从而接收到邻居小区的定位参考信号时，可以得到高的 SIR。由于本书不会介绍定位的相关内容，所以这里就不再做叙述了。

第4章　下行 L1/L2 控制信道

为了支持上下行数据传输，需要定义一些相关的下行控制信令。由于这些信令携带的信息来自于物理层（L1）或 MAC 层（L2），因此这些信令被称为下行 L1/L2 控制信令。下行 L1/L2 控制信令包括 PCFICH、PHICH 和 PDCCH。

本章首先会介绍 REG 的概念，并在随后的章节分别对 PCFICH、PHICH 和 PDCCH 进行介绍，以便大家对下行 L1/L2 控制信道有个全面的认识。

每个下行子帧（包括 TDD 中的 DwPTS 部分）被分成 2 部分：控制区域（control region）和数据区域（data region）。控制区域主要用于传输下行 L1/L2 控制信令，包括 PCFICH/PHICH/PDCCH；数据区域主要用于传输数据，包括 PSS/SSS、PBCH、PDSCH 和 PMCH。

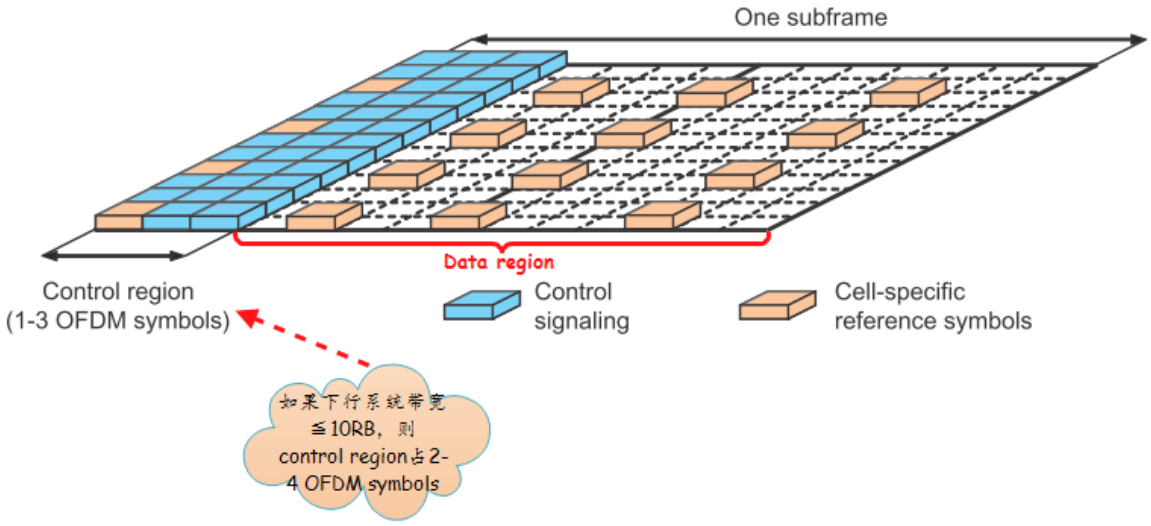


图 4-1：控制区域和数据区域

注意：只有下行才区分控制区域和数据区域，上行是不区分的。

4.1　REG

REG（Resource Element Group）用于定义如何将下行 L1/L2 控制信令映射到 RE 上。REG 是下行 L1/L2 控制信令进行物理资源分配的基本单位。一个 REG 中包含的 RE 集合取决于小区所配置的小区特定的参考信号的个数。

注意：只有下行 L1/L2 控制信令才有 REG 的概念。

REG 只存在于下行子帧的控制区域内，只用于下行 L1/L2 控制信令。由于小区特定的参考信号所占的 RE 不能用于传输下行 L1/L2 控制信令，且一个 REG 必须包含至少 4 个可用于传输下行 L1/L2 控制信令的 RE，且一个 REG 所包含的所有 RE 必须位于同一 RB 内，所以只要某个 OFDM 符号内包含了参考信号，则每个 RB 在该 OFDM 符号上只存在 2 个 REG。

由于下行 L1/L2 控制信令与小区特定的参考信号在相同的天线端口集合上发送，而小区特定的参考信号只位于天线端口 0~3 上，因此后面的介绍中涉及天线端口时只对应天线端口 0/1/2/3。

假设 $k_0 = n_{\mathrm{PRB}} \cdot N_{\mathrm{sc}}^{\mathrm{RB}}$ ，其中 $0 \le n_{\mathrm{PRB}} < N_{\mathrm{RB}}^{\mathrm{DL}}$ 。即 k_0 为 PRB 索引为 n_{PRB} 的 RB 在频域上的第一个 RE 所在的子载波位置。在编号为 n_{PRB} 的某个 PRB 内（如不做特殊说明，则对正常的循环前缀和扩展的循环前缀都适用），有：

情况一：一个子帧的第一个 slot 的第一个 OFDM 符号（对应 $l = 0$ ）包含了 2 个 REG，每个 REG 包含的 6 个 RE 的索引 $(k, l = 0)$ 分别为： $k = k_0 + 0, k_0 + 1, ..., k_0 + 5$ 和 $k = k_0 + 6, k_0 + 7, ..., k_0 + 11$。

情况二：如果使用 1 个或 2 个小区特定的参考信号的配置，一个子帧的第一个 slot 的第二个 OFDM 符号（对应 $l = 1$ ）包含了 3 个 REG，每个 REG 包含的 4 个 RE 的索引 $(k, l = 1)$ 分别为： $k = k_0 + 0, k_0 + 1, ..., k_0 + 3$ 、 $k = k_0 + 4, k_0 + 5, ..., k_0 + 7$ 和 $k = k_0 + 8, k_0 + 9, ..., k_0 + 11$。

情况三：如果使用 4 个小区特定的参考信号的配置，一个子帧的第一个 slot 的第二个 OFDM 符号（对应 $l = 1$ ）包含了 2 个 REG，每个 REG 包含的 6 个 RE 的索引 $(k, l = 1)$ 分别为： $k = k_0 + 0, k_0 + 1, ..., k_0 + 5$ 和 $k = k_0 + 6, k_0 + 7, ..., k_0 + 11$。

情况四：一个子帧的第一个 slot 的第三个 OFDM 符号（对应 $l = 2$ ）包含了 3 个 REG，每个 REG 包含的 4 个 RE 的索引 $(k, l = 2)$ 分别为 $k = k_0 + 0, k_0 + 1, ..., k_0 + 3$ 、 $k = k_0 + 4, k_0 + 5, ..., k_0 + 7$ 和 $k = k_0 + 8, k_0 + 9, ..., k_0 + 11$。

情况五：如果一个子帧使用正常的循环前缀，则该子帧的第一个 slot 的第四个 OFDM 符号（对应 $l = 3$ ）包含了 3 个 REG，每个 REG 包含的 4 个 RE 的索引 $(k, l = 3)$ 分别为 $k = k_0 + 0, k_0 + 1, ..., k_0 + 3$ 、 $k = k_0 + 4, k_0 + 5, ..., k_0 + 7$ 和 $k = k_0 + 8, k_0 + 9, ..., k_0 + 11$。

情况六：如果一个子帧使用扩展的循环前缀，则该子帧的第一个 slot 的第四个 OFDM 符号（对应 $l = 3$ ）包含了 2 个 REG，每个 REG 包含的 6 个 RE 的索引 $(k, l = 3)$ 分别为： $k = k_0 + 0, k_0 + 1, ..., k_0 + 5$ 和 $k = k_0 + 6, k_0 + 7, ..., k_0 + 11$。

控制区域至多包含第一个 slot 的前 4 个 OFDM 符号（对应 $l = 0 \sim 3$ ）。结合 36.211 的 Figure 6.10.1.2-1 和 Figure 6.10.1.2-2 可知：

- 对于 1/2/4 天线端口传输，第一个 OFDM 符号（对应 $l=0$）必然传输小区特定的参考信号，所以一个 RB 在该 OFDM 符号上包含 2 个 REG；（对应情况一）
- 对于 1/2 天线端口传输，第二个 OFDM 符号（对应 $l=1$）必然不传输小区特定的参考信号，所以一个 RB 在该 OFDM 符号上包含 3 个 REG；（对应情况二）
- 对于 4 天线端口传输，第二个 OFDM 符号（对应 $l=1$）必然传输小区特定的参考信号，所以一个 RB 在该 OFDM 符号上包含 2 个 REG；（对应情况三）
- 对于 1/2/4 天线端口传输，第三个 OFDM 符号（对应 $l=2$）必然不传输小区特定的参考信号，所以一个 RB 在该 OFDM 符号上包含 3 个 REG；（对应情况四）
- 对于 1/2/4 天线端口传输，如果使用正常的循环前缀，第四个 OFDM 符号（对应 $l=3$）必然不传输小区特定的参考信号，所以一个 RB 在该 OFDM 符号上包含 3 个 REG；（对应情况五）
- 对于 1/2/4 天线端口传输，如果使用扩展的循环前缀，第四个 OFDM 符号（对应 $l=3$）必然传输小区特定的参考信号，所以一个 RB 在该 OFDM 符号上包含 2 个 REG；（对应情况六）

前面介绍的是一个子帧内每个 RB 在控制区域内的每个 OFDM 符号上包含了多少个 REG。在实际计算在一个子帧上可用的 REG 数时，是将整个下行系统带宽内的所有 RB 在控制区域内的 REG 数都统计在内的。

这里我们以控制区域占 3 个 OFDM 符号为例，来看看如何计算某个小区在某个下行子帧上包含的 REG 数。

假设某个小区在下行使用 **2 天线端口**传输，在某个子帧上的控制区域占 3 个 OFDM 符号，下行系统带宽为 N_{RB}^{DL}（以频域上的 RB 数为单位），则小区在某个下行子帧上可用的 REG 总数为 (2+3+3)* N_{RB}^{DL}。（第一个 OFDM 符号上，每个 RB 包含 2 个 REG；后两个 OFDM 符号上，每个 RB 包含 3 个 REG）

假设某个小区在下行使用 **4 天线端口**传输，在某个子帧上的控制区域占 3 个 OFDM 符号，下行系统带宽为 N_{RB}^{DL}（以频域上的 RB 数为单位），则小区在某个下行子帧上可用的 REG 总数为 (2+2+3)* N_{RB}^{DL}。（前两个 OFDM 符号上，每个 RB 包含 2 个 REG；第三个 OFDM 符号上，每个 RB 包含 3 个 REG）

从前面的介绍中可以看出：（1）某个 REG 所包含的所有 RE 必定位于同一个 RB 内，且在时域上位于同一个 OFDM 符号内；（2）$4 \le$ 一个 REG 中包含的 RE 个数 ≤ 6；（3）一个 REG 中只

会用到 4 个 RE，对于只有一个小区特定的参考信号（即只使用一个天线端口：天线端口 0）的配置，第一个 OFDM 符号中（包括扩展的循环前缀中的第 4 个 OFDM 符号）除去参考信号所占的 RE，每个 REG 还包含了 5 个可用的 RE。因此在这种情况下，会假设按照 2 个天线端口（天线端口 0 和 1）来计算分配给小区特定的参考信号的 RE，这样就能算出所使用的 4 个 RE，即 6 个 RE 中有一个 RE 是未使用的。

【参考资料】

[1] TS 36.211 的 6.2.4 节 Resource-element groups

[2] TS 36.211 的 6.10.1 节 Cell-specific reference signals

4.2 PCFICH

PCFICH（Physical Control Format Indicator CHannel）用于通知 UE 对应下行子帧的控制区域的大小，即控制区域所占的 OFDM 符号（OFDM symbol）的个数。或者说，PCFICH 用于指示一个下行子帧中用于传输 PDCCH 的 OFDM 符号的个数。

4.2.1 PCFICH 介绍

每个小区在每个下行子帧有且仅有一个 PCFICH。

PCFICH 携带的信息为 CFI（Control Format Indicator），且 CFI 的取值范围为 1~3（即 CFI = 1、2 或 3；用 2 比特表示，CFI=4 为预留，不使用）。对于下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}} > 10$，控制区域所占的 OFDM 符号数为 1（CFI =1）或 2（CFI =2）或 3（CFI =3），即等于 CFI；对于下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}} \le 10$，控制区域所占的 OFDM 符号数为 2（CFI =1）或 3（CFI =2）或 4（CFI =3），即等于 CFI + 1。

对于 TDD 而言，子帧 1 和子帧 6 的控制区域至多只能占 2 个 OFDM 符号，这是因为在这些子帧中，PSS 要占据第三个 OFDM 符号。

见 36.211 的 Table 6.7-1。

Table 6.7-1: Number of OFDM symbols used for PDCCH.

Subframe	Number of OFDM symbols for PDCCH when $N_{RB}^{DL} > 10$	Number of OFDM symbols for PDCCH when $N_{RB}^{DL} \le 10$
Subframe 1 and 6 for frame structure type 2	1, 2	2
MBSFN subframes on a carrier supporting PDSCH, configured with 1 or 2 cell-specific antenna ports	1, 2	2
MBSFN subframes on a carrier supporting PDSCH, configured with 4 cell-specific antenna ports	2	2
Subframes on a carrier not supporting PDSCH	0	0
Non-MBSFN subframes (except subframe 6 for frame structure type 2) configured with positioning reference signals	1, 2, 3	2, 3
All other cases	1, 2, 3	2, 3, 4

PHICH duration 的配置限制了 CFI 取值范围的下限，也就是说，限制了控制区域至少需要占用的 OFDM 符号数。对于下行系统带宽 $N_{RB}^{DL} > 10$ 的小区而言，如果配置了 extended PHICH duration，UE 会认为 CFI 的值等于 PHICH duration（即等于 3），此时 UE 可以忽略 PCFICH 的值；对于下行系统带宽 $N_{RB}^{DL} \le 10$ 的小区而言，由于 CFI 指定的可用于控制区域的 OFDM 符号数可以为 4（见 36.212 的 5.3.4 节），大于 PHICH duration 可配置的最大值 3，如果此时配置了 extended PHICH duration，UE 还是要使用 PCFICH 指定的配置（即对于下行系统带宽 $N_{RB}^{DL} \le 10$ 的小区，CFI = 3 且配置了 extended PHICH duration 时，控制区域所占的 OFDM 符号数为 4）。即“**CFI 和 extended PHICH duration 相比较，取其大者**”（见 36.213 的 9.1.3 节）。关于 PHICH，见 4.3 节的介绍。

4.2.2 PCFICH 物理层处理

PCFICH 的物理层处理如图 4-2 所示。

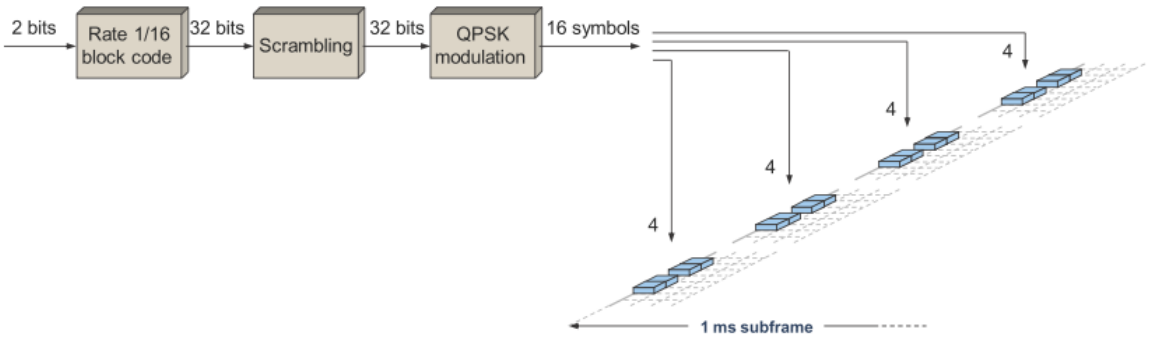


图 4-2：PCFICH 物理层处理

2 比特的 CFI 会先经过码率为 1/16 的信道编码，得到一个 32 比特的 codeword。（见 36.212 的 Table 5.3.4-1）

Table 5.3.4-1: CFI code words.

CFI	CFI code word < b_0, b_1, ..., b_{31} >
1	<0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1>
2	<1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0>
3	<1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1,0,1,1>
4 (Reserved)	<0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0>

接着，会使用小区和子帧特定的扰码对 32 比特的 codeword 进行加扰，以随机化小区间干扰（inter-cell interference）。（见 36.211 的 6.7.1 节）

然后，对加扰后的 32 个 scrambled bits 进行 QPSK 调制，得到 16 个调制符号（modulation symbol）。（见 36.211 的 6.7.2 节）

再把 16 个调制符号进行层匹配（layer mapping）和预编码（precoding）后，映射到对应的 RE 中去。（见 36.211 的 6.7.3 节）

只有 PCFICH 正确解码才能知道控制区域的大小，因此 PCFICH 总是放在每个子帧的第一个 OFDM 符号中发送。

PCFICH 的 16 个调制符号被分为 4 组，每组占一个 REG，每个 REG 包含 4 个可用于传输的 RE。这 4 个 REG 平均分布在整个下行系统带宽中，以获得频率分集增益。

每个 REG 包含的信息用 $z^{(p)}(i) = \left\langle y^{(p)}(4i), y^{(p)}(4i+1), y^{(p)}(4i+2), y^{(p)}(4i+3) \right\rangle$ 表示，i 表示每个 REG 的索引，其取值范围为 0 ~ 3。每个 REG 的起始 RE 所在的位置通过如下公式计算：

$$k \ \% \ (N_{\mathrm{RB}}^{\mathrm{DL}} N_{\mathrm{sc}}^{\mathrm{RB}})$$

其中：

$z^{(p)}(0)$ is mapped to the resource-element group represented by $k=\bar{k}$

$z^{(p)}(1)$ is mapped to the resource-element group represented by $k=\bar{k}+\lfloor N_{\mathrm{RB}}^{\mathrm{DL}}/2\rfloor\cdot N_{\mathrm{sc}}^{\mathrm{RB}}/2$

$z^{(p)}(2)$ is mapped to the resource-element group represented by $k=\bar{k}+\lfloor 2N_{\mathrm{RB}}^{\mathrm{DL}}/2\rfloor\cdot N_{\mathrm{sc}}^{\mathrm{RB}}/2$

$z^{(p)}(3)$ is mapped to the resource-element group represented by $k=\bar{k}+\lfloor 3N_{\mathrm{RB}}^{\mathrm{DL}}/2\rfloor\cdot N_{\mathrm{sc}}^{\mathrm{RB}}/2$

且有

$$\bar{k}=\left(N_{\mathrm{sc}}^{\mathrm{RB}}/2\right)\cdot\left(N_{\mathrm{ID}}^{\mathrm{cell}}\bmod 2N_{\mathrm{RB}}^{\mathrm{DL}}\right)$$

从上面的公式可以看出：

（1）$\lfloor N_{\mathrm{RB}}^{\mathrm{DL}}/2\rfloor\cdot N_{\mathrm{sc}}^{\mathrm{RB}}/2$约为 1/4 下行系统带宽，也就是说，相邻的 REG 在频域上相隔约 1/4 下行系统带宽；

（2）REG 的位置与小区的 Physical Cell ID（PCI）和下行系统带宽相关。在选择小区的 PCI 时，需要注意避免与邻居小区的 PCFICH 传输发生干扰，具体内容会在后续介绍。

PCFICH 与 PBCH 在相同的天线端口集合传输。如果使用多个天线端口传输，则只能使用传输分集（transmit diversity）。

4.2.3 载波聚合对 PCFICH 的影响

对于支持 CA（载波聚合）的 UE 而言，每个载波单元（component carrier，对应一个小区）都有各自的 PCFICH，即不同的载波单元可能有不同大小的控制区域，也即 PDSCH 的起始 OFDM 符号可能不同。因此，原则上 UE 需要在它被调度的每个载波单元上接收 PCFICH。不同的载波单元可能有不同的 PCI，所以不同载波单元的 PCFICH 的位置和扰码也可能不同。

与 PDCCH 一样，PCFICH 也是在控制区域传输的，因此，跨承载调度场景中影响 PDCCH 的小区间干扰（inter-cell interference）对于 PCFICH 也同样存在（见第 23 章关于“载波聚合”的介绍）。为了解决这个问题，Rel-10 可以通过 *CrossCarrierSchedulingConfig* 的 *pdsch-Start-r10* 字段指定任意跨载波调度的 PDSCH 的起始 OFDM 符号，而不需要通过解码对应载波单元上的 PCFICH 获得。

该机制并不妨碍 eNodeB 动态改变每个载波单元的控制区域的大小（虽然在许多异构网络场景中，为了实现 ICIC，通常会配置一个相对静态的控制区域大小）。

4.2.4 如何选择 CFI 以及 PCFICH 对 PCI 规划的影响

PCFICH 在每个被调度的 TTI 都会下发，并且是可以动态变化的。UE 在每个 TTI 都应该重新计算控制区域所占的 OFDM 符号的个数，并综合考虑下行系统带宽、CFI 的取值、PHICH duration 和 *pdsch-Start-r10* 等的影响。

控制区域所占的 OFDM 符号越少，对应 TTI 可用于用户数据传输的 OFDM 符号就越多，相应的吞吐量（throughput）就会越高，反之亦然。

控制区域所占的 OFDM 符号越少，对应 TTI 可用于 PDCCH 传输的 OFDM 符号就越少，则该 TTI 能够服务（调度）的 UE 数就越少，即容量（capacity）越小。

所以 CFI 的选择需要根据场景的需求，在吞吐量和容量之间取得平衡。

在规划小区的 PCI 时，通常会将 PSS 和 SSS 考虑在内（相邻小区避免使用相同的 PSS）。实际上，也需要将 PCFICH 在第一个 OFDM 符号中的位置考虑在内，PCIFCH 的位置对小区边界的吞吐量（throughput）有很大的影响。

在配置小区的 PCI 时，其中一个原则是避免与邻居小区的 PCFICH 在频域位置上（即 REG 的位置）发生重叠，这样能够提高 PCFICH 的解码性能。

从之前的介绍中可以看出，PCFICH 的位置取决于 PCI 和部署的下行系统带宽。PCI 有 504 种取值， PCI 不同的相邻小区可能会有重叠的 PCFICH 位置。如果相邻小区的 PCFICH 位置存在重叠，则小区边界的 UE 在解码 PCFICH 时会受到邻居小区的干扰。如果 UE 解码 PCFICH 出了问题，又会导致 PDCCH 和 PDSCH 解码失败，从而降低了小区边界的吞吐量。（更详细的内容可参见参考资料[5]）

图 4-3 是不同小区的 PCFICH 配置存在重叠的一个例子。其中 p、q、r 和 s 分别表示对应的 REG 编号。

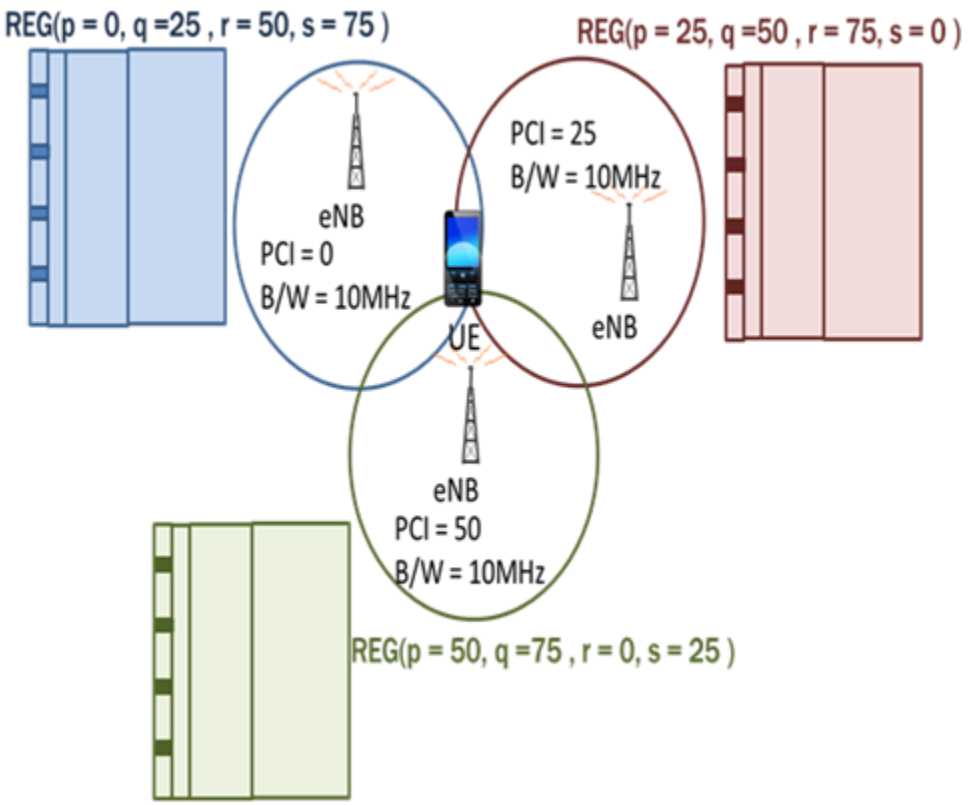


图 4-3：相邻小区的 PCFICH 位置部分重叠的例子

小结： 控制区域所占的 OFDM 符号数主要依赖于如下因素：（1）在吞吐量和容量之间取得平衡；（2）36.211 的 Table 6.7-1 中的限制因素；（3）PHICH duration 的配置；（4）载波聚合中 *pdsch-Start-r10* 的配置。

在配置小区的 PCI 时，其中一个原则时避免与邻居小区的 PCFICH 在频域位置上（即 REG 的位置）发生重叠。

【参考资料】

[1] TS 36.211 的 6.7 节 Physical control format indicator channel

[2] TS 36.212 的 5.3.4 节 Control format indicator

[3] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.4.1 节

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 9.3.3 节和 28.3.1.2 节

[5] 《Don't Forget PCFICH While Doing Physical Cell Identity (PCI) Planning》 by Manohara

4.3 PHICH

PHICH（Physical Hybrid-ARQ Indicator CHannel）用于对上行 PUSCH 传输的数据回应 HARQ ACK/NACK。每个 TTI 中的每个上行 TB 对应一个 PHICH，也就是说，当 UE 在某小区配置了上行空分复用时，需要 2 个 PHICH。

4.3.1 PHICH 资源介绍

小区是通过 *MasterInformationBlock* 的 *phich-Config* 字段来配置 PHICH 的。

```
MasterInformationBlock ::=          SEQUENCE {
    dl-Bandwidth                    ENUMERATED {
                                        n6, n15, n25, n50, n75, n100},
    phich-Config                    PHICH-Config, ------PHICH配置
    systemFrameNumber               BIT STRING (SIZE (8)),
    spare                           BIT STRING (SIZE (10))
}
```

```
PHICH-Config ::=                SEQUENCE {
    phich-Duration                  ENUMERATED {normal, extended},------配置PHICH
duration
    phich-Resource                  ENUMERATED {oneSixth, half, one, two}------配置
PHICH资源数
}
```

phich-Duration 指定了在控制区域内是使用的 1 个 OFDM 符号（对应 normal PHICH duration）还是 3（或 2）个 OFDM 符号（对应 extended PHICH duration）来发送 PHICH，对应 36.211 的 Table 6.9.3-1。

Table 6.9.3-1: PHICH duration in MBSFN and non-MBSFN subframes.

PHICH duration	Non-MBSFN subframes		MBSFN subframes on a carrier supporting PDSCH
	Subframes 1 and 6 in case of frame structure type 2	All other cases	
Normal	1	1	1
Extended	2	3	2

注：TDD 中，PSS 在子帧 1 和 6 的第 3 个 OFDM 符号上传输，该符号不能用于发送 PHICH，所以此时在 extended PHICH duration 下，只能使用 2 个 OFDM 符号来发送 PHICH。

eNodeB 通常会配置只使用第一个 OFDM 符号来发送 PHICH，这样即使 PCFICH 解码失败了，也不影响 PHICH 的解码。但在某些场景下，比如下行系统带宽较小的小区（如 1.4 MHz，总共只有 6 个 RB），其频域分集的增益要比下行系统带宽较大的小区（如 20 MHz）的小区要低。通过使用 extended PHICH duration，能提高时间分集的增益，从而提高 PHICH 的性能。

前一节已经介绍过，PHICH duration 的配置限制了 CFI 取值范围的下限，也就是说，限制了控制区域至少需要占用的 OFDM 符号数。对于下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}} > 10$ 的小区而言，如果配置了 extended PHICH duration，UE 会认为 CFI 的值等于 PHICH duration，此时 UE 可以忽略 PCFICH 的值；对于下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}} \le 10$ 的小区而言，由于 CFI 指定的可用于 control region 的 OFDM 符号数可以为 4（见 36.212 的 5.3.4 节），大于 PHICH duration 可配置的最大值 3，如果此时配置了 extended PHICH duration，UE 还是要使用 PCFICH 指定的配置。即“**CFI 和 extended PHICH duration 相比较，取其大者”**。（见 36.213 的 9.1.3 节）

phich-Resource 指定了控制区域中预留给 PHICH 的资源数，它决定了 PHICH group 的数目。

多个 PHICH 可以映射到相同的 RE 集合中发送，这些 PHICH 组成了一个 PHICH group，即多个 PHICH 可以复用到同一个 PHICH group 中。同一个 PHICH group 中的 PHICH 通过不同的正交序列（orthogonal sequence）来区分。即一个二元组 $\left(n_{\mathrm{PHICH}}^{\mathrm{group}}, n_{\mathrm{PHICH}}^{\mathrm{seq}}\right)$ 唯一指定一个 PHICH 资源，其中 $n_{\mathrm{PHICH}}^{\mathrm{group}}$ 为 PHICH group 索引，$n_{\mathrm{PHICH}}^{\mathrm{seq}}$ 为该 PHICH group 内的正交序列索引。

一个小区内可用的 PHICH group 数的计算方式如图 4-4 所示。

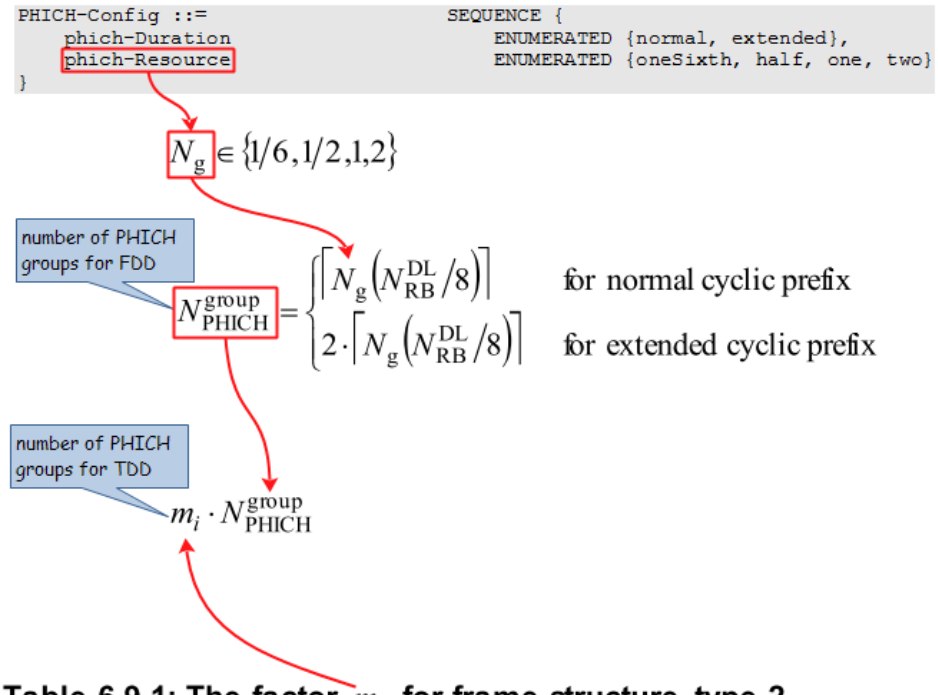


Table 6.9-1: The factor m_i for frame structure type 2.

Uplink-downlink configuration	Subframe number i									
	0	1	2	3	4	5	6	7	8	9
0	2	1	-	-	-	2	1	-	-	-
1	0	1	-	-	1	0	1	-	-	1
2	0	0	-	1	0	0	0	-	1	0
3	1	0	-	-	-	0	0	0	1	1
4	0	0	-	-	0	0	0	0	1	1
5	0	0	-	0	0	0	0	0	1	0
6	1	1	-	-	-	1	1	-	-	1

图 4-4：如何计算 PHICH group 的个数

注意：（1）$m_i = 2$ 的场景只出现在 TDD 0 这种配置下，此时对应子帧所需的 PHICH group 数量是 $m_i = 1$ 时的 2 倍。这是因为只有在 TDD 0 配置下，一个系统帧内的下行子帧数少于上行子帧数，此时在同一个下行子帧可能需要反馈 2 个上行子帧的 ACK/NACK 信息，所以需要 2 倍的 PHICH 资源；（2）控制区域中不一定包含 PHICH（例如 m_i =0 的情况）。

从图 4-4 可以看出：在下行系统带宽确定的情况下，对于 FDD 而言，PHICH group 数仅与 phich-Resource 的配置相关；而对于 TDD 而言，PHICH group 数不仅与 phich-Resource 的配置相关，还与 TDD 上下行配置（uplink-downlink configuration）以及子帧号相关。

N_g 越大，PHICH group 数越多，可复用的 UE 数也就越多，支持调度的上行 UE 数也就越多，但码间干扰也就越大，解调性能也就越差。与此同时，控制区域内可用于 PDCCH 的资源数就越少。

一个 PHICH group 可用的正交序列数见 36.211 的 Table 6.9.1-2。可以看出，对正常的循环前缀而言，一个 PHICH group 支持 8 个正交序列，即支持 8 个 PHICH 复用；对扩展的循环前缀而言，一个 PHICH group 支持 4 个正交序列，即支持 4 个 PHICH 复用。

Table 6.9.1-2: Orthogonal sequences $\left[w(0) \quad \cdots \quad w(N_{\mathrm{SF}}^{\mathrm{PHICH}}-1)\right]$ for PHICH.

Sequence index	Orthogonal sequence	
$n_{\mathrm{PHICH}}^{\mathrm{seq}}$	Normal cyclic prefix $N_{\mathrm{SF}}^{\mathrm{PHICH}}=4$	Extended cyclic prefix $N_{\mathrm{SF}}^{\mathrm{PHICH}}=2$
0	$[+1 \quad +1 \quad +1 \quad +1]$	$[+1 \quad +1]$
1	$[+1 \quad -1 \quad +1 \quad -1]$	$[+1 \quad -1]$
2	$[+1 \quad +1 \quad -1 \quad -1]$	$[+j \quad +j]$
3	$[+1 \quad -1 \quad -1 \quad +1]$	$[+j \quad -j]$
4	$[+j \quad +j \quad +j \quad +j]$	-
5	$[+j \quad -j \quad +j \quad -j]$	-
6	$[+j \quad +j \quad -j \quad -j]$	-
7	$[+j \quad -j \quad -j \quad +j]$	-

通过上面的介绍，我们可以计算出一个小区在某个下行子帧所包含的 PHICH 资源数：对应正常的循环前缀，其值为 $8*m_i \cdot N_{\mathrm{PHICH}}^{\mathrm{group}}$；对应扩展的循环前缀，其值为 $4*m_i \cdot N_{\mathrm{PHICH}}^{\mathrm{group}}$。（我们可以认为：在 FDD 下，$m_i=1$）

一个小区真正所需的 PHICH 资源总数取决于：（1）下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}}$；（2）每个 TTI 能够调度的上行 UE 数（只有被调度的上行 UE 才需要 PHICH）；（3）UE 是否支持空分复用（2 个上行 TB 就对应 2 个 PHICH）等。

PHICH 配置必须在 MIB 中发送的原因在于：SIB 是在 PDSCH 中发送的， PDSCH 资源是通过 PDCCH 来指示的， PDCCH 的盲检又与 PHICH 资源数的配置相关（见 4.4 节关于“PDCCH”的介绍），因此 UE 需要提前知道 PHICH 配置以便成功解码 SIB。

对于 FDD 而言，接收到 MIB 就可以计算出预留给 PHICH 的资源。（在 FDD 下，$m_i=1$）

对于 TDD 而言，UE 仅仅接收到 MIB 是不够的，UE 还需要知道 TDD 上下行配置和子帧号，以便确定 m_i 值。通过小区搜索过程，UE 已经知道了当前子帧号；但 UE 需要接收到 SIB1 后，通过 *SystemInformationBlockType1* 的 *tdd-Config* 的 *subframeAssignment* 字段才能知道 TDD 上下行配置。

问题来了：SIB1 在 PDSCH 中发送，需要先解码 PDCCH，且 PDCCH 的解码与 PHICH 资源数的计算相关；而 PHICH 资源数的计算又依赖于 SIB1 中指定的 TDD 上下行配置，这就形成了死锁。解决的方法是，UE 在接收 SIB1 时，会使用不同的 m_i 值（0~2，见图 4-4）去尝试盲检，直到成功解码出 SIB1 为止，从而得到 TDD 上下行配置。

4.3.2 PHICH 物理层处理

在物理层，eNodeB 会先对每个 HARQ 确认信息（1 比特：对应一个上行 TB 的 ACK/NACK）重复 3 遍（见 36.212 的 5.3.5 节），得到 3 比特的信息，接着使用 BPSK 调制和使用一个长为 4（对于扩展的循环前缀而言，长为 2）的正交序列进行扩频，再使用小区特定的搅扰序列进行加扰后，就得到 12 个加扰 symbol（见 36.211 的 6.9.1 节）。

多个 PHICH 映射同一个 PHICH group 时，是将多个 PHICH 映射到同一个 RE 的 symbol 相加来实现的。（对应 36.211 的 6.9.3 节中的公式 $\bar{y}^{(p)}(n) = \sum y_i^{(p)}(n)$ ）

每个 PHICH group 会映射到 3 个 REG 中，这 3 个 REG 是分开的，彼此间隔 1/3 下行系统带宽。12 个 symbol 如何映射到对应的 REG、层匹配、预编码、以及如何映射到 RE，详见 36.211 的 6.9.2 节和 6.9.3 节。

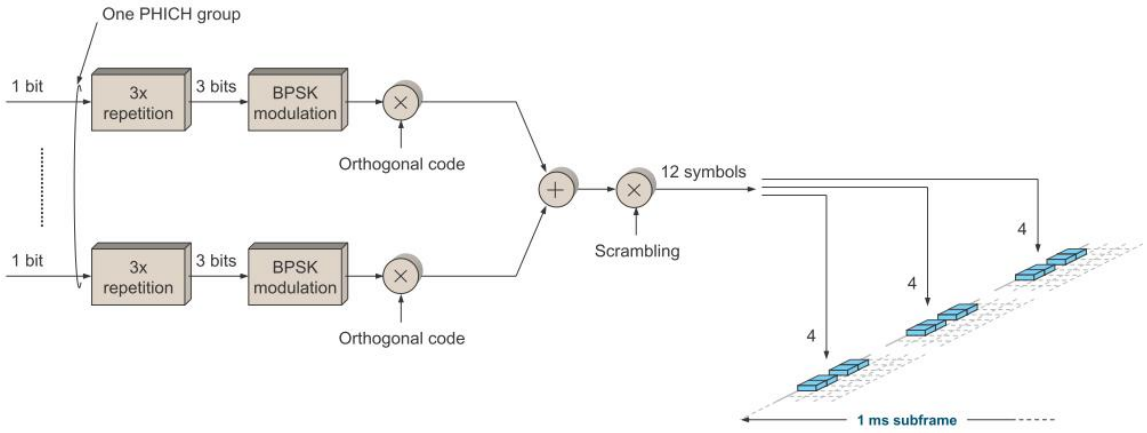


图 4-5：PHICH 物理层处理

在控制区域的第一个 OFDM 符号上，资源首先会分配给 PCFICH，PHICH 只能映射到没有被 PCFICH 使用的那些 RE 上。属于同一个 PHICH group 中的所有 PHICH 映射到相同的 RE 集合上；不同的 PHICH group 使用的 RE 集合是不同的。

PHICH 与 PBCH 在相同的天线端口集合传输。如果使用多个天线端口传输，则只能使用传输分集（transmit diversity）。

4.3.3 UE 如何确定其使用的 PHICH 资源

UE 如何确定 eNodeB 使用哪个 PHICH 资源来回应其上行数据的 ACK/NACK 呢？

在时域上，如果 UE 在子帧 n 发送 PUSCH，则 UE 会在子帧 $n+k_{PHICH}$ 检测对应的 PHICH。对于 FDD 而言，k_{PHICH} 总是等于 4；对于 TDD 而言，k_{PHICH} 是通过 36.213 的 Table 9.1.2-1 得到。在子帧绑定（TTI bundling）操作中，PHICH 资源是与所有绑定在一起的子帧中的最后一个子帧相对应的。

Table 9.1.2-1: k_{PHICH} for TDD

TDD UL/DL Configuration	subframe index *n*									
	0	1	2	3	4	5	6	7	8	9
0			4	7	6			4	7	6
1			4	6				4	6	
2			6					6		
3			6	6	6					
4			6	6						
5			6							
6			4	6	6			4	7	

在确定了在哪个子帧上接收对应的 PHICH 后，UE 需要确定所使用的 PHICH 资源，即确定二元组 $\left(n_{\mathrm{PHICH}}^{\mathrm{group}}, n_{\mathrm{PHICH}}^{\mathrm{seq}}\right)$。该二元组与 DCI format 0/4 指定的上行资源分配和 DMRS 的循环移位相关，计算公式如下：

$$n_{PHICH}^{group} = (I_{PRB_RA} + n_{DMRS}) \bmod N_{PHICH}^{group} + I_{PHICH} N_{PHICH}^{group}$$
$$n_{PHICH}^{seq} = (\lfloor I_{PRB_RA} / N_{PHICH}^{group} \rfloor + n_{DMRS}) \bmod 2N_{SF}^{PHICH}$$

其中，

n_{DMRS}：DCI format 0/4 中有一个字段叫 Cyclic shift for DM RS and OCC index（见 36.212 的 5.3.3.1.1 节），通过该字段查 36.213 的 Table 9.1.2-2，就得到对应的 n_{DMRS} 值。当然，此 DCI format 0/4 必须是最新的用于指示对应 PHICH 相关的 TB 所在的 PUSCH 资源的。如果同一 TB 没有相应的 DCI format 0/4，并且以下两个条件满足其一，n_{DMRS} 的值将为 0：

- 同一 TB 的初始 PUSCH 传输是半静态调度的；
- 同一 TB 的初始 PUSCH 传输是通过 RAR 调度的。

Table 9.1.2-2: Mapping between n_{DMRS} and the cyclic shift for DMRS field in PDCCH with uplink DCI format in [4]

Cyclic Shift for DMRS Field in PDCCH with uplink DCI format in [4]	n_{DMRS}
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

在上行 MU-MIMO 中，多个 UE 可能使用相同的时频资源来发送 PUSCH（接下来介绍的几个参数都相同），此时需要通过码分来进行复用，即不同的 UE 配置不同的 n_{DMRS} 值，从而不同 UE 使用了不同的 PHICH 资源。

N_{SF}^{PHICH}：是用于 PHICH 调制的 spreading factor 的大小。对于正常的循环前缀，其值为 4；对于扩展的循环前缀，其值为 2。（见 36.211 的 6.9.1 节）

I_{PRB_RA}：如果是 PUSCH 传输的第一个 TB，其值为 $I_{PRB_RA}^{lowest_index}$；如果是 PUSCH 传输的第二个 TB，其值为 $I_{PRB_RA}^{lowest_index}+1$。其中，$I_{PRB_RA}^{lowest_index}$ 为对应的 PUSCH 传输在第一个 slot 的最低 PRB 索引。即上行 SU-MIMO 中使用此字段来区分不同 TB 对应的 PHICH 资源。

N_{PHICH}^{group}：PHICH group 的个数，见之前的介绍。

I_{PHICH}：当 TDD 的上下行配置为 0 且 PUSCH 在子帧 4 或 9（对应回应 ACK/NACK 的下行子帧为 0 或 5，其 $m_i=2$，此时 2 个不同的上行子帧发送的 PUSCH 需要在同一个下行子帧回应 ACK/NACK）上发送时，其值为 1；其它情况下，其值为 0。

4.3.4 载波聚合对 PHICH 的影响

在载波聚合中，PHICH 与对应的上行 PUSCH 数据传输的 UL Grant 在同一个下行载波单元（Component Carrier，CC）上传输。这样做的原因在于异构网络的部署可能使得一些载波单元的控制区域受到较高的小区间干扰（inter-cell interference），这时候使用跨载波调度（cross-carrier scheduling）将某些载波单元的 PDCCH（此时对应 DCI format 0/4）在信道质量较好的其它载波单元上发送，能提高了 PDCCH 的解码效率。假如对于某个特定 UE，将载波单元 1 的 DCI format 0/4 放在载波单元 2 的控制区域上发送，说明载波单元 2 的信道质量比载波单元 1 好。此时把载波单元 1 的 PHICH 也放在载波单元 2 上发送，相应地也能提高 PHICH 的解码效率。

原则：**PHICH 与对应的上行数据传输的 UL Grant（DCI format 0/4）在同一个下行载波单元上传输。**

因此，当配置了跨载波调度时，一个下行载波单元可能需要携带多个上行载波单元的 PHICH，从而增加了 PHICH 冲突的可能性（因为 PHICH 资源与对应 PUSCH 传输的起始 PRB 相关，多个上行载波单元可能使用相同的起始 PRB）。为了降低冲突，可以将在同一下行载波单元的控制区域上传输的不同上行载波单元的 DMRS 的循环移位（即 n_{DMRS} ）配置成不同的值来偏移 PHICH 索引。

与此同时，eNodeB 调度器也可以在调度时为不同上行载波单元选择起始 PRB 不同的 PUSCH 资源。

【参考资料】

[1] TS 36.211 的 6.9 节 HARQ indicator (HI)

[2] TS 36.212 的 5.3.5 节 Physical hybrid ARQ indicator channel

[3] TS 36.213 的 9.1.2 节 PHICH Assignment Procedure

[4] TS 36.331 的 *PHICH-Config*

[5] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.4.2 节

[6] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 9.3.4 节和 28.3.1.3 节

[7] http://www.sharetechnote.com 中关于 PHICH 的介绍

[8] 《Physical Hybrid-ARQ Indicator Channel (PHICH)》by Hongyan Lei

4.4 PDCCH

PDCCH（Physical Downlink Control CHannel）主要用于：（1）向 UE 发送下行调度信息（36.321 中称之为 DL Assignment），以便 UE 接收 PDSCH；（2）向 UE 发送上行调度信息（36.321 中称之为 UL Grant），以便 UE 发送 PUSCH；（3）发送非周期性 CQI 上报请求；（4）通知 MCCH 变化；（5）发送上行功控命令；（6）HARQ 相关信息；（7）携带 RNTI：该信息隐式包含在 CRC 中等等。

PDCCH 携带的信息称为 DCI（Downlink Control Information）。各种 DCI format 及其携带的具体信息可参见 36.212 的 5.3.3 节。DCI 可能指示小区级的信息（使用 SI-RNTI / P-RNTI / RA-RNTI 加扰），也可能指示 UE 级的信息（使用 C-RNTI / SPS C-RNTI / Temporary C-RNTI 加扰）。

一个 PDCCH 只能携带一个某种 format 的 DCI。一个小区可以在上行和下行同时调度多个 UE，即一个小区可以在每个子帧发送多个调度信息。每个调度信息在独立的 PDCCH 上传输，也就是说，一个小区可以在一个子帧上同时发送多个 PDCCH。

PDCCH 固定使用 QPSK 调制。

一个 PDCCH 在 n 个连续的 CCE（Control Channel Element）上传输，每个 CCE 由 9 个 REG 组成。PDCCH 的起始 CCE 索引 i 必须满足 $i \bmod n = 0$ 且 i 的取值范围为 $0 \sim N_{CCE,k} - 1$，其中 $N_{CCE,k}$ 为子帧 k 中所有可供 PDCCH 使用的 CCE 总数。**即由 n 个 CCE 组成的 PDCCH，其起始位置所在的 CCE 号，必须为 n 的整数倍。**

PDCCH 所占的第一个 CCE 的 CCE 索引称为 n_{CCE}，在介绍下行 HARQ（对应 36.213 的 10.1 节）时会经常提到这个概念。

PDCCH 有 4 种 format (格式) {0，1，2，3}，分别对应聚合等级（Aggregation Level）{1，2，4，8}。聚合等级表示一个 PDCCH 占用的连续的 CCE 个数，即前面提到的 n。

eNodeB 会根据信道质量等因素来决定某个 PDCCH 使用的聚合等级。例如：如果 PDCCH 是发给某个下行信道质量很好的 UE（例如位于小区中心），则使用 1 个 CCE 来发送该 PDCCH 可能就足够了；如果 PDCCH 是发给某个下行信道质量很差的 UE（例如位于小区边缘），则可能需要使用 8 个 CCE 来发送该 PDCCH 以达到足够的健壮性。PDCCH 的功率也可以根据信道条件进行调整，eNodeB 可以将信道质量较好 UE 的 PDCCH 发射功率节省下来以分配给信道质量较差的 UE。（PDCCH 的链路自适应）

LTE 中支持的 PDCCH 格式如 36.211 的 Table 6.8.1-1 所示。

Table 6.8.1-1: Supported PDCCH formats.

PDCCH format	Number of CCEs	Number of resource-element groups	Number of PDCCH bits
0	1	9	72
1	2	18	144
2	4	36	288
3	8	72	576

注意：不要将 PDCCH format 和 DCI format 混淆，二者是不同的概念。

为什么需要多个聚合等级（PDCCH format）呢？

首先，为了支持不同的 DCI format 以提高资源利用率。由于 DCI 的大小与其格式和下行系统带宽相关，差异会很大。通过将不同聚合等级的 PDCCH 看做不同大小的容器来容纳不同大小的 DCI，可以提供灵活的资源利用粒度，而不是使用“一种聚合等级，适配所有 DCI format”的解决方案。

其次，为了适应不同的无线信道条件。DCI 的大小和 PDCCH 大小之间的比值可以指示有效码率（effective coding rate）。在 DCI format 固定的前提下，对于信道条件差的 UE，可以通过提高聚合等级来提高更健壮的编码和可靠性；而对于信道条件好的 UE，可以通过降低聚合等级来节省 CCE 资源。

再次，区别对待用于控制信息（如小区特定的信息）的 DCI 和用于用户数据的 DCI。更高的聚合等级可被用于控制信息的资源分配以提供更多保护，其聚合等级可以为 4 或 8。而用于 UE 特定数据的聚合等级可以为 1、2、4 和 8。

下行 DCI 包括 DCI format 1/1A/1B/1C/1D/2/2A/2B/2C；上行 DCI 包括 DCI format 0/4。无论是下行 DCI，还是上行 DCI，都是通过下行的 PDCCH 发送给 UE 的。

4.4.1 搜索空间

我们先从非载波聚合，即 UE 只有一个 serving cell 的场景开始介绍。

UE 会在 non-DRX 子帧监听 PDCCH candidates 集合，这意味着 UE 需要根据所要监听的 DCI format 来尝试解码该集合中的每一个 PDCCH。该集合被称为该 UE 的搜索空间（Search Space）。在聚合等级 $L \in \{1,2,4,8\}$ 上的搜索空间 $S_k^{(L)}$ 定义为 PDCCH candidates 的集合。

搜索空间分为公共搜索空间（Common search space）和 UE 特定的搜索空间（UE-specific search space）。公共搜索空间空间用于传输与 Paging、RA Response、BCCH 等相关的控制信息（小区级别的公共信息），该信息对所有 UE 来说都是一样的。UE 特定的搜索空间用于传输与 DL-SCH、UL-SCH 等相关的控制信息（UE 级别的信息）。但是当 UE 特定的搜索空间没有足够的可用资源

时，公共搜索空间也可以用于传输属于某个特定 UE 的控制信息。但公共搜索空间只能用于传输较小的 DCI format 0/1A/3/3A/1C。（注：DCI fomat 0/1A/3/3A 有相同的大小）

从 36.213 的 Table 9.1.1-1 可以看出，对于某种 DCI format，可能的 candidate 有 22（6 + 6 + 2 + 2 + 4 + 2）个。

Table 9.1.1-1: PDCCH candidates monitored by a UE.

Search space $S_k^{(L)}$			Number of PDCCH candidates $M^{(L)}$
Type	Aggregation level L	Size [in CCEs]	
UE-specific	1	6	6
	2	12	6
	4	8	2
	8	16	2
Common	4	16	4
	8	16	2

搜索空间 $S_k^{(L)}$ 内的某个 PDCCH candidate m 所占用的 CCEs 通过如下公式计算

$$L\cdot\left\{\left(Y_k+m\right)\bmod\left\lfloor N_{\mathrm{CCE},k}/L\right\rfloor\right\}+i$$

其中 $i=0,\cdots,L-1$ 且 $m=0,\cdots,M^{(L)}-1$。$M^{(L)}$ 为给定的搜索空间内需要监听的 PDCCH candidate 数，并如 36.213 的 Table 9.1.1-1 所示。

公共搜索空间从 CCE 0 开始；UE 特定的搜索空间的起始位置可以通过前面给出的公式（对应 $i=0$）计算得出。

对于公共搜索空间，Y_k 为 0。

对于 UE 特定的搜索空间 $S_k^{(L)}$（聚合等级为 L），Y_k 定义为

$$Y_k=\left(A\cdot Y_{k-1}\right)\bmod D$$

其中 $Y_{-1}=n_{\mathrm{RNTI}}\neq 0$，$A=39827$，$D=65537$ and $k=\lfloor n_s/2\rfloor$，n_s 为一个系统帧中的 slot 号（取值范围为 0~19）。

从上面的公式可以看出，搜索空间与 RNTI、子帧号相关。

图 4-6 是 UE A 和 UE B 的搜索空间的一个例子：

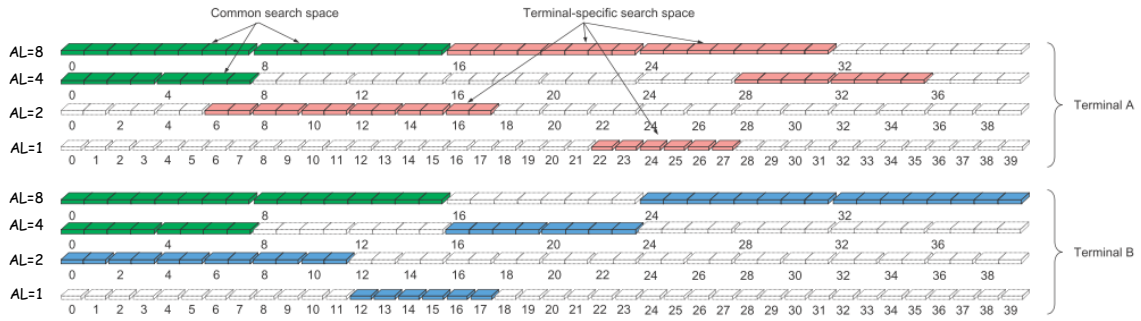


图 4-6：搜索空间举例（注：AL=4 的公共搜索空间的 candidate 数为 4，但图中只画了 2 个）

公共搜索空间和 UE 特定的搜索空间可能重叠，属于不同 UE 的 UE 特定的搜索空间也可能重叠。如果重叠的区域被一个 UE 占用，那么其它 UE 将不能再使用这些 CCE 资源。

UE 只会在聚合等级 4 和 8 上监听公共搜索空间，但会在聚合等级 1、2、4、8 上监听 UE 特定的搜索空间。

eNodeB 在调度时，会针对每个待调度的 UE，从对应的搜索空间中选择一个可用的 PDCCH candidate。如果能分配到 CCE 就调度，否则就不调度。发给不同 UE 的 PDCCH 可以有不同的聚合等级。

4.4.2 下行子帧可用的 CCE 数

一个下行子帧的控制区域由 PCFICH + PHICH + PDCCH + Reference Symbol 组成。

映射顺序：先映射 Reference Symbol，接着映射 PCFICH，然后是 PHICH，其映射的位置与小区配置有关，原则是尽量配置到不同符号不同子载波上。最后对剩下的 REG 将重新格式化，划分 CCE，映射 PDCCH。

因此，需要先排除 Reference Symbol、PCFICH、PHICH 所占的资源，才能确定某个下行子帧可用的 CCE 数。这里我们通过 4 个步骤来介绍如何确定某个下行子帧所包含的 CCE 总数。

步骤一：确定影响因素。

（1）参考信号

小区特定的参考信号：如果使用 1 或 2 天线端口传输，则小区特定的参考信号在频域上间隔 6 个子载波，时域上位于每个 slot 的第 1 个和倒数第 3 个 OFDM 符号上。如果使用 4 天线端口传输，

则除了在每个 slot 的第 1 个和倒数第 3 个 OFDM 符号上外，小区特定的参考信号还需要在每个 slot 的第 2 个 OFDM 符号上传输。（具体见 3.1 节关于“小区特定的参考信号”的介绍）

UE 特定的参考信号：不在天线端口 0~3 中传输，只随数据部分（data region）一起传输，且不插入小区特定的参考信号所位于的 OFDM 符号中，因此不占用控制区域的资源。（具体见 3.2 节关于“UE 特定的参考信号”的介绍）

（2）PCFICH

PCFICH 用于通知 UE 当前子帧的控制区域大小，一个小区在每个子帧有且仅有一个 PCFICH。PCFICH 位于子帧的第 1 个 OFDM 符号上，共占 4 个 REG。4 个 REG 在频域上的位置由小区的 PCI 决定。（具体见 4.2 节关于“PCFICH”的介绍）

（3）PHICH

通过读取 MIB 确定 PHICH group 的个数及其资源分布（从 UE 角度上看）。每个 PHICH group 映射到 3 个 REG，且 REG 间相隔近似 1/3 的下行系统带宽。通常位于第 1 个 OFDM 符号中传输，但也可以通过 *phich-Duration* 配置 PHICH 在控制区域的 3 个 OFDM 符号上发送。（具体见 4.3 节关于“PHICH”的介绍）

1 PHICH group = 8 PHICHs (正常的循环前缀)

1 PHICH group = 4 PHICHs (扩展的循环前缀)

（4）PDCCH

主要用于传输下行控制信息（以便 UE 正确接收 PDSCH）和 UL Grant（为 PUSCH 分配上行资源）。其分配以 CCE 为单位，1 CCE = 9 REG = 36 RE = 72 bits。

附：同步信号 PSS/SSS（见 36.211 的 6.11 节）

FDD 中，PSS 和 SSS 都随着子帧 0 和 5 的第一个 slot 传输，其中 PSS 位于该 slot 的最后一个 OFDM 符号上，SSS 位于该 slot 的倒数第二个 OFDM 符号上。

因此对于 FDD 而言，PSS/SSS 是不在控制区域内传输的。

TDD 中，PSS 随着子帧 1 和 6 的第三个 OFDM 符号传输，SSS 随着子帧 0 和 5 的最后一个 OFDM 符号传输。

因此对于 TDD 而言，子帧 1 和子帧 6 最多只能有 2 个 OFDM 符号，因为在这些子帧中，PSS 要占据第三个 OFDM 符号。

步骤二：确定某个下行子帧可用的 REG 总数。

（1）综合 PCFICH 等的因素，计算控制区域所占的 OFDM 符号数（具体见 4.2 节关于“PCFICH”的介绍）。

（2）一个下行子帧可用的 REG 总数的计算可参见 4.1 节中关于“REG”的介绍。

步骤三：确定 PCFICH 和 PHICH 所占用的 REG 数。

每个下行子帧的 PCFICH 固定占 4 个 REG。

一个 PHICH group 占 3 个 REG。每个子帧中包含的 PHICH group 总数 $N_{\text{PHICH}}^{\text{group}}$ 的计算可参见 4.3 节的介绍。PHICH 所占的 REG 个数为 $3*m_i \cdot N_{\text{PHICH}}^{\text{group}}$（对于 FDD 而言，$m_i$ 固定为 1）。

步骤四：确定 CCE 总数。

1 CCE = 9 REG = 36 RE = 72 bits

PDCCH 可用的 CCE 数 $N_{CCE} = \lfloor N_{REG} / 9 \rfloor$，其中 N_{REG} =（控制区域的所有 OFDM 符号中包含的 REG 总数 - PCFICH 占用的 REG 数（固定为 4）- PHICH 占用的 REG 数（不一定存在）。

关于 CCE 总数计算的具体例子可参见参考资料[8]和[9]中的介绍。

4.4.3 PDCCH 盲检

DCI 有多种格式（见 36.212 的 5.3.3.1 节），但 UE 事先并不知道接收到的 PDCCH 携带的是哪种格式的 DCI，也不知道该 DCI 使用哪个 PDCCH candidate 进行传输，所以 UE 必须进行 PDCCH 盲检以接收对应的 DCI。

步骤一：UE 需要计算可用于 PDCCH 的 CCE 总数。

UE 通过 PSS/SSS，确定了物理层小区 ID（PCI）和 frame timing（说得通俗一点，就是 subframe number #0 所在的位置，但此时还不知道 system frame number）。因为小区特定的参考信号及其 frequency shift（指定 RS 的位置）与 PCI 一一对应，所以间接确定了小区特定的参考信号及其在 RB 中的位置（见第 19 章关于“小区搜索过程”的介绍）。

接着 UE 就可以进行信道估计并进一步解调 PBCH，从而获取 system frame number、PHICH 占用的资源分布和天线端口数（见 20.1 节关于“MIB”的介绍）。再通过解调 PCFICH 获取 CFI，就知道了控制区域占用的 OFDM 符号数。

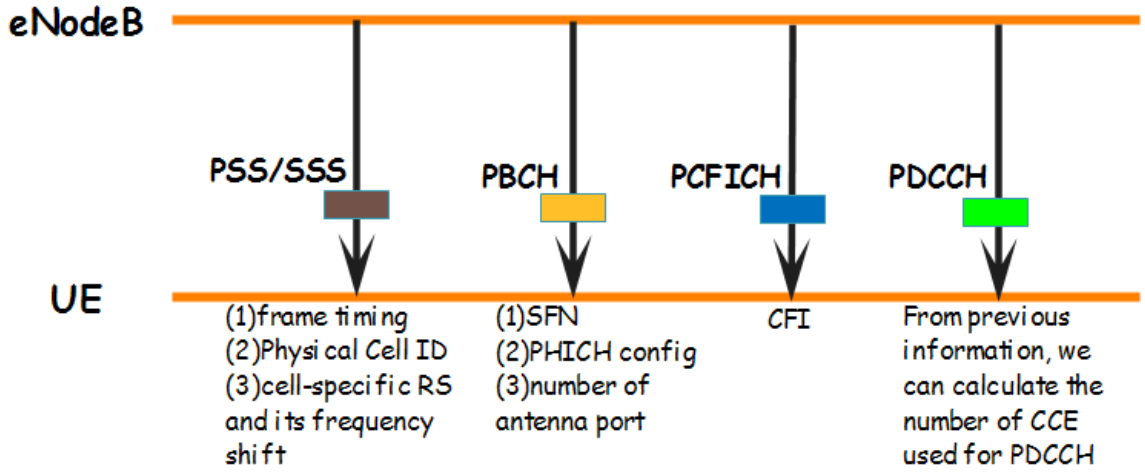


图 4-7：UE 启动时接收的信息

至此，PCFICH 的内容已经解调，PHICH 的分布由 PBCH 确定，参考信号分布取决于物理小区 ID 和 PBCH 使用的天线端口数，从而 PDCCH 在一个子帧内所能占用的 CCE 总数就可以确定了。

步骤二：进行 PDCCH 盲检。

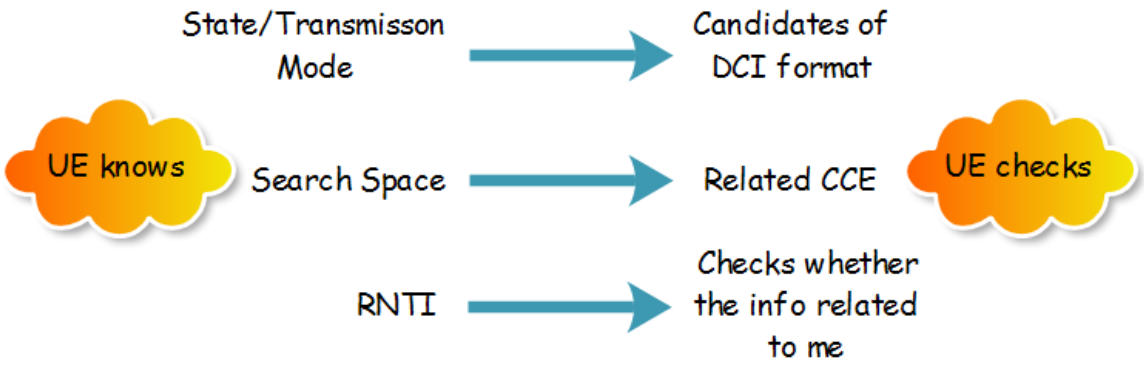


图 4-8：PDCCH 盲检时，UE 已经知道已经要确定的信息

虽然 UE 事先并不知道要接收的 PDCCH 携带的是哪种格式的 DCI，也不知道该 DCI 使用哪个 PDCCH candidate 进行传输，但 UE 知道自己处于何种状态以及在该状态下期待收到的 DCI 信息。

例如在 IDLE 态时 UE 期待收到 Paging；在发起 Random Access 后 UE 期待的是 RAR；在有上行数据待发送时期待 UL Grant 等。

并且 UE 知道自己的搜索空间，因此知道 DCI 可能分布在哪些 CCE 上。对于不同的期望信息，UE 尝试使用相应的 *X-RNTI*、可能的 DCI format、可能的聚合等级（Aggregation Level，简称 AL）去与属于自己的搜索空间内的 CCE 做 CRC 校验。如果 CRC 校验成功，那么 UE 就知道这个信息是自己需要的，也就知道相应的 DCI format，从而进一步解出 DCI 内容。

UE 不知道要收到的 PDCCH 使用哪种聚合等级，所以 UE 会把所有可能性都尝试一遍。例如：对于公共搜索空间，UE 需要分别按 Aggregation Level = 4 和 Aggregation Level = 8 来搜索。当按 AL = 4 盲检时，16 个 CCE 需要盲检 4 次，即有 4 个 PDCCH candidate；当按 AL = 8 盲检时，16 个 CCE 需要盲检 2 次，也就是有 2 个 PDCCH candidates；那么对于公共空间来说，一共有 4+2=6 个 PDCCH candidates。而对于 UE 特定的搜索空间，UE 需要分别按 Aggregation Level = 1、2、4、8 来盲检一遍，此时一共有 6+6+2+2=16 个 PDCCH candidates。（见 36.213 的 Table 9.1.1-1）

UE 在搜索空间进行盲检时，只需对可能出现的 DCI format 进行尝试解码，并不需要对所有的 DCI format 进行匹配。可能出现的 DCI format 取决于 UE 期望接收什么信息以及传输模式（见 36.213 的 7.1 节和 8.0 节）。例如：如果 UE 期待接收 DL-SCH 并使用 TM 3，当 UE 对使用 C-RNTI 扰码的 PDCCH 进行解码时，只会使用自己的 C-RNTI 对 DCI format 1A 和 DCI format 2A 进行尝试解码。如果同时该 UE 期望在该子帧内接收系统信息（SI），则会使用 SI-RNTI 对 DCI format 1A 和 DCI format 1C 进行尝试解码。更确切地说，UE 使用对应 DCI format 的 payload 长度来尝试盲检。

在成功解码 PDCCH 之前，UE 会在每一个可能的 PDCCH candidate 上尝试解码，一旦解码成功就停止解码过程。

为什么 UE 进行 PDCCH 盲检的总次数不超过 44 次？

从 36.213 的 Table 9.1.1-1 可以看出，对于某种 DCI format 进行盲检时，可能的 candidate 有 22 个。

从 36.213 的 7.1 节和 8.0 节可以看出，在某种传输模式或状态下（如随机接入时使用 RA-RNTI）解码时，可能的 DCI format 最多有 2 种。

因此 UE 进行 PDCCH 盲检的总次数不超过 44（22 * 2）次。

注：（1）Rel-10 中，如果 UE 配置了上行 MIMO（使用上行 TM 2，见 36.213 的 Table 8-3），则盲检次数还要额外增加 16 次（因为需要额外盲检 DCI format 4，而 DCI format 4 只在 UE 特定的搜索空间内传输），即总共的盲检次数将达到 60 次。（2）如果 UE 在上下行都有数据传输（需要

同时盲检上下行 DCI），且 UE 在上行配置的是 TM 1，即在上行只使用 DCI format 0，则由于 DCI format 0 和 DCI format 1A 的大小相同，并不会额外增加盲检次数。

另外，如果 UE 配置了 SPS 调度，还可能需要使用 SPS C-RNTI 去盲检 PDCCH。

可以看出，在一个下行子帧内，UE 实际盲检的次数可能超过 44 次。如果在一个下行子帧内，UE 期望同时接收下行数据、UL Grant 和系统信息。则对每一种信息，都会使用对应的 X-RNTI 在搜索空间内对相应的 DCI format 尝试解调。

4.4.4 载波聚合对 PDCCH 的影响

如果 UE 配置了载波聚合，则 UE 会在每个 non-DRX 子帧上对所有激活的 serving cell 的搜索空间进行监听。此时每个 serving cell 的搜索空间 $S_k^{(L)}$ 内的某个 PDCCH candidate m 所占用的 CCEs 通过如下公式计算

$$L\left\{(Y_k+m')\bmod\left\lfloor N_{\mathrm{CCE},k}/L\right\rfloor\right\}+i$$

对公共搜索空间而言，$m'=m$。对 UE 特定的搜索空间而言，如果 UE 在正在监听 PDCCH 的 serving cell 上配置了 CIF，则 $m'=m+M^{(L)}\cdot n_{CI}$，其中 n_{CI} 为 CIF 的值（跨载波调度时才存在）；而如果该 serving cell 上没有配置 CIF，则 $m'=m$。

可以看出，如果 UE 没有配置跨载波调度（CIF），则每个 serving cell 的搜索空间与 Rel-8 是一致的。

如果 UE 配置了跨载波调度（CIF），则与 Rel-8 相比，跨载波调度的 serving cell 在计算搜索空间时添加了偏移量 $M^{(L)}\cdot n_{CI}$。

UE 的某个 serving cell 发送的 PDCCH 是否存在 CIF 字段是通过 *cif-Presence-r10* 字段来控制的。

PCell 对应的 CIF 固定为 0；如果 CIF 的值不为 0，则该 PDCCH 对应 *sCellIndex-r10* 等于 CIF 的 SCell 上的资源。即 CIF 的值与 *SCellToAddMod-r10* 中的 *sCellIndex-r10* 值是一致的。

关于载波聚合以及 CIF，可参见第 23 章关于“载波聚合”的介绍。

对于某个 UE 而言，只在 PCell 上存在公共搜索空间。也就是说，UE 在 SCell 上只会监听 UE 特定的搜索空间，这是因为 UE 不会从 SCell 上接收 SI、Paging、RAR 以及 power-control commands for a group of UEs 等消息。

公共搜索空间上发送的 PDCCH 是不带 CIF 字段的。

对于 UE 特定的搜索空间，可以分为以下几种情况：

情况一：UE 在所有 serving cell 上都没有配置 CIF，此时 UE 会在每个激活的 serving cell 上监听各自 UE 特定的搜索空间上的 PDCCH。每个 serving cell 的 PDCCH 都在该 serving cell 对应的载波上传输，且所有的 PDCCH 都不带 CIF 字段。

情况二：UE 在 serving cell *c* 上配置了 CIF，此时该 serving cell 上的 PDCCH 必定不会在其它 serving cell 上发送。UE 会在该 serving cell 的 UE 特定的搜索空间上监听带 CIF 且使用 C-RNTI 加扰的 PDCCH。

情况三：UE 在 serving cell *c* 上没有配置 CIF 且没有配置为在另一个 serving cell 上发送 PDCCH，则 UE 会在该 serving cell 的 UE 特定的搜索空间上监听不带 CIF 字段的 PDCCH。

情况四：UE 在 PCell 上配置了 CIF，且配置了 SPS。此时 UE 会在 PCell 的 UE 特定的搜索空间上监听带 CIF 且使用 SPS C-RNTI 加扰的 PDCCH。需要说明的是：（1）PCell 上的 PDCCH 永远不会在其它 serving cell（SCell）上发送；（2）SPS 调度只存在于 PCell 上。

情况五：UE 的某个 SCell 被配置为在另一个 serving cell 上发送对应该 SCell 的带 CIF 的 PDCCH，此时 UE 不会在该 SCell 上监听 PDCCH。而在替该 SCell 发送 PDCCH 的另一个 serving cell 上，UE 至少会监听其自身的 PDCCH。

需要说明的是，以上情况并不是完全互斥的，UE 在某个 serving cell 上的配置可能同时满足几种情况。

对于 eNodeB 来说，它在发送带 CIF 的 PDCCH 时，知道该 PDCCH 对应哪个 serving cell，也知道该 PDCCH 可选的 PDCCH candidate 集合（与 n_{CI} 相关）。

但对于 UE 来说，它并不确定 PDCCH 中携带的 CIF 值是什么，即不确定哪个 serving cell 会给该 UE 发送 PDCCH。UE 只知道每个特定的 serving cell 给该 UE 发送的 PDCCH 上可能携带的 CIF 的集合，因此 UE 会在该 serving cell 上尝试所有可能的 CIF 值去盲检 PDCCH。

UE 只需要在 PCell 和已激活的 SCell 上盲检 PDCCH。

配置了 CA 的 UE 需要进行的盲检次数最多为 44 + 32 * numOfActivatedDlSCell（PCell 需要 44 次盲检；而 SCell 不需要盲检公共搜索空间，因此只需要做 32 次盲检）。

对于每一个配置了上行 MIMO（使用 DCI format 4）的 serving cell 来说，还要额外加上 16 次盲检。

4.4.5 DCI format

本节将详细介绍各种 DCI format 的每个字段的作用。主要是为了方便大家直接根据 DCI 中的某个字段找到本书对应的章节，以便了解其具体用途。

4.4.5.1 上行 DCI

上行 DCI 主要用于指示上行 PUSCH 传输，包括 DCI format 0 和 DCI format 4。

4.4.5.1.1 DCI format 0

- **Carrier indicator**－0 或 3 bit。该字段用于指示这个 DCI 调度的是哪一个 serving cell 上的资源。只有使能了载波聚合中的跨载波调度特性时，该字段才会存在。如果 *PhysicalConfigDedicated* 中的 *cif-Presence-r10* 字段设置为 TRUE，则 PCell 上发送的 DCI 的该字段为 3 比特。如果 *CrossCarrierSchedulingConfig-r10* 中的 *cif-Presence-r10* 字段设置为 TRUE，则对应的 SCell 上发送的 DCI 的该字段为 3 比特。见 4.4.4 节和 23.7.3 节的介绍。
- **Flag for format0/format1A differentiation** －1 bit。值为 0 时表示这个 DCI 是 DCI format 0；值为 1 时表示这个 DCI 是 DCI format 1A。由于 DCI format 0 和 DCI format 1A 的大小相同，所以二者需要使用这个字段予以区分，而其它 DCI format 并不需要这个字段。
- **Frequency hopping flag** －1 bit。用于指示是否使用 PUSCH 跳频，具体见 36.213 的 8.4 节。对于上行资源分配类型 1（不支持 PUSCH 跳频），此比特还用于对应资源分配字段（Resource block assignment and hopping resource allocation 字段）的 MSB（最高比特位）。见 9.2 节的介绍。
- **Resource block assignment and hopping resource allocation** － $\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{UL}}(N_{\mathrm{RB}}^{\mathrm{UL}}+1)/2) \rceil$ bits
    - 对于使用 PUSCH 跳频的情况（只有上行资源分配类型 0 支持 PUSCH 跳频）
        - N_{UL_hop} 个 MSB 比特用于指示使用的是 PUSCH 跳频类型 1 还是 PUSCH 跳频类型 2。如果使用 PUSCH 跳频类型 1，该值还指定了 $\tilde{n}_{PRB}(i)$ 的取值。
        - 剩余的 $\left(\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{UL}}(N_{\mathrm{RB}}^{\mathrm{UL}}+1)/2) \rceil - N_{\mathrm{UL_hop}} \right)$ 个比特指定了该上行子帧的第一个 slot 的资源分配。
        - 关于 PUSCH 跳频的处理，参见 9.2 节的介绍。
    - 对于使用上行资源分配类型 0 但不使用 PUSCH 跳频的情况
        - $\left(\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{UL}}(N_{\mathrm{RB}}^{\mathrm{UL}}+1)/2) \rceil \right)$ 比特。其解析见 9.1.1 节的介绍。
    - 对于使用上行资源分配类型 1 且不使用 PUSCH 跳频的情况
        - 先将 Frequency hopping flag 字段同本字段串联在一起，再根据 9.1.2 节的介绍进行解析。
- **Modulation and coding scheme and redundancy version**－5 bits。用于指定对应的 PUSCH 传输的 MCS、RV 以及用于确定 TB size。见 9.3 节的介绍。
- **New data indicator**－1 bit。用于判断当前传输是新传还是重传。见 12.2.1 节的介绍。

- **TPC command for scheduled PUSCH** – 2 bits。见 36.213 的 5.1.1.1 节。用于 PUSCH 的功控。
- **Cyclic shift for DM RS and OCC index** – 3 bits。见 36.211 的 5.5.2.1.1 节。用于支持 SU-MIMO 上区分不同层间的传输。并用于支持上行 MU-MIMO，通过给调度在相同时频资源上的不同 UE 分配不同的参考信号循环移位和 OCC，eNodeB 可以估计来自每个 UE 的上行信道并通过适当的处理来抑制 UE 间的干扰。具体见 7.1.2 节的介绍。
- **UL index** – 2 bits。只存在于 TDD 0 配置中。见 36.213 的 5.1.1.1 节、7.2.1 节、第 8 章和 8.4 节。对 TDD 0 而言，一个系统帧内的下行子帧数少于上行子帧数。因此，一个下行子帧可能需要同时给 2 个上行子帧发送 UL grant。该字段用于告诉 UE，当前的 UL grant 对应的是哪个（或 2 个）上行子帧的 PUSCH 传输。见 12.2.4.3 节的介绍。
- **Downlink Assignment Index (DAI)** – 2 bits。只存在于 TDD 1~6 配置中。见 36.213 的 7.3 节。eNodeB 使用该字段通知 UE 其期望 HARQ 反馈的下行传输数。见 13.3 节的介绍。
- **CSI request** – 1 or 2 bits。见 36.213 的 7.2.1 节。只有当 UE 配置了多于 1 个下行小区（下行载波聚合）且当对应的 DCI format 映射到 UE 特定的搜索空间（使用 C-RNTI 加扰）时，才会使用 2 比特。通过设置该字段，eNodeB 可以显式地要求 UE 在对应的 PUSCH 上发送一个非周期性 CSI 上报。见 10.4 节的介绍。
- **SRS request** – 0 or 1 bit。见 36.213 的 8.2 节。只有当正在调度的 PUSCH 对应的 DCI format 映射到 UE 特定的搜索空间（使用 C-RNTI 加扰）时，才会存在本字段。该字段用于触发 UE 是否进行非周期性 SRS 上报。见 7.2.3 节的介绍。
- **Resource allocation type** – 1 bit。该字段仅在上行系统带宽小于或等于下行系统带宽（即 $N_{\mathrm{RB}}^{\mathrm{UL}} \le N_{\mathrm{RB}}^{\mathrm{DL}}$ ）时存在。如果该字段不存在，则只支持上行资源分配类型 0。如果该字段存在，则其值为 0 时表示使用上行资源分配类型 0；其值为 1 时表示使用上行资源分配类型 1。Resource block assignment and hopping resource allocation 字段的依赖此字段进行解析。见 9.1 节的介绍。

4.4.5.1.2 DCI format 4

DCI format 4 用于 PUSCH 的调度。

- **Carrier indicator** – 0 或 3 bit。见 4.4.5.1.1 节的介绍。
- **Resource block assignment** – $\max\left(\left\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{UL}}(N_{\mathrm{RB}}^{\mathrm{UL}}+1)/2)\right\rceil, \left\lceil \log_2\left(\binom{\left\lceil N_{RB}^{UL}/P+1\right\rceil}{4}\right)\right\rceil\right)$ bits
  - 对于使用上行资源分配类型 0 的情况

  - 使用最低的 $\left(\left\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{UL}}(N_{\mathrm{RB}}^{\mathrm{UL}}+1)/2)\right\rceil\right)$ 个比特（LSB）。其解析见 9.1.1 节的介绍（或见 36.213 的 8.1.1 节）。

- 对于使用上行资源分配类型 1 的情况

  - 使用最低的 $\left\lceil \log_2\left(\binom{\left\lceil N_{RB}^{UL}/P+1\right\rceil}{4}\right)\right\rceil$ 个比特（LSB）。其解析见 9.1.2 节的介绍（或见 36.213 的 8.1.2 节）。

- **TPC command for scheduled PUSCH** – 2 bits。见 36.213 的 5.1.1.1 节。用于 PUSCH 的功控。

- **Cyclic shift for DM RS and OCC index** – 3 bits。用于支持 SU-MIMO 上区分不同层间的传输。并用于支持上行 MU-MIMO，通过给调度在相同时频资源上的不同 UE 分配不同的参考信号循环移位和 OCC，eNodeB 可以估计来自每个 UE 的上行信道并通过适当的处理来抑制 UE 间的干扰。具体见 7.1.2 节的介绍。

- **UL index** – 2 bits。只存在于 TDD 0 配置中。见 36.213 的 5.1.1.1 节、7.2.1 节、第 8 章和 8.4 节。对 TDD 0 而言，一个系统帧内的下行子帧数少于上行子帧数。因此，一个下行子帧可能需要同时给 2 个上行子帧发送 UL grant。该字段用于告诉 UE，当前的 UL grant 对应的是哪个（或 2 个）上行子帧的 PUSCH 传输。见 12.2.4.3 节的介绍。

- **Downlink Assignment Index (DAI)** – 2 bits。只存在于 TDD 1~6 配置中。见 36.213 的 7.3 节。eNodeB 使用该字段通知 UE 其期望 HARQ 反馈的下行传输数。见 13.3 节的介绍。

- **CSI request** – 1 or 2 bits。见 36.213 的 7.2.1 节。只有当 UE 配置了多于 1 个下行小区（下行载波聚合）且当对应的 DCI format 映射到 UE 特定的搜索空间（使用 C-RNTI 加扰，DCI format 4 只能映射到 UE 特定的搜索空间）时，才会使用 2 比特。通过设置该字段，eNodeB 可以显式地要求 UE 在对应的 PUSCH 上发送一个非周期性 CSI 上报。见 10.4 节的介绍。

- **SRS request** – 2 bits。见 36.213 的 8.2 节。该字段用于触发 UE 使用（至多 3 个）预先配置的设置之一来进行非周期性 SRS 上报。见 7.2.3 节的介绍。

- **Resource allocation type** – 1 bit。该字段的值为 0 时表示使用上行资源分配类型 0；值为 1 时表示使用上行资源分配类型 1。Resource block assignment 字段的依赖此字段进行解析。见 9.1 节的介绍。

对于传输块 TB 1 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PUSCH 传输的 MCS、RV 以及用于确定 TB size。见 9.3 节的介绍。

- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 12.2.1 节的介绍。

对于传输块 TB 2 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PUSCH 传输的 MCS、RV 以及用于确定 TB size。见 9.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 12.2.1 节的介绍。

- **Precoding information and number of layers** – 其所占 bit 数与 UE 侧的天线端口数相关，具体见 36.212 的 Table 5.3.3.1.8-1。该字段用于指定 PUSCH 传输所使用的预编码，其含义见 36.212 的 Table 5.3.3.1.8-2 和 Table 5.3.3.1.8-3。对于只使能了一个 codeword 的情况，Table 5.3.3.1.8-3 中的索引 24~39 只用于某个 TB 的重传，且要求该 TB 在前一次传输使用的是 2 层传输。用于 2 天线端口的 TPMI 指定了使用 36.211 的 Table 5.3.3A.2-1 中的哪个 codebook index。用于 4 天线端口的 TPMI 指定了使用 36.211 的 Table 5.3.3A.2-2（1 layer）、Table 5.3.3A.2-3（2 layers）、Table 5.3.3A.2-4（3 layers）和 Table 5.3.3A.2-5（4 layers）中的哪个 codebook index。见 9.4 节的介绍。

  如果 2 个 TB 都使能，则 TB 1 对应 codeword 0，TB 2 对应 codeword 1。如过只使能了一个 TB，则 TB 到 codeword 的映射关系见 36.212 的 Table 5.3.3.1.5-2。

Table 5.3.3.1.8-1: Number of bits for precoding information.

Number of antenna ports at UE	Number of bits for precoding information
2	3
4	6

Table 5.3.3.1.8-2: Content of precoding information field for 2 antenna ports

One codeword: Codeword 0 enabled Codeword 1 disabled		Two codewords: Codeword 0 enabled Codeword 1 enabled	
Bit field mapped to index	**Message**	**Bit field mapped to index**	**Message**
0	1 layer: TPMI=0	0	2 layers: TPMI=0
1	1 layer: TPMI=1	1-7	reserved
2	1 layer: TPMI=2		
…	…		
5	1 layer: TPMI=5		
6-7	reserved		

Table 5.3.3.1.8-3: Content of precoding information field for 4 antenna ports

One codeword: Codeword 0 enabled Codeword 1 disabled		Two codewords: Codeword 0 enabled Codeword 1 enabled	
Bit field mapped to index	**Message**	**Bit field mapped to index**	**Message**
0	1 layer: TPMI=0	0	2 layers: TPMI=0
1	1 layer: TPMI=1	1	2 layers: TPMI=1
…	…	…	…
23	1 layer: TPMI=23	15	2 layers: TPMI=15
24	2 layers: TPMI=0	16	3 layers: TPMI=0
25	2 layers: TPMI=1	17	3 layers: TPMI=1
…	…	…	…
39	2 layers: TPMI=15	27	3 layers: TPMI=11
40-63	reserved	28	4 layers: TPMI=0
		29 - 63	Reserved

4.4.5.2 下行 DCI

下行 DCI 主要用于发送下行调度分配信息，包括 DCI format 1/1A/1B/1C/1D/2/2A/2B/2C。

4.4.5.2.1 DCI format 1

- **Carrier indicator** – 0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Resource allocation header (resource allocation type 0 / type 1)** – 1 bit。值为 0 时表示使用下行资源分配类型 0；值为 1 时表示使用下行资源分配类型 1。如果下行系统带宽小于或等于 10 个 PRB，则不存在这个字段，此时使用下行资源分配类型 0。见 5.1 节的介绍。
- **Resource block assignment**（关于该字段的解析，见 5.1 节的介绍）
    - 对于下行资源分配类型 0（见 36.213 的 7.1.6.1 节）
        - 该字段包含 $\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \rceil$ 比特。
    - 对于下行资源分配类型 1（见 36.213 的 7.1.6.2 节）
        - 该字段的 $\lceil \log_2(P) \rceil$ 比特用于指定所选的 RBG 子集。
        - 该字段的 1 比特用于指定子集内的资源是否偏移，1 表示偏移，0 表示不偏移。
        - 该字段的剩余 $\left(\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \rceil - \lceil \log_2(P) \rceil - 1 \right)$ 比特用于指定资源分配。
- **Modulation and coding scheme** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **HARQ process number** – 3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍。

- **New data indicator**－1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version**－2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。
- **TPC command for PUCCH** －2 bits。用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。
- **Downlink Assignment Index (DAI)** －2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。

4.4.5.2.2 DCI format 1A

DCI format 1A 不仅可用于单个 PDSCH 码字的调度，还可被用作 PDCCH order 来触发随机接入过程。

- **Carrier indicator**－0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Flag for format0/format1A differentiation** －1 bit。见 4.4.5.1.1 节介绍。

只有当 DCI format 1A 的 CRC 使用 C-RNTI 来加扰，且剩余的字段按下面的介绍设置时，DCI format 1A 才会被用作 PDCCH order 来触发随机接入过程。（见第 21 章的介绍）

- **Localized/Distributed VRB assignment flag**－1 bit。该字段设置为‘0’。
- **Resource block assignment**－$\left\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \right\rceil$ bit。该字段的所有所有比特设置为 1。
- **Preamble Index**－6 bits。用于指定 preamble ID。见第 21 章的介绍。
- **PRACH Mask Index**－4 bits。见第 21 章的介绍。
- 剩下的所有比特都设置为 0。

否则，

- **Localized/Distributed VRB assignment flag**－1 bit。值为 0 时表示使用集中式 VRB，值为 1 时表示使用分布式 VRB。见 5.2 节的介绍。
- **Resource block assignment**－$\left\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \right\rceil$ bits。（关于该字段的解析，详见 5.1 节的介绍。或见 36.213 的 7.1.6.3 节）
    - 对于集中式 VRB：$\left\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \right\rceil$ 比特用于提供资源分配信息。
    - 对于分布式 VRB：

        - 如果 $N_{\mathrm{RB}}^{\mathrm{DL}} < 50$ ，或者 DCI format 1A 的 CRC 使用 RA-RNTI，P-RNTI 或 SI-RNTI 加扰，则 $\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \rceil$ 比特用于提供资源分配信息。
        - 否则，该字段的最高 1 比特指示 gap 值，值为 0 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},1}$ ，值为 1 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},2}$ 。而剩余的 $(\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \rceil - 1)$ 比特用于提供资源分配信息。

- **Modulation and coding scheme**－5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **HARQ process number**－3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍。
- **New data indicator**－1 bit。用于判断当前传输是新传还是重传。
    - 如果 DCI format 1A 的 CRC 使用 RA-RNTI，P-RNTI 或 SI-RNTI 加扰，
        - 如果 $N_{\mathrm{RB}}^{\mathrm{DL}} \geq 50$ 且“Localized/Distributed VRB assignment flag”字段设置为 1，即使用分布式 VRB，则该字段指示 gap 值，值为 0 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},1}$ ，值为 1 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},2}$ 。
        - 否则该字段是预留的。
    - 否则，该字段用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version**－2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。
- **TPC command for PUCCH**－2 bits。
    - 如果 DCI format 1A 的 CRC 使用 RA-RNTI，P-RNTI 或 SI-RNTI 加扰，
        - 该字段的最高 1 比特是预留的；
        - 该字段的最低 1 比特（LSB）用于指定 36.213 中定义的 TBS table 的列 $N_{\mathrm{PRB}}^{1\mathrm{A}}$ ；
        - 该字段的最低 1 比特的值为 0 时 $N_{\mathrm{PRB}}^{1\mathrm{A}} = 2$ ，否则 $N_{\mathrm{PRB}}^{1\mathrm{A}} = 3$。见 9.3.3 节的介绍。
    - 否则，该字段用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。
- **Downlink Assignment Index (DAI)**－2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。

- **SRS request** – 0 or 1 bit。见 36.213 的 8.2 节。只有当正在调度的 PDSCH 传输对应的 DCI format 映射到 UE 特定的搜索空间（使用 C-RNTI 加扰）时，才会存在本字段。该字段用于触发 UE 是否进行非周期性 SRS 上报。见 7.2.3 节的介绍。

注：如果 DCI format 1A 的 CRC 使用 RA-RNTI，P-RNTI 或 SI-RNTI 加扰，则以下 2 个字段是预留的：

- HARQ process number
- Downlink Assignment Index (DAI)

4.4.5.2.3 DCI format 1B

- **Carrier indicator** – 0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Localized/Distributed VRB assignment flag** – 1 bit。值为 0 时表示使用集中式 VRB，值为 1 时表示使用分布式 VRB。见 5.2 节的介绍。
- **Resource block assignment** – $\left\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \right\rceil$ bits。（关于该字段的解析，详见 5.1 节的介绍。或见 36.213 的 7.1.6.3 节）
    - 对于集中式 VRB：$\left\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \right\rceil$ 比特用于提供资源分配信息。
    - 对于分布式 VRB：
        - 如果 $N_{\mathrm{RB}}^{\mathrm{DL}} < 50$，则 $\left\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \right\rceil$ 比特用于提供资源分配信息。
        - 否则，该字段的最高 1 比特指示 gap 值，值为 0 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},1}$，值为 1 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},2}$。而剩余的 $\left(\left\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \right\rceil - 1\right)$ 比特用于提供资源分配信息。
- **Modulation and coding scheme** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **HARQ process number** – 3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。
- **TPC command for PUCCH** – 2 bits。用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。

- **Downlink Assignment Index –** 2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。
- **TPMI information for precoding –** 比特数见 36.212 的 Table 5.3.3.1.3A-1。其含义见 6.5.6 节中关于“TM 6”的介绍。
- **PMI confirmation for precoding –** 1 bit。其含义见 6.5.6 节中关于“TM 6”的介绍。

4.4.5.2.4 DCI format 1C

如果 DCI format 1C 用于非常紧凑的单 PDSCH 码字调度，则

- **1 bit 用于指示 gap 值。**值为 0 时表示 $N_{\text{gap}} = N_{\text{gap},1}$ ，值为 1 时表示 $N_{\text{gap}} = N_{\text{gap},2}$ 。
- 如果 $N_{\text{RB}}^{\text{DL}} < 50$ ，则不存在前面介绍用于指示 gap 的 1 比特值。
- **Resource block assignment –** $\left\lceil \log_2 \left(\left\lfloor N_{\text{VRB,gap1}}^{\text{DL}} / N_{\text{RB}}^{\text{step}} \right\rfloor \cdot \left(\left\lfloor N_{\text{VRB,gap1}}^{\text{DL}} / N_{\text{RB}}^{\text{step}} \right\rfloor + 1 \right) / 2 \right) \right\rceil$ bits 。其解析见 5.1.4 节的介绍。
- **Modulation and coding scheme –** 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。

否则，DCI format 1C 用于通知 MCCH 变化，有

- **Information for MCCH change notification–** 8 bits。见 36.331 的 5.8.1.3 节。
- 剩余的比特是预留的，其填充的长度等于 DCI format 1C 用于非常紧凑的单 PDSCH 码字调度时的长度。

4.4.5.2.5 DCI format 1D

- **Carrier indicator –** 0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Localized/Distributed VRB assignment flag –** 1 bit。值为 0 时表示使用集中式 VRB，值为 1 时表示使用分布式 VRB。见 5.2 节的介绍。
- **Resource block assignment –** $\left\lceil \log_2 (N_{\text{RB}}^{\text{DL}} (N_{\text{RB}}^{\text{DL}} + 1) / 2) \right\rceil$ bits。（关于该字段的解析，详见 5.1 节的介绍。或见 36.213 的 7.1.6.3 节）
    - 对于集中式 VRB：$\left\lceil \log_2 (N_{\text{RB}}^{\text{DL}} (N_{\text{RB}}^{\text{DL}} + 1) / 2) \right\rceil$ 比特用于提供资源分配信息。
    - 对于分布式 VRB：

- 如果 $N_{\mathrm{RB}}^{\mathrm{DL}} < 50$ ，则 $\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \rceil$ 比特用于提供资源分配信息。
- 否则，该字段的最高 1 比特指示 gap 值，值为 0 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},1}$ ，值为 1 时表示 $N_{\mathrm{gap}} = N_{\mathrm{gap},2}$ 。而剩余的 $(\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{DL}}(N_{\mathrm{RB}}^{\mathrm{DL}}+1)/2) \rceil - 1)$ 比特用于提供资源分配信息。

- **Modulation and coding scheme** －5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **HARQ process number** －3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍
- **New data indicator**－1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version**－2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。
- **TPC command for PUCCH** －2 bits。用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。
- **Downlink Assignment Index**－2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。
- **TPMI information for precoding**－比特数见 36.212 的 Table 5.3.3.1.4A-1。其含义见 6.5.5 节中关于“TM 5”的介绍。
- **Downlink power offset**－1 bit。其含义见 6.5.5 节中关于“TM 5”的介绍。

4.4.5.2.6 DCI format 2

- **Carrier indicator**－0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Resource allocation header (resource allocation type 0 / type 1)** －1 bit。值为 0 时表示使用下行资源分配类型 0；值为 1 时表示使用下行资源分配类型 1。如果下行系统带宽小于或等于 10 个 PRB，则不存在这个字段，此时使用下行资源分配类型 0。见 5.1 节的介绍。
- **Resource block assignment**（关于该字段的解析，见 5.1 节的介绍）
  - 对于下行资源分配类型 0（见 36.213 的 7.1.6.1 节）
    - 该字段包含 $\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \rceil$ 比特。
  - 对于下行资源分配类型 1（见 36.213 的 7.1.6.2 节）
    - 该字段的 $\lceil \log_2(P) \rceil$ 比特用于指定所选的 RBG 子集。

- 该字段的 1 比特用于指定子集内的资源是否偏移，1 表示偏移，0 表示不偏移。
- 该字段的剩余 $\left(\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \right\rceil - \left\lceil \log_2(P) \right\rceil - 1\right)$ 比特用于指定资源分配。

- **TPC command for PUCCH** – 2 bits。用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。
- **Downlink Assignment Index (DAI)** – 2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。
- **HARQ process number** – 3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍。
- **Transport block to codeword swap flag** – 1 bit。用于指定 TB 与 codeword 的对应关系，见 13.1.2 节的介绍。

对于传输块 TB 1 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

对于传输块 TB 2 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

- **Precoding information** – 比特数见 36.212 的 Table 5.3.3.1.5-3。其含义见 6.5.4 节中关于“TM 4”的介绍。

4.4.5.2.7 DCI format 2A

- **Carrier indicator** – 0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Resource allocation header (resource allocation type 0 / type 1)** – 1 bit。值为 0 时表示使用下行资源分配类型 0；值为 1 时表示使用下行资源分配类型 1。如果下行系统带宽小于或等于 10 个 PRB，则不存在这个字段，此时使用下行资源分配类型 0。见 5.1 节的介绍。

- **Resource block assignment**（关于该字段的解析，见 5.1 节的介绍）
    - 对于下行资源分配类型 0（见 36.213 的 7.1.6.1 节）
        - 该字段包含$\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \right\rceil$ 比特。
    - 对于下行资源分配类型 1（见 36.213 的 7.1.6.2 节）
        - 该字段的$\left\lceil \log_2(P) \right\rceil$ 比特用于指定所选的 RBG 子集。
        - 该字段的 1 比特用于指定子集内的资源是否偏移，1 表示偏移，0 表示不偏移。
        - 该字段的剩余$\left(\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \right\rceil - \left\lceil \log_2(P) \right\rceil - 1\right)$ 比特用于指定资源分配。
- **TPC command for PUCCH** – 2 bits。用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。
- **Downlink Assignment Index (DAI)** – 2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。
- **HARQ process number** – 3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍。
- **Transport block to codeword swap flag** – 1 bit。用于指定 TB 与 codeword 的对应关系，见 13.1.2 节的介绍。

对于传输块 TB 1 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

对于传输块 TB 2 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

- **Precoding information**－比特数见 36.212 的 Table 5.3.3.1.5A-1。其含义见 6.5.3 节中关于“TM 3”的介绍。

4.4.5.2.8 DCI format 2B

- **Carrier indicator**－0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Resource allocation header (resource allocation type 0 / type 1)** －1 bit。值为 0 时表示使用下行资源分配类型 0；值为 1 时表示使用下行资源分配类型 1。如果下行系统带宽小于或等于 10 个 PRB，则不存在这个字段，此时使用下行资源分配类型 0。见 5.1 节的介绍。
- **Resource block assignment**（关于该字段的解析，见 5.1 节的介绍）
    - 对于下行资源分配类型 0（见 36.213 的 7.1.6.1 节）
        - 该字段包含$\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \right\rceil$ 比特。
    - 对于下行资源分配类型 1（见 36.213 的 7.1.6.2 节）
        - 该字段的$\left\lceil \log_2(P) \right\rceil$ 比特用于指定所选的 RBG 子集。
        - 该字段的 1 比特用于指定子集内的资源是否偏移，1 表示偏移，0 表示不偏移。
        - 该字段的剩余$\left(\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \right\rceil - \left\lceil \log_2(P) \right\rceil - 1\right)$ 比特用于指定资源分配。
- **TPC command for PUCCH** －2 bits。用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。
- **Downlink Assignment Index (DAI)** －2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。
- **HARQ process number**－3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍。
- **scrambling identity**－1 bit。用于指定 n_{SCID} 值。见 6.5.8 节关于“TM 8”的介绍。
- **SRS request**－0 或 1 bit。只在 TDD 中存在，用于触发非周期性 SRS。见 7.2.3 节的介绍。

对于传输块 TB 1 而言，

- **Modulation and coding scheme and redundancy version**－5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator**－1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

对于传输块 TB 2 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

4.4.5.2.9 DCI format 2C

- **Carrier indicator** – 0 或 3 bit。见 4.4.5.1.1 节介绍。
- **Resource allocation header (resource allocation type 0 / type 1)** – 1 bit。值为 0 时表示使用下行资源分配类型 0；值为 1 时表示使用下行资源分配类型 1。如果下行系统带宽小于或等于 10 个 PRB，则不存在这个字段，此时使用下行资源分配类型 0。见 5.1 节的介绍。
- **Resource block assignment**（关于该字段的解析，见 5.1 节的介绍）
    - 对于下行资源分配类型 0（见 36.213 的 7.1.6.1 节）
        - 该字段包含 $\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \right\rceil$ 比特。
    - 对于下行资源分配类型 1（见 36.213 的 7.1.6.2 节）
        - 该字段的 $\left\lceil \log_2(P) \right\rceil$ 比特用于指定所选的 RBG 子集。
        - 该字段的 1 比特用于指定子集内的资源是否偏移，1 表示偏移，0 表示不偏移。
        - 该字段的剩余 $\left(\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \right\rceil - \left\lceil \log_2(P) \right\rceil - 1\right)$ 比特用于指定资源分配。
- **TPC command for PUCCH** – 2 bits。用于 PUCCH 的功控，见 36.213 的 5.1.2.1 节。该字段还可用于指示使用的 PUCCH 资源，具体见第 13 章的介绍。
- **Downlink Assignment Index (DAI)** – 2 bits。所有的 TDD 配置下都存在，但只用于 TDD 1~6。FDD 中不存在该字段。见 13.3 节的介绍。
- **HARQ process number** – 3 bits (FDD), 4 bits (TDD)。用于指定当前传输对应的 HARQ process。见第 11 章和第 13 章的介绍。
- **Antenna port(s), scrambling identity and number of layers** – 3 bit。用于指定 n_{SCID} 值、天线端口和层数信息。见 6.5.9 节关于“TM 9”的介绍。
- **SRS request**– 0 或 1 bit。只在 TDD 中存在，用于触发非周期性 SRS。见 7.2.3 节的介绍。

对于传输块 TB 1 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

对于传输块 TB 2 而言，

- **Modulation and coding scheme and redundancy version** – 5 bits。用于指定对应的 PDSCH 传输的 MCS 以及用于确定 TB size。见 5.3 节的介绍。
- **New data indicator** – 1 bit。用于判断当前传输是新传还是重传。见 13.2.1 节的介绍。
- **Redundancy version** – 2 bits。用于指定当前传输的冗余版本。见 13.1.1 节的介绍。

【参考资料】

[1] TS 36.211 的 6.8 节 Physical downlink control channel

[2] TS 36.212 的 5.3.3 节 Downlink control information

[3] TS 36.213 的 7.1 节 UE procedure for receiving the physical downlink shared channel

[4] TS 36.213 的 9.1.1 节 UE procedure for determining physical downlink control channel assignment

[5] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.4.3 节

[6] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 9.3.5 节、9.3.6 节和 28.3.1.1 节

[7] http://www.sharetechnote.com 中关于 PDCCH 的介绍

[8] 《PDCCH Dimensioning》 & 《PDCCH Construction》 by HONGYAN

[9] http://www.pewscorner.host-ed.me/LTE/lte_resource_grid.html

第5章　PDSCH

PDSCH 主要用于传输来自 DL-SCH 和 PCH 的数据，更确切地说，RAR、Paging、SIB、RRC 消息（不包括 MIB）和用户数据等最终会在 PDSCH 上传输。PDSCH 是在下行子帧的数据区域（data region）上传输的。

为了接收 PDSCH，UE 需要先解码 PDCCH。PDCCH 携带的 DCI 指定了在空口上如何传输 PDSCH。DCI 指定了 PDSCH 所占的资源、MCS、初传或重传相关信息、层、预编码等信息。

本章主要从 UE 的角度来介绍 UE 如何从 PDCCH 中获取到对应 PDSCH 的资源分配信息（5.1 节）和调制阶数（5.3.1 节），以及如何确定 PDSCH 所携带的 TB 的大小（5.3.2 节）。同时还会介绍分配的 PDSCH 资源是如何从 VRB 映射到 PRB 的（5.2 节）。

5.1　下行资源分配类型

本节主要介绍下行物理信道 PDSCH 的 3 种资源分配类型：Type 0、Type 1 和 Type 2。通过本节的介绍，大家可以知道 eNodeB 在当前的 PDSCH 传输中给 UE 分配了哪些 VRB 资源。

PDSCH 传输具体使用哪种资源分配类型取决于 eNodeB 所选的 DCI format 以及 DCI 内相关比特的配置。

每种下行 DCI format 支持哪种资源分配类型，以及有哪些与资源分配相关的比特，可参见 4.4.5 节中关于各种下行 DCI format 的介绍（或见 36.212 的 5.3.3 节）。本节会重点对与下行资源分配类型相关的 DCI format 字段进行分析。

表 5-1 描述了几种下行 DCI format 与下行资源分配类型的对应关系。

表 5-1：下行 DCI format 与下行资源分配类型的对应关系

DCI format	Type 0	Type 1	Type 2
1	Yes	Yes	No
1A	No	No	Yes
1B	No	No	Yes
1C	No	No	Yes
1D	No	No	Yes
2	Yes	Yes	No
2A	Yes	Yes	No
2B	Yes	Yes	No
2C	Yes	Yes	No

注意：（1）下行 PDSCH 资源是基于 VRB 而不是 PRB 分配的。当然，VRB 与 PRB 有一定的对应关系，具体见 36.211 的 6.2.3 节或见本章的 5.2 节。VRB 的相关概念会在 5.2 节予以介绍；（2）DCI format 1/2/2A/2B/2C 同时支持 Type 0 和 Type 1，具体使用哪种类型是通过对应 DCI 的 1 比特“Resource allocation header”字段来指定的（见图 5-1）。如果对应 DCI format 中的“Resource allocation header”字段设置为 0，则表示使用资源分配类型 0；否则表示使用资源分配类型 1。当下行系统带宽小于或等于 10 RB 时，“Resource allocation header”字段是不存在的，此时默认使用资源分配类型 0。

5.1.1 RBG 介绍

在介绍资源分配类型 Type 0 和 Type 1 之前，需要先介绍一下 RBG 的概念。

RBG（Resource Block Group，资源块组）是一组连续的集中式 VRB（localized VRB）。RBG 的大小（P，即每个 RBG 中包含的 VRB 数。最后一个 RBG 包含的 VRB 数可能小于 P）与下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}}$ 相关，对应关系见 36.213 的 Table 7.1.6.1-1。

Table 7.1.6.1-1: Type 0 Resource Allocation RBG Size vs. Downlink System Bandwidth

System Bandwidth $N_{\mathrm{RB}}^{\mathrm{DL}}$	RBG Size (*P*)
≤10	1
11 – 26	2
27 – 63	3
64 – 110	4

对应下行系统带宽 N_{RB}^{DL}，RBG 的总数 N_{RBG} 为：

$$N_{RBG} = \lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \rceil$$

其中，前 $\lfloor N_{\mathrm{RB}}^{\mathrm{DL}} / P \rfloor$ 个 RBG 的大小为 P；如果 $N_{RBG} \ \% \ P > 0$，则最后一个 RBG 的大小为 $N_{\mathrm{RB}}^{\mathrm{DL}} - P \cdot \lfloor N_{\mathrm{RB}}^{\mathrm{DL}} / P \rfloor$。RBG 的索引是从最低的频率开始，并按照频率递增的顺序进行编号的。

以下行系统带宽 N_{RB}^{DL} = 50 RB 为例，查 Table 7.1.6.1-1 可知，其 P 值为 3，RBG 的总数 N_{RBG} 为 17，前 16 个 RBG 各包含 3 个 VRB，最后一个 RBG 只包含 2 个 VRB。

5.1.2 资源分配类型 0（Resource allocation type 0）

在资源分配类型 0 中，DCI format 1/2/2A/2B/2C 通过一个 bitmap（如图 5-1 所示）来指示分配给 UE 的 RBG。bitmap 共包含 $N_{RBG}=\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/P\rceil$ 比特，每 1 比特对应 1 个 RBG，最高位表示 RBG 0，最低位表示 RBG $N_{RBG}-1$，依此类推。如果某个 RBG 分配给了某个 UE，则 bitmap 中对应比特置为 1；否则置为 0。

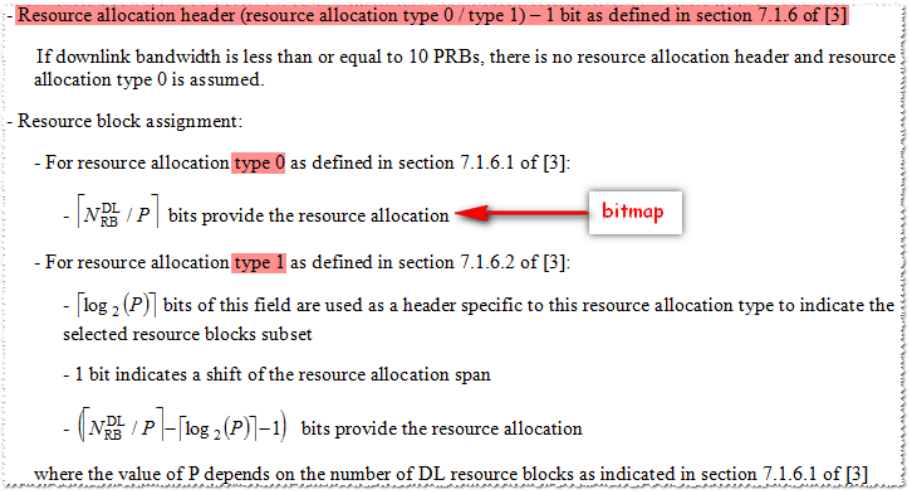


图 5-1：DCI format 1/2/2A/2B/2C 中与 Type 0 相关的字段

以小区下行系统带宽 25 RB 为例。

（1）通过查 36.213 的 Table 7.1.6.1-1 可以知道，RBG 大小 P = 2；

（2）RBG 的总数 $N_{RBG}=\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/P\rceil=\lceil 25/2\rceil=13$。其中前 12 个 RBG 的每个 RBG 大小为 2，最后一个 RBG 的大小为 1（如表 5-2 所示）；

表 5-2：资源分配类型 0 的 RBG 资源与 VRB 的对应关系（25 RB）

RBG 0	RBG 1	RBG 2	RBG 3	RBG 4	RBG 5	RBG 6	RBG 7	RBG 8	RBG 9	RBG 10	RBG 11	RBG 12
VRB 0, VRB 1	VRB 2, VRB 3	VRB 4, VRB 5	VRB 6, VRB 7	VRB 8, VRB 9	VRB 10, VRB 11	VRB 12, VRB 13	VRB 14, VRB 15	VRB 16, VRB 17	VRB 18, VRB 19	VRB 20, VRB 21	VRB 22, VRB 23	VRB 24

（3）即 bitmap 共包含 13 比特。

（4）假设分配给某 UE 的资源的 bitmap 为：1001110100010，则该 UE 被分配了 RBG 0、RBG 3、RBG 4、RBG 5、RBG 7、RBG 11（如图 5-2 所示）。

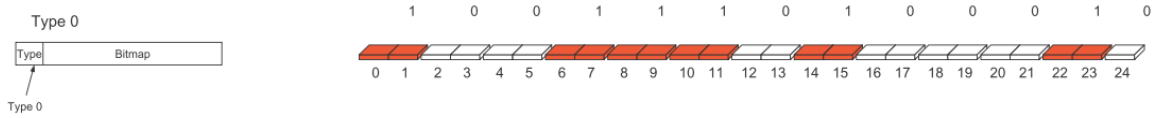


图 5-2：资源分配类型 0 的例子（25 RB）

从上面的例子可以看出：（1）资源分配类型 0 支持频域上的非连续 RB 分配；（2）调度的粒度比较粗：调度的最小单位是 RBG，对于较大的带宽而言，无法按照单个 RB 来分配资源。当 payload 较小时，可能会造成资源的浪费。

5.1.3 资源分配类型 1（Resource allocation type 1）

在资源分配类型 1 中，所有的 RBG 被分为 P 个子集，P 为 RBG 的大小（见 5.1.1 节）。每个 RBG 子集 p（$0 \le p < P$）包含从 RBG p 开始，间隔为 P 的所有 RBG。分配给某个 UE 的 VRB 资源必须来自于同一个子集。

在资源分配类型 1 中，DCI format 1/2/2A/2B/2C 通过 3 个域来指示分配给 UE 的 VRB（**注意**：与资源分配类型 0 不同，这里是 VRB，而不是 RBG），见图 5-3。

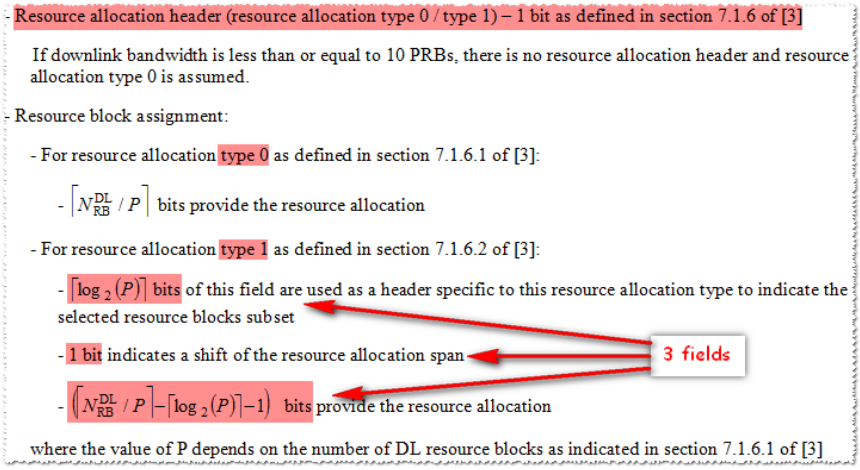


图 5-3：DCI format 1/2/2A/2B/2C 中与 Type 1 相关的字段

第一个域包含 $\lceil \log_2(P) \rceil$ 比特，用于指定所选的 RBG 子集，即 p 的值。

第二个域包含 1 比特（shift bit），用于指定子集内的资源是否偏移，1 表示偏移，0 表示不偏移。

第三个域包含一个 bitmap，bitmap 的每一比特对应所选 RBG 子集中的一个 VRB（**注意**：不是 RBG）。最高位表示子集中的第一个 VRB，最低位表示子集中的最后一个 VRB，依此类推。如果某个 VRB 分配给了某个 UE，则 bitmap 中对应比特置为 1；否则置为 0。bitmap 的大小，即 bitmap 包含的比特数 $N_{\mathrm{RB}}^{\mathrm{TYPE1}}$ 为

$$N_{\mathrm{RB}}^{\mathrm{TYPE1}}=\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/P\right\rceil-\left\lceil \log_2(P)\right\rceil-1$$

一个选定的 RBG 子集中的 VRB 起始于该子集中的最小 VRB 号 + 偏移量 $\Delta_{\mathrm{shift}}(p)$，并对应 bitmap 中的最高位。该偏移量以 VRB 的数量表示，并且是发生在选定的 RBG 子集内的偏移。如果 DCI 的资源块分配信息中的第二个域为 0，则 RBG 子集 p 的偏移 $\Delta_{\mathrm{shift}}(p)=0$；如果 DCI 的资源块分配信息中的第二个域为 1，则 RBG 子集 p 的偏移 $\Delta_{\mathrm{shift}}(p)=N_{\mathrm{RB}}^{\mathrm{RBG\ subset}}(p)-N_{\mathrm{RB}}^{\mathrm{TYPE1}}$，且 bitmap 中的最低比特位调整为对应 RBG 子集中的最后一个 VRB。

$N_{\mathrm{RB}}^{\mathrm{RBG\,subset}}(p)$ 为 RBG 子集 p 包含的 VRB 数，计算公式如下：

$$N_{\mathrm{RB}}^{\mathrm{RBG\ subset}}(p)=\begin{cases}\left\lfloor\dfrac{N_{\mathrm{RB}}^{\mathrm{DL}}-1}{P^2}\right\rfloor\cdot P+P & ,p<\left\lfloor\dfrac{N_{\mathrm{RB}}^{\mathrm{DL}}-1}{P}\right\rfloor \bmod P\\ \left\lfloor\dfrac{N_{\mathrm{RB}}^{\mathrm{DL}}-1}{P^2}\right\rfloor\cdot P+(N_{\mathrm{RB}}^{\mathrm{DL}}-1)\bmod P+1 & ,p=\left\lfloor\dfrac{N_{\mathrm{RB}}^{\mathrm{DL}}-1}{P}\right\rfloor \bmod P\\ \left\lfloor\dfrac{N_{\mathrm{RB}}^{\mathrm{DL}}-1}{P^2}\right\rfloor\cdot P & ,p>\left\lfloor\dfrac{N_{\mathrm{RB}}^{\mathrm{DL}}-1}{P}\right\rfloor \bmod P\end{cases}$$

对于 RBG 子集 p 而言，其 bitmap 中的每一比特 i（$i=0,1,\cdots,N_{\mathrm{RB}}^{\mathrm{TYPE1}}-1$）对应的 VRB 可通过如下公式计算：

$$n_{\mathrm{VRB}}^{\mathrm{RBG\ subset}}(p)=\left\lfloor\frac{i+\Delta_{\mathrm{shift}}(p)}{P}\right\rfloor P^2+p\cdot P+\left(i+\Delta_{\mathrm{shift}}(p)\right)\bmod P$$

关于子集和偏移量的概念，可能较难理解。这里举了个例子，以加深大家对资源分配类型 1 的理解。

以小区下行系统带宽 25 RB 为例。

（1）通过查 36.213 的 Table 7.1.6.1-1 可以知道，P = 2，即有 2 个子集：子集 0（从 RBG0 开始）和子集 1（从 RBG1 开始），每个子集中的 RBG 间隔为 P = 2，此时子集 0 和子集 1 包含的 RBG 集合如表 5-3 所示；

表 5-3：资源分配类型 1 中的子集（25 RB）

子集 0（$p = 0$）	RBG0	RBG2	RBG4	RBG6	RBG8	RBG10	RBG12
子集 1（$p = 1$）	RBG1	RBG3	RBG5	RBG7	RBG9	RBG11	

（2）$\lceil \log_2(P) \rceil = 1$，即第一个域使用 1 比特指定所选的 RBG 子集；

（3）第二个域使用 1 比特指定 RBG 子集中的资源是否偏移；

（4）bitmap 包含的比特数 $N_{\mathrm{RB}}^{\mathrm{TYPE1}} = \lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \rceil - \lceil \log_2(P) \rceil - 1$ = 13 -1 -1 = 11；即 bitmap 只能对应 11 个 VRB。

（5）每个 RBG 子集 p 包含的 VRB 数为

$N_{\mathrm{RB}}^{\mathrm{RBG\,subset}}(0)$	13
$N_{\mathrm{RB}}^{\mathrm{RBG\,subset}}(1)$	12

可以看出，bitmap 不足以表示每个子集中包含的所有 VRB。

（6）接下来，我们详细介绍第二个域，即 shift bit 对 bitmap 所表示的 VRB 的影响。

如果 shift bit 为 0，RBG 子集 p 的偏移 $\Delta_{\mathrm{shift}}(p) = 0$；

如果 shift bit 为 1，RBG 子集 p 的偏移为

$\Delta_{\mathrm{shift}}(0)$	2 （13 - 11）
$\Delta_{\mathrm{shift}}(1)$	1 （12 - 11）

从之前的分析可以看出，每个子集包含哪些 RBG 是确定的，也就是说，包含哪些 VRB 也是确定的。对应表 5-3，每个子集可用的 VRB 集合如图 5-4 所示：

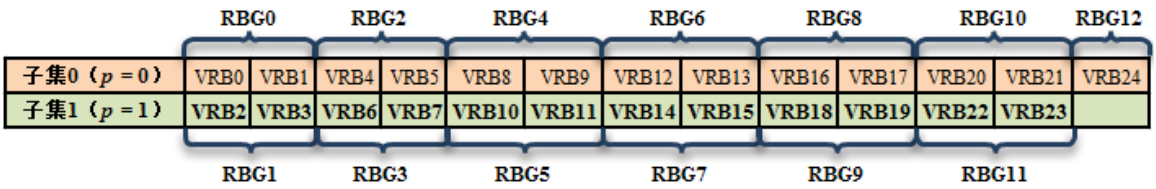


图 5-4：资源分配类型 1 中每个子集可用的 VRB 集合（25 RB）

当 shift bit = 0 时，根据下面的公式，可知道 bitmap（对于 25RB 带宽，共 11 比特）的每一个比特对应哪个 VRB。

$$n_{\mathrm{VRB}}^{\mathrm{RBG\ subset}}(p)=\left\lfloor \frac{i+\Delta_{\mathrm{shift}}(p)}{P} \right\rfloor P^2 + p\cdot P + \left(i+\Delta_{\mathrm{shift}}(p)\right) \bmod P$$

结果如下：

表 5-4：每个子集的 bitmap 中的每个比特对应的 VRB（25 RB, shift bit = 0）

i	VRB（子集 0）	VRB（子集 1）
0	0	2
1	1	3
2	4	6
3	5	7
4	8	10
5	9	11
6	12	14
7	13	15
8	16	18
9	17	19
10	20	22

从表 5-4 可以看出，如果 shift bit = 0（不发生偏移），每个子集的 bitmap 对应的 VRB，是从图 5-4 给定的 VRB 集合中的第一个 VRB 开始（对应子集 0，起始 VRB 为 VRB0；对应子集 1，起始 VRB 为 VRB2），顺序选取 11 个 VRB。

当 shift bit = 1 时，根据下面的公式，可知道 bitmap（对于 25RB 带宽，共 11 比特）的每一个比特对应哪个 VRB。

$$n_{\mathrm{VRB}}^{\mathrm{RBG\ subset}}(p)=\left\lfloor \frac{i+\Delta_{\mathrm{shift}}(p)}{P} \right\rfloor P^2 + p\cdot P + \left(i+\Delta_{\mathrm{shift}}(p)\right) \bmod P$$

结果如下：

表 5-5：每个子集的 bitmap 的每个比特表示的 VRB（25 RB, shift bit = 1）

i	VRB（子集 0）	VRB（子集 1）
0	4	3
1	5	6
2	8	7
3	9	10
4	12	11
5	13	14
6	16	15
7	17	18
8	20	19
9	21	22
10	24	23

从表 5-5 可以看出，如果 shift bit = 1（发生偏移），每个子集的 bitmap 对应的 VRB，是从图 5-4 给定的 VRB 集合中的第一个 VRB，加上偏移量开始（对应子集 0，偏移量 $\Delta_{\mathrm{shift}}(0)$ = 2，即在图 5-4 给定的 p = 0 的 VRB 集合中，往前移 2 个，得到起始 VRB 为 VRB4；对应子集 1，偏移量 $\Delta_{\mathrm{shift}}(1)$ = 1，即在图 5-4 给定的 p = 1 的 VRB 集合中，往前移 1 个，得到起始 VRB 为 VRB3），顺序选取 11 个 VRB。

图 5-5 介绍了使用资源分配类型 1 的例子（25 RB）：

上半部分对应：资源分配类型 1；子集 0；shift bit 为 0；bitmap 为 10011101000。即分配该 UE 的资源为：VRB0、VRB5、VRB8、VRB9、VRB13。

下半部分对应：资源分配类型 1；子集 0；shift bit 为 1；bitmap 为 10011101000。即分配该 UE 的资源为：VRB4、VRB9、VRB12、VRB13、VRB17。

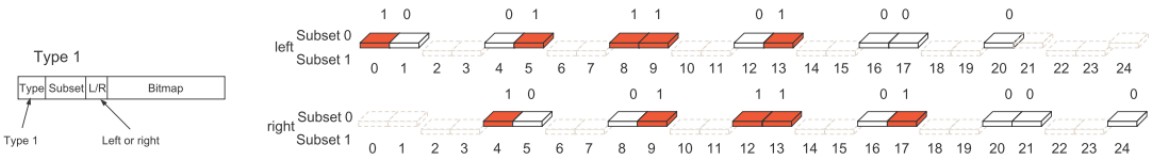


图 5-5：资源分配类型 1 的例子（25 RB）

关于资源分配类型 1 的更多例子，还可以参见参考资料[6]中的介绍。

从上面的例子可以看出：（1）资源分配类型 1 支持频域上的非连续 RB 分配；（2）与资源分配类型 0 相比，资源分配类型 1 支持粒度为 1 RB 的分配；（3）资源分配类型 0 和资源分配类型 1 使用相同的比特数来表示资源的分配；（4）bitmap 的比特数实际上比 RBG 子集中的 VRB 数要少，通过 shift bit，bitmap 才能覆盖所有的 VRB。

5.1.4 资源分配类型 2（Resource allocation type 2）

在资源分配类型 2 中，分配给 UE 的资源为一段连续的 VRB，其 VRB 可以是集中式（localized），也可以是分布式的（distributed）。

对于 **DCI format 1A/1B/1D** 而言，有一个比特（对应 Localized/Distributed VRB assignment flag 字段）用于指示是使用集中式 VRB（该比特为 0）还是使用分布式 VRB（该比特为 1）。

- Localized/Distributed VRB assignment flag – 1 bit as defined in 7.1.6.3 of [3]
- Resource block assignment – $\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil$ bits as defined in section 7.1.6.3 of [3]:
  - For localized VRB:
    $\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil$ bits provide the resource allocation
  - For distributed VRB:
    - If $N_{RB}^{DL} < 50$ or if the format 1A CRC is scrambled by RA-RNTI, P-RNTI, or SI-RNTI
      - $\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil$ bits provide the resource allocation
    - Else
      - 1 bit, the MSB indicates the gap value, where value 0 indicates $N_{gap} = N_{gap,1}$ and value 1 indicates $N_{gap} = N_{gap,2}$
      - $(\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil - 1)$ bits provide the resource allocation,

where N_{gap} is defined in [2].

图 5-6：DCI format 1A 中与 Type 2 相关的字段

对于集中式 VRB 分配而言，分配给一个 UE 的资源可以从 1 个 VRB 到整个下行系统带宽的所有 VRB。

如果 DCI format 1A 使用分布式 VRB 分配方式，且其 DCI 的 CRC 由 P-RNTI、RA-RNTI 或 SI-RNTI 加扰，则分配给对应 UE 的 VRB 数可以从 1 个到 $N_{\mathrm{VRB}}^{\mathrm{DL}}$ 个。（ $N_{\mathrm{VRB}}^{\mathrm{DL}}$ 的计算可见 36.211 的 6.2.3.2 节，或见 5.2 节介绍）

如果 DCI format 1B/1D 的 CRC 使用 C-RNTI 加扰，或 DCI format 1A 的 CRC 使用 C-RNTI、SPS C-RNTI 或 TC-RNTI 加扰，并且这些 DCI 使用分布式 VRB 分配方式，则当下行带宽为 6~49 RB 时，分配给对应 UE 的 VRB 数可以从 1 个到最多 $N_{\mathrm{VRB}}^{\mathrm{DL}}$ 个；则当下行带宽为 50~110 RB 时，分配给对应 UE 的 VRB 数可以从 1 个到最多 16 个。

对于 DCI format 1A/1B/1D 而言，资源分配由一个资源指示值 RIV 来表示。通过这个值，可以推导出分配给 UE 的起始 RB（ RB_{start} ）以及连续分配的 RB 的长度（ L_{CRBs} ）。计算公式如下：

如果 $(L_{CRBs}-1)\le\lfloor N_{RB}^{DL}/2\rfloor$ ，则 $RIV=N_{RB}^{DL}(L_{CRBs}-1)+RB_{start}$ ；否则 $RIV=N_{RB}^{DL}(N_{RB}^{DL}-L_{CRBs}+1)+(N_{RB}^{DL}-1-RB_{start})$ 。其中 $L_{CRBs}\ge 1$ 且不超过 $N_{VRB}^{DL}-RB_{start}$ 。

当 UE 收到一个 RIV 后，如何计算 RB_{start} 和 L_{CRBs} ？

通过 $\lfloor RVI/N_{RB}^{DL}\rfloor+RVI\%N_{RB}^{DL}$ 可以知道是 $(L_{CRBs}-1)\le\lfloor N_{RB}^{DL}/2\rfloor$ 还是 $(L_{CRBs}-1)>\lfloor N_{RB}^{DL}/2\rfloor$ ，并最终计算出 RB_{start} 和 L_{CRBs}。

由于 $L_{CRBs}\ge 1$ 且不超过 $N_{VRB}^{DL}-RB_{start}$ ，且必定有 $N_{VRB}^{DL}\le N_{RB}^{DL}$ ，故 $RB_{start}+L_{CRBs}\le N_{RB}^{DL}$ ，也就有

（1）当 $(L_{CRBs}-1)\le\lfloor N_{RB}^{DL}/2\rfloor$ 时，
$\lfloor RVI\ /\ N_{RB}^{DL}\rfloor+RVI\ \%\ N_{RB}^{DL}=RB_{start}+L_{CRBs}-1<N_{RB}^{DL}$ ；

（2）当 $(L_{CRBs}-1)>\lfloor N_{RB}^{DL}/2\rfloor$ 时，
$\lfloor RVI\ /\ N_{RB}^{DL}\rfloor+RVI\ \%\ N_{RB}^{DL}=2N_{RB}^{DL}-(RB_{start}+L_{CRBs})\ge N_{RB}^{DL}$ 。

UE 收到 RIV 后，计算 $\lfloor RVI\ /\ N_{RB}^{DL}\rfloor+RVI\ \%\ N_{RB}^{DL}$ 的值 x，

（1）如果 $x<N_{RB}^{DL}$ ，则得知 $(L_{CRBs}-1)\le\lfloor N_{RB}^{DL}/2\rfloor$ ，也就得到了最终结果
$RB_{start}=RIV\ \%\ N_{RB}^{DL}$ ， $L_{CRBs}=\lfloor RVI\ /\ N_{RB}^{DL}\rfloor+1$ ；

（2）如果 $x\ge N_{RB}^{DL}$ ，则得知 $(L_{CRBs}-1)>\lfloor N_{RB}^{DL}/2\rfloor$ ，也就得到了最终结果

$RB_{start} = N_{RB}^{DL} - RIV \ \% \ N_{RB}^{DL} - 1$，$L_{CRBs} = N_{RB}^{DL} - \lfloor RVI \ / \ N_{RB}^{DL} \rfloor + 1$；

图 5-7 介绍了 DCI format 1A/1B/1D 使用资源分配类型 2 的例子（25 RB）：起始 RB（RB_{start}）为 3，连续分配的 VRB 数（L_{CRBs}）为 8，$L_{CRBs} - 1 = 7 < \lfloor N_{RB}^{DL} / 2 \rfloor = \lfloor 25/2 \rfloor = 12$，所以 $RIV = N_{RB}^{DL}(L_{CRBs} - 1) + RB_{start} = 25*(8-1)+3 = 178$。

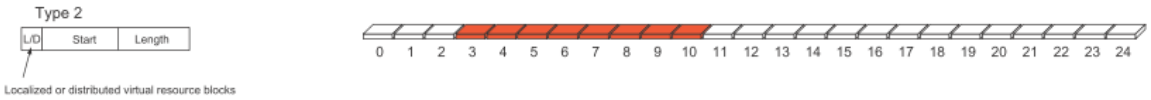


图 5-7：资源分配类型 2 的例子（25 RB）

DCI format 1C 只支持分布式 VRB 分配方式。对于 DCI format 1C 而言，分配给某个 UE 的资源可以从 $N_{\mathrm{RB}}^{\mathrm{step}}$ 个到最多 $\lfloor N_{\mathrm{VRB}}^{\mathrm{DL}} / N_{\mathrm{RB}}^{\mathrm{step}} \rfloor \cdot N_{\mathrm{RB}}^{\mathrm{step}}$ 个 VRB。其中 $N_{\mathrm{RB}}^{\mathrm{step}}$ 为增长的步进值，并与下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}}$ 相关（见 36.213 的 Table 7.1.6.3-1）。

Table 7.1.6.3-1: $N_{\mathrm{RB}}^{\mathrm{step}}$ values vs. Downlink System Bandwidth

System BW ($N_{\mathrm{RB}}^{\mathrm{DL}}$)	$N_{\mathrm{RB}}^{\mathrm{step}}$
	DCI format 1C
6-49	2
50-110	4

对于 DCI format 1C 而言，资源分配也是通过一个资源指示值 RIV 来表示。通过这个值，可以推导出分配给 UE 的起始 RB（$RB_{start} = 0, N_{\mathrm{RB}}^{\mathrm{step}}, 2N_{\mathrm{RB}}^{\mathrm{step}}, \ldots, (\lfloor N_{\mathrm{VRB}}^{\mathrm{DL}} / N_{\mathrm{RB}}^{\mathrm{step}} \rfloor - 1)N_{\mathrm{RB}}^{\mathrm{step}}$）以及连续分配的 RB 的长度（$L_{CRBs} = N_{\mathrm{RB}}^{\mathrm{step}}, 2N_{\mathrm{RB}}^{\mathrm{step}}, \ldots, \lfloor N_{\mathrm{VRB}}^{\mathrm{DL}} / N_{\mathrm{RB}}^{\mathrm{step}} \rfloor \cdot N_{\mathrm{RB}}^{\mathrm{step}}$）。计算公式如下：

如果 $(L'_{CRBs} - 1) \le \lfloor N'^{DL}_{VRB} / 2 \rfloor$，则 $RIV = N'^{DL}_{VRB}(L'_{CRBs} - 1) + RB'_{start}$；否则 $RIV = N'^{DL}_{VRB}(N'^{DL}_{VRB} - L'_{CRBs} + 1) + (N'^{DL}_{VRB} - 1 - RB'_{start})$。其中 $L'_{CRBs} = L_{CRBs} / N_{RB}^{step}$，$RB'_{start} = RB_{start} / N_{RB}^{step}$ 并且 $N'^{DL}_{VRB} = \lfloor N_{VRB}^{DL} / N_{RB}^{step} \rfloor$。而 $L'_{CRBs} \ge 1$ 且不超过 $N'^{DL}_{VRB} - RB'_{start}$。

对于 DCI format 1C 而言，UE 收到一个 RIV 后计算 RB_{start} 和 L_{CRBs} 的方式与 DCI format 1A/1B/1D 类似，这里就不做介绍了。

假设是在 DCI format 1C 中的资源分配且下行系统带宽为 25 RB，$RB_{start}=2$，$L_{CRBs}=8$，则有

$N_{\mathrm{RB}}^{\mathrm{step}}$	2
$N_{\mathrm{VRB}}^{\mathrm{DL}}=N_{\mathrm{VRB,\,gap1}}^{\mathrm{DL}}=2\cdot\min(N_{\mathrm{gap}},N_{\mathrm{RB}}^{\mathrm{DL}}-N_{\mathrm{gap}})$	2 * min(12, 25 - 12) = 24（见 36.211 的 6.2.3.2 节）
$L'_{CRBs}=L_{CRBs}/N_{RB}^{step}$	8 / 2 = 4
$RB'_{start}=RB_{start}/N_{RB}^{step}$	2 / 2 = 1
$N'^{DL}_{VRB}=\lfloor N^{DL}_{VRB}/N^{step}_{RB}\rfloor$	$\lfloor 24/2\rfloor=12$

因为 $(L'_{CRBs}-1)$ = 3 < $\lfloor N'^{DL}_{VRB}/2\rfloor$ = 6，所以 $RIV=N'^{DL}_{VRB}(L'_{CRBs}-1)+RB'_{start}$ = 12 * (4 - 1) + 1 = 37。

从上面的例子可以看出：（1）资源分配类型 2 只支持连续 VRB 的分配；（2）对于资源分配类型 2，DCI format 1A/1B/1D 与 DCI format 1C 的格式是不同的，DCI format 1C 多了步进的概念；（3）与资源分配类型 0/1 只支持集中式 VRB 分配不同，资源分配类型 2 既支持集中式 VRB 也支持分布式 VRB。

通过前面的介绍可以知道，UE 能够通过对应的 DCI 确定当前 PDSCH 传输使用的资源分配类型以及占用了哪些 VRB，因此也确定了分配给当前 PDSCH 传输的 VRB 数。又因为 VRB 数与 PRB 数相等，因此也就确定了当前传输分配的 PRB 数 N_{PRB}。N_{PRB} 会在计算 TB Size 时被用到（见 36.213 的 7.1.7 节）。

VRB 是逻辑上的虚拟资源块概念，还需要将其映射到实际的物理资源块 PRB 上，详细内容见 5.2 节介绍。

【参考资料】

[1] TS 36.211 的 6.2.3 节 Resource blocks

[2] TS 36.212 的 5.3.3 节 Downlink control information

[3] TS 36.213 的 7.1.6 节 Resource allocation

[4] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.4.4 节

[5] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 9.3.5.4 节

[6] 《Type 1 Resource Allocation in LTE》 by Prakash

[7] 《Resource Allocation Type》

5.2 VRB 到 PRB 的映射

在第 2 章中我们介绍了物理资源块，即 PRB（Physical Resource Block）。LTE 中还定义了虚拟资源块，即 VRB（Virtual Resource Block）。PDSCH 和 PUSCH 的资源都是以 VRB 对（VRB pair）为单位进行分配，然后再将其映射到 PRB 上的。

eNodeB 在决定将哪些 RB 分配给某个特定 UE 时，可能会将时域和频域相关的下行信道质量考虑在内。即信道相关的频选调度会将信道的变化，如由频率选择性衰落引起的信道变化等，考虑在内，然后将那些信道质量好的 RB（不一定连续）分配给该 UE，这样可能会显著地提升 UE 的速率以及整个小区的吞吐量。

频选调度要求 UE 上报下行信道质量给 eNodeB，这会带来较大的信令开销，同时需要保证 eNodeB 能够成功并及时地接收到下行信道质量信息，以避免收不到或收到过时的信息。在某些场景下，频选调度是不适用的：

- 对于低速业务，如语音业务，与频选相关的反馈信令会带来相对较大的开销，得不偿失；
- 在 UE 高速移动的场景下，如在高速运行的高铁上，很难或根本不可能跟踪实时信道质量，因而无法提供频选调度所需的信道质量精确度。

在这种情况下，一种可选的方案是将下行传输分布到频域内非连续的资源块上以获得频率分集增益，从而提高传输的可靠性。

下行资源分配类型 0 和 1 能够提供分布式资源分配，并且在很多情况下也能满足需求，但还是有一些缺陷：

- 无法满足单个 RB pair 的分布式分配需求，即其最小分配单元为 1 个 RB pair 的情况；
- 相对较大的 PDCCH 开销。例如：带宽为 100 RB 的情况下，资源分配类型 0 和 1 需要 $\lceil N_{\mathrm{RB}}^{\mathrm{DL}} / P \rceil = \lceil 100/4 \rceil = 25$ 比特用于指示下行资源，而资源分配类型 2 只需要

$\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil = \lceil \log_2(100(100+1)/2) \rceil = 13$ 比特。可以看出，资源分配类型 2 中用于指示下行资源的比特开销较小。（见 36.212 的 5.3.3.1 节）

资源分配类型 2 既能提供频域上连续的 RB 资源分配，也能提供频域上非连续的 RB 资源分配。资源分配类型 2 有相对较小的 PDCCH 开销，并在使用分布式 VRB 的情况下，允许单个 RB pair 实现频域上的分布式传输。

使用分布式 VRB 的主要目的是在子带的信道状况不可知或过时的情况下，用于最大化频域分集增益。

LTE 定义了 2 种类型的 VRB：集中式 VRB（localized VRB）和分布式 VRB（distributed VRB）。

VRB 与 PRB 大小相同。无论对于哪一种 VRB，同一子帧内的 2 个 slot 的一对 VRB（VRB pair）被看成一个整体，并被分配相同的 VRB number：n_{VRB}。

5.2.1 集中式 VRB

集中式 VRB 直接一一映射到 PRB，即有 $n_{PRB} = n_{VRB}$，并且不支持同一 RB pair 在 slot 间跳频，如图 5-8 所示。

集中式 VRB 支持频选相关的动态调度的大多数场景。

资源分配类型 0 和 1 只支持集中式 VRB。

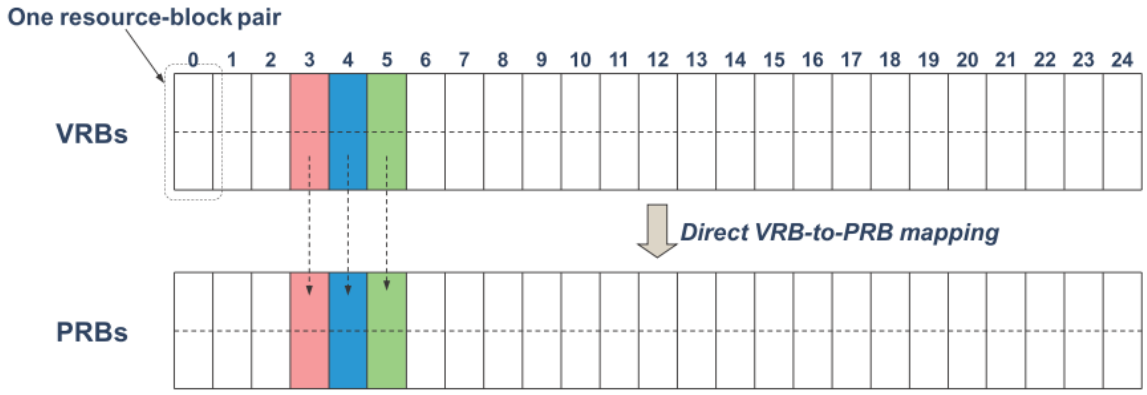


图 5-8：下行系统带宽 25 RB，集中式 VRB 下，VRB 到 PRB 的映射过程举例

5.2.2 分布式 VRB

下行资源分配类型 2 既支持集中式 VRB，也支持分布式 VRB。使用哪种类型是通过 DCI format 1A/1B/1D 的 Localized/Distributed VRB assignment flag 字段来配置的。（见 36.212 的 5.3.3.1 节）

DCI format 1C 只支持分布式 VRB。

接下来，我们会介绍分布式 VRB 映射到 PRB 的过程（以 $N_{\mathrm{RB}}^{\mathrm{DL}}=50$，$P=3$，$N_{\mathrm{gap}}=N_{\mathrm{gap},2}=9$ 为例）。

分布式 VRB 到 PRB 的映射过程主要分为 2 步：步骤一：交织（interleaving）：将连续的 VRB pair 映射到非连续的 PRB pair 上；步骤二：同一 VRB number 在 slot 间的跳频。

步骤一：交织（interleaving）

我们将交织过程细化成几个步骤：

（1）确定 N_{gap} 、P 值以及 VRB 个数。

Table 6.2.3.2-1: RB gap values.

System BW ($N_{\mathrm{RB}}^{\mathrm{DL}}$)	Gap (N_{gap})	
	1st Gap ($N_{\mathrm{gap},1}$)	2nd Gap ($N_{\mathrm{gap},2}$)
6-10	$\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/2 \rceil$	N/A
11	4	N/A
12-19	8	N/A
20-26	12	N/A
27-44	18	N/A
45-49	27	N/A
50-63	27	9
64-79	32	16
80-110	48	16

首先介绍一下 N_{gap}。N_{gap} 的取值与下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}}$ 相关，并在 36.211 的 Table 6.2.3.2-1 中定义。当 $6 \le N_{\mathrm{RB}}^{\mathrm{DL}} \le 49$ 时，只定义了一个 gap 值 $N_{\mathrm{gap},1}$，并且 $N_{\mathrm{gap}} = N_{\mathrm{gap},1}$。当 $50 \le N_{\mathrm{RB}}^{\mathrm{DL}} \le 110$ 时，定义了两个 gap 值 $N_{\mathrm{gap},1}$ 和 $N_{\mathrm{gap},2}$，并且 $N_{\mathrm{gap}} = N_{\mathrm{gap},1}$ 还是 $N_{\mathrm{gap}} = N_{\mathrm{gap},2}$ 取决于 DCI format 1A/1B/1D 的 Resource block assignment 字段的配置。如图 5-9 所示。

该bit决定是集中式VRB还是分布式VRB

- Localized/Distributed VRB assignment flag – 1 bit as defined in 7.1.6.3 of [3]

- Resource block assignment – $\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil$ bits as defined in section 7.1.6.3 of [3]:

  - For localized VRB:

    $\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil$ bits provide the resource allocation

  - For distributed VRB:

    - If $N_{RB}^{DL} < 50$ or if the format 1A CRC is scrambled by RA-RNTI, P-RNTI, or SI-RNTI

      - $\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil$ bits provide the resource allocation

    - Else 该bit决定是使用gap 1还是gap 2

      - 1 bit, the MSB indicates the gap value, where value 0 indicates $N_{gap} = N_{gap,1}$ and value 1 indicates $N_{gap} = N_{gap,2}$

      - $(\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil - 1)$ bits provide the resource allocation,

    where N_{gap} is defined in [2].

图 5-9：DCI format 1A/1B/1D 中与 VRB 分配相关的字段

VRB 的编号（VRB number）从 0 到 $N_{VRB}^{DL}-1$。VRB 个数 N_{VRB}^{DL} 的计算公式如图 5-10 所示：

$$N_{VRB}^{DL} = \begin{cases} N_{VRB,gap1}^{DL} = 2 \cdot \min(N_{gap}, N_{RB}^{DL} - N_{gap}); & \text{对应} N_{gap} = N_{gap,1} \\ N_{VRB,gap2}^{DL} = \lfloor N_{RB}^{DL} / 2N_{gap} \rfloor \cdot 2N_{gap}; & \text{对应} N_{gap} = N_{gap,2} \end{cases}$$

图 5-10：VRB 个数的计算公式

VRB 个数可能小于下行系统带宽中包含的 PRB 个数，多出来的 RB pair 是不能进行分布式传输的。例如：下行系统带宽为 25 RB 的情况下，VRB 的个数仅为 24，即第 25 个 RB pair 不能用于分布式传输。

在 5.1.1 节中，我们已经介绍了 RBG 的概念。这里我们依然会用到 RBG 中的 P 值概念（即 RBG 的大小），见 36.213 的 Table 7.1.6.1-1。

Table 7.1.6.1-1: Type 0 Resource Allocation RBG Size vs. Downlink System Bandwidth

System Bandwidth N_{RB}^{DL}	RBG Size (*P*)
≤10	1
11 – 26	2
27 – 63	3
64 – 110	4

从 36.211 的 Table 6.2.3.2-1 和 36.213 的 Table 7.1.6.1-1 可以看出：gap 的大小是 P^2 的整数倍，这样做的目的是为了使在同一个子帧内，分布式 VRB 资源分配与资源分配类型 0/1 能够平滑共存。

（2）确定 VRB 交织矩阵：“横放”

确定 N_{gap} 、P 值以及 VRB 个数 的目的是为了计算 VRB 交织矩阵。既然是矩阵，就需要明确该矩阵有几行几列，包含哪些信息。

一个 VRB 交织矩阵对应一个 VRB 交织单元。$\tilde{N}_{\text{VRB}}^{\text{DL}}$ 个连续的 VRB number 组成一个 VRB 交织单元，其中

$$\tilde{N}_{\text{VRB}}^{\text{DL}} = \begin{cases} N_{\text{VRB}}^{\text{DL}}; & \text{对应} N_{\text{gap}} = N_{\text{gap},1} \\ 2N_{\text{gap}}; & \text{对应} N_{\text{gap}} = N_{\text{gap},2} \end{cases}$$

图 5-11：VRB 交织单元中包含的 VRB 数

结合图 5-10、图 5-11 可以看出：一共有 $N_{\text{VRB}}^{\text{DL}}$ 个 VRB 需要交织，而每个交织单元只包含 $\tilde{N}_{\text{VRB}}^{\text{DL}}$ 个 VRB，所以交织单元的个数为 $N_{\text{VRB}}^{\text{DL}} / \tilde{N}_{\text{VRB}}^{\text{DL}}$。更进一步，对应 $N_{\text{gap}} = N_{\text{gap},1}$，只有 1 个交织单元；对应 $N_{\text{gap}} = N_{\text{gap},2}$，有 $\lfloor N_{\text{RB}}^{\text{DL}} / 2N_{\text{gap}} \rfloor$ 个交织单元。

交织矩阵共有 $N_{\text{row}} = \left\lceil \tilde{N}_{\text{VRB}}^{\text{DL}} / (4P) \right\rceil \cdot P$ 行，4 列。VRB number 会逐行写入矩阵，并逐列读出（“横放列取”或“横写列读”，“列取”会在后面介绍）。矩阵包含的元素个数 $4N_{\text{row}}$ 可能大于 $\tilde{N}_{\text{VRB}}^{\text{DL}}$，此时多

出来的元素会填充为 null，且填充为 null 的元素位于最后 $N_{\text{null}}/2$ 行的第 2 列和第 4 列。填充为 null 的元素个数为 $N_{\text{null}} = 4N_{\text{row}} - \tilde{N}_{\text{VRB}}^{\text{DL}}$ 。

我们以 $N_{\text{RB}}^{\text{DL}} = 50$ ，使用分布式 VRB 且 $N_{\text{gap}} = N_{\text{gap},2}$ 为例来介绍交织矩阵的生成。我们先计算出相关的值，见表 5-6。

表 5-6： $N_{\text{RB}}^{\text{DL}} = 50$ ， $N_{\text{gap}} = N_{\text{gap},2}$ 的例子

Field	Value
$N_{\text{RB}}^{\text{DL}}$	$= 50$
N_{gap}	$= N_{\text{gap},2} = 9$
P	$= 3$
$N_{\text{VRB}}^{\text{DL}}$	$= N_{\text{VRB, gap2}}^{\text{DL}} = \lfloor N_{\text{RB}}^{\text{DL}} / 2N_{\text{gap}} \rfloor \cdot 2N_{\text{gap}} = \lfloor 50/(2*9) \rfloor * 2 * 9 = 36$
$\tilde{N}_{\text{VRB}}^{\text{DL}}$	$= 2N_{\text{gap}} = 18$
N_{row}	$= \lceil \tilde{N}_{\text{VRB}}^{\text{DL}} / (4P) \rceil \cdot P = \lceil 18/(4*3) \rceil \cdot 3 = 6$
N_{null}	$= 4N_{\text{row}} - \tilde{N}_{\text{VRB}}^{\text{DL}} = 4*6 - 18 = 6$

基于表 5-6，生成的交织矩阵为（这是“横放”的过程，*表示 null。其实共有 2 个交织矩阵，第二个矩阵的处理是类似的，这里就不介绍了）：

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & * & 13 & * \\ 14 & * & 15 & * \\ 16 & * & 17 & * \end{bmatrix}$$

图 5-12：生成的交织矩阵

从前面的介绍可以看出，“横放”的过程实际是按照 VRB number 从小到大的顺序，逐行写入交织矩阵，并避开 null 元素的过程。

（3）确定 VRB 对应的 PRB：“列取”

“列取”是将 VRB number 是映射到 PRB number 的过程。在“列取”过程中，null 元素会被忽略。

我们先使用图 5-12 的例子来说明“列取”的过程中，VRB number 是怎么映射到 PRB number 的，然后再介绍该过程如何对应到公式中。

以图 5-12 为例，假设需要确定 VRB 12（$n_{\mathrm{VRB}}=12$）对应的 PRB，我们逐列读取该矩阵，遇*则跳过，直到读到的元素值为 12，即读取的顺序为（0, 4, 8, 12），到第 4 个元素得到值 12 的元素，而计数是从 0 开始的，所以 VRB 12 对应 PRB 3（$n_{\mathrm{PRB}}=3$）。

假设需要确定 VRB 15（$n_{\mathrm{VRB}}=15$）对应的 PRB，我们逐列读取该矩阵，遇*则跳过，直到读到的元素值为 15，即读取的顺序为（0, 4, 8, 12, 14, 16, 1, 5, 9, 2, 6, 10, 13, 15），到第 14 个元素得到值 15 的元素，而计数是从 0 开始的，所以 VRB 15 对应 PRB 13（$n_{\mathrm{PRB}}=13$）。

图 5-12 的矩阵“列取”后的 VRB number 到 PRB number 的映射关系如图 5-13 所示。

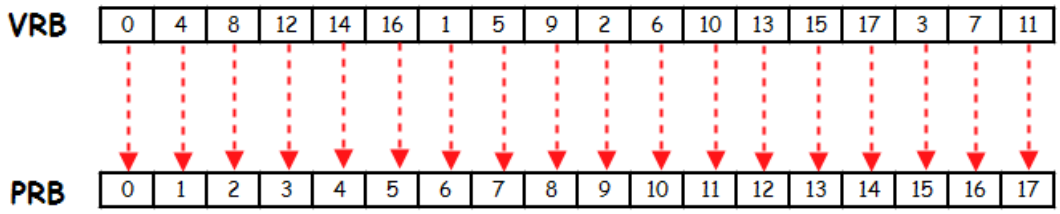


图 5-13：“横放列取”后的映射关系

上面过程看上去比较简单，但通过公式中反映出来就略显复杂了。在图 5-14 中，我对协议中公式做了一些标注，以便于大家理解。其实理解公式的一种好方式是自己把一些数据代入公式，然后算一遍，这样能够加深理解。

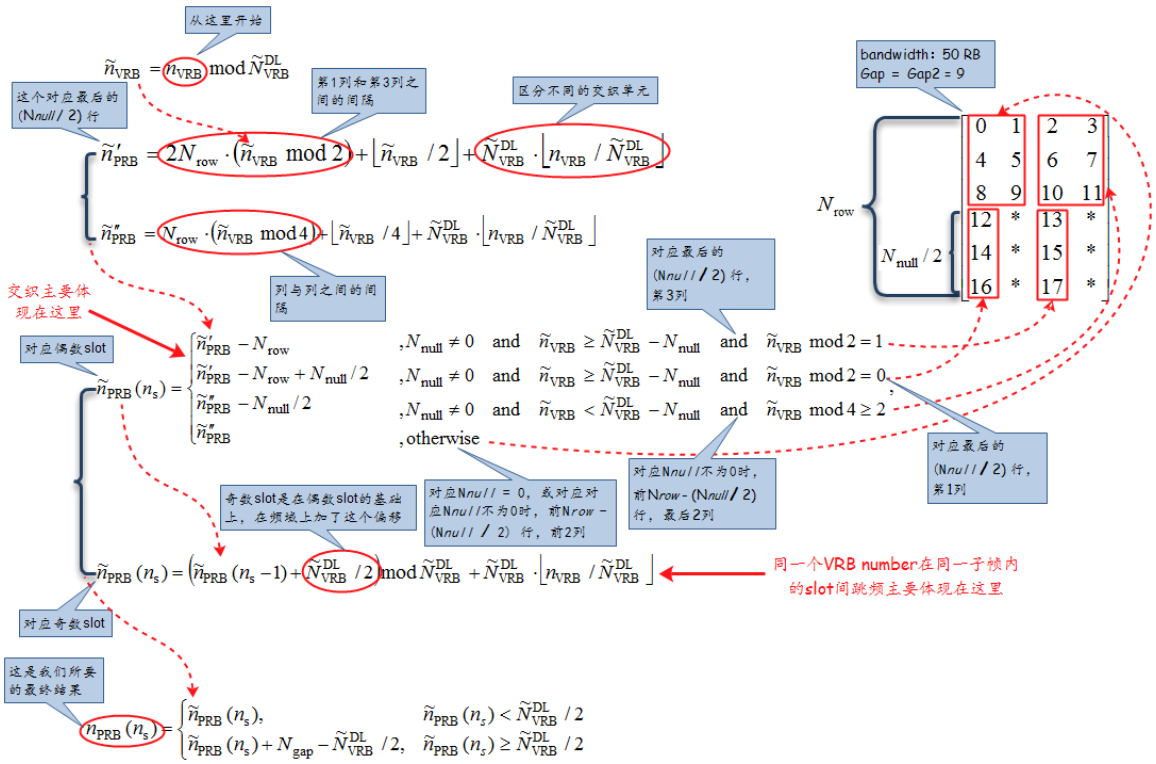


图 5-14：“横放列取”对应的公式（见 36.211 的 6.2.3.2 节）

步骤二：slot 间的跳频

在步骤一中，我们已经介绍了 VRB number 如何映射到 PRB number。但对应同一 VRB number，奇数 slot 上的 RB 会在偶数 slot 的基础上，在 VRB 交织单元内偏移 $\tilde{N}_{\mathrm{VRB}}^{\mathrm{DL}}/2$ 。（对应的公式见图 5-14）

slot 间的跳频使得 UE 在只分配了单个 RB pair 的情况下，也可以进行分布式传输，从而提高频率分集增益。

同样以 $N_{\mathrm{RB}}^{\mathrm{DL}}=50$ ，使用分布式 VRB 且 $N_{\mathrm{gap}}=N_{\mathrm{gap},2}$ 为例，VRB 12 在偶数 slot 上对应 PRB 3（ $n_{\mathrm{PRB}}=3$ ），但在奇数 slot 上对应 PRB 12（ $n_{\mathrm{PRB}}=12$ ）。

如果 UE 1 被分配了 VRB 3~5，UE 2 被分配了 VRB 11~13，则最终的映射结果如图 5-15 所示：

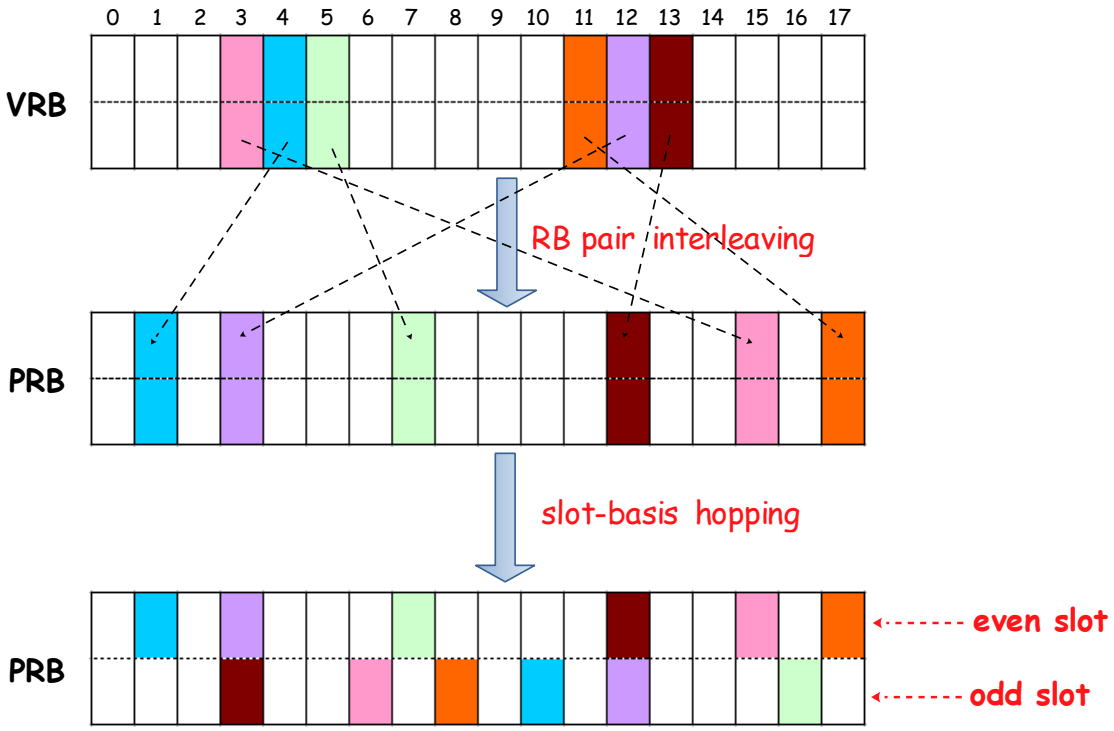


图 5-15： $N_{\mathrm{RB}}^{\mathrm{DL}}=50$ $N_{\mathrm{gap}}=N_{\mathrm{gap},2}$ 的例子

可以看出：（1）对于分布式 VRB 而言，连续的 VRB 映射到的 PRB 并不连续，即使是单一的 VRB pair 在频域上也是分开的；（2）slot 间的跳频只在 VRB 交织单元内进行，不会跳出其对应的交织单元之外。

附：关于 VRB 到 PRB 的映射，有很多专利文档，大家可以 google 一下。

通过 5.1 节和本节的介绍，UE 就确定了当前 PDSCH 传输占用的 PRB 数 N_{PRB} 以及具体占用了哪些 PRB。

【参考资料】

[1] TS 36.211 的 6.2.3 节 Resource blocks

[2] TS 36.212 的 5.3.3 节 Downlink control information

[3] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.1.2 节

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 9.2.2.1 节和 9.3.5.4 节

[5] 《Resource Allocation in LTE》

[6] 《Resource allocation type 2》

5.3 如何确定调制阶数和 TB size

UE 如何根据接收到的 DCI format 1/1A/1B/1C/1D/2/2A/2B/2C 来确定对应的 PDSCH 传输使用调制阶数 Q_m（modulation order）以及 TB size 呢？

首先，UE 会读取以下字段：

- 读取对应 DCI 中 5 比特的“Modulation and coding scheme”字段，得到 I_{MCS}；
- 根据 5.1 节的介绍，确定当前 PDSCH 传输占用的 PRB 数 N_{PRB}。

5.3.1 确定调制阶数

UE 在确定当前的 PDSCH 传输使用的调制阶数 Q_m 时，需要分 2 种情况来处理：

情况 1：对于使用 P-RNTI（对应 Paging 消息）、RA-RNTI（对应 Msg 2）、SI-RNTI（对应 SIB1 和 SI 消息）加扰的 DCI，其对应的 PDSCH 传输的 Q_m 固定为 2；

情况 2：对于除情况 1 外的其它情况，如使用 C-RNTI、SPS C-RNTI、TC-RNTI 加扰的 DCI，其对应的 PDSCH 传输的 Q_m 是通过 I_{MCS} 查 36.213 的 Table 7.1.7.1-1 得到的。

Table 7.1.7.1-1: Modulation and TBS index table for PDSCH

MCS Index I_{MCS}	Modulation Order Q_m	TBS Index I_{TBS}
0	2	0
1	2	1
2	2	2
3	2	3
4	2	4
5	2	5
6	2	6
7	2	7
8	2	8
9	2	9
10	4	9
11	4	10
12	4	11
13	4	12
14	4	13
15	4	14
16	4	15
17	6	15
18	6	16
19	6	17
20	6	18
21	6	19
22	6	20
23	6	21
24	6	22
25	6	23
26	6	24
27	6	25
28	6	26
29	2	reserved
30	4	
31	6	

5.3.2 确定 TB size

从 MAC 层发往物理层的数据是以传输块（Transport Block，TB）的形式组织的。UE 需要确定 TB 中包含了多少数据，即确定 TB size。

为了确定某个 TB 对应的 TB size，UE 首先需要确定该 TB 占用的 PRB 数 N_{PRB} 和 TBS 索引 I_{TBS}。其中 N_{PRB} 是由对应 DCI 中的相应字段指定的（见 5.1 节中关于“下行资源分配类型”的介绍）。

需要注意的是，在空分复用下，一个 TTI 会至多发送 2 个 TB，每个 TB 的 TB size 是独立计算的。但这 2 个 TB 的 N_{PRB} 是相同的。

这里需要分为以下 3 种情况分别处理：

情况 1：对于使用 P-RNTI、RA-RNTI、SI-RNTI 加扰的 DCI：

- 如果是 DCI format 1A：
    - 则将 N_{PRB} 设置为 $N_{\text{PRB}}^{\text{1A}}$，将 I_{TBS} 设置为 I_{MCS}，然后查 36.213 的 Table 7.1.7.2.1-1 得到对应 PDSCH 传输的 TB size。关于 $N_{\text{PRB}}^{\text{1A}}$（取值范围：2 和 3）的计算，可参见 36.212 的 5.3.3.1.3 节中关于 DCI format 1A 的介绍，或见本书的 4.4.5.2.2 节。
- 如果是 DCI format 1C：
    - 将 I_{TBS} 设置为 I_{MCS}，然后查 36.213 的 Table 7.1.7.2.3-1 得到对应 PDSCH 传输的 TB size。

注意：针对这种情况，（1）I_{TBS} 并不是通过查 36.213 的 Table 7.1.7.1-1 得到的；（2）使用 DCI format 1C 时，无需计算 N_{PRB}；（3）在后续介绍 SIB 时，我们会提到，物理层限制了 1 个 SIB 的最大 size。如果使用 DCI format 1C，则最大 size 为 1736 比特（217 byte），这可以从 36.213 的 Table 7.1.7.2.3-1 看出。如果使用 DCI format 1A，则最大 size 为 2216 比特（277 byte），此时 N_{PRB} 的最大值为 3，I_{TBS} 的最大值为 26，查 36.213 的 Table 7.1.7.2.1-1，可以看出其最大 size 为 2216。

情况 2：对于非特殊子帧（包括 FDD 下的子帧和 TDD 下的正常子帧）

如果 UE 接收到 DCI format 2/2A/2B/2C，且某个 TB 对应的“Modulation and coding scheme”对应的 $I_{\text{MCS}} = 0$，且当前传输的冗余版本 $rv_{idx} = 1$，则该 TB 是去使能的；否则该 TB 是使能的。

去使能的 TB 是不需要计算 TB size 的，因为它不传输任何东西。除去 DCI format 2/2A/2B/2C 中被去使能了的 TB，对于 $0 \le I_{\text{MCS}} \le 28$，UE 会先通过 I_{MCS} 查 36.213 的 Table 7.1.7.1-1 得到 TBS 索引 I_{TBS}；并根据 5.1 节中“下行资源分配类型”的介绍计算出当前 PDSCH 传输占用的 PRB 数 N_{PRB}，然后针对以下各种情况分别处理：

- 1 个 TB 不映射到多于 1 层的空分复用，即只映射到 1 层时，根据 36.213 的 7.1.7.2.1 节的介绍来得到对应 TB 的 TB Size；
- 1 个 TB 映射到 2 层的空分复用时，根据 36.213 的 7.1.7.2.2 节的介绍来得到对应 TB 的 TB Size；
- 1 个 TB 映射到 3 层的空分复用，则根据 36.213 的 7.1.7.2.4 节的介绍来得到对应 TB 的 TB Size；
- 1 个 TB 映射到 4 层的空分复用时，根据 36.213 的 7.1.7.2.5 节的介绍来得到对应 TB 的 TB Size；

注意：TM 9 支持至多 8 层传输，但协议中在计算 TB size 时，却不存在 1 个 TB 映射到多于 4 层的情况。这是因为 UE 使用多于 4 层进行传输时，必定存在 2 个 TB，每个 TB 映射到少于或等于 4 层上（从 36.211 的 Table 6.3.3.2-1 上可以看出）。这也从另一个方面说明了，每个 TB 的 TB size 是独立计算的。

对于 $29 \le I_{\mathrm{MCS}} \le 31$ 而言（此时为下行的自适应重传），UE 会根据针对同一 TB 的最近一个 $0 \le I_{\mathrm{MCS}} \le 28$ 的 PDCCH 来决定其使用的 TB size（此时重传不改变 TB size）。如果对于同一 TB，并没有对应 $0 \le I_{\mathrm{MCS}} \le 28$ 的 PDCCH，并且同一 TB 的初始 PDSCH 传输是半静态调度（SPS）时，其 TB size 会由最近一个半静态调度分配时使用 PDCCH 来决定。

情况 3：如果某个 TB 在 TDD 的特殊子帧上传输，即在 DwPTS 上传输，则其 N_{PRB} 定义为 $N_{PRB} = \max\left\{\left\lfloor N'_{\mathrm{PRB}} \times 0.75 \right\rfloor, \quad 1\right\}$，其中 N'_{PRB} 等于情况 2 中计算出的 N_{PRB}。即在计算 DwPTS 上的 PDSCH 传输使用的 PRB 数时，是按照正常子帧的 0.75 倍来计算的（最小为 1）。接下来的处理与情况 2 相同。

这里有个特例：对于带正常循环前缀的特殊子帧配置 0 和 5，或带扩展循环前缀的特殊子帧配置 0 和 4，特殊子帧的 DwPTS 是不用于传输 PDSCH 的。这是由于此时分配给 DwPTS 的 OFDM 符号数过少导致的。

接下来我们以 FDD 下， 20 MHz 下行带宽，UE 配置了 TM 3 为例，来介绍如何确定 TB size 以及理论上可达到的下行峰值速率。

假设小区使用 2×2（或 4×2）天线传输，则 UE 支持至多 2 层传输，在使用空分复用的情况下，传输 2 个 TB，每个 TB 对应 1 层。同时假设每个 TB 的 $I_{\mathrm{MCS}} = 28$，对应 $I_{\mathrm{TBS}} = 26$，并且整个下行系统带宽的 100 个 RB 都分配给 UE，即 $N_{\mathrm{PRB}} = 100$，由于每个 TB 的 TB size 都是独立计算的，且一个 TB 只对应 1 层，所以此时应该按照 36.213 的 7.1.7.2.1 节中的介绍来得到某个 TB 的 TB size，得到结果为 75376 比特。由于每个子帧传输 2 个 TB，所以这个值需要乘以 2 才是 eNodeB 在该子帧发送给 UE 的数据量。换算成每秒的数据量为 75376 * 2 * 1000 = 150752000 比特（约为 143 Mbit/s）。

假设小区使用 4×4 天线传输，则 UE 支持至多 4 层传输，在使用空分复用的情况下，传输 2 个 TB，每个 TB 对应 2 层。同时假设每个 TB 的 $I_{\mathrm{MCS}} = 28$，对应 $I_{\mathrm{TBS}} = 26$，并且整个下行系统带宽的 100 个 RB 都分配给 UE，即 $N_{\mathrm{PRB}} = 100$，由于每个 TB 的 TB size 都是独立计算的，且一个 TB 对应 2 层，所以此时应该按照 36.213 的 7.1.7.2.2 节中的介绍来得到某个 TB 的 TB size。首先根据(I_{TBS} , N_{PRB})查 36.213 的 Table 7.1.7.2.1-1 得到 TBS_L1=75376，然后根据 TBS_L1 查 36.213 的 Table 7.1.7.2.2-1 的到 TBS_L2=149776，TBS_L2 就是对应 TB 的 TB size。由于每个子帧传输 2 个

TB，所以这个值需要乘以 2 才是 eNodeB 在该子帧发送给 UE 的数据量。换算成每秒的数据量为 149776 * 2 * 1000 = 299552000 比特（约为 285 Mbit/s）。

对于 TDD 而言，由于下行子帧数的减少以及特殊子帧上传输的数据量较少，理论上的下行峰值熟虑也会相应地减少。

从前面的介绍可以看出，计算 TB size 的过程就是首先确定 N_{PRB} 和 I_{TBS} ，然后查表的过程。对应协议，可参见 36.213 的 7.1.7 节。

对于下行 PDSCH 传输，如果初传中的某个 TB 的有效信道码率高于 0.930，则 UE 不会去解码该 TB。其中有效码率定义为下行信息比特数（包含 CRC）除以 PDSCH 上的物理信道比特数。

本节是从 UE 的角度来介绍如何确定调制阶数和 TB size 的，eNodeB 侧如何决定这些值与其调度和 LA（Link Adaptation）等相关，不同厂商会有不同的实现，这里就不予以介绍了。

PDSCH 只在下行子帧的数据区域上传输。并且那些用于传输 PBCH、PSS/SSS、小区特定的参考信号、UE 特定的参考信号、MBSFN 参考信号以及 CSI-RS 参考信号的 RE 不能用于传输 PDSCH。

接下来，我们将进一步细化前面的例子，将更多因素考虑在内，来介绍如何选择 TBS。

这里我们以 TDD 1，特殊子帧配置 7（10:2:2），下行系统带宽 $N_{RB}^{DL} = 25RB$ (即信道带宽为 5MHz)，小区使用正常的循环前缀，且使用 2×2 天线传输为例，来介绍子帧 0（其它下行子帧的计算思路类似，这里就不做介绍了）如何选择最大的 TBS。为了简单，这里我们假定 CFI = 1，UE 使用 TM 3（这里不考虑除小区特定的参考信号外的其它参考信号），单层（rank=1）传输，并且整个下行系统带宽都分配给了该 UE。

首先我们来计算可用于 PDSCH 传输的 RE 数。如表 5-7 所示。

表 5-7：可用于 PDSCH 传输的 RE 数的计算举例

下行系统带宽 N_{RB}^{DL}	=25 RB
整个下行系统带宽包含的 RE 数	= 25 (RB) * 12 (RE/symbol) * 14 (symbols) = 4200
小区特定的参考信号（CRS）所占的 RE 数	2 天线端口传输，每个 RB 需要预留 16 个 RE 给 CRS 使用，则整个下行系统带宽内，需要预留 16(RE/RB) * 25（RB） = 400 个 RE 给 CRS 使用
控制区域（Control Region）所占的 RE 数	CFI = 1，即控制区域占 1 个 OFDM 符号，则有控制区域所占的 RE 数（这里要减去 CRS 所占的 RE，并且此配置下每

	个 RB 在第 1 个 slot 的第 1 个 OFDM 符号内有 4 个 RE 用于 CRS）为： 12 (RE/symbol) * 25 (RB) – 4 (RE/RB) * 25 (RB) = 200
SSS 所占的 RE 数	TDD 下，SSS 会在子帧 0 的最后一个 OFDM 符号上发送（与 CRS 没有重叠），并在频域上占 72 个子载波，所以这里有 72 个 RE 被 SSS 占用
PBCH 所占的 RE 数	PBCH 在时域上位于每个系统帧的子帧 0 的第 2 个 slot 的前 4 个 OFDM 符号上，并在频域上占据 72 个中心子载波（不含 DC）。PBCH 映射到 RE 时，用于参考信号的 RE 是不能用于 PBCH 的，并且不管天线端口配置如何，都是按假定使用 4 天线端口来排除预留给 CRS 的 RE 的（见 36.211 的 6.6.4 节。也就是说，PBCH 所占的区域内，每个 RB 里有 8 个 RE 是不能被 PBCH 使用的），并且这些 RE 不能用于传输 PDSCH。因此有 PBCH 占用的 RE 数为：6 (RB) * 12 (RE/symbol) * 4 (symbol) - 6(RB) * 8 (RE/RB) = 240。注意：第 2 个 slot 的第 2 个 OFDM 符号上有 4(RE/RB) * 6(RB) = 24 个 RE 是由于按 4 天线端口来计算而预留，且不能用于 PDSCH 传输的。
可用于 PDSCH 传输的 RE 数为	= 4200(ALL) - 400(CRS) - 200(Control Region) – 72(SSS) – 240(PBCH) – 24(reserved) = **3264**

假设使用最高阶调制 64QAM（即 $Q_m = 6$ ），则 PDSCH 上的物理信道比特数为 3264 * 6 = 19584。假设该 UE 的信道质量很好，选择的 $I_{\text{MCS}} = 28$ ，对应的 $I_{\text{TBS}} = 26$ ， $N_{\text{PRB}} = 25$ ，查 36.213 的 Table 7.1.7.2.1-1 得到 TBS = 18336。从 36.212 的 5.1.2 节可以看出，此时所需的 code block 数为 $C = \lceil B/(Z-L) \rceil = \lceil (18336+24)/(6144-24) \rceil = 3$。其中“B = TBS + 24 位 CRC”，L=24 是 CRC 的长度，每个 code block 都要各自加上 CRC。计算码率时，是将 CRC 计算在内的，所以这里要加上 4 * 24 = 96 比特的 CRC。因此此时的码率为：(18336+24*4) / 19584 ≈ 0.94117647 > 0.930，导致该 TB 的有效信道码率高于 0.930，因此不能选择 $I_{\text{MCS}} = 28$。【注：code rate = (TBS + 24 + 24 * code block 数) / (Q_m * 可用于 PDSCH 传输的 RE 数)】

接着往下选择 $I_{\text{MCS}} = 27$ ，计算出的码率为(15840 + 24*4) / 19584 ≈ 0.81372549 < 0.930，也就是说，此配置下的子帧 0， I_{MCS} 最高只能选择 27，传输的最大 TB 块为 15840 比特。

上述例子只是为了向大家展示选择合适的 TBS 的思路。在不同的配置下、不同的子帧上，需要考虑的因素各不相同，例如特殊子帧上要考虑 DwPTS 所占 OFDM 符号数，TM 7 下要将 UE 特定的参考信号所占的 RE 排除在外，以及考虑不同 UE category 的能力等等。

如果将一个系统帧内的每个下行子帧（包括 DwPTS）所能选择的最大 TBS 计算出来，就能较精确地计算出协议上定义的理论上的下行最大速率。

【参考资料】

[1] TS 36.211 的 6.3.5 节 Mapping to resource elements

[2] TS 36.213 的 7.1.7 节 Modulation order and transport block size determination

[3] http://www.sharetechnote.com/html/Handbook_LTE_CodeRate.html

[4] http://wenku.baidu.com/view/6c1fd2b4ce2f0066f533228c.html

第6章　传输模式（Transmission Mode）

本章主要介绍下行 PDSCH 传输中使用的各种 TM 模式。首先，我们会对 TM 模式做个简单介绍，以便大家对 TM 模式有个直观的认识（6.1 节）。接着我们会对于 TM 模式中涉及的一些概念进行介绍，如 TB/codeword/layer/precoding/antenna port 等（6.2 节）。接下来，我们将介绍下行的 2 种天线预编码处理方式：基于码本和基于非码本的预编码（6.3 节）。然后，我们会对 SU-MIMO 和 MU-MIMO 的概念进行阐述（6.4 节）。最后，我们会对 Rel-10 中涉及的每种 TM 模式进行介绍（6.5 节）。

6.1　TM 模式简介

在 LTE 中，多天线传输可以描述成将调制后的数据映射到不同的天线端口的过程。其输入为调制符号（使用 QPSK，16QAM，64QAM 调制，对应 1 个或 2 个 TB）；其输出为每个天线端口上的一系列符号，这些符号随后会应用到 OFDM 的调制器中，并映射到该天线端口的时频网格（即 RE）上。

在发射机和/或接收机按照不同的方式来使用多天线传输可以实现不同的目的：

- 在发射机和/或接收机使用多天线可用来提供额外的分集以对抗无线信道的衰落（“传输分集”，transmit diversity）。这种情况下，不同天线所经历的信道应该拥有低的互相关性，这意味着天线间的间距需要足够大（空间分集，spatial diversity），或需要使用不同的天线极化方向（极化分集，polarization diversity）。传输分集主要用于降低信道衰落。
- 在发射机和/或接收机可以按照某种特定的方式来使用多天线以“形成”一个完整的波束。例如，可以最大化目标接收机/发射机方向上的整体天线增益，或抑制特定的主要干扰信号。这种“波束赋形（beamforming）”可基于天线间高或低的衰落相关性来实现。波束赋形主要用于提高小区的覆盖（coverage）。
- 在发射机和接收机上同时使用多天线可用来建立多个并行的传输信道，这样可以提供非常高的带宽利用率而不会降低相关功率有效性。换句话说，可以在有限的带宽上提供很高的数据速率而不会大比例地降低覆盖。这通常被称为“空分复用（spatial multiplexing）”，有时也称为 MIMO（Multi-Input Multi-Output）。空分复用主要用于提高数据传输速率，数据被分为多个流，这些流同时发送。

在 eNodeB 侧，每个小区可以选择配置 1、2、4、8 根发射天线。

不同的多天线传输方案对应不同的传输模式（Transmission Mode，简称为 TM 模式）。到 Rel-10 为止，LTE 支持 9 种 TM 模式。它们的区别在于**天线映射的不同特殊结构**，以及**解调时所使用的不同参考信号**（小区特定的参考信号或 UE 特定的参考信号），以及**所依赖的不同 CSI 反馈类型**。

TM 1：单天线端口传输（使用 port 0），应用于单天线传输的场合。

TM 2：发射分集模式，适用于小区边缘信道情况比较复杂，干扰较大的情况，也可用于 UE 高速移动的情况。使用 2 或 4 天线端口。

发射分集是默认的多天线传输模式。它通过在不同的天线上发送相同的数据实现数据冗余，从而提高 SINR，使得传输更加可靠。

TM 3：大延迟分集的开环空分复用，适合 UE 高速移动的场景；使用 2 或 4 天线端口。

TM 4：闭环空间复用，适合信道条件较好的场合，用于提供较高的数据传输速率；使用 2 或 4 个天线端口。

TM 5：MU-MIMO 传输模式，主要用来提高小区的容量；使用 2 或 4 天线端口。TM 5 是 TM 4 的 MU-MIMO 版本。

TM 6：Rank 1 的传输，主要适用于小区边缘的情况；使用 2 或 4 天线端口。

TM 7：单流波束赋形，主要适用于小区边缘的 UE，能够有效对抗干扰，只使用 port 5。

TM 8：双流波束赋形，可用于小区边缘的 UE，也可用于其它场景。使用 port 7 和 port 8，每个 port 对应一个 UE 特定的参考信号，这 2 个参考信号通过 2 个正交的 OCC（orthogonal cover code）区分。在空分复用下，这 2 个 OCC 和对应的参考信号被用于这 2 层的传输。

TM 9：支持最多 8 层的传输，主要是为了提高数据传输速率。使用 port 7~14。

36.213 的 Table 7.1-5 指明了不同的 TM 模式（使用 C-RNTI 加扰）可能使用的 DCI format 及其对应的传输样式（Transmission scheme）。同时也指明了每种 DCI format 可能位于的搜索空间（Search Space）。

Table 7.1-5: PDCCH and PDSCH configured by C-RNTI

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 1	DCI format 1A	Common and UE specific by C-RNTI	Single-antenna port, port 0 (see subclause 7.1.1)
	DCI format 1	UE specific by C-RNTI	Single-antenna port, port 0 (see subclause 7.1.1)
Mode 2	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 1	UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
Mode 3	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 2A	UE specific by C-RNTI	Large delay CDD (see subclause 7.1.3) or Transmit diversity (see subclause 7.1.2)
Mode 4	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 2	UE specific by C-RNTI	Closed-loop spatial multiplexing (see subclause 7.1.4)or Transmit diversity (see subclause 7.1.2)
Mode 5	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 1D	UE specific by C-RNTI	Multi-user MIMO (see subclause 7.1.5)
Mode 6	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 1B	UE specific by C-RNTI	Closed-loop spatial multiplexing (see subclause 7.1.4) using a single transmission layer
Mode 7	DCI format 1A	Common and UE specific by C-RNTI	If the number of PBCH antenna ports is one, Single-antenna port, port 0 is used (see subclause 7.1.1), otherwise Transmit diversity (see subclause 7.1.2)
	DCI format 1	UE specific by C-RNTI	Single-antenna port, port 5 (see subclause 7.1.1)
Mode 8	DCI format 1A	Common and UE specific by C-RNTI	If the number of PBCH antenna ports is one, Single-antenna port, port 0 is used (see subclause 7.1.1), otherwise Transmit diversity (see subclause 7.1.2)
	DCI format 2B	UE specific by C-RNTI	Dual layer transmission, port 7 and 8 (see subclause 7.1.5A) or single-antenna port, port 7 or 8 (see subclause 7.1.1)
Mode 9	DCI format 1A	Common and UE specific by C-RNTI	Non-MBSFN subframe: If the number of PBCH antenna ports is one, Single-antenna port, port 0 is used (see subclause 7.1.1), otherwise Transmit diversity (see subclause 7.1.2) MBSFN subframe: Single-antenna port, port 7 (see subclause 7.1.1)
	DCI format 2C	UE specific by C-RNTI	Up to 8 layer transmission, ports 7-14 (see subclause 7.1.5B) or single-antenna port, port 7 or 8 (see subclause 7.1.1)

从 36.213 的 Table 7.1-5 可以看出，每个 TM 模式都由 2 种不同的传输样式（Transmission scheme）组成。其中一个是发射分集或单天线端口传输，而另一个是基于性能优先选择的传输样式。如 TM 3，有 2 种不同的传输样式：发射分集和 Large delay CDD。如果 eNodeB 能从 UE 获得足够的反馈信息，以及信道条件较好，则会选择 Large delay CDD 来发送 PDSCH，否则会使用发射分集。

这 9 种 TM 模式只用于 DL-SCH 传输。对于 BCH 和 L1/L2 控制信道，可以说使用单天线端口传输或传输分集，但通常不说使用某种 TM 模式。

eNodeB 可能会在以下几种情况下通过特定的 RRC 消息给 UE 配置所使用的 TM 模式：（1）初始接入（*RRCConnectionSetup*）；（2）RRC 连接重建（*RRCConnectionReestablishment*）；（3）RRC 连接重配置（*RRCConnectionReconfiguration*）；（4）小区切换（Handover）。

这些 RRC 消息中都可能包含了 *transmissionMode* 字段，用于指定 UE 所使用的 TM 模式，该字段是可选的。UE 没有接收到 eNodeB 明确指定的 TM 模式之前，会基于在 PBCH 检测中确定的天线端口数（见 20.1 节关于“MIB”的介绍）来决定默认使用的 TM 模式：如果 PBCH 天线端口数为 1，则默认使用 TM 1；否则默认使用 TM 2。（见 36.331 的 9.2.4 节）

在 eNodeB 发送初始接入过程的 Msg 4 之前，UE 还不知道使用的 TM 模式（只有在收到 Msg 4 之后，UE 才会知道配置的 TM 模式），所以此时 eNodeB 只会使用默认的 TM 模式（2 或 4 天线端口时使用 TM 2；单天线端口时使用 TM 1）。

当 UE 处于 RRC_CONNECTED 态时，其 TM 模式并不会频繁改变。如果因为某些原因导致 TM 模式需要发生变化（由 eNodeB 决定），eNodeB 会通过发送 *RRCConnectionReconfiguration* 消息来通知 UE 改变 TM 模式。

eNodeB 在给某个 UE 配置 TM 模式时，会参考 UE 的能力（UE capability）和 eNodeB 的处理能力。例如：不能给 category 1 UE 配置 TM 4，因为 category 1 UE 并不支持空分复用。如果 eNodeB 并不支持 TM 9，即使 UE 支持 TM 9，eNodeB 也不会配置。（关于 UE category，见 36.306 的 4.1 节）

TM 模式是 UE 特定的信息，同一小区内的不同 UE，可能配置了不同的 TM 模式。

配置了载波聚合的 UE，在不同的 serving cell 上可以使用不同的 TM 模式。

TM 模式的选择会在以下几个方面影响 UE 的性能：

（1）TM 模式决定了 UE 可以使用的 2 种传输样式（transmission scheme，见 36.213 的 Table 7.1-5）。当 UE 处于 RRC_CONNECTED 态时，eNodeB 会基于 UE 的反馈来决定使用哪种传输样式。并在信道条件发生变化时，同一 UE 的 PDSCH 传输可以在传输样式之间来回切换（此时只需

要改变使用的 DCI format 即可）。以 TM 3 为例，信道条件较好时，可以使用 Large delay CDD；信道条件较差时，可以切换到发射分集。

（2）TM 模式决定了 UE 需要盲检的 DCI format，以及影响搜索空间。以 TM 3 为例，UE 在盲检 PDCCH 时，只需检测 DCI format 1A（对应发射分集）和 DCI format 2A（对应 Large delay CDD），这些信息可以从 36.213 的 Table 7.1-5 的第 2 列和第 3 列得到。

（3）TM 模式影响并限制了 UE 需要上报的 CSI 信息（CQI/PMI/RI）。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的第 5 章和 10.3 节

[2] 《Why Should You Care about Transmission Modes?》 by RAY

[3] 《Transmission Mode》 by RAY

6.2 TB/codeword/layer/precoding/port

在具体介绍每个传输模式之前，先介绍几个相关概念。

数据或信令在发给物理层之前，要先经过 PDCP，RLC 和 MAC 层的处理。但从物理层的角度来看，它只从 MAC 层接收数据：MAC PDU（Protocol Data Unit）。

下行 PDSCH 的物理层处理及其对应的 3GPP 协议见图 6-1。

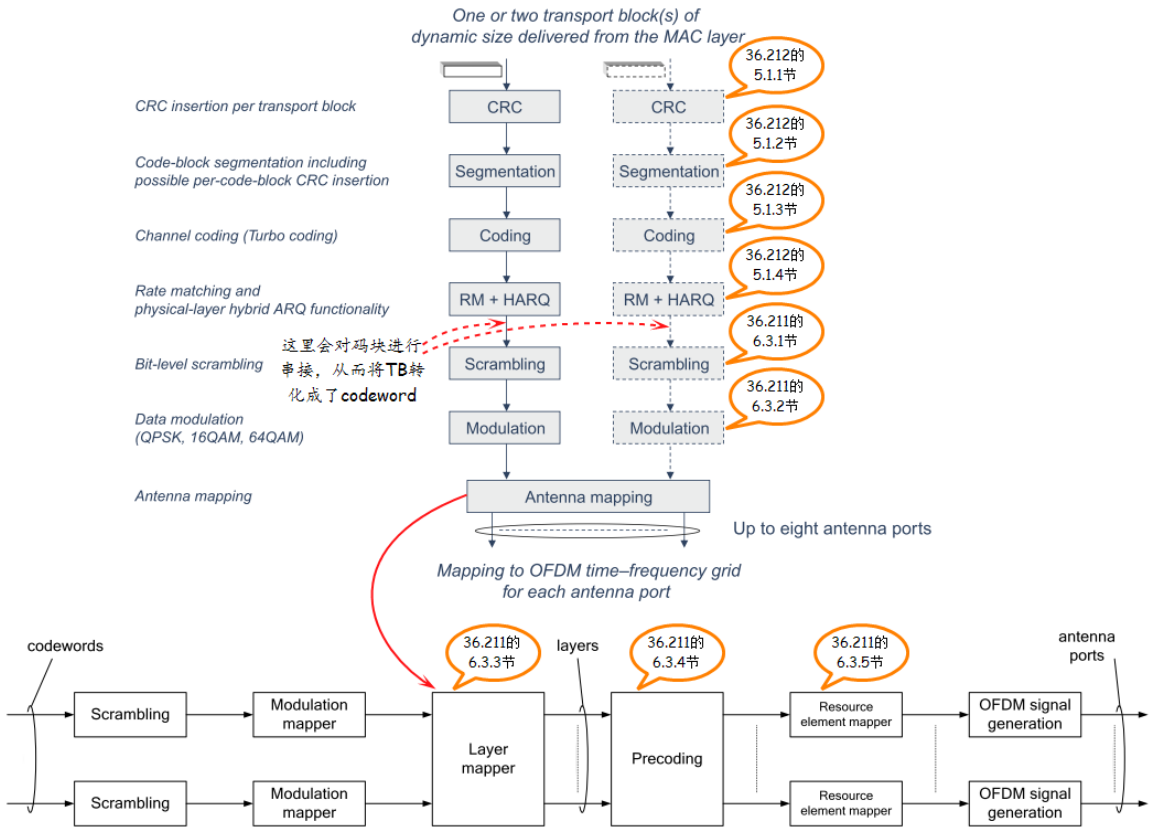


图 6-1：下行物理信道 PDSCH 处理流程，以及对应的 3GPP 协议章节

6.2.1 TB

从 MAC 层发往物理层的数据是以传输块（Transport Block，TB）的形式组织的。一个 TB 对应包含一个 MAC PDU 的数据块，这个数据块会在一个 TTI 内发送，同时也是 HARQ 重传的单位。如果 UE 不支持空分复用，则一个 TTI 至多会发送一个 TB；如果 UE 支持空分复用，则一个 TTI 至多会发送 2 个 TB。

注意：（1）协议中所说的一个 TTI 至多发送 2 个 TB，是从 UE 的角度来说的。也就说，对于某个 UE 而言，在下行传输（非载波聚合）时，eNodeB 在一个 TTI 内至多给该 UE 发送 2 个 TB；在上行传输时，该 UE 在一个 TTI 内至多给 eNodeB 发送 2 个 TB。而从 eNodeB 的角度来说，它可以在一个 TTI 调度多个 UE，所以 eNodeB 在一个 TTI 可以同时发送或接收多于 2 个 TB；（2）载波聚合下，UE 可能同时从多个小区接收数据，或给多个小区发送数据，因此在一个 TTI 可能发送或接收多于 2 个的 TB，但对应某一个小区，UE 在一个 TTI 可以发送或接收至多 2 个 TB。

6.2.2 codeword

一个码字（codeword）是对在一个 TTI 上发送的一个 TB 进行 CRC 插入、码块分割并为每个码块插入 CRC、信道编码、速率匹配之后，得到的数据码流。每个码字与一个 TB 相对应，因此一个 UE 在一个 TTI 至多发送 2 个码字。码字可以看作是带出错保护的 TB。

6.2.3 layer

对 1 个或 2 个码字进行加扰（scrambling）和调制（modulation）之后得到的复数符号（调制符号）进行层映射（layer mapping，见 36.211 的 6.3.3 节）后，会映射到一个或多个传输层（transmission layer，通常也称为 layer）。每层对应一条有效的数据流。

传输层的个数，即层数被称为“传输阶”或“传输秩（rank）”。传输秩是可以动态变化的。

层数必须小于或等于发射天线端口个数和接收天线端口个数二者的最小值，即“层数 ≤ min(发射天线端口数，接收天线端口数)”。

36.211 的 6.3.3 节介绍了以下几种情况如何进行层映射。

单天线端口传输：只有 1 个 codeword，只使用 1 层，codeword 直接映射到层上（见 36.211 的 6.3.3.1 节）；

传输分集：只有 1 个 codeword，使用 2 或 4 天线端口传输，分别对应 2 层或 4 层（但依然认为 rank = 1）。该 codeword 的所有调制符号按照轮询（round-robin）的方式逐一分布在不同的层上（见 36.211 的 6.3.3.3 节）；

空分复用：可能有 1 个或 2 个 codeword，可以被映射到 1~8 层。其层映射过程参见 36.211 的 6.3.3.2 节。

注意：虽然在 LTE 的规范中，发射分集也被描述成多层传输（见 36.211 的 6.3.3.3 节），但发射分集依然是 rank=1（单秩）的传输样式。

图 6-2 描述的是初传的 TB 到 1~4 layer 的映射。1 层传输只对应 1 个 TB，而 2 个 TB 对应 2 个或更多个传输层。

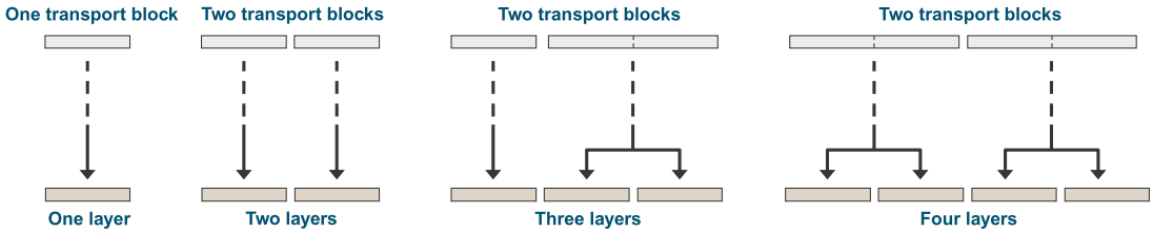


图 6-2：码字映射到层的举例（初传）

而对于 HARQ 重传，如果 2 个 TB 中只有其中 1 个需要重传，并且该 TB 在初传时映射到 2 层，则该 TB 的重传也会映射到 2 层。（36.212 中 5.3.3.1 节中支持 2 个 TB 的 DCI format 2/2A/2B/2C 有相关介绍）

经过层映射后，每层所包含的调制符号数是相同的，并等于在每个天线端口上发送的调制符号（symbol）数。

例如：在 2 个码字需要映射到 3 层的情况下。第二个码字（映射到第 2 层和第 3 层）对应的调制符号数是第一个码字（映射到第 1 层）的 2 倍。这是通过选择合适的 TB 大小和速率匹配功能来保证的。其映射方式如表 6-1 所示：

表 6-1：2 个 codeword 映射到 3 层

Number of layers	**Number of codewords**	**Codeword-to-layer mapping** $i = 0,1,...,M_{\text{symb}}^{\text{layer}} - 1$
3	2	$x^{(0)}(i) = d^{(0)}(i)$ $x^{(1)}(i) = d^{(1)}(2i)$ $x^{(2)}(i) = d^{(1)}(2i+1)$ $M_{\text{symb}}^{\text{layer}} = M_{\text{symb}}^{(0)} = M_{\text{symb}}^{(1)} / 2$

在 Rel-10 中，引入的 TM 9 至多支持 8 层传输，其层映射方式与前面的介绍类似。不同之处在于：（1）1 个 TB 可以映射到至多 4 层上；（2）在使用的层数大于 4 时，必定存在 2 个 TB；（3）对于 HARQ 重传，如果 2 个 TB 中只有其中 1 个需要重传，则该 TB 在初传时映射到几层，其重传也会映射到几层。

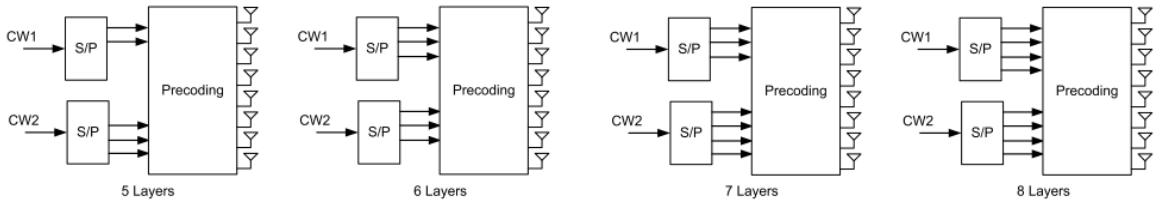


图 6-3：层数大于 4 时，码字到层的映射过程

可以将码字到层的映射可以看做是把一个码字等分成 N 份，每份放入独立的 1 层的过程。这里的 N 等于一个码字需要映射到的层数。

6.2.4 precoding

预编码（precoding）是使用预编码矩阵将层（layer）映射到天线端口（antenna port）的过程。预编码矩阵是 R×P 的矩阵，其中 R 为传输秩，即使用的传输层数，P 为天线端口的个数。

码本（codebook）是有限的可用预编码矩阵的集合。UE 只能从码本中选择合适的预编码矩阵（PMI）进行上报；而对于基于码本的预编码，eNodeB 只能从码本中选择合适的预编码矩阵，并通过 DCI 告诉 UE 对应的下行 PDSCH 传输所使用的预编码矩阵。

对于预编码，会在 6.5 节介绍具体的 TM 模式时予以重点描述。

6.2.5 antenna port

天线端口（antenna port）是逻辑上的概念，一个天线端口可以是一个物理发射天线，也可以是多个物理发射天线的合并。在这两种情况下，UE 的接收机（receiver）都不会去分解来自同一个天线端口的信号，因为从终端的角度来看，不管信道是由单个物理发射天线形成的，还是由多个物理发射天线合并而成的，这个天线端口对应的参考信号（Reference Signal）就定义了这个天线端口，终端都可以根据这个参考信号得到这个天线端口的信道估计。每个天线端口对应一个时频资源网格（time/frequency resource grid），有其独自的参考信号。一个天线端口就是一个信道，终端需要根据这个天线端口对应的参考信号进行信道估计和数据解调。

对于**下行**，一个小区支持的天线端口集合依赖于该小区的参考信号配置。

- 小区特定的参考信号（Cell-specific reference signal）：支持 1、2、4 个天线端口配置，并对应在天线端口 $p=0$ 、 $p\in\{0,1\}$ 和 $p\in\{0,1,2,3\}$ 上传输；
- MBSFN 参考信号：对应在天线端口 $p=4$ 上传输；
- UE 特定的参考信号（UE-specific reference signal）：在天线端口 $p=5$、 $p=7$、 $p=8$，或 $p\in\{7,8,9,10,11,12,13,14\}$ 中的一个或几个中传输；
- 定位参考信号（Positioning reference signal）：在天线端口 $p=6$ 上传输；
- CSI 参考信号：支持 1、2、4、8 个天线端口配置，并对应在天线端口 $p=15$ 、 $p=15,16$ 、 $p=15,...,18$ 和 $p=15,...,22$ 上传输。

TB、码字、传输层、天线端口之间的关系为：TB 数 = 码字数 ≤ 层数 ≤ 天线端口数

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.3.2 节

[2] 《Codewords and Layers and Ports, Oh My!》 by Don

[3] 《LTE 中 layer 和 codeword 的概念》

6.3 基于码本的预编码和基于非码本的预编码

LTE 支持 2 种方式进行下行天线预编码：基于码本的预编码（codebook-based precoding）和基于非码本的预编码（non-codebook-based precoding）。二者的主要区别在于前者使用小区特定的参考信号进行信道估计，而后者使用 UE 特定的参考信号进行信号估计。

TM 3/4/5/6 使用基于码本的预编码；TM 7/8/9 使用基于非码本的预编码。

6.3.1 基于码本的预编码

基于码本的预编码使用**小区特定的参考信号**来进行信道估计，一个小区至多有 4 个小区特定的参考信号，因此基于码本的预编码支持至多 4 个天线端口，即支持至多 4 层传输。

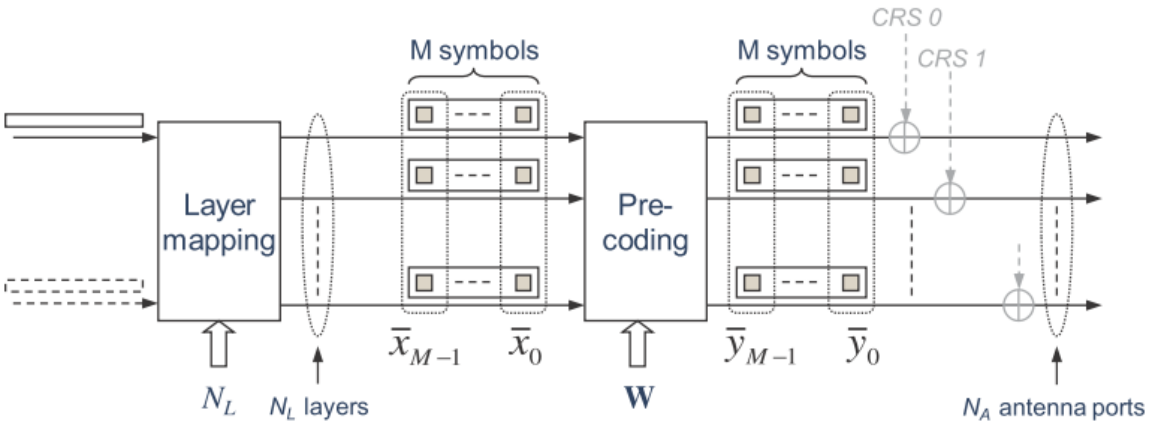


图 6-4：基于码本的预编码

图 6-4 为基于码本的预编码的基本处理流程。对应 1 个或 2 个 TB 的调制符号首先会通过层映射映射到 N_L 层（见 6.2.3 节），接着层会通过预编码映射到天线端口。

图 6-4 同样指明了对于基于码本的预编码，**小区特定的参考信号（CRS）是在预编码之后插入到对应的天线端口的**。因此，UE 必须明确地知道在发送端使用的是哪个预编码矩阵，以便正确地

处理接收到的信号。eNodeB 会通过对应的 DCI 明确地告诉 UE 当前 PDSCH 传输所使用的预编码信息。

基于码本的预编码有闭环预编码（closed-loop）和开环预编码（open-loop）之分。二者的区别主要在于预编码矩阵的结构不同，以及 eNodeB 如何选择预编码矩阵并通知 UE。

4.3.1.1 开环预编码（open loop precoding）

对于使用开环预编码的 TM 3，预编码不依赖于 UE 上报的建议使用的预编码矩阵（即 UE 并不上报 PMI），eNodeB 也不需要发送确切的信令来告诉 UE 使用哪个预编码矩阵，这也是“open loop”的由来。相反地，eNodeB 和 UE 使用相同的方式从一个预先定义的集合中选择一个预编码矩阵。只有 TM 3 使用开环预编码，具体可参见 6.5.3 节的介绍。

4.3.1.2 闭环预编码（close loop precoding）

在闭环预编码中，eNodeB 是基于 UE 的反馈来选择预编码矩阵的。基于对小区特定的参考信号的测量，UE 会选择一个合适的传输层数以及合适的预编码矩阵，并以 RI 和 PMI 的方式反馈给 eNodeB。RI 和 PMI 只是 UE 提供的一个建议，eNodeB 并不一定需要严格遵循 UE 提供的 RI/PMI 来进行传输。如果 eNodeB 重新选择了传输层数或预编码矩阵，则需要明确地告诉 UE。这也是“close loop”的由来。

为了限制上下行信令的大小，对应每个 rank 值，只定义了有限集合的预编码矩阵，该集合也称为码本（codebook）。基于码本的预编码中，UE（当上报 PMI 时）和 eNodeB（当选择一个预编码矩阵用于 PDSCH 传输时）都只会从对应的码本中选择一个预编码矩阵。当 UE 上报 PMI，或是 eNodeB 告诉 UE 所选择的预编码矩阵时，只需要通知对应的预编码矩阵索引即可。

TM 4/5/6 均使用闭环预编码。

对应 2 天线端口，可用的预编码矩阵集合（即码本）见 36.211 的 Table 6.3.4.2.3-1。对应 4 天线端口，可用的预编码矩阵集合（即码本）见 36.211 的 Table 6.3.4.2.3-2。

注：（1）后续介绍 TM 4 时，我们会看到，对于闭环的空分复用，当使用 2 天线端口且层数为 2 时，不能使用 36.211 的 Table 6.3.4.2.3-1 中的 codebook index 0。该矩阵只能用于开环空分复用；（2）闭环预编码也支持单层传输。

在闭环 MIMO 模式（对应闭环预编码）中，UE 需要向 eNodeB 提供信道状态的反馈：CQI/RI/PMI。这种反馈机制能够使发射机充分地了解信道状态，从而提供更高的速率。但是，只有在信道信息足够精确并且有丰富的多径环境的情况下，这种传输样式才能提供最好的性能。因此，闭环 MIMO 通常使用于低速移动的场景，例如 UE 固定在一个地方，或 UE 以步行的速度在移动。

而在高速移动的场景，通常会使用开环 MIMO（对应开环预编码），这是因为信道状态的反馈不够及时，PMI 被认为是不可靠的且不会被使用。而在 TDD 网络中，信道是相互的（上下行使用相同的链路和频段），因此可基于上行传输来得到更为精确的下行信道信息。因此与 FDD 网络相比，MIMO 能在更广的信道条件下提升 TDD 网络的性能。

6.3.2 基于非码本的预编码

基于非码本的预编码（non-codebook-based precoding）也只用于 DL-SCH 传输，并使用 UE 特定的参考信号进行解调。基于非码本的预编码用于 TM 7（只支持单层传输）、TM 8（支持至多 2 层传输）和 TM 9（支持至多 8 层传输）。

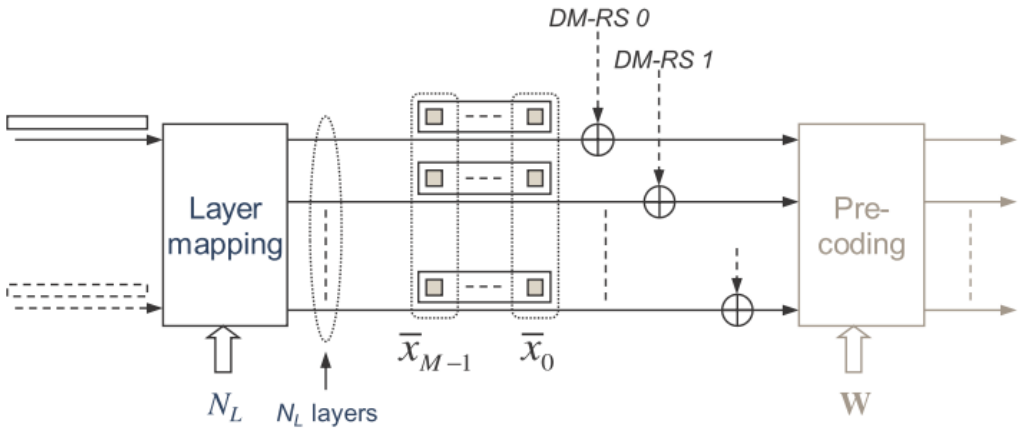


图 6-5：基于非码本的预编码

基于非码本的预编码与基于码本的预编码的不同之处在于：基于码本的预编码，其**小区特定的参考信号**是在预编码**之后**插入到每个天线端口的。而基于非码本的预编码，其 **UE 特定的参考信号**是在预编码**之前**插入到每个天线端口的。使用经过预编码后的 UE 特定的参考信号，允许接收侧解调和恢复传输层时，无需明确地知道发射端所使用的预编码信息。

基于预编码后的 UE 特定的参考信号进行的信道估计可以直接反映传输层所经历的信道，包括预编码信息，因此可以直接用于不同层的相干解调。也就是说，eNodeB 无需告诉 UE 任何关于预编码矩阵的信息，而只需要告诉 UE 所使用的层数，即 rank 值。因此，eNodeB 可以选择任意一个预编码矩阵，并且无需告诉 UE 所使用的预编码矩阵，这也是称为“基于非码本的预编码”的原因。但需要注意的是，基于非码本的预编码依然可以基于 UE 反馈的 PMI 来选择预编码矩阵。

虽然在 LTE 规范中，基于非码本的预编码并没有明确指定预编码信息，但 eNodeB 依然需要选择一个合适的预编码矩阵用于传输，如何选择取决于厂商的实现。基于非码本的预编码下有 2 种方式可用来选择预编码。

方式一：eNodeB 可以基于 SRS 等上行传输来估计上行信道状态，并依赖上下行信道的互益性，来选择用于下行传输的预编码矩阵。这在上下行使用相同频率的 TDD 时，通常更有意义。

方式二：eNodeB 可以基于 UE 的反馈来选择预编码矩阵（配置了 TM 7 的 UE 不上报 PMI，所以不能使用这种方式）。对于 TM 8/9 中，UE 的反馈方式可以与基于码本的闭环预编码类似，并且使用与基于码本的预编码相同的预编码矩阵集合（但扩展到支持至多 8 个天线端口）。并且对于使用 8 天线端口的 TM 9 而言，UE 的测量是基于 CSI-RS 而不是小区特定的参考信号进行的。

基于非码本的预编码同样可能使用协议中预先定义好的码本。但与基于码本的预编码不同的是，这些码本只在 UE 上报 PMI 时使用，而在实际的下行传输中，eNodeB 不会告知 UE，UE 也不会关心当前 PDSCH 传输使用的预编码矩阵。

基于非码本的预编码中，eNodeB 可以选择一个任意的预编码矩阵用于 PDSCH 传输也是图 6-5 中预编码部分使用阴影表示的原因。在 LTE 规范中，图 6-5 中的预编码部分是不可见的。更严格地说，在基于非码本的预编码中，天线端口与层直接一一映射。这可以从 36.211 的 6.3.4.4 节中看出，从层到天线端口的映射过程中，并未使用到预编码矩阵，层直接映射到天线端口中。其中的映射关系如下：

$$\begin{bmatrix} y^{(7)}(i) \\ y^{(8)}(i) \\ \vdots \\ y^{(6+\upsilon)}(i) \end{bmatrix} = \begin{bmatrix} x^{(0)}(i) \\ x^{(1)}(i) \\ \vdots \\ x^{(\upsilon-1)}(i) \end{bmatrix}$$

这里，预编码是在天线端口处理之后进行的。

表 6-2：基于码本的预编码和基于非码本的预编码的区别

基于码本的预编码	基于非码本的预编码
使用小区特定的参考信号来解调数据	使用 UE 特定的参考信号来解调数据
小区特定的参考信号是在预编码之后插入到每个天线端口的	UE 特定的参考信号是在预编码之前插入到每个天线端口的
eNodeB 需要告诉 UE 当前 PDSCH 传输使用的预编码信息（TM 3 的预编码方式固定，但还是要告诉 UE 使用层数信息）	eNodeB 不需要告诉 UE 当前 PDSCH 传输使用的预编码信息

需要强调的是，无论是基于码本的预编码，还是基于非码本的预编码，UE 在上报 PMI 时都必须从协议中预先定义好的码本中选择一个预编码矩阵来上报。eNodeB 在发送下行 PDSCH 数据时，基于码本的预编码所选择的预编码矩阵必须属于码本中的一员，并通过 DCI 告诉 UE；但基于非码本的预编码所选择的预编码矩阵可以属于，也可以不属于码本中的一员，并且不需要告知 UE。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.3.2 节和 10.3.3 节

[2] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 11.2.2.2 节

6.4 SU-MIMO 和 MU-MIMO

空分复用意味着多层传输，即将相同的时频资源分配给同一个 UE，并用于发送多个并行的传输。空分复用还要求在发送端和接收端同时存在多根天线，并加上发射机和接收机的信号处理，组合在一起以抑制不同层间的干扰。

空分复用（Spatial Multiplexing）通常也称为 MIMO（Multi-Input-Multi-Output）传输，以反映空分复用中的信道拥有多输入（multiple inputs，对应 eNodeB 侧有多根发射天线）多输出（multiple outputs，对应 UE 侧有多根接收天线），而更具体的术语是单用户 MIMO，即 SU-MIMO，以便同 MU-MIMO 进行区分。即通常所说的“空分复用”、“MIMO”专指 SU-MIMO。

空分复用通常要求相对高的 SINR，例如 15 dB 或更高。一个基站的覆盖范围内通常有很多激活的 UE，而且多个基站可能对邻居基站产生干扰，所以通常 SINR 会较低，这在多个相邻的小区使用相同的频段时更为明显，此时只有小区中心的 UE 才会有足够大的 SINR，以进行 SU-MIMO。SU-MIMO 能够提高单个 UE 的峰值速率。

LTE 下行支持 MIMO 技术进行空间维度的复用。空间复用支持单用户 MIMO(Single-User-MIMO，即 SU-MIMO)模式或者多用户 MIMO (Multiple-User-MIMO，即 MU-MIMO)模式。

SU-MIMO 和 MU-MIMO 都支持通过预编码的方法来降低或者控制空间复用数据流之间的干扰，从而改善 MIMO 技术的性能。

SU-MIMO 中，空间复用的数据流调度给一个单独的用户，以提升该用户的传输速率和频谱效率。在 SU-MIMO 中，分配给该 UE 的时频资源由该 UE 独占。

MU-MIMO 中，空间复用的数据流调度给多个用户，多个用户通过空分方式共享同一时频资源，系统可以通过空间维度的多用户调度获得额外的多用户分集增益。MU-MIMO 中，多个 UE 使用相同的时频资源，彼此之间通过空分方式予以区别（注意：这些 UE 可以共用相同的层）。这要求 eNodeB 使用多根发射天线来区分这 2 个或更多个不同的传输。同时，MU-MIMO 通常要求有更精确地 CSI 反馈。

在 LTE 中，MU-MIMO 并不要求 UE 知道是否在同样的时频资源上存在其它 UE 的传输，允许不同的 UE 只在部分资源上存在重合，并在理论上不要求 UE 拥有多根接收天线（但是 LTE 的性能要求通常假定 UE 侧至少存在 2 根接收天线）。并且在不同的 RB 上，可以有不同的复用 UE 数。

其实 SU-MIMO 和 MU-MIMO 对于 UE 来说是不可见的，但对 eNodeB 是可见的。eNodeB 在调度时，决定是否在同一时频资源上调度多个 UE。

SU-MIMO 能够显著提高某个 UE 的吞吐量；而 MU-MIMO 能够显著提高整个小区的吞吐量和容量。

在 LTE 中， TM 3/4/8/9 支持 SU-MIMO。

在 LTE 中， TM 5/8/9 支持 MU-MIMO，其中 TM 5 使用基于码本的预编码，TM 8/9 使用基于非码本的预编码。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.3.2 节和 10.3.3 节

[2] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 11.1.4 节

6.5 TM 模式

从本节起，我们将详细介绍 Rel-10 中的各种 TM 模式。

注：在具体介绍各种 TM 模式时，只会简单涉及 UE 如何上报 CSI（包括 CQI/RI/PMI），具体内容会在第 10 章介绍“CSI”时予以描述。

6.5.1 TM 1

表 6-3：TM 1，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 1	DCI format 1A	Common and UE specific by C-RNTI	Single-antenna port, port 0 (see subclause 7.1.1)
	DCI format 1	UE specific by C-RNTI	Single-antenna port, port 0 (see subclause 7.1.1)

前面已经介绍过，如果在 PBCH 检测中确定的天线端口数为 1，且 eNodeB 还未给 UE 配置其它 TM 模式的情况下，默认使用的 TM 模式为 TM 1。在这种特定的情况下，我们可以假定该小区只有一根发射天线。但在正常的 LTE 部署中，小区通常不会只有一根发射天线。

TM 1 使用单天线端口 port 0，这意味着即使小区拥有多根发射天线，发往对应 UE 的所有下行传输也只使用 port 0。这种传输样式限制了该 UE 的下行性能。由于不能使用多天线传输，该 UE 不能使用“发射分集”或“MIMO”技术，因此无法获得更可靠的信号传输（使用发射分集）和提高该 UE 的吞吐量（使用 MIMO）。

TM 1 意味着小区只能使用一根发射天线（transmit antenna）给该 UE 发送下行数据，但并不意味着 UE 只能使用一个接收天线（receive antenna）来接收数据。UE 可以使用 1 根接收天线（对应 SISO）或 2 根接收天线（对应 SIMO）。使用 SIMO，UE 能够使用接收分集技术来提高下行的覆盖（coverage）。

SISO 是 Single Input Single Output 的简称，对应在发射端（eNodeB）使用一根发射天线，在接收端（UE）使用一根接收天线。SIMO 是 Single Input Multiple Output 的简称，对应在发射端（eNodeB）使用一根发射天线，在接收端（UE）使用多根接收天线。

如果 UE 配置了 TM 1，则 UE 在接收使用 C-RNTI 加扰的下行数据时，只需要盲检 DCI format 1A（在公共搜索空间和 UE 特定的搜索空间中进行盲检）和 DCI format 1（只在 UE 特定的搜索空间中进行盲检）。

TM 1 也影响到 UE 的反馈，由于只使用单天线端口传输（意味着不支持空分复用，且只支持单层传输），因此 UE 无需反馈 PMI 和 RI。同时 TM 1 限制了 UE 只需基于一个天线端口来上报相关的 CQI。

只支持 TM 1 的小区通常是只用于单点覆盖的私有网络（无法使用发射分集，所以小区的 coverage 较小），该网络只能供较少的用户使用，且这些用户在上下行都不要求高速的服务。另一个应用场景是低速的设备监控应用，这种场景下，上下行的开销都不大。TM 1 还可用于便宜的 femtocell 上，这种小区只支持有限数量的 UE。

单天线端口传输中只有 1 个 codeword，只使用 1 层，codeword 直接映射到层上，且层上的数据直接映射到天线端口上。3GPP 协议中关于 TM 1 的处理见表 6-4。

表 6-4：TM 1，码字/层/预编码处理

传输模式 (Transmission Mode)	码字数 (No of Codewords)	层数 (Number of Layers)	预编码 (Precoding)	码本 (Codebook)	天线端口 (Antenna Ports)	说明
TM 1	1	1 (36.211 的 6.3.3.1 节)	36.211 的 6.3.4.1 节	无	单天线端口： port 0	单天线端口传输

【参考资料】

[1] 《Transmission Mode #1 ….Or is it?》

6.5.2 TM 2

表 6-5：TM 2，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 2	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 1	UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)

传输分集（transmit diversity）主要用于提高数据传输的可靠性。相同的数据在多根发射天线上冗余发送，称为发射分集（TX diversity），这可以提高 SINR。发射分集并不能提高吞吐量（throughput）。

发射分集可用于任何下行传输，尤其适用于无法通过链路自适应和（或）信道相关的调度来适应变化的信道情况的场景。如果 eNodeB 无法获知不同传输天线的下行信道信息，则多天线传输只能使用发射分集而无法使用波束赋形。为了使用发射分集，各天线经历的信道应该存在低相关性，这可以通过足够大的天线间距或使用不同的极化方向来实现。

发射分集并不要求紧凑的 PMI 反馈，因此能够支持高速移动的 UE。

对于 BCH 和 PCH 传输信道，以及 L1/L2 控制信令，发射分集是唯一使用的多天线传输机制。

如果小区配置了 2 个小区特定的参考信号，则 BCH/PCH 和 L1/L2 控制信令必须使用 2 天线发射分集；如果小区配置了 4 个小区特定的参考信号，则 BCH/PCH 和 L1/L2 控制信令必须使用 4 天线发射分集。对于 BCH/PCH 和 L1/L2 控制信令，eNodeB 不需要告诉 UE 它们使用的是哪种多天线传输模式，UE 可以通过 PBCH 盲检来确定（具体过程见 20.1 节关于“MIB”的介绍）。

当配置了 TM 2 时，DL-SCH 传输就使用发射分集。对于 TM 3~TM9，其多天线传输可以回退到“发射分集”。更特别地说，当使用 DCI format 1A 时，无论配置了什么 TM 模式（除 TM 1 外），就意味着使用发射分集。（见 36.213 的 Table 7.1-5）

发射分集使用小区特定的参考信号（cell-specific RS）来进行信道估计，所以使用发射分集的 PDSCH 数据总是与小区特定的参考信号在相同的天线端口上传输。

2 天线端口的发射分集基于 SFBC（Space-Frequency Block Coding）。SFBC 意味着 2 个连续的调制符号 S_i 和 S_{i+1} 直接映射到第一个天线端口频域上相邻的 RE 上。而在第二个天线端口上，频域交换和变化后的符号 $-S_{i+1}^{*}$ 和 S_i^{*} 映射到对应的 RE 上，其中*表示复共轭。如图 6-6 所示。

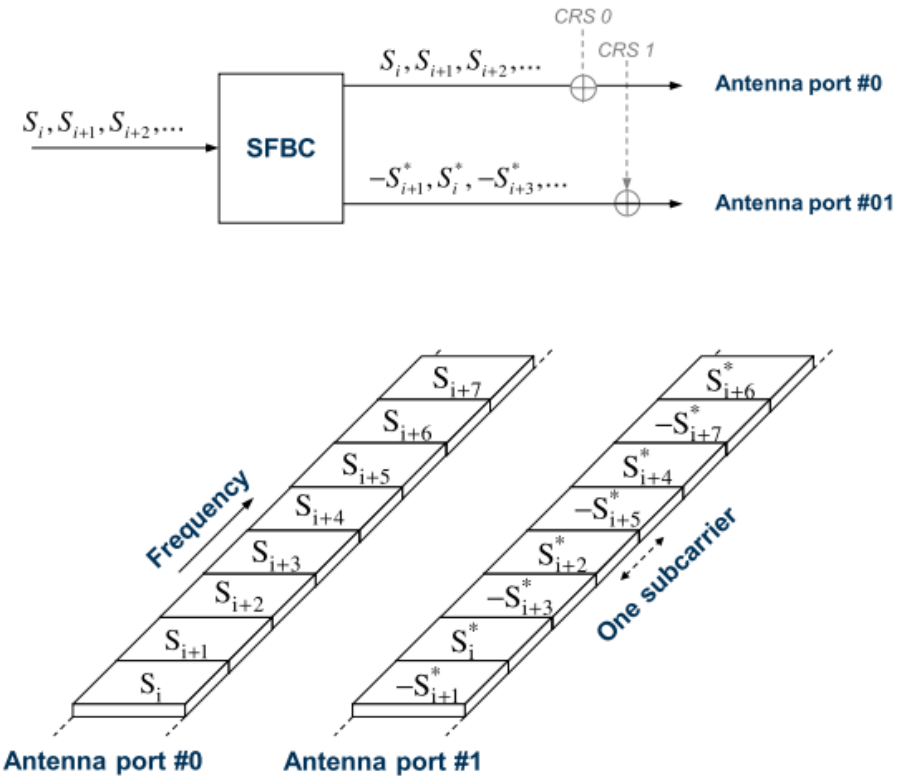


FIGURE 10.15

Transmit diversity for two antenna ports – SFBC.

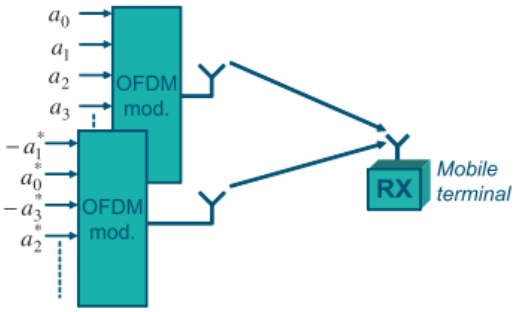


FIGURE 5.10

Space–Frequency Block Coding (SFBC) assuming two transmit antennas.

图 6-6：SFBC

4 天线端口的发射分集基于 SFBC（Space-Frequency Block Coding）和 FSTD（Frequency-Switched Transmit Diversity）的组合。组合的 SFBC/FSTD 意味着调制符号在不同的天线端口对

（port 0 和 port 2 为一对，port 1 和 port 3 为一对）之间通过交替传输的 SFBC 形式进行传输。在一对天线端口上用于传输的 RE，在另一对天线端口上不能用于传输。组合的 SFBC/FSTD 从某种意义上讲，以 4 个调制符号（对应频域上 4 个连续的 RE）为一组进行操作，这就是 REG（Resource Element Group）的由来，每个 REG 由 4 个频域上连续的 RE 组成。如图 6-7 所示。

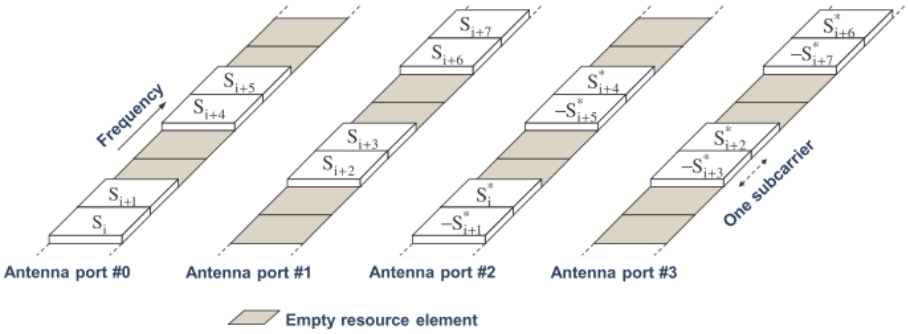


FIGURE 10.16

Transmit diversity for four antenna ports – combined SFBC/FSTD.

图 6-7：Combined SFBC/FSTD

注：关于发射分集的实现方式，可参见参考资料[1]。

前面已经介绍过，如果在 PBCH 检测中确定的天线端口数为 2 或 4，且 eNodeB 还未给 UE 配置其它 TM 模式的情况下，默认使用的 TM 模式为 TM 2（2 天线时使用 SFBC，4 天线时使用 SFBC 和 FSTD 的组合）。

TM 2 的 rank = 1，不支持空分复用，因此 UE 无需反馈 PMI 和 RI。但 TM 2 依然需要 CQI 的上报。

传输分集中只有 1 个 codeword，使用 2 或 4 天线端口传输，分别对应 2 层或 4 层（但依然认为 rank = 1）。该 codeword 的所有调制符号按照轮询（round-robin）的方式逐一分布在不同的层上（见 36.211 的 6.3.3.3 节）。每层的数据按照固定的方式映射到天线端口上，所以 eNodeB 无需告诉 UE 如何进行预编码。

3GPP 协议中关于 TM 2 的处理见表 6-6。

表 6-6：TM 2，码字/层/预编码处理

传输模式 (Transmission Mode)	码字数 (No of Codewords)	层数 (No of Layers)	预编码 (Precoding)	码本 (Codebook)	天线端口 (Antenna Ports)	说明

TM 2	1	**2 或 4（rank=1）** (36.211 的 6.3.3.3 节)	36.211 的 6.3.4.3 节	无	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	传输分集

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 5.4.1 节和 10.3.1 节

6.5.3 TM 3

表 6-7：TM 3，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 3	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 2A	UE specific by C-RNTI	Large delay CDD (see subclause 7.1.3) or Transmit diversity (see subclause 7.1.2)

TM 3 是开环空分复用（Open-Loop Spatial Multiplexing，OLSM），有时也称为“Open-Loop MIMO”或“Large Delay CDD”。

配置了 TM 3 的 UE 既支持健壮且可靠的“发射分集”，也支持称之为“Large-Delay CDD”的高吞吐量 MIMO。该 MIMO 支持 2/3/4 层传输。

与其它 MIMO 传输样式（TM 4/8/9）相比，TM 3 是最简单的。它所需要的 UE 反馈少于其它 MIMO 传输样式，因此 UE 的复杂度更低，eNodeB 实现起来也更简单。

基于 UE 反馈的 CQI 和 RI（TM 3 不需要反馈 PMI），eNodeB 中的调度器会动态地选择使用发射分集还是 Large Delay CDD 来给该 UE 发送下行数据。除非估计的下行信道质量高于一定的阈值且 UE 上报的 RI 大于 1，否则 UE 不会使用 MIMO 而会使用发射分集。

配置了 TM 3 的 UE，其上报的 CQI 是基于上报的 RI 的。如果上报的 RI = 1，则 CQI 是基于发射分集来上报的；如果上报的 RI = 2/3/4，则 CQI 是基于 Large-Delay CDD 来上报的。UE 侧进行信道估计时使用的是小区特定的参考信号。

前面已经介绍过，对于使用开环操作的 TM 3，预编码不依赖于 UE 上报的建议使用的预编码矩阵（即 UE 并不上报 PMI），eNodeB 也不需要发送确切的信令以告诉 UE 使用哪个预编码矩阵（这

也是“open-loop”的由来）。相反地，eNodeB 和 UE 使用相同的方式从一个预先定义的集合中选择一个预编码矩阵（见图 6-8）。

开环预编码的一个应用场景是 UE 在高速移动，这种场景下信道条件变化快，PMI 上报的延迟较大而导致无法得到精确的反馈。

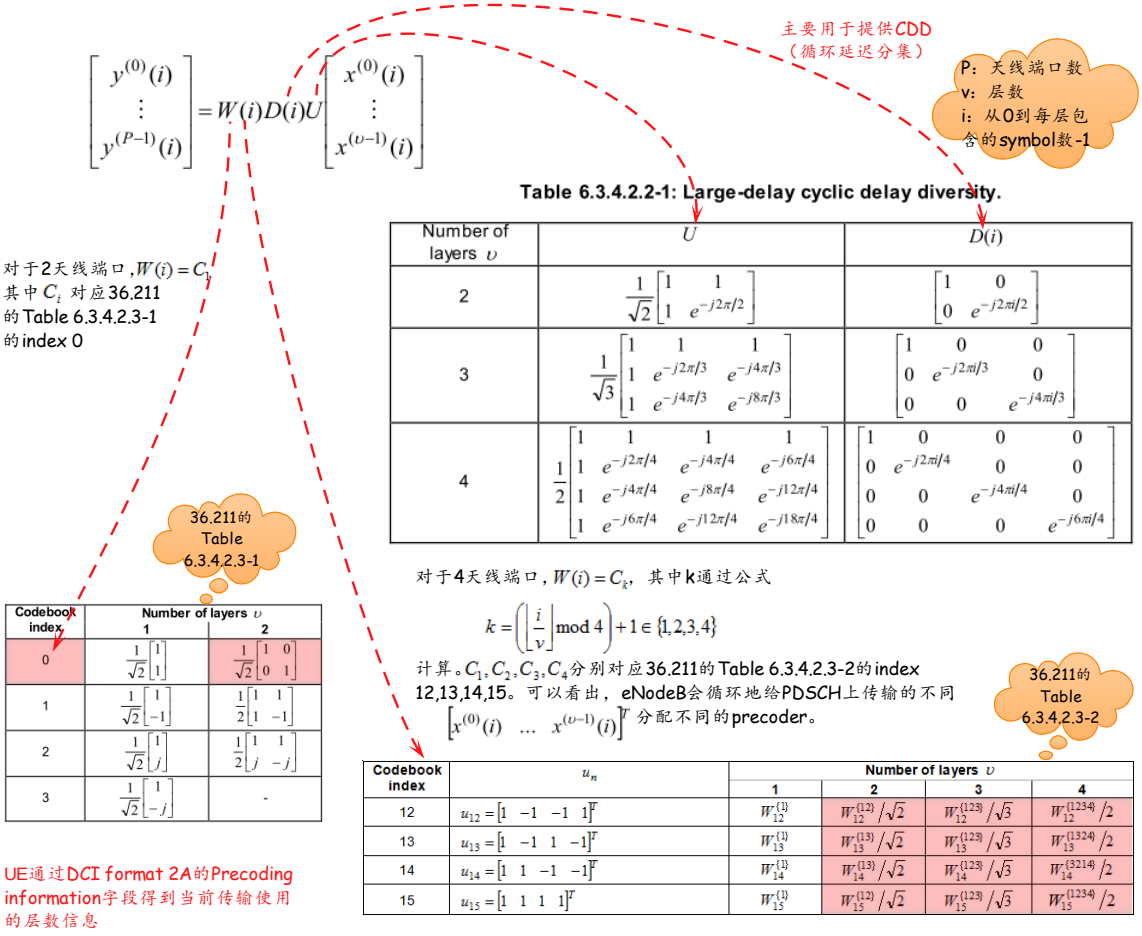


图 6-8：TM 3 的预编码（precoding）处理

从图 6-8 可以看出，对于 TM 3 的预编码，从层映射到天线端口的公式中，只要层数 v 确定了，矩阵 $D(i)$ 和 U 就确定了。而对于 2 天线端口，$W(i)$ 固定为 36.211 的 Table 6.3.4.2.3-1 中层数为 2，

index 为 0 对应的预编码矩阵；对于 4 天线端口，通过图 6-8 中的公式 $k = \left(\left\lfloor \frac{i}{\upsilon} \right\rfloor \bmod 4 \right) + 1 \in \{1,2,3,4\}$ 可以计算出 $W(i)$ 对应的预编码矩阵。

UE 可以通过盲检 PBCH 获得天线端口数， 并通过 DCI format 2A 的 precoding information 字段（2 天线端口传输时，根据使能的 TB 数）获得层数 v，因此 UE 可以通过这些信息知道对应的 PDSCH 传输所使用的预编码信息（如图 6-8），而不需要 eNodeB 发送确切的信令以告诉 UE 使用哪个预编码矩阵。

DCI format 2A 中的 precoding information 字段包含的比特数如 36.212 的 Table 5.3.3.1.5A-1 所示。可以看出，如果使用 2 天线端口传输，则不存在该字段。

Table 5.3.3.1.5A-1: Number of bits for precoding information.

Number of antenna ports at eNodeB	Number of bits for precoding information
2	0
4	2

DCI format 2A 中的 precoding information 字段按 36.212 的 Table 5.3.3.1.5A-2 来解释：

- 如果只使能了一个 TB，则 index 1 只用于支持对应 TB 的重传，且该 TB 在前一次传输中使用的是 2 层的 large delay CDD。
- 如果使用 2 天线端口传输，则 precoding information 字段是不存在的。此时，如果 2 个 TB 都使能了，则使用 2 层传输；如果只有 codeword 0 使能，而 codeword 1 去使能，则使用传输分集。

Table 5.3.3.1.5A-2: Content of precoding information field for 4 antenna ports.

One codeword: Codeword 0 enabled, Codeword 1 disabled	Two codewords: Codeword 0 enabled, Codeword 1 enabled

Bit field mapped to index	Message	Bit field mapped to index	Message
0	4 layers: Transmit diversity	0	2 layers: precoder cycling with large delay CDD
1	2 layers: precoder cycling with large delay CDD	1	3 layers: precoder cycling with large delay CDD
2	reserved	2	4 layers: precoder cycling with large delay CDD
3	reserved	3	reserved

注意：DCI format 2A 中的 precoding information 字段指定的是当前传输使用的层数，而不是指定预编码矩阵。

除了预编码，每一个天线上的信号都会有一个特殊的延迟（cyclic delay diversity，称为 CDD），这样人为地增加了频率分集。$D(i)U$ 的基本思想是对不同层看到的任何信道差异进行平均。

3GPP 协议中关于 TM 3 的处理见表 6-8。

表 6-8：TM 3，码字/层/预编码处理

传输模式 (Transmission Mode)	码字数 (No of Codewords)	层数 (No of Layers)	预编码 (Precoding)	码本 (Codebook)	天线端口 (Antenna Ports)	说明
TM 3	1	**2 或 4**（**rank=1**）(36.211 的 6.3.3.3 节)	36.211 的 6.3.4.3 节	无	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	传输分集
	2	**2~4** (36.211 的 6.3.3.2 节)	36.211 的 6.3.4.2.2 节	36.211 的 Table 6.3.4.2.3-1 和 Table 6.3.4.2.3-2	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	大延迟分集的开环空分复用

注：使用 Large Delay CDD 时，也可能只有一个 codeword，此时另一个 codeword 是去使能的（例如 index=1 的项用于重传时）。具体使用传输分集还是 Large Delay CDD 需要根据使用的 DCI format 来确定。

简单小结：配置了 TM 3 的 UE 会将 CQI 和建议使用层数（即 RI）上报给 eNodeB，eNodeB 中的调度器会动态地选择使用发射分集还是 Large Delay CDD 来给该 UE 发送下行数据，并通过对应的 DCI format 告知 UE。如果使用的是 Large Delay CDD，UE 会读取 DCI format 2A 中 precoding information 字段（2 天线端口传输时，是根据使能的 TB 数）来确定当前传输使用的层数（可以与 UE 上报的 RI 不同），然后根据图 6-8 的介绍来确定预编码矩阵，从而得到当前 PDSCH 传输的预编码信息。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.3.2.2 节

[2] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 11.2.2.2 节

[3] 《Transmission Mode 3 ….The first of the 4 MIMOs!》by RAY

6.5.4 TM 4

表 6-9：TM 4，使用 C-RNTI 加扰

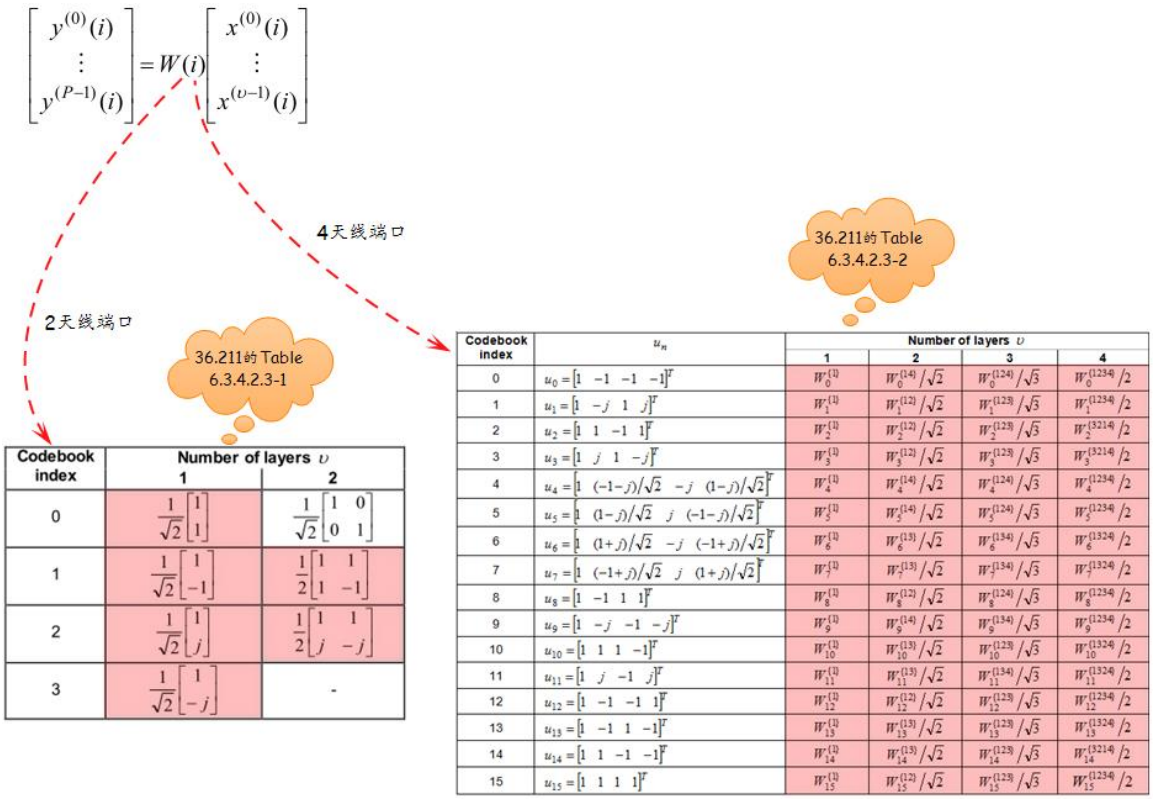
Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 4	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 2	UE specific by C-RNTI	Closed-loop spatial multiplexing (see subclause 7.1.4)or Transmit diversity (see subclause 7.1.2)

TM 4 是闭环空分复用（Closed Loop Spatial Multiplexing，CLSM）。与 TM 3 不同，TM 4 要求 UE 给 eNodeB 反馈 PMI 信息。这种反馈机制可以使得发送侧（eNodeB）先对数据进行预编码，以便优化无线信道的传输，并使得接收侧（UE）能够更加容易地将初始数据流区分开。TM 4 适用于信道条件较好的场景。

TM 4 要求 UE 反馈 CQI、PMI、RI。与 TM 3 类似，eNodeB 中的调度器会根据上报的 RI 和 CQI 等来动态地选择使用发射分集还是空分复用来给该 UE 发送下行数据。

TM 4 支持至多 4 层传输，至多 4 个天线端口（port 0~3）。UE 侧进行信道估计时使用的是小区特定的参考信号。UE 会根据信道条件从预先定义好的码本中选择一个预编码矩阵，并通过 PMI 反馈给 eNodeB。

TM 4 的预编码处理见图 6-9。



Codebook index	Number of layers υ 1	2
0	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$	$\frac{1}{\sqrt{2}}\begin{bmatrix}1&0\\0&1\end{bmatrix}$
1	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\1&-1\end{bmatrix}$
2	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\j\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\j&-j\end{bmatrix}$
3	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-j\end{bmatrix}$	-

Codebook index	u_n	Number of layers υ 1	2	3	4
0	$u_0 = [1\ -1\ -1\ -1]^T$	$W_0^{\{1\}}$	$W_0^{\{14\}}/\sqrt{2}$	$W_0^{\{124\}}/\sqrt{3}$	$W_0^{\{1234\}}/2$
1	$u_1 = [1\ -j\ 1\ j]^T$	$W_1^{\{1\}}$	$W_1^{\{12\}}/\sqrt{2}$	$W_1^{\{123\}}/\sqrt{3}$	$W_1^{\{1234\}}/2$
2	$u_2 = [1\ 1\ -1\ 1]^T$	$W_2^{\{1\}}$	$W_2^{\{12\}}/\sqrt{2}$	$W_2^{\{123\}}/\sqrt{3}$	$W_2^{\{3214\}}/2$
3	$u_3 = [1\ j\ 1\ -j]^T$	$W_3^{\{1\}}$	$W_3^{\{12\}}/\sqrt{2}$	$W_3^{\{123\}}/\sqrt{3}$	$W_3^{\{3214\}}/2$
4	$u_4 = [1\ (-1-j)/\sqrt{2}\ -j\ (1-j)/\sqrt{2}]^T$	$W_4^{\{1\}}$	$W_4^{\{14\}}/\sqrt{2}$	$W_4^{\{124\}}/\sqrt{3}$	$W_4^{\{1234\}}/2$
5	$u_5 = [1\ (1-j)/\sqrt{2}\ j\ (-1-j)/\sqrt{2}]^T$	$W_5^{\{1\}}$	$W_5^{\{14\}}/\sqrt{2}$	$W_5^{\{124\}}/\sqrt{3}$	$W_5^{\{1234\}}/2$
6	$u_6 = [1\ (1+j)/\sqrt{2}\ -j\ (-1+j)/\sqrt{2}]^T$	$W_6^{\{1\}}$	$W_6^{\{13\}}/\sqrt{2}$	$W_6^{\{134\}}/\sqrt{3}$	$W_6^{\{1324\}}/2$
7	$u_7 = [1\ (-1+j)/\sqrt{2}\ j\ (1+j)/\sqrt{2}]^T$	$W_7^{\{1\}}$	$W_7^{\{13\}}/\sqrt{2}$	$W_7^{\{134\}}/\sqrt{3}$	$W_7^{\{1324\}}/2$
8	$u_8 = [1\ -1\ 1\ 1]^T$	$W_8^{\{1\}}$	$W_8^{\{12\}}/\sqrt{2}$	$W_8^{\{124\}}/\sqrt{3}$	$W_8^{\{1234\}}/2$
9	$u_9 = [1\ -j\ -1\ -j]^T$	$W_9^{\{1\}}$	$W_9^{\{14\}}/\sqrt{2}$	$W_9^{\{134\}}/\sqrt{3}$	$W_9^{\{1234\}}/2$
10	$u_{10} = [1\ 1\ 1\ -1]^T$	$W_{10}^{\{1\}}$	$W_{10}^{\{13\}}/\sqrt{2}$	$W_{10}^{\{123\}}/\sqrt{3}$	$W_{10}^{\{1324\}}/2$
11	$u_{11} = [1\ j\ -1\ j]^T$	$W_{11}^{\{1\}}$	$W_{11}^{\{13\}}/\sqrt{2}$	$W_{11}^{\{134\}}/\sqrt{3}$	$W_{11}^{\{1324\}}/2$
12	$u_{12} = [1\ -1\ -1\ 1]^T$	$W_{12}^{\{1\}}$	$W_{12}^{\{12\}}/\sqrt{2}$	$W_{12}^{\{123\}}/\sqrt{3}$	$W_{12}^{\{1234\}}/2$
13	$u_{13} = [1\ -1\ 1\ -1]^T$	$W_{13}^{\{1\}}$	$W_{13}^{\{13\}}/\sqrt{2}$	$W_{13}^{\{123\}}/\sqrt{3}$	$W_{13}^{\{1324\}}/2$
14	$u_{14} = [1\ 1\ -1\ -1]^T$	$W_{14}^{\{1\}}$	$W_{14}^{\{13\}}/\sqrt{2}$	$W_{14}^{\{123\}}/\sqrt{3}$	$W_{14}^{\{3214\}}/2$
15	$u_{15} = [1\ 1\ 1\ 1]^T$	$W_{15}^{\{1\}}$	$W_{15}^{\{12\}}/\sqrt{2}$	$W_{15}^{\{123\}}/\sqrt{3}$	$W_{15}^{\{1234\}}/2$

图 6-9：TM 4 的预编码（precoding）处理

注：对于闭环的空分复用，当使用 2 天线端口且层数为 2 时，不能使用 36.211 的 Table 6.3.4.2.3-1 中的 codebook index 0。

eNodeB 是通过 DCI format 2 的 Precoding information 字段将当前 PDSCH 传输所使用的预编码矩阵告诉 UE 的。（见 36.212 的 5.3.3.1.5 节）

Precoding information 字段的解释与天线端口数和使能的 codeword 数相关。使用 2 天线端口时，对应 36.212 的 Table 5.3.3.1.5-4；使用 4 天线端口时，对应 36.212 的 Table 5.3.3.1.5-5。这些表中的 TPMI 对应 36.211 中的 Table 6.3.4.2.3-1 或 Table 6.3.4.2.3-2 中的 codebook index。

DCI format 2 中的 precoding information 字段包含的比特数如 36.212 的 Table 5.3.3.1.5-3 所示。

Table 5.3.3.1.5-3: Number of bits for precoding information.

Number of antenna ports at eNodeB	Number of bits for precoding information
2	3
4	6

如果只使能了一个 TB，则 36.212 的 Table 5.3.3.1.5-5 中的索引 18~34 只用于支持对应 TB 的重传，且该 TB 在前一次传输中使用的是 2 层的闭环空分复用。也即前一次传输中的 2 个 TB 中只有 1 个需要重传的情况。

对于 UE 上报的 PMI 建议，eNodeB 可以有 2 种选择：（1）eNodeB 采用 UE 最近一次上报的 PMI 建议来进行预编码。此时 eNodeB 在子帧 n 上发送的 PDSCH 使用的预编码，是根据子帧 n - 4 或 n - 4 之前上报的非周期性 CSI 中的最近一个 PMI 来选择的。并且对于非周期性 CSI 上报模式 2-2：如果被调度的 RB 属于上报的最优 M 个子带，则其预编码使用最近一个非周期性 CSI 上报中指示的最优 M 个子带的 PMI；否则其预编码使用最新一个非周期性 CSI 上报中指示的宽带 PMI。（关于模式 2-2，可参见第 10 章关于“CSI”的介绍）；（2）eNodeB 重新选择一个不同的预编码矩阵。

无论 eNodeB 如何选择，UE 都可以通过 Precoding information 字段查 36.212 中的 Table 5.3.3.1.5-4 或 Table 5.3.3.1.5-5 得到最终的结果。

TM 4 使用闭环预编码，更多内容可参见 6.3.1 节。

3GPP 协议中关于 TM 4 的处理见表 6-10。

表 6-10：TM 4，码字/层/预编码处理

传输模式 (Transmission Mode)	码字数 (No of Codewords)	层数 (No of Layers)	预编码 (Precoding)	码本 (Codebook)	天线端口 (Antenna Ports)	说明
TM 4	1	**2 或 4（rank=1）**(36.211 的 6.3.3.3 节)	36.211 的 6.3.4.3 节	无	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	传输分集
	1 或 2	**1~4**(36.211 的 6.3.3.2 节)	36.211 的 6.3.4.2.1 节	36.211 的 Table 6.3.4.2.3-1 和 Table 6.3.4.2.3-2	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	闭环空分复用

注：使用闭环空分复用时，也可能只有一个 codeword，此时另一个 codeword 是去使能的。具体使用传输分集还是闭环空分复用需要根据使用的 DCI format 来确定。

与开环的 TM 3 相比，闭环的 TM 4 需要利用信道条件信息来做传输决定。但 eNodeB 接收到的信道信息存在一定的延迟，过时的信道信息对调度来说是没有用的，因此在 UE 高速移动的情况下，由于信道条件变化很快，此时要谨慎使用 TM 4。同时 UE 上报的信道信息可能有限且并不精确（如 TM 4 下，UE 可能只上报一个 wideband PMI），所以此时 TM 4 的性能可能会有一些损失。

【参考资料】

[1] 《Choosing between open- and closed-loop MIMO in BTS systems》

[2] 《Performance Analysis of Closed and Open Loop MIMO in LTE》by Carsten F. Ball, Robert Mullner, Johann Lienhart and Hubert Winkler

6.5.5 TM 5

表 6-11：TM 5，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 5	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 1D	UE specific by C-RNTI	Multi-user MIMO (see subclause 7.1.5)

在 Rel-8 中，定义了 TM 5 用于支持 MU-MIMO。TM 5 是 TM 4 的 MU-MIMO 版本，同样使用基于码本的闭环预编码。但 TM 5 只使用单层（rank=1）传输，实现了“闭环的基于码本的波束赋形”。

TM 5 只使用单层（rank=1）传输意味着 UE 无需反馈 RI，但依然需要反馈 CQI 和 PMI。

TM 4 和 TM 5 之间的主要区别在于：在 TM 5 中，PDSCH 与小区特定的参考信号之间多了个额外的功率偏移 $\delta_{\text{power-offset}}$ dB。如 36.213 的 Table 7.1.5-1 所示。

Table 7.1.5-1: Mapping of downlink power offset field in DCI format 1D to the $\delta_{\text{power-offset}}$ value.

Downlink power offset field	$\delta_{\text{power-offset}}$ [dB]
0	$-10\log_{10}(2)$
1	0

通常，依赖于小区特定的参考信号进行信道估计的传输模式（TM 1~TM 6）会使用参考信号作为相位参考，同时参考信号还会作为功率/幅度参考来解调使用高阶调制（16QAM 和 64QAM）的信号传输。因此，为了正确地解调高阶调制，UE 需要知道小区特定的参考信号和 PDSCH 之间的功率偏移。（见 36.213 的 5.2 节）

eNodeB 会通过上层信令通知 UE 这个功率偏移以及小区特定的参考信号的 EPRE（energy per resource element），但是该偏移是小区特定的参考信号的功率与包含所有层的整个 PDSCH 功率之间的偏移。对于空分复用，整个 PDSCH 功率还需要在不同层间划分。

在纯粹的空分复用中（即无 MU-MIMO，对应 TM 3 和 TM 4），UE 知道使用的层数，因此间接地知道了小区特定的参考信号的功率与每层 PDSCH 功率之间的偏移。

但在 MU-MIMO 中，可用的总功率会在不同 UE 的下行传输中进行划分，每个传输只能使用较少的 PDSCH 功率。但是 UE 并不知道是否存在其它 UE 的并行传输，即不知道是否存在其它 UE 与其进行 MU-MIMO 配对，因此并不知道每 PDSCH 功率是否降低。为了告诉 UE 是否存在 PDSCH 功率的降低，所以在 TM 5 中，eNodeB 会通过 DCI format 1D 中的“Downlink power offset”字段，将 PDSCH 和小区特定的参考信号之间的额外功率偏移 $\delta_{\text{power-offset}}$ dB 信息，通知给配置了 TM 5 的 UE。如果 $\delta_{\text{power-offset}}$ 值为 0，说明不存在 MU-MIMO，对应的时频资源只分配给一个 UE；如果 $\delta_{\text{power-offset}}$ 值为 $-10\log_{10}(2)$ dB（约为-3 dB），说明存在其它 UE 一起进行 MU-MIMO。

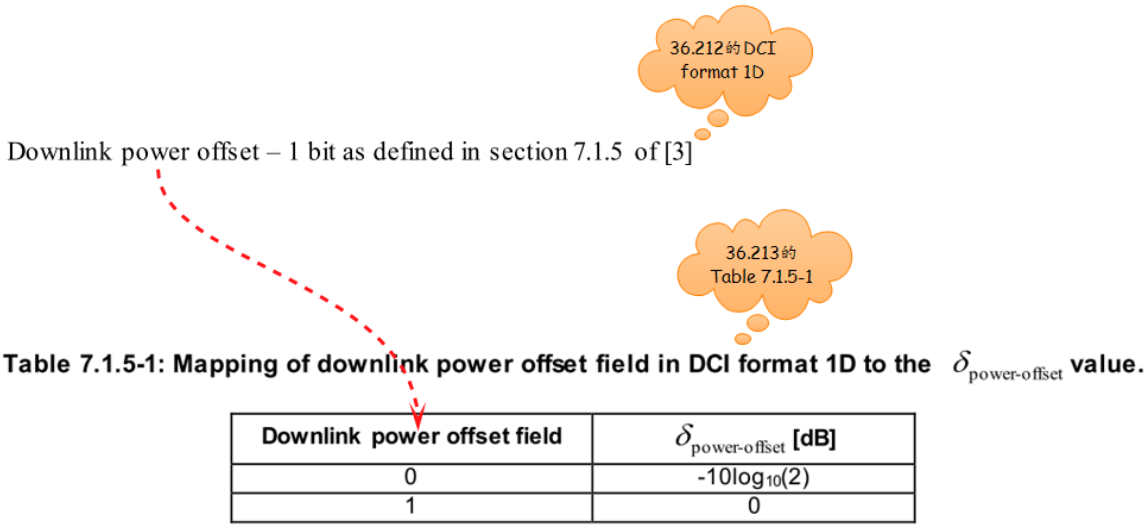


图 6-10：DCI format 1D 中的 Downlink power offset 字段

TM 5 只支持 rank = 1 的传输，并且只支持至多 2 个 UE 并行传输，即至多 2 个 UE 进行 MU-MIMO 配对，这是因为 TM 5 只定义了一个-10log$_{10}$(2) dB 的偏移。

对于某个 UE 而言，TM 5 只使用单层传输，并且一个 TTI 只支持一个 TB 传输，所以调制后的符号直接映射到层上，该层所包含的符号数等于调制后的符号数。（见 36.211 的 6.3.3.1 节）

TM 5 的预编码处理如图 6-11 所示。

$$\begin{bmatrix} y^{(0)}(i) \\ \vdots \\ y^{(P-1)}(i) \end{bmatrix} = W(i) \begin{bmatrix} x^{(0)}(i) \\ \vdots \\ x^{(\upsilon-1)}(i) \end{bmatrix}$$

UE如何确定使用的预编码矩阵

Table 5.3.3.1.4A-1: Number of bits for TPMI information.

Number of antenna ports at eNodeB	Number of bits
2	2
4	4

36.212的DCI format 1D

- TPMI information for precoding – number of bits as specified in Table 5.3.3.1.4A-1

TPMI information indicates which codebook index is used in Table 6.3.4.2.3-1 or Table 6.3.4.2.3-2 of [2] corresponding to the single-layer transmission.

36.211的Table 6.3.4.2.3-1

Codebook index	Number of layers υ 1	2
0	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$	$\frac{1}{\sqrt{2}}\begin{bmatrix}1&0\\0&1\end{bmatrix}$
1	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\1&-1\end{bmatrix}$
2	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\j\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\j&-j\end{bmatrix}$
3	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-j\end{bmatrix}$	-

36.211的Table 6.3.4.2.3-2

Codebook index	u_n	1	Number o 2
0	$u_0 = [1 \ -1 \ -1 \ -1]^T$	$W_0^{\{1\}}$	$W_0^{\{14\}}/\sqrt{2}$
1	$u_1 = [1 \ -j \ 1 \ j]^T$	$W_1^{\{1\}}$	$W_1^{\{12\}}/\sqrt{2}$
2	$u_2 = [1 \ 1 \ -1 \ 1]^T$	$W_2^{\{1\}}$	$W_2^{\{12\}}/\sqrt{2}$
3	$u_3 = [1 \ j \ 1 \ -j]^T$	$W_3^{\{1\}}$	$W_3^{\{12\}}/\sqrt{2}$
4	$u_4 = [1 \ (-1-j)/\sqrt{2} \ -j \ (1-j)/\sqrt{2}]^T$	$W_4^{\{1\}}$	$W_4^{\{14\}}/\sqrt{2}$
5	$u_5 = [1 \ (1-j)/\sqrt{2} \ j \ (-1-j)/\sqrt{2}]^T$	$W_5^{\{1\}}$	$W_5^{\{14\}}/\sqrt{2}$
6	$u_6 = [1 \ (1+j)/\sqrt{2} \ -j \ (-1+j)/\sqrt{2}]^T$	$W_6^{\{1\}}$	$W_6^{\{13\}}/\sqrt{2}$
7	$u_7 = [1 \ (-1+j)/\sqrt{2} \ j \ (1+j)/\sqrt{2}]^T$	$W_7^{\{1\}}$	$W_7^{\{13\}}/\sqrt{2}$
8	$u_8 = [1 \ -1 \ 1 \ 1]^T$	$W_8^{\{1\}}$	$W_8^{\{12\}}/\sqrt{2}$
9	$u_9 = [1 \ -j \ -1 \ -j]^T$	$W_9^{\{1\}}$	$W_9^{\{14\}}/\sqrt{2}$
10	$u_{10} = [1 \ 1 \ 1 \ -1]^T$	$W_{10}^{\{1\}}$	$W_{10}^{\{13\}}/\sqrt{2}$
11	$u_{11} = [1 \ j \ -1 \ j]^T$	$W_{11}^{\{1\}}$	$W_{11}^{\{13\}}/\sqrt{2}$
12	$u_{12} = [1 \ -1 \ -1 \ 1]^T$	$W_{12}^{\{1\}}$	$W_{12}^{\{12\}}/\sqrt{2}$
13	$u_{13} = [1 \ -1 \ 1 \ -1]^T$	$W_{13}^{\{1\}}$	$W_{13}^{\{13\}}/\sqrt{2}$
14	$u_{14} = [1 \ 1 \ -1 \ -1]^T$	$W_{14}^{\{1\}}$	$W_{14}^{\{13\}}/\sqrt{2}$
15	$u_{15} = [1 \ 1 \ 1 \ 1]^T$	$W_{15}^{\{1\}}$	$W_{15}^{\{12\}}/\sqrt{2}$

图 6-11：TM 5 的预编码处理

TM 5 使用基于码本的预编码，eNodeB 需要通过 DCI format 1D 的 TPMI information for precoding 字段告诉 UE 所使用的预编码矩阵对应的 codebook index。

3GPP 协议中关于 TM5 的处理见表 6-12。

表 6-12：TM 5，码字/层/预编码处理

传输模式 (Transmission Mode)	码字数 (No of Codewords)	层数 (No of Layers)	预编码 (Precoding)	码本 (Codebook)	天线端口 (Antenna Ports)	说明
TM 5	1	**2 或 4（rank=1）** (36.211 的 6.3.3.3 节)	36.211 的 6.3.4.3 节	无	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	传输分集
	1	**1** (36.211 的 6.3.3.1 节)	36.211 的 6.3.4.2.1 节	36.211 的 Table 6.3.4.2.3-1 和 Table 6.3.4.2.3-2	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	MU-MIMO

由于进行 MU-MIMO 的 2 个 UE 使用相同的时频资源，因此彼此之间存在着干扰。eNodeB 可以通过为不同 UE 选择合适的预编码矩阵来进行干扰控制。但从图 6-11 可以看出，TM 5 的预编码是基于非常有限的预编码矩阵集合进行的，因此只有在小区内的用户数较大时，才能实现有效地预编码（这样容易找到彼此间干扰较小的配对 UE）。

实际上，将单层数据映射到多个天线端口会产生“波束赋形”的效果。由于进行 MU-MIMO 的 2 个 UE 使用不同的预编码矩阵，所以形成的波束也不同。这会在介绍 TM 6 时予以说明。

eNodeB 在同时调度 2 个 UE 进行 MU-MIMO 时，需要保证这 2 个 UE 使用不同且彼此正交的预编码矩阵，这样才能将 2 个 UE 区分开。

在 Rel-8 中，SU-MIMO（对应 TM 3 和 TM 4）和 MU-MIMO（对应 TM 5）无法进行动态切换，而必须通过 RRC 消息显式地修改 TM 模式来实现（时延相对于动态切换来说要高得多）。事实上，Rel-8 中定义的 TM 5 只在某些特定的场景下才会使用，如在高负载的且带高相关性的 ULA（Uniform Linear Array）天线的小区内使用。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.3.4 节

[2] 《INTERFERENCE-AWARE RECEIVER DESIGN FOR MU-MIMO IN LTE: REAL-TIME PERFORMANCE MEASUREMENTS》，by Sebastian Wagner & Florian Kaltenberger

6.5.6 TM 6

表 6-13：TM 6，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 6	DCI format 1A	Common and UE specific by C-RNTI	Transmit diversity (see subclause 7.1.2)
	DCI format 1B	UE specific by C-RNTI	Closed-loop spatial multiplexing (see subclause 7.1.4) using a single transmission layer

TM 6 是使用单层（rank =1）的闭环预编码，用于支持基于码本的波束赋形。

TM 6 使用小区特定的参考信号来进行信道估计。

TM 6 是闭环空分复用（TM 4）的一个特例，与 TM 4 相比，TM 6 只能使用 1 层（即 rank = 1）进行传输。与使用传输分集的 TM 2 相比，TM 6 可以提高 1~2 dB 的增益。

TM 6 只使用单层（rank=1）传输意味着 UE 无需反馈 RI，但依然需要反馈 CQI 和 PMI。

额外添加一个单层传输的 TM 模式，而不是使用单层场景下的 TM 4 的原因在于：通过限制只能使用单层传输，上下行的信令开销会减少（通过比较 DCI format 1B 和 DCI format 2 可以看出下行信令开销的的降低，同时无需反馈 RI）。TM 6 可以配置给由于 SINR 低而导致无法使用多层传输的 UE。

TM 6 主要用于提高小区边缘的 UE 的速率，因此相关的 DCI 较小不会导致覆盖范围的降低。

TM 6 中，UE 反馈的 PMI 所选择的预编码所形成的波束赋形并不知道多 UE 的小区间干扰，这与 TM 7/TM 8 中基于 Angle of Arrival 的经典波束赋形是不同的。

对于某个 UE 而言，TM 6 只使用单层传输，并且一个 TTI 只支持一个 TB 传输，所以调制后的符号直接映射到层上，该层所包含的符号数等于调制后的符号数。（见 36.211 的 6.3.3.1 节）

TM 6 的预编码处理如图 6-12、图 6-13 所示。

- PMI confirmation for precoding – 1 bit as specified in Table 5.3.3.1.3A-2

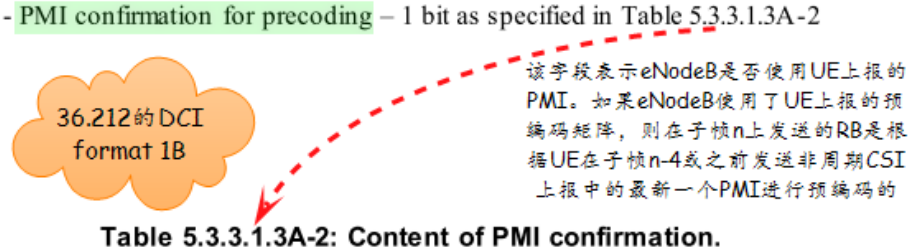


Table 5.3.3.1.3A-2: Content of PMI confirmation.

Bit field mapped to index	Message
0	Precoding according to the indicated TPMI in the TPMI information field
1	Precoding using the precoder(s) according to PMI(s) indicated in the latest aperiodic CSI report. For aperiodic CSI mode 2-2: - Precoding of scheduled resource blocks belonging to the reported preferred M subband(s), use precoder(s) according to the preferred M subband PMI(s) indicated in the latest aperiodic CSI report; - Precoding of scheduled resource blocks not belonging to the reported preferred M subband(s), precoding using a precoder according to the wideband PMI indicated in the latest aperiodic CSI report.

图 6-12：PMI confirmation for precoding 字段含义

$$\begin{bmatrix} y^{(0)}(i) \\ \vdots \\ y^{(P-1)}(i) \end{bmatrix} = W(i) \begin{bmatrix} x^{(0)}(i) \\ \vdots \\ x^{(\upsilon-1)}(i) \end{bmatrix}$$

UE如何确定使用的预编码矩阵

Table 5.3.3.1.3A-1: Number of bits for TPMI information.

Number of antenna ports at eNodeB	Number of bits
2	2
4	4

36.212的DCI format 1B

TPMI information for precoding – number of bits as specified in Table 5.3.3.1.3A-1

TPMI information indicates which codebook index is used in Table 6.3.4.2.3-1 or Table 6.3.4.2.3-2 of [2] corresponding to the single-layer transmission.

36.211的Table 6.3.4.2.3-1

Codebook index	Number of layers υ 1	2
0	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$	$\frac{1}{\sqrt{2}}\begin{bmatrix}1&0\\0&1\end{bmatrix}$
1	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\1&-1\end{bmatrix}$
2	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\j\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\j&-j\end{bmatrix}$
3	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-j\end{bmatrix}$	-

36.211的Table 6.3.4.2.3-2

Codebook index	u_n	1	Number 2
0	$u_0 = [1 \ -1 \ -1 \ -1]^T$	$W_0^{\{1\}}$	$W_0^{\{14\}}/\sqrt{2}$
1	$u_1 = [1 \ -j \ 1 \ j]^T$	$W_1^{\{1\}}$	$W_1^{\{12\}}/\sqrt{2}$
2	$u_2 = [1 \ 1 \ -1 \ 1]^T$	$W_2^{\{1\}}$	$W_2^{\{12\}}/\sqrt{2}$
3	$u_3 = [1 \ j \ 1 \ -j]^T$	$W_3^{\{1\}}$	$W_3^{\{12\}}/\sqrt{2}$
4	$u_4 = [1 \ (-1-j)/\sqrt{2} \ -j \ (1-j)/\sqrt{2}]^T$	$W_4^{\{1\}}$	$W_4^{\{14\}}/\sqrt{2}$
5	$u_5 = [1 \ (1-j)/\sqrt{2} \ j \ (-1-j)/\sqrt{2}]^T$	$W_5^{\{1\}}$	$W_5^{\{14\}}/\sqrt{2}$
6	$u_6 = [1 \ (1+j)/\sqrt{2} \ -j \ (-1+j)/\sqrt{2}]^T$	$W_6^{\{1\}}$	$W_6^{\{13\}}/\sqrt{2}$
7	$u_7 = [1 \ (-1+j)/\sqrt{2} \ j \ (1+j)/\sqrt{2}]^T$	$W_7^{\{1\}}$	$W_7^{\{13\}}/\sqrt{2}$
8	$u_8 = [1 \ -1 \ 1 \ 1]^T$	$W_8^{\{1\}}$	$W_8^{\{12\}}/\sqrt{2}$
9	$u_9 = [1 \ -j \ -1 \ -j]^T$	$W_9^{\{1\}}$	$W_9^{\{14\}}/\sqrt{2}$
10	$u_{10} = [1 \ 1 \ 1 \ -1]^T$	$W_{10}^{\{1\}}$	$W_{10}^{\{13\}}/\sqrt{2}$
11	$u_{11} = [1 \ j \ -1 \ j]^T$	$W_{11}^{\{1\}}$	$W_{11}^{\{13\}}/\sqrt{2}$
12	$u_{12} = [1 \ -1 \ -1 \ 1]^T$	$W_{12}^{\{1\}}$	$W_{12}^{\{12\}}/\sqrt{2}$
13	$u_{13} = [1 \ -1 \ 1 \ -1]^T$	$W_{13}^{\{1\}}$	$W_{13}^{\{13\}}/\sqrt{2}$
14	$u_{14} = [1 \ 1 \ -1 \ -1]^T$	$W_{14}^{\{1\}}$	$W_{14}^{\{13\}}/\sqrt{2}$
15	$u_{15} = [1 \ 1 \ 1 \ 1]^T$	$W_{15}^{\{1\}}$	$W_{15}^{\{12\}}/\sqrt{2}$

图 6-13：TM 6 的预编码处理

从图 6-12、图 6-13 可以看出，TM 6 使用基于码本的预编码，eNodeB 需要通过 DCI format 1B 的 PMI confirmation for precoding 字段和 TPMI information for precoding 字段告诉 UE 所使用的预编码矩阵。

对于 UE 上报的 PMI 建议，eNodeB 可以有 2 种选择：（1）eNodeB 采用 UE 最近一次上报的 PMI 建议来进行预编码。此时 eNodeB 在子帧 n 上发送的 PDSCH 使用的预编码，是根据子帧 n - 4 或 n - 4 之前上报的非周期性 CSI 中的最近一个 PMI 来选择的。并且对于非周期性 CSI 上报模式 2-2：如果被调度的 RB 属于上报的最优 M 个子带，则其预编码使用最近一个非周期性 CSI 上报中指

示的最优 M 个子带的 PMI；否则其预编码使用最新一个非周期性 CSI 上报中指示的宽带 PMI。（关于模式 2-2，可参见第 10 章关于“CSI”的介绍）。此时 eNodeB 会将“PMI confirmation for precoding”字段设置为 1，并且此时 UE 无需再关注“TPMI information for precoding”字段；（2）eNodeB 重新选择一个不同的预编码矩阵。此时 eNodeB 会将“PMI confirmation for precoding”字段设置为 0，并将“TPMI information for precoding”字段设置为所选的预编码矩阵对应的 codebook index（见图 6-13）。

3GPP 协议中关于 TM 6 的处理见表 6-14。

表 6-14：TM 6，码字/层/预编码处理

传输模式 (Transmission Mode)	码字数 (No of Codewords)	层数 (No of Layers)	预编码 (Precoding)	码本 (Codebook)	天线端口 (Antenna Ports)	说明
TM 6	1	**2 或 4（rank=1）** (36.211 的 6.3.3.3 节)	36.211 的 6.3.4.3 节	无	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	传输分集
	1	**1** (36.211 的 6.3.3.1 节)	36.211 的 6.3.4.2.1 节	36.211 的 Table 6.3.4.2.3-1 和 Table 6.3.4.2.3-2	2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	Rank=1 的闭环空分复用

接下来我们会介绍单层的预编码处理所产生的“波束赋形”效果。

以 2 天线端口传输为例，经过预编码处理后的波束赋形效果如图 6-14 所示。这是一种“隐式”的波束赋形效果，这与 TM 7/8/9 中“经典”的波束赋形是不同的（TM 7/8/9 会直接影响天线阵列，并使得波束指向小区的特定区域）。

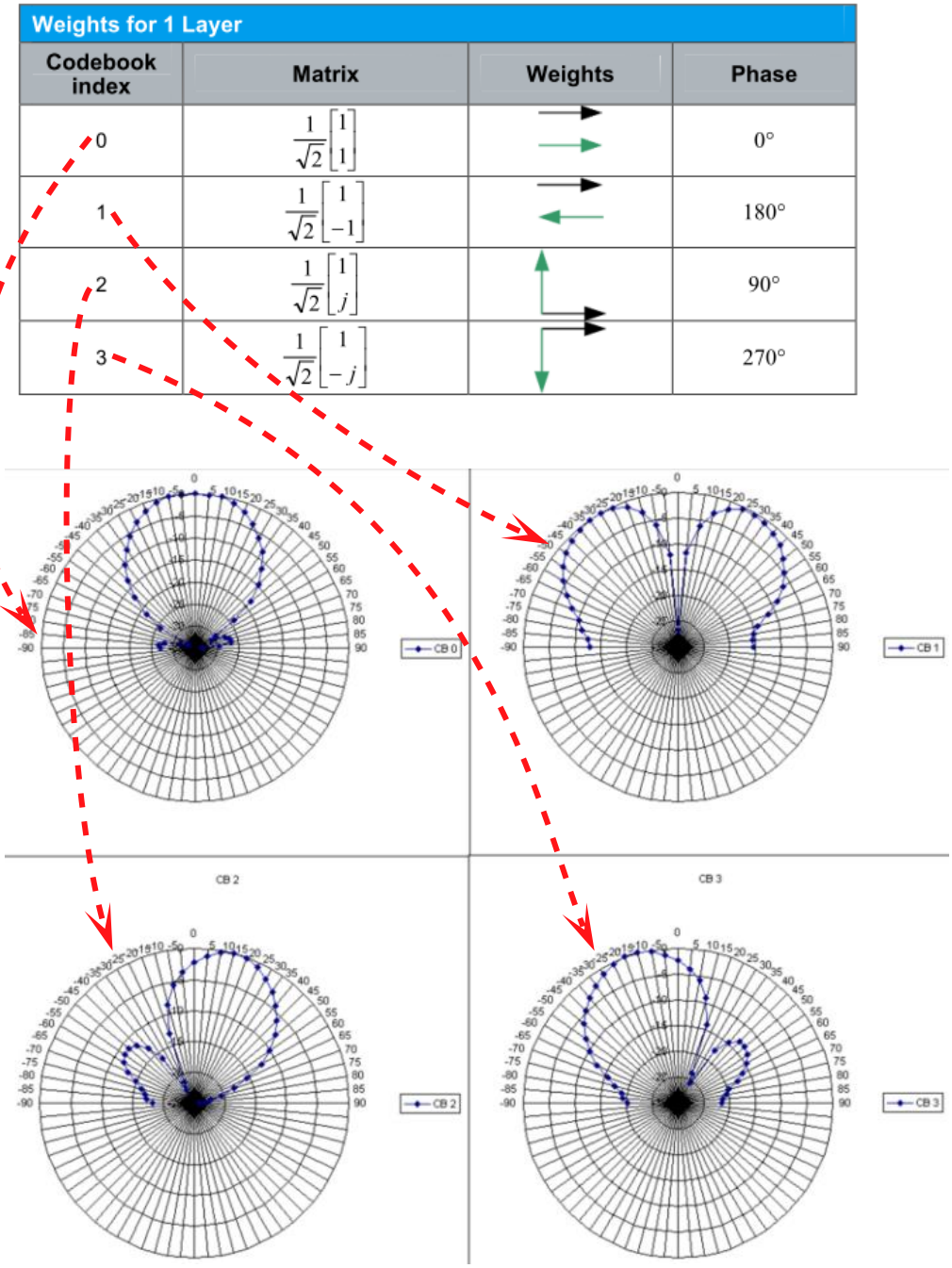


图 6-14：2 天线端口下，TM 6 的隐式波束赋形效果

对于 4 天线端口传输，会产生 16 种（对应 16 个 codebook index）不同的波束赋形效果，这里就不做介绍了。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.3.2.1 节和 10.4.4 节

[2] 《LTE Transmission Modes and Beamforming》by Bernhard Schulz

[3] 《LTE Transmission Modes and Beamforming, White Paper》的 3.2.6 节

6.5.7 TM 7

表 6-15：TM 7，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 7	DCI format 1A	Common and UE specific by C-RNTI	If the number of PBCH antenna ports is one, Single-antenna port, port 0 is used (see subclause 7.1.1), otherwise Transmit diversity (see subclause 7.1.2)
	DCI format 1	UE specific by C-RNTI	Single-antenna port, port 5 (see subclause 7.1.1)

TM 7 是 Rel-8 中引入的"单用户单流波束赋形"。TM 7 对于提高小区平均吞吐量及边缘吞吐量，并降低小区间干扰有着重要作用。但是单流波束赋形仅限于单流传输，其层映射和预编码是一一映射的，并且在 LTE 规范中只定义了波束赋形所需的天线端口 5，以及相应的 CSI 上报机制。

使用波束赋形意味着多根发射天线会将其发射功率集中指向目标 UE 的方向，这就抑制了其它传输带来的干扰，从而提高了 SIR，这对提高边缘 UE 的吞吐量是有益的。如果 eNodeB 无法获知不同传输天线的下行信道信息，更确切地说，是不同传输天线的相对信道相位（relative channel phase）的信息，则多天线传输只能使用发射分集而无法使用波束赋形。（这同样适用于 TM 8/9）

TM 7 是"经典" 的波束赋形，其无法提供任何分集以对抗无线信道衰落，但却能提高接收信号的强度。更多的关于波束赋形的介绍，可参见参考资料[1]。通常来说，波束赋形可以提升的接收端的信号强度，与发射天线的数目成正比。

TM 7 使用基于非码本的预编码，只支持单层（rank = 1）传输。TM 7 使用 UE 特定的参考信号来进行信道估计，并且固定使用天线端口 5。UE 特定的参考信号只在分配给 TM 7 的 UE 的资源块（RB）上传输，UE 根据 UE 特定的参考信号进行信道估计和数据解调。TM 7 中使用的 UE 特定的参考信号见 3.2.1 节的介绍。

UE 特定的参考信号一般用于波束赋形，此时， eNodeB 一般使用一个物理天线阵列来产生定向到某个特定 UE 的波束，这个波束代表一个不同的信道，因此需要根据 UE 特定的参考信号进行信道估计和数据解调。

对于 TM 7，UE 并不上报 PMI 和 RI，但需要上报 CQI。

由于 TM 7 只使用单层传输，一个 TTI 只支持一个 TB 传输，所以调制后的符号直接映射到层上。该层所包含的符号数等于调制后的符号数。（见 36.211 的 6.3.3.1 节）

TM 7 使用基于非码本的预编码，eNodeB 无需告诉 UE 当前的 PDSCH 传输使用的预编码矩阵。又由于 TM 7 固定使用单层传输，eNodeB 也无需告诉 UE 当前的 PDSCH 传输使用的层数。

LTE 规范中并未定义 TM 7 相关的预编码矩阵，更多内容可以参见 6.3.2 节关于“基于非码本的预编码”的介绍。

3GPP 协议中关于 TM 7 的处理见表 6-16。

表 6-16：TM 7，码字/层/预编码处理

传输模式 (Transmission Mode)	**码字数 (No of Codewords)**	**层数 (No of Layers)**	**预编码 (Precoding)**	**码本 (Codebook)**	**天线端口 (Antenna Ports)**	**说明**
TM 7	1	**1 或 2 或 4（rank=1）** (36.211 的 6.3.3.1 节或 6.3.3.3 节)	36.211 的 6.3.4.1 节或 6.3.4.3 节	无	单天线端口：port 0 2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	如果 PBCH 天线端口数为 1，则使用单天线端口 0 并同 TM 1 处理；否则使用传输分集
	1	**1** (36.211 的 6.3.3.1 节)	36.211 的 6.3.4.1 节	无	单天线端口：port 5	单流波束赋形

在 TM 7 中，eNodeB 需要计算最优的波束赋形权值（beamforming weighting），以生成指向某个特定 UE 的波束。eNodeB 通常会根据接收到的上行信号的方向（DoA：Direction of Arrival；或 AoA：Angle of Arrival）以及对上行信号的测量，来计算波束赋形的权值。具体算法的介绍大家可以 google 一下，这里就不做特别的介绍了。

协议中并未规定 eNodeB 使用哪种算法来确定波束赋形的参数，这取决于不同厂商的实现，同时 eNodeB 使用的物理天线数以及天线架构也取决于厂商自己的实现。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 5.4.2 节和 10.3.3 节

[2] 《LTE Transmission Modes and Beamforming, White Paper》的 3.2.7 节

[3] 《LTE 双流波束赋形技术研究》 by 董宏伟，张守霞

[4] http://bbs.cnttr.com/archiver/tid-276198.html，TM 7 如何在同一时刻生成 2 个波束

6.5.8 TM 8

表 6-17：TM 8，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 8	DCI format 1A	Common and UE specific by C-RNTI	If the number of PBCH antenna ports is one, Single-antenna port, port 0 is used (see subclause 7.1.1), otherwise Transmit diversity (see subclause 7.1.2)
	DCI format 2B	UE specific by C-RNTI	Dual layer transmission, port 7 and 8 (see subclause 7.1.5A) or single-antenna port, port 7 or 8 (see subclause 7.1.1)

在 Rel-9 中，新增了 TM 8 以支持“双流波束赋形”。TM 8 实现了波束赋形和空分复用的结合，进一步增大了系统吞吐量。

注：单流和双流的区别主要在于是否使用 MIMO 技术。如果不使用 MIMO 技术，就是单流波束赋形，如果使用 8 天线，则 8 个天线阵元都发射同样的数据形成 1 个波束。如果使用 MIMO 技术，就是双流波束赋形，如果使用 8 天线，就会分成 4+4 两组天线阵元，分别发射两组数据，形成 2 个波束，达到逻辑上的 2*2 MIMO。

TM 8 使用天线端口 7 和/或 8，并使用 UE 特定的参考信号来解调下行数据。此参考信号序列与 TM 7 使用的 UE 特定的参考信号序列是不同的，具体可参见 3.2.2 节的介绍。

从 36.212 的 5.3.3.1.5B 节关于 DCI fomat 2B 的介绍中可以看出 UE 如何确定在 TM 8 中使用的 TB 数/层数/天线端口：

- 如果 2 个 TB 都使能，则层数为 2（rank = 2），TB 1 对应 codeword 0，TB 2 对应 codeword 1。天线端口 7 和 8 用于空分复用。codeword 到 layer 的映射过程对应 36.211 的 Table 6.3.3.2-1 中层数为 2，codeword 数为 2 的行。
- 如果只有 1 个 TB 使能，则层数为 1（rank = 1），TB 到 codeword 的匹配关系见 36.212 的 Table 5.3.3.1.5-2。而使能的 TB 映射到单天线端口的对应关系与去使能的 TB 对应的“New data indicator”字段有关，其关系见 36.212 的 Table 5.3.3.1.5B-1。其 codeword 到 layer 的映射过程见 36.211 的 6.3.3.1 节。

Table 5.3.3.1.5-2: Transport block to codeword mapping (one transport block enabled).

transport block 1	transport block 2	codeword 0 (enabled)	codeword 1 (disabled)
enabled	disabled	transport block 1	-
disabled	enabled	transport block 2	-

Table 5.3.3.1.5B-1: Antenna port for single-antenna port transmission (one transport block disabled).

New data indicator of the disabled transport block	Antenna port
0	7
1	8

注：TM 8 中不存在单层传输映射到 2 个天线端口的情况。1 个 TB 只映射到 1 层，每层只映射到 1 个天线端口。

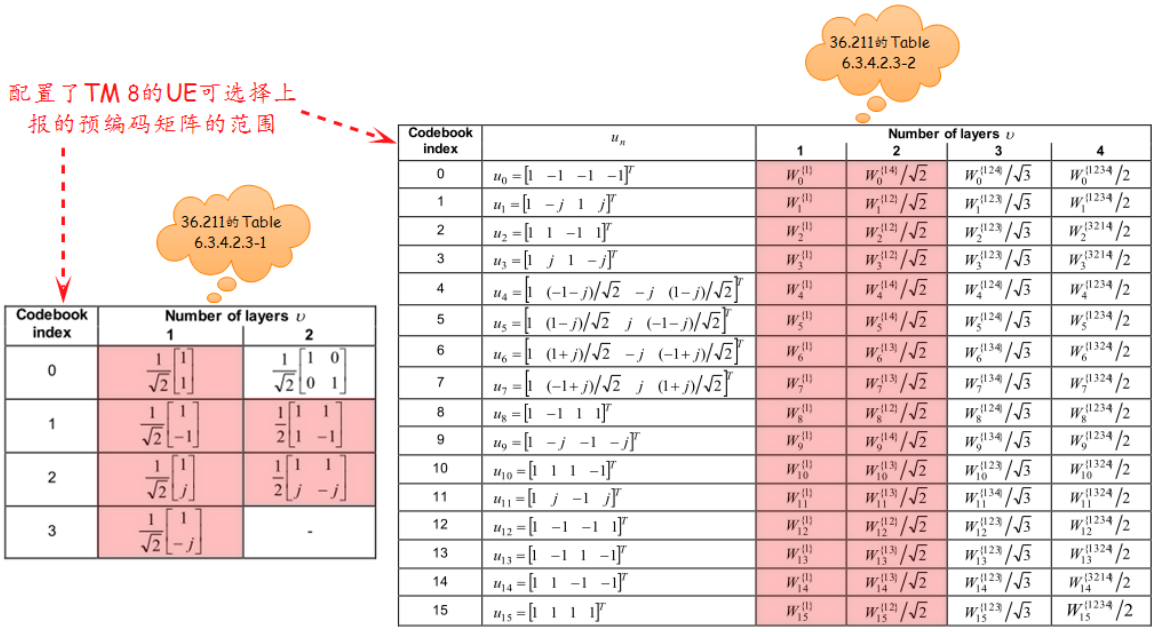
对于配置了除 TM 9 外的其它传输模式的 UE，上报的 CQI 值都是基于对小区特定的参考信号的测量得到的。（见 36.213 的 7.2.3 节）

TM 8 使用基于非码本的预编码，eNodeB 并不会告诉 UE 所使用的预编码矩阵。但配置了 PMI/RI 上报的 UE 还是可以通过上报 PMI/RI 来建议 eNodeB 使用某个预编码矩阵或 RI 配置，但 UE 不知道也没必要知道 eNodeB 会使用哪个预编码矩阵来传输 PDSCH 数据。

对于配置了 TM 8（包括下一节介绍的 TM 9）的 UE，只有当该 UE 配置了 PMI/RI 上报（通过 *pmi-RI-Report* 设置）时，才会上报 PMI。

Non-PMI/RI 反馈主要用于 TDD 系统中，因为 TDD 中，上下行使用相同的频率，得益于信道的互益性，eNodeB 可以从上行传输（主要是 SRS）推断出下行传输的信道信息，这样就能够节约上行的反馈开销（见参考资料[4]）。此时 UE 无需上报 PMI 和 RI，但依然需要上报 CQI。

配置了 TM 8 的 UE 可以上报的 PMI 的范围如图 6-15 所示：



Codebook index	Number of layers υ 1	2
0	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$	$\frac{1}{\sqrt{2}}\begin{bmatrix}1&0\\0&1\end{bmatrix}$
1	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\1&-1\end{bmatrix}$
2	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\j\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1&1\\j&-j\end{bmatrix}$
3	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-j\end{bmatrix}$	-

Codebook index	u_n	Number of layers υ 1	2	3	4
0	$u_0=\begin{bmatrix}1&-1&-1&-1\end{bmatrix}^T$	$W_0^{\{1\}}$	$W_0^{\{14\}}/\sqrt{2}$	$W_0^{\{124\}}/\sqrt{3}$	$W_0^{\{1234\}}/2$
1	$u_1=\begin{bmatrix}1&-j&1&j\end{bmatrix}^T$	$W_1^{\{1\}}$	$W_1^{\{12\}}/\sqrt{2}$	$W_1^{\{123\}}/\sqrt{3}$	$W_1^{\{1234\}}/2$
2	$u_2=\begin{bmatrix}1&1&-1&1\end{bmatrix}^T$	$W_2^{\{1\}}$	$W_2^{\{12\}}/\sqrt{2}$	$W_2^{\{123\}}/\sqrt{3}$	$W_2^{\{3214\}}/2$
3	$u_3=\begin{bmatrix}1&j&1&-j\end{bmatrix}^T$	$W_3^{\{1\}}$	$W_3^{\{12\}}/\sqrt{2}$	$W_3^{\{123\}}/\sqrt{3}$	$W_3^{\{3214\}}/2$
4	$u_4=\begin{bmatrix}1&(-1-j)/\sqrt{2}&-j&(1-j)/\sqrt{2}\end{bmatrix}^T$	$W_4^{\{1\}}$	$W_4^{\{14\}}/\sqrt{2}$	$W_4^{\{124\}}/\sqrt{3}$	$W_4^{\{1234\}}/2$
5	$u_5=\begin{bmatrix}1&(1-j)/\sqrt{2}&j&(-1-j)/\sqrt{2}\end{bmatrix}^T$	$W_5^{\{1\}}$	$W_5^{\{14\}}/\sqrt{2}$	$W_5^{\{124\}}/\sqrt{3}$	$W_5^{\{1234\}}/2$
6	$u_6=\begin{bmatrix}1&(1+j)/\sqrt{2}&-j&(-1+j)/\sqrt{2}\end{bmatrix}^T$	$W_6^{\{1\}}$	$W_6^{\{13\}}/\sqrt{2}$	$W_6^{\{134\}}/\sqrt{3}$	$W_6^{\{1324\}}/2$
7	$u_7=\begin{bmatrix}1&(-1+j)/\sqrt{2}&j&(1+j)/\sqrt{2}\end{bmatrix}^T$	$W_7^{\{1\}}$	$W_7^{\{13\}}/\sqrt{2}$	$W_7^{\{134\}}/\sqrt{3}$	$W_7^{\{1324\}}/2$
8	$u_8=\begin{bmatrix}1&-1&1&1\end{bmatrix}^T$	$W_8^{\{1\}}$	$W_8^{\{12\}}/\sqrt{2}$	$W_8^{\{124\}}/\sqrt{3}$	$W_8^{\{1234\}}/2$
9	$u_9=\begin{bmatrix}1&-j&-1&-j\end{bmatrix}^T$	$W_9^{\{1\}}$	$W_9^{\{14\}}/\sqrt{2}$	$W_9^{\{134\}}/\sqrt{3}$	$W_9^{\{1234\}}/2$
10	$u_{10}=\begin{bmatrix}1&1&1&-1\end{bmatrix}^T$	$W_{10}^{\{1\}}$	$W_{10}^{\{13\}}/\sqrt{2}$	$W_{10}^{\{123\}}/\sqrt{3}$	$W_{10}^{\{1324\}}/2$
11	$u_{11}=\begin{bmatrix}1&j&-1&j\end{bmatrix}^T$	$W_{11}^{\{1\}}$	$W_{11}^{\{13\}}/\sqrt{2}$	$W_{11}^{\{134\}}/\sqrt{3}$	$W_{11}^{\{1324\}}/2$
12	$u_{12}=\begin{bmatrix}1&-1&-1&1\end{bmatrix}^T$	$W_{12}^{\{1\}}$	$W_{12}^{\{12\}}/\sqrt{2}$	$W_{12}^{\{123\}}/\sqrt{3}$	$W_{12}^{\{1234\}}/2$
13	$u_{13}=\begin{bmatrix}1&-1&1&-1\end{bmatrix}^T$	$W_{13}^{\{1\}}$	$W_{13}^{\{13\}}/\sqrt{2}$	$W_{13}^{\{123\}}/\sqrt{3}$	$W_{13}^{\{1324\}}/2$
14	$u_{14}=\begin{bmatrix}1&1&-1&-1\end{bmatrix}^T$	$W_{14}^{\{1\}}$	$W_{14}^{\{13\}}/\sqrt{2}$	$W_{14}^{\{123\}}/\sqrt{3}$	$W_{14}^{\{3214\}}/2$
15	$u_{15}=\begin{bmatrix}1&1&1&1\end{bmatrix}^T$	$W_{15}^{\{1\}}$	$W_{15}^{\{12\}}/\sqrt{2}$	$W_{15}^{\{123\}}/\sqrt{3}$	$W_{15}^{\{1234\}}/2$

图 6-15：配置了 TM 8 的 UE 可选择上报的预编码矩阵的范围

注意：eNodeB 可选用的预编码矩阵并不局限于图 6-15 指定的范围。

从上面的介绍可以看出，TM 8 的预编码和波束赋形对 UE 来说是透明的；而传输使用的层数可以通过是否去使能其中 1 个 TB 来告诉 UE。

根据调度的 UE 情况的不同，双流波束赋形技术可分为“单用户双流波束赋形，使用 SU-MIMO”和“多用户双流波束赋形，使用 MU-MIMO”。

6.5.8.1 SU-MIMO

单用户双流波束赋形，是由 eNodeB 测量上行信道，得到上行信道信息后，再根据上行信道信息计算两个天线赋形权值，利用该赋形权值对要发射的两个数据流进行下行赋形。

单用户双流波束赋形使得单个用户在某一时刻可以进行两个数据流（2 个 TB）传输，同时获得赋形增益和空间复用增益，可以获得比单流波束赋形技术更大的传输速率，进而提高 UE 的峰值速率。此时，根据多天线理论可知，接收天线数不能小于空间复用的数据流数，即接收端（UE）至少需要有 2 根天线。

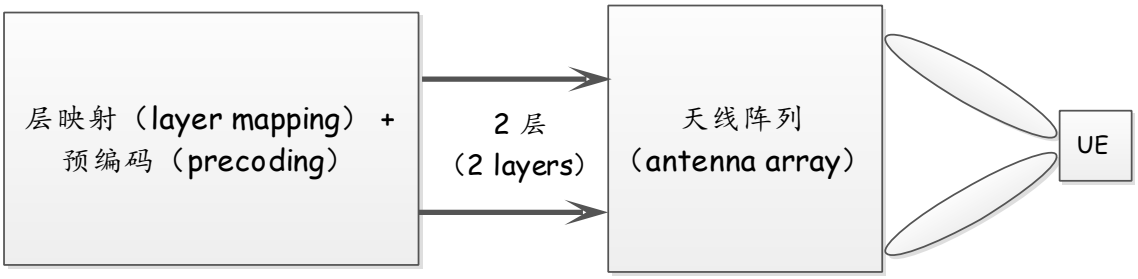


图 6-16：TM 8，使用 SU-MIMO

对于 TM 8 而言，在 SU-MIMO 中，eNodeB 分配给 UE 的频域资源由该 UE 独占。此时 eNodeB 在一个 TTI 内给该 UE 发送 2 个 TB，使用 2 层，并使用天线端口 7 和 8 来发送 PDSCH。在不同天线端口上使用不同的正交覆盖码（Orthogonal Cover Code，OCC）来定义相互正交的 UE 特定的参考信号，从而区分 2 个天线端口（或者说 2 层）上的数据。由于每个天线端口对应的正交覆盖码是固定的（见 36.211 的 Table 6.10.3.2-1），所以 eNodeB 没有必要通过 DCI format 2B 告诉 UE 所使用的正交覆盖码。

Table 6.10.3.2-1: The sequence $\overline{w}_p(i)$ for normal cyclic prefix.

Antenna port p	$\left[\overline{w}_p(0) \quad \overline{w}_p(1) \quad \overline{w}_p(2) \quad \overline{w}_p(3)\right]$
7	$\left[+1 \quad +1 \quad +1 \quad +1\right]$
8	$\left[+1 \quad -1 \quad +1 \quad -1\right]$

6.5.8.2 MU-MIMO

多用户双流波束赋形中，基站根据上行信道信息或者 UE 反馈的信息进行多用户匹配。多 UE 配对完成后，按照一定的准则生成天线赋形权值，利用得到的天线赋形权值为每一个 UE 进行赋形。

多用户双流波束赋形技术，利用了智能天线的波束定向原理和空间信道的不相关性，实现多 UE 的空分多址复用。此时，各 UE 占用相同的时频资源（或部分重叠），但两个 UE 接收不同的数据流。

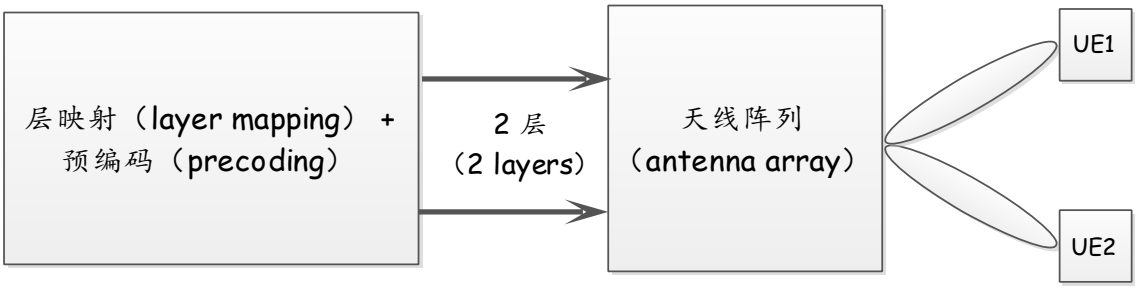


图 6-17：TM 8，使用 MU-MIMO

在多用户双流波束赋形模式下，多 UE 的配对可以是 2 个 rank = 1 的 UE 配对；也可以是 2 个 rank = 2 的 UE 配对；也可以是一个 rank = 1 和一个 rank = 2 的 UE 的配对。

前面已经介绍过，在不同天线端口上使用不同的正交覆盖码（Orthogonal Cover Code，OCC）来定义不同但相互正交的 UE 特定的参考信号，从而区分 2 个天线端口（或者说 2 层）上的数据。并且每个天线端口对应的正交覆盖码是固定的。

如果**配对的是两个 rank = 1 的 UE**（对应每个 UE 只有一个 TB），则可以给每个 UE 分配独占的一层，不同层间通过不同的 OCC 来区分。每个 UE 可以通过“设置去使能的 TB 对应的 New data indicator 字段”来将不同的 UE 分配到不同的层（天线端口）上。

表 6-18：TM 8 下配对的两个 rank = 1 的 UE 的配置

UE	去使能的 TB 对应的 New data indicator 字段	n_{SCID}	天线端口	层数
UE1	0	不关心	7	1
UE2	1	不关心	8	1

如果配对的是**两个 rank = 2 的 UE**，或者**配对的是一个 rank = 1 和一个 rank = 2 的 UE**，那么必然会在某层（或者说某个天线端口）上，2 个 UE 使用相同的 OCC，这时就无法区分这 2 个 UE 的参考信号了。为了解决这个问题，在 DCI format 2B 中新增了一个 1 比特的 Scrambling identity 字段。

从 36.211 的 6.10.3.1 节可以看出，除了 OCC，还可以将一个伪随机序列应用到 TM 8/9 使用的 UE 特定的参考信号序列上。在同一小区的同一子帧上，这个伪随机序列与 n_{SCID} 相关。在 TM 8

中，n_{SCID} 与 DCI format 2B 中 Scrambling identity 字段一一对应。其对应关系见 36.211 的 Table 6.10.3.1-1。

Table 6.10.3.1-1: Mapping of scrambling identity field in DCI format 2B to n_{SCID} values for antenna ports 7 and 8.

Scrambling identity field in DCI format 2B [3]	n_{SCID}
0	0
1	1

因此，在配对的是两个 rank = 2 的 UE，或者配对的是一个 rank = 1 和一个 rank = 2 的 UE 的情况下，可以给不同的 UE 分配不同的 n_{SCID} ，这样不同的 UE 使用不同的参考信号序列，就能够区分这 2 个 UE 的 PDSCH 传输了。

表 6-19：TM 8 下，两个 rank =2 的 UE 配对时的配置

UE	“scrambling identity”字段	n_{SCID}	天线端口	层数
UE1	0	0	7~8	2
UE2	1	1	7~8	2

可以看出，当 TM 8 用于 MU-MIMO 时，所需做的只是给不同 UE 分配不同的 UE 特定的参考信号。

对于 TM 8，eNodeB 无需修改 TM 模式就可在 SU-MIMO 和 MU-MIMO 之间来回切换（只要修改 DCI format 2B 中的相应字段即可）。

表 6-20：TM 8 的 SU-MIMO 和 MU-MIMO 配置

TM 8 的 SU-MIMO 和 MU-MIMO 配置	
SU-MIMO	UE 独占分配给它的资源，不同层间通过 OCC 来区分。eNodeB 在调度时，不将该 UE 独占的资源分配给其它 UE 即可。

MU-MIMO	（1）配对的是两个 rank = 1 的 UE：不同的 UE 使用不同层。eNodeB 在调度时可以将相同的时频资源分配给 2 个 UE，但不同的 UE 使用不同的层（天线端口），这可以通过将这 2 个 UE 去使能的 TB 对应的 New data indicator 字段设置成不同的值来实现。 （2）如果配对的是两个 rank = 2 的 UE，或者配对的是一个 rank = 1 和一个 rank = 2 的 UE：通过不同的 OCC 和 n_{SCID} 来区分。eNodeB 在调度时可以将相同的时频资源分配给 2 个 UE，但对不同的 UE，将对应的 Scrambling identity 字段设置成不同的值。此时同一个 UE 通过 OCC 来区分 2 层传输，而不同的 UE 通过 n_{SCID} 来区分在同一层上的传输。

双流波束赋形技术的出现，不仅能够最大限度地发挥波束赋形在覆盖容量上的优势，而且能够进一步提升系统的频谱效率，适合于各类室外场景（如城区、郊区）的覆盖，最大限度地满足运营商对覆盖性能和频谱效率提升的双重需求，同时有效降低网络内的干扰。

常用的调度方法是，基站同时对 2 个用户发送数据，每用户各占一流（给每个 UE 发送 1 个 TB）。当用户间的空间隔离度很好的情况下，基站侧可以同时对 2 个用户发送数据，每用户均使用双流波束赋形（给每个用户发送 2 个 TB），充分利用空分特性，进一步提高系统的峰值流量。

在低负载的情况下，可配对用户较少，所以可采用 SU-MIMO（单用户 MIMO）使 UE 获得更高数据峰值流量。MU-MIMO（多用户 MIMO）可在高负载时增加同时调度的用户数，从而提高小区峰值流量。

使用双流波束赋形的原因：对于 UE 而言，双流依然是使用 2 个 codeword。使用双流可以使不同的流使用各自独自的编码和调制方式。这样在使用空分复用的情况下，在 UE 上报更好的信道质量的流上，eNodeB 可以使用更高的 MCS；而在信道质量较差的流上，eNodeB 可以使用较低的 MCS（即更可靠的 MCS）。

3GPP 协议中关于 TM 8 的处理见表 6-21。

表 6-21：TM 8，码字/层/预编码处理

传输模式 (Transmission Mode)	码字数 (No of Codewords)	层数 (No of Layers)	预编码 (Precoding)	码本 (Codebook)	天线端口 (Antenna Ports)	说明
TM 8	1	**1 或 2 或 4（rank=1）** (36.211 的 6.3.3.1 节或 6.3.3.3 节)	36.211 的 6.3.4.1 节或 6.3.4.3 节	无	单天线端口：port 0 2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	如果 PBCH 天线端口数为 1，则使用单天线端口 0 并同 TM 1 处理；否则使用传输分集
	1 或 2	1 或 2 (36.211 的 6.3.3.2 节)	36.211 的 6.3.4.4 节	无 （上报 PMI 时的码本见图 6-15）	单天线端口：port 7 或 8 2 天线端口：port 7 和 8	双流波束赋形

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.3.4 节

[2] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 8.2.3 节

[3] 《LTE 双流波束赋形技术研究》 by 董宏伟，张守霞

[4] “R1-110285, Views on Supporting PMI Disabling in TM9” , Research In Motion, UK Limited.

6.5.9 TM 9

表 6-22：TM 9，使用 C-RNTI 加扰

Transmission mode	DCI format	Search Space	Transmission scheme of PDSCH corresponding to PDCCH
Mode 9	DCI format 1A	Common and UE specific by C-RNTI	Non-MBSFN subframe: If the number of PBCH antenna ports is one, Single-antenna port, port 0 is used (see subclause 7.1.1), otherwise Transmit diversity (see subclause 7.1.2) MBSFN subframe: Single-antenna port, port 7 (see subclause 7.1.1)
	DCI format 2C	UE specific by C-RNTI	Up to 8 layer transmission, ports 7-14 (see subclause 7.1.5B) or single-antenna port, port 7 or 8 (see subclause 7.1.1)

在 Rel-10 中，新增了 TM 9 以支持“8 流波束赋形”。

TM 9 设计的目的在于减少基站间的干扰以最大化信号的稳定性以及提供突发的性能要求。TM 9 可以提高频谱利用率（使用高阶 MIMO），提升小区边缘的数据传输速率，增大小区的覆盖以及有更好地干扰管理（使用波束赋形），并且能够在 SU-MIMO 和加强的 MU-MIMO 之间灵活地切换。

对于 TM 9，eNodeB 无需修改 TM 模式就可在 SU-MIMO 和 MU-MIMO 之间来回切换（只要修改 DCI format 2C 中的相应字段即可）。

TM 9 使用了 2 个新的参考信号： CSI-RS 和 DMRS（UE 特定的参考信号）。前者主要用于测量 CSI；而后者是 Rel-9 中新增的 UE 特定的参考信号的加强版，用于信道估计，并支持至多 8 层的传输。

注：CSI-RS 只用于 TM 9 中。

TM 9 使用天线端口 7~14，并使用 UE 特定的参考信号来解调下行数据。此参考信号序列是 TM 8 使用的 UE 特定的参考信号序列的扩展，以支持至多 8 天线端口传输。具体可参见 3.2.2 节。

对于配置了 TM 9 的 UE，只有当该 UE 配置了 PMI/RI 上报（通过 *pmi-RI-Report* 设置）时，才会上报 PMI。

对于配置了 TM9 且同时配置了 PMI/RI 上报的 UE，上报的 CQI 值是基于对 CSI-RS 的测量得到的；对于配置了 TM9 但没有配置 PMI/RI 上报的 UE，上报的 CQI 值是基于对小区特定的参考信号的测量得到的（见 36.213 的 7.2.3 节）。

从 36.213 的 Table 7.2.3-0 可以看出，对于配置了 TM9 但没有配置 PMI/RI 上报的 UE，其 CQI 是在假定使用单天线端口或传输分集的情况下得到的，但此时下行可能使用 8 根传输天线，而规范中并未定义 8 传输天线下的传输分集。此时的解决方法是使用天线虚拟化，具体可参见 Samsung 的一篇提案，见参考资料[4]。

Table 7.2.3-0: PDSCH transmission scheme assumed for CSI reference resource

Transmission mode	Transmission scheme of PDSCH
9	If the UE is configured without PMI/RI reporting: if the number of PBCH antenna ports is one, single-antenna port, port 0; otherwise transmit diversity If the UE is configured with PMI/RI reporting: if the number of CSI-RS ports is one, single-antenna port, port 7; otherwise up to 8 layer transmission, ports 7-14 (see subclause 7.1.5B)

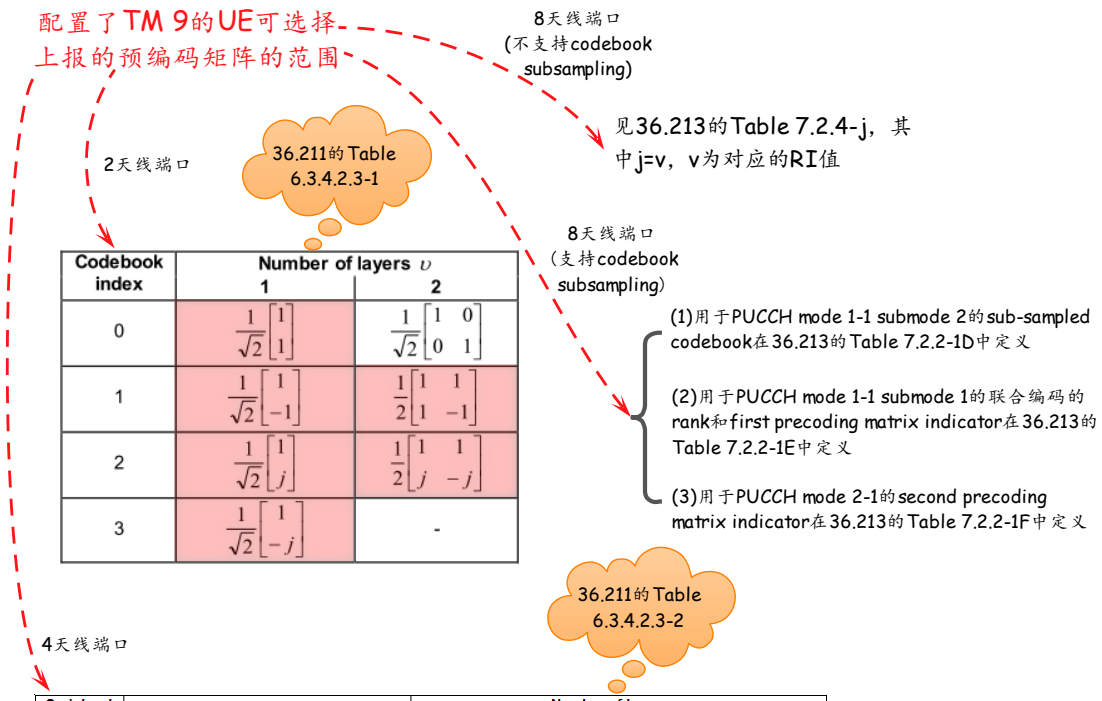
对于配置了 TM 9 的 UE，只有当该 UE 配置了 PMI/RI 上报且 CSI-RI 端口数大于 1 时，才会上报 PMI。

对于 8 CSI-RS 天线端口来说，用于 PMI 反馈的码本结构可以看做是一个 2 阶段的码本。对应的预编码矩阵 W 可以分解成 2 个矩阵的乘积，即$W = i_1 \cdot i_2$。其中i_1反映的是长期（long-term）的宽带（wideband）信道特性，i_2反映的是短期（long-term）的频率选择性信道特性。使用这种上报方式的主要好处是较低的反馈开销以及比单码本更好的性能。

当 UE 需要上报 PMI 时，UE 会反馈一个预编码矩阵以表明 eNodeB 使用的预编码（对 UE 透明的）和 UE 期望使用的预编码之间的偏差。此时，UE 反馈的是 2 个索引：i_1和i_2，而不是实际的 PMI。i_1和i_2的取值范围都是 0~15，这样可以提供更细粒度的反馈，从而提高 MU-MIMO 的性能。eNodeB 可以使用这 2 个索引来重新构造需要的预编码矩阵。用于 PMI 反馈的码本结构见图 6-18。

配置了 TM 9 的 UE，在使用 2 或 4 天线端口时，用于 PMI 反馈的码本结构与 Rel-8/ Rel-9 中定义的码本结构相同，而不使用前面介绍的“2 阶段码本”结构。如图 6-18 所示。

配置了 TM 9 的 UE，在使用 8 天线端口时，用于 PMI 反馈的码本结构见 36.213 的 Table 7.2.4-j，其中 j = v，且 v 等于对应的 RI 值。如果 UE 还支持 codebook subsampling（码本筛选，参见 10.5.5 节的介绍）时，还会有不同的处理。如图 6-18 所示。



Codebook index	Number of layers υ 1	2
0	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$	$\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}$
1	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$
2	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\j\end{bmatrix}$	$\frac{1}{2}\begin{bmatrix}1 & 1\\j & -j\end{bmatrix}$
3	$\frac{1}{\sqrt{2}}\begin{bmatrix}1\\-j\end{bmatrix}$	-

Codebook index	u_n	Number of layers υ 1	2	3	4
0	$u_0=[1\ -1\ -1\ -1]^T$	$W_0^{\{1\}}$	$W_0^{\{14\}}/\sqrt{2}$	$W_0^{\{124\}}/\sqrt{3}$	$W_0^{\{1234\}}/2$
1	$u_1=[1\ -j\ 1\ j]^T$	$W_1^{\{1\}}$	$W_1^{\{12\}}/\sqrt{2}$	$W_1^{\{123\}}/\sqrt{3}$	$W_1^{\{1234\}}/2$
2	$u_2=[1\ 1\ -1\ 1]^T$	$W_2^{\{1\}}$	$W_2^{\{12\}}/\sqrt{2}$	$W_2^{\{123\}}/\sqrt{3}$	$W_2^{\{3214\}}/2$
3	$u_3=[1\ j\ 1\ -j]^T$	$W_3^{\{1\}}$	$W_3^{\{12\}}/\sqrt{2}$	$W_3^{\{123\}}/\sqrt{3}$	$W_3^{\{3214\}}/2$
4	$u_4=[1\ (-1-j)/\sqrt{2}\ -j\ (1-j)/\sqrt{2}]^T$	$W_4^{\{1\}}$	$W_4^{\{14\}}/\sqrt{2}$	$W_4^{\{124\}}/\sqrt{3}$	$W_4^{\{1234\}}/2$
5	$u_5=[1\ (1-j)/\sqrt{2}\ j\ (-1-j)/\sqrt{2}]^T$	$W_5^{\{1\}}$	$W_5^{\{14\}}/\sqrt{2}$	$W_5^{\{124\}}/\sqrt{3}$	$W_5^{\{1234\}}/2$
6	$u_6=[1\ (1+j)/\sqrt{2}\ -j\ (-1+j)/\sqrt{2}]^T$	$W_6^{\{1\}}$	$W_6^{\{13\}}/\sqrt{2}$	$W_6^{\{134\}}/\sqrt{3}$	$W_6^{\{1324\}}/2$
7	$u_7=[1\ (-1+j)/\sqrt{2}\ j\ (1+j)/\sqrt{2}]^T$	$W_7^{\{1\}}$	$W_7^{\{13\}}/\sqrt{2}$	$W_7^{\{134\}}/\sqrt{3}$	$W_7^{\{1324\}}/2$
8	$u_8=[1\ -1\ 1\ 1]^T$	$W_8^{\{1\}}$	$W_8^{\{12\}}/\sqrt{2}$	$W_8^{\{124\}}/\sqrt{3}$	$W_8^{\{1234\}}/2$
9	$u_9=[1\ -j\ -1\ -j]^T$	$W_9^{\{1\}}$	$W_9^{\{14\}}/\sqrt{2}$	$W_9^{\{134\}}/\sqrt{3}$	$W_9^{\{1234\}}/2$
10	$u_{10}=[1\ 1\ 1\ -1]^T$	$W_{10}^{\{1\}}$	$W_{10}^{\{13\}}/\sqrt{2}$	$W_{10}^{\{123\}}/\sqrt{3}$	$W_{10}^{\{1324\}}/2$
11	$u_{11}=[1\ j\ -1\ j]^T$	$W_{11}^{\{1\}}$	$W_{11}^{\{13\}}/\sqrt{2}$	$W_{11}^{\{134\}}/\sqrt{3}$	$W_{11}^{\{1324\}}/2$
12	$u_{12}=[1\ -1\ -1\ 1]^T$	$W_{12}^{\{1\}}$	$W_{12}^{\{12\}}/\sqrt{2}$	$W_{12}^{\{123\}}/\sqrt{3}$	$W_{12}^{\{1234\}}/2$
13	$u_{13}=[1\ -1\ 1\ -1]^T$	$W_{13}^{\{1\}}$	$W_{13}^{\{13\}}/\sqrt{2}$	$W_{13}^{\{123\}}/\sqrt{3}$	$W_{13}^{\{1324\}}/2$
14	$u_{14}=[1\ 1\ -1\ -1]^T$	$W_{14}^{\{1\}}$	$W_{14}^{\{13\}}/\sqrt{2}$	$W_{14}^{\{123\}}/\sqrt{3}$	$W_{14}^{\{3214\}}/2$
15	$u_{15}=[1\ 1\ 1\ 1]^T$	$W_{15}^{\{1\}}$	$W_{15}^{\{12\}}/\sqrt{2}$	$W_{15}^{\{123\}}/\sqrt{3}$	$W_{15}^{\{1234\}}/2$

注：可参见36.213的7.2.4节

图 6-18：TM 9 中用于 PMI 反馈的预编码矩阵集合

对于 TM 9 而言，eNodeB 会通过 DCI format 2C 告知 UE 下行传输所使用的传输秩，但并不会指定预编码矩阵，因为 TM 9 使用基于非码本的预编码，其预编码矩阵对于 UE 来说是不可见的。

DCI format 2C 有一个字段：Antenna port(s), scrambling identity and number of layers，用于指定当前 PDSCH 传输使用的天线端口，层数以及 n_{SCID}。该字段的含义见 36.212 的 Table 5.3.3.1.5C-1。

Table 5.3.3.1.5C-1: Antenna port(s), scrambling identity and number of layers indication

One Codeword: Codeword 0 enabled, Codeword 1 disabled		Two Codewords: Codeword 0 enabled, Codeword 1 enabled	
Value	**Message**	**Value**	**Message**
0	1 layer, port 7, n_{SCID}=0	0	2 layers, ports 7-8, n_{SCID}=0
1	1 layer, port 7, n_{SCID}=1	1	2 layers, ports 7-8, n_{SCID}=1
2	1 layer, port 8, n_{SCID}=0	2	3 layers, ports 7-9
3	1 layer, port 8, n_{SCID}=1	3	4 layers, ports 7-10
4	2 layers, ports 7-8	4	5 layers, ports 7-11
5	3 layers, ports 7-9	5	6 layers, ports 7-12
6	4 layers, ports 7-10	6	7 layers, ports 7-13
7	Reserved	7	8 layers, ports 7-14

对于 TM 9 而言，如果 2 个 TB 都使能，则 TB 1 映射到 codeword 0，TB 2 映射到 codeword 1。如果其中一个 codeword 去使能，则 TB 到 codeword 的映射见 36.212 的 Table 5.3.3.1.5-2。

如果只使能了一个 TB，则 Table 5.3.3.1.5C-1 中 Value = 4，5，6 的项只用于支持对应 TB 的重传，且该 TB 在前一次传输中对应使用的是 2，3，4 层传输。

TM 9 可以支持至多 8 层的 SU-MIMO 传输和至多 4 层的 MU-MIMO 传输。

6.5.9.1 SU-MIMO

与 TM8 类似的是，TM 9 的单用户 8 流波束赋形，是由 eNodeB 测量上行信道，得到上行信道信息后，再根据上行信道信息计算 8 个天线的赋形权值，利用该赋形权值对要发射的（至多）8 个数据流进行下行赋形。

单用户 8 流波束赋形使得单个用户在某一时刻可以进行（至多）8 个数据流（2 个 TB）传输，同时获得赋形增益和空间复用增益，可以获得比单/双流波束赋形技术更大的传输速率，进而提高 UE 的峰值速率。此时，根据多天线理论可知，接收天线数不能小于空间复用的数据流数，即接收端（UE）至少需要有 8 根天线。

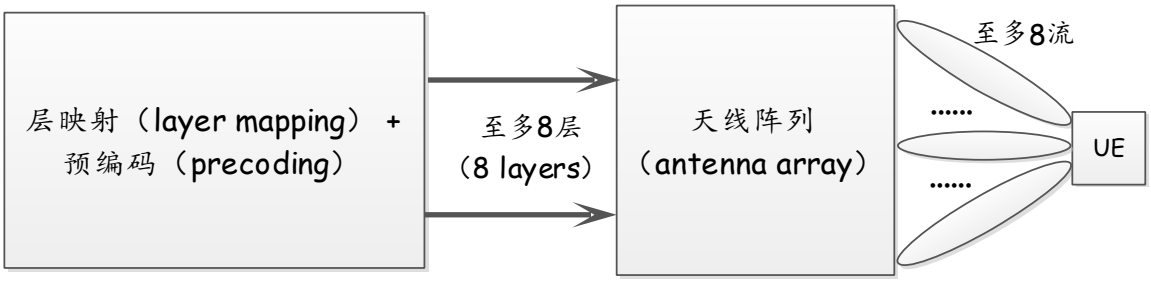


图 6-19：TM 9，使用 SU-MIMO

对于 TM 9 而言，在 SU-MIMO 中，eNodeB 分配给 UE 的频域资源由该 UE 独占。此时 eNodeB 在每个 TTI 上给该 UE 发送 2 个 TB，使用至多 8 层，并使用天线端口 7~14 来发送 PDSCH。在不同天线端口上使用不同的正交覆盖码（Orthogonal Cover Code，OCC）以及不同的 RE 资源来定义的相互正交的 UE 特定的参考信号（具体可参见 3.2.2 节的介绍），从而区分 8 个天线端口（或者说 8 层）上的 PDSCH 数据。由于每个天线端口上的 UE 特定的参考信号对应的正交覆盖码和所占的 RE 是固定的（见 36.211 的 Table 6.10.3.2-1），所以 eNodeB 没有必要通过 DCI format 2C 告诉 UE 其 UE 特定的参考信号所使用的正交覆盖码和所占的 RE。

Table 6.10.3.2-1: The sequence $\overline{w}_p(i)$ for normal cyclic prefix.

Antenna port p	$\left[\overline{w}_p(0) \quad \overline{w}_p(1) \quad \overline{w}_p(2) \quad \overline{w}_p(3)\right]$
7	$[+1 \quad +1 \quad +1 \quad +1]$
8	$[+1 \quad -1 \quad +1 \quad -1]$
9	$[+1 \quad +1 \quad +1 \quad +1]$
10	$[+1 \quad -1 \quad +1 \quad -1]$
11	$[+1 \quad +1 \quad -1 \quad -1]$
12	$[-1 \quad -1 \quad +1 \quad +1]$
13	$[+1 \quad -1 \quad -1 \quad +1]$
14	$[-1 \quad +1 \quad +1 \quad -1]$

6.5.9.2 MU-MIMO

多用户 8 流波束赋形中，基站根据上行信道信息或者 UE 反馈的信息进行多用户匹配。多 UE 配对完成后，按照一定的准则生成天线赋形权值，利用得到的天线赋形权值为每一个 UE 进行赋形。

多用户 8 流波束赋形技术，利用了智能天线的波束定向原理和空间信道的不相关性，实现多 UE 的空分多址复用。此时，各 UE 占用相同的时频资源，但多个 UE 接收不同的数据流。

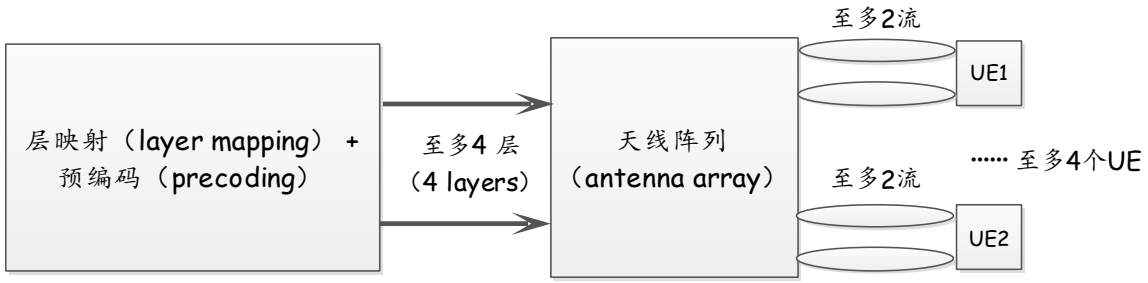


图 6-20：TM 9，使用 MU-MIMO

在多用户 8 流波束赋形模式下，多 UE 的配对可以是 2 或 4 个 rank = 1 的 UE 配对；也可以是 2 个 rank = 2 的 UE 配对；也可以是一个 rank = 1 和一个 rank = 2 的 UE 的配对。

对于 TM 9 而言，MU-MIMO 可以有 2 种实现形式：

（1）**支持至多 4 个 UE，每个 UE 使用 1 层**：eNoeB 将 4 个不同的 UE 特定的参考信号分配给并行传输的 4 个 UE，每个 UE 都不使用空分复用（即都只传输 1 个 codeword），都只使用 1 层传输。其实现方式如下：

在同一个调度的 TTI，eNodeB 给 4 个不同的 UE 分别发送了 DCI format 2C。每个 DCI format 2C 的“Antenna port(s), scrambling identity and number of layers”字段各不相同，其取值范围为 0~3。同时这 4 个 UE 都只使能了 1 个 codeword。即有

表 6-23：TM 9 下配对的 4 个 rank = 1 的 UE 的配置

UE	“Antenna port(s), scrambling identity and number of layers”字段	n_{SCID}	天线端口	层数
UE1	0	0	7	1
UE2	1	1	7	1
UE3	2	0	8	1
UE4	3	1	8	1

如果配对的是 2 个 rank=1 的 UE，可以通过将这 2 个 UE 的 DCI format 2C 中“Antenna port(s), scrambling identity and number of layers”字段分别配置成 0 和 2 来实现。

如果不同的 UE 使用相同的 n_{SCID} ，则不同的 UE 会使用不同的天线端口（port 7 或 port 8），即 UE 之间的数据会通过正交覆盖码（OCC）来区分；如果不同 UE 使用相同的天线端口，则 UE 之间会使用不同的 n_{SCID} 来区分彼此之间的数据。

复用 4 个 UE 意味着每个 UE 只能接收到原本 1/4 的功率，且 UE 还必须经受大量的 UE 间干扰。所以只有当小区内有大量高 SNR 的 UE，且由空分复用和多 UE 分集带来的速率增益超过了功率平分和增加的干扰带来的每 UE 速率降低时，才会使用 4 UE 配对的 MU-MIMO。由于在 MU-MIMO 中，调度超过 4 个 UE 会带来过度复杂的调度实现和 UE 实现，因此在真实的部署中，是几乎不会使用的。

从我的理解上看，TM 8 其实也支持至多 4 个 UE 进行 MU-MIMO。此时通过配置 n_{SCID} 和去使能某个 codeword，实现每个 UE 使用不同的“n_{SCID} + 天线端口”组合，从而使 UE 彼此区分开。当然，TM 9 也支持 6.5.8 节的表 6-18 中介绍的配对方式。

（2）**2 个 UE，每个 UE 至多 2 层**：eNodeB 将 4 个不同的参考信号分配给并行传输的 2 个 UE，每个 UE 都支持空分复用（即都传输 2 个 codeword），都使用 2 层传输。即这种方式将 MU-MIMO 和空分复用结合在一起。其实现方式如下：

在同一个调度的 TTI，eNodeB 给 2 个不同的 UE 分别发送了 DCI format 2C。每个 DCI format 2C 的“Antenna port(s), scrambling identity and number of layers”字段各不相同，其取值范围为 0~1。同时这 2 个 UE 都使能了 2 个 codeword。即有

表 6-24：TM 9 下配对的 2 个 rank = 2 的 UE 的配置

UE	“Antenna port(s), scrambling identity and number of layers”字段	n_{SCID}	天线端口	层数
UE1	0	0	7~8	2
UE2	1	1	7~8	2

注：也可以是一个 rank = 1 和一个 rank = 2 的 UE 进行配对，此时的配置方式与表 6-24 相同。

对于同一个 UE，使用正交覆盖码（OCC）来区分 port 7 和 port 8 中传输的数据；对于不同的 UE，使用 n_{SCID} 来区分二者的数据。

可以看出，当 TM 9（包括 TM 8）用于 MU-MIMO 时，所需做的只是给不同 UE 分配不同的 UE 特定的参考信号。

从上面的介绍可以看出，配置了 TM 9 的多个 UE 配对使用 MU-MIMO 时，都只能使用 port 7 和/或 port 8，而不能使用 port 9~14。

并且可以看出，TM 9 中，MU-MIMO 支持（1）至多 4 个 UE 在相同的时频资源上协同调度；（2）每个 UE 至多分配 2 层；（3）所有 UE 的层数加起来不能超过 4 层。

TM 8/9 用于 MU-MIMO 时，并不需要使用 TM 5 中提到的功率偏移。这是因为与 TM 5 使用小区特定的参考信号不同，在 TM 8/9 中，每个 PDSCH 传输都有各自的 UE 特定的参考信号集合，每个参考信号的功率成比例于层数和传输数，也就是说参考信号与每层的 PDSCH 功率的比值保持不变，所以不需要这个功率偏移值。

个人认为，TM8 和 TM 9 的 MU-MIMO 的应用场景、可配对的 UE 数以及层的划分都是相同的，只不过配置的 DCI format 中的字段不同罢了。

参考资料[3]对 MU-MIMO 做了一个系统性的介绍，对了解 MU-MIMO 的设计思路以及应用场景等有很大帮助，其中的参考资料涉及了许多相关的 3GPP 提案，值得一读。

3GPP 协议中关于 TM 9 的处理见表 6-25。

表 6-25：TM 9，码字/层/预编码处理

传输模式 (Transmission Mode)	**码字数 (No of Codewords)**	**层数 (No of Layers)**	**预编码 (Precoding)**	**码本 (Codebook)**	**天线端口 (Antenna Ports)**	**说明**
TM 9	1	**1 或 2 或 4（rank=1）** (36.211 的 6.3.3.1 节或 6.3.3.3 节)	36.211 的 6.3.4.1 节或 6.3.4.3 节	无	单天线端口：port 0 2 天线端口：port 0/1 4 天线端口：port 0/1/2/3	如果 PBCH 天线端口数为 1，则使用单天线端口 0 并同 TM 1 处理；否则使用传输分集

	1 或 2	**1~8** (36.211 的 6.3.3.2 节)	36.211 的 6.3.4.4 节	无 （上报 PMI 时的码本见图 6-18）	port 7~14	8 流波束赋形

【参考资料】

[1] 《LTE – The UMTS Long Term Evolution, 2nd Edition》的 29.1 节和 29.3 节

[2] 《LTE-A: Downlink Transmission Mode 9 (TM-9)》

[3] 《Recent Trend of Multiuser MIMO in LTE-Advanced》

[4] “R1-110086, Reference PDSCH Transmission Schemes for CQI Calculation in Rel-10” , Samsung

第7章　上行参考信号

本章主要介绍 LTE 中的上行参考信号。LTE 在上行定义了 2 种类型的上行参考信号：

- DeModulation Reference Signal（DMRS）：DMRS 主要用于 eNodeB 对上行物理信道进行信道估计，以便正确地解调 PUCCH 和 PUSCH；
- Sounding Reference Signal（SRS）：SRS 主要用于上行信道质量估计以便 eNodeB 进行上行的频选调度。SRS 还可用于估计上行 timing，且在假设下行/上行信道互益的情况（尤其是 TDD）下，利用信道对称性来估计下行信道质量。

在 7.1.1 节，我们会先对上行参考信号序列进行介绍，重点描述 group hopping 和 sequence hopping 的概念。在 7.1.2 节，我们会对 PUSCH 中使用的 DMRS 进行介绍，重点描述如何保证 DMRS 间的正交性。最后在 7.2 节，我们会对 SRS 做详尽的分析和介绍。

7.1　DMRS

本节主要介绍 DMRS，首先我们来介绍一下上行参考信号序列的生成。

7.1.1　上行参考信号序列

LTE 中的上行参考信号都是基于 Zadoff-Chu（ZC）序列生成的。

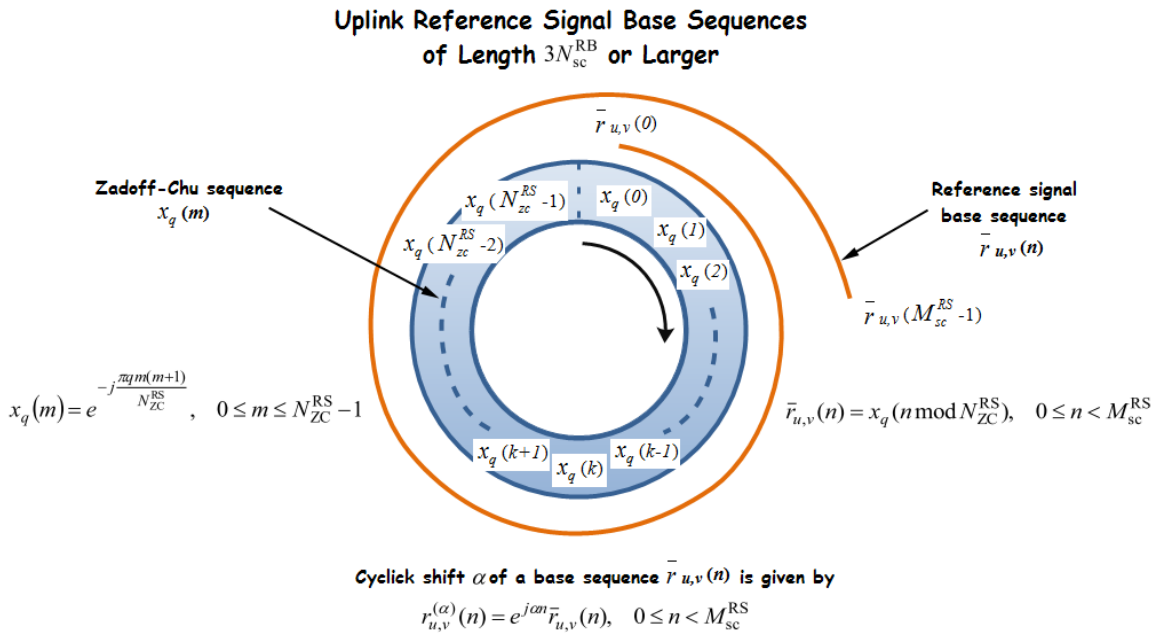


图 7-1：上行参考信号序列的生成

上行参考信号序列 $r_{u,v}^{(\alpha)}(n)$ 是对基序列（base sequence）$\bar{r}_{u,v}(n)$ 进行循环移位（cyclic shift）得到的。

首先我们来介绍如何生成基序列 $\bar{r}_{u,v}(n)$。这需要分两种情况处理：（1）基序列的长度小于 $3N_{sc}^{RB}$；（2）基序列的长度大于等于 $3N_{sc}^{RB}$。

（1）基序列的长度小于 $3N_{sc}^{RB}$（N_{sc}^{RB} 为一个 RB 包含的子载波数，其值固定为 12）：

此时使用特殊的 QPSK-based 序列，公式如下：

$$\bar{r}_{u,v}(n)=e^{j\varphi(n)\pi/4}, \quad 0\le n\le M_{sc}^{RS}-1$$

其中 $\varphi(n)$ 见 36.211 的 Table 5.5.1.2-1（对应 $M_{sc}^{RS}=N_{sc}^{RB}$）和 Table 5.5.1.2-2（对应 $M_{sc}^{RS}=2N_{sc}^{RB}$）。可以看出，对应每种长度，有 30 个可用的基序列，这 30 个序列被分为 30 组，分别对应 Table 5.5.1.2-1 和 Table 5.5.1.2-2 的 u 值。

（2）基序列的长度大于等于 $3N_{sc}^{RB}$：

此时可以将基序列的生成过程分解成以下步骤：确定参考信号的长度 -> 确定 Zadoff-Chu 序列的长度 -> 生成 Zadoff-Chu 序列 –> 生成基序列。

步骤一：确定参考信号的长度 M_{sc}^{RS}（子载波的个数）。DMRS 的序列长度计算见 7.1.2 节,SRS 的序列长度计算见 7.2 节。

步骤二：确定 Zadoff-Chu 序列的长度 N_{zc}^{RS}。N_{zc}^{RS} 为小于等于 M_{sc}^{RS} 的最大质数。

步骤三：生成 Zadoff-Chu 序列。q^{th} root Zadoff-Chu 序列定义为

$$x_q(m) = e^{-j\frac{\pi q m(m+1)}{N_{ZC}^{RS}}}, \quad 0 \le m \le N_{ZC}^{RS} - 1$$

其中，q 定义为

$$q = \lfloor \bar{q} + 1/2 \rfloor + v \cdot (-1)^{\lfloor 2\bar{q} \rfloor}$$
$$\bar{q} = N_{ZC}^{RS} \cdot (u+1)/31$$

这里 u 为组号（group number），v 为序列号（base sequence number）。具体见后续说明。

步骤四：生成基序列。使用下面的公式计算基序列：

$$\bar{r}_{u,v}(n) = x_q(n \bmod N_{ZC}^{RS}), \quad 0 \le n < M_{sc}^{RS}$$

从上面的公式可以看出，基序列是通过对长度为 N_{ZC}^{RS} 的 Zadoff-Chu 序列首尾相接，循环转圈得到的（起点为 0，走 M_{sc}^{RS} -1 步），见图 7-1。

基序列是对 Zadoff-Chu 序列进行循环扩展的得到的。以参考信号的长度 M_{sc}^{RS} 为 60 为例，N_{zc}^{RS} 为 59，此时可用的基序列数为 58（59-1）；以参考信号的长度 M_{sc}^{RS} 为 72 为例，N_{zc}^{RS} 为 71，此时可用的基序列数为 70（71-1）。即对应某种长度的参考信号序列，可用的基序列数为“小于参考信号长度的最小质数减 1”。

通过对前面介绍的基序列进行循环移位（cyclic shift）就得到对应的参考信号序列。

$$r_{u,v}^{(\alpha)}(n) = e^{j\alpha n}\bar{r}_{u,v}(n), \quad 0 \le n < M_{sc}^{RS}$$

7.1.1.1 group hopping（组跳）

对于每种参考信号序列长度，都有至少 30 个基序列可以使用。这些序列被分为 30 组，其取值范围是 0~29（这就是 u 的含义）。对于长度小于（不包含）72 的参考信号序列而言，可用的基序列数为 30 个，因此每个 group 只包含 1 个基序列（这就是 v 的含义，此时其值固定为 0）；对于长度大于或等于 72 的参考信号序列而言，可用的基序列数大于 60 个，此时每个 group 包含 2 个基序列（这就是 v 的含义，此时其值为 0 或 1）。

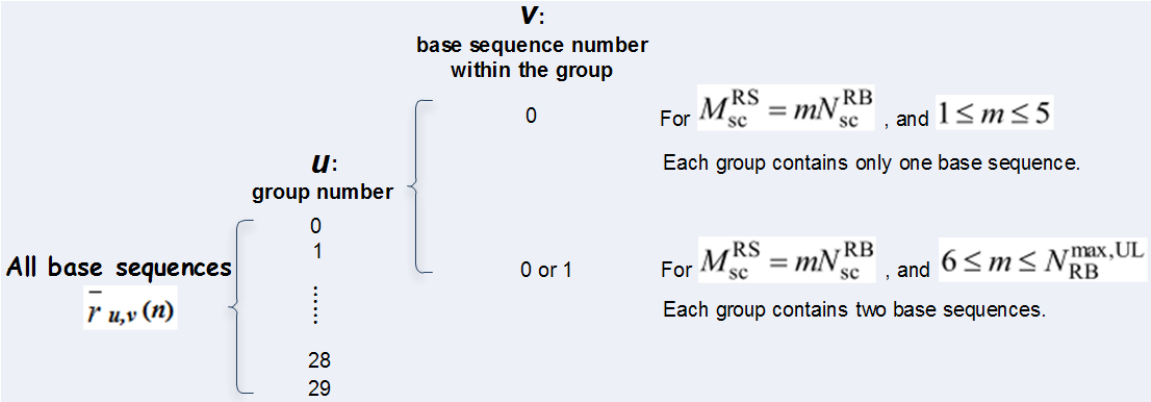


图 7-2：u 和 v 的取值范围

每个 slot 上使用的组号（group number）u 的计算方式见图 7-3：

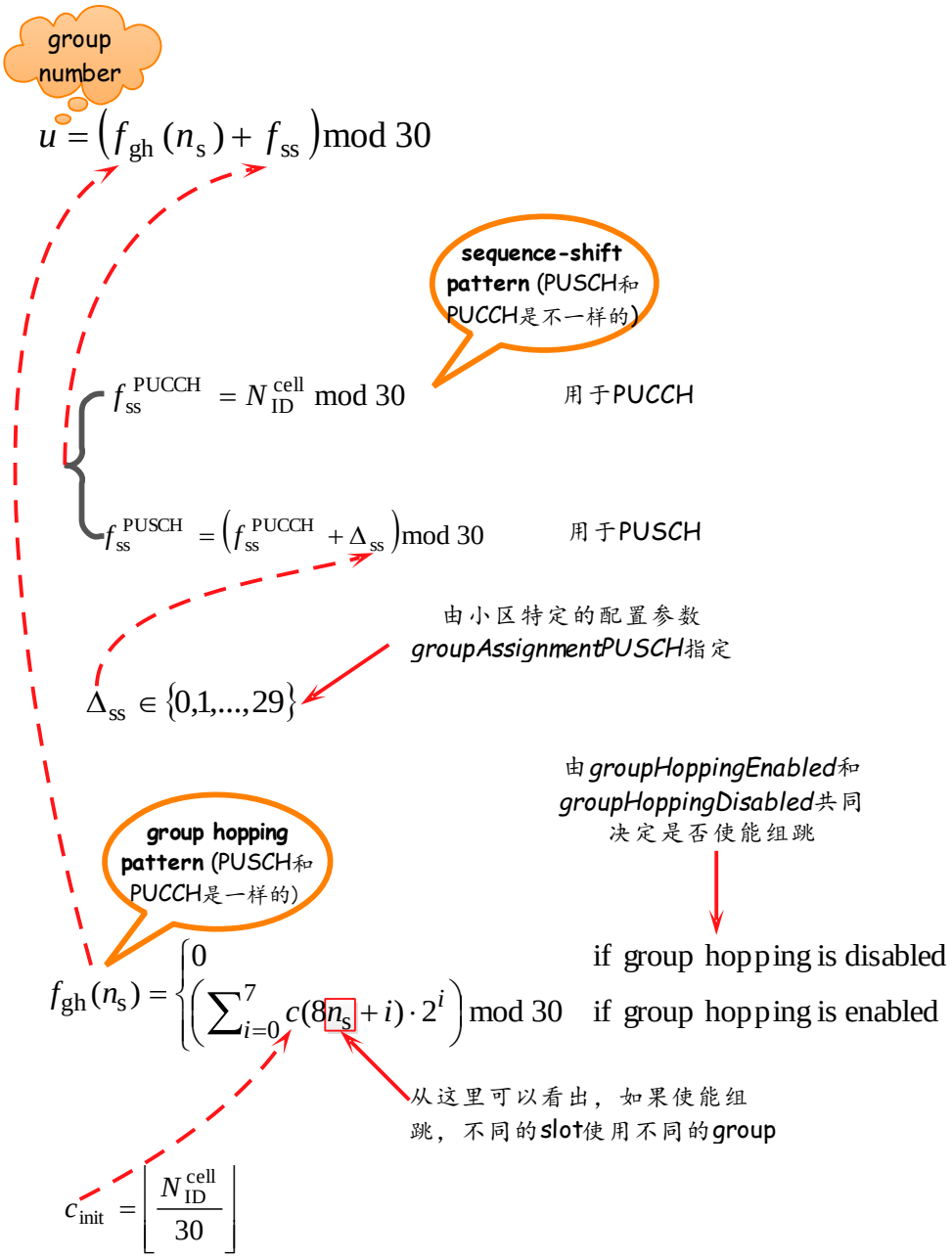


图 7-3：组号 u 的计算公式

同一小区内，不同的 slot 可以使用相同的 group，也可以使用不同的 group。如果不同的 slot 使用不同的 group（即使用不同的 u 值），就形成 **group hopping（组跳**，见 36.211 的 5.5.1.3 节）。

PUSCH 和 PUCCH 中的 DMRS 是否使能组跳可以通过**小区特定**的配置参数 *Group-hopping-enabled*（对应 36.331 中的 *groupHoppingEnabled*）来指定。对于 PUSCH 来说，还可以使用一个 **UE 特定**的参数 *Disable-sequence-group-hopping*（对应 36.331 中的 *groupHoppingDisabled-r10*）来配置是否去使能某个特定 UE 的组跳，此时不需要关心是否在小区级的配置中使能了组跳。即对于某个 UE 来说，该 UE 级的配置会覆盖小区级的配置。

从图 7-3 可以看出，如果组跳是去使能的，那么 $f_{gh}(n_s)$ 固定为 0，而 f_{ss} 只与 PCI 和小区特定的配置参数 *groupAssignmentPUSCH* 相关，所以此时对于小区内的所有 UE 而言，在任意 slot 上都使用相同的 u 值，即组号是相同的。

而如果组跳是使能的，$f_{gh}(n_s)$ 的值与 PCI 和 slot 号相关，此时 u 的值是随着 slot 号的变化而变化的，从而形成了组跳。

7.1.1.2 sequence hopping（序列跳变）

在同一小区内，对于长度大于 72 的参考信号序列（此时每个 group 包含 2 个基序列），不同的 slot 可以使用不同的 base sequence number（v 值不同），从而形成了 **sequence hopping**（见 36.211 的 5.5.1.4 节）。

对于长度 $M_{sc}^{RS} < 6N_{sc}^{RB}$ 的参考信号序列来说，base sequence number v 的值固定为 0。

对于长度 $M_{sc}^{RS} \geq 6N_{sc}^{RB}$ 的参考信号序列来说，在 slot n_s 上使用的 base sequence number v 通过下面的公式计算：

$$v = \begin{cases} c(n_s) & \text{if group hopping is disabled and sequence hopping is enabled} \\ 0 & \text{otherwise} \end{cases}$$

其中伪随机序列会在每个无线帧的开始处进行初始化，且 $c_{init} = \left\lfloor \frac{N_{ID}^{cell}}{30} \right\rfloor \cdot 2^5 + f_{ss}^{PUSCH}$。

小区特定的配置参数 *Sequence-hopping-enabled*（对应 36.331 中的 *sequenceHoppingEnabled*）决定了是否使能 sequence hopping。

PUSCH（PUCCH 的长度必定小于 6 个 RB，所以不支持 sequence hopping）中的 DMRS 是否使能 sequence hopping 可以通过小区特定的配置参数 *Sequence-hopping-enabled*（对应 36.331 中的 *sequenceHoppingEnabled*）来指定的。对于 PUSCH 来说，还可以使用一个 UE 特定的参数 *Disable-sequence-group-hopping*（对应 36.331 中的 *groupHoppingDisabled-r10*）来配置是否去使能某个特定

UE 的 sequence hopping，此时不需要关心是否在小区级的配置中使能了 sequence hopping。即对于某个 UE 来说，该 UE 级的配置会覆盖小区级的配置。

与此同时，只有组跳去使能的情况下才能使能 sequence hopping。

而如果 sequence hopping 是使能的，v 的值与 PCI 和 slot 号相关，此时 v 的值是随着 slot 号的变化而变化的，从而形成了 sequence hopping。

使用 group hopping 和 sequence hopping 的主要目的都是为了随机化小区间（inter-cell）的参考信号干扰。

7.1.2 DMRS：Demodulation Reference Signal

DMRS 主要用于 eNodeB 对上行物理信道进行信道估计，以便正确地解调 PUCCH 和 PUSCH。

由于没有必要估计 PUSCH/PUCCH 所占频带外的信道质量，所以 DMRS 总是与相应的物理信道（PUSCH 或 PUCCH）一起传输，并且覆盖相同的频带。 即 DMRS 的带宽（对应参考信号序列的长度）等于对应的 PUSCH/PUCCH 的带宽（以子载波的个数计算）。因此，随 PUSCH 一起传输的 DMRS 序列长度为 12 的整数倍，随 PUCCH 一起传输的 DMRS 序列长度固定为 12。

对于 PUSCH 而言，DMRS 位于每个 slot 的倒数第 4 个 SC-FDMA 符号上。

对于 PUCCH 而言，不同的 PUCCH format 有不同的分布。其在每个 slot 上所占的 SC-FDMA 符号见 36.211 的 Table 5.5.2.2.2-1（l 是从 0 开始计算的，每个 slot 内的取值范围是 0~6（对应正常的循环前缀）或 0~5（扩展的循环前缀），相关的图见第 8 章关于“PUCCH”的介绍）。

Table 5.5.2.2.2-1: Demodulation reference signal location for different PUCCH formats.

PUCCH format	Set of values for l	
	Normal cyclic prefix	Extended cyclic prefix
1, 1a, 1b	2, 3, 4	2, 3
2, 3	1, 5	3
2a, 2b	1, 5	N/A

DMRS 会占用所在 SC-FDMA 符号上的所有 RE，即上行参考信号与其它上行传输在时分上进行复用，而不是像下行参考信号一样使用频分复用，这样做的原因可参见参考资料[4]中的介绍。

对于 PUCCH 来说，可能存在多个 UE 在相同的时频资源上同时发送 PUCCH 的情况，此时不同 UE 使用的 DMRS 必须是相互正交的。

对于 PUSCH 来说，在不考虑 MU-MIMO 的情况下，同一 TTI 内分配给不同 UE 的 RB 集合是不同的，这就保证不同 UE 的 DMRS 在频域上已经区分开了。但在 SU-MIMO 中，同一 UE 会在多于

1 层上发送数据，不同层上使用的不同 DMRS 需要彼此正交，以避免层间干扰。在 MU-MIMO 中，配对的 UE 可能被分配在相同的 RB 上传输，这样就需要定义相互正交的 DMRS，以避免 UE 间的干扰。

也就是说，对于上行传输来说，以下场景需要定义多个相互正交的 DMRS：（1）同一小区的多个 UE 使用相同的时频资源同时进行上行传输，包括 PUCCH 传输和上行 MU-MIMO；（2）同一个 UE 使用了上行空分复用（SU-MIMO），此时不同层上的传输使用不同的 DMRS，这可从 36.211 的 Table 5.5.2.1.1-1 看出。

如果不同的 DMRS 使用不同的参考信号序列来定义的话，则它们拥有相对低但并不是完全互斥的相关性，也就是说，不是完全正交的。但如果不同的 DMRS 是基于**相同的参考信号序列**来生成的，并使用不同的循环移位（cyclic shift），就能使得彼此间是完全正交而互不干扰的。与此同时，也可以使用正交覆盖码（Orthogonal Cover Code， OCC）来定义相互正交的 DMRS。

7.1.2.1 用于 PUSCH 的 DMRS

本节的核心内容在于如何配置 DCI format 0/4 中的 Cyclic shift for DMRS and OCC index 字段以及配置 36.331 中的 *groupHoppingDisabled-r10* 来保证不同的 DMRS 之间的正交性。如何选择循环移位和 OCC 是本节的重点所在。

用于 PUSCH 的 DMRS 参考信号序列通过如下公式生成：

$$r_{\mathrm{PUSCH}}^{(\lambda)}\left(m\cdot M_{\mathrm{sc}}^{\mathrm{RS}}+n\right)=w^{(\lambda)}(m)r_{u,v}^{(\alpha_\lambda)}(n)$$

其中 $\begin{matrix}m=0,1\\ n=0,\ldots,M_{\mathrm{sc}}^{\mathrm{RS}}-1\end{matrix}$ 且 $M_{\mathrm{sc}}^{\mathrm{RS}}=M_{\mathrm{sc}}^{\mathrm{PUSCH}}$。这里 $w^{(\lambda)}(m)$ 定义了用于 DMRS 的正交覆盖码（Orthogonal Cover Code， OCC），α_λ 定义了用于 DMRS 的循环移位。

PUSCH 中的 DMRS 中使用的循环移位 α_λ 如图 7-4 所示：

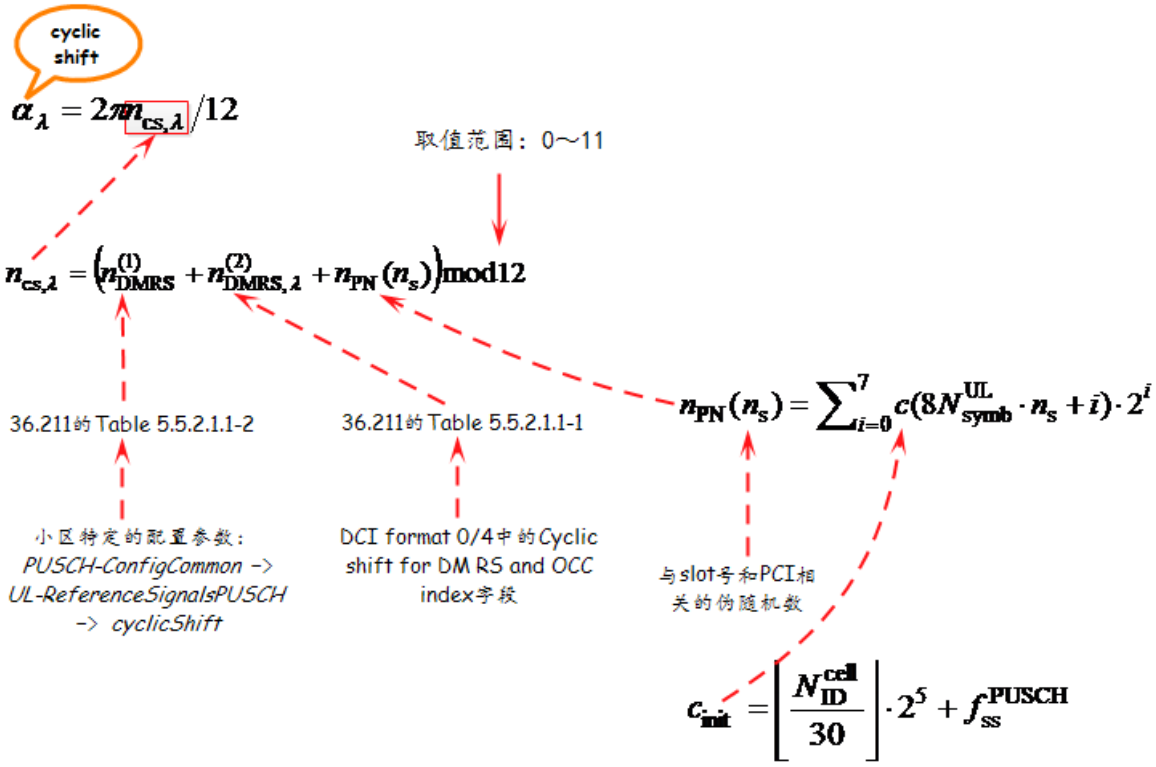


图 7-4：用于 PUSCH 的 DMRS 的循环移位 α_λ 的计算公式

可以看出：（1）由于 $n_{cs,\lambda}$ 是 mod 12 得到的，其取值范围是 0~11，也就是说对于同一个 base sequence，至多只有 12 种循环移位取值{mπ/6，其中 m 的取值范围是 0~11}；（2）$n_{DMRS}^{(1)}$ 与 $n_{PN}(n_s)$ 的计算与 UE 无关，所有在同一小区同一上行子帧被调度的 UE 所计算出的这 2 个值是相同的。而 $n_{DMRS,\lambda}^{(2)}$ 是与某个特定的 UE 相关的，该值（以及 OCC）是由最近发给某个特定 UE 的，对应同一 TB 的 PUSCH 传输的 DCI format 0/4 的 Cyclic shift for DMRS and OCC index 字段指定，并通过查 36.211 的 Table 5.5.2.1.1-1 得到的。也就是说，对于上行动态调度，非自适应重传与前一次传输（未必是“初传”）使用相同的 Cyclic shift for DMRS and OCC index（注意：这里说的是 Cyclic shift for DMRS and OCC index 相同，但并不意味着 $n_{DMRS,\lambda}^{(2)}$ 和 OCC 是相同的，后续会详细介绍）。而上行初传和自适应重传都有相应的 DCI format 0/4 伴随着发送，因此直接使用该 DCI 携带的 Cyclic shift for DMRS and OCC index 字段来获取 $n_{DMRS,\lambda}^{(2)}$ 和 OCC。

在频率选择性信道下，为了保持正交，并不是所有的 12 种循环移位都能够使用，而只有部分子集可以使用。例如，可能只有 6 种取值{0，2π/6，4π/6，…，10π/6} 或更少的值可被使用。使用的循环移位之间的间隔越大，对信道的频率选择性就越不敏感，也就越容易成功解码。

为了保持使用不同循环移位的参考信号的正交性，要求接收到的上行参考信号必须是时间对齐的，这也是为什么需要使用上行同步，以保证同一小区的不同 UE 的上行传输在时间上对齐的原因。

如果 eNodeB 没有设置 UE 特定的 RRC 配置参数 *Activate-DMRS-with OCC*（对应 *dmrs-WithOCC-Activated-r10*），或者当前 PUSCH 传输对应的 TB 的最近一个上行 DCI 使用 TC-RNTI 加扰（即 Msg3），则 DCI format 0 指定的正交序列 $w^{(\lambda)}(m)$ 固定为 $\left[w^{\lambda}(0) \quad w^{\lambda}(1)\right] = \left[1 \quad 1\right]$；否则当前 PUSCH 传输对应的 TB 使用的正交序列由对应的最近一个 DCI format 0/4 的 Cyclic shift for DMRS and OCC index 字段指定，并通过查 36.211 的 Table 5.5.2.1.1-1 得到。

Table 5.5.2.1.1-1: Mapping of Cyclic Shift Field in uplink-related DCI format to $n^{(2)}_{\mathrm{DMRS},\lambda}$ and $\left[w^{(\lambda)}(0) \quad w^{(\lambda)}(1)\right]$.

Cyclic Shift Field in uplink-related DCI format [3]	$n^{(2)}_{\mathrm{DMRS},\lambda}$				$\left[w^{(\lambda)}(0) \quad w^{(\lambda)}(1)\right]$			
	$\lambda=0$	$\lambda=1$	$\lambda=2$	$\lambda=3$	$\lambda=0$	$\lambda=1$	$\lambda=2$	$\lambda=3$
000	0	6	3	9	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$
001	6	0	9	3	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$
010	3	9	6	0	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$
011	4	10	7	1	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$
100	2	8	5	11	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$
101	8	2	11	5	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$
110	10	4	1	7	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$
111	9	3	0	6	$\left[1 \quad 1\right]$	$\left[1 \quad 1\right]$	$\left[1 \quad -1\right]$	$\left[1 \quad -1\right]$

可以说，在一个小区内，上行 DCI 的 Cyclic shift for DMRS and OCC index 字段决定了该 UE 的 PUSCH 传输使用的 DMRS 的循环移位和 OCC。

对于上行初传和自适应重传来说，都伴随着发送上行 DCI 来指示其如何进行传输，因此每一层使用的循环移位和 OCC 都可以通过对应上行 DCI 的 Cyclic shift for DMRS and OCC index 字段直接指定。

但对于非自适应重传（如 PHICH 触发的重传）来说，并没有对应的上行 DCI，因此此时需要定义一种规则以便 UE 获取对应的循环移位和 OCC。对于非自适应重传，当前 PUSCH 传输使用的 Cyclic shift for DMRS and OCC index 是由发给该 UE 的最近一个对应同一 TB 的 DCI format 0/4 指定的。

对于上行空分复用（TM 2），如果 2 个 TB 都是 NACK，或第一个 TB（TB 1）是 NACK，则非自适应重传使用的循环移位和 OCC 与前一次传输是相同的。但如果第一个 TB（TB 1）是 ACK，第二个 TB（TB 2）是 NACK，则虽然 Cyclic shift for DMRS and OCC index 与前一次传输相同，但非自适应重传使用的循环移位和 OCC 与前一次传输是不同的。

图 7-5 是 Cyclic shift for DMRS and OCC index 配置为 000 时，TB 2 的非自适应重传使用的循环移位和 OCC 的举例，注意 TB 2 的初传和重传使用的循环移位和 OCC 的不同。（注：这里 OCC 为 0 时表示 $\begin{bmatrix}1 & 1\end{bmatrix}$，OCC 为 1 时表示 $\begin{bmatrix}1 & -1\end{bmatrix}$）

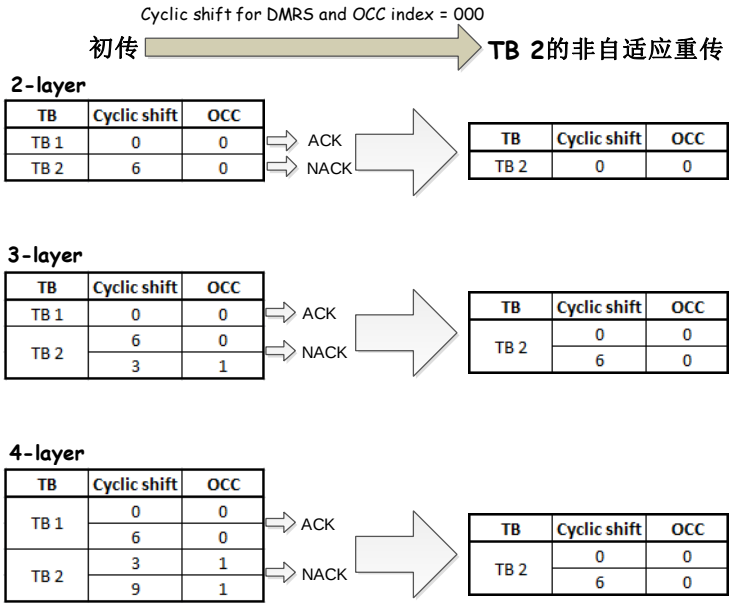


图 7-5：Cyclic shift for DMRS and OCC index=000 时，TB 2 的非自适应重传使用的循环移位和 OCC

如果同一 TB 的 PUSCH 传输并没有上行 DCI 并且满足下面条件之一，则会使用 Table 5.5.2.1.1-1 的第一行来获取 $n_{\mathrm{DMRS},0}^{(2)}$ 和 $w^{(\lambda)}(m)$。

- 如果同一 TB 的初始 PUSCH 传输是半静态调度（SPS）的；或者
- 如果同一 TB 的初始 PUSCH 传输是由 RAR（random access response）调度的。

也就是说，对于 SPS 调度或 RAR 调度的 PUSCH 传输（此时不支持多层传输，也不支持 MU-MIMO）的非自适应重传，则会使用 Table 5.5.2.1.1-1 的第一行来获取 $n_{\mathrm{DMRS},0}^{(2)}$ 和 $w^{(\lambda)}(m)$（其中 rank=1）。

协议中并没有明确说明由 SPS 调度和 RAR 调度的初始 PUSCH 传输使用哪种 $n_{\mathrm{DMRS},0}^{(2)}$ 和 $w^{(\lambda)}(m)$。但结合协议的介绍，此时应该也会使用 Table 5.5.2.1.1-1 的第一行来获取 $n_{\mathrm{DMRS},0}^{(2)}$ 和 $w^{(\lambda)}(m)$（其中 rank=1）。

Rel-10 支持上行多天线端口传输，并支持上行空分复用（SU-MIMO）。上行空分复用意味着需要使用多层传输，此时每层需要使用一个 DMRS，以区分不同层间的传输。由于 Rel-10 中的上行空分复用支持至多 4 层传输，因此一个 UE 需要支持传输至多 4 个 DMRS。这些 DMRS 可以通过以下方式区分：

- 使用不同的循环移位（如 36.211 的 Table 5.5.2.1.1-1 所示，不同的层分配了不同的 $n_{\mathrm{DMRS},\lambda}^{(2)}$，从而得到不同的循环移位。其中 λ 对应某个层）；
- 使用不同的正交覆盖码（Orthogonal Cover Codes，OCC）。如 36.211 的 Table 5.5.2.1.1-1 所示，某些层之间分配了不同的 $\left[w^{(\lambda)}(0) \quad w^{(\lambda)}(1)\right]$，从而保证了这些层之间的正交性。由于一个子帧内只能有 2 个 SC-FDMA 符号用于传输 DMRS，所以 $\left[w^{(\lambda)}(0) \quad w^{(\lambda)}(1)\right]$ 只能取值 $\left[1 \quad 1\right]$ 或 $\left[1 \quad -1\right]$。

这些正交的参考信号同样用于上行 MU-MIMO，即不同的 UE 使用相同的时频资源来传输 PUSCH 的场景。

接下来，我们将讨论如何配置 Cyclic shift for DMRS and OCC index（即选择合适的循环移位和 OCC），以最大程度地保证 DMRS 之间的正交性。这里有几条原则（这也是设计 36.211 的 Table 5.5.2.1.1-1 时考虑的原则）：

（1）对于循环移位来说，需要保证不同层间的循环移位值相差越大越好，以最小化层间（inter-layer）干扰。对于 SU-MIMO 来说，不同 UE 占用不同的 RB 集合，彼此之间已经通过频域区分了，此时只要保证同一 UE 内的不同层间传输的 DMRS 的循环移位相差尽可能大。对于 MU-MIMO 来说，需要保证不同 UE 在所有层上不能分配相同循环移位的同时，使得循环移位相差尽可能大。

（2）在 SU-MIMO 中，OCC 可与循环移位一起提供层间的正交性。在 MU-MIMO 中，OCC 可用于提供不同 UE 间的正交性。

对于 SU-MIMO，为了提供不同层间的最大正交性，不同的 OCC 应该应用到拥有相对近的循环移位值的 layer-pair 上。例如 layer #0 和 layer #1（彼此之间的循环移位值相差 6）使用同一个 OCC，而 layer #2 和 layer #3（彼此之间的循环移位值相差 6，但与 layer #0 和 layer #1 的循环移位值相差 3）使用另一个 OCC。

前面已经介绍过，同一 base 参考信号序列的不同循环移位才能用来定义多个正交的 DMRS。如果进行 MU-MIMO 配对的 UE 分配了不同的带宽，则不同的 UE 会使用不同的 base 参考信号序列（因为序列的长度不同）。此时 base 参考信号序列不同，使用循环移位并不能保证不同 UE 的 DMRS 间的正交性，因此需要使用 OCC 来区分不同 UE 的 DMRS。这就要求不同的 UE 不能使用相同的 OCC（同时，还是应该保证循环移位值相差尽可能大）。

但如果进行 MU-MIMO 的 UE 分配了相同的带宽，则不同的 UE 会使用相同的 base 参考信号序列（因为序列长度相同）。此时与 SU-MIMO 类似，OCC 可与循环移位一起提供不同 UE 的层间正交性。但依然要求不同的 UE 不能使用相同的 OCC，以提供最大的正交性。

也就是说，对于 MU-MIMO，为了提供不同 UE 间的最大正交性，不同的 OCC 应该应用到不同的 UE 上，并且每个 UE 的所有层应该使用相同的 OCC。

基于前面的介绍，表 7-1 中举了几个常见场景下的配置举例，如此配置的目的都是为了尽可能地保证 DMRS 间的最大正交性。

表 7-1：SU/MU-MIMO 参考信号配置举例

SU-/MU-MIMO 场景			layer #0	layer #1	layer #2	layer #3	Cyclic shift for DMRS and OCC index
SU-MIMO：rank=2， 1 UE	UE 1	$n_{\mathrm{DMRS},\lambda}^{(2)}$	0	6			000
		OCC	$[1\ \ 1]$	$[1\ \ 1]$			
SU-MIMO：rank=3，1 UE	UE 1	$n_{\mathrm{DMRS},\lambda}^{(2)}$	0	6	3		000
		OCC	$[1\ \ 1]$	$[1\ \ 1]$	$[1\ \ -1]$		
SU-MIMO：rank=4,，1 UE	UE 1	$n_{\mathrm{DMRS},\lambda}^{(2)}$	0	6	3	9	000
		OCC	$[1\ \ 1]$	$[1\ \ 1]$	$[1\ \ -1]$	$[1\ \ -1]$	
MU-MIMO：2 个 UE，每个 UE 的 rank=1	UE 1	$n_{\mathrm{DMRS},\lambda}^{(2)}$	0				000
		OCC	$[1\ \ 1]$				
	UE 2	$n_{\mathrm{DMRS},\lambda}^{(2)}$	6				001
		OCC	$[1\ \ -1]$				
MU-MIMO：2 个 UE，每个 UE 的 rank=2	UE 1	$n_{\mathrm{DMRS},\lambda}^{(2)}$	0	6			000
		OCC	$[1\ \ 1]$	$[1\ \ 1]$			
	UE 2	$n_{\mathrm{DMRS},\lambda}^{(2)}$	3	9			010

		OCC	$[1 \ -1]$	$[1 \ -1]$			
MU-MIMO：2 个 UE，每个 UE 的 rank=4	UE 1	$n^{(2)}_{\mathrm{DMRS},\lambda}$	4	10	7	1	011
		OCC	$[1 \ 1]$	$[1 \ 1]$	$[1 \ 1]$	$[1 \ 1]$	
	UE 2	$n^{(2)}_{\mathrm{DMRS},\lambda}$	8	2	11	5	101
		OCC	$[1 \ -1]$	$[1 \ -1]$	$[1 \ -1]$	$[1 \ -1]$	

对于 MU-MIMO 中，一个 rank=3 的 UE 和一个 rank=1 的 UE 进行配对，或 2 个 rank=3 的 UE 进行配对的场景并不常见，这里不予以介绍。如果有这个需要的话，遵循前面介绍的原则即可。

如果上行 MU-MIMO 中配对的 UE 分配了不同的带宽，并且使能了 group hopping 或 sequence hopping，则使用 OCC 也不能保证不同 UE 的 DMRS 间的正交性。在这种场景下，配对的 UE 需要去使能 group hopping 和 sequence hopping。

在 Rel-8/ Rel-9 中，group hopping 和 sequence hopping 的去使能是小区级的配置。如果此时将整个小区的 group hopping 和 sequence hopping 都去使能了，会导致没有配置“不同带宽的 UE 配对的 MU-MIMO”的其它 UE 丢失了小区间的干扰随机化。因此在 Rel-10 中，新增了 UE 级别的 group hopping 和 sequence hopping 的去使能，这也就是 7.1.1 节介绍的 *Disable-sequence-group-hopping*（对应 36.331 中的 *groupHoppingDisabled-r10*）的由来以及作用。

Rel-10 中，分配给 UE 的上行 PUSCH 资源可能不是连续的，此时 DMRS 的参考信号序列长度是基于分配给 UE 的所有 RB 来计算的，然后再分段放在对应的 PUSCH 所在的 RB 上传输。

7.1.2.2 用于 PUCCH 的 DMRS

对于 PUCCH 中使用的 DMRS，其长度固定为 12，此时保证在同一 RB pair 上传输 PUCCH 的不同 UE 间的 DMRS 彼此正交即可。具体见 36.211 的 5.5.2.2 节，本书就不再做介绍了。

【参考资料】

[1] TS 36.211 的 5.5 节 Reference signals

[2] TS 36.212 的 5.3.3.1.1 节 Format 0

[3] TS 36.212 的 5.3.3.1.8 节 Format 4

[4] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.2 节

[5] 《LTE – The UMTS Long Term Evolution, 2nd Edition》的 15 章和 29.2.1 节

[6] “R1-104906, OCC and CS for UL DMRS in SU/MU-MIMO” , Panasonic

[7] “R1-104638, Considerations on uplink DMRS design in LTE-Advanced” , Pantech

7.2 SRS

eNodeB 使用 SRS 来估计不同频段的上行信道质量。eNodeB 侧的调度器可以根据“上行信道状态估计”，将瞬时信道状态好的那些 RB 分配给 UE 的上行 PUSCH 传输，同时可以选择不同的传输参数（如：瞬时数据速率），以及选择对应上行多天线传输相关的不同参数（即用于“上行频率选择性调度”）。SRS 还可用于估计上行 timing，且在假设下行/上行信道互益的情况（尤其是 TDD）下，利用信道对称性来估计下行信道质量。

与 DMRS 不同，SRS 并不一定要与任何上行物理信道一起传输。但如果 SRS 和其它物理信道（如 PUSCH）一起传输时，SRS 通常会覆盖一个不同且通常更大的频带。

LTE 定义了 2 种类型的 SRS 传输：

- 周期性 SRS（periodic SRS，对应 trigger type 0）：Rel-8 中引入，通过 RRC 配置。
- 非周期性 SRS（aperiodic SRS，对应 trigger type 1）：Rel-10 中引入。对于 FDD，eNodeB 可以通过 DCI format 0/4/1A 触发 UE 发送非周期性 SRS。对于 TDD，eNodeB 可以通过 DCI format 0/4/1A/2B/2C 触发 UE 发送非周期性 SRS。（使用对应 DCI 中的 SRS request 字段）

如果 UE 需要在同一 serving cell 的同一子帧发送周期性 SRS 和非周期性 SRS，则 UE 只会发送非周期性 SRS。

载波聚合中，UE 可以同时在不同的 serving cell 上发送 SRS。一个 UE 可以在每个 serving cell 上都配置周期性 SRS 和非周期性 SRS。UE 在不同的 serving cell 上，可以有不同的 SRS 配置参数。SRS 配置是可选的。

```
SoundingRS-UL-ConfigCommon ::=      CHOICE {------小区特定的配置参数
    release                             NULL,
    setup                               SEQUENCE {
        srs-BandwidthConfig                 ENUMERATED {bw0, bw1, bw2, bw3, bw4, bw5,
bw6, bw7},------C_SRS，见36.211的Table 5.5.3.2-1，Table 5.5.3.2-2，Table 5.5.3.2-3和Table 5.5.3.2-4。
        srs-SubframeConfig                  ENUMERATED {
                                                sc0, sc1, sc2, sc3, sc4, sc5, sc6, sc7,
                                                sc8, sc9, sc10, sc11, sc12, sc13, sc14,
sc15},------一个系统帧（10 ms）内可用于发送SRS的子帧号的集合。见36.211的Table 5.5.3.3-1（FDD）和Table
5.5.3.3-2（TDD）。
```

```
        ackNackSRS-SimultaneousTransmission BOOLEAN, ------见36.213的8.2节，是否同时发送
ACK/NACK/SR（只针对PUCCH）和SRS。会影响到使用short PUCCH还是Normal PUCCH。SCell并不使用该字段（因为
载波聚合中，只有PCell会发送PUCCH）。
        srs-MaxUpPts                        ENUMERATED {true}           OPTIONAL    --
Cond TDD------是否重配置UpPTS的 m^max_SRS,0 ，见36.211的5.5.3.2节
    }
}

SoundingRS-UL-ConfigDedicated ::=   CHOICE{------ 用于周期性SRS，UE特定的配置参数
    release                             NULL,
    setup                               SEQUENCE {
        srs-Bandwidth                       ENUMERATED {bw0, bw1, bw2, bw3},------ B_SRS ，见
36.211的Table 5.5.3.2-1，Table 5.5.3.2-2，Table 5.5.3.2-3和Table 5.5.3.2-4
        srs-HoppingBandwidth                ENUMERATED {hbw0, hbw1, hbw2, hbw3},------
b_hop 。用于指定是否进行SRS跳频，见36.211的5.5.3.2节
        freqDomainPosition                  INTEGER (0..23), ------ n_RRC 。
        duration                            BOOLEAN,------用于指定UE是只发送一个SRS（FALSE），
还是无限地发送周期性SRS（直到去使能SRS）
        srs-ConfigIndex                     INTEGER (0..1023), ------I_SRS。用于指定周期性SRS的上
报周期 T_SRS 和在周期内的子帧偏移 T_offset
        transmissionComb                    INTEGER (0..1), ------ k̄_TC 。用于指定周期性SRS的“梳齿”
        cyclicShift                         ENUMERATED {cs0, cs1, cs2, cs3, cs4, cs5,
cs6, cs7}------ n^cs_SRS ，见36.211的5.5.3.1节。
    }
}

SoundingRS-UL-ConfigDedicated-v1020 ::= SEQUENCE {
    srs-AntennaPort-r10                 SRS-AntennaPort------ N_p 。用于指定周期性SRS使用的天线端
口数
}

SoundingRS-UL-ConfigDedicatedAperiodic-r10 ::=  CHOICE{------ 用于非周期性SRS，UE特定的配置参数
    release                             NULL,
    setup                               SEQUENCE {
        srs-ConfigIndexAp-r10               INTEGER (0..31), ------I_SRS。用于指定非周期性SRS的上
报周期 T_SRS,1 和在周期内的子帧偏移 T_offset,1
        srs-ConfigApDCI-Format4-r10         SEQUENCE (SIZE (1..3)) OF SRS-ConfigAp-r10
    OPTIONAL,--Need ON
        srs-ActivateAp-r10                  CHOICE {
                release                         NULL,
                setup                           SEQUENCE {
                    srs-ConfigApDCI-Format0-r10         SRS-ConfigAp-r10,
                    srs-ConfigApDCI-Format1a2b2c-r10        SRS-ConfigAp-r10,
                    ...
                }
        }                                                                   OPTIONAL    -
- Need ON
    }
}

SRS-ConfigAp-r10 ::= SEQUENCE {
    srs-AntennaPortAp-r10               SRS-AntennaPort, ------ N_p 。用于指定非周期性SRS使用的天
线端口数
```

```
    srs-BandwidthAp-r10                 ENUMERATED {bw0, bw1, bw2, bw3},------ B_SRS ，见36.211
的Table 5.5.3.2-1，Table 5.5.3.2-2，Table 5.5.3.2-3和Table 5.5.3.2-4
    freqDomainPositionAp-r10            INTEGER (0..23), ------ n_RRC 。
    transmissionCombAp-r10              INTEGER (0..1), ------ k̄_TC 。用于指定非周期性SRS的“梳齿”
    cyclicShiftAp-r10                   ENUMERATED {cs0, cs1, cs2, cs3, cs4, cs5, cs6,
cs7}------ n_SRS^cs ，见36.213的5.5.3.1节
}

SRS-AntennaPort ::=                 ENUMERATED {an1, an2, an4, spare1}

-- ASN1STOP
```

对于 TDD 而言，如果一个 serving cell 的 UpPTS 包含 1 个 SC-FDMA 符号（SC-FDMA symbol），则该符号可用于 SRS 传输；如果一个 serving cell 的 UpPTS 包含 2 个 SC-FDMA 符号，则这 2 个符号都可用于 SRS 传输，且可以同时分配给同一 UE。

7.2.1 SRS 子帧配置

如果 SRS 在某个子帧上发送，则 SRS 将占据该子帧的最后一个 SC-FDMA 符号上，因此 SRS 和 DMRS 位于不同的 SC-FDMA 符号上。如果最后一个 SC-FDMA 符号分配给了 SRS，则该符号不能用于 PUSCH 传输，在最差的情况下（每个子帧都有 SRS 传输），SRS 会占用大约 7%（1/14）的开销。

一个小区在一个系统帧（10 ms）内可用于发送 SRS 的子帧号的集合必须满足：

- $\lfloor n_s/2 \rfloor \bmod T_{SFC} \in \Delta_{SFC}$：这个公式计算的是在一个系统帧（10ms）内，有哪些子帧可以用于发送 SRS。其中 $\lfloor n_s/2 \rfloor$ 为子帧号；可以将该公式看做是把一个 10ms 的系统帧分为 $(10 / T_{SFC})$ 份，而 Δ_{SFC} 为在每一份中的子帧偏移。（T_{SFC} 和 Δ_{SFC} 的取值见图 7-6）
- 对于 TDD 而言，只有上行子帧或 UpPTS 可以用于发送 SRS。（TDD 中，子帧 0 和子帧 5 必定为下行子帧，所以必定不会用于发送 SRS）

一个小区内可用于发送 SRS 的子帧号的集合是通过 IE：*SoundingRS-UL-ConfigCommon* 的 *srs-SubframeConfig* 字段来配置，这是一个小区级的配置（通过 SIB2 下发）。

srs-SubframeConfig 与 36.211 的 Table 5.5.3.3-1（用于 FDD）和 Table 5.5.3.3-2（用于 TDD）的对应关系如图 7-6 所示。与协议中的这 2 个 Table 相比，我在最后添加了 1 列，给出了每种配置下的每个系统帧内可用于发送 SRS 的子帧集合。（见 36.211 的 5.5.3.3 节）

cell-specific configuration

```
SoundingRS-UL-ConfigCommon ::=      CHOICE {
    release                             NULL,
    setup                               SEQUENCE {
        srs-BandwidthConfig                 ENUMERATED {bw0, bw1, bw2, bw3, bw4, bw5, bw6, bw7},
        srs-SubframeConfig                  ENUMERATED {
                                                sc0, sc1, sc2, sc3, sc4, sc5, sc6, sc7,
                                                sc8, sc9, sc10, sc11, sc12, sc13, sc14, sc15},
        ackNackSRS-SimultaneousTransmission BOOLEAN,
        srs-MaxUpPts                        ENUMERATED {true}           OPTIONAL    -- Cond TDD
    }
}
```

This table is for FDD

Table 5.5.3.3-1: Frame structure type 1 sounding reference signal subframe configuration.

srs-SubframeConfig	Binary	Configuration Period T_{SFC} (subframes)	Transmission offset Δ_{SFC} (subframes)	Subframes that can be used to transmit SRS in a radio frame
0	0000	1	{0}	{0,1,2,3,4,5,6,7,8,9}
1	0001	2	{0}	{0,2,4,6,8}
2	0010	2	{1}	{1,3,5,7,9}
3	0011	5	{0}	{0,5}
4	0100	5	{1}	{1,6}
5	0101	5	{2}	{2,7}
6	0110	5	{3}	{3,8}
7	0111	5	{0,1}	{0,1,5,6}
8	1000	5	{2,3}	{2,3,7,8}
9	1001	10	{0}	{0}
10	1010	10	{1}	{1}
11	1011	10	{2}	{2}
12	1100	10	{3}	{3}
13	1101	10	{0,1,2,3,4,6,8}	{0,1,2,3,4,6,8}
14	1110	10	{0,1,2,3,4,5,6,8}	{0,1,2,3,4,5,6,8}
15	1111	reserved	reserved	reserved

This table is for TDD

Table 5.5.3.3-2: Frame structure type 2 sounding reference signal subframe configuration.

srs-SubframeConfig	Binary	Configuration Period T_{SFC} (subframes)	Transmission offset Δ_{SFC} (subframes)	Subframes that can be used to transmit SRS in a radio frame
0	0000	5	{1}	{1,6}
1	0001	5	{1, 2}	{1,2,6,7}
2	0010	5	{1, 3}	{1,3,6,8}
3	0011	5	{1, 4}	{1,4,6,9}
4	0100	5	{1, 2, 3}	{1,2,3,6,7,8}
5	0101	5	{1, 2, 4}	{1,2,4,6,7,9}
6	0110	5	{1, 3, 4}	{1,3,4,6,8,9}
7	0111	5	{1, 2, 3, 4}	{1,2,3,4,6,7,8,9}
8	1000	10	{1, 2, 6}	{1,2,6}
9	1001	10	{1, 3, 6}	{1,3,6}
10	1010	10	{1, 6, 7}	{1,6,7}
11	1011	10	{1, 2, 6, 8}	{1,2,6,8}
12	1100	10	{1, 3, 6, 9}	{1,3,6,9}
13	1101	10	{1, 4, 6, 7}	{1,4,6,7}
14	1110	reserved	reserved	reserved
15	1111	reserved	reserved	reserved

See 5.5.3.3 of TS 36.211

图 7-6：一个 10ms 的系统帧内可用于传输 SRS 的子帧号集合

从 36.211 的 Table 5.5.3.3-1 / Table 5.5.3.3-2 可以看出，有些 *srs-SubframeConfig* 的值是 reserved 的（FDD 下，*srs-SubframeConfig* = 15 是 reserved 的；TDD 下，*srs-SubframeConfig* = 14/15 是 reserved 的）。如果某小区配置的 *srs-SubframeConfig* 值是 reserved 的，那么该小区的 SRS 功能就是

完全关闭的（即所有 UE 都不发送 SRS）。例如当小区主要服务于高速移动的 UE 时，就可以关闭 SRS 功能。

为什么需要 SRS 子帧配置，而且该配置还是小区级的呢？

一个 UE 在某个特定子帧上发送的 SRS 可能与小区内其它 UE 的 PUSCH 传输在频域上相互重叠。为了避免不同 UE 的 SRS 传输和 PUSCH 传输之间发生冲突，所有 UE 都应该避免在发送了 SRS 的子帧的最后一个 SC-FDMA 符号上发送 PUSCH。因此，小区内的所有 UE 应该知道在哪些子帧集合上，可能会有 UE 发送 SRS，这样所有 UE 就能避免在这些子帧的最后一个 SC-FDMA 符号上发送 PUSCH。这就是为什么需要 SRS 子帧配置，且该配置是小区级的原因。

7.2.2 周期性 SRS

周期性 SRS 传输每隔一定的周期发送一次。eNodeB 通过 *srs-ConfigIndex*（UE 特定的配置参数）给 UE 配置 I_{SRS}。UE 通过 I_{SRS} 查 36.213 的 Table 8.2-1（用于 FDD）或 Table 8.2-2（用于 TDD），可以得到周期性 SRS 的上报周期 T_{SRS} 和在周期内的子帧偏移 T_{offset}。

Table 8.2-1: UE Specific SRS Periodicity T_{SRS} and Subframe Offset Configuration T_{offset} for trigger type 0, FDD

SRS Configuration Index I_{SRS}	SRS Periodicity T_{SRS} (ms)	SRS Subframe Offset T_{offset}
0 – 1	2	I_{SRS}
2 – 6	5	I_{SRS} – 2
7 – 16	10	I_{SRS} – 7
17 – 36	20	I_{SRS} – 17
37 – 76	40	I_{SRS} – 37
77 – 156	80	I_{SRS} – 77
157 – 316	160	I_{SRS} – 157
317 – 636	320	I_{SRS} – 317
637 – 1023	reserved	reserved

Table 8.2-2: UE Specific SRS Periodicity T_{SRS} and Subframe Offset Configuration T_{offset} for trigger type 0, TDD

SRS Configuration Index I_{SRS}	SRS Periodicity T_{SRS} (ms)	SRS Subframe Offset T_{offset}
0	2	0, 1
1	2	0, 2
2	2	1, 2
3	2	0, 3
4	2	1, 3
5	2	0, 4
6	2	1, 4
7	2	2, 3
8	2	2, 4
9	2	3, 4
10 – 14	5	I_{SRS} – 10
15 – 24	10	I_{SRS} – 15
25 – 44	20	I_{SRS} – 25
45 – 84	40	I_{SRS} – 45
85 – 164	80	I_{SRS} – 85
165 – 324	160	I_{SRS} – 165
325 – 644	320	I_{SRS} – 325
645 – 1023	reserved	reserved

周期 T_{SRS} 是每个 serving cell 级别的配置，其值属于集合{2, 5, 10, 20, 40, 80, 160, 320} ms。“每个 serving cell 级别”包含了 2 层含义：

（1）接入到同一 serving cell 的所有 UE 的 T_{SRS} 是相同的。假设 FDD 下的某个 serving cell 配置的 SRS 周期 $T_{SRS} = 10ms$ ，则该 serving cell 给接入到该小区的 UE 配置的 I_{SRS} 取值范围必须为“7~16”（见 36.213 的 Table 8.2-1）。

（2）如果 UE 配置了多个 serving cell（载波聚合），则对应不同的 serving cell，可以有不同的 T_{SRS} 配置。

对于“TDD 且 SRS 周期 $T_{SRS} > 2$ 的小区”和“FDD 小区” 而言，用于发送周期性 SRS 的子帧必须满足

$$(10 \cdot n_f + k_{SRS} - T_{offset}) \bmod T_{SRS} = 0$$

对于 TDD 下，SRS 周期 T_{SRS} 配置为 2 ms 的 SRS 传输而言，其发送的子帧必须满足

$$(k_{SRS} - T_{offset}) \bmod 5 = 0$$

注：对于 SRS 周期 T_{SRS} 配置为 2 ms 的情况，公式中是 mod 5 而不是 mod 2，这说明每个 UE 上报 SRS 的实际周期是 5 ms；并且从 36.213 的 Table 8.2-2 可以看出，此时每个半帧内（5 ms）只有 2 个 T_{offset}，说明每个半帧内只有 2 个 SRS 资源。

对于 FDD 而言，k_{SRS} 为每个系统帧内的子帧号（$k_{SRS}=\{0,1,...,9\}$）。对于 TDD 而言，k_{SRS} 在 36.213 的 Table 8.2-3 定义。

Table 8.2-3: k_{SRS} for TDD

	subframe index *n*										
	0	1		2	3	4	5	6		7	8	9
		1st symbol of UpPTS	2nd symbol of UpPTS					1st symbol of UpPTS	2nd symbol of UpPTS			
k_{SRS} in case UpPTS length of 2 symbols		0	1	2	3	4		5	6	7	8	9
k_{SRS} in case UpPTS length of 1 symbol		1		2	3	4		6		7	8	9

图 7-7 是 TDD 1 下，小区的 SRS 周期设置为 5 ms，且 UpPTS 占 2 个符号的一个例子。图 7-7 中给出了子帧号、k_{SRS} 和 I_{SRS} 的对应关系。在这个例子中，由于上行子帧数的限制，I_{SRS} 不能取值 14。

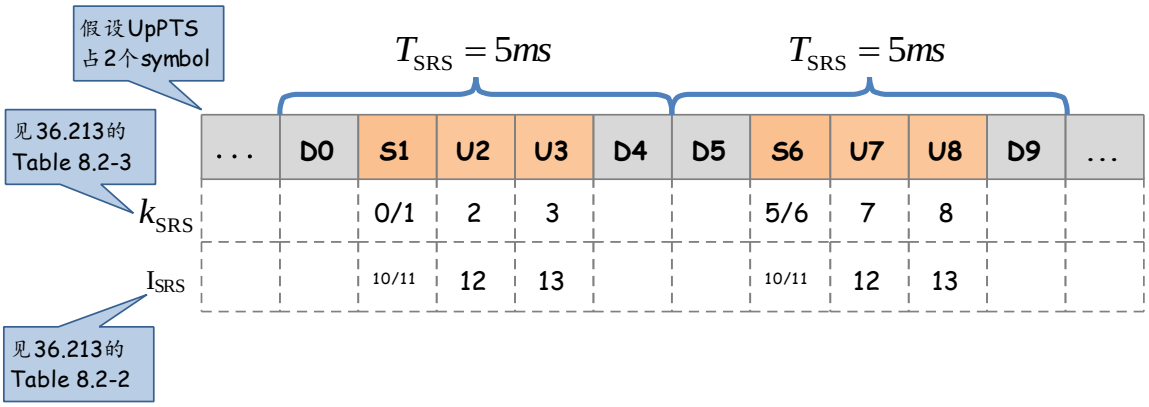


图 7-7：举例

I_{SRS} 的取值还会受到 *srs-SubframeConfig* 的影响。前面已经介绍过，*srs-SubframeConfig* 决定了一个小区在一个系统帧（10 ms）内可用于发送 SRS 的子帧号的集合，不属于这个集合内的子帧是不能用于发送 SRS 的。假设在上面的例子中，TDD 小区内只有特殊子帧 1 和 6 可用于发送 SRS（对应 *srs-SubframeConfig* = 0，见 36.211 的 Table 5.5.3.3-2），则 I_{SRS} 只能取值 10 或 11。

对于周期性 SRS，还有一个 UE 级的配置 *duration*，用于指定 UE 是只发送一个 SRS 后就不再发送，还是无限地周期性发送 SRS（直到 SRS 去使能为止）。

7.2.3 非周期性 SRS

非周期性 SRS 是通过 DCI format 0/4/1A/2B/2C 的 SRS request 字段来触发的，且非周期性 SRS 不支持跳频。

对于非周期性 SRS 且使用 DCI format 4 的情况，eNodeB 可通过 *srs-ConfigApDCI-Format4* 给 UE 配置 3 组 SRS 参数。UE 使用 DCI format 4 的 2 比特 SRS request 字段查 36.213 的 Table 8.1-1 可知道所选的那组 SRS 参数。

Table 8.1-1: SRS request value for trigger type 1 in DCI format 4

Value of SRS request field	Description
’00’	No type 1 SRS trigger
‘01’	The 1st SRS parameter set configured by higher layers
‘10’	The 2nd SRS parameter set configured by higher layers
‘11’	The 3rd SRS parameter set configured by higher layers

在非周期性 SRS 且使用 DCI format 0 的情况下，eNodeB 只会通过 *srs-ConfigApDCI-Format0* 给 UE 配置 1 组 SRS 参数。

在非周期性 SRS 且使用 DCI format 1A/2B/2C 的情况下，eNodeB 只会通过 *srs-ConfigApDCI-Format1a2b2c-r10* 给 UE 配置 1 组 SRS 参数。（注：个人认为使用 DCI format 1A/2B/2C 来触发非周期性 SRS 只用在 TDD 且 TM 8/9 配置了 Non-PMI/RI 上报的场景）

DCI format 0/1A/2B/2C 包含了 1 比特的 SRS request 字段，如果该字段值为 1，则触发非周期性 SRS 上报；否则不触发非周期性 SRS 上报。

注：只有 DCI format 4 的 SRS request 字段包含 2 比特的信息，其它 DCI format 的 SRS request 字段至多包含 1 比特的信息。

对于非周期性 SRS，eNodeB 通过 *srs-ConfigIndexAp-r10*（UE 特定的配置参数）给 UE 配置 I_{SRS}。UE 通过 I_{SRS} 查 36.213 的 Table 8.2-4（用于 FDD）或 Table 8.2-5（用于 TDD），可以得到 SRS 周期 $T_{SRS,1}$ 和在周期内的子帧偏移 $T_{offset,1}$。

注：非周期性 SRS 中，I_{SRS} 的取值也会受到 *srs-SubframeConfig* 的影响。

Table 8.2-4: UE Specific SRS Periodicity $T_{SRS,1}$ and Subframe Offset Configuration $T_{offset,1}$ for trigger type 1, FDD

SRS Configuration Index I_{SRS}	SRS Periodicity $T_{SRS,1}$ (ms)	SRS Subframe Offset $T_{offset,1}$
0 – 1	2	I_{SRS}
2 – 6	5	I_{SRS} – 2
7 – 16	10	I_{SRS} – 7
17 – 31	reserved	reserved

Table 8.2-5: UE Specific SRS Periodicity $T_{SRS,1}$ and Subframe Offset Configuration $T_{offset,1}$ for trigger type 1, TDD

SRS Configuration Index I_{SRS}	SRS Periodicity $T_{SRS,1}$ (ms)	SRS Subframe Offset $T_{offset,1}$
0	2	0, 1
1	2	0, 2
2	2	1, 2
3	2	0, 3
4	2	1, 3
5	2	0, 4
6	2	1, 4
7	2	2, 3
8	2	2, 4
9	2	3, 4
10 – 14	5	I_{SRS} – 10
15 – 24	10	I_{SRS} – 15
25 – 31	reserved	reserved

与周期性 SRS 类似的，非周期性 SRS 中的周期 $T_{SRS,1}$ 也是每个 serving cell 级别的配置，但其值属于集合{2, 5, 10} ms。

如果 UE 在 serving cell c 上配置了非周期性 SRS，且没有配置 CIF（非跨载波调度），则 UE 会基于 serving cell c 上接收到的 PDCCH 的 SRS request 字段，来决定是否在 serving cell c 上发送 SRS。

如果 UE 在 serving cell c 上配置了非周期性 SRS，且配置了 CIF（跨载波调度），则 UE 会基于接收到的 PDCCH 的 SRS request 字段，来决定是否在 CIF 字段指定的 serving cell c 上发送 SRS。

也就是说，PDCCH 调度的是哪个 serving cell 的 PUSCH/PDSCH 数据，其触发的非周期性 SRS 就在对应的 serving cell 上发送。

如果 UE 在 serving cell c 上配置了非周期性 SRS，且在 serving cell c 的子帧 n 上检测到了 SRS 请求，则 UE 会在满足以下条件的第一个子帧上发送非周期性 SRS：

- 该子帧满足 $n+k, k \geq 4$ ，且
- 对于“TDD 且 $T_{\text{SRS},1} > 2$ 的小区”和“FDD 小区” 而言，用于发送非周期性 SRS 的子帧必须满足 $(10 \cdot n_f + k_{\text{SRS}} - T_{offset,1}) \bmod T_{\text{SRS},1} = 0$。
- 对于“TDD 且 $T_{\text{SRS},1} = 2$ 的 SRS 传输”而言，其发送的子帧必须满足 $(k_{\text{SRS}} - T_{offset,1}) \bmod 5 = 0$。

对于 FDD 而言， k_{SRS} 为每个系统帧内的子帧号（ $k_{\text{SRS}} = \{0,1,...,9\}$ ）。对于 TDD 而言， k_{SRS} 在 36.213 的 Table 8.2-3 定义。（可参见图 7-7 的举例）

7.2.4 SRS 频域资源

在频域上，SRS 传输应该覆盖调度器感兴趣的频带。这可以通过 2 种方式实现：

- 通过发送一个足够大的“宽带 SRS（wideband SRS）”，来覆盖整个感兴趣的频带。
- 通过发送多个“窄带 SRS（narrowband SRS）”，并在频域上进行跳频，然后将一连串发送的 SRS 联合起来，就能覆盖整个感兴趣的频带。

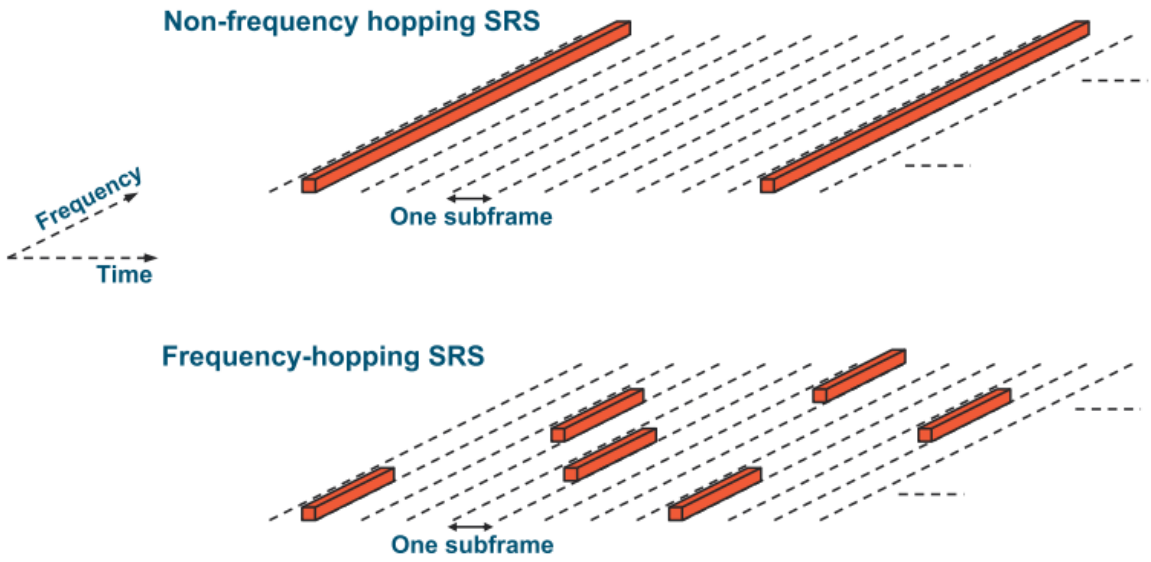


图 7-8：非跳频 SRS 和跳频 SRS 之间的比较

使用宽带（非跳频）SRS 传输的主要好处在于只使用 1 个 SRS 传输（在一个 SC-FDMA 符号内）就可以把整个感兴趣的频带都上报给 eNodeB。由于只要子帧的最后一个 SC-FDMA 符号要用于发送 SRS（不管是宽带 SRS 还是窄带 SRS），则整个 SC-FDMA 符号都不能用于该小区内所有 UE 的上行数据传输，所以从资源利用的角度上看，使用宽带 SRS 传输的效率更高，此时使用更少的 SC-FDMA 符号就可以探测整个带宽。对比上图，窄带 SRS 需要使用 4 个 SC-FDMA 符号（在不同的子帧上）才能把整个频带上报给 eNodeB。

但在上行路径损耗较高的情况下，宽带 SRS 传输可能导致相对较低的接收功率谱密度（这个好理解：发射功率固定的情况下，将功率平均分配给越大的带宽，则每个 RE 分配到的功率越小），这会恶化信道估计的结果。在这种情况下，使用多个窄带 SRS 是个更好的选择，这样可以使可用的传输功率集中在更窄的频率范围内，并且在频带内进行跳频，提高增益。

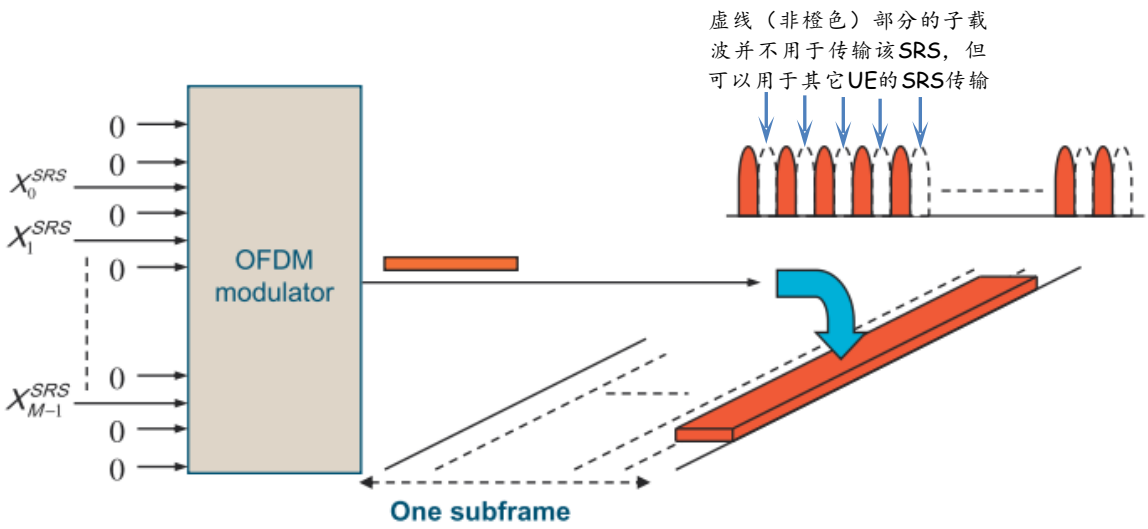


图 7-9：频域上参考信号的映射

不同 UE 的 SRS 可以在同一子帧且相同的 RB 集合上发送，但彼此之间可以通过使用不同的“梳齿”来区分。协议中使用 $\bar{k}_{TC} \in \{0,1\}$ 来指定某个 UE 发送的 SRS 所使用的梳齿，并通过 *transmissionComb* 或 *transmissionComb-ap* 字段来配置的。例如图 7-9 中，实线（橙色）部分的子载波用于某个 UE 的 SRS 传输，而虚线（非橙色）部分的子载波可用于其它 UE 的 SRS 传输。

也就是说，SRS 的参考信号序列是每隔一个子载波映射的，从而形成了一个“梳状”的频谱。

接下来，我们从 3GPP 协议的角度来介绍 SRS 的频域资源。

首先，我们来介绍 SRS 带宽，即每个 SRS 在频域上占多少个 RB。

$M_{sc,b}^{RS}$ 指定了 SRS 参考信号序列的长度，间接地说明了该 SRS 在频域上占多少个 RB。

$$M_{sc,b}^{RS} = m_{SRS,b} N_{sc}^{RB} / 2$$

其中 $b = B_{SRS}$ ， $m_{SRS,\,b}$ 见 36.211 的 Table 5.5.3.2-1 至 Table 5.5.3.2-4（每个 Table 对应不同的上行系统带宽）。 $m_{SRS,\,b}$ 实际上指定了每个 SRS 在频域上所占的 RB 数。公式中“除以 2”的原因是 SRS 的参考信号序列是每隔一个子载波映射的，即“梳状”频谱导致的。

$m_{SRS,\,b}$ 与上行系统带宽 N_{RB}^{UL} 、小区特定的配置参数 $C_{SRS} \in \{0,1,2,3,4,5,6,7\}$（通过 *srs-BandwidthConfig* 配置）和 UE 特定的配置参数 $B_{SRS} \in \{0,1,2,3\}$（通过 *srs-Bandwidth* 配置）相关。见 36.211 的 Table 5.5.3.2-1 至 Table 5.5.3.2-4。

Table 5.5.3.2-1: $m_{\mathrm{SRS},\,b}$ and N_b, $b=0,1,2,3$, values for the uplink bandwidth of $6 \le N_{\mathrm{RB}}^{\mathrm{UL}} \le 40$.

SRS bandwidth configuration	SRS-Bandwidth $B_{\mathrm{SRS}}=0$		SRS-Bandwidth $B_{\mathrm{SRS}}=1$		SRS-Bandwidth $B_{\mathrm{SRS}}=2$		SRS-Bandwidth $B_{\mathrm{SRS}}=3$	
C_{SRS}	$m_{\mathrm{SRS},\,0}$	N_0	$m_{\mathrm{SRS},\,1}$	N_1	$m_{\mathrm{SRS},\,2}$	N_2	$m_{\mathrm{SRS},\,3}$	N_3
0	36	1	12	3	4	3	4	1
1	32	1	16	2	8	2	4	2
2	24	1	4	6	4	1	4	1
3	20	1	4	5	4	1	4	1
4	16	1	4	4	4	1	4	1
5	12	1	4	3	4	1	4	1
6	8	1	4	2	4	1	4	1
7	4	1	4	1	4	1	4	1

Table 5.5.3.2-4: $m_{\mathrm{SRS},\,b}$ and N_b, $b=0,1,2,3$, values for the uplink bandwidth of $80 < N_{\mathrm{RB}}^{\mathrm{UL}} \le 110$.

SRS bandwidth configuration	SRS-Bandwidth $B_{\mathrm{SRS}}=0$		SRS-Bandwidth $B_{\mathrm{SRS}}=1$		SRS-Bandwidth $B_{\mathrm{SRS}}=2$		SRS-Bandwidth $B_{\mathrm{SRS}}=3$	
C_{SRS}	$m_{\mathrm{SRS},\,0}$	N_0	$m_{\mathrm{SRS},\,1}$	N_1	$m_{\mathrm{SRS},\,2}$	N_2	$m_{\mathrm{SRS},\,3}$	N_3
0	96	1	48	2	24	2	4	6
1	96	1	32	3	16	2	4	4
2	80	1	40	2	20	2	4	5
3	72	1	24	3	12	2	4	3
4	64	1	32	2	16	2	4	4
5	60	1	20	3	4	5	4	1
6	48	1	24	2	12	2	4	3
7	48	1	16	3	8	2	4	2

对于 UpPTS 而言，如果存在小区特定的参数 *srs-MaxUpPts*（值为 TRUE），则 $m_{\mathrm{SRS},\,0}$ 将重配置为 $m_{\mathrm{SRS},0}^{\max} = \max_{c \in C}\left\{m_{\mathrm{SRS},0}^{c}\right\} \le \left(N_{\mathrm{RB}}^{\mathrm{UL}} - 6N_{\mathrm{RA}}\right)$；否则 $m_{\mathrm{SRS},0}^{\max} = m_{\mathrm{SRS},0}$。其中 c 是 SRS BW 配置，$N_{\mathrm{RA}}$ 是对应 UpPTS 中包含的 format 4 PRACH 数，这里的 6 表示的是 1 个 PRACH 资源在频域上占 6 个 RB。

由于 SRS 关心的区域并不包含上行系统带宽边界处用于 PUCCH 传输的区域，所以最大 SRS 带宽（宽带 SRS）小于整个上行系统带宽。宽带 SRS 占据的是整个带宽的中心 $m_{\mathrm{SRS},\,0}$ 个 RB。

SRS 所对应的频域资源通常不包含用于传输 PUCCH format 2/2a/2b 的 RB，即二者不会在频域上产生冲突。

当 $B_{\mathrm{SRS}}=0$ 时，UE 将使用宽带 SRS 传输（也即 $m_{\mathrm{SRS},\,0}$ 是最大的 SRS 带宽）；当 $B_{\mathrm{SRS}}>0$（除了这种情况：对应 $6 \le N_{\mathrm{RB}}^{\mathrm{UL}} \le 40$、$C_{\mathrm{SRS}}=7$、$m_{\mathrm{SRS},\,0}=4$，只使用宽带 SRS，见 36.211 的 Table 5.5.3.2-4）时，UE 将使用窄带 SRS 传输。

C_{SRS} 和 B_{SRS} 共同决定了 SRS 带宽的大小和频域上分为几份。以 $N_{\mathrm{RB}}^{\mathrm{UL}}=100$、$C_{\mathrm{SRS}}=0$ 为例，参照 36.211 的 Table 5.5.3.2-4 可知：假设 $B_{\mathrm{SRS}}=0$，则使用宽带 SRS，每个 SRS 的带宽为 96 RB，

频域上只分为 1（$N_0=1$）份；假设 $B_{\mathrm{SRS}}=1$，则使用窄带 SRS，每个 SRS 的带宽为 48 RB，频域上被分为 2（$N_0*N_1=2$）份；假设 $B_{\mathrm{SRS}}=2$，则使用窄带 SRS，每个 SRS 的带宽为 24 RB，频域上被分为 4（$N_0*N_1*N_2=4$）份；假设 $B_{\mathrm{SRS}}=3$，则使用窄带 SRS，每个 SRS 的带宽为 4 RB，频域上被分为 24（$N_0*N_1*N_2*N_3=24$）份。

可以看出，SRS 传输的带宽总是 4 RB 的整数倍（对应 $m_{\mathrm{SRS},3}$），因此 SRS 的参考信号序列的长度总是 24 的整数倍（4 个 RB 对应 48 个子载波，由于 SRS 是梳状结构，因此 48 / 2 = 24 才是其长度的基本单位）。

接着，我们来介绍 SRS 传输在频域上的起始位置。

SRS 传输在频域上的起始位置 $k_0^{(p)}$ 通过如下方式计算：

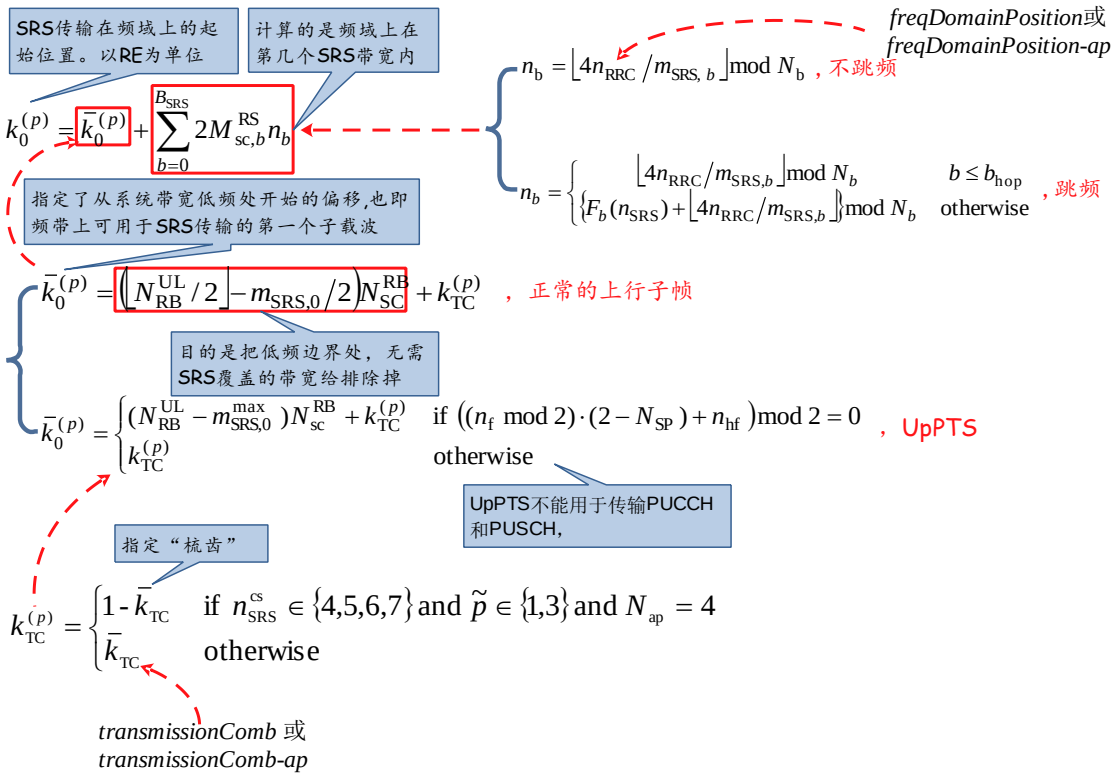


图 7-10：SRS 传输的频域起始位置

符号	说明
n_{f}	系统帧号
n_{hf}	一个系统帧由 2 个半帧组成。如果 UpPTS 位于第 1 个半帧上，则 n_{hf} 等于 0；如果 UpPTS 位于第 2 个半帧上，则 n_{hf} 等于 1。
N_{SP}	一个系统帧内 DL 到 UL 的 switch point 的个数。对于 5 ms 的“Downlink-to-Uplink switch-point periodicity”而言，其值为 2；对于 10 ms 的“Downlink-to-Uplink switch-point periodicity”而言，其值为 1（见 36.211 的 Table 4.2-2）。
N_{ap}	用于 SRS 传输的天线端口数
$\tilde{p}$	天线端口索引，见 36.211 的 Table 5.2.1-1

这里我们把公式分为 $\bar{k}_0^{(p)}$ 和 $\sum_{b=0}^{B_{\mathrm{SRS}}} 2M_{\mathrm{sc},b}^{\mathrm{RS}} n_b$ 两部分来分析。

$\bar{k}_0^{(p)}$ 表示了从上行系统带宽的低频处开始的可用于 SRS 传输的偏移，也即频带上可用于 SRS 传输的第一个子载波的位置，或者说宽带 SRS 的起始子载波所在的位置。

对于正常的子帧而言，$\bar{k}_0^{(p)} = \left(\left\lfloor N_{\mathrm{RB}}^{\mathrm{UL}} / 2 \right\rfloor - m_{\mathrm{SRS},0}/2\right) N_{\mathrm{SC}}^{\mathrm{RB}} + k_{\mathrm{TC}}^{(p)}$。其中 $\left(\left\lfloor N_{\mathrm{RB}}^{\mathrm{UL}} / 2 \right\rfloor - m_{\mathrm{SRS},0}/2\right) N_{\mathrm{SC}}^{\mathrm{RB}}$ 是为了把上行系统的低频处，SRS 不关心的用于 PUCCH 传输的区域排除掉。$k_{\mathrm{TC}}^{(p)}$ 的值为 0 或 1，用于确定使用哪个梳齿。图 7-11 是正常子帧，$N_{\mathrm{RB}}^{\mathrm{UL}} = 100$，$C_{\mathrm{SRS}} = 0$ 的一个例子，可以看出，如果 $k_{\mathrm{TC}}^{(p)}$ 的值为 0，则 $\bar{k}_0^{(p)} = 24$；如果 $k_{\mathrm{TC}}^{(p)}$ 的值为 1，则 $\bar{k}_0^{(p)} = 25$。

$\sum_{b=0}^{B_{\mathrm{SRS}}} 2M_{\mathrm{sc},b}^{\mathrm{RS}} n_b$ 可以看做是把宽带 SRS 等分成 N 份后，所取的那一份。其中 n_b 为频域位置索引，除非 RRC 连接重配置，否则其值是保持不变的；n_{RRC} 是通过 *freqDomainPosition*（用于周期性 SRS）和 *freqDomainPosition-ap*（用于非周期性 SRS）配置的，决定了 n_b 的值，见图 7-10。更确切地说，n_{RRC} 决定了 SRS 传输在频域上的起始位置。

图 7-11 是正常子帧，$N_{\mathrm{RB}}^{\mathrm{UL}} = 100$，$C_{\mathrm{SRS}} = 0$ 的一个例子（为了简单，省略了 $B_{\mathrm{SRS}} = 3$ 的场景），大家可以取个 n_{RRC} 值，同时查 36.211 的 Table 5.5.3.2-4，把对应的值代入图 7-10 中的公式计算一遍，就可以得到图 7-11 的结果。

注：由于 N_0 值固定为 1，所以可以计算出 n_0 值固定为 0。同时，$2M_{\mathrm{sc},b}^{\mathrm{RS}}$ 等于一个 SRS 带宽。

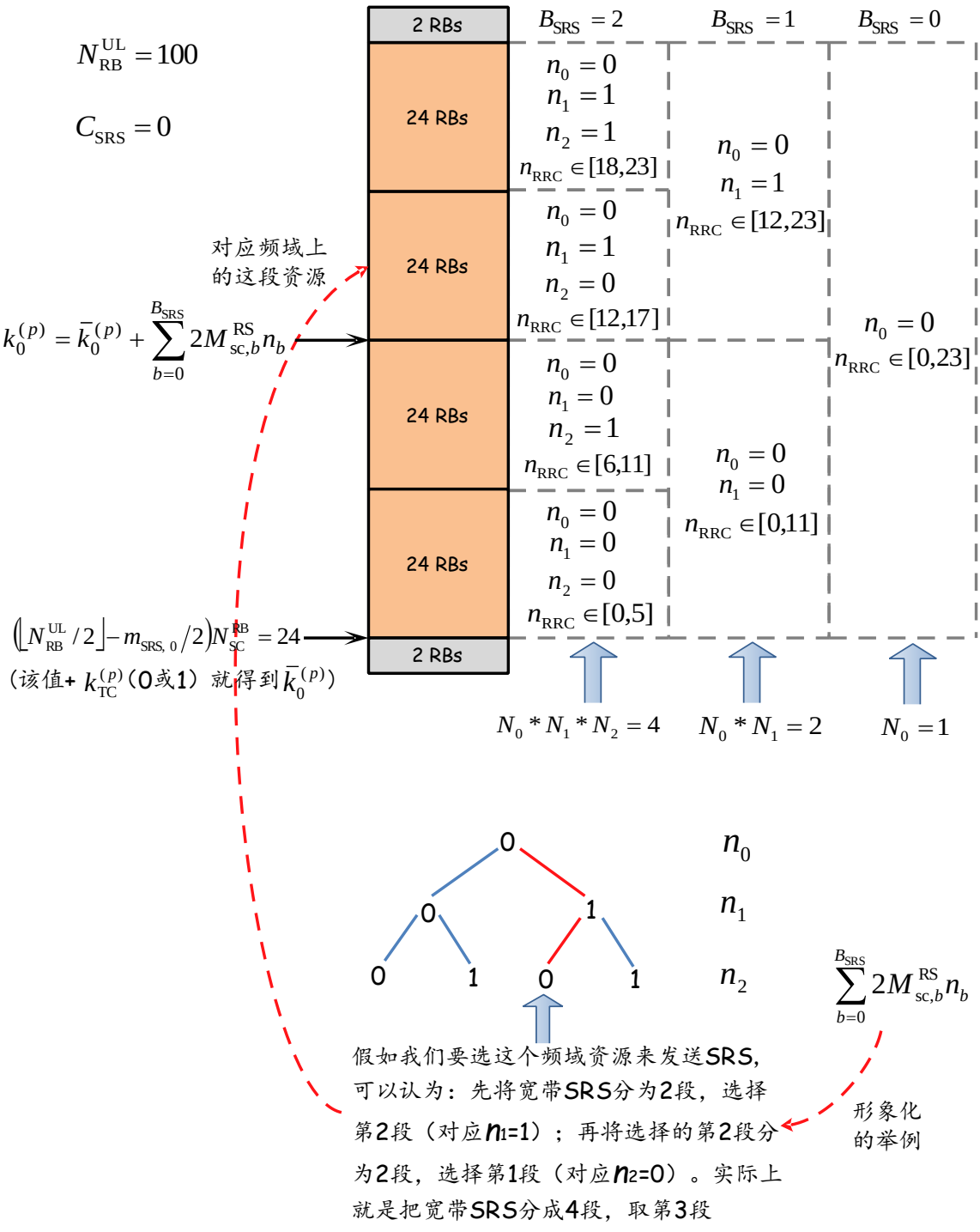


图 7-11：正常子帧，$N_{RB}^{UL}=100$，$C_{SRS}=0$ 的例子

这里就不对 UpPTS 上的 SRS 进行介绍了，其基本思路类似。但需要注意的是 UpPTS 不能用于发送 PUCCH 和 PUSCH，只能用于发送 SRS 和 PRACH。

最后，我们来介绍“梳齿”的概念。

从前面的介绍可以看出，不同 UE 可以在相同子帧、相同 RB 集合上发送 SRS，但彼此之间使用不同的 $\bar{k}_{TC}$ 予以区分。

从前面的介绍可以看出，非周期性 SRS 和周期性 SRS 的频域结构是相同的，但是配置方式（RRC 消息字段）有所不同。

7.2.5 SRS 跳频

SRS 的跳频是通过参数 $b_{\text{hop}} \in \{0,1,2,3\}$ 配置的，该值是通过 UE 级的 RRC 参数 *srs-HoppingBandwidth* 来配置的。

如果没有使能 SRS 跳频（ $b_{\text{hop}} \geq B_{\text{SRS}}$ ），则频率位置索引 n_b 的值固定为 $n_{\text{b}} = \lfloor 4n_{\text{RRC}} / m_{\text{SRS},\,b} \rfloor \bmod N_{\text{b}}$（除非进行 RRC 连接重配置），其中 n_{RRC} 是通过 *freqDomainPosition*（周期性 SRS）和 *freqDomainPosition-ap*（非周期性 SRS）配置的。

如果使能了 SRS 跳频（ $b_{\text{hop}} < B_{\text{SRS}}$ ），则频率位置索引 n_b 的值为

$$n_b = \begin{cases} \lfloor 4n_{\text{RRC}}/m_{\text{SRS},b} \rfloor \bmod N_b & b \leq b_{\text{hop}} \\ \{F_b(n_{\text{SRS}}) + \lfloor 4n_{\text{RRC}}/m_{\text{SRS},b} \rfloor\} \bmod N_b & \text{otherwise} \end{cases}$$

其中 N_b 见 36.211 的 Table 5.5.3.2-1 至 Table 5.5.3.2-4。$F_b(n_{\text{SRS}})$ 按下面的公式定义：

$$F_b(n_{\text{SRS}}) = \begin{cases} (N_b/2)\left\lfloor \dfrac{n_{\text{SRS}} \bmod \Pi_{b'=b_{\text{hop}}}^{b} N_{b'}}{\Pi_{b'=b_{\text{hop}}}^{b-1} N_{b'}} \right\rfloor + \left\lfloor \dfrac{n_{\text{SRS}} \bmod \Pi_{b'=b_{\text{hop}}}^{b} N_{b'}}{2\Pi_{b'=b_{\text{hop}}}^{b-1} N_{b'}} \right\rfloor & \text{if } N_b \text{ even} \\ \lfloor N_b/2 \rfloor \left\lfloor n_{\text{SRS}} / \Pi_{b'=b_{\text{hop}}}^{b-1} N_{b'} \right\rfloor & \text{if } N_b \text{ odd} \end{cases}$$

其中 $N_{b_{\text{hop}}} = 1$（无论 N_b 取何值），且有

$$n_{\mathrm{SRS}}=\begin{cases}2N_{\mathrm{SP}}n_{\mathrm{f}}+2\left(N_{\mathrm{SP}}-1\right)\left\lfloor\dfrac{n_{\mathrm{s}}}{10}\right\rfloor+\left\lfloor\dfrac{T_{\mathrm{offset}}}{T_{\mathrm{offset_max}}}\right\rfloor, & \text{for 2 ms SRS periodicit y of frame structure type 2}\\ \left\lfloor\left(n_{\mathrm{f}}\times 10+\left\lfloor n_{\mathrm{s}}/2\right\rfloor\right)/T_{\mathrm{SRS}}\right\rfloor, & \text{otherwise}\end{cases}$$

n_{SRS} 计算的是 UE 特定的 SRS 传输数目，T_{SRS} 和 T_{offset} 见 7.2.2 节介绍。$T_{\mathrm{offset_max}}$ 为特定 SRS 子帧偏移配置下的最大 T_{offset} 值。

需要注意的是，窄带 SRS 并不意味着一定会跳频。如果窄带 SRS 不跳频，则 UE 就只会固定上报某一段 RB 的 SRS 信息。

7.2.6 Cyclic Shift

类似于 DMRS，多个 UE 可以在同一子帧使用相同的 RB、相同的“梳齿 $\bar{k}_{TC}$ ”来发送 SRS，并使用同一 base sequence 的不同循环移位（码分复用）来保证不同 SRS 传输之间的正交性。对 SRS 而言，定义了 8 个不同的循环移位值 $n_{\mathrm{SRS}}^{\mathrm{cs}}=\{0,1,2,3,4,5,6,7\}$。不同 UE 的循环移位是通过 UE 级的配置参数 *cyclicShift*（周期性 SRS）和 *cyclicShiftAp-r10*（非周期性 SRS）下发给 UE。

7.2.7 SRS 的冲突处理

当 SRS 与 PUSCH 传输在同一 SC-FDMA 符号发生冲突时，UE 将不发送 SRS。例如：UE 在一个 serving cell 上要发送 SRS，而在另一个 serving cell 上要发送 PUSCH 且使用了最后一个 SC-FDMA 符号，此时会丢弃 SRS。

当 SRS 与携带 ACK/NACK 和/或 SR 的 PUCCH（PUCCH format 1/1a/1b/3）在同一子帧发生冲突时，如果 *ackNackSRS-SimultaneousTransmission* 设置为 FALSE，则 UE 不会发送 SRS；如果 *ackNackSRS-SimultaneousTransmission* 设置为 TRUE，则 UE 会同时发送 SRS 和 PUCCH，且发送的 PUCCH 使用短格式（shortened format，即该子帧第 2 个 slot 的最后一个 SC-FDMA 符号不用于发送 PUCCH，见 36.211 的 5.4.1 节和 5.4.2A 节）。

ackNackSRS-SimultaneousTransmission 决定了 UE 是否支持在同一子帧上传输 SRS 和用于反馈 ACK/NACK 的 PUCCH。如果支持，则在 PCell 的小区特定的 SRS 子帧上（见 7.2.1 节），UE 将使用短 PUCCH 格式（shortened PUCCH format）来传输 ACK/NACK 和 SR，即使在该子帧上没有 UE 发送 SRS 时，也是如此；否则，UE 会使用正常的 PUCCH format 1/1a/1b 和正常的 PUCCH format 3 来发送 ACK/NACK 和 SR。

由于 UE 只会在 PCell 上发送 PUCCH，所以对于 SCell 来说，并不关心 *ackNackSRS-SimultaneousTransmission*。

对于同一个 UE 而言，当其在任意一个 serving cell 上传输的 SRS 与其要发送的携带 ACK/NACK 和/或 SR 的正常 PUCCH（normal PUCCH format 1/1a/1b/3，不使用短格式，即 *ackNackSRS-SimultaneousTransmission*=FALSE）在同一子帧发生冲突时，UE 将不发送 SRS。

当周期性 SRS（type 0）与 PUCCH format 2/2a/2b 传输在同一子帧发生冲突时，UE 将不发送周期性 SRS。

当非周期性 SRS（type 1）与 PUCCH format 2a/2b 传输，或与带 HARQ 传输的 PUCCH format 2 在同一子帧发生冲突时，UE 将不发送非周期性 SRS。

当非周期性 SRS（type 1）与不带 HARQ 传输的 PUCCH format 2 在同一子帧发生冲突时，UE 将不发送不带 HARQ 传输的 PUCCH format 2。

注：从 36.213 的 10.1.1 节可以看出，带 HARQ 传输的 PUCCH format 2 只在扩展的循环前缀的情况下使用。且在扩展的循环前缀下，不支持 PUCCH format 2a/2b。

从 36.211 的 5.4.1 节和 5.4.2A 节可以看出，PUCCH format 1/1a/1b 和 PUCCH format 3 在编码时，其调制后的符号在不同的 SC-FDMA symbol 上存在冗余（也即同一个调制符号，如 d(0)，在多个 SC-FDMA symbol 上存在传输），因此当最后一个 SC-FDMA symbol 被 SRS 占用时，可以通过其它 SC-FDMA symbol 上的数据来解码 PUCCH format 1/1a/1b 和 PUCCH format 3，这也是这 2 种 PUCCH 格式存在短格式的原因。

但从 36.211 的 5.4.2 节可以看出，对于 PUCCH format 2/2a/2b 而言，20-bit 的 CQI 信息，QPSK 调制成 10 个符号（d(0)~d(9)），分别放到 10 个 SC-FDMA symbol 上传输（其它 SC-FDMA symbol 用于传输 DMRS），如果最后一个 SC-FDMA symbol 被 SRS 占用了，则 UE 只能在 9 个 SC-FDMA symbol 上发送其中的 9 个调制符号，这样就丢了最后一个调制符号 d(9)。由于不存在冗余，eNodeB 无法正确解码出 PUCCH format 2/2a/2b。这也是不存在短格式的 PUCCH format 2/2a/2b，以及 PUCCH format 2/2a/2b 不能与 SRS 同时传输的原因。

在 UpPTS 上，当 SRS 传输所占的资源与 preamble format 4 的 PRACH 区域相重叠，或超出了 serving cell 配置的上行系统带宽的范围时，UE 将不发送 SRS。

当 SRS 传输与 RAR 中的 UL grant 指定的 PUSCH 传输（或同一 TB 的重传），即 SRS 与基于竞争的随机接入中的 Msg3 或 Msg3 的重传，在同一子帧上发生冲突时，UE 将不会发送 SRS。

SRS 的冲突处理可见 36.213 的 8.2 节。

【参考资料】

[1] TS 36.211 的 5.5.3 节 Sounding reference signal

[2] TS 36.213 的 8.2 节 UE sounding procedure

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 15.6 节和 29.2.2 节

[4] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.2.2 节

[5] http://www.sharetechnote.com/html/Handbook_LTE_SRS.html

第8章　PUCCH

PUCCH 主要用于传输上行 L1/L2 控制信息以支持上下行数据传输。本章将按以下步骤来介绍 PUCCH。在 8.1 节，我们会对 PUCCH 做一个简单的介绍。在 8.2 节，我们会介绍各种 PUCCH format 中使用的码分复用，主要是关于循环移位和正交覆盖码的使用。而从 8.3 节到 8.6 节，我们会对各种 PUCCH format 进行详细介绍，并在 8.5 节介绍混合 PUCCH 的概念。在 8.7 节，我们会对 Rel-10 中提出的 SORTD 概念进行描述。最后在 8.8 节，我们会从资源分配的角度来描述如何管理和分配这些 PUCCH 资源。

8.1　PUCCH 简介

UE 需要发送必要的上行 L1/L2 控制信息以支持上下行数据传输。上行 L1/L2 控制信息（Uplink Control Information，UCI）包括

- SR（Scheduling Request）：用于向 eNodeB 请求上行 UL-SCH 资源。
- HARQ ACK/NACK：对在 PDSCH 上发送的下行数据进行 HARQ 确认。
- CSI：Channel State Information，包括 CQI、PMI、RI 等信息。用于告诉 eNodeB 下行信道质量等，以帮助 eNodeB 进行下行调度。

与下行控制信息（DCI）相比，为什么上行控制信息（UCI）只需要携带这么少的信息呢？这里秉承了一个原则：UE 只需要告诉 eNodeB 不知道的信息。

由于上行调度是在 eNodeB 侧实现的，与上行资源分配相关的信息（Resource block assignment、MCS 等）是由 eNodeB 通过 UL grant 告诉 UE 的，且对应该 UL grant，UE 在哪个上行子帧发送数据是固定的（对应关系见 36.213 的 Table 8-2），所以 eNodeB 知道 UE 会在哪个上行子帧的哪些 RB 上使用哪种 MCS 发送数据，而不需要 UE 通知它。

而与 HARQ 相关的信息：

- 由于上行调度是在 eNodeB 实现的，NDI 和 TB size 也是由 eNodeB 通过 UL grant 发送给 UE 的，所以 eNodeB 知道这些信息；
- 由于上行 HARQ 是同步（synchronous）的，eNodeB 和 UE 都可以根据初传/重传与 ACK/NACK 的 timing 关系推导出上行 HARQ process ID；
- UL HARQ 中使用的 RV（Redundancy version）遵循一个预先定义好的模式，eNodeB 事先也是知道的：
    - 初传（Initial transmission）的 RV 值固定为 0；

- 如果重传（retransmission）是由 PHICH 触发而不是由 PDCCH（UL Grant）触发（上行非自适应重传），则 RV 值为序列的下一个 RV 值（顺序为 0, 2, 3, 1。见 36.321 的 5.4.2.2 节）；
- 如果重传（retransmission）是由 PDCCH（UL Grant）触发（上行自适应重传），则 RV 是由指示重传的 PDCCH（DCI format 0/4）中的 Modulation and coding scheme and redundancy version 字段决定的（使用其中保留的 3 种组合，见 36.213 的 8.6.1 节）。

而一些下行独有的一些东西，如 DAI、TPC command for PUCCH、SRS request 之类的配置，UCI 就更没有必要发送了。

即使 UE 没有被分配任何 UL-SCH 资源，UE 也可能需要发送上行控制信息。取决于 UE 是否已经被分配了上行 UL-SCH 资源，LTE 定义了 2 种不同的方式用于发送上行 L1/L2 控制信息。

- 如果 UE 在当前子帧没有被分配用于发送 UL-SCH 数据的上行资源，则 UE 使用 PUCCH 来发送上行 L1/L2 控制信息。对于载波聚合（Carrier Aggregation，CA）而言，PUCCH 只能在 PCell 上传输。这也意味着，多个载波单元发送的下行数据的 HARQ 确认信息需要在同一个上行载波单元（对应 PCell）中发送。
- 如果 UE 在当前子帧被分配了用于发送 UL-SCH 数据的上行资源，则 UE 使用 PUSCH 来发送上行 L1/L2 控制信息。如果 UE 已经被分配了 UL-SCH 资源，就没必要再发送 SR 了。在载波聚合中，所有载波单元的 L1/L2 控制信息必须在同一个上行载波单元（并不一定是 PCell）中传输，这点与 PUCCH 类似。

UE 在一个上行子帧内需要发送的上行控制信息可能不同，因此 LTE 定义了几种不同的 PUCCH format（如表 8-1 所示）。

表 8-1：PUCCH format 简介

PUCCH format	Modulation scheme	Number of coded bits per subframe	Information
1	N/A	N/A	SR
1a	BPSK	1	1-bit ACK/NACK with/without SR
1b	QPSK	2	2-bit ACK/NACK with/without SR
1b with channel selection	QPSK	2	up to 4-bit ACK/NACK
2	QPSK	20	20-bit CSI
2	QPSK	20	20-bit CSI + 1 or 2-bit ACK/NACK (only for extended CP)

2a	QPSK + BPSK	21	20-bit CSI + 1-bit ACK/NACK (only support normal CP)
2b	QPSK + QPSK	22	20-bit CSI + 2-bit ACK/NACK (only support normal CP)
3	QPSK	48	For FDD, up to 10-bit ACK/NACK; For TDD, up to 20-bit ACK/NACK
3	QPSK	48	For FDD, up to 10-bit ACK/NACK + 1-bit SR; For TDD, up to 20-bit ACK/NACK + 1-bit SR

对于 PUCCH 而言，有几个很重要的限制，并对如何发送上行控制信息有很大影响：

- 一个 UE 在一个 TTI 内至多发送一个 PUCCH；
- 对 TDD 而言，PUCCH 不能在特殊子帧的 UpPTS 域中发送，即不能在特殊子帧上发送 PUCCH；
- 载波聚合中，PUCCH 只能在 PCell 上发送。

在 Rel-8/ Rel-9 中，是不允许同一 UE 在同一子帧同时发送 PUCCH 和 PUSCH 的。

与 Rel-8/ Rel-9 不同，在 Rel-10 中，允许同一 UE 在同一子帧同时发送 PUCCH 和 PUSCH，这是通过 IE：*PUCCH-ConfigDedicated-v1020* 的 *simultaneousPUCCH-PUSCH-r10* 字段来配置的。通常只有在 UE 有足够的功率且不影响覆盖的情况下，才会配置 UE 同时发送 PUCCH 和 PUSCH。

在 Rel-8/ Rel-9 中，上行并不支持多天线传输，因此只支持单天线端口发送 PUCCH。

与 Rel-8 不同，在 Rel-10 中，支持 2 天线端口发送 PUCCH，用以实现 PUCCH 的传输分集，该技术被称作 SORTD（Spatial Orthogonal-Resource Transmit Diversity）。在 8.7 节，我们会详细介绍 SORTD 技术。

【参考资料】

[1] TS 36.211 的 5.4 节 Physical uplink control channel

[2] TS 36.213 的第 10 章 Physical uplink control channel procedures

[3] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.4 节

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.3 节

8.2 PUCCH 中的码分复用

PUCCH 在频域上通常被配置成位于上行系统带宽的边缘。一个 PUCCH 在一个上行子帧内占 2 个 slot，每个 slot 在频域上占 12 个子载波（subcarrier），即 1 个 RB。为了提供频域分集，PUCCH 在 slot 的边界“跳频”：即在同一子帧内，PUCCH 前后两个 slot 的 PRB 资源分别位于可用的频谱资源的两端（这 2 个 PRB 组成了一个 RB pair，但在频域资源上，我们称其占一个 RB），而中间的整块频谱资源用于传输 PUSCH（如图 8-1 所示）。这样的设计不仅能够提供 PUCCH 的频率分集增益，还不会打散上行频谱，保证了上行传输的单载波特性。（更多的优点可以见参考资料[3]中的介绍）

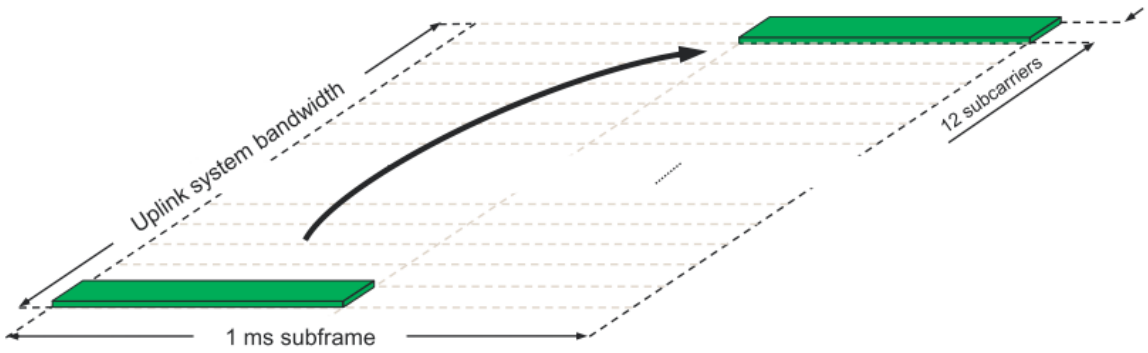


图 8-1：PUCCH 资源的 RB 分布

一个 UE 在一个子帧中独占一个 RB pair 来发送 PUCCH 太过奢侈了，为了有效地利用资源，同一小区的多个 UE 可以共享同一个 RB pair 来发送各自的 PUCCH。这是通过正交码分复用（orthogonal Code Division Multiplexing，CDM）来实现的：在频域上使用循环移位（cyclic shift，也称作相位旋转 phase rotation，这是同一种技术的 2 种不同说法）；在时域上使用正交序列（orthogonal sequence）。

不同的 PUCCH format，可能使用不同的 CDM 技术（如表 8-2 所示）。

表 8-2：不同 PUCCH format 所使用的不同的 CDM 技术

PUCCH format	CDM
1/1a/1b	cyclic shift + orthogonal sequence

2/2a/2b	cyclic shift
3	orthogonal sequence

频域上的 CDM 是通过对一个长为 12 的小区特定的频域序列（一个 PUCCH 频域上占 1 个 RB，共 12 个子载波，因此序列长度为 12。该序列与长度为 12 的上行参考信号序列相同）进行循环移位，生成不同的正交序列，再分配给不同的 UE 来实现的。即 PUCCH 在一个 RB 内至多支持 12 个循环移位（对应循环移位索引 0 ~ 11）。

然而对 PUCCH format 1/1a/1b 而言，在频率选择性信道下，为了保持正交，并不是所有的 12 种循环移位都能够使用。典型情况下，可认为小区至多有 6 个可用的循环移位。而小区间干扰可能导致这个数目变得更小。该数目是通过 *deltaPUCCH-Shift* 来配置（这个参数会在介绍 PUCCH format 1/1a/1b 时予以说明）。

表 8-3：不同 PUCCH format 在一个 RB 内所能使用的 cyclic shift 的个数

PUCCH format	Number of cyclic shifts in a RB
1/1a/1b	12 (*deltaPUCCH-Shift* = 1) 6 (*deltaPUCCH-Shift* = 2) 4 (*deltaPUCCH-Shift* = 3)
2/2a/2b	12

时域上的 CDM 是通过将一个 RB 内用于传输 PUCCH 的所有 SC-FDMA 符号乘以一个正交序列（orthogonal sequence）来实现。不同的 UE 在同一 RB 上发送 PUCCH 时使用不同的正交序列，从而保证了相互间的正交性。

表 8-4：不同 PUCCH format 在一个 RB 内所能使用的 orthogonal sequence 的个数

PUCCH format	Number of orthogonal sequences in a RB
1/1a/1b	Normal CP: 3 Extended CP: 2
3	Normal PUCCH format 3: 5 Shortened PUCCH format 3: 4

使用循环移位和正交序列，能够保证同一小区内（intra-cell）的 UE 之间的正交性，却无法避免来自不同小区（inter-cell）的 UE 的干扰，这是因为不同小区使用的 PUCCH 序列并不一定正交。为了随机化 inter-cell 干扰，序列的循环移位会根据一个衍生于该小区 PCI 的跳变模式，随着每个 slot 的每个 SC-FDMA 符号进行变化。

每个小区使用的长为 12 的小区特定的频域序列（基序列）与该小区的 PCI 有关，并且在每个 SC-FDMA 符号上使用的序列都是对基序列进行循环移位生成的。循环移位的偏移值与 slot 号（n_s：取值范围 0~19）以及 symbol 号（l：正常的循环前缀下，其取值范围为 0~6；扩展的循环前缀下，其取值范围为 0~5）都有关系。循环移位受下面这个公式的影响（见 36.211 的 5.4 节）：

$$n_{\mathrm{cs}}^{\mathrm{cell}}(n_{\mathrm{s}},l)=\sum_{i=0}^{7}c(8N_{\mathrm{symb}}^{\mathrm{UL}}\cdot n_{\mathrm{s}}+8l+i)\cdot 2^{i}$$

其中 $c(i)$ 是一个伪随机数序列（在 36.211 的 7.2 节中定义），且 $c_{\mathrm{init}}=N_{\mathrm{ID}}^{\mathrm{cell}}$（即等于 PCI）。

后面介绍 PUCCH 1、PUCCH 2 资源时，会看到上面的公式是如何影响循环移位的取值的。

为了进一步随机化 inter-cell 干扰，正交序列和循环移位还会在 slot 间跳变（见参考资料[2]）。这也会在后续予以介绍。

图 8-2 举了一个例子。假定共 2 个小区：cell A 和 cell B。在正常的循环前缀（normal CP）下，每个小区只能使用 6 个循环移位，即一个 RB 总共有 3 * 6 = 18 个 PUCCH 1 资源。在 cell A 中，若一个 UE 使用了 PUCCH 1 资源 3（对应橙色线圈起来的），对应（cyclic shift，orthogonal sequence）组合：在第一个 slot 使用（6, 0），在第二个 slot 使用（11, 1）。若该 UE 使用了 PUCCH 1 资源 11（对应绿色线圈起来的），则对应（cyclic shift，orthogonal sequence）组合：在第一个 slot 使用（11, 1），在第二个 slot 使用（8, 2）。在 cell B 中，虽然同样是使用 PUCCH 1 资源 3 和 11，但 PUCCH 1 资源对应的的循环移位和正交序列与 cell A 是不同的，从而随机化了不同小区间的干扰。

Phase rotation [multiples of 2π/12]	Number of cover sequence					
	Even-numbered slot			Odd-numbered slot		
	0	1	2	0	1	2
0	0		12	12		16
1		6			14	
2	1		13	6		10
3		7			8	
4	2		14	0		4
5		8			2	
6	3		15	13		17
7		9			15	
8	4		16	7		11
9		10			9	
10	5		17	1		5
11		11			3	

Cell A

Phase rotation [multiples of 2π/12]	Number of cover sequence					
	Even-numbered slot			Odd-numbered slot		
	0	1	2	0	1	2
0		11			3	
1	0		12	12		16
2		6			14	
3	1		13	6		10
4		7			8	
5	2		14	0		4
6		8			2	
7	3		15	13		17
8		9			15	
9	4		16	7		11
10		10			9	
11	5		17	1		5

Cell B

图 8-2：两个不同的小区，2 个 PUCCH 资源的循环移位和正交序列跳变的例子

需要说明的是，除了 PUCCH format 1/1a/1b 和 PUCCH format 2/2a/2b 可以混合在同一个 RB 中传输外，其它任何情况下，1 个 RB 只能用于传输一种格式的 PUCCH，即 1 个 RB 或只能用于传输 PUCCH format 1/1a/1b，或只能用于传输 PUCCH format 2/2a/2b，或只能用于传输 PUCCH format 3。但多个 UE 可以在同一 TTI 内使用同一 RB 来发送相同格式的 PUCCH，并使用上面介绍的 CDM 技术予以区分。

不同的 PUCCH format 如何使用这些 CDM 技术，会在后续具体介绍每种 PUCCH format 时予以介绍。

【参考资料】

[1] TS 36.211 的 5.4 节 Physical uplink control channel

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.4 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.3 节

8.3 PUCCH format 1/1a/1b

本节主要介绍 PUCCH format 1/1a/1b，主要内容包括：PUCCH format 1/1a/1b 携带的信息、PUCCH format 1/1a/1b 如何调制到 RB 中、PUCCH 1 资源表示、*deltaPUCCH-Shift* 对循环移位的影响、一个 RB 包含多少个 PUCCH 1 资源的计算、如何解读 36.211 的 5.4.1 节和 5.4.3 节的那些公式等等。

需要说明的是：（1）本节介绍的内容只适用于 UE 接入一个小区的场景，即不牵涉载波聚合（Carrier Aggregation，CA）。在载波聚合中，UE 如何反馈 HARQ ACK/NACK 会在后续介绍 PUCCH format 1b with channel selection（见 13.5.1 节）和 PUCCH format 3（见 8.6 节和 13.5.2 节）中予以说明；（2）本节介绍的内容不涉及半静态调度（SPS）；（3）本节介绍的内容不涉及 PUCCH format 1/1a/1b 与 PUCCH format 2/2a/2b 混合的 PUCCH，这会在 8.5 节中介绍。

PUCCH format 1 只携带 SR 信息。UE 只有在请求上行 PUSCH 资源的时候，才需要发送 SR，其它时候 UE 不发送 SR 以节约电量和减少干扰。因此与 HARQ 确认信息不同，并没有明确的比特用于发送 SR，而是通过分配给 UE 的用于 SR 传输的 PUCCH 1 资源上是否存在能量来表示是否存在 SR。

PUCCH format 1a 只传输 1 比特的 ACK/NACK 信息，使用 BPSK 调制，对应单小区、下行单 codeword 的场景。如果 UE 同时需要发送 SR，还可以携带 SR 信息。

PUCCH format 1b 传输 2 比特的 ACK/NACK 信息，使用 QPSK 调制，对应单小区、下行双 codeword 的场景。如果 UE 同时需要发送 SR，还可以携带 SR 信息。

表 8-5：PUCCH format 1/1a/1b

PUCCH format	Modulation scheme	Number of coded bits per subframe	Information
1	N/A	N/A	SR
1a	BPSK	1	1-bit ACK/NACK with/without SR
1b	QPSK	2	2-bit ACK/NACK with/without SR

我们将 PUCCH format 1/1a/1b 使用的 PUCCH 资源统称为 PUCCH 1 资源，PUCCH 1 资源通过一个资源索引 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 表示。通过该索引，能够得到该资源所在的 RB（n_{PRB}），所使用的循环移位（$\alpha_{\tilde{p}}(n_{\mathrm{s}},l)$）和正交序列（$n_{\mathrm{oc}}^{(\tilde{p})}(n_{\mathrm{s}})$）。这里 $\tilde{p}$ 表示天线端口号，有时也写成 p，或根本就不写。

分配给某个 UE 的 SR 资源（$n_{\mathrm{PUCCH,SRI}}^{(1,p)}=n_{\mathrm{PUCCH}}^{(1,p)}$）是通过 IE: *SchedulingRequestConfig* 的 *sr-PUCCH-ResourceIndex* 字段配置的。该 SR 资源即为 PUCCH 1 资源，一旦 eNodeB 给某个 UE 分配了一个 PUCCH 1 资源用于发送 SR，则该资源周期性的每 n 个子帧（SR 的周期）出现一次，并在出现的那个子帧上被该 UE 独占，即使 UE 在该子帧上并不发送 SR。

对于 UE 只接入一个 serving cell（非载波聚合）的场景，用于发送 HARQ ACK/NACK 信息的 PUCCH format 1a/1b 所使用的 PUCCH 1 资源索引 $n_{PUCCH}^{(1,p)}$ 是通过一个以 PDCCH 的第一个 CCE（first CCE）作为入参的函数得到的，该 PDCCH 是用于调度与 HARQ ACK/NACK 信息对应的下行数据的（见 36.213 的 10.1.2.1 节和 10.1.3.1 节）。因此，并不需要在下行调度信息中明确地指出 PUCCH 1 资源的信息，从而降低了开销。

经过调制以后，PUCCH format 1a 和 PUCCH format 1b 都只生成一个调制信号 d_{HARQ} 。其星座图匹配方式如图 8-3 所示：

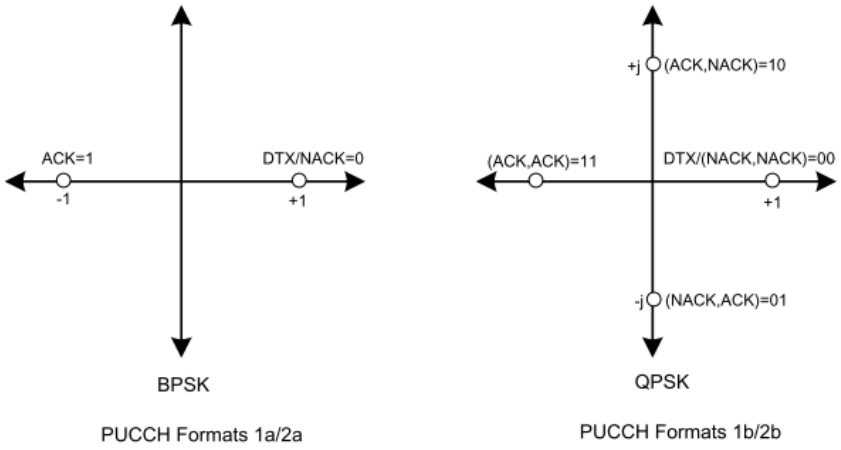


图 8-3：HARQ ACK/NACK 的星座图匹配方式

当 UE 只需要发送 SR 时，SR 与 NACK 使用相同的星座点。

当 UE 需要同时发送 SR 和 HARQ ACK/NACK 时，ACK/NACK 会在 SR 资源上发送，而不是在根据 first CCE 计算出的 PUCCH 1 资源（这里我们将其称为 ACK/NACK 资源）上发送。此时 ACK/NACK 资源上不发送任何东西，即是没有能量的。eNodeB 通过比较 ACK/NACK 资源和 SR 资源上检测到的能量，就知道 UE 是否发送了 SR。

首先，由于 PDSCH 传输和对应的 ACK/NACK 有固定的 timing 关系，所以 eNodeB 知道 UE 会在哪些子帧上发送 ACK/NACK。又由于对应的 PDCCH 也是由 eNodeB 发给 UE 的，所以 eNodeB 知道所使用的 ACK/NACK 资源。

其次，由于 SR 资源和 SR 的周期是由 eNodeB 分配给 UE 的，所以 eNodeB 知道 UE 可能会在哪些子帧上使用哪个 SR 资源来发送 SR。但 eNodeB 并不能确定 UE 是否在该子帧发送了 SR。

因此，eNodeB 知道 UE 在哪些子帧上可能同时发送 SR 和 ACK/NACK。如果在该子帧上，UE 同时发送了 SR 和 ACK/NACK，则二者会在 SR 资源上传输，eNodeB 发现 SR 资源上的能量明显比 ACK/NACK 资源上的能量高，就知道 UE 发送了 SR，并在 SR 资源上去接收 ACK/NACK。如果在该子帧上，UE 只发送了 ACK/NACK，则会在 ACK/NACK 资源上传输，此时 SR 资源上不发送任何东西，eNodeB 发现 ACK/NACK 资源上的能量明显比 SR 资源上的能量高，就知道 UE 没有发送 SR，并在 ACK/NACK 资源上去接收 ACK/NACK。

注意：（1）这里并不排除有其它更高级的联合编码 SR 和 ACK/NAKC 的方式；（2）在 PUCCH format 3 中，同时发送 SR 和 HARQ ACK/NACK 时，处理方式与 PUCCH format 1a/1b 是不同的，这会在 8.6 节予以介绍。

如果没有收到有效的 DL-SCH 相关的控制信令（即没有收到对应下行数据的 PDCCH），则 PUCCH 上什么也不传输（对应 DTX）。这样做除了可以避免占用不必要的 PUCCH 资源外，还可以使 eNodeB 在接收的 PUCCH 资源上检测 3 种状态：ACK，NACK，DTX。由于 NACK 和 DTX 的处理是不同的，所以这种检测是必要的：对于 NACK 来说，会重传额外的 parity bit 以增加冗余；而对于 DTX 来说，UE 很可能已经丢失了 systematic bit 的初始传输，所以比重传额外的 parity bit 更好的做法是重传 systematic bit（对于丢失初传的 PDCCH 的情况下更是如此）。

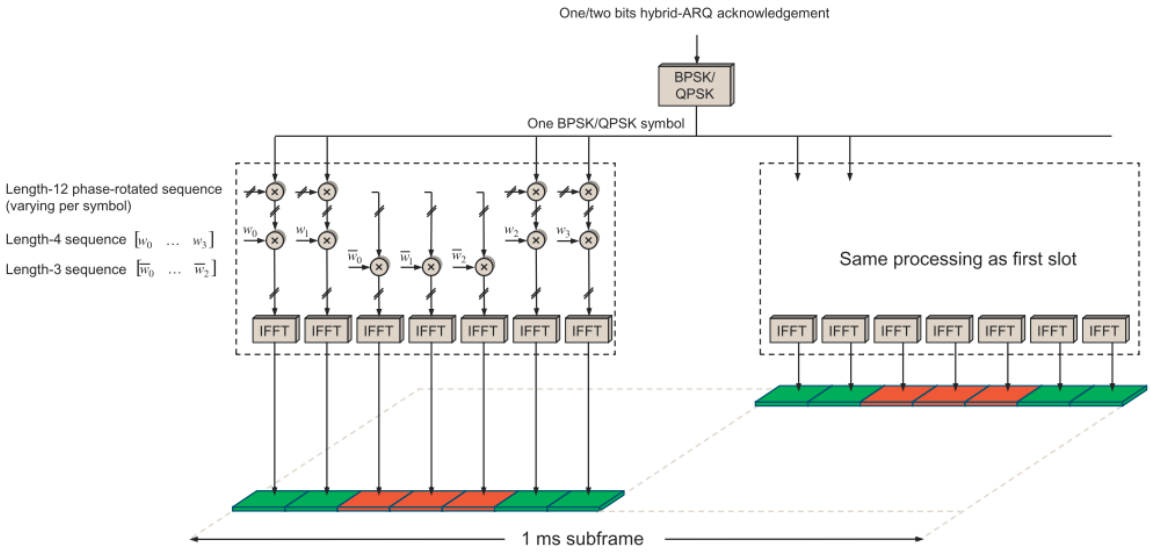


图 8-4：PUCCH format 1/1a/1b（正常的循环前缀）

对于正常的循环前缀而言，每个 slot 有 7 个 SC-FDMA 符号，每个 SC-FDMA 符号都会发送一个长为 12 的序列（对应 1 个 RB 有 12 个子载波）。前面已经介绍过，该序列是通过对一个长为 12 的小区特定的频域序列进行循环移位得到的（一个 RB 在频域内至多支持 12 个循环移位）。其中 3 个 SC-FDMA 符号被用于发送 DMRS，而剩下的 4 个 SC-FDMA 符号被用于发送 PUCCH format 1/1a/1b，如图 8-4 所示。

对于扩展的循环前缀而言，每个 slot 有 6 个 SC-FDMA 符号，其中中间的 2 个 SC-FDMA 符号被用于发送 DMRS，而剩下的 4 个 SC-FDMA 符号被用于发送 PUCCH format 1/1a/1b。

为了提高每个 RB 上可复用的 PUCCH format 1/1a/1b 的 UE 数，在时域上，调制符号会乘以一个长为 4 的正交序列（orthogonal sequence，对应 4 个可用于传输 PUCCH format 1/1a/1b 的 SC-FDMA 符号）。PUCCH format 1/1a/1b 只定义了 3 个而不是 4 个相互正交的正交序列，这是因为 DMRS 同样使用正交序列，以便 eNodeB 为特定的 UE 进行信道质量估计。但正常的循环前缀的情况下，一个 slot 中只有 3 个 SC-FDMA 符号可用于传输 DMRS，对应 3 个正交序列。为了满足 DMRS 的需求，因此 PUCCH format 1/1a/1b 只能使用 3 个相互正交的长度为 4 的正交序列。（正常的循环前缀下，一个 RB 在时域内至多支持 3 个正交序列；扩展的循环前缀下，这个数字为 2，道理同上）。

也就是说，正常的循环前缀下，最多允许 3 * 12 = 36（对于扩展的循环前缀而言，这个数值为 2 * 12 = 24）个 UE 在同一 RB 内发送 PUCCH format 1/1a/1b，彼此间通过循环移位和正交序列予以区分。

前面已经介绍过，对于 PUCCH 1/1a/1b 而言，在实际部署中，12 个循环移位并不一定都能够使用。PUCCH format 1/1a/1b 在每个 RB 上可用的循环移位数是由 $\Delta_{\text{shift}}^{\text{PUCCH}}$ 决定，并通过小区特定的参数 IE：*PUCCH-ConfigCommon* 的 *deltaPUCCH-Shift* 字段进行配置。*deltaPUCCH-Shift* 取值范围为（1，2，3），对应可用的循环移位数为 12，6 和 4，经过循环移位后的序列之间相互正交。

一个长为 12 的小区特定的频域序列至多有 12 种循环移位值（对应循环移位索引 0 ~ 11），如果 *deltaPUCCH-Shift* = 1，说明相邻的循环移位间的间隔为 1，对应可用的循环移位的数目为 12（12 / 1 = 12，对应图 8-5）；如果 *deltaPUCCH-Shift* = 2，说明相邻的循环移位间的间隔为 2，对应可用的循环移位的数目为 6（12 / 2 = 6，对应图 8-6）；如果 *deltaPUCCH-Shift* = 3，说明相邻的循环移位间的间隔为 3，对应可用的循环移位的数目为 4（12 / 3 = 4，对应图 8-7）。

注意：*deltaPUCCH-Shift* 只应用于 PUCCH format 1/1a/1b（包括混合 PUCCH 中的 PUCCH format 1/1a/1b），而不能应用于 PUCCH format 2/2a/2b。

	Orthogonal code sequence index, n_{oc}		
Cyclic shift index, n_{RS}^{PUCCH}	$n_{oc} = 0$	$n_{oc} = 1$	$n_{oc} = 2$
0	0	12	24
1	1	13	25
2	2	14	26
3	3	15	27
4	4	16	28
5	5	17	29
6	6	18	30
7	7	19	31
8	8	20	32
9	9	21	33
10	10	22	34
11	11	23	35

图 8-5：正常的循环前缀下，$\Delta_{\text{shift}}^{\text{PUCCH}} = 1$，1 个 RB 内共 36 个 PUCCH 1 资源

Cyclic shift index, n_{RS}^{PUCCH}	Orthogonal code sequence index, n_{oc}		
	$n_{oc}=0$	$n_{oc}=1$	$n_{oc}=2$
0	0		12
1		6	
2	1		13
3		7	
4	2		14
5		8	
6	3		15
7		9	
8	4		16
9		10	
10	5		17
11		11	

图 8-6：正常的循环前缀下，$\Delta_{shift}^{PUCCH}=2$，1 个 RB 内共 18 个 PUCCH 1 资源

Cyclic shift index, n_{RS}^{PUCCH}	Orthogonal code sequence index, n_{oc}		
	$n_{oc}=0$	$n_{oc}=1$	$n_{oc}=2$
0	0		
1		4	
2			8
3	1		
4		5	
5			9
6	2		
7		6	
8			10
9	3		
10		7	
11			11

图 8-7：正常的循环前缀下，$\Delta_{shift}^{PUCCH}=3$，1 个 RB 内共 12 个 PUCCH 1 资源

注意：图 8-5、图 8-6、图 8-7 分别来自[3]的 Table 16.5、Table 16.6、Table 16.7。但大家能注意到，Table 16.7 和图 8-7 是不一样的。Table 16.7 指示一个 RB 内只有 10 个资源索引，但实际上应该有 12 个，所以我认为原书中是不正确的，就改成了图 8-7 的样子。如果大家发现我理解错了，欢迎指正。

小结：一个用于发送 PUCCH format 1/1a/1b 的 RB，所包含的 PUCCH 1 资源数 $N^{(1)}_{PUCCH\ ,RB}$ ，也即可复用的 UE 数为：

$$N^{(1)}_{PUCCH\ ,RB} = c * P$$

其中，c 表示正交序列的数目，对于正常的循环前缀而言，其值为 3；对于扩展的循环前缀而言，其值为 2。P 表示可用的循环移位的数目，其值为 $N^{RB}_{sc} / \Delta^{\text{PUCCH}}_{\text{shift}}$ 。 N^{RB}_{sc} 的值固定为 12，表示 1 个 RB 频域上包含了 12 个子载波。

如果某个小区共配置了 $N^{(1)}_{RB}$ 个 RB 用于发送 PUCCH format 1/1a/1b，则该小区可用的 PUCCH format 1 资源总数（这里不包含混合 PUCCH）为 $N^{(1)}_{RB} * N^{(1)}_{PUCCH\ ,RB}$ 。

前面已经提到过，通过 PUCCH 1 资源索引 $n^{(1,\tilde{p})}_{\text{PUCCH}}$ ，能够得到该资源所在的 RB（ n_{PRB} ），所使用的循环移位（ $\alpha_{\tilde{p}}(n_s,l)$ ）和正交序列（ $n^{(\tilde{p})}_{\text{oc}}(n_s)$ ），即该索引唯一地指定一个三元组（ n_{PRB} ， $\alpha_{\tilde{p}}(n_s,l)$ ， $n^{(\tilde{p})}_{\text{oc}}(n_s)$ ）。这里我将 36.211 的 5.4.1 节和 5.4.3 节用于计算该三元组的公式串起来，并添加了一些注释，以方便大家学习。

如果大家仔细对照公式来看协议的话，可以看出之前介绍的知识点基本都可以从这些公式反映出来。

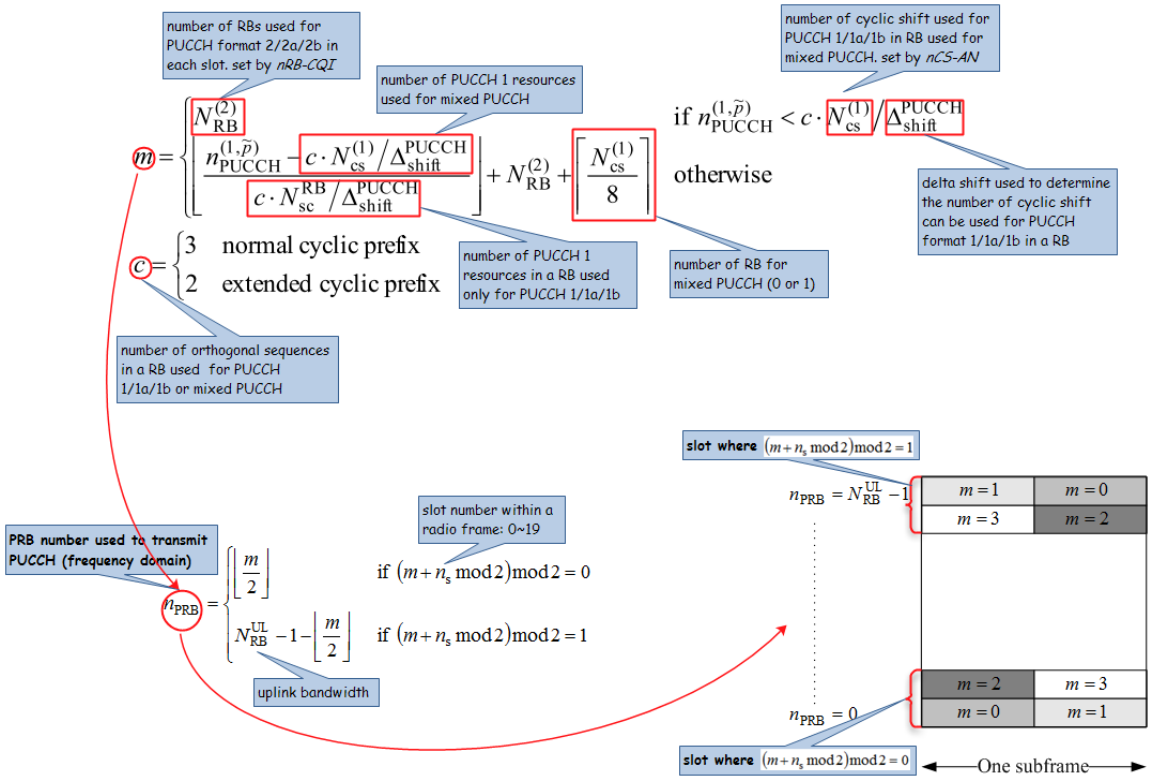


图 8-8：通过 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 确定 PUCCH 1 资源所在的 RB

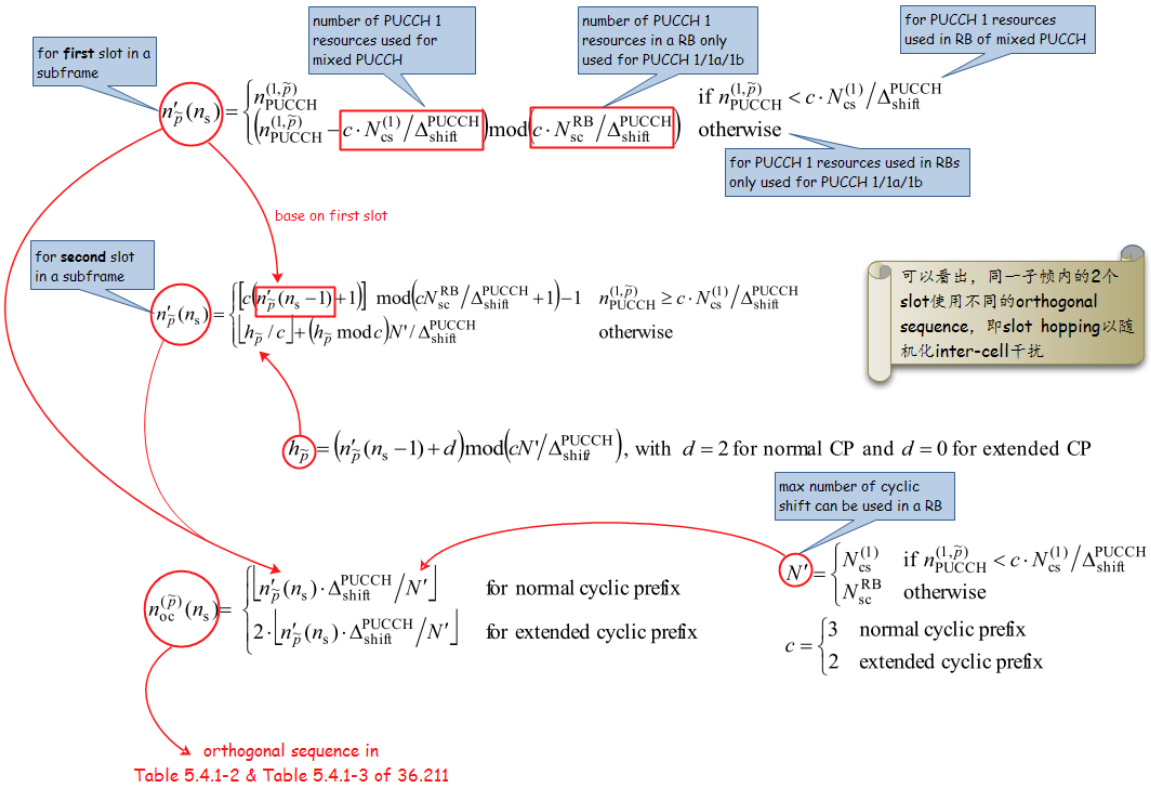


图 8-9：通过 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 确定 PUCCH 1 资源所使用的正交序列

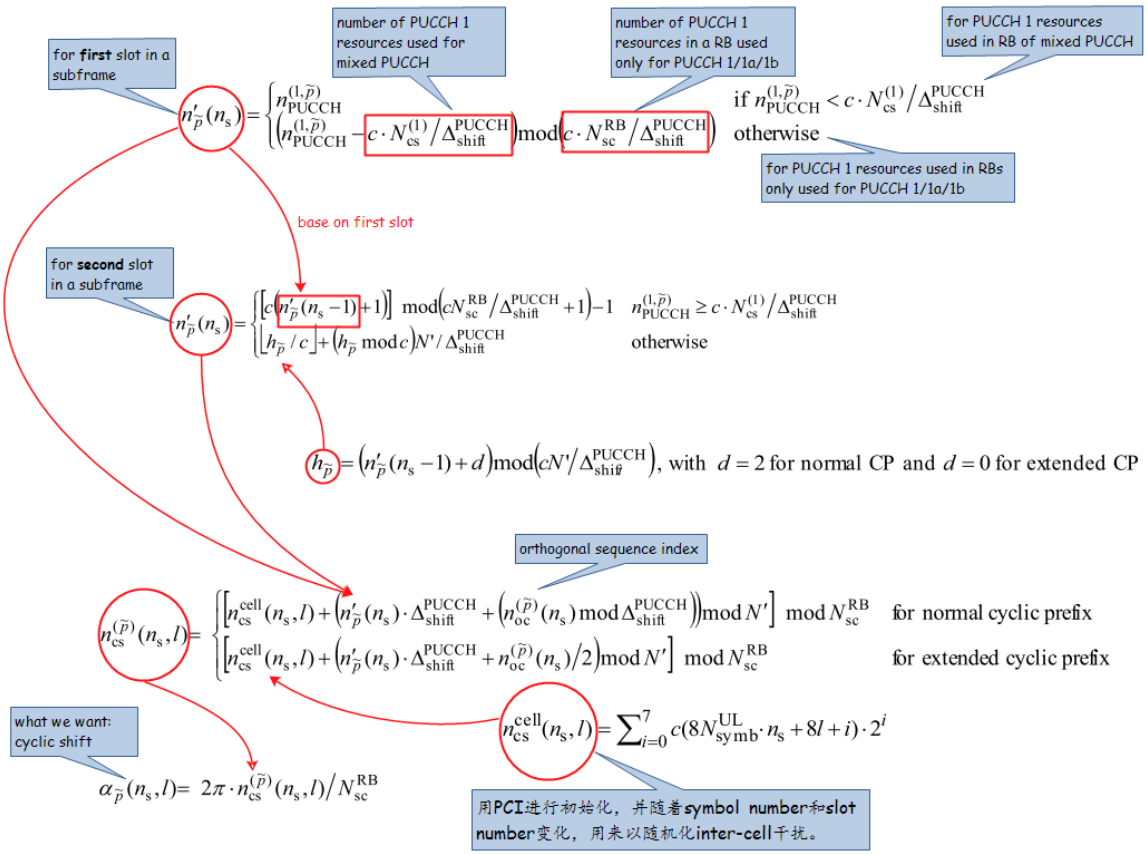


图 8-10：通过 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 确定 PUCCH 1 资源所使用的循环移位

【参考资料】

[1] TS 36.211 的 5.4 节 Physical uplink control channel

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.4 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.3 节

8.4 PUCCH format 2/2a/2b

本节主要介绍 PUCCH format 2/2a/2b，主要内容包括：PUCCH format 2/2a/2b 携带的信息、PUCCH format 2/2a/2b 如何调制到 RB 中、PUCCH 2 资源表示、一个 RB 包含多少个 PUCCH 2 资源的计算、如何解读 36.211 的 5.4.2 节和 5.4.3 节的那些公式等等。

PUCCH format 2：对于正常的循环前缀而言，只携带 20 比特的 CSI 信息；对于扩展的循环前缀而言，不仅携带 20 比特的 CSI 信息，还携带 1 比特或 2 比特的 HARQ ACK/NACK 信息。二者均采用 QPSK 调制。

PUCCH format 2a：携带 20 比特的 CSI 信息（QPSK 调制）和 1 比特的 ACK/NACK 信息（对应单小区、下行单 codeword 的场景，使用 BPSK 调制）。

PUCCH format 2b：携带 20 比特的 CSI 信息（QPSK 调制）和 2 比特的 ACK/NACK 信息（对应单小区、下行双 codeword 的场景，使用 QPSK 调制）。

PUCCH format 2a/2b 只支持正常的循环前缀。

表 8-6：PUCCH format 2/2a/2b

PUCCH format	Modulation scheme	Number of coded bits per subframe	Information
2	QPSK	20	20-bit CSI
2	QPSK	20	20-bit CSI + 1 or 2-bit ACK/NACK (only for extended CP)
2a	QPSK + BPSK	21	20-bit CSI + 1-bit ACK/NACK (only for normal CP)
2b	QPSK + QPSK	22	20-bit CSI + 2-bit ACK/NACK (only for normal CP)

PUCCH format 2/2a/2b 只用于发送周期性的 CQI 上报；而 PUSCH 用于非周期性的 CQI 上报。如果在某一个周期性发送 CQI 的子帧上，UE 被调度了 PUSCH 资源，则周期性的 CQI 会在 PUSCH 上传输，而不是在 PUCCH 上传输（见参考资料[3]的 10.2.1 节）。

在载波聚合中，每个载波单元都需要发送各自的 CSI，因此存在多个周期性 CSI 上报的情况。通过在时域上将每个载波单元的 CSI 错开，保证不同的载波单元在不同的子帧上上报 CSI，从而避免了 CSI 的冲突。

eNodeB 可以控制 UE 在哪个子帧上发送 SR，也可以控制 UE 在哪个子帧上发送 CSI，因此 eNodeB 通过特定的配置，可以避免 UE 在同一子帧发送 SR 和 CSI。但如果无法通过特定的配置保证 UE 在不同的子帧发送 SR 和 CSI 时，UE 会在冲突发生的时候，选择发送 SR，而丢弃 CSI。这是因为缺少一个 CSI，只会影响调度和速率匹配的精确度；但缺少一个 SR，UE 就无法发送上行数据。也就是说，UE 不会同时发送 SR 和 CSI，如果二者产生冲突，则发送 SR，丢弃 CSI。

UE 可以同时发送 CSI 和 HARQ ACK/NACK，对应 PUCCH format 2a/2b。但 eNodeB 也可以将 UE 配置成在同一子帧中需要同时发送 HARQ ACK/NACK 和 CSI 时，只发送 HARQ ACK/NACK，而丢弃 CSI，这是通过 IE：*CQI-ReportPeriodic* 的 *simultaneousAckNackAndCQI* 字段来配置的。在载波聚合中，如果 UE 在同一子帧中需要同时发送 CSI 与 PUCCH format 3，则只发送 PUCCH format 3，而丢弃 CSI。

我们将 PUCCH format 2/2a/2b 使用的 PUCCH 资源统称为 PUCCH 2 资源，PUCCH 2 资源通过一个资源索引 $n_{\mathrm{PUCCH}}^{(2,\tilde{p})}$ 表示。通过该索引，能够得到该资源所在的 RB（ n_{PRB} ）和所使用的循环移位（ $\alpha_{\tilde{p}}(n_s,l)$ ）。这里 $\tilde{p}$ 表示天线端口号，有时也写成 p，或根本就不写。

分配给某个 UE 的 CQI 资源（ $n_{\mathrm{PUCCH}}^{(2,p)}$ ）是通过 UE 级的配置参数 IE: *CQI-ReportPeriodic* 的 *cqi-PUCCH-ResourceIndex* 字段配置的。

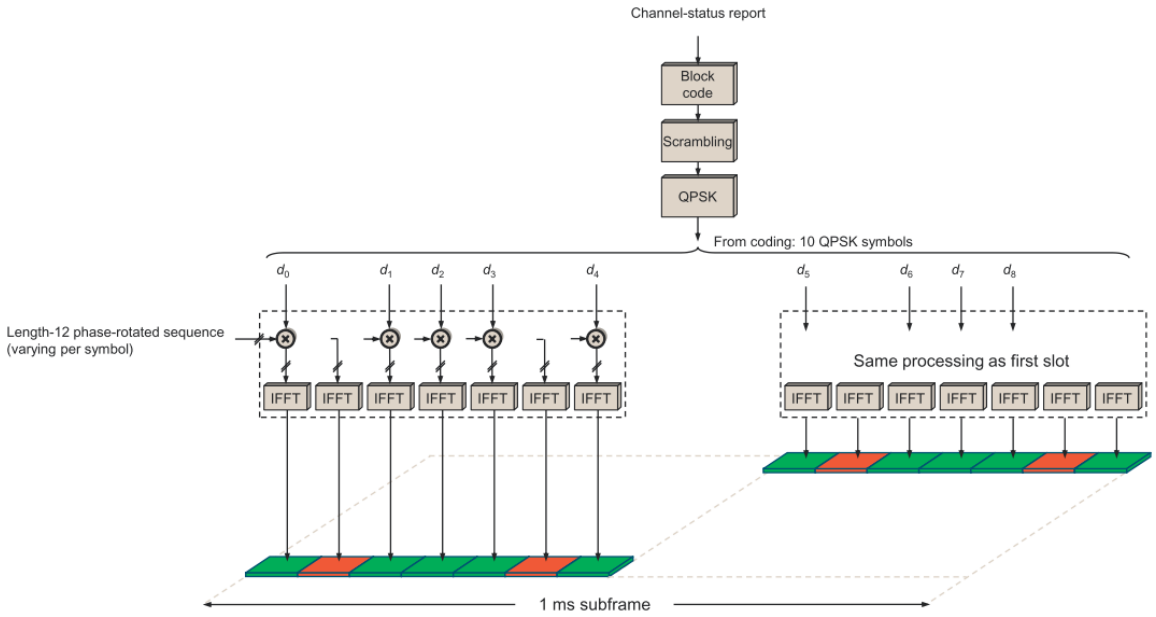


图 8-11：PUCCH format 2（正常的循环前缀，图中漏了一个 d9）

从图 8-11 可以看出，对于正常的循环前缀而言，每个 slot 有 7 个 SC-FDMA 符号，用于传输 PUCCH format 2 的 RB，其每个 slot 的第 2 和第 6 个 SC-FDMA 符号用于传输 DMRS，而剩余的 5 个 SC-FDMA 符号被用于传输 PUCCH format 2。

对于扩展的循环前缀而言，每个 slot 只有 6 个 SC-FDMA 符号，此时只有 1 个 SC-FDMA 符号用于传输 DMRS（第 4 个 SC-FDMA 符号，见图 8-14），而剩余的 5 个 SC-FDMA 符号用于传输 PUCCH format 2。

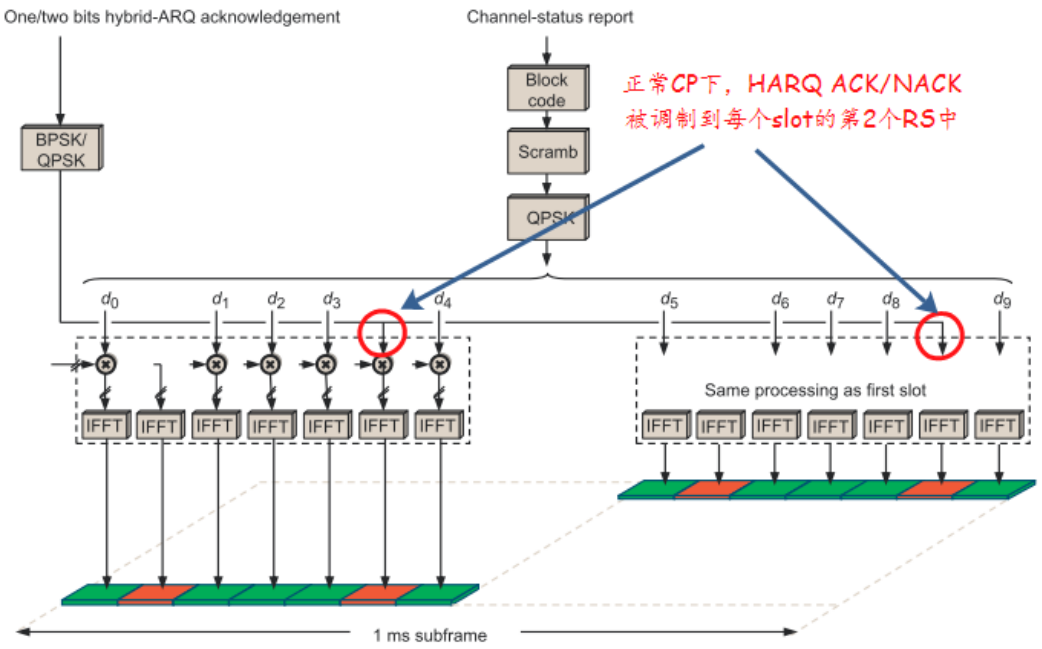


图 8-12：PUCCH format 2a/2b（正常的循环前缀）

从图 8-12 可以看出，与 PUCCH format 2 不同，PUCCH format 2a/2b 多了 1 比特或 2 比特的 HARQ ACK/NACK 信息。1 比特 HARQ ACK/NACK 使用 BPSK 调制，2 比特 HARQ ACK/NACK 使用 QPSK 调制，最终都得到 1 个调制符号 d_{HARQ}。其中 ACK 被编码为‘1’，NACK 被编码为 0。

在正常的循环前缀情况下，HARQ 信息 d_{HARQ} 会被调制到每个 slot 的第 2 个 RS 中去。HARQ ACK/NACK 的星座图匹配方式与 8.3 节介绍的方式相同。

eNodeB 如何解码调制在第二个 RS 中的 HARQ 确认信息呢？在低多普勒（low-Doppler）环境下，一个 slot（0.5 ms）内的信道变化很小，这时可以只使用第一个 RS 作为相位参考。而为了提高 CSI 检测的信道估计质量，另一种可行的方式是使用每个 slot 的第一个 RS 作为信道估计，来解码第二个 RS 中的 HARQ 确认信息。一旦 HARQ 确认信息被成功解码后，将第二个 RS 中的调制信息去除，然后就可以使用无 HARQ 情况下的解码方式进行解码 CSI 了。

前面说的两种方式都只适用于低多普勒环境，对于高多普勒环境下，由于信道变化很快，HARQ 确认信息和 CSI 需要联合解码。（详见参考资料[2]的 14.4.1.4 节和[3]的 16.3.4.1 节）

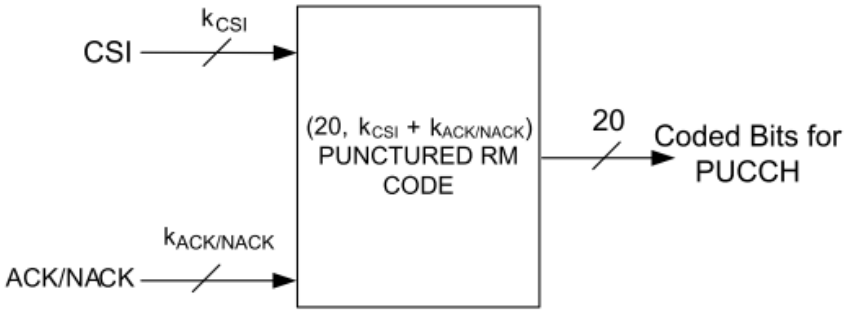


图 8-13：扩展的循环前缀下，HARQ ACK/NACK 与 CSI 的联合编码

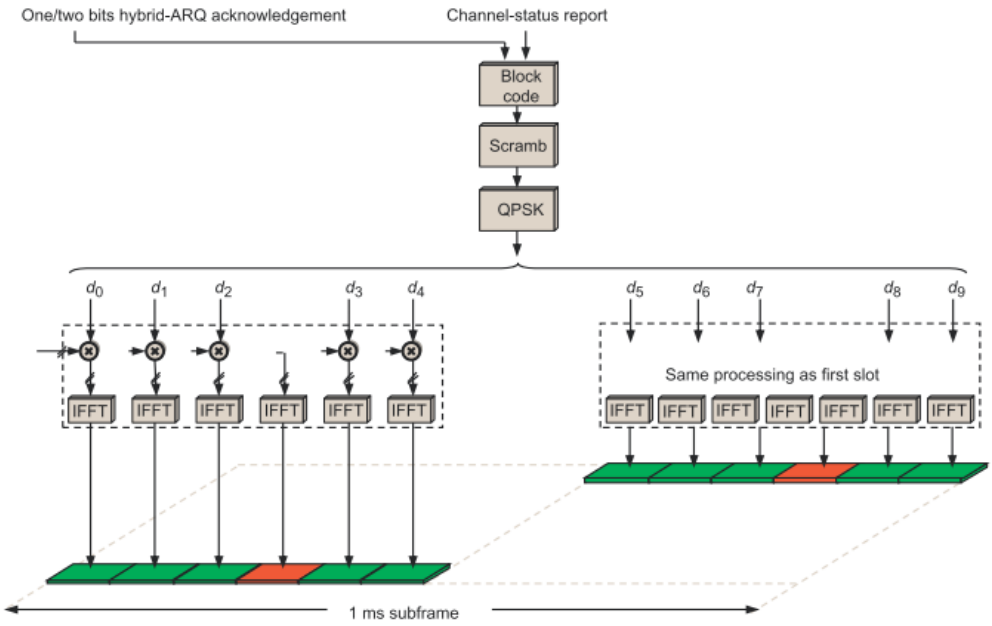


图 8-14：PUCCH format 2（扩展的循环前缀）

在扩展的循环前缀情况下（此时每个 slot 只有 1 个 SC-FDMA 符号用于 DMRS），1 比特或 2 比特的 ACK/NACK 将与 CSI 联合编码生成一个（20, k_{CQI} + $k_{ACK/NACK}$）的 Reed-Muller-based block code（如图 8-13 所示），得到的 20 比特信息将按照图 8-14 所示的方式传输。（具体见 36.212 的 5.2.3.4 节的介绍）

PUCCH format 2/2a/2b 与 PUCCH format 1/1a/1b 使用相同的小区特定的 DMRS 序列，但对于 PUCCH format 2/2a/2b 而言，同一 RB 上不同的 PUCCH 2 资源只能通过循环移位来区分，而不使用正交序列来区分。

一个 RB 在频域内至多支持 12 个循环移位，也就是说，一个 RB 中包含了 12 个 PUCCH 2 资源。（**注意**：从协议的角度上看，对于 PUCCH format 2/2a/2b 而言，12 个循环移位索引都可以使用，并不受 *deltaPUCCH-Shift* 字段影响）

某小区内可用于发送 PUCCH format 2/2a/2b 的 RB 数使用 $N_{\mathrm{RB}}^{(2)}$ 表示，$N_{\mathrm{RB}}^{(2)}$ 是通过 IE：*PUCCH-ConfigCommon* 的 *nRB-CQI* 字段来配置的。

小结：一个用于发送 PUCCH format 2/2a/2b 的 RB，所包含的 PUCCH format 2 资源数 $N_{PUCCH,RB}^{(2)}=12$。

如果某个小区共配置了 $N_{RB}^{(2)}$ 个 RB 用于发送 PUCCH format 2/2a/2b，则该小区可用的 PUCCH format 2 资源总数（这里不计算混合 PUCCH）为 $12*N_{RB}^{(2)}$。

注意：在实际部署中，对于 PUCCH format 2/2a/2b，通常并不是 12 个循环移位（对应 12 个 PUCCH 2 资源）都能够使用，这取决于 eNodeB 的实现。此时只要 eNodeB 在分配 PUCCH 2 资源时，在同一子帧内，不将同一 RB 内相邻的 PUCCH 2 资源（此时循环移位也相邻）分配给不同的 UE 就行了。这是由 eNodeB 在分配 PUCCH 2 资源时保证的，在协议中并没有体现。

前面已经提到过，通过 PUCCH 2 资源索引 $n_{\mathrm{PUCCH}}^{(2,\tilde{p})}$，能够得到该资源所在的 RB（$n_{\mathrm{PRB}}$）和所使用的循环移位（$\alpha_{\tilde{p}}(n_{\mathrm{s}},l)$），即该索引唯一地指定一个二元组（$n_{PRB}$，$\alpha_{\tilde{p}}(n_{\mathrm{s}},l)$）。这里我将 36.211 的 5.4.2 节和 5.4.3 节用于计算该二元组的公式串起来，并添加了一些注释，以方便大家学习。

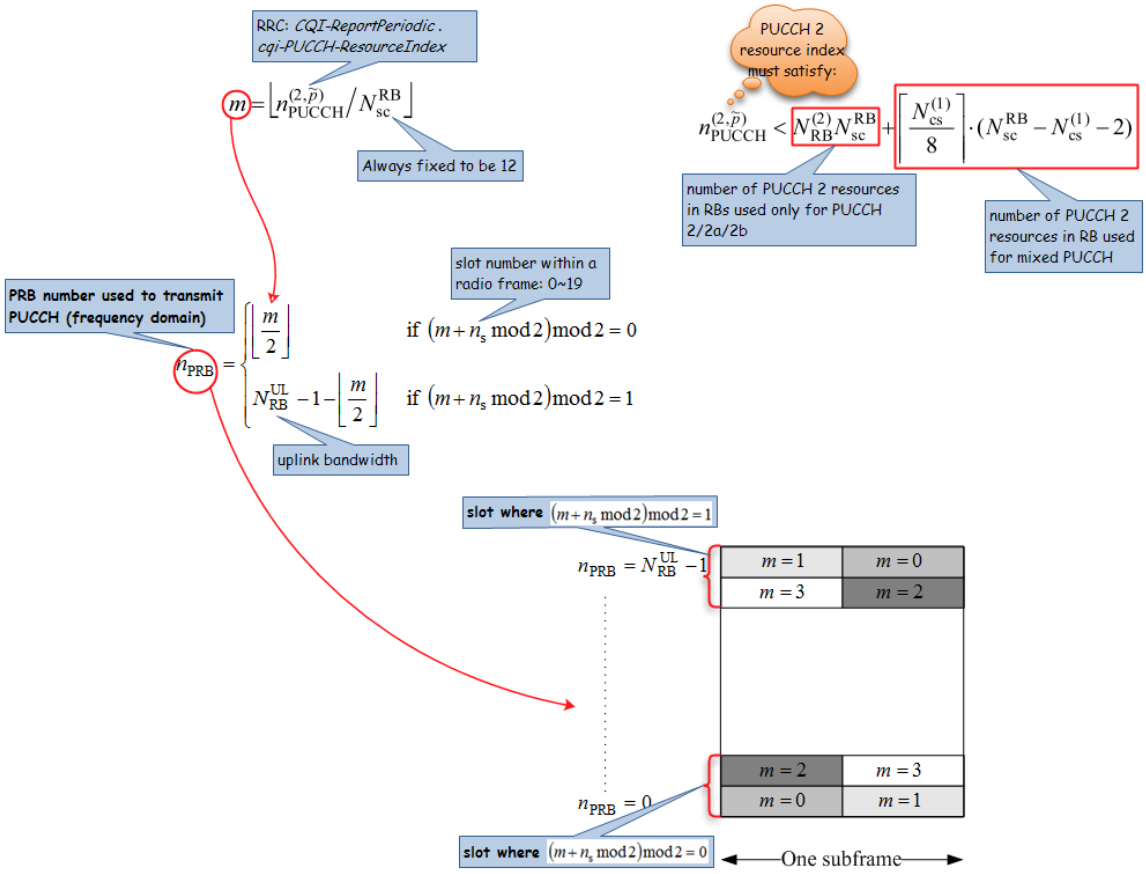


图 8-15：通过 $n_{PUCCH}^{(2,\tilde{p})}$ 确定 PUCCH 2 资源所在的 RB

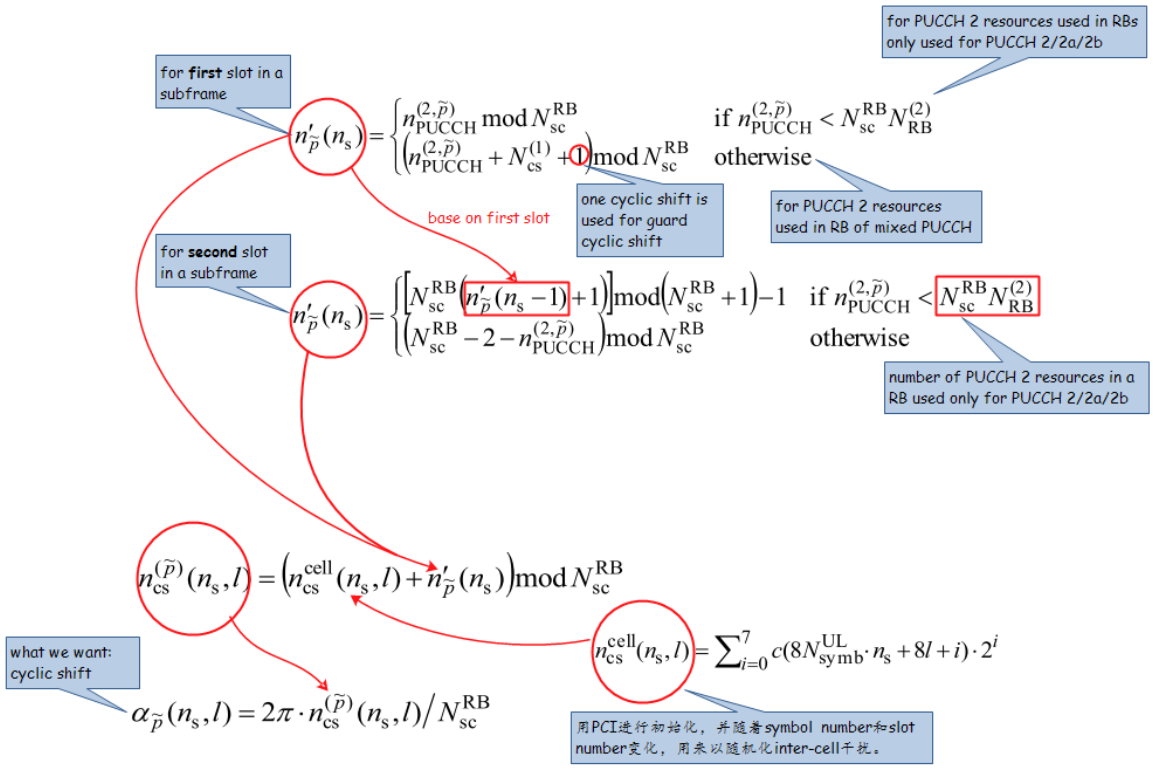


图 8-16：通过 $n^{(2,\tilde{p})}_{\mathrm{PUCCH}}$ 确定 PUCCH 2 资源所使用的循环移位

【参考资料】

[1] TS 36.211 的 5.4 节 Physical uplink control channel

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.4 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.3 节

8.5 PUCCH format 1/1a/1b 与 PUCCH format 2/2a/2b 混合的 PUCCH RB

对于较小的上行系统带宽，例如 1.4 MHz，如果用于发送 CSI（PUCCH format 2/2a/2b）和 ACK/NACK（PUCCH format 1/1a/1b）的 PUCCH 使用不同的 RB，控制信令的开销就会变得很大（1.4 MHz，共 6 个 RB，此时要分出 2 个 RB 给 PUCCH 用）。因此，LTE 支持 PUCCH format 1/1a/1b 和 PUCCH format 2/2a/2b 混合的 PUCCH RB，以减少控制信令的开销。这是通过将一个 RB 内的部分循环移位分配给 PUCCH format 1/1a/1b，剩余的部分循环移位分配给 PUCCH format 2/2a/2b 来实现的。

对于 10MHz 或 20MHz 上行系统带宽而言，通常不使用混合 PUCCH。

$N_{cs}^{(1)}$ 表示混合 PUCCH（PUCCH format 1/1a/1b 和 PUCCH format 2/2a/2b 混合的 PUCCH，或称为 mixed PUCCH）的 RB 中用于 PUCCH format 1/1a/1b 的循环移位（cyclic shift）的个数，取值范围为{0, 1, ..., 7}。如果 $N_{cs}^{(1)} = 0$，则不存在混合 PUCCH。一个子帧至多只有一个 RB 可用于传输混合 PUCCH。$N_{cs}^{(1)}$ 是通过小区级的配置参数 IE：*PUCCH-ConfigCommon* 中的 *nCS-AN* 字段来配置的。

混合 PUCCH 中的可用于 PUCCH format 1/1a/1b 的循环移位数同样受到 Δ_{shift}^{PUCCH} 的影响，且 $N_{cs}^{(1)}$ 必须是 Δ_{shift}^{PUCCH} 的整数倍。以 $\Delta_{shift}^{PUCCH} = 2$，$N_{cs}^{(1)} = 6$ 为例，混合 PUCCH 中可用于 PUCCH format 1/1a/1b 的循环移位数为 3。

剩余的用于 PUCCH format 2/2a/2b 的循环移位数为 $N_{sc}^{RB} - N_{cs}^{(1)} - 2$。减 2 的原因是有 2 个循环移位索引要作为 guard cyclic shift，不能用于发送 PUCCH。以 $\Delta_{shift}^{PUCCH} = 2$，$N_{cs}^{(1)} = 6$ 为例，混合 PUCCH 中可用于 PUCCH format 2/2a/2b 的循环移位数为 12 - 6 -2 = 4。

表 8-7 是混合 PUCCH RB 中的 12 个循环移位分类。

表 8-7：混合 PUCCH RB 中的循环移位分类

Cyclic shift index	Cyclic shift index allocation
0 1 2 . . . $N_{cs}^{(1)}$ -1	Format 1/1a/1b (HARQ ACK / NACK, SR) cyclic shifts
$N_{cs}^{(1)}$	**Guard cyclic shift**
$N_{cs}^{(1)}$ +1 . . .	Format 2 / 2a / 2b (CSI) cyclic shifts

10	
11	**Guard cyclic shift**

注意：表 8-7 与参考资料[3]中的 Table 16.8 是不同的。 [3]中的 Table 16.8 认为 $N_{cs}^{(1)}+1$ 是 guard cyclic shift，而我认为循环移位索引 $N_{cs}^{(1)}$ 才是 guard cyclic shift。我得出这个结论的依据是 36.211 的 5.4 节中关于 $N_{cs}^{(1)}$ 的描述：

The variable $N_{cs}^{(1)}$ denotes the number of cyclic shift used for PUCCH formats 1/1a/1b in a resource block used for a mix of formats 1/1a/1b and 2/2a/2b.

$N_{cs}^{(1)}$ 表示混合 PUCCH（PUCCH format 1/1a/1b 和 PUCCH format 2/2a/2b 混合的 PUCCH）的 RB 中用于 PUCCH format 1/1a/1b 的循环移位（cyclic shitf）的个数。而循环移位索引从 0 开始，所以范围应该 0~ $N_{cs}^{(1)}$ -1。从 36.211 的 5.4.1 节和 5.4.2 节中计算混合 PUCCH 中 PUCCH 1 或 PUCCH 2 资源的循环移位时使用的公式也能够看出端倪。

小结：混合 PUCCH 中，PUCCH 1 的资源数为 $\left\lceil \frac{N_{cs}^{(1)}}{8} \right\rceil . c \cdot N_{cs}^{(1)} / \Delta_{shift}^{PUCCH}$，PUCCH 2 的资源数为 $\left\lceil \frac{N_{cs}^{(1)}}{8} \right\rceil \cdot (N_{sc}^{RB} - N_{cs}^{(1)} - 2)$。如果 $N_{cs}^{(1)} = 0$，则不存在混合 PUCCH。

结合 8.3 节的介绍，某个小区可用的 PUCCH 1 资源总数（包含混合 PUCCH）为 $N_{RB}^{(1)} * N_{PUCCH,RB}^{(1)} + \left\lceil \frac{N_{cs}^{(1)}}{8} \right\rceil . c \cdot N_{cs}^{(1)} / \Delta_{shift}^{PUCCH}$。某个小区可用的 PUCCH 2 资源总数（包含混合 PUCCH）为 $12 * N_{RB}^{(2)} + \left\lceil \frac{N_{cs}^{(1)}}{8} \right\rceil \cdot (N_{sc}^{RB} - N_{cs}^{(1)} - 2)$

关于混合 PUCCH 中 PUCCH 1 和 PUCCH 2 资源的计算，可以参见 8.3 节（PUCCH format 1/1a/1b）的图 8-8、图 8-9、图 8-10 以及 8.4 节（PUCCH format 2/2a/2b）的图 8-15、图 8-16。

【参考资料】

[1] TS 36.211 的 5.4 节 Physical uplink control channel

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.4 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.3.6 节

8.6 PUCCH format 3

在下行载波聚合（Carrier Aggregation，CA）中，多个载波单元（Component Carrier，CC，对应一个 serving cell）可能在同一个 TTI 内存在下行数据传输。**PUCCH 只能在 PCell 上传输**，且**每个 UE 在每个子帧上只能发送一个 PUCCH**，因此 UE 可能需要在 PCell 的一个上行子帧上，使用一个 PUCCH，对所有 serving cell 相应的下行子帧中传输的 PDSCH 进行 HARQ ACK/NACK 回应。

对于 **FDD** 而言，在一个上行子帧中，最多需要反馈 10 个 ACK/NACK 比特的信息。（一个 UE 至多可配置 5 个 serving cell，如果每个 serving cell 都配置了 MIMO，则每个 serving cell 需要 2 个 ACK/NACK 比特，所以至多需要传输 5 * 2 =10 个 ACK/NACK 比特的信息）

对于 **TDD** 而言，在一个上行子帧中，可能需要反馈多于 20 个 ACK/NACK 比特的信息。（例如：一个 UE 至多可配置 5 个 serving cell，如果每个 serving cell 都配置了 MIMO，则每个 serving cell 需要 2 个 ACK/NACK 比特。在 TDD 中，N 个下行子帧数据可能需要在同一个上行子帧进行 ACK/NACK 回应（我们将 N 称为 ACK/NACK bundling window 的大小，见 36.213 的 Table 10.1.3.1-1。例如：在 TDD 2 下的子帧 2，N = 4，此时所需要的 ACK/NACK 比特信息的数量为 5 * 2 * 4 = 40）

Table 10.1.3.1-1: Downlink association set index K：$\{k_0, k_1, \cdots k_{M-1}\}$ for TDD

UL-DL Configuration	Subframe *n*									
	0	1	2	3	4	5	6	7	8	9
0	-	-	6	-	4	-	-	6	-	4
1	-	-	7, 6	4	-	-	-	7, 6	4	-
2	-	-	8, 7, 4, 6	-	-	-	-	8, 7, 4, 6	-	-
3	-	-	7, 6, 11	6, 5	5, 4	-	-	-	-	-
4	-	-	12, 8, 7, 11	6, 5, 4, 7	-	-	-	-	-	-
5	-	-	13, 12, 9, 8, 7, 5, 4, 11, 6	-	-	-	-	-	-	-
6	-	-	7	7	5	-	-	7	7	-

Rel-8 中，PUCCH 都是针对单个 serving cell 设计的，无法满足载波聚合的需求。为了解决这个问题，Rel-10 新增了 2 个 PUCCH format 以支持载波聚合：

- PUCCH format 1b with channel selection：使用 PUCCH format 1b 携带的 2 比特信息，并选择几个特定 PUCCH 1 资源的某一个，来指示所要发送的 ACK/NACK 信息。其只支持不超过 4 个 ACK/NACK 比特信息且不多于 2 个 serving cell 的场景；

- PUCCH format 3：可支持至多 5 个 serving cell 且 UE 在每个 serving cell 都配置了 MIMO 的场景。**注意**：PUCCH format 3 也能够支持 2 个 serving cell 的场景，具体是使用 PUCCH format 3 还是 PUCCH format 1b with channel selection 取决于 eNodeB 对 *pucch-Format-r10* 的配置。

载波聚合下，使用这两种 format 中的哪一个是通过 IE：*PUCCH-ConfigDedicated-v1020* 来指定的。

```
PUCCH-ConfigDedicated-v1020 ::=     SEQUENCE {
    pucch-Format-r10                    CHOICE {        ----这是个CHOICE，因此对于某个UE来说，只能在PUCCH format 3和PUCCH format 1b with channel selection中选一个。
        format3-r10                                 SEQUENCE {
            n3PUCCH-AN-List-r10 SEQUENCE (SIZE (1..4)) OF INTEGER (0..549)  OPTIONAL,   -- Need ON
            twoAntennaPortActivatedPUCCH-Format3-r10        CHOICE {
                release                                         NULL,
                setup                                           SEQUENCE {
                    n3PUCCH-AN-ListP1-r10   SEQUENCE (SIZE (1..4)) OF INTEGER (0..549)
                }
            }                                                                   OPTIONAL    -- Need ON
        },
        channelSelection-r10                SEQUENCE {
            n1PUCCH-AN-CS-r10                   CHOICE {
                release                             NULL,
                setup                               SEQUENCE {
                    n1PUCCH-AN-CS-List-r10              SEQUENCE (SIZE (1..2)) OF N1PUCCH-AN-CS-r10
                }
            }                                                                   OPTIONAL    -- Need ON
        }
    }                                                                           OPTIONAL,   -- Need OR
    twoAntennaPortActivatedPUCCH-Format1a1b-r10     ENUMERATED {true}       OPTIONAL,   -- Need OR
    simultaneousPUCCH-PUSCH-r10                     ENUMERATED {true}       OPTIONAL,   -- Need OR
    n1PUCCH-AN-RepP1-r10                            INTEGER (0..2047)       OPTIONAL    -- Need OR
}

N1PUCCH-AN-CS-r10   ::= SEQUENCE (SIZE (1..4)) OF INTEGER (0..2047)
```

本节只介绍 PUCCH 3 资源的分配，不涉及 PUCCH format 1b with channel selection，也不会介绍 UE 如何使用 PUCCH 来回应 ACK/NACK，这会在第 4 篇介绍 HARQ 时予以详细说明。

PUCCH format 3 支持 48 编码比特（而不是真正反馈的 ACK/NACK 比特）。真正反馈的 ACK/NACK 比特**取决于配置的 serving cell 数、每一个 serving cell 使用的传输模式（transmission mode）**，在 TDD 中还与 ACK/NACK bundling window 的大小相关：

- 对于 FDD 而言，支持最多 10 个 ACK/NACK 比特的负载；

- 对于 TDD 而言，支持最多 20 个 ACK/NACK 比特的负载。如果多个下行子帧对应一个上行子帧，且需要反馈的 ACK/NACK 比特数大于 20，则每一个 serving cell 的下行子帧的 2 个 codeword 需要做 ACK/NACK 比特的“spatial bundling”处理：同一 serving cell 的同一下行子帧的 2 个 codeword 对应的 ACK/NACK 做逻辑与（logical AND）操作。

Rel-10 中，PUCCH format 3 可以携带最多 21 比特的信息（例如：码率为 0.4375），其中 20 比特用于 ACK/NACK 信息，剩余的 1 比特用于发送 SR。

表 8-8： PUCCH format 3

PUCCH format	Modulation scheme	Number of coded bits per subframe	Information
3	QPSK	48	For FDD, up to 10-bit ACK/NACK; For TDD, up to 20-bit ACK/NACK
3	QPSK	48	For FDD, up to 10-bit ACK/NACK + 1-bit SR; For TDD, up to 20-bit ACK/NACK + 1-bit SR

TDD 中，每个 serving cell 需要传输多少个 ACK/NACK 比特见 36.213 的 7.3 节，图 8-17 举了一个例子，仅供参考。

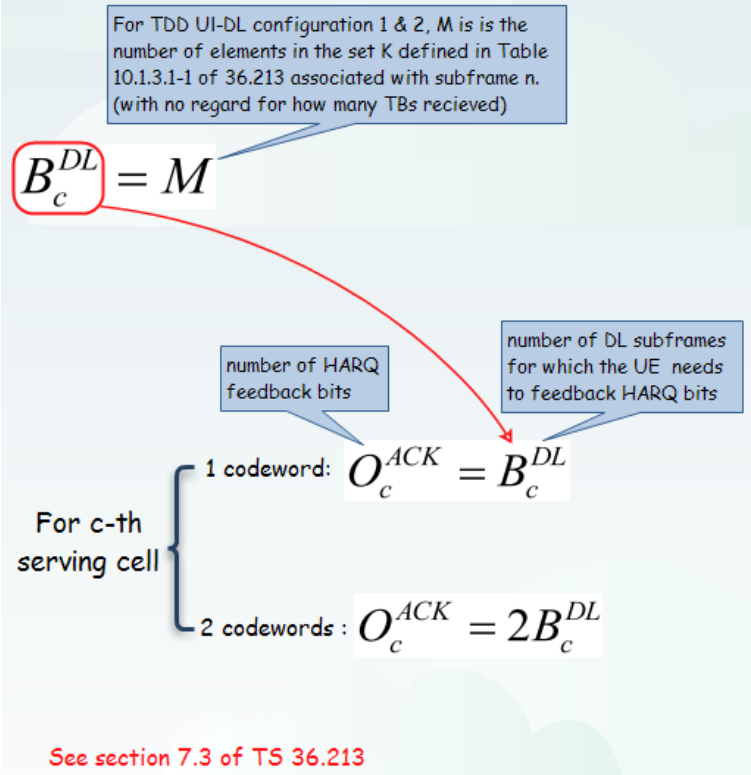


图 8-17：TDD 中，针对某个 serving cell，UE 在某个上行子帧需要发送的 ACK/NACK 比特数

所有 ACK/NACK 比特会按照下行 serving cell index 的递增顺序连接在一起的。如果传输模式为 TM 1/2/5/6/7（1 codeword），则每个 serving cell 只需要 1 比特的 HARQ ACK/NACK 信息；如果传输模式为 TM 3/4/8/9（MIMO，2 codewords），则每个 serving cell 需要 2 比特的 HARQ ACK/NACK 信息。（详见 36.212 的 5.2.3.1 节）

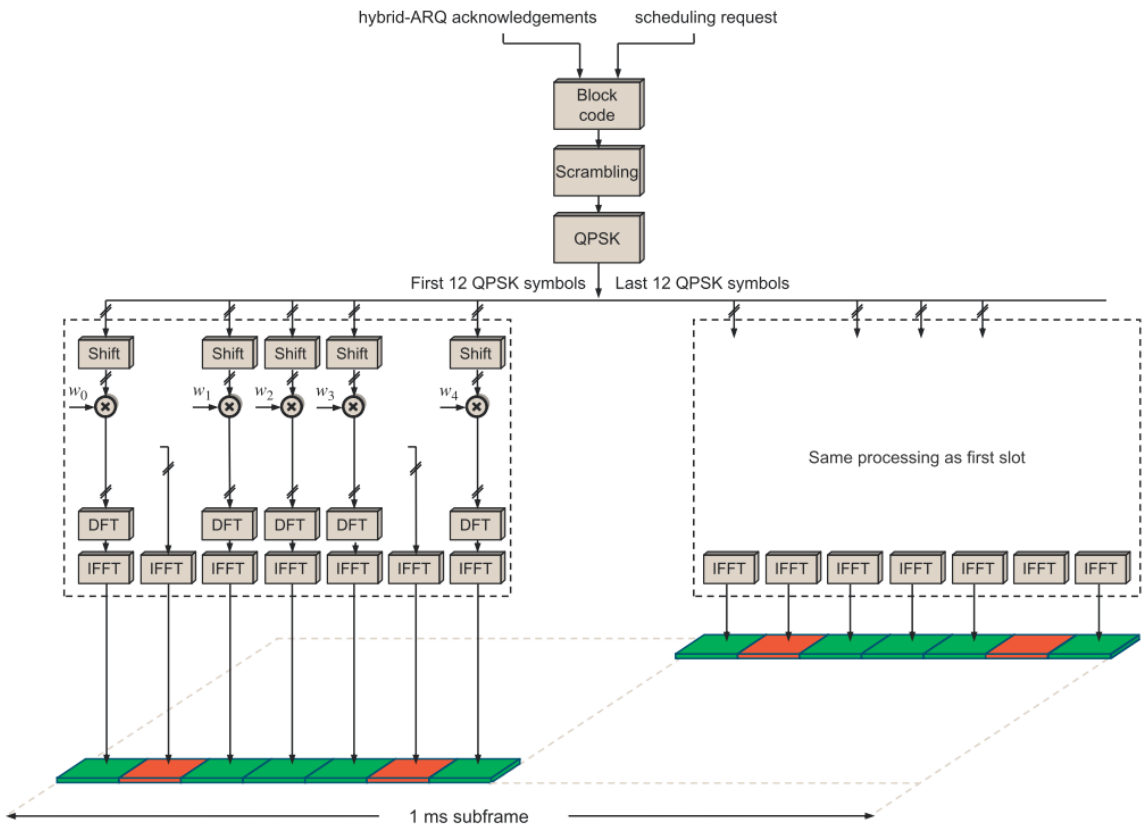


图 8-18：PUCCH format 3（正常的循环前缀）

从图 8-18 可以看出，与 PUCCH format 2 类似，对于正常的循环前缀而言，用于传输 PUCCH format 3 的 RB，每个 slot 有 7 个 SC-FDMA 符号，其每个 slot 的第 2 和第 6 个 SC-FDMA 符号用于传输 DMRS，而剩余的 5 个 SC-FDMA 符号用于传输 PUCCH format 3。

对于扩展的循环前缀而言，每个 slot 只有 6 个 SC-FDMA 符号，此时只有 1 个 SC-FDMA 符号用于传输 DMRS（对应第 4 个 SC-FDMA 符号），而剩余的 5 个 SC-FDMA 符号用于传输 PUCCH format 3。

对于 PUCCH format 3 而言，同一 RB 上不同的 PUCCH 3 资源只通过时域上的正交序列（othorgenal sequence）来区分，而不使用频域上的循环移位（cyclic shift）来区分。

我们将 PUCCH format 3 使用的 PUCCH 资源统称为 PUCCH 3 资源，PUCCH 3 资源通过一个资源索引 $n_{\mathrm{PUCCH}}^{(3,\tilde{p})}$ 表示。通过该索引，能够得到该资源所在的 RB（n_{PRB}），以及所使用的正交序列（$n_{\mathrm{oc}}^{(\tilde{p})}(n_{\mathrm{s}})$）。这里 $\tilde{p}$ 表示天线端口号，有时也写成 p，或根本就不写。

分配给某个 UE 的几个候选 PUCCH 3 资源是通过 IE：*PUCCH-ConfigDedicated-v1020* 的 *n3PUCCH-AN-List-r10*（对应 port 0）和 *n3PUCCH-AN-ListP1-r10*（对应 port 1）来配置的。eNodeB

会在随后的调度中，使用 PDCCH 中的 TPC 字段来告知 UE 使用这几个候选资源中的哪一个。（具体见 13.5.2 节关于“PUCCH format 3”的介绍）

对于 PUCCH format 3 而言，由于一个 slot 在时域内至多有 5 个 SC-FDMA 符号可用于发送 PUCCH format 3，所以使用的正交序列的长度为 5，且序列中的每一个元素对应其中一个 SC-FDMA 符号。因此，一个 RB pair 中至多包含了 5 个 PUCCH 3 资源，即至多支持 5 个 UE 复用（此时的 PUCCH 称之为 normal PUCCH format 3）。

需要注意的是，当某个上行子帧配置了 SRS，则第二个 slot 的最后一个 SC-FDMA 符号需要预留给 SRS 使用，因此第二个 slot 只能使用一个长为 4 的正交序列。即当配置了 SRS，一个 RB pair 中至多包含了 4 个 PUCCH 3 资源，即至多支持 4 个 UE 复用（此时的 PUCCH 称之为 shortened PUCCH format 3）。

为了提高 high-Doppler 场景下的性能，第二个 slot 使用的正交序列与第一个 slot 使用的不同，但第二个 slot 使用的正交序列是基于第一个 slot 的正交序列得到的（见图 8-20）。

小结：一个用于发送 PUCCH format 3 的 RB，所包含的 PUCCH 3 资源数 $N_{PUCCH,RB}^{(3)}$ 为 5（normal PUCCH format 3）或 4（shortened PUCCH format 3）。

如果某个小区共配置了 $N_{RB}^{(3)}$ 个 RB 用于发送 PUCCH format 3，则该小区在某个上行子帧内可用的 PUCCH 3 资源总数取决于该子帧是否发送 SRS：如果不发送 SRS，则为 PUCCH 3 资源总数为 $5*N_{RB}^{(3)}$；如果发送 SRS，则为 PUCCH 3 资源总数为 $4*N_{RB}^{(3)}$。

如果在同一子帧需要同时发送 SR 和 PUCCH format 3，则 SR 会复用到 PUCCH format 3 资源中，而不是使用 *sr-PUCCH-ResourceIndex* 指定的 PUCCH 1 资源。这点与需要同时发送 SR 和 PUCCH format 1a/1b 时是不一样的（见 36.213 的 10.1.5 节）。

前面已经提到过，通过 PUCCH 3 资源索引 $n_{\mathrm{PUCCH}}^{(3,\tilde{p})}$，能够得到该资源所在的 RB（$n_{\mathrm{PRB}}$）和所使用的正交序列（$n_{\mathrm{oc}}^{(\tilde{p})}(n_{\mathrm{s}})$），即该索引唯一地指定一个二元组（$n_{PRB}$，$n_{\mathrm{oc}}^{(\tilde{p})}(n_{\mathrm{s}})$）。这里我将 36.211 的 5.4.2A 节和 5.4.3 节用于计算该二元组的公式串起来，并添加了一些注释，以方便大家学习。

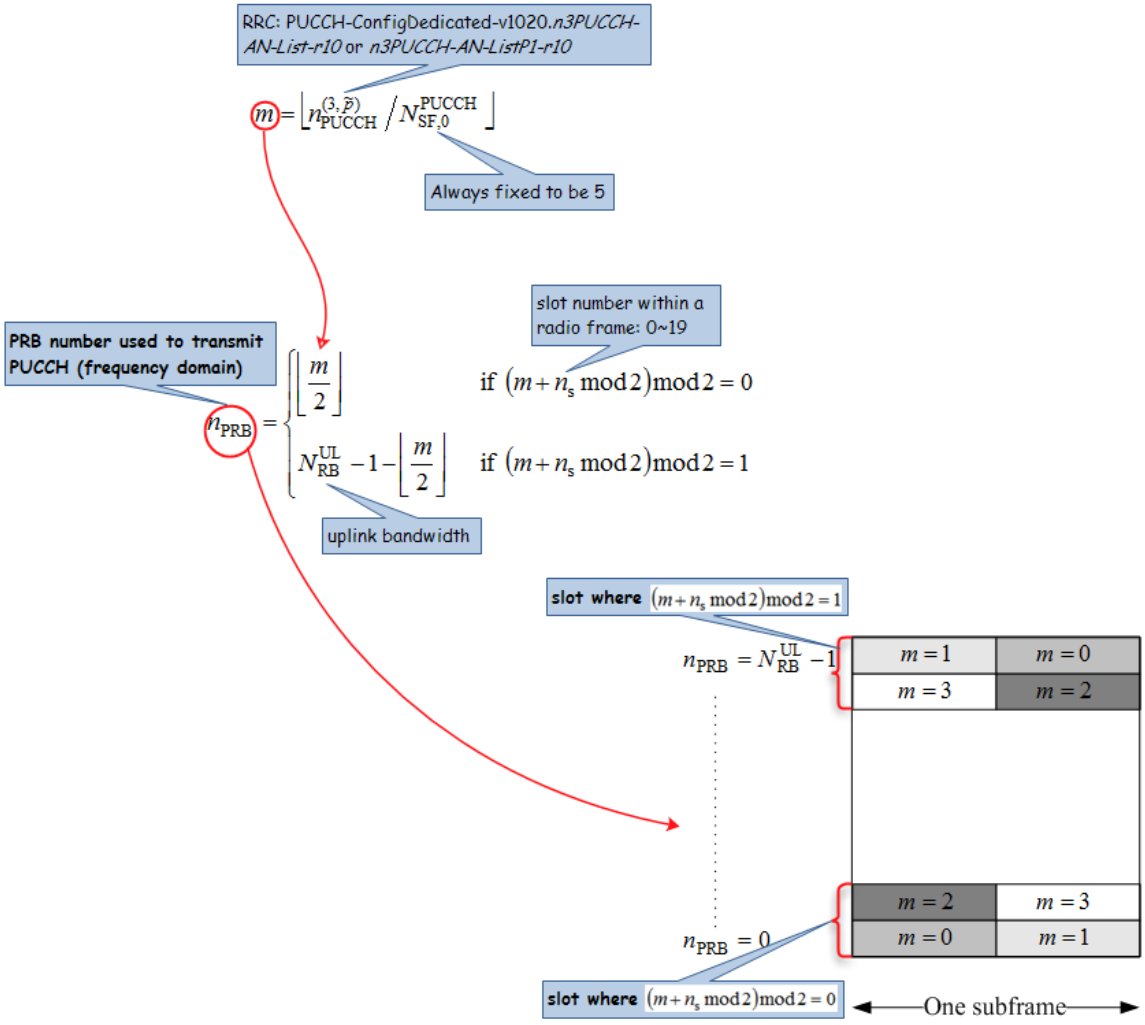


图 8-19：通过 $n_{PUCCH}^{(3,\tilde{p})}$ 确定 PUCCH 3 资源所在的 RB

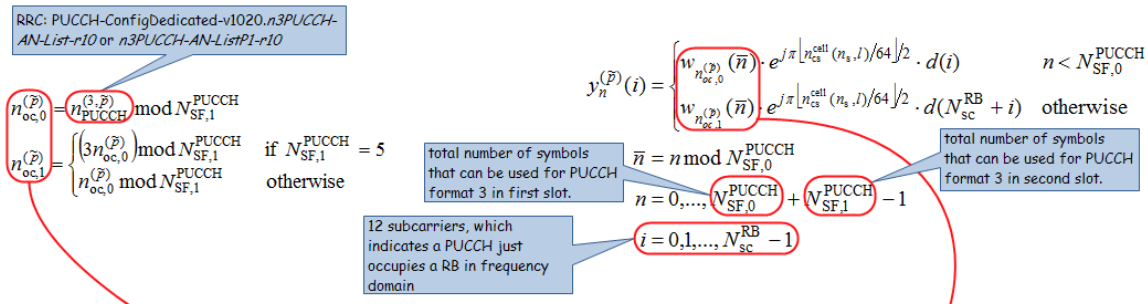


Table 5.4.2A-1: The orthogonal sequence $w_{n_{oc}}(i)$.

Sequence index n_{oc}	Orthogonal sequence $\left[w_{n_{oc}}(0) \ \cdots \ w_{n_{oc}}(N_{SF}^{PUCCH}-1)\right]$	
	$N_{SF}^{PUCCH}=5$	$N_{SF}^{PUCCH}=4$
0	$[1 \ 1 \ 1 \ 1 \ 1]$	$[+1 \ +1 \ +1 \ +1]$
1	$[1 \ e^{j2\pi/5} \ e^{j4\pi/5} \ e^{j6\pi/5} \ e^{j8\pi/5}]$	$[+1 \ -1 \ +1 \ -1]$
2	$[1 \ e^{j4\pi/5} \ e^{j8\pi/5} \ e^{j2\pi/5} \ e^{j6\pi/5}]$	$[+1 \ +1 \ -1 \ -1]$
3	$[1 \ e^{j6\pi/5} \ e^{j2\pi/5} \ e^{j8\pi/5} \ e^{j4\pi/5}]$	$[+1 \ -1 \ -1 \ +1]$
4	$[1 \ e^{j8\pi/5} \ e^{j6\pi/5} \ e^{j4\pi/5} \ e^{j2\pi/5}]$	-

for second slot of shortened PUCCH format 3 transmitted with SRS

图 8-20：通过 $n_{PUCCH}^{(3,\tilde{p})}$ 确定 PUCCH 3 资源所使用的正交序列

需要注意的是：从图 8-19 可以看出，在计算 n_{PRB} 时，$n_{PUCCH}^{(3,\tilde{p})}$ 固定除以 5，也就是说，是按一个 RB 中有 5 个 PUCCH 3 资源来计算的。而从图 8-20 可以看出，在计算 $n_{oc}^{(\tilde{p})}(n_s)$ 时，是根据第二个 slot 中可用于 PUCCH format 3 的 SC-FDMA 符号数来做 mod 计算的，也就是说，normal PUCCH format 3 下，一个 RB 中的 5 个 PUCCH 3 资源索引都可以使用，shortened PUCCH format 3 下，一个 RB 中的 5 个 PUCCH 3 资源索引只有 4 个可以使用。

举个例子：假设某子帧的 $n_{PRB}=5$ 的 RB pair 用于发送 PUCCH format 3，则该 RB 内的 PUCCH 3 资源索引 $n_{PUCCH}^{(3,\tilde{p})}$ 集合为{ 50, 51, 52, 53, 54 }。如果该子帧内同时需要发送 SRS，即使用 shortened PUCCH format 3 的情况下，$N_{SF,1}^{PUCCH}=4$，通过图 8-20 的公式，得到该 RB 内的 PUCCH 3 资源索引对应的 $n_{oc}^{(\tilde{p})}(n_s)$ 为{ 2, 3, 0, 1, 2 }。也就是说，对应资源索引 50 和 54，二者在同一个 RB 内，使用相同正交序列，这是不允许的，因为无法保证正交性。

因此，在一个 shortened PUCCH format 3 的 RB 内，有 5 个 PUCCH 3 资源索引 $n_{PUCCH}^{(3,\tilde{p})}$，但其中只有 4 个能够被使用，剩下的 1 个必须预留而不能分配给任何 UE。

注：本节内容不考虑半静态调度的场景。

【参考资料】

[1] TS 36.211 的 5.4.2A 节 PUCCH format 3

[2] TS 36.212 的 5.2.3.1 节 Channel coding for UCI HARQ-ACK

[3] TS 36.213 的 7.3 节 UE procedure for reporting HARQ-ACK

[4] TS 36.213 的 10 章 Physical uplink control channel

[5] TS 36.331 的 6.3.2 节 IE: *PUCCH-Config*

[6] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.4.1.3 节

[7] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 28.3.2.1 节

8.7 SORTD

当 UE 支持多天线传输时，如果 PUCCH 的传输能够充分利用多天线以及对应的天线功率放大器，就能够充分利用功率资源并实现最大的分集增益。为了实现额外的分集，在 Rel-10 中，引入了 PUCCH 的 2 天线传输分集机制：Space Orthogonal-Resource Transmit Diversity，简称为 SORTD。该机制同时向后兼容 Rel-8 的 PUCCH 设计。

SORTD 的基本思想是在不同的天线端口上使用不同的正交资源（即不同的 PUCCH 资源：RB、循环移位或正交序列不同）来传输同一 UE 的同一上行控制信息（UCI）。

可以看出，SORTD 只应用于 PUCCH。且与 Non-SORTD 传输机制相比，SORTD 需要使用 2 倍的 PUCCH 资源。

对于 UE 有 4 根物理天线，则会使用特定实现的天线虚拟化。从本质上说，就是使用一个透明的机制将 2 天线端口的信号映射到 4 根物理天线上。

接下来我们会介绍，针对不同的 PUCCH format（或者说 PUCCH 应用场景），如何确定 SORTD 中不同天线端口（P0 和 P1）使用的不同 PUCCH 资源？

（1）PUCCH format 1，即 SR 资源：eNodeB 通过 RRC 消息中的如下字段：*sr-PUCCH-ResourceIndex* 和 *sr-PUCCH-ResourceIndexP1-r10* 来分别配置天线端口 P0 和 P1 使用的 PUCCH 1 资源。（见 36.331 的 *SchedulingRequestConfig*）

（2）PUCCH format 1a/1b，即 ACK/NACK 资源。（非载波聚合场景）

对于 FDD，天线端口 P0 和 P1 使用的 2 个 PUCCH 1 资源分别通过 n_{CCE} 和 $n_{CCE}+1$ 得到。（见 36.213 的 10.1.2.1 节）

对于 TDD 并使用 HARQ bundling 的场景，天线端口 P0 和 P1 使用的 2 个 PUCCH 1 资源分别通过 n_{CCE} 和 $n_{CCE}+1$ 得到。（见 36.213 的 10.1.3.1 节）

对于 TDD 并使用 HARQ multiplexing 的场景，PUCCH 传输不支持传输分集，即不支持 SORTD。

（3）PUCCH format 2/2a/2b，即 CSI 资源：对于周期性 CSI 上报，eNodeB 通过 RRC 消息中的如下字段：*cqi-PUCCH-ResourceIndex-r10* 和 *cqi-PUCCH-ResourceIndexP1-r10* 来分别配置天线端口 P0 和 P1 使用的 PUCCH 2 资源。

非周期性 CSI 上报使用 PUSCH 传输，因此不支持 SORTD。

（4）PUCCH format 1b with channel selection：在 Rel-10 中，PUCCH format 1b with channel selection 并不支持传输分集，即不支持 SORTD。

（5）PUCCH format 3：对于使用 PUCCH format 3 的场景，下行 DCI（除了 PCell 上发送的第一个下行 DCI）中的 TPC command for PUCCH 字段将指示从 4 个 PUCCH 3 资源中选择 1 个（2 天线传输时是一对）用于回复 ACK/NACK。该 4 个 PUCCH 3 资源是通过 *format3-r10* 的 *n3PUCCH-AN-List-r10* 来配置的；对于 2 天线传输，第二个天线端口的 4 个 PUCCH 1 资源通过 *n3PUCCH-AN-ListP1-r10* 配置。（见 36.213 的 10.1.2.2.2 节和 10.1.3.2.2 节）

（6）SPS：对于下行传输，如果接收到的下行 DCI 指示 DL SPS 激活，则 DCI 中的 TPC command for PUCCH 字段将指示从 4 个 PUCCH 1 资源中选择 1 个（2 天线传输时是一对）用于回复 ACK/NACK。该 4 个 PUCCH 1 资源是通过 SPS-ConfigDL 的 *n1PUCCH-AN-PersistentList* 来配置的；对于 2 天线传输，第二个天线端口的 4 个 PUCCH 1 资源通过 *n1PUCCH-AN-PersistentListP1-r10* 配置。（见 36.213 的 9.2 节）

（7）ACK/NACK Repetition：如果 UE 配置了 ACK/NACK Repetition（协议中称为 HARQ-ACK Repetition），eNodeB 会通过 RRC 消息中的如下字段：*n1PUCCH-AN-Rep* 和 *n1PUCCH-AN-RepP1-r10* 来分别配置天线端口 P0 和 P1 使用的 PUCCH 1 资源（见 36.331 的 ackNackRepetition）。具体处理可参见 13.7 节关于“HARQ-ACK Repetition”的介绍。

确定了 2 个天线端口使用的 PUCCH 资源，以及需要发送的 PUCCH 信息后，剩下的就是传输分集的处理了。

如果传输天线是不相关的，则 SORTD 可以提供最好的分集性能，但会消耗更多的正交资源（Rel-8/ Rel-9 只使用 1 个资源，而 Rel-10 中可能使用 2 个资源），从而可能降低 PUCCH 的容量。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.3.3 节

[2] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 29.4.2 节

[3] “R1-090786, PUCCH TxD Schemes for LTE-A” , LG Electronics

8.8 PUCCH 资源分配

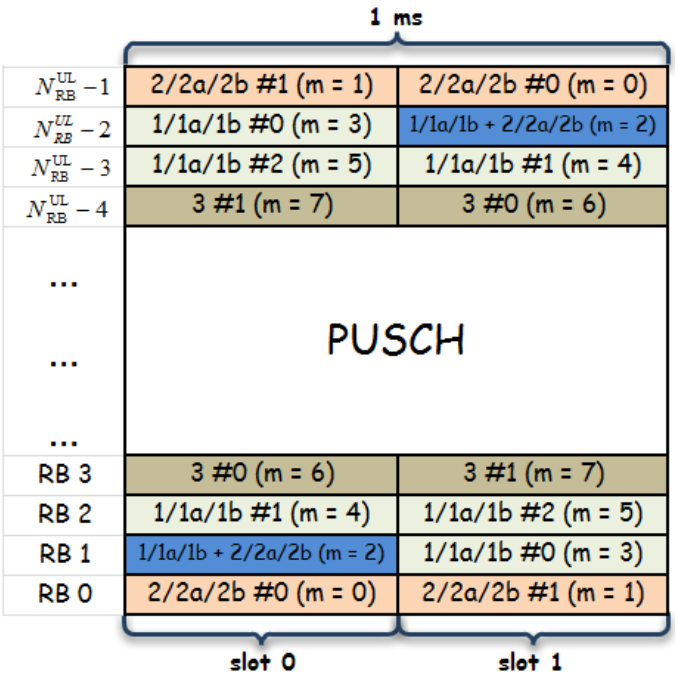


图 8-21：一个典型的 PUCCH format 与 PRB 的映射关系的例子

一个典型的 PUCCH 物理资源映射的例子如图 8-21 所示： PUCCH format 2/2a/2b 在频带的边界（对应图 8-21 中 m = 0、1 的 RB），紧接着一个 PUCCH format 1/1a/1b 与 PUCCH format 2/2a/2b 混合的 PUCCH（对应图 8-21 中 m = 2 的 RB。混合 PUCCH 可能存在也可能不存在），而后跟着 PUCCH format 1/1a/1b（对应图 8-21 中 m = 3、4、5 的 RB），最后是 PUCCH format 3（对应图 8-21 中 m = 6、7 的 RB）。（之后通常紧接着 PRACH 资源，也就是说，PUCCH 资源的分配会影响到 $n_{\mathrm{PRBoffset}}^{\mathrm{RA}}$ 的取值）

在同一子帧内，一个 PUCCH 的前后两个 slot 的 PRB 资源分别位于可用的频谱资源的两端，形成一个 PRB pair。我们将每个上行子帧中用于传输 PUCCH format 1/1a/1b 的所有 PRB pair 的个数用 $N_{RB}^{(1)}$ 表示；用于传输 PUCCH format 2/2a/2b 的所有 PRB pair 的个数用 $N_{RB}^{(2)}$ 表示；用于传输 PUCCH format 3 的所有 PRB pair 的个数用 $N_{RB}^{(3)}$ 表示；用于传输 PUCCH format 1/1a/1b 与 PUCCH format 2/2a/2b 混合的 PUCCH 的所有 PRB pair 的个数用 $N_{RB}^{(mix)}$ 表示；用于传输 PUCCH 的所有 PRB pair 的个数称为 PUCCH 资源大小，用 $N_{\mathrm{RB}}^{\mathrm{PUCCH}}$ 来表示。有 $N_{\mathrm{RB}}^{\mathrm{PUCCH}}=N_{\mathrm{RB}}^{(1)}+N_{\mathrm{RB}}^{(2)}+N_{\mathrm{RB}}^{(3)}+N_{\mathrm{RB}}^{(\mathrm{mix})}$。

以图 8-21 为例：有 $N_{\mathrm{RB}}^{\mathrm{PUCCH}}=8$，$N_{RB}^{(1)}=3$，$N_{RB}^{(2)}=2$，$N_{RB}^{(3)}=2$，$N_{RB}^{(mix)}=1$。

eNodeB 在分配 PUCCH 资源时，需要考虑以下要素：

- 上行系统带宽 N_{RB}^{UL}：带宽越大，可用的资源越多，可以分配给 PUCCH 的 RB 数就越多。例如：对于上行系统带宽为 6 RB 的小区而言，如果在某个上行子帧需要发送 PRACH（1 个 PRACH 资源在频域上占用 6 个 RB），则该子帧不能用于发送 PUCCH。
- 每个 RB 包含的 PUCCH 资源数，以及不同的 PUCCH 资源所占用的 RB 数，二者共同决定了该小区在一个上行子帧所包含的不同 PUCCH format 的资源总量。而该资源总量，又是由系统容量来决定的。针对不同的 PUCCH format，会有不同的计算方式，这会在后续章节予以介绍。
- 系统容量：如根据小区支持的 SR 用户数确定用于 SR 的 PUCCH 1 资源数（PUCCH format 1）；根据每个 TTI 能调度的下行用户数确定用于发送 ACK/NACK 的 PUCCH 1 资源数（PUCCH format 1a/1b，这还和 CCE 数相关）；根据 CA 场景下支持 PUCCH format 1b with channel selection 的用户数确定用于发送 ACK/NACK 的 PUCCH 1 资源数（对应 *PUCCH-ConfigDedicated-v1020* 中 *channelSelection-r10* 字段的配置）；根据小区支持的周期性 CSI 上报的用户数确定用于发送 CQI/PMI/RI 的 PUCCH 2 资源数（PUCCH format 2/2a/2b）；根据 CA 场景下支持 PUCCH format 3 的用户数确定用于发送 ACK/NACK 的 PUCCH 3 资源数（对应 *PUCCH-ConfigDedicated-v1020* 中 *format3-r10* 字段的配置）；

- eNodeB 或 UE 的解码能力等：这是设备厂商和终端厂商的问题，不同的厂商有不同的实现。例如：虽然协议规定 1 个 RB 允许携带 12 个 PUCCH 2 资源，但为了提高解码效率，eNodeB 可能只会给 UE 分配其中的 6 个 PUCCH 2 资源，彼此之间间隔一个循环移位。
- UE 是否支持上行多天线传输：对于 PUCCH 来说，是否支持 SORTD。如果 UE 支持 SORTD，则分配给该 UE 的 PUCCH 资源需要翻倍（由 1 个变为 2 个）。

需要说明的是，只要不同的 UE 被分配了不同的 PUCCH 资源索引，就可以保证 UE 之间相互正交。

接下来，我们以由外往里的顺序（PUCCH 2 -> mixed PUCCH -> PUCCH 1 -> PUCCH 3）来介绍不同 PUCCH format 的资源分配。

8.8.1 PUCCH 2 资源分配

PUCCH 2 资源用于携带 PUCCH format 2/2a/2b，并用于发送周期性的 CSI 上报。

小区在每个上行子帧需要多少个 RB pair 来发送 PUCCH format 2/2a/2b 会受到以下因素的影响。

- CSI 的周期：周期越大，意味着更多的上行子帧可用于发送 CSI。
- 周期内有多少个上行子帧可用于发送 PUCCH format 2/2a/2b：FDD 下，所有子帧都可以用于发送周期性 CSI。TDD 下，只有正常的上行子帧可用于发送周期性 CSI，下行子帧和特殊子帧是不能用于发送周期性 CSI 的。
- 每个 RB pair 包含多少个可用的 PUCCH 2 资源：一个 RB 内至多有 12 个 PUCCH 2 资源，但通常并不是 12 个 PUCCH 2 资源都能够使用，可能只使用其中的 6 个或 4 个 PUCCH 2 资源。
- 如果存在混合 PUCCH，还要考虑对应的 RB 内有多少个 PUCCH 2 资源。
- 小区的 PUCCH 2 容量，即小区支持最多给多少个 UE 分配 PUCCH 2 资源。容量越大，要求的 PUCCH 2 资源数就越多。

综合上述考虑，就可以确定在每个上行子帧需要多少个 RB pair 用于发送周期性 CSI。具体例子可参见 10.5.3 节关于“PUCCH 2 的容量”的介绍。

小区会通过 *PUCCH-ConfigCommon* 的 *nRB-CQI* 字段（对应 $N_{RB}^{(2)}$）告诉 UE 在一个上行子帧需要预留多少个 RB pair 用于发送 PUCCH format 2/2a/2b。

从图 8-15 可以看出，当 $n_{\mathrm{PUCCH}}^{(2,\tilde{p})}=0$ 时，对应 m=0，在一个子帧的第一个 slot 上，PUCCH 2 资源位于 $n_{PRB}=0$ 的 RB 上。也就是说，PUCCH 2 资源索引 0 对应的起始 RB 总是位于频域的第一个 RB 上。

我们以图 8-21 为例来说明 PUCCH 2 资源的分配（不考虑混合 PUCCH）。小区在一个上行子帧上使用 2 个 RB pair 来发送 PUCCH format 2/2a/2b。假设每个 RB 只能复用 4 个 PUCCH 2 资源，此时循环移位可取值{0,3,6,9}（更确切地说，不同 PUCCH 2 资源的循环移位值相差为 3 即可），则整个小区在该子帧至多支持 8 个 UE 在同一上行子帧同时发送周期性 CSI。此时小区可使用的 PUCCH 2 资源索引集合可以为{0，3，6，9，12，15，18，21}。

注：（1）可以看出，即使一个 RB 只能复用 4 个 PUCCH 2 资源，在分配 PUCCH 2 资源时，依然是按每个 RB 内有 12 个 PUCCH 2 资源索引来计算的。这与计算 PUCCH 1 资源时是不同的，假设一个 RB 只能复用 4 个 PUCCH 1 资源，则每个 RB 内只有 4 个 PUCCH 1 资源索引。（2）如果有一个 RB 用于发送混合 PUCCH，可以将小区在每个上行子帧需要预留的 PUCCH 2 资源数减去混合 PUCCH 中的 PUCCH 2 资源数，再按上面的介绍来计算用于 PUCCH 2 的 RB 数。

需要说明的是，只有处于 RRC_CONNECTED 态且上行同步的 UE，才会被分配 PUCCH 2 资源。

分配给 UE 的 PUCCH 2 资源是可回收的。以下几种情况下，小区会回收已经分配给 UE 的 PUCCH 2 资源，回收后的 PUCCH 2 资源可以分配给其它 UE 使用。

- *CQI-ReportPeriodic* 配置为 release；
- UE 上行不同步。从 36.321 的 5.2 节可以看出，如果上行不同步（*timeAlignmentTimer* 超时），UE 会释放 PUCCH 资源。
- 超过了 SR 的重发次数。从 36.321 的 5.4.4 节可以看出，如果 SR 的发送次数超过了 SR 的最大重传次数（*dsr-TransMax*），UE 会释放 PUCCH 资源。

【参考资料】

[1] 《Format 2: PUCCH Capacity Calculations》 by Lauro

8.8.2 混合 PUCCH 资源分配

从 8.5 节可以看出，混合 PUCCH 中包含的 PUCCH 1 资源数为 $\left\lceil \frac{N_{cs}^{(1)}}{8} \right\rceil \cdot c \cdot N_{cs}^{(1)} / \Delta_{shift}^{PUCCH}$，包含的 PUCCH 2 资源数为 $\left\lceil \frac{N_{cs}^{(1)}}{8} \right\rceil \cdot (N_{sc}^{RB} - N_{cs}^{(1)} - 2)$。混合 PUCCH 至多占 1 个 RB。如果 $N_{cs}^{(1)} = 0$，则不存在混合 PUCCH。

8.8.3 PUCCH 1 资源分配

PUCCH 1 资源用于携带 PUCCH format 1/1a/1b。一个小区内 PUCCH 1 资源根据不同的用途，可以分为以下几类：

（1）用于发送 SR 的 PUCCH 1 资源：通过 *sr-PUCCH-ResourceIndex*（和 *sr-PUCCH-ResourceIndexP1-r10*）给 UE 分配的资源。我们将小区在一个上行子帧分配给这部分的 PUCCH 1 资源数称为 $N_{PUCCH}^{(SR)}$；

（2）用于发送 SPS 调度相关的 ACK/NACK 信息的 PUCCH 1 资源：通过 *n1PUCCH-AN-PersistentList*（和 *n1PUCCH-AN-PersistentListP1-r10*）给 UE 分配的资源。我们将小区在一个上行子帧分配给这部分的 PUCCH 1 资源数称为 $N_{PUCCH}^{(SPS)}$；

（3）PUCCH format 1b with channel selection 中用于信道选择的 PUCCH 1 资源：通过 *channelSelection-r10* 的 *n1PUCCH-AN-CS-List-r10* 字段给 UE 分配的资源。我们将小区在一个上行子帧分配给这部分的 PUCCH 1 资源数称为 $N_{PUCCH}^{(CS)}$；

（4）用于发送与动态调度相关的 ACK/NACK 信息的 PUCCH 1 资源：通过对应 PDCCH 的 first CCE 计算出的资源。我们将小区在一个上行子帧分配给这部分的 PUCCH 1 资源数称为 $N_{PUCCH}^{(A/N)}$。

注：PUCCH format 1b with channel selection 中可能同时使用到（2）、（3）、（4）中的 PUCCH 1 资源，具体见 13.5.1 节的介绍。

由于这几类 PUCCH 1 资源可复用在同一个 RB 内，所以我们首先来统计每种类型的资源需要的 PUCCH 1 资源数。

在每个上行子帧需要预留多少个 PUCCH 1 资源来发送 SR，可参见第 14 章关于“SR”的介绍。

对于 PUCCH format 1b with channel selection 中用于信道选择的 PUCCH 1 资源，跟 1 个上行子帧内需要对多少个配置了 PUCCH format 1b with channel selection 的 UE 进行 ACK/NACK 反馈相关。假设在一个上行子帧需要对至多 4 个配置了 PUCCH format 1b with channel selection 的 UE 进行 ACK/NACK 反馈，则配置的 *n1PUCCH-AN-CS-List-r10* 包含 4 组（共 8 个）PUCCH 1 资源即可。此

时整个小区配置了 PUCCH format 1b with channel selection 的 UE 数可以大于 4 个，eNodeB 对这些 UE 都配置相同的 *n1PUCCH-AN-CS-List-r10* 值，但在一个反馈 ACK/NACK 的上行子帧对应的所有下行子帧上，eNodeB 可以做到只调度至多 4 个使用 PUCCH format 1b with channel selection 的 UE（其它 UE 也可被调度，但不使用 channel selection 资源），此时 eNodeB 通过对应的 DCI 给不同的 UE 分配不同的 TPC 值即可。

与 PUCCH format 1b with channel selection 类似，eNodeB 会通过 *n1PUCCH-AN-PersistentList*（和 *n1PUCCH-AN-PersistentListP1-r10*）给 UE 配置 4 个（对）PUCCH 1 资源用于反馈下行 SPS 调度对应的 ACK/NACK，并通过对应 DCI 的 TPC 字段从 4 个（对）PUCCH 1 资源中选择出 1 个用于反馈 ACK/NACK。关于下行 SPS 调度相关的 PUCCH 1 资源计算，可参见第 24 章关于“SPS”的介绍。

动态调度：从 36.213 的 10.1 节可以看出，通过 first CCE 计算出的 PUCCH 1 资源是在偏移了 $N_{\mathrm{PUCCH}}^{(1)}$ 的基础计算出的，也即用于动态调度的 PUCCH 1 资源的最小值为 $N_{\mathrm{PUCCH}}^{(1)}$。$N_{\mathrm{PUCCH}}^{(1)}$ 实际可以配置成前面介绍的（1）、（2）、（3）这 3 种情况下分配的所有 PUCCH 1 资源之和，即 $N_{\mathrm{PUCCH}}^{(1)} = N_{PUCCH}^{(SR)} + N_{PUCCH}^{(SPS)} + N_{PUCCH}^{(CS)}$。动态调度的 PUCCH 1 资源在计算时“$+N_{\mathrm{PUCCH}}^{(1)}$”的目的是为了避免其与这 3 类 PUCCH 1 资源发生冲突。

这里我们分别对 FDD 和 TDD 下的动态调度中的 PUCCH 1 资源分配进行介绍。

从 36.213 的 10.1.2 节可以看出，FDD 下，动态调度下的 PUCCH 1 资源的计算公式为“$n_{\mathrm{PUCCH}}^{(1)} = n_{\mathrm{CCE}} + N_{\mathrm{PUCCH}}^{(1)}$”或“$n_{\mathrm{PUCCH}}^{(1)} = n_{\mathrm{CCE}} + 1 + N_{\mathrm{PUCCH}}^{(1)}$”。由于是 $N_{\mathrm{PUCCH}}^{(1)}$ 由小区级的配置参数 *n1PUCCH-AN* 决定的，对于该小区的所有 UE 来说是固定的。因此 n_{CCE} 的取值范围就决定了一个上行子帧内用于动态调度的 PUCCH 1 资源的取值范围。n_{CCE} 是对应的 PDCCH 所占的第一个 CCE 的索引，一个子帧内有多少个 CCE，就决定了 n_{CCE} 的取值范围。一个下行子帧包含多少个 CCE 的计算方式可参见 4.4.2 节中关于“下行子帧可用的 CCE 数”的介绍。

假设 FDD 下的小区下行系统带宽为 10 MHz，某个下行子帧的控制区域占 1 个 OFDM 符号，PHICH duration 设为 normal 且 Ng = 1/6 的情况下，计算出的 CCE 资源数为 43 个，此时一个上行子帧用于动态调度的 PUCCH 1 资源数为 44（43 + 1）个，这里需要+1 的原因在于从 36.213 的 10.1.2 节可以看出，PUCCH 1 资源索引在某些场景下需要使用“$n_{\mathrm{CCE}} + 1$”来计算。

从 36.213 的 10.1.3 节可以看出，TDD 下，动态调度下的 PUCCH 1 资源的计算公式为“$n_{\mathrm{PUCCH}}^{(1)} = (M - m - 1) \cdot N_c + m \cdot N_{c+1} + n_{\mathrm{CCE}} + N_{\mathrm{PUCCH}}^{(1)}$”或“$n_{\mathrm{PUCCH}}^{(1)} = (M - m - 1) \cdot N_c + m \cdot N_{c+1} + n_{\mathrm{CCE}} + 1 + N_{\mathrm{PUCCH}}^{(1)}$”。与 FDD 相比，计算公式中多了“$(M - m - 1) \cdot N_c + m \cdot N_{c+1}$”部分。因此与 FDD 相比，TDD 下可能需要更多的 PUCCH 1 资源。

从之前的介绍可以看出，一个上行子帧需要的 PUCCH 1 资源数为 $N_{PUCCH\,1} = N_{PUCCH}^{(SR)} + N_{PUCCH}^{(SPS)} + N_{PUCCH}^{(CS)} + N_{PUCCH}^{(A/N)}$。而用于 PUCCH format 1/1a/1b 的 RB pair 数为 $N_{RB}^{(1)} = \left\lceil (N_{PUCCH\,1} - \left\lceil \frac{N_{\mathrm{cs}}^{(1)}}{8} \right\rceil . c \cdot N_{\mathrm{cs}}^{(1)} / \Delta_{\mathrm{shift}}^{\mathrm{PUCCH}}) \ / \ (c \cdot N_{\mathrm{sc}}^{\mathrm{RB}} / \Delta_{\mathrm{shift}}^{\mathrm{PUCCH}}) \right\rceil$。其中 $\left\lceil \frac{N_{\mathrm{cs}}^{(1)}}{8} \right\rceil . c \cdot N_{\mathrm{cs}}^{(1)} / \Delta_{\mathrm{shift}}^{\mathrm{PUCCH}}$ 为混合 PUCCH 中包含的 PUCCH 1 资源数，$c \cdot N_{\mathrm{sc}}^{\mathrm{RB}} / \Delta_{\mathrm{shift}}^{\mathrm{PUCCH}}$ 为一个 RB pair 中包含的 PUCCH 1 资源数。

需要说明的是，$\Delta_{\mathrm{shift}}^{\mathrm{PUCCH}}$ 会对同一 PUCCH 1 资源索引 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 所在的 RB 产生影响。我们以正常的循环前缀下，不存在混合 PUCCH，且 PUCCH 1 资源索引取值 0~35 为例。假设 $\Delta_{\mathrm{shift}}^{\mathrm{PUCCH}}$ 值为 1，对应一个 RB 内有 36 个 PUCCH 1 资源，对照图 8-8 可以看出，资源索引 0~35 会都位于同一个 RB pair 内，此时同一 RB pair 内相邻的 PUCCH 1 索引（如索引 5 和索引 6）之间的循环移位相差为 1。但如果 $\Delta_{\mathrm{shift}}^{\mathrm{PUCCH}}$ 值为 2，则一个 RB 内有 18 个 PUCCH 1 资源，对照图 8-8 可以看出，资源索引 0~17 会位于同一个 RB pair 内，而资源索引 18~35 会位于另一个 RB pair 内，此时同一 RB pair 内相邻的 PUCCH 1 索引（如索引 5 和索引 6）之间的循环移位相差为 2。

从图 8-8 还可以看出，当 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})} = 0$ 时，无论小区是否支持混合 PUCCH，在一个子帧的第一个 slot 上，PUCCH 1 资源总是位于 $n_{PRB} = \lfloor N_{RB}^{(2)} / 2 \rfloor$ 或 $n_{PRB} = N_{RB}^{UL} - 1 - \lfloor N_{RB}^{(2)} / 2 \rfloor$ 的 RB 上，而不是位于 $n_{PRB} = 0$ 的 RB 上，这与 PUCCH 2 资源和 PUCCH 3 资源是不同的。也就是说，PUCCH 1 资源索引 0 对应的起始 RB 总是将 PUCCH 2 所占的 RB 剔除后开始计算的。而 PUCCH 2 资源索引 0 和 PUCCH 3 资源索引 0 总是在 $n_{PRB} = 0$ 的 RB 上。

由于 UE 只有在收到下行调度时（包括动态调度和半静态调度）才会使用 PUCCH 1 资源来回应 ACK/NACK，而每个小区在 1 个 TTI 内能够调度的最大 UE 数通常是固定的，因此，小区中 UE 数增加，并不需要增加用于 PUCCH format 1a/1b 的 PUCCH 资源总量。

8.8.4 PUCCH 3 资源分配

一个小区需要预留多少个 PUCCH 3 资源，跟 1 个上行子帧内需要对多少个配置了 PUCCH format 3 的 UE 数进行 ACK/NACK 反馈相关。假设在一个上行子帧需要对至多 4 个配置了 PUCCH format 3 的 UE 数进行 ACK/NACK 反馈，则配置的 *n3PUCCH-AN-List-r10* 包含 4 个（支持 SORTD 时是 8 个）PUCCH 3 资源即可。此时整个小区配置了 PUCCH format 3 的 UE 数可以大于 4 个，eNodeB 对这些 UE 都配置相同的 *n3PUCCH-AN-List-r10* 值，但在一个反馈 ACK/NACK 的上行子帧对应的所有下行子帧上，eNodeB 可以做到只调度至多 4 个使用 PUCCH format 3 的 UE（其它 UE 也可被调度，但不使用 PUCCH 3 资源），此时 eNodeB 通过对应的 DCI 给不同的 UE 分配不同的 TPC 值即可。

如果在一个上行子帧需要对至多 8 个配置了 PUCCH format 3 的 UE 数进行 ACK/NACK 反馈，则所需的 PUCCH 3 资源就需要翻倍。

可以看出，PUCCH 3 资源分配与 PUCCH format 1 with channel selection 中用于信道选择的那部分 PUCCH 1 资源的分配方式类似。

在 8.6 节中我们已经介绍过，normal PUCCH format 3 下，一个 RB 中的 5 个 PUCCH 3 资源索引都可以使用；shortened PUCCH format 3 下，一个 RB 中的 5 个 PUCCH 3 资源索引只有 4 个可以使用。所以 PUCCH format 3 和 SRS 冲突时，需要避免将一个 RB 内的第 5 个 PUCCH 3 资源分配给 UE。一个可行的做法是只固定使用同一个 RB 内的 4 个 PUCCH 3 资源，而不使用剩余的 1 个 PUCCH 3 资源。

我们以图 8-21 为例，来介绍哪些 PUCCH 3 资源可分配给 UE。从图 8-19 中可以看出，PUCCH 3 资源索引 0 总是在 $n_{PRB}=0$ 的 RB 上。但从图 8-21 可以看出，位于带宽边缘的 6 个 RB pair 已经被 PUCCH format 2/2a/2b、混合 PUCCH、PUCCH format 1/1a/1b 所占用。因此映射到这些 RB pair 上的 PUCCH 3 资源都不能分配给 UE。再结合图 8-19 和图 8-20，我们可以知道，此时分配给 UE 的 PUCCH 3 资源索引必须从 5 * 6=30 开始。

也就是说，一个小区在一个上行子帧上可使用的 PUCCH 3 资源的起始资源索引为 $5*(N_{RB}^{(1)}+N_{RB}^{(2)}+N_{RB}^{(mix)})$，小于该值的 PUCCH 3 资源索引不能分配给 UE。这里 5 表示一个 RB pair 中包含了 5 个 PUCCH 3 资源索引。

8.8.5 PUCCH 资源特殊配置

小区在进行 PUCCH 资源分配时，并不能够保证所有 PUCCH 资源（包括所有的 PUCCH 类型）正好占用偶数个 RB pair。如果 PUCCH 资源占用的 RB pair 数为奇数，就会出现图 8-22 的情况。与图 8-21 相比，图 8-22 中有一个“m = 奇数”（图中为 m = 7）的 RB pair 没被 PUCCH 占用。小区在进行资源分配时，需要对这个“m = 7”的 RB pair 做特殊处理，通常这个 RB pair 既不被 PRACH 所使用，也不被 PUSCH 所使用。

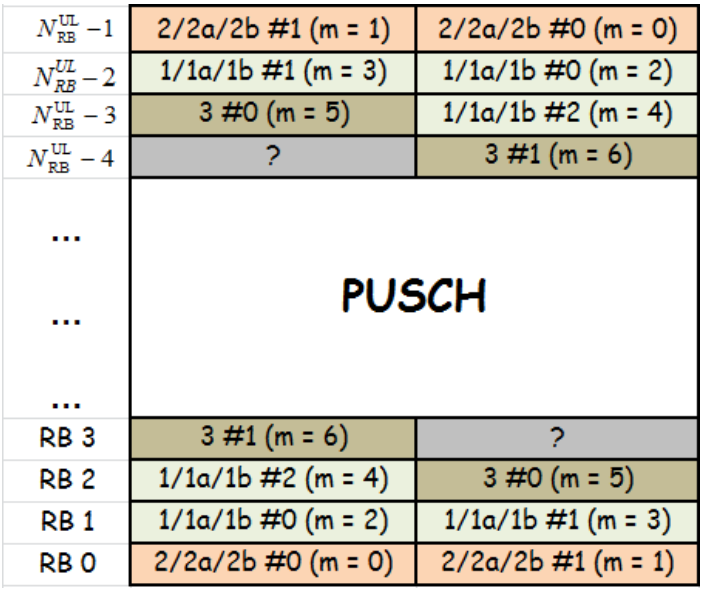


图 8-22：一个“m = 奇数”的 RB pair 没被 PUCCH 占用的场景

eNodeB 还可以选择将低功率的 PUSCH 传输（例如：来自小区中心的 UE）调度在 PUCCH 区域的外围 RB，即位于带宽的边缘上，而内部的 PUCCH 区域用于 PUCCH 传输。此时通过将高功率的 PUCCH 传输（例如：来自小区边缘的 UE）稍微地离开频带的边缘，可以有效地降低某些频带上的带外发射（out-of-band emissions）。其原则就是将所分配的 PUCCH 资源远离那些要减轻带外发射的频带。

简单地说，就是将上行系统带宽边缘的 RB 用于 PUSCH 传输，并相应地将 PUCCH 所占的 RB 往带宽中心移动。这种实现方式被称为 PUCCH over-dimensioning。图 8-23 是其中的一个例子。

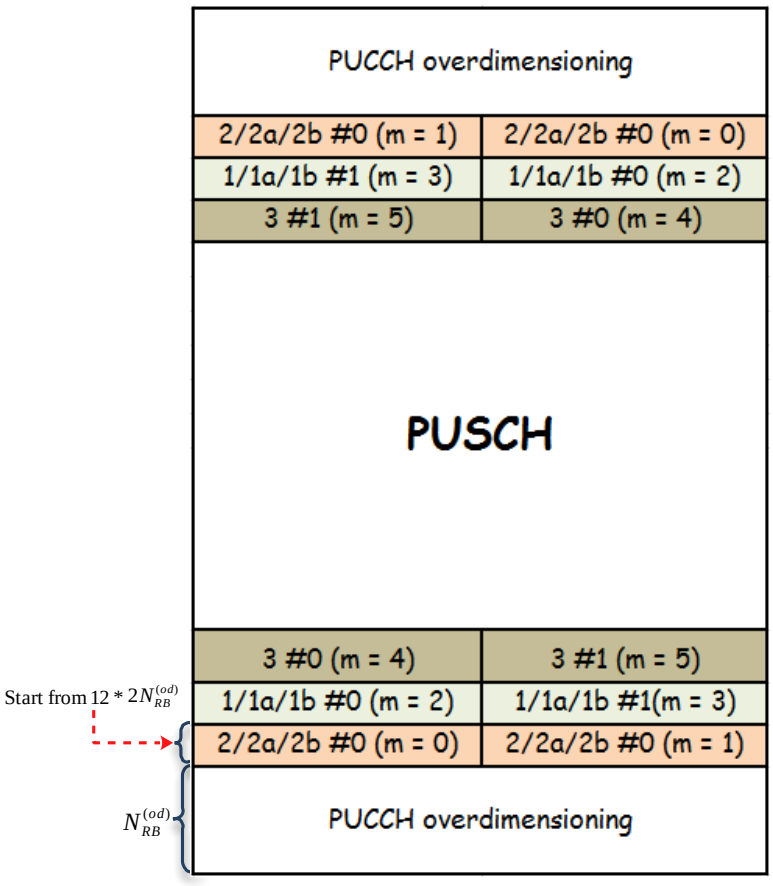


图 8-23：PUCCH overdimensioning 举例

由于在 PUCCH overdimensioning 中，PUCCH 所占的资源并不位于带宽的边缘，所以在计算 PUCCH 2 资源所占的 RB 数 $N_{RB}^{(2)}$ 时，需要将带宽边缘用于 PUCCH overdimensioning 的 RB 和用于 PUCCH 2 的 RB 都包含在内。假设小区在带宽的边缘各有 $N_{RB}^{(od)}$ 个 RB 用于 PUCCH overdimensioning，并有 $N_{RB}^{(PUCCH2)}$ 个 RB pair 用于 PUCCH format 2/2a/2b，则有 $N_{RB}^{(2)} = 2N_{RB}^{(od)} + N_{RB}^{(PUCCH2)}$，此时 $N_{RB}^{(2)}$ 为小区所要配置的 *nRB-CQI* 值。但 eNodeB 在给 UE 分配 PUCCH 2 资源时，其资源索引是从 $12*2N_{RB}^{(od)}$ 开始的，这样就保证了 PUCCH overdimensioning 所占的 RB 上不会有 PUCCH 传输。

虽然混合 PUCCH 、PUCCH 1 资源和 PUCCH 3 资源所占的 RB 也往带宽中心移动，但其计算方式并不需要做改变。

【参考资料】

[1] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.3.1 节

[2] 《DEPLOYMENT OF LTE UL SYSTEM FOR ARBITRARY SYSTEM BANDWIDTHS VIA PUCCH CONFIGURATION》

[3] 《用于在无线通信系统中控制 UE 发射的方法和装置》 by J·蒙托霍， W·陈， P·加尔， A·梅朗

第9章　PUSCH

PUSCH 主要用于传输来自 UL-SCH 的数据，更确切地说， RRC 消息和用户数据等最终会在 PUSCH 上传输。

为了发送 PUSCH，UE 需要首先解码 PDCCH。PDCCH 携带的 DCI（DCI format 0/4）指定了在空口上如何传输 PUSCH。DCI 指定了 PUSCH 所占的资源、MCS、初传或重传相关信息、层以及预编码等信息。

本章主要从 UE 的角度来介绍 UE 如何从 PDCCH 中获取到对应 PUSCH 的资源分配信息（9.1 节）、调制阶数（9.3.1 节）和冗余版本（9.3.2 节），以及如何确定 PUSCH 所携带的 TB 的大小（9.3.3 节）。同时在 9.2 节，我们会介绍 PUSCH 传输如何通过跳频来提高分集增益。在 Rel-10 中，新增了对上行多天线传输的支持，在本章的最后，我们会对 PUSCH 的多天线传输进行介绍（9.4 节）。

9.1　上行资源分配类型

本节主要介绍上行物理信道 PUSCH 的 2 种资源分配类型：Type 0 和 Type 1。

具体使用哪种上行资源分配类型取决于所使用的上行 DCI（DCI format 0/4）以及 DCI 内的 “Resource allocation type”字段的配置。

- 如果上行 DCI format 中不存在“Resource allocation type”字段，则只支持上行资源分配类型 0；
- 如果上行 DCI format 中的“Resource allocation type”字段为 0，则表示使用上行资源分配类型 0；
- 如果上行 DCI format 中的“Resource allocation type”字段为 1，则表示使用上行资源分配类型 1。

表 9-1：DCI format 0/4 与上行资源分配类型的对应关系

DCI format	“Resource allocation type” 字段 (1 bit)	资源分配类型
0	0	Type 0
	1	Type 1
	Not present when $N_{\mathrm{RB}}^{\mathrm{UL}} > N_{\mathrm{RB}}^{\mathrm{DL}}$	Type 0

4	0	Type 0
	1	Type 1

注意：分配给某个 UE 的 PUSCH 传输的 RB 数只能为 2、3 和 5 的乘积（见 36.211 的 5.3.3 节），即必须满足 $2^x \cdot 3^y \cdot 5^z$，其中 x、y 和 z 必须为非负整数。因此分配给某个 UE 的 PUSCH 传输的 RB 数必须属于以下集合：

1	2	3	4	5	6	8	9	10	12	15	16
18	20	24	25	27	30	32	36	40	45	48	50
54	60	64	72	75	80	81	90	96	100		

图 9-1：PUSCH 传输可用的资源分配集合

9.1.1 上行资源分配类型 0（uplink resource allocation type 0）

上行资源分配类型 0 的处理与用于 DCI format 1A/1B/1D 的下行资源分配类型 2 的处理基本一样，而上行 PUSCH 传输使用的是上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}}$ 而不是下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}}$ 进行计算。（详见 5.1.4 节关于“资源分配类型 2”的介绍）

对于 DCI format 0，其资源分配是通过“Resource block assignment and hopping resource allocation”字段指定的。对于 DCI format 4，其资源分配是通过“Resource block assignment”字段指定的。

9.1.2 上行资源分配类型 1（uplink resource allocation type 1）

上行资源分配类型 1 是 Rel-10 中引入的，以便为 UE 分配频域上不连续的 PUSCH 资源。

上行资源分配类型 1 中的资源分配信息指示给 UE 分配 2 个 RB 集合，每个集合包含 1 个或多个连续的大小为 P 的 RBG。P 表示 RBG 中包含的连续的 RB 个数，其值见 36.213 的 Table 7.1.6.1-1（把下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}}$ 替换成上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}}$）。

注意：上行资源分配类型 1 是以 RBG 为分配单位，而不是以 RB 为分配单位的。

Table 7.1.6.1-1: Type 0 Resource Allocation RBG Size vs. Downlink System Bandwidth

System Bandwidth N_{RB}^{DL}	RBG Size (*P*)
≤10	1
11 – 26	2
27 – 63	3
64 – 110	4

如果使用上行资源分配类型 1，则 DCI format 0/4 中的资源分配字段（“Resource block assignment”字段）表示的是一个 combinatorial index *r*。

combinatorial index *r* 指定了 RB 集合 1 的起始 RBG 和结束 RBG 的索引 s_0 和 s_1-1，以及 RB 集合 2 的起始 RBG 和结束 RBG 的索引 s_2 和 s_3-1。*r* 的计算公式为：

$$r=\sum_{i=0}^{M-1}\left\langle \begin{matrix} N-s_i \\ M-i \end{matrix} \right\rangle$$

其中 M = 4 且 $N=\left\lceil N_{\mathrm{RB}}^{\mathrm{UL}}/P \right\rceil+1$。$s_i$ 的排序属性和范围如下（可参见 36.213 的 7.2.1 节）：

集合 $\{s_i\}_{i=0}^{M-1}$, $(1\le s_i\le N,\quad s_i<s_{i+1})$包含了 M 个已排序的 RBG 索引，即有 $S_0<S_1<S_2<S_3$。

$\left\langle \begin{matrix} x \\ y \end{matrix} \right\rangle=\begin{cases} \begin{pmatrix} x \\ y \end{pmatrix} & x\ge y \\ 0 & x<y \end{cases}$ 是扩展的二项式系数，会生成一个唯一的 $r\in\left\{0,\cdots,\begin{pmatrix} N \\ M \end{pmatrix}-1\right\}$。其中 $\begin{pmatrix} n \\ k \end{pmatrix}=\frac{n!}{k!\times(n-k)!}$ 为组合的计算公式。

如果起始 RBG 与结束 RBG 的索引相同，则只给该 RB 集合分配了一个 RBG。

一个 combinatorial index *r* 包含 $\left\lceil \log_2\left(\begin{pmatrix} \left\lceil N_{RB}^{UL}/P+1 \right\rceil \\ 4 \end{pmatrix}\right) \right\rceil$ 比特。

如果 UL grant 中的资源分配字段（“Resource block assignment”字段）所包含的比特数

- 小于完全表示 *r* 所需的比特数：则 *r* 的 LSB 将占用资源分配字段的所有比特，而 *r* 的剩余比特将为 0；
- 大于完全表示 *r* 所需的比特数：则 *r* 将占用 UL grant 中的资源分配字段的 LSB。

上行资源分配类型 1 并不支持 PUSCH 跳频，其频域分集是通过使用 2 个不同的 RB 集合来实现的。

图 9-2 是使用上行资源分配类型 1 的一个例子。

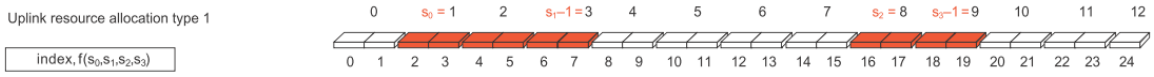


图 9-2：上行资源分配类型 1 的一个例子

图 9-2 的例子中，上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}}=25$，参见 36.213 的 Table 7.1.6.1-1 可知，P = 2。参见 5.1.1 节关于 RBG 的介绍可知，此时 RBG 的总数 $N_{RBG}=\lceil N_{\mathrm{RB}}^{\mathrm{UL}}/P\rceil=\lceil 25/2\rceil=13$，即 RBG 的索引范围为 0 ~ 12。

图 9-2 中分配给 UE 的 RB 集合 1 的起始 RGB 索引为 1，结束 RBG 索引为 3，即有 $s_0=1,\ s_1=4$；分配给 UE 的 RB 集合 2 的起始 RGB 索引为 8，结束 RBG 索引为 9，即有 $s_2=8,\ s_3=10$。

由于 M = 4 且 $N=\lceil N_{\mathrm{RB}}^{\mathrm{UL}}/P\rceil+1=14$。所以有

$$r=\sum_{i=0}^{M-1}\left\langle\begin{matrix}N-s_i\\M-i\end{matrix}\right\rangle=\binom{13}{4}+\binom{10}{3}+\binom{6}{2}+\binom{4}{1}=715+120+15+4=854$$

关于 combinatorial index 的介绍，以及如何通过 combinatorial index 反推出 S_i，可参见参考资料[7]中的介绍。

DCI format 0/4 中的资源分配字段提供的是用于上行传输的 VRB，即通过上面的介绍计算出来的是 n_{VRB}，而 VRB 到 PRB 的映射过程：

- 如果使用上行资源分配类型 1，则用于 PUSCH 传输的 PRB 与 VRB 是直接一一映射的，即有 $n_{\mathrm{PRB}}=n_{\mathrm{VRB}}$；
- 如果使用上行资源分配类型 0，且上行 PUSCH 跳频是去使能的，则用于 PUSCH 传输的 PRB 与 VRB 是直接一一映射的，即有 $n_{\mathrm{PRB}}=n_{\mathrm{VRB}}$；
- 如果使用上行资源分配类型 0，且上行 PUSCH 跳频是使能的，且使用的跳频类型 1（type 1），则 VRB 到 PRB 的映射过程见下一节关于“PUSCH 跳频类型 1”中的介绍；
- 如果使用上行资源分配类型 0，且上行跳频是使能的，且使用的跳频类型 2（type 2），则 VRB 到 PRB 的映射过程见下一节关于“PUSCH 跳频类型 2”中的介绍。

通过上述过程，UE 知道了为当前的 PUSCH 传输分配了哪些 PRB，因此也就知道了所分配的 PRB 数 N_{PRB}。

【参考资料】

[1] TS 36.211 的 5.3.4 节 Mapping to physical resources

[2] TS 36.212 的 5.3.3.1.1 节 Format 0

[3] TS 36.212 的 5.3.3.1.8 节 Format 4

[4] TS 36.213 的 8.1 节 Resource Allocation for PDCCH with uplink DCI Format

[5] 《4G LTE/LTE-Advanced for Mobile Broadband》的 10.4.5.1 节

[6] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 28.3.6.2 节

[7] 《Combinatorial number system》 from wikipdedia

[8] 《LTE Resource Allocation Tool》：一个 LTE 资源分配计算工具

9.2 上行跳频

LTE 中，PUSCH 传输通过跳频可以提高分集。在上一节我们已经介绍过，上行资源分配类型 1 通过将 2 个 RB 集合分布在频域上不同的位置来实现频域分集，且不支持 PUSCH 跳频。而上行资源分配类型 0 分配的是频域上连续的 VRB，需要通过跳频的方式将 VRB 映射到 PRB，从而实现频率分集。

PUSCH 跳频（PUSCH frequency hopping）的**触发条件**为：只有当 DCI format 0 中的“Frequency hopping flag”字段设置为 1，且使用上行资源分配类型 0（“Resource allocation type”字段设置为 0）时，UE 才会进行 PUSCH 跳频，否则 UE 不会进行 PUSCH 跳频。

DCI format 4 是不支持 PUSCH 跳频的。

LTE 中定义了 2 种 PUSCH 跳频类型：

- 类型 1（Type 1）：基于 UL grant 中明确的跳频信息进行跳频；
- 类型 2（Type 2）：根据小区特定的跳频/镜像样式进行基于 subband 的跳频。

首先，如何确定是使用类型 1 还是使用类型 2 的跳频呢？

DCI format 0 有一个资源分配字段（“Resource block assignment and hopping resource allocation”字段），如果使能跳频的话，该字段最高 1 或 2 比特（MSB，用 N_{UL_hop} 表示，称之为 hopping bit）是用于指示跳频信息的，具体选择 1 比特 还是 2 比特与上行系统带宽相关，如下表所示。（见 36.213 的 Table 8.4-1）

Table 8.4-1: Number of Hopping Bits N_{UL_hop} vs. System Bandwidth

System BW N_{RB}^{UL}	#Hopping bits for 2nd slot RA (N_{UL_hop})
6-49	1
50-110	2

hopping bit 用于指示使用的 PUSCH 跳频类型是类型 1 还是类型 2。如果使用的是类型 1，hopping bit 还决定了 $\tilde{n}_{PRB}(i)$，如下表所示。（见 36.213 的 Table 8.4-2）

Table 8.4-2: PDCCH DCI Format 0 Hopping Bit Definition

System BW N_{RB}^{UL}	Number of Hopping bits	Information in hopping bits	$\tilde{n}_{PRB}(i)$
6 – 49	1	0	$\left(\left\lfloor N_{RB}^{PUSCH}/2\right\rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$,
		1	Type 2 PUSCH Hopping
50 – 110	2	00	$\left(\left\lfloor N_{RB}^{PUSCH}/4\right\rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$
		01	$\left(-\left\lfloor N_{RB}^{PUSCH}/4\right\rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$
		10	$\left(\left\lfloor N_{RB}^{PUSCH}/2\right\rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$
		11	Type 2 PUSCH Hopping

从 36.213 的 Table 8.4-2 可以看出，LTE 中可用的上行系统带宽与 hopping bit 取值以及 PUSCH 跳频类型的对应关系为：

表 9-2：DCI format 0/4 与上行资源分配类型的对应关系

上行系统带宽(RB)	hopping bit 取值	跳频类型
	0	Type 1

6、15、25	1	Type 2
50、75、100	00	Type 1
	01	Type 1
	10	Type 1
	11	Type 2

DCI format 0 中的资源分配字段除去用于跳频信息的 1 或 2 比特后，剩余的比特数为 $y=\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{UL}}(N_{\mathrm{RB}}^{\mathrm{UL}}+1)/2)\rceil - N_{UL_hop}$。剩余的比特携带的是 RIV 值，用于计算 RB_{START}和 L_{CRBs} 。（详见 9.1.1 节关于“上行资源分配类型 0”的介绍）

接着，我们来确定是跳频的范围？

既然是 PUSCH 跳频，那跳频的范围只能限定在那些用于 PUSCH 的 RB 上，而不能跳频到那些用于 PUCCH 的 RB 上，否则会把 PUCCH 的数据给破坏了。

在进行上行资源分配时，PUCCH 通常位于上行频域的两端。这里有 2 种可能：偶数个 RB pair 分配给 PUCCH，和奇数个 RB pair 分配给 PUCCH。图 9-3 是这 2 种情况的举例：

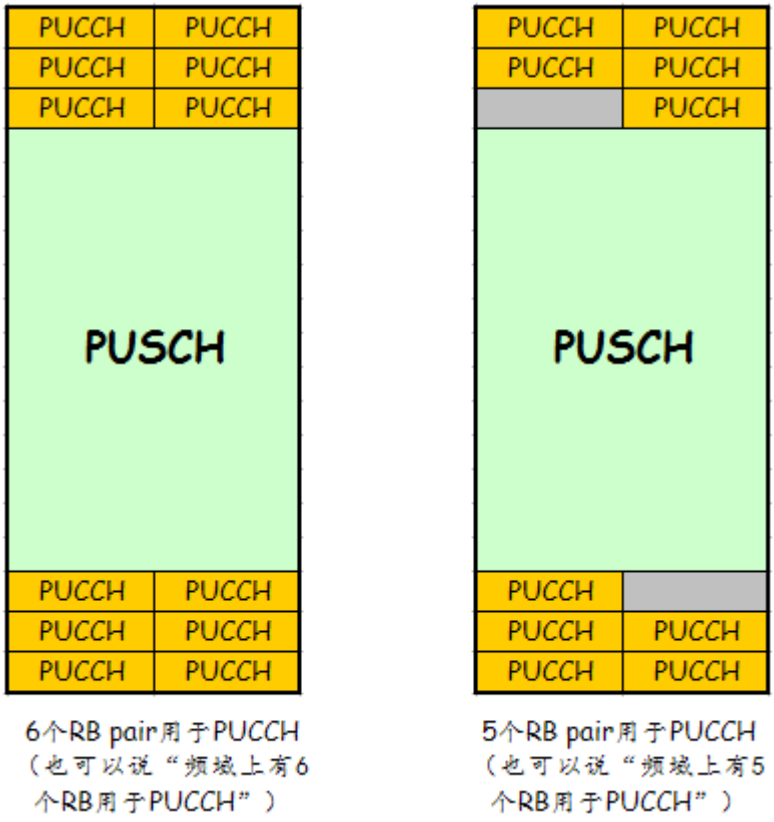


图 9-3：上行频域分配举例

如果只分配了奇数个 RB pair 给 PUCCH，则会有额外的一个 RB pair 不能用于 PUSCH 跳频，如图 9-3 中右边标记为灰色的部分。

为了计算出那些用于 PUSCH 的 RB，需要把 PUCCH 所占的 RB pair（以及灰色的部分）给剔除掉。

协议中使用 N_{RB}^{HO} 表示 PUCCH 在频域上所占的 RB 数 N_{RB}^{PUCCH}（或者说 RB pair 数），其值是通过 *pusch-hoppingOffset* 配置的。

需要剔除掉的不能用于 PUSCH 传输的 RB 数 $\tilde{N}_{RB}^{HO}$ 的计算公式为：

$$\begin{cases} \tilde{N}_{RB}^{HO} = N_{RB}^{HO} + 1; & N_{RB}^{HO} \text{是奇数时} \\ \tilde{N}_{RB}^{HO} = N_{RB}^{HO}; & N_{RB}^{HO} \text{是偶数时} \end{cases}$$

可用于 PUSCH 传输的 RB 数 N_{RB}^{PUSCH}（称之为跳频带宽：hopping bandwidth）按如下方式计算：

$$N_{RB}^{PUSCH}=\begin{cases} N_{RB}^{UL}-\tilde{N}_{RB}^{HO}-\left(N_{RB}^{UL} \bmod 2\right) & \text{Type 1 PUSCH hopping} \\ N_{RB}^{UL} & \text{Type 2 } N_{sb}=1 \text{ PUSCH hopping} \\ N_{RB}^{UL}-\tilde{N}_{RB}^{HO} & \text{Type 2 } N_{sb}>1 \text{ PUSCH hopping} \end{cases}$$

从上面的公式也可以看出，跳频带宽并不能覆盖整个上行系统带宽，这是因为上行频带的两端要预留一部分 RB 给 PUCCH 使用。

进行 PUSCH 跳频的 UE 会从同一 TB 的最近一个 DCI format 0 的资源分配字段（“Resource block assignment and hopping resource allocation”字段）中，确定用于一个子帧的第一个 slot（*S1*）的 PUSCH 资源分配（RA），该 RA 包含了该子帧的最小索引 PRB（lowest index PRB）信息 $n_{PRB}^{S1}(n)$。

注：“最近一个 DCI format 0”主要是针对上行非自适应重传而言的。新传或上行自适应重传都有一个对应的 DCI，用于明确地指定如何进行传输。而对于上行非自适应重传，必须依赖对应同一 TB 的最近一个 DCI format 0 来决定如何进行 PUSCH 跳频。

如果针对同一 TB，没有相应的 PDCCH，UE 会基于如下方式确定其跳频类型：

- 如果同一 TB 的初始 PUSCH 传输是半静态调度的，则 UE 会从最近接收到的用于上行半静态调度的 PDCCH 中获取跳频信息；（该 PDCCH 中与跳频相关字段的解析与动态调度中的 DCI format 0 是一致的）
- 如果 PUSCH 传输是由 RAR 中的授权指定的，则 UE 会从用于同一 TB 的 RAR 中的授权中获取跳频信息。（可参见 21.3.2 节的介绍）

eNodeB 还可以通过小区特定的配置参数 *hoppingMode* 来设置上行跳频所支持的跳频模式（hopping mode）：

- “intra and inter-subframe hopping”：即在“同一个子帧内和不同子帧之间”同时进行跳频。同一个子帧内的跳频是在 2 个 slot 之间进行的，它提供了一个 codeword 内的频率分集；

- “inter-subframe hopping”：在子帧之间进行跳频，同一子帧的 2 个 slot 之间是不进行跳频的。这种模式为同一 TB 的不同 HARQ 重传之间提供了频率分集。

9.2.1 PUSCH 跳频类型 1

跳频类型 1 使用 DCI format 0 中明确指定的跳变偏移（hopping offset）进行跳频。

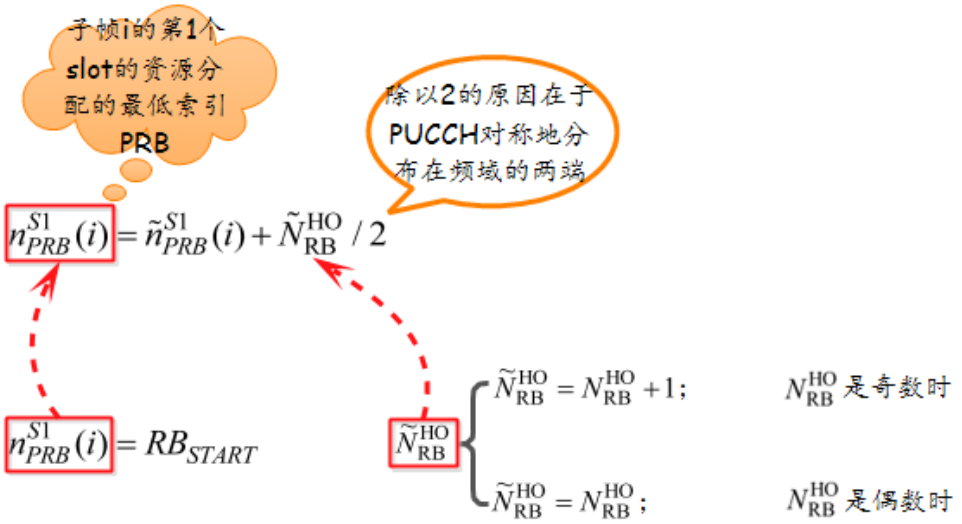
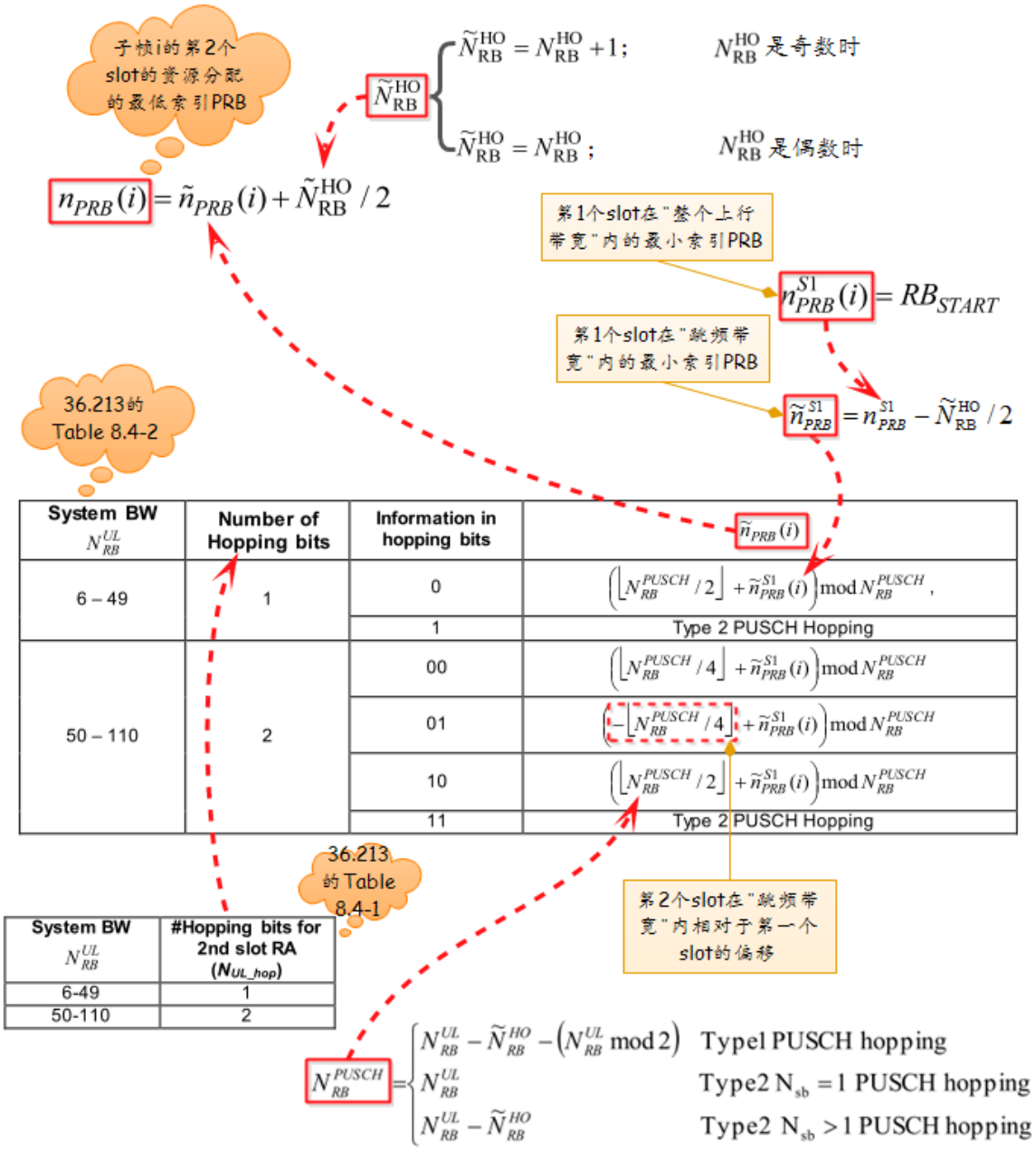


图 9-4：第 1 个 slot 中起始 PRB 的计算

$n_{PRB}^{S1}(n)$ 是用于一个子帧的第一个 slot（*S1*）的 PUSCH 资源分配的最小索引 PRB（lowest index PRB），其值等于 RB_{START}。$\tilde{n}_{PRB}^{S1}$是第一个 slot 中用于发送 PUSCH 的 RB 内（用于发送 PUCCH 的 RB 不计算在内）的最小索引 PRB，其取值范围为 $0 \le \tilde{n}_{PRB}^{S1} < N_{RB}^{PUSCH}$（图 9-6 的例子中，指出了 $\tilde{n}_{PRB}^{S1}=0$ 和 $\tilde{n}_{PRB}^{S1}=N_{RB}^{PUSCH}-1$所对应的 PRB）。



System BW N_{RB}^{UL}	Number of Hopping bits	Information in hopping bits	$\tilde{n}_{PRB}(i)$
6 – 49	1	0	$\left(\lfloor N_{RB}^{PUSCH}/2 \rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$,
		1	Type 2 PUSCH Hopping
50 – 110	2	00	$\left(\lfloor N_{RB}^{PUSCH}/4 \rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$
		01	$\left(-\lfloor N_{RB}^{PUSCH}/4 \rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$
		10	$\left(\lfloor N_{RB}^{PUSCH}/2 \rfloor + \tilde{n}_{PRB}^{S1}(i)\right) \bmod N_{RB}^{PUSCH}$
		11	Type 2 PUSCH Hopping

System BW N_{RB}^{UL}	#Hopping bits for 2nd slot RA (N_{UL_hop})
6-49	1
50-110	2

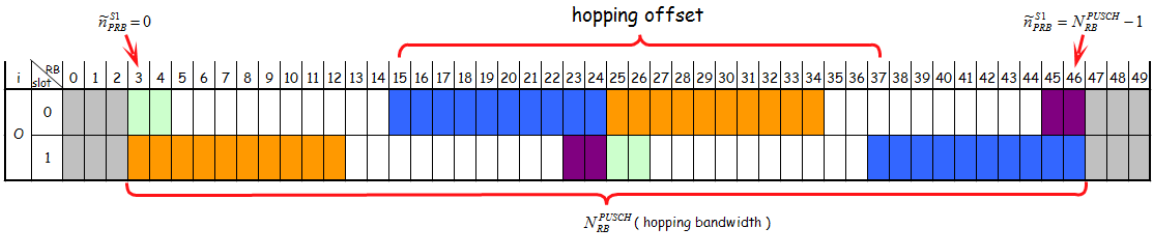
图 9-5：第 2 个 slot 中起始 PRB 的计算

图 9-5 是第 2 个 slot 的起始 PRB 的计算方式。可以看出，第二个 slot 是在第一个 slot 的基础上进行大约 1/2、+1/4、-1/4 跳频带宽的偏移。

用于 PUSCH 传输的 PRB 集合由 L_{CRBs} 个连续的 RB 组成，在第一个 slot 是从 PRB 索引 $n_{PRB}^{S1}(i)$ 开始的，在第二个 slot 是从 PRB 索引 $n_{PRB}(i)$ 开始的。

图 9-6 是在上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}}=50$，6 个 RB pair 用于 PUCCH 传输，且 hopping bit 的取值为“10”的情况下，使用 PUSCH 跳频类型 1 的一个例子。

Type 1 PUSCH hopping（example）	
Field	Value
$N_{\mathrm{RB}}^{\mathrm{UL}}$	= 50
$N_{\mathrm{RB}}^{\mathrm{HO}}$	= 6
$\tilde{N}_{\mathrm{RB}}^{\mathrm{HO}}$	$=N_{\mathrm{RB}}^{\mathrm{HO}}=6$
N_{UL_hop}	= 2（假设这里取值为“10”）
y	$=\lceil \log_2(N_{\mathrm{RB}}^{\mathrm{UL}}(N_{\mathrm{RB}}^{\mathrm{UL}}+1)/2)\rceil - N_{UL_hop}=9$
N_{RB}^{PUSCH}	$=N_{RB}^{UL}-\tilde{N}_{RB}^{HO}-(N_{RB}^{UL} \bmod 2)=50-6-(50 \bmod 2)=44$
可分配的最大连续 RB 数	$\lfloor 2^y / N_{\mathrm{RB}}^{\mathrm{UL}} \rfloor=\lfloor 512/50 \rfloor=10$
Hopping offset	$\lfloor N_{RB}^{PUSCH}/2 \rfloor=\lfloor 44/2 \rfloor=22$



PUCCH
UE1
UE2
UE3
UE4

UE	RIV	RB_{START}	L_{CRBs}	$n_{PRB}^{S1}(i)$	$\tilde{n}_{PRB}^{S1}$	$n_{PRB}(i)$
UE1	53	3	2	3	0	25
UE2	465	15	10	15	12	37
UE3	475	25	10	25	22	3
UE4	95	45	2	45	42	23

图 9-6：Type 1 intra-subframe PUSCH hopping

PUSCH 跳频类型 1 的公式看似复杂，但其要表达的意思很简单：第一个 slot 不跳频。第二个 slot 在第一个 slot 的基础上，在“跳频带宽”范围内发生$\lfloor N_{RB}^{PUSCH}/2 \rfloor$、或$\lfloor N_{RB}^{PUSCH}/4 \rfloor$、或$-\lfloor N_{RB}^{PUSCH}/4 \rfloor$的偏移。

前面介绍的是基于“intra-subframe”的处理。如果 *hoppingMode* 配置成 “inter-subframe”（不同子帧之间），则第一个 slot 的 RA 应用于 CURRENT_TX_NB 为偶数的子帧，第二个 slot 的 RA 应用于 CURRENT_TX_NB 为奇数的子帧。也就是说，“inter-subframe”模式下，同一子帧上的同一 RB 在 2 个 slot 上的索引是相同的；“intra-subframe”模式下，同一子帧上的同一 RB 在 2 个 slot 上的索引是不同的。

每个上行 HARQ process 都会维护一个状态变量 CURRENT_TX_NB，该变量表示当前 HARQ buffer 中的某个 MAC PDU 已经发生的传输次数（对应同一个 TB，包括初传和重传）。如果是新传，HARQ process 会将该变量设置为 0；每发生一次重传，该变量会加 1。当达到最大重传次数时，该变量会清零。（见 36.321 的 5.4.2.2 节）

9.2.2 PUSCH 跳频类型 2

跳频类型 2 是根据小区特定的跳频/镜像样式（hopping/mirroring pattern）进行基于 subband 的跳频。其使用的是一个预先定义的与小区 PCI 相关的伪随机数跳频样式。

对于 PUSCH 跳频类型 2，跳频带宽（hopping bandwidth）会被划分为大小相等的多个 subband，每个 subband 由多个连续的 RB 组成。

subband 的个数 N_{sb} 是通过 *n-SB* 配置的。而每个 subband 包含的 RB 数 $N_{\text{RB}}^{\text{sb}}$ 通过如下方式计算：

$$N_{\text{RB}}^{\text{sb}} = \begin{cases} N_{\text{RB}}^{\text{UL}} & N_{\text{sb}} = 1 \\ \left\lfloor \left(N_{\text{RB}}^{\text{UL}} - N_{\text{RB}}^{\text{HO}} - N_{\text{RB}}^{\text{HO}} \bmod 2 \right) / N_{\text{sb}} \right\rfloor & N_{\text{sb}} > 1 \end{cases}$$

$\left(N_{\text{RB}}^{\text{UL}} - N_{\text{RB}}^{\text{HO}} - N_{\text{RB}}^{\text{HO}} \bmod 2 \right)$计算的是可用于 PUSCH 传输的 RB 数 N_{RB}^{PUSCH}，即跳频带宽所包含的 RB 数，其计算结果跟之前介绍的 N_{RB}^{PUSCH} 是一致的。

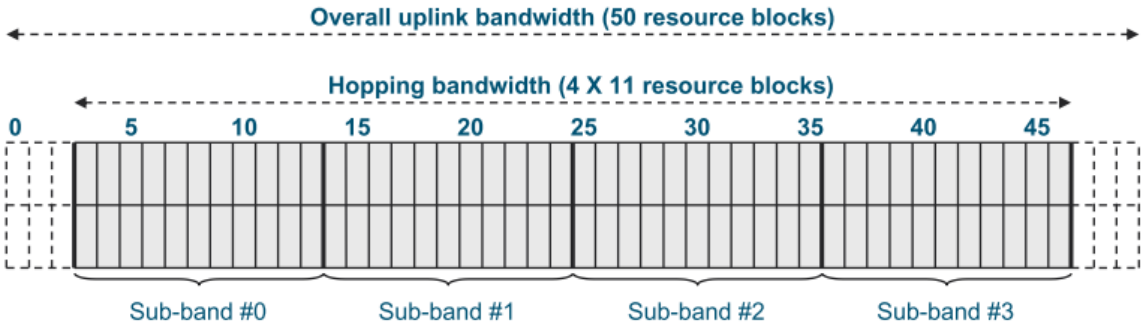


图 9-7：PUSCH 跳频中的 subband 划分的一个例子

图 9-7 是 subband 划分的一个例子。整个上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}}=50$ ，配置了 4 个 subband，每个 subband 包含 11 个 RB，频域的两端共有 6 个 RB 用于传输 PUCCH。

PUSCH 跳频类型 2 中，用于 PUSCH 传输的 PRB，是通过将 UL grant 中指定的 VRB 偏移多个 subband 得到的，且不同的 slot 可以有不同的偏移量。

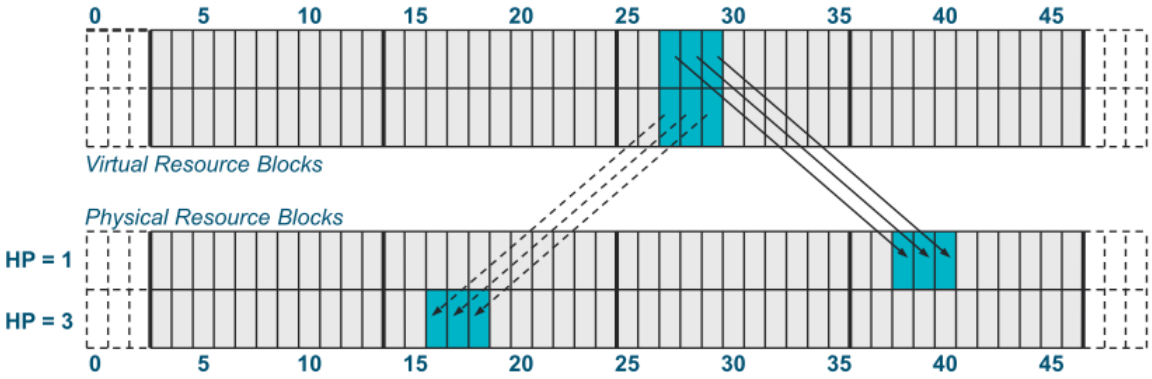


图 9-8：PUSCH 跳频类型 2 一个例子

图 9-8 是 PUSCH 跳频类型 2 的一个例子，一个 UE 被分配了 VRB 27、28 和 29。在第 1 个 slot，往右偏移 1 个 subband，得到 PRB 38、39 和 40。在第 2 个 slot，往右偏移 3 个 subband（等价于往左偏移一个 subband，即偏移是“环绕”的），得到 PRB 16、17 和 18。

由于跳频样式是小区特定的，即该小区内的所有 UE 都遵循相同的跳频样式，而不同的 UE 会分配非重叠的 VRB，因此其物理资源 PRB 也不会发生重叠。

除了跳频样式外，小区还定义了一个小区特定的镜像样式。 $f_{\mathrm{m}}(i) \in \{0,1\}$ 决定是否使用镜像。镜像是基于一个 slot 生效的，如果某个 slot 使用镜像的话（ $f_{\mathrm{m}}(i)=1$ ），则该 slot 的每个 subband 内的 RB 编号是“从右往左”依次递增的；如果某个 slot 不使用镜像的话（ $f_{\mathrm{m}}(i)=0$ ），则该 slot 的每个 subband 中的 RB 编号是“从左往右”依次递增编号的。如图 9-9 所示。

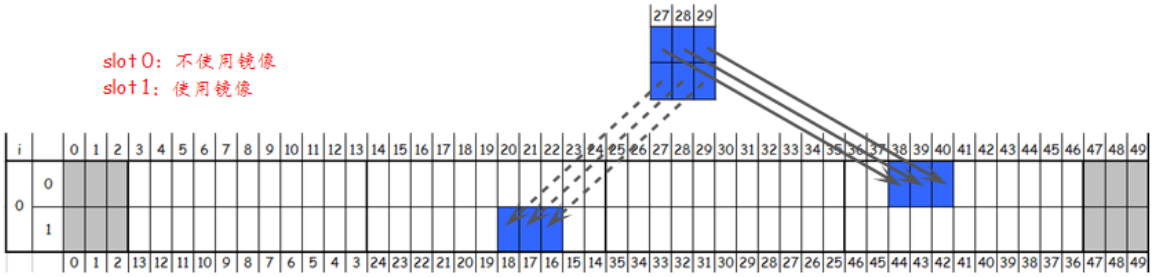


图 9-9：镜像的一个例子

图 9-9 是镜像的一个例子：第 1 个 slot 没有使用镜像，第 2 个 slot 使用了镜像。一个 UE 被分配了 VRB 27、28 和 29。在第 1 个 slot，往右偏移 1 个 subband，得到 PRB 38、39 和 40。在第 2 个 slot，往右偏移 3 个 subband（等价于往左偏移一个 subband，即偏移是“环绕”的），得到 PRB 16、17 和 18，但由于第二个 slot 使用了镜像，因此实际分配的是 PRB 20、21 和 22。

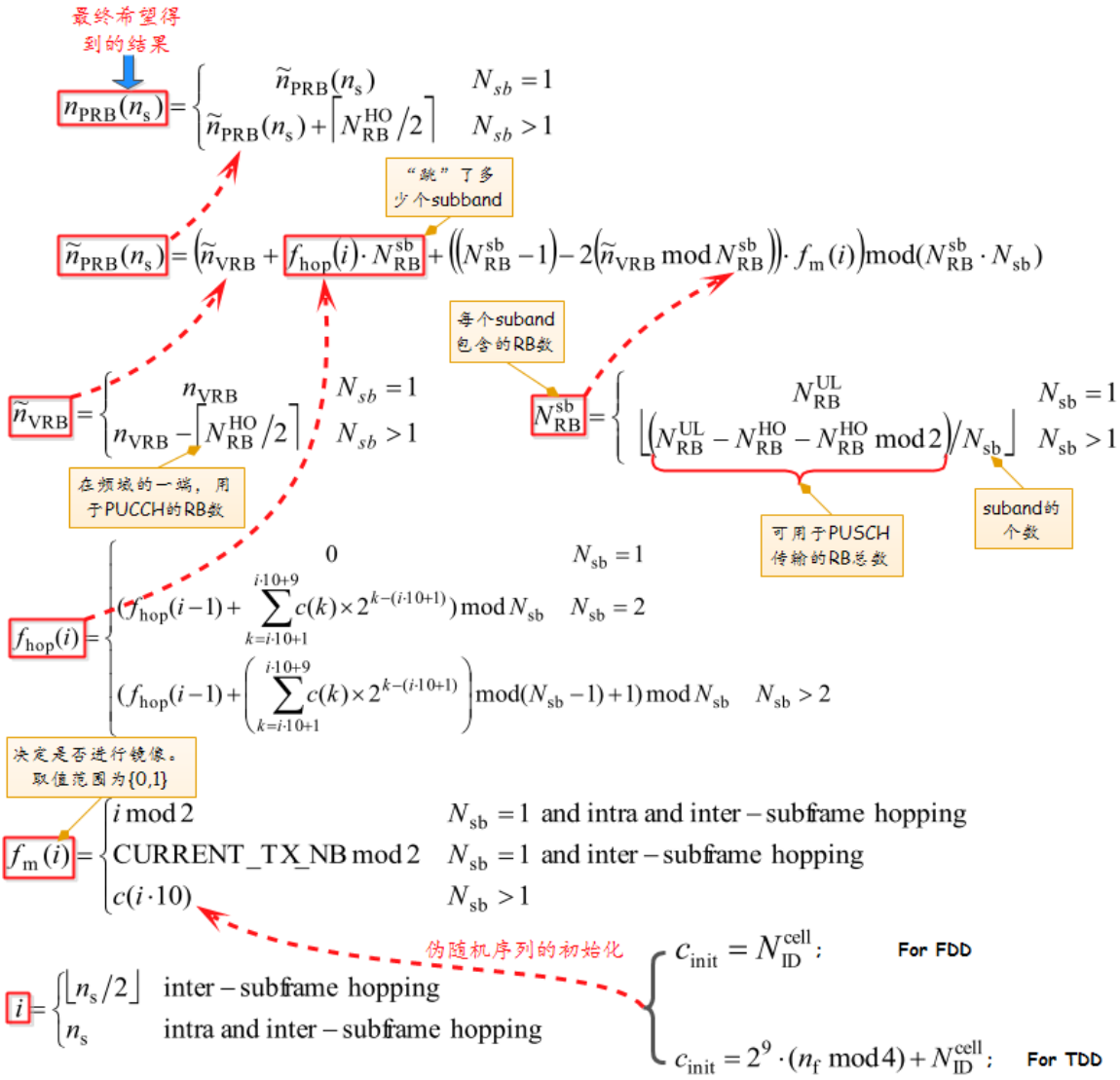


图 9-10：Type 2 PUSCH hopping

图 9-10 是 PUSCH 跳频类型 2 的 PRB 的计算公式，可以看出，跳频样式和镜像样式都与 PCI 相关，因此在邻居小区间通常是不同的。而且跳频/镜像样式的周期变化对应一个系统帧（10 ms）。

而且可以看出，如果 $N_{sb}=1$，那么 $f_{hop}(i)=0$ 时，只能进行镜像（当 $f_m(i)=1$ 时）。

对于 TDD，如果 UE 还没有获取到 SFN，则 UE 不会传输跳频类型为 Type 2 且 $N_{sb}>1$ 的 PUSCH。

对于跳频类型 2，同样可以使用 *hoppingMode* 来配置 PUSCH 跳频是“inter-subframe”的还是“intra and inter-subframe”的。

由于有 1 或 2 比特（hopping bit）需要被用于指示跳频信息，因此限制了分配给某个 UE 的连续 RB 数：

- 可以分配给 Type 1 跳频的连续 RB 数不能大于 $\lfloor 2^y / N_{\mathrm{RB}}^{\mathrm{UL}} \rfloor$ 个。
- 可以分配给 Type 2 跳频的连续 RB 数不能大于 $\min(\lfloor 2^y / N_{\mathrm{RB}}^{\mathrm{UL}} \rfloor, \lfloor N_{RB}^{PUSCH} / N_{sb} \rfloor)$ 个。

可以看出，对于 Type 2 跳频，带宽一定的情况下，如果 subband 的个数增加，则可分配给一个 UE 的连续 RB 数将变少；与此同时，分配给一个 UE 的所有 RB 只能位于同一个 subband 中，而不能跨 subband 分配。

类型 1 和类型 2 的比较：

从上面的介绍可以看出，通过改变 DCI format 0 中的对应字段，可以在跳频类型 1 和跳频类型 2 之间动态切换。

对于 PUSCH 跳频类型 1，有 3 个不同的跳频选项（偏移：1/2、1/4、-1/4 跳频带宽），在使用“intra and inter-subframe hopping”时，其变化周期为 1 个子帧（即在同一子帧的 2 个 slot 之间进行跳频）；在使用“inter-subframe hopping”时，其变化周期为 2 个子帧（即在同一 TB 的奇数次传输和偶数次传输之间进行跳频）。

而 PUSCH 跳频类型 2，提供了使用不同样式的跳频和镜像，且变化周期为 10 个子帧（即同一系统帧内的不同子帧间是不同的）。因此与 PUSCH 跳频类型 1 相比，提供了更多的频率分集。

与此同时，PUSCH 跳频类型 2 是小区特定的，而邻居小区的 PCI 通常是不同的，这也可以为降低小区间干扰（inter-cell interference）提供帮助。

注：公式中并未禁止 PUSCH 跳频到 PRACH 所占的 RB 资源的情况，这是因为协议上是允许 PRACH 和 PUSCH 在同一 RB 上传输的。

【参考资料】

[1] TS 36.211 的 5.3.4 节 Mapping to physical resources

[2] TS 36.213 的 8.4 节 UE PUSCH Hopping procedure

[3] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.1.3 节

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.2.1.2 节

[5] 《Clarification for PUSCH hopping type 1》by Ericsson

[6] 《Frequency Hopping in LTE Uplink》

[7] 《LTE uplink frequency hopping》

9.3 如何确定调制阶数、冗余版本和 TB size

UE 如何根据接收到的 DCI format 0/4 来确定对应的 PUSCH 传输使用调制阶数（modulation order）、冗余版本（redundancy version）以及 TB size 呢？

首先，UE 需要确定如下信息：

- 读取“modulation and coding scheme and redundancy version”字段，得到 I_{MCS} ；
- 读取“CSI request”字段；
- 根据 9.1 节中关于“上行资源分配类型”的介绍（或见 36.213 的 8.1 节），计算出分配给该 UE 的 PRB 数 N_{PRB} ；
- 计算出控制信息的编码符号数。

9.3.1 确定调制阶数

UE 发送的 PUSCH 所使用的调制阶数 Q_m 取决于以下几个因素：（1）UE 是否有支持使用 64QAM 调制 PUSCH 的能力，见 36.306 的 Table 4.1-2；（2）eNodeB 基于某些考虑，可以配置小区内的 UE 不使用 64QAM 来调制 PUSCH。eNodeB 可以通过 SIB2 的 *enable64QAM* 字段配置小区内的 UE 是否使能 64QAM 调制（小区级的配置）；（3）通过 I_{MCS} 查 36.213 的 Table 8.6.1-1 得到的 Q_m' ；（4）UE 是否使能 TTI Bundling。

Table 8.6.1-1: Modulation, TBS index and redundancy version table for PUSCH

MCS Index I_{MCS}	Modulation Order Q'_m	TBS Index I_{TBS}	Redundancy Version rv_{idx}
0	2	0	0
1	2	1	0
2	2	2	0
3	2	3	0
4	2	4	0
5	2	5	0
6	2	6	0
7	2	7	0
8	2	8	0
9	2	9	0
10	2	10	0
11	4	10	0
12	4	11	0
13	4	12	0
14	4	13	0
15	4	14	0
16	4	15	0
17	4	16	0
18	4	17	0
19	4	18	0
20	4	19	0
21	6	19	0
22	6	20	0
23	6	21	0
24	6	22	0
25	6	23	0
26	6	24	0
27	6	25	0
28	6	26	0
29	reserved		1
30			2
31			3

如果$0 \le I_{MCS} \le 28$，则调制阶数Q_m按照如下方式确定：

- 如果 UE 支持使用 64QAM 来调制 PUSCH，且 eNodeB 没有配置只使用 QPSK 和 16QAM 来调制 PUSCH，则调制阶数Q_m等于通过I_{MCS}查 36.213 的 Table 8.6.1-1 得到的Q'_m。
- 如果 UE 不支持使用 64QAM 来调制 PUSCH，或 eNodeB 配置了只使用 QPSK 和 16QAM 来调制 PUSCH，则调制阶数Q_m等于$Q_m = \min(4, Q'_m)$。注：（1）UE 在上行是否支持 64QAM 调制与 *ue-Category* 相关，其对应关系见 36.306 的 Table 4.1-2；（2）eNodeB 可以通过 IE: *enable64QAM* 告诉 UE，该小区是否支持 64QAM 来调制 PUSCH。
- 如果 *ttiBundling* 设置为 TRUE（即 UE 使能了 TTI bundling），则分配给该 UE 的上行资源必须满足$N_{PRB} \le 3$，且其调制模式必须为$Q_m = 2$。

如果 $29 \le I_{\mathrm{MCS}} \le 31$，则调制阶数 Q_m 按照如下方式确定：

- 如果“使用 DCI format 0 且 $I_{\mathrm{MCS}} = 29$”，或“使用 DCI format 4，但只使能了 1 个 TB，且使能的 TB 的 $I_{\mathrm{MCS}} = 29$，且使用的传输层数为 1”；并且同时满足以下条件，则调制模式设为 $Q_m = 2$：
    - “CSI request”字段为 1 比特，且该比特设置为触发非周期性 CSI 上报，且 $N_{\mathrm{PRB}} \le 4$；或
    - “CSI request”字段为 2 比特，且根据 36.213 的 Table 7.2.1-1A，触发了 1 个 serveing cell 的非周期性 CSI 上报，且 $N_{\mathrm{PRB}} \le 4$；或
    - “CSI request”字段为 2 比特，且根据 36.213 的 Table 7.2.1-1A，触发了多于 1 个 serveing cell 的非周期性 CSI 上报，且 $N_{\mathrm{PRB}} \le 20$。
- 否则，UE 会根据针对同一 TB 的最近一次收到的 $0 \le I_{\mathrm{MCS}} \le 28$ 的 DCI format 0/4 来决定调制阶数 Q_m。如果对于同一 TB，并没有对应 $0 \le I_{\mathrm{MCS}} \le 28$ 的 DCI format 0/4，此时其调制阶数会按照如下方式确定：
    - 当同一 TB 的初始 PUSCH 传输是半静态调度（SPS）的时，则使用最近一次半静态调度使用的 PDCCH 来决定当前传输的调制阶数；或者
    - 当 PUSCH 传输是由 RAR 中的 UL grant 指定的时，则同一 TB 的调制阶数也由 RAR 指定。（见 36.213 的 6.2 节）

9.3.2 确定冗余版本 RV

当 UE 收到 DCI format 0/4 时，会通过 I_{MCS} 查 36.213 的 Table 8.6.1-1 得到对应 PUSCH 传输使用的冗余版本（rv_{idx}）。

9.3.3 确定 TB size

对于 $0 \le I_{\mathrm{MCS}} \le 28$ 而言，如果 DCI format 4 中的 TB 没有被去使能，则 UE 会先通过 I_{MCS} 查 36.213 的 Table 8.6.1-1 得到 TBS 索引 I_{TBS}，然后结合之前计算出的 N_{PRB}，按照 36.213 的 7.1.7.2.1 节（1 个 TB 只映射到 1 层）或 7.1.7.2.2 节（1 个 TB 映射到 2 层）的介绍查表得到 TB size。

对于 $29 \le I_{\mathrm{MCS}} \le 31$ 而言，

- 如果“使用 DCI format 0 且 $I_{\mathrm{MCS}} = 29$”，或“使用 DCI format 4，但只使能了 1 个 TB，且使能的 TB 的 $I_{\mathrm{MCS}} = 29$，且使用的传输层数为 1”；并且同时满足以下条件，则 UE 并不会发送

UL-SCH 数据，即不发送 TB，而只会发送用于当前 PUSCH 上报模式的控制信息（UCI）：

  - “CSI request”字段为 1 比特，且该比特设置为触发非周期性 CSI 上报，且 $N_{\mathrm{PRB}} \le 4$；或
  - “CSI request”字段为 2 比特，且根据 36.213 的 Table 7.2.1-1A，触发了 1 个 serveing cell 的非周期性 CSI 上报，且 $N_{\mathrm{PRB}} \le 4$；或
  - “CSI request”字段为 2 比特，且根据 36.213 的 Table 7.2.1-1A，触发了多于 1 个 serveing cell 的非周期性 CSI 上报，且 $N_{\mathrm{PRB}} \le 20$。

- 否则，UE 会根据针对同一 TB 的 $0 \le I_{\mathrm{MCS}} \le 28$ 的初始 DCI format 0/4 来决定 TB size（注意：因为上行重传不能改变 TB size，所以这里使用的是“初传”对应的 DCI format 0/4；但在决定调制阶数时，使用的是“最近一次传输”对应的 DCI format 0/4）。如果对于同一 TB，并没有对应 $0 \le I_{\mathrm{MCS}} \le 28$ 的初始 DCI format 0/4，此时其 TB size 会按照如下方式确定：
  - 当同一 TB 的初始 PUSCH 传输是半静态调度（SPS）的时，则使用最近一次半静态调度使用的 PDCCH 来决定当前传输的 TB size；或者
  - 当 PUSCH 传输是由 RAR 中的 UL grant 指定的时，则同一 TB 的 TB size 也由 RAR 指定。

需要注意的是，在上行空分复用（TM 2）下，UE 在一个 TTI 会至多发送 2 个 TB，每个 TB（每个 TB 只能映射到至多 2 层）的 TB size 是独立计算的。但这 2 个 TB 的 N_{PRB} 是相同的。

UE 接收到 DCI format 4，如果某个 TB 对应的“$I_{\mathrm{MCS}} = 0$ 且 $N_{\mathrm{PRB}} > 1$”或“$I_{\mathrm{MCS}} = 28$ 且 $N_{\mathrm{PRB}} = 1$”，则该 TB 是去使能的；否则该 TB 是使能的。（见 36.213 的 8.6.2 节）

那些用于传输 DMRS 以及 SRS 的 SC-FDMA 符号不能用于传输 PUSCH。

与下行相比，在选择上行的 TBS 时，需要考虑的因素更少，也更加简单。PUSCH 传输时需要扣除的 RE 包括：

- 对于 PUSCH 而言，DMRS 位于每个 slot 的倒数第 4 个 SC-FDMA 符号上。因此对应 PUSCH 传输，每个 TTI 内有 2 个 SC-FDMA 符号被用于 DMRS，这部分 RE 需要剔除。
- 在频域上，PUCCH 占用的 RB 不能用于传输 PUSCH。

- 在频域上，当某个上行子帧需要传输 PRACH 时，需要排除 PRACH 所占的 RB。因为 PRACH 所占的 RB 通常不用于发送 PUSCH。

这里我们以 TDD 2，上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}} = 100RB$ (即信道带宽为 20MHz)，小区使用正常的循环前缀，且上行使用单天线端口传输为例，来介绍上行子帧 2（其它上行子帧的计算思路类似，这里就不做介绍了）如何选择最大的 TBS。假设 UE 在上行支持的最高调制阶数为 16QAM（$Q_m = 4$），且整个上行系统带宽都分配给了该 UE。同时假设在该子帧，分配了 4 个 RB 给 PUCCH 使用，且分配了 6 个 RB 给 PRACH 使用。

此时可分配给 PUSCH 传输的 RB 数为：100 – 4(PUCCH) – 6(PRACH) = 90。由于每个子帧内有 2 个 SC-FDMA 符号要预留给 DMRS 使用，所以每个 TTI 内可用的 SC-FDMA 符号数为 14 -2 =12。也就是说，在该子帧内，可用于 PUSCH 传输的 RE 数为：90(RB) * 12(symbol) * 12(RE/symbol) = 12960。由于 UE 在上行支持的最高调制阶数为 16QAM（$Q_m = 4$），因此有 PUSCH 上的物理信道比特数为 4 * 12960 = 51840。

与下行 PDSCH 传输不同，协议中并未要求上行 PUSCH 传输的初传 TB 的有效信道码率不能高于 0.930，当然其有效信道码率不能超过 1。此时的 $N_{\mathrm{PRB}} = 90$，通过查 36.213 的 Table 7.1.7.2.1-1 可知，小于 51840 的第一个 TBS 值为 51024，对应 $I_{\mathrm{TBS}} = 23$，$I_{\mathrm{MCS}} = 25$。

从 36.212 的 5.1.2 节可以看出，此时所需的 code block 数为 $C = \lceil B/(Z-L) \rceil = \lceil (51024+24)/(6144-24) \rceil = 9$。其中“B = TBS + 24 位 CRC”，L=24 是 CRC 的长度，每个 code block 都要各自加上 CRC。计算码率时，是将 CRC 计算在内的，所以这里要加上 10 * 24 = 240 比特的 CRC。此时的码率为：(51024 + 24*10) / 51840 ≈ 0.988888889。虽说此码率小于 1，但通常还是认为这个值过高了。

我们继续往下选择 $I_{\mathrm{MCS}} = 24$，对应 $I_{\mathrm{TBS}} = 22$，查 36.213 的 Table 7.1.7.2.1-1 得到 TBS = 48936，此时所需的 code block 数为 $C = \lceil B/(Z-L) \rceil = \lceil (48936+24)/(6144-24) \rceil = 8$。此时的码率为：(48936 + 24*9) / 51840 ≈0.94814815。这是一个可接受的值（eNodeB 在选择 TBS 时，通常需要考虑更多因素，如不同 UE category 的能力等）。

需要说明的是，对应 $I_{\mathrm{MCS}} = 24$，查 36.213 的 Table 8.6.1-1 可知，此时的 $Q_m' = 6$。也就是说，虽然 UE 最高支持 $Q_m = 4$，但并不影响选择 TBS 时，其 I_{MCS} 对应的 Q_m' 落在大于 4 的区间内。

如果将一个系统帧内的每个上行子帧所能选择的最大 TBS 计算出来，就能较精确地计算出协议上定义的理论上的上行最大速率。

【参考资料】

[1] TS 36.213 的 8.6 节 Modulation order, redundancy version and transport block size determination

[2] http://wenku.baidu.com/view/6c1fd2b4ce2f0066f533228c.html

9.4 上行 TM 模式

在 Rel-8/ Rel-9 中，上行只支持单天线传输。而在 Rel-10 中引入了上行多天线传输，以支持上行 PUSCH 空分复用以及 PUCCH 的传输分集。本章主要介绍 Rel-10 中的上行 PUSCH 传输。

Rel-10 中对于 PUSCH 传输也引入了传输模式（Transmission Mode，TM）的概念（见 36.213 的 Table 8-3）：

TM 1：只支持单天线端口传输。使用 DCI format 0 来指示上行传输。

TM 2：支持 2 或 4 天线端口传输，即上行的闭环空分复用，此时使用 DCI format 4 来指示上行传输。但在信道条件较差时，也可以动态地切换到单天线端口传输，此时使用 DCI format 0 来指示上行传输（这里不使用上行传输分集的原因可参见参考资料[2]中的介绍）。

注：从 36.213 的 Table 8-5 和 Table 8-6 可以看出，无论是 SPS 调度的上行传输（使用 SPS C-RNTI 加扰），还是使用 TC-RNTI 加扰的 Msg3 传输，都只能使用 DCI format 0 来指示其上行 PUSCH 传输，也就是说此时只支持单天线端口传输而不支持上行空分复用。

对于 PUSCH 传输，多天线传输基于天线预编码，如图 9-11 所示。与下行的基于非码本的预编码类似，DMRS 也是在预编码之前插入到每个天线端口的，其中每层对应一个的已经预编码处理过的 DMRS。上行多天线传输支持至多 4 个天线端口，允许至多 4 层空分复用。

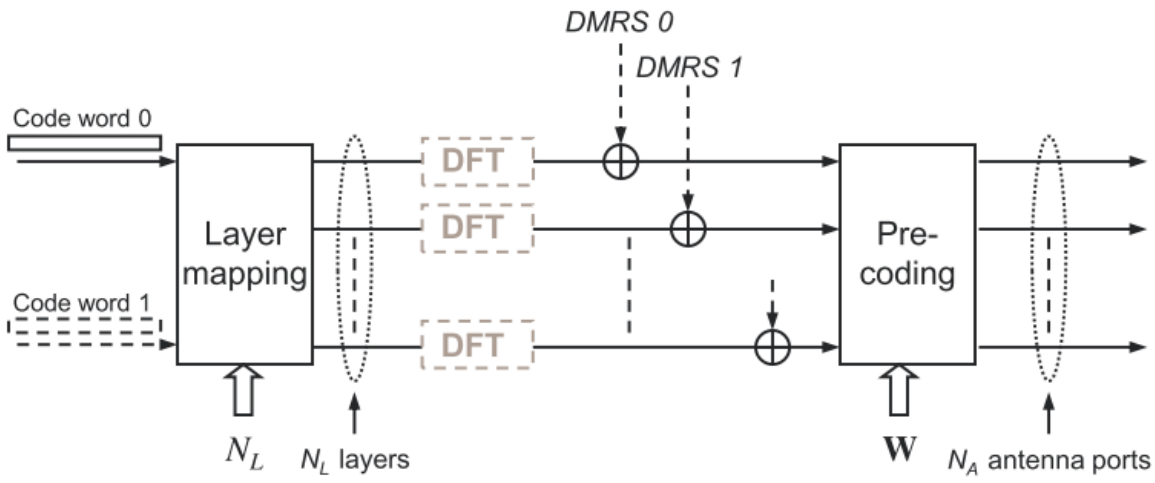


图 9-11：基于预编码的上行多天线传输

对于上行传输来说，每个 UE 在一个 TTI 至多发送 2 个 TB（如果配置了上行载波聚合，每个 TTI 在每个小区至多发送 2 个 TB）。其 TB 到层的映射与下行类似：对于初传而言，1 层传输只对应 1 个 TB，而 2 个 TB 对应 2 个或更多传输层。而对于 HARQ 重传，一个 TB 可能映射到多层传输。

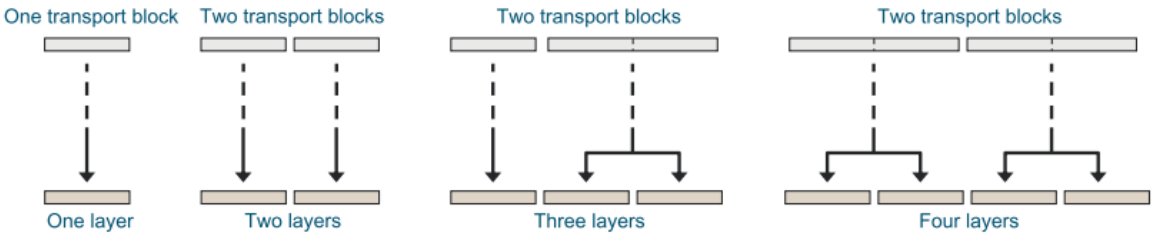


图 9-12：码字映射到层的举例（初传）

经过层映射后，每层所包含的调制符号数是相同的，并等于在每个天线端口上发送的调制符号数。上行的层映射处理可见 36.211 的 5.3.2A 节，其处理与下行的层映射处理并无太大区别。

对于上行 HARQ，每个 TB 是独立进行 HARQ 确认的，并使用不同的 PHICH 索引。在 4.3 节介绍 PHICH 时，我们已经说过，UE 在确定使用的 PHICH 资源时，会使用 I_{PRB_RA} 来计算二元组 $\left(n_{\mathrm{PHICH}}^{\mathrm{group}}, n_{\mathrm{PHICH}}^{\mathrm{seq}}\right)$。如果使用上行空分复用，则对于第一个 TB，$I_{PRB_RA}$ 的值等于 $I_{PRB_RA}^{lowest_index}$，对于第二个 TB，$I_{PRB_RA}$ 的值等于 $I_{PRB_RA}^{lowest_index}+1$。其中，$I_{PRB_RA}^{lowest_index}$ 为对应的 PUSCH 传输在第一个

slot 的最低 PRB 索引。通过 $I_{PRB_RA}^{lowest_index}$，就能够区分出同一上行子帧发送的 2 个 TB 所使用的不同 PHICH 索引。

与下行的基于非码本的预编码处理不同，图 9-11 中的预编码处理部分并未进行阴影化处理，即上行预编码处理对于 3GPP 的协议规范来说，是可见的。

在下行的基于非码本的预编码中，天线匹配的预编码部分在协议规范中是没有定义的，eNodeB 可以使用任意的预编码用于下行传输。由于下行使用的是经过预编码处理后的 UE 特定的参考信号，UE 不需要知道 eNodeB 侧所使用的预编码矩阵也能够恢复出不同层的传输。

对于上行传输来说，使用的也是经过预编码处理后的 DMRS，实际上 eNodeB 也不需要知道 UE 侧所使用的预编码矩阵就能恢复出不同层上的传输。但是，上行的预编码矩阵是由 eNodeB 来选择，并通过 DCI format 4 的 Precoding information and number of layers 字段告诉 UE 的（DCI format 0 只支持单天线端口、单层传输，因此不需要告诉 UE 预编码信息）。UE 接收到调度结果后，会按照 eNodeB 选择的预编码矩阵来进行预编码处理。因此在协议规范中，上行的预编码处理部分是可见的。

为了控制 DCI format 4 的大小，协议中只支持有限数量的预编码矩阵集合。如果上行使用 2 天线端口传输，其可使用的预编码矩阵见 36.211 的 Table 5.3.3A.2-1（可以看出，此时共有 6 个 rank=1 矩阵，1 个 rank=2 矩阵）；如果上行使用 4 天线端口传输，其可使用的预编码矩阵见 36.211 的 Table 5.3.3A.2-2 至 Table 5.3.3A.2-5（可以看出，此时共有 24 个 rank=1 矩阵，16 个 rank=2 矩阵，12 个 rank=3 矩阵，1 个 rank=4 矩阵）。对于全 rank 的传输，即传输层数等于天线端口数的情况，只定义了一个 N×N 的单位矩阵，其中 N 为天线端口数。

协议中定义的用于上行传输的所有预编码矩阵，每一行有且仅有一个非零的元素。即预编码矩阵在将层映射到天线端口时，至多只有一层会映射到每个天线端口上，也就是说，每个天线端口只会传输某一层的数据，而不会传输其它层的数据（大家将预编码矩阵代入 36.211 的 5.3.3A.2 节介绍的层到天线端口的映射公式，就能得出这个结论）。这样能够保持每个天线端口上传输的信号的立方度量特性。

为了选择一个合适的预编码矩阵，eNodeB 需要通过测量 UE 上报的 SRS 以获取上行信道信息。SRS 是直接在不同的天线端口上传输且不进行预编码处理的，因此 SRS 直接反应了每个天线端口的信道，并且不包含任何预编码信息。基于接收到的 SRS，eNodeB 可以选择一个合适的传输层数，以及对应的上行预编码矩阵，并将结果通过 DCI format 4 告诉 UE。

SRS 传输和 PUSCH 传输可以使用相同的天线端口数，例如 SRS 和 PUSCH 都使用 2 个天线端口；也可以使用不同的天线端口数，例如上行 PUSCH 传输只使用 2 个天线端口，而 eNodeB 却可以配置 SRS 来检测 4 个天线端口（对应可能存在 4 层传输的情况）的信道条件。在这种情况下，SRS 在不同的天线端口集合上传输，以便在信道条件允许的情况下，eNodeB 可以将上行 PUSCH 传输从 2 层传输切换到 4 层传输。

对于 SU-MIMO，即分配给 UE 的时频资源只由该 UE 独占的场景，需要区分不同层之间的传输，这是通过每层上使用一个相互正交的 DMRS 来实现的。

与下行 MU-MIMO 类似，上行 MU-MIMO 意味着多个 UE 使用相同的上行时频资源来发送上行数据，这要求 eNodeB 使用多根接收天线来区分这 2 个或更多个不同的传输。

上行 MU-MIMO 可以获得与 SU-MIMO 类似的系统吞吐量（throughput），而无需 UE 侧使用多根发射天线，从而降低了 UE 的实现复杂度。

上行 MU-MIMO 并不要求 UE 知道是否在同样的时频资源上存在其它 UE 的传输，并允许不同的 UE 只在部分资源上存在重合。当然 eNodeB 必须在合适的信道条件下才会配对 UE。

从实现上看，支持上行 MU-MIMO 只要求显式地给配对的 UE 分配相互正交的 DRMS 就可以了。MU-MIMO 中使用的 DMRS 结构与 SU-MIMO 类似，都是基于 OCC 和循环移位来区分不同 UE 使用的 DMRS。通过给不同的 UE 配置合适的 Cyclic shift for DMRS and OCC index（通过 DCI format 0/4），来指示使用的 OCC 和循环移位的组合。

MU-MIMO 同时允许配对 UE 中的一个或几个 UE 使用多于一层，即同时允许某些 UE 使用 SU-MIMO。此时只要保证分配给这些 UE 的 DMRS 彼此正交即可。

关于上行 SU-MIMO 和 MU-MIMO 的 DMRS 配置，可参见 7.1.2 节关于“DMRS”的介绍。在该节中，详细介绍了如何配置 DMRS 以支持上行 SU-MIMO 和 MU-MIMO。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.3 节

[2] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 29.4.1 节

第10章 CSI

无线信道条件通常是在不断变化的。例如：UE 从一个信道条件好的区域移动到一个信道条件差的区域；或者 UE 在两旁有高大的建筑物的街道上移动，那么沿信号传播方向的街道上信号将增强，而垂直于传播方向的街道上信号将减弱。

假设在信道质量很好的情况下，下行使用了 QPSK 而不是更高阶的调制方式（16QAM/64QAM）进行传输的话，对频谱的利用率就会变低，并造成吞吐量的下降。而假设在信道质量很差的情况下，使用了高阶调制（16QAM/64QAM）而不是 QPSK 调制的话，就会造成过多的重传。无论是哪种情况，都没能有效地利用无线资源。

为了更好地适应无线信道的变化，UE 可以通过 CSI 将下行信道质量信息上报给 eNodeB，以便 eNodeB 为 UE 选择更可靠的 MCS，更少打孔（less puncturing）以及选择更好的时频资源。

CSI（Channel State Information）是 UE 上报给 eNodeB 的信道状态信息，由 CQI（Channel Quality Indicator）、PMI（Precoding Matrix Indicator）、PTI（Precoding Type Indicator）和 RI（Rand Indication）组成，其所占的时频资源是由 eNodeB 来控制的。

UE 通过测量接收到的下行参考信号（小区特定的参考信号或 CSI-RS）来获取 CSI 信息，并上报给 eNodeB。eNdoeB 在下行调度时会将信道质量考虑在内。而某个 UE 上报的 CSI 既可以用于反映那些分配给该 UE 的 PRB 的信道质量，也是可以用于反映那些没有分配给该 UE 的 PRB 的信道质量信息。UE 特定的参考信号不能用于 CSI 测量的原因在于，它们仅在使用“波束赋形”传输（TM 7/8/9）时的被调度 RB 上存在。而小区特定的参考信号和 CSI-RS 是在整个频带内发送，并独立于分配给某个 UE 的 RB 的，因此可用于 CSI 测量。

eNodeB 并不一定遵循 UE 上报的下行信道状态报告来进行下行传输。相反的，UE 上报的只是**建议** eNodeB 使用的传输阶数（层数）和预编码矩阵，以及 eNodeB 不应该超过的最高调制和编码方案（MCS）。eNodeB 可以使用 CSI 报告的建议，也可以重新选择自己的传输参数，eNodeB 会通过 DCI 将当前传输真正使用的 MCS、层数和预编码矩阵等信息告诉 UE。

DL-SCH 传输的调制和编码方案可以并且通常不同于 UE 上报的 CQI，这是因为调度器需要考虑一些额外的信息，而这些信息是 UE 在上报 CQI 时所不知道的。例如，eNodeB 给某个 UE 分配下行 RB 资源时，需要将其它 UE 考虑在内。eNodeB 还需要考虑等待传输的数据量，在只有少量数据需要传输并且有足够的 RB 可以分配给 UE 的时候，即使信道条件允许，也不需要选择一个很高的码率和 MCS 等。

在 LTE 中，属于同一个 codeword 的所有 RB 使用相同的 MCS，即使这个 codeword 被映射到多个 layer 上。但不同的 codeword 可以使用不同的 MCS。

如果 UE 配置了多于一个 serving cell（载波聚合），则 UE 只会上报激活的 serving cell 的 CSI。这是因为非激活的 serving cell 上不存在下行传输，所以不需要上报 CSI。

CSI 上报可以是周期性的（periodic），也可以是非周期性的（aperiodic）。

在 Rel-10 中，是允许 UE 在同一子帧同时上报 PUCCH 和 PUSCH 的。但在之前的版本中（Rel-8/Rel-9），是不允许的。

如果 UE 没有配置同时发送 PUSCH 和 PUCCH（通过 *simultaneousPUCCH-PUSCH-r10* 配置），并且在周期性 CSI 上报的子帧上没有分配 PUSCH 资源，则 UE 会在 PUCCH 上发送周期性 CSI。

如果 UE 没有配置同时发送 PUSCH 和 PUCCH，并且在同一个子帧内，UE 需要同时发送上行数据和周期性 CSI，则周期性 CSI 在分配了 PUSCH 资源且拥有最小 *ServCellIndex* 的小区上的 PUSCH 传输，并使用与基于 PUCCH 的周期性 CSI 上报一样的格式。

非周期性 CSI 只能在 PUSCH 上传输。对于非周期性的 CQI/PMI 上报，只有在配置的 CSI 反馈类型支持 RI 上报的情况下才会上报 RI。

如果在同一个子帧内，需要同时发送周期性 CSI 和非周期性 CSI，则 UE 只会发送非周期性 CSI。

周期性 CSI 上报和非周期性 CSI 上报，均提供了下行信道质量的估计，但二者在上报内容的详细程度和使用上有很大区别。由于非周期性 CSI 上报在 PUSCH 上传输，而且 PUSCH 比 PUCCH 能够承载更多的内容，因此非周期性 CSI 上报的内容比起 PUCCH 上传输的周期性 CSI 上报来说更加详细，也就更大。同时非周期性 CSI 上报是有需要时才传输的，其开销并不是一个大问题。最后，eNodeB 请求 UE 上报一个非周期性 CSI 报告时，很可能需要发送大量的数据，而周期性 CSI 上报是不管 UE 是否会在不久的将来被调度都会被发送的，从这里来看，非周期性 CSI 上报的开销也是可接受的。

表 10-1 是对各种 TM 模式下，CSI 所需上报内容的一个小结：

表 10-1：各种 TM 模式下，UE 是否需要上报 CQI/PMI/RI

TM 模式	CQI	PMI	RI
TM 1	Yes	No	No
TM 2	Yes	No	No
TM 3	Yes	No	Yes
TM 4	Yes	Yes	Yes

TM 5	Yes	Yes	No
TM 6	Yes	Yes	No
TM 7	Yes	No	No
TM 8	Yes	Yes or No	Yes or No
TM 9	Yes	Yes or No	Yes or No

【参考资料】

[1] http://www.sharetechnote.com/html/Handbook_LTE_CSI.html

10.1 RI

RI 是 UE 建议 eNodeB 在下行传输中使用的传输阶数（rank），即建议下行传输使用的层数（layer）。

只有在空分复用下，UE 才需要发送 RI 以指示可用的传输层（transmission layer）数目；而其它情况下，RI 总是等于 1。因此只有在 TM 3/4，以及配置了 PMI/RI 上报的 TM 8/9 情况下，UE 才需要上报 RI。

在整个有效的带宽内，UE 只会上报一个 RI 值，也即 RI 上报是频率非选择性的。RI 无法使用频率选择性的原因在于 LTE 中某个 UE 的所有层上的数据都在相同的 RB 集合中传输。（通过 36.212 中 DCI 格式中可以看出，只有一个字段用于指定分配的 RB 资源，而不是为每个 TB，或每个 layer 分别分配 RB 资源）

当 UE 配置了 TM 4 或带 PMI/RI 上报的 TM8/9 时，在每个 RI 上报间隔内，UE 会根据对应的 eNodeB 天线配置和 UE category，从支持的 RI 集合中选择一个 RI 上报给 eNodeB。

当 UE 配置了 TM 3 时，在每个 RI 上报间隔内，UE 会根据对应的 eNodeB 天线配置和 UE category，上报一个 RI 给 eNodeB，以支持 eNodeB 在传输分集和 large delay CDD 中进行选择（RI = 1 时，指示使用传输分集；RI > 1 时，指示使用 large delay CDD）。

关于 layer，可进一步参见 6.2.3 节关于“layer”的介绍。

10.2 PMI

PMI 是 UE 建议 eNodeB 在下行传输中使用的预编码矩阵（precoder matrix）。该预编码矩阵是在假设使用“上报的 RI 指示的层数”的基础上进行选择的。

10.2.1 UE 上报 PMI

预编码矩阵的选择是可以频率选择性的，也即 UE 可以对下行频谱的不同部分（整个下行系统带宽，或其子集）建议使用不同的预编码矩阵。UE 可以同时上报 1 个或多个 PMI。

只有 TM 4/5/6 和配置了 PMI/RI 上报的 TM 8/9 支持 PMI 上报。

配置了 TM8 或 TM9 的 UE，是否支持 PMI/RI 上报是通过 *pmi-RI-Report* 配置的。

对于配置了 TM 9 的 UE 而言，除了配置支持 PMI/RI 上报外，还需要 CSI-RS 天线端口数大于 1，UE 才会上报 PMI。（见 36.213 的 7.2.4 节）

UE 上报的 PMI 信息只能从协议（36.211 的 6.3.4 节，或 36.213 的 Table 7.2.2-1D~7.2.2-1F）定义的码本（codebook）中选取。

不同的 TM 模式，对应的码本可能不同。对于 TM 4/5/6/8/9，可选的预编码矩阵集合可参见 6.5 节关于“TM 模式”的介绍。

eNodeB 还可以通过 *codebookSubsetRestriction* 限制 UE 可选择的预编码矩阵的范围（是前面介绍的预编码集合的子集，即码本的子集），以避免上报的预编码矩阵在某些天线配置下无法使用，协议中称为 codebook subset restriction（码本子集限制，见 36.213 的 7.2 节）。

codebookSubsetRestriction 是一个 bitmap。对于一个特定的预编码码本及其相关联的 TM 模式，该 bitmap 指定了 UE 在配置的特定 TM 模式下，认为的 eNodeB 可使用的所有预编码码本子集。

Bitmap 中的每个比特与一个预编码矩阵相对应。Bitmap 的大小等于 UE 在配置的 TM 模式和所使用的天线端口数下可用的所有预编码矩阵数。

只有 TM 3/4/5/6 和配置了 PMI/RI 上报的 TM 8/9 支持码本子集限制。

```
AntennaInfoDedicated ::=            SEQUENCE {
    transmissionMode                    ENUMERATED {
                                            tm1, tm2, tm3, tm4, tm5, tm6,
                                            tm7, tm8-v920},
    codebookSubsetRestriction           CHOICE {------TM 3/4/5/6的码本子集限制
        n2TxAntenna-tm3                     BIT STRING (SIZE (2)),
        n4TxAntenna-tm3                     BIT STRING (SIZE (4)),
        n2TxAntenna-tm4                     BIT STRING (SIZE (6)),
        n4TxAntenna-tm4                     BIT STRING (SIZE (64)),
        n2TxAntenna-tm5                     BIT STRING (SIZE (4)),
        n4TxAntenna-tm5                     BIT STRING (SIZE (16)),
```

```
        n2TxAntenna-tm6                     BIT STRING (SIZE (4)),
        n4TxAntenna-tm6                     BIT STRING (SIZE (16))
    }       OPTIONAL,                                                   -- Cond
TM
    ue-TransmitAntennaSelection         CHOICE{
            release                         NULL,
            setup                           ENUMERATED {closedLoop, openLoop}
    }
}

AntennaInfoDedicated-v920 ::=       SEQUENCE {
    codebookSubsetRestriction-v920      CHOICE {------TM 8的码本子集限制
        n2TxAntenna-tm8-r9                  BIT STRING (SIZE (6)),
        n4TxAntenna-tm8-r9                  BIT STRING (SIZE (32))
    }       OPTIONAL                                                    -- Cond
TM8
}

AntennaInfoDedicated-r10 ::=        SEQUENCE {
    transmissionMode-r10                ENUMERATED {
                                            tm1, tm2, tm3, tm4, tm5, tm6, tm7, tm8-v920,
                                            tm9-v1020, spare7, spare6, spare5, spare4,
                                            spare3, spare2, spare1},
    codebookSubsetRestriction-r10       BIT STRING          OPTIONAL,           -- Cond
TMX------TM 9的码本子集限制
    ue-TransmitAntennaSelection     CHOICE{
        release                         NULL,
        setup                           ENUMERATED {closedLoop, openLoop}
    }
}
```

每种传输模式对应的 *codebookSubsetRestriction* 包含的比特数如 36.213 的 Table 7.2-1b 所示。该比特数与对应的 TM 模式在某种天线端口配置下支持的最大预编码矩阵数相同。以 TM 4 为例，从 6.5.4 节的图 6-9 可以看出，2 天线端口下，至多支持 6 个预编码矩阵，所以 *n2TxAntenna-tm4* 的大小（包含的比特数）为 6；4 天线端口下，至多支持 64 个预编码矩阵，所以 *n4TxAntenna-tm4* 的大小为 64。

Table 7.2-1b: Number of bits in codebook subset restriction bitmap for applicable transmission modes.

	Number of bits A_c		
	2 antenna ports	4 antenna ports	8 antenna ports
Transmission mode 3	2	4	
Transmission mode 4	6	64	
Transmission mode 5	4	16	
Transmission mode 6	4	16	
Transmission mode 8	6	32	
Transmission mode 9	6	64	109

codebookSubsetRestriction 对应的比特串 $a_{A_c-1},\ldots,a_3,a_2,a_1,a_0$ 中，a_0 是 LSB，a_{A_c-1} 是 MSB。如果某个比特值为 0，则该比特相关联的预编码矩阵对应的 PMI 和 RI 是不允许被上报的。

TM 3：

- 2 天线端口：bitmap 包含 2 比特。比特 a_1 与 36.211 的 Table 6.3.4.2.3-1 中的层数为 2 且码本索引为 0 的预编码矩阵相对应。比特 a_0 与 36.211 的 6.3.4.3 节介绍的 2 天线端口传输分集相对应。
- 4 天线端口：bitmap 包含 4 比特。比特 $a_{\upsilon-1},\upsilon=2,3,4$ 与 36.211 的 Table 6.3.4.2.3-2 中的层数对应为 υ 且码本索引为 12，13，14，15 的预编码矩阵相对应。比特 a_0 与 36.211 的 6.3.4.3 节介绍的 4 天线端口传输分集相对应。

TM 3 并不需要上报 PMI，所以这里的码本子集限制并不是限制 PMI 上报的。TM 3 中 *codebookSubsetRestriction* 只是对 RI 的上报进行限制。

举例：假设 UE 配置了 TM 3，且使用 4 天线端口，且 *codebookSubsetRestriction* 为‘1011’，则 UE 可上报的 RI 的取值范围为 RI=4、RI=2、RI=1（传输分集），而不能上报 RI=3。

TM 4：

- 2 天线端口：bitmap 包含 6 比特。每个比特与对应的层数以及码本索引的对应关系见 36.213 的 Table 7.2-1c。

Table 7.2-1c: Association of bits in *codebookSubSetRestriction* bitmap to precoders in the 2 antenna port codebook of Table 6.3.4.2.3-1 in [3].

Codebook index i_c	Number of layers υ	
	1	2
0	a_0	-
1	a_1	a_4
2	a_2	a_5
3	a_3	-

- 4 天线端口：bitmap 包含 64 比特。比特 $a_{16(\upsilon-1)+i_c}$ 与层数为 υ 且码本索引为 i_c 的预编码相对应。其中 i_c（取值范围：0~15）见 36.211 的 Table 6.3.4.2.3-2。

注：具体的预编码矩阵数也可参见 6.5.4 节关于“TM 4”的介绍。

这里我们可以看出，bitmap 包含的比特数与该天线端口数下所有可能选取的预编码矩阵数是相等的，每个比特对应其中的一个预编码矩阵。

举例：假设 UE 配置了 TM 4，且使用 2 天线端口，且 *codebookSubsetRestriction* 为‘111011’，则 UE 可上报 RI = 2 且码本索引为 1 和 2 的预编码矩阵（对应的 PMI = 0 和 1），以及 RI = 1 且码本索引为 3、1 和 0 的预编码矩阵。

TM 5/6：

- 2 天线端口：bitmap 包含 4 比特。比特 a_{i_c} 与层数为 $\upsilon=1$ 且码本索引为 i_c 的预编码矩阵相对应。其中 i_c（取值范围：0~3）见 36.211 的 Table 6.3.4.2.3-1。
- 4 天线端口：bitmap 包含 16 比特。比特 a_{i_c} 与层数为 $\upsilon=1$ 且码本索引为 i_c 的预编码矩阵相对应。其中 i_c（取值范围：0~15）见 36.211 的 Table 6.3.4.2.3-2。

注：具体的预编码矩阵数也可参见 6.5.5 节关于“TM 5”和 6.5.6 节关于“TM 6”的介绍。

TM 8：

- 2 天线端口：bitmap 包含 6 比特。每个比特与对应的层数以及码本索引的对应关系见 36.213 的 Table 7.2-1c。
- 4 天线端口：bitmap 包含 32 比特。比特 $a_{16(\upsilon-1)+i_c}$ 与层数为 υ（$\upsilon=1,2$）且码本索引为 i_c 的预编码矩阵相对应。其中 i_c（取值范围：0~15）见 36.211 的 Table 6.3.4.2.3-2。

注：具体的预编码矩阵数也可参见 6.5.8 节关于“TM 8”的介绍。可以看出，TM 8 使用与 TM 4 类似的码本集合，只不过 TM 8 至多支持 2 层传输。

TM 9：

- 2 天线端口：bitmap 包含 6 比特。每个比特与对应的层数以及码本索引的对应关系见 36.213 的 Table 7.2-1c。
- 4 天线端口：bitmap 包含 64 比特。比特 $a_{16(\upsilon-1)+i_c}$ 与层数为 υ 且码本索引为 i_c 的预编码矩阵相对应。其中 i_c（取值范围：0~15）见 36.211 的 Table 6.3.4.2.3-2。
- 8 天线端口：bitmap 包含 109 比特。比特 $a_{f1(\upsilon-1)+i_{c1}}$ 与层数为 υ（$\upsilon \in \{1,2,3,4,5,6,7,8\}$）且码本索引为 i_{c1} 的预编码矩阵相对应，其中 $f1(\cdot)=\{0,16,32,36,40,44,48,52\}$。比特 $a_{53+g1(\upsilon-1)+i_{c2}}$ 与层数为 υ（$\upsilon \in \{1,2,3,4\}$）且码本索引为 i_{c2} 的预编码矩阵相对应，其中 $g1(\cdot)=\{0,16,32,48\}$（注：对于 $\upsilon \in \{5,6,7,8\}$，i_2 固定为 0，所以这里只需指定层数小于 5 的值）。其中码本索引 i_{c1} 和 i_{c2} 见 36.213 的 Table 7.2.4-1 ~Table 7.2.4-8，分别对应 υ=1、2、3、4、5、6、7 或 8。（注意：这几张表在 36.213-aa0 中定义，而 36.213-a30 中这几张表是在 36.211 中定义的）

注：可以看出，当使用 2 或 4 天线端口传输时，TM 9 使用与 TM 4 类似的码本集合。而使用 8 天线端口传输时，TM 9 中新定义了一套矩阵集合。

关于 UE 侧如何计算 PMI/RI，可参考[4]中的介绍。

10.2.2 eNodeB 选择预编码矩阵

对于 UE 上报的 PMI 建议，eNodeB 有 2 种选择：

（1）eNodeB 可以采用 UE 最近一次上报的 PMI 建议，此时 eNodeB 只需要在下行 DCI 中发送一个确认比特以告诉 UE，你建议的 PMI 已经被我采纳并用于下行传输。一旦 UE 收到了这个确认信息，就会使用它自己建议的配置来解调和解码对应的 DL-SCH 传输。由于 UE 进行 PMI 计算时可以是频率选择性的，遵循 UE 建议的预编码矩阵的 eNodeB 可能需要针对不同的 RB 集合使用不同的预编码矩阵。

（2）eNodeB 可以选择一个不同的预编码矩阵，但必须在下行 DCI 中明确地把这个信息告诉 UE。UE 会使用这个预编码矩阵来解调和解码对应的 DL-SCH 传输。为了降低下行 DCI 的比特数，下行 DCI 只会携带一个预编码矩阵信息，也就是说，此时的预编码是非频率选择的，下行 DCI 中指示所有的 RB 都必须使用这个预编码矩阵。eNodeB 也可以选择使用不同的层数，此时 UE 假定只使用每个建议的预编码矩阵中的部分列。

只有在 TM 4/5/6 中，eNodeB 才需要将选择的预编码矩阵信息通过 DCI 告诉 UE。具体如何通知，可以参见 6.5 节中关于 TM 4/5/6 的介绍。

【参考资料】

[1] TS 36.211 的 6.3.4 节 Precoding

[2] TS 36.213 的 7.2 节UE procedure for reporting Channel State Information (CSI)

[3] 《LTE: UE PMI Calculation》---UE 如何确定 PMI

[4] 《Calculation of the Spatial Preprocessing and Link Adaption Feedback for 3GPP UMTS/LTE》

[5] 下面的链接中介绍了一些预编码矩阵的内容，能解答大家一些疑惑：
http://www.sharetechnote.com/html/BasicProcedure_LTE_MIMO.html

10.3 CQI

UE 使用 CQI 来告诉 eNodeB 的调度器，UE 所看到的下行信道质量信息。如果使用了 MIMO 传输，CQI 会包含所需的 MIMO 相关的反馈。

CQI 是在使用建议的 RI 和 PMI，且 PDSCH 传输的 BLER 最高为 10%的情况下，使用的最高 MCS（Modulation and Coding Scheme）。

使用 CQI 来反馈下行信道质量而不是其它（例如 SNR）的原因在于这里将不同 UE 的接收能力考虑在内。这同时也简化了 UE 的测试，在使用 CQI 指示的 MCS 的情况下，如果 BLER 大于 10%，则测试失败。CQI 可以看做是对 SINR 的一种测量，但又不仅仅是 SINR。CQI 的测量需要将 SINR 以及 UE 的接收机的能力考虑在内。对于那些采用了先进信号处理算法（例如：干扰消除技术）的 UE 而言，可以上报一个更高的信道质量，并且依赖于 eNodeB 的调度算法，可以接收更高的数据速率。

关于 UE 如何将 SINR 映射到 CQI，可参考[4]中的实现。其基本思路是：在特定条件下，每次取 1~15 的中的一个 CQI 值，在对应的调制方式和码率下，调整 SNR 值，使其达到可接受的 10% BLER，则该 SNR 值就对应所选的 CQI 值。注意：在不同的条件下（例如不同的 TM 模式、或不同的天线配置），该对应关系可能不同，需要区别对待。

CQI 索引及其代表的含义见 36.213 的 Table 7.2.3-1。

Table 7.2.3-1: 4-bit CQI Table

CQI index	modulation	code rate x 1024	efficiency
0	out of range		
1	QPSK	78	0.1523
2	QPSK	120	0.2344
3	QPSK	193	0.3770
4	QPSK	308	0.6016
5	QPSK	449	0.8770
6	QPSK	602	1.1758
7	16QAM	378	1.4766
8	16QAM	490	1.9141
9	16QAM	616	2.4063
10	64QAM	466	2.7305
11	64QAM	567	3.3223
12	64QAM	666	3.9023
13	64QAM	772	4.5234
14	64QAM	873	5.1152
15	64QAM	948	5.5547

注：efficiency = code rate / 1024 * Qm（这里的 code rate 指的是 Table 7.2.3-1 的第三列值。结果需要四舍五入，取小数点后 4 位）。其中 QPSK 对应的 Qm = 2；16QAM 对应的 Qm = 4；64QAM 对应的 Qm = 6。

对于配置了 TM9 且同时配置了 PMI/RI 上报的 UE，上报的 CQI 值是基于对 CSI-RS 的测量得到的；对于配置了 TM9 但没有配置 PMI/RI 上报的 UE，上报的 CQI 值是基于对小区特定的参考信号的测量得到的；对于配置了除 TM 9 外的其它传输模式的 UE，上报的 CQI 值是基于对小区特定的参考信号的测量得到的。（36.213 的 7.2.3 节）

eNodeB 在选择 MCS，即选择调制阶数（modulation order）和 TB size 时，会保证对应的码率（code rate）尽可能地接近 CQI index 指示的码率。如果有多个“调制阶数和 TB size 的组合”对应的码率同样接近 CQI index 指示的码率，则 eNodeB 会选择 TB size 最小的那个组合。

参考资料[6]中举了一些例子来介绍 eNodeB 在选择 MCS 和 TB size 如何计算实际的码率，并在参考资料[7]中给出了详细解释。但要注意的是，实际的处理过程可能会稍微复杂一些，例如，在计算每个 RB 可用的 RE 数时，需要将用于参考信号、PBCH、PSS/SSS 和控制区域等所占的 RE 排除在外等等。

10.3.1 CSI 参考资源（CSI reference resource）

我们将在本节介绍 UE 会在哪些子帧（时域）的哪些 RB（频域）以及那些空间资源（空间域）上测量下行信道质量，以计算 CQI。

在 3GPP 协议中，用于测量 CSI 的资源称为 CSI reference resource。一个 CSI reference resource 由时域、频域和空间域 3 部分唯一确定。

- 在**频域**上，CSI reference resource 定义为需要上报的 CQI 对应的 band（wideband 或 subband，见后续章节介绍）所在的 PRB 集合；
- 在**时域**上，CSI reference resource 定义为一个单一的下行子帧 $n - n_{CQI_ref}$，其中 n 对应需要上报 CSI 的上行子帧（见后续章节介绍），n_{CQI_ref} 通过如下方式定义：
    - 对于周期性 CSI 上报，n_{CQI_ref} 是大于或等于 4 的最小值，且计算出的 $n - n_{CQI_ref}$ 是一个有效的下行子帧；
    - 对于非周期性 CSI 上报，且该上报是由上行 DCI 中的 CSI request 字段触发的情况，通过 n_{CQI_ref} 计算出的下行子帧与发送该上行 DCI 的下行子帧为同一个有效的下行子帧；
    - 对于非周期性 CSI 上报，且该上报是由 RAR 中的 CSI request 字段触发的情况，n_{CQI_ref} 等于 4，且下行子帧 $n - n_{CQI_ref}$ 是一个有效的下行子帧，其中下行子帧 $n - n_{CQI_ref}$ 位于发送 RAR 的下行子帧之后。
- 而在**空间域**上，CSI reference resource 定义为需要上报的 CQI 基于的 RI 和 PMI。

如果满足以下条件，则认为一个 serving cell 内的一个下行子帧是有效的：

- 对于该 UE 而言，该子帧为一个下行子帧；
- 除了 TM 9，该子帧不是一个 MBSFN 子帧；
- 如果该子帧是 TDD 中的特殊子帧，则该子帧包含的 DwPTS 域的长度不能小于或等于 $7680 \cdot T_s$；（因为这种情况下 DwPTS 不能用于 PDSCH 传输）
- 该子帧不能位于该 UE 配置的 measurement gap 内；
- 对于周期性 CSI 上报且配置了“CSI 子帧集合（CSI subframe set）”的 UE 而言，该子帧必须是该集合中的一员。

如果一个 serving cell 的 CSI reference resource 不是一个有效的下行子帧，则 UE 不会在上行子帧 n 向该 serving cell 发送 CSI 上报。

这里需要说明的是，对于配置了 8 天线端口的 TM9 且同时配置了 PMI/RI 上报的 UE，上报的 CQI 值是基于对 CSI-RS 的测量得到的，所以此时有效的下行子帧还必须是发送了 CSI-RS 的下行子帧。

10.3.2 CSI 子帧集合

在前一节中，我们已经介绍了一个有效的下行子帧必须属于 “CSI 子帧集合（CSI subframe set）”。CSI 子帧集合是在 Rel-10 中引入的。首先，我们来介绍一下“CSI 子帧集合（CSI subframe set）”的应用场景。

在第 23 章关于“载波聚合”的介绍中我们会看到，在异构网络（见 23.5 节）中，macro cell（宏基站）和 pico-cell（小基站）使用同一频率时，macro cell 会对 pico cell 造成很大的干扰。其中一种有效的方式是使用跨载波调度来避免小区间的干扰。实际上，LTE 还提出了一种小区间干扰协调的方法，将 macro cell 的功率限定在某些特定的子帧上。通俗点讲，就是在某些子帧上，macro cell 不发送下行数据（可能会发送一些下行控制信令，见后文介绍），这样可以将 macro cell 的发射功率设定在较低的水平，对 pico cell 的干扰也就较小，这些受保护的子帧被称为“保护子帧（protected subframes）”。在保护子帧上，pico cell 上的 UE 的数据和控制信息传输，将经历更少的来自 macro cell 的干扰，因此即使在高干扰的区域，pico cell 依然可以使用这些子帧来给 UE 发送数据。与传统的 ICIC 相比，这种限制不仅应用在数据传输（PDSCH），还应用在控制区域（control region）上。如图 10-1 所示，8 个子帧中有 2 个子帧被配置为保护子帧。

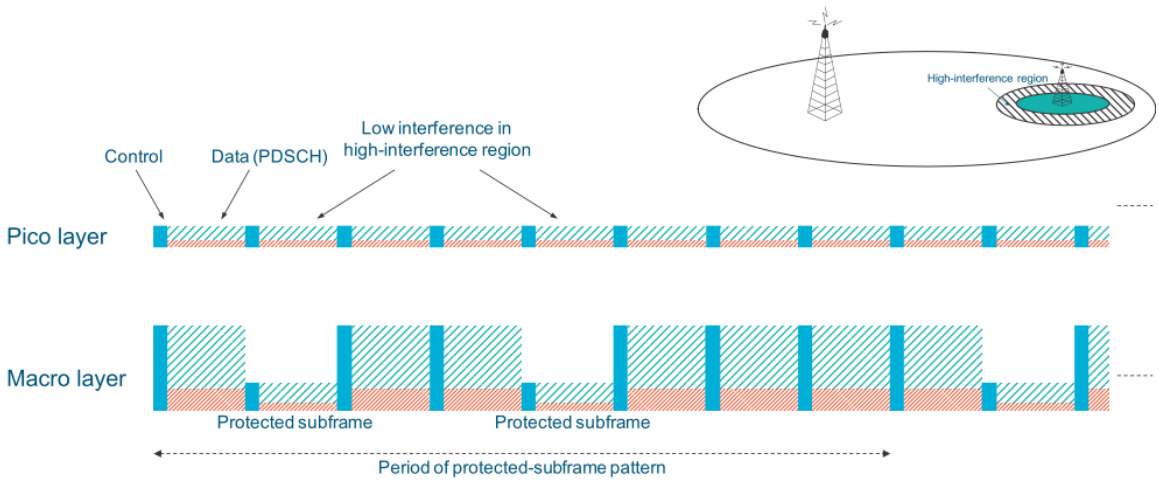


图 10-1：保护子帧

为了支持上面介绍的保护子帧，需要在不同小区之间发送“procected subframe pattern”的信息，以便彼此之间知道保护子帧的集合。保护子帧集合是可以动态变化的，这取决于 macro cell 和 pico cell 可进行协调调度的时间粒度。

macro cell 在保护子帧上并不需要完全不发送任何控制信息，macro cell 可以发送一些跟上行传输相关的有限的控制信息，如 uplink grant 和/或 PHICH 传输，这样就不会对上行调度产生太大的影响。只要 macro cell 上传输的控制信息是有限的，并且只占用了整个控制区域很小的一部分，则对 pico cell 的高干扰区域内的 UE 的干扰可以保持在一个可接受的水平。

即使在保护子帧上没有 uplink grant 和 PHICH 传输，procected subframe pattern 的配置也应该尽可能最小化对上行调度的影响，这是通过让 procected subframe pattern 与上行 HARQ 的 8 子帧 timing（对 FDD 而言）相匹配来实现的。对于 FDD 来说，该 pattern 是与 40ms（4 个系统帧）对齐的；而对于 TDD 来说，这取决于上下行配置，这可见 *MeasSubframePattern-r10*。

对 pico cell 上的 UE 来说，在保护子帧和非保护子帧之间所受到的干扰水平可能有变化很大，如果不对保护子帧和非保护子帧进行区分，而是联合起来进行 CSI 测量，则很难精确地反映任意一种子帧类型的干扰水平。因此，可以在保护子帧集合和非保护子帧集合上，分别给 UE 配置不同的 CSI 测量子集（CSI measurement subset，对应协议中介绍的 $C_{CSI,0}$ 和 $C_{CSI,1}$），以反映二者不同的干扰水平。

eNodeB 可以通过 *csi-MeasSubframeSet1-r10*（对应集合 $C_{CSI,0}$）和 *csi-MeasSubframeSet2-r10*（对应集合 $C_{CSI,1}$）给 UE 配置分别在哪些下行子帧上进行 CSI 测量（即上面介绍的“CSI measurement subset”）。对应的结构为 *MeasSubframePattern-r10*：

```
MeasSubframePattern-r10 ::= CHOICE {
    subframePatternFDD-r10              BIT STRING (SIZE (40)),-----用于FDD
    subframePatternTDD-r10              CHOICE {
        subframeConfig1-5-r10                   BIT STRING (SIZE (20)), -----用于TDD配置1~5
        subframeConfig0-r10                     BIT STRING (SIZE (70)), -----用于TDD配置0
        subframeConfig6-r10                     BIT STRING (SIZE (60)), -----用于TDD配置6
        ...
    },
    ...
}
```

MeasSubframePattern 用于限制时域上的测量资源，也就是指定在哪些子帧进行测量，而不在其它子帧进行测量。其中第一个，也就最左边的比特对应满足 SFN mod x = 0 系统帧中的子帧 #0，其

中 SFN 是 PCell 中的系统帧号，且 x 等于 bit string 的大小除以 10。比特值为 1 时表示将在对应的子帧上进行测量。

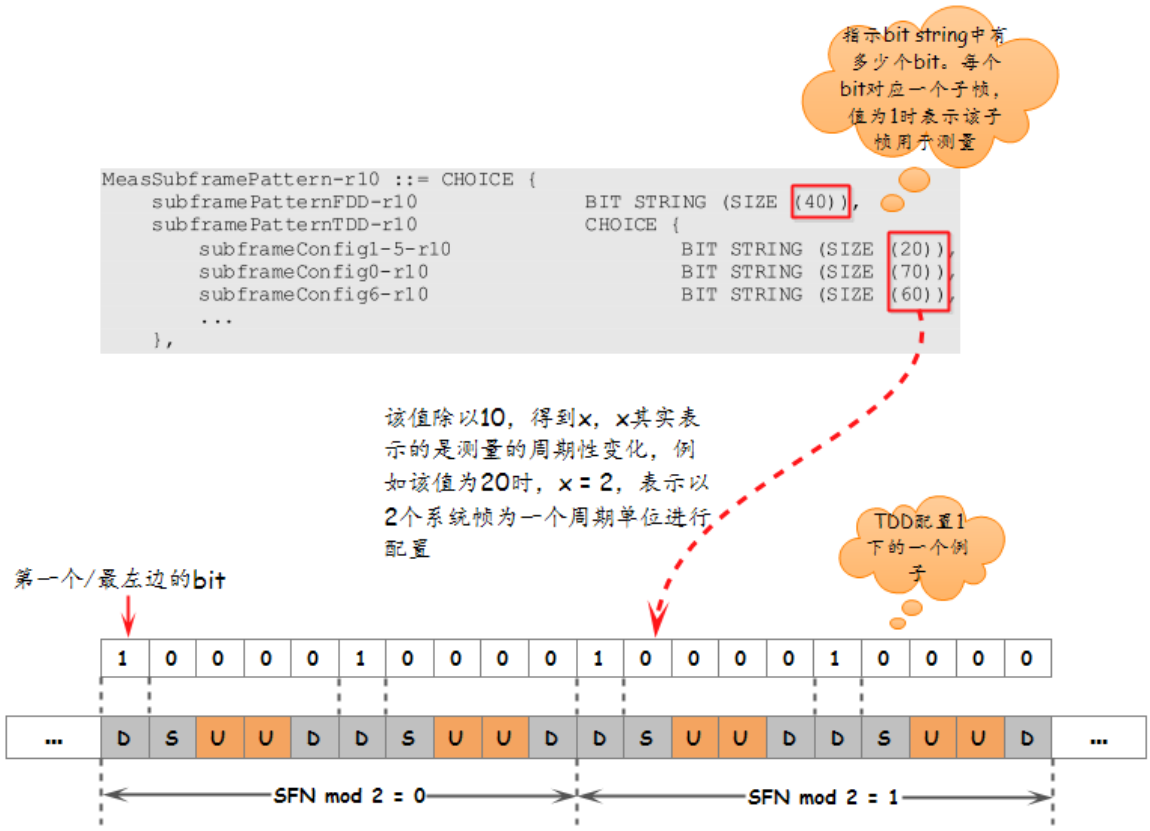


图 10-2：MeasSubframePattern 的一个例子

图 10-2 是测量子帧配置的一个例子：TDD 1 下，bit string 的大小为 20 比特，即以 2 个系统帧为一个周期单位。UE 会在该周期内的第 1、6、11、16 个子帧进行测量。

如果 UE 被配置成为多于一个 CSI 子帧集（CSI subframe set）进行上报，则 *cqi-pmi-ConfigIndex* 和 *ri-ConfigIndex* 对应子帧集 1（*csi-MeasSubframeSet1*）的 CQI/PMI 和 RI 周期以及相对偏移；*cqi-pmi-ConfigIndex2-r10* 和 *ri-ConfigIndex2-r10* 对应子帧集 2（*csi-MeasSubframeSet2*）的 CQI/PMI 和 RI 周期以及相对偏移。

也就是说，在 *cqi-pmi-ConfigIndex*、*ri-ConfigIndex* 指定的资源上上报的 CQI/PMI/RI，是对 *csi-MeasSubframeSet1* 指定的下行子帧进行测量的结果；在 *cqi-pmi-ConfigIndex2-r10*、*ri-ConfigIndex2-*

r10 指定的资源上上报的 CQI/PMI/RI，是对 *csi-MeasSubframeSet2* 指定的下行子帧进行测量的结果。二者可分别对应保护子帧和非保护子帧。

关于 CSI 子帧集合，在 36.300 的 16.1.5.1 节和 K.1.2 节也有介绍。

前面已经介绍过，保护子帧的集合可能是动态变化的，但更新 CSI 测量子集的信息又不是很灵活（通过 RRC 信令或上层消息配置的时间开销通常较大）。因此在实际应用中，对应保护子帧的 CSI 测量子集可能只包含所有保护子帧的一个子集，这部分集合通常是相对静止不变的。同样的，对应非保护子帧的 CSI 测量集合可能只包含所有非保护子帧的一个子集。

同时需要注意的是，在某些场景下，pico cell 会同时位于 2 个 macro cell 的边界处，并同时受到这 2 个 macro cell 的干扰，如果 2 个 macro cell 有不同的配置且只有部分的保护子帧发生重叠，则 pico cell 的调度以及 CSI 测量集合的配置需要将这 2 个 macro cell 的保护子帧集合的配置考虑在内。

【参考资料】

[1] TS 36.300 的 16.1.5.1 节 UE configurations for time domain ICIC

[2] TS 36.300 的 K.1.2 节 Pico scenario

[3] TS 36.213 的 7.2.3 节 Channel quality indicator (CQI) definition

[4] 《Downlink SNR to CQI Mapping for Different Multiple Antenna Techniques in LTE》

[5] 《4G LTE/LTE-Advanced for Mobile Broadband》的 13.4.1 节

[6] http://www.sharetechnote.com/html/Handbook_LTE_CQI.html

[7] http://www.sharetechnote.com/html/Handbook_LTE_CodeRate.html

10.4 非周期性 CSI 上报（Aperiodic CSI Reporting）

从 eNodeB 的角度来看，只有当小区有下行数据需要发送（此时才会关心下行信道的相关信息）时，才会要求 UE 发送非周期性 CSI 上报。当然，不同的厂商可能有不同的实现。

eNodeB 是通过 *CQI-ReportAperiodic-r10* 来配置非周期性 CSI 上报的相关参数的。

```
CQI-ReportAperiodic-r10 ::=     CHOICE {
    release                             NULL,
    setup                               SEQUENCE {
        cqi-ReportModeAperiodic-r10         CQI-ReportModeAperiodic,-----用于指定非周期性CSI上报模式
        aperiodicCSI-Trigger-r10            SEQUENCE {-----指示了当UE配置了1个或多个SCell时，需要为哪些serving cell上报非周期性的CSI报告。见后面的介绍。
            trigger1-r10                    BIT STRING (SIZE (8)),
            trigger2-r10                    BIT STRING (SIZE (8))
        }                                                               OPTIONAL    -
- Need OR
    }
}

CQI-ReportModeAperiodic ::=             ENUMERATED {-----指定非周期性CSI上报模式。

                                            rm12, rm20, rm22, rm30, rm31,
                                            spare3, spare2, spare1
}
```

10.4.1 触发方式

eNodeB 通过 DCI format 0/4（见 36.212 的 5.3.3.1.1 节和 5.3.3.1.8 节），或 RAR 中的 UL grant 的 CSI request 字段（见 36.213 的 6.2 节）来通知 UE 是否进行非周期性 CSI 上报。

10.4.1.1 DCI format 0/4 触发的非周期性 CSI 上报

DCI format 0/4 中有一个 CSI 字段，用来指示 UE 在对应的 PUSCH 传输中是否将非周期性 CSI 上报给 eNodeB。

- **CSI request** – 1 or 2 bits as defined in section 7.2.1 of [3]. The 2-bit field only applies to UEs that are configured with more than one DL cell and when the corresponding DCI format is mapped onto the UE specific search space given by the C-RNTI as defined in [3]

图 10-3：DCI format 0/4 中的 CSI request 字段

从图 10-3 可以看出，DCI format 0/4 中的 CSI 字段可能为 1 比特也可能为 2 比特。只有当 UE 配置了多于一个下行小区，即下行使用载波聚合（CQI 反馈的是下行信道质量，所以这里对应的是下

行，而不是上行），并且对应的 DCI 映射到 UE 特定的搜索空间（而不是公共的搜索空间。并且 DCI format 4 只能映射到 UE 特定的搜索空间）时，CSI request 字段才会使用 2 比特。

如果 DCI format 0/4 中的 CSI request 字段为 1 比特，并且该字段设置为 1，则该 DCI 调度的是哪个小区的资源， UE 就上报那个小区的 CSI 信息。

如果 DCI format 0/4 中的 CSI request 字段为 2 比特，则 UE 根据 36.213 的 Table 7.2.1-1A 来决定为哪些下行小区上报非周期性 CSI，如图 10-4 所示。

Table 7.2.1-1A: CSI Request field for PDCCH with uplink DCI format in UE specific search space

Value of CSI request field	Description
'00'	No aperiodic CSI report is triggered
'01'	Aperiodic CSI report is triggered for serving cell c
'10'	Aperiodic CSI report is triggered for a 1st set of serving cells configured by higher layers
'11'	Aperiodic CSI report is triggered for a 2nd set of serving cells configured by higher layers

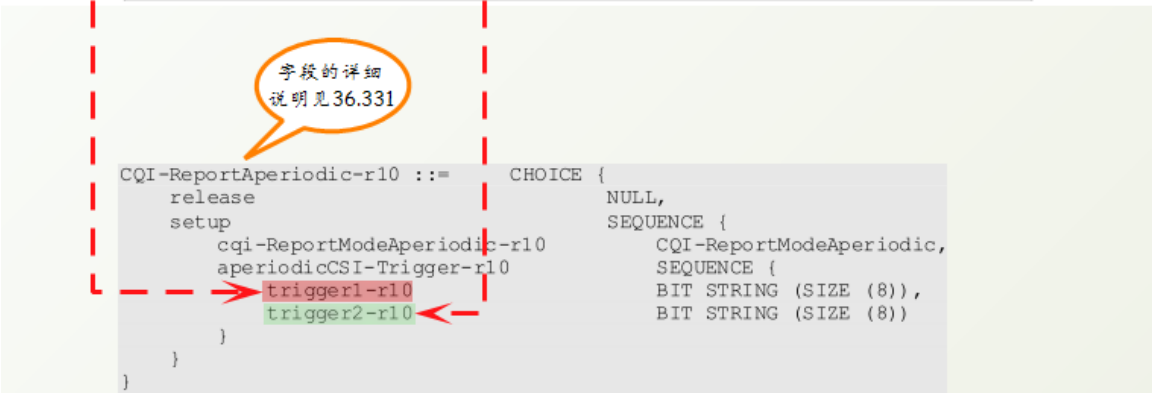


图 10-4：CSI request 字段为 2 比特时的解析

trigger1-r10 和 *trigger2-r10* 指示了当 UE 配置了 1 个或多个 SCell 时，需要为哪些 serving cell 上报非周期性的 CSI 报告。其中 *trigger1-r10* 与配置为“10”的 CSI request 字段对应，*trigger2-r10* 与配置为“11”的 CSI request 字段对应。也就是说，eNodeB 可以动态地选择让 UE 为哪些 serving cell 上报非周期性 CSI。

trigger1-r10 和 *trigger2-r10* 中最左边的比特 0 对应 *ServCellIndex*=0 的小区，紧接着的比特 1 对应 *ServCellIndex*=1 的小区，依次类推。每个比特可被设置为 0（表示不给该小区上报非周期性

CSI），也可被设置为 1（表示给该小区上报非周期性 CSI），但至多只有 5 个比特可以被设置为 1，对应载波聚合中只能给 UE 配置至多 5 个 serving cell。

如果 DCI format 0/4 的 CSI request 字段设置为“01”，则 UE 会上报该 DCI 调度的资源所在的 serving cell 上的非周期性 CSI。

如果 UE 在子帧 n 收到的 DCI format 0/4 中的 CSI request 字段指示需要上报 CSI，则 UE 会在子帧 n + k 的 PUSCH 上发送非周期性的 CSI 报告。

对于 FDD 而言，k = 4。

对于 TDD configuration 1~6，k 值见 36.213 的 Table 8-2。

Table 8-2 *k* for TDD configurations 0-6

TDD UL/DL Configuration	subframe number *n*									
	0	1	2	3	4	5	6	7	8	9
0	4	6				4	6			
1		6			4		6			4
2				4					4	
3	4								4	4
4									4	4
5									4	
6	7	7				7	7			5

对于 TDD configuration 0：（1）如果 UL index 字段的 MSB 设置为 1 且 LSB 设置为 0，则 k 值见 36.213 的 Table 8-2；（2）如果 UL index 字段的 MSB 设置为 0 且 LSB 设置为 1，则 k 值等于 7；（3）如果 UL index 字段的 MSB 和 LSB 均设置为 1，则 k 值见 36.213 的 Table 8-2。

UE 不希望在同一下行子帧收到多于 1 个非周期性 CSI 上报请求。也就是说，在载波聚合中，如果 UE 在同一下行子帧收到多个 UL Grant（即 DCI format 0/4），则其中只有一个 UL Grant 的 CSI request 字段能够设置成非 0。

10.4.1.2 RAR 触发的非周期性 CSI 上报

RAR 中的 UL grant 有一个 1 比特的 CSI request 字段，用来指示 UE 在对应的 PUSCH 传输（Msg3）中是否将非周期性 CSI 上报给 eNodeB。

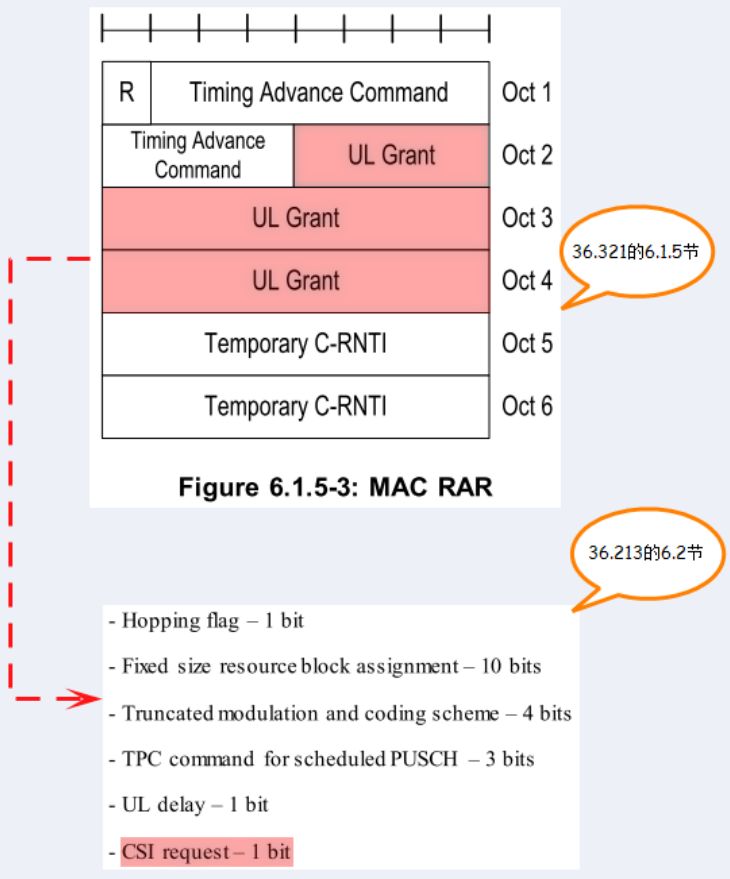


图 10-5：MAC RAR 中的 UL Grant 字段中的 CSI request 比特

对于**基于非竞争**的随机接入而言，RAR 中的 CSI request 字段用于决定在对应的 PUSCH 传输中是否包含非周期性的 CQI/PMI/RI 上报。而对于**基于竞争**的随机接入而言，RAR 中的 CSI request 字段是预留的，即没有任何作用。

如果 UE 在子帧 n 收到的 RAR 中的 UL Grant 的 CSI request 字段指示需要上报 CSI，并且该字段不是预留的（CSI request 字段设置为 1，且为基于非竞争的随机接入），则

- 如果 RAR 中的 UL Grant 的 UL delay 字段设置为 0，则 UE 应该在上行子帧 n + k_1 上报非周期性 CSI。其中 k = k_1（ $k_1 \geq 6$，对于 FDD 而言，就是 n + 6；对于 TDD 而言，n + 6 可能不是上行子帧，所以 $k_1 \geq 6$），也就是子帧 n + 6 算起的第一个可用的上行子帧。
- 如果 RAR 中的 UL Grant 的 UL delay 字段设置为 1，则 UE 会在 n + k_1 之后的第一个可用上行子帧来上报非周期性 CSI，也就是从子帧 n + 6 算起的第二个可用的上行子帧。

【参考资料】

[1] TS 36.213 的 5.3.3.1.1 节 Format 0

[2] TS 36.213 的 5.3.3.1.8 节 Format 4

[3] TS 36.213 的 6.2 节 Random Access Response Grant

[4] TS 36.213 的 7.2.1 节 Aperiodic CSI Reporting using PUSCH

10.4.2 上报模式

10.4.2.1 TM 模式与非周期性 CSI 上报模式的关系

非周期性 CSI 上报中包含哪些内容与 UE 配置的 CSI 上报模式相关。LTE 支持 3 种非周期性 CSI 上报模式，如 36.213 的 Table 7.2.1-1 所示。

Table 7.2.1-1: CQI and PMI Feedback Types for PUSCH CSI reporting Modes

		PMI Feedback Type		
		No PMI	**Single PMI**	**Multiple PMI**
PUSCH CQI Feedback Type	**Wideband (wideband CQI)**			Mode 1-2
	UE Selected (subband CQI)	Mode 2-0		Mode 2-2
	Higher Layer-configured (subband CQI)	Mode 3-0	Mode 3-1	

UE 使用的非周期性 CSI 上报模式是通过 *cqi-ReportModeAperiodic* 来配置的。不同的 TM 模式，支持不同的非周期性 CSI 上报模式。也就是说，TM 模式的配置限制了 *cqi-ReportModeAperiodic* 的取值范围。

```
CQI-ReportModeAperiodic ::=             ENUMERATED {
                                            rm12, rm20, rm22, rm30, rm31,
cqi-ReportModeAperiodic                     spare3, spare2, spare1
```

TM模式限制了非周期性CSI上报可选的上报模式

非周期性CSI上报模式与TM模式的对应关系

TM模式	Reporting Mode				
	Wideband CQI	UE Selected CQI		Higher Layer-configured CQI	
	1-2 (Multiple PMI)	2-0 (No PMI)	2-2 (Multiple PMI)	3-0 (No PMI)	3-1 (Single PMI)
TM 1		Yes		Yes	
TM 2		Yes		Yes	
TM 3		Yes		Yes	
TM 4	Yes		Yes		Yes
TM 5					Yes
TM 6	Yes		Yes		Yes
TM 7		Yes		Yes	
TM 8	**Yes** (with PMI/RI reporting)	**Yes** (without PMI/RI reporting)	**Yes** (with PMI/RI reporting)	**Yes** (without PMI/RI reporting)	**Yes** (with PMI/RI reporting)
TM 9	**Yes** (with PMI/RI reporting且CSI-RS天线端口数大于1)	**Yes** (without PMI/RI reporting，或CSI-RS天线端口数等于1)	**Yes** (with PMI/RI reporting且CSI-RS天线端口数大于1)	**Yes** (without PMI/RI reporting，或CSI-RS天线端口数等于1)	**Yes** (with PMI/RI reporting且CSI-RS天线端口数大于1)

图 10-6：TM 模式与非周期性 CSI 上报模式的对应关系

注：（1）TM 1/2/7 无需上报 PMI/RI。TM 3 是开环的空分复用，其预编码矩阵使用固定的计算方式，也无需上报 PMI。

（2）对于 TM8，如果 UE 配置了 PMI/RI 上报（*pmi-RI-Report* 设置成 setup），则支持 CSI 上报模式 1-2、2-2、 3-1；如果 UE 没有配置 PMI/RI 上报，则支持 CSI 上报模式 2-0、3-0；

（3）对于 TM9，如果 UE 配置了 PMI/RI 上报（*pmi-RI-Report* 设置成 setup）并且 CSI-RS 天线端口数大于 1，则支持 CSI 上报模式 1-2、2-2、3-1；如果 UE 没有配置 PMI/RI 上报，或 CSI-RS 天线端口数等于 1，则支持 CSI 上报模式 2-0、3-0；

（4）如果某个小区的下行系统带宽 $N_{RB}^{DL} \le 7$，则该小区不支持在 PUSCH 上报 CSI（即不支持非周期性 CSI 上报模式）。

可以看出，非周期性 CSI 上报的 3 种模式主要关注于 CQI 和 PMI 的上报，而 RI 的上报与具体的 TM 模式相关：只有在 TM 3/4，以及配置了 PMI/RI 上报的 TM 8/9 情况下，UE 才会上报 RI。这是因为只有在空分复用下，UE 才需要发送 RI 以指定层数。对于其它 TM 模式，RI 总是等于 1。

10.4.2.2 subband 划分

LTE 中，UE 不仅可以基于整个下行系统带宽上报 CQI/PMI，也可以基于某个子带（subband）上报 CQI/PMI。Subband 的划分规则如下：

把整个下行系统带宽划分成多个大小相等的 subband（最后一个 subband 可能较小），这些 subband 的集合称为集合 S。每个 subband 由 k 个连续的 PRB 组成，其中 k 与下行系统带宽相关。由于下行系统带宽未必能被 k 整除，所以集合 S 中的最后一个 subband 包含的 PRB 数可能小于 k。下行系统带宽 N_{RB}^{DL} 所包含的 subband 数为 $N = \lceil N_{RB}^{DL} / k \rceil$。这些 subband 从最低的频率处开始，按照频率递增的顺序进行索引。对于不同的非周期性 CSI 上报模式，k 值可能不同，见后续介绍。

10.4.2.3 上报模式

在介绍上报模式之前，我们先来介绍一下非周期性 CSI 上报中每个 CQI/PMI/RI 元素所占的比特数：

- **Wideband CQI**：使用 4 比特表示 codeword 0 的 wideband 信道质量信息。如果存在 codeword 1（且需要上报对应的 CQI）的话，则额外需要 4 比特表示 codeword 1 的 wideband 信道质量信息；（每个 4 比特信息与 36.213 的 Table 7.2.3-1 对应）
- **Subband CQI**：
  - 如果使用的是 **Higher Layer-configured subband feedback**，则每个 codeword 的**每个 subband CQI** 各使用 2 比特的 subband differential CQI offset level 信息，来反映每个 subband CQI 相对于对应 codeword 的 wideband CQI 的偏移值。
  - 如果使用的是 **UE-selected subband feedback**，则**每个 codeword** 上各使用 2 比特的 differential CQI offset level 信息，来反映所选择的 M 个 subband 的 CQI 值相对于对应 codeword 的 wideband CQI 的偏移值。同时，还需要 L 比特来指示所选的 M 个 subband 的所在位置（即 subband 索引）。

- **RI**：对于使用至多 2 层传输的 UE 而言，使用 1 比特表示 RI；对于使用至多 4 层传输的 UE 而言，使用 2 比特表示 RI；对于使用至多 8 层传输的 UE 而言，使用 3 比特表示 RI；
- **PMI**（见 36.213 的 7.2.4 节）：
    - 对于 2 和 4 天线端口传输，每个 PMI 值对应 36.211 的 Table 6.3.4.2.3-1 或 Table 6.3.4.2.3-2 的 codebook index。所以有
        - 对于 2 天线端口（$\{0,1\}$或$\{15,16\}$）传输，且 RI 值为 1 的情况，PMI 的取值范围为 $n \in \{0,1,2,3\}$，对应 36.211 的 Table 6.3.4.2.3-1 中 $\upsilon = 1$ 且 codebook index 为 n 的项。可以看出，4 种可选值，因此使用 2 比特表示；
        - 对于 2 天线端口（$\{0,1\}$或$\{15,16\}$）传输，且 RI 值为 2 的情况，PMI 的取值范围为 $n \in \{0,1\}$，对应 36.211 的 Table 6.3.4.2.3-1 中 $\upsilon = 2$ 且 codebook index 为 n + 1 的项。可以看出，2 种可选值，因此使用 1 比特表示；（注：对于使用闭环空分复用的 TM 模式，当层数为 2 时，codebook index 0 不能被使用，这也是为什么这里使用 n+1 而不是 n 的原因）
        - 对于 4 天线端口（$\{0,1,2,3\}$或$\{15,16,17,18\}$）传输，PMI 的取值范围为 $n \in \{0,1,\cdots,15\}$，对应 36.211 的 Table 6.3.4.2.3-2 中 υ 等于对应的 RI 值，且 codebook index 为 n 的项。可以看出，每种 RI 下，有 16 种可选值，因此使用 4 比特表示；
    - 对于 8 天线端口传输（只用于 TM 9），每个 PMI 值对应 36.213 的 Table 7.2.4-1 至 Table 7.2.4-8 的成对 codebook index。有
        - 对于 8 天线端口 $\{15,16,17,18,19,20,21,22\}$，first PMI（$i_1$）的值 $i_1 \in \{0,1,\cdots, f(\upsilon)-1\}$ 和 second PMI（i_2）的值 $i_2 \in \{0,1,\cdots, g(\upsilon)-1\}$对应 36.213 的 Table 7.2.4-*j* 中的 codebook indices i_1 和 i_2，其中 υ 等于对应 RI 值，*j* = υ，$f(\upsilon) = \{16,16,4,4,4,4,4,1\}$（即 i_1 对应需要的比特数为 $\{4,4,2,2,2,2,2,0\}$）和 $g(\upsilon) = \{16,16,16,8,1,1,1,1\}$（即 i_2 对应需要的比特数为 $\{4,4,4,3,0,0,0,0\}$）。

注：非周期性 CSI 中并不使用 PTI。

接下来，我们将详细介绍非周期性 CSI 上报的各种上报模式，并结合前面的每个 CQI/PMI/RI 元素所占的比特数，来确定每次上报中需要上报哪些信息，以及这些信息总共包含的比特数。

（1）**Wideband feedback**：通过一个 CQI 值（每个 codeword 各上报一个 CQI）反映小区整个下行系统带宽的平均信道质量。但 PMI 上报依然是频率选择性的，即通过把整个下行带宽划分成多个大小相等的 subband（最后一个 subband 可能较小），每个 subband 由多个连续的 RB 组成（每个 subband 的大小见 36.213 的 Table 7.2.1-3），且 UE 为每个 subband 上报一个 PMI 值。对于支持空分复用的 TM 模式而言，CQI 和 PMI 是根据 RI 指示的 channel rank 来计算的；非空分复用下，假定 rank=1。这种模式不提供任何关于频域的信息。

- **Mode 1-2**：（1 表示上报的是 Wideband 的 CQI，2 表示上报多个 PMI 值）
  - UE 会为每个 codeword 上报一个宽带 CQI 值，该 CQI 值反映的是整个下行带宽的平均信道质量；
  - UE 会为每个 subband 上报一个 PMI 值，该 PMI 对应的预编码矩阵必须从码本子集中选择；
  - 在“带 8 个 CSI-RS 端口的 TM 9”配置下，UE 并不会为每个 subband 上报一个 PMI 值，而是为整个下行带宽上报一个第一 PMI 值 i_1，并为每个 subband 上报一个第二 PMI 值 i_2；
  - 对于 TM 4/8/9（支持空分复用），上报的 CQI/PMI 是基于上报的 RI 所指示的 rank 来计算的；对于其它的 TM 模式（只有 TM 6，不支持空分复用），上报的 CQI/PMI 是基于 rank = 1 来计算的；
  - Subband 的划分见 10.4.2.2 节的介绍。其中 subband 的大小（即 k 值）与下行系统带宽相关。对应关系见 36.213 的 Table 7.2.1-3。

Table 7.2.1-3: Subband Size (*k*) vs. System Bandwidth

System Bandwidth N_{RB}^{DL}	Subband Size (*k*)
6 - 7	NA
8 - 10	4
11 - 26	4
27 - 63	6
64 - 110	8

mode 1-2 实际需要发送哪些信息，这些信息占多少比特等可参见 36.212 的 5.2.2.6.1 节。从该节内容可以看出，具体上报哪些信息与特定的 TM 和天线端口配置相关。

这里我们以 36.212 的 Table 5.2.2.6.1-1A 为例，来说明一下如何理解这些表，这样大家就能够更清晰地理解每种上报模式下需要发送哪些信息（CQI/PMI）。（RI 见 36.212 的 Table 5.2.2.6.1-2）

36.212 的 Table 5.2.2.6.1-1A 对应的是 8 天线端口下的带 PMI/RI 上报的 TM 9。通过前面的介绍，我们有

表 10-2：36.212 的 Table 5.2.2.6.1-1A 解析

域	说明
Wideband CQI codeword 0	UE 会为每个 codeword（这里是 codeword 0）上报一个宽带 CQI 值，而每个 wideband CQI 占 4 比特。
Wideband CQI codeword 1	UE 会为每个 codeword（这里是 codeword 1）上报一个宽带 CQI 值，而每个 wideband CQI 占 4 比特。但当 rank=1 时，只存在 codeword 0，此时不存在此字段。
Wideband first PMI i_1	对应 rank 从 1 到 8，i_1 需要的比特数为 $\{4,4,2,2,2,2,2,0\}$，而 UE 只会为整个下行带宽上报一个第一 PMI 值 i_1，所以对应每种 rank，最终需要发送的比特数为 $\{4,4,2,2,2,2,2,0\}$。
Subband second PMI i_2	对应 rank 从 1 到 8，i_2 需要的比特数为 $\{4,4,4,3,0,0,0,0\}$，而 UE 会为每个 subband 上报一个第二 PMI 值 i_2，subband 的个数为 N，所以对应每种 rank，最终需要发送的比特数为 $\{4N,4N,4N,3N,0,0,0,0\}$。

从上面的介绍可以看出，计算最终需要上报多少比特信息的过程，就是先确定 TM 模式、天线端口数、配置的上报模式和使用的 RI，然后根据对应的上报模式需要上报的信息，以及每个 CQI/PMI 元素所占的比特数得到最终需要发送的比特数。

（2）**Higher Layer-configured subband feedback**：eNodeB 决定 UE 应该为哪些 subband 生成 CQI 报告。UE 会上报一个反映整个下行系统带宽的平均信道质量的 CQI 以及为每一个配置的 subband 上报一个 CQI。取决于 TM 模式的不同，PMI/RI 也会随着报告上报。

Subband 的划分见 10.4.2.2 节的介绍。其中 subband 的大小（即 k 值）与下行系统带宽相关。对应关系见 36.213 的 Table 7.2.1-3。

Table 7.2.1-3: Subband Size (*k*) vs. System Bandwidth

System Bandwidth N_{RB}^{DL}	Subband Size (*k*)
6 - 7	NA
8 - 10	4
11 - 26	4
27 - 63	6
64 - 110	8

subband CQI 使用一个 2 比特值（每个 codeword 独自编码）来表示相对于 wideband CQI 的偏移。计算公式如下：

subband differential CQI offset level = subband CQI index – wideband CQI index

2 比特的 subband differential CQI value 与 subband differential offset level 之间的对应关系见 36.213 的 Table 7.2.1-2：

Table 7.2.1-2: Mapping subband differential CQI value to offset level

Subband differential CQI value	Offset level
0	0
1	1
2	≥2
3	≤-1

- **Mode 3-0**：（3 表示上报的是 Higher Layer-configured subband 的 CQI，0 表示不上报 PMI 值）
  - UE 会上报一个反映整个下行系统带宽的平均信道质量的 wideband CQI 值；
  - UE 会为每个 subband 上报一个 subband CQI 值；
  - 即使 RI > 1，wideband CQI 和 subband CQI 反映的都是第一个 codeword 的信道质量；

- 对于 TM 3，上报的 CQI 值是基于上报的 RI 所指示的 rank 来计算的；对于其它的 TM 模式，上报的 CQI 值是基于 rank = 1 计算的。

mode 3-0 实际需要发送哪些信息，这些信息占多少比特等可参见 36.212 的 Table 5.2.2.6.2-1。

- **Mode 3-1**：（3 表示上报的是 Higher Layer-configured subband 的 CQI，1 表示只反馈 1 个 PMI 值）
  - UE 会为整个下行系统带宽上报一个 PMI；
  - UE 会为每个 codeword 上报一个宽带 CQI 值，该 CQI 值反映的是整个下行带宽的平均信道质量，且假定该 CQI 在所有 subband 只使用一个预编码矩阵；
  - UE 会为每个 codeword 的每个 subband 上报一个 subband CQI 值，且假定该 CQI 在所有 subband 只使用一个预编码矩阵；
  - 在“带 8 个 CSI-RS 端口的 TM 9”配置下，UE 并不会上报对应整个下行系统带宽的 PMI，而是会对应该 PMI，上报一个第一 PMI 值 i_1，以及上报一个第二 PMI 值 i_2；
  - 对于 TM 4/8/9，上报的 CQI/PMI 是基于上报的 RI 所指示的 rank 来计算的；对于其它的 TM 模式，上报的 CQI/PMI 是基于 rank = 1 计算的。

mode 3-1 实际需要发送哪些信息，这些信息占多少比特等可参见 36.212 的 Table 5.2.2.6.2-2 和 Table 5.2.2.6.2-2A。

（3）**UE-selected subband feedback**：UE 选择最好的 M 个 subband（每个 subband 的大小为 k）进行上报。上报的信息包括：所选择 M 个 subband 索引、一个反映所选择的 M 个 subband 的平均信道质量的 CQI（注意：不是为选择的每个 subband 上报一个 CQI）、一个反映整个下行系统带宽的平均信道质量的 CQI。取决于 TM 模式的不同，PMI/RI 也会随着报告上报。

M 和 k 的取值与下行系统带宽的范围相关，其对应关系见 36.213 的 Table 7.2.1-5。对比 wideband reports 中使用的 Table 7.2.1-3 可知，UE-selected reports 划分 subband 的粒度更细，即在相同的下行系统带宽下，其 k 值更小，subband 数更多。subband 的划分见 10.4.2.2 节的介绍。

Table 7.2.1-5: Subband Size (*k*) and Number of Subbands (*M*) in S vs. Downlink System Bandwidth

System Bandwidth $N_{\mathrm{RB}}^{\mathrm{DL}}$	Subband Size *k* (RBs)	*M*
6 – 7	NA	NA
8 – 10	2	1
11 – 26	2	3
27 – 63	3	5
64 – 110	4	6

对于 UE 选择的 subband 上报模式，UE 还会上报一个 combinatorial index *r* 以反映所选择的 M 个 subband 的位置。*r* 的定义如下：

$$r = \sum_{i=0}^{M-1} \left\langle \begin{matrix} N - s_i \\ M - i \end{matrix} \right\rangle$$

集合 $\{s_i\}_{i=0}^{M-1}$, $(1 \le s_i \le N, \quad s_i < s_{i+1})$包含了 M 个已排序的 subband 索引。

$\left\langle \begin{matrix} x \\ y \end{matrix} \right\rangle = \begin{cases} \binom{x}{y} & x \ge y \\ 0 & x < y \end{cases}$ 是扩展的二项式系数，会生成一个唯一的 $r \in \left\{0, \cdots, \binom{N}{M} - 1\right\}$。其中 $\binom{n}{k} = \frac{n!}{k! \times (n-k)!}$ 为组合的计算公式。

用于指示所选的 M 个 subband 所在位置（即 subband 索引）的比特数为 $L = \left\lceil \log_2 \binom{N}{M} \right\rceil$ 。

用于反映所选的 M 个 subband 的 CQI 使用一个 2 比特的值（每个 codeword 独自编码）来表示相对于 wideband CQI 的偏移。计算公式如下：

differential CQI offset level = M selected subbands CQI index – wideband CQI index

2 比特的 differential CQI value 与 differential CQI offset level 之间的对应关系见 36.213 的 Table 7.2.1-4：

Table 7.2.1-4: Mapping differential CQI value to offset level

Differential CQI value	Offset level
0	≤1
1	2
2	3
3	≥4

- **Mode 2-0**：（2 表示上报的是 UE-selected subband 的 CQI，0 表示不上报 PMI 值）
  - UE 会选择最好的 M 个 subband 进行上报，每个 subband 的大小为 k。所选的 M 个 subband 对应的位置通过 combinatorial index *r* 进行上报；
  - UE 会上报一个反映所选择的 M 个 subband 的平均信道质量的 CQI。该 CQI 反映的是第一个 codeword 的信道质量，即使在 RI > 1 的情况下也是如此；
  - UE 会上报一个反映整个下行系统带宽的平均信道质量的 wideband CQI 值。该 CQI 反映的是第一个 codeword 的信道质量，即使在 RI > 1 的情况下也是如此；
  - 对于 TM 3，上报的 CQI 是基于上报的 RI 所指示的 rank 来计算的；对于其它的 TM 模式，上报的 CQI 是基于 rank = 1 计算的。

mode 2-0 实际需要发送哪些信息，这些信息占多少比特等可参见 36.212 的 Table 5.2.2.6.3-1。

- **Mode 2-2**：（第一个 2 表示上报的是 UE-selected subband 的 CQI，第二个 2 表示上报多个 PMI 值）
  - UE 会选择最好的 M 个 subband 进行上报，每个 subband 的大小为 k。所选的 M 个 subband 对应的位置通过 combinatorial index *r* 进行上报；
  - UE 会为每个 codeword 上报一个反映整个下行系统带宽的平均信道质量的 wideband CQI 值；
  - UE 会为每个 codeword 上报一个反映所选择的 M 个 subband 的平均信道质量的 CQI 值；且假定该 CQI 在这 M 个 subband 中的每一个 subband 上，都使用一个相同预编码矩阵；
  - 如果 UE 没有配置“带 8 个 CSI-RS 端口的 TM 9”，则 UE 会上报一个反映整个下行系统带宽的最好 PMI，以及一个反映所选择的 M 个 subband 的最好 PMI；

- 如果 UE 配置了“带 8 个 CSI-RS 端口的 TM 9”，UE 会为整个下行带宽上报一个第一 PMI（i_1）和一个第二 PMI（i_2），并会为所选的 M 个 subband 上报另一个第二 PMI（i_2），即会上报 3 个 PMI；
- UE 上报的 PMI 会从码本子集（codebook subset）中选取。
- 对于 TM 4/8/9，上报的 CQI/PMI 是基于上报的 RI 所指示的 rank 来计算的；对于其它的 TM 模式，上报的 CQI/PMI 是基于 rank = 1 计算的。

mode 2-2 实际需要发送哪些信息，这些信息占多少比特等可参见 36.212 的 Table 5.2.2.6.3-2 和 Table 5.2.2.6.3-2A。

注：前面介绍的主要是 CQI/PMI 的上报，而关于 RI 上报所占的比特数，可分别参见 36.212 的 Table 5.2.2.6.1-2、Table 5.2.2.6.2-3、Table 5.2.2.6.3-3。

【参考资料】

[1] TS 36.212 的 5.2.2.6 节 Channel coding of control information

[2] TS 36.213 的 7.2.1 节 Aperiodic CSI Reporting using PUSCH

[3] http://www.sharetechnote.com/html/Handbook_LTE_CQI_PMI_FeedbackType.html

10.4.3 上行控制信息映射到 PUSCH 的方式

如果 UE 需要在某个子帧的 PUSCH 上发送上行数据（即在对应子帧有 UL grant），并且同时需要发送上行控制信息（ACK/NACK、CQI、PMI、RI），则上行控制信息将与数据复用在一起，共同在 PUSCH 上传输。

由于 SR 只有在 UE 没有 UL grant 的情况，才会用于向 eNodeB 申请 PUSCH 资源，所以在已经有 PUSCH 资源的情况下，UE 就不必发送 SR 了。而 BSR 属于 MAC PDU 的一部分（BSR MAC control element），是作为上行数据的一部分进行传输的。因此，SR 和 BSR 的上报并不在本章的讨论范畴之内。

当上行控制信令（ACK/NACK、CQI、PMI、RI）在 PUSCH 上传输时，其映射到 RE 的方式如图 10-7 所示：

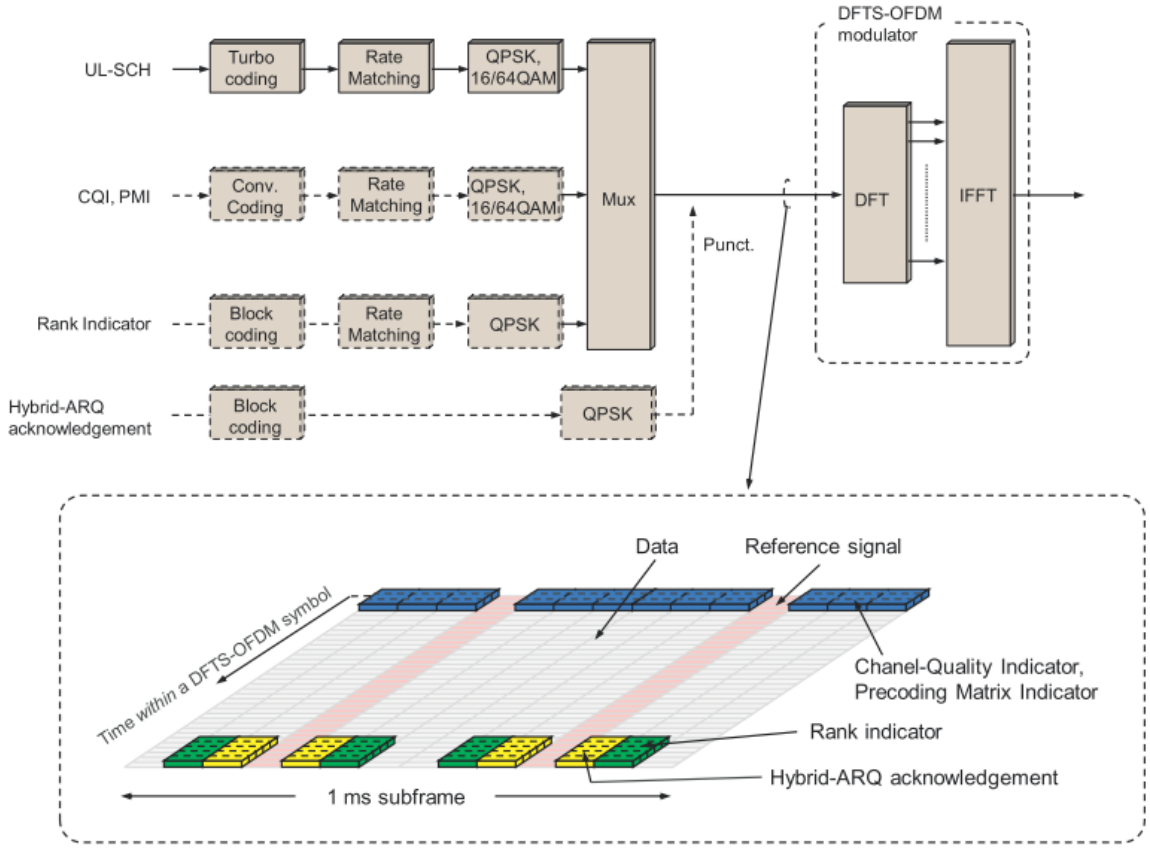


图 10-7：PUSCH 上控制信令和数据的复用

UE 上报的 ACK/NACK 对于下行传输的正确处理很重要，对于 1 或 2 比特的 ACK/NACK，固定使用健壮的 QPSK 调制，对于需要发送更多比特 ACK/NACK 的场景，则使用与数据相同的调制模式。

与数据传输不同，ACK/NACK 并不能通过重传和强有力的信道编码来处理这些变化。因此 ACK/NACK 在紧靠着 DMRS 的 RE 上传输，从而使得信道估计能够更加精确。这在高多普勒频移的条件下尤为重要，因为此时信道条件可能在一个 slot 内发生变化。

由于下行数据传输与对应的 ACK/NACK 有固定的 timing 关系（见第 13 章关于“下行 HARQ”的介绍），因此 eNodeB 能够知道何时去接收来自 UE 的 ACK/NACK，因此能够正确地解码 ACK/NACK 和数据部分。但是存在 eNodeB 发送了用于下行调度的 PDCCH，但 UE 没有收到（DTX）的情况，此时 UE 并不会回复 ACK/NACK，但 eNodeB 却期望收到 ACK/NACK，从而导致 UE 编码和 eNodeB 解码方式的不匹配。也就是说，如果此时速率匹配依赖于 ACK/NACK 是否发送，则可能导致 UL-SCH 解码失败。为了避免这种错误，在 UL-SCH 的比特流中，HARQ ACK/NACK 是被打孔的，从而那些未被打孔的比特并不受是否存在 ACK/NACK 的影响。

CQI 和 PMI 时分复用到 PUSCH 中，并使用与数据部分相同的调制模式。信道状态报告主要用于中低等程度的多普勒频移中，其信道质量相对稳定，因此并不需要特别的匹配方式。

RI 同 CQI/PMI 的映射方式不同，但同 ACK/NACK 类似，位于靠近 DMRS 的 RE 上，从而使得 RI 相比 CQI/PMI 来说更加健壮。这样做的原因在于正确解码 CQI/PMI 的前提是已经正确解码 RI。从调制的角度上看，RI 固定使用 QPSK 调制。

如果上行使用空分复用，PUSCH 上会同时传输 2 个 TB，此时 CQI 和 PMI 会复用到使用最高 MCS 的那个编码 TB 上，并在该 TB 所映射到的每一层上使用前面介绍的复用方式进行复用。这样就能让 CQI 和 PMI 在信道条件最好的层（1 或 2 层）上传输。也就是说，MCS 相对较差的 TB 对应的层上是不会发送 CQI 和 PMI 的。

如果上行使用空分复用，ACK/NACK 和 RI 会在所有的层上重复传输，并在每一层上使用与单层传输时相同的复用方式来与编码数据进行复用。此时在每一层上都传输相同的信息，但不同层会使用不同的加扰，从而提供了分集增益。

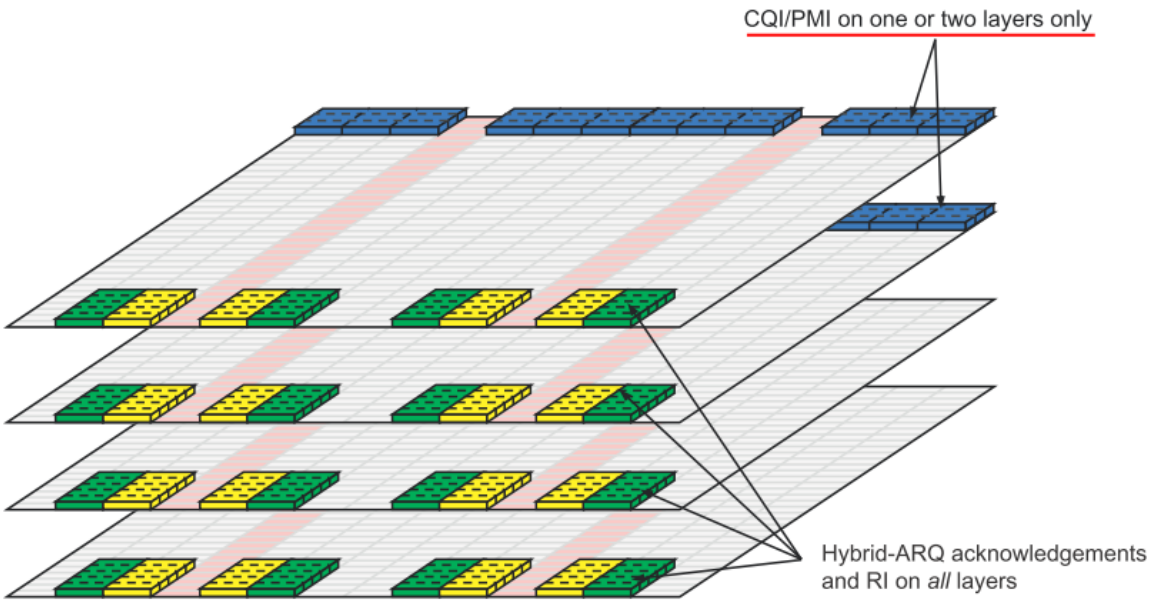


图 10-8：上行空分复用下，CQI/PMI、RI、ACK/NACK 与 PUSCH 的复用

上行控制信息与 PUSCH 如何进行复用的详细内容，可见 36.211 的 5.2.2.6 节。

【参考资料】

[1] TS 36.212 的 5.2.2.6 节 Channel coding of control information

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.4.2 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.4 节

10.5 周期性 CSI 上报（Periodic CSI Reporting）

eNodeB 是通过 *CQI-ReportPeriodic* 来配置周期性 CSI 上报的相关参数的。每个下行载波单元都可以独立地配置各自的周期性 CSI 参数。

```
CQI-ReportPeriodic ::=      CHOICE {
    release                         NULL,
    setup                           SEQUENCE {
```

```
        cqi-PUCCH-ResourceIndex             INTEGER (0..1185), -----CQI资源索引（PUCCH 2资源）
        cqi-pmi-ConfigIndex                 INTEGER (0..1023), -----指定了周期性CQI/PMI上报的
周期periodicity以及在周期内的偏移
        cqi-FormatIndicatorPeriodic         CHOICE {-----用于指定周期性CSI上报模式
            widebandCQI                         NULL,
            subbandCQI                          SEQUENCE {
                k                                   INTEGER (1..4)
            },
        },
        ri-ConfigIndex                      INTEGER (0..1023)   OPTIONAL,               -
- Need OR-----指定了RI上报的周期periodicity以及相对偏移。
        simultaneousAckNackAndCQI           BOOLEAN-----是否允许在同一子帧同时发送ACK/NACK和CSI
    }
}

CQI-ReportPeriodic-r10 ::=      CHOICE {
    release                             NULL,
    setup                               SEQUENCE {
        cqi-PUCCH-ResourceIndex-r10         INTEGER (0..1184), -----CQI资源索引（PUCCH 2资
源）。用于天线端口0
        cqi-PUCCH-ResourceIndexP1-r10       INTEGER (0..1184)               OPTIONAL,   -
- Need OR-----CQI资源索引（PUCCH 2资源）。用于天线端口1，只有在支持SORTD的情况下才存在。
        cqi-pmi-ConfigIndex             INTEGER (0..1023), -----指定了周期性CQI/PMI上报的周期
periodicity以及在周期内的偏移。对应子帧集1（csi-MeasSubframeSet1）
        cqi-FormatIndicatorPeriodic-r10     CHOICE {,-----用于指定周期性CSI上报模式
            widebandCQI-r10                     SEQUENCE {
                csi-ReportMode-r10      ENUMERATED {submode1, submode2}     OPTIONAL    -
- Need OR          -----对应36.213的PUCCH_format1-1_CSI_reporting_mode。配置了带8个CSI-RS port的TM 9
的UE，可以用此字段对Mode 1-1进一步配置子模式1（submode 1）或子模式2（submode 2）
            },
            subbandCQI-r10                      SEQUENCE {
                k                               INTEGER (1..4),
                periodicityFactor-r10               ENUMERATED {n2, n4}
            }
        },
        ri-ConfigIndex                  INTEGER (0..1023)       OPTIONAL,               -
- Need OR-----指定了RI上报的周期periodicity以及相对偏移。对应子帧集1（csi-MeasSubframeSet1）
        simultaneousAckNackAndCQI       BOOLEAN, -----是否允许在同一子帧同时发送ACK/NACK和CSI。
只对PCell生效
        cqi-Mask-r9                     ENUMERATED {setup}      OPTIONAL,               -
- Need OR-----用于限制CQI/PMI/PTI/RI是否只能在DRX周期的on-duration期间上报。对所有serving cell都生效
        csi-ConfigIndex-r10             CHOICE {
            release                         NULL,
            setup                           SEQUENCE {
                cqi-pmi-ConfigIndex2-r10
        INTEGER (0..1023), -----指定了周期性CQI/PMI上报的周期periodicity以及在周期内的偏移。对应子帧集
2（csi-MeasSubframeSet2）
                ri-ConfigIndex2-r10             INTEGER (0..1023)       OPTIONAL        -
- Need OR-----指定了RI上报的周期periodicity以及相对偏移。对应子帧集2（csi-MeasSubframeSet2）
            }
        }       OPTIONAL                                                            -
- Need ON
    }
}
```

如果 UE 没有配置同时传输 PUSCH 和 PUCCH，或者如果 UE 配置了同时传输 PUSCH 和 PUCCH 但并没有 PUSCH 传输，则在同一子帧上需要同时传输 CSI 和 SR 时，UE 会发送 SR，而 CSI 会被丢弃。

36.213 的 10.1 节介绍了当 CSI 与 HARQ-ACK 冲突时 UE 的行为以及所使用的 PUCCH format（或见本书的 13.6.2 节）。

如果 *ttiBundling* 设置为 TRUE（即使能了该 UE 的 TTI bundling 功能），并且在 TTI bundle 的某个子帧上，同时需要发送 UL-SCH 和周期性 CSI 上报，则 UE 会丢弃该周期性 CSI 上报，而不是将其复用到 PUSCH 中去。当 UE 配置了 TTI bundling，就不应该配置“同时发送 PUCCH 和 PUSCH”。（见 36.213 的 7.2.2 节）

CQI/PMI 的计算基于最后一个上报的 RI，如果缺失了最后一个上报的 RI，UE 会根据 *codebookSubsetRestriction* 中隐含的可用最小 RI 来计算 CQI/PMI。如果配置了多于一个 CSI 子帧集合（见 10.3.2 节）用于上报，则 CQI/PMI 的计算需要基于相同的子帧集合中最后一个上报的 RI。

在载波聚合中，为每个小区上报的周期性 CSI 是通过在时间上使用不同的 offset 来避免不同小区间的冲突的。

这里还要介绍一下 DRX（可参见第 16 章关于“DRX”的介绍）对处于 RRC_CONNECTED 态的 UE 的周期性 CSI 上报的影响。36.331 中定义了一个字段 *cqi-Mask*，用于限制 CQI/PMI/PTI/RI 是否只能在 DRX 周期的 on-duration 期间上报。具体地说，如果 *cqi-Mask* 的值为 setup，且定时器 *onDurationTimer* 没有运行时，UE 不能上报在 PUCCH 上传输的 CQI/PMI/RI/PTI（即此时只能在 *onDurationTimer* 运行期间上报周期性 CSI）；而如果没有设置 *cqi-Mask*，且当 UE 不在激活期内时，UE 不能上报在 PUCCH 上传输的 CQI/PMI/RI/PTI（即此时只能在激活期上报周期性 CSI）。（见 36.321 的 5.7 节）

10.5.1 PUCCH reporting type

与 PUSCH 相比，PUCCH 只能携带有限的信息，这意味着周期性 CSI 上报中的不同类型信息可能无法在同一个子帧中发送给 eNodeB。因此，UE 在不同的时间点上，可能上报不同类型的信息。对于使用 PUCCH 的 CSI 上报模式，定义多种 CQI/PMI 和 RI 上报类型，每种上报类型的大小与 PUCCH 上报模式的对应关系见 36.213 的 Table 7.2.2-3：

- Type 1：支持 UE 选择的 subband 的 CQI 反馈

- Type 1a：支持 subband CQI 和第二 PMI 反馈
- Type 2/2b/2c：支持 wideband CQI 和 PMI 反馈
- Type 2a：支持 wideband PMI 反馈
- Type 3：支持 RI 反馈
- Type 4：支持 wideband CQI 反馈
- Type 5：支持 RI 和 wideband PMI 反馈
- Type 6：支持 RI 和 PTI 反馈

在这里，我们同样先介绍一下周期性 CSI 上报中每个 CQI/PMI/RI 元素所占的比特数：

- **Wideband CQI**：使用 4 比特表示 codeword 0 的 wideband 信道质量信息。如果存在 codeword 1（且需要上报对应的 CQI）的话，额外使用 3 比特表示 codeword 1 的 wideband 信道质量信息（3-bit wideband spatial differential CQI）；（4 比特信息与 36.213 的 Table 7.2.3-1 对应，而 3-bit wideband spatial differential CQI 信息与 36.213 的 Table 7.2-2 对应）
- **Subband CQI**：使用 4 比特表示 codeword 0 的 subband 信道质量信息。如果存在 codeword 1（且需要上报对应的 CQI）的话，额外使用 3 比特表示 codeword 1 的 subband 信道质量信息（additional 3-bit subband spatial differential CQI value for codeword 1 offset level）。同时，还需要 L 比特表示所选择的 subband 的索引（L 的计算见 10.5.4 节以及后续介绍）；
- **RI**：对于使用至多 2 层传输的 UE 而言，使用 1 比特表示 RI；对于使用至多 4 层传输的 UE 而言，使用 2 比特表示 RI；对于使用至多 8 层传输的 UE 而言，使用 3 比特表示 RI；（可见 Type 3 的描述）
- **PMI**（见 36.213 的 7.2.4 节）：
    - 对于 2 和 4 天线端口传输，每个 PMI 值对应 36.211 的 Table 6.3.4.2.3-1 或 Table 6.3.4.2.3-2 的 codebook index。所以有
        - 对于 2 天线端口（$\{0,1\}$或$\{15,16\}$）传输，且 RI 值为 1 的情况，PMI 的取值范围为 $n \in \{0,1,2,3\}$，对应 36.211 的 Table 6.3.4.2.3-1 中 $\upsilon = 1$ 且 codebook index 为 n 的项。可以看出，4 种可选值，因此使用 2 比特表示；
        - 对于 2 天线端口（$\{0,1\}$或$\{15,16\}$）传输，且 RI 值为 2 的情况，PMI 的取值范围为 $n \in \{0,1\}$，对应 36.211 的 Table 6.3.4.2.3-1 中 $\upsilon = 2$ 且 codebook index 为 n + 1 的项。可以看出，2 种可选值，因此使用 1 比特表示；（注：对于使用闭环空分复用的 TM 模式，当层数为 2 时，codebook index 0 不能被使用，这也是为什么这里使用 n+1 而不是 n 的原因）

- 对于 4 天线端口（$\{0,1,2,3\}$ 或{15,16,17,18}）传输，PMI 的取值范围为 $n \in \{0,1,\cdots,15\}$，对应 36.211 的 Table 6.3.4.2.3-2 中 υ 等于对应的 RI 值，且 codebook index 为 n 的项。可以看出，每种 RI 下，有 16 种可选值，因此使用 4 比特表示；

  - 对于 8 天线端口传输（只用于 TM 9），每个 PMI 值对应 36.213 的 Table 7.2.4-1 至 Table 7.2.4-8 的成对 codebook index。有

    - 对于 8 天线端口 $\{15,16,17,18,19,20,21,22\}$，first PMI（$i_1$）的值 $i_1 \in \{0,1,\cdots, f(\upsilon)-1\}$ 和 second PMI（i_2）的值 $i_2 \in \{0,1,\cdots, g(\upsilon)-1\}$ 对应 36.213 的 Table 7.2.4-*j* 中的 codebook indices i_1 和 i_2，其中 υ 等于对应 RI 值，*j* = υ，$f(\upsilon) = \{16,16,4,4,4,4,4,1\}$（即 i_1 对应需要的比特数为 $\{4,4,2,2,2,2,2,0\}$）和 $g(\upsilon) = \{16,16,16,8,1,1,1,1\}$（即 i_2 对应需要的比特数为 $\{4,4,4,3,0,0,0,0\}$）。

    - 而在以下几种情况下（具体会在 10.5.5 节中介绍），有

      - 用于 PUCCH mode 1-1 submode 2 的 sub-sampled codebook 在 36.213 的 Table 7.2.2-1D 中定义（用于第一和第二预编码矩阵索引 i_1 和 i_2），所使用的比特数也能从该表中得到（$\{4,4,4,4,2,2,2,0\}$）。

      - 用于 PUCCH mode 1-1 submode 1 的联合编码的 rank 和第一预编码矩阵索引 i_1 在 36.213 的 Table 7.2.2-1E 中定义，所使用的比特数也能从该表中得到（$\{3,3,1,1,1,1,1,0\}$）。

      - 用于 PUCCH mode 2-1 的 sub-sampled codebook 的第二预编码矩阵索引 i_2 在 36.213 的 Table 7.2.2-1F 中定义，所使用的比特数也能从该表中得到（$\{4,2,2,2,0,0,0,0\}$）。

- **PTI**：其值为 0 或 1，因此使用 1 比特表示。

我对 36.213 的 Table 7.2.2-3 做了一些注释，解释了每种上报类型的 payload 包含了哪些信息，以及这些信息包含了多少比特。

Table 7.2.2-3: PUCCH Reporting Type Payload size per PUCCH Reporting Mode and Mode State

PUCCH Reporting Type	Reported	Mode State	PUCCH Reporting Modes			
			Mode 1-1 (bits/BP)	Mode 2-1 (bits/BP)	Mode 1-0 (bits/BP)	Mode 2-0 (bits/BP)
1	Sub-band CQI	RI = 1	NA	4+L （4-bit subband CQI + L-bit index）	NA	4+L （4-bit subband CQI + L-bit index）
		RI > 1	NA	7+L （4-bit subband CQI +3-bit subband spatial differential CQI + L-bit index）	NA	4+L （4-bit subband CQI + L-bit index）
1a （可以从 36.213 的 Table 7.2.2-1F 得出 subband second PMI 使用的 bit 数）	Sub-band CQI / second PMI	8 antenna ports RI = 1	NA	8+L （4-bit subband CQI + L-bit index + 4-bit subband second PMI）	NA	NA
		8 antenna ports 1 < RI < 5	NA	9+L （4-bit subband CQI +3-bit subband spatial differential CQI + L-bit index + 2-bit subband	NA	NA

				second PMI）		
		8 antenna ports RI > 4	NA	7+L （4-bit subband CQI +3-bit subband spatial differential CQI + L-bit index + 0-bit subband second PMI）	NA	NA
2	Wideband CQI/PMI	2 antenna ports RI = 1	6 （4-bit wideband CQI + 2-bit PMI）	6 （4-bit wideband CQI + 2-bit PMI）	NA	NA
		4 antenna ports RI = 1	8 （4-bit wideband CQI + 4-bit PMI）	8 （4-bit wideband CQI + 4-bit PMI）	NA	NA
		2 antenna ports RI > 1	8 （4-bit wideband CQI + 3-bit wideband spatial differential CQI + 1-bit PMI）	8 （4-bit wideband CQI + 3-bit wideband spatial differential CQI + 1-bit PMI）	NA	NA
		4 antenna ports RI > 1	11 （4-bit wideband CQI + 3-bit	11 （4-bit wideband CQI + 3-	NA	NA

			wideband spatial differential CQI + 4-bit PMI）	bit wideband spatial differential CQI + 4-bit PMI）		
2a（可以从 36.211 的 Table 6.3.4.2.3-3~10 的 i_1 得出使用的 bit 数）	Wideband first PMI	8 antenna ports RI < 3	NA	4（4-bit wideband first PMI）	NA	NA
		8 antenna ports 2 < RI < 8	NA	2（2-bit wideband first PMI）	NA	NA
		8 antenna ports RI = 8	NA	0	NA	NA
2b（可以从 36.211 的 Table 6.3.4.2.3-3~10 的 i_2 得出 wideband second PMI 使用的 bit 数）	Wideband CQI / second PMI	8 antenna ports RI = 1	8（4-bit wideband CQI + 4-bit wideband second PMI）	8（4-bit wideband CQI + 4-bit wideband second PMI）	NA	NA
		8 antenna ports 1 < RI < 4	11（4-bit wideband CQI + 3-bit wideband spatial differential CQI + 4-bit wideband second PMI）	11（4-bit wideband CQI + 3-bit wideband spatial differential CQI + 4-bit wideband second PMI）	NA	NA
		8 antenna ports RI = 4	10	10	NA	NA

			（4-bit wideband CQI + 3-bit wideband spatial differential CQI + 3-bit wideband second PMI）	（4-bit wideband CQI + 3-bit wideband spatial differential CQI + 3-bit wideband second PMI）		
		8 antenna ports RI > 4	7 （4-bit wideband CQI + 3-bit wideband spatial differential CQI + 0-bit wideband second PMI）	7 （4-bit wideband CQI + 3-bit wideband spatial differential CQI + 0-bit wideband second PMI）	NA	NA
2c (first PMI / second PMI 所占的 bit 数见 36.213 的 Table 7.2.2-1D)	Wideband CQI / first PMI / second PMI	8 antenna ports RI = 1	8 （4-bit wideband CQI + 4-bit first PMI/second PMI）	NA	NA	NA
		8 antenna ports 1 < RI ≤ 4	11 （4-bit wideband CQI + 3-bit wideband spatial differential CQI + 4-bit first	NA	NA	NA

			PMI/second PMI）			
		8 antenna ports 4 < RI ≤ 7	9 （4-bit wideband CQI + 3-bit wideband spatial differential CQI + 2-bit first PMI/second PMI）	NA	NA	NA
		8 antenna ports RI = 8	7 （4-bit wideband CQI + 3-bit wideband spatial differential CQI + 0-bit first PMI/second PMI）	NA	NA	NA

3	RI	2/4 antenna ports, 2-layer spatial multiplexing	1 （至多 2 层，所以使用 1 bit 表示）	1 （至多 2 层，所以使用 1 bit 表示）	1 （至多 2 层，所以使用 1 bit 表示）	1 （至多 2 层，所以使用 1 bit 表示）
		8 antenna ports, 2-layer spatial multiplexing	1 （至多 2 层，所以使用 1 bit 表示）	NA	NA	NA
		4 antenna ports, 4-layer spatial multiplexing	2 （至多 4 层，所以使用 2 bits 表示）	2 （至多 4 层，所以使用 2 bits 表示）	2 （至多 4 层，所以使用 2 bits 表示）	2 （至多 4 层，所以使用 2 bits 表示）
		8 antenna ports, 4-layer spatial multiplexing	2 （至多 4 层，所以使用 2 bits 表示）	NA	NA	NA
		8-layer spatial multiplexing	3 （至多 8 层，所以使用 3 bits 表示）	NA	NA	NA

4	Wideband CQI	RI = 1 or RI>1	NA	NA	4 （使用 4 bit 表示 wideband CQI）	4 （使用 4 bit 表示 wideband CQI）
5 (所占的 bit 数见 36.213 的 Table 7.2.2-1E)	RI/ first PMI	8 antenna ports, 2-layer spatial multiplexing	4 (见 36.213 的 Table 7.2.2-1E)	NA	NA	NA
		8 antenna ports, 4 and 8-layer spatial multiplexing	5 (见 36.213 的 Table 7.2.2-1E)			
6	RI/PTI	8 antenna ports, 2-layer spatial multiplexing	NA	2 （1-bit RI + 1-bit PTI）	NA	NA
		8 antenna ports, 4-layer spatial multiplexing	NA	3 （2-bit RI + 1-bit PTI）	NA	NA
		8 antenna ports, 8-layer spatial multiplexing	NA	4 （3-bit RI + 1-bit PTI）	NA	NA

如果同一小区内，使用 PUCCH 上报类型 3/5/6 的 CSI 上报（包含 RI），与使用 PUCCH 上报类型 1/1a/2/2a/2b/2c/4 的 CSI 上报（不包含 RI）发生了冲突，则 PUCCH 上报类型 1/1a/2/2a/2b/2c/4 的 CSI 上报会由于优先级较低而被丢弃。（即 RI 上报的优先级高于 CQI/PMI 上报的优先级。这个好理解，毕竟 CQI/PMI 是基于 RI 来确定的）

如果 UE 配置了多于一个 serving cell（载波聚合），则 UE 在任一给定的子帧上只会上报其中一个 serving cell 的 CSI 报告。

如果在某一给定的子帧上，一个 serving cell 使用 PUCCH 上报类型 3/5/6/2a 的 CSI 上报（包含 RI 或 PMI，但不包含 CQI），与另一个 serving cell 使用的 PUCCH 上报类型 1/1a/2/2b/2c/4 的 CSI 上报（包含 CQI）发生了冲突，则 PUCCH 上报类型 1/1a/2/2b/2c/4 的 CSI 上报会由于优先级较低而被丢弃。（即 RI/PMI 上报的优先级高于 CQI 上报的优先级。这个也好理解，毕竟 CQI 是在某一特定 PMI/RI 的基础上进行测量的）

如果在某一给定的子帧上，一个 serving cell 使用的 PUCCH 上报类型 2/2b/2c/4 的 CSI 上报（wideband CQI 上报），与另一个 serving cell 使用的 PUCCH 上报类型 1/1a 的 CSI 上报（subband CQI 上报）发生了冲突，则 PUCCH 上报类型 1/1a 的 CSI 上报会由于优先级较低而被丢弃。（即 wideband CQI 上报的优先级高于 subband CQI 上报的优先级）

如果在某一给定的子帧上，不同 serving cell 的 CSI 上报使用的 PUCCH 上报类型拥有相同的优先级，则会上报拥有最小 *ServCellIndex* 的 serving cell 的 CSI，而其它 serving cell 的 CSI 将会被丢弃。

从上面的介绍可以看出，载波聚合下，在任意一个给定的子帧上，UE 只会给一个 serving cell 上报周期性 CSI。也就是说，在给不同的 serving cell 配置周期性 CSI 的周期和偏移时，应该尽量避免不同 serving cell 上的 CSI 上报在同一子帧上发生冲突。

再结合 10.5.4 节的介绍，就能够确定每种周期性 CSI 上报需要上报哪些内容。

10.5.2 如何配置周期

对于每一个 serving cell 来说，其 *cqi-pmi-ConfigIndex*（$I_{CQI/PMI}$）指定了周期性 CQI/PMI 上报的周期 periodicity（N_{pd}，以子帧为计量单位）以及在周期内的偏移（$N_{OFFSET,CQI}$，以子帧为计量单位）。对于 FDD，见 36.213 的 Table 7.2.2-1A；对于 TDD，见 36.213 的 Table 7.2.2-1C。

Table 7.2.2-1A: Mapping of $I_{CQI/PMI}$ to N_{pd} and $N_{OFFSET,CQI}$ for FDD.

$I_{CQI/PMI}$	Value of N_{pd}	Value of $N_{OFFSET,CQI}$
$0 \le I_{CQI/PMI} \le 1$	2	$I_{CQI/PMI}$
$2 \le I_{CQI/PMI} \le 6$	5	$I_{CQI/PMI} - 2$
$7 \le I_{CQI/PMI} \le 16$	10	$I_{CQI/PMI} - 7$
$17 \le I_{CQI/PMI} \le 36$	20	$I_{CQI/PMI} - 17$
$37 \le I_{CQI/PMI} \le 76$	40	$I_{CQI/PMI} - 37$
$77 \le I_{CQI/PMI} \le 156$	80	$I_{CQI/PMI} - 77$
$157 \le I_{CQI/PMI} \le 316$	160	$I_{CQI/PMI} - 157$
$I_{CQI/PMI} = 317$	Reserved	
$318 \le I_{CQI/PMI} \le 349$	32	$I_{CQI/PMI} - 318$
$350 \le I_{CQI/PMI} \le 413$	64	$I_{CQI/PMI} - 350$
$414 \le I_{CQI/PMI} \le 541$	128	$I_{CQI/PMI} - 414$
$542 \le I_{CQI/PMI} \le 1023$	Reserved	

Table 7.2.2-1C: Mapping of $I_{CQI/PMI}$ to N_{pd} and $N_{OFFSET,CQI}$ for TDD.

$I_{CQI/PMI}$	Value of N_{pd}	Value of $N_{OFFSET,CQI}$
$I_{CQI/PMI} = 0$	1	$I_{CQI/PMI}$
$1 \le I_{CQI/PMI} \le 5$	5	$I_{CQI/PMI} - 1$
$6 \le I_{CQI/PMI} \le 15$	10	$I_{CQI/PMI} - 6$
$16 \le I_{CQI/PMI} \le 35$	20	$I_{CQI/PMI} - 16$
$36 \le I_{CQI/PMI} \le 75$	40	$I_{CQI/PMI} - 36$
$76 \le I_{CQI/PMI} \le 155$	80	$I_{CQI/PMI} - 76$
$156 \le I_{CQI/PMI} \le 315$	160	$I_{CQI/PMI} - 156$
$316 \le I_{CQI/PMI} \le 1023$	Reserved	

注： TDD 下的周期性 CQI/PMI 上报，不同的 TDD 配置，其周期配置有一定的限制。

- 上报周期 $N_{pd}=1$ 只能用于 TDD 配置 0/1/3/4/6，此时系统帧内的所有上行子帧都用于 CQI/PMI 上报；
- 上报周期 $N_{pd}=5$ 只能用于 TDD 配置 0/1/2/6（因为只有 TDD 0/1/2/6 使用 5 ms 的下行到上行的切换周期：Downlink-to-Uplink Switch-point periodicity，见 36.211 的 Table 4.2-2）；
- 上报周期 $N_{pd}=\{10,20,40,80,160\}$ 可用于所有的 TDD 配置。

ri-ConfigIndex（I_{RI}）指定了 RI 上报的周期 periodicity（M_{RI}）以及相对偏移（$N_{OFFSET,RI}$）。见 36.213 的 Table 7.2.2-1B。

RI 的相对偏移 $N_{OFFSET,RI}$ 的取值范围为 $\{0,-1,...,-(N_{pd}-1)\}$。

Table 7.2.2-1B: Mapping of I_{RI} to M_{RI} and $N_{OFFSET,RI}$.

I_{RI}	**Value of** M_{RI}	**Value of** $N_{OFFSET,RI}$
$0 \le I_{RI} \le 160$	1	$-I_{RI}$
$161 \le I_{RI} \le 321$	2	$-(I_{RI}-161)$
$322 \le I_{RI} \le 482$	4	$-(I_{RI}-322)$
$483 \le I_{RI} \le 643$	8	$-(I_{RI}-483)$
$644 \le I_{RI} \le 804$	16	$-(I_{RI}-644)$
$805 \le I_{RI} \le 965$	32	$-(I_{RI}-805)$
$966 \le I_{RI} \le 1023$	Reserved	

前面已经介绍过，如果 UE 被配置成为多于一个 CSI 子帧集（CSI subframe set）进行上报，则 *cqi-pmi-ConfigIndex* 和 *ri-ConfigIndex* 对应子帧集 1（*csi-MeasSubframeSet1*）的 CQI/PMI 和 RI 周期以及相对偏移；*cqi-pmi-ConfigIndex2-r10* 和 *ri-ConfigIndex2-r10* 对应子帧集 2（*csi-MeasSubframeSet2*）的 CQI/PMI 和 RI 周期以及相对偏移。

（1）如果 UE 配置了 wideband CQI/PMI 上报：

- 用于上报 CQI/PMI 的子帧必须满足：

$$(10\times n_f+\lfloor n_s/2\rfloor-N_{OFFSET,CQI})\bmod(N_{pd})=0$$

- 如果配置了 RI 上报，则 RI 上报的周期是 N_{pd} 的 M_{RI} 倍（以子帧为计量单位）。用于上报 RI 的子帧必须满足：

$$(10 \times n_f + \lfloor n_s / 2 \rfloor - N_{OFFSET,CQI} - N_{OFFSET,RI}) \bmod (N_{pd} \cdot M_{RI}) = 0$$

（2）如果 UE 同时配置了 wideband CQI/PMI 和 subband CQI 上报：

- 用于上报 wideband CQI/PMI 和 subband CQI 的子帧必须满足：

$$(10 \times n_f + \lfloor n_s / 2 \rfloor - N_{OFFSET,CQI}) \bmod N_{pd} = 0$$

  - 如果没有传输 PTI（因为没有配置）或最近一次传输的 PTI = 1
    - wideband CQI/wideband PMI（或者 TM 9 中的 wideband CQI/wideband second PMI）上报的周期为 $H \cdot N_{pd}$，其上报的子帧必须满足：

$$(10 \times n_f + \lfloor n_s / 2 \rfloor - N_{OFFSET,CQI}) \bmod (H \cdot N_{pd}) = 0$$

      整数 H 定义为 $H = J \cdot K + 1$，其中 J 为 bandwidth part 的个数（见 10.5.4 节介绍）；K 是通过 *cqi-FormatIndicatorPeriodic-r10* -> *subbandCQI-r10* -> K 来配置的。

    - 在每 2 个连续的 wideband CQI/wideband PMI（或者 TM 9 中的 wideband CQI/wideband second PMI）上报之间，剩余的可用于上报的 $J \cdot K$ 个子帧，将按顺序被用于 subband CQI 的上报，这些 subband CQI 是将所有的 bandwidth part（共 J 个）轮询 K 遍得到的，即在一个周期（$H \cdot N_{pd}$）中，每个 bandwidth part 对应的 subband CQI 都会被上报 K 次。系统帧号（SFN）由 1023 转变为 0 时，在 2 个连续的 wideband CQI/wideband PMI 之间可用于上报的子帧数可能小于 $J \cdot K$ 个，此时 UE 不会在下一次的 2 个连续的 wideband CQI/wideband PMI 之前，上报剩余的还未来得及上报的 subband CQI。Bandwidth part 的一次轮询是从 bandwidth part 0 开始，一次递增至 bandwidth part *J-1* 的。（注意：每次只会选择一个 bandwidth part 中的一个 subband 进行上报）
  - 如果最近一次传输的 PTI = 0
    - wideband first PMI 上报的周期为 $H' \cdot N_{pd}$，其上报的子帧必须满足：

$$(10 \times n_f + \lfloor n_s / 2 \rfloor - N_{OFFSET,CQI}) \bmod (H' \cdot N_{pd}) = 0$$

      其中 H' 是通过 *periodicityFactor-r10* 配置的。

- 在每 2 个连续的 wideband first PMI 上报之间，剩余的可用于上报的子帧将用于 wideband second PMI 和 wideband CQI 的上报。

注：从这里可以看出，周期性 CQI/PMI 上报的机会每 N_{pd} 个 TTI 都会出现一次，但每个上报机会里上报的内容可能有所不同。

- 如果配置了 RI 上报，则 RI 上报的周期是 wideband CQI/PMI 周期 $H \cdot N_{pd}$ 的 M_{RI} 倍（以子帧为计量单位）。并且上报的 RI 与上报的 wideband CQI/PMI 以及 subband CQI 使用相同的 PUCCH 循环移位资源。用于上报 RI 的子帧必须满足：

$$\left(10 \times n_f + \lfloor n_s / 2 \rfloor - N_{OFFSET,CQI} - N_{OFFSET,RI}\right) \bmod \left(H \cdot N_{pd} \cdot M_{RI}\right) = 0$$

其中 n_f 为系统帧号；n_s 为一个系统帧内的 slot 号，取值范围为 0~19；$\lfloor n_s / 2 \rfloor$ 的值对应子帧号。

我们以图 10-9 为例来介绍周期性 CSI 配置。

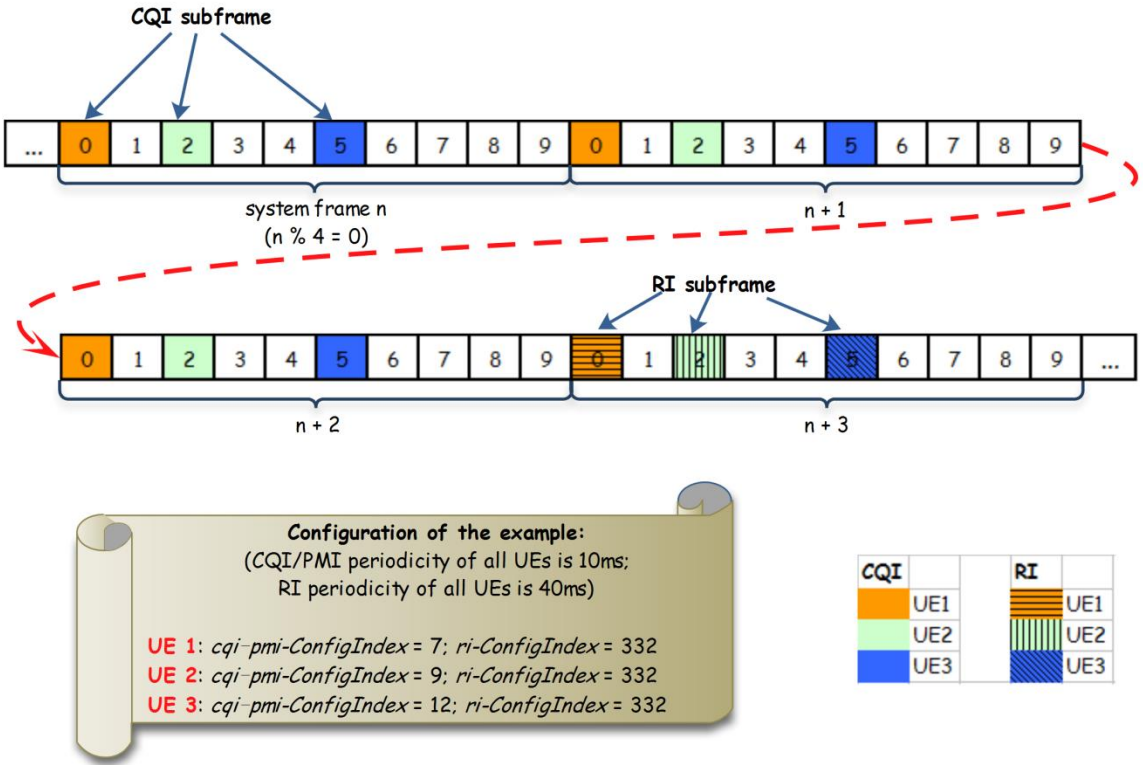


图 10-9：FDD 下，使用 wideband CQI/PMI 上报的例子

图 10-9 是 FDD 下，使用 wideband CQI/PMI 上报的一个例子。其中 *cqi-pmi-ConfigIndex* 的取值范围是 7~16，查 36.213 的 Table 7.2.2-1A 可知，UE 每隔 10 ms 需要发送一次 CQI/PMI；*ri-ConfigIndex* 的取值范围是 322~482，查 Table 7.2.2-1B 可知，UE 每隔 4 * 10 = 40 ms 需要发送一次 RI。

可以看出，当同一子帧需要同时传输 CQI/PMI 和 RI 时，会优先发送 RI。即 RI 上报的优先级高于 CQI/PMI 上报的优先级。

可以看出，*cqi-pmi-ConfigIndex* 和 *ri-ConfigIndex* 的值越小，则 CQI/PMI/RI 上报的周期越小，上报的频率也更高。这样 eNodeB 能够得到每个 UE 的更精确的下行信道状态信息，使得 eNodeB 在下行调度中能得到更精确的输入，进而能为下行传输选择更精确的 MCS 等，但相应地需要占用更多的 PUCCH 2 资源。

但对于某些场景而言，比如信道条件变化非常快的场景（如高铁），CSI 反馈（类似于“传输->反馈->再传输”的循环）太慢，此时更好的做法是平均化反馈信息（并将 HARQ 反馈考虑在内），同时使用较低的 MCS。

而在另一些场景下，例如要求相对低的吞吐量以满足 QoS 需求，如果此时使用快速的自适应会要求 CSI 反馈的周期变小，因此会要求更多的 PUCCH 2 资源用于反馈信息，这样做不是非常有效的。此时更好的做法是 CSI 反馈使用较大的周期，以便实现相对较好的开销和吞吐量。

【参考资料】

[1] TS 36.213 的 7.2.2 节 Periodic CSI Reporting using PUCCH

[2] http://www.sharetechnote.com/html/Handbook_LTE_CQI_PMI_RI_ReportingConfiguation.html （注意：这一章 “Wideband vs Subband”一节中关于 k 的介绍是错的）

[3] 《LTE Reporting: CQI reporting period and DL throughput》

10.5.3 PUCCH 2 的容量（capacity）计算

周期性 CSI 上报所使用的 PUCCH 2 资源 $n_{\mathrm{PUCCH}}^{(2,\tilde{p})}$ 是 UE 特定的，并通过 *cqi-PUCCH-ResourceIndex*（Rel-10 中使用 *cqi-PUCCH-ResourceIndex-r10* 或 *cqi-PUCCH-ResourceIndexP1-r10*）配置。无论是 wideband CQI/PMI 上报，还是同时配置了 wideband CQI/PMI 和 subband CQI 上报，UE 的 PUCCH 2 资源都是每隔 N_{pd} ms 出现一次的，只是上报的内容（CQI/PMI 或 RI）可能有所不同。也就是说，一个 UE 在一个周期内只会在其中的一个上行子帧上占用某一个 PUCCH 2 资源来发送周期性 CSI。

小区内的所有 UE 通常会配置相同的 CSI 上报周期。

从前面的介绍可以看出，与 CQI/PMI 上报相比，RI 上报的频率通常较低。这反映了合适的层数通常变化较慢，而影响到预编码矩阵和 MCS 选择的信道变化（channel variation）通常变化较快。

在小区 PUCCH 2 资源容量不变的情况下，*cqi-pmi-ConfigIndex* 和 *ri-ConfigIndex*（更确切地说，是周期）的值越小，其上报的频率越快，则要求每个子帧上需要分配更多的 RB 来传输 PUCCH format 2/2a/2b，但这样就减少了可用于 PUSCH 传输的 RB 数。

eNodeB 会通过 *nRB-CQI* 告诉 UE 用于周期性 CQI 上报（使用 PUCCH format 2/2a/2b）的 RB 数。

假设使用 FDD 的某小区，*nRB-CQI* 的值为 2，每个 RB 中可用的 PUCCH 2 资源数为 4（在介绍 PUCCH format 2/2a/2b 时，我们说过，一个 RB 内至多有 12 个 PUCCH 2 资源，但 12 个 PUCCH 2 资源并不一定都能够使用），其 CQI/PMI 上报的周期为 10 ms（对应 10 个 $I_{CQI/PMI}$，一个周期内有 10 个上行子帧可用于发送 PUCCH），这 10 ms 内有 10 个上行子帧，则该小区可分配用于周期性 CQI 上报的 PUCCH 2 资源的 UE 总数为：2 (RB) * 4 (UE/RB) * 10(subframe) = 80。也就是说，该小区可同时给至多 80 个 UE 分配 PUCCH 2 资源。

但在小区 PUCCH 2 资源容量不变（同样为 80 UE），每个 RB 中可用的 PUCCH 2 资源数依然为 4 的情况下，将 CQI/PMI 上报的周期变为 20 ms（对应 20 个 $I_{CQI/PMI}$），则 *nRB-CQI* 的值设为 1 就足够了。

如果是 TDD，一个周期内可用的上行子帧数变少，因此在小区容量不变且配置不变的情况下，每个子帧需要预留更多的 PUCCH 2 资源用于周期性 CSI 上报。

在这个例子中，我们既未考虑 SORTD，也未考虑混合 PUCCH 的情况。如果还存在混合 PUCCH，需要将混合 PUCCH 中的 PUCCH 2 资源数考虑在内。

注意：虽然在 8.4 节关于“PUCCH format 2/2a/2b”的介绍中，我们说过每个 RB 至多包含 12 个 PUCCH 2 资源，但在实际应用中，未必 12 个资源都会被使用，因为需要将 UE 间的干扰、eNodeB 的解码能力等因素考虑在内。例如：选择同一 RB 内的 4 个 PUCCH 2 资源分配给 4 个 UE， UE 间的 PUCCH 2 资源的循环移位相差为 3，可以降低干扰，并提高解码的效率和可靠性。

基于前面的介绍，我们来计算小区在一个上行子帧需要预留多少个 PUCCH 2 资源给周期性 CSI 使用。假设小区的 PUCCH 2 资源容量，即支持的 CQI 用户数为 N_{CQI}^{user} ，每个系统帧（10 ms）内有 $N_{UL}^{subfraem}$ 个上行子帧可用于发送 PUCCH，CSI 的上报周期为 N_{pd} ，则每个上行子帧需要预留的 PUCCH 2 资源数为 $N_{PUCCH}^{(2)} = \left\lceil N_{CQI}^{user} / (N_{pd} / 10 \ * \ N_{UL}^{subframe}) \right\rceil$。

如果每个 RB 可复用 N_{RB}^{PUCCH2} 个 PUCCH 2 资源，则每个上行子帧需要预留 $N_{RB}^{(2)} = \left\lceil (N_{PUCCH}^{(2)} - \left\lceil \frac{N_{\text{cs}}^{(1)}}{8} \right\rceil \cdot (N_{\text{sc}}^{\text{RB}} - N_{\text{cs}}^{(1)} - 2)) / N_{RB}^{PUCCH\ 2} \right\rceil$ 个 RB pair 用于发送 PUCCH format 2/2a/2b。其中 $\left\lceil \frac{N_{\text{cs}}^{(1)}}{8} \right\rceil \cdot (N_{\text{sc}}^{\text{RB}} - N_{\text{cs}}^{(1)} - 2)$ 为混合 PUCCH 中包含的 PUCCH 2 资源数（实际使用的可能是其子集，此时将该值替换成这个子集中的元素个数即可），具体见 8.5 节的介绍。$N_{RB}^{(2)}$ 即为 *PUCCH-ConfigCommon* 的 *nRB-CQI* 字段所要设置的值。

需要注意的是，前面介绍的计算公式的前提是 UE 不支持 SORTD。如果 UE 支持 SORTD，分配给某个 UE 的 PUCCH 2 资源数需要翻倍（由 1 变成 2）。

根据之前的介绍，可以通过以下途径来提高小区的 PUCCH 2 资源容量，即提高小区内可进行周期性 CSI 上报的 UE 的总数（需要在这些因素之间进行权衡）：

- 提高 *nRB-CQI*，即提高每个上行子帧用于发送 PUCCH format 2/2a/2b 的 RB 数（但这会降低 PUSCH 容量，即降低可用于 PUSCH 传输的 RB 数）；
- 提高 CQI 上报的周期（即提高 *cqi-pmi-ConfigIndex* 的值，例如将周期由 10 ms 提高到 20 ms），但这可能会影响 CQI/PMI 上报的精度，进而影响到上行的吞吐量（throughput）；
- 提升每个 RB 中可复用的 UE 数，例如：选择同一 RB 内的 6 个 PUCCH 2 资源分配给 6 个 UE， UE 间的 PUCCH 2 资源的循环移位相差为 2。但这要求 eNodeB 有更强的解码能力。

【参考资料】

[1] 《CQI-PMI-ConfigIndex, RI-Configindex and Their Impact on Throughput》

[2] 《Format 2: PUCCH Capacity Calculations》

10.5.4 周期性 CSI 上报的几种模式

周期性 CSI 上报中包含哪些内容与 UE 配置的 CSI 上报模式相关。LTE 支持 2 种周期性 CSI 上报模式，如 36.213 的 Table 7.2.2-1 所示。

Table 7.2.2-1: CQI and PMI Feedback Types for PUCCH CSI reporting Modes

		PMI Feedback Type	
		No PMI	**Single PMI**
PUCCH CQI Feedback Type	**Wideband (wideband CQI)**	Mode 1-0	Mode 1-1
	UE Selected (subband CQI)	Mode 2-0	Mode 2-1

如果 UE 配置了带 8 个 CSI-RS port 的 TM 9，则可以通过 *csi-ReportMode-r10*（即 *PUCCH_format1-1_CSI_reporting_mode*）对 Mode 1-1 进一步配置子模式 1（submode 1）或子模式 2（submode 2）。（具体见 10.5.5 节介绍）

注意：由于 PUCCH 负载有限，周期性 CSI 不支持在上报的子帧上上报多个 PMI（联合编码的除外）。

UE 使用的周期性 CSI 上报模式是通过 *cqi-FormatIndicatorPeriodic* 来配置的。不同的 TM 模式，支持不同的周期性 CSI 上报模式。也就是说，TM 模式的配置限制了 *cqi-FormatIndicatorPeriodic* 的取值范围。

```
cqi-FormatIndicatorPeriodic          CHOICE {
    widebandCQI                          NULL,
    subbandCQI                           SEQUENCE {
        k                                    INTEGER (1..4)
    }
```

TM模式限制了周期性CSI上报可选的上报模式

周期性CSI上报模式与TM模式的对应关系				
TM模式	Reporting Mode			
	Wideband CQI		UE Selected CQI	
	1-0 (No PMI)	1-1 (Single PMI)	2-0 (No PMI)	2-1 (Single PMI)
TM 1	Yes		Yes	
TM 2	Yes		Yes	
TM 3	Yes		Yes	
TM 4		Yes		Yes
TM 5		Yes		Yes
TM 6		Yes		Yes
TM 7	Yes		Yes	
TM 8	**Yes** (without PMI/RI reporting)	**Yes** (with PMI/RI reporting)	**Yes** (without PMI/RI reporting)	**Yes** (with PMI/RI reporting)
TM 9	**Yes** (without PMI/RI reporting，或CSI-RS天线端口数等于1)	**Yes** (with PMI/RI reporting且CSI-RS天线端口数大于1)	**Yes** (without PMI/RI reporting，或CSI-RS天线端口数等于1)	**Yes** (with PMI/RI reporting且CSI-RS天线端口数大于1)

图 10-10：TM 模式与周期性 CSI 上报的对应关系

注：（1）TM 1/2/7 无需上报 PMI/RI。TM 3 是开环的空分复用，其预编码矩阵使用固定的计算方式，所以也无需上报 PMI；

（2）对于 TM8 而言，如果 UE 配置了 PMI/RI 上报，则支持 CSI 上报模式 1-1、2-1；如果 UE 没有配置 PMI/RI 上报，则支持 CSI 上报模式 1-0、2-0；

（3）对于 TM9 而言，如果 UE 配置了 PMI/RI 上报且 CSI-RS 天线端口数大于 1，则支持 CSI 上报模式 1-1、2-1；如果 UE 没有配置 PMI/RI 上报或 CSI-RS 天线端口数 等于 1，则支持 CSI 上报模式 1-0、2-0；

（4）下行系统带宽 $N_{\mathrm{RB}}^{\mathrm{DL}} \le 7$ 的小区不支持周期性上报模式 Mode 2-0 和 Mode 2-1。也就是说，此时只支持 wideband CQI 上报；

（5）从图 10-10 可以看出，只需要使用 *cqi-FormatIndicatorPeriodic* 指定是 wideband 上报还是 subband 上报，就能够确定使用哪种周期性 CSI 上报模式。这是因为从 36.213 的 7.2.2 节中关于 TM 模式与周期性 CSI 上报模式的对应关系中可以看出，wideband CQI 上报和 subband CQI 上报分别只对应一种周期性 CSI 上报模式。也就是说，TM 模式（TM 8/9 还与是否进行 PMI/RI 上报相关）限定了可选的周期性 CSI 上报模式范围（只有 2 个：一个为 wideband 的，另一个为 subband 的），接着使用 *cqi-FormatIndicatorPeriodic* 就能确定使用的是 wideband 的那个，还是 subband 的那个。

接下来，我们将详细介绍周期性 CSI 上报的各种上报模式。

（1）**Wideband feedback**：UE 会上报一个反映整个下行系统带宽的平均信道质量的 CQI。如果使能了 PMI 上报，则上报一个反映整个下行系统带宽的 PMI。

- **Mode 1-0**：（1 表示上报的是 Wideband 的 CQI，0 表示不上报 PMI 值）
  - 在上报 RI 的子帧（只用于 TM 3）
    - UE 会上报一个包含一个 RI 的 type 3 报告；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-3，以确定其具体的内容）
  - 在上报 CQI 的子帧
    - UE 会上报一个包含一个 wideband CQI 值的 type 4 报告。该 wideband CQI 表示的是第一个 codeword 的信道质量，而不管 RI 是否大于 1；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-1）
    - 对于 TM 3，CQI 的计算是基于最后一个上报的周期性 RI 的；而对于其它传输模式，CQI 是基于 rank = 1 来计算的。
- **Mode 1-1**：（1 表示上报的是 Wideband 的 CQI，1 表示只反馈 1 个 PMI 值）
  - 在上报 RI 的子帧（只用于 TM 4/8/9）
    - UE 会上报一个包含一个 RI 的 type 3 报告；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-3）
  - 在上报 RI 和一个第一 PMI 的子帧（只用于配置了 8 CSI-RS 端口的 submode 1 的 TM 9）
    - UE 会上报一个包含联合编码的 RI 和 wideband first PMI 的 type 5 报告，该 PMI 选自对应的码本子集；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-3A）
    - wideband first PMI 是基于上报的周期性 RI 来计算的。

- 在上报 CQI/PMI 的子帧（只用于除“8 CSI-RS 端口的 TM 9 配置”外的其它传输模式配置）
  - UE 会从对应的码本子集中选择一个反映整个下行系统带宽的预编码矩阵；
  - UE 会上报一个 type 2 的报告，该报告包含如下信息：（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-2）
    - 一个反映整个下行系统带宽信道质量的 wideband CQI；
    - 所选择的一个 PMI（wideband PMI）；
    - 如果 RI > 1，还会上报一个额外的 3-bit wideband spatial differential CQI，以反映 codeword 1 的宽带信道质量信息。其中 Codeword 1 offset level = wideband CQI index for codeword 0 – wideband CQI index for codeword 1。
    - 3-bit wideband spatial differential CQI value 与 offset level 的对应关系见 36.213 的 Table 7.2-2；
  - 对于 TM 4/8/9（支持空分复用），上报的 CQI/PMI 是基于上报的 RI 所指示的 rank 来计算的；对于其它的 TM 模式，上报的 CQI/PMI 是基于 rank = 1 来计算的；
- 在上报 wideband CQI/second PMI 的子帧（只用于配置了 8 CSI-RS 端口的 submode 1 的 TM 9）
  - UE 会从对应的码本子集中选择一个反映整个下行系统带宽的预编码矩阵；
  - UE 会上报一个 type 2b 的报告，该报告包含如下信息：（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-2A）
    - 一个反映整个下行系统带宽信道质量的 wideband CQI；
    - 所选择的一个 PMI（wideband second PMI）；
    - 如果 RI > 1，还会上报一个额外的 3-bit wideband spatial differential CQI，以反映 codeword 1 的宽带信道质量信息。
  - 上报的 wideband second PMI 是基于最近上报的周期性 RI 和 wideband first PMI 来计算的；而上报的 wideband CQI 是基于最近上报的周期性 RI 和所选择的预编码矩阵来计算的；
- 在上报 wideband CQI/first PMI/second PMI 的子帧（只用于配置了 8 CSI-RS 端口的 submode 2 的 TM 9）
  - UE 会从对应的码本子集中选择一个反映整个下行系统带宽的预编码矩阵；

- UE 会上报一个 type 2c 的报告，该报告包含如下信息：（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-2B）
  - 一个反映整个下行系统带宽信道质量的 wideband CQI；
  - Wideband first PMI 和 Wideband second PMI，对应于所选择的预编码矩阵，见 36.213 的 7.2.4 节；
  - 如果 RI > 1，还会上报一个额外的 3-bit wideband spatial differential CQI，以反映 codeword 1 的宽带信道质量信息。
- 上报的 wideband first PMI、wideband second PMI 和 wideband CQI 是基于最近上报的周期性 RI 来计算的；

（2）**UE-selected reports**：与非周期性 CSI 上报不同的是，在周期性 CSI 上报中，整个下行系统带宽被划分成 1 到 4 个 bandwidth part（简称 BP），每个 BP 由多个 subband 组成。对于每个 BP，UE 会选择该 BP 内最好的 subband 予以上报。subband 的大小的取值范围为 4~8 个 RB。由于 PUCCH 支持的载荷是有限的，所以在周期内会轮询所有的 BP，并在一个子帧只会上报一个 BP 的 wideband CQI/PMI（使能的话），以及该 BP 上最好的 subband 和该 subband 的 CQI。如果使能 RI 上报的话，RI 只能在其它子帧上报。

对于 UE-selected subband CQI 上报，某个 serving cell 在某个子帧的 CQI 上报对应的是整个下行系统带宽的某 1 个或某几个特定部分的信道质量，这些部分被称为 bandwidth part（简称为 BP）。这些 subband 从最低的频率开始，按照频率递增的顺序进行索引。

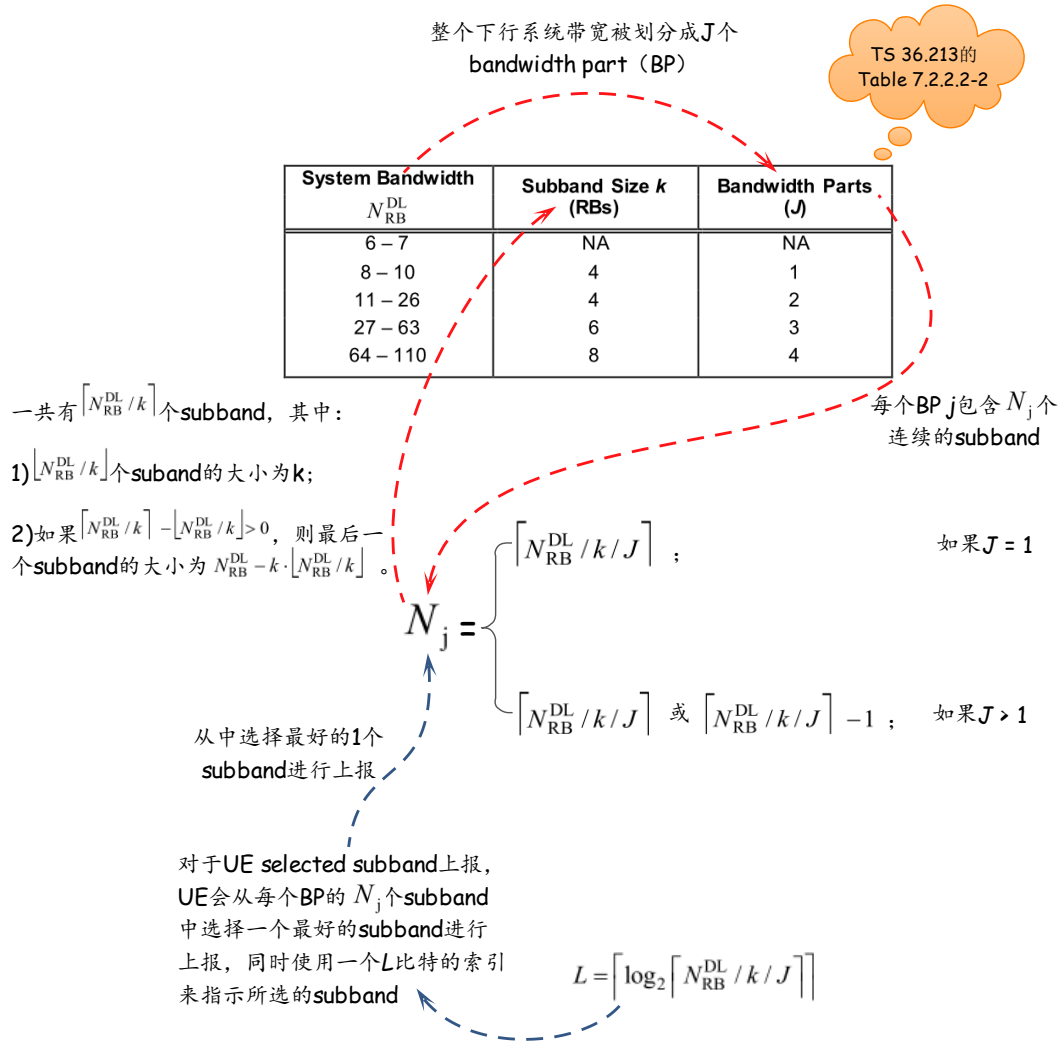


图 10-11：UE-selected subband CQI 上报中的 BJ 划分

注：从图 10-11 可以看出，对于下行系统带宽为 6 RB 的小区而言，只支持 wideband CQI 上报。

J 表示 bandwidth part 的个数，由于会从每个 bandwidth part 选择一个 subband 进行上报，所以 J 也指定了需要上报的 subband CQI 的个数。

以 N_{RB}^{DL} = 100 RB 为例，从 36.213 的 Table 7.2.2-2 可以看出，J = 4， k = 8：

（1）整个下行系统带宽分成 J = 4 个 BP（对应 j = 0~3）；

（2）一共有$\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/k \right\rceil = \lceil 100/8 \rceil = 13$ 个 subband。其中前 12 个 subband 的大小为 8 个 RB，最后一个 subband 的大小为 4 个 RB；

（3）每个 BP 包含$\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/k/J \right\rceil = \lceil 100/8/4 \rceil = 4$ 或 $\left\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/k/J \right\rceil - 1 = \lceil 100/8/4 \rceil - 1 = 3$ 个连续的 subband（BP j = 0 包含 N_j =4 个 subband；BP j = 1~3 包含 N_j = 3 个 subband）；

（4）UE 使用 $L = \left\lceil \log_2 \left\lceil N_{\mathrm{RB}}^{\mathrm{DL}}/k/J \right\rceil \right\rceil$ = 2 个比特的索引来指示从某个 BP 中选择上报的 subband。

- **Mode 2-0**：（2 表示上报的是 UE-selected CQI，0 表示不上报 PMI 值）
  - 在上报 RI 的子帧（只用于 TM 3）
    - UE 会上报一个包含一个 RI 的 type 3 报告；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-4）
  - 在上报 wideband CQI 的子帧
    - UE 会上报一个包含一个 wideband CQI 值的 type 4 报告。该 wideband CQI 表示的是第一个 codeword（codeword 0）的信道质量，而不管 RI 是否大于 1；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.1-1）
    - 对于 TM 3，CQI 的计算基于最后一个上报的周期性 RI 的；而对于其它传输模式，CQI 是基于 rank = 1 来计算的。
  - 在上报 selected subband CQI 的子帧
    - UE 会从每个 BP 的 N_j 个 subband 中选择一个最好的 subband；
    - UE 会上报一个“包含一个 CQI 值和对应的 L 比特的 subband 索引的 type 1 报告”，该 CQI 反映了在前面选出来的 subband 的信道质量。每个 BP 的 type 1 报告会在后续的上报机会中轮流上报。该 subband CQI 表示的是第一个 codeword 的信道质量，而不管 RI 是否大于 1；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-1）
    - 对于 TM 3，所选择的 subband 和对应的 CQI 的计算是基于最后一个上报的周期性 RI 的；而对于其它传输模式，这些值是基于 rank = 1 来计算的。
- **Mode 2-1**：（2 表示上报的是 UE-selected CQI，1 表示只反馈 1 个 PMI 值）

- 在上报 RI 的子帧（只用于 TM 4/8 以及配置的 CSI-RS 端口数为 2 或 4 的 TM 9）
  - UE 会上报一个包含一个 RI 的 type 3 报告；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-4 和 Table 5.2.3.3.2-4A）
- 在上报 RI 的子帧（只用于配置的 CSI-RS 端口数为 8 的 TM 9）
  - UE 会上报一个包含一个 RI 和 PTI 的 type 6 报告（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-4A）
- 在上报 wideband CQI/PMI 的子帧（只用于除“8 CSI-RS 端口的 TM 9 配置”外的其它传输模式配置）
  - UE 会从对应的码本子集中选择一个反映整个下行系统带宽的预编码矩阵；
  - 在接下来的每个上报机会中，UE 会上报一个 type 2 报告，该报告包含如下信息：（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-3 和 Table 5.2.3.3.2-3A）
    - 一个反映整个下行系统带宽信道质量的 wideband CQI；
    - 所选择的一个 PMI（wideband PMI）；
    - 如果 RI > 1，还会上报一个额外的 3-bit wideband spatial differential CQI，以反映 codeword 1 的宽带信道质量信息。
  - 对于 TM 4/8/9（支持空分复用），上报的 CQI/PMI 是基于最近一个上报的周期性 RI 所指示的 rank 来计算的；对于其它的 TM 模式，上报的 CQI/PMI 是基于 rank = 1 来计算的。
- 在上报 wideband first PMI 的子帧（只用于 “8 CSI-RS 端口的 TM 9 配置”且最近上报的 PTI = 0）
  - UE 会从对应的码本子集中选择一组对应 wideband first PMI 的预编码矩阵；
  - 在接下来的每个上报机会中，UE 会上报一个包含 wideband first PMI 的 type 2a 报告，该 PMI 对应前面选择的预编码矩阵集合；（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-3B 中 PTI=0 的项）
  - Wideband first PMI 是基于最近一个上报的 RI 所指示的 rank 来计算的。
- 在上报 wideband CQI/second PMI 的子帧（只用于 “8 CSI-RS 端口的 TM 9 配置”且最近上报的 PTI = 1）
  - UE 会从对应的码本子集中选择一个反映整个下行系统带宽的预编码矩阵；

- 在接下来的每个上报机会中，UE 会上报一个 type 2b 的报告，该报告包含如下信息：（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-3B 中 PTI=1 的项）
    - 一个反映整个下行系统带宽信道质量的 wideband CQI；
    - 所选择的一个 PMI（wideband second PMI）；
    - 如果 RI > 1，还会上报一个额外的 3-bit wideband spatial differential CQI，以反映 codeword 1 的宽带信道质量信息。
- 上报的 wideband second PMI 是基于最近上报的周期性 RI 和 wideband first PMI 来计算的；而上报的 wideband CQI 是基于最近上报的周期性 RI 和所选择的预编码矩阵来计算的。

- 在上报 selected subband CQI 的子帧（只用于除“8 CSI-RS 端口的 TM 9 配置”外的其它传输模式配置）
    - UE 会从每个 BP 的 N_j 个 subband 中选择最好的一个 subband；
    - UE 会在接下来的每个上报机会上报一个“type 1 报告”，该报告包括（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-2 和 Table 5.2.3.3.2-2A）
        - 一个反映在前面选出来的 subband 的信道质量且对应 codeword 0 的 CQI 值，以及对应的 L 比特的 subband 索引；
        - 如果 RI > 1，还会上报一个额外的用于 codeword 1 的 3-bit subband spatial differential CQI，用于反映 codeword 1 的窄带信道质量信息。其中 Codeword 1 offset level = subband CQI index for codeword 0 – subband CQI index for codeword 1；
        - 3-bit subband spatial differential CQI value 与 offset level 的对应关系见 36.213 的 Table 7.2-2；
    - 对于 TM 4/8/9（支持空分复用），上报的 subband 的选择以及 CQI 值是基于最近一次上报的周期性 wideband PMI 和 RI 来计算的；对于其它的 TM 模式，这些值是基于最近一次上报的周期性 wideband PMI 和 rank = 1 来计算的。

- 在上报 wideband CQI/second PMI 的子帧（只用于 “8 CSI-RS 端口的 TM 9 配置”且最近上报的 PTI = 0）
    - UE 会从对应的码本子集中选择一个反映整个下行系统带宽的预编码矩阵；
    - UE 会上报一个 type 2b 的报告，该报告包含如下信息：（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-2B 中 PTI=0 的项）

- 一个反映整个下行系统带宽信道质量的 wideband CQI，该 CQI 假定使用前面选择的预编码矩阵；
- 所选择的一个 PMI（wideband second PMI）；
- 如果 RI > 1，还会上报一个额外的 3-bit wideband spatial differential CQI。

▪ 上报的 wideband second PMI 是基于最近上报的周期性 RI 和 wideband first PMI 来计算的；而上报的 wideband CQI 是基于最近上报的周期性 RI 和所选择的预编码矩阵来计算的。

○ 在上报 wideband CQI/second PMI 的子帧（只用于 “8 CSI-RS 端口的 TM 9 配置”且最近上报的 PTI = 1）

▪ UE 会从每个 BP 的 N_j 个 subband 中选择最好的一个 subband；

▪ 在接下来的每个上报机会中，UE 会为每个 BP 上报一个 “type 1a 报告”，该报告包括（可参见 10.5.1 节，也可参见 36.212 的 Table 5.2.3.3.2-2B 中 PTI=1 的项）

- 一个反映在前面选出来的 subband 的信道质量且对应 codeword 0 的 CQI 值，以及对应的 L 比特的 subband 索引；
- 如果 RI > 1，还会上报一个额外的用于 codeword 1 的 3-bit subband spatial differential CQI。其中 Codeword 1 offset level = subband CQI index for codeword 0 – subband CQI index for codeword 1；
  - 3-bit subband spatial differential CQI value 与 offset level 的对应关系见 36.213 的 Table 7.2-2；
  - 该 CQI 假定使用对应所选的 second PMI 和最近上报的 first PMI 的预编码矩阵，以及在对应 BP 上所选的 subband 上传输。
- 一个从码本子集中选出的预编码矩阵的 second PMI，并假定数据在对应 BP 上所选的 subband 上传输；

▪ 上报的 subband second PMI 是基于最近上报的周期性 RI 和 wideband first PMI 来计算的；而选择的 subband 和 CQI 值是基于最近上报的周期性 RI 和所选择的预编码矩阵来计算的。

一个典型的混合使用周期性和非周期性 CSI 上报的方式是：使用轻量级的周期性 CSI 上报，例如提供 wideband CQI 上报但不提供 PMI 上报（mode 1-0）；而当有发给某个特定的 UE 的下行数据到达时，eNodeB 才会要求该 UE 上报非周期性 CSI，例如：频率选择性的 CQI 和 PMI（mode 3-1）。

小结：我们可以按照如下步骤来确定 UE 在周期性的 CSI 资源上需要上报的内容：（1）结合 UE 配置的 TM 模式，以及对应的“*cqi-FormatIndicatorPeriodic*”，就可以确定对应的周期性 CSI 上报模式；（2）同时结合 10.5.2 节的介绍，就知道哪些子帧上需要上报 CQI/PMI，哪些子帧上需要上报 RI/PTI；（3）接着结合本节的介绍，就知道需要上报哪种类型的“type XX 报告”；（4）将本节介绍的各种的“type XX 报告”跟 10.5.1 中介绍的 Table 7.2.2-3 以及上面介绍的 36.212 的各个 Table 结合起来，就能够清楚地知道在各种场景下（与天线配置、上报的 RI 等相关）需要上报哪些信息，以及这些信息占多少比特。

10.5.5 “8 天线端口”的 TM9 的周期性 CSI 上报

结合前面的介绍，这里我们需要重点介绍一下 8 天线端口传输下的 TM 9 如何上报 CQI/PMI/RI 的。

在 6.5.9 节关于“TM9”的介绍中我们可以看到，在 TM 9 中，UE 通常通过 i_1 和 i_2 来反馈预编码矩阵的信息。但为了减少上行的开销，8 CSI-RS 端口下还支持以下周期性 CSI 上报子模式。

10.5.5.1 Wideband 周期性 CSI 上报

从 10.5.4 节可以看出，wideband 周期性 CSI 上报中，RI/PMI/CQI 上报通过 2 个不同的 PUCCH 上报类型来反馈：一个包括 RI，另一个包含 CQI/PMI 信息。对于 8 CSI-RS 端口的 TM 9 的 wideband 周期性 CSI 上报，且其 PUCCH 上报模式为 Mode 1-1 时，还有 2 种可选的子模式配置：submode 1 和 submode 2。

如果 UE 配置了带 8 个 CSI-RS port 的 TM 9，则 eNodeB 可以通过 *csi-ReportMode-r10*（即 *PUCCH_format1-1_CSI_reporting_mode*）对 Mode 1-1 进一步配置成子模式 1（submode 1）或子模式 2（submode 2）。

在 submode 1 中，wideband first PMI i_1 会包含在第一个上报中，并与 RI 联合编码后一起上报（type 5）；而 wideband CQI 和 wideband second PMI i_2 会一起传输（type 2b）。

在 submode 2 中，第一个上报中依然只包括 RI（type 3），而所有的 PMI 信息（包括 i_1 和 i_2）会包含在第二个上报中，并与 CQI 一起上报（type 2c）。但由于 PUCCH 的负载有限，需要对原码本进行筛选，取其子集（subsampling），以便第二个上报能够匹配 PUCCH 可容纳的比特数。

submode 2 最小化了 RI 的码率（因为 RI 单独传输），使得 RI 的传输更为可靠，而 submode 1 避免了对 PMI 码本进行重新筛选（subsampling）。如何取舍取决于 eNodeB 的实现。

10.5.5.2 Subband 周期性 CSI 上报

i_1 对应的 PMI 反馈具有相对的时间不变性，即 i_1 通常在相对较长的时间内保持不变。因此，可以通过降低 i_1 的上报频率，使得更多的资源用于 i_2 的上报。此时 CQI/ PMI/ RI 上报通过 3 个不同的 PUCCH 上报类型来反馈。

从 10.5.4 节可以看出，对于 8 CSI-RS 端口的 TM 9 的 subband 周期性 CSI 上报中，RI/PMI/CQI 上报通过 3 个不同的 PUCCH 上报类型来反馈：第一个包括 RI 和一个 1 比特的 PTI（type 6）。PTI 与 RI 一起上报，用于 UE 告诉 eNodeB 第二个和第三个上报的内容：

- 如果 PTI 的值为 0，则第二个上报包含 i_1（type 2a），第三个上报包含 wideband 的 i_2 和相关的 CQI（type 2b）。
- 而如果 UE 认为 i_1 从前一次上报后就没有变化，PTI 的值就会设为 1，此时第二个上报包含 wideband 的 i_2 和相关的 CQI（type 2b），第三个上报包含 subband 的 i_2 和相关的 CQI（type 1a），并且第三个上报将上报多次，每个 subband 上报一次。

可以看出，如果 i_1 的值是保持不变的，可以通过将 PTI 的值设置为 1，来提高 i_2 的上报频率。图 10-12 是使用 PTI 来进行 subband 周期性 CSI 上报的例子。

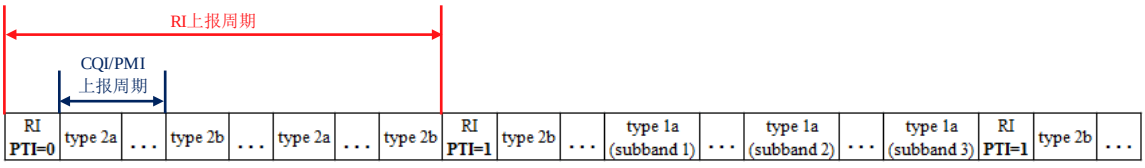


图 10-12：使用 PTI 来进行 subband 周期性 CSI 上报

从这里我们可以看出，只有在 8 天线端口传输（TM 9），并且只在周期性 CSI 上报中才会使用 PTI。

【参考资料】

[1] TS 36.212 的 5.2.3.3 节 Channel coding for UCI channel quality information

[2] TS 36.213 的 7.2 节 UE procedure for reporting Channel State Information (CSI)

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 29.3.3 节

10.6 小结

本章试着从以下 3 个方面来介绍 CSI 上报：

（1）CQI/PMI/RI 的定义：主要介绍了 CQI/PMI/RI 的含义和作用。

（2）UE 如何测量 CQI/PMI/RI：简单介绍了 UE 侧如何测量 CQI/PMI/RI 信息，如 CSI 参考资源等。

（3）UE 如何上报 CQI/PMI/RI：主要介绍了码本子集配置，周期性 CSI 上报和非周期性 CSI 上报等。重点关注了如何触发对应的 CSI 上报，各种上报模式下在对应的上报子帧上需要上报的内容，以及使用哪些时频资源进行上报等内容。

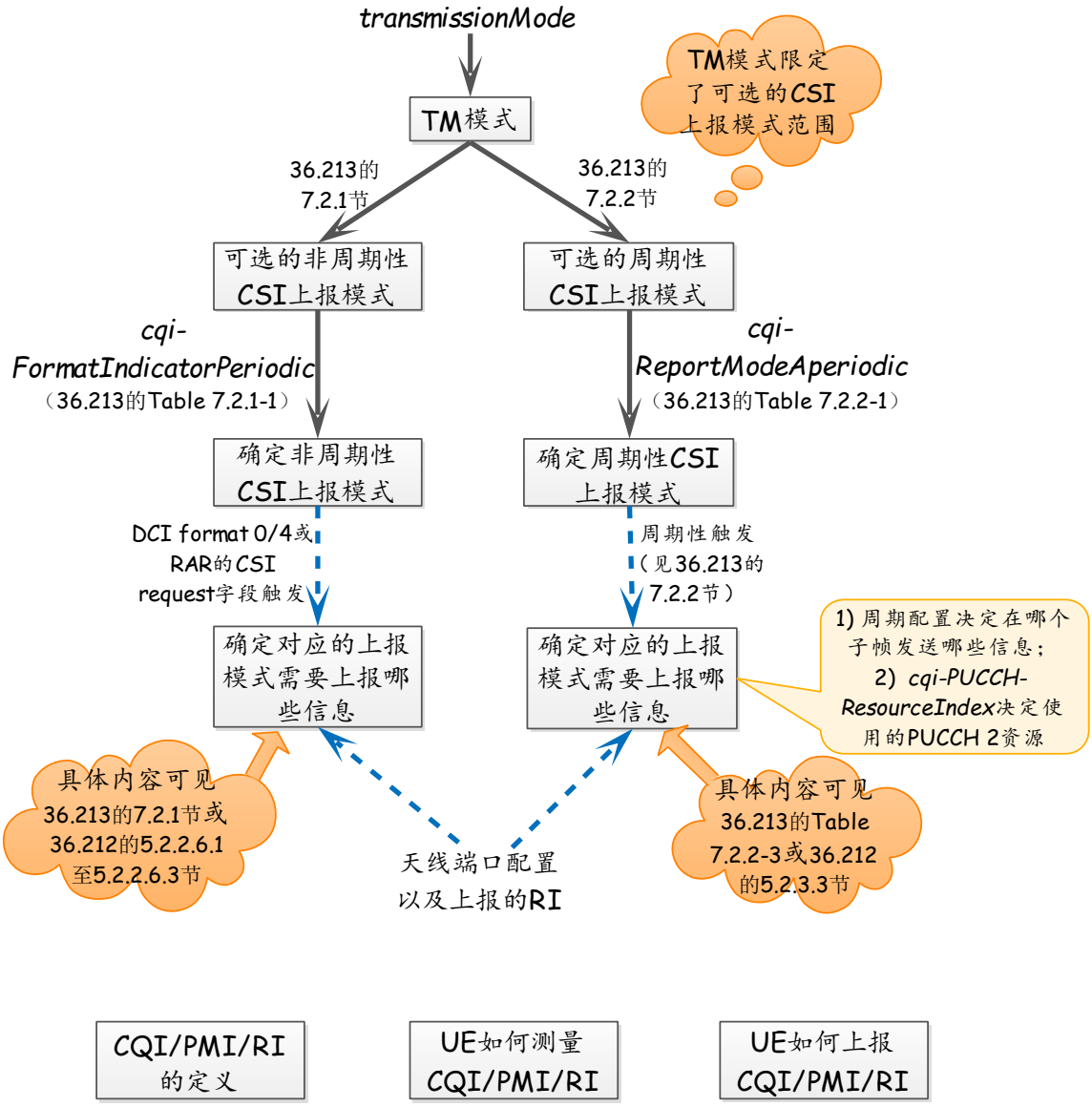


图 10-13：CSI 小结

从图 10-13 可以看出，TM 模式决定了可选的 CSI 上报模式的范围，而配置的 CSI 上报模式同 TM 模式、天线端口配置以及上报的 RI 共同决定了在上报的子帧上需要上报哪些信息。

第11章 HARQ 介绍

LTE 中有 2 级重传处理机制：MAC 层的 HARQ 机制，以及 RLC 层的 ARQ（只针对 AM 模式数据传输）机制。

丢失或出错的数据的重传主要是由 MAC 层的 HARQ 机制处理的，并由 RLC 的重传功能进行补充。

MAC 层的 HARQ 机制能够提供快速重传，RLC 层的 ARQ 机制能够提供可靠的数据传输。二者结合可以实现较小的往返时间、适度的反馈开销以及可靠传输的要求。

HARQ 机制的目标在于实现非常快速的重传，其反馈出错率大概在 1%左右。对于某些业务，如 TCP 传输（要求丢包率小于10^{-5}），HARQ 反馈的出错率显然过高了。对于这类业务， RLC 层的重传处理能进一步降低反馈出错率。与 HARQ 相比，RLC 状态报告并不会频繁传输，因此获得10^{-5}甚至更低丢包率的可靠性开销并不大。

第 11 章至第 13 章主要介绍 HARQ 处理，而 RLC 层的重传会在第 26 章予以介绍。

11.1 带软合并的 HARQ（HARQ with soft combining）

HARQ（Hybrid Automatic Repeat reQuest），混合式自动重传请求，是一种结合 FEC（Forward Error Correction）与 ARQ（Automatic Repeat reQuest）方法的技术。

FEC 通过添加冗余信息，使得接收端能够纠正一部分错误，从而减少重传的次数。

对于 FEC 无法纠正的错误，接收端会通过 **ARQ 机制**请求发送端重发数据。接收端使用检错码，通常为 CRC 校验，来检测接收到的数据包是否出错。如果无错，则接收端会发送一个肯定的确认（ACK）给发送端，发送端收到 ACK 后，会接着发送下一个数据包。如果出错，则接收端会丢弃该数据包，并发送一个否定的确认（NACK）给发送端，发送端收到 NACK 后，会重发相同的数据。

前面介绍的 ARQ 机制采用丢弃数据包并请求重传的方式。虽然这些数据包无法被正确解码，但其中还是包含了有用的信息，如果丢弃了，这些有用的信息就丢失了。通过使用 **HARQ with soft combining**（带软合并的 HARQ），接收到的错误数据包会保存在一个 HARQ buffer 中，并与后续接收到的重传数据包进行合并，从而得到一个比单独解码更可靠的数据包（“软合并”的过程）。然后对合并后的数据包进行解码，如果还是失败，则重复“请求重传，再进行软合并”的过程。

根据重传的比特信息与原始传输是否相同，HARQ with soft combining 分为 chase combining 和 incremental redundancy（增量冗余）两类。chase combining 中重传的比特信息与原始传输相同；incremental redundancy 中重传的比特信息不需要与原始传输相同。本书不会对 chase combing 做进一步介绍，这是因为 LTE 中只使用 incremental redundancy 机制。

在 incremental redundancy 中，每一次重传并不需要与初始传输相同。相反，会生成多个 coded 比特的集合，每个集合都携带相同的信息。当需要重传时，通常会传输与前一次不同的 coded bit 集合，接收端会把重传的数据与前一次传输的数据进行合并。每次重传的 coded bit 集合称为一个冗余版本（Redundancy Version，RV）。

图 11-1 是一个增量冗余的例子：

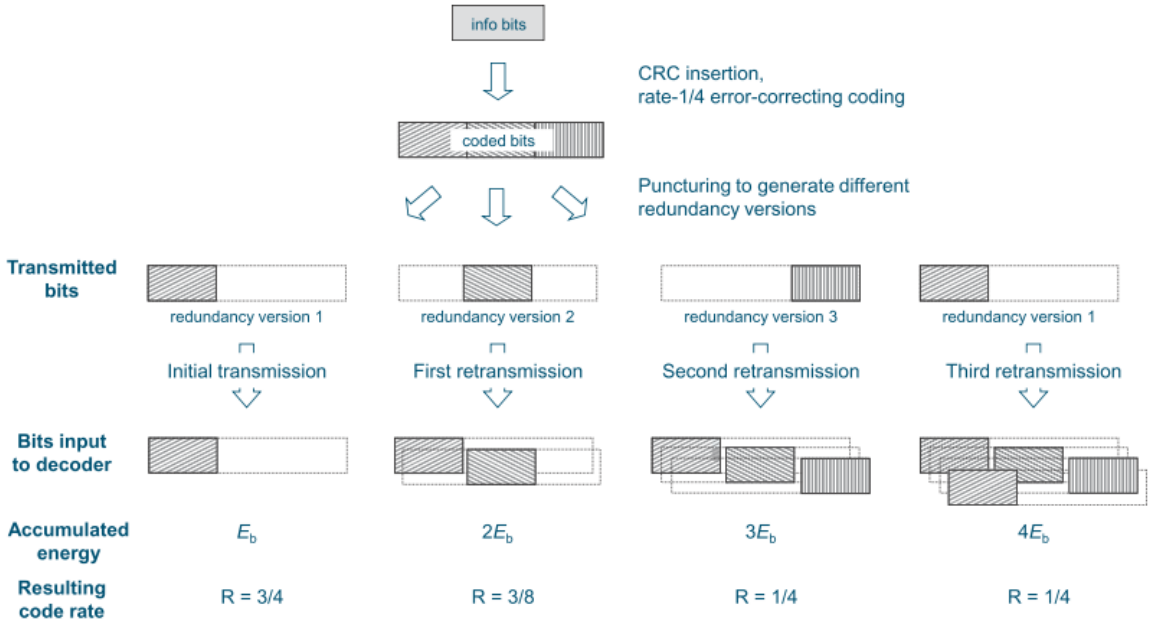


图 11-1：增量冗余的例子

由于重传可能携带了不包含在前次传输中的额外奇偶校验比特（parity bit），所以重传的码率会降低。每次重传可以包含与初始传输数目不同的 coded bit，且不同重传的调制方式也可以不同。

这里我们以 DL-SCH 的物理层处理步骤来简单介绍一下冗余版本 RV 的生成过程。UL-SCH 的处理与之类似。

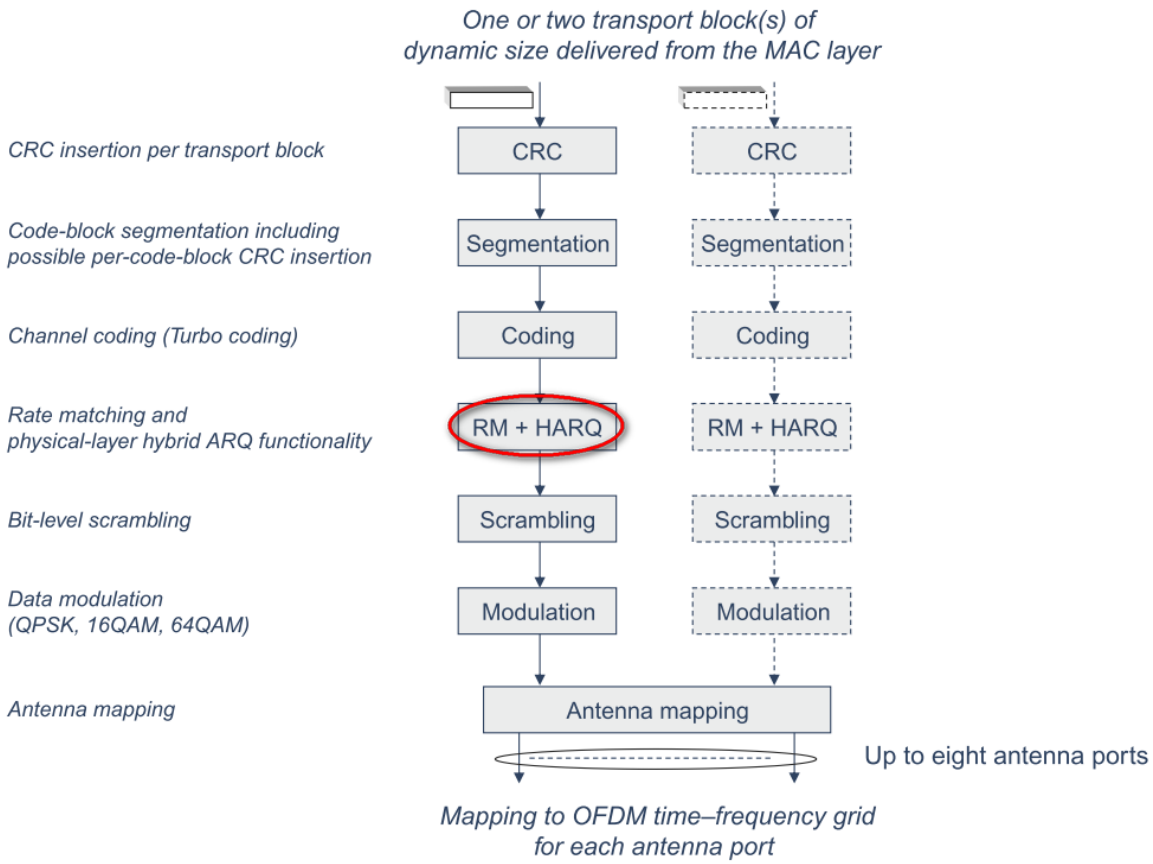


图 11-2：DL-SCH 的物理层处理

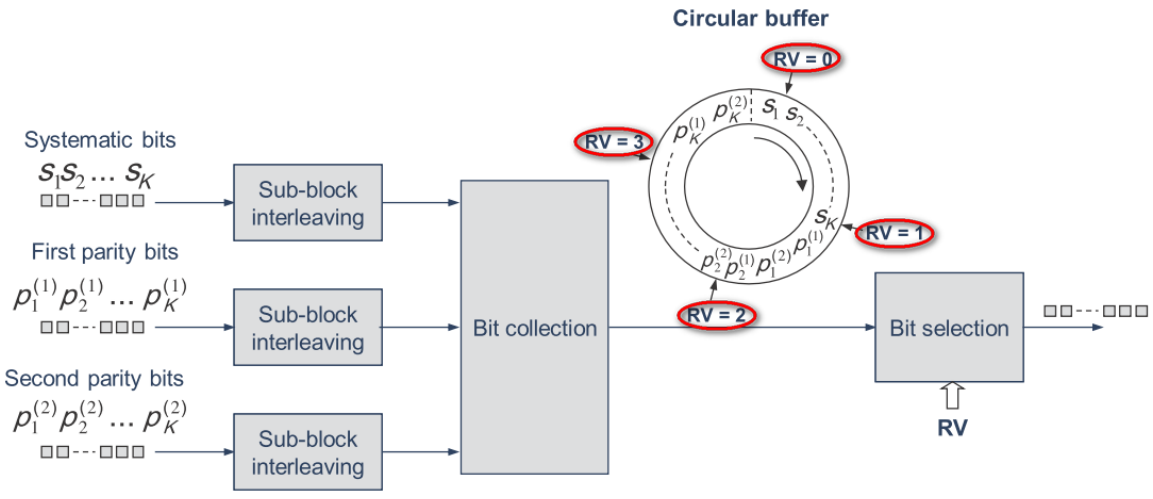


图 11-3：速率匹配和 HARQ 功能

从图 11-2、图 11-3 可以看出，在 Turbo 编码之后，会输出 systematic bits，first parity bits 和 second parity bits。这些输出会先进行交织，然后插入一个环形缓冲器（circular buffer）中。其中，systematic bits 会先插入，然后 first parity bits 和 second parity bits 交替着插入。

不同的 RV 意味着从环形缓冲器的不同起始位置来提取要发送的比特。

对于 DL-SCH 和 UL-SCH 而言，其信道编码采用了 Turbo 编码。Turbo 编码中的 systematic bit 比 parity bit 更重要，在初传(initial transmission)中至少需要包含所有的 systematic bit 和部分 parity bit；而在重传(retransmission)中，会包含初传中没有的 parity bit。如果初传（注意：这里是“第一次传输”）的接收质量很差或根本没收到，此时重传只带 parity bit 的 RV 不如重传包含全部（或部分）systematic bit 的 RV 来得性能好。这里有两种不同的否定应答：NACK 和 DTX。NACK 要求重传额外的 parity bit，而 DTX 要求重传 systematic bit（也就是说，针对 RV=0 的初传，如果 HARQ 反馈为 DTX，则重传的 RV 依然为 0）。总之，基于之前传输尝试的信号质量来决定重传中包含多少 systematic bit 和 parity bit 是很重要的。

HARQ 是通过校验 CRC 来判断接收到的数据包是否出错，并且校验 CRC 是在软合并之后进行的。如果 CRC 校验成功，则接收端会发送肯定的反馈（ACK）；如果 CRC 校验失败，则接收端会发送否定的反馈（NACK）。

HARQ 功能同时跨越物理层和 MAC 层。其中发送端生成不同的冗余版本（选用哪个 RV 是由 MAC 层告诉物理层的）以及接收端进行软合并、CRC 校验是由物理层负责的。在接收端，HARQ buffer 通常位于物理层中，这是因为物理层需要对接收到的数据进行软合并和解码处理。

HARQ protocol 是 MAC 层的功能，HARQ protocol 在发送端和接收端都存在。发送端的 HARQ 操作包括传输和重传 TB、接收并处理 ACK/NACK 等；接收端的 HARQ 操作包括接收 TB，生成 ACK/NACK 等。

只有 DL-SCH 和 UL-SCH 传输支持 HARQ，其它信道是不支持 HARQ 的。

需要说明的是，如果将 ACK 解码成 NACK，只会在 MAC 层重传一个已经正确解码的 TB。而 NACK 解码成 ACK，则 MAC 层实际并没有成功接收到 TB，但却发给了 RLC 层，这就要依赖 RLC 的重传来找回丢失了的 TB，因此会带来一定的延迟。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 6.3、6.4、10.1.1.4 节

[2] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 9.2.1 节

[3] 《Forward error correction》 by 维基百科

[4] 《混合式自动重送请求》by 维基百科

11.2 HARQ process

HARQ 使用 stop-and-wait protocol（停等协议）来发送数据。

在停等协议中，发送端发送一个 TB 后，就停下来等待确认信息。接收端会使用 1 比特的信息对该 TB 进行肯定（ACK）或否定（NACK）的确认。但是每次传输后发送端就停下来等待确认，会导致吞吐量很低。因此 LTE 使用多个并行的 stop-and-wait process：当一个 HARQ process 在等待确认信息时，发送端可以使用另一个 HARQ process 来继续发送数据。如图 11-4 所示。

这些 HARQ process 共同组成了一个 HARQ 实体（HARQ entity），这个实体结合了停等协议，同时允许数据的连续传输。

每个 UE 都有一个 HARQ 实体。但在载波聚合中，一个 UE 对应每个载波单元（Component Carrier）都有各自的 HARQ 实体。

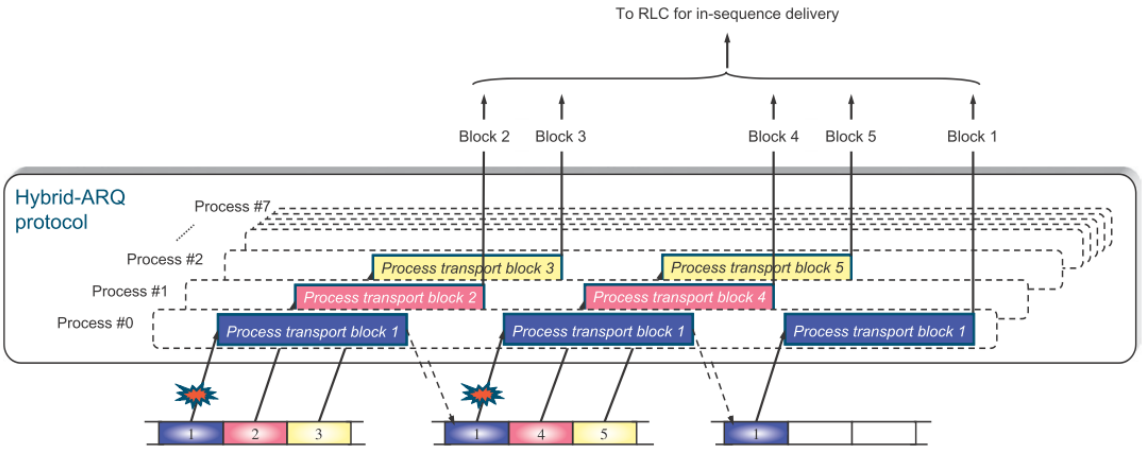


图 11-4：多个并行的 HARQ process

每个 HARQ process 在一个 TTI 只处理一个 TB（Transport Block，传输块）。

每个 HARQ process 在接收端都需要有独立的 HARQ buffer 以便对接收到的数据进行软合并。

在空分复用中，一个 TTI 会并行传输 2 个 TB，每个 TB 有各自独立的 HARQ 确认信息，并使用不同的 HARQ process 来处理。此时 1 个 HARQ 实体包含 2 个 HARQ process 集合。

使用多个并行的 stop-and-wait process 可能导致接收端的 MAC 层送往 RLC 层的数据是乱序的。如图 11-4 所示，传输块 5 在传输块 1 之前成功解码，导致传输块 5 先于传输块 1 送往 RLC 层，从而出现数据的乱序。因此，RLC 层需要对接收到的数据进行重排序。在载波聚合中，RLC 层同样需要负责数据的重排序，这是因为 RLC 层对载波聚合不可见，而每个载波单元有独立的 HARQ 实体，导致一个 RLC 层需要从多个 HARQ 实体中接收数据，而接收自多个 HARQ 实体的数据很可能是乱序的。

当发送端收到一个确认信息（ACK/NACK）后，需要知道该确认信息对应的 HARQ process，这是通过确认信息与传输的数据之间固定的 timing 关系来确定的。这对上行、下行都适用。注意：这里是针对某次传输与对应的 ACK/NACK 之间的 timing 关系的，而同步/异步是针对重传和前一次传输（初传或重传）之间的 timing 关系的。

对于 HARQ，只有在对应同一 TB，也即同一 HARQ process 的基础上讨论“初传”、“重传”才有意义。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 8.2.2.3 节

11.3 HARQ 相关概念

HARQ 有下行 HARQ 和上行 HARQ 之分：下行 HARQ 针对 DL-SCH 数据，上行 HARQ 针对 UL-SCH 数据。下行 HARQ 和上行 HARQ 是相互独立的，处理的方式也不相同。

本节会介绍一些与 HARQ 处理相关的概念，具体的处理方式会在第 12 章（上行 HARQ）和第 13 章（下行 HARQ）予以详细说明。

HARQ process number：也称为 HARQ process ID，唯一地指定一个 HARQ process。

NDI：New Data Indicator 的缩写。每个 HARQ process 会保存一个 NDI 值，该值使用 1 比特来指示被调度的数据是新传还是重传。如果同一 HARQ process 的 NDI 值与之前相比发生了变化（NDI toggled），则表示当前传输是一个新的 TB 的初传，否则（NDI not toggled）表示当前传输是同一个 TB 的重传。

有些 DCI 中包含了一个“New data indicator”字段，通常也称为 NDI，并与某个 HARQ process 相关联。但该字段用于推导对应 HARQ process 的 NDI。即“DCI 中携带的 NDI”与“HARQ process 的 NDI”的概念并不完全相同。（上行 NDI 的处理见 36.321 的 5.4 节；下行 NDI 的处理见 36.321 的 5.3 节）

RV：Redundancy Version 的缩写。用于指示当前传输所使用的冗余版本，其取值范围为 0~3。

MCS：Modulation and Coding Scheme 的缩写。DCI 使用 5 比特的 MCS index（0~31）来指示当前传输所使用的调制和编码样式，并影响到 TBS 的选择。MCS 共计 32 种组合，其中 3 种组合（索引为 29~31）是预留的，且这 3 种组合只用于重传。

使用这 3 种预留组合的前提是 UE 成功地接收了初传中的 DCI（使用 MCS index 0~28）。如果初传的 HARQ 反馈为 DTX，即接收端没有收到对应的 DCI，则发送端需要在接下来的一次重传中使用 DCI 显式地指明所使用的资源和 MCS，且此时不能使用 MCS index 29~31。

TBS：Transport Block Size 的缩写。重传的 TBS 通常是不变的，并与初传的 TBS 保持一致，否则无法进行软合并处理。

PRB 数和 PRB 所在的频域位置：虽然重传的 TBS 通常保持不变，但下行重传以及上行自适应重传的 PRB 数、PRB 所在的频域位置和 MCS 是可以改变的，这提供了调度的灵活性。

表 11-1：HARQ 信息

HARQ 信息	DCI 中的比特数	下行 HARQ	上行 HARQ
NDI: New Data Indicator	1 比特	对于每个新传的 TB，NDI 会翻转。当 UE 收到一个下行分配（下行 DCI）时，UE 会检查 NDI 以决定当前传输是“与同一 HARQ process 的 HARQ buffer 中保存的之前接收到的数据进行软合并（重传，NDI 没有翻转）”，还是“清空 HARQ buffer（新传，NDI 翻转）”。	当 UE 收到一个 UL grant（上行 DCI）时，UE 会检查 NDI 以决定该 UL grant 是用于重传（NDI 没有翻转），还是用于新传（NDI 翻转）。
PRB 资源和 TBS	PRB 资源是通过对应 DCI 的“Resource block assignment”字段分配的。但 TBS 还跟 MCS 相关。（见 5.3 节和 9.3 节的介绍）	重传使用的 PRB 资源和调制阶数是可以改变的。但 TBS 通常应该与初传保持一致。（从 36.213 的 Table 7.1.7.1-1 可以看出，MCS 索引 29~31 对应的 TBS index 是预留的）。但从 36.321 的 5.3.2.2 节可以看出，如果重传的 TBS 与该 TB 的前一次传输的 TBS 不一致，UE 会使用当前数据替换该 TB 在 HARQ buffer 中的内容，也就是说，此时无法进行软合并处理。	对于非自适应重传，使用的 PRB 资源和 TBS 与前一次传输相同。对于自适应重传，使用的 PRB 资源可以改变，但 TBS 和调制阶数应该与初传保持一致。（从 36.213 的 Table 8.6.1-1 可以看出，MCS 索引 29~31 对应的 TBS index 和 Modulation Order 都是预留的）
HARQ process number	FDD 下使用 3 比特；TDD 下使用 4 比特。（只存在于下行 DCI 中，且 DCI format 1C 中不存在该字段）	下行 HARQ 是异步的，eNodeB 需要通知 UE 使用哪个 HARQ process 来处理接收到的数据。	上行 DCI 中并不存在该字段。上行 HARQ 是同步的，其 HARQ process number 可以根据 timing 关系推导出。

RV: Redundancy Version	2 比特	指示当前传输使用的冗余版本。	指示当前传输使用的冗余版本。对于非自适应重传，RV 的选择顺序为“0, 2, 3, 1”。对于自适应重传，RV 是根据 DCI format 0/4 中的“Modulation and coding scheme and redundancy version”字段得到的。（MCS 索引 29~31 预留给 RV 使用）
MCS: Modulation and Coding Scheme	5 比特。指定了调制阶数，码率和 TBS 等信息。	见 5.3 节的介绍	见 9.3 节的介绍

【参考资料】

[1] TS 36.213 的 7.1.7 节 Modulation order and transport block size determination

[2] TS 36.213 的 8.6 节 Modulation order, redundancy version and transport block size determination

[3] TS 36.321 的 5.3 节 DL-SCH data transfer

[4] TS 36.321 的 5.4 节 UL-SCH data transfer

[5] 《4G LTE/LTE-Advanced for Mobile Broadband》的 12.1 节

11.4 同步/异步、自适应/非自适应

HARQ 协议在时域上分为同步（synchronous）和异步（asynchronous）两类；在频域上分为自适应（adaptive）和非自适应（non-adaptive）两类。

注意：同步/异步、自适应/非自适应是针对同一 TB（使用同一 HARQ process）的初传和重传而言的，而不是针对某次传输与 ACK/NACK 之间的关系的，虽然二者之间存在紧密的联系。

异步 HARQ（asynchronous HARQ）意味着重传可以发生在任一时刻，也意味着能以任意顺序使用 HARQ process。**同步 HARQ**（synchronous HARQ）意味着重传只能在前一次传输之后的固定时刻发送，也意味着在某个特定的子帧，只能使用某个特定的 HARQ process。

同步 HARQ 的好处在于 HARQ process 号可以直接从系统帧号/子帧号中推导出来，而无需显式地发送 HARQ process number；异步 HARQ 的好处在于重传调度更加灵活。

自适应 HARQ（adaptive HARQ）意味着可以改变重传所使用的 PRB 资源以及 MCS。**非自适应 HARQ**（non-adaptive HARQ）意味着重传必须与前一次传输（新传或前一次重传）使用相同的 PRB 资源和 MCS。

在 LTE 中，下行使用异步、自适应 HARQ；上行使用同步 HARQ，但重传可以是自适应的，也可以是非自适应的。

表 11-2：上下行重传与同步/异步、自适应/非自适应的对应关系

HARQ	**下行**	**上行**
时域	异步	同步
频域	自适应	自适应或非自适应

注意：（1）异步/同步、自适应/非自适应都是针对重传与前一次传输（包括新传和前一次重传）之间的关系的，而不是针对某次传输（包括新传和重传）与 ACK/NACK 之间的关系的。

（2）“初传”也称为“新传”，是指某个 TB 的第一次传输；“重传”是指同一个 TB 的除初传外其它次传输；“前一次传输”是同一个 TB 的相对于当前传输的前一次传输。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 12.1 节

11.5 BLER

在无线网络中，一个设备（如 eNodeB）是按块（block，也就是 TB）向另一个设备（如 UE）发送数据的。发送端使用块中的数据计算出一个 CRC，并随着该块一起发送到接收端。接收端根据收到的数据计算出一个 CRC，并与接收到的 CRC 进行比较，如果二者相等，接收端就认为成功地收到了正确的数据，并向发送端回复一个“ACK”；如果二者不相等，接收端就认为收到了错误的数据，并向发送端回复一个“NACK”，以要求发送端重传该块。如果在某个特定的期间内，发送端没有收到接收端的回复，则发送端假定之前发送的块没有到达接收端，发送端自动重发该块。(MAC 层的 HARQ 处理)

BLER（block error rate），即误块率，是出错的块在所有发送的块中所占的百分比（只计算初传的 block）。在实际应用中，某一特定百分比（如：LTE 中数据信道的 BLER 要求为 10%以下）的 BLER 并不总是必须的，因为可以重传出错的块并通过特殊的处理（如软合并等），使得接收端正确解出收到的数据。需要测量和计算 BLER 时，在发送端就能够完成，因为可以通过收到的 NACK 数来计算 BLER。

在 LTE 中，控制信道的目标 BLER 为 1%，数据信道的目标 BLER 为 10%。当 BLER 不超过 10%时，UE 将向 eNodeB 上报它所能解码的最高 MCS（通过 CQI 上报）。

LTE 在无 HARQ 重传情况下误块率指标为 10%，加入 HARQ 重传后误帧率（FER）大概为 1%，再加上 RLC 层的 ARQ 后性能可以提升到 10^-5 数量级。

例：假设发送了 500 个 block 的数据，其中 499 个 block 回复 ACK，1 个 block 回复 NACK，则 BLER 为 1 / 500 = 0.002 * 100% = 0.2%（从这个例子可以看出，计算 BLER 时，是不把重传的 block 的 ACK/NACK 计算在内的）。

【参考资料】

[1] 《How to Calculate the Block Error Rate》 by Steve McDonnell

第12章 上行 HARQ

上行 HARQ 是对 UE 发往 eNodeB 的 PUSCH 数据进行确认（Acknowledge）以及重传的处理流程。

本章主要介绍上行 HARQ 的处理。但并不包含 TTI bundling 的上行 HARQ 处理，这部分内容会在第 25 章介绍“TTI bundling”时予以介绍。

本章重点围绕着以下几个问题来阐述上行 HARQ 的处理流程：

- 如何判断是重传还是新传？
- 何时清空 HARQ buffer？
- 如何确定 HARQ process？
- 如何确定 RV？
- 如何确定 timing 关系？
- 上行 HARQ 中的自适应和非自适应重传处理有何不同？

12.1 上行 HARQ 介绍

图 12-1 是上行数据传输的一个简单示意图（不包含除 SR 外的其它 PUCCH 传输，也不包含 DMRS）。

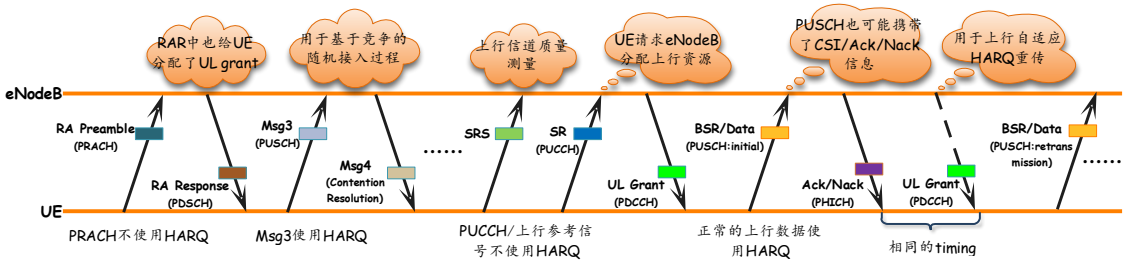


图 12-1：上行数据传输

前面已经介绍过，只有 UL-SCH 支持上行 HARQ，而 UL-SCH 在物理信道 PUSCH 上传输，也即只有 PUSCH 上传输的数据支持上行 HARQ。

上行 PUSCH 传输支持 2 种传输模式：TM 1（只支持单天线端口传输）和 TM2（支持空分复用）。

对于 FDD 而言，如果上行配置的传输模式只支持单天线端口传输，则有 8 个上行 HARQ process；如果上行配置的传输模式支持空分复用，则上行 HARQ process 数将翻倍，为 16 个。

对于 TDD 而言，如果上行配置的传输模式只支持单天线端口传输，则不同的 TDD 上下行配置对应的上行 HARQ process 数见 36.213 的 Table 8-1 所示（见下表）；如果上行配置的传输模式支持空分复用，则上行 HARQ process 数将翻倍。

这里我们不考虑 TTI bundling，即子帧绑定（subframe bundling）的场景。

Table 8-1: Number of synchronous UL HARQ processes for TDD

TDD UL/DL configuration	Number of HARQ processes for normal HARQ operation	Number of HARQ processes for subframe bundling operation
0	7	3
1	4	2
2	2	N/A
3	3	N/A
4	2	N/A
5	1	N/A
6	6	3

对于上行传输，每个 UE（而不是每个无线承载）会被配置一个最大传输次数。Msg 3 的最大传输次数是通过 *RACH-ConfigCommon* 的 *maxHARQ-Msg3Tx* 字段来配置的；而除 Msg3 外的其它上行 HARQ process 允许的最大传输次数是通过 *MAC-MainConfig* 的 *maxHARQ-Tx* 字段来配置的。

eNodeB 使用 PHICH 来反馈对应 PUSCH 传输的 ACK/NACK，一个上行 TB 对应一个 PHICH 资源。关于 PHICH，请参见 4.3 节的介绍。

12.2 自适应/非自适应、同步

前面已经介绍过，在 LTE 中，上行使用同步 HARQ，但重传可以是自适应的，也可以是非自适应的。

上行 HARQ 使用同步（synchronous）、非自适应（non-adaptive）的目的是为了降低开销。由于上行重传总是发生在可预知的子帧上（例如：在 FDD 下，重传总是发生在前一次传输的 8 个子帧之后；TDD 下跟具体的 TDD 上下行配置相关，见 12.2.4 节介绍），所以根据 timing 关系可以直接推导出使用的 HARQ process。并且在非自适应重传时，重传与**前一次传输**（注意：不是“新传”，这可以在 36.300 的 9.1 节和 36.321 的 5.4.2.2 节找到答案）使用相同的 PRB 资源和 MCS。因此下行只需要 PHICH 这一种控制信令，而不需要 PDCCH，从而降低了开销。

而上行 HARQ 有时使用自适应（adaptive）重传是为了避免分割上行频域资源或避免与随机接入的资源发生碰撞。此时 eNodeB 不仅会发送 PHICH，还会发送 PDCCH 以指示重传所使用的新的 PRB 资源和 MCS。

上行 DCI 包括 DCI format 0/4，在 PDCCH 上传输。如果没有特别说明，本章提到的 PDCCH 指的都是携带了上行 DCI 的 PDCCH。

再次强调，对于 HARQ，只有在对应同一 TB，也即同一 HARQ process 的基础上讨论“初传”、“重传”才有意义。

12.2.1 判断新传/重传

eNodeB 会对接收到的上行数据进行 CRC 校验，以决定是否需要重传。如果 CRC 校验成功，且 UE 还有上行数据发送，则进行新传；如果 CRC 校验失败，则进行重传。

而在 UE 侧，需要根据对应 HARQ process 的 NDI 值是否发生翻转来决定是进行新传还是重传。

如果 UE 要在 PUSCH 上发送数据，需要满足以下两个条件之一或同时满足这两个条件：

- 收到一个有效的 UL Grant：该 UL grant 可以来自动态调度的 PDCCH、或来自 RAR，或通过 SPS（半静态调度）配置。
- 收到一个 PHICH 且指示为 NACK：对应非自适应重传。

注：RAR 指示的 Msg3 传输的上行 HARQ 处理与正常的 HARQ 处理（对应动态调度或半静态调度）有些不同，这部分内容会在后续章节中予以介绍。

对于 Msg3 的传输：

Msg3 的初传使用的资源是由 RAR 来指定的；而 Msg3 的自适应重传是由 TC-RNTI 加扰的 PDCCH（DCI format 0/4）来指定的，并且在上行，使用 TC-RNTI 加扰的 PDCCH 只会用于 Msg3 的重传。所以有

- 如果 UE 收到的 UL grant 来自 RAR（使用 RA-RNTI 加扰），则对应的 PUSCH 传输（Msg3）是新传；
- 如果 UE 收到使用 TC-RNTI 加扰的 PDCCH，则对应的 PUSCH 传输（Msg3）是自适应重传（见 12.2.3 节）。也就是说，TC-RNTI 加扰的 PDCCH 携带的 NDI 字段不作为判断 Msg3 是新传还是重传的依据；

- 如果 UE 只收到一个 PHICH 且指示为 NACK，则对应的 PUSCH 传输（Msg3）是非自适应重传（见 12.2.3 节）。

对于 SPS 的传输（关于 SPS 的介绍，见第 24 章）：

- 如果 UE 在 PCell 上收到使用 SPS C-RNTI 加扰的指示上行 SPS 激活的 PDCCH，且该 DCI 携带的 NDI 字段为 0，则认为对应 HARQ process 的 NDI 发生翻转，且对应的 PUSCH 传输是新传；
- 如果 UE 在 PCell 上收到使用 SPS C-RNTI 加扰的指示上行 SPS 激活的 PDCCH，且该 DCI 携带的 NDI 字段为 1，则认为对应 HARQ process 的 NDI 没有翻转，且对应的 PUSCH 传输是自适应重传；
- 如果是上行 SPS 激活后的周期性 PUSCH 传输，则认为对应 HARQ process 的 NDI 发生翻转，该 PUSCH 传输是新传；
- 如果 UE 收到使用 C-RNTI 加扰的 PDCCH，且同一 HARQ process 的前一次收到的 PDCCH 是使用 SPS C-RNTI 加扰的（即前面介绍的 3 种情况），则认为对应 HARQ process 的 NDI 发生翻转（此时会忽略使用 C-RNTI 加扰的 PDCCH 中携带的 NDI 字段的值），且对应的 PUSCH 传输是新传；
- 如果 UE 只收到一个 PHICH 且指示为 NACK，则对应的 PUSCH 传输是非自适应重传。

从 36.321 的 5.4.1 节可以看出，UE 优先尝试接收使用 TC-RNTI/C-RNTI 加扰的 PDCCH 或 RAR；如果没有接收到，才会去尝试接收使用 SPS C-RNTI 加扰的 PDCCH。

对于动态调度（使用 C-RNTI 加扰）的传输：

- 如果 UE 收到使用 C-RNTI 加扰的 PDCCH，且该 DCI 携带的 NDI 字段与对应 HARQ process 的前一次传输相比发生了翻转，则对应的 PUSCH 传输是新传；
- 如果 UE 收到使用 C-RNTI 加扰的 PDCCH，且该 DCI 携带的 NDI 字段与对应 HARQ process 的前一次传输相比没有翻转，则对应的 PUSCH 传输是自适应重传；
- 如果 UE 收到使用 C-RNTI 加扰的 PDCCH，且对应的 HARQ process 的 HARQ buffer 是空的（即第一次使用该 HARQ process 来传输），则对应的 PUSCH 传输是新传；
- 如果 UE 只收到一个 PHICH 且指示为 NACK，则对应的 PUSCH 传输是非自适应重传。

本节的结论是对 36.321 的 5.4 节进行分析后得出的。

12.2.2 TB 使能/去使能以及与 codeword 的对应关系

针对同一个 UE，上行非空分复用情况下，一个 TTI 只能发送一个 TB，该 TB 与一个 codeword 一一对应。

而针对同一个 UE，上行空分复用（上行 TM 2，见 36.213 的 8.0 节）情况下，情况略显复杂。UE 配置了空分复用的情况下，并不是在每个调度的 TTI 都能发送 2 个 codeword。

对于上行 PUSCH 传输而言，TB 去使能（disabled）的情况有 2 种：

情况一：当 UE 配置了 TM 2，且接收到的是 DCI format 0 时，UE 会认为需要发送的 PUSCH 只对应 TB 1（transport block 1），而 TB 2（transport block 2）是去使能的；（见 36.213 的 8.0 节）

情况二：UE 接收到 DCI format 4，如果某个 TB 对应的“$I_{\mathrm{MCS}}=0$ 且 $N_{\mathrm{PRB}}>1$”或“$I_{\mathrm{MCS}}=28$ 且 $N_{\mathrm{PRB}}=1$”，则该 TB 是去使能的；否则该 TB 是使能的。（见 36.213 的 8.6.2 节）

TB 与 codeword 的对应关系如下：

如果 2 个 TB 都使能，则 TB 1 与 codeword 0 对应，TB 2 与 codeword 1 对应。

如果 2 个 TB 中的一个是去使能的（对应前面介绍的情况二），则 TB 与 codeword 的对应关系见 36.212 的 Table 5.3.3.1.5-2。从下表可以看出，使能的那个 TB 都只对应 codeword 0。

Table 5.3.3.1.5-2: Transport block to codeword mapping (one transport block enabled).

transport block 1	transport block 2	codeword 0 (enabled)	codeword 1 (disabled)
enabled	disabled	transport block 1	-
disabled	enabled	transport block 2	-

12.2.3 上行 HARQ 自适应/非自适应/延迟重传处理

前一节已经介绍了如何判断是进行新传还是重传。这一节我们介绍如何判断使用哪种方式进行重传。

表 12-1：上行 HARQ 处理（参见 36.300 的 9.1 节）

UE 收到的 PHICH	UE 收到的指示上行传输的 PDCCH	UE 的行为	重传类型
ACK or NACK	新传 (NDI 翻转)	根据收到的 PDCCH 进行新传，并将 HARQ buffer 中的数据替换为新的 MAC PDU	无
ACK or NACK	重传 (NDI 没有翻转)	根据收到的 PDCCH 进行重传	自适应重传
ACK	无	无传输，也不进行重传。不清空 HARQ buffer，数据依然保存在 HARQ buffer 中。UE 需要接收到一个新的 PDCCH 才能恢复重传	无 （延迟重传）
NACK	无	重传使用与“前一次传输”相同的频域资源和相同的传输格式	非自适应重传

表 12-1 指出了 UE 如何判断是进行新传、自适应重传、非自适应重传还是不进行任何传输（“延迟重传”）。

（1）如果 UE 在某个子帧只收到 PHICH，且其值为 NACK，则使用非自适应重传。

（2）如果 UE 在某个子帧只收到 PHICH，且其值为 ACK，则不进行任何传输。

（3）给定某个 HARQ process，无论收到的 PHICH 指示的是 ACK 还是 NACK，只要同时还收到 PDCCH，则 UE 会忽略 PHICH 而使用 PDCCH 来决定如何进行下一次传输（新传或重传）。

（4）给定某个 HARQ process，即使接收到的 PHICH 指示为 ACK，也不会清空 HARQ buffer。此时，还需要通过在当前子帧或后续子帧中接收到的 PDCCH 中的 NDI 来决定是进行重传（NDI 没有翻转），还是进行新传（NDI 翻转）。也即，是否清空 HARQ buffer 是由对应 HARQ process 的 NDI 来决定的。如果 UE 只收到了 ACK，那么接下来的那次重传只能进行自适应重传、而不能进行非自适应重传。

UE 只是遵循 eNodeB 的调度要求来进行传输，实际选用哪种方式是由 eNodeB 决定的。对于上行 HARQ 重传，eNodeB 有 3 种处理方式：

1、**非自适应重传**：“前一次传输”占用的资源与其它资源没有冲突，且重传与 measurement gap 没有冲突时，eNodeB 会在 PHICH 上发送 NACK，且不发送 PDCCH。

2、**自适应重传**：当满足以下条件之一，且 eNodeB 成功地为该 UE 分配了 PDCCH/PUSCH 资源时，eNodeB 会使用自适应的方式来重传上行数据。

- 重传所占的资源与 PRACH 资源、MSG3 所占的资源相冲突（此时重传的优先级较低）时，或
- 为了避免分割上行频域资源，或
- 此次重传是由 HARQ 反馈为 DTX 的初传触发的，或
- 对“延迟重传”的重新调度 。

此时，eNodeB 会在 PHICH 上发送 ACK，且会发送 PDCCH。

上行自适应重传中，在 PHICH 上发送 ACK 而不是 NACK 的原因在于：如果 eNodeB 在自适应重传时，发送的是 NACK 和 PDCCH，那么当 UE 只收到了 NACK，却丢失了 PDCCH 时，由于 UE 并不知道 eNodeB 是否发送了 PDCCH，UE 会根据收到的 NACK 信息来进行非自适应重传，这样可能造成资源的冲突。如果 eNodeB 在自适应重传时，发送的是 ACK 和 PDCCH，那么当 UE 只收到了 ACK，却丢失了 PDCCH 时，UE 不会进行任何传输，这样就不会造成资源冲突的问题。

3、**只反馈 ACK，不进行重传**（“延迟重传”）：当 eNodeB 判断 UE 需要进行自适应重传，但却无法给该重传分配 PDCCH/PUSCH 资源时，或当重传与 measurement gap 相冲突时，eNodeB 会在 PHICH 上发送 ACK 而不发送 PDCCH，以告诉 UE 不进行上行 HARQ 重传（见 36.300 的 9.1 节）。eNodeB 会在之后选择一个合适的时机，使用自适应重传的方式通知 UE 重新发送重传。

需要说明的是，本节中关于 eNodeB 侧上行 HARQ 重传的处理，仅供大家参考，不同厂商在 eNodeB 侧的实现可能略有不同。

接下来，我们会介绍在上行 HARQ 重传中如何选取 RV。

如果是**非自适应重传**，UE 只会收到 PHICH 中指示的 NACK 信息，而不会显式地收到 RV 信息。上行 HARQ 是同步的，RV 的选择遵循一个固定的顺序：0，2，3，1。新传使用的 RV 由 PDCCH 显式指定，其值为 0；若之后 UE 收到 NACK（但没收到 PDCCH），则会使用前一次传输对应的下一个 RV 版本（0 -> 2 -> 3 -> 1 -> 0 -> …）来发起重传，依此类推。

如果是**自适应重传**，则 UE 不仅会收到 PHICH，还会收到 PDCCH。如果需要重传，UE 会根据 PDCCH 的指示来选择 RV（见 36.213 的 Table 8.6.1-1。通常 eNodeB 也会按 0，2，3，1 顺序来选取 RV）。如果 PDCCH 中指示的 MCS index 为 0~28，则 RV = 0 并使用 Table 8.6.1-1 中指示的真正的 MCS。

Table 8.6.1-1 中的 MCS index 29~31 是预留给上行 HARQ 重传用的，用于指示自适应重传使用的 RV，且不指定真正的 MCS 信息。如果 MCS index 为 29~31，则该重传的 MCS 遵循前一次传输使用的 MCS（或者说，最近一次接收到的 MCS index 为 0~28 的 PDCCH），TBS 和调制方式都与前一次传输相同。

注：上行自适应重传的 MCS index 只能选择 29~31，因为只有这 3 种组合才能指定不同的 RV；而 MCS index 0~28 的 RV 固定为 0。

在 PDCCH 调度的重传中使用哪种 RV 需要在增量冗余增益和稳定性之间取得平衡。从增量冗余的角度上看，改变重传之间的 RV 值有益于充分利用来自增量冗余的增益。但如果丢失了初传的 PDCCH 信息，则意味着需要重新发送 PDCCH 且 RV 设置为 0，以便显式地指示 MCS 和 TBS 等信息。

如果上行同时支持自适应和非自适应重传，则要求对应同一上行子帧的 PHICH 和 PDCCH 拥有相同的 timing，即在同一下行子帧上发送。如果不满足该条件，则 UE 不知道是该听从 PHICH 还是该等待 PDCCH 而不管 PHICH，实现的复杂度会大增。

12.2.4 上行 HARQ 同步处理

上行 HARQ 重传在时域上使用同步的方式，相邻的 2 次传输（初传与重传，重传与重传）之间有固定的 timing 关系。

接下来，我们分 FDD、TDD 1~6、TDD 0 三种配置来介绍上行 HARQ 在时域上的同步关系。每种配置都包含 2 部分：

- PDCCH/PHICH 与对应的 PUSCH 传输之间的 timing 关系；
- PUSCH 传输与对应的 PHICH（ACK/NACK）之间的 timing 关系。

RAR 与对应的 Msg3 传输之间的 timing 关系比较特殊，我们会在 12.2.4.4 节介绍这部分内容。

12.2.4.1 FDD

对 FDD 而言，如果 UE 在子帧 n 收到了 PDCCH（DCI format 0/4，对应新传或自适应重传）或 PHICH（只收到 NACK，对应非自适应重传），则 UE 会在子帧 n + 4 发送对应的 PUSCH。（见 36.213 的 8.0 节）

对 FDD 而言，如果 UE 在子帧 n 收到了 PHICH，则该 PHICH 对应 UE 在上行子帧 n - 4 发送的 PUSCH。（见 36.213 的 8.3 节）

如图 12-2 所示。

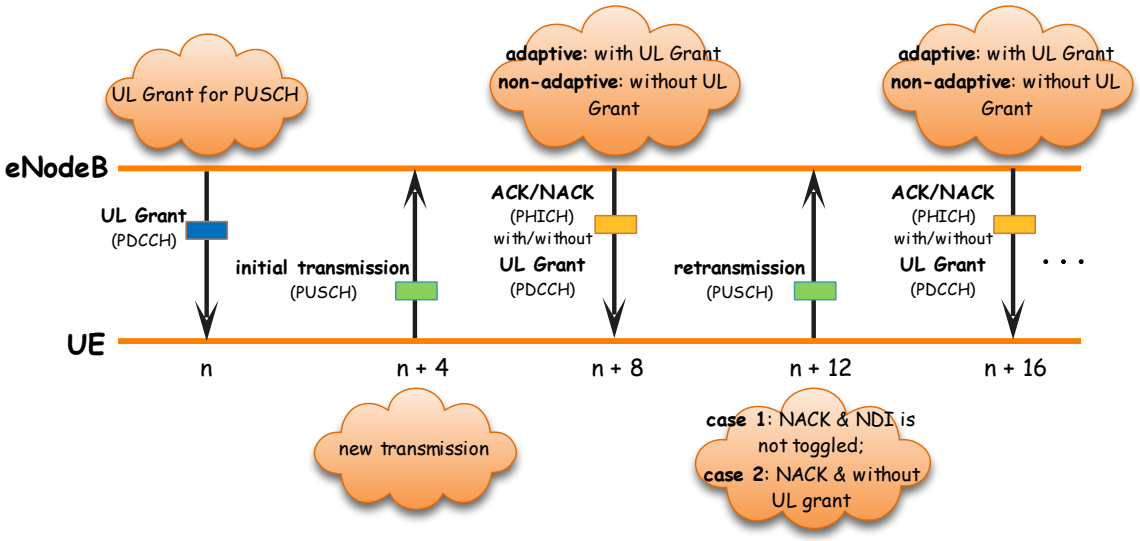


图 12-2：FDD 中的上行传输，PDCCH、PUSCH、ACK/NACK 之间的 timing 关系

子帧 n、n+4、n+8、n+12、n+16 都对应同一 HARQ process。只要确定了子帧 n 所使用的 HARQ process number，根据 timing 关系，也就知道后续子帧 n+4、n+8、n+12、n+16 所使用的 HARQ process。TDD 的情况类似，但是 timing 关系略有不同。

注：每个 TTI 只对应一个 HARQ process，空分复用的情况下是 2 个，每个 HARQ process 对应一个 TB。

从上面的介绍可以看出，初传和重传、重传和重传之间固定相隔 8 个子帧，即有同一 TB 的初传和重传所在子帧的（SFN * 10 + subframeNumber）% 8 值相等，在实现时，我们可以使用该值作为对应的 HARQ process number。

12.2.4.2 TDD 1~6

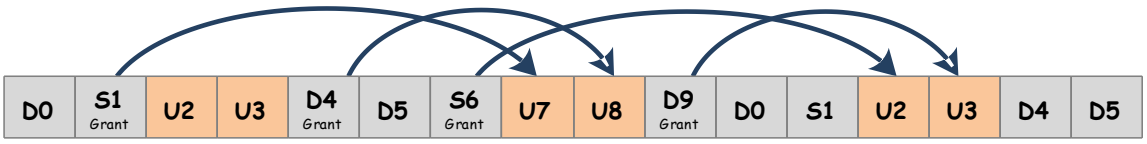
对 TDD 1~6 而言，如果 UE 在子帧 n 收到了 PDCCH（DCI format 0/4，对应新传或自适应重传）或 PHICH（只收到 NACK，对应非自适应重传），则 UE 会在子帧 n + k 发送对应的 PUSCH。其中 k 的值见 36.213 的 Table 8-2。（见 36.213 的 8.0 节）

Table 8-2 *k* for TDD configurations 0-6

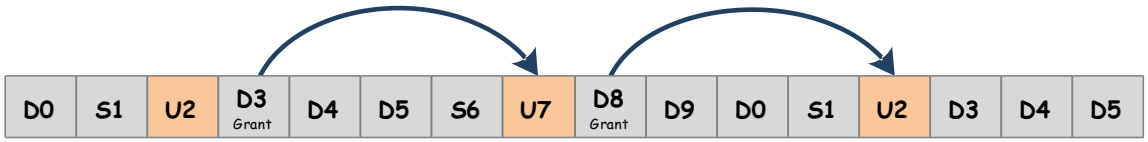
	subframe number *n*

TDD UL/DL Configuration	0	1	2	3	4	5	6	7	8	9
0	4	6				4	6			
1		6			4		6			4
2				4					4	
3	4								4	4
4									4	4
5									4	
6	7	7				7	7			5

图 12-3 给出了 TDD 1 和 TDD 2 下，PDCCH/PHICH 与对应的 PUSCH 传输之间的 timing 关系。以 TDD 1 为例，如果 UE 在子帧 1 收到了 PDCCH（或 PHICH），则 UE 会在子帧 7 发送 PUSCH（或重传）；如果 UE 在子帧 9 收到了 PDCCH（或 PHICH），则 UE 会在下一系统帧的子帧 3 发送 PUSCH（或重传）。



TDD Configuration 1 (DL:UL=3:2)



TDD Configuration 2 (DL:UL=4:1)

图 12-3：TDD 1/2 下，PDCCH/PHICH 与对应的 PUSCH 传输之间的 timing 关系

（对应 36.213 的 Table 8-2）

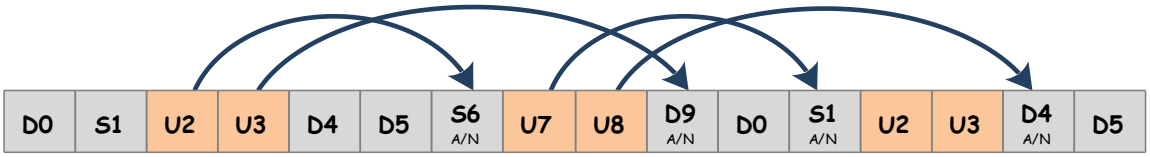
对 TDD 1~6 而言，如果 UE 在子帧 n 收到了 PHICH，则该 PHICH 对应 UE 在上行子帧 n - k 发送的 PUSCH。其中 k 的值见 36.213 的 Table 8.3-1（见下表）。（见 36.213 的 8.3 节）

Table 8.3-1 *k* for TDD configurations 0-6

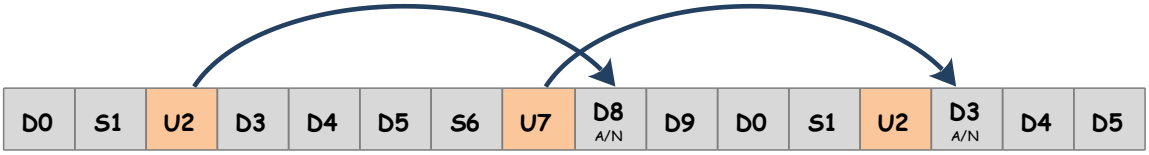
TDD UL/DL Configuration	subframe number *i*									
	0	1	2	3	4	5	6	7	8	9
0	7	4				7	4			

1		4			6		4			6
2				6					6	
3	6								6	6
4									6	6
5									6	
6	6	4				7	4			6

图 12-4 给出了 TDD 1 和 TDD 2 下，PUSCH 传输与对应的 PHICH（ACK/NACK）之间的 timing 关系。以 TDD 1 为例，如果 UE 在子帧 2 发送了 PUSCH，则 UE 会在子帧 6 接收对应的 PHICH；如果 UE 在子帧 7 发送了 PUSCH，则 UE 会在下一系统帧的子帧 1 接收对应的 PHICH。



TDD Configuration 1 (DL:UL=3:2)



TDD Configuration 2 (DL:UL=4:1)

图 12-4：TDD 1/2 下，PUSCH 传输与对应的 PHICH（ACK/NACK）之间的 timing 关系

（对应 36.213 的 Table 8.3-1）

图 12-5 是 TDD 1 的一个例子：假如 UE 在下行子帧 1 收到 PDCCH，对照图 12-3 可知，UE 会在上行子帧 7 发送 PUSCH，进一步对照图 12-4 可知，UE 会在下行子帧 1 接收 PHICH（和 PDCCH）。如果需要重传，对照图 12-5 可知，UE 会在上行子帧 7 上进行重传，如此反复！这就是一个完整的 HARQ timing 关系。

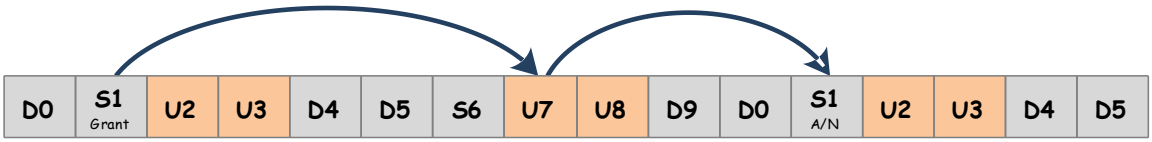


图 12-5：TDD 1 下的上行 HARQ timing 举例

对照图 12-3 和 12-4 可以看出，TDD 1 和 TDD 2（典型的 TDD 配置）中的初传和重传都位于相同的子帧号上，因此在实现时，我们可以将 HARQ process number 和子帧号一一绑定。如表 12-2 和表 12-3 所示。

表 12-2：TDD 1 下，子帧号与 HARQ process number 的映射关系举例

TDD Configuration 1										
子帧号	0	1	2	3	4	5	6	7	8	9
HARQ process number			#0	#1				#2	#3	

表 12-3：TDD 2 下，子帧号与 HARQ process number 的映射关系举例

TDD Configuration 2										
子帧号	0	1	2	3	4	5	6	7	8	9
HARQ process number			#0					#1		

12.2.4.3 TDD 0

对 TDD 0 而言，一个系统帧内的下行子帧数少于上行子帧数。因此，一个下行子帧可能需要同时给 2 个上行子帧发送 UL grant，为了实现该功能，DCI format 0/4 新增了一个 2 比特的字段：UL index（见 36.212 的 5.3.3.1.1 节和 5.3.3.1.8 节）。与此同时，一个下行子帧可能需要同时反馈 2 个上行子帧的 ACK/NACK 信息，为了将不同上行子帧对应的 PHICH 区分开，又新增了 I_{PHICH} 的概念（见 36.213 的 9.1.2 节）。

注意：只有 TDD 0 下的上行子帧 4 或 9 的 PUSCH 传输对应 $I_{PHICH}=1$；其它情况下（包括 FDD 和其它 TDD 上下行配置），$I_{PHICH}=0$。

对 TDD 0 而言，如果 UE 在子帧 n 收到的 PDCCH 的 UL index 的 MSB 置为 1，或在子帧 0 或 5 接收到的 PHICH 对应 $I_{PHICH}=0$（换句话说，不是对子帧 4 或 9 发送的 PUSCH 传输的 HARQ 反馈），则 UE 会在子帧 n + k 发送对应的 PUSCH，其中 k 的值见 36.213 的 Table 8-2。

对 TDD 0 而言，如果 UE 在子帧 n 收到的 PDCCH 的 UL index 的 LSB 置为 1，或在子帧 0 或 5 接收到的 PHICH 对应 $I_{PHICH}=1$（换句话说，是对子帧 4 或 9 发送的 PUSCH 传输的 HARQ 反馈），或在子帧 1 或 6 接收到 PHICH，则 UE 会在子帧 n + 7 发送对应的 PUSCH。

对 TDD 0 而言，如果 UE 在子帧 n 收到的 PDCCH 的 UL index 的 MSB 和 LSB 均置为 1，则 UE 会在子帧 n + k 和 n + 7 都发送对应的 PUSCH。（见 36.213 的 8.0 节）

如图 12-6 所示。

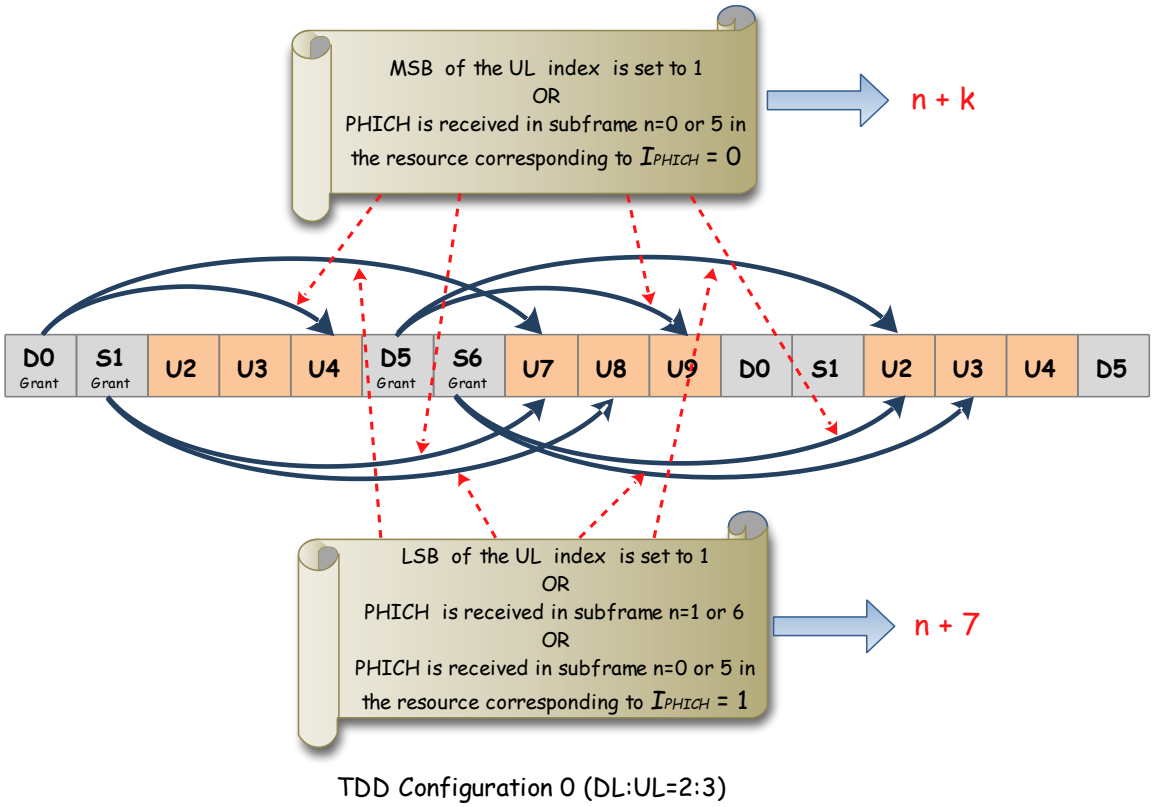


图 12-6：TDD 0 下，PDCCH/PHICH 与对应的 PUSCH 传输之间的 timing 关系

从图 12-6 中可以看出：在 TDD 0 下，（1）任意一个下行子帧发送的 UL grant 都可能对应 2 个上行子帧发送的 PUSCH；（2）UE 在某个上行子帧使用的资源可能来自 2 个下行子帧的 UL grant。例如：如果 UE 在下行子帧 0 收到的 UL grant 的 LSB 置为 1，同时在下行子帧 1 收到的 UL grant 的 MSB 置为 1，则对应上行子帧 7，会收到 2 个 UL grant。但实际上，eNodeB 在调度时应该避免出现这种情况，UE 在一个上行子帧上使用的 PUSCH 资源只应由一个 UL grant 来指定。

对 TDD 0 而言，如果 UE 在对应 $I_{PHICH}=0$ 资源的子帧 n 收到了 PHICH，则该 PHICH 对应 UE 上行子帧 n - k 发送的 PUSCH 数据，其中 k 的值见 36.213 的 Table 8.3-1；如果 UE 在对应 $I_{PHICH}=1$ 资源的子帧 n 收到了 PHICH，则该 PHICH 对应 UE 上行子帧 n - 6 发送的 PUSCH 数据。其中 k 值见 36.213 的 Table 8.3-1。（见 36.213 的 8.3 节）

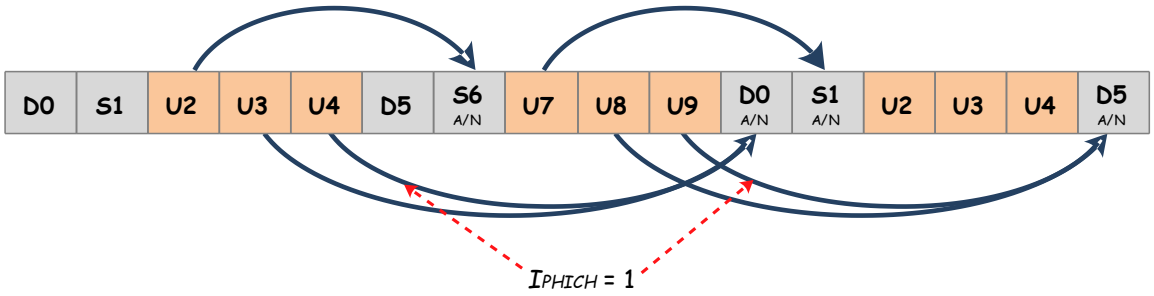


图 12-7：TDD 0 下，PUSCH 传输与对应的 ACK/NACK 之间的 timing 关系

（对应 36.213 的 Table 8.3-1）

从图 12-7 可以看出，对 TDD 0 而言，下行子帧 0 可能需要同时反馈在上行子帧 3（$I_{PHICH}=0$）和 4（$I_{PHICH}=1$）发送的 PUSCH 数据的 ACK/NACK 信息；下行子帧 5 可能需要同时反馈上行子帧 8（$I_{PHICH}=0$）和 9（对应 $I_{PHICH}=1$）发送的 PUSCH 数据的 ACK/NACK 信息。

12.2.4.4 RAR 与 Msg3 之间的 timing 关系

Msg3 的初传的 UL grant 由 RAR 指定，该 UL grant 与对应 Msg3 传输之间的 timing 关系跟 PDCCH 与对应的 PUSCH 传输之间的 timing 关系是不同的。

如果 UE 在子帧 n 成功地接收到自己的 RAR（接收到使用 RA-RNTI 加扰的 PDCCH，且对应的 DL-SCH 传输块中有一个 RAR 的 preamble，与该 UE 之前传输的 preamble 相同），则 UE 会在 n + k_1（其中 $k_1 \geq 6$）开始的第一个可用上行子帧（对于 FDD 而言，就是 n + 6；对于 TDD 而言，n + 6 可能不是上行子帧，所以 k_1 可能≥ 6）发送 Msg3。RAR 所带的 UL grant 中包含一个 1 比特的字段 UL delay，如果该值为 0，则 n + k_1 为第一个可用于 Msg3 的上行子帧；如果该值为 1，则 UE 会在 n + k_1 之后的第一个可用上行子帧来发送 Msg3。（见 36.213 的 6.1.1 节）

图 12-8 是 FDD 下 MSG3 的 timing 关系的举例；图 12-9 是 TDD 1 下 MSG3 的 timing 关系的举例。

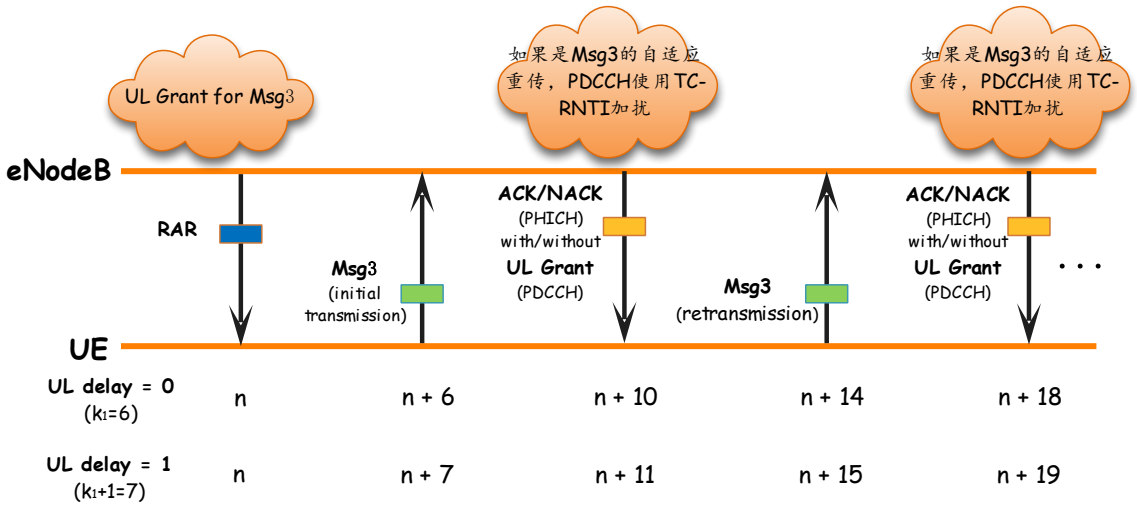
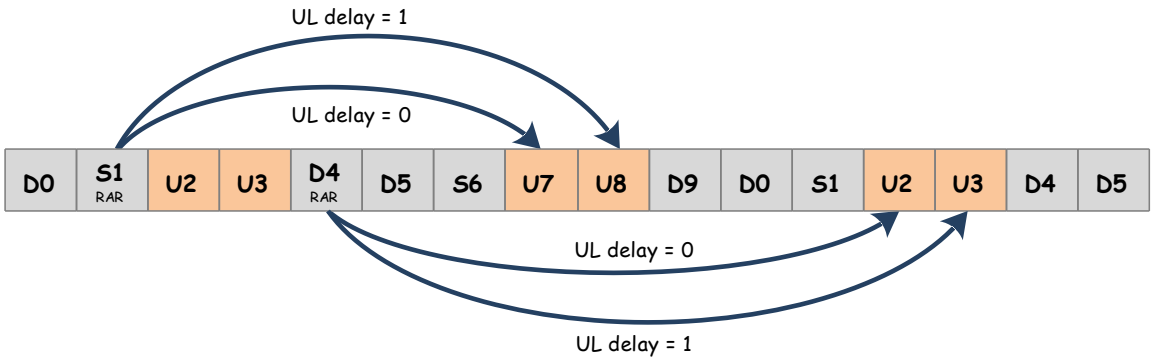


图 12-8：FDD 下，Msg3 传输的 timing 关系

（注意与图 12-2 相比，timing 关系的变化）



TDD Configuration 1 (DL:UL=3:2)

图 12-9：TDD 1 下，RAR 与对应的 Msg3 传输之间的 timing 关系

（注意与图 12-3 相比，timing 关系的变化）

对于正常的上行传输，UE 在子帧 n 收到 UL grant 后，在子帧 n + 4 或之后的第一个上行子帧发送上行数据，其 UL grant 是在 PDCCH 上传输的。但对于 Msg3 来说，UE 在子帧 n 收到 RAR 后，是在子帧 n + 6 或之后的第一个上行子帧（不考虑 UL delay 为 1 的情况）发送 Msg3 的。这是因为 RAR（包含 Msg3 的 UL grant）是在 PDSCH 而不是在 PDCCH 上传输的，所以 UE 需要更多的时间去确定 UL grant、传输格式等。

图 12-10 是 FDD 下的上行 HARQ 处理的一个例子。TDD 除了 timing 关系不同外，其它处理是一样的。

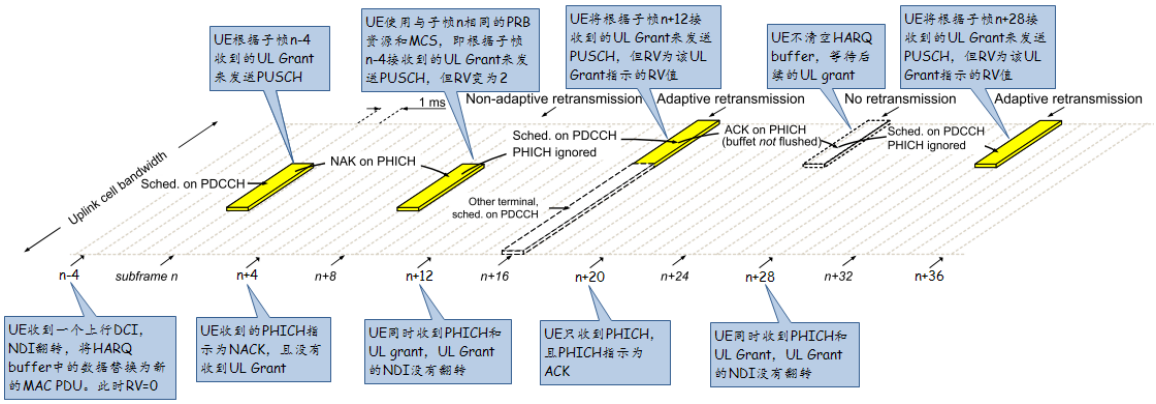


图 12-10：上行非自适应和自适应 HARQ 操作（FDD）

【参考资料】

[1] TS 36.211 的 6.9 节 Physical hybrid ARQ indicator channel

[2] TS 36.212 的 5.3.3.1 节 DCI formats

[3] TS 36.213 的 8.0 节 UE procedure for transmitting the physical uplink shared channel

[4] TS 36.213 的 8.3 节 UE HARQ-ACK procedure

[5] TS 36.213 的 8.6 节 Modulation order, redundancy version and transport block size determination

[6] TS 36.300 的第 9 章 ARQ and HARQ

[7] TS 36.321 的 5.4 节 UL-SCH data transfer

[8] 《4G LTE/LTE-Advanced for Mobile Broadband》的 6.3、6.4、8.2.2.3 节、12 章

[9] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 10.3.2 节

[10] 《HARQ in TD-LTE》by Sekhar

[11] 《TDD 中的 HARQ 进程》

[12] 《HARQ process》

第13章 下行 HARQ

下行 HARQ 是对 eNodeB 发往 UE 的 PDSCH 数据进行确认（Acknowledge）以及重传的处理流程。

本章主要介绍下行 HARQ 的处理，重点围绕着以下几个方面来阐述下行 HARQ 的处理流程：

- 如何判断是重传还是新传？
- 下行的异步、自适应处理；
- DAI；
- HARQ bundling 和 HARQ multiplexing；
- 载波聚合下的下行 HARQ 处理，主要包括 PUCCH format 1b with channel selection 和 PUCCH format 3；
- 下行 HARQ 的冲突处理，主要在 ACK/NACK 与 SR/CSI 的冲突处理；
- HARQ-ACK Repetition；
- 广播 HARQ。

对于下行 HARQ，有几种类型的下行传输需要 UE 反馈 ACK/NACK：

1、动态调度的下行传输：UE 收到一个使用 C-RNTI 或 TC-RNTI（对应随机接入过程中的 Msg4）加扰的 PDCCH 和 PDSCH，该 PDCCH 对应一个 PDSCH，二者是在同一下行子帧中同时传输；

2、SPS 调度的下行传输：分为以下 3 种情况。

2.a：UE 收到一个使用 SPS C-RNTI 加扰的指示下行 SPS 激活的 PDCCH，该 PDCCH 对应一个 PDSCH（可能是新传也可能是重传），二者是在同一下行子帧中同时传输。这种情况与动态调度的情况类似；

2.b：UE 收到一个使用 SPS C-RNTI 加扰的指示下行 SPS 释放的 PDCCH，该 PDCCH 并不指定 PDSCH 资源；

2.c：在配置的周期性 SPS 子帧上发送的 PDSCH，但该 PDSCH 并不伴随着 PDCCH 一起传输。

以下情况 UE 是不需要反馈 ACK/NACK 的：对于带正常的循环前缀的特殊子帧配置 0 和 5，或带扩展的循环前缀的特殊子帧配置 0 和 4，特殊子帧的 DwPTS 是不用于传输 PDSCH 的，所以对应该特殊子帧，UE 无需反馈 ACK/NACK。

对于下行 HARQ，UE 可在不同的物理信道上传输需要反馈的 ACK/NACK：

1、UE 有动态调度的 PUSCH 资源：如果在当前需要反馈 ACK/NACK 的上行子帧，UE 被动态地分配了上行 PUSCH 资源，则 ACK/NACK 会在 PUSCH 上传输。该 PUSCH 有对应的 DCI format 0/4。

2、无 PUSCH 资源：如果在当前需要反馈 ACK/NACK 的上行子帧， UE 没有上行调度，即不发送 PUSCH，则 UE 会在 PUCCH 上传输 ACK/NACK。

3、SPS 调度的上行传输：分为以下 2 种情况

3.a：如果 UE 在当前子帧上发送的 PUSCH 对应一个指示上行 SPS 激活的 DCI format 0/4（可能是新传也可能是重传），且需要在该子帧上反馈 ACK/NACK，则 UE 会在 PUSCH 上传输 ACK/NACK。这种情况与动态调度的情况类似。

3.b：如果 UE 在配置的周期性 SPS 子帧上发送 PUSCH，且需要在该子帧上反馈 ACK/NACK，则 UE 会在 PUSCH 上传输 ACK/NACK。

注：本章会涉及一些与 SPS（半静态调度）相关的概念，如果大家不了解 SPS，建议先阅读第 24 章。

13.1 下行 HARQ 介绍

图 13-1 是下行数据传输的一个简单示意图。

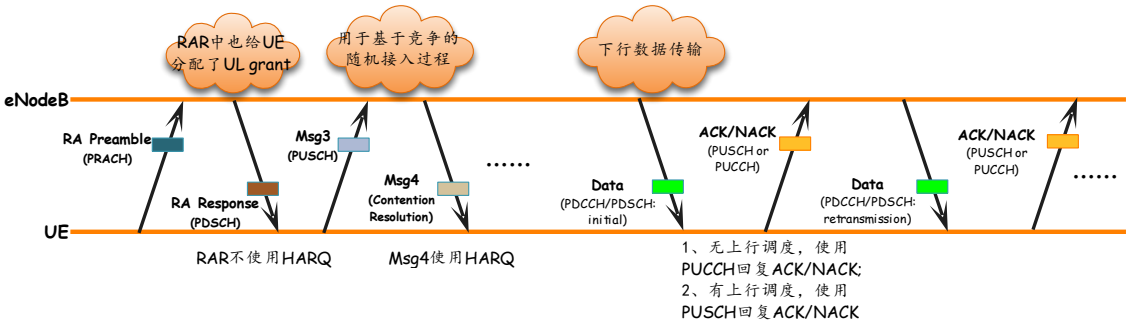


图 13-1：下行数据传输

下行 HARQ 使用异步（asynchronous）、自适应（adaptive）的方式意味着重传可能发生在任意时刻和频域上的任意位置。下行 HARQ 使用异步、自适应方式的原因是为了避免与系统信息、

MBSFN 子帧之类的传输产生冲突。通过在时域或频域上重新调度重传，避免与系统信息、MBSFN 子帧在无线资源上发生重叠。也就是说，下行重传总是通过 PDCCH 来重新调度的。

下行 PDSCH 传输对应的 ACK/NACK 在 PUCCH 或 PUSCH 上发送。

对 TDD 而言， 特殊子帧的 DwPTS 可用于发送 PCFICH/PHICH/PDCCH 和 PDSCH。这也意味着，特殊子帧可用于发送 UL grant（即上行 DCI，在 PDCCH 上传输）和对应 PUSCH 数据的 ACK/NACK（在 PHICH 上传输）等。

这里有个特例：对于带正常的循环前缀的特殊子帧配置 0 和 5，或带扩展的循环前缀的特殊子帧配置 0 和 4（见 36.211 的 Table 4.2-1），特殊子帧的 DwPTS 是不能用于传输 PDSCH 的（见 36.213 的 7.1.7 节）。这些配置下，DwPTS 的持续时间都很短，只占 3 个 OFDM 符号，所以不用于传输 PDSCH。

对 TDD 而言， UpPTS 可用于发送 SRS 和 PRACH（preamble format 4），但不能用于发送 PUCCH 和 PUSCH（见 36.213 的 8.2 节；36.211 的 5.4 节和 5.7 节）。这是因为 UpPTS 的持续时间都很短，所以不用于传输 PUCCH 和 PUSCH。也就是说，特殊子帧不能用于发送对应于 PDSCH 传输的 ACK/NACK。因此，对于下行 PDSCH 传输而言，只能使用非特殊子帧来发送对应的 ACK/NACK。

通常 eNodeB 和 UE 侧的解码处理时间至少为 3ms。

下行 HARQ 的最大重传次数是由 eNodeB 确定的，不同的厂家可能有不同的实现，UE 并不知道下行的最大重传次数（注：UE 通过 *maxHARQ-Msg3Tx* 和 *maxHARQ-Tx* 能够得知上行传输的最大重传次数），UE 只会根据收到的 NDI 来决定是进行软合并（重传）还是清空 HARQ buffer（新传）。

13.1.1 下行 DCI 中与 HARQ 相关的字段

对于 PDSCH 传输而言，UE 根据接收到的 PDCCH（下行 DCI）来决定如何处理下行 HARQ。

下行 DCI 包括 DCI format 1/1A/1B/1C/1D/2/2A/2B/2C。

如果没有特别说明，本章提到的 PDCCH 指的都是携带了下行 DCI 的 PDCCH。

表 13-1：下行 DCI 中与 HARQ 相关的字段

（Yes 表示存在该字段，No 表示不存在该字段）

字段	DCI format								
	1	**1A**	**1B**	**1C**	**1D**	**2**	**2A**	**2B**	**2C**
HARQ process number	Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes

第 1 个 TB	Modulation and coding scheme	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
	New data indicator	Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes
	Redundancy version	Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes
第 2 个 TB	Modulation and coding scheme	No	No	No	No	No	Yes	Yes	Yes	Yes
	New data indicator	No	No	No	No	No	Yes	Yes	Yes	Yes
	Redundancy version	No	No	No	No	No	Yes	Yes	Yes	Yes
TPC command for PUCCH		Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes
Downlink Assignment Index		Yes	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes
Transport block to codeword swap flag		No	No	No	No	No	Yes	Yes	No	No

表 13-1 列出了下行 DCI 中与 HARQ 相关的字段：

（1）DCI format 1/1A/1B/1D 不支持下行空分复用，只指示 1 个 TB；DCI format 2/2A/2B/2C 用于支持下行空分复用，可指示 2 个 TB；DCI format 1C 不支持 HARQ。

（2）空分复用中的每个 TB 包含各自独立 MCS/RV/NDI，并使用独立的 HARQ process 来处理。在下行空分复用中，2 个 TB 使用不同的 HARQ process，但在下行 DCI format 中，只有一个 HARQ process number 字段，该字段用于指示第一个 TB 对应的 HARQ process，而第二个 TB 对应的 HARQ process 可以通过该 HARQ process number 字段间接得到（例如：将 HARQ process number 字段 + number of HARQ process 就得到第二个 TB 对应的 HARQ process）。对上行传输而言，DCI format 0/4 是没有 HARQ process number 字段的。

（3）空分复用中的每个 TB 拥有各自的 RV/NDI 的目的在于：允许只重传其中一个 TB。也就是说空分复用的 2 个 TB 是不相关的，当其中一个 TB 出错时，另一个成功接收的 TB 并不需要重传，且成功接收 TB 的 HARQ process 可以用于下一次新传。（上行的处理也是一样的）

（4）36.213 的 Table 7.1.7.1-1 中的 MCS index 29~31 是预留给下行 HARQ 重传用的，但不用于指示下行重传使用的 RV。

与上行 HARQ 使用 MCS index 29~31 来指示自适应重传的 RV 不同（从 36.212 的 5.3.3 节可以看出，DCI format 0/4 是没有 Redundancy version 字段的），下行 DCI 中包含了 Redundancy version 字段，用于直接指示下行传输的 RV。这也是 36.213 的 Table 7.1.7.1-1 与 Table 8.6.1-1 相比，没有 Redundancy Version 这一列的原因。

如果 MCS index 为 29~31，则该下行重传与前一次传输使用相同的 TBS，但其调制阶数（Modulation Order）是可以变化的（这点与上行 HARQ 重传不同，上行 HARQ 中，重传的调制阶数是不变的）。

当然，使用这 3 种预留组合的前提是 UE 成功地接收了初传中的 DCI（使用 MCS index 0~28），否则（即初传的 HARQ 反馈为 DTX 时）重传中需要显式地指定 TBS，即此时不能使用 MCS index 29~31。

（5）下行重传的 TBS 通常是不变的。但是对应下行 HARQ 的同一个 TB 的重传，如果当前的 TBS 与最近一次收到的 TBS 不一致，则 UE 会使用当前收到的数据替换掉 HARQ buffer 中的原有数据（见 36.213 的 5.3.2.2 节）。

关于 Downlink Assignment Index (DAI)和 TPC command for PUCCH 字段，会在后续介绍 UE 如何反馈 ACK/NACK 时予以介绍。

对于 UE 而言，下行 HARQ 处理可以总结成一句话："一切听下行 DCI 的安排"！

13.1.2 TB 使能/去使能以及与 codeword 的对应关系

针对同一个 UE，非空分复用情况下，一个 TTI 只能发送一个 TB，该 TB 与一个 codeword 一一对应。

而针对同一个 UE，在空分复用情况下，情况略显复杂。UE 配置了空分复用的情况下，并不是在每个调度的 TTI 都能发送 2 个 codeword。

对于下行 PDSCH 传输而言，TB 去使能（disabled）的情况有 2 种：

情况一：当 UE 配置了 TM 3/4/8/9，且接收到的是 DCI format 1A 时，UE 会认为 eNodeB 发送的 PDSCH 只对应 TB 1（transport block 1），而 TB 2（transport block 2）是去使能的；（见 36.213 的 7.1 节）

情况二：UE 接收到 DCI format 2/2A/2B/2C，如果某个 TB 对应的 "Modulation and coding scheme "字段对应的 $I_{\mathrm{MCS}} = 0$（见 36.213 的 Table 7.1.7.1-1），且当前传输的冗余版本 $rv_{idx} = 1$，则该 TB 是去使能的；否则该 TB 是使能的。（见 36.213 的 7.1.7.2 节）

TB 与 codeword 的对应关系如下：

通常情况下，TB 1 与 codeword 0 对应，TB 2 与 codeword 1 对应。但以下情况较为特殊。

对于 DCI format 2/2A（用于支持空分复用，DCI format 2B/2C 没有该字段），有一个 1 比特的字段：Transport block to codeword swap flag。该字段的作用在于平均 2 个 codeword 间的信道质量。

如果 2 个 TB 都使能，则 TB 与 codeword 的对应关系见 36.212 的 Table 5.3.3.1.5-1。

Table 5.3.3.1.5-1: Transport block to codeword mapping (two transport blocks enabled).

transport block to codeword swap flag value	codeword 0 (enabled)	codeword 1 (enabled)
0	transport block 1	transport block 2
1	transport block 2	transport block 1

如果 2 个 TB 中的一个是去使能的（对应前面介绍的情况二），则 Transport block to codeword swap flag 字段是预留但无意义的，此时 TB 与 codeword 的对应关系见 36.212 的 Table 5.3.3.1.5-2。从下表可以看出，使能的那个 TB 都只对应 codeword 0。

Table 5.3.3.1.5-2: Transport block to codeword mapping (one transport block enabled).

transport block 1	transport block 2	codeword 0 (enabled)	codeword 1 (disabled)
enabled	disabled	transport block 1	-
disabled	enabled	transport block 2	-

13.2 自适应、异步

下行使用自适应 HARQ（adaptive HARQ）意味着可以改变重传所使用的 PRB 资源以及 MCS，也意味着 eNodeB 需要重新调度下行重传。

下行使用异步 HARQ（asynchronous HARQ）意味着重传可以发生在任一时刻，也意味着能以任意顺序使用 HARQ process。这也是下行 DCI 中需要包含 HARQ process number 的原因，eNodeB 需要明确地告诉 UE 当前传输所对应的 HARQ process。

13.2.1 判断新传/重传

UE 会根据对应 HARQ process 的 NDI 值是否发生翻转来决定收到的数据是新传还是重传。

UE 会对接收到的下行数据进行 CRC 校验，以决定是回复 ACK 还是 NACK。如果 CRC 校验成功，则 UE 会回复 ACK；如果 CRC 校验失败，则 UE 会回复 NACK。

eNodeB 根据 UE 反馈的 ACK/NACK 来决定是进行重传还是新传。

再次强调，对于 HARQ，只有在对应同一 TB，也即同一 HARQ process 的基础上讨论“初传”、“重传”才有意义。

对于动态调度（使用 TC-RNTI/C-RNTI 加扰）的传输：

- 如果 UE 收到使用 TC-RNTI 加扰的 PDCCH，且该 DCI 是对应该 TC-RNTI 的第一个下行 DCI，则认为对应 HARQ process 的 NDI 发生翻转，且对应的 PDSCH 传输是新传；
- 如果 UE 收到使用 C-RNTI 加扰的 PDCCH，且对应的 HARQ process 的 HARQ buffer 是空的（即第一次使用该 HARQ process 来传输），则对应的 PDSCH 传输是新传；
- 如果 UE 收到使用 TC-RNTI/C-RNTI 加扰的 PDCCH，且该 DCI 携带的 NDI 字段与对应 HARQ process 的前一次传输相比发生了翻转，则对应的 PDSCH 传输是新传；
- 如果 UE 收到使用 TC-RNTI/C-RNTI 加扰的 PDCCH，且该 DCI 携带的 NDI 字段与对应 HARQ process 的前一次传输相比没有翻转，则对应的 PDSCH 传输是重传。

注：在下行传输中，只有随机接入过程中的 Msg 4 可能使用 TC-RNTI 加扰。具体参见 21.3 节关于“步骤四：eNodeB 发送 contention resolution”的介绍。

对于 SPS 的传输（关于 SPS 的介绍，见第 24 章）：

- 如果 UE 在 PCell 上收到使用 SPS C-RNTI 加扰的指示下行 SPS 激活的 PDCCH，且该 DCI 携带的 NDI 字段为 0，则认为对应 HARQ process 的 NDI 发生翻转，且对应的 PDSCH 传输是新传；
- 如果 UE 在 PCell 上收到使用 SPS C-RNTI 加扰的指示下行 SPS 激活的 PDCCH，且该 DCI 携带的 NDI 字段为 1，则认为对应 HARQ process 的 NDI 没有翻转，且对应的 PDSCH 传输是重传；
- 如果是下行 SPS 激活后的周期性 PDSCH 传输，则认为对应 HARQ process 的 NDI 发生翻转，该 PDSCH 传输是新传；
- 如果 UE 收到使用 C-RNTI 加扰的 PDCCH，且同一 HARQ process 的前一次收到的 PDCCH 是使用 SPS C-RNTI 加扰的（即前面介绍的 3 种情况），则认为对应 HARQ process 的 NDI 发生翻转（此时会忽略使用 C-RNTI 加扰的 PDCCH 中携带的 NDI 字段的值），且对应的 PDSCH 传输是新传。

从 36.321 的 5.3.1 节可以看出，UE 优先尝试接收使用 TC-RNTI/C-RNTI 加扰的 PDCCH；如果没有接收到，才会去尝试接收使用 SPS C-RNTI 加扰的 PDCCH。

注：由于 RAR 不支持 HARQ，所以其传输必定为新传，并且没有重传的概念。

本节的结论是对 36.321 的 5.3 节进行分析后得出的。

13.2.2 下行 HARQ 的 timing 关系

在下行传输中，PDSCH 总是伴随着 PDCCH 在同一个下行子帧上传输（配置的周期性 SPS 子帧上的 PDSCH 传输例外）。

下行 HARQ 的初传和重传、重传和重传之间没有固定的 timing 关系，但是 PDSCH 传输与对应的 ACK/NACK 之间有固定的 timing 关系。

13.2.2.1 FDD

对于 FDD 而言，在如果 UE 在下行子帧 n – 4 上检测到 PDSCH 传输，或检测到一个指示下行 SPS 释放的 PDCCH，则 UE 会在上行子帧 n 回复 ACK/NACK。

13.2.2.2 TDD

与 FDD 下一个上行子帧发送的 ACK/NACK 只对应一个下行子帧发送的 PDSCH 不同，在 TDD 中，多个下行子帧发送的 PDSCH 可能需要在同一个上行子帧回复 ACK/NACK。

对于 TDD 而言，如果 UE 在下行子帧 n – k 上检测到 PDSCH 传输，或检测到一个指示下行 SPS 释放的 PDCCH，则 UE 会在上行子帧 n 回复 ACK/NACK。其中 $k \in K$ ，且 k 在 36.213 的 Table 10.1.3.1-1 中定义。

Table 10.1.3.1-1: Downlink association set index $K: \{k_0, k_1, \cdots k_{M-1}\}$ for TDD

UL-DL Configuration	Subframe *n*									
	0	1	2	3	4	5	6	7	8	9
0	-	-	6	-	4	-	-	6	-	4
1	-	-	7, 6	4	-	-	-	7, 6	4	-
2	-	-	8, 7, 4, 6	-	-	-	-	8, 7, 4, 6	-	-
3	-	-	7, 6, 11	6, 5	5, 4	-	-	-	-	-
4	-	-	12, 8, 7, 11	6, 5, 4, 7	-	-	-	-	-	-
5	-	-	13, 12, 9, 8, 7, 5, 4, 11, 6	-	-	-	-	-	-	-
6	-	-	7	7	5	-	-	7	7	-

从 Table 10.1.3.1-1 可以看出，多个下行子帧发送的 PDSCH 可能在同一上行子帧回复 ACK/NACK。同一个上行子帧对应的所有的下行子帧组成了集合 K，集合中共有 M 个元素，K 中的每一个元素称为一个 k。不同的下行子帧对应不同的 k，按照表中出现的先后顺序依次标注为

$k_0, k_1, ..., k_{M-1}$，每个元素的下标记为 m。这里我们将集合 K 中所有的下行子帧称之为一个“HARQ 反馈窗口”。注意：这里介绍的概念针对特定的 TDD 上下行配置的某个特定上行子帧才有意义。

例如：TDD 1 下，上行子帧 2（对应子帧 n）用于回复 eNodeB 在下行子帧 n-7 和 n-6（这 2 个下行子帧组成一个 HARQ 反馈窗口）发送的 PDSCH 的 ACK/NACK，其对应的集合 K 为{7, 6}，M 值为 2。其中 $k_0 = 7, k_1 = 6$，k_0 对应的 m=0，k_1 对应 m=1。

TDD 2 下，上行子帧 7（对应子帧 n）用于回复 eNodeB 在下行子帧 n-8、n-7、n-4 和 n-6（这 4 个下行子帧组成一个 HARQ 反馈窗口）发送的 PDSCH 的 ACK/NACK，其对应的集合 K 为{8, 7, 4, 6}，，M 值为 4。其中 $k_0 = 8, k_1 = 7, k_2 = 4, k_3 = 6$，$k_0$ 对应的 m=0，k_1 对应 m=1，k_2 对应的 m=2，k_3 对应 m=3。

注意：3GPP 在介绍下行 HARQ 时，经常会用到这些概念，在这里做统一介绍，后续不再特别说明。

图 13-2 给出了 TDD 1 和 2 下，PDSCH 传输与对应的 ACK/NACK 之间的 timing 关系。

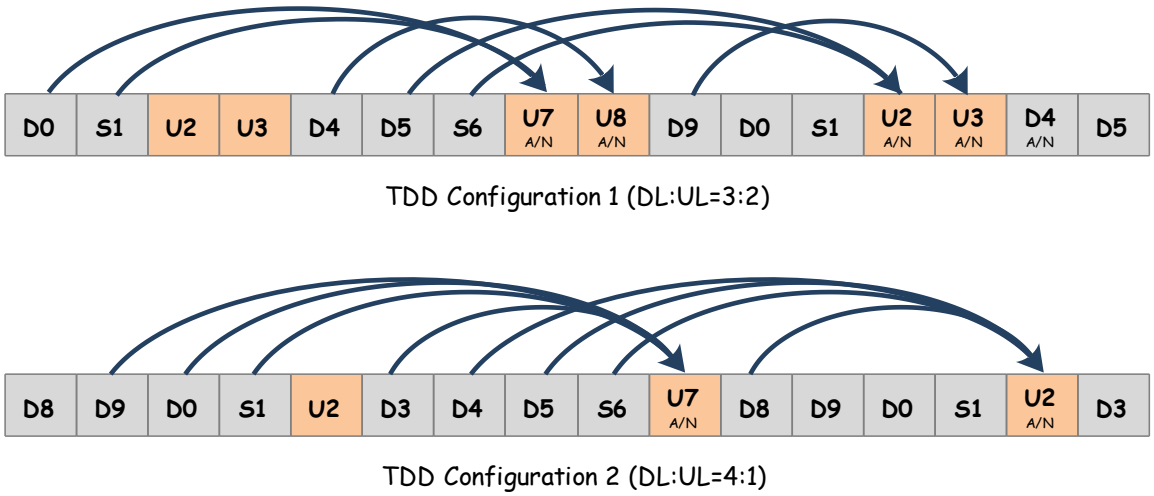
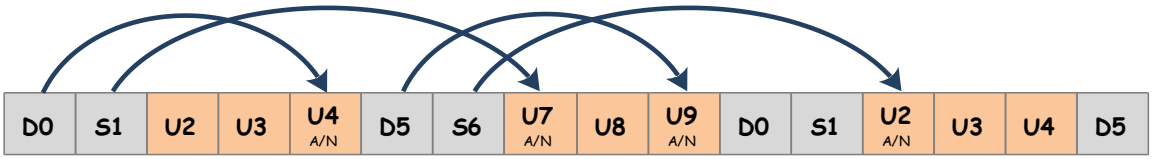


图 13-2：TDD 1/2 下，PDSCH 传输与对应的 ACK/NACK 之间的 timing 关系

图 13-3 给出了 TDD 0 下，PDSCH 传输与对应的 ACK/NACK 之间的 timing 关系。



TDD Configuration 0 (DL:UL=2:3)

图 13-3：TDD 0 下，PDSCH 传输与对应的 ACK/NACK 之间的 timing 关系

注意：下行 HARQ 是异步的，重传与前一次传输之间、重传与前一传输的 ACK/NACK 之间没有固定的 timing 关系。但某一次传输与其对应的 ACK/NACK 之间是有固定的 timing 关系的。

13.3 DAI

PDCCH 传输并不是 100%可靠，UE 可能丢失某些下行 DCI，从而导致 HARQ 出错率变高。对于 TDD 而言，当 HARQ 反馈窗口内的某些下行 DCI 丢失时，UE 可能会错误地反馈 ACK。例如：在 HARQ bundling 下，eNodeB 在某个下行子帧发送了 PDCCH 和对应 PDSCH，但 UE 都没检测到，那么 UE 会基于 HARQ 反馈窗口内的其它下行子帧来做 HARQ bundling 处理。如果其它下行子帧的数据都成功接收，这时就可能错误的反馈 ACK。但实际上在 HARQ bundling 中，只要有一个子帧的下行 DCI 丢失，UE 就应该反馈 NACK。

为了避免这类问题，PDCCH 中引入了一个 Downlink Assignment Index (DAI)字段，用于告诉 UE 在 HARQ 反馈窗口内有多少个子帧包含下行传输。在 HARQ bundling（见 13.4.1 节）中，这可以帮助 UE 检测到是否丢失了下行 DCI，并避免出现丢失了某些下行 DCI 却反馈 ACK 的情况。在 HARQ multiplexing（见 13.4.2 节）中，DAI 可以帮助 UE 确定需要反馈多少比特的 ACK/NACK 信息。

FDD 中，一个上行子帧只需要对一个下行子帧的 PDSCH 回应 ACK/NACK，因此没有 DAI 的概念。

在 TDD 0 中，一个系统帧内的上行子帧数多于下行子帧数，一个上行子帧只需要对一个下行子帧的 PDSCH 回应 ACK/NACK，因此也没有 DAI 的概念。

只有 TDD 1~6 支持 DAI。类似的原因，只有 TDD 1~6 支持 HARQ bundling 和 HARQ multiplexing。

TDD 1~6，UE 只配置了一个 serving cell，且没有配置 PUCCH format 3 的情况下， UE 在子帧 $n-k'$ 收到了 DCI format 0/4（ k' 在 36.213 的 Table 7.3-Y 中定义，见下表），我们使用该 DCI 中的 DAI 字段查 36.213 的 Table 7.3-X 得到的值，称为 V_{DAI}^{UL}。

V_{DAI}^{UL} 表示在所有的 n - k 的子帧集合中（即 HARQ 反馈窗口内）发送了 PDSCH 的子帧数（对应本章开头介绍的场景 1、2.a、2.c）和指示了下行 SPS 释放的 PDCCH 的子帧数（对应本章开头介绍的场景 2.b）的总和。

注意：这里统计的是“PDSCH 子帧数+指示下行 SPS 释放的 PDCCH 子帧数”。这里的 PDSCH 传输可能有对应的 PDCCH（对应 2 种场景：动态调度的 PDCCH + 指示下行 SPS 激活的 PDCCH，该 PDCCH 对应一个 PDSCH），也可能没有对应的 PDCCH（在配置的周期性 SPS 子帧上发送的 PDSCH）。如果在 n – k 的子帧集合中，既没有 PDSCH 传输，也没有指示下行 SPS 释放的 PDCCH，也即没有下行数据传输，则 UE 期待收到的 DCI format 0/4 中的 DAI 字段对应的 V_{DAI}^{UL} 设置为 4。DCI format 0/4 中的 DAI 字段与 V_{DAI}^{UL} 的对应关系见 36.213 的 Table7.3-X。（见 36.213 的 7.3 节）

Table 7.3-X: Value of Downlink Assignment Index

DAI MSB, LSB	V_{DAI}^{UL} or V_{DAI}^{DL}	Number of subframes with PDSCH transmission and with PDCCH indicating DL SPS release
0,0	1	1 or 5 or 9
0,1	2	2 or 6
1,0	3	3 or 7
1,1	4	0 or 4 or 8

Table 7.3-Y: Uplink association index *k*' for TDD

TDD UL/DL Configuration	subframe number *n*									
	0	1	2	3	4	5	6	7	8	9
1			6	4				6	4	
2			4					4		
3			4	4	4					
4			4	4						
5			4							
6			7	7	5			7	7	

注：比较 36.213 的 Table 7.3-Y 和 Table 10.1.3.1-1 可以看出，DCI format 0/4 发送时所在的下行子帧（对应 36.213 的 Table 7.3-Y）一定对应 36.213 的 Table 10.1.3.1-1 中的 HARQ 反馈窗口里的最后一个下行子帧，或比该下行子帧还要靠后。因此，eNodeB 发送 DCI format 0/4 时，就已经知道了 HARQ 反馈窗口里有多少个子帧发送了下行数据。

TDD 1~6 下，当 UE 配置了多于一个 serving cell，或者当 UE 只配置了一个 serving cell 并同时配置 PUCCH format 3 时，UE 在子帧 $n-k'$ 收到了 DCI format 0/4（ k' 在 36.213 的 Table 7.3-Y 中定义），我们使用该 DCI 中的 DAI 字段查 36.213 的 Table 7.3-Z 得到的值，称为 W_{DAI}^{UL}。

3GPP 中并未说明 eNodeB 侧如何设置 W_{DAI}^{UL} 的值。实际上 W_{DAI}^{UL} 表示在所有的 n - k 的子帧集合中，发送了最多“PDSCH 和指示了下行 SPS 释放的 PDCCH”的那个 serving cell 上发送了下行传输的子帧数。也就是说，W_{DAI}^{UL} 取所有 serving cell 中的最大 V_{DAI}^{UL} 值（这里假定所有的 serving cell 都发送了 V_{DAI}^{UL}。实际上对于某个 UE 来说，在所有 serving cell、同一子帧收到的 DCI format 0/4 中的 DAI 字段应该配置相同的值）。举个例子，假设 UE 配置了 2 个 serving cell，则有 $W_{DAI}^{UL} = \max(V_{DAI,0}^{UL}, V_{DAI,1}^{UL})$。其中 $V_{DAI,c}^{UL}$ 表示第 c 个 serving cell 的 V_{DAI}^{UL} 值。

这里的 PDSCH 传输可能有对应的 PDCCH，也可能没有对应的 PDCCH。如果在 n – k 的子帧集合中，既没有 PDSCH 传输，也没有指示下行 SPS 释放的 PDCCH（也即没有下行数据传输），则中的 DAI 字段对应的 W_{DAI}^{UL} 设置为 4。DCI format 0/4 中的 DAI 字段与 W_{DAI}^{UL} 的对应关系见 36.213 的 Table7.3-Z。（见 36.213 的 7.3 节）

Table 7.3-Z: Value of W_{DAI}^{UL} determined by the DAI field in DCI format 0/4

DAI MSB, LSB	W_{DAI}^{UL}
0,0	1
0,1	2
1,0	3
1,1	4

对于同一个 UE 而言，使用上行载波聚合时，在不同 serving cell 上收到的调度同一个上行子帧的 UL grant 必须携带相同的 W_{DAI}^{UL} 值。（见参考资料[2]的介绍）

以图 13-4 为例，假设 UE 配置了 2 个 serving cell，并且 2 个 serving cell 都使用 TDD 2，在反馈 ACK/NACK 的上行子帧 7 对应的 M = 4。对应 PCell，在下行子帧 9，0，1 上有下行传输（其 $V_{DAI}^{UL}=3$）；对应 SCell，在下行子帧 0，1 上有下行传输（其 $V_{DAI}^{UL}=2$），此时在下行子帧 3 上发送的 DCI format 0/4 上的 DAI 字段为“10”，对应 $W_{DAI}^{UL}=3$。（如果在 SCell 也需要发送 DCI format 0/4，则其 DAI 字段也必须设置为“10”）

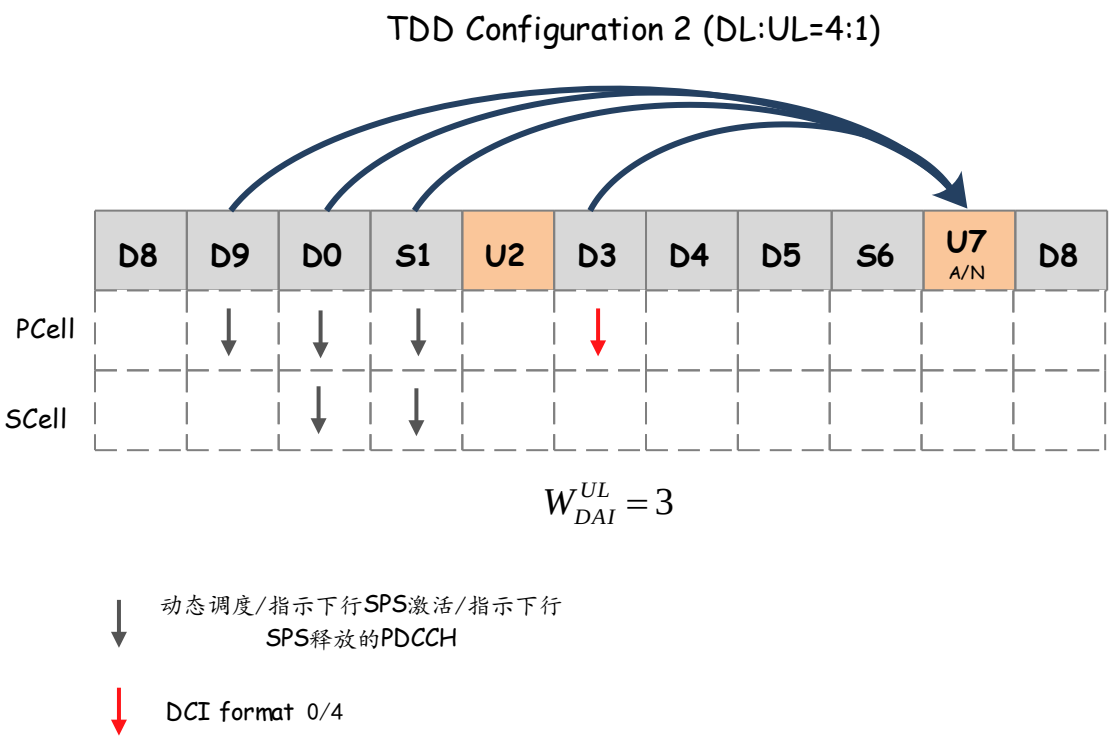


图 13-4：TDD 2 下，W_{DAI}^{UL} 举例

在 TDD 1~6 下，DCI format 1/1A/1B/1D/2/2A/2B/2C 中的 DAI 字段指示了每个配置的 serving cell 到当前子帧（该子帧位于 HARQ 反馈窗口内）为止，发送了多少个带 PDSCH 传输的 PDCCH（对应本章开头介绍的场景 1 和 2a）和指示下行 SPS 释放的 PDCCH（对应本章开头介绍的场景 2b），该值会在子帧之间更新。也就是说，该 DAI 值指示了到当前子帧为止，在某个 serving cell 的 HARQ 反馈窗口内发送了多少个下行 DCI。$V_{DAI,c}^{DL}$ 为 UE 在 serving cell c 的子帧 $n-k_m$ 检测到的 DCI format 1/1A/1B/1D/2/2A/2B/2C 中的 DAI 字段查 36.213 的 Table 7.3-X 得到的值。其中 k_m 为在集合 K 中的检测到 DCI format 1/1A/1B/1D/2/2A/2B/2C 的最小值，也即 k_m 对应集合 K 中最后一个检测到 DCI format 1/1A/1B/1D/2/2A/2B/2C 的下行子帧。如果 UE 只配置了一个 serving cell，$V_{DAI,c}^{DL}$ 中的 c 可以忽略。注意：这里统计的是 PDCCH 数，并不统计本章开头介绍的场景 2c，并且这是一个累加值。

对所有的 TDD 配置而言，$U_{DAI,c}$ 指示了 UE 在 serving cell c 的所有 n - k（其中 $k \in K$）子帧中，接收到的所有带 PDSCH 传输的下行 PDCCH 数（对应本章开头介绍的场景 1 和 2.a）和指示下行 SPS 释放的 PDCCH 数（对应本章开头介绍的场景 2.b）之和。注意：这里统计的是 PDCCH 数，并不统计场景 2c。

对所有的 TDD 配置而言，N_{SPS}指示了 UE 在所有 n - k（其中 $k \in K$ ）子帧中，接收到的没有 PDCCH 的 PDSCH 传输数（对应本章开头介绍的场景 2.c）。由于只有 PCell 支持 SPS，所以这里没有 serving cell c 的要求。又由于 SPS 的周期最小为 10 ms，而 n - k（其中 $k \in K$ ）必然落在同一个 10 ms 范围内，也就是说，在一个 HARQ 反馈窗口内，没有 PDCCH 的 PDSCH 传输至多可能出现 1 次。因此，N_{SPS}的值只能为 0 或 1。注意：这里统计的是 PDSCH 数。

表 13-2：DAI 相关概念

符号	含义	应用场景	来源
V_{DAI}^{UL}	PDSCH 子帧数 + 指示下行 SPS 释放的 PDCCH 子帧数 （eNodeB 告诉 UE 的）	TDD 1~6，UE 只配置了一个 serving cell，且没有配置 PUCCH format 3 的情况	DCI format 0/4 （上行 DCI）
W_{DAI}^{UL}	$W_{DAI}^{UL} = \max\{V_{DAI,c}^{UL}\}$ （eNodeB 告诉 UE 的）	在 TDD 1~6 下，当 UE 配置了多于一个 serving cell，或者当 UE 只配置了一个 serving cell 并同时配置 PUCCH format 3 时	DCI format 0/4 （上行 DCI）
$V_{DAI,c}^{DL}$	带 PDSCH 的 PDCCH 数 + 指示下行 SPS 释放的 PDCCH （eNodeB 告诉 UE 的）	TDD 1~6（每个 serving cell 都有这样一个值）	DCI format 1/1A/1B/1D/2/2A/2B/2C（下行 DCI）
$U_{DAI,c}$	带 PDSCH 的 PDCCH 数 + 指示下行 SPS 释放的 PDCCH （UE 接收时统计的，而不是 eNodeB 告诉 UE 的）	所有的 TDD 配置（每个 serving cell 都有这样一个值）	UE 根据接收到的 PDCCH 进行统计
N_{SPS}	没有 PDCCH 的 PDSCH 传输数 （UE 接收时统计的，而不是 eNodeB 告诉 UE 的）	所有的 TDD 配置	UE 根据接收到的没有 PDCCH 的 PDSCH 进行统计

注：（1）表 13-2 中介绍的概念针对同一个 HARQ 反馈窗口才有意义；（2）V_{DAI}^{UL}反映了 eNodeB 在 HARQ 反馈窗口内有多少个子帧发送了 PDSCH 和指示下行 SPS 释放的 PDCCH；W_{DAI}^{UL}反映了 eNodeB 在 HARQ 反馈窗口内发送了最多“PDSCH 和指示下行 SPS 释放的 PDCCH”的那个 serving cell 的发送了下行传输的子帧数；$U_{DAI,c} + N_{SPS}$反映了 UE 在 serving cell c 的 HARQ 反馈窗口内实际在多少个子帧接收到了 PDSCH 和指示下行 SPS 释放的 PDCCH。（3）UE 可能丢失 eNodeB 在 HARQ 反馈窗口内发送的最后一个下行 DCI，在这种情况下，UE 并不能根据$V_{DAI,c}^{DL}$就判定 eNodeB 在某个 serving cell 的 HARQ 反馈窗口内有多少个子帧发送了 PDSCH 和指示下行 SPS 释放的 PDCCH。

图 13-5 是 TDD 2 下，使用上行子帧 2 回复 ACK/NACK 的一些例子（不包含W_{DAI}^{UL}），这些例子中都假定 UE 没有丢失任何下行传输。结合表 13-2，大家可以对 DAI 的概念有更加深入的理解。

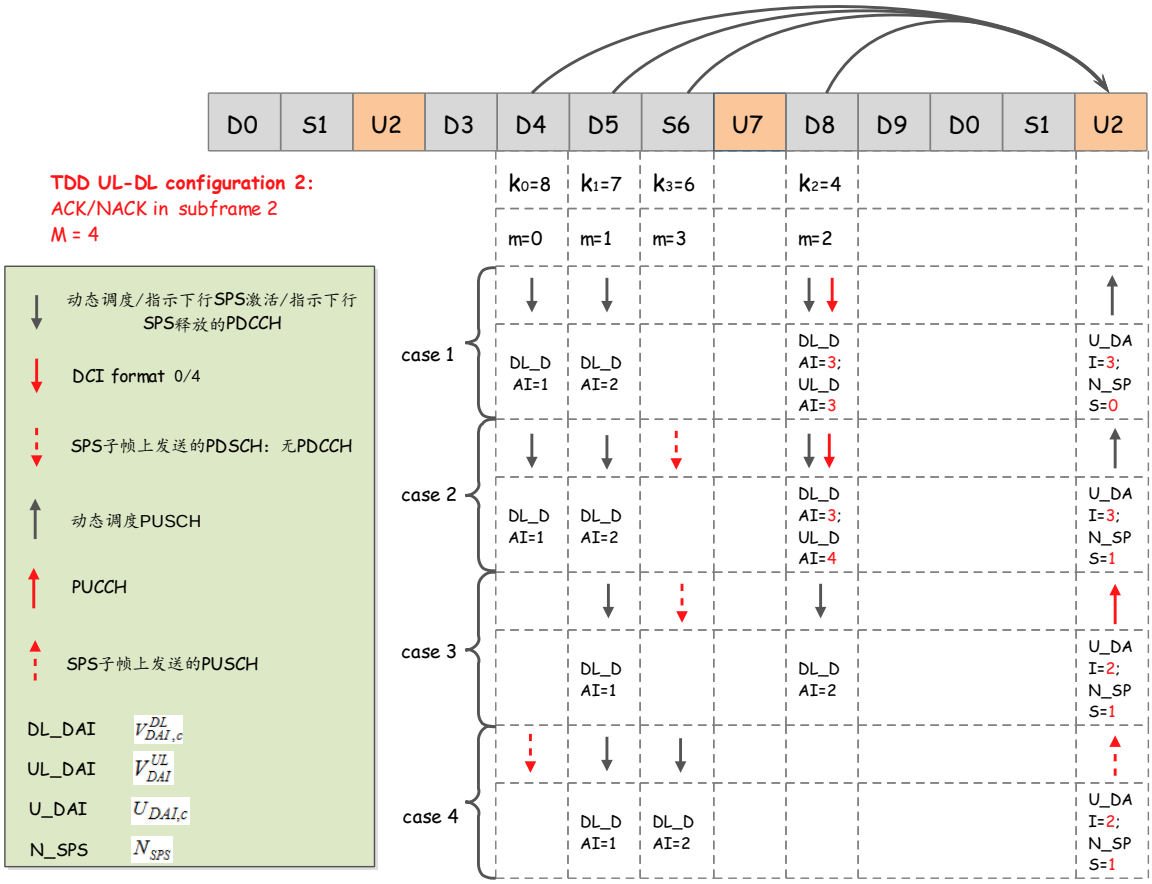


图 13-5：TDD 2 下，关于 DAI 的一些例子

【参考资料】

[1] TS 36.213 的 7.3 节 UE procedure for reporting HARQ-ACK

[2] “R1-112325, Draft CR for clarification on UL DAI value for multiple UL grants in TDD”, LG Electronics

13.4 HARQ bundling 和 HARQ multiplexing

对于 TDD 1~6，如果 UE 不支持聚合超过 1 个 serving cell，即没有使用载波聚合的情况下，RRC 层可以给 UE 配置 2 种 ACK/NACK 反馈模式。（见 36.213 的 10.1.3 节）

- HARQ-ACK bundling（或称为 HARQ bundling、ACK/NACK bundling）
- HARQ-ACK multiplexing（或称为 HARQ multiplexing、ACK/NACK multiplexing）

UE 使用 HARQ bundling 还是 HARQ multiplexing 是通过 *PUCCH-ConfigDedicated* 的 *tdd-AckNackFeedbackMode* 字段来配置的。该字段对 PUCCH 和 PUSCH 上回复的 ACK/NACK 均有效。

可以看出，只有 TDD 1~6 且非载波聚合的情况下，才有 HARQ bundling 和 HARQ multiplexing 的概念。

13.4.1 HARQ bundling

HARQ bundling：将同一 serving cell 的多个下行子帧的对应同一 codeword 的 ACK/NACK 做逻辑与（logical AND）操作，最终得到 1 比特（非空分复用，使用 PUCCH format 1a）或 2 比特（空分复用，使用 PUCCH format 1b）的 ACK/NACK 信息。做 HARQ bundling 操作的是属于同一 HARQ 反馈窗口内的$U_{DAI} + N_{SPS}$个 PDSCH 传输和指示下行 SPS 释放 PDCCH 的子帧，那些没有发送 PDCCH/PDSCH 的子帧是不考虑在内的。

注意：HARQ bundling 只用于单个 cell 的场景。也就是说，载波聚合下并不使用 HARQ bundling。

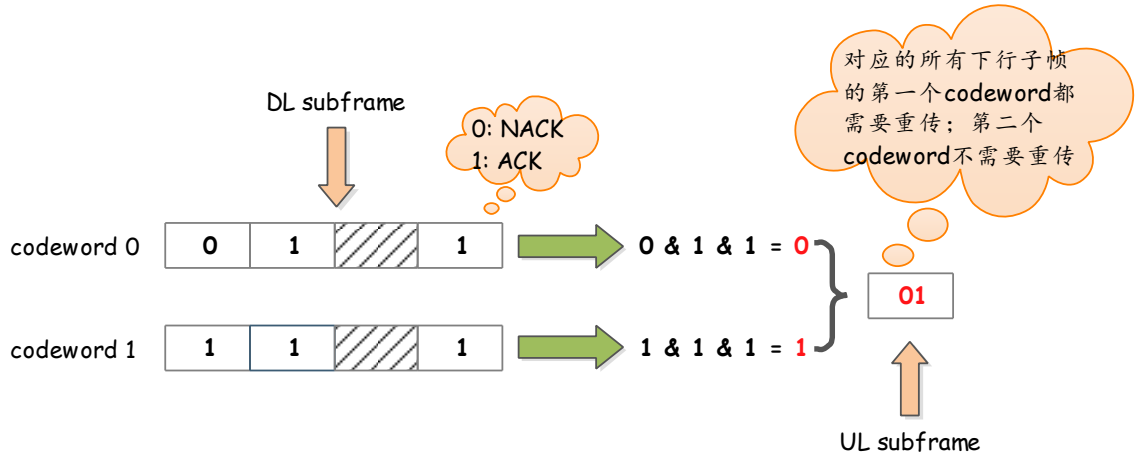


图 13-6：HARQ bundling 的一个例子

图 13-6 是 HARQ bundling 的一个例子，空分复用的 UE 在 4 个下行子帧接收到的 PDSCH 对应在同一个上行子帧回 ACK/NACK。eNodeB 在第 1、2、4 个子帧发送了 PDSCH，对应 codeword 0，HARQ 反馈分别为 NACK/ACK/ACK，则有 0 & 1 & 1 = 0；对应 codeword 1， HARQ 反馈分别为 ACK/ACK/ACK，则有 1 & 1 & 1 = 1。则 UE 在对应的 PUCCH/PUSCH 上反馈 ACK/NACK 时，会携带 2 比特的信息 01，即对应的所有下行子帧的第一个 codeword 都需要重传，而第二个 codeword 不需要重传。

假定图 13-6 的其他条件不变，eNodeB 在第 3 个子帧（对应阴影部分）也发送了 PDCCH 和 PDSCH，但 UE 没有检测到，此时 UE 应该反馈 00。但由于 UE 不知道 eNodeB 是否在第三个子帧发送了数据，根据 HARQ bundling 的原理，UE 还是会反馈 01，这就不正确了。为了解决这类问题，LTE 提出了 DAI 的概念，在后面的章节中，我们会看到 UE 如何通过使用 DAI 来判断是否丢失了某些传输。

注意：协议中的提到 “spatial HARQ-ACK bundling（或 spatially-bundled）”与“HARQ-ACK bundling”不是同一个概念。

“spatial HARQ-ACK bundling（或 spatially-bundled）”是指将同一小区，同一下行子帧发送的 2 个 codeword 对应的 2 个 ACK/NACK 做逻辑与（logical AND）处理，得到 1 比特 的 ACK/NACK 信息。它主要应用于 HARQ multiplexing、PUCCH format 1b with channel selection 和 PUCCH format 3。spatial HARQ-ACK bundling 是对单个下行子帧的处理，而 HARQ bundling/multiplexing 是对整个 HARQ 反馈窗口的处理。

使用 HARQ bundling 的场景有 3 种（见 36.213 的 7.3 节）：

（1）*tdd-AckNackFeedbackMode* 设置为 *bundling*；

（2）*tdd-AckNackFeedbackMode* 设置为 *multiplexing*，但用于反馈 ACK/NACK 的上行子帧对应的 M 值为 1（M 的取值见 36.213 的 Table 10.1.3.1-1），即一个上行子帧对应一个下行子帧的场景。这种场景至多需要反馈 2 比特（空分复用）的 ACK/NACK，如果使用 HARQ multiplexing，至多只能携带 1 比特的信息，反而不如不使用 HARQ bundling，而将 2 比特的信息都反馈给 eNodeB（使用 PUCCH format 1b）。

（3）如果是 TDD 5 且 UE 不支持聚合超过 1 个 serving cell（即非载波聚合条件下），则该 UE 只支持 HARQ bundling。（见 36.213 的 10.1.3 节）

对于 HARQ bundling，当 UE 配置了 TM 3/4/8/9 并且 ACK/NACK 在 PUSCH 上传输（注意：不包含 PUCCH 上反馈 ACK/NACK 的情况），则 UE 总是假定 codeword 0 和 1 都使能，并最终生成 2 比特的 ACK/NACK 信息。如果 UE 在 HARQ 反馈窗口内只在 codeword 0 检测到 PDSCH 传输，则 UE 为 codeword 1 生成 NACK。

通过之前的介绍我们可以看出，对于 HARQ bundling，只要 UE 在 HARQ 反馈窗口内有一个 TB 没有正确接收（NACK/DTX），对应该 TB 所在的 codeword，eNodeB 就应该收到 NACK 或根本没收到 HARQ 反馈，并重传所有下行子帧对应该 codeword 的数据。

由于只需要传输 1 或 2 比特的 ACK/NACK 信息，HARQ bundling 能够提高上行控制信令的 UL coverage 和 capacity（尤其对小区边缘的 UE）。而不足之处在于 eNodeB 无法确认哪一个或几个下行子帧解码失败，只好把所有下行子帧的数据都重传一遍，这会导致吞吐量（throughput）的下降。

对于 HARQ bundling，UE 会检测在 HARQ 反馈窗口内是否有下行 DCI（即，下行分配：downlink assignment）丢失的情况。如果 UE 使用 PUSCH 来回复 ACK/NACK，还会计算一个参数 N_{bundled}。N_{bundled} 用于 36.212 的 5.2.2.6 节中与 UL-SCH 相关的信道编码。对于 TDD 0 而言，如果 UE 在子帧 n - k 收到了 PDSCH，则 N_{bundled} 为 1。其它配置下的计算方式见后续介绍。

使用 HARQ bundling 时，UE 如何判断是否丢了下行 DCI?（与协议里介绍的相比，我尽量把各种丢失下行传输的场景都考虑在内，而不仅仅是丢失下行 DCI）

场景一：在 TDD 1~6 下，UE 在子帧 n 没有发送 PUSCH（也即没有收到 DCI format 0/4，没有收到 V_{DAI}^{UL}。此时 UE 使用 PUCCH 来回复 ACK/NACK），如果 $U_{DAI} > 0$ 且 $V_{DAI}^{DL} \neq (U_{DAI} - 1) \bmod 4 + 1$，则 UE 检测到至少有一个下行 DCI 丢了。

注：“$\mathrm{mod}\,4+1$”的原因在于 HARQ 反馈窗口内的下行子帧数可能为 0~9，对照 36.213 的 Table 7.3-X，V_{DAI}^{UL} 和 V_{DAI}^{DL} 的取值范围是 1~4，所以需要做此处理。实际上，只有 TDD 5 会出现 HARQ 反馈窗口内的下行子帧数大于 4 的情况。

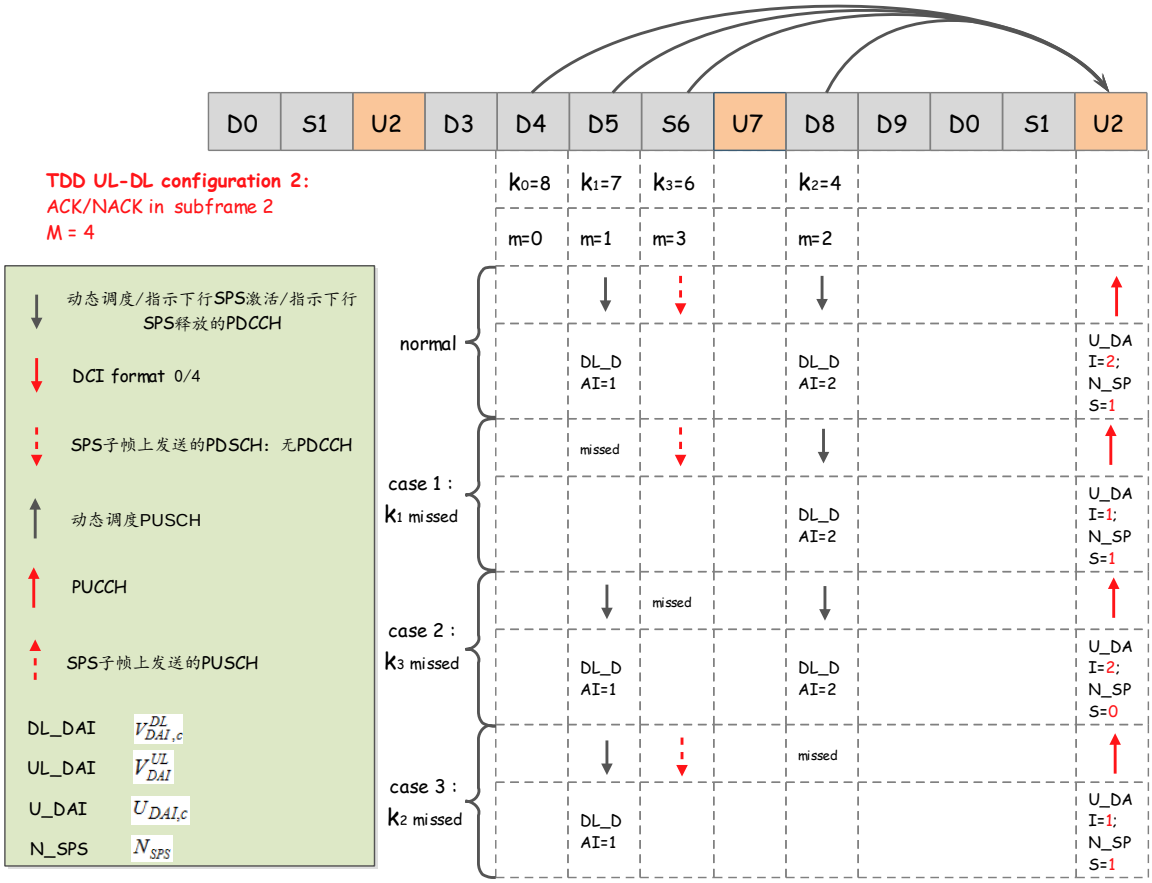


图 13-7：场景一举例

图 13-7 是场景一的几个例子。normal/case 1/case 2/case 3 条件下，eNodeB 发送的所有 PDCCH/PDSCH 都与 normal 条件下是一致的。但在 case 1/2/3 中，UE 可能丢失某些 PDCCH/PDSCH。

- **case 1**：HARQ 反馈窗口内最后一个下行 DCI 被成功接收，但之前的某些子帧内丢失了下行 DCI。此时 $U_{DAI}>0$ 且 $V_{DAI}^{DL}\neq(U_{DAI}-1)\mathrm{mod}\,4+1$，UE 能够检测到下行传输丢失，直接在 PUCCH 1 资源上回复 NACK。

- **case 2**：HARQ 反馈窗口内所有下行 DCI 都被成功接收，但丢失了周期性 SPS 子帧下发的 PDSCH。此时$U_{DAI} > 0$且$V_{DAI}^{DL} = (U_{DAI} - 1) \bmod 4 + 1$，UE 没有检测到丢失下行 DCI。UE 通过下行 SPS 的配置以及 SPS 激活的时间，知道在子帧 6 上是有 PDSCH 的，但却没有收到。而做 HARQ bundling 操作的下行子帧是包含周期性 SPS 子帧的，所以 UE 此时应该在 PUCCH 1 资源上回复 NACK。

- **case 3**：HARQ 反馈窗口内最后一个下行 DCI 没有成功接收，但之前子帧的下行传输都成功接收了。此时$U_{DAI} > 0$且$V_{DAI}^{DL} = (U_{DAI} - 1) \bmod 4 + 1$，UE 没有检测到丢失下行 DCI，所以 UE 会根据之前子帧的下行传输做 HARQ bundling 处理，并把 ACK/NACK 结果反馈给 eNodeB。但此时 UE 反馈的 ACK/NACK 信息可能是错误的， eNodeB 应该能够检测出这种错误。eNodeB 如何检测这种错误将在场景四中做详细介绍。（这种情况 UE 使用 PUCCH 1 资源来反馈 ACK/NACK）

场景二：在 TDD 1~6 下，UE 需要发送 PUSCH，且该 PUSCH 传输是由 DCI format 0/4 指定的（即动态调度的或 SPS 激活子帧调度的），如果$V_{DAI}^{UL} \neq (U_{DAI} + N_{SPS} - 1) \bmod 4 + 1$，则 UE 检测到至少有一个下行传输丢了，且会对所有的 codewords 回复 NACK，并设置$N_{\text{bundled}} = V_{DAI}^{UL} + 2$。如果没有检测到下行 DCI 丢了，则$N_{\text{bundled}} = V_{DAI}^{UL}$。如果$U_{DAI} + N_{SPS} = 0$且$V_{DAI}^{UL} = 4$（即认为没有下行传输），则 UE 不会回复 ACK/NACK。

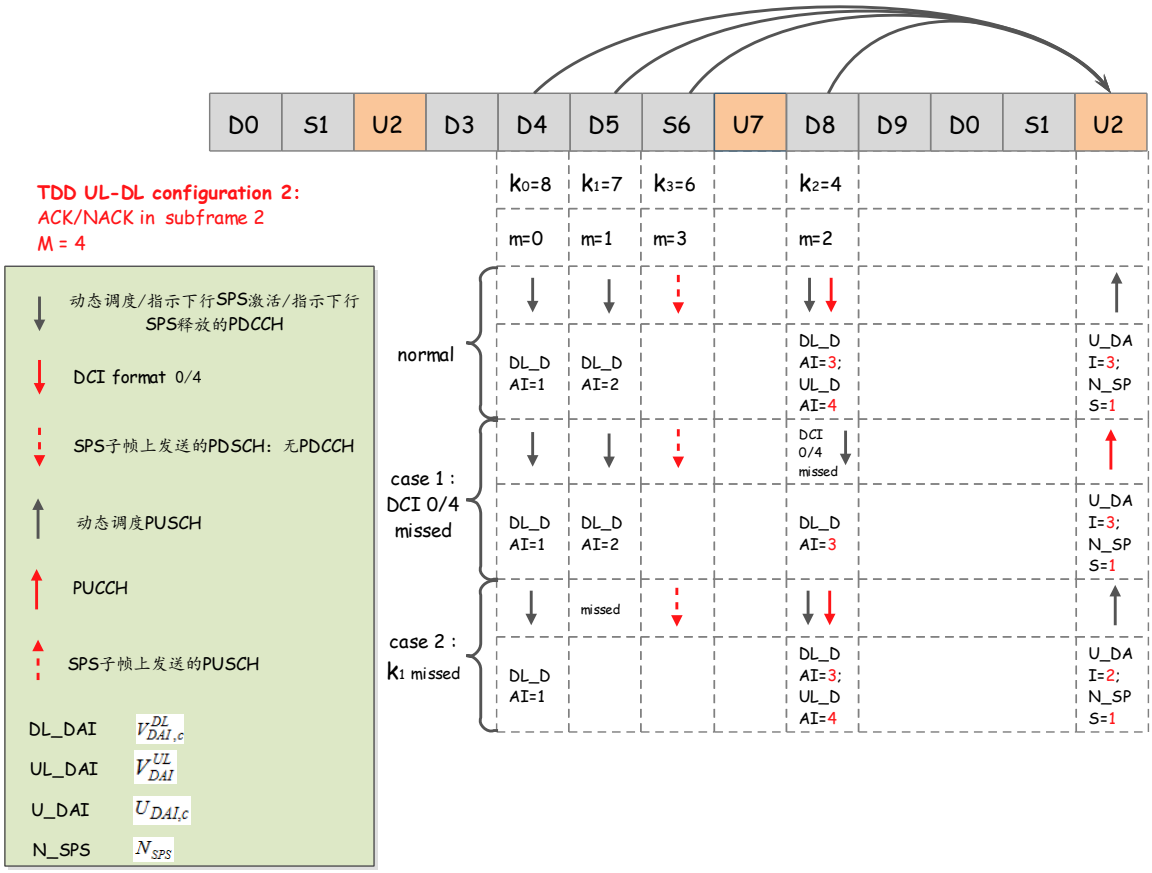


图 13-8：场景二举例

图 13-8 是场景二的一个例子。normal/case 1/case 2 条件下，eNodeB 发送的所有 PDCCH/PDSCH 都与 normal 条件下是一致的。但在 case 1/2 中，UE 可能丢失某些 PDCCH/PDSCH。

- **case 1**：DCI format 0/4 丢失。由于 UE 根本就没有收到 DCI format 0/4，所以也就不知道 eNodeB 是否发送了 DCI format 0/4。UE 会根据当前接收到的信息来反馈 ACK/NACK，此时 $V_{DAI}^{DL} = (U_{DAI} - 1) \bmod 4 + 1$，所以 UE 会在 **PUCCH** 反馈 ACK/NACK，并且 eNodeB 在期待的 PUSCH 上是接收不到 ACK/NACK。这是由于 UE 没有收到 DCI format 0/4 而导致反馈 ACK/NACK 的资源变化（PUSCH 变成了 PUCCH），而不是因为下行没有发送成功，个人觉得，eNodeB 在发现调度的 PUSCH 资源上没有收到上行数据时，应该尝试去 PUCCH 上解码 ACK/NACK。（也就是说这种情况下，eNodeB 应该按照场景一的方式来处理）

- **case 2**：DCI format 0/4 成功传输，但某些下行传输（PDCCH 或 PDSCH）丢失。无论哪个下行传输丢失，都会有$V_{DAI}^{UL} \neq (U_{DAI} + N_{SPS} - 1) \bmod 4 + 1$，此时 UE 能够检测到下行传输丢失，所以直接在 PUSCH 上对所有的 codewords 回复 NACK 即可。

场景三：在 TDD 1~6 下，UE 需要发送 PUSCH，但该 PUSCH 传输并不是由 DCI format 0/4 指定的（配置的周期性上行 SPS 子帧上发送的。在 HARQ 反馈窗口内没有收到 DCI format 0/4，即没有收到V_{DAI}^{UL}），如果$U_{DAI} > 0$且$V_{DAI}^{DL} \neq (U_{DAI} - 1) \bmod 4 + 1$，则 UE 检测到至少有一个下行 DCI 丢了，且会对所有的 codewords 回复 NACK。无论是否丢失下行 DCI，UE 都会设置 $N_{\text{bundled}} = (U_{DAI} + N_{SPS})$。如果$U_{DAI} + N_{SPS} = 0$（即认为没有下行传输），则 UE 不会回复 ACK/NACK。

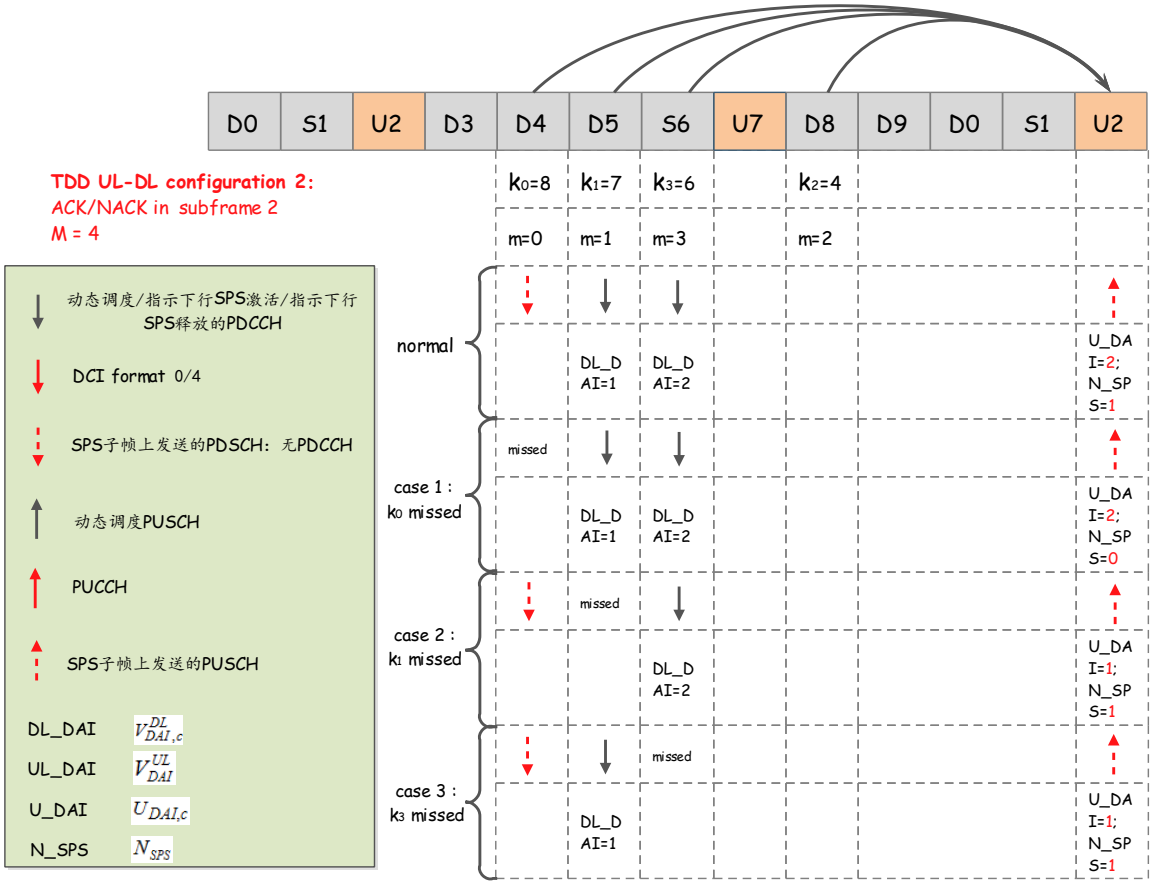


图 13-9：场景三举例

图 13-9 是场景三的一个例子。normal/case 1/case 2/case 3 条件下，eNodeB 发送的所有 PDCCH/PDSCH 都与 normal 条件下是一致的。但在 case 1/2/3 中，UE 可能丢失某些 PDCCH/PDSCH。

- **case 1**：HARQ 反馈窗口内所有下行 DCI 都被成功接收，但丢失了周期性 SPS 子帧下发的 PDSCH。此时$U_{DAI}>0$且$V_{DAI}^{DL}=(U_{DAI}-1)\bmod 4+1$，UE 没有检测到丢失下行 DCI。UE 通过下行 SPS 的配置以及 SPS 激活的时间，知道在子帧 4 上是有 PDSCH 的，但 UE 没有收到。而做 HARQ bundling 的子帧是包含周期性 SPS 子帧的，所以此时应该在 PUSCH 资源上回复 NACK。

- **case 2**：HARQ 反馈窗口内最后一个下行 DCI 被成功接收，但在之前的某些子帧上丢失了下行 DCI。此时$U_{DAI}>0$且$V_{DAI}^{DL}\neq(U_{DAI}-1)\bmod 4+1$，此时 UE 能够检测到下行传输丢失，所以直接在 PUSCH 上对所有的 codewords 回复 NACK 即可。

- **case 3**：HARQ 反馈窗口内最后一个下行 DCI 没有成功接收，但之前子帧的下行传输都成功接收了。此时$U_{DAI}>0$且$V_{DAI}^{DL}=(U_{DAI}-1)\bmod 4+1$，UE 没有检测到丢失下行 DCI，所以 UE 会根据之前子帧的下行传输做 HARQ bundling 处理，并把 ACK/NACK 结果反馈给 eNodeB。但此时 UE 反馈的 ACK/NACK 信息可能是错误的， eNodeB 应该能够检测出这种错误。eNodeB 如何检测这种错误将在场景四中做详细介绍。（这种场景 UE 使用 PUSCH 资源来反馈 ACK/NACK）

对于前面介绍的三种场景，可以将判断原则简单地归纳为：V_{DAI}^{UL}、 W_{DAI}^{UL}和$V_{DAI,c}^{DL}$是 eNodeB 告诉 UE 我发了多少东西，$U_{DAI,c}$和N_{SPS}是 UE 实际收到了多少东西，如果两者不相等，必定是 UE 丢了某些东西。

场景四：这种场景最为特殊。假如 UE 丢失的是 HARQ 反馈窗口内的最后一个（或几个，但原理是相同的）下行 DCI，但之前的下行 DCI 都成功接收了，且 UE 没有收到 DCI format 0/4。此时有$U_{DAI}>0$且$V_{DAI}^{DL}=(U_{DAI}-1)\bmod 4+1$，根据之前的介绍，UE 认为没有丢失下行 DCI，所以会依据成功接收到下行 DCI 的情况来做 HARQ bundling 处理，并将 ACK/NACK 反馈给 eNodeB（但实际上此时应该为 NACK）。如果 eNodeB 接收到了该 HARQ 反馈并且该反馈为 ACK 的话，就可能错误地处理。

第四种场景又分为两种情况：（1）UE 在 PUCCH 上回复 ACK/NACK，对应场景一的 case3；（2）UE 在周期性 SPS 子帧上的 PUSCH 上回复 ACK/NACK，对应场景三的 case3。

（1）UE 在 PUCCH 上回复 ACK/NACK，对应场景一的 case3

对于第四种场景的第（1）种情况，首先需要介绍 HARQ bundling 下使用的 PUCCH 1 资源是如何计算的。

对于 TDD HARQ bundling，或者 *tdd-AckNackFeedbackMode* 设置为 *multiplexing* 并且反馈 ACK/NACK 的上行子帧对应的 M 值为 1，UE 会使用 PUCCH format 1a/1b 来回复 ACK/NACK，其资源 $n_{\mathrm{PUCCH}}^{(1,p)}$ 通过如下方式计算：

如果在 HARQ 反馈窗口内，UE 接收到有 PDCCH 指示的 PDSCH（动态调度或下行 SPS 激活）或指示下行 SPS 释放的 PDCCH，则 UE 会选择一个 c 值，该值属于{0, 1, 2, 3}且满足 $N_c \le n_{\mathrm{CCE}} < N_{c+1}$，然后使用公式 $n_{\mathrm{PUCCH}}^{(1,p=p_0)} = (M-m-1)\cdot N_c + m\cdot N_{c+1} + n_{\mathrm{CCE}} + N_{\mathrm{PUCCH}}^{(1)}$ 来计算在天线端口 p0 上使用的 PUCCH 1 资源，使用公式 $n_{\mathrm{PUCCH}}^{(1,p=p_1)} = (M-m-1)\cdot N_c + m\cdot N_{c+1} + n_{\mathrm{CCE}} + 1 + N_{\mathrm{PUCCH}}^{(1)}$ 来计算在天线端口 p1（如果配置了 2 天线传输 PUCCH format 1a/1b 的话）上使用的 PUCCH 1 资源。其中为 $N_{\mathrm{PUCCH}}^{(1)}$ 为 RRC 层配置的小区级参数（*n1PUCCH-AN*），$N_c = \max\left\{0, \left\lfloor [N_{\mathrm{RB}}^{\mathrm{DL}}\cdot(N_{\mathrm{sc}}^{\mathrm{RB}}\cdot c - 4)]/36 \right\rfloor\right\}$，$n_{\mathrm{CCE}}$ 是 UE 在 HARQ 反馈窗口内**最后接收到**的下行 DCI 所在的第一个 CCE 的 CCE number，公式中的 m 为该子帧所对应的 m 值。

对应图 13-7 的 normal 和 case 3 的情况，M 和 $N_{\mathrm{PUCCH}}^{(1)}$ 是相同的。normal 情况下，最后一个接收到 PDCCH 的子帧为 k2，即 m=2；case 3 情况下，最后一个接收到 PDCCH 的子帧为 k1，即 m=1。而参见 36.213 的 9.1.1 节，UE 特定的搜索空间的计算与子帧号相关，所以 normal 和 case 3 情况下得到的搜索空间是不同的，同时在每个子帧会重新分配 PDCCH，这基本保证 normal 和 case 3 情况下 n_{CCE} 是不同的。进一步，所选的 N_c、N_{c+1} 可能是不同的。结合上面的 PUCCH 1 资源计算公式，我们可以认为，normal 和 case 3 情况下计算出的 PUCCH 1 资源是不同的。

我们将 normal 情况下使用的 PUCCH 1 资源称为 A，case 3 情况下使用的 PUCCH 1 资源称为 B。normal 和 case 3 情况下，eNodeB 发送的数据都是一样的，eNodeB 会期待在资源 A 上接收 ACK/NACK。但 UE 在 case 3 情况下会使用资源 B 来发送 ACK/NACK。eNodeB 是不会在资源 B 上来接收 ACK/NACK 的，那么 eNodeB 就没有收到 ACK/NACK，即认为是 DTX，会重传所有数据。这正符合我们的要求。

简单地说，对应场景一的 case 3，由于 eNodeB 检测的 PUCCH 1 资源和 UE 使用的 PUCCH 1 资源是不一致的，而 eNodeB 又不会在 UE 使用的 PUCCH 1 资源上去解码 ACK/NACK，从而导致 eNodeB 没有收到 ACK/NACK，并重传所有的数据，这符合我们的要求。

（2）UE 在 SPS 子帧上的 PUSCH 上回复 ACK/NACK，对应场景三的 case3

与场景一的 case 3 可以通过使用不同的 PUCCH 1 资源来检测丢失最后一个下行传输不同，场景三的 case 3 是使用周期性 SPS 子帧上的 PUSCH 来传输的，即资源是固定的。

但从 36.213 的 7.3 节可以看出，场景三的 case 3 多计算了一个值 $N_{\text{bundled}} = (U_{DAI} + N_{SPS})$。对于场景三的 normal 情况，$N_{\text{bundled}} = 3$；对于场景三的 case 3 情况，$N_{\text{bundled}} = 2$。

而从 36.212 的 5.2.2.6 节可以看出，N_{bundled} 与 PUSCH 上发送的 ACK/NACK 的加扰序列的选择相关。在场景三的 case 3 下，eNodeB 会使用 $N_{\text{bundled}} = 3$ 对应的加扰序列去解码 ACK/NACK，但 UE 使用的是 $N_{\text{bundled}} = 2$ 对应的加扰序列来加扰 ACK/NACK，这样 eNodeB 就无法成功解码 ACK/NACK，会重传所有的数据，这符合我们的要求。

13.4.2 HARQ multiplexing

HARQ multiplexing：将同一 serving cell 的同一下行子帧发送的 2 个 codeword 对应的 ACK/NACK 做逻辑与（logical AND）操作（**注意**：在协议中，这称为“spatially bundled”或“spatial HARQ-ACK bundling”处理，与 HARQ bundling 不是同一个概念），得到 1 比特 的 ACK/NACK 信息。做 HARQ multiplexing 操作的是属于同一 HARQ 反馈窗口内的下行子帧，UE 根据有多少个子帧发送了下行数据，或 M 值的大小，来决定发送多少比特的 ACK/NACK 信息。

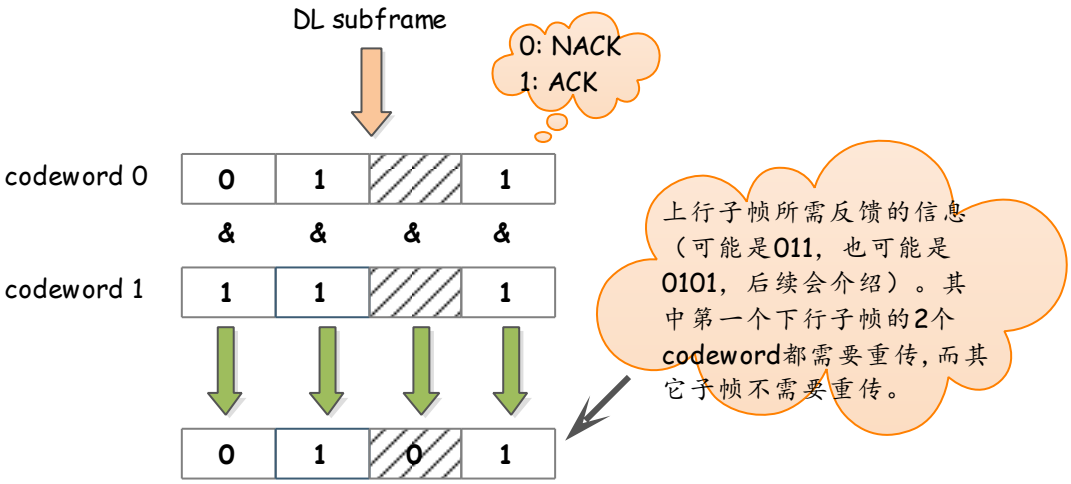


图 13-10：HARQ multiplexing 的一个例子

图 13-10 是 HARQ multiplexing 的一个例子，空分复用的 UE 在 4 个下行子帧接收到的 PDSCH 对应在同一个上行子帧回复 ACK/NACK。eNodeB 在第 1、2、4 个下行子帧发送了 PDSCH，对应第 1 个子帧，2 个 codeword 的 HARQ 反馈分别为 NACK/ACK，则有 0 & 1 = 0；对应第 2 个子帧，2 个 codeword 的 HARQ 反馈分别为 ACK/ACK，则有 1 & 1 = 1；对应第 3 个子帧，没有下行传输，则有 0 & 0 = 0（注意：这个 0 可能发送，也可能不发送，这会在后续介绍）；对应第 4 个子

帧，2 个 codeword 的 HARQ 反馈分别为 ACK/ACK，则有 1 & 1 = 1。则在对应的 PUCCH/PUSCH 上反馈 ACK/NACK 时，会携带 3 比特（或 4 比特）的 ACK/NACK 信息 011（或 0101），即第 1 个下行子帧的 2 个 codeword 都需要重传。

使用 HARQ multiplexing 的场景：*tdd-AckNackFeedbackMode* 设置为 *multplexing*，且反馈 ACK/NACK 的上行子帧对应的 M 值为大于 1（M 的取值见 36.213 的 Table 10.1.3.1-1），即一个上行子帧对应多个下行子帧的场景。（见 36.213 的 7.3 节）

如果是 TDD 5 且 UE 不支持聚合超过 1 个 serving cell（即非载波聚合条件下），则该 UE 只支持 HARQ bundling，而不支持 HARQ multiplexing。（见 36.213 的 10.1.3 节）

接下来，我们会介绍使用 HARQ multiplexing 时，UE 需要发送多少个 ACK/NACK 比特，以及每个比特与各个下行子帧的对应关系。在 PUSCH 或 PUCCH 上传输 ACK/NACK 的处理方式是不同的。

13.4.2.1 在 PUSCH 上发送 ACK/NACK（见 36.213 的 7.3 节）

如果 UE 需要在上行子帧 n 的 PUSCH 上反馈 ACK/NACK，UE 会确定需要反馈的 ACK/NACK 比特总数 O^{ACK}，以及每一个比特 o_n^{ACK} 的 ACK/NACK 值，其中 $n = 0, \ldots, O^{ACK} - 1$。

场景一：UE 需要发送 PUSCH，且该 PUSCH 是由 DCI format 0/4 指定的（动态调度，或上行 SPS 激活的 PDCCH 对应的 PUSCH），也即在 HARQ 反馈窗口内收到了 V_{DAI}^{UL}。此时 $O^{ACK} = V_{DAI}^{UL}$。

如果在下行子帧 n - k 发送的 PDSCH 有一个对应的 PDCCH，或发送的是指示下行 SPS 释放的 PDCCH，则该子帧对应的 ACK/NACK 在比特 $o_{DAI(k)-1}^{ACK}$ 上反馈，其中 *DAI(k)*是 UE 在下行子帧 n – k 检测到的 DCI format 1A/1B/1D/2/2A/2B/2C 上携带的 DAI 值。

如果 $N_{SPS} > 0$，即在周期性的下行 SPS 子帧上有 PDSCH 传输，且该 PDSCH 没有对应的 PDCCH，则该子帧对应的 ACK/NACK 在比特 $o_{O^{ACK}-1}^{ACK}$（即最后一个比特）上发送。

如果 eNodeB 在某个下行子帧发送了带 PDCCH 的 PDSCH，或发送了指示下行 SPS 释放的 PDCCH（即携带了 *DAI(k)*），但 UE 没有收到（即丢失了下行传输），则该下行子帧携带的 *DAI(k)* 对应的 ACK/NACK 比特会设置成 NACK。

如果 $U_{DAI} + N_{SPS} = 0$ 且 $V_{DAI}^{UL} = 4$（即认为没有任何下行传输），则 UE 不会反馈任何 ACK/NACK。

只要 UE 在 HARQ 反馈窗口内收到了 DCI format 0/4，即收到了V_{DAI}^{UL}，就知道 eNodeB 到底在多少个下行子帧发送了 PDSCH 或指示下行 SPS 释放的 PDCCH，因此也就知道了 *DAI(k)* 的最大值以及需要发送多少比特的 ACK/NACK 信息。而通过下行 SPS 配置以及检测到的 DCI format 1A/1B/1D/2/2A/2B/2C，UE 知道哪些下行子帧是接收到下行传输的（可能 ACK，也可能 NACK）以及该子帧对应的 ACK/NACK 比特所在的位置（通过 *DAI(k)*得到）。同时，下行数据都是 eNodeB 发送的，eNodeB 也知道 UE 应该回复多少个 ACK/NACK 比特，以及每个 ACK/NACK 比特与发送的下行子帧的对应关系。通过解码 ACK/NACK 信息，就知道哪些子帧的数据需要重传。

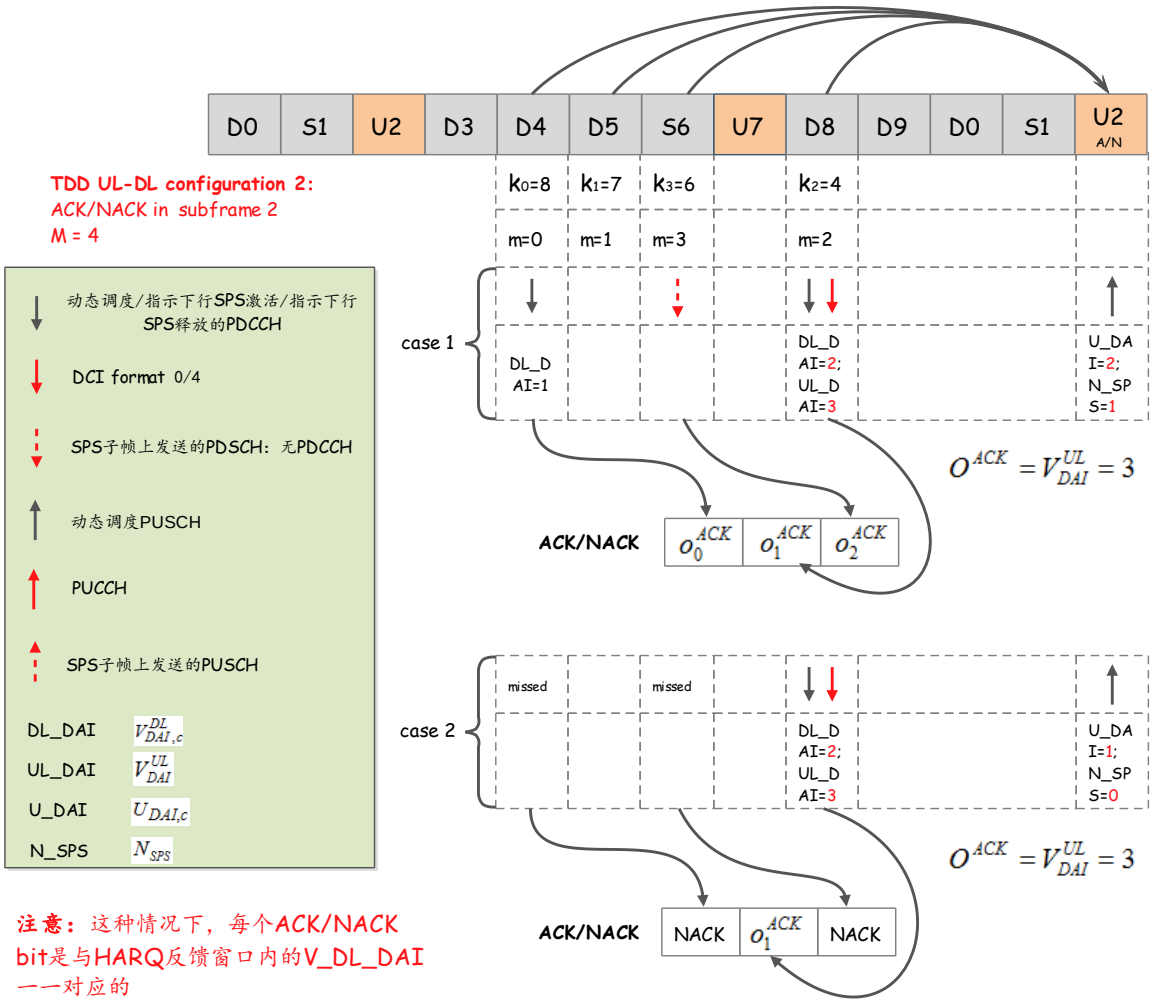


图 13-11：场景一举例

图 13-11 是场景一的一个例子。case 1/case 2 条件下，eNodeB 发送的所有 PDCCH/PDSCH 都是一致的。但在 case 2 中，UE 可能丢失某些 PDCCH/PDSCH。

- **case 1**：UE 成功接收到所有下行数据，并成功解码 DCI format 0/4 而得到V_{DAI}^{UL}。这样 UE 就确定了需要反馈的 ACK/NACK 比特数$O^{ACK}=V_{DAI}^{UL}=3$。每一比特与一个下行子帧一一对应，UE 根据解码情况在对应的比特上反馈 ACK/NACK。

- **case 2**：丢失了某些下行数据，但成功解码 DCI format 0/4 而得到V_{DAI}^{UL}。UE 首先确定了需要反馈的 ACK/NACK 比特数$O^{ACK}=V_{DAI}^{UL}=3$。每一比特与一个下行子帧一一对应，对于接收到下行数据的子帧，UE 根据解码情况在对应的比特上反馈 ACK/NACK。对于那些没有接收到下行数据的子帧（UE 其实也不确定丢失的是哪些下行子帧的数据，但这并不影响 UE 正确反馈 ACK/NACK），直接回复 NACK。

场景二：UE 需要发送 PUSCH，但该 PUSCH 传输并不是由 DCI format 0/4 指定的（即周期性上行 SPS 子帧上发送的 PUSCH，不包含上行 SPS 激活的 PDCCH 对应的 PUSCH）。也即在 HARQ 反馈窗口内没有收到 DCI format 0/4，没有收到V_{DAI}^{UL}。由于 UE 无法获知 eNodeB 到底发送了多少个下行传输，也不知道在 HARQ 反馈窗口内是否丢失了下行传输，所以它会把 HARQ 反馈窗口内的每个下行子帧（不管该子帧是否发送了数据，没有接收到数据的比特直接设置为 NACK）的 ACK/NACK 都反馈给 eNodeB，eNodeB 只要在它关心的比特（对应发送了下行数据的子帧）上去检测 ACK/NACK 即可。所以此时$O^{ACK}=M$。

每个 ACK/NACK 比特o_i^{ACK}对应 HARQ 反馈窗口内的一个下行子帧$n-k_i$。

如果 UE 在 HARQ 反馈窗口内的某个下行子帧上没有检测到任何 PDSCH 传输或指示下行 SPS 释放的 PDCCH（即丢失了下行传输），则该下行子帧对应的 ACK/NACK 比特会设置成 NACK。

如果$U_{DAI}+N_{SPS}=0$（即认为没有下行传输），则 UE 不会回复任何 ACK/NACK。

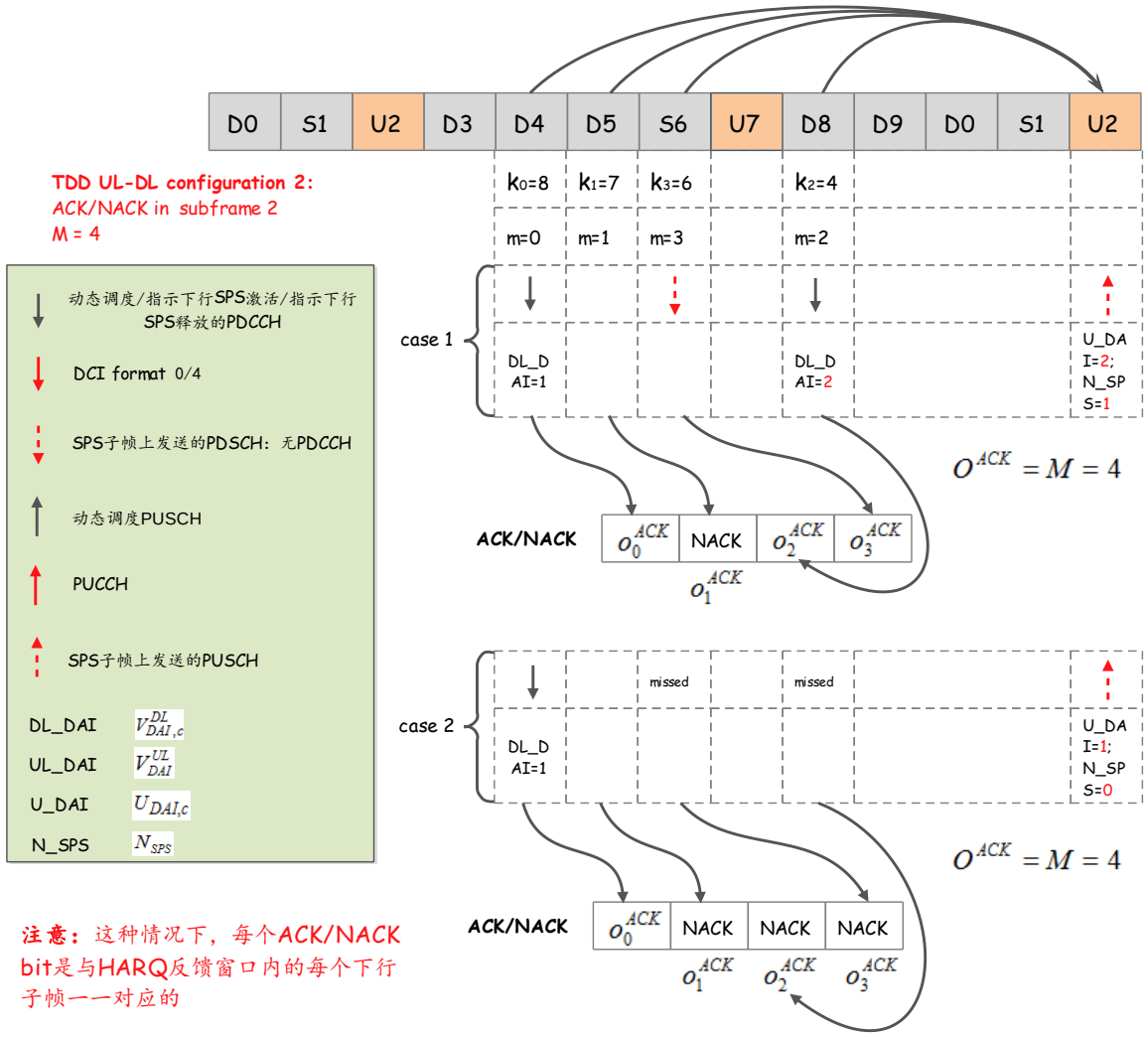


图 13-12：场景二举例

图 13-12 是场景二的一个例子。case 1/case 2 条件下，eNodeB 发送的所有 PDCCH/PDSCH 都是一致的。但在 case 2 中，UE 可能丢失某些 PDCCH/PDSCH。

case 1/2 下，由于每个反馈的比特都对应一个下行子帧，UE 只要在接收到下行数据的子帧上反馈 ACK/NACK 即可，其它子帧上直接反馈 NACK。

与 HARQ bundling 需要判断是否丢失了下行 DCI 不同，HARQ multiplexing 只需要把接收到下行数据的子帧对应的 ACK/NACK 反馈给 eNodeB 即可，对那些没有接收到下行数据的子帧直接反馈

NACK。而 eNodeB 只需要关心那些发送了下行数据的子帧对应的 ACK/NACK 信息，从而决定哪些子帧需要重传。

13.4.2.2 在 PUCCH 上发送 ACK/NACK（见 36.213 的 10.1.3.1 节）

如果 UE 需要在上行子帧 n 反馈 ACK/NCK，且在子帧 n - k 没有收到 DCI format 0/4，且子帧 n 不是周期性的上行 SPS 子帧，则 UE 会在 PUCCH 上反馈对应下行传输的 ACK/NACK。

TDD 下，如果 UE 只配置了一个 serving cell 且配置了 HARQ multiplexing，则对于 M > 1 的上行子帧 n，$n_{\mathrm{PUCCH},i}^{(1)}$ 是对应下行子帧 $n-k_i$ 计算出的 PUCCH 1 资源，而 HARQ-ACK(i)对应下行子帧 $n-k_i$ 发送的数据的 ACK/NACK。即下标 i 与 m（m 见 13.2.2 节介绍）值是一一对应的。

如果在 HARQ 反馈窗口内，UE 在下行子帧 $n-k_i$ 接收到有 PDCCH 指示的 PDSCH（动态调度或下行 SPS 激活）或指示下行 SPS 释放的 PDCCH，则 UE 会然后使用公式 $n_{\mathrm{PUCCH},i}^{(1)}=(M-i-1)\cdot N_c+i\cdot N_{c+1}+n_{\mathrm{CCE},i}+N_{\mathrm{PUCCH}}^{(1)}$ 来计算一个 PUCCH 1 资源。其中 c 值属于{0, 1, 2, 3}且满足 $N_c\le n_{\mathrm{CCE}}<N_{c+1}$，$N_c=\max\left\{0,\left\lfloor[N_{\mathrm{RB}}^{\mathrm{DL}}\cdot(N_{\mathrm{sc}}^{\mathrm{RB}}\cdot c-4)]/36\right\rfloor\right\}$，$N_{\mathrm{PUCCH}}^{(1)}$ 为 RRC 层配置的小区级参数（通过 *n1PUCCH-AN* 配置），$n_{\mathrm{CCE},i}$ 是 UE 在下行子帧 $n-k_i$ 发送的下行 DCI 所在的第一个 CCE 的 CCE number。

如果在 HARQ 反馈窗口内，UE 在下行子帧 $n-k_i$ 接收到无 PDCCH 指示的 PDSCH（周期性下行 SPS 子帧发送的 PDSCH），则 UE 会根据配置的 *n1PUCCH-AN-PersistentList* 以及最近一个指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段查 36.213 的 Table 9.2-2 得到一个 PUCCH 1 资源 $n_{\mathrm{PUCCH},i}^{(1)}$。

从上面的介绍的可以看出，对应 HARQ 反馈窗口内的每一个接收到下行传输的子帧，都会计算出一个 PUCCH 1 资源 $n_{\mathrm{PUCCH},i}^{(1)}$。其中 i 表示该下行子帧是 HARQ 反馈窗口内的第几个下行子帧（与 m 值一一对应）。

对于使用 PUCCH 反馈的 ACK/NACK 来说，HARQ 反馈窗口内有多少个下行子帧（由 M 表示，范围 1~4），UE 就可能需要反馈多少个 ACK/NACK 比特。但是 PUCCH format 1b 至多携带 2 比特的信息，可能不足以表示所有的信息。因此在 HARQ multiplexing 中，会计算出多个 PUCCH 1 资源，然后 UE 从这些 PUCCH 1 资源中选择一个（M 选 1，称为“channel selection”，这是在 UE 侧完成的）用于发送 PUCCH format 1b，这样就能携带除 PUCCH format 1b 的 2 比特以外的额外信息。

个人对“channel selection”的理解：步骤一：eNodeB 侧和 UE 侧都会先计算出 N 个 PUCCH 1 资源，且二者的计算结果相同（UE 侧计算 N 个 PUCCH 1 资源所需的信息都是由 eNodeB 告诉 UE 的。但 UE 可能丢失某些下行传输，所以 UE 侧计算出来的资源数可能会少于 N 个）；步骤二：UE 根

据所要发送的 ACK/NACK 信息（根据 ACK/NACK 查表）选择 N 个 PUCCH 1 资源中的一个来发送 PUCCH format 1b（N 选 1，因此额外携带了一些信息）；步骤三：eNodeB 不知道 UE 会使用 N 个 PUCCH 1 资源中的哪一个来发送 ACK/NACK 信息，所以会对 N 个资源进行逐个检查，直到检查到对应的 ACK/NACK 信息，这样就知道是 N 种组合中的哪一个，也就得到额外携带的信息，再加上 PUCCH format 1b 本身携带的 2 比特信息，就可以表示更多的信息了（这个过程是通过查表，得到 ACK/NACK 的）。

channel selection 实际上就是 UE 从 N 个 PUCCH 1 资源中选取一个的过程，但这个资源不是随机选择的，而是根据所要反馈的 ACK/NACK 信息，查表得到的。这里要求 eNodeB 和 UE 查的是同一张表。channel selection 主要应用在 HARQ multiplexing 和 PUCCH format 1b with channel selection。

使用 HARQ multiplexing 时，查看的表可能是 Tables 10.1.3-2（M=2），10.1.3-3（M=3）和 10.1.3-4（M=4），也可能是 Tables 10.1.3-5（M=2），10.1.3-6（M=3）和 10.1.3-7（M=4）。使用哪个集合是通过 *PUCCH-ConfigDedicated-v1020* 的 *pucch-Format-r10* 的 *channelSelection-r10* 字段来配置的。对于 TDD 而言，如果 UE 只配置了 PCell，则 *channelSelection-r10* 字段指示 HARQ multiplexing 基于 Tables 10.1.3-5（M=2），10.1.3-6（M=3）和 10.1.3-7（M=4）来反馈 ACK/NACK。

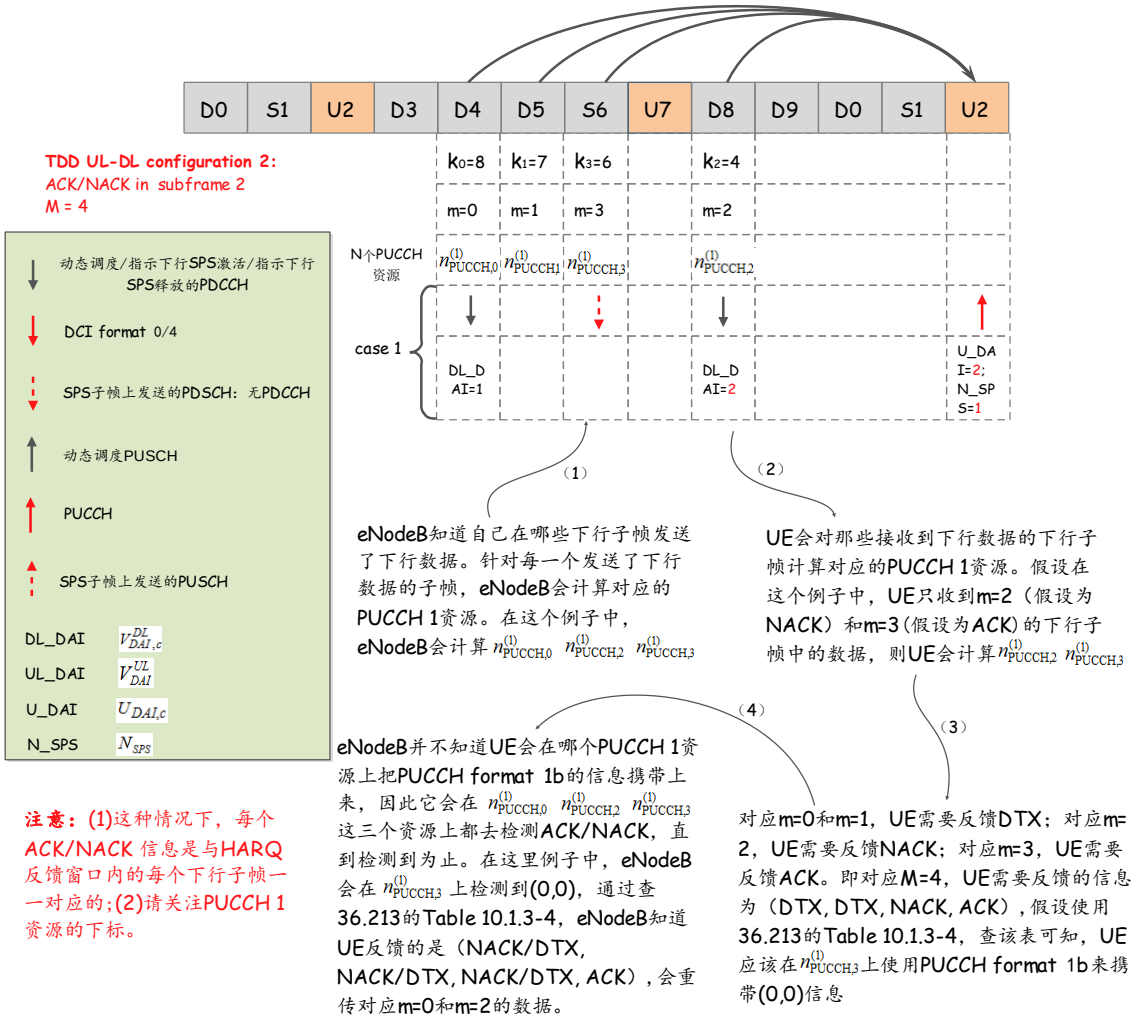


图 13-13：HARQ multiplexing 举例

图 13-13 是 TDD 下使用 HARQ multiplexing 的一个例子。注意计算出的 PUCCH 1 资源的下标及其对应的子帧。

表 13-3 是 HARQ multiplexing 中 UE 反馈的 ACK/NACK 比特与下行子帧的对应关系的小结。

表 13-3 HARQ multiplexing 中 UE 反馈的 ACK/NACK 比特与下行子帧的对应关系

场景	ACK/NACK 比特数	ACK/NACK 比特与下行子帧的对应关系

在 HARQ 反馈窗口内收到 DCI format 0/4，在 PUSCH 上反馈 ACK/NACK	$O^{ACK} = V_{DAI}^{UL}$	每个 ACK/NACK 比特与下行 DCI 内携带的 DAI（即 *DAI(k)*）对应；周期性下行 SPS 子帧上的 PUSCH 传输位于最后一个比特上
在周期性的上行 SPS 子帧的 PUSCH 上反馈 ACK/NACK	$O^{ACK} = M$	每个 ACK/NACK 比特与 HARQ 反馈窗口内的一个下行子帧对应
在 PUCCH 上反馈 ACK/NACK	M	每个 ACK/NACK 比特与 HARQ 反馈窗口内的一个下行子帧对应

【参考资料】

[1] 36.213 的 7.3 节 UE procedure for reporting HARQ-ACK

[2] 36.213 的 10.1 节 UE procedure for determining physical uplink control channel assignment

[3] 36.331 的 *tdd-AckNackFeedbackMode*

[4] 《TDD LTE 中的 ACK/NACK Bundling 和 Multiplexing》

[5] 《HARQ in TD-LTE》

[6] 《Link level investigation of ACK/NACK bundling for LTE TDD》

[7] 《Method and Apparatus for Improving ACK/NACK Bundling》

13.5 载波聚合下的下行 HARQ

本节会对载波聚合下的下行 HARQ 处理进行介绍。重点介绍使用 PUCCH 来反馈 ACK/NACK，包括 PUCCH format 1b with channel selection 和 PUCCH format 3 的处理。

在 8.6 节，我们已经介绍过，为了支持载波聚合，Rel-10 中新增了 2 种 PUCCH 的 ACK/NACK 反馈方式：

- **PUCCH format 1b with channel selection**：使用 PUCCH format 1b 携带的 2 比特信息，并选择几个特定 PUCCH 1 资源的某一个，来指示所要发送的 ACK/NACK 信息。其只支持不超过 4 个 ACK/NACK 比特信息且不多于 2 个 serving cell 的场景；
- **PUCCH format 3**：可支持至多 5 个 serving cell 且 UE 在每个 serving cell 都配置了 MIMO 的场景。**注意**：PUCCH format 3 也能够支持 2 个 serving cell 的场景，具体是使用 PUCCH

format 3 还是 PUCCH format 1b with channel selection 取决于 eNodeB 对 *pucch-Format-r10* 的配置。

载波聚合下，使用这两种 format 中的哪一个是通过 IE：*PUCCH-ConfigDedicated-v1020* 来指定的。

```
PUCCH-ConfigDedicated-v1020 ::=     SEQUENCE {
    pucch-Format-r10                    CHOICE {        ----这是个CHOICE，因此对于某个UE来说，只能在PUCCH format 3和PUCCH format 1b with channel selection中选一个。
        format3-r10                                 SEQUENCE {
            n3PUCCH-AN-List-r10 SEQUENCE (SIZE (1..4)) OF INTEGER (0..549)  OPTIONAL,   -- Need ON       ---- eNodeB给UE配置的4个PUCCH 3资源，用于天线端口0。
            twoAntennaPortActivatedPUCCH-Format3-r10        CHOICE {
                release                                         NULL,
                setup                                           SEQUENCE {
                    n3PUCCH-AN-ListP1-r10   SEQUENCE (SIZE (1..4)) OF INTEGER (0..549) ----eNodeB给UE配置的4个PUCCH 3资源，用于天线端口1，只在2天线传输时使用。
                }
            }                                                                   OPTIONAL    -- Need ON
        },
        channelSelection-r10                SEQUENCE {
            n1PUCCH-AN-CS-r10                   CHOICE {
                release                             NULL,
                setup                               SEQUENCE {
                    n1PUCCH-AN-CS-List-r10              SEQUENCE (SIZE (1..2)) OF N1PUCCH-AN-CS-r10       ----用于PUCCH format 1b with channel selection的PUCCH 1资源列表。在SCell上配置的传输模式支持空分复用或TDD下M大于1的情况下，才会使用2个list。
                }
            }                                                                   OPTIONAL    -- Need ON
        }
    }                                                                           OPTIONAL,   -- Need OR
    twoAntennaPortActivatedPUCCH-Format1a1b-r10     ENUMERATED {true}       OPTIONAL,   -- Need OR
    simultaneousPUCCH-PUSCH-r10                     ENUMERATED {true}       OPTIONAL,   -- Need OR
    n1PUCCH-AN-RepP1-r10                            INTEGER (0..2047)       OPTIONAL    -- Need OR
}

N1PUCCH-AN-CS-r10   ::= SEQUENCE (SIZE (1..4)) OF INTEGER (0..2047) ----用于PUCCH format 1b with channel selection的PUCCH 1资源。
```

13.5.1 PUCCH format 1b with channel selection

PUCCH format 1b with channel selection 包含配置至多 4 个 PUCCH 1 资源（称之为“channel”），选择这些资源中的一个可指示额外的 ACK/NACK 信息。

对于 FDD 而言，使用 PUCCH format 1b with channel selection 可以携带 2 个 serving cell 上的所有 ACK/NACK 信息。而对于 TDD 而言，如果需要反馈的 ACK/NACK 比特数大于 4，则同一 serving

cell 在同一个下行子帧上发送的 2 个 codeword（空分复用情况下）对应的 ACK/NACK 比特需要先做 spatial bundling 处理。如果 spatial bundling 处理之后的 ACK/NACK 比特数依然大于 4，则还要进行时域上的 bundling 处理（这种情况只在 TDD 5 下可能存在）。

13.5.1.1 FDD

对于 FDD，UE 需要反馈的 ACK/NACK 比特数取决于配置的 serving cell 数和每个 serving cell 上配置的下行 TM 模式。

对于 FDD 下，配置了 2 个 serving cell 并使用 PUCCH format 1b with channel selection 的 UE 而言，UE 会根据需要反馈的 ACK/NACK 信息，查看 36.213 的 Table 10.1.2.2.1-3、Table 10.1.2.2.1-4、Table 10.1.2.2.1-5 后，从 A 个 PUCCH 1 资源（ $n^{(1)}_{\mathrm{PUCCH},\,j}$ ，其中 $0 \le j \le A-1$ 且 $A \in \{2,3,4\}$ ）中选择一个 $n^{(1)}_{\mathrm{PUCCH}}$ 来发送 PUCCH format 1b，并携带 2 比特的信息 $b(0)b(1)$。

HARQ-ACK(j) 对应某个 serving cell 上发送的一个 TB，或一个指示下行 SPS 释放的 PDCCH，需要反馈的 ACK/NACK/DTX。其对应关系见 36.213 的 Table 10.1.2.2.1-1。

Table 10.1.2.2.1-1: Mapping of Transport Block and Serving Cell to HARQ-ACK(*j*) for PUCCH format 1b HARQ-ACK channel selection

A	***HARQ-ACK(j)***			
	HARQ-ACK(0)	HARQ-ACK(1)	HARQ-ACK(2)	HARQ-ACK(3)
2	TB1 Primary cell	TB1 Secondary cell	NA	NA
3	TB1 Serving cell1	TB2 Serving cell1	TB1 Serving cell2	NA
4	TB1 Primary cell	TB2 Primary cell	TB1 Secondary cell	TB2 Secondary cell

从上表可以看出，FDD 下，每个 TB（或指示下行 SPS 释放的 PDCCH）都对应一个 ACK/NACK 比特。我们可以认为“A = 在 PCell 上配置的 TM 模式支持的最大 TB 数 + 在 SCell 上配置的 TM 模式支持的最大 TB 数”。

当 UE 在某个 serving cell 下配置的传输模式（TM 模式）支持 2 个 TB，但一个 PDSCH 传输只携带一个 TB，或只传输一个指示下行 SPS 释放的 PDCCH 时， 2 个 TB 会使用相同的 ACK/NACK 信息。也就是说对于支持 2 个 TB 的传输模式，即使只发送了一个 TB，也必须按照 2 个 TB 来反馈 ACK/NACK，并且 2 个 TB 使用相同的反馈。

注意：UE 在 PCell 和 SCell 可以使用不同的 TM 模式。在 36.213 的 Table 10.1.2.2.1-1 中，A = 3 时，对应哪个 serving cell 使用空分复用，HARQ-ACK(0)和 HARQ-ACK(1)就与该 serving cell 的 2 个 TB 相对应。

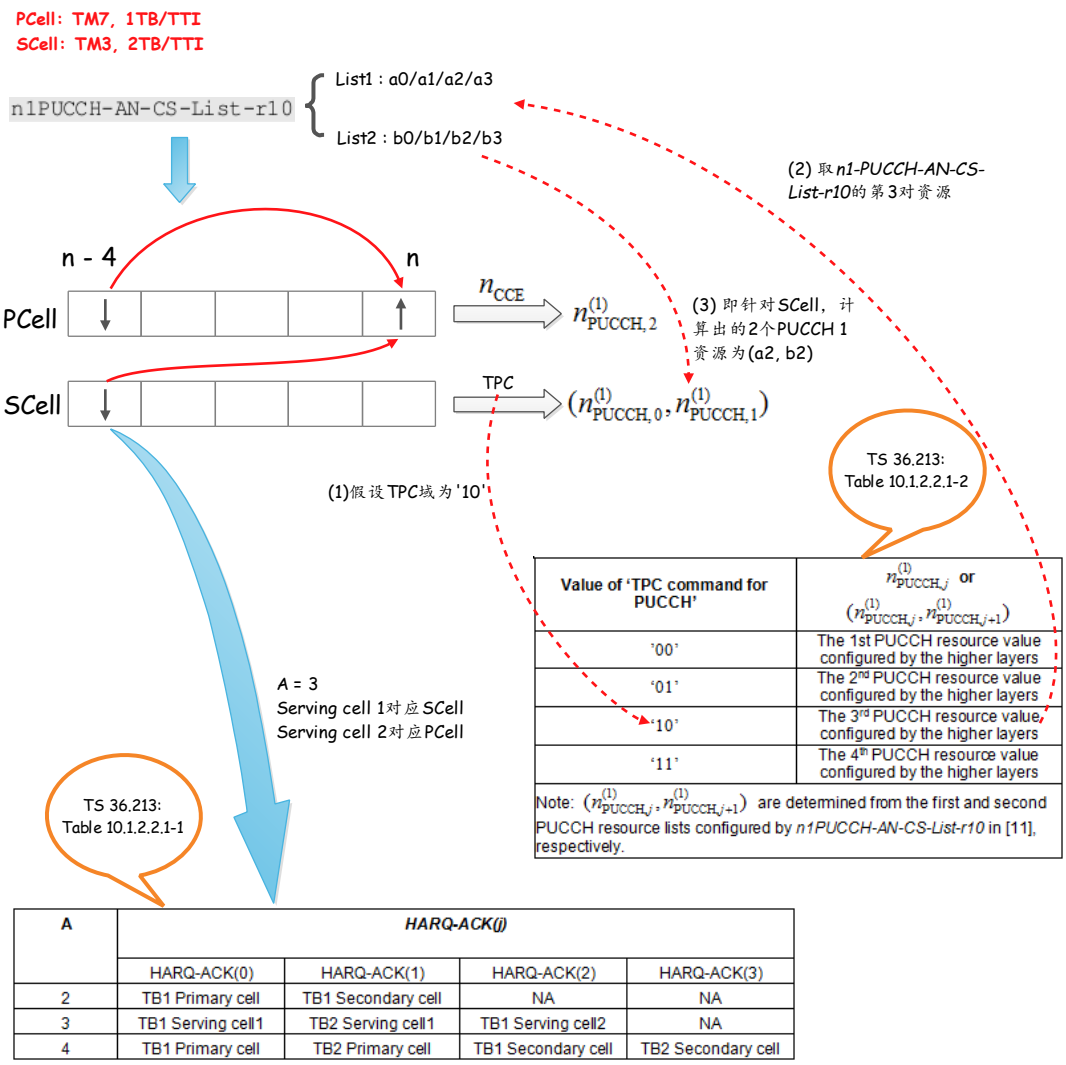
接下来我们就来介绍 FDD 下，UE 使用 PUCCH format 1b with channel selection 来反馈 ACK/NACK 的流程。

步骤一：UE 首先需要确定 A（最大值为 4）个与 Table 10.1.2.2.1-1 中的 HARQ-ACK(j) 对应的 PUCCH 1 资源 $n^{(1)}_{\mathrm{PUCCH},\,j}$ ，其中 $0 \le j \le A-1$。

- **情况 1**：如果 UE 在 PCell 的子帧 n-4 上检测到带 PDCCH 的 PDSCH 传输（动态调度或下行 SPS 激活子帧）或指示下行 SPS 释放的 PDCCH，则有 $n^{(1)}_{\mathrm{PUCCH},\,\mathrm{j}} = n_{\mathrm{CCE}} + N^{(1)}_{\mathrm{PUCCH}}$。如果 UE 在 PCell 上配置的传输模式支持 2 个 TB，则 $n^{(1)}_{\mathrm{PUCCH},\,j+1} = n_{\mathrm{CCE}} + 1 + N^{(1)}_{\mathrm{PUCCH}}$。 $N^{(1)}_{\mathrm{PUCCH}}$ 为 RRC 层配置的小区级参数（*n1PUCCH-AN*），n_{CCE} 是对应的下行 PDCCH 所在的第一个 CCE 的 CCE number。（**注**：1、这种情况只针对 PCell 上的下行传输；2、如果 UE 在 PCell 不支持空分复用，而在 SCell 上支持空分复用，参考 36.213 的 Table 10.1.2.2.1-1，对应 A=3，这里计算出的资源为 $n^{(1)}_{\mathrm{PUCCH},\,2}$，即 j=2；而在其它时候，这里计算出的资源为 $n^{(1)}_{\mathrm{PUCCH},\,0}$ 和 $n^{(1)}_{\mathrm{PUCCH},\,1}$，其中 $n^{(1)}_{\mathrm{PUCCH},\,1}$ 只在 PCell 上支持同时传输 2 个 TB 的情况下才存在）

- **情况 2**：如果 UE 在 PCell 的子帧 n-4 上检测到不带 PDCCH 的 PDSCH 传输（周期性下行 SPS 子帧上发送的 PDSCH），则 UE 会根据配置的 *n1PUCCH-AN-PersistentList* 以及最近一个指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段查 36.213 的 Table 9.2-2 得到一个 PUCCH 1 资源 $n^{(1)}_{\mathrm{PUCCH},\,j}$。如果 UE 在 PCell 上配置的传输模式支持最多 2 个 TB，则 $n^{(1)}_{\mathrm{PUCCH},\,j+1} = n^{(1)}_{\mathrm{PUCCH},\,j} + 1$。

- **情况 3**：如果 UE 在 SCell 的子帧 n-4 上检测到带 PDCCH 的 PDSCH 传输（动态调度，UE 在 SCell 上不支持 SPS），UE 会根据该 PDCCH 的 TPC 字段查 Table 10.1.2.2.1-2，然后从 *pucch-Format-r10* 的 *channelSelection-r10* 的 *n1PUCCH-AN-CS-List-r10* 字段指定的几个资源中选择 1 个 PUCCH 1 资源，作为 $n^{(1)}_{\mathrm{PUCCH},\,j}$；或 UE 在 SCell 上配置的传输模式支持 2 个 TB 的情况下，选择 2 个 PUCCH 1 资源，作为 $(n^{(1)}_{\mathrm{PUCCH},\,j}, n^{(1)}_{\mathrm{PUCCH},\,j+1})$。



Value of 'TPC command for PUCCH'	$n^{(1)}_{PUCCH,j}$ or $(n^{(1)}_{PUCCH,j}, n^{(1)}_{PUCCH,j+1})$
'00'	The 1st PUCCH resource value configured by the higher layers
'01'	The 2nd PUCCH resource value configured by the higher layers
'10'	The 3rd PUCCH resource value, configured by the higher layers
'11'	The 4th PUCCH resource value configured by the higher layers

Note: $(n^{(1)}_{PUCCH,j}, n^{(1)}_{PUCCH,j+1})$ are determined from the first and second PUCCH resource lists configured by *n1PUCCH-AN-CS-List-r10* in [11], respectively.

A	*HARQ-ACK(j)*			
	HARQ-ACK(0)	HARQ-ACK(1)	HARQ-ACK(2)	HARQ-ACK(3)
2	TB1 Primary cell	TB1 Secondary cell	NA	NA
3	TB1 Serving cell1	TB2 Serving cell1	TB1 Serving cell2	NA
4	TB1 Primary cell	TB2 Primary cell	TB1 Secondary cell	TB2 Secondary cell

图 13-14：FDD 下，PUCCH format 1b with channel selection 的一个例子

图 13-14 是 FDD 下，PUCCH format 1b with channel selection 的一个例子。假设 UE 在 PCell 上配置了 TM7，一个 TTI 只能发送 1 个 TB；在 SCell 上配置了 TM3，一个 TTI 可以发送 2 个 TB。并且 eNodeB 通过 *n1-PUCCH-AN-CS-List-r10* 字段给 UE 配置了 2 个 PUCCH 1 资源列表，每个列表分别包含 4 个 PUCCH 1 资源，分别为 a0/a1/a2/a3 和 b0/b1/b2/b3。即有 A=3，HARQ-ACK(0)和 HARQ-ACK(1)分别对应 SCell 的 TB1 和 TB2，HARQ-ACK(2)对应 PCell 的 TB1。基于 SCell 计算出的 2 个 PUCCH 1 资源为 $(n^{(1)}_{PUCCH,0}, n^{(1)}_{PUCCH,1})$；基于 PCell 计算出的 1 个 PUCCH 1 资源为 $n^{(1)}_{PUCCH,2}$。

本例中，$n^{(1)}_{PUCCH,2}$ 是基于 PCell 上接收到的 PDCCH 所占的第一个 CCE 的 CCE 索引得到的，$(n^{(1)}_{PUCCH,0}, n^{(1)}_{PUCCH,1})$ 是基于 SCell 上接收到的 PDCCH 上的 TPC 字段，查 36.213 的 Table 10.1.2.2.1-2，得知应该使用 *n1PUCCH-AN-CS-List-r10* 中的那一对资源（*n1PUCCH-AN-CS-List-r10* 配置了 2

个 list，这一对资源分别来自不同的 list），这里我们假设 TPC 字段为“10”，最终得到的 $(n^{(1)}_{\mathrm{PUCCH},0}, n^{(1)}_{\mathrm{PUCCH},1})$ 为(a2, b2)。

步骤二：根据需要反馈的 ACK/NACK 信息以及在步骤一中计算出的 A 个 PUCCH 1 资源，UE 会查看 36.213 的 Table 10.1.2.2.1-3、Table 10.1.2.2.1-4、Table 10.1.2.2.1-5 以决定从 A 个 PUCCH 1 资源中选择哪一个来发送 PUCCH format 1b，以及 PUCCH format 1b 中携带的 2 比特信息。

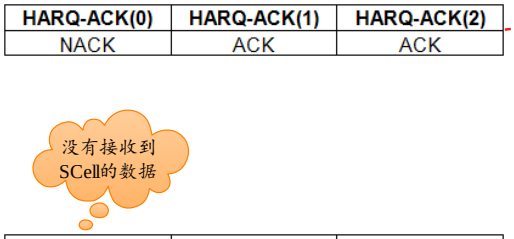


Table 10.1.2.2.1-4: Transmission of Format 1b HARQ-ACK channel selection for $A = 3$

HARQ-ACK(0)	HARQ-ACK(1)	HARQ-ACK(2)	$n^{(1)}_{PUCCH}$	$b(0)b(1)$
ACK	ACK	ACK	$n^{(1)}_{PUCCH,1}$	1,1
ACK	NACK/DTX	ACK	$n^{(1)}_{PUCCH,1}$	1,0
NACK/DTX	ACK	ACK	$n^{(1)}_{PUCCH,1}$	0,1
NACK/DTX	NACK/DTX	ACK	$n^{(1)}_{PUCCH,2}$	1,1
ACK	ACK	NACK/DTX	$n^{(1)}_{PUCCH,0}$	1,1
ACK	NACK/DTX	NACK/DTX	$n^{(1)}_{PUCCH,0}$	1,0
NACK/DTX	ACK	NACK/DTX	$n^{(1)}_{PUCCH,0}$	0,1
NACK/DTX	NACK/DTX	NACK	$n^{(1)}_{PUCCH,2}$	0,0
NACK	NACK/DTX	DTX	$n^{(1)}_{PUCCH,0}$	0,0
NACK/DTX	NACK	DTX	$n^{(1)}_{PUCCH,0}$	0,0
DTX	DTX	DTX	No Transmission	

图 13-15：FDD 下，PUCCH format 1b with channel selection 的一个例子

图 13-15 紧接着图 13-14 的例子。假设 UE 需要回复的 ACK/NACK 信息为（NACK, ACK, ACK），则通过查 36.213 的 Table 10.1.2.2.1-4 可知，UE 应该使用 $n^{(1)}_{\mathrm{PUCCH},1}$ 将 PUCCH format 1b 携带的 2 比特信息“01”反馈给 eNodeB。另假设 UE 需要回复的 ACK/NACK 信息为（DTX, DTX, NACK），则通过查 36.213 的 Table 10.1.2.2.1-4 可知，UE 应该使用 $n^{(1)}_{\mathrm{PUCCH},2}$ 将 PUCCH format 1b 携带的 2 比特信息“00”反馈给 eNodeB。

图 13-16 总结了 FDD 下，PUCCH format 1b with channel selection 宏观上的处理流程。

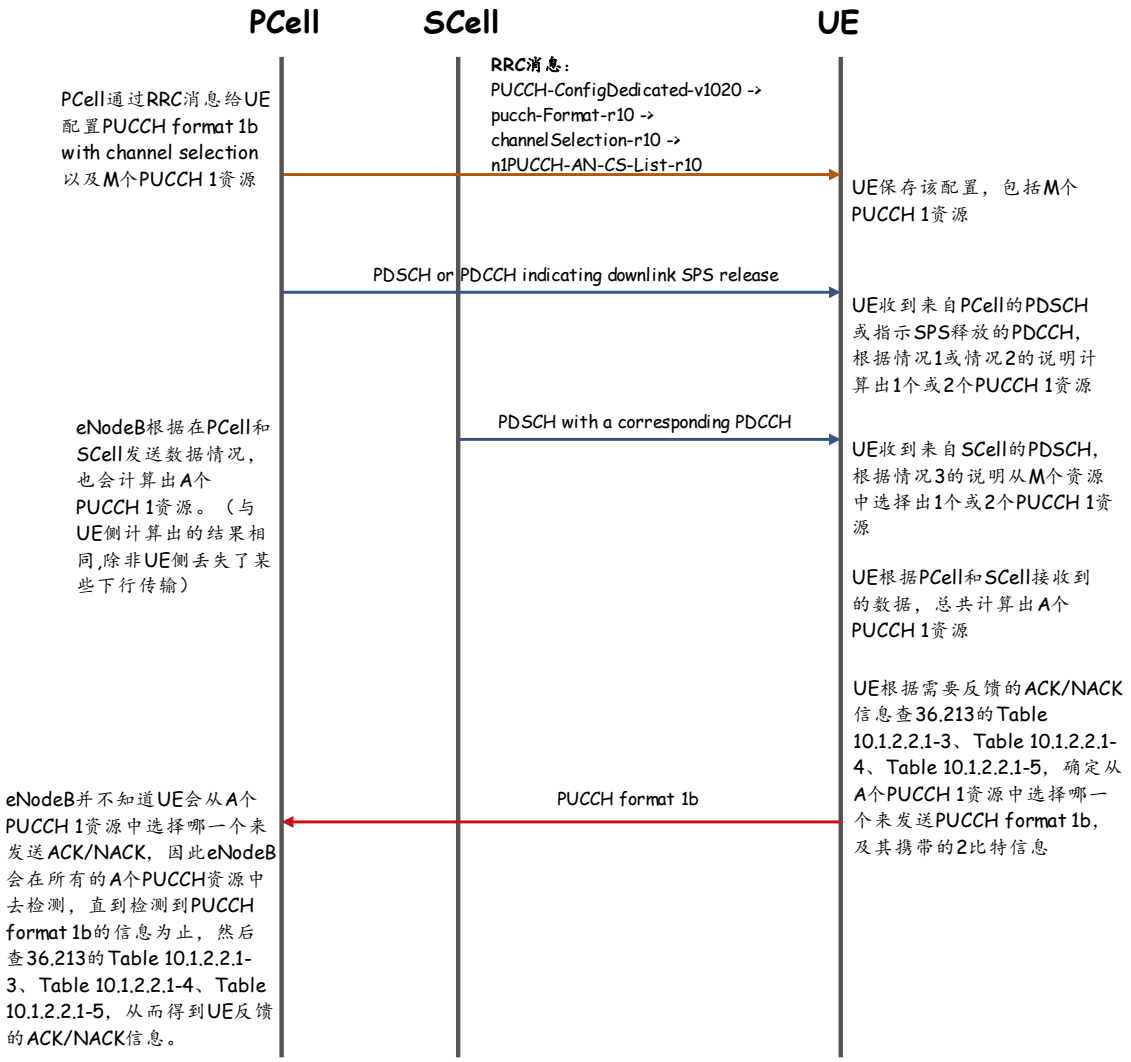


图 13-16：FDD 下，PUCCH format 1b with channel selection 的处理流程

13.5.1.2 TDD

相比 FDD，TDD 下 PUCCH format 1b with channel selection 的处理略复杂，但基本思路是一致的。

TDD 5 不支持 PUCCH format 1b with channel selection。

对于 TDD 下的 PUCCH format 1b with channel selection，需要区分反馈 ACK/NACK 的上行子帧对应的 $M \le 2$ 和 $M > 2$ 的场景。

首先，我们来介绍一下某个需要反馈 ACK/NACK 的上行子帧对应的 $M \le 2$ 的场景。

对于 TDD 下，配置了 2 个 serving cell 并使用 PUCCH format 1b with channel selection，且需要回复 ACK/NACK 的上行子帧 n 对应的 $M=1$ 的 UE 而言，UE 会基于配置的 serving cell 数以及每个 serving cell 上的下行传输模式来决定所要反馈的 ACK/NACK 比特数 O。如果 UE 在某个 serving cell 上配置的传输模式支持最多 2 个 TB，则该 serving cell 使用 2 个 ACK/NACK 比特，否则使用 1 个 ACK/NACK 比特。

对于 TDD 下，配置了 2 个 serving cell 并使用 PUCCH format 1b with channel selection，且需要回复 ACK/NACK 的上行子帧 n 对应的 $M \le 2$ 的 UE 而言，UE 会根据需要反馈的 ACK/NACK 信息，查看 36.213 的 Table 10.1.3.2-1、Table 10.1.3.2-2、Table 10.1.3.2-3 后，从 A 个 PUCCH 1 资源（$n^{(1)}_{\mathrm{PUCCH},\,j}$，其中 $0 \le j \le A-1$ 且 $A \in \{2,3,4\}$）中选择一个 $n^{(1)}_{\mathrm{PUCCH}}$ 来发送 PUCCH format 1b，并携带 2 比特的信息 $b(0)b(1)$。

对于 $M=1$ 的上行子帧 n，HARQ-ACK(j) 对应某个 serving cell 上发送的一个 TB，或一个指示下行 SPS 释放的 PDCCH，需要反馈的 ACK/NACK/DTX。其对应关系见 36.213 的 Table 10.1.2.2.1-1。（注意：这种情况下，一个比特与某个 serving cell 上的一个 TB 对应）

对于 $M=2$ 的上行子帧 n，HARQ-ACK(j) 对应某个 serving cell 上的某个子帧发送的一个 PDSCH 传输，或一个指示下行 SPS 释放的 PDCCH，需要反馈的 ACK/NACK/DTX。其对应关系见 36.213 的 Table 10.1.3.2-4。也就是说，这种情况下，同一 serving cell 在同一个下行子帧上发送的 2 个 codeword（空分复用情况下）对应的 ACK/NACK 比特需要先做 spatial bundling 处理。（注意：这种情况下，一个比特与某个 serving cell 上的一个下行子帧对应）

Table 10.1.3.2-4: Mapping of subframes on each serving cell to HARQ-ACK(*j*) for PUCCH format 1b HARQ-ACK channel selection for TDD with $M=2$

A	***HARQ-ACK(j)***			
	HARQ-ACK(0)	HARQ-ACK(1)	HARQ-ACK(2)	HARQ-ACK(3)
4	The first subframe of Primary cell	The second subframe of Primary cell	The first subframe of Secondary cell	The second subframe of Secondary cell

对比 Table 10.1.3.2-4 和 Table 10.1.2.2.1-1 可以看出，对应 M=2，A 必然等于 4；对应 M=1，A 可能为 2、3、4。

步骤一：对于 TDD，UE 会确定 A 个 PUCCH 1 资源 $n^{(1)}_{\mathrm{PUCCH},\,j}$，该资源与 HARQ-ACK($j$)对应，其中 $0 \le j \le A-1$，对于 $M=1$，HARQ-ACK(j)的含义见 36.213 的 Table 10.1.2.2.1-1；对于 $M=2$，HARQ-ACK(j)的含义见 36.213 的 Table 10.1.3.2-4。

- **情况 1**：如果 UE 在 PCell 的子帧 $n-k_m$ 上检测到带 PDCCH 的 PDSCH 传输（动态调度或下行 SPS 激活子帧）或指示下行 SPS 释放的 PDCCH，则有 $n^{(1)}_{\mathrm{PUCCH},j}=(M-m-1)\cdot N_c+m\cdot N_{c+1}+n_{\mathrm{CCE},m}+N^{(1)}_{\mathrm{PUCCH}}$。其中 c 属于 {0, 1, 2, 3} 且有 $N_c\le n_{\mathrm{CCE},m}<N_{c+1}$，$N_c=\max\left\{0,\left\lfloor[N^{\mathrm{DL}}_{\mathrm{RB}}\cdot(N^{\mathrm{RB}}_{\mathrm{sc}}\cdot c-4)]/36\right\rfloor\right\}$。$N^{\mathrm{DL}}_{\mathrm{RB}}$ 为 PCell 的下行系统带宽，$N^{(1)}_{\mathrm{PUCCH}}$ 为 RRC 层配置的小区级参数（*n1PUCCH-AN*），n_{CCE} 是对应的下行 PDCCH 所在的第一个 CCE 的 CCE number。对于 M=1 的上行子帧 n，且配置的传输模式支持 2 个 TB，则会计算出第二个 PUCCH 1 资源 $n^{(1)}_{\mathrm{PUCCH},j+1}=(M-m-1)\cdot N_c+m\cdot N_{c+1}+n_{\mathrm{CCE},m}+1+N^{(1)}_{\mathrm{PUCCH}}$。（**注**：1、这种情况只针对 PCell 上的下行传输；2、从上面的介绍可以看出，对于 M=2 的上行子帧，或对于 M=1 的上行子帧且下行配置了支持 2 个 TB 的传输模式，都会计算出 2 个 PUCCH 1 资源。前者计算出的 2 个 PUCCH 1 资源与所对应的 2 个下行子帧相关；后者计算出的 2 个 PUCCH 1 资源中，第一个与所对应的下行子帧相关，第二个在第一个的基础上加 1 得到）

- **情况 2**：如果 UE 在 PCell 的子帧 $n-k$ 上检测到不带 PDCCH 的 PDSCH 传输（周期性下行 SPS 子帧上发送的 PDSCH），则 UE 会根据配置的 *n1PUCCH-AN-PersistentList* 以及最近一个指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段查 36.213 的 Table 9.2-2 得到一个 PUCCH 1 资源 $n^{(1)}_{\mathrm{PUCCH},j}$。（**注意**：与 FDD 下的这种场景可能会计算出 2 个 PUCCH 1 资源不同，TDD 下的这种场景只会计算出 1 个 PUCCH 1 资源）

- **情况 3**：如果 UE 在 SCell 的子帧 n - k 上检测到带 PDCCH 的 PDSCH 传输（动态调度，UE 在 SCell 上不支持 SPS），UE 会根据该 PDCCH 的 TPC 字段查 36.213 的 Table 10.1.2.2.1-2，从 *pucch-Format-r10* 的 *channelSelection-r10* 的 *n1PUCCH-AN-CS-List-r10* 字段指定的几个资源中选择 1 个 PUCCH 1 资源，作为 $n^{(1)}_{\mathrm{PUCCH},j}$。如果 UE 位于 M=1 的上行子帧 n 且 SCell 上配置的 TM 模式支持 2 个 TB 的情况下，或位于 M = 2 的上行子帧 n 的情况下，则会选择 2 个 PUCCH 1 资源，作为 $(n^{(1)}_{\mathrm{PUCCH},j},n^{(1)}_{\mathrm{PUCCH},j+1})$。UE 会认为在 SCell 的子帧 n – k 上收到的所有 PDCCH 的 TPC 字段指示相同的 PUCCH 1 资源，即 SCell 上收到的所有 PDCCH 的 TPC 字段是相同的。

步骤二：根据需要反馈的 ACK/NACK 信息以及在步骤一中计算出的 A 个 PUCCH 1 资源，UE 会查看 36.213 的 Table 10.1.3.2-1、Table 10.1.3.2-2、Table 10.1.3.2-3 以决定从 A 个 PUCCH 1 资源中选择哪一个来发送 PUCCH format 1b，以及 PUCCH format 1b 中携带的 2 比特信息。

接着，我们来介绍一下某个需要反馈 ACK/NACK 的上行子帧对应的 $M>2$ 的场景。

步骤一：对于 TDD 下，配置了 2 个 serving cell 并使用 PUCCH format 1b with channel selection，且需要回复 ACK/NACK 的上行子帧 n 对应的 $M>2$ 的 UE 而言，假设 $n^{(1)}_{\mathrm{PUCCH},i}$， $0\le i\le 3$ 为衍生自上行子帧 n 对应的 M 个下行子帧传输的 PUCCH 1 资源。其中，$n^{(1)}_{\mathrm{PUCCH},0}$ 和 $n^{(1)}_{\mathrm{PUCCH},1}$ 与 PCell 上的 PDSCH 传输或指示下行 SPS 释放的 PDCCH 相关联；$n^{(1)}_{\mathrm{PUCCH},2}$ 和 $n^{(1)}_{\mathrm{PUCCH},3}$ 与 SCell 上的 PDSCH 传输相关联。（ $M>2$ 的情况下一定要计算出 4 个 PUCCH 1 资源，即 A=4 ）

对于 **PCell**，

- 如果 UE 在 PCell 的子帧 $n-k$ 上检测到不带 PDCCH 的 PDSCH 传输（周期性下行 SPS 子帧上发送的 PDSCH ）
  - 则 UE 会根据配置的 *n1PUCCH-AN-PersistentList* 以及最近一个指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段查 36.213 的 Table 9.2-2 得到 $n^{(1)}_{\mathrm{PUCCH},0}$ ；
  - 如果 UE 在 PCell 的子帧 $n-k_m$ 上检测到带 PDCCH 的 PDSCH 传输（动态调度或下行 SPS 激活子帧）且其 DAI 值为 1，或指示下行 SPS 释放的 PDCCH 且其 DAI 值为 1，则有 $n^{(1)}_{\mathrm{PUCCH},1}=(M-m-1)\cdot N_c+m\cdot N_{c+1}+n_{\mathrm{CCE},m}+N^{(1)}_{\mathrm{PUCCH}}$。其中 c 属于 {0, 1, 2, 3} 且有 $N_c\le n_{\mathrm{CCE},m}<N_{c+1}$，$N_c=\max\left\{0,\left\lfloor[N^{\mathrm{DL}}_{\mathrm{RB}}\cdot(N^{\mathrm{RB}}_{\mathrm{sc}}\cdot c-4)]/36\right\rfloor\right\}$。$N^{(1)}_{\mathrm{PUCCH}}$ 为 RRC 层配置的小区级参数（*n1PUCCH-AN*），n_{CCE} 是在子帧 $n-k_m$ 收到的下行 PDCCH 所在的第一个 CCE 的 CCE number。（注意：这种情况下，并不关心 DAI 大于 1 的 PDCCH ）
  - HARQ-ACK(0)对应不带 PDCCH 的 PDSCH 传输（周期性下行 SPS 子帧上发送的 PDSCH ）的 ACK/NACK/DTX。对于 $1\le j\le M-1$，如果 UE 接收到的带 PDCCH 的 PDSCH 传输的 DAI 值等于“j”，或 UE 接收到的一个指示下行 SPS 释放的 PDCCH 的 DAI 值等于“j”，则 HARQ-ACK(j)为其对应的 ACK/NACK/DTX，否则 HARQ-ACK(j)设置为 DTX。
- 否则（即在 HARQ 反馈窗口内没有检测到任何不带 PDCCH 的 PDSCH 传输），
  - 如果 UE 在 PCell 的子帧 $n-k_m$ 上检测到带 PDCCH 的 PDSCH 传输（动态调度或下行 SPS 激活子帧）且其 DAI 值为 1 或 2，或指示下行 SPS 释放的 PDCCH 且其 DAI 值为 1 或 2，则有 $n^{(1)}_{\mathrm{PUCCH},i}=(M-m-1)\cdot N_c+m\cdot N_{c+1}+n_{\mathrm{CCE},m}+N^{(1)}_{\mathrm{PUCCH}}$。其中 c 属于 {0, 1, 2, 3} 且有 $N_c\le n_{\mathrm{CCE},m}<N_{c+1}$，$N_c=\max\left\{0,\left\lfloor[N^{\mathrm{DL}}_{\mathrm{RB}}\cdot(N^{\mathrm{RB}}_{\mathrm{sc}}\cdot c-4)]/36\right\rfloor\right\}$，$N^{(1)}_{\mathrm{PUCCH}}$ 为 RRC 层配置的小区级参数（*n1PUCCH-AN*），n_{CCE} 是在子帧 $n-k_m$ 收到的下行 PDCCH 所在的第一个 CCE 的 CCE number。$i=0$ 对应 DAI 值为 1 的

PDCCH；$i=1$对应 DAI 值为 2 的 PDCCH。（注意：这种情况下，并不关心 DAI 值大于 2 的 PDCCH）

- 对于$0 \le j \le M-1$，如果 UE 收到的一个 PDSCH 传输对应的 PDCCH 的 DAI 值等于“j+1”，或一个指示下行 SPS 释放的 PDCCH 的 DAI 值等于“j+1”，则 HARQ-ACK(j)为其对应的 ACK/NACK/DTX；否则 HARQ-ACK(j)设置为 DTX。

对于 SCell，

- 如果 UE 在 SCell 的子帧$n-k_m$上检测到 PDSCH 传输，但其对应的 PDCCH 在 PCell 上传输（跨载波调度的情况），且其 DAI 值为 1 或 2，则有 $n^{(1)}_{\mathrm{PUCCH},i}=(M-m-1)\cdot N_c+m\cdot N_{c+1}+n_{\mathrm{CCE},m}+N^{(1)}_{\mathrm{PUCCH}}$。其中 c 属于 {0, 1, 2, 3} 且有 $N_c \le n_{\mathrm{CCE},m} < N_{c+1}$，$N_c=\max\left\{0, \left\lfloor [N^{\mathrm{DL}}_{\mathrm{RB}}\cdot(N^{\mathrm{RB}}_{\mathrm{sc}}\cdot c-4)]/36 \right\rfloor\right\}$，$N^{\mathrm{DL}}_{\mathrm{RB}}$为 PCell 的下行系统带宽，$N^{(1)}_{\mathrm{PUCCH}}$ 为 RRC 层配置的小区级参数（*n1PUCCH-AN*），n_{CCE}是在子帧 $n-k_m$ 收到的下行 PDCCH 所在的第一个 CCE 的 CCE number。$i=2$ 对应 DAI 值为 1 的 PDCCH；$i=3$对应 DAI 值为 2 的 PDCCH。
- 如果 UE 在 SCell 的子帧 n – k 上检测到带 PDCCH 的 PDSCH 传输（非跨载波调度，PDSCH 和 PDCCH 都在同一 SCell 上），UE 会根据该 PDCCH 的 TPC 字段查 36.213 的 Table 10.1.2.2.1-2，从 *pucch-Format-r10* 的 *channelSelection-r10* 的 *n1PUCCH-AN-CS-List-r10* 字段指定的几个资源中选择 1 对 PUCCH 1 资源，作为 $n^{(1)}_{\mathrm{PUCCH},2}$ 和 $n^{(1)}_{\mathrm{PUCCH},3}$。UE 会认为在 SCell 的子帧 n – k 上收到的所有 PDCCH 的 TPC 字段指示相同的 PUCCH 1 资源，即 SCell 上收到的所有 PDCCH 的 TPC 字段是相同的。
- 对于$0 \le j \le M-1$，如果 UE 收到的一个 PDSCH 传输对应的 PDCCH 的 DAI 值等于“j+1”，则 HARQ-ACK(j)为其对应的 ACK/NACK/DTX；否则 HARQ-ACK(j)设置为 DTX。

步骤二：根据需要反馈的 ACK/NACK 信息以及在步骤一中计算出的 A 个 PUCCH 1 资源，UE 会查看 36.213 的 Table 10.1.3.2-5（对应 M=3）或 Table 10.1.3.2-6（对应 M=4）以决定从 A 个 PUCCH 1 资源中选择哪一个来发送 PUCCH format 1b，以及 PUCCH format 1b 中携带的 2 比特信息 $b(0),b(1)$。

从上面的介绍可以看出，PCell 通过 *n1PUCCH-AN-CS-List-r10* 给 UE 配置的 PUCCH 1 资源只是被 UE 用来计算 A 个 PUCCH 1 资源中的某 1 个或 2 个（通过 TPC 指定），UE 并不一定使用 *n1PUCCH-AN-CS-List-r10* 中指定的资源来回复 ACK/NACK。

对于 PUCCH format 1b with channel selection 而言，PCell 上收到的 TPC 字段只用于功控，SCell 上收到的 TPC 字段只用于选择 PUCCH 1 资源。在 HARQ 反馈窗口内，只有 SCell 上有下行数据传输，UE 才可能会使用 *n1PUCCH-AN-CS-List-r10* 中的某一个（对）资源。

实际上，可能不是所有的 A 个 PUCCH 1 资源都会被计算。例如只有 SCell 上有下行数据时，就不会计算与 PCell 相关的 PUCCH 1 资源。

TDD 和 FDD 对于 PUCCH format 1b with channel selection 的处理流程和思想基本类似。不同的地方主要在于：（1）计算 A 个 PUCCH 1 资源的方式不同；（2）查的表不同。

PUCCH format 1b with channel selection 的难点在于如何计算出 A 个 PUCCH 1 资源，以及 HARQ-ACK(j)与下行传输的对应关系。理清这些问题，理解起来也就不那么复杂了。

由于 TDD 的处理思想与 FDD 类似，所以在 TDD 中，就不画图说明了。

这里需要再强调一下，在载波聚合中，PUCCH 只能在 PCell 上发送，所以上面计算的所有 PUCCH 1 资源都是针对 PCell 上的 PUCCH 资源。

13.5.2 PUCCH format 3

接下来，我们继续介绍 PUCCH format 3 的处理。

13.5.2.1 FDD

对于 FDD，如果 UE 配置了 PUCCH format 3，UE 将会在上行子帧 n 使用 PUCCH 3 资源 $n_{\mathrm{PUCCH}}^{(3,\tilde{p})}$ 或 PUCCH 1 资源 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 来回复 ACK/ANCK。规则如下：（见 36.213 的 10.1.2.2.2 节）

情况一：如果 UE 只在 PCell 的子帧 n - 4 上，只检测到一个带 PDCCH 的 PDSCH 传输，或检测到一个指示下行 SPS 释放的 PDCCH（UE 没有在 SCell 收到任何下行传输），则 UE 会使用 PUCCH format 1a/1b 来回复 ACK/NACK。天线端口 P0 使用的 PUCCH 1 资源为 $n_{\mathrm{PUCCH}}^{(1,\tilde{p}_0)} = n_{\mathrm{CCE}} + N_{\mathrm{PUCCH}}^{(1)}$；当使用 2 天线传输时，天线端口 P1 使用的 PUCCH 1 资源为 $n_{\mathrm{PUCCH}}^{(1,\tilde{p}_1)} = n_{\mathrm{CCE}} + 1 + N_{\mathrm{PUCCH}}^{(1)}$。（见图 13-17）

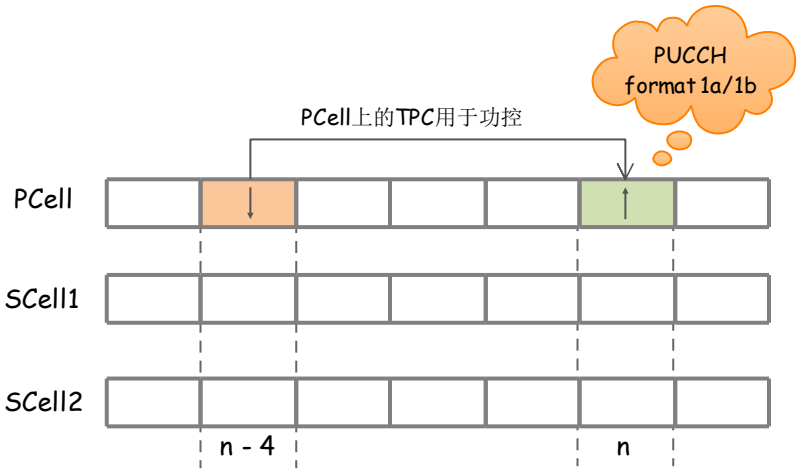


图 13-17：FDD 下，情况一、二举例

情况二：如果 UE 只在 PCell 的子帧 n - 4 上，检测到一个不带 PDCCH 的 PDSCH 传输（UE 没有在 SCell 收到任何下行传输），则 UE 会使用 PUCCH format 1a/1b 来回复 ACK/NACK。使用的 PUCCH 1 资源 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 会根据配置的 *n1PUCCH-AN-PersistentList* 和 *n1PUCCH-AN-PersistentListP1-r10*（2 天线传输时）以及最近一个指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段查 36.213 的 Table 9.2-2 得到。（类似图 13-17，但此时没有 TPC 字段）

情况三：如果 UE 在 SCell 的子帧 n - 4 上检测到带 PDCCH 的 PDSCH 传输（SCell 上只有动态调度，不存在 SPS 调度），则 UE 会使用 PUCCH format 3 来回复 ACK/NACK。使用的 PUCCH 3 资源 $n_{\mathrm{PUCCH}}^{(3,\tilde{p})}$ 是通过如下方式得到的：eNodeB 之前通过 *n3PUCCH-AN-List-r10*（对应天线端口 P0）和 *n3PUCCH-AN-ListP1-r10*（2 天线传输时，对应天线端口 P1），给 UE 配置了 4 个（或 4 对）PUCCH 3 资源。UE 根据 SCell 上收到的 PDCCH 里的 TPC command for PUCCH 字段查 36.213 的 Table 10.1.2.2.2-1，从 4 个（或 4 对）PUCCH 3 资源中选择一个来回复 ACK/NACK。UE 在同一子帧的所有 SCell 接收到的下行 DCI 的 TPC 字段必须是相同的，即指定相同的 PUCCH 3 资源。这种场景下，只要 SCell 有下行传输，就使用 PUCCH 3 资源，而无需关注 PCell 是否有下行传输。（见图 13-18）

Table 10.1.2.2.2-1: PUCCH Resource Value for HARQ-ACK Resource for PUCCH

Value of 'TPC command for PUCCH'	$n_{\text{PUCCH}}^{(3,\tilde{p})}$
'00'	The 1st PUCCH resource value configured by the higher layers
'01'	The 2nd PUCCH resource value configured by the higher layers
'10'	The 3rd PUCCH resource value configured by the higher layers
'11'	The 4th PUCCH resource value configured by the higher layers

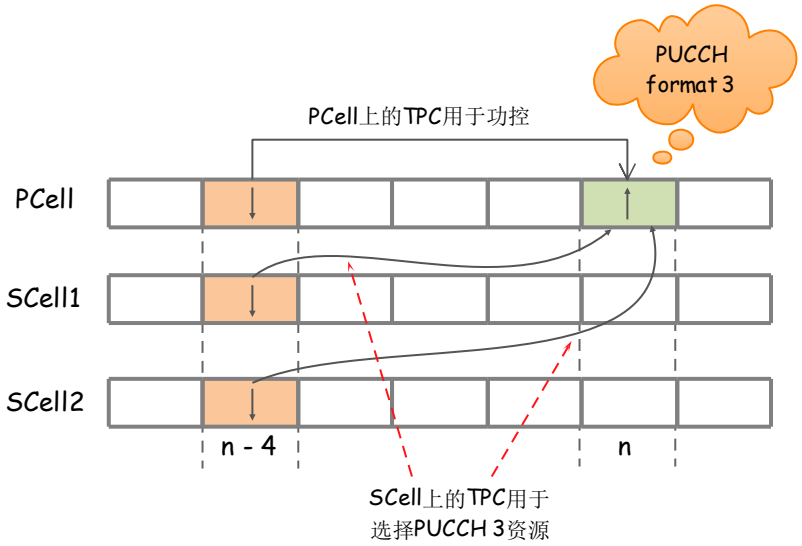


图 13-18：FDD 下，情况三举例

注：（1）对于 FDD，除了指示下行 SPS 激活的 DCI，PCell 上收到的下行 DCI 的 TPC command for PUCCH 字段都是用于 PUCCH 功控的，所有 SCell 上收到的所有下行 DCI 的 TPC command for PUCCH 字段都是用于指示选择哪个 PUCCH 3 资源的，且 UE 在所有 SCell 的同一子帧接收到的 TPC command for PUCCH 字段都必须是相同的；（2）对于 FDD，只有 SCell 上有 PDSCH 传输，才会使用 PUCCH format 3，如果只有 PCell 上有 PDSCH 传输或指示下行 SPS 释放的 PDCCH，则使用 PUCCH format 1a/1b；（3）与 PUCCH format 1b with channel selection 中 PUCCH 1 的资源由 UE 来选择不同，PUCCH format 3 中 PUCCH 3 资源的选择是由 eNodeB 来完成的，并通过 TPC 字段告知 UE。

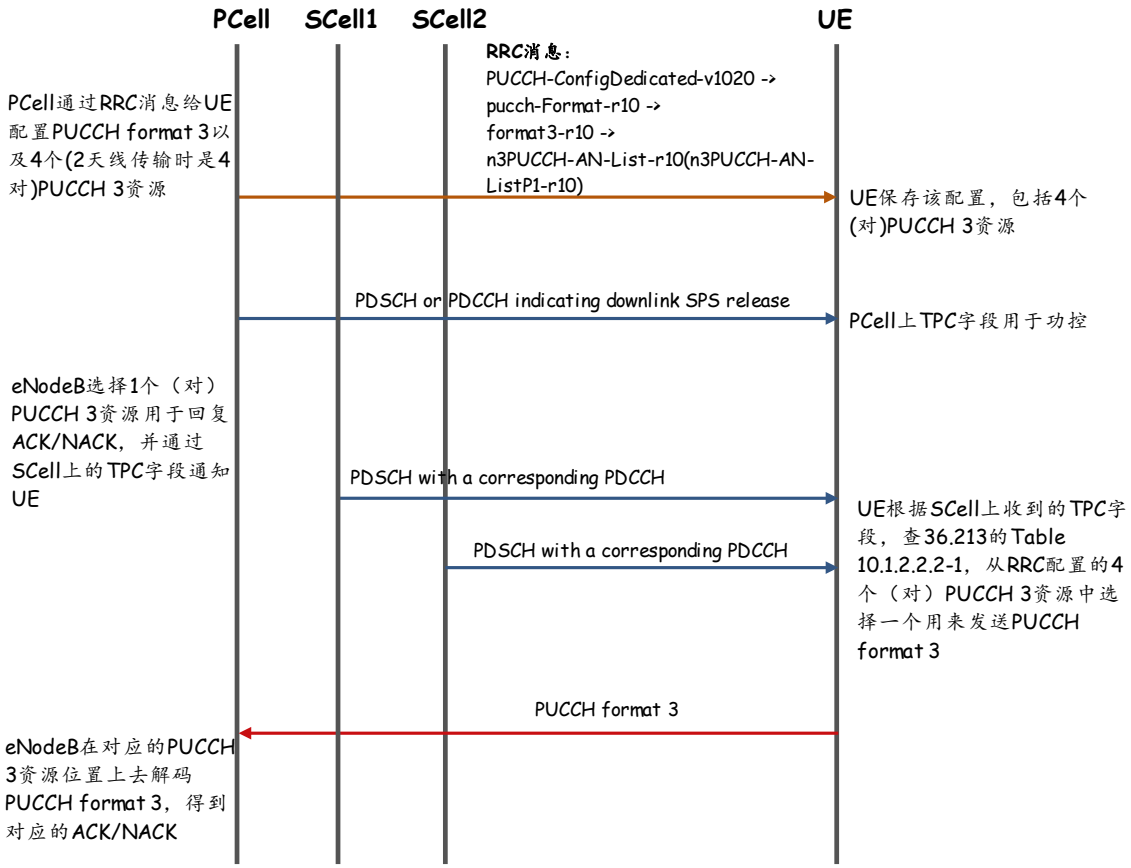


图 13-19：FDD 下，PUCCH format 3 的处理流程

对于 FDD 而言，在一个上行子帧中，最多需要反馈 10 个 ACK/NACK 比特的信息。（一个 UE 至多可配置 5 个 serving cell，如果每个 serving cell 都配置了 MIMO，则每个 serving cell 需要 2 个 ACK/NACK 比特，所以至多需要传输 10 个 ACK/NACK 比特的信息）

对于 FDD 而言，按照 index 由低到高的顺序将每个 serving cell 所要反馈的 ACK/NACK 比特串联在一起，就得到了 PUCCH format 3 中携带的信息。

13.5.2.2 TDD

对于 TDD，如果 UE 配置了 PUCCH format 3，且上行子帧 n 对应的 $M \geq 1$，且 UE 配置了多于一个 serving cell，UE 将会在上行子帧 n 使用 PUCCH 3 资源 $n_{\mathrm{PUCCH}}^{(3,\tilde{p})}$ 或 PUCCH 1 资源 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 来回复 ACK/ANCK。规则如下：（见 36.213 的 10.1.3.2.2 节）

情况一：如果 UE 只在 PCell 的子帧 $n-k_m$ 上，只检测到一个带 PDCCH 的 PDSCH 传输且该 PDCCH 中带的 DAI 值为 1，或只检测到指示下行 SPS 释放的 PDCCH 且该 PDCCH 中带的 DAI 值

为 1（UE 没有在 SCell 收到任何下行传输），则 UE 会使用 PUCCH format 1a/1b 来回复 ACK/NACK。使用的 PUCCH 1 资源 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 为：对于天线端口 P0，$n_{\mathrm{PUCCH}}^{(1,\tilde{p}_0)} = (M-m-1)\cdot N_c + m\cdot N_{c+1} + n_{\mathrm{CCE},m} + N_{\mathrm{PUCCH}}^{(1)}$；当使用 2 天线传输时，对于天线端口 P1，$n_{\mathrm{PUCCH}}^{(1,\tilde{p}_1)} = n_{\mathrm{PUCCH}}^{(1,\tilde{p}_0)} + 1$。也就是说，如果在 HARQ 反馈窗口内，eNodeB 只在 PCell 上只发送一个下行传输（带 PDCCH 的 PDSCH 传输或指示下行 SPS 释放的 PDCCH，此时 DAI 值为 1），则 UE 会使用 PUCCH format 1a/1b。（见图 13-20）

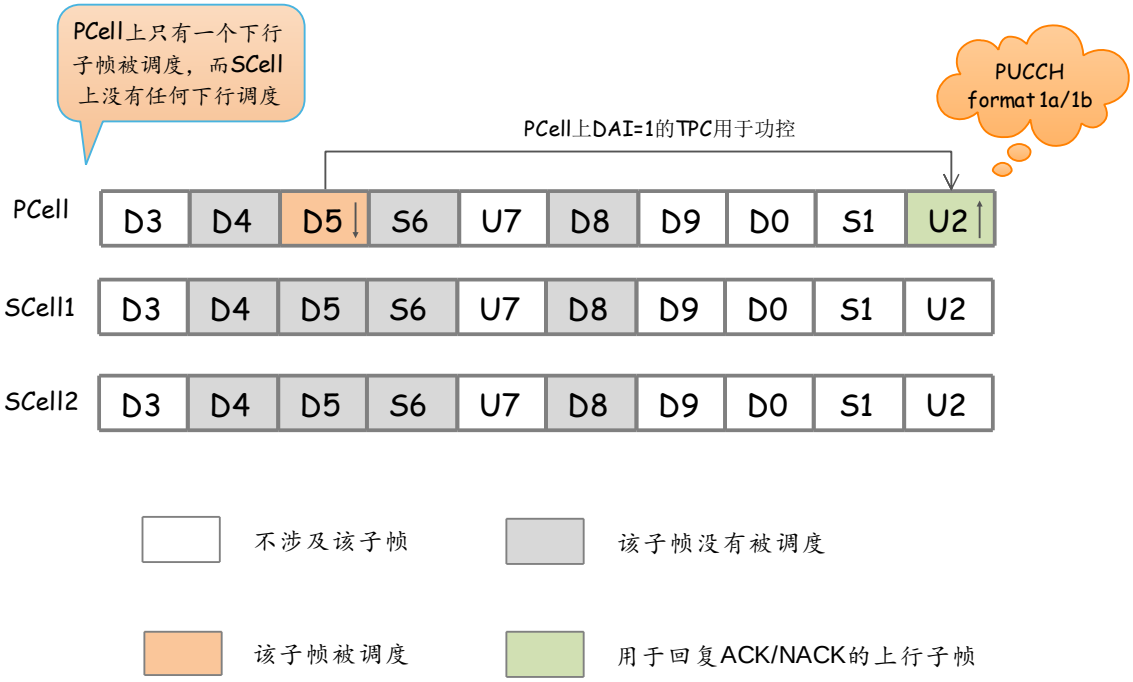


图 13-20：TDD 2 下，情况一举例

情况二：如果 UE 只在 PCell 的子帧 n – k 上，只检测到一个不带 PDCCH 的 PDSCH 传输，并且没有检测到指示下行 SPS 释放的 PDCCH（UE 没有在 SCell 收到任何下行传输），则 UE 会使用 PUCCH format 1a/1b 来回复 ACK/NACK。使用的 PUCCH 1 资源 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 会根据配置的 *n1PUCCH-AN-PersistentList* 和 *n1PUCCH-AN-PersistentListP1-r10* 以及最近一个指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段查 36.213 的 Table 9.2-2 得到。前面已经介绍过，只有 PCell 支持 SPS，且 SPS 的周期最小为 10 ms，且 n - k（其中 $k \in K$）必然落在同一个 10 ms 范围内，因此，在一个 HARQ 反馈窗口内，没有 PDCCH 的 PDSCH 传输至多出现 1 次。因此这里可以理解为，如果只在 PCell 上，只发送了一个不带 PDCCH 的 PDSCH，则使用 PUCCH format 1a/1b。（类似图 13-20，但此时没有 TPC 字段）

情况三：对于 $M>1$，且 UE 在 PCell 的子帧 n–k 上检测到一个不带 PDCCH 的 PDSCH 传输，且 UE 只在 PCell 的子帧 $n-k_m$ 上检测到一个额外的带 PDCCH 的 PDSCH 传输且该 PDCCH 中带的 DAI 值为 1，或检测到指示下行 SPS 释放的 PDCCH 且该 PDCCH 中带的 DAI 值为 1（UE 没有在 SCell 收到任何下行传输），则 UE 会根据 36.213 的 Table 10.1.3.2-1（对应 $A=2$）和 Table 10.1.3.2-2（对应 $A=3$），从 A 个 PUCCH 1 资源中选择出 1 个用于发送 PUCCH format 1b。如果 UE 在 PCell 上配置的 TM 模式支持 2 个 TB，则 A=3，否则 A=2。这种场景下，eNodeB 至多有 2 个下行子帧发送了下行传输，一个是 SPS 子帧上的 PDSCH（从 36.213 的 Table 7.1-6 看，SPS 调度是不支持空分复用的，即在一个 TTI 只传输一个 TB），对应 HARQ-ACK(0)；一个是动态调度或下行 SPS 激活子帧上传输的 PDSCH 或指示下行 SPS 释放的 PDCCH，可能支持空分复用（动态调度的情况），即支持 2 个 TB，对应 HARQ-ACK(1)和 HARQ-ACK(2)，所以 A 的值最大为 3，最小为 2。这 A 个 PUCCH 1 资源是通过如下方式计算的：

- $n^{(1)}_{\mathrm{PUCCH},0}$ 是会根据配置的 *n1PUCCH-AN-PersistentList* 以及最近一个指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段查 36.213 的 Table 9.2-2 得到。
- $n^{(1)}_{\mathrm{PUCCH},1}=(M-m-1)\cdot N_c+m\cdot N_{c+1}+n_{\mathrm{CCE},m}+N^{(1)}_{\mathrm{PUCCH}}$。如果 UE 在 PCell 上配置的 TM 模式支持 2 个 TB，则 $n^{(1)}_{\mathrm{PUCCH},2}=n^{(1)}_{\mathrm{PUCCH},1}+1$。

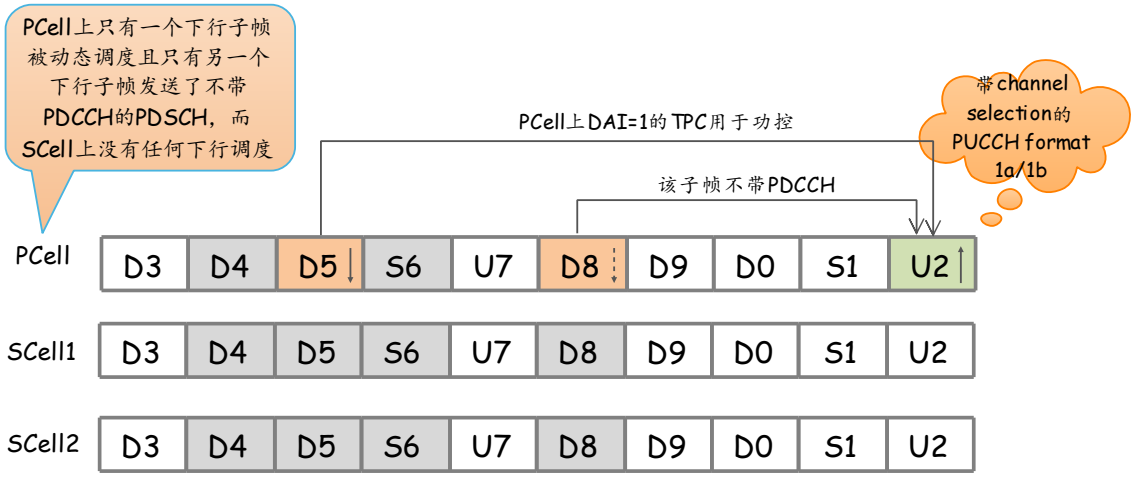


图 13-21：TDD 2 下，情况三举例

情况四：对于 $M>1$，且 UE 仅在 PCell 的子帧 $n-k_m$ 上检测到一个带 PDCCH 的 PDSCH 且该 PDCCH 上的 DAI 值大于 1，或检测到一个指示下行 SPS 释放的 PDCCH 且该 PDCCH 上的 DAI 值大于 1，则 UE 会使用 PUCCH format 3 来回复 ACK/NACK。使用的 PUCCH 3 资源 $n^{(3,\tilde{p})}_{\mathrm{PUCCH}}$ 是通过

如下方式得到的：eNodeB 之前通过 *n3PUCCH-AN-List-r10*（对应天线端口 P0）和 *n3PUCCH-AN-ListP1-r10*（2 天线传输时，对应天线端口 P1），给 UE 配置了 4 个（或 4 对）PUCCH 3 资源。此时 UE 根据 PCell 上收到的 DAI 值大于 1 的 PDCCH 里的 TPC command for PUCCH 字段查 36.213 的 Table 10.1.2.2.2-1，从 4 个（或 4 对）PUCCH 3 资源中选择一个来回复 ACK/NACK。UE 在某个子帧，在 PCell 接收到的所有 DAI 值大于 1 的下行 DCI 的 TPC 字段必须是相同的，即需要指定相同的 PUCCH 3 资源。这种场景下，UE 在 PCell 上至少接收到 2 个带 PDCCH 的 PDSCH 传输（必然有一个 DAI > 1），此时使用 PUCCH format 3 来回复 ACK/ANCK。（见图 13-22）

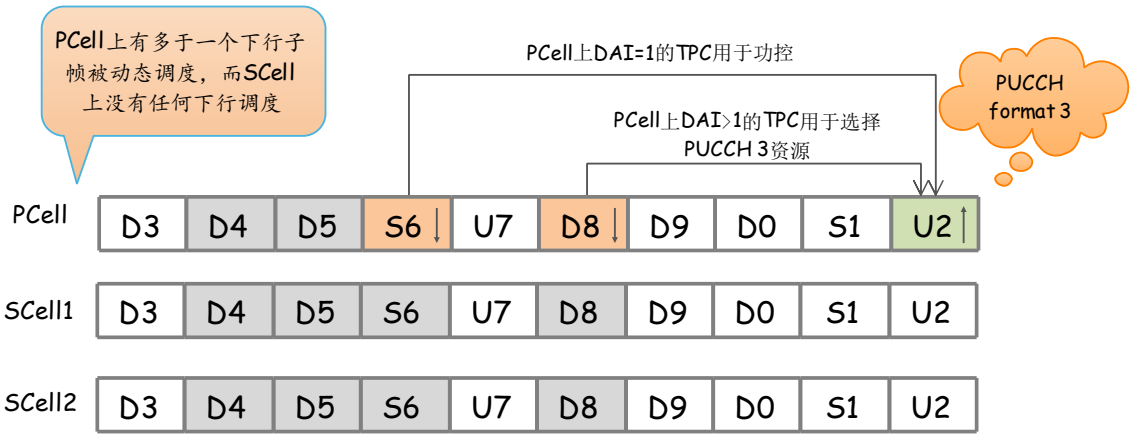


图 13-22：TDD 2 下，情况四举例

情况五：如果 UE 在 SCell 的在子帧 n - k 上（即在 HARQ 反馈窗口内）检测到带 PDCCH 的 PDSCH 传输（SCell 上只有动态调度，不存在 SPS 调度），则 UE 会使用 PUCCH format 3 来回复 ACK/NACK。使用的 PUCCH 3 资源是通过如下方式得到的：eNodeB 之前通过 *n3PUCCH-AN-List-r10*（对应天线端口 P0）和 *n3PUCCH-AN-ListP1-r10*（2 天线传输时，对应天线端口 P1），给 UE 配置了 4 个（或 4 对）PUCCH 3 资源。此时 UE 根据 SCell 上收到的 PDCCH 里的 TPC command for PUCCH 字段查 36.213 的 Table 10.1.2.2.2-1，从 4 个（或 4 对）PUCCH 3 资源中选择一个来回复 ACK/NACK。UE 在某个子帧，在所有 SCell 接收到的下行 DCI 的 TPC 字段必须是相同的，即需要指定相同的 PUCCH 3 资源。该 TPC 字段也必须与前面介绍的 PCell 上接收到的 DAI 大于 1 的 PDCCH 中的 TPC 字段相同。（见图 13-23 和图 13-24）

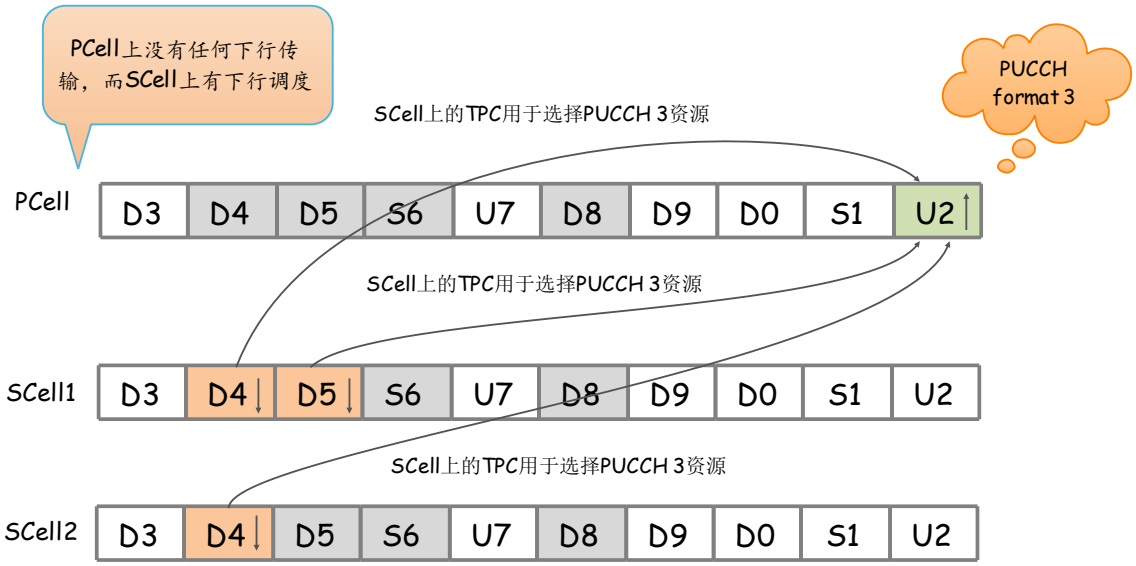


图 13-23：TDD 2 下，情况五举例

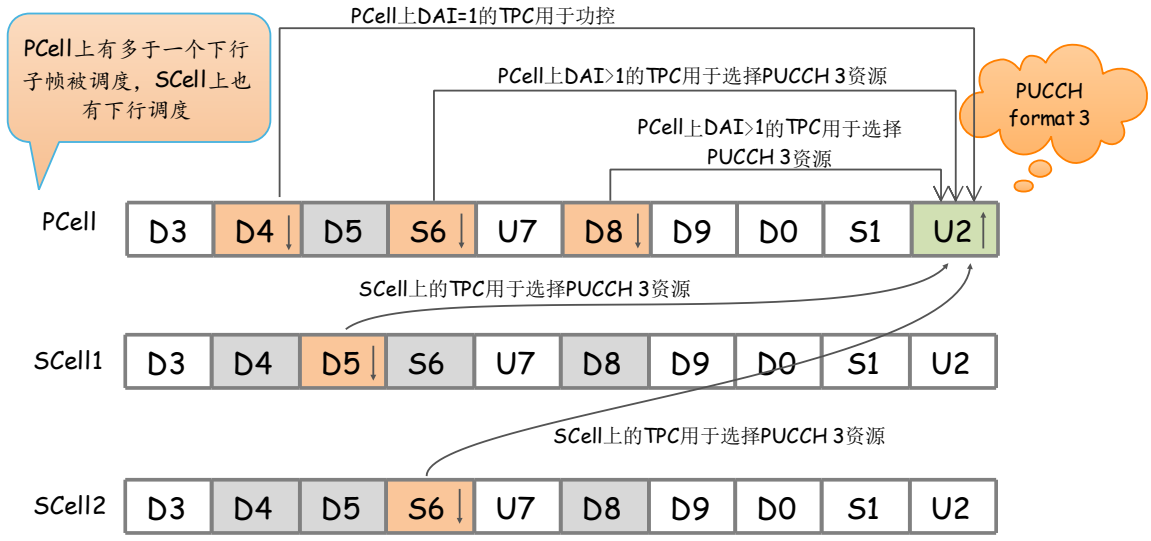


图 13-24 ：TDD 2 下，情况五举例

注：（1）对于 TDD，PCell 上收到的 DAI = 1 的下行 DCI 的 TPC command for PUCCH 字段（除指示下行 SPS 激活的 DCI 中的 TPC command for PUCCH 字段外）都是用于 PUCCH 功控的；而 PCell 上收到的 DAI > 1 的下行 DCI 的 TPC command for PUCCH 字段和所有 SCell 上收到的所有下行 DCI 的 TPC command for PUCCH 字段都是用于指示选择哪个 PUCCH 3 资源的，且 UE 在 PCell

上收到的 DAI > 1 的下行 DCI 的所有 TPC command for PUCCH 字段与 SCell 接收到的 TPC command for PUCCH 字段必须是相同的；（2）对于 TDD，PUCCH format 3 的触发条件为“（PCell 上动态调度 + 下行 SPS 激活的 PDCCH 数）大于等于 2”或“SCell 上有 PDSCH 传输”，或二者兼有之；（3）与 PUCCH format 1b with channel selection 中 PUCCH 1 的资源由 UE 来选择不同，PUCCH format 3 中 PUCCH 3 资源的选择是由 eNodeB 来完成的，并通过 TPC 字段告知 UE。

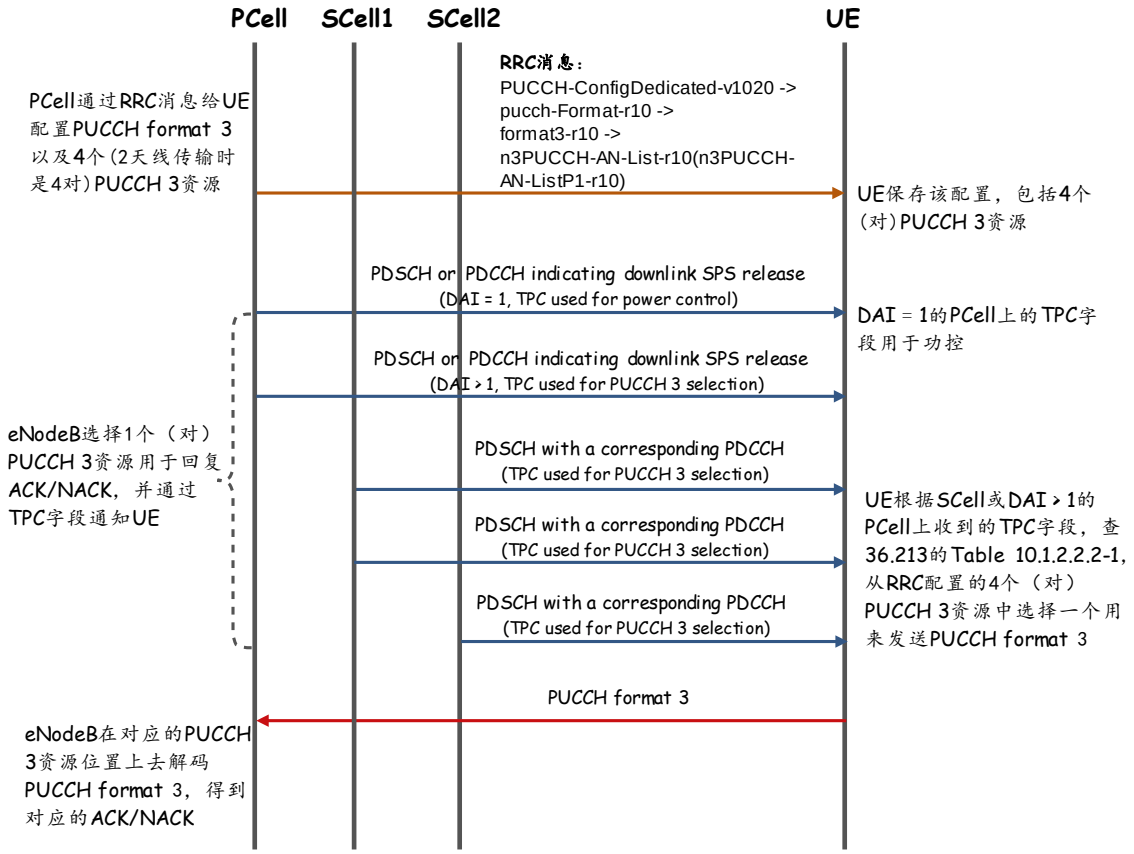


图 13-25：TDD 下，PUCCH format 3 的处理流程

对于 TDD，当 UE 配置了 PUCCH format 3 时，第 c 个 serving cell 的 HARQ 反馈比特为 $o_{c,0}^{ACK}\ o_{c,1}^{ACK},...,o_{c,O_c^{ACK}-1}^{ACK}$，其中 c≥0。当 UE 在第 c 个 serving cell 配置的 TM 模式只支持 1 个 TB 或应用了 spatial HARQ-ACK bundling 时，$O_c^{ACK} = B_c^{DL}$；而其它情况下（第 c 个 serving cell 配置的 TM 模式支持 2 个 TB），$O_c^{ACK} = 2B_c^{DL}$，其中 B_c^{DL} 为 UE 在第 c 个 serving cell 中需要反馈 ACK/NACK 比特的下行子帧数。（见 36.213 的 7.3 节）

- 当 UE 在 PUCCH 上传输时，如果在 HARQ 反馈窗口内的特殊子帧不是“带正常的循环前缀且特殊子帧配置为 0 和 5”，或不是“带扩展的循环前缀且特殊子帧配置为 0 和 4”（这两种情况下的特殊子帧的 DwPTS 是不能用于传输 PDSCH 的，即不用反馈 ACK/NACK），则 $B_c^{DL} = M$ ；否则， $B_c^{DL} = M - 1$。

- 对于 TDD 0（一个上行子帧对应一个下行子帧），或一个不带 DCI format 0/4 的 PUSCH 传输（在周期性上行 SPS 子帧上发送的 PUSCH，不包含上行 SPS 激活的 PDCCH 对应的 PUSCH），如果在 HARQ 反馈窗口内的特殊子帧不是“带正常的循环前缀且特殊子帧配置为 0 和 5”，或不是“带扩展的循环前缀且特殊子帧配置为 0 和 4”，则 $B_c^{DL} = M$ ；否则，$B_c^{DL} = M - 1$。如果在 HARQ 反馈窗口内，UE 没有收到任何 PDSCH 或指示下行 SPS 释放的 PDCCH，则 UE 不会反馈 ACK/NACK。

- 对于 TDD 1/2/3/4/6 并且 PUSCH 传输是基于检测到的 DCI format 0/4 的（动态调度或上行 SPS 激活），则 UE 会假定 $B_c^{DL} = W_{DAI}^{UL}$。如果在 HARQ 反馈窗口内，UE 没有收到任何 PDSCH 或指示下行 SPS 释放的 PDCCH，并且 $W_{DAI}^{UL} = 4$，则 UE 不会反馈 ACK/NACK。

- 对于 TDD 5 且 PUSCH 传输是基于检测到的 DCI format 0/4 的（动态调度或上行 SPS 激活），则 UE 会假定 $B_c^{DL} = W_{DAI}^{UL} + 4\left\lceil \left(U - W_{DAI}^{UL}\right)/4 \right\rceil$。其中 U 表示在所有配置的 serving cell 中的最大 U_c 值，而 U_c 是 UE 在第 c 个 serving cell 中的 HARQ 反馈窗口内收到的 PDSCH 数和指示下行 SPS 释放的 PDCCH 数的总和。如果在 HARQ 反馈窗口内，UE 没有收到任何 PDSCH 或指示下行 SPS 释放的 PDCCH，并且 $W_{DAI}^{UL} = 4$，则 UE 不会反馈 ACK/NACK。

关于 serving cell c 所对应的 serving cell（即与 *sCellIndex-r10* 的对应关系）见第 23.1 节中关于“PCell / SCell / Serving Cell”的介绍。

对于 TDD 且 UE 配置了 PUCCH format 3，

- 对于 TDD 1~6，UE 在第 c 个 serving cell 的子帧 n–k 收到的带 PDCCH 的 PDSCH 传输，或收到的指示下行 SPS 释放的 PDCCH，在 第 c 个 serving cell 配置的 TM 模式只支持 1 个 TB（或使用了 spatial HARQ-ACK bundling）的情况下对应比特 $o_{c,DAI(k)-1}^{ACK}$ ；在第 c 个 serving cell 配置的 TM 模式支持 2 个 TB 的情况下对应比特 $o_{c,2DAI(k)-2}^{ACK}$ 和 $o_{c,2DAI(k)-1}^{ACK}$。其中 *DAI(k)*是 UE 在下行子帧 n–k 检测到的 DCI format 1A/1B/1D/2/2A/2B/2C 上携带的 DAI 值，且 $o_{c,2DAI(k)-2}^{ACK}$ 和 $o_{c,2DAI(k)-1}^{ACK}$ 分别对应 codeword 0 和 codeword 1。如果 $N_{SPS} > 0$，即在下行 SPS 子帧上有 PDSCH 传输，且该 PDSCH 没有对应的 PDCCH，则该子帧对应的 ACK/NACK 在比特 $o_{O^{ACK}-1}^{ACK}$（即最后一个比特）上发送。如果 eNodeB 在某个下行子帧发送了带 PDCCH 的 PDSCH，或发送了指示下行 SPS 释放的 PDCCH（即携带了 *DAI(k)*），但

UE 没有收到（即丢失了下行传输），则 UE 会将该 *DAI(k)*对应的 ACK/NACK 比特会设置成 NACK。

- 对于 TDD 0，UE 在第 c 个 serving cell 的子帧 n–k 收到的带 PDCCH 的 PDSCH 传输，或收到的指示下行 SPS 释放的 PDCCH，在 第 c 个 serving cell 配置的 TM 模式只支持 1 个 TB（或使用了 spatial HARQ-ACK bundling）的情况下对应比特 $o_{c,0}^{ACK}$ ；在第 c 个 serving cell 配置的 TM 模式支持 2 个 TB 的情况下对应比特 $o_{c,0}^{ACK}$ 和 $o_{c,1}^{ACK}$ ，其中 $o_{c,0}^{ACK}$ 和 $o_{c,1}^{ACK}$ 分别对应 codeword 0 和 codeword 1。对应某个 HARQ 反馈比特，如果没有检测到任何 PDSCH 传输或没有检测到指示下行 SPS 释放的 PDCCH，则该比特会设置成 NACK。

从上面的介绍可以看出，对于 TDD 且配置了 PUCCH format 3 用于反馈 ACK/NACK 的情况，“带正常的循环前缀且特殊子帧配置为 0 和 5”，或“带扩展的循环前缀且特殊子帧配置为 0 和 4”的特殊子帧，在计算要反馈的 ACK/NACK 比特数时是排除在外的，该特殊子帧也不会用于发送指示下行 SPS 释放的 PDCCH。

对于 TDD 而言，在一个上行子帧中，可能需要反馈多于 20 个 ACK/NACK 比特的信息。（例如：一个 UE 至多可配置 5 个 serving cell，每个 serving cell 都配置了 MIMO，此时每个 serving cell 需要 2 个 ACK/NACK 比特。在 TDD 2 下的子帧 2，此时所需要的 ACK/NACK 比特信息的数量最多为 5 * 2 * 4 = 40）

对于 TDD 而言，PUCCH format 3 支持最多 20 个 ACK/NACK 比特的负载。如果多个下行子帧对应一个上行子帧，且需要反馈的 ACK/NACK 比特数大于 20，则对于每一个支持空分复用的 serving cell 的下行子帧的 2 个 codeword 需要做 ACK/NACK 比特的“spatial bundling”处理（同一 serving cell 的同一下行子帧的 2 个 codeword 对应的 ACK/NACK 做 logical AND 操作）。然后再按照 index 由低到高的顺序将每个 serving cell 所要反馈的 ACK/NACK 比特串联在一起，就得到了 PUCCH format 3 中携带的信息。具体可见 36.212 的 5.2.3.1 节，这里就不做详细介绍了。

【参考资料】

[1] TS 36.213 的 7.3 节 UE procedure for reporting HARQ-ACK

[2] TS 36.213 的 10.1.1 节 PUCCH format information

13.6 下行 HARQ 的冲突处理

之前介绍的都是单独传输 HARQ ACK/NACK 时的处理。当 UE 需要在同一子帧使用 PUCCH 来同时传输 ACK/NACK、SR、CSI 时，会有不同的处理方式。接下来，我们会分别从 FDD 和 TDD 的角度来描述 ACK/NACK 与 SR/CSI 冲突时的处理。

13.6.1 ACK/NACK 与 SR 的冲突处理

首先，我们分别从 FDD 和 TDD 的角度来描述 ACK/NACK 与 SR 冲突时的处理。

13.6.1.1 FDD

FDD 下，UE 需要在同一上行子帧同时发送 SR 和 ACK/NACK，且 UE 被分配了 SR 资源（PUCCH format 1 资源）时，分为以下几种场景：（见 36.213 的 7.3 节）

场景 1：FDD 下，UE 需要发送 PUCCH format 1a/1b（ACK/ANCK），如果同时需要发送 SR，则使用 SR 资源来发送 PUCCH format 1a/1b（携带 SR 信息）；如果不需要发送 SR，则使用分配的 PUCCH format 1a/1b 资源来发送 PUCCH format 1a/1b（不携带 SR 信息）。

场景 2：FDD 下，UE 需要发送 PUCCH format 1b with channel selection，如果此时不需要发送 SR，则使用 PUCCH format 1b with channel selection 中选择出的 PUCCH 1 资源来发送 ACK/NACK（见本章的 13.5.1.1 节或 36.213 的 10.1.2.2.1 节）；如果同时需要发送 SR，则 UE 先要对每个 serving cell 的 ACK/NACK 进行“spatially-bundled”处理（注：这里与情况一有所不同），然后使用 SR 资源来发送 PUCCH format 1b（携带 SR 信息。注意：此时没有 channel selection 的信息，只有 PUCCH format 1b 携带的 2 比特信息），此时 PUCCH format 1b 携带的 2 比特信息 $b(0),b(1)$ 按照如下方式计算：

- 如果某个 serving cell 上只有一个 TB 或一个指示下行 SPS 释放的 PDCCH，则该 serving cell 的一个 ACK/NACK 比特正好对应一个 TB 或一个指示下行 SPS 释放的 PDCCH；
- 如果某个 serving cell 上有 2 个 TB，则对应这 2 个 TB 的 2 个 ACK/NACK 比特会先进行“spatially-bundled”处理，得到对应该 serving cell 的 1 比特信息；
- 如果某个 serving cell 上既没有收到 PDSCH 传输，也没有收到一个指示下行 SPS 释放的 PDCCH，则对应该 serving cell 的 1 比特信息设置为 NACK。

从上面的介绍可以看出，最终每个 serving cell 都有 1 比特的 ACK/NACK 信息。其中 PCell 上的 ACK/NACK 比特对应 $b(0)$，SCell 上的 ACK/NACK 比特对应 $b(1)$。

场景 3：FDD 下，UE 需要同时发送 PUCCH format 3 和 SR，则 SR 会复用到 PUCCH format 3 中并使用分配的 PUCCH format 3 资源发送。但需要注意的是：如果 UE 配置了 PUCCH format 3，但只在 PCell 上存在 PDSCH 传输，或只在 PCell 上存在一个指示下行 SPS 释放的 PDCCH（SCell 上不存在任何 PDSCH 传输），则 UE 会使用 PUCCH format 1a/1b 来发送 ACK/NACK，此时与 SR 同时传输的处理与场景 1 相同。

13.6.1.2 TDD

TDD 下，UE 需要在同一上行子帧同时发送 SR 和 ACK/NACK，且 UE 被分配了 SR 资源（PUCCH format 1 资源）时，分为以下几种场景：（见 36.213 的 7.3 节）

场景 1：TDD 下， UE 配置了 HARQ bundling，或 HARQ multiplexing，或 PUCCH format 1b with channel selection：如果此时不需要发送 SR，则 UE 会根据本章的 13.4 节和 13.5.1.2 节的介绍，在分配给 UE 的 HARQ 资源上发送 PUCCH format 1a/1b；如果需要在同一子帧发送 ACK/NACK 和 SR，则使用 SR 资源来发送 PUCCH format 1b，并携带 2 比特的 $b(0),b(1)$ 信息。其中 $b(0),b(1)$ 的值是基于 $N_{SPS}+\sum_{c=0}^{N_{cells}^{DL}-1}U_{DAI,c}$ 个 HARQ 回应中包含多少个 ACK（注意：不包含 NACK），然后查 36.213 的 Table 7.3-1 得到的。其中同一子帧上的多个 codeword 的 ACK/NACK 需要做 spatial HARQ bundling 处理，即一个子帧只会对应一个 ACK 或一个 NACK。对于 TDD 1~6，如果 $\sum_{c=0}^{N_{cells}^{DL}-1}U_{DAI,c}>0$，并且某个 serving cell 的 $V_{DAI,c}^{DL}\neq(U_{DAI,c}-1)\bmod 4+1$，则 UE 检测到丢失了至少一个下行分配（即下行 DCI）。

注：关于 N_{SPS} 和 $U_{DAI,c}$ 的定义，可参见 13.3 节的介绍。

Table 7.3-1: Mapping between multiple HARQ-ACK responses and $b(0),b(1)$

Number of ACK among multiple ($N_{SPS}+\sum_{c=0}^{N_{cells}^{DL}-1}U_{DAI,c}$) HARQ-ACK responses	$b(0),b(1)$
0 or None (UE detect at least one DL assignment is missed)	0, 0
1	1, 1
2	1, 0
3	0, 1
4	1, 1
5	1, 0
6	0, 1
7	1, 1
8	1, 0
9	0, 1

从前面的介绍可以看出，TDD 下，需要同时发送 SR 和“1 或 2 比特的 ACK/NACK”时，UE 会通过携带 2 比特的 $b(0),b(1)$ 信息，并使用 PUCCH format 1b（携带 SR）来表示，因此 TDD 下并不存在携带 SR 信息的 PUCCH format 1a。这从 36.213 的 10.1.1 节关于 Format 1a 的介绍也可以看出：

- Format 1a for 1-bit HARQ-ACK or in case of FDD for 1-bit HARQ-ACK with positive SR（只有FDD下，才存在带SR的PUCCH format 1a）

场景 2：TDD 下，UE 需要同时发送 PUCCH format 3 和 SR，则 SR 会复用到 PUCCH format 3 中并使用分配的 PUCCH format 3 资源发送。但需要注意的是：即使 UE 配置了 PUCCH format 3，在以下几种情况（在 13.5.2.2 节也有介绍）下，UE 会使用 PUCCH format 1a/1b 来发送 ACK/NACK。而如果此时需要同时发送 ACK/NACK 和 SR，则会按照场景 1 的介绍来处理。

- 如果 UE 只在 PCell 的子帧 $n-k_m$ 上，只检测到一个带 PDCCH 的 PDSCH 传输且该 PDCCH 中带的 DAI 值为 1，或只检测到指示下行 SPS 释放的 PDCCH 且该 PDCCH 中带的 DAI 值为 1（UE 没有在 SCell 收到任何下行传输）；
- 如果 UE 只在 PCell 的子帧 n – k 上，只检测到一个不带 PDCCH 的 PDSCH 传输（下行周期性 SPS 子帧上发送的 PDSCH），并且没有检测到指示下行 SPS 释放的 PDCCH（UE 没有在 SCell 收到任何下行传输）；
- 对于 $M>1$，且 UE 在 PCell 的子帧 n – k 上检测到一个不带 PDCCH 的 PDSCH 传输，且 UE 只在 PCell 的子帧 $n-k_m$ 上检测到一个额外的带 PDCCH 的 PDSCH 传输且该 PDCCH 中带的 DAI 值为 1，或检测到指示下行 SPS 释放的 PDCCH 且该 PDCCH 中带的 DAI 值为 1（UE 没有在 SCell 收到任何下行传输）。

注：PUCCH format 1a/1b 是否携带 SR 是通过在传输时使用不同的 PUCCH 1 资源（是使用 SR 资源，还是使用 ACK/NACK 资源）来表示的。具体处理可参见 8.3 节的介绍。

13.6.2 ACK/NACK 与 CSI 的冲突处理

接下来，我们将介绍 UE 需要在同一上行子帧同时发送 CSI 和 ACK/NACK，且 UE 被分配了 CSI 资源（周期性的 PUCCH 2 资源）时，如何进行 ACK/NACK 和 CSI 的传输。

当在一个发送 PUSCH 的子帧上，需要同时发送一个周期性的 CSI 上报和 ACK/NACK 时，如果 *simultaneousPUCCH-PUSCH*（RRC 配置是否同时发送 PUCCH 和 PUSCH）设置为 FALSE（即不允许同时发送 PUCCH 和 PUSCH），则周期性的 CSI 会与 ACK/NACK 复用在一起，并在 PUSCH 上传输；否则如果 *simultaneousPUCCH-PUSCH* 设置为 TRUE（即允许同时发送 PUCCH 和 PUSCH），则 ACK/NACK 会在 PUCCH 上传输，而周期性的 CSI 会在 PUSCH 上传输。（既适用于 FDD，也适用于 TDD）。

UE 只配置了一个 serving cell 并且没有配置 PUCCH format 3，当在一个没有发送 PUSCH 的子帧上，需要同时发送一个周期性的 CSI 上报和 ACK/NACK 时，如果 *simultaneousAckNackAndCQI*（RRC 配置是否同时发送 CQI 和 ACK/NACK）设置为 TRUE，则 ACK/NACK 会和周期性 CSI 上报复用在一起，并使用 PUCCH format 2a/2b 发送（此时 TDD 下的处理见后面介绍的**场景 1**）；如果 *simultaneousAckNackAndCQI* 设置为 FALSE，则丢弃 CSI，并使用 PUCCH format 1a/1b 来发送 ACK/NACK（既适用于 FDD，也适用于 TDD）。

在 FDD 下，UE 配置了多于一个 serving cell，当在一个没有发送 PUSCH 的子帧上，需要同时发送一个周期性的 CSI 上报和 ACK/NACK 时，如果 *simultaneousAckNackAndCQI* 设置为 TRUE，并且如果 ACK/NACK 对应只在 PCell 上传输的一个 PDSCH 或指示下行 SPS 释放的 PDCCH（UE 没有在 SCell 收到任何下行传输），则 ACK/NACK 会和周期性 CSI 上报复用在一起，并使用 PUCCH format 2a/2b 发送；否则丢弃 CSI，只发送 ACK/NACK。

在 TDD 下，UE 只配置了一个 serving cell 并且配置了 PUCCH format 3，当在一个没有发送 PUSCH 的子帧上，需要同时发送一个周期性的 CSI 上报和 ACK/NACK 时，如果 *simultaneousAckNackAndCQI* 设置为 TRUE，则周期性 CSI 上报是丢弃还是与 ACK/NACK 复用在一起，根据后面介绍的**场景 3** 来处理；如果 *simultaneousAckNackAndCQI* 设置为 FALSE，则丢弃 CSI，而使用 PUCCH format 1a/1b 来发送 ACK/NACK。

在 TDD 下，UE 配置了多于一个 serving cell，当在一个没有发送 PUSCH 的子帧上，需要同时发送一个周期性的 CSI 上报和 ACK/NACK 时，如果 *simultaneousAckNackAndCQI* 设置为 TRUE，则周期性 CSI 上报是丢弃还是与 ACK/NACK 复用在一起，根据后面介绍的**场景 1**、**场景 2** 或**场景 3** 来处理；否则丢弃 CSI，只发送 ACK/NACK。

场景 1：TDD 下，当需要在 PUCCH 上同时发送 ACK/NACK 和 CSI 时，如果 UE 配置了 HARQ bundling，或 HARQ multiplexing，或 PUCCH format 1b with channel selection，并且 UE 只在 PCell 的子帧 n – k 上收到的 PDSCH 和/或指示下行 SPS 释放的 PDCCH，则对于正常的循环前缀，UE 会使用 PUCCH format 2b 来传输 CSI 和 $b(0),b(1)$；对于扩展的循环前缀，UE 会根据 36.212 的 5.2.3.4 节的介绍使用 PUCCH format 2 来传输 CSI 和 $b(0),b(1)$，其中 a_0'', a_1'' 被 $b(0),b(1)$ 所替代。其中 $b(0),b(1)$ 的值是基于 $N_{SPS} + \sum_{c=0}^{N_{cells}^{DL}-1} U_{DAI,c}$ 个 HARQ 回应中包含多少个 ACK（注意：不包含 NACK），然后查 36.213 的 Table 7.3-1 得到的。其中同一子帧上的多个 codeword 的 ACK/NACK 需要做 spatial HARQ bundling 处理，即一个子帧只会对应一个 ACK 或一个 NACK。对于 TDD 1~6，如果 $\sum_{c=0}^{N_{cells}^{DL}-1} U_{DAI,c} > 0$，并且某个 serving cell 的 $V_{DAI,c}^{DL} \neq (U_{DAI,c} - 1) \bmod 4 + 1$，则 UE 检测到丢失了至少一个下行分配。

注：关于 N_{SPS} 和 $U_{DAI,c}$ 的定义，可参见 13.3 节的介绍。

场景 2：TDD 下，当 UE 需要在同一上行子帧使用 PUCCH 来发送 ACK/NACK 和 CSI 时，如果 UE 配置了 PUCCH format 1b with channel selection，并且在 SCell 的子帧 n – k 上至少收到了一个 PDSCH，则 UE 会丢弃 CSI，并按照本章的 13.5.1.2 节的介绍来发送 ACK/NACK。

场景 3：TDD 下，如果 UE 需要在同一上行子帧发送 ACK/NACK 和 CSI，且配置了 PUCCH format 3，则当满足下面的条件时，UE 会根据 36.212 的 5.2.3.4 节的介绍使用 PUCCH format 2/2a/2b

来同时发送 CSI 和 ACK/NACK；否则 UE 会丢弃 CSI 并根据 13.5.2.2 节的介绍来发送 ACK/NACK（即 PUCCH format 3 和 CSI 不能同时发送）。

- 如果 UE 只在 PCell 的子帧 $n-k_m$ 上，只检测到一个带 PDCCH 的 PDSCH 传输且该 PDCCH 中带的 DAI 值为 1，或只检测到指示下行 SPS 释放的 PDCCH 且该 PDCCH 中带的 DAI 值为 1（UE 没有在 SCell 收到任何下行传输）。
- 如果 UE 只在 PCell 的子帧 n – k 上，只检测到一个不带 PDCCH 的 PDSCH 传输，并且没有检测到指示下行 SPS 释放的 PDCCH（UE 没有在 SCell 收到任何下行传输）。

无论是 FDD 还是 TDD，如果 UE 在一个没有发送 PUSCH 的子帧上，需要同时发送 SR，ACK/NACK 和周期性的 CSI，则 UE 会丢弃 CSI，并按照 ACK/NACK 和 SR 同时传输的场景来处理。

【参考资料】

[1] TS 36.213 的 7.3 节 UE procedure for reporting HARQ-ACK

[2] TS 36.213 的 10.1.1 节 PUCCH format information

13.7 HARQ-ACK Repetition

eNodeB 可以将 UE 配置成在多个连续的上行子帧上重复发送对应同一个 TB 的 ACK/NACK。这样做有益于保证位于小区边界的功率受限的 UE 的 ACK/NACK 信号的可靠性。这个特性称为 HARQ-ACK Repetition（又称为 ACK/NACK Repetition）。

注意：重复发送的是同一 ACK/NACK，而不是某个 TB（重复发送同一个 TB 对应的特性是 TTI bundling，详细内容见第 24 章关于“TTI bundling”的介绍）。

HARQ-ACK Repetition 只适用于使用 PUCCH 反馈 ACK/NACK 的场景。

HARQ-ACK Repetition 是通过 UE 特定的参数 *ackNackRepetition* 来配置的。

```
PUCCH-ConfigDedicated ::=           SEQUENCE {
    ackNackRepetition               CHOICE{
        release                         NULL,
        setup                           SEQUENCE {
            repetitionFactor                ENUMERATED {n2, n4, n6, spare1},------
ACK/NACK重复传输的次数，包含对应初传的ACK/NACK传输在内
            n1PUCCH-AN-Rep                  INTEGER (0..2047) },------HARQ-ACK
Repetition中，在天线端口0上用于回复ACK/NACK的PUCCH 1资源
```

```
        }
    },
    tdd-AckNackFeedbackMode             ENUMERATED {bundling, multiplexing}
    OPTIONAL   -- Cond TDD
}


PUCCH-ConfigDedicated-v1020 ::=     SEQUENCE {

    ......

    n1PUCCH-AN-RepP1-r10                        INTEGER (0..2047)     OPTIONAL   -
- Need OR},------HARQ-ACK Repetition中，在天线端口1上用于回复ACK/NACK的PUCCH 1资源
}
```

如果 UE 使能了 HARQ-ACK Repetition，UE 会重复发送任一 ACK/NACK 传输 N_{ANRep} 次，其中 N_{ANRep} 是通过 *repetitionFactor* 配置的，且该值将对应初传的 ACK/NACK 传输统计在内。

如果 UE 接收到没有带 PDCCH 的 PDSCH 传输（周期性配置的下行 SPS 子帧上发送的 PDSCH），UE 会使用配置给 SPS 的 PUCCH 1 资源来发送对应的 ACK/NACK 反馈 N_{ANRep} 次。该资源通过如下方式指定：

如果 UE 接收到的 DCI 指示 DL SPS 激活，则 DCI 中的 TPC command for PUCCH 字段将指示从 4 个 PUCCH 1 资源中选择 1 个（2 天线传输时是一对）用于回复 ACK/NACK。该 4 个 PUCCH 1 资源是通过 SPS-ConfigDL 的 *n1PUCCH-AN-PersistentList* 来配置的；对于 2 天线传输，第二个天线端口的 4 个 PUCCH 1 资源通过 *n1PUCCH-AN-PersistentListP1-r10* 配置。

如果 UE 接收到的 PDSCH 传输有一个对应的 PDCCH，或接收到一个指示下行 SPS 释放的 PDCCH，则对应初传的 ACK/NACK 使用的 PUCCH 1 资源会基于对应 PDCCH 的 first CCE index 计算得出（详见 36.213 的 10.1.2 节和 10.1.3 节），而后续的 $N_{\text{ANRep}}-1$ 次重复的 ACK/NACK 反馈会使用 $n^{(1,\tilde{p})}_{\text{PUCCH, ANRep}}$ 资源来传输。其中 $n^{(1,\tilde{p})}_{\text{PUCCH, ANRep}}$ 是通过 *n1PUCCH-AN-Rep*（对应天线端口 0）和 *n1PUCCH-AN-RepP1-r10*（2 天线传输时，对应天线端口 1）来配置的。

在 FDD 或 TDD 下， HARQ-ACK Repetition 都只适用于 UE 配置了一个 serving cell 的场景，即不支持载波聚合。对于 TDD，HARQ-ACK Repetition 只适用于 HARQ bundling。

HARQ-ACK Repetition 并不支持载波聚合。这是因为它会影响下行的性能，并且载波聚合通常假定控制信令的传输并不会遇到传输功率不足的问题。

对于 FDD，如果 UE 使能了 HARQ-ACK Repetition，且在子帧 n - 4 检测到了 PDSCH 传输，且 UE 没有在子帧 n 发送对应下行子帧 $n-N_{\text{ANRep}}-3, \ldots, n-5$ 的 PDSCH 传输的 ACK/NACK（重复发送的），则：

- UE 在上行子帧 n，$n+1$，…，$n+N_{\text{ANRep}}-1$ 的 PUCCH 上只会发送 ACK/NACK（对应子帧 n - 4 上检测到的 PDSCH 传输）；
- UE 不会在上行子帧 n，$n+1$，…，$n+N_{\text{ANRep}}-1$ 上发送任何其它信号。
- 对应在下行子帧 $n-3$，…，$n+N_{\text{ANRep}}-5$ 上检测到的 PDSCH 传输，UE 不会发送任何重复的 ACK/NACK，即不做 HARQ-ACK Repetition 处理，但第一个 ACK/NACK 还是要发送的。也就是说，此时对应下行子帧 $n-3$，…，$n+N_{\text{ANRep}}-5$ 的 PDSCH 传输，只会发送一个 ACK/NACK。（注：用于发送重复 ACK/NACK 的 PUCCH 1 资源只有一个或一对，该资源不能同时给 2 个下行子帧发送的 PDSCH 反馈 ACK/NACK）

图 13-26 是 FDD 下，HARQ-ACK Repetition 的一个例子，其中 $N_{\text{ANRep}}=4$。假设配置了 HARQ-ACK Repetition 的 UE 只在子帧 n - 4 和 n – 3 收到了 PDSCH 传输，则 UE 会在子帧 n、n + 2、n + 3 重复发送对应子帧 n - 4 的 ACK/NACK；而在子帧 n + 1 只会发送对应子帧 n - 3 的 ACK/NACK。

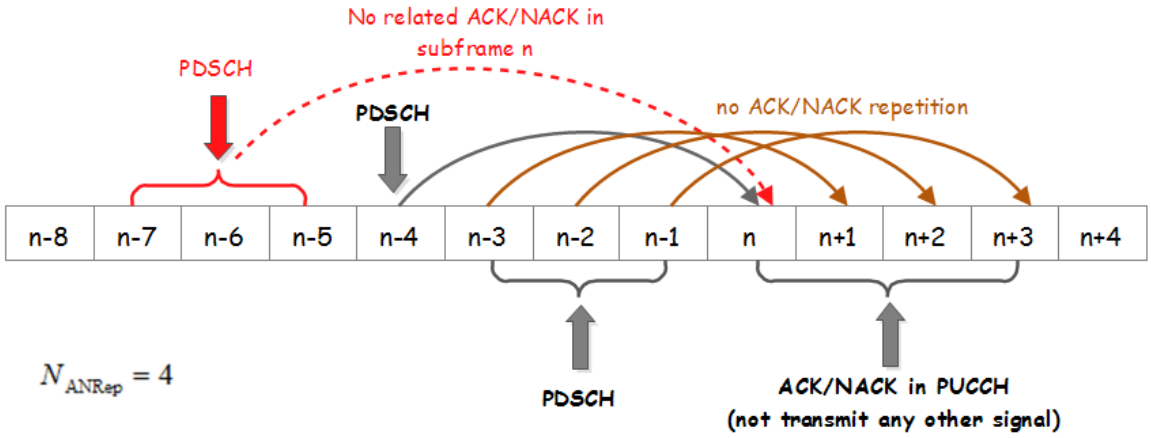


图 13-26：FDD 下的 HARQ-ACK Repetition 的例子，其中 repetitionFactor = 4

在 TDD 下，如果 UE 使能了 HARQ-ACK Repetition，且在子帧 n - k 检测到了 PDSCH 传输，其中 $k \in K$ 且 K 在 36.213 的 Table 10.1.3.1-1 中定义（即在 HARQ 反馈窗口内），且 UE 没有在子帧 n 发送对应早于子帧 n – k 发送的 PDSCH 传输的 ACK/NACK（重复发送的），则：

- UE 在上行子帧 n 以及后续的 $N_{\text{ANRep}}-1$ 个上行子帧（标记为 n_1，…，$n_{N_{\text{ANRep}}-1}$）的 PUCCH 上只会发送 ACK/NACK（对应子帧 n - k 上检测到的 PDSCH 传输）；

- UE 不会在上行子帧 n，n_1，…，$n_{N_{ANRep}-1}$ 上发送任何其它信号；
- 对应在下行子帧 n_i-k 上检测到 PDSCH 传输，其中 $k \in K_i$ 且 K_i 对应上行子帧 n_i（$1 \le i \le N_{ANRep}-1$）并在 36.213 的 Table 10.1.3.1-1 中定义（即在 HARQ 反馈窗口内），则 UE 不会发送任何重复的 ACK/NACK，即不做 HARQ-ACK Repetition 处理，但第一个 ACK/NACK 还是要发送的。也就是说，此时对应下行子帧 n_i-k 的 PDSCH 传输，只会发送一个 ACK/NACK。

TDD 下，HARQ-ACK Repetition 处理与 FDD 类似，只是 timing 有所不同而已，这里就不举例了。

【参考资料】

[1] TS 36.213 的 10.1.4 节 HARQ-ACK Repetition procedure

[2] TS 36.213 的 10.2 节 Uplink HARQ-ACK timing

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.5 节和 28.3.2.1 节

[4] 《Method and system for dynamic ACK/NACK repetition for robust downlink MAC PDU transmission in LTE》

13.8 广播 HARQ

HARQ 只支持 DL-SCH 和 UL-SCH 的传输。广播类的消息（如 Paging、MIB、RAR 等）是针对多个 UE 的，所以并不支持 HARQ。

但 SI 消息也有 HARQ，该 HARQ 没有用来指示是否成功接收的反馈信息，即无 ACK/NACK，也无需 NDI。

SI 消息使用专用的广播 HARQ process 来接收和处理。

如果 UE 需要读取 SI 消息，UE 会基于 RRC 所给的调度信息在 PCell 上接收使用 SI-RNTI 加扰的 PDCCH（DCI format 1A/1C）：如果是 DCI format 1A，则该 DCI 携带了 RV 信息，直接使用该 RV 信息；如果是 DCI format 1C，则该 DCI 不带 RV 信息，此时会根据如下规则来确定 RV：当前 TTI 的 RV 通过公式 RV_K = ceiling(3/2*k) modulo 4 来计算，其中 k 与系统信息的类型相关：

- 对于 SIB1 而言，k = (SFN/2) modulo 4，其中 SFN 为系统帧号；

- 对于其它 SI 消息而言，$k = i \bmod 4$，i 的取值范围为 $i = 0,1,\ldots, n_s^w - 1$，其中 i 为 SI 窗口 n_s^w 内的子帧号。

关于广播 HARQ process，可参见 36.321 的 5.3.1 节。

注意：虽然同一 MIB（PBCH 使用卷积码而不是 Turbo 码）在 4 个 TTI 内发送的数据可以进行合并，但其并没有 HARQ 的概念，也就没有 RV 的概念。

【参考资料】

[1] TS 36.321 的 5.3.1 节 DL Assignment reception

第14章 SR

如果 UE 没有上行数据要传输，eNodeB 并不需要为该 UE 分配上行资源，否则会造成资源的浪费。因此， UE 需要先告诉 eNodeB 自己是否有上行数据需要传输， eNodeB 才能决定是否给 UE 分配上行资源。为此 LTE 提供了一个上行调度请求（Scheduling Request，SR）的机制。

UE 通过 SR 告诉 eNodeB 是否需要上行资源以便用于 UL-SCH 传输，但并不会告诉 eNodeB 有多少上行数据需要发送（这是通过 BSR 上报的）。eNodeB 收到 SR 后，给 UE 分配多少上行资源取决于 eNodeB 的实现，通常的做法是至少分配足够 UE 发送 BSR 的资源。

eNodeB 不知道 UE 什么时候需要发送上行数据，即不知道 UE 什么时候会发送 SR。因此，eNodeB 需要在已经分配的 SR 资源上检测是否有 SR 上报。

在载波聚合中，无论配置了多少个上行载波单元（component carrier），都只需要 1 个 SR 就够了，毕竟 SR 的作用只是告诉 eNodeB，本 UE 有上行数据要发送了，你看着给点上行资源吧！由于 PUCCH 只在 PCell 上发送，而 SR 只在 PUCCH 上发送，因此 SR 只在 PCell 上发送。

本章并不介绍 SR 如何编码并在 PUCCH 上传输，这部分内容可参见 8.3 节关于“PUCCH format 1/1a/1b”的介绍。

需要明确的是，只有处于 RRC_CONNECTED 态且保持上行同步的 UE 才会发送 SR；且 SR 只能用于请求**新传**数据（而不是重传数据）的 UL-SCH 资源。

UE 是因为没有上行 PUSCH 资源才发送 SR 的，所以 UE 只能在 PUCCH 上发送 SR。eNodeB 可以为每个 UE 分配一个专用的 SR 资源（PUCCH format 1）用于发送 SR。该 SR 资源是周期性的，每 n 个子帧出现一次。SR 的周期是通过 IE：*SchedulingRequestConfig* 的 *sr-ConfigIndex* 字段配置的。

由于 SR 资源是 UE 专用且由 eNodeB 分配的，因此 SR 资源与 UE 一一对应且 eNodeB 知道具体的对应关系。也就是说，UE 在发送 SR 信息时，并不需要指定自己的 ID（C-RNTI），eNodeB 通过 SR 资源的时频位置，就知道是哪个 UE 请求上行资源。SR 资源是通过 IE：*SchedulingRequestConfig* 的 *sr-PUCCH-ResourceIndex* 字段配置的。

```
SchedulingRequestConfig ::=     CHOICE {
    release                         NULL,
    setup                           SEQUENCE {
        sr-PUCCH-ResourceIndex              INTEGER (0..2047),------SR使用的PUCCH 1资源，对应port 0
        sr-ConfigIndex                  INTEGER (0..157), ),------配置SR的周期以及在周期内的偏移

        dsr-TransMax                    ENUMERATED {
                                            n4, n8, n16, n32, n64, spare3,
spare2, spare1}),------SR的最大传输次数
```

```
    }
}

SchedulingRequestConfig-v1020 ::=   SEQUENCE {
    sr-PUCCH-ResourceIndexP1-r10    INTEGER (0..2047)        OPTIONAL       --
Need OR),------使用2天线端口来发送PUCCH时，在Port 1上使用的PUCCH 1资源，见8.7节关于“SORTD”的介绍
}
```

UE 在某些情况下可能没有 SR 资源。**场景一**：从 36.331 可以看出，*SchedulingRequestConfig* 是一个 UE 级的可选的 IE（optional），默认为 release。如果 eNodeB 不给某 UE 配置 SR（这取决于不同厂商的实现），则该 UE 只能通过随机接入过程来获取 UL grant（在 RAR 中分配）。是否配置 SR 主要影响用户面的延迟，并不影响上行传输的功能。

场景二：当 UE 丢失了上行同步，它也会释放 SR 资源，如果此时有上行数据要发送，也需要触发随机接入过程。（见 36.321 的 5.4.4 节）

从上面的描述可以看出，当 UE 没有被分配 SR 资源时，基于竞争的随机接入过程可以替代 SR 的功能用于申请上行资源。但这只适用于低密集度的上行资源请求的情况。

从 36.213 的 10.1.1 节可以看出，只有 PUCCH format 1/1a/1b 和 PUCCH format 3 可用于发送 SR。也就是说，不能在同一子帧同时发送 SR 和周期性的 CSI（使用 PUCCH format 2/2a/2b），如果发生了这种情况，UE 会丢弃 CSI 而只发送 SR。

sr-PUCCH-ResourceIndex 指定了 UE 在哪个 PUCCH format 1 资源上发送 SR。SR 资源用 $n_{\text{PUCCH, SRI}}^{(1,p)}$ 表示，其值与 PUCCH format 1 的资源索引 $n_{\text{PUCCH}}^{(1,p)}$ 相等。

如果在同一子帧上，需要同时发送 SR 和 PUCCH format 3（HARQ ACK/NACK），则 SR 会复用到 PUCCH format 3 发送中（见 36.212 的 5.2.3.1 节），而不是在 *sr-PUCCH-ResourceIndex* 指定的 PUCCH format 1 资源上发送。（关于 PUCCH 资源，可见第 8 章关于“PUCCH”的介绍）

sr-ConfigIndex 指定了 SR 的传输周期 $SR_{\text{PERIODICITY}}$ 和 SR 在该周期内的子帧偏移 $N_{\text{OFFSET,SR}}$，对应 36.213 的 Table 10.1.5-1。满足如下条件的上行子帧才能够用于发送 SR：

$$\left(10\times n_f + \lfloor n_s/2 \rfloor - N_{\text{OFFSET,SR}}\right) \bmod SR_{\text{PERIODICITY}} = 0$$

其中 n_f 为系统帧号；n_s 为一个系统帧内的 slot 号，取值范围为 0~19；$\lfloor n_s/2 \rfloor$ 的值对应子帧号。

从上面的公式可以看出，$\bmod SR_{PERIODICITY}$ 保证了每个 UE 对应的 SR 资源在每 $SR_{PERIODICITY}$ 个子帧出现一次（但 UE 只在有上行数据要发送却没有上行资源时，才用该资源来发送 SR）。$-N_{OFFSET,SR}$ 指定了每个 UE 对应的 SR 资源在其周期内的第几个子帧发送。

SR 资源配置如图 14-1 所示：

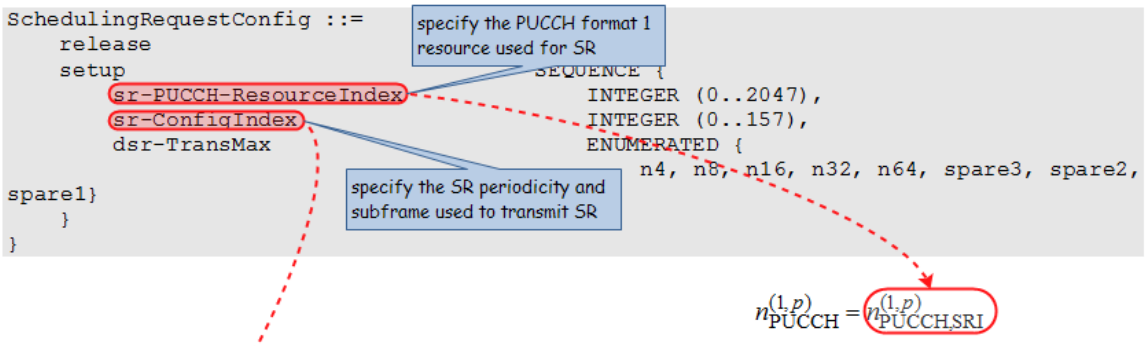


Table 10.1.5-1: UE-specific SR periodicity and subframe offset configuration

SR configuration Index I_{SR}	SR periodicity (ms) $SR_{PERIODICITY}$	SR subframe offset $N_{OFFSET,SR}$
0 – 4	5	I_{SR}
5 – 14	10	$I_{SR}-5$
15 – 34	20	$I_{SR}-15$
35 – 74	40	$I_{SR}-35$
75 – 154	80	$I_{SR}-75$
155 – 156	2	$I_{SR}-155$
157	1	$I_{SR}-157$

图 14-1：SR 资源

可以看出，*sr-ConfigIndex* 和 *sr-PUCCH-ResourceIndex* 共同决定了一个唯一的 SR 资源。该资源只能分配给一个 UE，但只有当 UE 有上行数据需要发送但却没有上行资源时才会被使用。

图 14-2 是 SR 周期配置的一个例子，3 个 UE 的周期都为 10ms，但在周期内的子帧偏移各不相同。

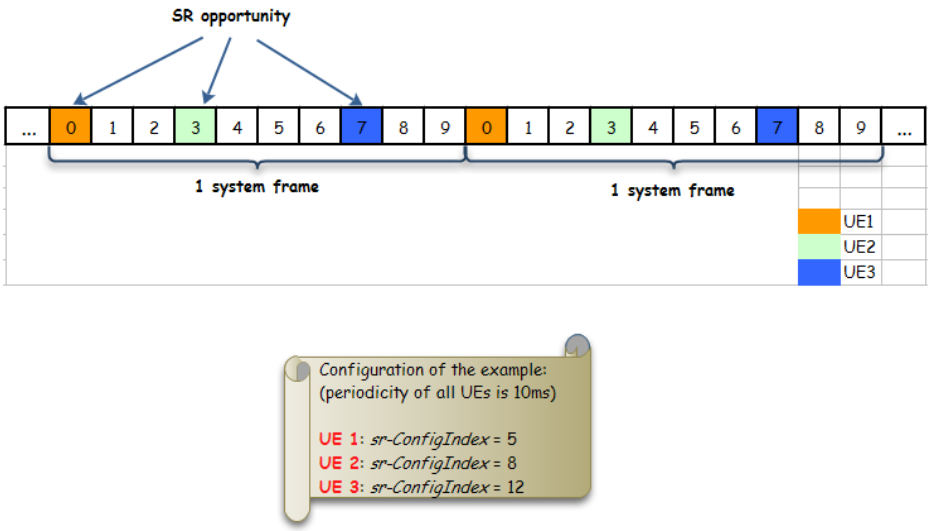


图 14-2：SR 周期配置的一个例子

如图 14-3 所示，当有上行数据到达并触发 SR 时，UE 会选择分配给它的下一个可用的 SR 资源来发送 SR。

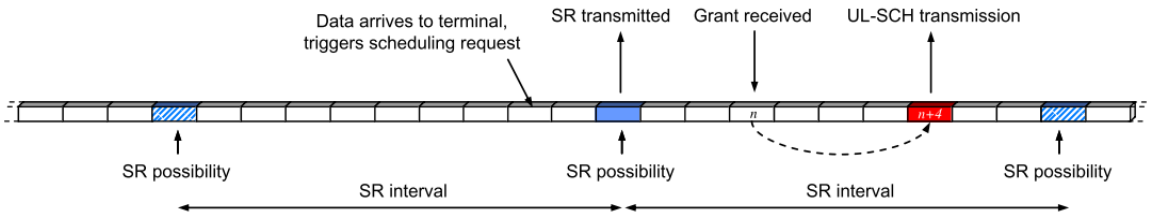


图 14-3：SR 传输

UE 发送 SR 以后，无法确定 eNodeB 什么时候会下发 UL Grant，这取决于上行资源的调度以及优先级等。如果 UE 等待超时（超时时间由 *sr-ProhibitTimer* 决定）就重发 SR，重发次数超过了 SR 的最大重传次数（由 IE：*SchedulingRequestConfig* 的 *dsr-TransMax* 决定）就会触发随机接入。（见 36.321 的 5.4.4 节）

小区内的所有 UE 通常会配置相同的 SR 周期。SR 周期的值越小，SR 上报的延时就越小，但在小区 SR 用户容量不变的情况下，每个子帧上需要预留给 SR 的 PUCCH 1 资源就越多。反之亦然。

周期性 SR 配置所使用的 PUCCH 1 资源 $n_{\mathrm{PUCCH}}^{(1,\tilde{p})}$ 是 UE 特定的，并通过 *sr-PUCCH-ResourceIndex* 配置，该资源每隔 $SR_{\mathrm{PERIODICITY}}$ ms 出现一次的。也就是说，一个 UE 在一个周期内只会在其中的一个上行子帧上占用某一个 PUCCH 1 资源。

接下来，我们将计算小区在一个上行子帧需要预留多少个 PUCCH 1 资源给 SR 使用。假设小区支持的 SR 用户数为 N_{SR}^{user} ，每个系统帧（10 ms）内有 $N_{UL}^{subfraem}$ 个上行子帧可用于发送 PUCCH，SR 的上报周期为 $SR_{\mathrm{PERIODICITY}}$ ，则每个上行子帧需要预留给 SR 的 PUCCH 1 资源数为 $N_{PUCCH}^{(SR)} = \lceil N_{SR}^{user} / (SR_{PERIODICITY} / 10 * N_{UL}^{subframe}) \rceil$。

假设使用 FDD 的某小区，需要支持同时给 80 个 UE 分配 SR 资源。如果 SR 周期为 10 ms（对应 10 个 I_{SR}，一个周期内有 10 个上行子帧可用于发送 PUCCH），则每个上行子帧需要预留 80 / (10 / 10 * 10) = 8 个 PUCCH 1 资源用于发送 SR。如果 SR 周期为 20 ms（对应 20 个 I_{SR}，一个周期内有 10 个上行子帧可用于发送 PUCCH），则每个上行子帧需要预留 80 / (20 / 10 * 10) = 4 个 PUCCH 1 资源用于发送 SR。

TDD 下，一个周期内可用的上行子帧数变少，因此在相同的配置下，每个上行子帧需要预留更多的 PUCCH 1 资源用于发送 SR。

需要注意的是，前面介绍的计算公式的前提是 UE 不支持 SORTD。如果 UE 支持 SORTD，分配给一个 UE 的 SR 资源数需要翻倍（由 1 变成 2）。

【参考资料】

[1] TS 36.213 的 10.1.5 节 Scheduling Request (SR) procedure

[2] TS 36.321 的 5.4.4 节 Scheduling Request

[3] TS 36.331 的 6.3.2 节 IE: *SchedulingRequestConfig*

[4] 《4G LTE/LTE-Advanced for Mobile Broadband》的 13.2.2.2 节

[5] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 16.3.7 节

第15章 BSR

在第 14 章中我们已经介绍到，UE 通过 SR 向 eNodeB 请求上行资源时，只指明了其是否有上行数据需要发送，而没有指明自己需要发送多少上行数据。UE 需要通过 BSR（Buffer Status Report）告诉 eNodeB，其上行 buffer 里有多少数据需要发送，以便 eNodeB 决定给该 UE 分配多少上行资源。

根据业务的不同，UE 可能建立大量的无线承载（radio bearer，每个 bearer 对应一个逻辑信道），如果为每一个逻辑信道上报一个 BSR，会带来大量的信令开销。为了避免这种情况，LTE 引入了 LCG（Logical Channel Group）的概念，并将每个逻辑信道放入一个 LCG（共 4 个）中。UE 基于 LCG 来上报 BSR，而不是为每个逻辑信道上报一个 BSR。

某个逻辑信道所属的 LCG 是在逻辑信道建立时通过 IE: *LogicalChannelConfig* 的 *logicalChannelGroup* 字段来设置的 。

```
LogicalChannelConfig ::=            SEQUENCE {
    ul-SpecificParameters               SEQUENCE {
        priority                            INTEGER (1..16),
        prioritisedBitRate                  ENUMERATED {
                                                kBps0, kBps8, kBps16, kBps32, kBps64,
kBps128,
                                                kBps256, infinity, kBps512-v1020,
kBps1024-v1020,
                                                kBps2048-v1020, spare5, spare4, spare3,
spare2,
                                                spare1},
        bucketSizeDuration                  ENUMERATED {
                                                ms50, ms100, ms150, ms300, ms500, ms1000,
spare2,
                                                spare1},
        logicalChannelGroup                 INTEGER (0..3)          OPTIONAL            -
- Need OR------逻辑信道所属的LCG
    }       OPTIONAL,                                                                   -
- Cond UL
    ...,
    [[  logicalChannelSR-Mask-r9            ENUMERATED {setup}      OPTIONAL        --
Cond SRmask
    ]]
}
```

将逻辑信道分组是为了提供更好的 BSR 上报机制。将那些有相似调度需求的逻辑信道放入同一 LCG 中，并通过 short BSR 上报其 buffer 状态。

如何分组取决于 eNodeB 的算法实现（例如：将相同 QCI/priority 的逻辑信道放入同一 LCG 中）。即上行的 QoS 管理是由 eNodeB 负责的。

由于 UE 的 LCG 和逻辑信道的配置是由 eNodeB 控制的，所以 eNodeB 知道每个 LCG 包含哪些逻辑信道以及这些逻辑信道的优先级。虽然 eNodeB 无法知道一个单独的逻辑信道的 buffer 状态，但由于同一 LCG 中的逻辑信道有着类似的 QoS/priority 需求，所以基于 LCG 来上报 buffer 状态也可以使得上行调度提供合适的调度结果。

15.1 BSR MAC Control Element

BSR 通过 MAC 层的 BSR MAC Control Element 上报，并包含 2 种格式：

- Short BSR 或 Truncated BSR 格式：只上报一个 LCG 的 BSR。其格式由一个 LCG ID 域和一个对应的 Buffer Size 域组成。如图 15-1 所示。

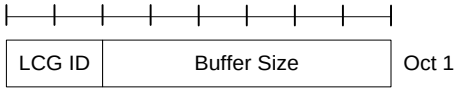


图 15-1：Short BSR 和 Truncated BSR MAC control element

- Long BSR 格式：包含了 4 个 Buffer Size 域，对应 LCG ID 0~3。即该格式会将所有 LCG 的 Buffer Size 一起上报给 eNodeB。如图 15-2 所示

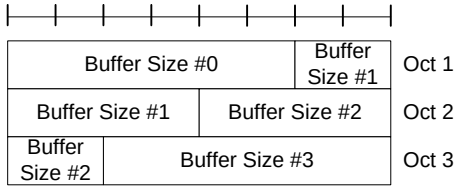


图 15-2：Long BSR MAC control element

LCG ID 域长为 2 比特，指定了上报的 buffer 状态对应的 LCG，其值与 IE: *LogicalChannelConfig* 的 *logicalChannelGroup* 字段对应。

Buffer Size 域长为 6 比特，指定了 UE 在发送这个 BSR 的 TTI 内的所有 MAC PDU 都生成以后，对应 LCG 的所有逻辑信道的 RLC 层和 PDCP 层中剩余的可用于传输的有效数据的总和（见 36.322 的 4.5 节和 36.323 的 4.5 节）。该数据量以字节（byte）为单位，但不将 RLC 头部和 MAC 头部信息计算在内。

从 36.321 的 6.1.3.1 节可以看出，eNodeB 收到 BSR 后，通过 Buffer Size 域得到一个 index，再用这个 index 查 Table 6.1.3.1-1 或 Table 6.1.3.1-2，可以得到 UE 真正需要发送的“近似”上行数据量。因为 UE 在发送 BSR 时，无法确定后续发送的上行数据中会有哪些 RLC 头部和 MAC 头部，所以计算时不将 RLC 头部和 MAC 头部信息计算在内。而从 Table 6.1.3.1-1 和 Table 6.1.3.1-2 可以看

出，通过 Buffer Size 域得到的 index 对应的是一个 Buffer Size 的范围，而不是一个精确的 Buffer Size，因此是一个“近似”上行数据量。

在 Rel-10 中，配置了载波聚合的 UE 可能存在多个上行载波单元同时发送数据，以及引入了上行 MIMO，BSR 指示的数据量也可能远大于 Rel-8 中指示的数据量，因此在 Rel-10 中，LTE 额外提供了一个 BSR 值的表（见 36.321 的 Table 6.1.3.1-2）。具体使用 Table 6.1.3.1-1 还是 Table 6.1.3.1-2 是通过 IE：*MAC-MainConfig* 的 *extendedBSR-Sizes-r10* 字段来配置的。

一个 BSR MAC control element 与一个 MAC subheader 对应。BSR 对应的 MAC subheader 中的 LCID 域如图 15-3 所示：（见 36.321 的 Table 6.2.1-2）

Table 6.2.1-2 Values of LCID for UL-SCH

Index	LCID values
00000	CCCH
00001-01010	Identity of the logical channel
01011-11000	Reserved
11001	Extended Power Headroom Report
11010	Power Headroom Report
11011	C-RNTI
11100	Truncated BSR
11101	Short BSR
11110	Long BSR
11111	Padding

图 15-3：UL-SCH 的所有 LCID 值

注意：LCID 与 LCG ID 是不同的。LCID 是用来唯一地指定 MAC PDU 中的一个 MAC SDU 或 MAC control element 或 padding 的。而 LCG ID 是用来指定逻辑信道所属的逻辑信道组 ID 的，只用于 BSR 上报。

15.2 BSR 的触发方式

当如下事件发生时，将会触发 BSR 上报：

1、UE 的上行数据 buffer 为空且有新数据到达：当所有 LCG 的所有逻辑信道都没有可发送的上行数据时，如果此时属于任意一个 LCG 的任意一个逻辑信道有数据变得可以发送，则 UE 会触发 BSR 上报。例如：UE 第一次发送上行数据。该 BSR 被称为“Regular BSR”;

2、高优先级的数据到达：如果 UE 已经发送了一个 BSR，并且正在等待 UL grant，此时有更高优先级的数据（即该数据所属的逻辑信道【而不是 LCG】比任意一个 LCG 的逻辑信道的优先级都要高）需要传输，则 UE 会触发 BSR 上报。该 BSR 被称为“Regular BSR”；

3、UE 周期性地向 eNodeB 更新自己的 buffer 状态：eNodeB 通过 IE：*MAC-MainConfig* 的 *periodicBSR-Timer* 字段为 UE 配置了一个 timer（配置成“infinity”则去使能该 timer），如果该 timer 超时，UE 会触发 BSR 上报。例如：当 UE 需要上传一个大文件时，数据到达 UE 传输 buffer 的时间与 UE 收到 UL grant 的时间是不同步的，也就是说 UE 在发送 BSR 和接收 UL grant 的同时，还在不停地往上行传输 buffer 里填数据，因此 UE 需要不停地更新需要传输的上行数据量。该 BSR 被称为“Periodic BSR”；

4、为提高 BSR 的健壮性，LTE 提供了一个重传 BSR 的机制：这是为了避免 UE 发送了 BSR 却一直没有收到 UL grant 的情况。eNodeB 通过 IE：*MAC-MainConfig* 的 *retxBSR-Timer* 字段为 UE 配置了一个 timer，当该 timer 超时且 UE 的任意一个 LCG 的任意一个逻辑信道里有数据可以发送时，将会触发 BSR。该 BSR 被称为“Regular BSR”。

5、废物再利用：当 UE 有上行资源且发现需要发送的数据不足以填满该资源时，多余出来的比特会作为 padding bit 而被填充一些无关紧要的值。与其用作 padding bit，还不如用来传 BSR 这些有用的数据。所以当 padding bit 的数量等于或大于“BSR MAC control element + 对应的 subheader”的大小时，UE 会使用这些比特来发送 BSR。该 BSR 被称为“Padding BSR”。

从上面的介绍可以看出，只有当某个逻辑信道属于某个 LCG 时，它的数据才会被统计到对应的 BSR 中并上报给 eNodeB；对于不属于任一 LCG 的逻辑信道（*logicalChannelGroup* 字段是 OPTIONAL 的），其数据不会被统计，不会影响任何 BSR 行为。

如果至少触发了一个 BSR 且该 BSR 没有被取消且 UE 已经在该 TTI 内收到了用于新传数据的 UL grant，则 UE 会

- 生成 BSR MAC control element；
- 除非所有生成的 BSR 均为 Truncated BSR，否则 UE 会启动或重启 periodicBSR-Timer；
- 启动或重启 retxBSR-Timer；

当触发了 Regular BSR，但 UE 没有足够的 UL-SCH 资源用于发送 BSR 时，UE 会发送 SR 请求；而当触发了 Periodic 或 Padding BSR，但 UE 没有足够的 UL-SCH 资源用于发送 BSR 时，UE 不会发送 SR 请求。

通常，SR 机制是针对整个 UE 的所有逻辑信道的，但在 Rel-9 中，LTE 还提供了一种基于逻辑信道进行 SR 请求的机制。对于 eNodeB 创建的每一个逻辑信道，都有一个 *logicalChannelSR-Mask-r9* 字段，用于指定当该逻辑信道有新数据到达时，是否触发 SR。

对于 Regular 和 Periodic BSR 而言，如果在该 TTI 内有多于 1 个 LCG 中有数据需要发送，则上报 Long BSR；否则上报 Short BSR。

对于 Padding BSR 而言，当 padding bit 的数量等于或大于“Short BSR +对应的 subheader”的大小但小于“Long BSR + 对应的 subheader”的大小时，如果在该 TTI 内有多于 1 个 LCG 中有数据需要发送，则将有数据要发送且优先级最高的逻辑信道所在的 LCG 的 BSR 上报给 eNodeB，该 BSR 格式为 Truncated BSR；如果该 TTI 内只有 1 个 LCG 有数据需要发送，则发送 Short BSR。

对于 Padding BSR 而言，当 padding bit 的数量等于或大于“Long BSR + 对应的 subheader”的大小时，发送 Long BSR。

即使有多个事件触发了 BSR，一个 MAC PDU 至多也只能包含一个 MAC BSR control element（如果一个 TTI 内有多个 MAC PDU，如在空分复用或载波聚合的情况下，则每个 MAC PDU 都能携带一个 MAC control element）。其中 Regular BSR 和 Periodic BRS 的优先级要高于 Padding BSR，即优先传输 Regular/Periodic BSR。

当 UE 收到一个新传数据的 UL grant 时，都会重启 *retxBSR-Timer*。

如果在某个子帧内收到的 UL grant 能够容纳 UL buffer 里的所有 pending data，却不足以容纳额外的 BSR MAC control element 和其对应的 subheader 时，所有已经触发的 BSR 都会被取消。

当一个 BSR 包含在一个待传输的 MAC PDU 里时，所有已经触发的 BSR 都会被取消。

UE 在每个 TTI 至多只能发送一个 Regular/Periodic BSR。如果 UE 在一个 TTI 内需要发送多个 MAC PDU，则它可以在任意一个不包含 Regular/Periodic BSR 的 MAC PDU 里携带一个 padding BSR。

UE 在一个 TTI 里传输的所有 BSR 反映的是这个 TTI 内的所有 MAC PDU 都生成以后的 buffer 状态。每个 LCG 在每个 TTI 内至多只能上报一个 buffer 状态值（注意：不是 BSR）。假如 UE 在一个 TTI 内上报了多个 BSR，且其中几个 BSR 都上报了同一个 LCG 的 buffer 状态，那么对应同一 LCG 的这些 buffer 状态值必须是相同的。

需要注意的是，一个 Padding BSR 是不允许取消那些已经触发的 Regular/Periodic BSR 的。Padding BSR 只能为某个特定的 MAC PDU 而触发，且当这个 MAC PDU 生成后，该 trigger 就被取消了。

有意思的是，UE 上报的 BSR 与 UE 如何处理 UL grant 没有直接的关系。UE 是基于逻辑信道优先级给无线承载分配资源的，与逻辑信道属于哪个 LCG 没有任何关系。例如：某个 UE 为了发送 HTTP 请求（假定该业务对应的逻辑信道属于 LCG 2），已经发送了 BSR。在 UE 收到对应 UL grant 之前，新出现了一个 RRC 消息。由于 RRC 消息对应的逻辑信道的优先级比 HTTP 请求对应的逻辑信道的优先级要高，所以当 UE 收到 UL grant（该 UL grant 是因为 HTTP 请求才予以分配的）

后，UE 会将上行资源优先分配给 RRC 消息，而不是分配给 HTTP 请求。（具体可参见第 17 章关于“MAC 复用和逻辑信道优先级”的介绍）

注：（1）CCCH、SRB1、SRB2 默认属于 LCG 0（在 36.331 中搜索 *logicalChannelGroup*，能看出协议中是有明确说明的）；（2）RRC 消息在 SRB 上传输且 SRB 默认属于 LCG 0，比 LCG 2 的优先级要高。

【参考资料】

[1] TS 36.300 的 11.3 节 Measurements to Support Scheduler Operation

[2] TS 36.321 的 5.4.5 节 Buffer Status Reporting

[3] TS 36.321 的 6.1.3.1 节 Buffer Status Report MAC Control Elements

[4] TS 36.331 的 *periodicBSR-Timer*、*retxBSR-Timer*、*logicalChannelGroup*

[5] 《4G LTE/LTE-Advanced for Mobile Broadband》的 13.2.2.2 节

[6] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 4.4.2.2 节

[7] 《Buffer Status Reports and Uplink Scheduling》 by John

第16章 DRX

本章主要介绍处于 RRC_CONNECTED 态下的 UE 的 DRX 处理流程。结合 3GPP 协议，介绍了几个 timer 的作用，同时还简单介绍了载波聚合对 DRX 的影响。

16.1 DRX 介绍

基于包的数据流通常是突发性的，在一段时间内有数据传输，但在接下来的一段较长时间内没有数据传输。在没有数据传输的时候，可以通过停止接收 PDCCH（此时会停止 PDCCH 盲检）来降低功耗，从而提升电池使用时间。这就是 DRX（Discontinuous Reception，非连续接收）的由来。

DRX 的基本机制是为处于 RRC_CONNECTED 态的 UE 配置一个 DRX cycle。DRX cycle 由“*On Duration*”和“Opportunity for DRX”组成：在“On Duration”时间内，UE 监听并接收 PDCCH（激活期）；在“Opportunity for DRX”时间内，UE 不接收 PDCCH 以减少功耗（休眠期）。

从图 16-1 可以看出，在时域上，时间被划分成一个个连续的 DRX Cycle。

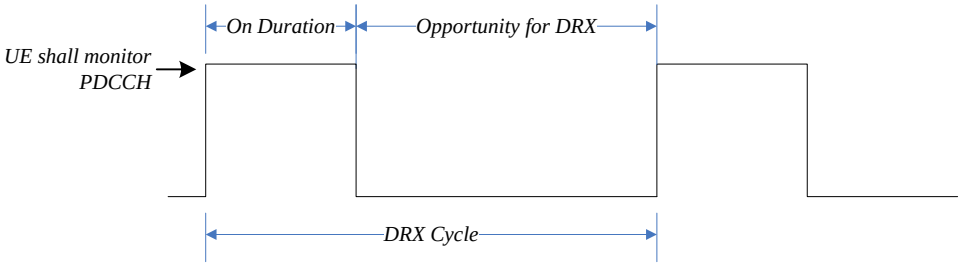


图 16-1：DRX cycle

注意：处于休眠期的 UE，只是不接收 PDCCH，但是可以接收来自其它物理信道的数据，如 PDSCH、ACK/NACK 等。例如：在 SPS 调度中，处于休眠期的 UE 可以接收周期性配置的下行子帧上发送的 PDSCH 数据。

eNodeB 通过 *DRX-Config* 来配置某个 UE 的 DRX 相关参数。

```
DRX-Config ::=                          CHOICE {
    release                                 NULL,
    setup                                   SEQUENCE {
        onDurationTimer                         ENUMERATED {
                                                    psf1, psf2, psf3, psf4, psf5, psf6,
                                                    psf8, psf10, psf20, psf30, psf40,
                                                    psf50, psf60, psf80, psf100,
```

```
                                                psf200},-------从一个DRX Cycle的起始处算起，连续
监听的PDCCH子帧数。
        drx-InactivityTimer                     ENUMERATED {
                                                    psf1, psf2, psf3, psf4, psf5, psf6,
                                                    psf8, psf10, psf20, psf30, psf40,
                                                    psf50, psf60, psf80, psf100,
                                                    psf200, psf300, psf500, psf750,
                                                    psf1280, psf1920, psf2560, psf0-v1020,
                                                    spare9, spare8, spare7, spare6,
                                                    spare5, spare4, spare3, spare2,
                                                    spare1},-------当UE成功解码一个指示初传的UL或DL
用户数据的PDCCH后，持续处于激活态的连续PDCCH子帧数。
        drx-RetransmissionTimer                 ENUMERATED {
                                                    psf1, psf2, psf4, psf6, psf8, psf16,
                                                    psf24, psf33}, -------从UE期待收到DL重传的子帧
（HARQ RTT之后）开始，连续监听的PDCCH子帧数。
        longDRX-CycleStartOffset            CHOICE {
            sf10                                INTEGER(0..9),
            sf20                                INTEGER(0..19),
            sf32                                INTEGER(0..31),
            sf40                                INTEGER(0..39),
            sf64                                INTEGER(0..63),
            sf80                                INTEGER(0..79),
            sf128                               INTEGER(0..127),
            sf160                               INTEGER(0..159),
            sf256                               INTEGER(0..255),
            sf320                               INTEGER(0..319),
            sf512                               INTEGER(0..511),
            sf640                               INTEGER(0..639),
            sf1024                              INTEGER(0..1023),
            sf1280                              INTEGER(0..1279),
            sf2048                              INTEGER(0..2047),
            sf2560                              INTEGER(0..2559)
        },-------指定了longDRX-Cycle和drxStartOffset。
        shortDRX                                SEQUENCE {
            shortDRX-Cycle                          ENUMERATED  {
                                                        sf2, sf5, sf8, sf10, sf16, sf20,
                                                        sf32, sf40, sf64, sf80, sf128, sf160,
                                                        sf256, sf320, sf512, sf640},-------指定了
short DRX Cycle持续的子帧数，即short DRX Cycle的大小。
            drxShortCycleTimer                      INTEGER (1..16)-------指定了UE在多长的时间内，
使用的是short DRX Cycle。该值为shortDRX-Cycle的倍数。
        }       OPTIONAL                                                    -- Need OR
    }
}
```

DRX cycle 的选择需要考虑电池节约与延迟之间的平衡。从一个方面讲，长 DRX 周期有益于延长 UE 的电池使用时间；例如网页浏览过程中，当用户正在阅读已经下载好的网页时，UE 持续接收下行数据是对资源的浪费。从另一个方面讲，当有新的数据传输时，一个更短的 DRX 周期有益于更快的响应，例如用户请求另一个网页或者进行 VoIP 通话时。为了满足上述需求，每个 UE 可以配置两个 DRX cycle：*shortDRX-Cycle* 和 *longDRX-Cycle*。如果 UE 配置了 *shortDRX-Cycle*，则 *longDRX-Cycle* 应该配置为 *shortDRX-Cycle* 的倍数。但在任一时刻，UE 只能使用其中一种配置。

drxStartOffset 指定 DRX cycle 的起始子帧，***longDRX-Cycle*** 指定了一个 long DRX cycle 占多少个子帧（即连续的“子帧数”），这两个参数都是由 ***longDRX-CycleStartOffset*** 字段确定的。

onDurationTimer 指定了从 DRX cycle 的起始子帧算起，需要监听 PDCCH 的连续“PDCCH 子帧数”。

对于 DRX，需要注意“连续的子帧数”与“连续的 PDCCH 子帧数”的区别。FDD 中，PDCCH 子帧可以是任意子帧；但在 TDD 中，PDCCH 子帧只包含下行子帧和包含 DwPTS 的子帧，这是因为只有下行子帧才有可能传输 PDCCH。

DRX 中定义了多个定时器（timer），有些指定的是“连续的子帧数”，而另一些指定的是“连续的 PDCCH 子帧数”。在 TDD 中，如果某个定时器指定的是“连续的 PDCCH 子帧数”，则上行子帧是不统计在该定时器的持续时间中的，此时该定时器实际持续的“子帧数”可能大于其指定的“PDCCH 子帧数”。（见表 16-1）

在大多数情况下，当一个 UE 在某个子帧被调度并接收或发送数据后，很可能在接下来的几个子帧内继续被调度，如果要等到下一个 DRX cycle 再来接收或发送这些数据将会带来额外的延迟。为了降低这类延迟，UE 在被调度后，会持续处于激活期，即会在配置的激活期内持续监听 PDCCH。其实现机制是：每当 UE 被调度以初传数据时，就会启动（或重启）一个定时器 *drx-InactivityTimer*，UE 将一直处于激活态直到该定时器超时。***drx-InactivityTimer*** 指定了当 UE 成功解码一个指示初传的 UL 或 DL 用户数据的 PDCCH 后，持续处于激活态的连续 PDCCH 子帧数。即当 UE 有初传数据被调度时，该定时器就启动或重启一次。**注意：**（1）这里是初传而不是重传，即指示重传的 PDCCH 并不会重启该定时器；（2）周期性的 SPS 子帧上发送的 PDSCH 虽然是初传，但并没有伴随着传输 PDCCH，因此该 PDSCH 并不会重启该定时器；（3）*drx-InactivityTimer* 指定的是连续的“PDCCH 子帧数（下行子帧）”，而不是连续的“子帧数”。

HARQ 重传并不关心 DRX cycle，配置了 DRX 的 UE 与没有配置 DRX 时使用相同的方式来接收/发送 HARQ 反馈和重传。上行使用同步方式，前一次传输与重传之间有固定的 timing 关系。下行使用异步方式，前一次传输与重传之间没有固定的 timing 关系，因此 LTE 定义了一个时间窗（HARQ RTT Timer），允许 UE 从前一次下行传输算起，并持续该时间窗之后，才开始监听下行的重传。

为了允许 UE 在 HARQ RTT 期间内休眠，每个 DL HARQ process 定义了一个 “HARQ RTT(Round Trip Time) timer”。当某个下行 HARQ process 的 TB 解码失败时，UE 可以假定至少在“HARQ RTT”子帧后才会有重传，因此当 HARQ RTT timer 正在运行时，UE 没必要监听 PDCCH。当 HARQ RTT timer 超时，且对应 HARQ process 接收到的数据没有被成功解码时，UE 会为该 HARQ process 启动一个 *drx-RetransmissionTimer*。当该 timer 运行时，UE 会监听用于 HARQ 重传的 PDCCH。*drx-RetransmissionTimer* 的长度与 eNodeB 调度器的灵活度要求相关。如果是要达到最优的电池消耗，就要求 eNodeB 在 HARQ RTT timer 超时之后，立即调度 HARQ 重传，这就也要求 eNodeB 为此预留无线资源，此时 *drx-RetransmissionTimer* 也就可以配得短些。***drx-RetransmissionTimer*** 指定了从 UE 期待收到 DL 重传的子帧（HARQ RTT 之后）开始，连续监听的“PDCCH 子帧数”。注意：这里针对的是“下行重传”，而不是“上行重传”。

对 FDD 而言，HARQ RTT Timer 的大小固定为 8 个子帧。对 TDD 而言，HARQ RTT Timer 的大小为 k + 4 个子帧，其中 k 值为下行传输与对应 HARQ 反馈之间的时间间隔（k 值见 36.213 的 Table 10.1.3.1-1）。

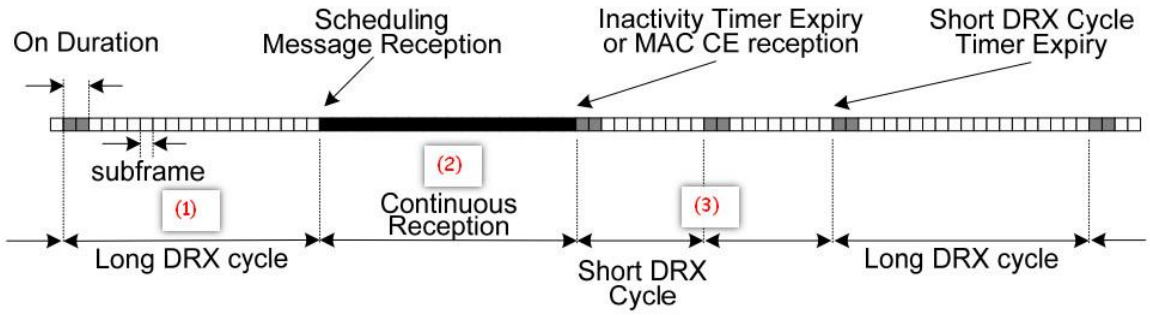


图 16-2：DRX 流程

当 UE 在“On Duration”期间收到一个调度消息（指示初传的 PDCCH）时，UE 会启动一个“*drx-InactivityTimer*”并在该 timer 运行期间的每一个下行子帧监听 PDCCH。当“*drx-InactivityTimer*”运行期间收到一个调度信息（指示初传的 PDCCH）时，UE 会重启该 timer。（对应图 16-2 中标红为(2)的部分）

当“*drx-InactivityTimer*”超时或收到 DRX Command MAC control element 时：（1）如果 UE 没有配置 short DRX cycle，则直接使用 long DRX cycle；（2）如果 UE 配置了 short DRX cycle，UE 会使用 short DRX cycle 并启动（或重启）“***drxShortCycleTimer***”，当“*drxShortCycleTimer*”超时，UE 使用 long DRX cycle。（对应图 16-2 中标红为(3)的部分）

如果 UE 当前使用 short DRX cycle，且[(SFN * 10) + subframe number] modulo (shortDRX - Cycle) = (drxStartOffset) modulo (shortDRX-Cycle)；或者 UE 当前使用 long DRX cycle，且[(SFN * 10) + subframe number] modulo (longDRX-Cycle) = drxStartOffset，则启动“*onDurationTimer*”。（对应图 16-2 中标红为(1)的部分）

注：*drxShortCycleTimer* 启动后，只说明当前使用 short DRX cycle，但此时未必启动了 DRX short cycle。DRX short cycle 是与 *onDurationTimer* 同时启动的。类似的，long DRX cycle 也是与 *onDurationTimer* 同时启动的。

当 UE 配置了 DRX cycle 时，UE 处于激活期的时间包括：

- *onDurationTimer*、或 *InactivityTimer*、或 *drx-RetransmissionTimer*、或 *mac-ContentionResolutionTimer* 正在运行时；

- UE 已经在 PUCCH 上发送了 SR，且该 SR 当前处于 pending 态；
- UE 的 HARQ buffer 存在数据，并等待 HARQ 重传的 UL grant 时；
- UE 成功接收了用于响应非 UE 选择的 preamble 的 RAR，却没有收到指示初传（使用 C-RNTI）的 PDCCH 时。

DRX 是 UE 级别的特性，而不是基于每个无线承载来配置的。

当 UE 配置了 DRX 时， UE 只能在“激活期”的时间内发送周期性 CQI。eNodeB 在使用 RRC 来配置周期性 CQI 上报时，可以进一步地限制 UE 只能在“on-duration”的时间内发送 CQI。

表 16-1 结合 36.213 的 5.7 节总结了关于各种 DRX 相关的 timer 启动和停止的触发条件。

表 16-1：与 DRX 相关 timer 的启动和停止

Timer	Start（Restart）	Stop	“子帧数”或“ PDCCH 子帧数”
onDurationTimer	UE 当前使用 short DRX cycle，且[(SFN * 10) + subframe number] modulo (shortDRX-Cycle) = (drxStartOffset) modulo (shortDRX-Cycle)；或者 UE 当前使用 long DRX cycle，且[(SFN * 10) + subframe number] modulo (longDRX-Cycle) = drxStartOffset	（1）收到 DRX Command MAC control element；（2）timer 超时	PDCCH 子帧数
drx-InactivityTimer	收到用于调度 new transmission 的 PDCCH（DL 和 UL 的均可）	（1）收到 DRX Command MAC control element；（2）timer 超时	PDCCH 子帧数
drx-RetransmissionTimer	HARQ RTT Timer 超时且对应 HARQ process 的 buffer 中的数据没有成功解码	（1）收到指示下行传输的 PDCCH；（2）当前子帧是周期性配置的下行 SPS 子帧（此时没有伴随着传输 PDCCH）；（3）timer 超时	PDCCH 子帧数
drxShortCycleTimer	当配置了 Short DRX cycle 时，如果 *drx-InactivityTimer* 超时，或收到 DRX Command MAC control element，则启动或重启 *drxShortCycleTimer*，并开始使用 Short DRX cycle	Timer 超时，此时开始使用 Long DRX cycle	子帧数
HARQ RTT timer	（1）UE 收到一个指示下行传输的 PDCCH；（2）当前子帧是周期性配置的下行 SPS 子帧（此时没有伴随着传输 PDCCH）	timer 超时	子帧数

注：*longDRX-Cycle* 和 *shortDRX-Cycle* 指定的是连续的“子帧数”。

除了 HARQ RTT timer 和 *drx-RetransmissionTimer* 是每个 DL HARQ process 都有一个外，其它的 timer 是每个 UE 只有一个。

从表 16-1 可以看出，当任一 timer 启动时，不会影响其它 timer 的运行。也即，UE 处于激活态的最短时间为 *onDurationTimer* 指定的时间，而最长时间是不定的。

需要说明的是，对于 eNodeB 的调度器而言，需要知道 UE 何时处于激活期，何时处于休眠期，以便只在激活期调度该 UE。

16.2 载波聚合（CA）对 DRX 的影响

Rel-8 中定义的 DRX 过程同样适用于 Rel-10。

如果配置了一个或多个 SCell，则所有的 serving cells 使用相同的 DRX 操作：

- 对于所有的 DL 载波单元（component carrier）而言，PDCCH 监测的激活时间是相同的；
- 当 UE 处于休眠期时，所有的载波单元都不接收数据；
- 当 UE 被激活时，所有 activated 的载波单元都将被激活以接收数据。

虽然 DRX 降低了 UE 的功耗，但 CA 可能进一步提高功耗，因此，LTE 提供了载波单元的 activation/deactivation 机制。（详见 23.3 节关于“SCell 激活/去激活”的介绍）

【参考资料】

[1] TS 36.321 的 5.7 节 Discontinuous Reception (DRX)

[2] TS 36.321 的 6.1.3.3 节 DRX Command MAC Control Element

[3] TS 36.300 的 12 章 DRX in RRC_CONNECTED

[4] 36.331 的 *DRX-Config*

[5] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 4.4.2.5 节

[6] 《4G LTE/LTE-Advanced for Mobile Broadband》的 13.2.6 节

第17章 MAC 复用与逻辑信道优先级

MAC 层负责将多个逻辑信道（logic channel）复用到同一个传输信道（transport channel）上。UE 在每个 TTI 只能发送一个 MAC PDU（不考虑空分复用和载波集合的情况），但可能来自多个逻辑信道的 RLC SDU 需要放到这同一个 MAC PDU 上，这就是 MAC 复用的由来。如图 17-1、图 17-2 所示，多个逻辑信道可能复用到同一个传输信道上。

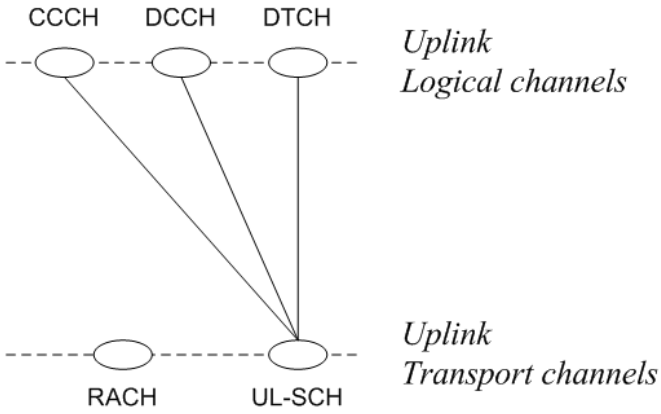


图 17-1：上行信道复用

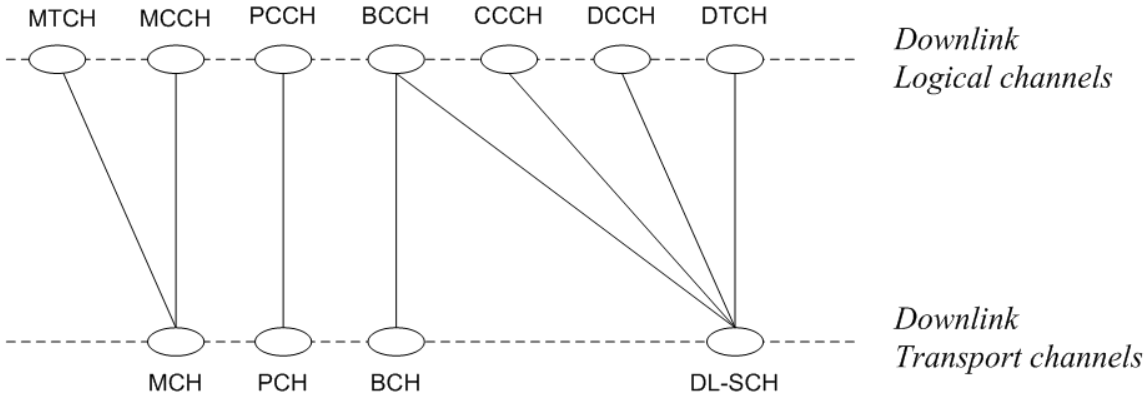


图 17-2：下行信道复用

对于下行而言，MAC 复用（multiplexing）和逻辑信道优先级（logical channel prioritization）是留给 eNodeB 来实现的。不同的厂商可能有不同的实现。

对于上行而言，UE 利用分配到的无线资源创建 MAC PDU 并传输的过程是完全规范化的，这就保证接入同一小区的所有 UE 遵循相同的规范，以一致的方式满足每个配置的无线承载的 QoS。

本章主要介绍 UE 侧的 MAC 层复用实现，同时会在本章的最后对各种 MAC PDU 进行介绍。

17.1 逻辑信道优先级与 MAC 复用

eNodeB 是基于 per-UE 而不是 per-bearer 分配上行无线资源的，哪些无线承载的数据能够放入分配的无线资源中传输是由 UE 决定的。

基于 UL grant 分配的上行无线资源，UE 需要决定包含在新的 MAC PDU 中的每个逻辑信道的数据总量，如果必要的话，UE 还要为 MAC control element 分配资源。换句话说，eNodeB 通过 UL grant 分配给某 UE 的上行资源是确定的。该 UE 基于 RRC 信令 *LogicalChannelConfig* 给定的配置，以及协议规定的规则，来决定放置哪些逻辑信道的数据以及每个逻辑信道放置多少数据。

MAC PDU 只有一个，但要复用的逻辑信道却有多个，这就要求为每个逻辑信道分配一个优先级。最高优先级的逻辑信道的数据优先包含在 MAC PDU 中，接着是次高优先级的逻辑信道的数据，以此类推，直到分配的 MAC PDU 已满或没有更多的数据要发送。每个逻辑信道的优先级是由 *LogicalChannelConfig* 的 *priority* 字段决定的，值越小，优先级越高。

但这种分配方式可能使得高优先级的逻辑信道始终占据着 eNodeB 分配给该 UE 的无线资源，从而导致低优先级的逻辑信道被“饿死”。为了避免这种情况，LTE 引入了优先比特率（PBR：Prioritised Bit Rate）的概念，即在给逻辑信道分配资源之前，配置好各个逻辑信道的数据数率，从而为每个逻辑信道提供了最小数据速率保证，避免了低优先级的逻辑信道被“饿死”。PBR 是通过 *LogicalChannelConfig* 的 *prioritisedBitRate* 字段决定的。

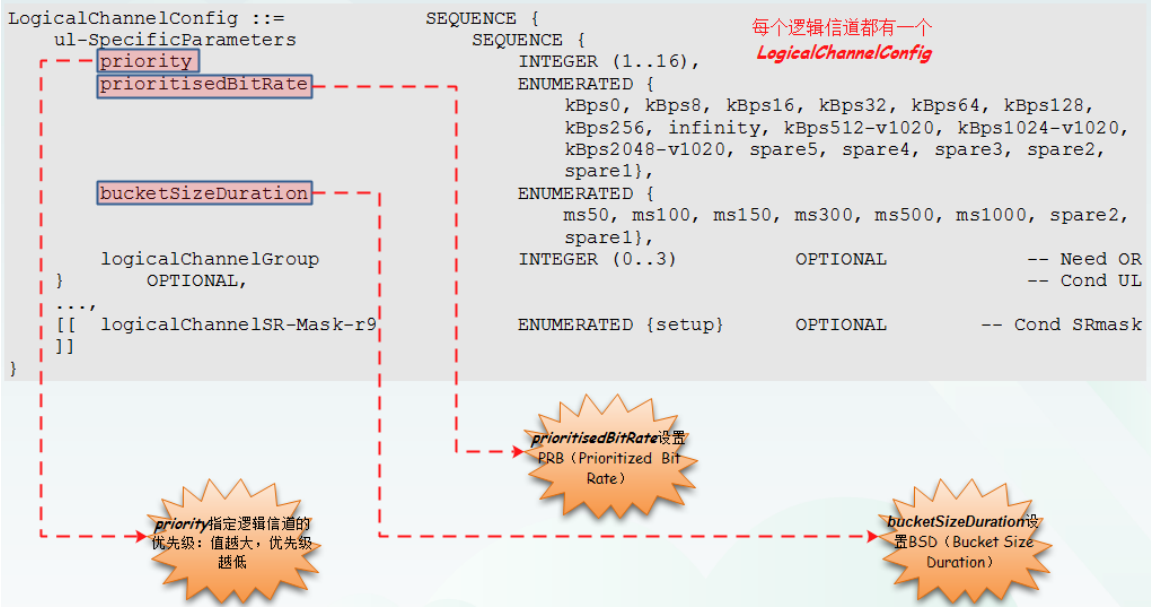


图 17-3：逻辑信道配置

MAC 层使用类似于 token bucket（令牌桶）的算法实现 MAC 复用。该算法的基本思想是基于令牌桶内是否有令牌以及令牌的多少来确定是否发送某逻辑信道的数据，并控制组装在 MAC PDU 中的该逻辑信道的数据量。

BSD（Bucket Size Duration）决定了令牌桶的“深度”。它与 PBR 共同决定了令牌桶的最大容量 PBR × BSD。令牌桶的最大容量限制了每个逻辑信道可以挂起（pending，即缓存在 buffer 中）的数据总量。

UE 为每个逻辑信道 j 维护一个变量 Bj，该变量指示了令牌桶里当前可用的 token 数，且每个 token 对应 1 Byte 的数据。Bj 在逻辑信道建立时初始化为 0，且每个 TTI 增加 PBR × TTI（如果 *prioritisedBitRate* 指定的 PRB 为 kBps8，则 PRB 为 8 kBps，即每 TTI 往令牌桶内注入 8 kBps * 1ms = 8 Byte 的 token）。Bj 的值不能超过桶的最大容量 PBR × BSD（以 BSD=500ms 为例，最大容量为 8kBps * 500ms = 4k Byte）。

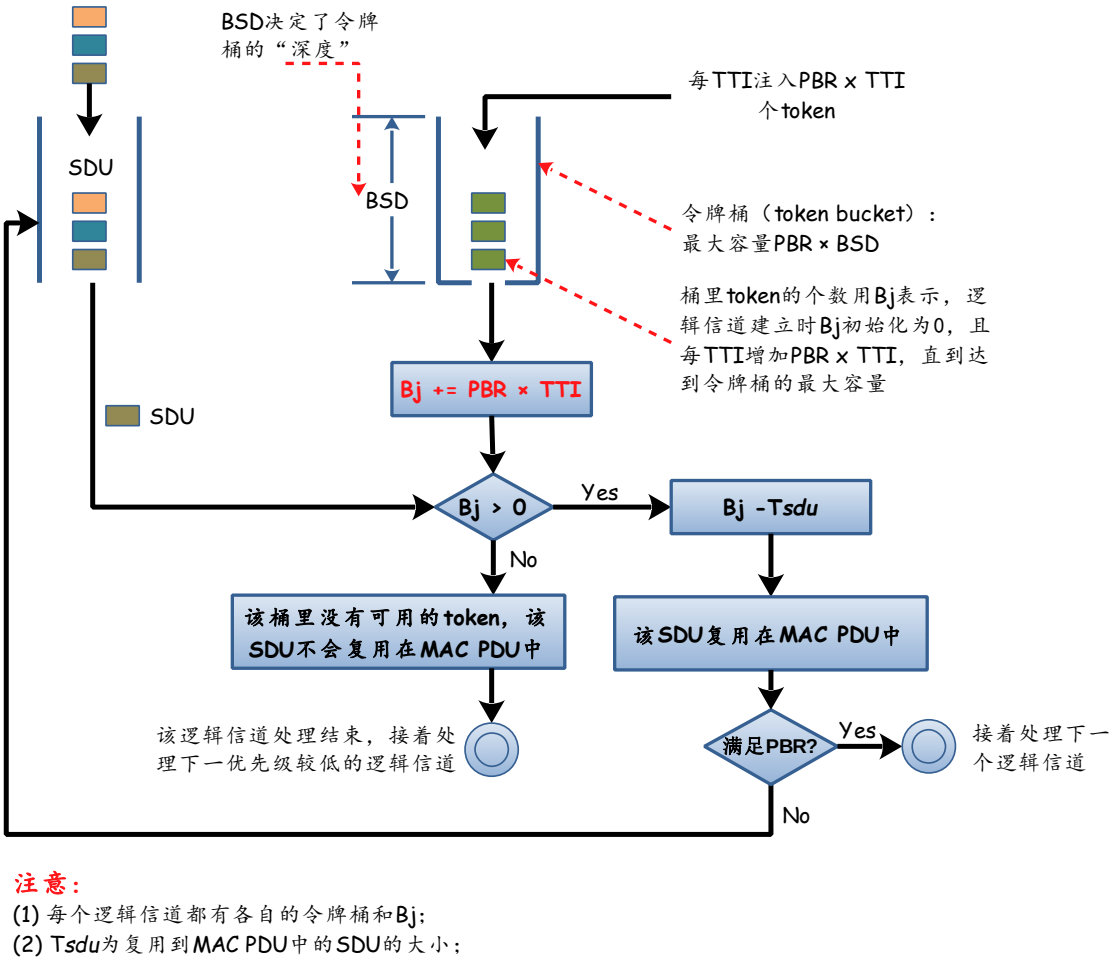


图 17-4：逻辑信道 j 的 MAC PDU 复用过程

当有**新传**（注意：不是重传）的数据时，UE 会按照如下步骤进行逻辑信道优先级化（Logical Channel Prioritization）：

Step1: 对于所有 Bj > 0 的逻辑信道，按照优先级递减顺序组包，每个逻辑信道分配的无线资源只能满足 PBR 的要求。当某个逻辑信道的 PBR 配置成无穷大（“infinity”）时，只有当这个逻辑信道的资源得到满足后，才会考虑比它优先级低的逻辑信道。

Step2：Bj 减去逻辑信道 j 在步骤 1 里复用到 MAC PDU 的所有 MAC SDUs 的大小。（参考实现：对于逻辑信道 j，每传输一个 RLC SDU，先比较 Bj 是否大于 0。如果 Bj 大于 0，则往 MAC PDU 中添加该 SDU。然后将 Bj 减去该 SDU 的大小 T*sdu*，并确定是否满足 PBR 的要求。如此反复，直到 Bj 小于 0，或满足 PBR 要求，则接着处理下一逻辑信道）

Step3：如果前两步执行完还剩有上行资源的话，则不管 Bj 的大小，把剩余的资源按照逻辑信道优先级分配给各个逻辑信道。只有当所有高优先级的逻辑信道的数据都发送完毕且 UL grant 还未

耗尽的情况下，低优先级的逻辑信道才能得到服务。即此时 UE 最大化高优先级的逻辑信道的数据传输。

与此同时，UE 还应遵循如下原则：（1）如果整个 RLC SDU 能够填入剩余的资源中，则不应对该 SDU 进行分段；（2）如果 UE 对逻辑信道中的 RLC SDU 进行分段，则应根据剩余资源的大小，尽量填入最大分段；（3）UE 应最大化数据的传输；（4）如果某个无线承载被挂起，则不应传输该无线承载对应逻辑信道的数据。

如果所有的逻辑信道的 PBR 都设置成 0 kBps，则会按照严格的优先级顺序来组包。此时 UE 会最大限度地满足更高优先级的数据的传输。

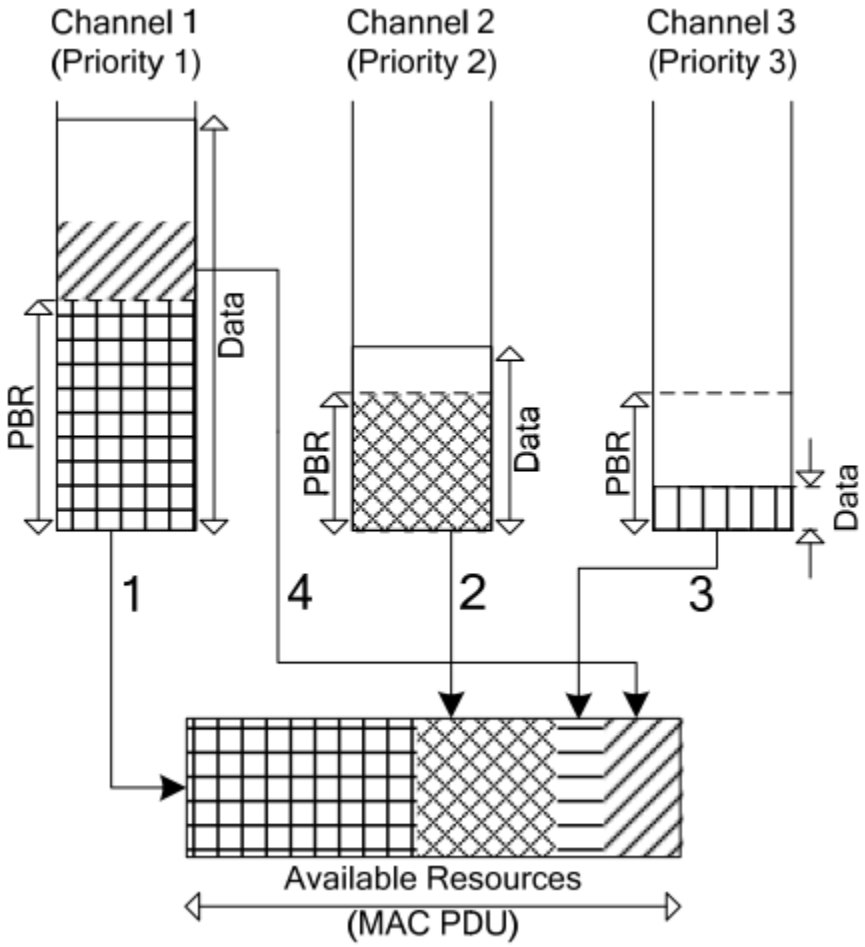


图 17-5：MAC 复用过程举例

如图 17-5 所示：Channel 1 优先级最高，所以其数据优先放入 MAC PDU 中，Channel 2 次之，Channel 3 最后。所有逻辑信道都处理完后，还有剩余的无线资源，则优先放置 Channel 1 的数据。

按照令牌桶的原理，当某 RLC SDU 的大小大于 Bj（即 Bj – size(RLC SDU) < 0）时，令牌桶中没有足够的令牌用于发送该 RLC SDU，此时该 SDU 应该放在 RLC 队列中，直到有足够的令牌后再进行传输。但如果 SDU 缓存在队列中的时间过长，又可能导致队列溢出，又导致了后续 SDU 的丢弃。权衡之下，LTE 允许 Bj < 0 的情况，即通过“借贷”的方式，将 SDU 先发出去，后续再“还贷”，贷款没还清之前，不准发送 SDU。

即当 Bj > 0（令牌桶中有令牌），但 Bj – sizeof(RLC SDU) < 0 时，数据不会放到队列中，而是采取“借贷”的方式获取到足够的 token（此时 Bj 减去 sizeof(RLC SDU)后，其值小于 0），把数据复用到 MAC PDU 中。只有将借到的 token 还完后（Bj 每 TTI 增加 PBR × TTI，直到 Bj > 0），该逻辑信道才能进行后续的数据传输。

该机制使得某个 TTI 给某个高优先级的逻辑信道分配更多的资源，使其 Bj < 0。但到了下一个或几个 TTI，由于 Bj 增加 PBR × TTI 后依然小于 0，导致逻辑信道无资格再发送数据，从而保证若干个 TTI 内的公平。出来借，迟早是要还得嘛！

前面主要介绍了单个逻辑信道的令牌桶算法实现。对于多个逻辑信道，其信道优先级 *priority* 应该根据信息重要程度的不同，考虑以下相对优先级（按从高到低的顺序排列）：

- 用于 C-RNTI 或来自 UL-CCCH 的数据的 MAC control element；
- 用于除“padding BSR”外的 BSR 的 MAC control element；
- 用于 PHR 或扩展 PHR 的 MAC control element；
- 除 UL-CCCH 的数据外，来自任意逻辑信道的数据；
- 用于“padding BSR”的 MAC control element。

简单地做个小结：eNodeB 通过限制令牌桶容量 PBR × BSD、令牌添加速度 PBR × TTI 来限制每个逻辑信道的速率，来保证低优先级逻辑信道的数据传输以及一定的流控作用。

可以看出，PBR 越大，每 TTI 往令牌桶里放置的 token 越多，逻辑信道也就能够更快地发送数据。而 PBR × BSD 决定了某个 TTI 内，逻辑信道能够发送的最大数据量（Bj 等于 PBR × BSD 且不考虑导致 Bj < 0 的情况）

载波聚合下的逻辑信道优先级处理：不同的载波单元可能提供相似的 QoS，因此允许 UE 自由决定如何将上行数据映射到不同 CC 上分配的上行资源上。当多个 serving cell 在同一子帧上给 UE 分配了上行资源（UL grant）时，协议并未要求 UE 先处理哪个 serving cell 的 UL grant，以及是采

用联合处理还是串行处理，这都交由 UE 自己来实现。但无论采取哪种方式，都需要满足每个逻辑信道的长期 PRB 要求。

17.2 MAC PDU

从 36.321 的 6.1 节可以看出，MAC PDU 分为几种类型：

- Transparent MAC PDU：用于传输来自 PCCH 或 BCCH 的数据，对 MAC 层是透明的，即 MAC 层不做任何处理，直接透传。
- Random Access Reponse MAC PDU：简称为 RAR MAC PDU，该 RAR MAC PDU 为随机接入过程的 Msg 2，用于发送 RAR。
- 除上面介绍的 2 种 MAC PDU 外的用于 DL-SCH 和 UL-SCH 的 MAC PDU：用于绝大多数来自 RRC 层的数据和用户数据的传输。多个无线承载的数据（以及 Mac Control Element）可能复用在同一个 MAC PDU 中。

注：MCH 对应的 MAC PDU 不在本书的讨论之列。

17.2.1 用于 DL-SCH 和 UL-SCH 的 MAC PDU

首先，我们来介绍一下除 Transparent MAC PDU 和 RAR MAC PDU 外的用于 DL-SCH 和 UL-SCH 的 MAC PDU。该 MAC PDU 的结构如 36.321 的 Figure 6.1.2-3 所示。

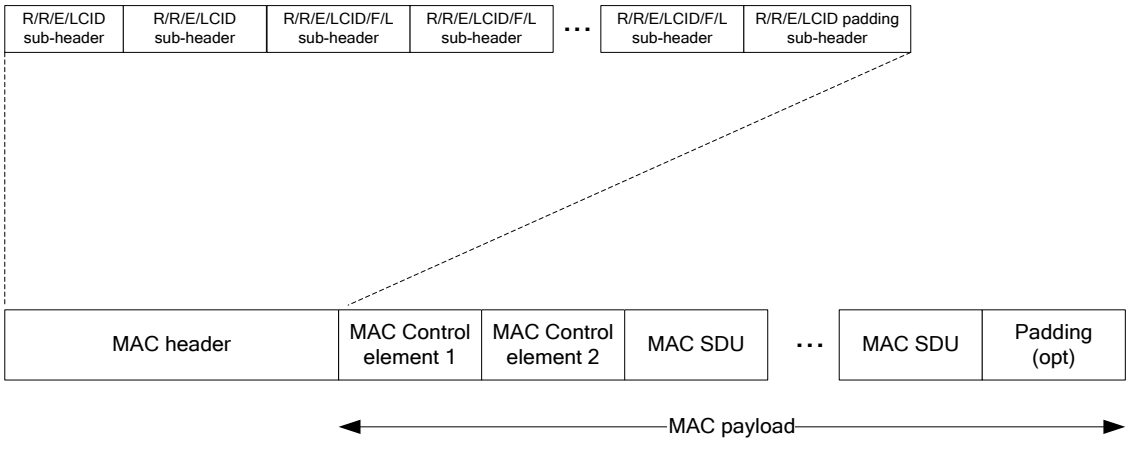


Figure 6.1.2-3: Example of MAC PDU consisting of MAC header, MAC control elements, MAC SDUs and padding

从 36.321 的 Figure 6.1.2-3 可以看出，一个 MAC PDU 由 1 个 MAC 头（MAC header）+ 0 个或多个 MAC SDU + 0 个或多个 MAC CE（Control Element）+ 可能存在的 padding 组成。

MAC header 由一个或多个 MAC subheader 组成。每个 subheader 对应一个 MAC PDU，或一个 MAC CE，或 padding。

MAC PDU 中的 subheader 包含以下几种类型：

- 除了 MAC PDU 内的最后一个 subheader 和用于固定长度的 MAC CE 的 subheader 外，其它 subheader 有 6 个域组成：R/R/E/LCID/F/L，如 36.321 的 Figure 6.1.2-1 所示。
- MAC PDU 内的最后一个 subheader 和用于固定长度的 MAC CE 的 subheader 由 4 个域组成：R/R/E/LCID，如 36.321 的 Figure 6.1.2-2 所示。哪些 MAC CE 是固定长度的，哪些 MAC CE 是非固定长度的，可以参见 36.321 的 6.1.3 节中关于各种 MAC CE 的介绍。
- 对应 padding 的 subheader 也由 4 个域组成：R/R/E/LCID，如 36.321 的 Figure 6.1.2-2 所示。但当需要 1-byte 或 2-byte 的 padding 时，会使用 1 个或 2 个 MAC PDU subheader（也由这 4 个域组成）替代 padding 的作用，其应用场景会在后续介绍。

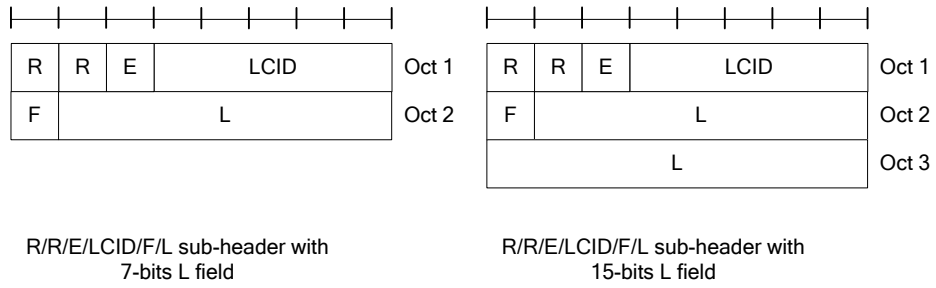


Figure 6.1.2-1: R/R/E/LCID/F/L MAC subheader

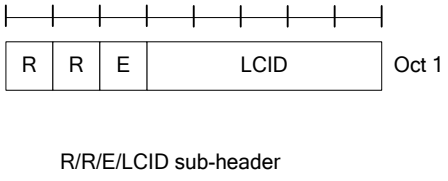


Figure 6.1.2-2: R/R/E/LCID MAC subheader

MAC PDU subheader 的顺序与对应的 MAC SDU、MAC CE 以及 padding 出现的顺序是一致的。

MAC CE 总是位于 MAC SDU 之前。

Padding 总是位于 MAC PDU 的最后（此时其 LCID 域对应 padding，即为 11111，并且 E=0）。但这里有一个特例：当需要 1-byte 或 2-byte 的 padding 时，会使用 1 个或 2 个 MAC PDU subheader 替代 padding 的作用，并将其放在 MAC PDU 的开头，即位于任何其他所有 subheader 的前面（为了描述的方便，我将其称之为 special padding subheader，此时其 LCID 域对应 padding，即为 11111，并且 E=1）。此时 special padding subheader 已经起到 padding 的作用，它没有对应的有效负载部分。也就是说，当需要 1 或 2 byte padding 的时候，这 1 个或 2 个 special padding subheader 已经替代了 padding 的作用，此时在 MAC PDU 的末尾，不会再有用于 padding 的比特了。为什么当需要 1-byte 或 2-byte 的 padding 时，要这样特殊处理的原因可参见参考资料[7]的介绍或[8]中的 References 的几个相关提案。

subheader 中有一个 LCID 域，LCID 是 Logical Channel ID 的简称，用于指示该 subheader 对应的 MAC SDU、或对应的 MAC CE 的类型、或对应的 padding。如 36.321 的 Table 6.2.1-1（用于 DL-SCH）和 Table 6.2.1-2（用于 UL-SCH）所示。MAC PDU 中的每个 MAC SDU、MAC CE 或 padding 都有一个对应的 LCID 域。另外的，当需要使用 special padding subheader 时，会有 1 个或 2 个额外的 LCID 域。

Table 6.2.1-1 Values of LCID for DL-SCH

Index	LCID values
00000	CCCH
00001-01010	Identity of the logical channel
01011-11010	Reserved
11011	Activation/Deactivation
11100	UE Contention Resolution Identity
11101	Timing Advance Command
11110	DRX Command
11111	Padding

Table 6.2.1-2 Values of LCID for UL-SCH

Index	LCID values
00000	CCCH
00001-01010	Identity of the logical channel
01011-11000	Reserved
11001	Extended Power Headroom Report
11010	Power Headroom Report
11011	C-RNTI
11100	Truncated BSR
11101	Short BSR
11110	Long BSR
11111	Padding

这里需要对取值范围为 00000~01010 的 LCID 做一下说明。

00000 对应 CCCH，由于 SRB0 在 CCCH 上传输，因此其对应的 LCID 固定为 0（即 00000）。

00001~01010 对应的是逻辑信道的标识，其含义如下：

- 36.331 中的 *SRB-ToAddMod -> srb-Identity* 指示了 SRB1 和 SRB2 对应的 LCID，SRB1 对应的值为 1（即 00001），SRB2 对应的值为 2（即 00010）。SRB1 和 SRB2 对应的逻辑信道标识可见 36.331 的 9.1.2 节。
- 36.331 中的 *DRB-ToAddMod -> logicalChannelIdentity* 指示了 DRB 对应的 LCID，其取值范围是 3~10（即 00011~01010）。

[6]中举了几个例子来详细介绍了 MAC 复用后的结果，以及如何解析这些 MAC PDU。感兴趣的话可以了解一下。

17.2.2 Transparent MAC PDU

如果一个 MAC PDU 用于传输来自 PCCH 或 BCCH 的数据，则该 MAC PDU 只能包含来自一个逻辑信道的数据。也就是说，PCCH 或 BCCH 的数据不能和其它逻辑信道的数据复用在同一个 MAC PDU 中传输，因此 PCCH 和 BCCH 并不需要使用 LCID 域，也不需要 MAC subheader。其实 PCCH 和 BCCH 使用的是 transparent MAC PDU（见 36.321 的 6.1.4 节），该 MAC PDU 只包含一个独立的 MAC SDU。UE 通过 P-RNTI 或 SI-RNTI 就能够区分所携带的数据。简而言之，PCCH 或 BCCH 在 MAC 层是透传的（见 36.300 的 6.1.2 节）。

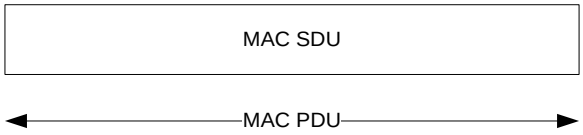


Figure 6.1.4-1: Example of MAC PDU (transparent MAC)

17.2.3 RAR MAC PDU

对于传输 Random Access Response 的 MAC PDU 而言，其只传输 RAR 相关的信息，而不能与 UL-SCH、PCCH、BCCH 等进行复用。关于 RAR MAC PDU 的介绍可见本书的 21.3.2 节，或见 36.213 的 6.1.5 节。

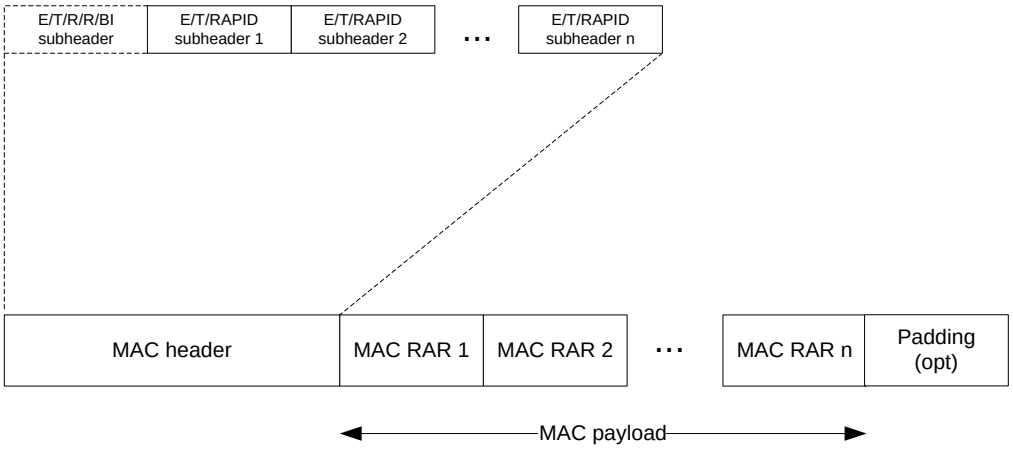


Figure 6.1.5-4: Example of MAC PDU consisting of a MAC header and MAC RARs

可以看出，eNodeB 在做下行调度时，会有不同的调度实体（不考虑 MBMS）：

- BCCH：包含 SIB1 和其它 SI 消息。（这里不包含 MIB。因为 MIB 的时频资源是固定，并在 PBCH 上传输）
- PCCH：包含 Paging 消息。
- RAR：即随机接入过程中的 Msg2。
- 每个 UE 调度的 DL-SCH 数据。

不同的调度实体不能复用在同一 MAC PDU 上传输，并需要使用独立的 PDCCH 来指示其在 PDSCH 上如何传输。

【参考资料】

[1] TS 36.321 的 5.4.3 节 Multiplexing and assembly

[2] TS 36.321 的 6.1 节 Protocol Data Units

[3] 《LTE – The UMTS Long Term Evolution, 2nd Edition》的 4.4.2.6 节

[4] 《LTE 系统 MAC 复用实体研究与设计》 冯川，李小文

[5] 《Logical channel prioritization procedure for generating multiple uplink transport blocks》

[6] http://www.sharetechnote.com/html/MAC_LTE.html#Examples_Good_MAC_PDU

[7] 《一种 MAC 层分组数据单元的配置方法和装置》 by 赵毅，谌丽，许芳丽

[8] “R2-080936 , MAC Padding”, Ericsson.

第18章 上行同步

本章将介绍 LTE 的上行同步过程。主要涉及：（1）为何需要上行同步；（2）eNodeB 如何测量上行定时提前量并下发 Timing Advance Command；（3）eNodeB 和 UE 如何判断上行失步（eNodeB 侧只会做一些原理性的介绍，不同厂家的实现可能不同）。

上行传输的一个重要特征是不同 UE 在时频上正交多址接入（orthogonal multiple access），即来自同一小区的不同 UE 的上行传输之间互不干扰。

为了保证上行传输的正交性，避免小区内（intra-cell）干扰，eNodeB 要求来自同一子帧但不同频域资源（不同的 RB）的不同 UE 的信号到达 eNodeB 的时间基本上是对齐的。eNodeB 只要在循环前缀（Cyclic Prefix）范围内接收到 UE 所发送的上行数据，就能够正确地解码上行数据，因此上行同步要求来自同一子帧的不同 UE 的信号到达 eNodeB 的时间都落在循环前缀范围之内。

为了保持使用不同循环移位的上行参考信号之间的正交性，也要求接收到的上行参考信号必须是时间对齐的，这也是需要使用上行同步，以保证同一小区的不同 UE 的上行传输在时间上对齐的原因。

为了保证接收侧（eNodeB 侧）的时间同步，LTE 提出了上行定时提前（Uplink Timing Advance）的机制。

在 UE 侧看来，timing advance 本质上是接收到下行子帧的起始时间与传输上行子帧的时间之间的一个负偏移（negative offset）。eNodeB 通过适当地控制每个 UE 的偏移，可以控制来自不同 UE 的上行信号到达 eNodeB 的时间。对于离 eNodeB 较远的 UE，由于有较大的传输延迟，就要比离 eNodeB 较近的 UE 提前发送上行数据。

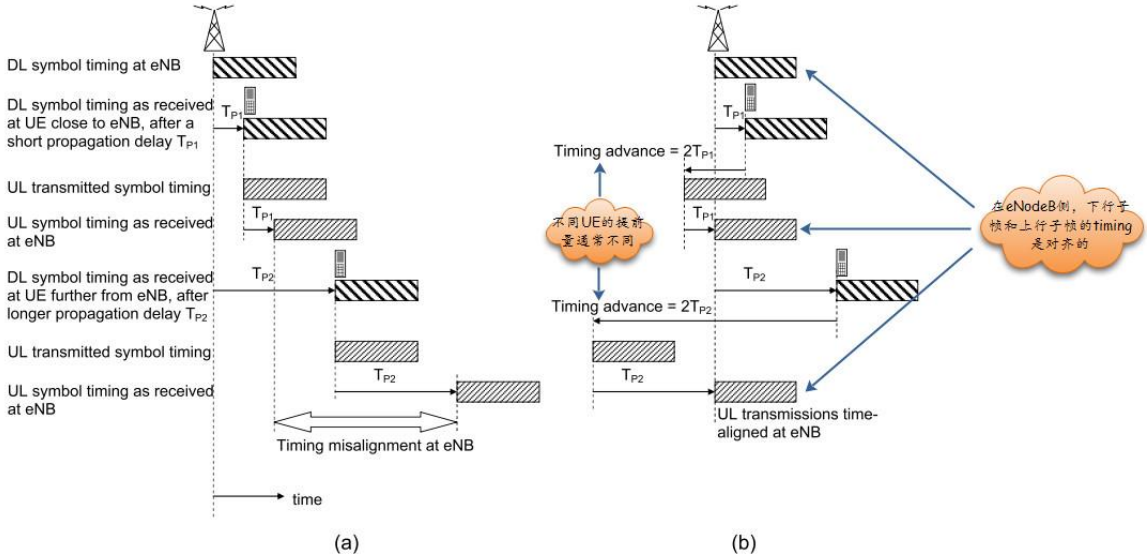


Figure 18.1: Time alignment of uplink transmissions:
(a) without timing advance; (b) with timing advance.

图 18-1：上行传输的 timing 对齐

图 18-1 的(a)中指出了不进行上行定时提前所造成的影响。

从图 18-1 的(b)中可以看出，eNodeB 侧的上行子帧和下行子帧的 timing 是相同的，而 UE 侧的上行子帧和下行子帧的 timing 之间有偏移。

同时可以看出：不同 UE 有各自不同的上行 timing advance，也即上行 timing advance 是 UE 级的配置。

处于上行失步（OUT_OF_SYNC）状态的 UE，还是可以接收下行数据的，但是不能发送上行数据。

前面介绍了为什么需要做上行 timing advance，接下来我们来介绍 eNodeB 如何测量上行信号以得到每个 UE 的上行定时提前量以及如何下发 Timing Advance Command 给 UE。

eNodeB 通过两种方式给 UE 发送 Timing Advance Command：

（1）在随机接入过程中，eNodeB 通过测量接收到的 preamble 来确定 timing advance 值，并通过 RAR 的 Timing Advance Command 字段（共 11 比特，对应 TA 索引值 T_A 的范围是 0~1282）发送给 UE。

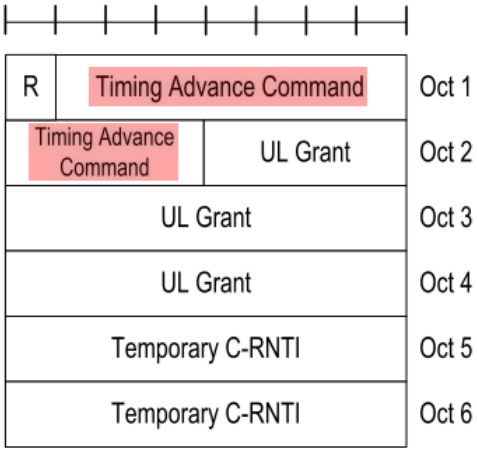


Figure 6.1.5-3: MAC RAR

图 18-2：MAC RAR

上行同步的粒度为16 T_s（0.52 μs）。对于随机接入而言，T_A 值乘以16 T_s，就得到相对于当前上行 timing 所需的实际调整值 $N_{TA} = T_A \times 16$（单位为 T_s）。关于 T_s，见本书 2.1 节的介绍。

上行 timing 的不确定性正比于小区半径，每 1 km 有大约 6.7μs 的传输延迟（6.7μs / km），LTE 中小区最大半径为 100 km，故最大传输延迟接近 0.67 ms。上行同步的粒度为16 T_s（0.52 μs），故 T_A 的最大值约为(0.67 * 1000)/0.52 ≈1288。（T_A 的最大值为 1282，应该是更精确的计算，但计算方法就是这样的，当然还要将解码时间考虑在内）

我称这个过程为“**初始上行同步过程**”。

（2）在 RRC_CONNECTED 态，eNodeB 需要维护 timing advance 信息。

虽然在随机接入过程中，UE 与 eNodeB 取得了上行同步，但上行信号到达 eNodeB 的 timing 可能会随着时间发生变化：

- 高速移动中的 UE，例如运行中的高铁上的 UE，其与 eNodeB 的传输延迟会不断变化；
- 当前传输路径消失，切换到新的的传输路径。例如在建筑物密集的城市，走到建筑的转角时，这种情况就很可能发生；
- UE 的晶振偏移，长时间的偏移累积可能导致上行定时出错；
- 由于 UE 移动而导致的多普勒频移等。

因此，UE 需要不断地更新其上行定时提前量，以保持上行同步。LTE 中，eNodeB 使用一种闭环机制来调整上行定时提前量。

eNodeB 基于测量对应 UE 的上行传输来确定每个 UE 的 timing advance 值。因此，只要 UE 有上行传输， eNodeB 就可以用来估计 timing advance 值。理论上，UE 发送的任何信号（SRS/DMRS/CQI/ACK/NACK/PUSCH 等）都可用于测量 timing advance。

如果某个特定 UE 需要校正，则 eNodeB 会发送一个 Timing Advance Command 给该 UE，要求其调整上行传输 timing。该 Timing Advance Command 是通过 Timing Advance Command MAC control element 发送给 UE 的。

Timing Advance Command MAC control element 由 LCID 值为 11101（见 36.321 的 Table 6.2.1-1）的 MAC PDU subheader 指示，且其结构如下（R 表示预留比特，设为 0）：

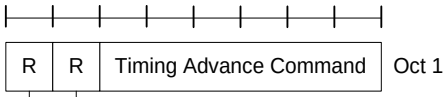


Figure 6.1.3.5-1: Timing Advance Command MAC control element

可以看出，Timing Advance Command 字段共 6 比特，对应 TA 索引值 T_A 的范围是 0~63。

UE 侧会保存最近一次 timing advance 调整值 $N_{TA,old}$，当 UE 收到新的 Timing Advance Command 而得到 T_A 后，会计算出最新的 timing advance 调整值 $N_{TA,new} = N_{TA,old} + (T_A - 31) \times 16$（单位为 T_s）。

我称这个过程为“**上行同步更新过程**”。

如果 UE 在子帧 n 收到 Timing Advance Command，则 UE 会从子帧 n+6 开始应用该 timing 调整值。

如果 UE 在子帧 n 和子帧 n+1 发送的 PUCCH/PUSCH/SRS 由于 timing 调整的原因出现重叠，则 UE 将完全发送子帧 n 的内容，而不发送子帧 n+1 中重叠的部分。

UE 收到 Timing Advance Command 后，会调整 PCell 的 PUCCH/PUSCH/SRS 的上行发送时间。而 SCell 的 PUSCH/SRS（SCell 不发送 PUCCH）的上行发送时间调整量与 PCell 相同。（见 36.213 的 4.2.3 节，注意：Rel-11 中的机制会有不同）

从前面的介绍可以看出，在 Rel-10 中，PCell 和 SCell 共用一条 Timing Advance Command。在载波聚合中，UE 可能需要往多个小区（或称为 component carrier）发送上行数据，在理论上，由于不

同小区的物理位置（inter-band CA）可能不同，每个小区都需要给该 UE 发送各自的 Timing Advance Command。但是这种类型的部署并不常见，载波聚合的小区通常物理位置上相近且同步，因此为了简化 LTE 的设计，所有聚合的小区共用一条 timing advance command。

注：Rel-11 中，不同小区可以使用不同的 TA，但本书只涉及 Rel-10 的内容。

前面已经介绍过，上行定时提前的调整量是相对于接收到的下行子帧的 timing 的，因此在 UE 没有收到 Timing Advance Command 的时候，UE 需要跟踪下行 timing 的变化，以便自动调整上行传输的 timing。（详见 36.133 的 7.1.2 节）

接下来，我们介绍 UE 在 MAC 层如何判断上行同步/失步（详见 36.321 的 5.2 节）：

eNodeB 会通过 RRC 信令给 UE 配置一个 timer（在 MAC 层，称为 *timeAlignmentTimer*），UE 使用该 timier 在 MAC 层确定上行是否同步。

需要注意的是：该 timer 有 Cell-specific 级别和 UE-specific 级别之分。eNodeB 通过 *SystemInformationBlockType2* 的 *timeAlignmentTimerCommon* 字段来配置的小区特定级别的 timer；eNodeB 通过 *MAC-MainConfig* 的 *timeAlignmentTimerDedicated* 字段来配置 UE 特定级别的 timer。

如果 UE 配置了 UE 特定的 timer，则 UE 使用该 timer 值，否则 UE 使用小区特定的 timer 值。

当 UE 收到 Timing Advance Command（来自 RAR 或 Timing Advance Command MAC control element），UE 会启动或重启该 timer。如果该 timer 超时，则认为上行失步，UE 会清空 HARQ buffer，通知 RRC 层释放 PUCCH/SRS，并清空任何配置的 DL assignment 和 UL grant。

当该 timer 在运行时，UE 认为上行是同步的；而当该 timer 没有运行，即上行失步时，UE 在上行只能发送 preamble。

还有一种情况下，UE 认为上行同步状态由“同步”变为“不同步”：非同步 Handover。

最后，我们介绍 eNodeB 是如何处理 UE 的上行同步呢？由于不同的厂商实现方式可能不同，这里只介绍一些可借鉴的做法。

（1）由于 UE 必须在 *timeAlignmentTimer* 超时之前接收到 Timing Advance Command，否则会认为上行失步。所以 eNodeB 需要保证在该 timer 时间范围内（通常要比该 timer 小，因为要预留一些时间给传输延迟和 UE 编解码等）给 UE 发送 Timing Advance Command，以便 UE 更新上行定时并重启该 timer。所以 eNodeB 必须保存最近一次**成功**地给该 UE 发送了 Timing Advance Command（即 eNodeB 收到了对应下行传输的 ACK）的子帧号，以便计算该时间范围。

（2）从（1）中可以看出，在 eNodeB 侧在 MAC 层也应该为每个 UE 维护一个类似 *timeAlignmentTimer* 的 timer，以保证在该 timer 超时之前给 UE 发送 Timing Advance Command。

eNodeB 何时启动/重启该 timer 呢？个人认为可以在 UE 随机接入成功中后启动，并在收到对应 Timing Advance Command MAC control element 的 ACK/NACK 后重启。注意 timer 的起始位置应该从最近一次**成功**地给该 UE 发送了 Timing Advance Command 的子帧（而不是收到对应 ACK 的子帧）。

（3）从上面的介绍可以看出， UE 在子帧 n 收到 Timing Advance Command 后，会从子帧 n + 6 才开始应用该 timing 调整值。也就是说，eNodeB 在子帧 n 发送了某个 UE 的 Timing Advance Command 之后，在子帧 n + 6 之前（不包括 n + 6 子帧）的时间内，是不会去测量该 UE 的上行 timing 的。

（4）在子帧 n + 6 之后，eNodeB 可能需要测量多个上行 timing 瞬时值以作平均处理，以便得到最终的调整量，也就是说，eNodeB 可能在 n + 6 子帧后的某段时间内，是不会发送 Timing Advance Command 的。当测量完毕后，eNodeB 在之后的某个子帧将 Timing Advance Command MAC control element 发给 UE。

（5）eNodeB 在物理层（L1 层）应该也会判断 UE 在上行是否同步（例如：物理层会根据 UL 信号来计算 SINR（也用于估算 TA 值），如果算出的 SINR 值过低，物理层就会认为 UL 失步），如果上行不同步，应告知 MAC 层。

关于 eNodeB 侧的上行同步处理，大家可以参考一下中兴的一篇专利，详见参考资料[9]。

【参考资料】

[1] TS 36.133 的第 7 章 Timing and signalling characteristics

[2] TS 36.213 的 4.2.3 节 Transmission timing adjustments

[3] TS 36.300 的 5.2.7.3 节 Uplink timing control

[4] TS 36.300 的 10.1.2.7 节 Timing Advance

[5] TS 36.321 的 5.2 节 Maintenance of Uplink Time Alignment

[6] TS 36.321 的 6.1.3.5 节 Timing Advance Command MAC Control Element

[7] 《4G LTE/LTE-Advanced for Mobile Broadband》的 11.5 节

[8] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 18.2.1 节

[9] 《LTE 的时间提前调整方法及基站》 ---中兴的一篇专利，介绍 eNodeB 侧如何处理上行同步的。

第19章 小区搜索过程（Cell Search Procedure）

UE 要接入 LTE 网络，必须经过小区搜索、获取小区系统信息、随机接入等过程。

本章将介绍小区搜索过程。主要涉及 PSS/SSS 以及 UE 通过 PSS/SSS 能够得到哪些有用的信息。

小区搜索的主要目的：（1）与小区取得频率和符号同步（下行同步）；（2）获取系统帧 timing，即下行帧的起始位置；（3）确定小区的 PCI（Physical-layer Cell Identity）。

UE 不仅需要在开机时进行小区搜索，为了支持移动性（mobility），UE 会不停地搜索邻居小区、取得同步并估计该小区信号的接收质量，从而决定是否进行切换（handover，当 UE 处于 RRC_CONNECTED 态）或小区重选（cell re-selection，当 UE 处于 RRC_IDLE 态）。

LTE 一共定义了 504 个不同的 PCI（对应协议 36.211 中的 N_{ID}^{cell}，取值范围 0 ~ 503），且每个 PCI 对应一个特定的下行参考信号序列。所有 PCI 的集合被分成 168 个组（对应协议 36.211 中的 $N_{ID}^{(1)}$，取值范围 0 ~ 167），每组包含 3 个小区 ID（对应协议 36.211 中的 $N_{ID}^{(2)}$，取值范围 0 ~ 2）。即有

$$N_{ID}^{cell} = 3N_{ID}^{(1)} + N_{ID}^{(2)}$$

为了支持小区搜索，LTE 定义了 2 个下行同步信号：PSS（Primary Synchronization Signal，主同步信号）和 SSS（Secondary Synchronization Signal，辅同步信号）。

对于 TDD 和 FDD 而言，这 2 类同步信号的结构是完全一样的，但在帧中的时域位置有所不同。

对于 FDD 而言，PSS 在子帧 0 和 5 的第一个 slot 的最后一个 OFDM 符号上发送；SSS 与 PSS 在同一子帧同一 slot 上发送，但 SSS 位于倒数第二个 OFDM 符号上，比 PSS 提前一个 OFDM 符号。（见图 19-1）

对于 TDD 而言，PSS 在子帧 1 和 6 的第三个 OFDM 符号上发送；而 SSS 在子帧 0 和 5 的最后一个 OFDM 符号上发送，比 PSS 提前 3 个符号。（见图 19-1）

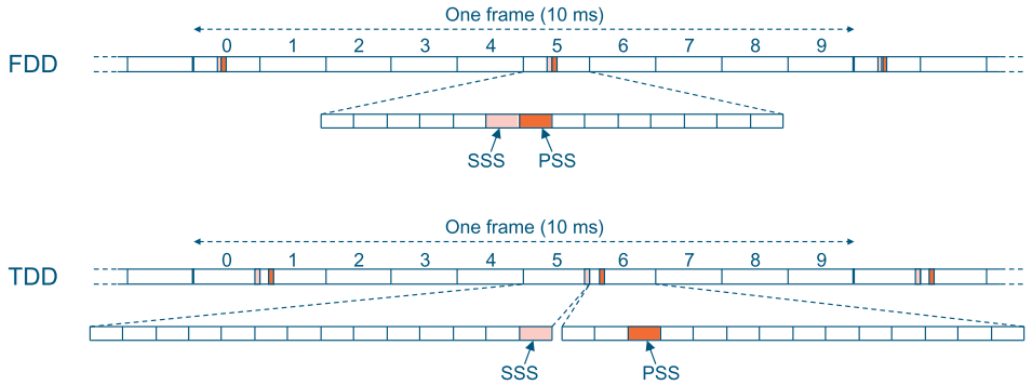


图 19-1：FDD 或 TDD 中，PSS/SSS 的时域位置

UE 开机时并不知道小区系统带宽的大小，但它知道自己支持的频带和带宽（见 36.101）。为了使 UE 能够尽快检测到系统的频率和符号同步信息，无论下行系统带宽的大小是多少，PSS 和 SSS 都位于中心的 72 个子载波上（即中心的 6 个 RB 上，不包含 DC。实际只使用了频率中心 DC 周围的 62 个子载波，两边各留了 5 个子载波用作保护波段）。UE 会在其支持的 LTE 频率的中心频点附近去尝试接收 PSS 和 SSS。

关于 PSS/SSS 在时频上的位置，详见 36.211 的 6.11.1.2 节和 6.11.2.2 节。公式比较简单，这里就不做介绍了。

PSS 使用长度为 63 的 Zadoff-Chu 序列（中间有 DC 子载波，所以实际上传输的长度为 62），加上边界额外预留的用作保护频段的 5 个子载波，形成了占据中心 72 个子载波（不包含 DC）的 PSS。（见图 19-2）

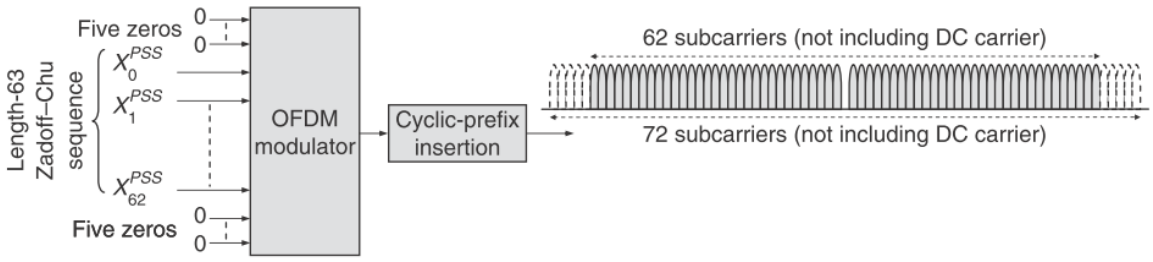


图 19-2：PSS 的结构

PSS 有 3 个取值，对应 3 种不同的 Zadoff-Chu 序列，每种序列对应一个 $N_{ID}^{(2)}$。某个小区的 PSS 对应的序列由该小区的 PCI 决定，即 N_{ID}^{cell} % 3。从 36.211 的 6.11.1.1 可以看出，不同的 $N_{ID}^{(2)}$ 对应不同的 Root index u，进而决定了不同 Zadoff-Chu 序列。

Table 6.11.1.1-1: Root indices for the primary synchronization signal.

$N_{\mathrm{ID}}^{(2)}$	**Root index** u
0	25
1	29
2	34

UE 为了接收 PSS，会使用 36.211 中 Table 6.11.1.1-1 指定的 Root index u 来尝试解码 PSS，直到其中某个 Root index u 成功解出 PSS 为止。这样，UE 就知道了该小区的 $N_{ID}^{(2)}$。又由于 PSS 在时域上的位置是固定的（见图 19-1），因此 UE 又可以得到该小区的 5 ms timing（一个系统帧内有两个 PSS，且这两个 PSS 是相同的，因此 UE 不知道解出的 PSS 是第一个还是第二个，所以只能得到 5 ms timing）。

综上所述，通过 PSS，UE 可以得到如下信息：

- $N_{ID}^{(2)}$
- 5 ms timing

SSS 由 2 个长度为 31 的 m-sequence 交织而成，形成长为 63 的序列（中间有 DC 子载波，所以实际上传输的长度为 62），加上边界额外预留的用作保护频段的 5 个子载波，形成了占据中心 72 个子载波（不包含 DC）的 SSS。且从图 19-1 可以看出，无论是 FDD 还是 TDD，SSS 都在子帧 0 和 5 上传输。

LTE 中，SSS 的设计有其特别之处：

- 2 个 SSS（SSS_1 位于子帧 0，SSS_2 位于子帧 5）的值来源于 168 个可选值的集合，其对应 168 个不同 $N_{ID}^{(1)}$；（见 36.211 的 Table 6.11.2.1-1，$N_{ID}^{(1)} = N_{ID}^{cell} / 3$）
- SSS_1 的取值范围与 SSS_2 是不同的，因此允许 UE 只接收一个 SSS 就检测出系统帧 10 ms 的 timing（即子帧 0 所在的位置）。这样做的原因在于，小区搜索过程中，UE 会搜索多个小区，搜索的时间窗可能不足以让 UE 检测超过一个 SSS。

SSS 的结构如图 19-3 所示，SSS_1 是由 2 个长度为 31 的 m-sequence X 和 Y 交织而成的，每个都可以取 31 个不同的值（实际上是同一 m-sequences 的 31 种不同的偏移，对应 36.211 的 Table

6.11.2.1-1 的 m_0 和 m_1 ）。在同一个小区中， sss_2 与 sss_1 使用的是相同的 2 个 m-sequence，不同的是，在 sss_2 中，这 2 个 sequence（X 和 Y）在频域上交换了一下位置，从而保证了 sss_1 和 sss_2 属于不同的集合。（36.211 的 6.11.2.1 中计算 d(2n)和 d(2n+1)的那个公式，可以很好地说明 sss_1 和 sss_2 的不同。对于 sss_1 而言，偶数位偏移 m_0 位，奇数位偏移 m_1 位；对于 sss_2 而言，偶数位偏移 m_1 位，奇数位偏移 m_0 位。而从 Table 6.11.2.1-1 可以看出，(m_0, m_1)组成一个取值，且 m_0 一定小于 m_1 ，因此， sss_1 和 sss_2 的取值范围必定不同）

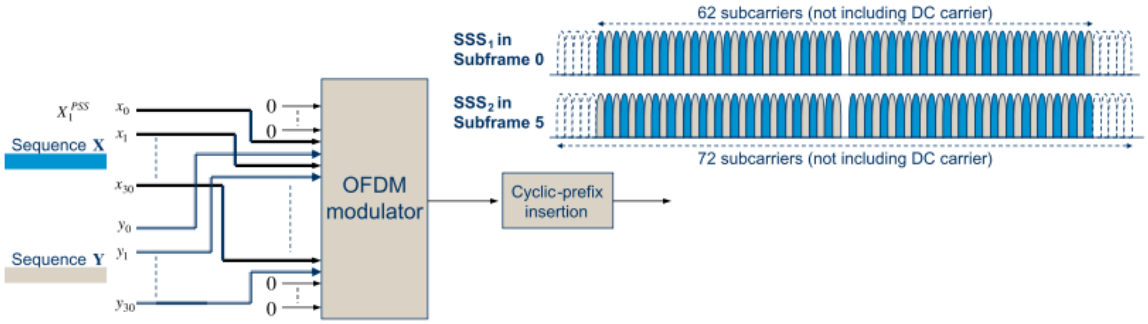


图 19-3：SSS 的结构

下面介绍 UE 是如何解码 SSS 的。

步骤一：UE 知道 PSS 后，就知道了 SSS 可能的位置。为什么说是可能呢？

首先，UE 在检测到 SSS 之前，还不知道该小区是工作在 FDD 还是 TDD 模式下。如果 UE 同时支持 FDD 和 TDD，则会在 2 个可能的位置上（见图 19-1）去尝试解码 SSS。如果在 PSS 的前一个 OFDM 符号上检测到 SSS，则小区工作在 FDD 模式下；如果在 PSS 的前 3 个 OFDM 符号上检测到 SSS，则小区工作在 TDD 模式下。如果 UE 只支持 FDD 或 TDD，则只会在相应的位置上去检测 SSS，如果检测不到，则认为不能接入该小区。（通过检测 SSS，UE 知道小区是工作在 FDD 模式还是 TDD 模式下）

其次，SSS 的确切位置还和循环前缀（Cyclic Prefix，CP）的长度有关（如图 19-4、图 19-5 所示）。在此阶段，UE 还不知道小区的循环前缀配置（正常的循环前缀还是扩展的循环前缀），因此会在这两个可能的位置去盲检 SSS。（通过检测 SSS，UE 知道小区的循环前缀配置）

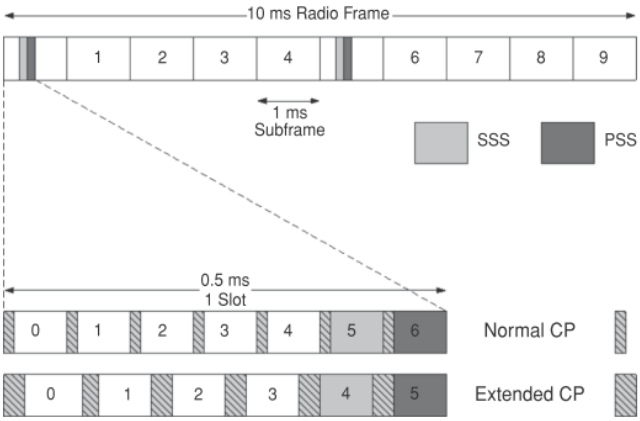


图 19-4：FDD 模式下，PSS/SSS 的帧和 slot 在时域上的结构

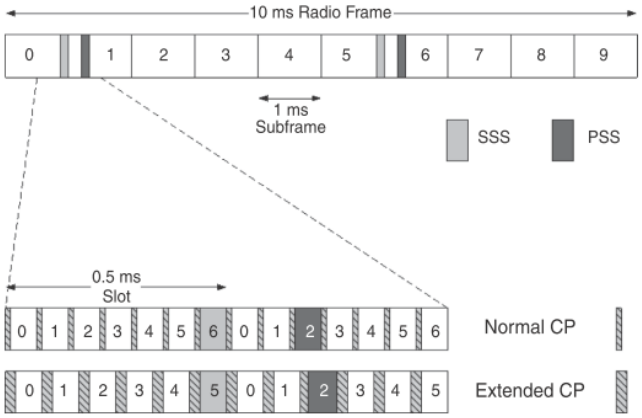


图 19-5：TDD 模式下，PSS/SSS 的帧和 slot 在时域上的结构

步骤二：UE 会在 SSS 可能出现的位置（如果 UE 同时支持 FDD 和 TDD，则至多有 4 个位置），根据 36.211 中 6.11.2.1 节里的公式、Table 6.11.2.1-1 中可能出现的 168 种取值、以及 X 与 Y 交织的顺序（以便确定是 sss_1 还是 sss_2，其实都能体现在公式里）等，盲检 SSS。

如果成功解码出 SSS（当然也知道了该 SSS 是 sss_1 还是 sss_2），就确定了 168 种取值之一（Table 6.11.2.1-1 中的 m_0 和 m_1），也就确定了 $N_{ID}^{(1)}$。确定了 SSS 是 sss_1 还是 sss_2，也就确定了该 SSS 是位于子帧 0 还是子帧 5，进而也就确定了该系统帧中子帧 0 所在的位置，即 10 ms timing。

综上所述，通过 SSS，UE 可以得到如下信息：

- $N_{ID}^{(1)}$，加上检测 PSS 时得到的 $N_{ID}^{(2)}$，也就得到了小区的 PCI；
- 由于小区特定的参考信号及其时频位置与 PCI 是一一对应的，因此也就知道了该小区的下行小区特定的参考信号及其时频位置；
- 10 ms timing，即系统帧中子帧 0 所在的位置（此时还不知道系统帧号，需要进一步解码 PBCH）；
- 小区是工作在 FDD 还是 TDD 模式下；
- 循环前缀配置：是正常的循环前缀还是扩展的循环前缀。

在多天线传输的情况下，同一子帧内，PSS 和 SSS 总是在相同的天线端口上发射，而在不同的子帧上，则可以利用多天线增益，在不同的天线端口上发射。

如果是初始同步（此时 UE 还没有驻留或连接到一个 LTE 小区），在检测完同步信号之后，UE 会解码 PBCH，以获取最重要的系统信息。

如果是识别邻居小区，UE 并不需要解码 PBCH，而只需要基于最新检测到的小区参考信号来测量下行信号质量水平，以决定是进行小区重选（UE 处于 RRC_IDLE 态）还是 handover（UE 处于 RRC_CONNECTED 态。此时 UE 会通过 RSRP 将这些测量结果上报给服务小区，决定是否进行 handover）。

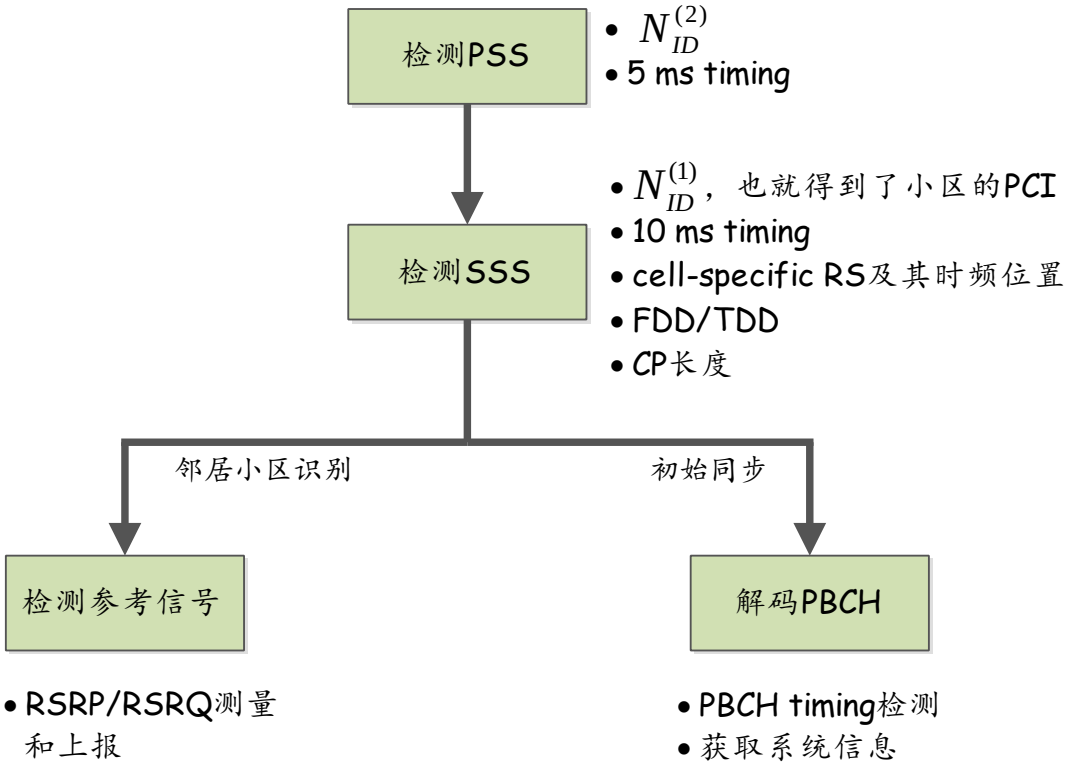


图 19-6：小区搜索过程的各个阶段所获取到的信息

在 LTE 网络规划中，需要避免相邻小区的“PCI mod 3”、“PCI mod 6”、“PCI mod 30”的值相等。即所谓的避免“模 3 干扰”、“模 6 干扰”、“模 30 干扰”。

- 模 3 干扰：由于 $\mathrm{PCI} = N_{ID}^{cell} = 3N_{ID}^{(1)} + N_{ID}^{(2)}$，如果相邻小区的 PCI mod 3 的值相同，那么相邻小区的 PSS 相同，就会造成 PSS 的相互干扰。
- 模 6 干扰：在时域位置固定的情况下，下行小区特定的参考信号在频域有 6 个频率移位（frequency shift）。如果相邻小区的 PCI mod 6 值相同，那么下行小区特定的参考信号在频域上的位置会重叠，就会造成参考信号间的相互干扰。
- 模 30 干扰：在 PUSCH 信道中携带了 DMRS 和 SRS 信息，这两个参考信号对于信道估计和解调非常重要。它们是由 30 组基本的 ZC 序列构成，即有 30 组不同的序列组合。如果相邻小区的 PCI mod 30 值相同，则会使用相同的 ZC 序列，就会造成上行 DMRS 和 SRS 的相互干扰。

【参考资料】

[1] TS 36.211 的 6.11 节 Synchronization signals

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 14.1 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的第 7 章

[4] 《LTE 中的小区搜索过程》

[5] 《LTE 小区搜索过程学习总结》

[6] “R1-060516, Cell Search procedure of EUTRA TDD system for the initial synchronization”, CATT, RITT

第20章 系统信息（System Information）

小区搜索过程之后，UE 已经与小区取得下行同步，得到小区的 PCI 以及检测到系统帧的 timing（即 10ms timing）。接着，UE 需要获取到小区的系统信息（System Information），这样才能知道该小区是如何配置的，以便接入该小区并在该小区内正确地工作。

系统信息是小区级别的信息，即对接入该小区的所有 UE 生效。系统信息可分为 *MasterInformationBlock*（MIB）和多个 *SystemInformationBlock*（SIB），每个系统信息包含了与某个功能相关的一系列参数集合。系统信息类型包括：

表 20-1：系统信息类型

类型	内容	周期（ms）	传输信道
MIB	包含一些数量有限但最重要也最频繁发送的参数，UE必须使用这些参数来获取其它的系统信息。见36.331的IE: *MasterInformationBlock*	40	BCH
SIB1	包含用来判断某小区是否适合用于小区选择的参数，以及其它SIB的时域调度信息。见36.331的IE: *SystemInformationBlockType1*	80	DL-SCH
SIB2	包含了公共的无线资源配置信息，对所有UE通用	由SIB1配置，见36.331的IE: *SystemInformationBlockType1*的字段: *SchedulingInfoList*	
SIB3~SIB8	包含了intra-frequency，inter-frefquency以及inter-RAT小区重选相关的参数		
SIB9	包含home eNodeB的名字		
SIB10~SIB12	包含地震、ETWS和CMAS警告消息		
SIB13	包含与一个或多个MBSFN区域相关的MBMS控制信息		

并不是所有的 SIB 都必须存在。例如对于运营商部署的基站而言，就不需要 SIB9；如果某小区不提供 MBMS，就不需要 SIB13。

小区会不断地广播这些系统信息。

有 3 种类型的 RRC 消息用于传输系统信息：MIB 消息、SIB1 消息、一个或多个 SI 消息。

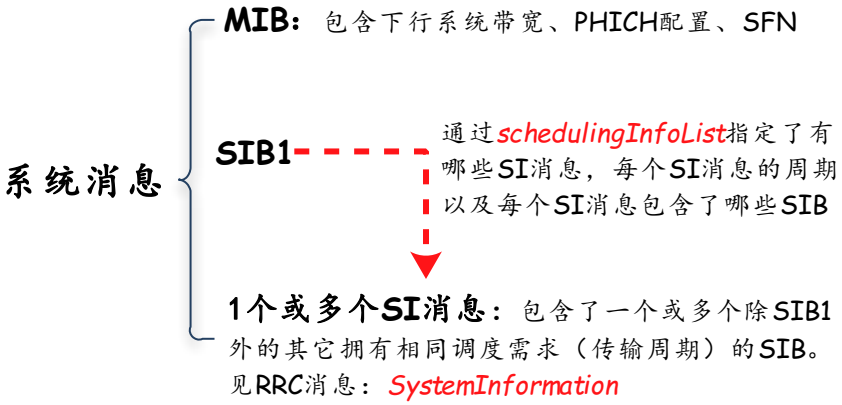


图 20-1：3 类用于发送系统信息的 RRC 消息

小区是通过逻辑信道 BCCH 向该小区内的所有 UE 发送系统信息的。

从图 20-2、图 20-3、图 20-4 可以看出，逻辑信道 BCCH 会映射到传输信道 BCH 和 DL-SCH。其中，BCH 只用于传输 MIB 信息，并映射到物理信道 PBCH；DL-SCH 用于传输各种 SIB 信息，并映射到物理信道 PDSCH。

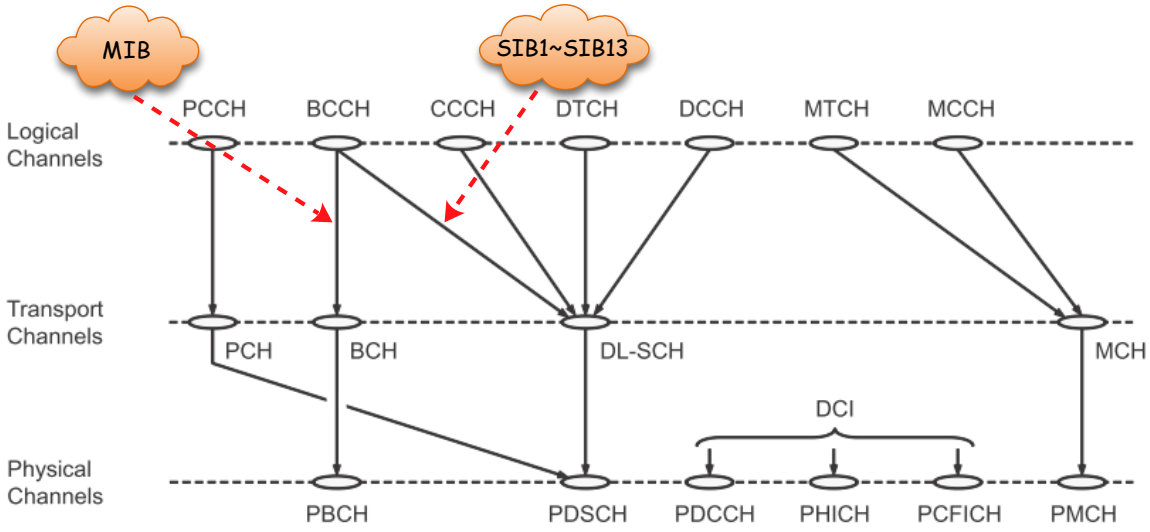


图 20-2：下行信道匹配

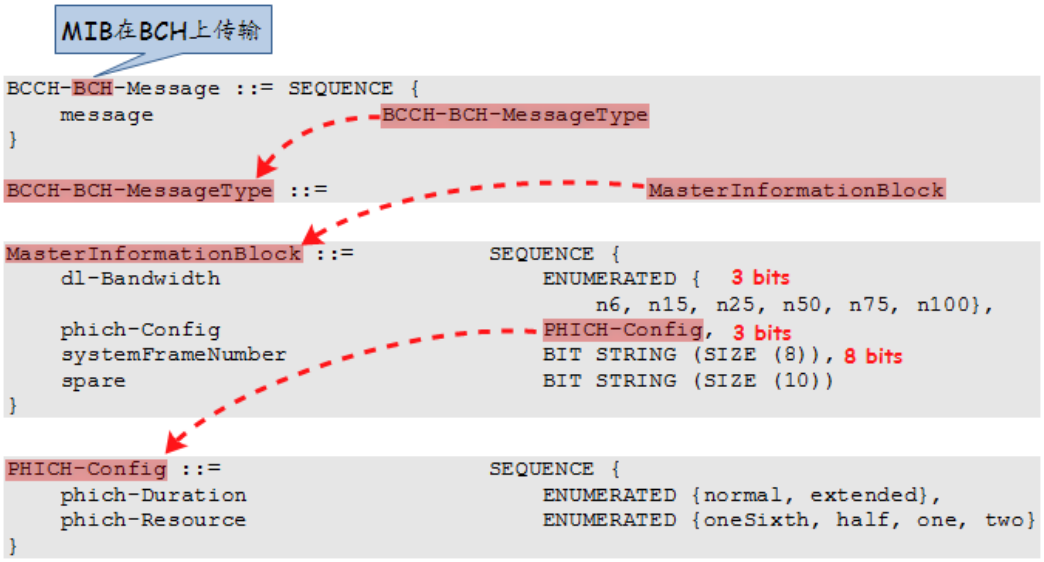


图 20-3：从 RRC 信令中可以看出：MIB 在 BCH 上传输

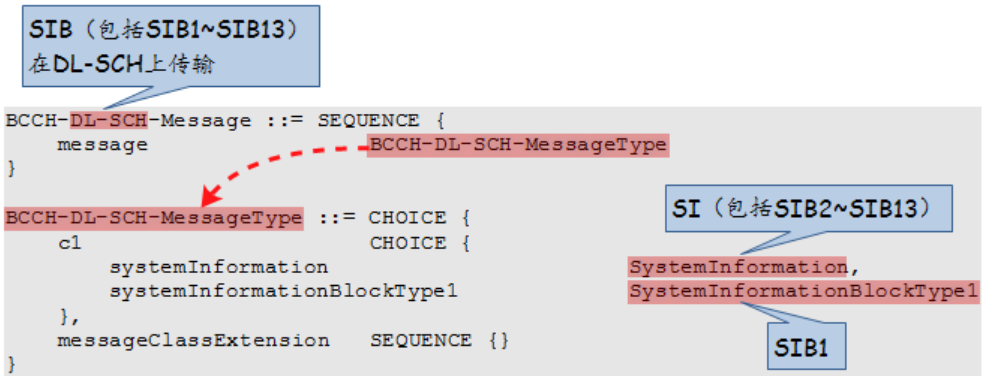


图 20-4：从 RRC 信令中可以看出：SIB 在 DL-SCH 上传输

20.1 MIB

对于 MIB 而言，除了 RRC 消息（*MasterInformationBlock*）里所带的信息外，它还有额外一些信息要告诉 UE。本节所要介绍的，就是小区会通过 MIB 告诉 UE 哪些信息，以及 UE 是如何检测到这些信息的。

UE 通过检测 PBCH，能得到以下信息：

（1）通过接收到的 *MasterInformationBlock* 可以知道小区的**下行系统带宽**、**PHICH 配置**（详见 4.3 节关于“PHICH”的介绍）、系统帧号（System Frame Number，SFN。更确切地说，获取到的是 **SFN 的高 8 位**，最低 2 位需要在 PBCH 盲检时得到，这会在后面介绍）。

（2）**小区特定的天线端口（cell-specific antenna port）的数目**：1 或 2 或 4。

（3）**用于 L1/L2 控制信令（包括 PCFICH、PHICH、PDCCH）的传输分集模式（transmit-diversity scheme）**：PBCH 和 L1/L2 控制信令都只能使用单天线传输或传输分集，如果使用传输分集， PBCH 和 L1/L2 控制信令会使用相同的多天线传输分集模式。

从图 20-2 可以看出，MIB 会在物理信道 PBCH 上传输。PBCH 时域上位于每个系统帧的子帧 0 的第 2 个 slot 的前 4 个 OFDM 符号上，并在频域上占据 72 个中心子载波（不含 DC）。对应 RE 不能用于发送 DL-SCH 数据。如图 20-5 所示。

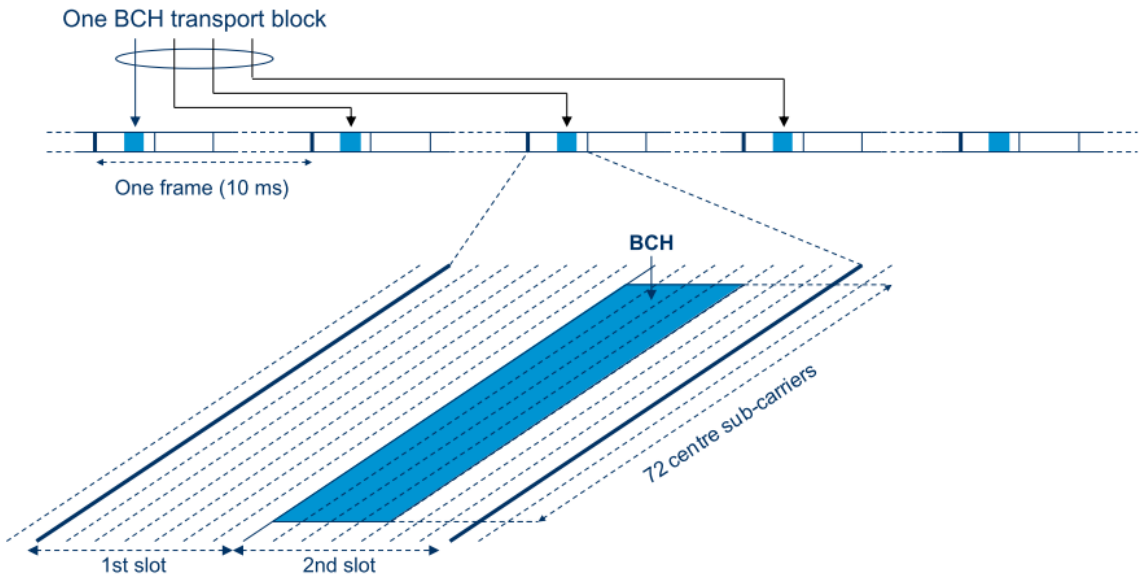


图 20-5：BCH 传输信道的资源映射

把 BCH 传输限制在 72 个中心的子载波，而不考虑小区的下行系统带宽的原因在于：UE 在接收 BCH 时并不知道小区的下行系统带宽。因此 UE 在第一次接收 BCH 时，可以假定小区的下行系统带宽等于可能的最小下行系统带宽（6 个 RB，对应 72 个子载波）。在解码了 MIB 后，UE 就能从 MIB 中得到实际的下行系统带宽。

MIB 在 40ms 周期内重复 4 次，每一次发送的 PBCH 都携带相同的 coded bit，也就是说，每一次都是可以独自解码的。因此，在信道质量（SIR）足够好的情况下，UE 可能只接收这 40ms 内的其中一个，就能够成功解码出 PBCH 的内容；如果不行，就与下一个 10ms 发送的 PBCH 的内容进行软合并，再进行解码，直到成功解码出 PBCH。

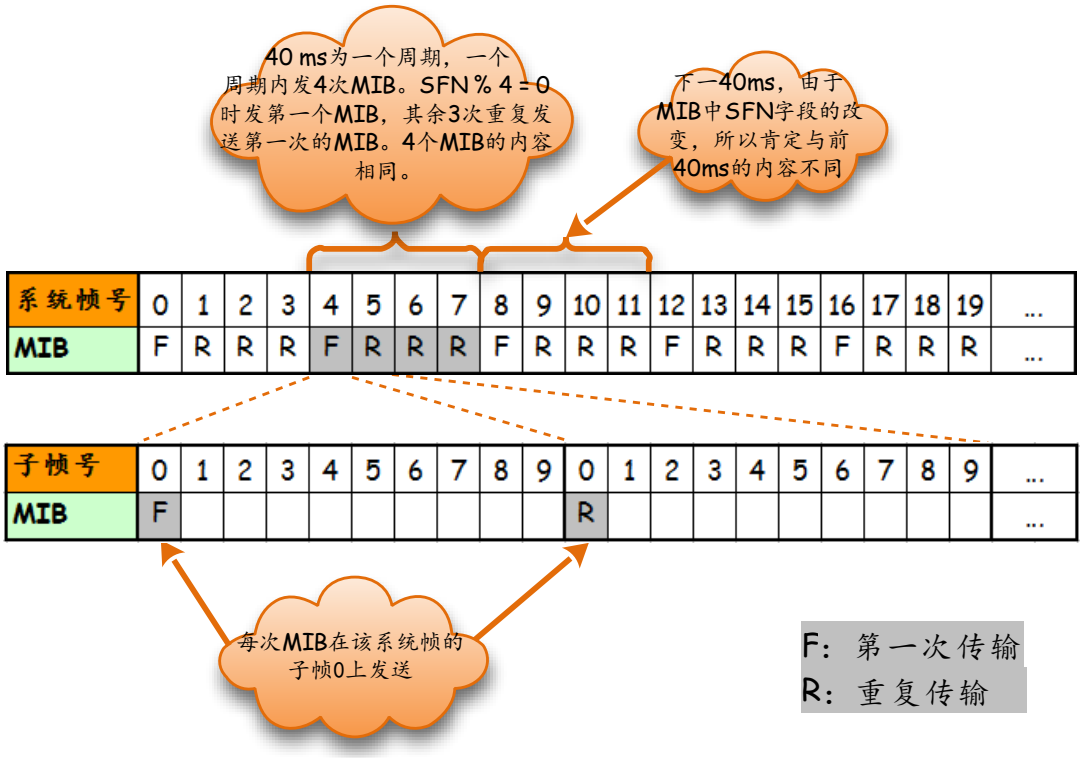


图 20-6：MIB 在时域上的调度

前面已经说过，通过 MIB，UE 只能获取到 SFN 的高 8 位，最低 2 位（也就是 40ms timing）是通过盲检 PBCH 得到的。40ms 内每次发送的 PBCH 会使用不同 scrambling and bit position（即共有 4 个不同的 phase of the PBCH scrambling code），并且每 40ms 会重置一次。

UE 可以通过使用 **4** 个可能的 phase of the PBCH scrambling code 中的每一个去尝试解码 PBCH，如果解码成功，也就知道了小区是在 40ms 内的第几个系统帧发送 MIB，即知道了 SFN 的最低 2 位。（参考资料[5]和[6]中介绍了检测 SFN 最低 2 位的几种策略，有兴趣的可以了解一下）

PBCH 的多天线传输只能使用传输分集，而且在 2 天线端口传输时，只能使用 SFBC；4 天线端口传输时，只能使用 combined SFBC/FSTD。UE 使用 **3** 种不同的 CRC mask（见 36.212 的 Table

5.3.1.1-1）来盲检 PBCH，可得到天线端口数目，而天线端口数目与传输分集模式一一对应（1 天线端口 对应单天线端口传输；2 天线端口 对应 SFBC；4 天线端口对应 combined SFBC/FSTD），因此当 UE 成功解码 PBCH 时，就知道了小区特定的天线端口数以及用于 L1/L2 控制信令的传输分集模式。（关于 SFBC、FSTD 的说明，详见参考资料[4]的 5.4.1.4 节和 10.3.1.2 节）

Table 5.3.1.1-1: CRC mask for PBCH.

Number of transmit antenna ports at eNodeB	PBCH CRC mask $< x_{ant,0}, x_{ant,1}, \ldots, x_{ant,15} >$
1	<0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0>
2	<1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1>
4	<0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1>

PBCH 有 3 种天线端口组合（1/2/4）和 4 种不同扰码（phase）组合，所以做盲检 PBCH 最多有 12 种可能组合。

图 20-7 给出了 MIB 在物理层的编码处理和映射流程。

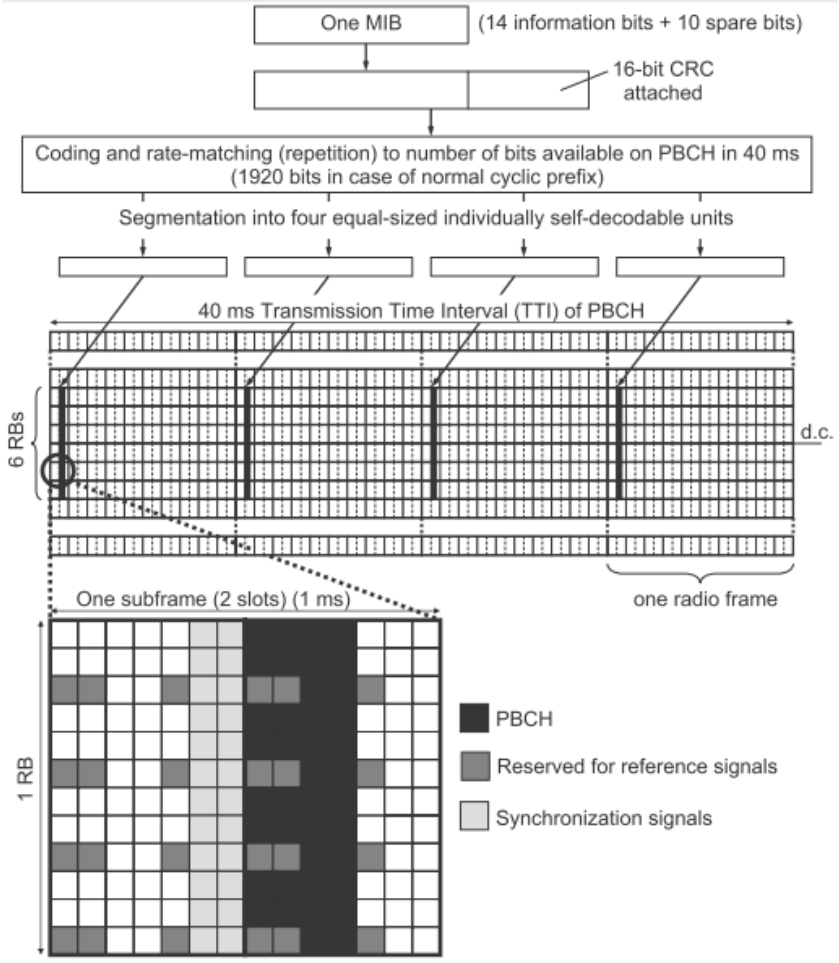


图 20-7：PBCH 结构

20.2 SIB

在 BCH 中传输的 MIB 只包含非常有限的系统信息。而更多的系统信息是通过在 DL-SCH 中传输的 SIB 来告诉 UE 的。

对于各类 SIB 而言，RRC 消息里所带的信息就是其要告诉 UE 的所有信息。各类 SIB 的内容可以参见 36.331 中对应的 RRC 消息。

与 MIB 所在的时频位置固定不同，SIB1 和 SI 消息都在 PDSCH 上传输，且 SIB1 和 SI 消息所占的 RB（频域上的位置）及其传输格式等是动态调度的，并由 SI-RNTI 加扰的 PDCCH 来指示。也就是说，UE 需要先在子帧上盲检使用 SI-RNTI 加扰的 PDCCH，才能知道该子帧是否存在 SI 消息（SIB1 在时域上的位置固定）。小区可以根据具体情况灵活地改变 SIB1 和 SI 所占的带宽、使用的 RB 集合以及传输格式等。

SIB1 的周期为 80ms，且在该周期内 SFN % 2 = 0 的系统帧的子帧 5 上重复发送同一 SIB1。SIB1 在时域上的位置是固定的，但在频域上的位置可能发生变化，并由对应的 PDCCH 来指示。

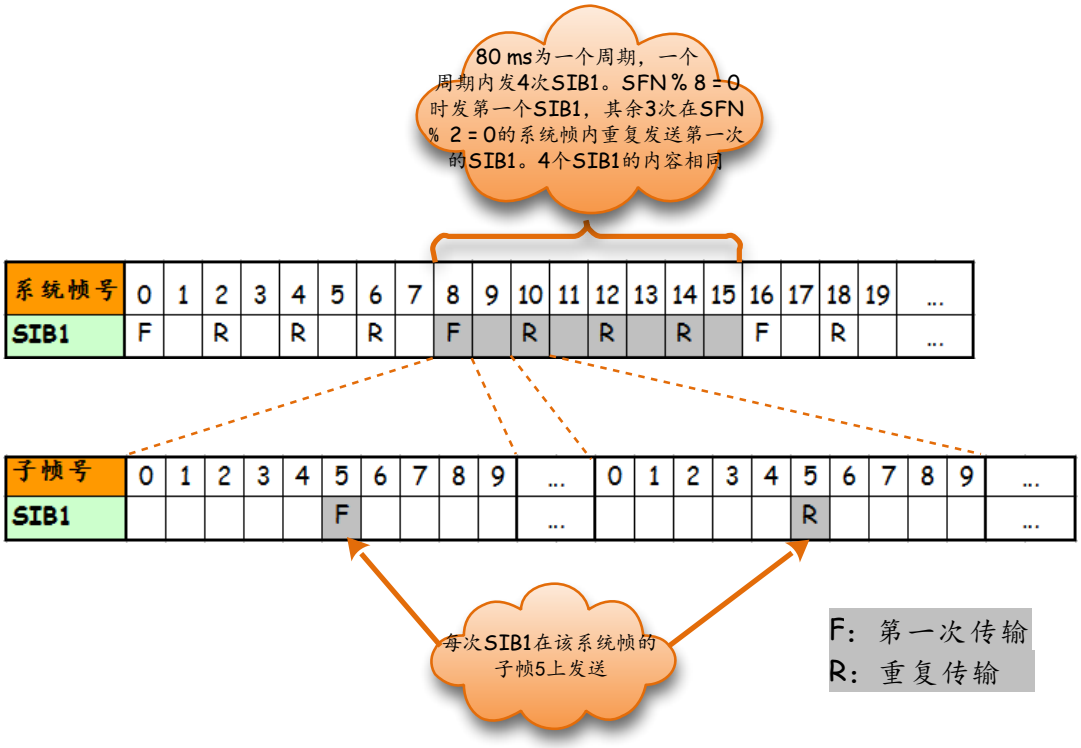


图 20-8：SIB1 在时域上的调度

每个 SI 消息包含了一个或多个除 SIB1 外的拥有相同调度需求的 SIB（这些 SIB 有相同的传输周期）。例如：2 个传输周期都是 160ms 的 SIB（如图 20-10 中的 SIB2 和 SIB3）可以放在同一个 SI 消息中，但传输周期为 320ms 的其它 SIB（如图 20-10 中的 SIB4 和 SIB5）必须放在不同的 SI 消息中。

单个 SI 消息中包含的信息比特不能超过一个 TB 块所能传输的最大比特数。

物理层限制了 1 个 SIB（个人觉得更好的描述是 SI 和 SIB1）的最大 size。如果使用 DCI format 1C，则最大 size 为 1736 比特（217 byte）；如果使用 DCI format 1A，则最大 size 为 2216 比特（277 byte）。

一个 SI 消息包含哪些 SIB 是通过 SIB1 中的 *schedulingInfoList* 指定的。每个 SIB 只能包含在一个 SI 消息中，且 SIB2 总是放在 *schedulingInfoList* 指定的 SI 列表的第一个 SI 消息项中，所以 *schedulingInfoList* 中并不指定 SIB2 所在的 SI。

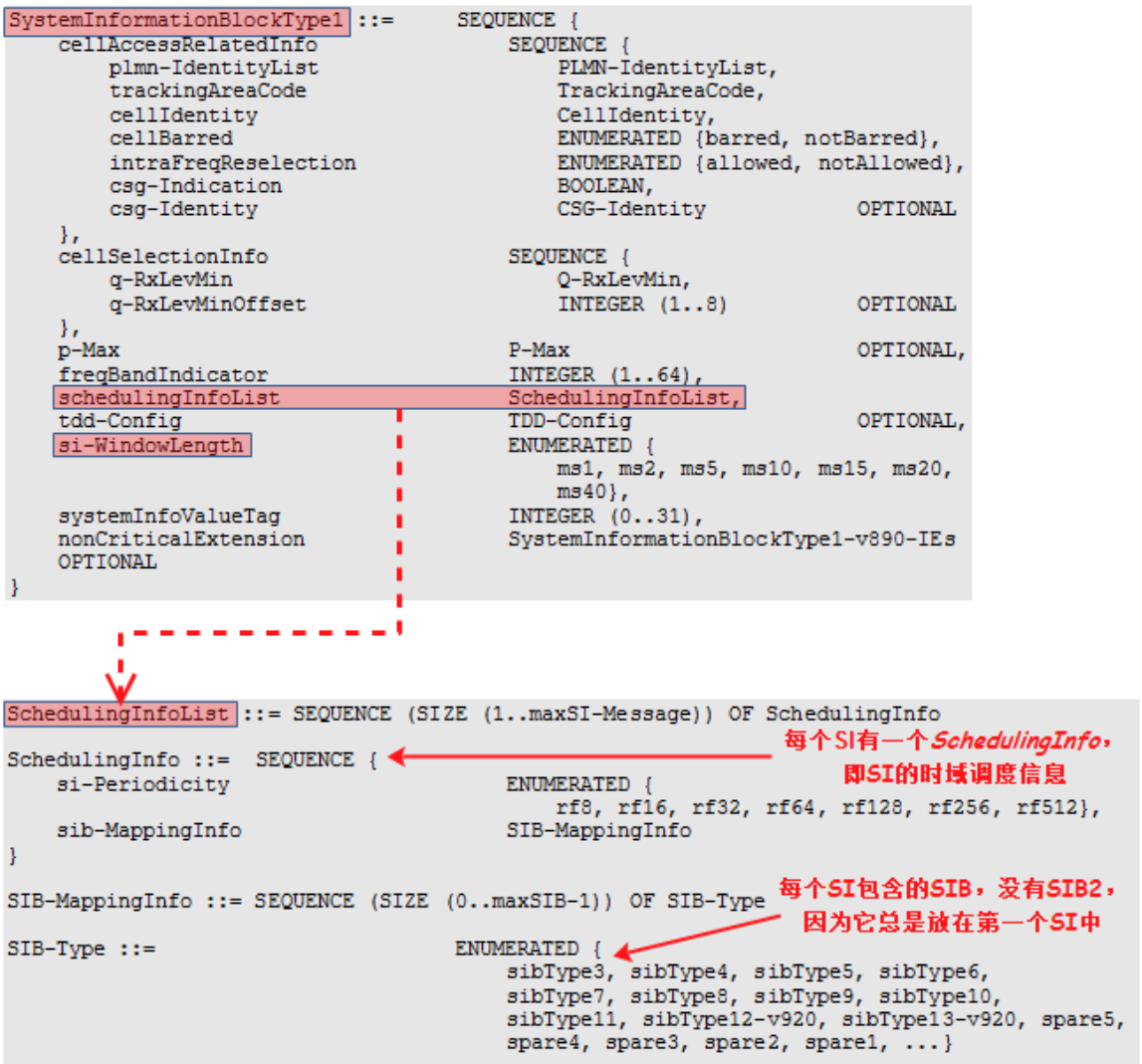


图 20-9：SIB1 信息（包含了 SI 的调度信息）

每个 SI 消息只在一个 SI 窗口（SI-windows）中传输：（1）一个 SI 消息跟一个 SI 窗口相关联，该 SI 窗口内只能发这个 SI 消息且可以重复发送多次（发多少次，在哪些子帧上发送等，取决于 eNodeB 的实现），但不能发送其它 SI 消息；（2）SI 窗口之间是紧挨着的（如果相邻的话），既不重叠，也不会有空隙；（3）所有 SI 消息的 SI 窗口长度都相同；（4）不同 SI 消息的周期是相互独立的。

前面我们已经介绍过 MIB 和 SIB1 的时域调度，接下来我们会详细介绍 SI 消息的时域调度。

首先需要确认每个 SI 消息对应的 SI 窗口的起始位置以及 SI 窗口的长度。

SI 窗口的长度由 *SystemInformationBlockType1* 的 *si-WindowLength* 字段指定，其以 ms 为单位。

SystemInformationBlockType1 的 *schedulingInfoList* 指定了 SI 消息的列表，每个 SI 消息在该列表中的顺序以 n 表示（从 1 开始）。假如 *schedulingInfoList* 中指定了 4 个 SI 消息，则会有 4 个连续的 SI 窗口用于发送这 4 个 SI 消息，而 n 表明了 SI 消息在第几个 SI 窗口。

此时每个 SI 消息有一个 x = (n - 1) * w，其中 w 为 *si-WindowLength*。可以看出，x 是以 ms 为单位的。

则 SI 窗口的起始帧满足 SFN % T = FLOOR(x / 10)，其中 T 为对应 SI 消息的周期，由 *si-Periodicity* 指定（其以系统帧为单位，即以 10ms 为单位）。SFN % T 保证了 SI 的周期，FLOOR(x / 10)确定 SI 窗口在周期内的起始系统帧（一个系统帧为 10ms，所以有 x / 10）。

SI 窗口的起始子帧为#a，其中 a = x % 10。

从公式可以看出，x 决定了 SI 窗口在该 SI 周期内的起始帧和起始子帧； SFN % T 保证了 SI 窗口在 SI 周期内只出现一次；而 x = (n - 1) * w 保证了 SI 窗口之间紧挨（如果相邻的话），不重叠，没有空隙。（SI 窗口起始帧和起始子帧的的计算，详见 36.331 的 5.2.3 节）

SI 窗口确定了以后，eNodeB 会决定在该窗口内调度多少次同一 SI，不同厂商的实现可能不同。但某些子帧不能用于调度 SI 消息：

- SFN % 2 = 0 的系统帧内的子帧 5，该子帧用于 SIB1 的传输；
- 任一 MBSFN 子帧；
- TDD 中的上行子帧。

图 20-10 是一个关于 SI 调度的例子。

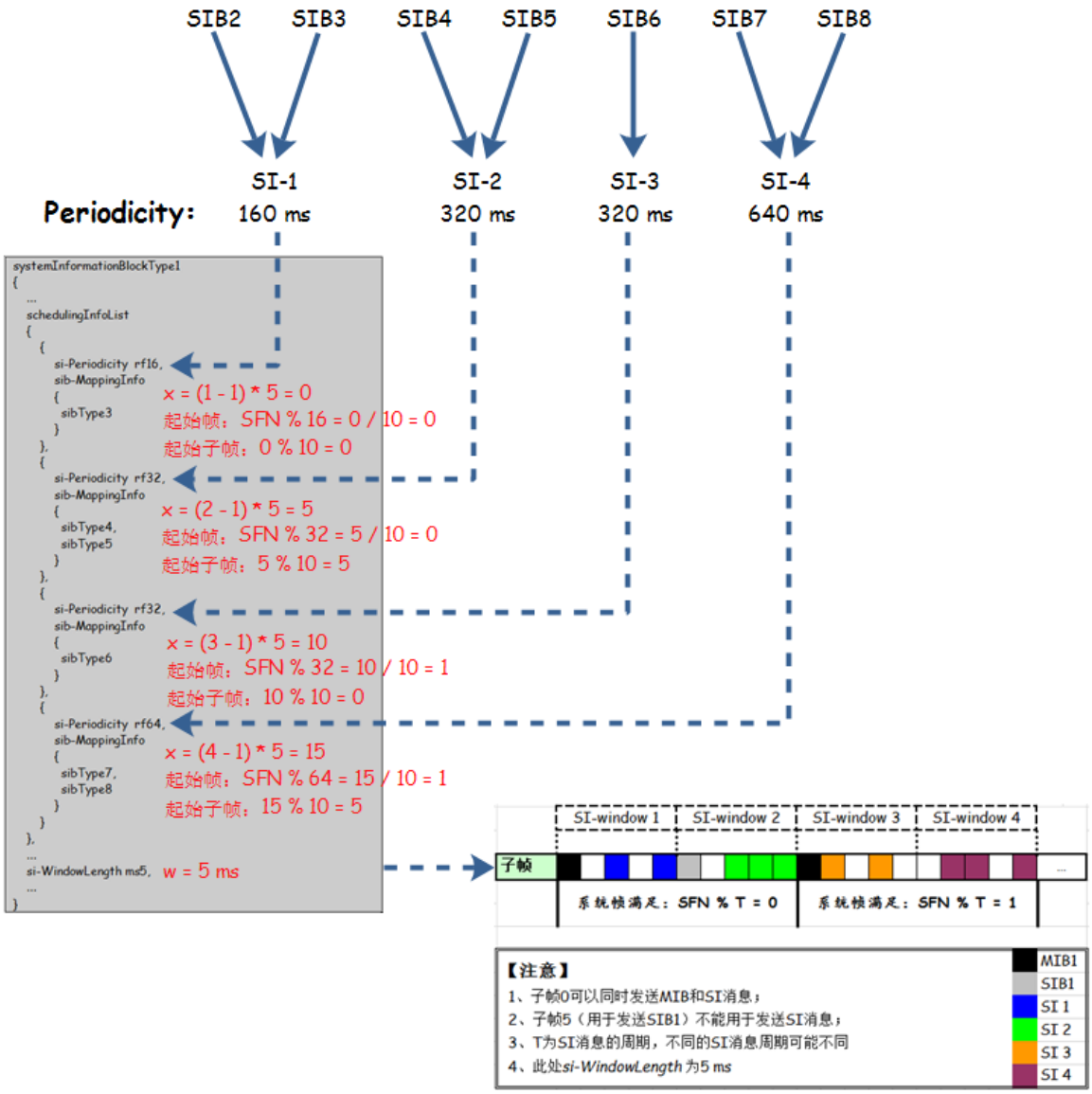


图 20-10：SI 调度的一个例子

可以看出，SI 不需要在时间窗内的连续子帧上传输。并且某个子帧上是否存在 SI 消息，是通过 SI-RNTI 加扰的 PDCCH 来指示的。

这里计算的是 SI 窗口的起始帧和起始子帧，并不是计算 SI 消息具体所在的系统帧和子帧。SI 消息是在 SI 窗口内发送的，UE 需要在 SI 窗口内盲检使用 SI-RNTI 加扰的 PDCCH，才能知道对应子帧上是否有 SI 消息。

在 SI 较小而下行系统带宽较大的情况下，一个子帧可能足以发送该 SI，但在其它情况下，可能需要使用多个子帧来发送一个 SI 消息。在后一种情况，会将整个 SI 消息进行信道编码后分成多份，然后放在多个子帧（不要求是连续子帧）上传输。而不是先分割成多份，然后独立地信道编码后传输。

注意：对于 SIB1 和 SI 消息，其调制阶数 Q_m 固定为 2。（见 36.213 的 7.1.7.1 节）

小结：MIB 和 SIB1 在时域上的位置和周期是固定的，而 SI 消息在时域上的位置和周期是由 SIB1 指定的。eNodeB 只会通过 *SystemInformationBlockType1* 告诉 UE 有哪些 SI，每个 SI 包含了哪些 SIB，这些 SI 会在哪个 SI 窗口发送以及 SI 窗口的时域位置和长度，但不会告诉 UE 在 SI 窗口的哪些子帧调度了该 SI。当 UE 需要某个 SIB 时，它就会在该 SIB 对应的 SI 消息对应的 SI 窗口的每个子帧（从 SI 窗口的起始子帧开始，共持续 *si-WindowLength* 个子帧，但不包含那些不能调度 SI 的子帧），使用 SI-RNTI 去尝试解码，直到成功接收到 SI 消息为止。

20.3 系统信息有效性和变更通知

系统信息（除了 ETWS 和 CMAS，对应 SIB10、SIB11 和 SIB12）的变更只发生在某些特定的系统帧，这里用到了变更周期的概念。

变更周期（modification period）的起始系统帧必须满足 SFN mod $m = 0$，其中 m 是组成一个变更周期的系统帧数。一个变更周期包含 $m = modificationPeriodCoeff * defaultPagingCycle$ 个系统帧。这是通过 SIB2 来设置的。

系统信息可能会在一个变更周期内传输多次，但在同一变更周期内，系统信息的内容不会发生变化。

当小区修改了某些系统信息时，它会先在一个变更周期内通知 UE 系统信息将发生变化（但并不发送更新后的内容），然后在紧接着的下一个变更周期，小区才会发送更新后的系统信息。如 36.331 的 Figure 5.2.1.3-1 所示（不同的颜色表示不同的系统信息）。

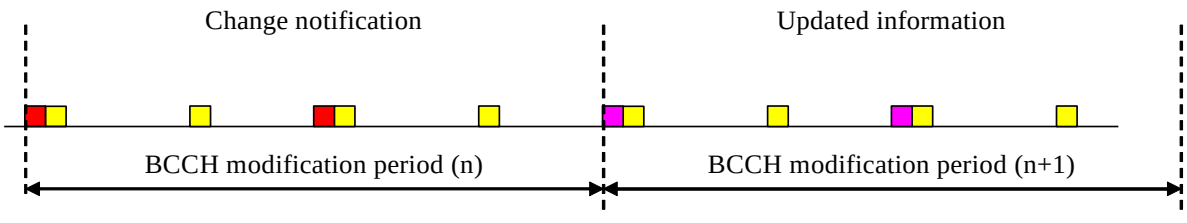


Figure 5.2.1.3-1: Change of system Information

UE 收到了一个系统信息变更通知（change notification）后，会从下一个变更周期的开始处就去接收新的系统信息。UE 在收到新的系统信息之前，会继续使用旧的系统信息。如果一个重要的参数发生改变，可能会对通信造成严重影响，但由于任何服务的中断都很短且不会频繁发生，所以认为结果是可以接受的。

小区有 2 种方式通知 UE 系统信息发生了变化：

- Paging 消息包含了一个 *systemInfoModification* 字段，用于指示 SI 是否发生了变化；
- SIB1 中包含了一个 *systemInfoValueTag* 字段，每当 SI 消息发生变化时，*systemInfoValueTag* 的值会加 1。

小区可以使用 Paging 消息来通知处于 RRC_IDLE 态和 RRC_CONNECTED 态的 UE 系统信息发生了变化。Paging 消息中包含了一个 *systemInfoModification* 字段，如果 UE 接收到的 Paging 消息包含该字段，则表示系统信息（除 SIB10、SIB11 和 SIB12 外）将在下一个变更周期发生变化。

SIB1（*SystemInformationBlockType1*）中包含了一个字段 *systemInfoValueTag*（取值范围 0~31），用于指示 SI 消息是否发生了变化。UE 可以使用这个字段来检验之前保存的 SI 消息是否依然有效（例如从小区覆盖之外回到小区覆盖的范围内）。如果该字段发生了变化，则 UE 认为所保存的系统信息是无效的，需要重新读取；否则认为保存的系统信息依然有效。另外，UE 会认为从接收到 SI 消息那一刻算起的 3 个小时之内，如果 *systemInfoValueTag* 未变化，则所保存的系统信息是有效的。即保存的 SI 消息的有效期为 3 个小时。

systemInfoValueTag 是除 MIB，SIB1，SIB10，SIB11 和 SIB12 之外的所有 SIB 所共用的。MIB 和 SIB1 的改变是通过 UE 获取这些消息来检测的。

某些系统信息，如 ETWS 信息、CMAS 信息以及类似 CDMA2000 系统时间的参数等发生变化时，小区是不会更新 SIB1 中的 *systemInfoValueTag* 值的，Paging 消息中也不会包含 *systemInfoModification* 字段。

UE 可以通过以下 2 种方式来确定保存的系统信息是否依然有效：

- 校验 SIB1 中的 *systemInfoValueTag* 字段是否发生变化；或
- 在每个变更周期，如果 UE 在变更周期内没有收到 Paging，则会尝试寻找 *systemInfoModification* 至少 *modificationPeriodCoeff* 次。如果 UE 在一个变更周期内没有收到 Paging 消息，则 UE 认为在接下来的一个变更周期范围不会发生系统信息的改变。如果处于 RRC_CONNECTED 态的 UE 在一个变更周期内收到了一个 Paging 消息，则 UE 会通过是否存在 *systemInfoModification* 来推断接下来的一个变更周期内，系统信息（除了 ETWS 和 CMAS 信息）是否会发生变化。

小区只会通知 UE 系统信息发生了变化，并不会告诉 UE 哪些系统信息发生了变化。所以 UE 收到了系统信息发生变化的指示后，会根据 SIB1 中的调度信息去重新接收所有 SI 消息（除 SIB10、SIB11、SIB12 外）。

对于处于 RRC_CONNECTED 态且支持 ETWS 和/或 CMAS 的 UE 而言，应该在每 *defaultPagingCycle* 个系统帧的时间内至少读取 Paging 消息一次，以确定是否存在 ETWS 和/或 CMAS 通知。

Paging 消息的 *etws-Indication* 字段用于通知 UE 开始接收 ETWS primary 通知（对应 SIB10）和/或 ETWS secondary 通知（对应 SIB11）。

Paging 消息的 *cmas-Indication* 字段用于通知 UE 开始接收 CMAS 通知（对应 SIB12）。

20.4 载波聚合对系统信息的影响

UE 只会在 PCell 上从广播消息中获取系统信息。对于 SCell 而言，其系统信息是在添加 SCell 时，通过 *RRCConnectionReconfiguration* 的 *SCellToAddMod-r10* 下发给 UE 的。如果某个 SCell 的系统信息发生改变，eNodeB 会让 UE 先释放该 SCell，然后重新添加该 SCell 以通知 UE 系统信息的变化。这从 36.331 中的 5.3.10.3b 中可以看出，在添加 SCell（SCell addition）时，是应用 *radioResourceConfigCommonSCell* 的配置的，但在修改 SCell（SCell modification）的信息时，是不应用 *radioResourceConfigCommonSCell* 的配置的，所以需要先删除再添加 SCell 以通知 UE 该 SCell 系统信息的变化。

注意：eNodeB 在 *RRCConnectionReconfiguration* 配置的系统信息可以与该 SCell 广播的系统信息不同。

【参考资料】

[1] TS 36.211 的 6.6 节 Physical broadcast channel

[2] TS 36.212 的 5.3.1 节 Broadcast channel

[3] TS 36.331 的 5.2 节 System Information

[4] 《4G LTE/LTE-Advanced for Mobile Broadband》的 14.2 节

[5] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 3.2.2 节和 9.2.1 节

[6] 《PBCH: How quickly can a UE read the MIB?》

[7] 《PBCH: Does the MIB tell the UE how many antennas are used in the cell?》

[8] 《SI 的调度》

[9] 《LTE 系统中 UE 接收系统消息解析》

第21章 随机接入过程（Random Access Procedure）

在小区搜索过程之后，UE 已经与小区取得了下行同步，因此 UE 能够接收下行数据。但 UE 只有与小区取得上行同步，才能进行上行传输。UE 通过随机接入过程（Random Access Procedure）与小区建立连接并取得上行同步。

随机接入的主要目的：（1）获得上行同步；（2）为 UE 分配一个唯一的标识 C-RNTI。

随机接入过程通常由以下 6 类事件之一触发：(见 36.300 的 10.1.5 节)

1. 初始接入时建立无线连接：UE 会从 RRC_IDLE 态到 RRC_CONNECTED 态；
2. RRC 连接重建过程（RRC Connection Re-establishment procedure）：以便 UE 在无线链路失败（Radio Link Failure）后重建无线连接。
3. 切换（handover）：此时 UE 需要与新的小区建立上行同步。
4. RRC_CONNECTED 态下，下行数据到达（此时需要回复 ACK/NACK）时，上行处于“不同步”状态；
5. RRC_CONNECTED 态下，上行数据到达（例：需要上报测量报告或发送用户数据）时，上行处于“不同步”状态或没有可用的 PUCCH 资源用于 SR 传输（此时允许已经处于上行同步状态的 UE 使用 RACH 来替代 SR 的作用）；
6. RRC_CONNECTED 态下，为了定位 UE，需要 timing advance。

随机接入过程还有一个特殊的用途：如果 PUCCH 上没有配置专用的 SR 资源时，随机接入还可作为一个 SR 来使用。

随机接入过程有 2 种不同的方式：

1. 基于竞争（Contention based）：应用于之前介绍的前 5 种事件；
2. 基于非竞争（Non-Contention based 或 Contention-Free based）：只应用于之前介绍的 (3)、(4) 、(6)三种事件。

21.1 preamble

随机接入过程的步骤一就是 UE 发送 random access preamble。Preamble 的主要作用是告诉 eNodeB 有一个随机接入请求，并使得 eNodeB 能估计其与 UE 之间的传输时延，以便 eNodeB 校准 uplink timing 并将校准信息通过 timing advance command 告知 UE。

Preamble 在 PRACH 上传输。eNodeB 会通过广播系统信息 SIB2 来通知所有的 UE，允许在哪些时频资源上传输 preamble。（由 *prach-ConfigIndex* 和 *prach-FreqOffset* 字段决定，详见 36.211 的 5.7 节）

每个小区有 64 个可用的 preamble 序列，UE 会选择其中一个（或由 eNodeB 指定）在 PRACH 上传输。这些序列可以分成两部分，一部分用于基于竞争的随机接入，另一部分用于基于非竞争的随机接入。用于基于竞争的随机接入的 preamble 序列又可分为两组：group A 和 group B（group B 可能不存在）。eNodeB 是通过 *RACH-ConfigCommon*（SIB2）来发送这些配置的。如图 21-1 所示。

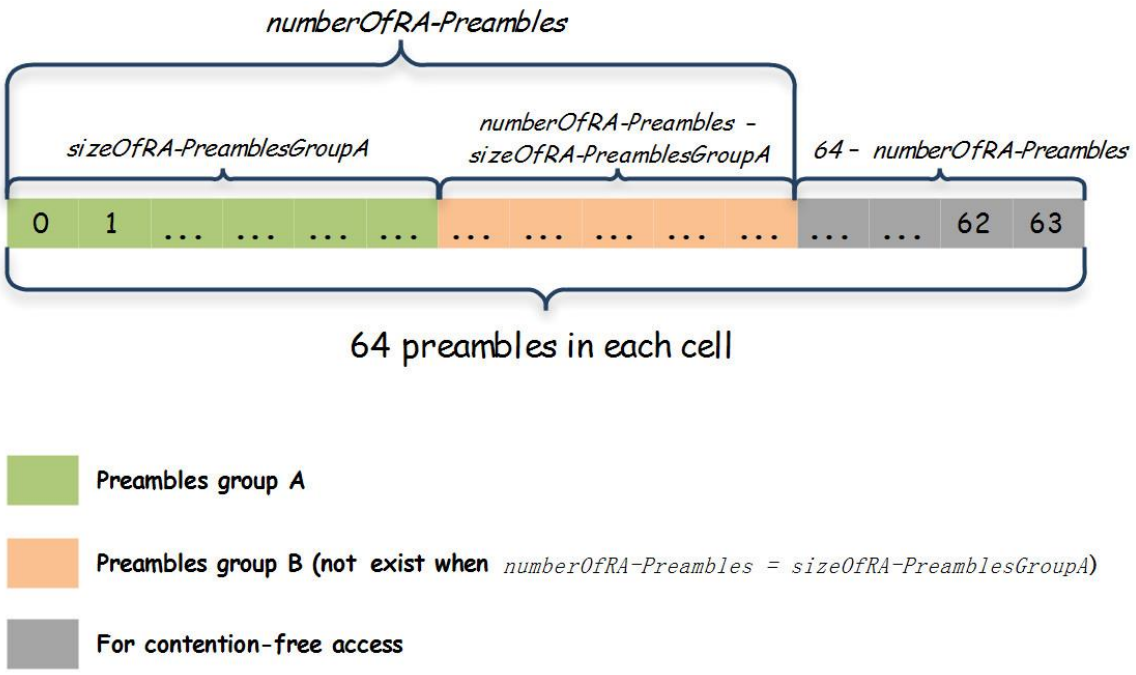


图 21-1：random access preambles

将用于基于竞争的随机接入的 preamble 序列分成 2 组 group A 和 group B 的目的是为了加入一定的先验信息，以便 eNodeB 在 RAR 中给 Msg3 分配适当的上行资源。如果 UE 接入时估计后续的 Msg3 可能比较大（大于 *messageSizeGroupA*），并且路径损耗 pathloss 小于 $P_{CMAX,c}$ – *preambleInitialReceivedTargetPower – deltaPreambleMsg3 – messagePowerOffsetGroupB*，则使用 group B 中的 preamble；否则使用 group A 中的 preamble。这样 eNodeB 就能够根据收到的 preamble 知道该 preamble 所属的 group，从而了解 Msg3 的大致资源需求。如果不分组，就应采用较高的 UL grant 配置，可能损失一些上行效率。（关于 preamble 的选择：详见 36.321 的 5.1.2 节）

Group A/B 中的 preamble 序列本身并没有太大区别，只有它们的划分才是有意义的，用于告诉 eNodeB 后续的资源需求。

如果 UE 进行的是基于非竞争的随机接入，例如 handover，使用的 preamble 是由 eNodeB 直接指定的（见 36.331 的 *RACH-ConfigDedicated*，以及 36.212 的 5.3.3.1.3 节中关于 PDCCH order 的介绍）。为了避免冲突，此时使用的 preamble 是除 group A 和 group B 外的预留 preamble。

简单地说：eNodeB 通过广播 SIB2 发送 *RACH-ConfigCommon*，告诉 UE preamble 的分组、Msg3 大小的阈值、功率配置等信息。UE 发起随机接入时，根据可能的 Msg3 大小以及 pathloss 等，选择合适的 preamble。

21.2 PRACH 资源

在 LTE 中，提到信道的资源时，通常都会涉及时域（system frame、subframe、slot、symbol、周期等）、频域（起始 RB、所占的 RB 数，是否跳频等）、码分复用（正交覆盖码 orthogonal cover code、循环移位 cyclic shift）等。

PRACH 用于传输 random access preamble。

某小区可用的 PRACH 时频资源是由 SIB2 的 *prach-ConfigIndex* 和 *prach-FrequencyOffset* 字段决定的。一旦这两个字段确定了，对接入该小区的所有 UE 而言，preamble 的格式（format）和可选的 PRACH 资源就固定了。

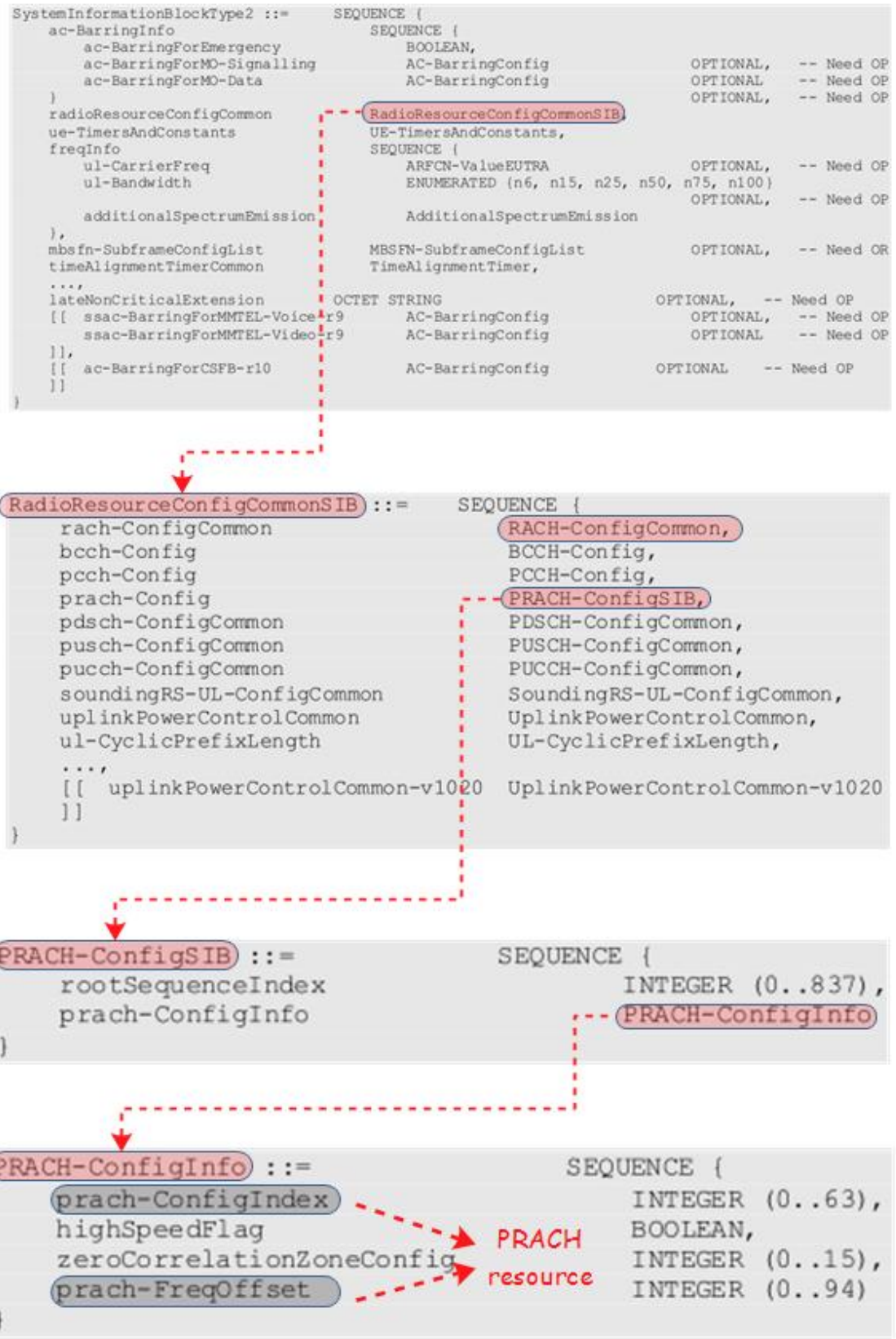

SystemInformationBlockType2 ::= SEQUENCE {
ac-BarringInfo SEQUENCE {
ac-BarringForEmergency BOOLEAN,
ac-BarringForMO-Signalling AC-BarringConfig OPTIONAL, -- Need OP
ac-BarringForMO-Data AC-BarringConfig OPTIONAL -- Need OP
} OPTIONAL, -- Need OP
radioResourceConfigCommon RadioResourceConfigCommonSIB,
ue-TimersAndConstants UE-TimersAndConstants,
freqInfo SEQUENCE {
ul-CarrierFreq ARFCN-ValueEUTRA OPTIONAL, -- Need OP
ul-Bandwidth ENUMERATED {n6, n15, n25, n50, n75, n100}
OPTIONAL, -- Need OP
additionalSpectrumEmission AdditionalSpectrumEmission
},
mbsfn-SubframeConfigList MBSFN-SubframeConfigList OPTIONAL, -- Need OR
timeAlignmentTimerCommon TimeAlignmentTimer,
...,
lateNonCriticalExtension OCTET STRING OPTIONAL, -- Need OP
[[ssac-BarringForMMTEL-Voice-r9 AC-BarringConfig OPTIONAL, -- Need OP
ssac-BarringForMMTEL-Video-r9 AC-BarringConfig OPTIONAL -- Need OP
]],
[[ac-BarringForCSFB-r10 AC-BarringConfig OPTIONAL -- Need OP
]]
}
RadioResourceConfigCommonSIB ::= SEQUENCE {
rach-ConfigCommon RACH-ConfigCommon,
bcch-Config BCCH-Config,
pcch-Config PCCH-Config,
prach-Config PRACH-ConfigSIB,
pdsch-ConfigCommon PDSCH-ConfigCommon,
pusch-ConfigCommon PUSCH-ConfigCommon,
pucch-ConfigCommon PUCCH-ConfigCommon,
soundingRS-UL-ConfigCommon SoundingRS-UL-ConfigCommon,
uplinkPowerControlCommon UplinkPowerControlCommon,
ul-CyclicPrefixLength UL-CyclicPrefixLength,
...,
[[uplinkPowerControlCommon-v1020 UplinkPowerControlCommon-v1020
]]
}
PRACH-ConfigSIB ::= SEQUENCE {
rootSequenceIndex INTEGER (0..837),
prach-ConfigInfo PRACH-ConfigInfo
}
PRACH-ConfigInfo ::= SEQUENCE {
prach-ConfigIndex INTEGER (0..63),
highSpeedFlag BOOLEAN,
zeroCorrelationZoneConfig INTEGER (0..15),
prach-FreqOffset INTEGER (0..94)
}
PRACH
resource

图 21-2：指定 PRACH 时频资源的 RRC 信令

每个 PRACH 资源在频域上占用 6 个连续 RB 的带宽，这正好等于 LTE 支持的最小上行带宽。因此，不管小区的传输带宽有多大，都可以使用相同的 RA preamble 结构。

虽然协议中明确规定，并不禁止 eNodeB 在预留给 PRACH 的 RB 上调度其它上行数据（36.213 的 6.1 节），但为了 PRACH 解码的可靠性， eNodeB 通常不会在预留给 PRACH 的 RB 上调度其它上行数据。

小结：PRACH 在频域上占 6 个连续的 RB。

preamble 由 2 部分组成：循环前缀（CP）和 preamble 序列（如 36.211 的 Figure 5.7.1-1 所示）。其中循环前缀的长度用 T_{CP} 表示，preamble 序列的长度用 T_{SEQ} 表示。

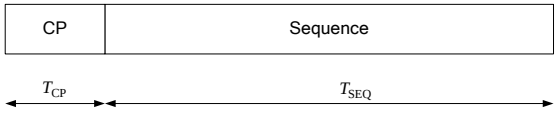


Figure 5.7.1-1: Random access preamble format.

preamble 在时域上的长度取决于配置。（如下表所示，见 36.211 的 5.7.1 节）

Table 5.7.1-1: Random access preamble parameters.

Preamble format	T_{CP}	T_{SEQ}
0（1 subframe）	$3168 \cdot T_s$	$24576 \cdot T_s$
1（2 subframes）	$21024 \cdot T_s$	$24576 \cdot T_s$
2（2 subframes）	$6240 \cdot T_s$	$2 \cdot 24576 \cdot T_s$
3（3 subframes）	$21024 \cdot T_s$	$2 \cdot 24576 \cdot T_s$
4*（for TDD only）	$448 \cdot T_s$	$4096 \cdot T_s$

* Frame structure type 2 and special subframe configurations with UpPTS lengths $4384 \cdot T_s$ and $5120 \cdot T_s$ only.

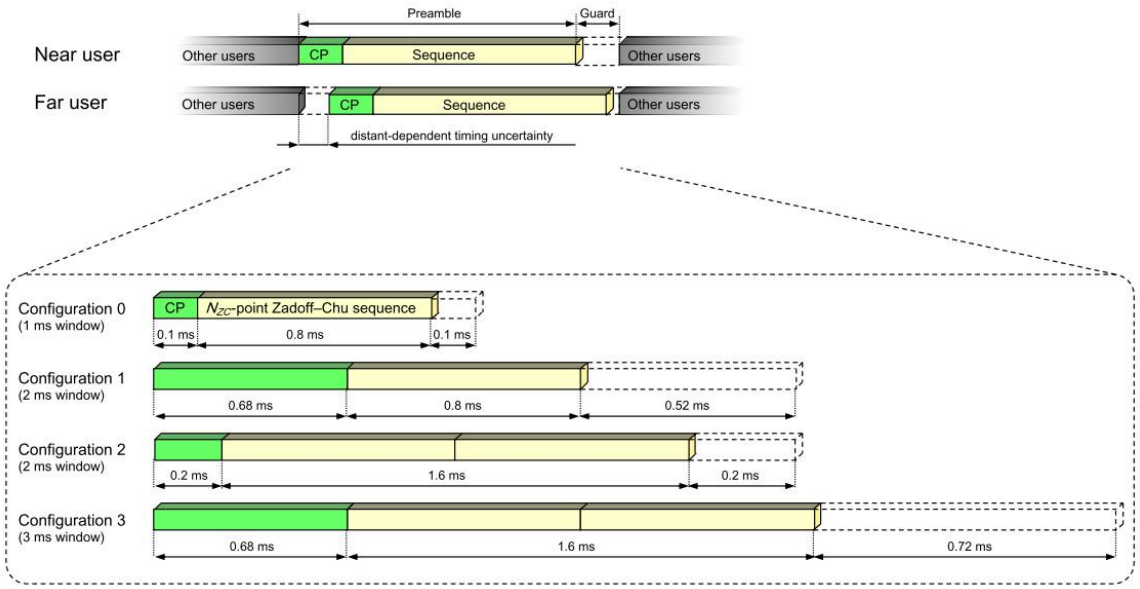


图 21-3：不同的 preamble 格式

从图 21-3 可以看出，不同格式的 preamble 在时域上所占的连续子帧数是不一样的，format 0 占 1 个子帧，format 1 和 format 2 占 2 个子帧，format 3 占 3 个子帧。

不同的 preamble 可能有不同循环前缀（Cyclic Prefix，CP）。循环前缀越大，对延迟的容忍度就越大。

在随机接入之前，UE 的上行同步还未建立，并且 UE 在小区中的位置并不确定，所以此时上行 timing 是不确定的。上行 timing 的不确定性正比于小区半径，每 1 km 有大约 6.7μs 的传输延迟（6.7μs / km）。为了应对这种不确定性，并且为了避免对接下来的未用于随机接入的子帧产生干扰，preamble 的传输中使用了一个保护时间（Guard Time，简称为 GT）。从 36.211 的 Table 5.7.1-1 可以看出，$T_{\mathrm{CP}}+T_{\mathrm{SEQ}}$ 的值是小于 preamble 占据的连续子帧的大小，例如对于 preamble format 0，其 $T_{\mathrm{CP}}+T_{\mathrm{SEQ}}=27744\cdot T_s < 30720\cdot T_{\mathrm{s}}=1ms$。而多出来的时间就作为保护时间 GT，例如对于 preamble format 0，其保护时间 GT 为 $30720\cdot T_{\mathrm{s}}-27744\cdot T_s=2976\cdot T_s$。

小区的半径正比于保护时间 GT 的大小，GT 越大，小区半径越大。preamble format 0 可以支持大约 14.53 km 的小区半径，如果需要支持更大的小区，就必须使用 preamble format 1~3。小区半径的具体计算方式（还与 preamble 序列的循环移位有关）可见参考资料[7]的介绍。

对于 TDD，还支持额外的 preamble 格式：format 4。该配置只用于特殊子帧的 UpPTS 字段，且只支持长度为 $4384\cdot T_{\mathrm{s}}$ 或 $5120\cdot T_{\mathrm{s}}$ 的 UpPTS 字段（见 36.211 的 Table 4.2-1）。由于 format 4 及其保护时间 GT 的长度明显小于前面介绍的 format 0~3，format 4 只支持覆盖范围很小的小区。

小结：时域上占的连续的 subframe 数：1、2、3、UpPTS；占据子帧内的所有 slot 和所有 SC-FDMA 符号（symbol）。配置时需要考虑小区的覆盖范围以及资源的使用（preamble 越大，可用于传输上行数据的资源就越少）。

前面已经介绍了 preamble 在时域和频域上所占的资源大小，接下来我们来讨论 preamble 在时域和频域上的位置。

对 **FDD** 而言，只支持 preamble format 0~3，且每个子帧至多只有一个 PRACH 资源，即多个随机接入请求只在时域上存在复用。36.211 的 Table 5.7.1-2 指定了 format 以及允许传输 preamble 的子帧配置，这是通过 *prach-ConfigIndex* 来指定的。假如 UE 接收到的 *prach-ConfigIndex* 配置为 12，则该 UE 可以选择任意（any）系统帧的（0，2，4，6，8）这 5 个子帧中的某一个来传输 format 0 的 preamble。假如 UE 接收到的 *prach-ConfigIndex* 配置为 18，则该 UE 只能选择在偶数（even）系统帧的子帧 7 来传输 format 1 的 preamble。

```
PRACH-ConfigSIB ::=                 SEQUENCE {
    rootSequenceIndex                   INTEGER (0..837),
    prach-ConfigInfo                    PRACH-ConfigInfo
}

PRACH-Config ::=                    SEQUENCE {
    rootSequenceIndex                   INTEGER (0..837),
    prach-ConfigInfo                    PRACH-ConfigInfo
}

PRACH-ConfigSCell-r10 ::=               SEQUENCE {
    prach-ConfigIndex-r10                   INTEGER (0..63)
}

PRACH-ConfigInfo ::=                SEQUENCE {
    prach-ConfigIndex                   INTEGER (0..63),
    highSpeedFlag                       BOOLEAN,
    zeroCorrelationZoneConfig           INTEGER (0..15),
    prach-FreqOffset                    INTEGER (0..94)
}
```

FDD

Table 5.7.1-2: Frame structure type 1 random access configuration for preamble formats 0-3.

PRACH Configuration Index	Preamble Format	System frame number	Subframe number	PRACH Configuration Index	Preamble Format	System frame number	Subframe number
0	0	Even	1	32	2	Even	1
1	0	Even	4	33	2	Even	4
2	0	Even	7	34	2	Even	7
3	0	Any	1	35	2	Any	1
4	0	Any	4	36	2	Any	4
5	0	Any	7	37	2	Any	7
6	0	Any	1, 6	38	2	Any	1, 6
7	0	Any	2, 7	39	2	Any	2, 7
8	0	Any	3, 8	40	2	Any	3, 8
9	0	Any	1, 4, 7	41	2	Any	1, 4, 7
10	0	Any	2, 5, 8	42	2	Any	2, 5, 8
11	0	Any	3, 6, 9	43	2	Any	3, 6, 9
12	0	Any	0, 2, 4, 6, 8	44	2	Any	0, 2, 4, 6, 8
13	0	Any	1, 3, 5, 7, 9	45	2	Any	1, 3, 5, 7, 9
14	0	Any	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	46	N/A	N/A	N/A
15	0	Even	9	47	2	Even	9
16	1	Even	1	48	3	Even	1
17	1	Even	4	49	3	Even	4
18	1	Even	7	50	3	Even	7
19	1	Any	1	51	3	Any	1
20	1	Any	4	52	3	Any	4
21	1	Any	7	53	3	Any	7
22	1	Any	1, 6	54	3	Any	1, 6
23	1	Any	2, 7	55	3	Any	2, 7
24	1	Any	3, 8	56	3	Any	3, 8
25	1	Any	1, 4, 7	57	3	Any	1, 4, 7
26	1	Any	2, 5, 8	58	3	Any	2, 5, 8
27	1	Any	3, 6, 9	59	3	Any	3, 6, 9
28	1	Any	0, 2, 4, 6, 8	60	N/A	N/A	N/A
29	1	Any	1, 3, 5, 7, 9	61	N/A	N/A	N/A
30	N/A	N/A	N/A	62	N/A	N/A	N/A
31	1	Even	9	63	3	Even	9

图 21-4：FDD 下，*prach-ConfigIndex* 与 Table 5.7.1-2 的对应关系

对 **FDD** 而言，preamble 在频域上的起始 RB（ $n_{\mathrm{PRB}}^{\mathrm{RA}}$ ）等于 *prach-FrequencyOffset* 指定的值（用 $n_{\mathrm{PRBoffset}}^{\mathrm{RA}}$ 表示，且满足条件 $0 \le n_{\mathrm{PRBoffset}}^{\mathrm{RA}} \le N_{\mathrm{RB}}^{\mathrm{UL}} - 6$ ）。

对 **TDD** 而言，每个上行子帧可以有多个 PRACH 资源，这是因为 TDD 中每个系统帧内的上行子帧数更少，从而要求每个上行子帧发送更多的 RA 请求（即 preamble）。在 TDD 中，每个 10 ms 的系统帧内至多可发送 6 个 RA 请求（见 36.211 的 Table 5.7.1-3 的 D_{RA} ）。

对 **TDD** 而言，preamble 在时域上的配置也是通过 *prach-ConfigIndex* 来指定的，且对应的表为 36.211 的 Table 5.7.1-3 和 Table 5.7.1-4。其中 D_{RA} 表示 UE 在一个 10ms 的系统帧内有多少次随机接入的机会。r_{RA} 在协议中没有介绍，在网上也看到过说这个字段没有用处的，但其出处应该是 ZTE 的提案《Time & Frequency Location Mapping for TDD PRACH》，有兴趣的大家可以去研究一下，顺便也能够了解 36.211 的 Table 5.7.1-3 和 Table 5.7.1-4 是如何通过计算得来的。

对 **TDD** 而言，36.211 的 Table 5.7.1-4 指定了 preamble 的时频位置。四元组 $(f_{\mathrm{RA}}, t_{\mathrm{RA}}^{(0)}, t_{\mathrm{RA}}^{(1)}, t_{\mathrm{RA}}^{(2)})$ 唯一指定一个特定的随机接入资源。$t_{\mathrm{RA}}^{(0)} = 0,1,2$ 指定了 preamble 可以选择在哪些系统帧上发送（0：所有帧；1：偶数帧；2：奇数帧）。$t_{\mathrm{RA}}^{(1)} = 0,1$ 指定 preamble 是位于一个系统帧的前半帧还是后半帧（0：前半帧；1：后半帧）。$t_{\mathrm{RA}}^{(2)}$ 指定 preamble 起始的上行子帧号，该子帧号位于两个连续的 downlink-to-uplink switch point 之间，且从 0 开始计数（见图 21-5）。format 4 是个例外，其 $t_{\mathrm{RA}}^{(2)}$ 标记为(*)。

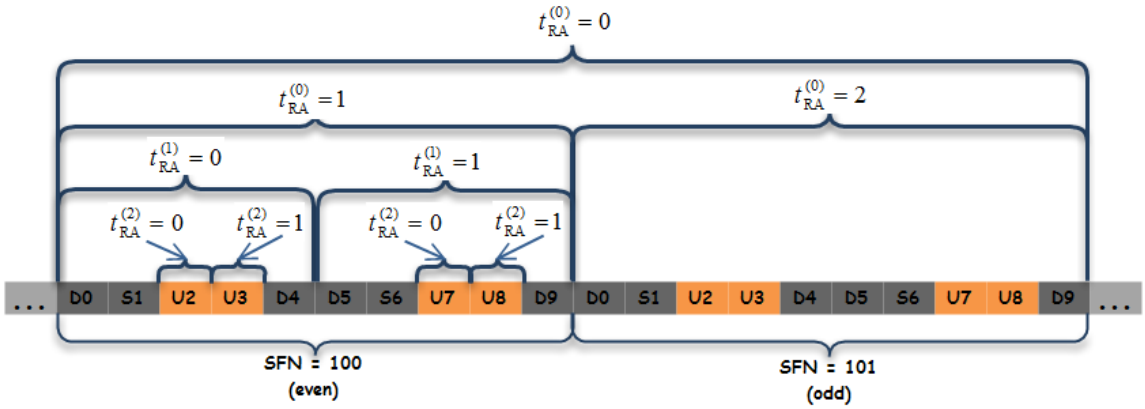


图 21-5：PRACH 的时域资源配置（preamble format 0~3、TDD Configuration 1）

对于 format 4 而言，其起始子帧是特殊帧，无 $t_{\mathrm{RA}}^{(2)}$ 配置。

这样，通过 *prach-ConfigIndex* 指定的 PRACH configuration index，UE 就得到了可能的 $t_{\mathrm{RA}}^{(0)}$、$t_{\mathrm{RA}}^{(1)}$、$t_{\mathrm{RA}}^{(2)}$ 配置，从而知道可以在哪些子帧上发送 preamble。

对于 **TDD** 而言，preamble 在频域上的起始 RB 是由 *prach-ConfigIndex* 和 *prach-FrequencyOffset* 确定的。通过 *prach-ConfigIndex* 查 36.211 的 Table 5.7.1-4 得到 f_{RA}（频域的偏移，单位是 6 个 RB

），通过 *prach-FrequencyOffset* 可以得到 $n_{\text{PRBoffset}}^{\text{RA}}$ ，再通过如下公式，可以得到 format 0~3 的 preamble 在频域上的起始 RB：

$$n_{\text{PRB}}^{\text{RA}} = \begin{cases} n_{\text{PRB offset}}^{\text{RA}} + 6\left\lfloor \frac{f_{\text{RA}}}{2} \right\rfloor, & \text{if } f_{\text{RA}} \bmod 2 = 0 \\ N_{\text{RB}}^{\text{UL}} - 6 - n_{\text{PRB offset}}^{\text{RA}} - 6\left\lfloor \frac{f_{\text{RA}}}{2} \right\rfloor, & \text{otherwise} \end{cases}$$

为了保证上行单载波的频域资源连续性，PRACH 资源通常分布在上行带宽的两端上（“低高频位置交错”）。起始位置由 $n_{\text{PRBoffset}}^{\text{RA}}$ 确定，一般紧挨着 PUCCH 资源的位置。公式中的数字 6 是为了保证 preamble 在频域占 6 个连续的 RB。

```
PRACH-ConfigSIB ::=              SEQUENCE {
    rootSequenceIndex                INTEGER (0..837),
    prach-ConfigInfo                 PRACH-ConfigInfo
}

PRACH-Config ::=                   SEQUENCE {
    rootSequenceIndex                INTEGER (0..837),
    prach-ConfigInfo                 PRACH-ConfigInfo
}

PRACH-ConfigSCell-r10 ::=            SEQUENCE {
    prach-ConfigIndex-r10                INTEGER (0..63)
}

PRACH-ConfigInfo ::=               SEQUENCE {
    prach-ConfigIndex                INTEGER (0..63),
    highSpeedFlag                    BOOLEAN,
    zeroCorrelationZoneConfig        INTEGER (0..15),
    prach-FreqOffset                 INTEGER (0..94)
}
```

$$n_{\text{PRB}}^{\text{RA}} = \begin{cases} n_{\text{PRB offset}}^{\text{RA}} + 6\left\lfloor \frac{f_{\text{RA}}}{2} \right\rfloor, & \text{if } f_{\text{RA}} \bmod 2 = 0 \\ N_{\text{RB}}^{\text{UL}} - 6 - n_{\text{PRB offset}}^{\text{RA}} - 6\left\lfloor \frac{f_{\text{RA}}}{2} \right\rfloor, & \text{otherwise} \end{cases}$$

图 21-6： prach-FreOffset

对于 format 4 而言，起始 RB 的计算公式如下：

$$n_{\text{PRB}}^{\text{RA}} = \begin{cases} 6 f_{\text{RA}}, & \text{if } \left((n_{\text{f}} \bmod 2) \times (2 - N_{\text{SP}}) + t_{\text{RA}}^{(1)}\right) \bmod 2 = 0 \\ N_{\text{RB}}^{\text{UL}} - 6(f_{\text{RA}} + 1), & \text{otherwise} \end{cases}$$

其中 n_{f} 是系统帧号， N_{SP} 是该系统帧内 downlink-to-uplink switch point 的个数。

对应某个特定的 TDD 上下行配置和特定 PRACH configuration Index 配置，查 36.211 的 Table 5.7.1-4，得到四元组 $(f_{\mathrm{RA}}, t_{\mathrm{RA}}^{(0)}, t_{\mathrm{RA}}^{(1)}, t_{\mathrm{RA}}^{(2)})$ 集合。该集合中 $t_{\mathrm{RA}}^{(0)}$、$t_{\mathrm{RA}}^{(1)}$、$t_{\mathrm{RA}}^{(2)}$ 都相等（对应同一上行子帧），但 f_{RA} 不相等的元素有几个，就说明了该子帧上有几个 PRACH 资源。例如对于 TDD 2，PRACH configuration Index 等于 15 的配置，四元组(0,0,0,0)、(1,0,0,0)、(2,0,0,0)对应同一上行子帧上的 3 个 PRACH 资源，四元组(0,0,1,0)、(1,0,1,0)对应另一个上行子帧上的 2 个 PRACH 资源。而对于 TDD 0，PRACH configuration Index 等于 18 的配置，对应的每个上行子帧上都只有一个 PRACH 资源。

小结：对于 FDD 而言，通过 *prach-ConfigIndex* 查 36.211 的 Table 5.7.1-2 得到 preamble format 以及可以用于传输 preamble 的系统帧和子帧号，从而确定可选的时域资源。通过 *prach-FrequencyOffset* 得到在频域上的起始 RB，从而确定频域资源（FDD 在某个子帧上只有一个频域资源，因此是固定的）。

对于 TDD 而言，通过 *prach-ConfigIndex* 查 36.211 的 Table 5.7.1-3 和 Table 5.7.1-4 得到 preamble format 以及可以用于传输 preamble 的四元组 $(f_{\mathrm{RA}}, t_{\mathrm{RA}}^{(0)}, t_{\mathrm{RA}}^{(1)}, t_{\mathrm{RA}}^{(2)})$，其中 $t_{\mathrm{RA}}^{(0)}$、$t_{\mathrm{RA}}^{(1)}$、$t_{\mathrm{RA}}^{(2)}$ 确定时域上可用于传输 preamble 的系统帧和子帧号，从而确定可选的时域资源。通过 *prach-FrequencyOffset* 得到 $n_{\mathrm{PRBoffset}}^{\mathrm{RA}}$，并与 f_{RA} 共同确定了可选的频域资源（TDD 在某个子帧上可能存在多个频域资源，所以是可选择的）。

如果 eNodeB 没有明确指定使用哪一个时频资源，则 UE 选择这些时频资源中的哪一个，是由 UE 的实现决定的。对于第一次发起的随机接入（而不是因为接入失败而发起的 preamble 重发），个人觉得可以选用时域上最接近的子帧，而频域上随机选择一个资源进行传输 preamble。对于由接入失败而发起的 preamble 重发，其时域资源（timing）的选择有点特殊，这会在后续章节中予以介绍。

简单地说：eNodeB 通过广播 SIB2 发送 *prach-ConfigIndex* 和 *prach-FrequencyOffset*，从而确定该小区可用于传输 preamble 的时频资源集合。UE 发起随机接入时，从中选择一个资源（也可能由 eNodeB 指定）发送 preamble。因为 eNodeB 不知道 UE 会在哪个时频资源上发送 preamble，所以会在指示的所有 preamble 时频资源上检测并接收 preamble。

从前面的介绍可以看出，UE 至少要在接收到 MIB、SIB1 和 SIB2（不一定要接收 SIB3~SIB13）后，才能发起随机接入过程。

21.3 随机接入过程

无论是基于竞争的随机接入，还是基于非竞争的随机接入，UE 都只能在其 PCell 上发起随机接入。

对于基于竞争的随机接入而言，其整个过程包含 4 个步骤：

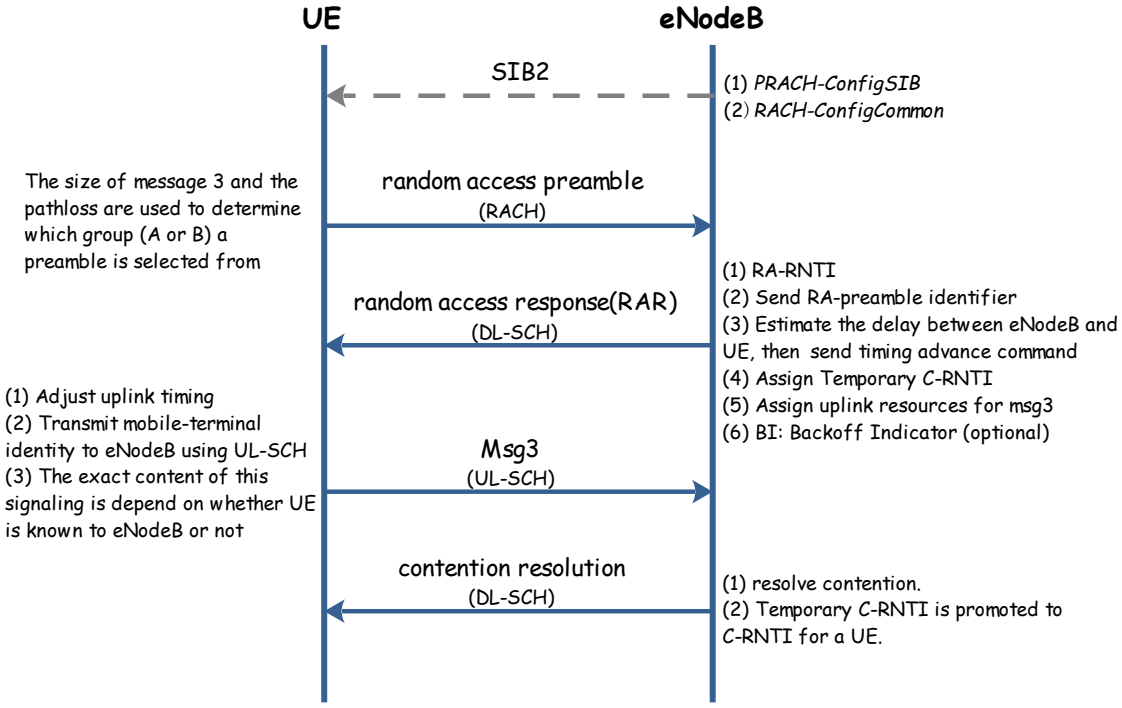


图 21-7：基于竞争的随机接入

对于基于非竞争的随机接入而言，其整个过程包含 2 个步骤：

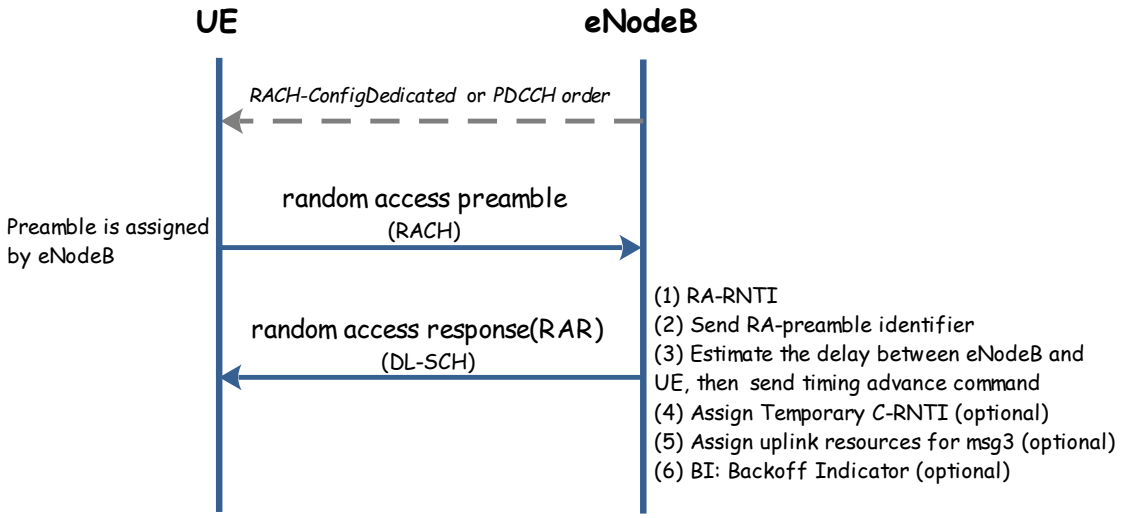


图 21-8：基于非竞争的随机接入

21.3.1 步骤一：UE 发送 preamble

UE 发送 random access preamble 给 eNodeB，以告诉 eNodeB 有一个随机接入请求，同时使得 eNodeB 能估计其与 UE 之间的传输时延并以此校准上行 timing。

触发随机接入过程的方式有以下 3 种（具体会在 21.4 节介绍）：

（1）PDCCH order 触发：eNodeB 通过特殊的 DCI format 1A 告诉 UE 需要重新发起随机接入，并告诉 UE 应该使用的 Preamble Index 和 PRACH Mask Index；

（2）MAC 子层触发：UE 自己选择 preamble 发起随机接入；

（3）上层触发：如初始接入，RRC 连接重建，handover 等。

UE 要成功发送 preamble，需要：（1）选择 preamble index；（2）选择用于发送 preamble 的 PRACH 资源；（3）确定对应的 RA-RNTI；（4）确定目标接收功率 PREAMBLE_RECEIVED_TARGET_POWER。

1、选择 preamble index

基于竞争的随机接入，其 preamble index 是由 UE 随机选择的。

UE 首先要确定选择的是 group A 还是 group B 中的 preamble。如果存在 preamble group B，且 Msg3 的大小大于 *messageSizeGroupA*，且 pathloss 小于 $P_{CMAX,c}$ – *preambleInitialReceivedTargetPower - deltaPreambleMsg3 – messagePowerOffsetGroupB*，则选择 group B；否则选择 group A。

如果 UE 在之前发送过 Msg3 且接入失败，则再次尝试接入时使用的 preamble 应该与第一次发送 Msg3 时使用的 preamble 属于相同的 group。

确定了 group 之后，UE 从该 group 中随机选择一个 preamble 并将 PRACH Mask Index 设置为 0。

而基于非竞争的随机接入中的 preamble index 是由 eNodeB 指定的。

对于基于非竞争的随机接入而言，eNodeB 通过为 UE 分配一个专用的 preamble index 来避免冲突的发生并指定一个 PRACH Mask Index。

eNodeB 分配 preamble index 和 PRACH Mask Index 的方式有两种：（1）通过 *RACH-ConfigDedicated* 的 *ra-PreambleIndex* 和 *ra-PRACH-MaskIndex* 字段设置（handover 过程）；（2）在 PDCCH order 触发的随机接入中，通过 DCI format 1A 的 Preamble Index 和 PRACH Mask Index 字段来设置（下行数据到达或定位）。

按理说，既然要使用基于非竞争的随机接入过程，eNodeB 分配的 preamble index 就不应该为 0（0 是用于基于竞争的随机接入的。个人认为此时不应使用 group A 和 group B 的任一 preamble，但协议中只针对 0 做了特别说明）。但如果 eNodeB 分配了 0 值，则实际的 preamble index 交由 UE 按照基于竞争的随机接入方式随机选择一个 preamble（个人认为这种情况主要针对 eNodeB 已经没有可用的非竞争 preamble，或 eNodeB 配置时根本没有为非竞争的随机接入预留 preamble 的场景）。

2、选择用于发送 preamble 的 PRACH 资源

基于 *prach-ConfigIndex*、PRACH Mask Index 以及物理层的 timing 限制，UE 会先确定下一个包含 PRACH 资源的可用子帧（在确定下一个可用的 PRACH 子帧时，UE 还需要将可能出现的 measurement gap 考虑在内）。

prach-ConfigIndex 指定了时域上可用的 PRACH 资源集合。

PRACH Mask Index 定义了某个 UE 可以在系统帧内的哪些 PRACH 上发送 preamble（见 36.321 的 Table 7.3-1，值为 0 表示所有可用的 PRACH 资源）。在基于非竞争的随机接入中，eNodeB 可以通过该 mask 直接指定 UE 在某个特定的 PRACH 上发送 preamble，从而保证不会与其它 UE 发生冲突。

Table 7.3-1: PRACH Mask Index values

PRACH Mask Index	Allowed PRACH (FDD)	Allowed PRACH (TDD)
0	All	All
1	PRACH Resource Index 0	PRACH Resource Index 0
2	PRACH Resource Index 1	PRACH Resource Index 1
3	PRACH Resource Index 2	PRACH Resource Index 2
4	PRACH Resource Index 3	PRACH Resource Index 3
5	PRACH Resource Index 4	PRACH Resource Index 4
6	PRACH Resource Index 5	PRACH Resource Index 5
7	PRACH Resource Index 6	Reserved
8	PRACH Resource Index 7	Reserved
9	PRACH Resource Index 8	Reserved
10	PRACH Resource Index 9	Reserved
11	Every, in the time domain, k 1st PRACH Resource Index in subframe	Every, in the time domain, even PRACH opportunity 1st PRACH Resource Index in subframe
12	Every, in the time domain, odd PRACH opportunity 1st PRACH Resource Index in subframe	Every, in the time domain, odd PRACH opportunity 1st PRACH Resource Index in subframe
13	Reserved	1st PRACH Resource Index in subframe
14	Reserved	2nd PRACH Resource Index in subframe
15	Reserved	3rd PRACH Resource Index in subframe

以 *ra-PRACH-MaskIndex* = 3 为例，查 36.321 的 Table 7.3.1 可知，对应 PRACH Resource Index 2，即 preamble 应该在系统帧内的第三个 PRACH 资源发送。PRACH Resource Index 是一个系统帧内的 PRACH 资源的编号，从 0 开始并以 PRACH 资源在 36.211 的 Table 5.7.1-2 和 Table 5.7.1-4 中出现的先后来排序。（以 *prach-ConfigIndex* = 12 为例，如果是 FDD，查 36.211 的 Table 5.7.1-2 可知，只在子帧 0、2、4、6、8 上存在 PRACH 资源，则 PRACH Resource Index 2 对应子帧 4 上的 PARCH 资源；如果是 TDD，且 UL/DL configuration 为 1，查 36.211 的 Table 5.7.1-4 可知，PRACH Resource Index 2 对应四元组（0,0,1,0）上的 PARCH 资源）

PRACH Mask Index 可以为 0，这说明 eNodeB 只为 UE 分配了 preamble，但 PRACH 资源还需 UE 自己选择。

物理层的 timing 限制在 36.213 的 6.1.1 中定义：

如果 UE 在子帧 n 接收到一个 RAR MAC PDU，但对应 TB 中没有一个响应与其发送的 preamble 对应，则 UE 应该准备好在不迟于子帧 n + 5 的时间内重新发送 preamble。

如果 UE 在子帧 n 没有接收到一个 RAR MAC PDU，其中子帧 n 为 RAR 窗口的最后一个子帧，则 UE 应该准备好在不迟于子帧 n + 4 的时间内重新发送 preamble。

如果随机接入过程是由 PDCCH order 在子帧 n 触发，则 UE 将在从子帧 $n+k_2$ 算起，第一个有可用 PRACH 的子帧中发送，其中 $k_2 \geq 6$。

至此，已经选定 PRACH 所在的子帧，接下来，我们开始选择频域上的位置。

在 TDD 模式且 PRACH Mask Index 为 0 的情况下：如果 eNodeB 指定了 *ra-PreambleIndex* 且其值不为 0，则在之前确定的子帧上随机选择一个 PRACH；否则在之前确定的子帧及其后续的两个子帧（共 3 个子帧）内随机选择一个 PRACH。

如果是 FDD 模式或 PRACH Mask Index 不为 0，则根据 PRACH Mask Index 选择一个 PRACH。

3、确定对应的 RA-RNTI

preamble 的时频位置决定了 RA-RNTI 的值，UE 发送了 preamble 之后，会在 RAR 时间窗内根据这个 RA-RNTI 值来监听对应的 PDCCH。RA-RNTI 的计算会在步骤二中介绍。

4、确定目标接收功率 PREAMBLE_RECEIVED_TARGET_POWER

preamble 的目标接收功率 PREAMBLE_RECEIVED_TARGET_POWER 通过下面的公式计算（见 36.321 的 5.1.3 节）：

preambleInitialReceivedTargetPower + DELTA_PREAMBLE + (PREAMBLE_TRANSMISSION_COUNTER – 1) * *powerRampingStep*

其中 *preambleInitialReceivedTargetPower* 是 eNodeB 期待接收到的 preamble 的初始功率。DELTA_PREAMBLE 与 preamble format 相关，其值见 36.321 的的 Table 7.6-1。而 *powerRampingStep* 是每次接入失败后，下次接入时提升的发射功率。

而 preamble 的实际发射功率 P_{PRACH} 的计算公式为

$$P_{PRACH} = \min\{ P_{CMAX,c}(i),\ \text{PREAMBLE_RECEIVED_TARGET_POWER} + PL_c \}_[\text{dBm}]$$

其中，$P_{CMAX,c}(i)$ 是 UE 在 PCell 的子帧 i 上所配置的最大输出功率，PL_c 是 UE 通过测量 PCell 的小区特定的参考信号得到的下行路径损耗。

21.3.2 步骤二：eNodeB 发送 Random Access Response

UE 发送了 preamble 之后，将在 RAR 时间窗（RA Response window）内监听 PDCCH，以接收对应 RA-RNTI 的 RAR（此时不考虑可能出现的 measurement gap）。如果在此 RAR 时间窗内没有接收到 eNodeB 回复的 RAR，则认为此次随机接入过程失败。

RAR 时间窗起始于发送 preamble 的子帧（如果 preamble 在时域上跨多个子帧，则以最后一个子帧计算）+ 3 个子帧，并持续 *ra-ResponseWindowSize* 个子帧。

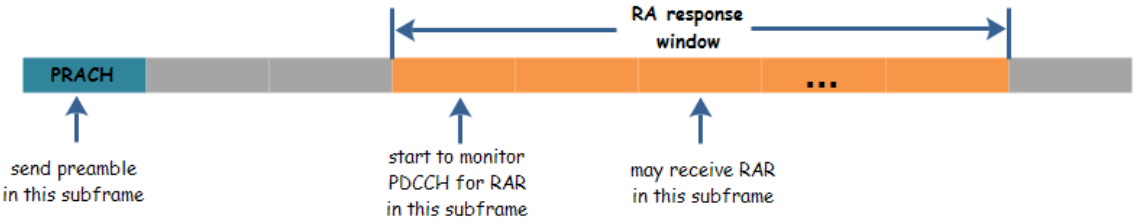


图 21-9：RA response window

与 preamble 相关联的 RA-RNTI 通过如下公式计算：

$$RA\text{-}RNTI = 1 + t_id + 10 * f_id$$

其中，t_id 是发送 preamble 的 PRACH 所在的第一个子帧号（$0 \le t_id < 10$），f_id 是在该子帧发送 preamble 的 PRACH 在频域上的索引（$0 \le f_id < 6$）。对于 FDD 而言，每个子帧只有一个 PRACH 资源，因此 f_id 固定为 0。（RA-RNTI 的计算见 36.321 的 5.1.4 节）

某个 UE 发送的 preamble 时频位置是确定的，eNodeB 在解码 preamble 时，也获得了该 preamble 的时频位置，进而知道了 RAR 中需要使用的 RA-RNTI。

当 UE 成功地接收到一个 RAR（使用前面介绍的 RA-RNTI 来解码），且该 RAR 中的 preamble index 与 UE 发送的 preamble index 相同时，则认为成功接收了 RAR，此时 UE 就可以停止监听 RAR 了。

接下来，我会从 Random Access Response 的 MAC PDU 构成的角度来介绍 RAR 携带的信息。

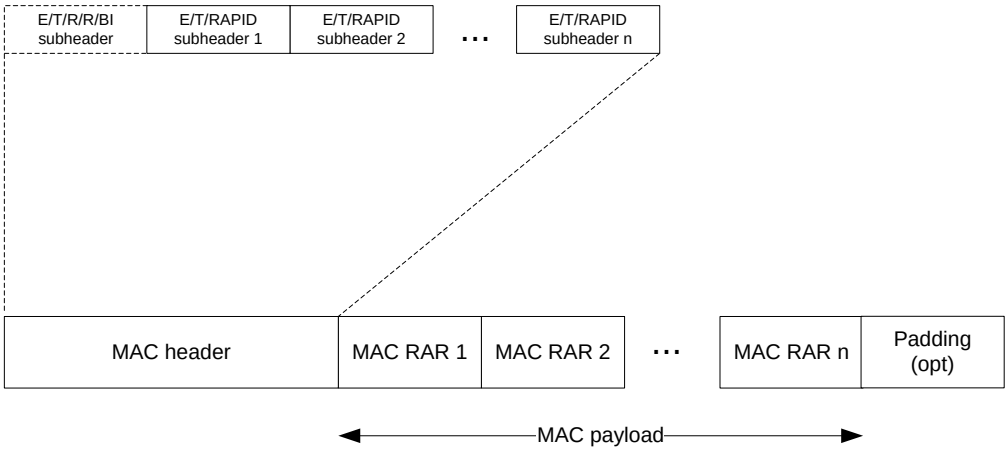


图 21-10：由 MAC 头和 MAC RARs 组成的 RAR MAC PDU

从图 21-10 可以看出，RAR MAC PDU 由 1 个 MAC 头（MAC header）+ 0 个或多个 MAC RAR（MAC Random Access Response，简称为 RAR）+ 可能存在的 padding 组成。

注意 RAR MAC PDU 与 RAR 的区别：1 个 RAR MAC PDU 包含 1 个或多个 RAR。

从 RAR MAC PDU 的结构可以看出，如果 eNodeB 在同一 PRACH 资源上检测到来自多个 UE 的随机接入请求，则使用一个 RAR MAC PDU 就可以对这些接入请求进行响应，每个随机接入请求（对应一个 preamble index）的响应对应一个 RAR。

如果多个 UE 在同一 PRACH 资源（时频位置相同，使用同一 RA-RNTI）发送 preamble，则对应的 RAR 复用在同一 RAR MAC PDU 中。

RAR MAC PDU 在 DL-SCH 上传输，并用以 RA-RNTI 加扰的 PDCCH 来指示。前面已经介绍过，使用相同 PRACH 资源发送 preamble（preamble 不一定需要相同）的所有 UE 都监听相同的 RA-RNTI 加扰的 PDCCH，并接收相同的 RAR MAC PDU，但不同 preamble index 对应不同的 RAR。

由于 RAR MAC PDU 只能使用一个 RA-RNTI 加扰，这也意味着使用不同 PRACH 资源（时频位置不同）发送的 preamble 对应的 RAR 不能复用到同一个 RAR MAC PDU 中。

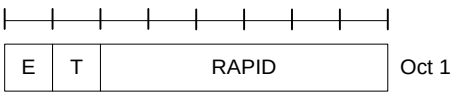


Figure 6.1.5-1: E/T/RAPID MAC subheader



Figure 6.1.5-2: E/T/R/R/BI MAC subheader

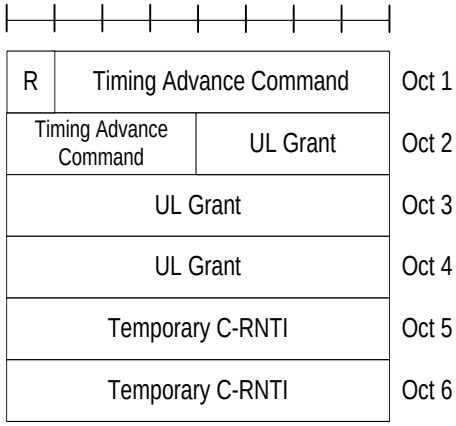


Figure 6.1.5-3: MAC RAR

MAC header 由一个或多个 MAC subheader 组成。除了 Backoff Indicator subheader 外，每个 subheader 对应一个 RAR。如果包含 Backoff Indicator subheader，则该 subheader 只出现一次，且位于 MAC header 的第一个 subheader 处。

如果 UE 收到了一个 Backoff Indicator subheader，则会保存一个 backoff 值，该值等于该 subheader 中的 BI 值；否则 UE 会将 backoff 值设为 0。

BI（Backoff Indicator）指定了 UE 重发 preamble 前需要等待的时间范围（取值范围见 36.321 的 7.2 节）。如果 UE 在 RAR 时间窗内没有接收到 RAR，或接收到的 RAR 中没有一个 preamble 与自己的相符合，则认为此次 RAR 接收失败。此时 UE 需要等待一段时间后，再发起随机接入。等待的时间为在 0 至 BI 指定的等待时间区间内选取一个随机值。（注：如果在步骤四中，冲突解决失败，也会有这样的后退机制）

需要注意的是： BI 指定的 UE 重发 preamble 前需要等待的时间可能与前面介绍的物理层 timing 存在冲突。（具体如何选择发送 preamble 的子帧，取决于 UE 的实现，协议中并没有给出答案！我只在一篇文章中有相关介绍，大家可以参考一下，见参考资料[6]。另外一种可能的解释是，物理层 timing 只是“准备好”发送，实际发送的时间还是由 MAC 层来决定。）

BI 的取值从侧面反映了小区的负载情况，如果接入的 UE 多，则该值可以设置得大些；如果接入的 UE 少，该值就可以设置得小些。

RAPID 为 Random Access Preamble IDentifier 的简称，为 eNodeB 在检测 preamble 时得到的 preamble index。如果 UE 发现该值与自己发送 preamble 时使用的索引相同，则认为成功接收到对应的 RAR。

11 比特的 Timing advance command 用于指定 UE 上行同步所需要的时间调整量。（可参见第 18 章关于“上行同步”的介绍）

20 比特 UL grant 指定了分配给 Msg3 的上行资源。当有上行数据传输时，例如需要解决冲突，eNodeB 在 RAR 中分配的 grant 不能小于 56 比特（见 36.321 的 5.1.4 节）。

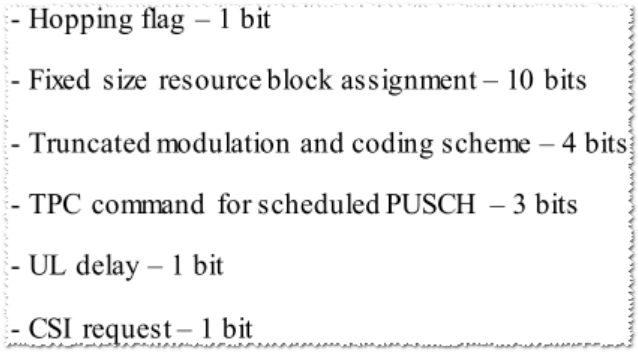


图 21-11：20-bit UL grant

对于 RAR 指定的 PUSCH 传输及其同一 TB 的 PUSCH 重传，都只能使用上行单天线端口传输。

如果对应的 RAR grant 中的“Hopping flag”字段设置为 1，并且上行资源分配类型为 0，则 Msg3 的传输会进行 PUSCH 跳频；否则不进行上行跳频。（从协议上看，RAR 指定的上行资源，只能使用上行资源分配类型为 0）。

10 比特的“Fixed size resource block assignment”字段的解析如下：

- 如果上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}} \le 44$，则只使用该字段的最低 b 个比特，其中 $b = \left\lceil \log_2\left(N_{\mathrm{RB}}^{\mathrm{UL}} \cdot \left(N_{\mathrm{RB}}^{\mathrm{UL}}+1\right)/2\right)\right\rceil \le \left\lceil \log_2\left(44 \cdot (44+1)/2\right)\right\rceil = 10$。其解析方式与正常的 DCI format 0 的“Resource block assignment and hopping resource allocation”字段相同。

- 如果上行系统带宽 $N_{\mathrm{RB}}^{\mathrm{UL}} > 44$，10 比特的该字段会分为 2 部分：$N_{UL_hop}$ 个跳频比特（hopping bit）和剩余的 $10 - N_{UL_hop}$ 比特。如果“Hopping flag”字段设置为 1，则 N_{UL_hop} 个跳频比特的定义如 36.213 的 Table 8.4-1 所示；如果“Hopping flag”字段设置为 0，则 N_{UL_hop} 为 0，即此时不存在跳频比特。由于 10 比特并不足以表示所有的资源分配情况，因此协议中

规定会在 N_{UL_hop} 个跳频比特中插入 b 个为“0”的比特，其中 $b=\left(\left\lceil \log_2\left(N_{\mathrm{RB}}^{\mathrm{UL}}\cdot\left(N_{\mathrm{RB}}^{\mathrm{UL}}+1\right)/2\right)\right\rceil-10\right)$。也就是说，此时认为该字段包含 N_{UL_hop} + b + (10 – N_{UL_hop})比特，然后与正常的 DCI format 0 的“Resource block assignment and hopping resource allocation”字段同样的方式进行解析。

关于 PUSCH 跳频，可参见 9.2 节关于“上行跳频”的介绍。

4 比特的“Truncated modulation and coding scheme”字段的取值范围为 0~15，这说明 RAR 对应的 PUSCH 传输（Msg3）的 I_{MCS} 的取值范围为 0~15，其定义见 36.213 的 Table 8.6.1-1。

3 比特的“TPC command for scheduled PUSCH”字段用于设置 PUSCH 的功率，其定义见 36.213 的 Table 6.2-1。而如何使用该字段，可参见 36.213 的 5.1 节中关于上行功控的介绍。

1 比特的“CSI request”字段：对于基于非竞争的随机接入而言，RAR 中的 CSI request 字段用于决定在对应的 PUSCH 传输中是否包含非周期性的 CQI/PMI/RI 上报。而对于基于竞争的随机接入而言，RAR 中的 CSI request 字段是预留的，即没有任何作用。

关于 RAR 里 20 比特 UL grant 的详细说明，可参见 36.213 的 6.2 节。

在随机接入过程中，如果用于同一 preamble group 的 RAR 中 UL grant 指定的资源大小与随机接入过程中第一次分配的 UL grant 不同，则 UE 的行为是未定义，换句话说，就不应该出现这种情况。

TC-RNTI 用于 UE 和 eNodeB 的后续传输。冲突解决后，该值可能变成 C-RNIT。

UE 随机选择一个 preamble 用于随机接入，就可能导致多个 UE 同时选择同一 PRACH 资源和同一个 preamble，从而导致冲突的出现（使用相同的 RA-RNTI 和 preamble，因此还不确定 RAR 是对哪个 UE 的响应），这时需要一个冲突解决机制来解决这个问题。冲突的存在也是 RAR 不使用 HARQ 的原因之一。（基于竞争的随机接入）

如果 UE 使用专用的 preamble（*ra-PreambleIndex* 是由 eNodeB 指定的，且该值不为 0）用于随机接入则不会有冲突，也就不需要后续的冲突解决处理，随机接入过程也就到此结束了。（基于非竞争的随机接入）

如果接入过程失败，则 UE 将在上次发射功率的基础上，提升功率 *powerRampingStep* 来发送下次 preamble，以提高发射成功的概率。

简单地说：UE 通过 RAR 所带的 RA-RNTI 和 preamble index 来确定是否成功接收到自己想要的 RAR，然后再进行后续处理。

21.3.3 步骤三：UE 发送 Msg3

基于非竞争的随机接入， preamble 是某个 UE 专用的，所以不存在冲突；又因为该 UE 已经拥有在接入小区内的唯一标志 C-RNTI，所以也不需要 eNodeB 给它分配 C-RNTI。因此，只有基于竞争的随机接入才需要步骤三和步骤四。

注：（1）使用基于非竞争的随机接入的 UE 必定原本处于 RRC_CONNECTED 态；（2）handover 时，UE 在目标小区使用的 C-RNTI 是通过 *RRCConnectionReconfiguration* 中的 *MobilityControlInfo* 的 *newUE-Identity* 来配置的。

之所以将第 3 条消息称为 Msg3 而不是某一条具体消息的原因在于，根据 UE 状态的不同和应用场景的不同，这条消息也可能不同，因此统称为 Msg3，即第 3 条消息。

RAR 与对应的 Msg3 之间的 timing 关系可参见 12.2.4.4 节的介绍。

Msg3 在 UL-SCH 上传输，使用 HARQ，且 RAR 中带的 UL grant 指定的用于 Msg3 的 TB 大小至少为 80 比特。（见 36.300 的 10.1.5.1 节）

Msg3 中需要包含一个重要信息：每个 UE 唯一的标志。该标志将用于步骤四的冲突解决。

对于处于 RRC_CONNECTED 态的 UE 来说，其唯一标志是 C-RNTI。

对于非 RRC_CONNECTED 态的 UE 来说，将使用一个来自核心网的唯一的 UE 标志（S-TMSI 或一个随机数）作为其标志。此时 eNodeB 需要先与核心网通信，才能响应 Msg3。

当 UE 处于 RRC_CONNECTED 态但上行不同步时，UE 有自己的 C-RNTI，在随机接入过程的 Msg3 中，UE 会通过 C-RNTI MAC control element 将自己的 C-RNTI 告诉 eNodeB，eNodeB 在步骤四中使用这个 C-RNTI 来解决冲突。

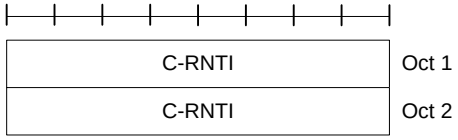


Figure 6.1.3.2-1: C-RNTI MAC control element

当 UE 在随机接入过程中使用上行 CCCH 来发送 Msg3 消息时， UE 还没有 C-RNTI，此时 UE 会使用来自核心网的 UE 标志（S-TMSI 或一个随机数）。步骤四中，eNodeB 会通过发送 UE Contention Resolution Identity MAC Control Element（携带了这个 UE 标志）来解决冲突。

注意：（1）UE Contention Resolution Identity MAC Control Element 是在步骤四中使用的。（2）在 36.331 中搜索“UL-CCCH-Message”，就可以知道有哪些上行 RRC 消息使用 CCCH 信道。当前只有 RRC 连接请求和 RRC 连接重建请求这两条消息使用上行 CCCH。

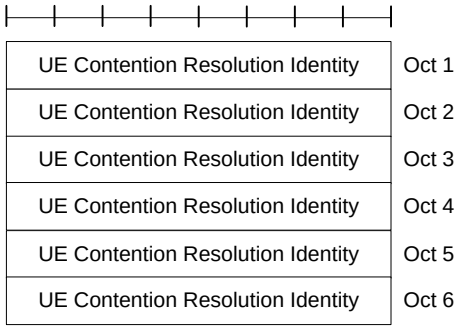


Figure 6.1.3.4-1: UE Contention Resolution Identity MAC control element

与随机接入的触发事件对应起来，Msg3 携带的信息如下：

1、如果是初次接入（initial access），Msg3 为在 CCCH 上传输的 RRC Connection Request，且至少需要携带 NAS UE 标志信息。

2、如果是 RRC 连接重建（RRC Connection Re-establishment），Msg3 为 CCCH 上传输的 RRC Connection Re-establishment Request，且不携带任何 NAS 消息。

3、如果是切换（handover），Msg3 为在 DCCH 上传输的经过加密和完整性保护的 RRC Handover Confirm，必须包含 UE 的 C-RNTI，且如果可能的话，需要携带 BSR。

4、对于其它触发事件，则至少需要携带 C-RNTI。

上行传输通常使用 UE 特定的信息，如 C-RNTI，对 UL-SCH 的数据进行加扰。但此时冲突还未解决，加扰不能基于 C-RNTI，而只能使用 TC-RNTI。也就是说，Msg3 只会使用 TC-RNTI 进行加扰。

21.3.4 步骤四：eNodeB 发送 contention resolution

在步骤三中已经介绍过，UE 会在 Msg3 有携带自己唯一的标志： C-RNTI 或来自核心网的 UE 标志（S-TMSI 或一个随机数）。eNodeB 在冲突解决机制中，会在 Msg4（我们把步骤四的消息称为 Msg4）中携带该唯一的标志以指定胜出的 UE。而其它没有在冲突解决中胜出的 UE 将重新发起随机接入。

eNodeB 会通过 PCell 上使用 C-RNTI 加扰的 PDCCH，或 DL-SCH 上传输的 UE Contention Resolution Identity MAC control element 来指明哪个 UE 在冲突解决中胜出。

UE 发送了 Msg3 后，会启动一个 *mac-ContentionResolutionTimer*，并在 Msg3 进行 HARQ 重传时，重启该 timer。在该 timer 超时或停止之前，UE 会一直监听 PDCCH。

如果 UE 监听到了 PDCCH，且 UE 在发送 Msg3 时携带了 C-RNTI MAC control element，则在以下 2 种情况下，UE 认为冲突解决成功（即该 UE 成功接入，此时 UE 会停止 *mac-ContentionResolutionTimer*，并丢弃 TC-RNTI。**注意**：这 2 种情况下 TC-RNTI 不会提升为 C-RNTI）：

（1）随机接入过程由 MAC 子层触发，且 UE 在 Msg4 中接收到的 PDCCH 由 Msg3 携带的 C-RNTI 加扰，且给新传的数据分配了上行资源；

（2）随机接入过程由 PDCCH order 触发，且 UE 在 Msg4 中接收到的 PDCCH 由 Msg3 携带的 C-RNTI 加扰。

如果 Msg3 在 CCCH 发送，且在 Msg4 中接收到的 PDCCH 由 RAR 中指定的 TC-RNTI 加扰，则当成功解码出的 MAC PDU 中包含的 UE Contention Resolution Identity MAC control element 与 Msg3 发送的 CCCH SDU 匹配时，UE 会认为随机接入成功并将自己的 C-RNTI 设置成 TC-RNTI。（只要成功解码 RAR MAC PDU，就停止 *mac-ContentionResolutionTimer*，并不需要等待冲突解决成功。**注意**：这种情况下 TC-RNTI 会提升为 C-RNTI）

如果 *mac-ContentionResolutionTimer* 超时，UE 会丢弃 TC-RNTI 并认为冲突解决失败。

如果冲突解决失败，UE 需要

（1）清空 Msg3 对应的 HARQ buffer；

（2）将 PREAMBLE_TRANSMISSION_ COUNTER 加 1，如果此时 PREAMBLE_TRANSMISSION_ COUNTER = *preambleTransMax* + 1，则通知上层随机接入发生了问题（Random Access problem indication）；

（3）在 0~BI 值之间随机选择一个 backoff time，UE 延迟 backoff time 后，再发起随机接入。

如果 UE 接入成功，UE 会

（1）如果收到 *ra-PreambleIndex* 和 *ra-PRACH-MaskIndex*，则丢弃；

（2）清空 Msg3 对应的 HARQ buffer。

对于 Msg4 而言，也使用 HARQ，但不需要与 Msg3 同步。从前面的介绍可以看出，对于初始接入和无线链路失效而言，Msg4 使用 TC-RNTI 加扰，且使用 RLC-TM 模式；而对处于 RRC_CONNECTED 态的 UE 而言，Msg4 使用 C-RNTI 加扰。

简单地说：

（1）如果 UE 原本就处于 RRC_CONNECTED 态，则该 UE 在小区内有唯一的标志 C-RNTI。步骤三中，Msg3 会通过 C-RNTI MAC control element 把这个 C-RNTI 带给 eNodeB；步骤四中，如果此 UE 在冲突解决中胜出，eNodeB 就使用这个 C-RNTI 对 PDCCH 进行加扰。UE 收到以此 C-RNTI 加扰的 PDCCH，就知道自己接入成功了。

（2）如果 UE 原本不处于 RRC_CONNECTED 态，则该 UE 在小区内不存在 C-RNTI，其唯一标志就是来自核心网（S-TMSI 或一个随机数）。步骤三中，Msg3 会将该唯一标志带给 eNodeB；步骤四中，如果此 UE 在冲突解决中胜出，eNodeB 会通过 UE Contention Resolution Identity MAC Control Element 将步骤三的信息发回给 UE，UE 比较 Msg3 和 Msg4，发现二者匹配，就知道自己接入成功了。

21.4 各类触发事件下的随机接入过程

本节将介绍各种触发事件是如何触发随机接入过程的，主要以信令流程图的方式予以说明。将本节内容与之前介绍的内容结合起来，有助于更深刻地理解随机接入过程。

触发随机接入过程的事件有 6 种，见之前介绍。

触发随机接入过程的方式有 3 种：（1）PDCCH order 触发；（2）MAC sublayer 触发；（3）上层触发。

由 PDCCH order 发起的随机接入过程（“initiated by a PDCCH order”）只有在如下场景才会发生：（1）eNodeB 要发送下行数据时，发现丢失了 UE 的上行同步，它会强制 UE 重新发起随机接入过程以获取正确的时间调整量；（2）UE 定位。

这时 eNodeB 会通过特殊的 DCI format 1A 告诉 UE 需要重新发起随机接入，并告诉 UE 应该使用的 Preamble Index 和 PRACH Mask Index。（见 36.212 的 5.3.3.1.3 节、36.213 的 Table 8-4）

Format 1A is used for random access procedure initiated by a PDCCH order only if format 1A CRC is scrambled with C-RNTI and all the remaining fields are set as follows:

- Localized/Distributed VRB assignment flag – 1 bit is set to '0'

- Resource block assignment – $\lceil \log_2(N_{RB}^{DL}(N_{RB}^{DL}+1)/2) \rceil$ bits, where all bits shall be set to 1

- Preamble Index – 6 bits

- PRACH Mask Index – 4 bits, [5]

- All the remaining bits in format 1A for compact scheduling assignment of one PDSCH codeword are set to zero

图 21-12：DCI format 1A 用于 PDCCH order 时的格式

对应的信令流程如下（注：UE 定位的处理流程与基于非竞争的下行数据到达场景类似）：

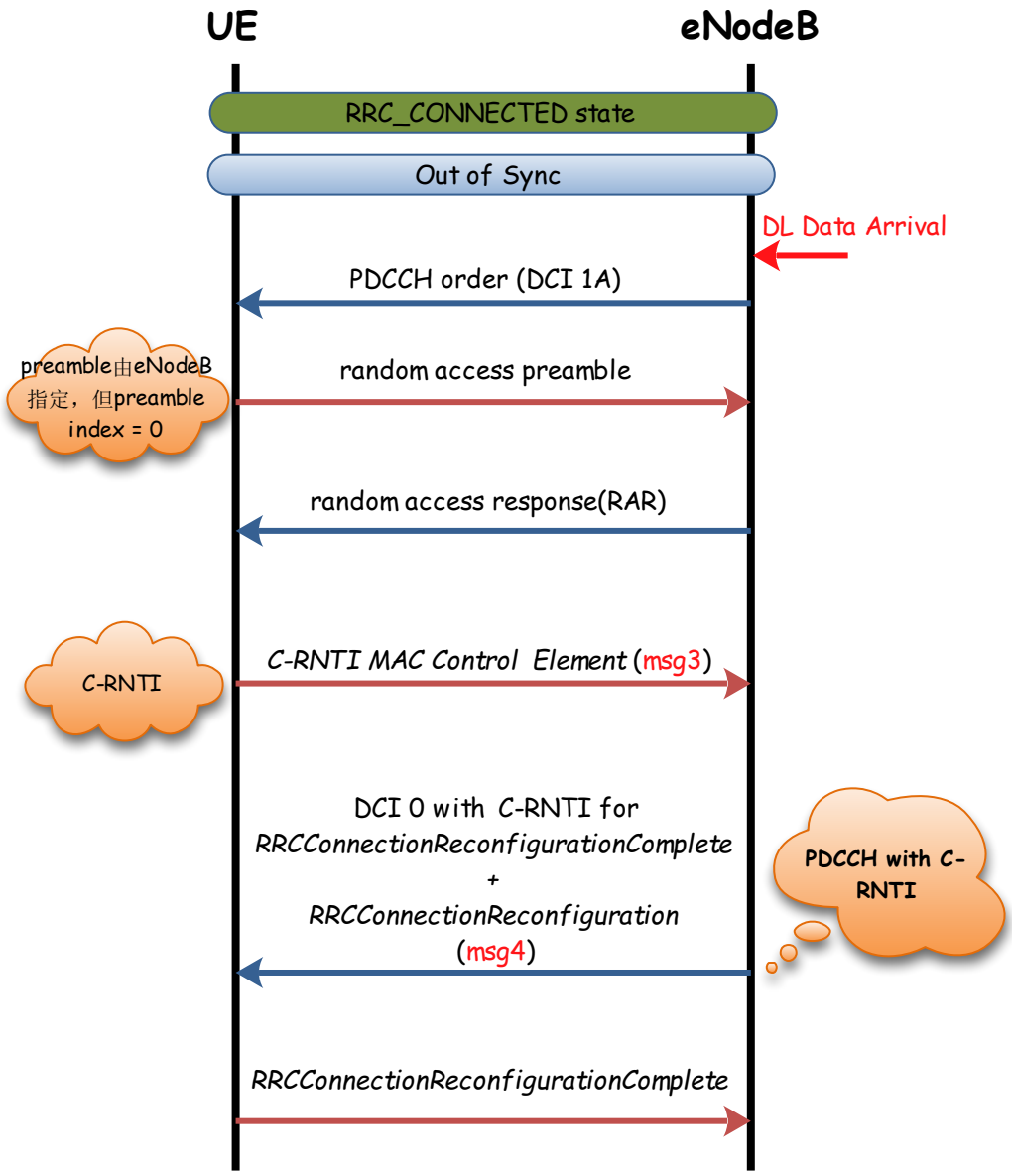


图 21-13：下行数据到达（基于竞争）

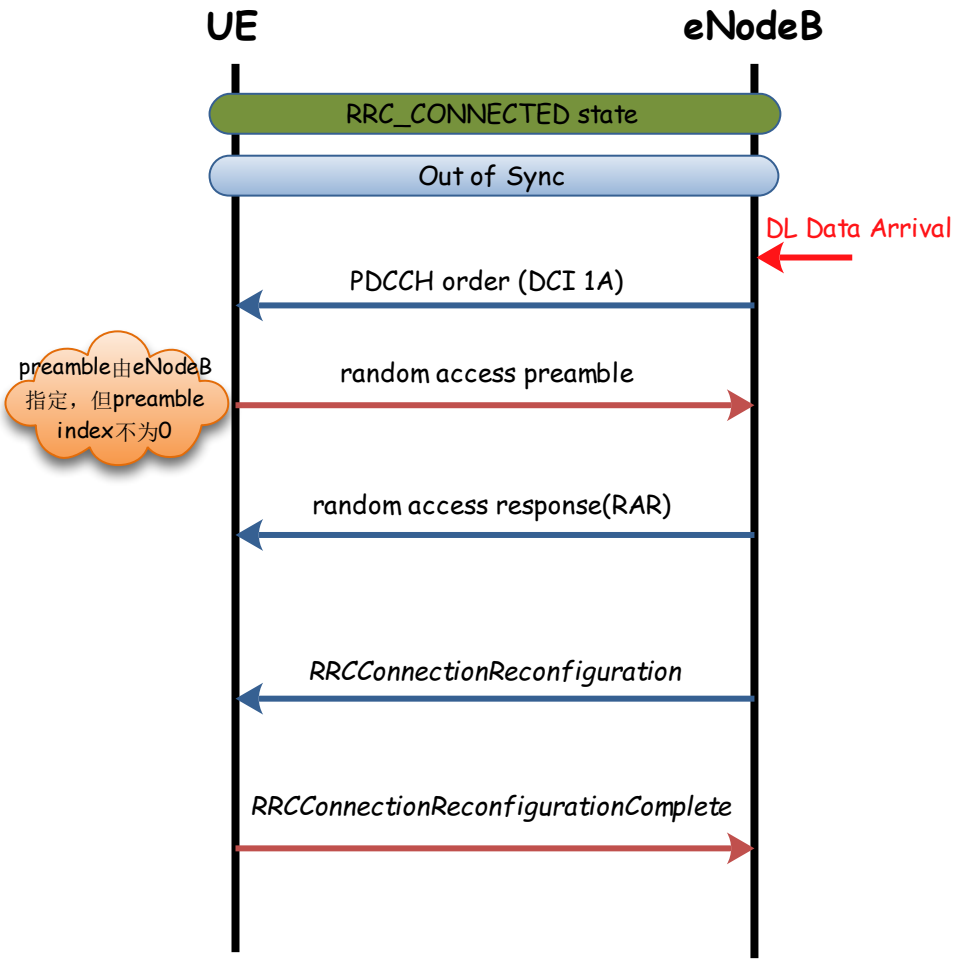


图 21-14：下行数据到达（基于非竞争）

由 MAC sublayer 发起随机接入过程的场景有：UE 有上行数据要发送，但在任意 TTI 内都没有可用于发送 SR 的有效 PUCCH 资源。此时上行数据传输的流程变为：

（1）UE 发送 preamble；

（2）eNodeB 回复 RAR，RAR 携带了 UL grant 信息；

（3）UE 开始发送上行数据。

什么情况下 UE 可能没有 SR 资源呢？场景一：从 36.331 可以看出，*SchedulingRequestConfig* 是一个 UE 级的可选的 IE（optional），默认为 release。如果 eNodeB 不给某 UE 配置 SR（这取决于不同厂商的实现），则该 UE 只能通过随机接入来获取 UL grant。因此，是否配置 SR 主要影响用户面的延迟，并不影响上行传输的功能！

场景二：当 UE 丢失了上行同步，它也会释放 SR 资源，如果此时有上行数据要发送，也需要触发随机接入过程。

从上面的描述可以看出，当 UE 没有被分配 SR 资源时，基于竞争的随机接入可以替代 SR 的功能用于申请上行资源。但这只适用于低密集度的上行资源请求的情况。

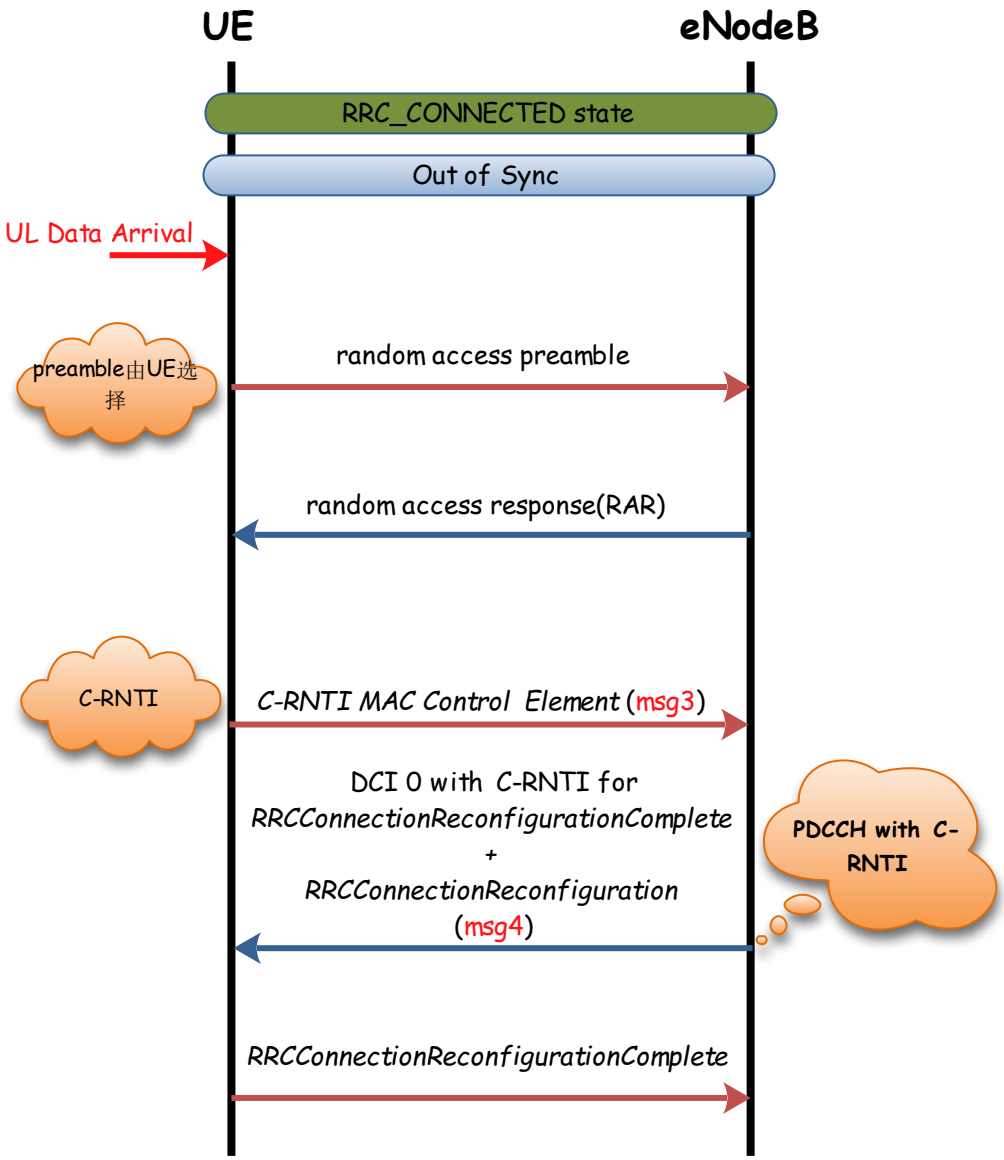


图 21-15：上行数据到达（基于竞争）

上层触发的随机接入过程包括：（1）初始接入；（2）RRC 连接重建；（3）切换。

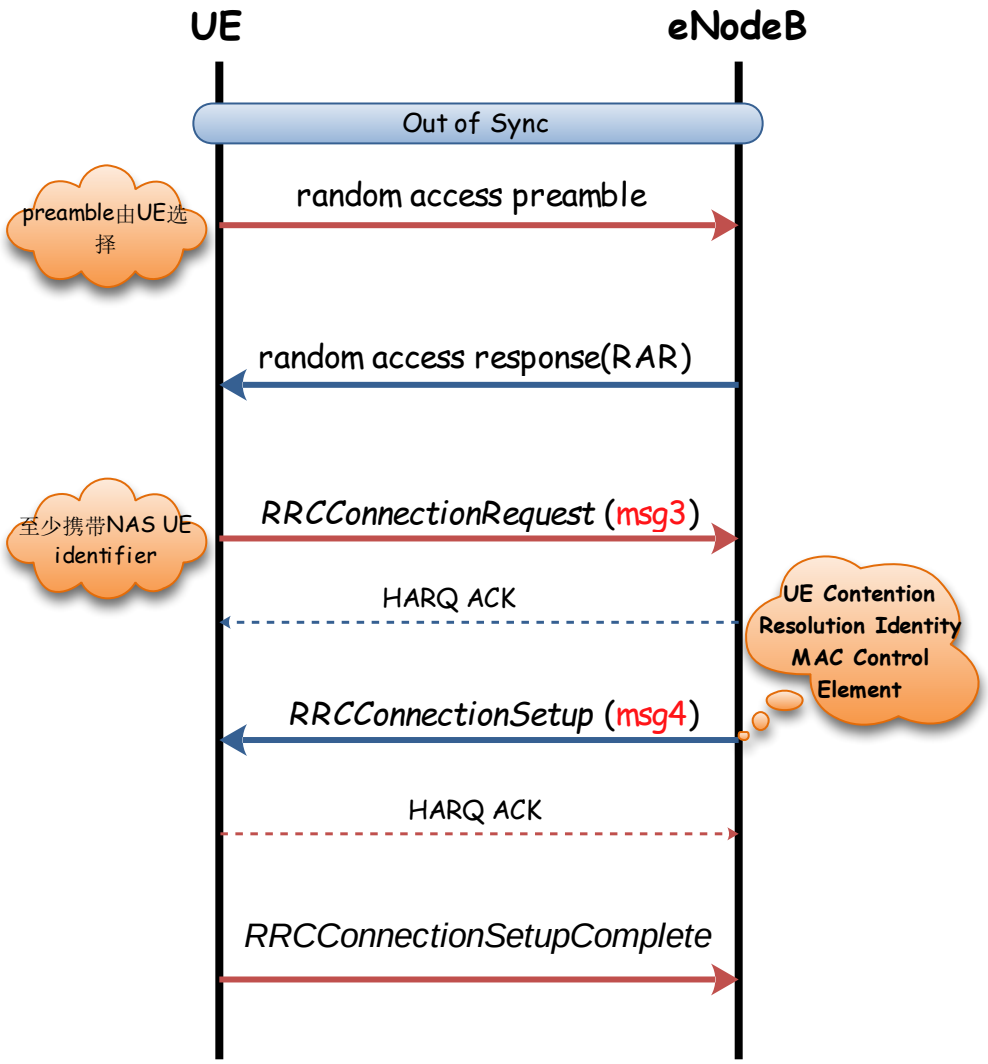


图 21-16：初始接入（initial access）

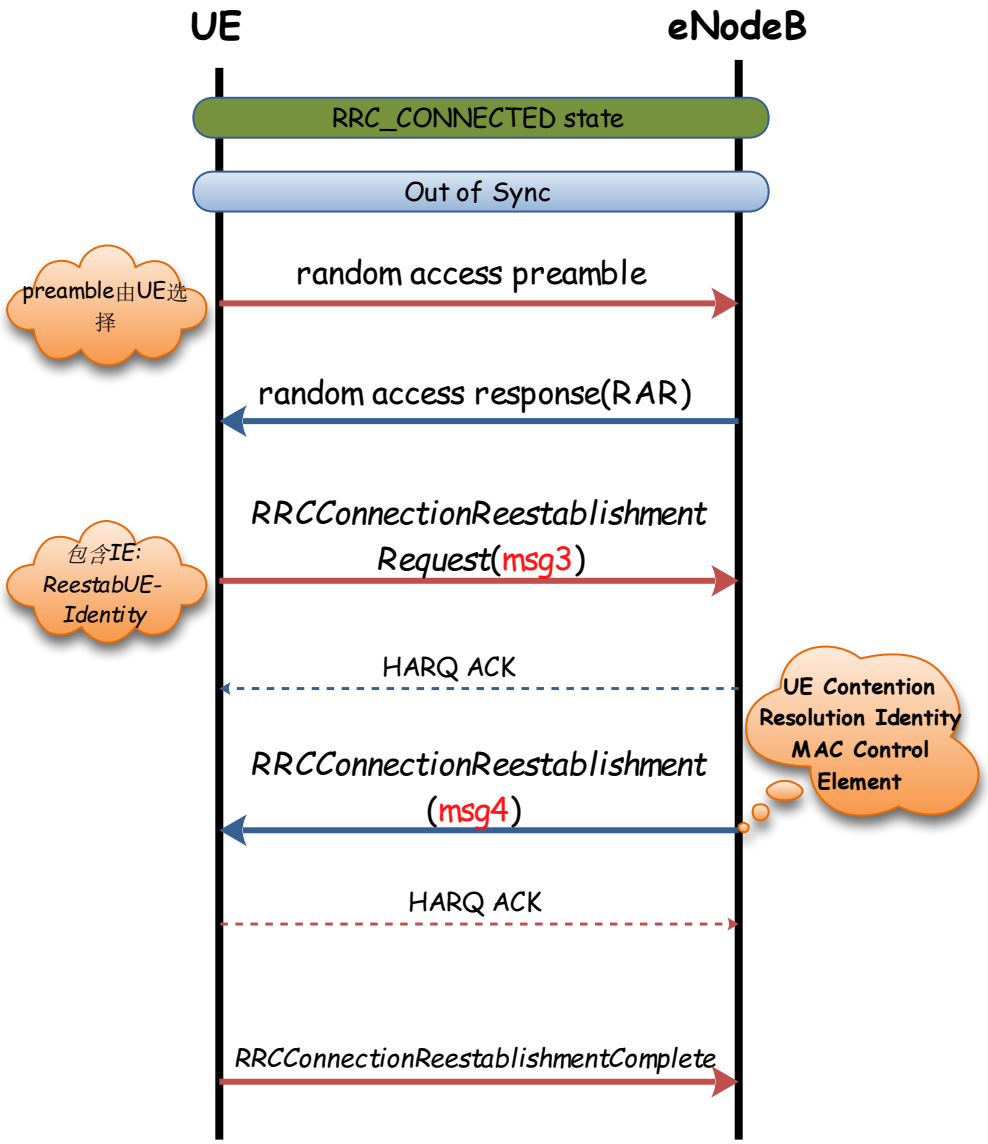


图 21-17：RRC 连接重建（RRC Reestablishment）

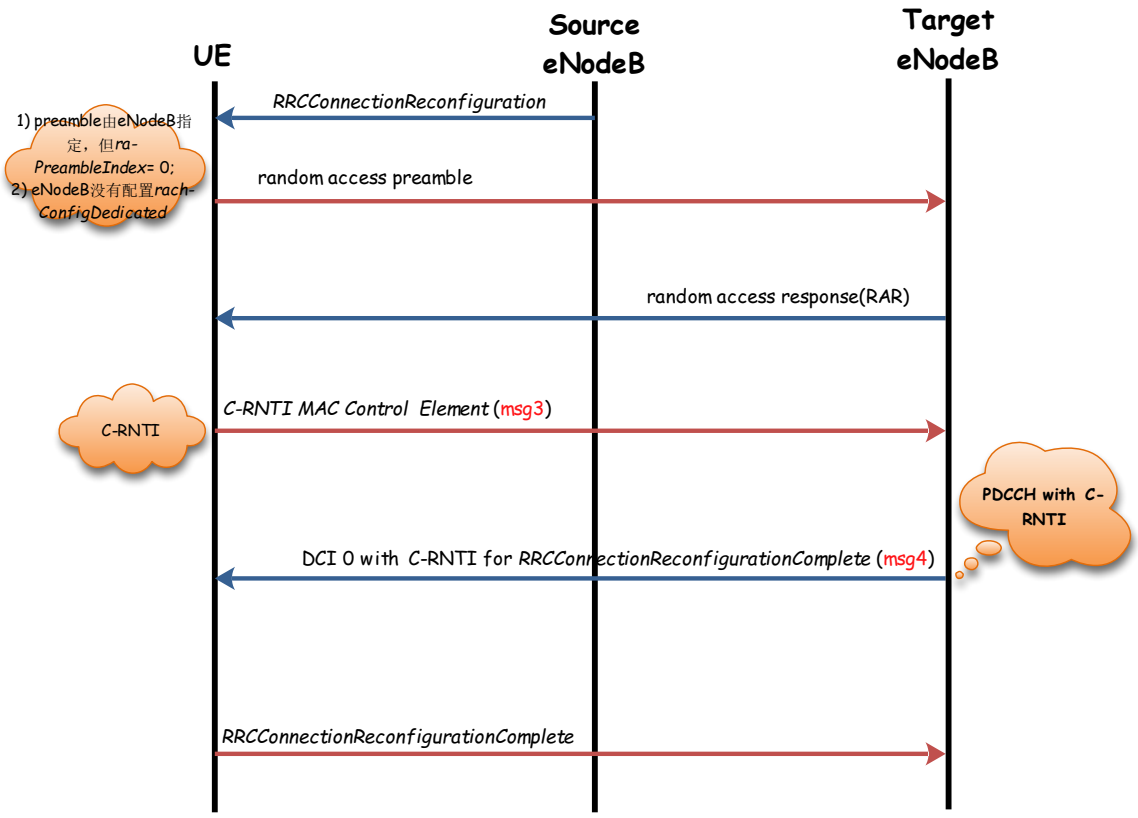


图 21-18：handover（基于竞争）

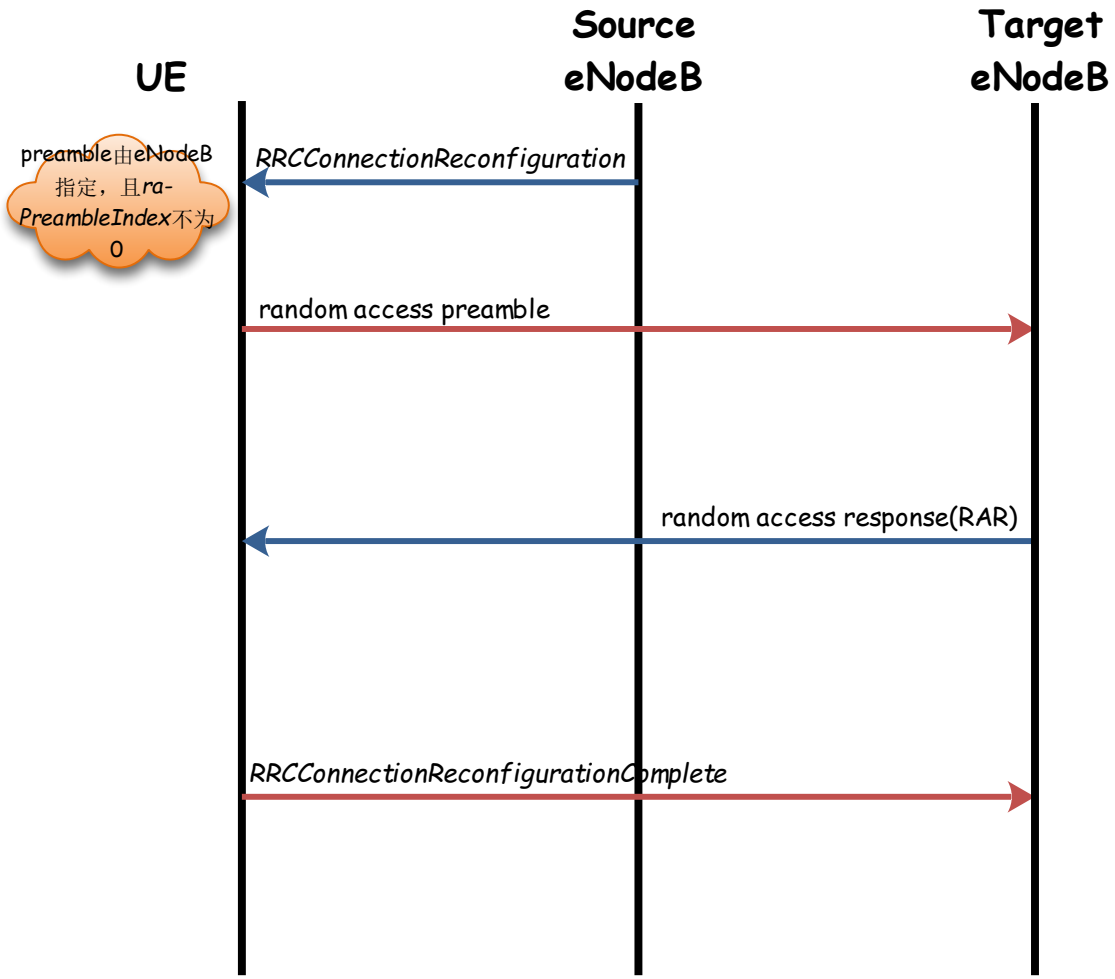


图 21-19：handover（基于非竞争）

对于 handover，如果是基于竞争的随机接入，其 Msg3 应该是 C-RNTI MAC Control Element + BSR MAC Control Element + PHR MAC Control Element。

之前已经介绍过，RAR 中 UL grant 指定的上行资源最小为 56 比特。对于 C-RNTI MAC Control Element，8 比特用于 MAC subheader，16 比特用于 C-RNTI MAC Control Element，共 24 比特，还剩于 32 比特。

我在第 17 章“MAC 复用和逻辑信道优先级”中介绍过，MAC 复用的优先级为： C-RNTI MAC Control Element and UL-CCCH > Regular/Periodic BSR MAC Control Element > PHR MAC Control Element > 除了 UL-CCCH 外的其它逻辑信道的数据 > padding BSR MAC control element。所以在剩余的 32 比特里会优先放置 BSR 和 PHR：

- BSR：8 比特用于 MAC subheader，8 比特用于 BSR MAC Control Element

- PHR：8 比特用于 MAC subheader，8 比特用于 PHR MAC Control Element

但当 RAR 中的 UL grant 指定的上行资源足够大，除了容纳 C-RNTI + BSR + PHR 外，还能够容纳 *RRCConnectionReconfigurationComplement* 消息时 ，则 Msg3 可以为 C-RNTI + BSR + PHR + *RRCConnectionReconfigurationComplement*。

对于 handover，如果 eNodeB 在重配置消息中，根本不带 *rach-ConfigDedicated* 字段（该字段是 OPTIONAL，即可选的），则 UE 发起基于竞争的随机接入。

```
MobilityControlInfo ::=     SEQUENCE {
    targetPhysCellId                    PhysCellId,
    carrierFreq                         CarrierFreqEUTRA                    OPTIONAL,   -- Cond HO-
toEUTRA
    carrierBandwidth                    CarrierBandwidthEUTRA               OPTIONAL,   -- Cond HO-
toEUTRA
    additionalSpectrumEmission          AdditionalSpectrumEmission          OPTIONAL,   -- Cond HO-
toEUTRA
    t304                                ENUMERATED {
                                            ms50, ms100, ms150, ms200, ms500, ms1000,
                                            ms2000, spare1},
    newUE-Identity                      C-RNTI,
    radioResourceConfigCommon           RadioResourceConfigCommon,
    rach-ConfigDedicated                RACH-ConfigDedicated                OPTIONAL,   -- Need OP
    ...
}
```

This field is OPTIONAL, so eNodeB may not config it, then UE will use contention-based RA

图 21-20：*MobilityControlInfo*

21.5 preamble 传输达到最大传输次数的处理

从 UE 的角度上看，随机接入过程可能遇到以下问题而导致随机接入失败：

- UE 没有收到其发送的 preamble 对应的 RAR（没有收到 RAR MAC PDU，或收到的 RAR MAC PDU 中没有对应该 preamble 的 RAR）；
- UE 发送了 Msg3，但没有收到 Msg4；
- UE 收到了 Msg4，但该 UE 不是冲突解决的胜利者。

如果某次随机接入失败了，UE 会重新发起随机接入。在 36.321 中，介绍到一个字段 *preambleTransMax*，该字段指定了 preamble 的最大传输次数。当 UE 发送的 preamble 数超过 *preambleTransMax* 时，协议要求 MAC 层发送一个 random access problem indication 到上层（通常是 RRC 层），但 MAC 层并不会停止发送 preamble。也就是说，MAC 层被设计成“无休止”地发送 preamble，而出现“UE 发送的 preamble 数超过 *preambleTransMax*”时如何处理是由上层（RRC 层）决定的。

也就是说，无论是发生上面介绍的哪种情况，MAC 层都会“无休止”地发送 preamble 以期望能成功接入小区。

在收到 MAC 层的 random access problem indication 后，RRC 层的行为取决于触发随机接入的场景：

场景一：RRC 连接建立。此时 UE 通过 RRC timer T300 来控制，当该 timer 超时（即 RRC 连接建立失败）时，UE 的 RRC 层会停止随机接入过程（此时会重置 MAC，释放 MAC 配置。而从 36.321 的 5.9 节可知，重置 MAC 会停止正在进行的随机接入过程），并通知上层 RRC 连接建立失败。（见 36.331 的 5.3.3.6 节）

场景二：RRC 连接重建。此时通过 RRC timer T301 和 T311 来控制，当该 timer 超时（即 RRC 连接重建失败）时，UE 的 RRC 层会停止 MAC 层的随机接入过程，并进入 RRC_IDLE 态。从 36.331 的 5.3.12 节可以看出，UE 进入 RRC_IDLE 态之前会重置 MAC。（见 36.331 的 5.3.7.6 节和 5.3.7.7 节）

场景三：handover。此时通过 RRC timer T304 来控制，当该 timer 超时（即 handover 失败）时，UE 的 RRC 层会停止之前的随机接入过程（如果之前分配了专用的用于非竞争随机接入的 preamble，则此时认为该 preamble 不再有效），然后触发 RRC 连接重建过程。（见 36.331 的 5.3.5.6 节）

场景四：RRC_CONNECTED 态下，下行数据到达时，上行处于“不同步”状态（包括“定位 UE”的场景）；或上行数据到达时，上行处于“不同步”状态或没有可用的 PUCCH 资源用于 SR 传输。由于这种场景下只有 MAC 层能感知到发生了随机接入过程，而 RRC 层是不知道的，所以当 RRC 层收到 random access problem indication 时，会认为无线链路失效（Radio Link Failure）了，此时如果 AS 安全还没被激活，UE 会进入 RRC_IDLE 态；否则的话，UE 会发起 RRC 连接重建流程（见 36.331 的 5.3.11.3 节）

从上面的介绍可以看出，在场景一、场景二、场景三中，RRC 层会忽略 MAC 层送上来的 random access problem indication，而是根据对应的 timer 来决定是否停止之前的随机接入过程。只有场景四下，RRC 层会对 random access problem indication 进行处理。

【参考资料】

[1] RACH 介绍：http://www.sharetechnote.com/

[2] 一些随机接入问题及解答：http://lteuniversity.com/

[3] 《4G LTE/LTE-Advanced for Mobile Broadband》的 14.3 节

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 17 章

[5] 《Random Access Supervision - Part 1》和《Random Access Supervision - Part 2》by John M.

[6] 《LTE 随机接入(很全).docx》

[7] 《Cell Size Configuration in Random Access Procedure (I) - Preamble Format》和《Cell Size Configuration in RACH (II) – Cyclic Shift》 by Hongyan

[8] 3GPP 协议 v10

TS 36.211 的 5.7 节 Physical random access channel

TS 36.213 的第 6 章 MAC Layer Procedure related to RACH Process.

TS 36.300 的 10.1.5 节 Overall description of RACH Process. 先阅读这个.

TS 36.321 的 5.1 节 MAC Layer process of random access procedure.

TS 36.331 的 6.3.1 节 System information blocks & SystemInformationBlockType2

TS 36.331 的 6.3.2 节 Radio resource control information elements - PRACH-Config

TS 36.331 的 6.3.2 节 Radio resource control information elements - RACH-ConfigCommon

TS 36.331 的 6.3.2 节 Radio resource control information elements - RACH-ConfigDedicated

第22章 寻呼（Paging）

当需要向处于 ECM-IDLE 态（UE 在 MME 中的状态）的 UE 发送下行数据时，MME 会向 UE 所注册的所有 TA（Tracking Area）内的所有 eNodeB 发送一条 PAGING 消息（MME 发往 eNodeB），然后 eNodeB 会通过空口发送一条 Paging 消息（eNodeB 发往 UE）以寻呼 UE。处于 RRC_IDLE 态的 UE 收到 Paging 消息后，可能会发起一个 RRC 连接建立过程以便接收呼叫。

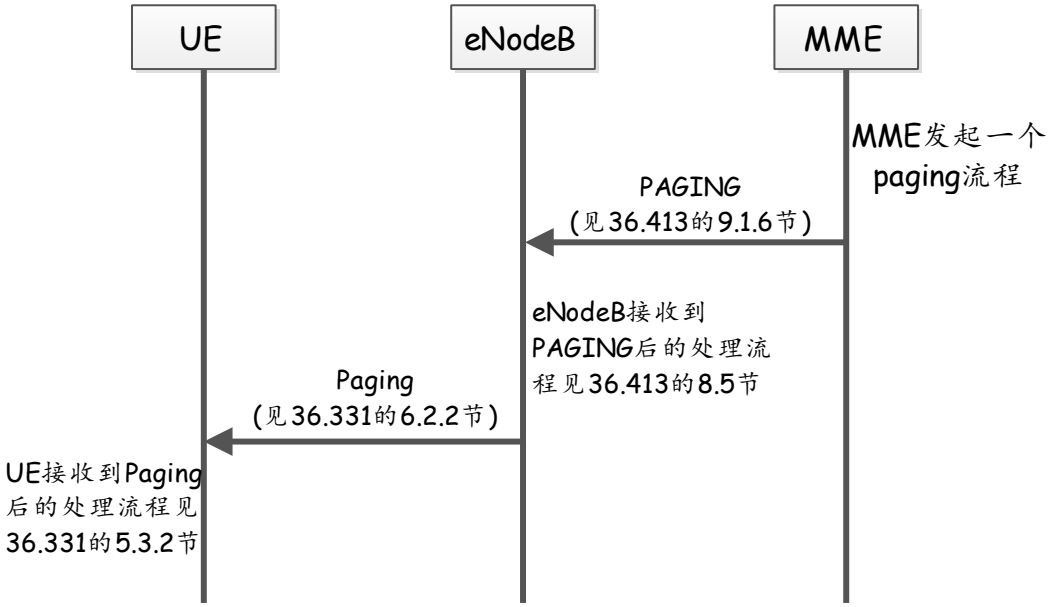


图 22-1：寻呼流程

处于 RRC_IDLE 态的 UE：（1）在 eNodeB 上并没有 RRC 上下文，即 eNodeB 并不知道是否存在该 UE；（2）UE 与 eNodeB 之间不存在 RRC 连接；（3）UE 在所处的 TA（Tracking Area）内已经被分配了一个唯一的标识；（4）UE 已经在 MME 注册，且在 MME 中存在上下文；（5）UE 同核心网之间不存在 NAS 信令的连接。

本章主要关注 Paging 消息在空口的处理，即主要关注 eNodeB 发往 UE 的 Paging 消息处理。

Paging 消息的作用包括：（1）向处于 RRC_IDLE 态的 UE 发送呼叫请求；（2）通知处于 RRC_IDLE 和 RRC_CONNECTED 态的 UE，系统信息发生了变化；（3）通知 UE 开始接收 ETWS primary 通知和/或 ETWS secondary 通知；（4）通知 UE 开始接收 CMAS 通知。

Paging 的后 3 个作用已经在 20.3 节“系统信息有效性和变更通知”中介绍了，本章只介绍 Paging 的第 1 个作用。

PCH（Paging CHannel）用于传输来自 PCCH 逻辑信道的 Paging 信息。PCH 传输信道直接映射到 PDSCH 物理信道，与 DL-SCH 数据共同参与下行调度。Paging 消息的内容如下：

```
Paging ::=                  SEQUENCE {
    pagingRecordList                PagingRecordList                    OPTIONAL,   --
Need ON------被寻呼的UE列表
    systemInfoModification          ENUMERATED {true}                   OPTIONAL,   --
Need ON------如果存在，指示一个除SIB10、SIB11、SIB12外的系统消息变更
    etws-Indication                 ENUMERATED {true}                   OPTIONAL,   --
Need ON------如果存在，指示一个ETWS primary notification和/或ETWS secondary notification
    nonCriticalExtension                Paging-v890-IEs                     OPTIONAL
}

Paging-v890-IEs ::=         SEQUENCE {
    lateNonCriticalExtension        OCTET STRING                            OPTIONAL,   -
- Need OP
    nonCriticalExtension            Paging-v920-IEs                         OPTIONAL
}

Paging-v920-IEs ::=         SEQUENCE {
    cmas-Indication-r9              ENUMERATED {true}                   OPTIONAL,   --
Need ON------如果存在，指示一个CMAS notification
    nonCriticalExtension                SEQUENCE {}                         OPTIONAL    -
- Need OP
}

PagingRecordList ::=                SEQUENCE (SIZE (1..maxPageRec)) OF PagingRecord

PagingRecord ::=                    SEQUENCE {
    ue-Identity                         PagingUE-Identity,------指示被寻呼的UE的NAS ID，唯一地
指定一个UE
    cn-Domain                           ENUMERATED  {ps, cs},
    ...
}

PagingUE-Identity ::=               CHOICE {
    s-TMSI                              S-TMSI,
    imsi                                IMSI, ------International Mobile Subscriber Identity的简称。
一个全球唯一的永久性的用户标识
    ...
}

IMSI ::=                            SEQUENCE (SIZE (6..21)) OF IMSI-Digit

IMSI-Digit ::=                      INTEGER (0..9)
```

只有处于 RRC_IDLE 态的 UE，才会读取 *pagingRecordList* 中的信息。

为了接收来自 eNodeB 的 Paging 消息，处于 RRC_IDLE 态的 UE 会监听使用 P-RNTI（值为 0xFFFE）加扰的 PDCCH。

Paging 支持非连续接收（Discontinuous Reception，DRX），使得处于 RRC_IDLE 态的 UE 只在预先定义好的时间段内“醒来”以接收 Paging 消息，而在其它时间可以保持“休眠”状态，这样就能够降低功耗，提升 UE 的电池使用时间。

注意：本章介绍的是处于 RRC_IDLE 态下的 UE 的 DRX 处理。这与处于 RRC_CONNECTED 态下的 UE 的 DRX 处理是不同的（具体见第 16 章关于“DRX”的介绍）。

Paging 的时频资源

Paging 消息所占的频域资源是由 P-RNTI 加扰的 PDCCH（见 36.213 的 Table 7.1-2）指定的。

而在时域上，UE 只会在其 Paging 周期内的某个特定帧（称为 PF）的特定子帧（称为 PO）去尝试接收 Paging 消息。

PO（Paging Occasion）是一个子帧，在该子帧上可能会有使用 P-RNTI 加扰，并指示 Paging 消息的 PDCCH。当使用了 DRX，UE 在每个 DRX cycle 上只需要检测 1 个 PO，也就是说，对应每个 UE，在每个 Paging 周期内只有 1 个子帧可用于发送 Paging。DRX cycle 与 Paging 周期是同一概念。

PF（Paging Frame）是一个无线帧，该帧可能包含一个或多个 PO。

PF 是满足如下公式的系统帧：

$$SFN \bmod T = (T \operatorname{div} N) * (UE_ID \bmod N)$$

使用索引 i_s 查下面的 2 张表（见 36.304 的 7.2 节），可得到 PO。其中 i_s 通过下面的公式得到：

$$i_s = \operatorname{floor}(UE_ID/N) \bmod Ns$$

FDD:

Ns	PO when i_s=0	PO when i_s=1	PO when i_s=2	PO when i_s=3
1	9	N/A	N/A	N/A
2	4	9	N/A	N/A
4	0	4	5	9

TDD (all UL/DL configurations):

Ns	PO when i_s=0	PO when i_s=1	PO when i_s=2	PO when i_s=3
1	0	N/A	N/A	N/A
2	0	5	N/A	N/A
4	0	1	5	6

对于某个 UE 来说，PF 就是用于发送 Paging 的系统帧，PO 就是该 PF 内用于发送 Paging 的子帧。

表 22-1：PF/PO 公式的相关参数说明

参数	说明
T_c（以系统帧为单位）	小区特定的默认 DRX cycle。通过 SIB2 的 IE: *PCCH-Config* -> *defaultPagingCycle* 来配置的
T_{ue}（以系统帧为单位）	UE 特定的 DRX cycle。由 MME 通过 IE: Paging DRX 来配置的（见 36.413 的 9.2.1.16 节）
T（以系统帧为单位）	UE 最终使用的 DRX cycle。 $T = \min(T_c, T_{UE})$。如果 MME 没有配置 UE 特定的 DRX cycle，则使用默认的 DRX cycle，即 *defaultPagingCycle*
nB	取值：4T, 2T, T, T/2, T/4, T/8, T/16, T/32。实际上 nB 表示在每个 DRX cycle 内包含了多少个 PO。 （通过 SIB2 的 IE: *PCCH-Config* -> *nB* 配置）
N	$N = \min(T, nB)$。实际上 N 表示在每个 DRX cycle 内包含了多少个 PF
Ns	$Ns = \max(1, nB/T)$。实际上 Ns 表示在每个 PF 内包含了多少个 PO
UE_ID	IMSI mod 1024

通过上面的介绍可以看出：T div N 相当于把一个 DRX cycle 等分成 N 份后，每一份包含的系统帧数；UE_ID mod N 相当于取 N 等份里的第“UE_ID mod N”（取值范围为 0 ~ N-1）份，而 PF 为其中的第一个系统帧。

36.331 中的 PCCH-Config 定义：

```
PCCH-Config ::=                     SEQUENCE {
    defaultPagingCycle                  ENUMERATED {
                                            rf32, rf64, rf128, rf256},------默认的DRX cycle，
对应36.304中定义的“T”。其中rf32对应32个系统帧，rf64对应64个系统帧，以此类推
    nB                                  ENUMERATED {
```

```
                                                fourT, twoT, oneT, halfT, quarterT,
oneEighthT,
                                                oneSixteenthT, oneThirtySecondT}------nB用于推导
PF和PO（见36.304）。fourT对应4 * T，twoT对应2 * T，以此类推
}
```

36.413 中 Paging DRX 定义：

IE/Group Name	Presence	Range	IE type and reference	Semantics description	Criticality	Assigned Criticality
Paging DRX	M		ENUMERATED(32, 64, 128, 256, …)		-	

IMSI 是一连串的 10 进制数字（0..9），IMSI 在公式中被解释成 10 进制整数，其中第 1 位为最高位，以此类推。例如：IMSI = 12（digit 1 = 1，digit 2 = 2），计算时该 IMSI 被解释成 10 进制数“12”，而不是“1 * 16 + 2 = 18”。

如果 UE 没有 IMSI，例如 UE 因为没有 USIM 而只能进行紧急呼叫，则该 UE 的 UE_ID = 0。

从 UE 的角度上看，短的 DRX cycle 会增加功耗，这是因为 UE 需要频繁地醒来以监听 P-RNTI 加扰的 PDCCH。

接下来，我们通过举几个例子来说明如何计算 UE 的 PF 和 PO。

例 1：假设 FDD 下，某个 UE 的配置如表 22-2 所示：

表 22-2：某个 UE 的 Paging 配置举例

UE 的配置	
IMSI	404685505601234
T_c	128
T_{ue}	32
T	$T = \min(T_c, T_{UE}) = 32$
nB	= T/2 = 16
计算 PF 和 PO	

UE_ID	IMSI mod 1024 = 722
N	$N = \min(T,\ nB) = 16$
Ns	$Ns = \max(1,\ nB/T) = 1$
i_s	i_s = floor(UE_ID/N) mod Ns = 0
PF	满足 SFN mod T= (T div N)*(UE_ID mod N) 。 计算出 PF 需满足：SFN mod 32=4
PO	通过 Ns 和 i_s 查表可知，子帧号为 9

从计算结果可以看出：

- 该 UE 需要每 320 ms（T = 32 * 10ms）醒来一次，并尝试接收 Paging 消息（即该 UE 的 DRX cycle 为 32 个系统帧）。
- 在该 UE 的 DRX cycle 内，每 2 个系统帧（ $N = 16$ ）中会有一个 PF 可用于 Paging。该 UE 对应的 DRX cycle 内有 16 个 PO（nB = 16）。不同的 UE 基于其不同的 UE_ID，可以在这 16 个 PO 中选择 1 个来接收 Paging。
- 每个 PF 内只有 1 个 PO（ $Ns = 1$ ）。
- 根据预先定义好的用于 FDD 的样式，对于 $Ns = 1$ ，i_s=0，PF 内的子帧 9 用于 PO。

对应表 22-2 中的 UE，其 PF、PO 如图 22-2 所示：

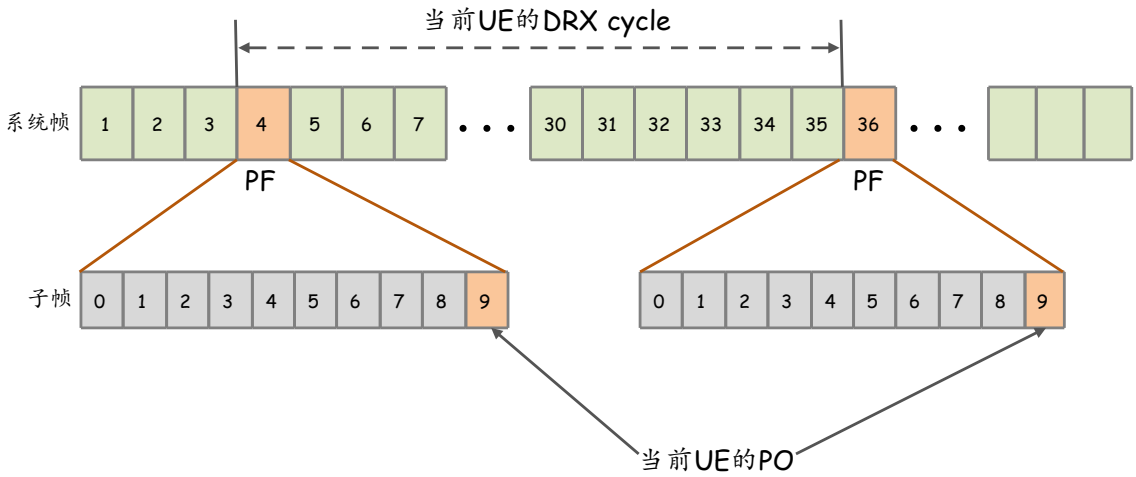


图 22-2：IMSI=404685505601234、$T = 32$ 、nB = 16 的 UE

例 2：假设 FDD 下，某个 UE 的配置如表 22-3 所示：

表 22-3：某个 UE 的 Paging 配置举例

UE 的配置	
IMSI	404685505601234
T_c	128
T_{ue}	32
T	$T = \min(T_c,\ T_{UE}) = 32$
nB	= 4T = 128
计算 PF 和 PO	
UE_ID	IMSI mod 1024 = 722
N	$N = \min(T,\ nB) = 32$
Ns	$Ns = \max(1,\ nB/T) = 4$
i_s	i_s = floor(UE_ID/N) mod Ns = 2
PF	满足 SFN mod T= (T div N)*(UE_ID mod N)。计算出 PF 需满足：SFN mod 32=18
PO	通过 Ns 和 i_s 查表可知，子帧号为 5

从计算结果可以看出：

- 该 UE 需要每 320 ms（T = 32 * 10ms）醒来一次，并尝试接收 Paging 消息（即该 UE 的 DRX cycle 为 32 个系统帧）。
- 在该 UE 的 DRX cycle 内，每个系统帧都是 1 个 PF（$N = 32$）。该 UE 对应的 DRX cycle 内有 128 个 PO（nB = 128）。不同的 UE 基于其不同的 UE_ID，可以在这 128 个 PO 中选择 1 个来 接收 Paging。

- 每个 PF 内有 4 个 PO（ $Ns = 4$ ）。
- 根据预先定义好的用于 FDD 的样式，对于 $Ns = 4$ ，i_s=2，PF 内的子帧 5 用于 PO。

对应表 22-3 中的 UE，其 PF、PO 如图 22-3 所示：

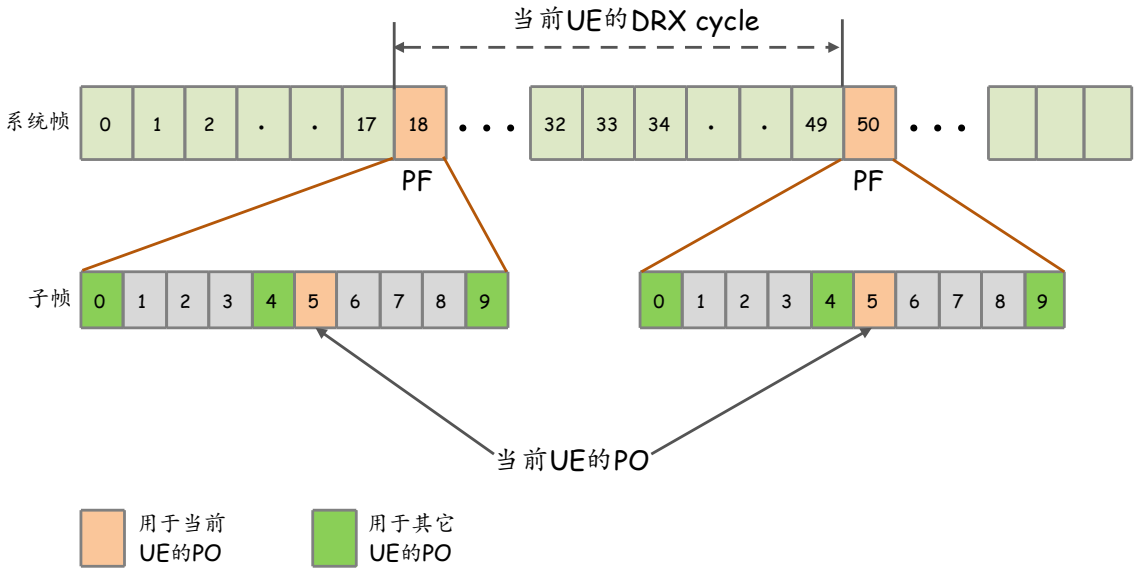


图 22-3：IMSI=404685505601234、$T = 32$ 、nB = 128 的 UE

UE 在 DRX cycle 内并不知道上行 timing，所以不能进行 HARQ 确认，也不能进行 HARQ 软合并。即 Paging 不支持 HARQ。

Paging 消息在 SRB0 上传输，并使用 RLC-TM 模式。

eNodeB 侧同样需要计算特定 UE 的 DRX cycle、PF 和 PO，这样 eNodeB 才能只在对应的 PO 上给 UE 发送 Paging 消息。

小结：处于 RRC_IDLE 态的 UE 在 DRX cycle 内的绝大部分时间里是休眠的，而只在对应 PF 的对应 PO 上醒来，去监听使用 P-RNTI 加扰的 PDCCH。如果 UE 在 PO 上检测到使用 P-RNTI 加扰的 PDCCH，UE 会去读取 *pagingRecordList* 中的每一个 *PagingRecord*，*PagingRecord* 中包含了被寻呼的 UE 的标志 *ue-Identity*。如果 UE 发现自己的 UE 标志与某个 *ue-Identity* 一致，就会把 *ue-Identity*

和 *cn-Domain* 发往上层继续处理。如果 UE 没有找到一个与其 UE 标志一致的 *ue-Identity*，UE 会丢弃接收到的 Paging 消息，并进入休眠。

【参考资料】

[1] TS 36.331 的 5.3.2 节 System information acquisition

[2] TS 36.413 的 8.5 节 Paging

[3] TS 36.304 的第 7 章 Paging

[4] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 3.4 节

[5] 《4G LTE/LTE-Advanced for Mobile Broadband》的 14.4 节

[6] 《Discontinuous Reception (DRX) in RRC_IDLE: Part 1》和

《Discontinuous Reception (DRX) in RRC_IDLE: Part 2》

第23章 载波聚合（Carrier Aggregation）

为了满足 LTE-A 下行峰速 1 Gbps，上行峰速 500 Mbps 的要求，需要提供最大 100 MHz 的传输带宽，但由于这么大带宽的连续频谱的稀缺，LTE-A 提出了载波聚合的解决方案。

载波聚合（Carrier Aggregation，CA）是将 2 个或更多的载波单元（Component Carrier， CC）聚合在一起以支持更大的传输带宽（最大为 100MHz）。

实际上，每个载波单元对应一个独立的小区。通常可以将 1 个载波单元等同于 1 个小区。

每个载波单元的最大带宽为 20 MHz。

为了高效地利用零碎的频谱，载波聚合支持不同载波单元之间的聚合（如图 23-1 所示）

- 相同或不同带宽的载波单元
- 同一频带内，邻接或非邻接的载波单元
- 不同频带内的载波单元

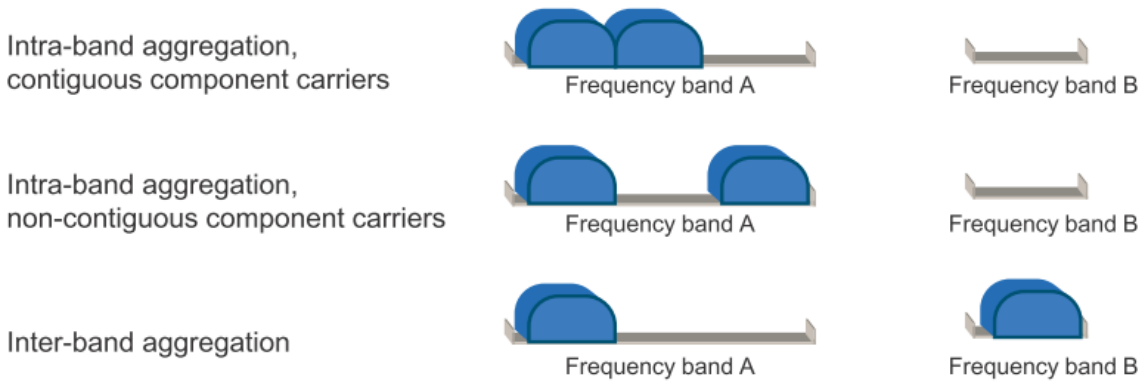


图 23-1：载波聚合

从基带（baseband）实现角度来看，这几种情况是没有区别的。这主要影响射频单元实现的复杂性。

载波聚合对核心网是不可见的。

使用载波聚合的另一个原因来自于对异构网络（heterogeneous network）的支持。后续会在 23.5 节中对异构网络进行介绍。

Rel-10 中的所有载波单元都是后向兼容的（backward-compatible），即同时支持 Rel-8/Rel-9 的 UE。

- R10 版本 UE 支持载波聚合，能够同时从多个载波单元接收数据，以及往多个载波单元发送数据；
- R8 版本 UE 只支持在一个 serving cell 内，从一个载波单元接收数据，以及往一个载波单元发送数据。

简单地说，没有配置载波聚合的 UE 只能与一个小区进行收发数据的操作。而配置了载波聚合之后，UE 能够同时与多个小区进行收发数据的操作，因此能够提高 UE 的吞吐量。

23.1 PCell / SCell / Serving Cell

PCell（Primary Cell，主小区）是 UE 进行初始连接建立的小区，或进行 RRC 连接重建的小区，或是在 handover 过程中指定的主小区。PCell 负责与 UE 之间的 RRC 通信。PCell 对应的载波单元称为 PCC（Primary Component Carrier）。其中，PCell 的下行载波称为 DL PCC，PCell 的上行载波称为 UL PCC。

SCell（Secondary Cell，辅小区）是在 RRC 重配置时添加的，用于提供额外的无线资源，SCell 与 UE 之间不存在任何 RRC 通信。SCell 对应的载波单元称为 SCC（Secondary Component Carrier）。其中，SCell 的下行载波称为 DL SCC，SCell 的上行载波称为 UL SCC。

PCell 是在连接建立（connection establishment）时确定的。SCell 是在初始安全激活流程（initial security activation procedure）之后，通过 RRC 连接重配置消息 *RRCConnectionReconfiguration* 添加/修改/释放的。

Serving cell，顾名思义，是为 UE 提供服务（上下行传输）的小区。如果 UE 处于 RRC_CONNECTED 态但并未配置 CA，则该 UE 只有一个 serving cell，即 PCell；如果 UE 处于 RRC_CONNECTED 态且配置了 CA，则该 UE 的 serving cell 集合包括 PCell 和所有的 SCell。

也就是说，serving cell 既可以指代 PCell，也可以指代 SCell。PCell 或 SCell 同时也是一个 serving cell。

配置了 CA 的 UE 可与 1 个 PCell 和至多 4 个 SCell（见 36.331 的 6.4 节的 *maxSCell-r10*）相连。某个 UE 的 serving cell 集合至多包含 5 个 serving cell（见 36.331 的 6.4 节的 *maxServCell-r10*）。

CA 是 UE 级的特性，不同的 UE 可能有不同的 PCell/SCell 以及 serving cell 集合。同一个小区，对某个 UE 而言可能是 PCell，但对另一个 UE 而言可能是 SCell。

配置了 CA 的 UE 在所有的 serving cell 内使用相同的 C-RNTI。

对于某个 UE 而言，每个载波单元（小区）都有一个对应的索引。PCell 的索引固定为 0，而每个 SCell 的索引是通过 UE 特定的 IE：*SCellToAddMod-r10 -> sCellIndex-r10* 发给 UE 的。

在 3GPP 协议中，还会涉及到 serving cell *c* 的概念，这里的 *c* 代表的是配置的第 *c* 个 serving cell 的概念。对于 PCell，*c* 值固定为 0；对于 SCell，*c* 与 *sCellIndex-r10* 存在一一对应关系，并按照 *sCellIndex-r10* 由小到大的顺序映射（见 36.212 的 5.2.3.1 节）。

PCell: c = 0

SCell:	sCellIndex-r10	1	3	4	6
	c	1	2	3	4

图 23-2：cell index 与 c 的映射关系举例

图 23-2 是 cell index 与 c 的映射关系的一个例子。假设 eNodeB 通过 *SCellToAddModList-r10* 按顺序给 UE 配置了 *sCellIndex-r10* 为 1、3、4、6 的 SCell，则这些 SCell 分别对应第 1、2、3、4（对应上面介绍的 *c*）个 serving cell。而 PCell 对应第 0（即 *c* = 0）个 serving cell。

在 Rel-10 中，某个 UE 聚合的载波单元通常来自同一个 eNodeB 且这些载波单元是同步的。

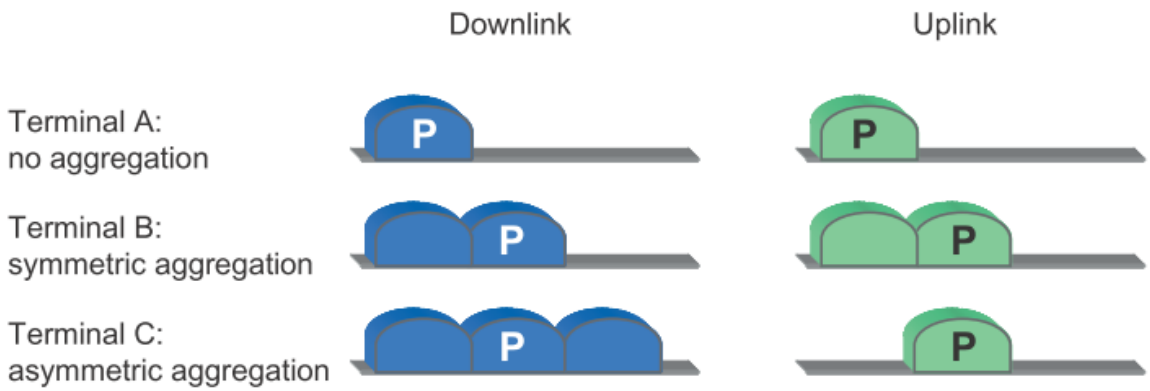


图 23-3：CA 配置举例（“P”代表 PCC）

载波单元的配置需要满足如下要求：

- 所有的载波单元需要有相同的双工模式（duplex scheme）；
- 下行载波单元的个数根据该 UE 的下行聚合能力来配置；
- 上行载波单元的个数根据该 UE 的上行聚合能力来配置；
- 对于某个 UE 而言，配置的上行载波单元数不能大于下行载波单元数。这是因为，如果某个 SCell 的上行资源被使用，则该 SCell 的下行资源也一定被使用。反过来，如果某个 SCell 的下行资源被使用，该 SCell 的上行资源却未必被使用。
- 在典型的 TDD 部署中，上行和下行的载波单元个数是一样的，并且不同载波单元的上下行配置也应该是相同的。但只要下行到上行的切换时间（downlink-uplink switch time）足够大，特殊子帧的配置可以不同。

连续的载波单元之间的中心频率间隔必须是 300 kHz 的整数倍。这是为了兼容 Rel-8 的 100 kHz frequency raster，并保证子载波的 15 kHz spacing，从而取的最小公倍数（详见 36.300 的 5.5 节）。

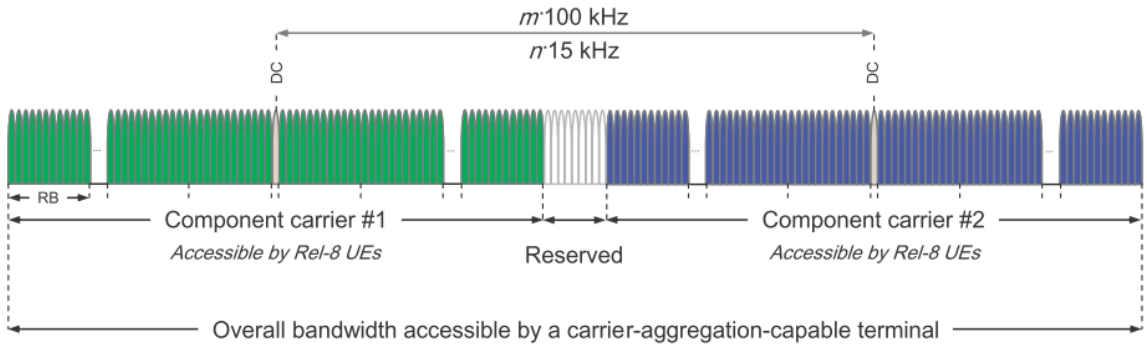


图 23-4：不同载波单元间中心频率的间隔

与非 CA 的场景类似，通过 *SystemInformationBlockType2* 的 *ul-CarrierFreq* 和 *ul-Bandwidth* 字段，可以指定下行 PCell 对应的上行 PCC（仅 FDD 需配置该字段）。这样做的目的是无需明确指定，就知道通过下行传输的某个 UL grant 与哪个上行 CC 相关联。

Rel-10 中，CA 支持的频带见 36.101 的 Table 5.5A-1 和 Table 5.5A-2。

Table 5.5A-1: Intra-band contiguous CA operating bands

E-UTRA CA Band	E-UTRA Band	Uplink (UL) operating band BS receive / UE transmit F_{UL_low} – F_{UL_high}			Downlink (DL) operating band BS transmit / UE receive F_{DL_low} – F_{DL_high}			Duplex Mode
CA_1	1	1920 MHz	–	1980 MHz	2110 MHz	–	2170 MHz	FDD
CA_40	40	2300 MHz	–	2400 MHz	2300 MHz	–	2400 MHz	TDD

Table 5.5A-2: Inter-band CA operating bands

E-UTRA CA Band	E-UTRA Band	Uplink (UL) operating band BS receive / UE transmit F_{UL_low} – F_{UL_high}			Downlink (DL) operating band BS transmit / UE receive F_{DL_low} – F_{DL_high}			Duplex Mode
CA_1-5	1	1920 MHz	–	1980 MHz	2110 MHz	–	2170 MHz	FDD
	5	824 MHz	–	849 MHz	869 MHz	–	894 MHz	

Rel-10 中，CA 中每个载波单元支持的带宽类型见 36.101 的 Table 5.6A.1-1 和 Table 5.6A.1-2。

Table 5.6A.1-1: E-UTRA CA configurations and bandwidth combination sets defined for intra-band contiguous CA

E-UTRA CA configuration / Bandwidth combination set				
E-UTRA CA configuration	Component carriers in order of increasing carrier frequency Allowed channel bandwidths for carrier [MHz]	Component carriers in order of increasing carrier frequency Allowed channel bandwidths for carrier [MHz]	Maximum aggregated bandwidth [MHz]	Bandwidth combination set
CA_1C	15	15	40	0
	20	20		
CA_40C	10	20	40	0
	15	15		
	20	10, 20		
NOTE 1: The CA configuration refers to an operating band and a CA bandwidth class specified in Table 5.6A-1 (the indexing letter). Absence of a CA bandwidth class for an operating band implies support of all classes. NOTE 2: For the supported CC bandwidth combinations, the CC downlink and uplink bandwidths are equal				

Table 5.6A.1-2: E-UTRA CA configurations and bandwidth combination sets defined for inter-band CA

E-UTRA CA configuration / Bandwidth combination set									
E-UTRA CA configuration	**E-UTRA bands**	**1.4 MHz**	**3 MHz**	**5 MHz**	**10 MHz**	**15 MHz**	**20 MHz**	**Maximum aggregated bandwidth [MHz]**	**Bandwidth combination set**
CA_1A-5A	1				Yes			20	0
	5				Yes				
NOTE 1: The CA Configuration refers to a combination of an operating band and a CA bandwidth class specified in Table 5.6A-1 (the indexing letter). Absence of a CA bandwidth class for an operating band implies support of all classes. NOTE 2: For each band combination, all combinations of indicated bandwidths belong to the set NOTE 3: For the supported CC bandwidth combinations, the CC downlink and uplink bandwidths are equal									

eNodeB 可以通过 RRC 消息 *UECapabilityEnquiry* 询问 UE 的能力，UE 会通过 *UECapabilityInformation* 将自己的无线接入能力告诉 eNodeB。UE 会在 *UECapabilityInformation* 中将 *ca-BandwidthClassUL-r10* 和 *ca-BandwidthClassDL-r10* 信息发送给 eNodeB，该信息指明了 UE 支持的最大载波单元数，以及可聚合的 RB 数等。其值对应 36.101 的 Table 5.6A-1。

Table 5.6A-1: CA bandwidth classes and corresponding nominal guard bands

CA Bandwidth Class	Aggregated Transmission Bandwidth Configuration	Maximum number of CC	Nominal Guard Band BW_{GB}
A	$N_{RB,agg} \le 100$	1	$0.05BW_{Channel(1)} - 0.5\Delta f_1$
B	$N_{RB,agg} \le 100$	2	FFS
C	$100 < N_{RB,agg} \le 200$	2	$0.05\ max(BW_{Channel(1)},BW_{Channel(2)}) - 0.5\Delta f_1$
D	$200 < N_{RB,agg} \le [300]$	FFS	FFS
E	$[300] < N_{RB,agg} \le [400]$	FFS	FFS
F	$[400] < N_{RB,agg} \le [500]$	FFS	FFS
NOTE 1: $BW_{Channel(1)}$ and $BW_{Channel(2)}$ are channel bandwidths of two E-UTRA component carriers according to Table 5.6-1 and $\Delta f_1 = \Delta f$ for the downlink with Δf the subcarrier spacing while $\Delta f_1 = 0$ for the uplink.			

FFS：For Further Study，供进一步研究。

23.2 添加/修改/删除 SCell

添加/修改/删除 SCell 主要是通过 RRC 连接重配置命令来完成的，添加/修改的 SCell 通过 *sCellToAddModList-r10* 指定，删除的 SCell 通过 *sCellToReleaseList-r10* 指定。

```
RRCConnectionReconfiguration-v1020-IEs ::= SEQUENCE {
    sCellToReleaseList-r10              SCellToReleaseList-r10          OPTIONAL,   --
Need ON-------需要删除的SCell列表
```

```
    sCellToAddModList-r10               SCellToAddModList-r10           OPTIONAL,    --
Need ON-------需要添加/修改的SCell列表
    nonCriticalExtension                SEQUENCE {}                     OPTIONAL     --
Need OP
}
```

23.2.1 添加/修改 SCell：（见 36.331 的 5.3.10.3b 节）

对于包含在 *SCellToAddModList* 的每个 *SCellIndex*，

- 如果 UE 当前配置中没有该 *SCellIndex*（**添加**）

  1）添加该 SCell，根据 *SCellToAddMod-r10* 的配置的添加对应 SCell。

  2）将该 SCell 的状态设置为 deactivated 态（见 23.3 节介绍）。

- 如果 UE 当前配置中有该 SCellIndex（**修改**）

  1）根据接收到的 *SCellToAddMod-r10* 中的 *radioResourceConfigDedicatedSCell* 来修改 SCell 的配置。

23.2.2 删除 SCell：（见 36.331 的 5.3.10.3a）

删除 SCell 有 2 种触发方式：

1）收到 *SCellToReleaseList*：删除当前 UE 配置中与 *SCellToReleaseList* 携带的 *SCellIndex* 相等的 SCell；

2）RRC 连接重建：删除当前 UE 配置中的所有 SCell。

23.3 SCell 激活/去激活（Activation/Deactivation）

为了更好地管理配置了 CA 的 UE 的电池消耗，LTE 提供了 SCell 的激活/去激活机制（见 36.300 的 11.2 节、36.321 的 5.13 节以及 36.213 的 4.3 节）。

PCell 是不支持激活/去激活的。也就是说， UE 的 PCell 总是处于激活态。

当 SCell **激活**时，UE 可以在对应的载波单元内：

- 发送 SRS；
- 上报 CQI/PMI/RI/PTI；
- 检测用于该 SCell 和在该 SCell 上传输的 PDCCH。

当 SCell **去激活**时，UE 在对应的载波单元内 ：

- 不发送 SRS；
- 不进行 CQI 测量，也不上报 CQI/PMI/RI/PTI；
- 不传输上行数据（包括 pending 的重传数据）；
- 不检测用于该 SCell 和在该 SCell 上传输的 PDCCH；
- 可以用于上行功控的路损测量的参考，但是测量的频率降低，以便降低功率消耗。

重配消息中不带 mobility 控制信息时，新添加到 serving cell 集合中的 SCell 初始为去激活的（deactivated）；而原本就在 serving cell 集合中 SCell（未变化或重配置），不改变他们原有的激活状态。

重配消息中带 mobility 控制信息时（例如 handover 时），所有的 SCell 均为去激活态。

UE 的激活/去激活机制是基于 Activation/Deactivation MAC Control Element 和 deactivation timers（去激活定时器）的结合来实现的。

基于 MAC CE 的 SCell 激活/去激活操作是由 eNodeB 控制的，基于去激活定时器的 SCell 去激活操作是由 UE 控制。

1、基于 MAC CE 的 SCell 激活/去激活

Activation/Deactivation MAC CE 的格式如 36.321 的 Figure 6.1.3.8-1 所示。其对应的 MAC subheader 的 LCID 为 11011。

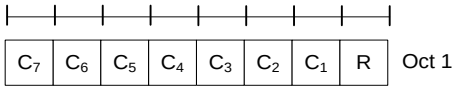


Figure 6.1.3.8-1: Activation/Deactivation MAC control element

C_i 对应 *sCellIndex-r10* 配置为 i 的 SCell 的激活/去激活状态。如果 C_i 设置为 1，表示对应的 SCell 被激活；如果 C_i 设置为 0，表示对应的 SCell 被去激活。

3GPP 协议并没有明确指出 eNodeB 应该在什么时候激活/去激活 SCell，这取决于厂商的实现（通常跟 UE 的负载相关）。

2、基于去激活定时器的 SCell 去激活

UE 会为每个 SCell 维护一个去激活定时器 *sCellDeactivationTimer*，对应某个 UE 的所有 SCell，*sCellDeactivationTimer* 的值是相同的。该值可以配置成“infinity”，即去使能基于 timer 的 SCell 去激活，此时 UE 无法控制 SCell 的去激活。

当在去激活定时器指定的时间内，UE 没有在对应的 SCell 上收到数据或 PDCCH 消息，则该 SCell 将去激活。这也是 UE 可以自动将某 SCell 去激活的唯一情况。

SCell 的信道质量并不作为 UE 判断是否去激活的条件。

注：UE 不能激活 SCell，只有 eNodeB 可以激活某个 UE 的 SCell。UE 只有在配置的 *sCellDeactivationTimer* 不为“infinity”的情况下，才能控制是否将 SCell 去激活。

当 UE 在子帧 n 收到某个 SCell 的激活命令时，对应的激活操作将在 n + 8 子帧启动。

当 UE 在子帧 n 收到去激活命令或某个 SCell 的 deactivation timer 超时，除了 CSI 报告对应的操作（停止上报）在 n+8 子帧完成外，其它操作必须在 n+8 子帧内完成。（见 36.213 的 4.3 节）

23.4 CA 的部署场景

本节将介绍载波聚合的 5 种典型部署场景。假设 F1 和 F2 是 2 个可被聚合的载波，分别对应不同的小区。（见 36.300 的 Annex J）

场景#1

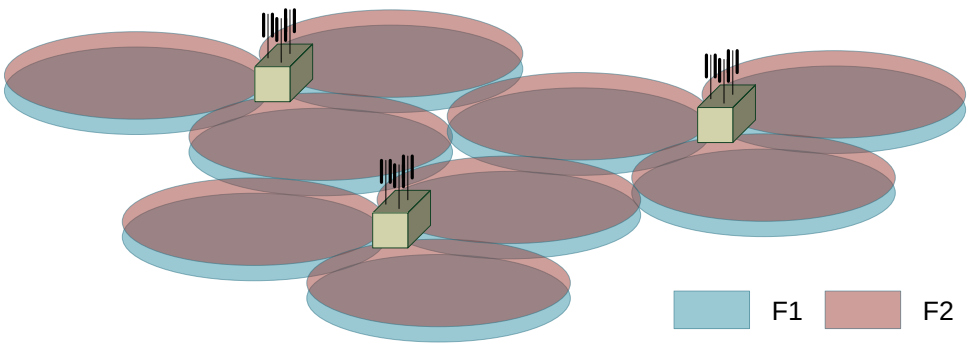


图 23-5：场景#1

场景#1 中，F1 和 F2 的位置相同、相互重叠，且提供基本相同的覆盖范围（coverage）。二者都提供足够的覆盖且都支持移动性（mobility）。可能的场景是 F1 和 F2 载波是同一频带内的载波，例如 2 GHz、800 MHz 等。

场景#2

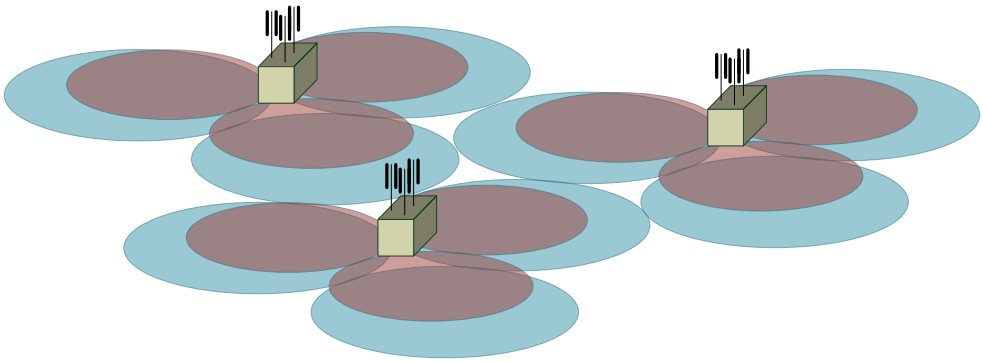


图 23-6：场景#2

场景#2 中，F1 和 F2 小区的位置相同、相互重叠，但由于 F2 的路径损耗比 F1 更大，其覆盖范围比 F1 更小。只有 F1 提供足够的覆盖，而 F2 被用于进一步提高吞吐量（throughput）。此时移动性是基于 F1 的覆盖进行的。可能的场景是 F1 和 F2 载波是不同频带上的载波，例如 F1 = {800 MHz ，2 GHz} 并且 F2 = {3.5 GHz}。

场景#3

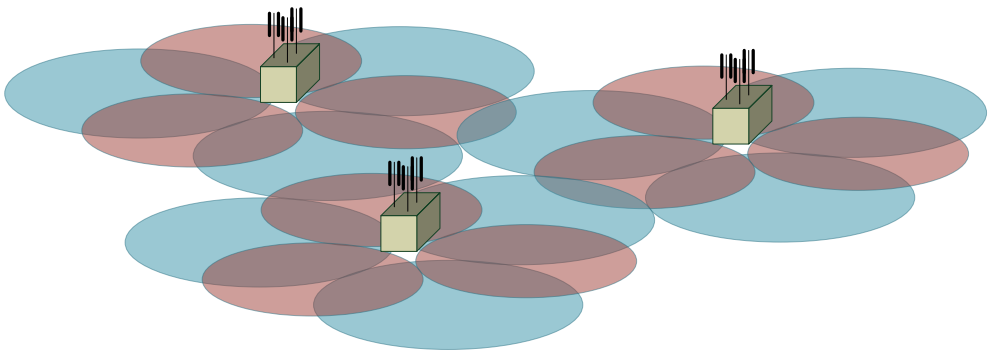


图 23-7：场景#3

场景#3 中，F1 和 F2 小区的位置相同，但 F2 的天线指向 F1 的小区边界（二者的天线方向不同），以此提升小区边界的吞吐量（throughput）。F1 提供了足够的覆盖，但 F2 由于可能存在的更大的路径损耗的原因，导致可能存在空洞（hole）。此时移动性是基于 F1 的覆盖进行的。可能的场景是 F1 和 F2 载波是不同频带上的载波，例如 F1 = {800 MHz ，2 GHz} 并且 F2 = {3.5 GHz}。对于那些处于 F1 和 F2 的重叠覆盖区域的 UE，可以聚合来自相同 eNodeB 的 F1 和 F2 载波，以提高吞吐量。

场景#4

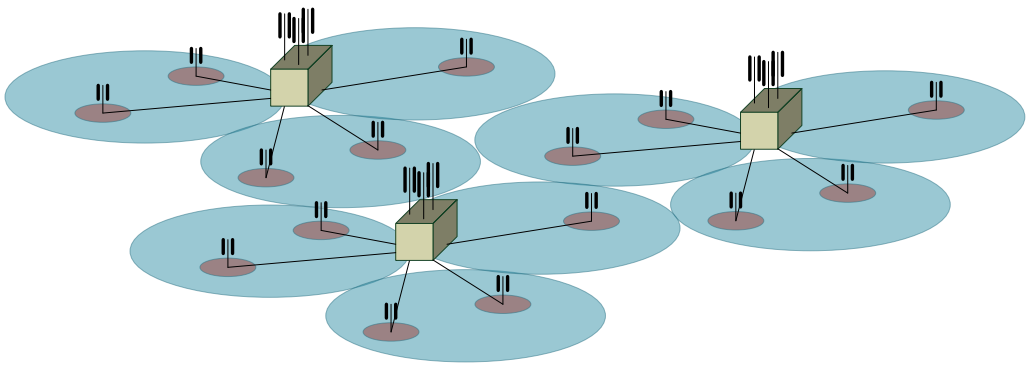


图 23-8：场景#4

场景#4 中，F1 提供广域的覆盖（macro coverage），而 F2 是通过 RRH（Remote Radio Head）设备被用于某些用户密集的热点（hot spot）区域，以提升该区域的吞吐量。F1 提供足够的覆盖，此时移动性是基于 F1 的覆盖进行的。可能的场景是 F1 和 F2 载波是不同频带上的载波，例如 F1 = {800 MHz ，2 GHz} 并且 F2 = {3.5 GHz}。

场景#5

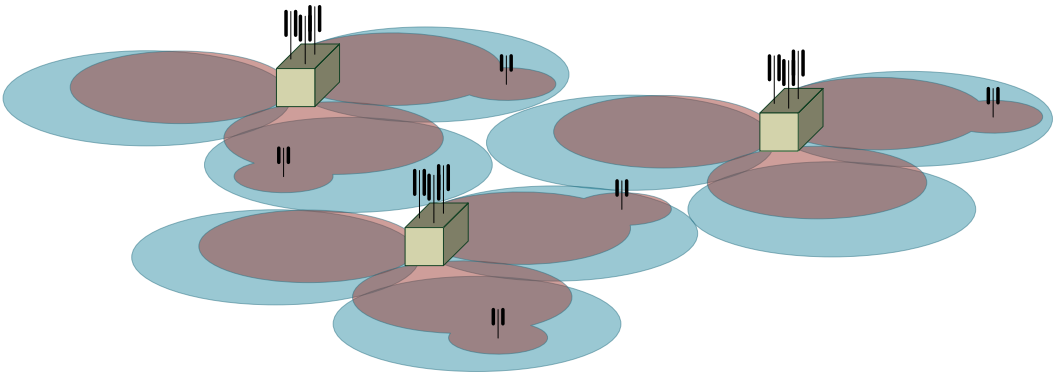


图 23-9：场景#5

场景#5 与场景#2 类似，F1 和 F2 提供不同的覆盖范围。但 F2 上进一步部署了频选中继器（frequency selective repeater），以扩大 F2 的覆盖范围。对于那些处于 F1 和 F2 的重叠覆盖区域的 UE，可以聚合来自相同 eNodeB 的 F1 和 F2 载波，以提高吞吐量。

以上这 5 种载波聚合的场景，适用于下行载波和上行载波。但如果场景#4 和场景#5 用于上行载波，可能会导致 UE 在不同小区的上行 timing 不同步。为了简单起见，Rel-10 中不考虑在聚合的不同小区之间使用不用的 TA（Timing Advance）值的情况。也就是说，假定聚合的小区之间上行 timing 是同步的。换句话说，Rel-10 中，下行载波支持上面介绍的全部 5 种配置，但上行载波只优先考虑场景#1、场景#2 和场景#3。

当部署了 CA 时，聚合的多个小区之间的帧 timing、SFN、TDD 配置必须是对齐的。

23.5 异构网络（heterogeneous network）

异构网络的一个典型例子是，在宏基站的小区覆盖范围内部署低功率的小基站（femto base station）来作为宏基站的补充，以提高系统容量和支持更高的速率，如图 23-10 所示。这 2 个不同输出功率的小区重叠在一起，构成了异构网络，并形成了不同的小区层（cell layer）。

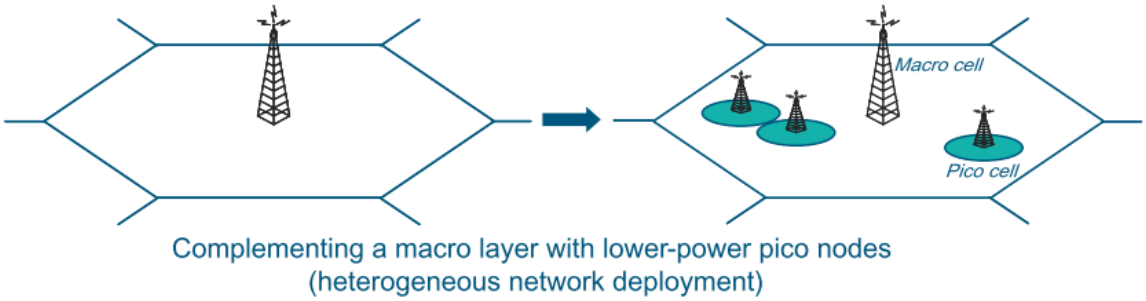


图 23-10：用低功率的皮小区（pico-cell）来补充宏小区

如果在不同的小区层间使用不同的频率，可以避免层间的强干扰。但在 LTE 中，不同的小区层使用不同的频率会导致不必要的频率碎片。因此在 LTE 的异构网络部署中，不同小区可以使用不同的传输功率，但使用相同的频率。如果二者使用相同的频率，宏基站会对小基站造成很大的干扰，此时 inter-layer interference（实际上也是某种类型的 inter-cell interference）会比处于同一层的 inter-cell interference 更加严重。

在异构网络中，对于 PDSCH 传输而言，可以通过小区间干扰协调（ICIC）的方式来避免干扰。但对于无法进行动态调度的下行 L1/L2 控制信道（PCFICH/PHICH/PDCCH）而言，通常使用跨载波调度（cross-carrier scheduling）的方式来避免小区间的干扰（见 4.4.4 节和 23.7.3 节）。

23.6 CA 对物理层的影响

每个载波单元有独立的物理层（physical layer），包括控制信令（control signaling）、调度（scheduling）和 HARQ 重传。

每个载波单元有一个独立的控制区域。

每个载波单元有一个独立的 DL-SCH 或 UL-SCH。

23.6.1 PCFICH

见 4.2.3 节关于“载波聚合对 PCFICH 的影响”的介绍。

23.6.2 PHICH

见 4.3.4 节关于“载波聚合对 PHICH 的影响”的介绍。

23.6.3 PDCCH

见 4.4.4 节关于“载波聚合对 PDCCH 的影响”的介绍。

23.6.4 对上行 L1/L2 Control Signaling 的影响

非同时传输 UL-SCH 和上行 L1/L2 控制信令的场景：对于配置了上行 CA 的 UE 而言，PUCCH 只能在 PCell 上传输。这也意味着多个下行 CC 传输的数据可能需要在同一个上行 CC 进行确认（acknowledge）。由于 PCell 是 UE 级别的概念，从网络侧（eNodeB）角度看，可能在多个 CC 上接收 PUCCH。

同时传输 UL-SCH 和上行 L1/L2 控制信令的场景：如果支持上行 CA，所有 CC 的上行控制信令只能在一个上行 CC 且只能该上行 CC 传输。

由于 Rel-8 中的 PUCCH 并不支持携带大量的 ACK/NACK 比特，所以在 Rel-10 中，新增了 PUCCH format 1b with channel selection 和 PUCCH format 3 的处理。具体见 13.5 节关于“载波聚合下的下行 HARQ”的介绍。

23.7 CA 对 MAC 层的影响（见 36.300 的 6.4 节）

eNodeB 侧的 MAC 层负责对无线资源的动态调度，以及 HARQ 处理。

23.7.1 对 HARQ 的影响（见 36.300 的 11 章）

CA 主要对 MAC 层可见。UE 对应每个 serving cell，都有各自独立的 HARQ entity。

数据的聚合发生在 MAC 层，MAC 层负责将数据划分到不同的载波单元上。

每个 serving cell 的上行和下行都有一个独立的 HARQ entity。并且在没有使用空分复用（spatial multiplexing）的情况下，每个 serving cell 在每个 TTI 产生 1 个 TB（空分复用的情况下，每个 TTI 产生 2 个 TB）。每个 TB 及其可能的 HARQ 重传只与一个 serving cell 匹配。

在不同 serving cell 上传输的 TB 有各自独立的 MCS（modulation and coding scheme）以及 MIMO 模式（即不同的 serving cell 可以使用不同的 TM 模式）。因此，每个 serving cell 有各自独立的 LA（link adaptation）以适应不同的信道质量。

系统允许不同的 serving cell 使用不同的传输功率配置，因此可以有不同的 coverage。

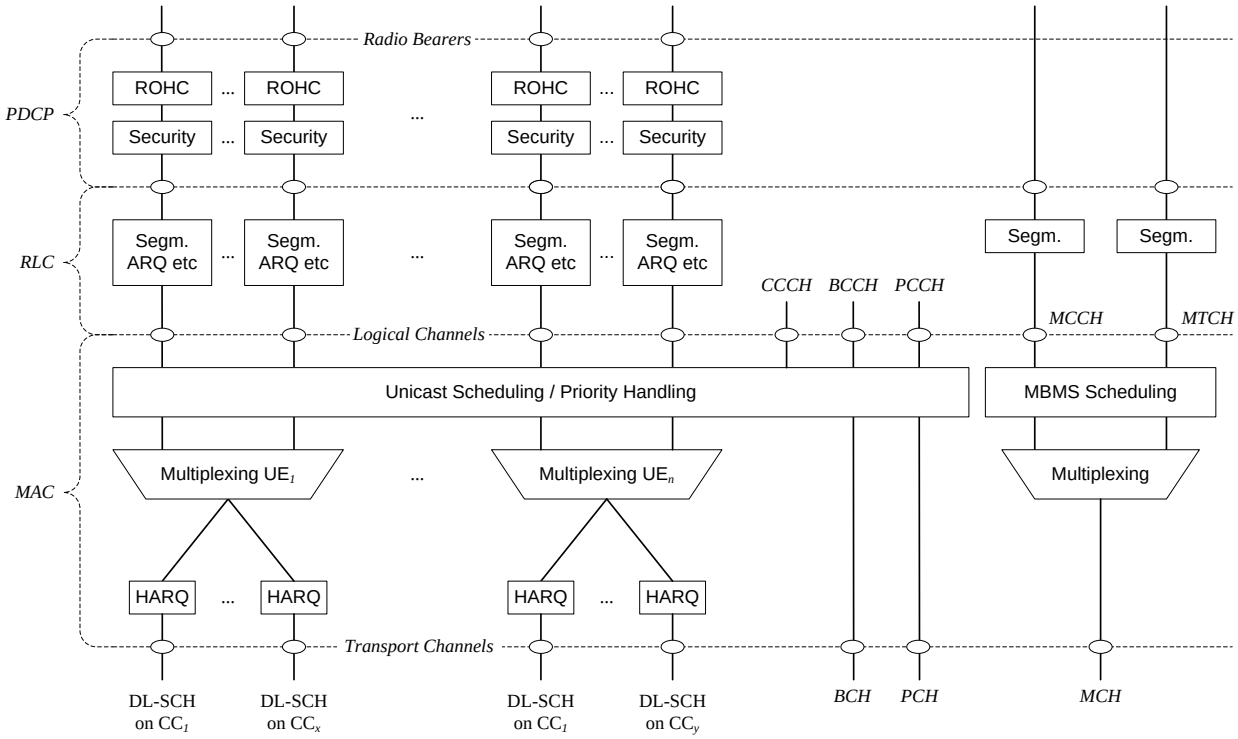


Figure 6.4-1: Layer 2 Structure for DL with CA configured

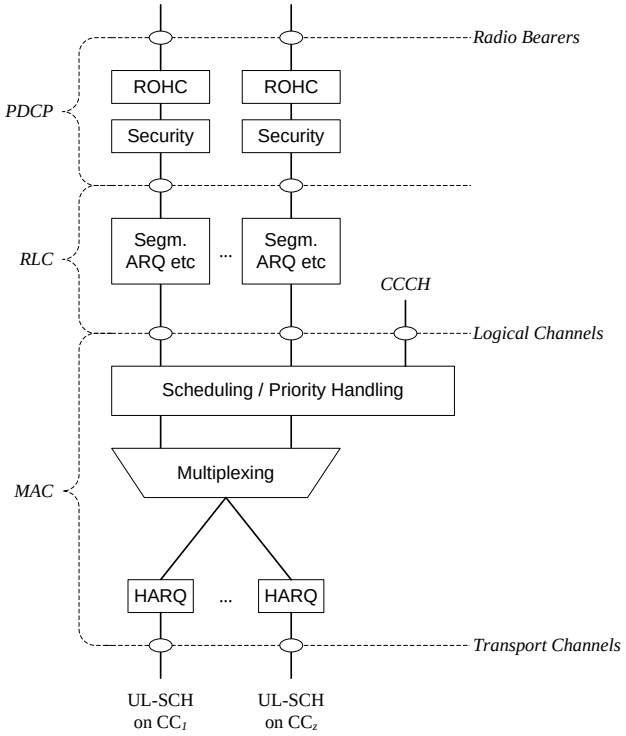


Figure 6.4-2: Layer 2 Structure for UL with CA configured

从 36.300 的 Figure 6.4-1 和 Figure 6.4-2 可以看出，对于同一个 UE，不同的 serving cell 虽然使用不同 HARQ entity，但是多个 serving cell 可以使用同一个复用（Multiplexing）单元。

关于 CA 下的 HARQ 处理，可参见 13.5 节关于“载波聚合下的下行 HARQ”的介绍。

23.7.2 对调度的影响（见 36.300 的 11 章）

如果 UE 配置了 CA，则该 UE 可以同时（同一个 TTI）在多个 serving cell 被调度。但任何时刻，至多只存在 1 个随机接入过程。

从 36.300 的 Figure 6.4-1 和 Figure 6.4-2 可以看出，一个全局的调度器实体（scheduler entity）可以负责所有 UE 以及它们对应的所有 CC 上的资源调度，但是也可以使用 per-CC 调度器，这取决于 eNodeB 的实现。

但从理论上，一个全局的调度器会比 per-CC 调度器提供更好的性能（避免了不同 CC 间相关调度信息的通信等）。

基于 CIF（Carrier Indicator Field）的跨载波调度（Cross-carrier scheduling）允许一个 serving cell 的 PDCCH 调度另一个 serving cell 上的无线资源。即控制信息在一个载波单元上传输，而对应的数据在另一个载波单元上传输。限制如下：

- 跨载波调度不适用于 PCell，PCell 总是通过它自身的 PDCCH 进行调度；
- 当某个 SCell 配置了 PDCCH，则跨载波调度不适用于该 SCell；
- 当某个 SCell 没有配置 PDCCH 时，则该 SCell 的跨载波调度总是通过另一个 serving cell 的 PDCCH 进行调度。

当 UE 配置了 CA 时，只能在 PCell 上为该 UE 半静态调度（SPS）上行资源或下行资源，也只有 PCell 上使用 C-RNTI 加扰的 PDCCH 分配的动态资源可以覆盖 SPS 调度的资源。也即 SPS 调度只存在于 PCell 上。

23.7.3 跨载波调度（cross-carrier scheduling）

在 LTE-A 中，跨载波调度的主要作用是在异构网络中为 PDCCH 提供 ICIC（Inter-Cell Interference Coordination）支持。一个典型异构网络场景如下：

一个 macro cell 和一个 small cell 共享 2 个下行 CC：CC1 和 CC2。small cell 的 2 个 CC 都在低传输功率下工作，macro cell 的 CC1 在高传输功率工作， CC2 在低传输功率工作。macro cell 在 CC1

上的传输对 small cell 的 CC1 有很大的干扰。因此，在 small cell 上，使用 CC2 上的 PDCCH 来跨载波调度传输在 CC1 上的数据是有好处的。

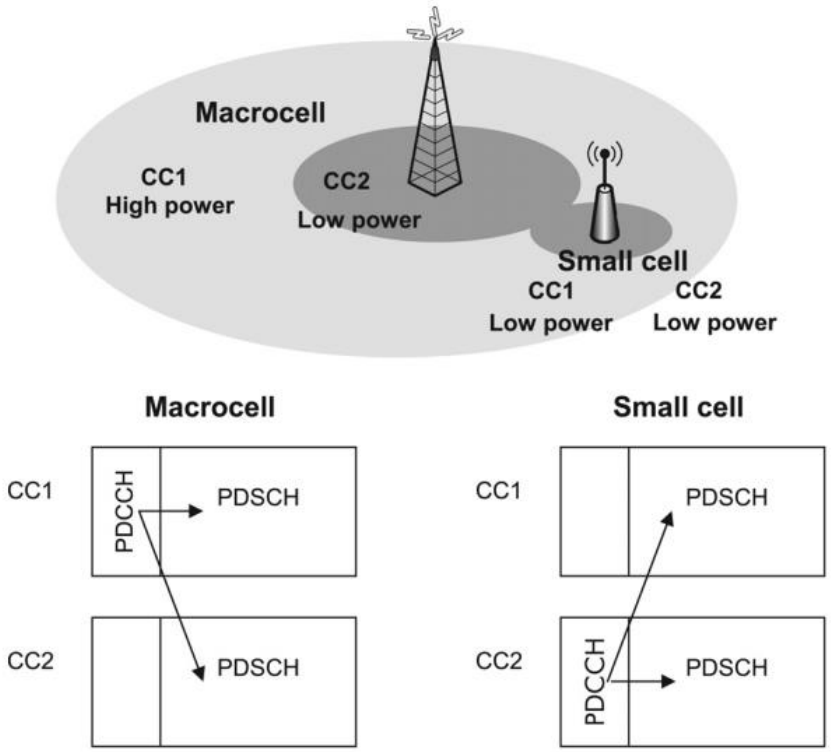


图 23-11：一个典型的异构网络部署：macro cell 和 small cell 共享 2 个 CCs

如果不支持跨载波调度，与 Rel-8 一样，每个小区上发送的 PDCCH 携带的信息对应本小区的下行资源分配或上行资源分配。

如果支持跨载波调度，则允许一个小区上的 PDCCH 调度另一个小区上的无线资源。即 PDCCH 在一个小区上传输，但对应的 PDSCH 或 PUSCH 在另一个小区上传输。

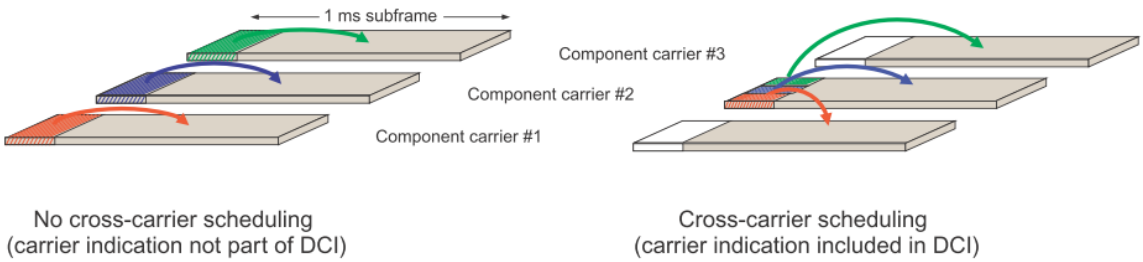


图 23-12：非跨载波调度与跨载波调度

eNodeB 是通过 *RRCConnectionReconfiguration* 来配置某个 UE 的跨载波调度的。其通过 IE: *crossCarrierSchedulingConfig-r10* 可以配置 UE 的每个载波单元在跨载波调度中的行为。（即每个载波单元可独立地配置是否使能跨载波调度以及在其它哪个载波单元上进行调度）。

```
CrossCarrierSchedulingConfig-r10 ::=        SEQUENCE { ------对应某个SCell的配置，这里将该SCell称为本小区
    schedulingCellInfo-r10              CHOICE {
        own-r10                             SEQUENCE {                  -- No cross carrier scheduling
            cif-Presence-r10                            BOOLEAN ------该值为TRUE时，表示本小区发送的PDCCH带CIF字段；否则表示本小区发送的PDCCH不带CIF字段
        },---------------- 如果CHOICE选择的是own-r10，则本小区的的PDSCH/PUSCH资源对应的PDCCH必定在同一小区的载波单元上发送。
        other-r10                               SEQUENCE {                  -- Cross carrier scheduling
            schedulingCellId-r10                ServCellIndex-r10,------本小区的PDCCH在schedulingCellId-r10指定的其它小区上发送
            pdsch-Start-r10                     INTEGER (1..4)------指定本小区的PDSCH的起始OFDM符号
        }----------------- 如果CHOICE选择的是other-r10，则本小区的PDSCH/PUSCH资源对应的PDCCH必定在其它小区的载波单元上发送。
    }
}
```

如果 UE 配置了跨载波调度，则该 UE 的 serving cell 可能的配置包括：

配置一：某个 serving cell 既不跨载波调度其它 serving cell 上的资源，也不被其它 serving cell 跨载波调度。即该 serving cell 只发送本小区的 PDCCH。

配置二：某个 serving cell 跨载波调度其它 serving cell 上的资源，此时该 serving cell 的资源也只能在本 serving cell 上调度，而不能被其它 serving cell 跨载波调度。即该 serving cell 既发送本小区的 PDCCH，也发送其它小区的 PDCCH。

配置三：某个 serving cell 被其它 serving cell 跨载波调度，此时该 serving cell 不能跨载波调度其它 serving cell 上的资源。即该 serving cell 既不发送本小区的 PDCCH，也不发送其它小区的 PDCCH。

从上面的介绍中可以看出，如果 UE 配置了跨载波调度，则某个小区发送的 PDCCH 可能对应本小区的 PDSCH/PUSCH 资源，也可能对应其它小区的 PDSCH/PUSCH 资源。因此在 Rel-10 中，

PDCCH（或者说 DCI）新增了一个“Carrier indicator”字段（简称为 CIF 字段），用于指定该 PDCCH 对应哪个小区的 PDSCH/PUSCH 资源，该字段只在跨载波调度中存在。

如果 CIF 的值为 0，则该 PDCCH 对应 PCell 上的资源；如果 CIF 的值不为 0，则该 PDCCH 对应“*sCellIndex-r10* = CIF”的 SCell 上的资源。即 CIF 的值与 *SCellToAddMod-r10* 中的 *sCellIndex-r10* 值是一一对应并且相等的。

UE 的某个 serving cell 发送的 PDCCH 是否存在 CIF 字段是通过 *cif-Presence-r10* 字段来控制的。该字段可能存在于 *PhysicalConfigDedicated*（对应 PCell 上的配置）或 *CrossCarrierSchedulingConfig-r10*（对应 SCell 上的配置）中。

在某一小区上传输的数据，UE 仅会在一个载波单元（同一个小区或跨载波调度时不同的小区）上收到对应的 PDCCH。

如果没有配置 CIF 字段，则在某一 serving cell 收到的 PDCCH 对应同一 serving cell 的 PUSCH 或 PDSCH 传输。

注意：（1）CIF 字段只存在于 UE 特定搜索空间的 PDCCH 中，公共搜索空间的 PDCCH 是不存在 CIF 字段的。

（2）支持跨载波调度与否，会导致 DCI 格式有所不同，UE 盲检时应根据具体的情况予以区分。

如果某个 SCell 上的 PDCCH 需要在其它 serving cell 上发送，则 eNodeB 会通过 *CrossCarrierSchedulingConfig-r10* 中的 *schedulingCellId-r10* 告知 UE，该 SCell 的 PDCCH 在 *schedulingCellId-r10* 对应的 serving cell 上发送。

这里我们举几个例子来简单说明如何配置跨载波调度。

例 1：假设 UE 不配置跨载波调度，则对应每个 serving cell 的 PDCCH 都在本小区的载波上发送。此时对应每个 serving cell，既不配置 *cif-Presence-r10*，也不配置 *CrossCarrierSchedulingConfig-r10*。此时每个小区发送的 PDCCH 都不带 CIF 字段。

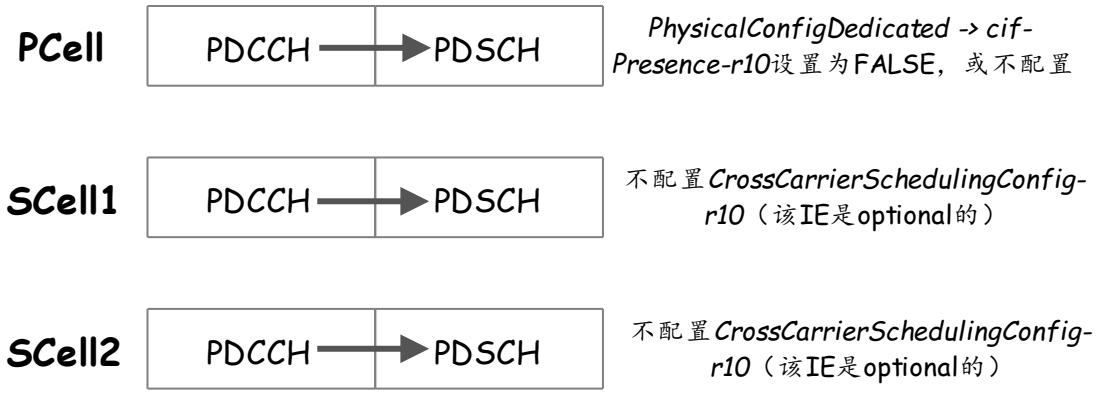


图 23-13：例 1

例 2：假设 UE 配置了跨载波调度。PCell 既调度本小区的资源，又跨载波调度 SCell1 的资源。其 *PhysicalConfigDedicated -> cif-Presence-r10* 要设置为 TRUE，其发送的 PDCCH 带 CIF 字段。

SCell1 既不调度本小区的资源，也不调度其它小区的资源，其资源在 PCell 上调度。此时 *CrossCarrierSchedulingConfig-r10 -> schedulingCellId-r10* 需要设置为 0，说明该小区对应的 PDCCH 在 PCell 上发送。

SCell2 调度本小区的资源，但不调度其它小区的资源。该小区不配置 *CrossCarrierSchedulingConfig-r10*，其发送的 PDCCH 不带 CIF 字段。

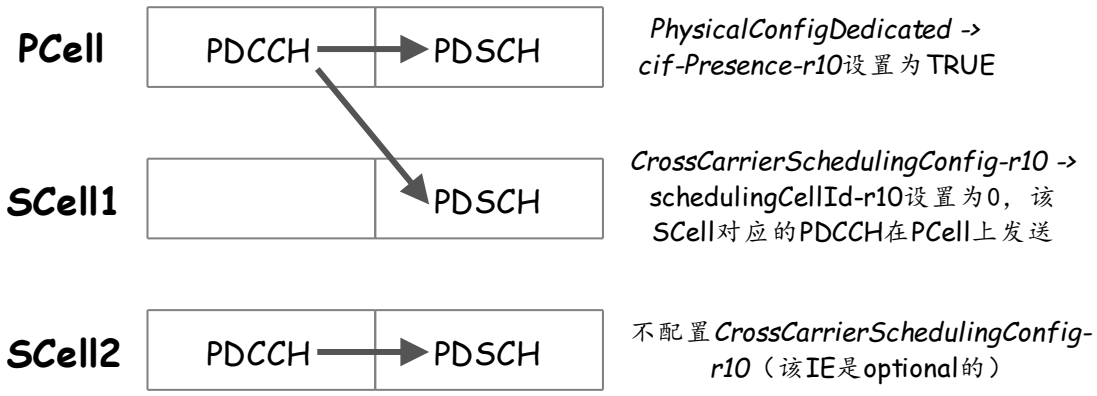


图 23-14：例 2

例 3：假设 UE 配置了跨载波调度。PCell 调度本小区的资源，但不调度其它小区的资源。该小区 *PhysicalConfigDedicated -> cif-Presence-r10* 可以设置为 FALSE，或根本不做配置，其发送的 PDCCH 不带 CIF 字段。

SCell1 既不调度本小区的资源，也不调度其它小区的资源，其资源在 SCell2 上调度。此时 SCell1 的 *CrossCarrierSchedulingConfig-r10 -> schedulingCellId-r10* 需要设置为 SCell2 对应的 *sCellIndex-r10*，说明该小区对应的 PDCCH 在 SCell2 上发送。

SCell2 既调度本小区的资源，又跨载波调度 SCell1 的资源。其 *CrossCarrierSchedulingConfig-r10 -> cif-Presence-r10* 要设置为 TRUE，其发送的 PDCCH 带 CIF 字段。

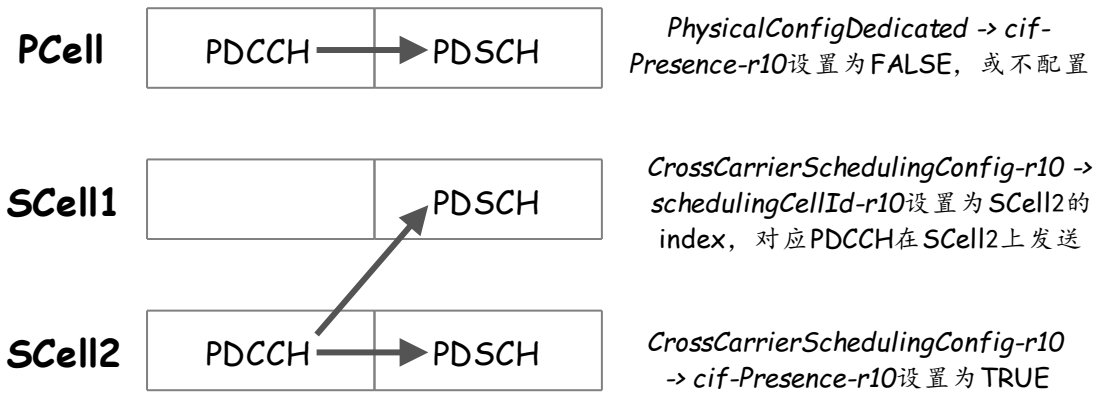


图 23-15：例 3

注意：前面介绍的 PCell/SCell 是针对同一个 UE 而言的，其对应的配置也是针对该 UE 的。而同一小区在同一 TTI 发送的针对不同 UE 的多个 PDCCH 中，一部分可携带 CIF 字段，而另一部分可不携带 CIF 字段。

23.8 CA 对 RLC/PDCP 层的影响

CA 对于 RLC 层和 PDCP 层是不可见的。但由于可能增大的数据量，需要扩大 RLC 的 buffer 大小。

23.9 CA 对 RRC 层的影响

处于 idle 态（RRC_IDLE）的 UE，其相关流程仅与 PCell 相关。CA 只针对 RRC_CONNECTED 的 UE 而言。

当 UE 配置了 CA 后，该 UE 仅与 PCell 存在 RRC 连接。PCell 控制该 UE 的所有 serving cell 配置，并提供安全输入（security input）、NAS mobility 信息以及 SI 信息。

- UE 的 PCell 只能在 handover 过程中发生变化（例如：伴随着 security key 变化和 RACH 过程）。也就是说，只有 PCell 能进行 handover 处理，SCell 是不进行 handover 的。
- 只有当 PCell 遇到 RLF（Radio Link Failure）时，才会触发重建（Re-establishment）流程。当 SCell 遭遇 RLF 时，并不会触发重建流程。
- NAS 信息来自 PCell。（因为 UE 只与 PCell 存在 RRC 连接）
- UE 并不需要监测 DL SCC 的无线链路。eNodeB 可以通过对应某 SCell 的 CQI 报告和/或 RRM measurement 报告检测到该 SCC 的链路质量。

eNodeB 可以在 PCell 上通过 RRC 重配置消息添加、删除、重配置 SCell。

当添加一个新的 SCell 时，PCell 会使用专用的 RRC 信令（见 *radioResourceConfigCommonSCell-r10*）来发送该 SCell 需要的所有系统信息， UE 并不需要直接从 SCell 中获取广播的系统信息。

当 UE 处于连接态（RRC_CONNECTED），且 SCell 的 SI 信息发生变化时，需要通过 RRC 重配置消息通知 UE。

在 intra-LTE handover 过程中，源 PCell 可以选取无线质量最好的几个小区并以质量递减的顺序生成 SCell 列表，然后发给目标 PCell。目标 PCell 自己决定是否使用以及使用其中的哪些 SCell。此过程是由 eNodeB 控制的，UE 无法自行删除 SCell。

23.10 CA 对 random access 过程的影响（见 36.300 的 10.1.5 节）

在 CA 中，UE 只在 PCell 进行随机接入。更确切地说，发生随机接入的小区即为该 UE 的 PCell。在任何时候，UE 只会发生一个随机接入过程，这与 UE 的配置和载波聚合能力无关。

随机接入过程不关注小区大小以及 serving cell 的个数（当配置了 CA 时）。

当配置了 CA 时，只有基于冲突的随机接入过程的第 4 步（contention resolution）可以由 PCell 进行跨载波调度，其它步骤（无论基于竞争还是非竞争）都只能在 PCell 上进行。

除了上述所说的第 4 步可以由 PCell 进行跨载波调度外，可以认为随机接入过程与 Rel-8 是没有任何区别的。

23.11 CA 对 DRX 的影响

见 16.2 节关于“载波聚合（CA）对 DRX 的影响”的介绍。

23.12 CA 对 BSR 的影响

在一个子帧内，多于一个 serving cell 可以发送 Regular 或 Periodic BSR 时，UE 可以选择在其中一个 serving cell 上发送 BSR。

在 UE 配置了多个上行载波单元的场景中，BSR 指示的数据量可能远大于 Rel-8 中指示的数据量，因此在 Rel-10 中，额外提供了一个 BSR 值的表（见 36.321 的 Table 6.1.3.1-2）。具体使用哪个表是通过 RRC 消息 *MAC-MainConfig* 的 *extendedBSR-Sizes-r10* 字段来配置的。

注：本章并没有涵盖载波聚合带来的所有影响，但在介绍其它特性时，如果涉及到载波聚合，会特别介绍的。

【参考资料】

[1] 《4G LTE/LTE-Advanced for Mobile Broadband》的 9.3 节、10.4.6 节和 17.4 节

[2] 《LTE – The UMTS Long Term Evolution, 2nd Edition》的第 28 章

第24章 SPS

VoLTE 的实现主要涉及以下方面。

- 专用的承载（Dedicated bearer）：使得 VoLTE 语音流与其它业务流区分开，并赋予 VoLTE 语音流更高的 QoS 级别；
- RoHC：RoHC 能够显著地降低 VoLTE 语音包的大小（接近 50%），从而降低了对资源的要求，这样就剩余了更多的资源可分配给其它 UE。
- SPS：SPS 能够降低调度周期性发送且较小的 VoLTE 语音包的开销（主要是 PDCCH 的开销），从而使得更多的资源可用于调度额外的 UE。
- DRX：DRX 可以在 VoLTE 通话期间，节约 UE 的电池消耗。这是通过允许 UE 在接收语音包的空隙关闭其接收机来实现的。（见第 16 章）
- TTI bundling：TTI bundling 通过使用多个绑定的 TTI 来传输同一个 TB，以降低语音包的出错率，从而提高小区边缘 UE 的上行 VoIP 覆盖。

本章以及第 25 章主要介绍 SPS 和 TTI bundling 的相关内容。同时不会涉及 SRVCC 和 CSFB 的实现，大家有兴趣的话可以去查阅一些相关资料。

Semi-Persistent Scheduling，简称 SPS，半永久性调度，又称为半静态调度。

与动态调度时每个 TTI 为 UE 分配一次无线资源不同（通过 PDCCH 指定），SPS 允许半静态配置无线资源，并将该资源周期性地分配给某个特定 UE。

简单地说，eNodeB 在某个 TTI 使用 SPS C-RNTI 加扰的 PDCCH 指定 UE 所使用的无线资源（这里将其称为 SPS 资源），每过一个周期，UE 就使用该 SPS 资源来收或发数据。eNodeB 无需在该子帧（这里将其称为 SPS 子帧）重新下发 PDCCH 来指定分配的资源。

由于 SPS 有“一次分配，多次使用”的特点，不需要在每个 TTI 都为 UE 下发 DCI（包括上行或下行的 DCI），从而降低了对应的 PDCCH 开销。

SPS 对数据包小，大小基本不变且周期性的业务很有用，如 VoIP。这类业务的 timing 和所需的无线资源都是可预知的。

只有 PCell 支持 SPS。这是因为 SPS 主要应用于低速业务，负载低，没有必要使用多个载波单元。

配置了 SPS 调度的 UE 可以同时进行动态调度，所以需要区分 PDCCH 是用于动态调度还是 SPS 调度。因此，配置了 SPS 调度的 UE 有 2 个标志：一个“正常的”C-RNTI，用于动态调度，在随机接入过程中分配；另一个是 SPS C-RNTI，用于 SPS 调度，通过 *SPS-Config* 的 *semiPersistSchedC-RNTI* 字段配置。

UE 的半静态调度是通过 IE: *SPS-Config* 配置的。

```
SPS-Config ::=SEQUENCE {
    semiPersistSchedC-RNTI          C-RNTI                OPTIONAL,         --
Need OR     ------Semi-Persistent Scheduling C-RNTI
    sps-ConfigDL                SPS-ConfigDL          OPTIONAL,         -- Need
ON          ------用于下行SPS
    sps-ConfigUL                SPS-ConfigUL          OPTIONAL          -- Need
ON          ------用于上行SPS
}

SPS-ConfigDL ::=  CHOICE{
    release                     NULL,       ------释放下行SPS
    setup                       SEQUENCE {
        semiPersistSchedIntervalDL          ENUMERATED {
                                            sf10, sf20, sf32, sf40, sf64, sf80,
                                            sf128, sf160, sf320, sf640, spare6,
                                            spare5, spare4, spare3, spare2,
                                            spare1},------指定下行SPS的周期
        numberOfConfSPS-Processes           INTEGER (1..8),------ 用于指定下行SPS的HARQ
process个数，上行没有这个配置
        n1PUCCH-AN-PersistentList           N1PUCCH-AN-PersistentList,------用于回复
ACK/NACK的4个PUCCH 1资源，用于天线端口0
        ...,
        [[  twoAntennaPortActivated-r10     CHOICE {
                release                     NULL,
                setup                       SEQUENCE {
                    n1PUCCH-AN-PersistentListP1-r10 N1PUCCH-AN-PersistentList------用
于回复ACK/NACK的4个PUCCH 1资源，用于天线端口1
                }
            }                                                       OPTIONAL   -
- Need ON
        ]]
    }
}

SPS-ConfigUL ::=  CHOICE {
    release                     NULL,       ------释放上行SPS
    setup                       SEQUENCE {
        semiPersistSchedIntervalUL          ENUMERATED {
                                            sf10, sf20, sf32, sf40, sf64, sf80,
                                            sf128, sf160, sf320, sf640, spare6,
                                            spare5, spare4, spare3, spare2,
                                            spare1},------指定上行SPS的周期
        implicitReleaseAfter            ENUMERATED {e2, e3, e4, e8},------如果UE连续
implicitReleaseAfter次发送的MAC PDU不包含MAC SDU，则上行SPS会自动释放。e2代表连续2次，依次类推
        p0-Persistent                   SEQUENCE {
```

```
            p0-NominalPUSCH-Persistent          INTEGER (-126..24),
            p0-UE-PUSCH-Persistent              INTEGER (-8..7)
        }       OPTIONAL,                                           -- Need OP------用
于功控，本书不做介绍
        twoIntervalsConfig                  ENUMERATED {true}       OPTIONAL,  -
- Cond TDD        ------是否支持two-intervals-SPS，只存在于TDD的上行
        ...
    }
}

N1PUCCH-AN-PersistentList ::=       SEQUENCE (SIZE (1..4)) OF INTEGER (0..2047)
```

UE 配置了 SPS 后，还不能使用，必须使用 SPS C-RNTI 加扰的 PDCCH 进行激活。（详见 36.213 的 9.2 节）

eNodeB 通过 SPS C-RNTI 加扰的 PDCCH 来激活/释放 UE 的 SPS。

只有满足如下条件，UE 才会验证接收到的 PDCCH 是否用于 SPS 的激活或释放：

- PDCCH 的 CRC 校验位使用 SPS C-RNTI 进行加扰；
- NDI 域（new data indicator）设置为 0。对于 DCI format 2/2A/2B/2C 而言，NDI 域是针对使能了的那个 TB 而言的。

注：SPS 是不支持空分复用的，所以对于 DCI format 2/2A/2B/2C 而言，只会使能一个 TB。

满足了上述条件后，UE 还会根据 36.213 的 Table 9.2-1（SPS 激活）和 Table 9.2-1A（SPS 释放）进一步验证如下字段：

Table 9.2-1: Special fields for Semi-Persistent Scheduling Activation PDCCH Validation

	DCI format 0	DCI format 1/1A	DCI format 2/2A/2B/2C
TPC command for scheduled PUSCH	set to ‘00’	N/A	N/A
Cyclic shift DM RS	set to ‘000’	N/A	N/A
Modulation and coding scheme and redundancy version	MSB is set to ‘0’	N/A	N/A
HARQ process number	N/A	FDD: set to ‘000’ TDD: set to ‘0000’	FDD: set to ‘000’ TDD: set to ‘0000’
Modulation and coding scheme	N/A	MSB is set to ‘0’	For the enabled transport block: MSB is set to ‘0’

Redundancy version	N/A	set to ‘00’	For the enabled transport block: set to ‘00’

Table 9.2-1A: Special fields for Semi-Persistent Scheduling Release PDCCH Validation

	DCI format 0	DCI format 1A
TPC command for scheduled PUSCH	set to ‘00’	N/A
Cyclic shift DM RS	set to ‘000’	N/A
Modulation and coding scheme and redundancy version	set to ‘11111’	N/A
Resource block assignment and hopping resource allocation	Set to all ‘1’s	N/A
HARQ process number	N/A	FDD: set to ‘000’ TDD: set to ‘0000’
Modulation and coding scheme	N/A	set to ‘11111’
Redundancy version	N/A	set to ‘00’
Resource block assignment	N/A	Set to all ‘1’s

如果对应 DCI format 中的所有字段都按 Table 9.2-1 或 Table 9.2-1A 所示来设置，则验证成功，否则验证失败。

如果验证成功，UE 会将接收到的 DCI 当做一个有效的 SPS 激活/释放来处理；如果验证失败，UE 会认为接收到的 DCI 使用了一个不匹配的 CRC。

注意：DCI 中的其它字段含义没有改变，SPS 使用的无线资源（称为 SPS 资源）和 MCS 等就是通过用于激活 SPS 的 DCI 来指示的。

（1）从 36.213 的 Table 9.2-1 可以看出，指示 SPS 激活的 PDCCH 中的“Resource block assignment”（用于下行）和“Resource block assignment (and hopping resource allocation)”（用于上行）字段是用于指示无线资源的，也就是说，对于下行，该 PDCCH 是与对应的 PDSCH 一起发送的；对于上行，该 PDCCH 还指定了 PUSCH 的资源。但从 36.213 的 Table 9.2-1A 可以看出，指示 SPS 释放的 PDCCH 中的“Resource block assignment”（用于下行）和“Resource block assignment (and hopping resource allocation)”（用于上行）字段为“全 1”，即该字段是不用于指示无线资源的。简单地说：SPS 激活时可以同时传输 PDSCH 和 PUSCH（有一定的 timing），但 SPS 释放时不能同时传输 PDSCH 和 PUSCH。

（2）从 36.213 的 Table 9.2-1 可以看出，指示 SPS 激活的 PDCCH 的 5 比特“Modulation and coding scheme (and redundancy version)”字段的 MSB 为 0，即 I_{MCS} 的取值范围为 0~15，而不是通常的 0~31。

（3）从 36.213 的 Table 9.2-1 可以看出，SPS 调度在上行并不支持 DCI format 4，SPS 调度在下行使用 DCI format 2/2A/2B/2C 时，只有一个 TB 是使能的，所以说 SPS 是不支持空分复用的。并且从 36.213 的 Table 8-5 可以看出，SPS 调度在上行只支持单天线端口传输（DCI format 0），从 36.213 的 Table 7.1-6 可以看出，SPS 调度在下行只支持单天线端口传输或传输分集，这也说明了 SPS 并不支持空分复用。

在激活了 SPS 后，UE 就可以周期性地使用配置的 SPS 资源来接收和发送数据。

对于下行 SPS 传输，需要指定用于回复 ACK/NACK 的 PUCCH 1 资源。如果 UE 接收到的 DCI 指示 DL SPS 激活，则 DCI 中的 TPC command for PUCCH 字段将指示从 4 个 PUCCH 1 资源中选择 1 个用于回复 ACK/NACK。该 4 个 PUCCH 1 资源是 eNodeB 通过 *SPS-ConfigDL* 的 *n1PUCCH-AN-PersistentList* 配置给 UE 的（对于 2 天线传输，第二个天线端口的 4 个 PUCCH 1 资源通过 *n1PUCCH-AN-PersistentListP1-r10* 配置）。

Table 9.2-2: PUCCH Resource value for Downlink Semi-Persistent Scheduling

Value of 'TPC command for PUCCH'	$n_{\mathrm{PUCCH}}^{(1,p)}$
'00'	The first PUCCH resource value configured by the higher layers
'01'	The second PUCCH resource value configured by the higher layers
'10'	The third PUCCH resource value configured by the higher layers
'11'	The fourth PUCCH resource value configured by the higher layers

简单地说，就是 eNodeB 先通过 *n1PUCCH-AN-PersistentList*（和 *n1PUCCH-AN-PersistentListP1-r10*）给 UE 配置 4 个（对）可选的 PUCCH 1 资源。然后在 DL SPS 激活时，通过 DCI 中的 TPC command for PUCCH 字段告诉 UE 具体使用哪一个（对）PUCCH 1 资源来回复 ACK/NACK。

小区在每个上行子帧需要预留多少个 PUCCH 1 资源用于 SPS 调度的 HARQ 反馈？

我们先以 FDD 为例来计算小区在一个上行子帧需要预留多少个 PUCCH 1 资源用于回复下行 SPS 调度的 ACK/NACK。假设小区需要支持 80 个 UE 同时进行语音通话，下行 SPS 调度的周期为 20 ms。FDD 下，每个子帧都可用于回复 ACK/NACK，因此在不考虑 SORTD 的情况下，每个上行子帧至多有$\lceil 80/(20/10*10)\rceil = 4$个 SPS 调度的 UE 需要反馈 ACK/NACK。此时 eNodeB 可以给这 80 个 UE 都配置相同的 *n1PUCCH-AN-PersistentList* 值，即所有的 UE 都配置相同的 4 个 PUCCH 1

资源，然后给需要在同一上行子帧进行 ACK/NACK 反馈的不同 UE（至多 4 个）分配不同的 TPC 值，就可以保证在同一上行子帧回复 ACK/NACK 的 UE 使用不同的 PUCCH 1 资源了。

如果小区需要支持 160 个 UE 同时进行语音通话，则所需的 PUCCH 1 资源就需要翻倍，每个上行子帧至多有$\lceil 160/(20/10*10)\rceil = 8$个 SPS 调度的 UE 需要反馈 ACK/NACK。此时 eNodeB 需要给其中 80 个 UE 配置一个 *n1PUCCH-AN-PersistentList* 值，而给另外 80 个 UE 配置另一个 *n1PUCCH-AN-PersistentList* 值，并且这 2 个 list 包含的 PUCCH 1 资源集合各不相同。然后在同一上行子帧进行 ACK/NACK 反馈的不同 UE（至多 8 个）分为 2 组，每组包含的 UE 数不超过 4 个，且每组配置不同的 list。通过给同一组内的不同 UE 分配不同的 TPC 值就可以保证在同一上行子帧回复 ACK/NACK 的 UE 使用不同的 PUCCH 1 资源了。

如果是 TDD，一个周期内可用的上行子帧数变少，因此在小区容量不变且配置不变的情况下，每个子帧需要使用更多的 PUCCH 1 资源用于 SPS 调度的 ACK/NACK 反馈。

假设支持的 SPS 用户数为 N_{SPS}^{user} ，每个系统帧（10 ms）内有 $N_{UL}^{subfraem}$ 个上行子帧可用于发送 PUCCH，下行 SPS 调度的周期为 N_{pd}（等于 *semiPersistSchedIntervalDL*，见后续介绍），则每个上行子帧需要预留的 PUCCH 1 资源数为 $N_{PUCCH}^{(SPS)} = \left\lceil N_{SPS}^{user}/(N_{pd}/10*N_{UL}^{subframe}) \right\rceil$。如果需要支持 SORTD，这个数值还需要乘以 2。

24.1 下行 SPS

如果配置并激活了 DL SPS，则 UE 会认为在满足下面公式的子帧里（称为 SPS 子帧）被分配了 SPS 下行资源（此时无需收到 PDCCH）：

(10 * SFN + subframe) = [(10 * $SFN_{start\ time}$ + $subframe_{start\ time}$) + N * *semiPersistSchedIntervalDL*] modulo 10240

其中 N > 0（初始为 0，每过一个周期加 1），$SFN_{start\ time}$ 和 $subframe_{start\ time}$ 为 UE 接收到指示下行 SPS 激活的 PDCCH 的系统帧号和子帧号。modulo 10240 是因为系统帧使用 10 比特表示，取值范围为 0~1023，每隔 1024 个系统帧需要做一次反转。*semiPersistSchedIntervalDL* 指明 UE 每隔 *semiPersistSchedIntervalDL* 个子帧被分配一次 DL SPS 资源，即 DL SPS 的周期。

当 UE 接收到指示下行 SPS 激活的 PDCCH 时，其对应的 HARQ process ID 是通过下面的公式计算的（DCI 中的 HARQ process number 域被置成 0）：

HARQ Process ID = [floor(CURRENT_TTI/semiPersistSchedIntervalDL)] modulo numberOfConfSPS-Processes

其中，CURRENT_TTI=[(SFN * 10) + subframe number] 。

对于下行 HARQ，只有新传会使用 SPS 资源，重传需要通过 SPS C-RNIT 加扰的 PDCCH 显式地分配资源，这是因为下行 HARQ 使用异步、自适应的方式。其中的 HARQ process number 字段指示了进行重传的 HARQ process。

这部分内容主要参见 36.321 的 5.10 节。

24.2 上行 SPS

如果配并激活了 UL SPS，则 UE 会认为在满足下面公式的子帧里（称为 SPS 子帧）被分配了 SPS 上行资源（此时无需收到 PDCCH）：

$$(10 * SFN + subframe) = [(10 * SFN_{start\ time} + subframe_{start\ time}) + N * semiPersistSchedIntervalUL + Subframe_Offset * (N\ modulo\ 2)]\ modulo\ 10240$$

$SFN_{start\ time}$ 和 $subframe_{start\ time}$ 为 UE 接收到指示上行 SPS 激活的 PDCCH 所指定的将用来发送上行数据的系统帧号和子帧号（对于 FDD，是子帧 n + 4；对于 TDD 而言，与上下行配置相关，可参见 12 章关于“上行 HARQ”的介绍。需要注意的是：这里的处理与下行略有不同）。

与下行相比，上行 SPS 增加了偏移量 Subframe_Offset * (N modulo 2)，该偏移量只存在于 TDD 中。如果 SPS-ConfigUL 的 *twoIntervalsConfig* 字段（只存在于 TDD 中）配置为 true，则 Subframe_Offset 根据 36.321 的 Table 7.4-1 来配置；否则 Subframe_Offset 的值为 0。（上行的 two-intervals-SPS 特性）

Table 7.4-1: Subframe_Offset values

TDD UL/DL configuration	Position of initial Semi-Persistent grant	Subframe_Offset value (ms)
0	N/A	0
1	Subframes 2 and 7	1
	Subframes 3 and 8	-1
2	Subframe 2	5
	Subframe 7	-5
3	Subframes 2 and 3	1
	Subframe 4	-2
4	Subframe 2	1
	Subframe 3	-1
5	N/A	0
6	N/A	0

two-intervals-SPS 特性用于避免第 n 个包的非自适应重传与第 n+1 个包的初传产生冲突，关于该特性的详细描述，可参见参考资料[12]的 3.3 节。

对于下行重传和上行自适应重传，由于需要重新发送 PDCCH，可以通过调度避免第 n 个包的重传与第 n+1 个包初传发生冲突。

如果 UE 在分配的上行 SPS 资源上，连续 *implicitReleaseAfter* 次发送的 MAC PDU 不包含 MAC SDU，则会自动释放上行 SPS 并清除配置的 UL grant。

在清除了 UL grant 后，需要重传的 SPS 数据并不受影响。

与下行不同，上行 HARQ 使用同步的方式，因此并不需要指定 HARQ process ID。并且对上行重传而言，可以使用 SPS 调度分配的资源，也可以使用 SPS C-RNTI 加扰的 PDCCH 重新指定资源。

注意：无论对于上行 SPS 还是下行 SPS，重传时使用的 SPS C-RNTI 加扰的 PDCCH 并不会重新配置 SPS 资源，也就是说，该 PDCCH 只在该子帧有效。

这部分内容主要参见 36.321 的 5.10 节。

从前面的介绍可以看出，SPS C-RNTI 不仅用于 SPS 激活过程中的 PDCCH，还用于 SPS 调度 TB 的 HARQ 重传的 PDCCH。为了区分这两种情况，使用了 NDI 字段：如果是配置 SPS 资源（SPS 激活），则 NDI 设置为 0，并认为 NDI 翻转；如果是 HARQ 重传，NDI 设置为 1，并认为 NDI 没有翻转。（见 36.321 的 5.3.1 节和 5.4.1 节）

如果 UE 收到的 PDCCH 使用 C-RNTI 加扰，并且对应的 HARQ process 前一次接收到的 PDCCH 使用 SPS C-RNTI 加扰（可能是 SPS 激活，也可能是 SPS 的重传），则无论 NDI 域的值是 0 或 1，都认为 NDI 翻转，即新传。

UE 激活了 SPS 后，仍然会监听用于动态调度的 PDCCH。当在 SPS 子帧上检测到用于动态调度的 PDCCH 时，该 PDCCH 分配的资源会取代 SPS 调度分配的资源，即动态调度的优先级更高。但需要注意的是，动态调度的 PDCCH 只对该子帧有效，并不影响后续的 SPS 子帧。这种做法在 SPS 调度分配的资源不够时是有用的，例如 UE 进行 VoIP 的同时在浏览网页，当下载网页时，可能需要更多的资源块，这时 SPS 调度分配的资源不够用，就需要动态调度更多的资源。还有一种可能，当 SPS 资源与 PRACH 资源冲突时，使用动态调度临时分配资源以避免冲突。

UE 激活了 SPS 后，可能在之后的过程中由于码率变化（无线信道条件发生变化）等原因，需要重新配置 SPS 资源。例如当 VoIP 服务的码率升高，eNodeB 需要重新下发一个新的使用 SPS C-RNTI 加扰的下行 DCI 或上行 DCI（格式与 SPS Activation 的 PDCCH 相同），以便分配更大的 SPS 资源。重新分配的过程可以发生在任意子帧，此时 $SFN_{start\ time}$ 和 $subframe_{start\ time}$ 也会更新，后续的 SPS 子帧将使用重新分配的 SPS 资源。

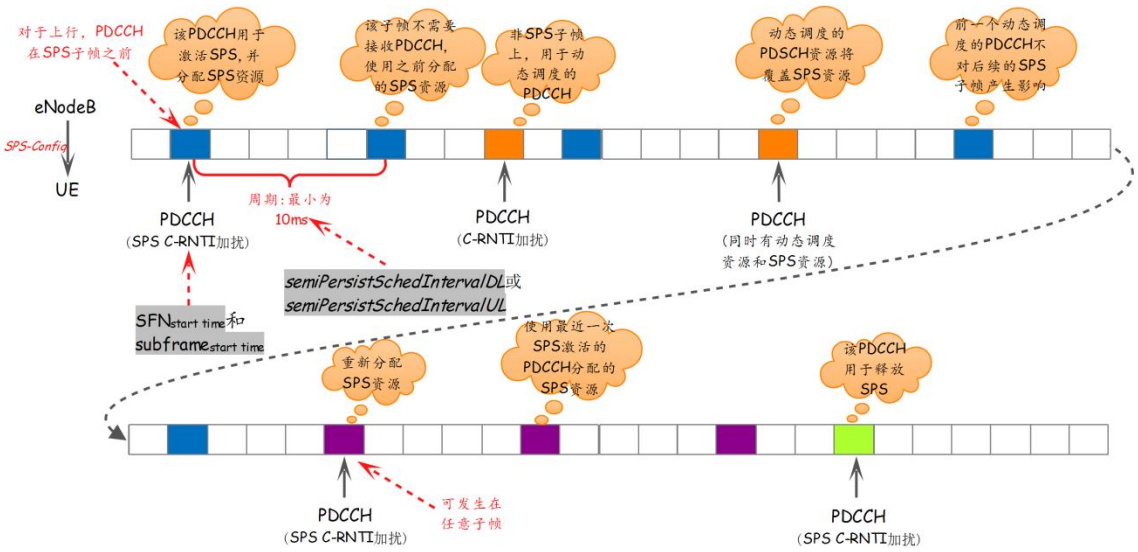


图 24-1：SPS 调度

配置了 SPS 的 UE，其 PDSCH/PUSCH 何时使用 C-RNTI，或是使用 SPS C-RNTI 来加扰可参见 36.213 的 7.1 节和 8.0 节。

这里，我将用于指示 SPS 激活的 PDCCH 所在的子帧称为“SPS 激活子帧”，将用于指示 SPS 释放的 PDCCH 所在的子帧称为“SPS 释放子帧”，而其它周期性发送 PDSCH（虽然下行 SPS 激活子帧也携带 PDSCH，但这里不包含在内）或 PUSCH 的 SPS 相关子帧称为“SPS 子帧”。这个概念会在介绍下行 HARQ 时用到。

与 SPS 相关的 PDCCH 应该在 DRX 的激活期发送给 UE。并且在 SPS 调度中，处于 DRX 休眠期的 UE 可以接收周期性配置的下行子帧上发送的 PDSCH 数据。

UE 在 measurement gap 期间既不会接收 DL-SCH，也不会发送 UL-SCH，此时即使配置了 SPS，UE 也不会在周期性的 SPS 子帧上接收 DL-SCH 或发送 UL-SCH。

结合前面的介绍可以看出，对于 UE 而言，SPS 的释放方式有以下几种：

（1）UE 收到指示 SPS 释放的 PDCCH：a）如果是下行，UE 会释放之前保存的用于下行 SPS 的下行分配（DL assignment），并且在 *timeAlignmentTimer* 正在运行的时候（UE 认为上行保持同步），UE 会针对这个指示下行 SPS 释放的 PDCCH 上报 ACK；b）如果是上行，UE 会释放之前保存的用于上行 SPS 的 UL grant（如果存在的话），但与下行不同的是，UE 并不会针对这个指示上行 SPS 释放的 PDCCH 回复 ACK/NACK。

注意：此时并不删除 RRC 层的 SPS 配置，即 SPS 仍然可以再次激活。

（2）UE 收到的 RRC 消息中的 *SPS-ConfigDL* 或 *SPS-ConfigUL* 配置成 release 时，UE 会释放对应的 SPS，并清除 SPS 配置。

（3）前面 2 种方式都是 eNodeB 显式地通知 UE 释放 SPS。对于上行 SPS，还存在一种隐式的释放方式：如果 UE 在分配的上行 SPS 资源上，连续 *implicitReleaseAfter* 次发送的 MAC PDU 不包含 MAC SDU，则会自动释放上行 SPS 并清除配置的 UL grant。这种方式主要是为了避免 UE 没有收到指示上行 SPS 释放的 PDCCH，而一直发送上行包。（UE 并不会对指示上行 SPS 释放的 PDCCH 回复 ACK/NACK，所以当 UE 丢失了这个 PDCCH 时，eNodeB 并不知道，因此也就不会重传这个 PDCCH）

小结：eNodeB 通过 *SPS-Config* 为特定 UE 配置 SPS 功能，然后使用 SPS C-RNTI 加扰的 PDCCH（上行/下行分别配置，NDI = 0）来激活该功能并分配 SPS 资源，之后 UE 周期性地使用该 SPS 资源来接收或发送数据。如果需要重传，下行 HARQ 需要使用 SPS C-RNTI 加扰的 PDCCH（NDI = 1）来指定重传所使用的资源；上行 HARQ 可以使用之前分配的 SPS 资源，也可以需要使用 SPS C-RNTI 加扰的 PDCCH（NDI = 1）来重新指定重传所使用的资源。

如果需要重新分配 SPS 资源，eNodeB 会下发一个使用 SPS C-RNTI 加扰的 PDCCH（上行/下行分别配置，NDI = 0）来重新激活 SPS。

如果 UE 在 SPS 子帧上收到以 C-RNTI 加扰的 PDCCH，即动态调度，则该 PDCCH 分配的资源将在该 TTI 覆盖 SPS 资源。

当 SPS 业务结束时，eNodeB 会下发一个使用 SPS C-RNTI 加扰的 PDCCH（NDI = 0）来释放 SPS。

【参考资料】

[1] TS 36.213 的 7.1 节 UE procedure for receiving the physical downlink shared channel

[2] TS 36.213 的 8.0 节 UE procedure for transmitting the physical uplink shared channel

[3] TS 36.213 的 9.2 节 PDCCH validation for semi-persistent scheduling

[4] TS 36.300 的 11.1 节 Basic Scheduler Operation

[5] TS 36.321 的 5.3.1 节 DL Assignment reception

[6] TS 36.321 的 5.4.1 节 UL Grant reception

[7] TS 36.321 的 5.10 节 Semi-Persistent Scheduling

[8] TS 36.331 的 *SPS-Config*

[9] 《4G LTE/LTE-Advanced for Mobile Broadband》的 13.2.3 节

[10] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 4.4.2 节

[11] 《LTE 中的半静态调度传输》

[12] 《Research and Solution of Semi-persistent Scheduling Problem in LTE System》

第25章 TTI bundling

不同的 UE 可能有不同的最大发射功率。有些 UE 的最大发送功率较低，其上行覆盖（uplink coverage）也就相对较小。对于某些业务，如位于小区边缘的 UE 的 VoIP 业务，在 1 ms 子帧内发送的数据可能无法得到可接受的出错率。TTI bundling 的目的就是为了提高小区边缘 UE 的上行 VoIP 覆盖。根据一些已知的仿真结果，上行使用 TTI bundling 能够带来 4 dB 的增益。

对于 VoIP 业务而言，其 QoS 要求：（1）延迟不超过 50ms；（2）包出错率应低于 1%。

在通常的传输中（无 TTI bundling，称之为 normal HARQ，后续会用到这个概念），一个 TB 会转换成多个冗余版本（Redundancy Version，RV），并在某个子帧上发送第一个 RV，而后续的 RV 是否发送取决于前一次传输的 ACK/NACK。图 25-1 是 FDD 下的 normal HARQ 举例。

对于小区边界的某些 UE 而言，其最大发射功率较低，重传的次数可能过多而导致 VoIP 业务的延迟超过 50 ms。

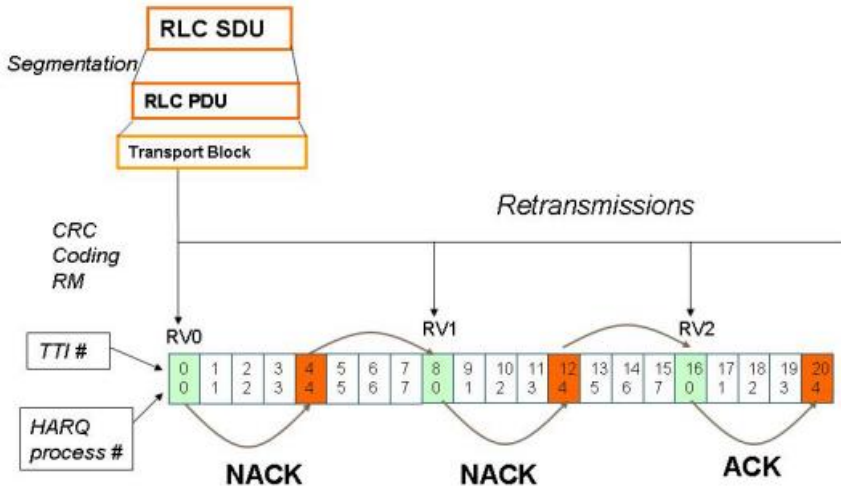


图 25-1：FDD 的 normal HARQ

常见的降低延迟并提高覆盖的解决方案是将 RLC SDU 分段，并在连续的 TTI 内发送（如图 25-2）。但这种方式会带来额外的头部开销（RLC/MAC header）和更多的控制信令（PDCCH）开销，同时 HARQ 反馈的出错率可能达到 1.2%，高于 VoIP 要求的 1%出错率。（详见参考资料[8]，里面有关于使用这种方案的缺点的详细说明）

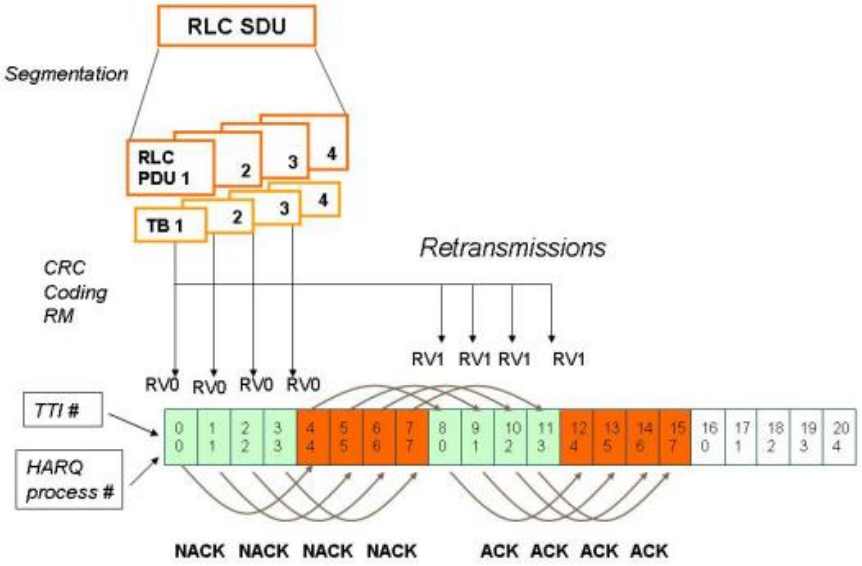


图 25-2: 将 RLC SDU 分段成多个 RLC PDUs，并使用 HARQ 来传输对应的 TB

为了解决上面提到的问题，LTE 提出了 TTI bundling 的解决方案。

TTI bundling 是在多个**连续**的上行子帧上多次发送同一个 TB（Transport Block），而无需等待 ACK/NACK 的技术。

注意：只有 UL-SCH 数据传输才支持 TTI bundling 功能。

在 TTI bundling 中，对应同一 TB 的不同 RV 可以在连续的子帧中发送，而不需要等待回应的 ACK/NACK。当对应该 TB 的所有传输都接收并处理完后，将会发送一个联合的 ACK/NACK。即在连续的子帧接收同一 TB 的多次传输（不同的 RV），并做软合并处理后，使用一个 ACK/NACK 做统一的回应。

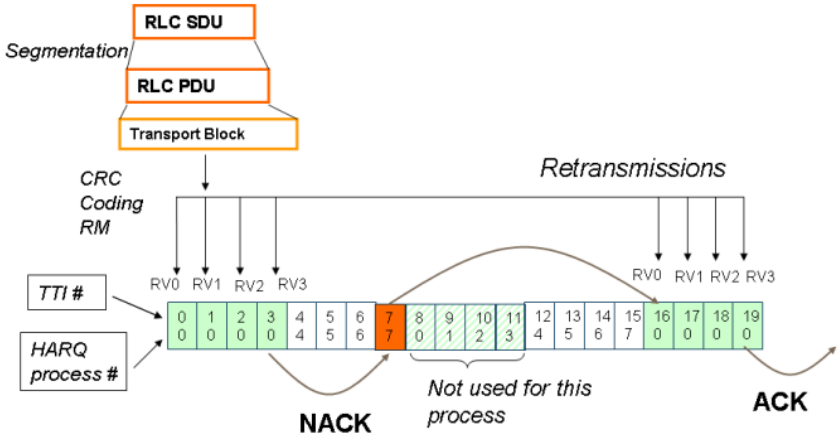


图 25-3：FDD 下的 TTI bundling

连续接收多个 RV 做软合并处理，明显比处理一个 RV 时的出错概率要低。

是否使能 TTI bundling 是通过 IE：*MAC-MainConfig -> ul-SCH-Config* 的 *ttiBundling* 字段来配置的。如果 *ttiBundling* 设置为 TRUE，则使能 TTI bundling；如果 *ttiBundling* 设置为 FALSE，则不使能 TTI bundling。

eNodeB 何时使能某个 UE 的 TTI bundling 呢？其中一种实现方式是 eNodeB 在某个给定的时间段内，通过接收对应 UE 的 power headroom 来计算该 UE 的可用功率是否低于某个阈值（例如：发射功率已接近 UE 的最大发射功率，但 SINR 值依旧很低），从而决定是否使能 TTI bundling 功能。或者是在发现 UE 处于小区边界（例如：很低的 SINR）并且 UE 使用类似于语音的业务（QCI=1）时，使能该 UE 的 TTI bundling 功能。

只有 FDD 和 TDD 0/1/6，才支持 TTI bundling。对于其它 4 种 TDD 上下行配置，由于一个系统帧内的上行子帧数小于 4 个，所以不支持 TTI bundling。

Table 4.2-2: Uplink-downlink configurations.

Uplink-downlink configuration	Downlink-to-Uplink Switch-point periodicity	Subframe number									
		0	1	2	3	4	5	6	7	8	9
0	5 ms	D	S	U	U	U	D	S	U	U	U
1	5 ms	D	S	U	U	D	D	S	U	U	D
2	5 ms	D	S	U	D	D	D	S	U	D	D
3	10 ms	D	S	U	U	U	D	D	D	D	D
4	10 ms	D	S	U	U	D	D	D	D	D	D
5	10 ms	D	S	U	D	D	D	D	D	D	D
6	5 ms	D	S	U	U	U	D	S	U	U	D

对于 TDD 而言，是不能同时使能 TTI bundling 和 SPS 的。（这是针对 Rel-10 以及之前版本而言的，后续版本可能会发生变化）

如果 UE 配置了 1 个或多个上行 SCell（不是下行 SCell），则 UE 不能配置 TTI bundling。（FDD 和 TDD 都适用，即上行载波聚合不支持 TTI bundling）

随机接入过程中的 MSG3 传输是不使用 TTI bundling 的。

当配置了 TTI bundling，参数 TTI_BUNDLE_SIZE（其值为 4。对于 TDD 而言，连续的 4 个上行子帧中间隔着下行子帧和特殊子帧）指定了绑定在一起的 TTI 数，这些 TTI 组成了一个 TTI bundle。属于同一 TTI bundle 的每一次传输（每个 TTI）都由同一个 HARQ process 来处理。

属于同一 TTI bundle 的多个子帧使用同一个 PDCCH（DCI format 0）来指示分配的上行资源。

当 UE 配置了 TTI bundling 以后，就不存在 normal HARQ 操作了 。也就是说，UE 收到的任一 UL Grant 都是对应一个 TTI bundle（不管该业务是不是 VoIP）。不存在 TTI bundling 和 normal HARQ 并存的情况。

只有对应 TTI bundle 的最后一个 TTI，会收到一个 HARQ ACK/NACK 反馈，而不管这个 TTI 是否发送数据（例如：当发生了 measurement gap）。也就是说，TTI bundle 内的所有 TTI 传输作为一个整体，统一反馈 HARQ ACK/NACK。也就是说，如果 UE 配置了 TTI bundling，则对应的 PHICH 资源与 TTI bundle 中的最后一个子帧相关联。

TTI bundle 的重传依然是一个 TTI bundle。

如果 UE 配置了 TTI bundling，则分配给该 UE 的上行资源不能多于 3 个 PRB，即 $N_{\mathrm{PRB}} \le 3$。同时其调制阶数（modulation order）必须配置成 $Q_m = 2$。（见 36.213 的 8.6.1 节）

FDD 下，如果 UE 配置了 TTI bundling，则有 4 个上行 HARQ process。

TDD 下，如果 UE 配置了 TTI bundling，则不同 TDD UL/DL configuration 下的上行 HARQ process 数见 36.213 的 Table 8-1。

Table 8-1: Number of synchronous UL HARQ processes for TDD

TDD UL/DL configuration	Number of HARQ processes for normal HARQ operation	Number of HARQ processes for subframe bundling operation
0	7	3
1	4	2
2	2	N/A
3	3	N/A
4	2	N/A
5	1	N/A
6	6	3

对于 FDD 而言，如果 UE 配置了 TTI bundling，当 UE 在子帧 n 收到了 DCI format 0（即 UL grant），并且（and，对应自适应重传）/或者（or，对应非自适应重传）在子帧 n - 5 收到了 PHICH，则 TTI bundle 内的第一个 PUSCH 会根据 PDCCH 或 PHICH 信息在子帧 n + 4 上传输。（见 36.213 的 8.0 节）

如图 25-3，UE 在 TTI #7 收到了 NACK（PHICH），将在 TTI #16 发送对应的重传。在 TTI #12，UE 可能收到 UL grant（对应自适应重传），也可能没有收到 UL grant（对应非自适应重传）。

对于 TDD Configuration 1 和 6 而言，如果 UE 配置了 TTI bundling，当 UE 在子帧 n 收到了 DCI format 0（即 UL grant），并且/或者在子帧 n - *l* 收到了 PHICH（*l* 值见 36.213 的 Table 8-2a），则 TTI bundle 内的第一个 PUSCH 会根据 PDCCH 或 PHICH 信息在子帧 n + k 上传输（k 值见 36.213 的 Table 8-2）。

对于 TDD Configuration 0 而言，如果 UE 配置了 TTI bundling，当 UE 在子帧 n 收到了 DCI format 0（即 UL grant），并且/或者在子帧 n - *l* 收到了 PHICH（*l* 值见 36.213 的 Table 8-2a）：（1）如果 DCI format 0 中的 UL index 字段的 MSB 设置为 1，或者 $I_{PHICH} = 0$，则 TTI bundle 内的第一个 PUSCH 会根据 PDCCH（UL grant）或 PHICH 信息在子帧 n + k 上传输（k 值见 36.213 的 Table 8-2）；（2）如果 DCI format 0 中的 UL index 字段的 LSB 设置为 1，或者 $I_{PHICH} = 1$，则 TTI bundle 内的第一个 PUSCH 会根据 PDCCH（UL grant）或 PHICH 信息在子帧 n + 7 上传输。

Table 8-2 *k* for TDD configurations 0-6

TDD UL/DL Configuration	subframe number *n*									
	0	1	2	3	4	5	6	7	8	9
0	4	6				4	6			
1		6			4		6			4
2				4					4	
3	4								4	4
4									4	4
5									4	
6	7	7				7	7			5

Table 8-2a *l* for TDD configurations 0, 1 and 6

TDD UL/DL Configuration	subframe number *n*									
	0	1	2	3	4	5	6	7	8	9
0	9	6				9	6			
1		2			3		2			3
6	5	5				6	6			8

接下来我们将介绍 eNodeB 在 TTI bundling 的调度中需要注意的事项。

当一个新的 TTI bundle 被调度时，需要避免与已经存在的 TTI bundle 的重传发生冲突。

对于同一个 UE 来说，一个 TTI 只能传输一个 TB（小区边界的 UE 通常不适用于空分复用的场景）。如果在同一子帧，需要重传一个旧的 bundle 和传输一个新的 bundle，就会产生冲突。如图 25-4 所示，一个新的 bundle 不能在 TTI #13 至 TTI #15 被调度（对应该 bundle 的最后一个 PUSCH 在 TTI #16 至 TTI #18 上发送），否则会与旧的 bundle 1 的重传产生冲突。

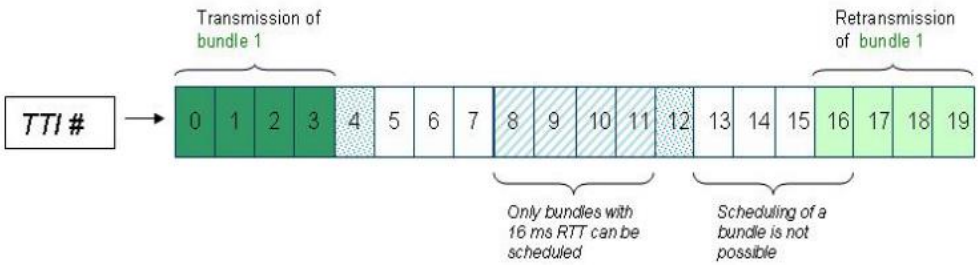


图 25-4：单个 UE 或多个 UE 的资源调度限制

对于同一子帧调度多个 UE，则需要避免多个 UE 被分配同样的频域资源。如图 25-4 所示，配置了 TTI bundling 的 UE，不能在 TTI #13 至 TTI #15（对应 bundle 的最后一个 PUSCH 在 TTI #16 至 TTI #18 上发送）调度使用已经被其它 UE 使用的相同频域资源（图中为 bundle 1 使用的频域资源）。同时，对于使用 normal HARQ 的 UE，其 RTT 为 8 ms，因此在 TTI #8 至 TTI #11（对应的重传将在子帧 TTI #16 至 TTI #19 上传输），也同样不能与 bundle 1 的重传使用相同的频域资源。

在调度过程中，要考虑到上述限制。并且在 eNodeB 侧，可能存在不同的调度 TTI bundling 的策略：

（1）同步调度策略（synchronous scheduling）：在这种策略下，所有 TTI bundle 的传输时间是同步的。以图 25-5 为例，所有的新的 TTI bundle 只能在每 4 个 TTI 处开始调度，即图中的 TTI #0、#4、#8、#12 …。这种方法的好处是调度更加直接，而且在网络负载较高的情况下，所有 TTI 上都可利用起来。通常 eNodeB 会使用这种方式。

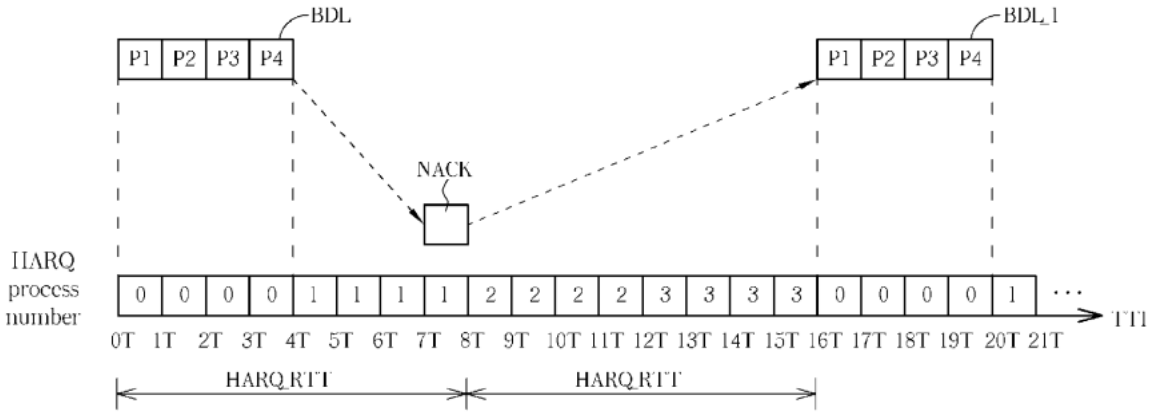


图 25-5：FDD 下，同步调度 TTI bundling 的例子

（2）非同步调度策略（non-synchronous scheduling）：在这种策略下，新的 TTI bundle 或 normal HARQ 在保证不冲突的情况下，可以在任意子帧被调度。即不同 UE 的 bundled HARQ process 和 normal HARQ process 可以任意混合。这种方法减少了调度的延时，但资源的有效分配算法比起同步调度策略来说更加复杂。

上边的介绍以 FDD 为主，TDD 的处理思想是类似的，只不过 timing 关系更复杂一些，这里就不做详细介绍了！

注：建议大家看看 参考资料[8]，本章的很多理解来自这篇文章。

从上面的介绍中可以看出，TTI bundling 有几个主要的限制：

- TTI bundling 只能用于上行传输；
- 如果 UE 使用了 TTI bundling，则该 UE 在一个 TTI 内至多只能被分配 3 RB；
- 如果 UE 使用了 TTI bundling，则该 UE 只能使用 QPSK 来发送上行数据；
- 如果 UE 使用了 TTI bundling，则该 UE 的所有的无线承载（不仅仅包括用于语音传输的 DRB）都只能使用 TTI bundling。

【参考资料】

[1] TS 36.213：搜索“TTI bundling”和“subframe bundling”

[2] TS 36.321 的 5.4.2.1 节 HARQ entity

[3] 《TTI Bundling in FDD and TD-LTE》

[4] 《TTI Bundling and VoIP Performance in LTE》

[5] 《LTE FDD 中的 TTI Bundling》

[6] 《VoLTE: Semi-Persistent Scheduling (SPS) and TTI Bundling》

[7] 《Method and apparatus of handling TTI bundling》

[8] 《LTE coverage improvement by TTI bundling》 这个必须读！！！

第26章 RLC

RLC 层位于 PDCP 层和 MAC 层之间。它通过 SAP（Service Access Point）与 PDCP 层进行通信，并通过逻辑信道与 MAC 层进行通信。每个 UE 的每个逻辑信道都有一个 RLC 实体（RLC entity）。RLC 实体从 PDCP 层接收到的数据，或发往 PDCP 层的数据被称作 RLC SDU（或 PDCP PDU）。RLC 实体从 MAC 层接收到的数据，或发往 MAC 层的数据被称作 RLC PDU（或 MAC SDU）。

RLC 层主要负责（见 36.322）：

- **分段/串联和重组 RLC SDU**（concatenation/segmentation/reassembly，只适用于 UM 和 AM 模式）：RLC PDU 的大小是由 MAC 层指定的，其大小通常并不等于 RLC SDU 的大小，所以在发送端需要分段/串联 RLC SDU 以便其匹配 MAC 层指定的大小。相应地，在接收端需要对之前分段的 RLC SDU 进行重组，以便恢复出原来的 RLC SDU 并按序递送（in-sequence delivery）给上层。

- 通过 **ARQ** 来进行纠错（只适用于 AM 模式）：MAC 层的 HARQ 机制的目标在于实现非常快速的重传，其反馈出错率大概在 1%左右。对于某些业务，如 TCP 传输（要求丢包率小于 10^{-5}），HARQ 反馈的出错率就显得过高了。对于这类业务，RLC 层的重传处理能够进一步降低反馈出错率。

- 对 RLC data PDU 进行**重排序**（reordering，只适用于 UM 和 AM 模式）：MAC 层的 HARQ 操作可能导致到达 RLC 层的报文是乱序的，所以需要 RLC 层对数据进行重排序。重排序是根据序列号（Sequence Number，SN）的先后顺序对 RLC data PDU 进行排序的。

- **复包检重测**（duplicate detection，只适用于 UM 和 AM 模式）：出现重复包的最大可能性为发送端反馈了 HARQ ACK，但接收端错误地将其解释为 NACK，从而导致了不必要的 MAC PDU 重传。

- 对 RLC data PDU 进行**重分段**（resegmentation，只适用于 AM 模式）：当 RLC data PDU（注意：这里不是 SDU）需要重传时，可能需要进行重分段。例如，当 MAC 层指定的大小小于需要重传的原始 RLC data PDU 的大小时，就需要对原始 RLC data PDU 进行重分段。

按序递送（in-sequence delivery）指的是 RLC 实体的接收端必须按序将重组（reassembly）好的 SDU 发送给 PDCP 层，也就是说，SDU n 必须在 SDU n+1 之前发送给 PDCP 层。其基本思想是将接收到的 RLC PDU（假设其 SN = x）放在接收 buffer（reception buffer）中，直到较小 SN（小于 x）的所有 PDU 都已成功接收并递送给 PDCP 层。只有当拥有较小 SN 的所有 RLC PDU 都用于重

组 SDU 后，下一个 RLC PDU 才会被使用。例如对于类似 VoLTE 的流应用，要求接收到的数据的顺序与它们被发送时的顺序是一致的，否则可能造成声音的紊乱。

RLC 层的功能是由 RLC 实体来实现的。一个 RLC 实体可以配置成以下 3 种模式之一：

- Transparent Mode（TM）：对应 TM RLC 实体，简称 TM 实体。该模式可以认为是空的 RLC，因为这种模式下只提供数据的透传（pass through）功能。
- Unacknowledged Mode（UM）：对应 UM RLC 实体，简称 UM 实体。该模式提供除重传和重分段外的所有 RLC 功能，因此提供了一种不可靠的传输服务。
- Acknowledged Mode（AM）：对应 AM RLC 实体，简称 AM 实体。通过出错检测和重传，AM 模式提供了一种可靠的传输服务。该模式提供了所有的 RLC 功能。

除 TM 模式对应的逻辑信道外，每个逻辑信道对应的 RLC 实体的模式（在 UM 模式和 AM 模式之间进行选择）是在无线承载建立时，eNodeB 通过相关 RRC 消息的 RLC-Config 字段来配置的。在 36.331 中，搜索“RLC-SAP”，能看到各种 RRC 消息所使用的 RLC 模式（以及 SRB）。

每种模式支持的 RLC 层功能见表 26-1（见 36.322 的 4.4 节）。

表 26-1：每种模式支持的 RLC 功能

RLC 功能	TM	UM	AM
传输上层 PDU	Yes	Yes	Yes
使用 ARQ 进行纠错	No	No	Yes
对 RLC SDU 进行分段、串联和重组	No	Yes	Yes
对 RLC data PDU 进行重分段	No	No	Yes
对 RLC data PDU 进行重排序	No	Yes	Yes
重复包检测	No	Yes	Yes
RLC SDU 丢弃处理	No	Yes	Yes
RLC 重建	Yes	Yes	Yes
协议错误检测	No	No	Yes

26.1 TM 模式

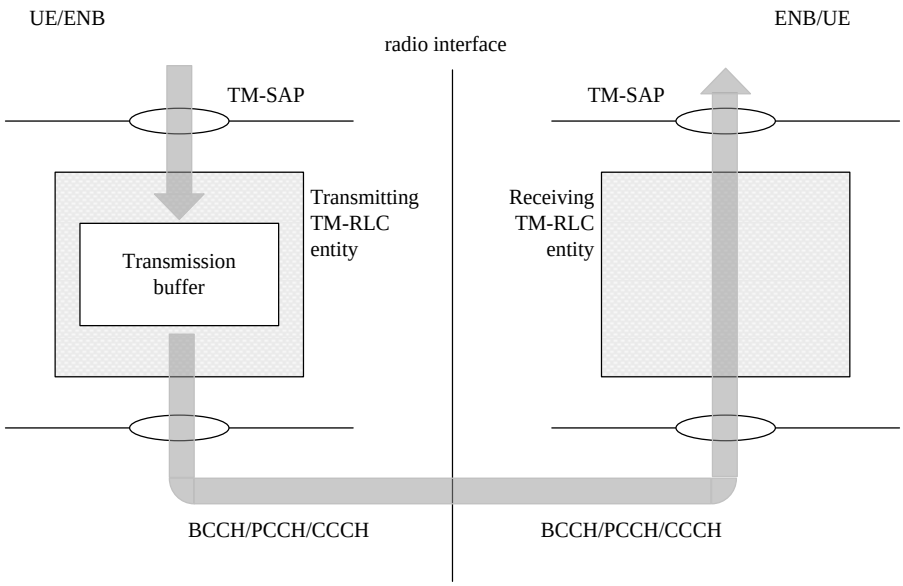


Figure 4.2.1.1.1-1: Model of two transparent mode peer entities

TM 模式下，RLC 实体只进行透传，不对 RLC SDU 进行分段和串联，也不添加任何头部信息。TM 模式通过逻辑信道 BCCH、PCCH 和 DL/UL CCCH 来接收/发送 RLC PDU。

在 eNodeB 或 UE 侧，一个 TM 实体只能接收或发送数据，而不能同时收发数据，即 TM 实体只提供单向的数据传输服务。

从 36.322 的 Figure 4.2.1.1.1-1 可以看出，在发送端，一个 TM 实体只由一个保存 RLC SDU 的传输 buffer 组成。当 MAC 层告诉该 TM 实体有一个传输机会时，TM 实体会将传输 buffer 中的 1 个 RLC SDU 直接发送给 MAC 层，而不做任何修改。在接收端，一个 TM 实体直接将从 MAC 层接收到的 RLC PDU 发送给 PDCP 层。

对于 TM 实体来说，一个 RLC SDU 等同于一个 RLC PDU。协议中，TM 实体传输的 PDU 被称为 TMD PDU。TMD PDU 只由一个 Data 域组成，并不包含任何 RLC 头部。见 36.322 的 Figure 6.2.1.2-1。

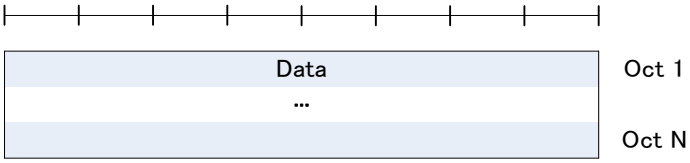


Figure 6.2.1.2-1: TMD PDU

只有那些无需 RLC 配置的 RRC 消息会使用 TM 模式，如系统消息 SI、Paging 消息以及使用 SRB0 的 RRC 消息。DRB 不支持 TM 模式，只支持 UM 模式或 AM 模式。

26.2 UM 模式

UM 模式不提供重传和重分段功能，其提供的是一种不可靠的服务。UM 模式常用于实时性要求较高的业务，如 VoIP 等，这种业务允许有一定的错包或丢包，但对延迟较为敏感，同时要求按序传输，并丢弃重复报文。点到多点的服务，如 MBMS，由于没有可用的反馈路径而不能使用 AM 模式，也使用 UM 模式。

UM 模式通过逻辑信道 DL/UL DTCH、MCCH 或 MTCH 来接收/发送 RLC PDU。

与 TM 模式类似，一个 UM 实体只能接收或发送数据，而不能同时收发数据。UM 实体只提供单向的数据传输服务。

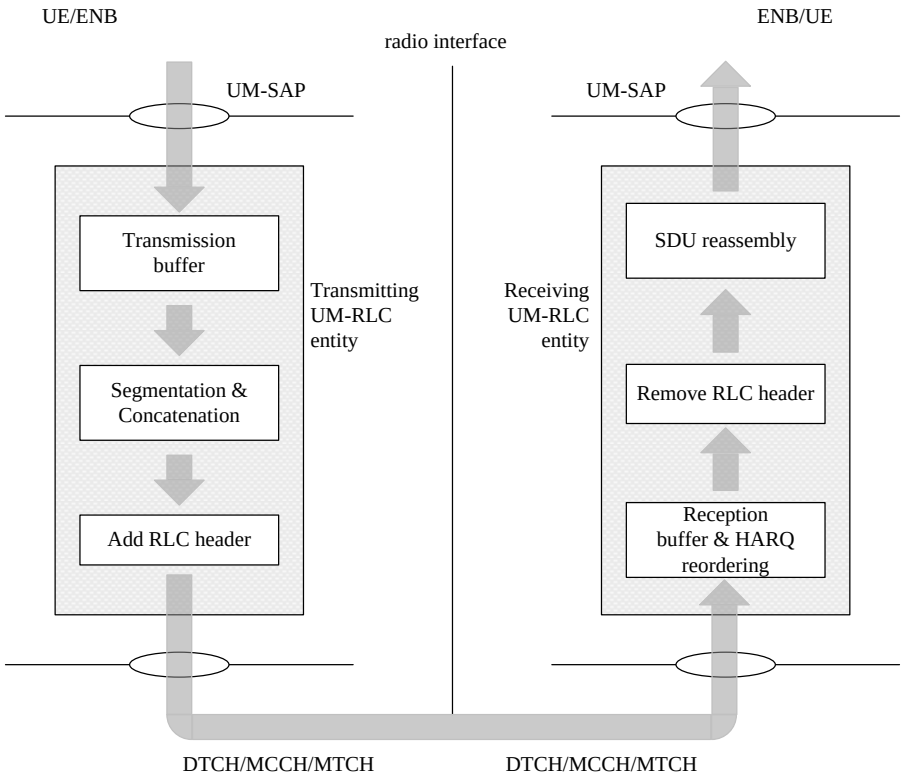


Figure 4.2.1.2.1-1: Model of two unacknowledged mode peer entities

26.2.1 发送端

UM 实体在发送端需要做 2 件事：（1）将来自上层（PDCP 层）的 RLC SDU 缓存在传输 buffer（transmission buffer）中；（2）在 MAC 层通知其发送 RLC PDU 时，分段/串联 RLC SDU 以生成 RLC PDU，并赋予合适的 SN 值，然后将生成的 RLC PDU 发给 MAC 层。

UM 实体在发送端会维护如下变量：

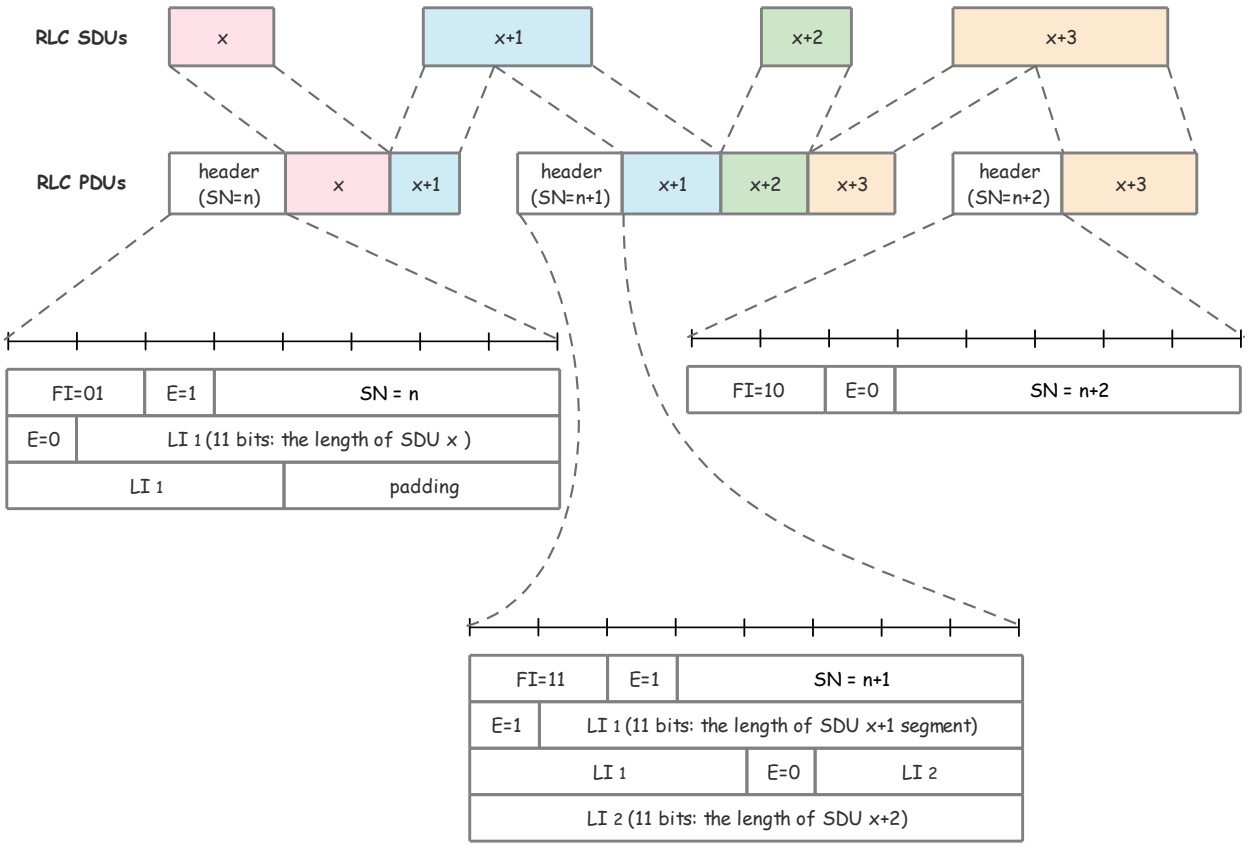
- VT(US)：该变量保存了下一个新生成的 UMD PDU 将被赋予的 SN 值。该变量初始化时为 0，并在 UM 实体发送一个 SN = VT(US)的 UMD PDU 给 MAC 层时更新。简单地说，“VT(US) – 1”等于最近一个已经发送的 UMD PDU 的 SN 值。

26.2.1.1 分段和串联

UM 实体发送 RLC PDU 的前提条件是：MAC 层通知 UM 实体发送一个 RLC PDU，即通知 UM 实体有一个传输机会。MAC 层同时会告诉 UM 实体在这次传输机会中，可以传输的 RLC PDU 的总大小。

由 MAC 层指定的 RLC PDU 的大小通常并不等于 RLC SDU 的大小，所以在发送端需要分段/串联 RLC SDU 以便生成的 RLC PDU 匹配 MAC 层指定的大小。

生成一个 RLC PDU 时，是按照 RLC SDU 到达 UM 实体的顺序来逐个将 RLC SDU 放入 RLC PDU 中的。



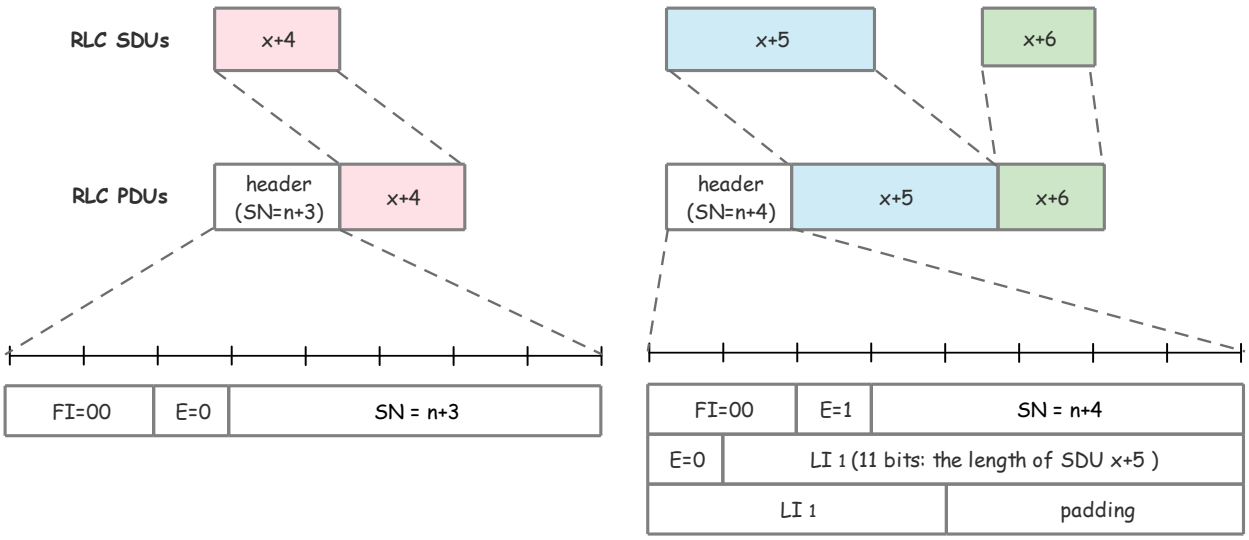


图 26-1：UM 分段和串联功能举例

图 26-1 是 UM 分段和串联功能的一个例子。UM 实体的传输 buffer 里有 7 个 RLC SDU，对应 SDU x 至 SDU x+6。当 MAC 层通知该 UM 实体发送一个 RLC PDU，并指定该 PDU 的大小时，UM 实体发现该大小只能容纳整个 SDU x 和部分 SDU x+1（以及生成的 RLC header）。此时 UM 实体会将 SDU x+1 分段，并将分段的前半部分与 SDU x 串联起来，再加上相应的 RLC header 后，生成 SN = n 的 RLC PDU，并将其发给 MAC 层。

在 MAC 层通知的下一个传输机会里，UM 实体发现 MAC 层指定的大小只能容纳下 SDU x+1 的后半部分、整个 SDU x+2 以及部分 SDU x+3（以及生成的 RLC header）。此时该实体会将 SDU x+3 分段，并将 SDU x+1 的后半部分、SDU x+2 以及 SDU x+3 的前半部分串联起来，再加上相应的 RLC header 后，生成 SN = n+1 的 RLC PDU，并将其发给 MAC 层。

在接下来的传输机会里，UM 实体发现 MAC 层指定的大小正好能容纳下 SDU x+3 的后半部分及其相应的 RLC header。此时该实体会将 SDU x+3 的后半部分加上相应的 RLC header 后，生成 SN = n+2 的 RLC PDU，并发给 MAC 层。

在生成 SN = n+3 的 PDU 时，MAC 层指定的大小正好能容纳下 SDU x+4 及其相应的 RLC header。此时无需分段，也不需要串联，将 SDU x+4 加上相应的 RLC header 后，生成 SN = n+3 的 RLC PDU，然后将其发给 MAC 层即可。

在接下来的传输机会里，MAC 层指定的大小正好能容纳下 SDU x+5、SDU x+6 及其相应的 RLC header。此时无需分段，只需要将 SDU x+5 和 SDU x+6 串联起来，加上相应的 RLC header 后，生成的 SN = n+4 的 RLC PDU，然后将其发给 MAC 层即可。

26.2.1.2 UMD PDU

结合图 26-1 的例子，我们进一步介绍 UMD PDU 的结构及其相关字段的作用。

UMD PDU 由 2 部分组成：Data 域和 header（头部）。

Data 域由一个或多个“Data field element”组成。一个“Data field element”对应一个 RLC SDU 或一个 RLC SDU 分段。Data field element 是按照 RLC SDU 到达 RLC 实体的先后顺序映射到 Data 域的。例如：图 1 中 SN = n 的 RLC PDU 的 Data 域包含了 2 个“Data field element”，分别对应 SDU x 和 SDU x+1 的分段。SN = n+1 的 RLC PDU 的 Data 域包含了 3 个“Data field element”，分别对应 SDU x+1 的分段、SDU x+2 和 SDU x+3 的分段。

一个 RLC PDU 的 Data 域按照顺序由“0 个或 1 个 SDU 分段 + 0 个或多个 SDU + 0 个或 1 个 SDU 分段”组成。也就是说，SDU 分段只可能出现在 Data 域的最开始或者最后。

header 由固定部分（每个 PDU 都有）和可能存在的扩展部分组成。

固定部分（fixed part）由一个 FI（2 比特）、一个 E（1 比特）和一个 SN（5 比特或 10 比特）字段组成，其本身是字节对齐（byte-aligned）的。

SN 唯一指定了一个 UMD PDU。SN 的长度由 RRC 层下发的 *sn-FieldLength* 字段指定，其值可以为 5 比特，也可以为 10 比特。如果使用 5 比特的 SN，则固定部分长为 1 字节；如果使用 10 比特的 SN，则固定部分长为 2 字节（多出来的 3 比特是预留的，用 R1 表示）。

扩展部分（extension part）由 1 个或多个“E（1 比特） + LI（11 比特）”组成。只有当 Data field element 的个数多于 1 个时，才存在扩展部分（extension part）。除了最后一个 Data field element 外，其它 Data field element 都有一个对应的“E + LI”。如果 LI 的个数为奇数，则最后一个 LI 之后需要添加 4 比特的 padding。可以看出，扩展部分中“E + LI”的个数等于 Data field element 的个数减去 1。

关于 UMD PDU 的结构，可参见 36.322 的 6.2.1.3 节。（或见下图。图中最左边的比特对应 MSB，最右边的比特对应 LSB）

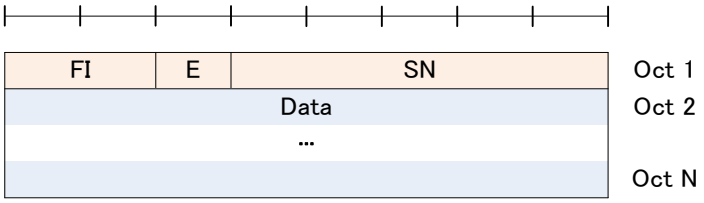


Figure 6.2.1.3-1: UMD PDU with 5 bit SN (No LI)

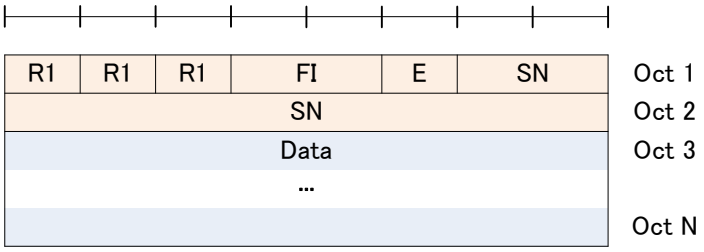


Figure 6.2.1.3-2: UMD PDU with 10 bit SN (No LI)

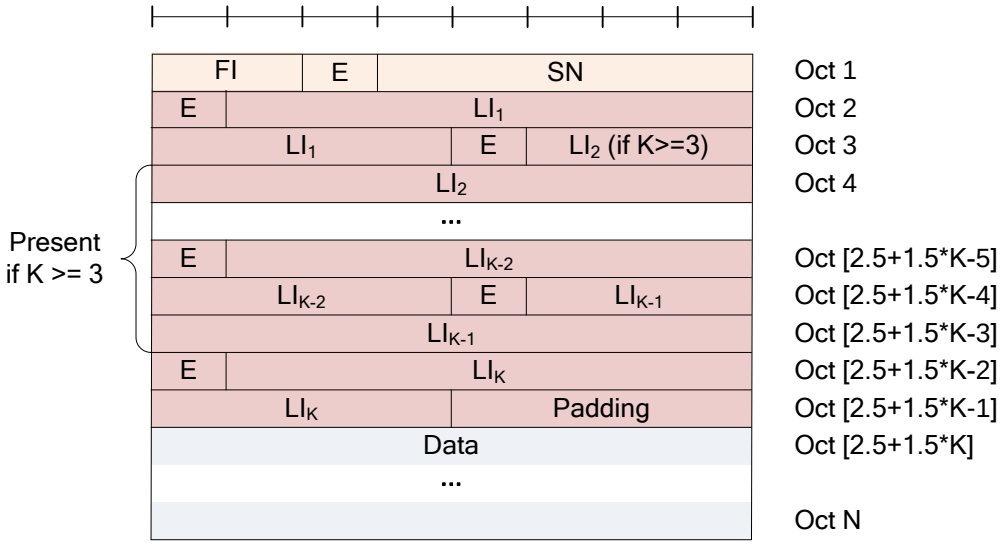


Figure 6.2.1.3-3: UMD PDU with 5 bit SN (Odd number of LIs, i.e. K = 1, 3, 5, …)

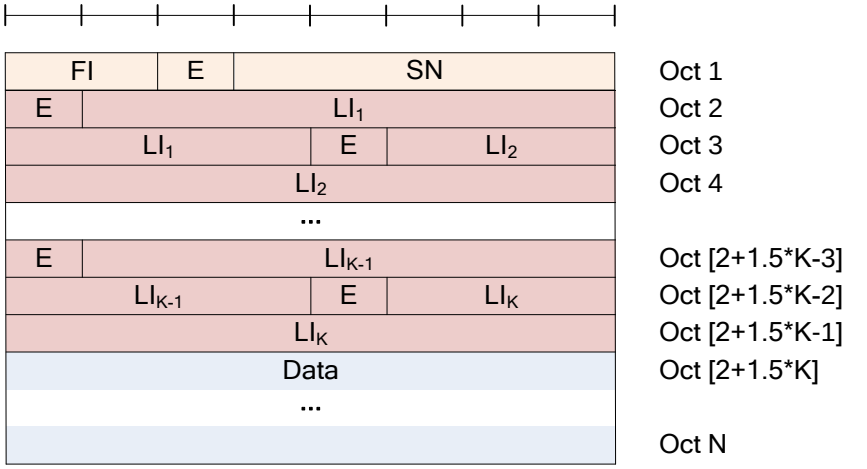


Figure 6.2.1.3-4: UMD PDU with 5 bit SN (Even number of LIs, i.e. K = 2, 4, 6, …)

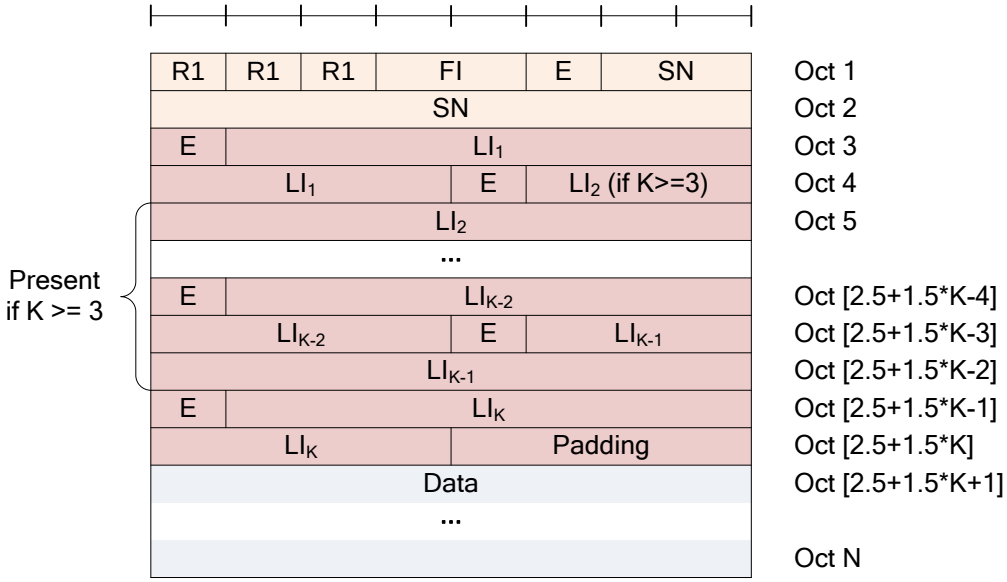


Figure 6.2.1.3-5: UMD PDU with 10 bit SN (Odd number of LIs, i.e. K = 1, 3, 5, …)

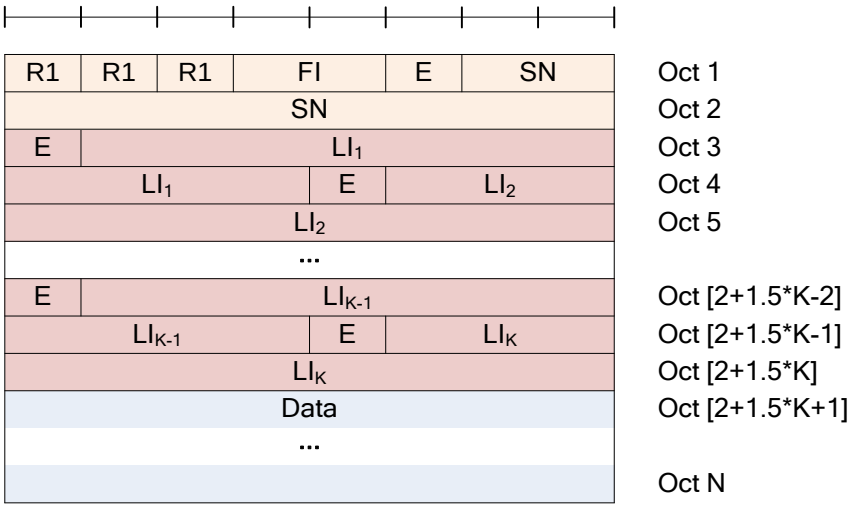


Figure 6.2.1.3-6: UMD PDU with 10 bit SN (Even number of LIs, i.e. K = 2, 4, 6, …)

E 是 Extension bit 的意思，该字段用于指示“固定部分”或“E + LI”之后紧接着的是 Data 域，还是一个“E + LI”。值为 0 表示之后紧跟着 Data 域，值为 1 表示之后紧跟着一个“E + LI”。

LI 是 Length Indicator 的意思，该字段用于指示对应的 Data field element（SDU 或 SDU 分段）的长度（以字节为单位）。

FI 是 Framing Info 的意思，该字段用于指示在 Data 域的开始或结束位置的 Data field element 是不是一个 RLC SDU 分段。该字段长为 2 比特，高比特位表示 Data 域的第一个字节是否是一个 RLC SDU 的第一个字节，低比特位表示 Data 域的最后一个字节是否是一个 RLC SDU 的最后一个字节（0 对应“是”，1 对应“否”）。这 2 比特可以分别对应不同的 RLC SDU，也可以对应同一 RLC SDU。

图 26-1 假设使用 5 比特的 SN 域。SN = n 的 RLC PDU 由 2 个 Data field element（SDU x 和 SDU x+1 的分段）组成，扩展部分包含 1 个“E + LI”，因此固定部分的 E 设置为 1，表示固定部分之后紧接着 1 个“E + LI”。扩展部分的“E + LI”中的 E 值为 0，表示其后紧接着 Data 域，LI（对应 LI_1）指示了 Data 域中 SDU x 的长度。由于 Data 域的第一个字节对应 SDU x 的第一个字节，Data 域的最后一个字节并不对应 SDU x+1 的最后一个字节，所以 FI 的值设置为 01。

SN = n+1 的 RLC PDU 由 3 个 Data field element（SDU x+1 的分段、SDU x+2 和 SDU x+3 的分段）组成，扩展部分包含 2 个“E + LI”，因此固定部分的 E 设置为 1，表示固定部分之后紧接着 1 个“E + LI”。扩展部分的第一个“E + LI”中的 E 值为 1，表示其后紧接着 1 个“E + LI”，LI（对应 LI_1）指

示了 Data 域中 SDU x+1 的分段的长度。扩展部分的第二个“E + LI”中的 E 值为 0，表示其后紧接着 Data 域，LI（对应 LI_2）指示了 Data 域中 SDU x+2 的长度。由于 Data 域的第一个字节并不对应 SDU x+1 的第一个字节，Data 域的最后一个字节也不对应 SDU x+3 的最后一个字节，所以 FI 的值设置为 11。

SN = n+2 的 RLC PDU 由 1 个 Data field element（SDU x+3 的分段）组成，扩展部分包含 0 个“E + LI”，因此固定部分的 E 设置为 0，表示固定部分之后紧接着 Data 域。由于 Data 域的第一个字节并不对应 SDU x+3 的第一个字节，Data 域的最后一个字节对应 SDU x+3 的最后一个字节，所以 FI 的值设置为 10。

接着对比 SN = n+3 和 SN = n+4 的 PDU 可以看出，虽然二者的 FI 都设置为 00，但 SN = n+3 的 PDU 的 FI 字段的 2 比特对应的是同一 SDU，而 SN = n+4 的 PDU 的 FI 字段的 2 比特分别对应不同的 SDU。

包括 UMD PDU 以及后面将介绍到的 AMD PDU、AMD PDU segment 和 STATUS PDU 在内，其对应的 RLC PDU 总是字节对齐的，并且 PDU 的最后不存在 padding。

从上面的介绍可以看出，RLC header 里并不指定 Data 域中最后一个 Data field element 的大小。这是因为 RLC PDU 的长度是由 MAC 指定的，该长度会在 MAC PDU 中对应该 RLC PDU 的 subheader 中的 L 字段中体现（见 36.321 的 6.1.2 节和 6.2.1 节）。“RLC PDU 的长度 - RLC PDU header 的长度 - 所有 LI 之和”即为最后一个 Data field element 的长度。

当 UM 实体要发送一个新的 UMD PDU 给 MAC 层时，它会将该 PDU 的 SN 设置成 VT(US)，然后将 VT(US)加 1。

26.2.2 接收端

UM 实体在接收端主要做几件事：（1）对分段的 RLC SDU 进行重组（reassembly），以便恢复出原来的 RLC SDU 并发往 PDCP 层；（2）对 RLC PDU 进行重排序（reordering）；（3）检测并丢弃重复包（duplicate detection）。

如果接收端收到的 RLC PDU 是乱序的，则需要先进行重排序。由于 MAC 层使用多个 HARQ process 来处理 HARQ，因此乱序到达是不可避免的（可参见 11.2 节的介绍）。乱序到达的 RLC PDU 会先保存在接收 buffer 中，直到之前的 RLC PDU 都已成功接收并递送给 PDCP 层。（同样适用于 AM 模式）

如图 26-2 所示，MAC 层的 HARQ 处理导致了 RLC PDU 以 PDU 8、PDU 6 和 PDU 9 的顺序到达接收端的 RLC 层。此时接收端会对接收到的 PDU 进行重排序处理，并将 PDU 以 PDU 6、PDU 8 和 PDU 9 的顺序保存在接收 buffer 中。

在重排序期间，通过校验收到的 RLC PDU 的 SN 值，接收端可以知道是否收到了重复包。重复包将被丢弃，从而保证上层不会收到重复的数据。（同样适用于 AM 模式）

接收端需要检测 MAC 层是否丢失了某个 RLC PDU，并避免过度的重排序延迟。简单地说，接收端只会等待还未收到的 RLC PDU 一段时间，等不到就不等了。重排序定时器 *t_Reordering* 决定了在多长时间内等待一个还未收到的 PDU。每个 UM 实体只有一个 *t_Reordering*。使用该定时器的目的是为了检测 MAC 层是否丢失了某个 RLC PDU，如果在 *t_Reordering* 指定的时间内没有收到该 PDU，则接收端认为该 PDU 已经丢失了，且 UM 实体不会再去尝试接收这些已经丢失了的 PDU。（同样适用于 AM 模式。但不同的是，AM 模式会要求对端重传丢失了的 PDU，而不是直接丢弃）

只有当一个 RLC SDU 的所有分段都存在于接收 buffer 里时，才能从保存的 RLC PDU 中重组出该 SDU。只要有部分分段没有被接收到，该 RLC SDU 的所有数据都会被丢弃。也就是说，如果某个 RLC SDU 的一个或几个分段所在的 RLC PDU 丢失了，而导致包含其某个分段的另一个 RLC PDU 无法重组出该 RLC SDU，则收到的 RLC SDU 分段将被丢弃（但使用收到的 PDU 成功重组出的其它 SDU 要发送给 PDCP 层）。（同样适用于 AM 模式。但不同的是，AM 模式会要求对端重传丢失了的 PDU）

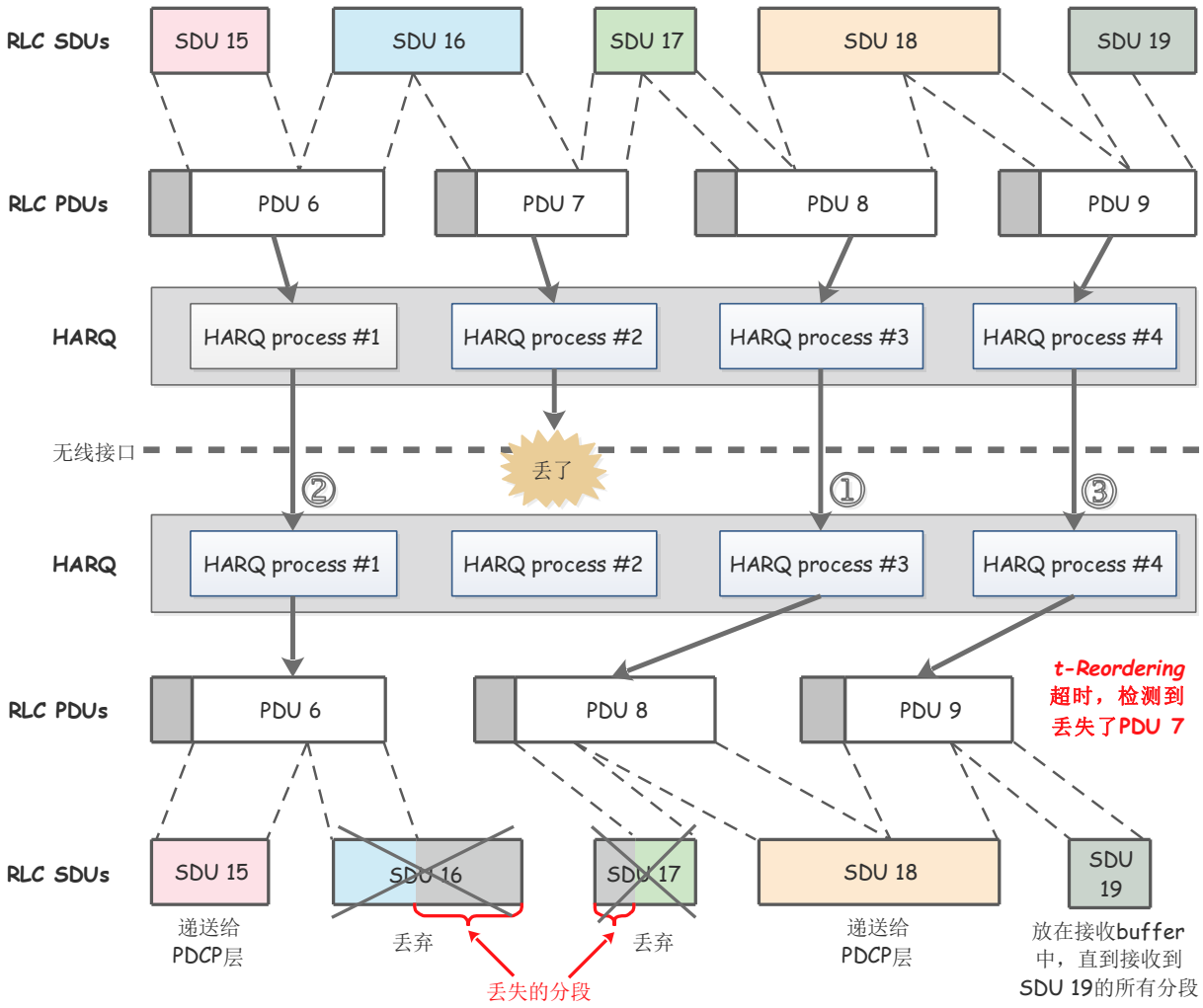


图 26-2：UM 模式中，PDU 丢失检测及处理

如图 26-2 所示，当接收端收到 PDU 8 时，会启动定时器 *t_Reordering*。由于在该定时器超时之前，PDU 7 还没收到，因此接收端认为 PDU 7 丢失了。接收端在重组 RLC SDU 时，由于 PDU 7 的丢失导致了 SDU 16 和 SDU 17 的部分分段丢失，从而无法重组出完整的 SDU 16 和 SDU 17，因此 SDU 16 和 SDU 17 的已接收分段将被丢弃。SDU 15 和 SDU 18 是完全接收的，所以会被递送给 PDCP 层。SDU 19 的已接收分段会继续放在接收 buffer 中，直到接收到 SDU 19 的所有分段（最终也有可能被丢弃）。

重排序和重复包检测功能并不会应用在使用 MCCH 或 MTCH 的 UM 实体上，因为这些信道在 MAC 层并不使用 HARQ 操作。

26.2.2.1 接收端相关变量

在详细介绍 UM 模式接收端处理流程之前，我们需要先介绍接收端的相关变量。UM 实体在接收端会维护如下变量：

VR(UR)： UE 接收状态变量（UM receive state variable）。该变量保存了等待重排序的最早一个 UMD PDU 的 SN 值。该变量初始化时为 0。接收端认为 SN 小于 VR(UR)的 UMD PDU 都已被成功接收（即使没有成功接收，也认为小于 VR(UR)的 UMD PDU 已经丢失而不再去接收了），VR(UR)对应重排序窗口内还未接收到的拥有最小 SN 的 UMD PDU。

VR(UX)：UM *t-Reordering* 状态变量（UM *t-Reordering* state variable）。该变量保存了触发 *t-Reordering* 的 UMD PDU 的 SN 值的下一个 SN。当启动 *t-Reordering*（同时会更新 VR(UX)）时，说明有小于 VR(UX)的 UMD PDU 还未接收到，此时需要等待这些 UMD PDU 以便进行重排序。

VR(UH)：UM 最高接收状态变量（UM highest receiving state variable）。该变量保存的 SN 值等于所有已经接收到的 UMD PDU 中，拥有最高 SN 的那个 UMD PDU 的 SN 值，再加 1。该变量初始化时为 0。该变量对应重排序窗口的上边界（不包含 VR(UH)）。简单地说，“VR(UH) – 1”等于已接收的拥有最高 SN 的 UMD PDU 的 SN 值。

Reordering window：重排序窗口。如果一个 SN 满足(VR(UH) – UM_Window_Size) ≤ SN < VR(UH)，则该 SN 位于重排序窗口内；否则该 SN 位于重排序窗口外。UM RLC 接收实体不会去接收“SN < (VR(UH) – UM_Window_Size)”的 UMD PDU，也就是说，接收端认为 SN 小于重排序窗口下边界的 UMD PDU 都已经成功接收。重排序窗口指定了在不向前移动该窗口的前提下，能够接收的 PDU 的数量。如果重排序窗口向前移动，则位于窗口之外的任一 PDU，不管其状态如何，都需要进行重组，并将生成的 SDU 按序发往 PDCP 层。

UM_Window_Size：对应重排序窗口的大小。当使用 5 比特 SN 时，UM_Window_Size = 16；当使用 10 比特 SN 时，UM_Window_Size = 512；当 UM RLC 接收实体用于 MCCH 或 MTCH 时，UM_Window_Size = 0。

图 26-3 是除 UR(X)外的 UM 接收端相关变量的一个例子。

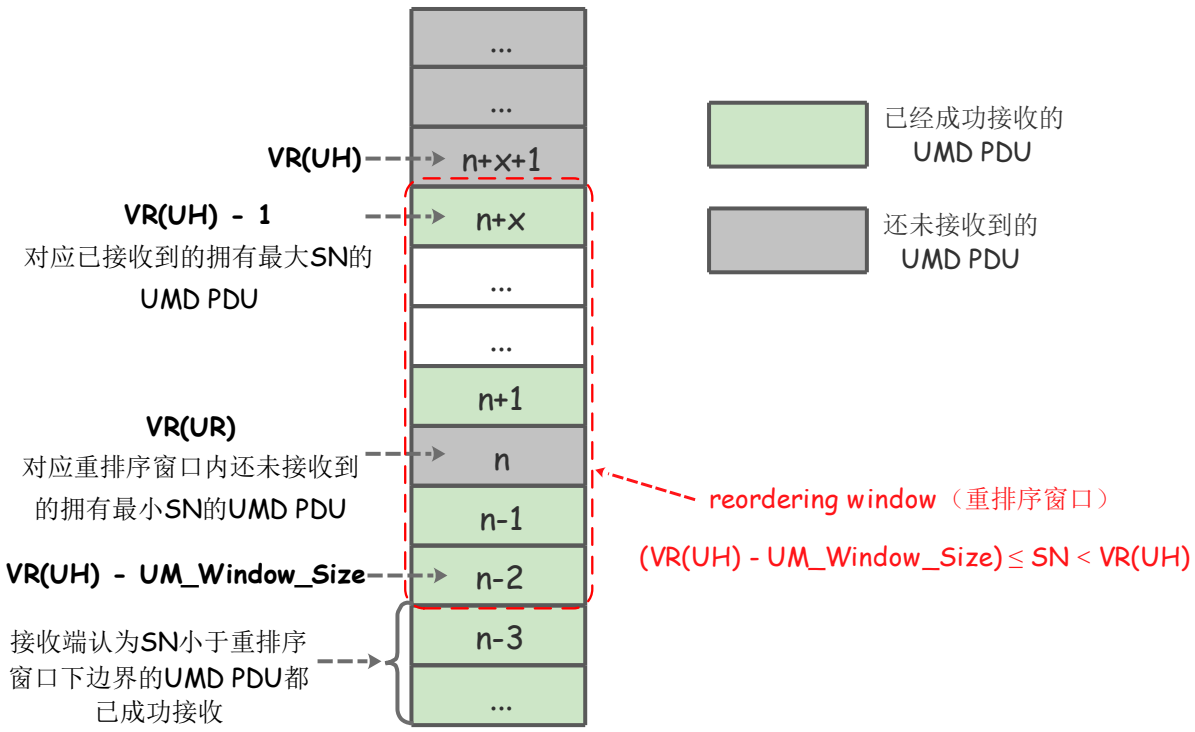


图 26-3：除 VR(UX)外的 UM 接收端相关变量

26.2.2.2 丢弃处理

当 UM 实体从 MAC 层收到一个 UMD PDU 时，它会判断是丢弃该 PDU 还是将其放入接收 buffer 中。

当接收端收到一个 SN = x 的 UMD PDU 时，在以下 3 种情况下该 PDU 会被丢弃：

情况 1：如果 VR(UR) < x < VR(UH)，且之前已经成功接收到 SN = x 的 UMD PDU，则该 PDU 是重复包，予以丢弃。如图 26-4 的 case 1。

情况 2：由于接收端认为 SN < VR(UR)的 PDU 都已经成功接收，所以当(VR(UH) – UM_Window_Size) ≤ x < VR(UR)时，该 PDU 也会被丢弃。如图 26-4 的 case 2。

情况 3：接收端不会去接收重排序窗口之外，且 SN < (VR(UH) – UM_Window_Size)的 UMD PDU，所以 x < (VR(UH) – UM_Window_Size)的 PDU 将被丢弃。如图 26-4 的 case 3。

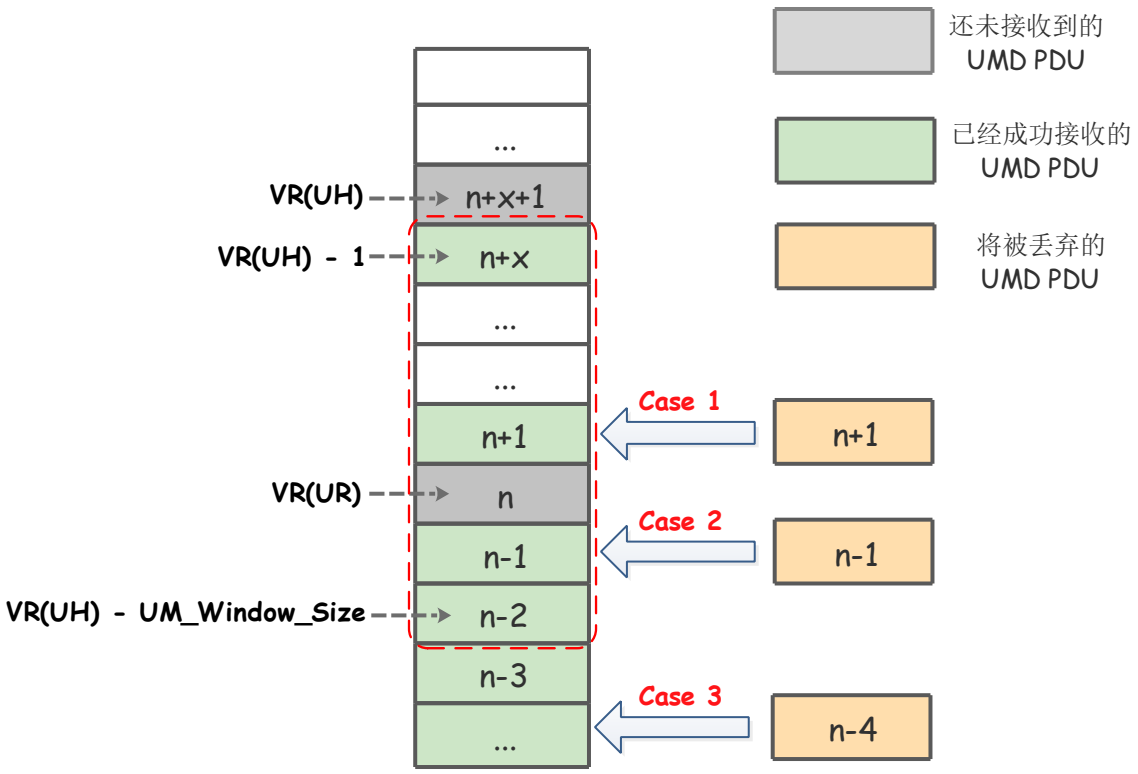


图 26-4：将被丢弃的 UMD PDU 举例

而在其它情况下，接收端会将 UMD PDU 放入接收 buffer 中，并按照下一节的介绍进行处理。

26.2.2.3 接收 buffer 中的 UMD PDU 的处理

当一个 SN = x 的 UMD PDU 放入接收 buffer 时，接收端会按如下步骤进行操作。

步骤 1：如果 x 位于重排序窗口之外（此时 x 一定是超出了重排序窗口的上边界，即 x ≥ VR(UH)），

- 将 VR(UH)设置成 x + 1。
- 由于 VR(UH)的更新，重排序窗口也将相应地向前移动，此时必定会有一些 UMD PDU 移到了重排序窗口之外。因此需要对重排序窗口之外的 UMD PDU 进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。
- 如果此时 VR(UR)也移到了重排序窗口之外，则将 VR(UR)设置成(VR(UH) – UM_Window_Size)，即设置成重排序窗口的下边界。其中一种场景发生在接收端一直没有

收到 SN = VR(UR)的 UMD PDU，但不断地收到新的位于重排序窗口上边界之外的 UMD PDU，从而 VR(UH)不断前移，直至 VR(UR)落到重排序窗口之外。

步骤 2：如果在接收 buffer 中包含了一个 SN = VR(UR)的 UMD PDU，则

- 将 VR(UR)更新成“SN > 当前 VR(UR)”且还未接收到的第一个 UMD PDU 的 SN 值。可以认为“SN < 更新后的 VR(UR)”的 UMD PDU 都已成功接收；
- 对“SN < 更新后的 VR(UR)”的 UMD PDU 进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。

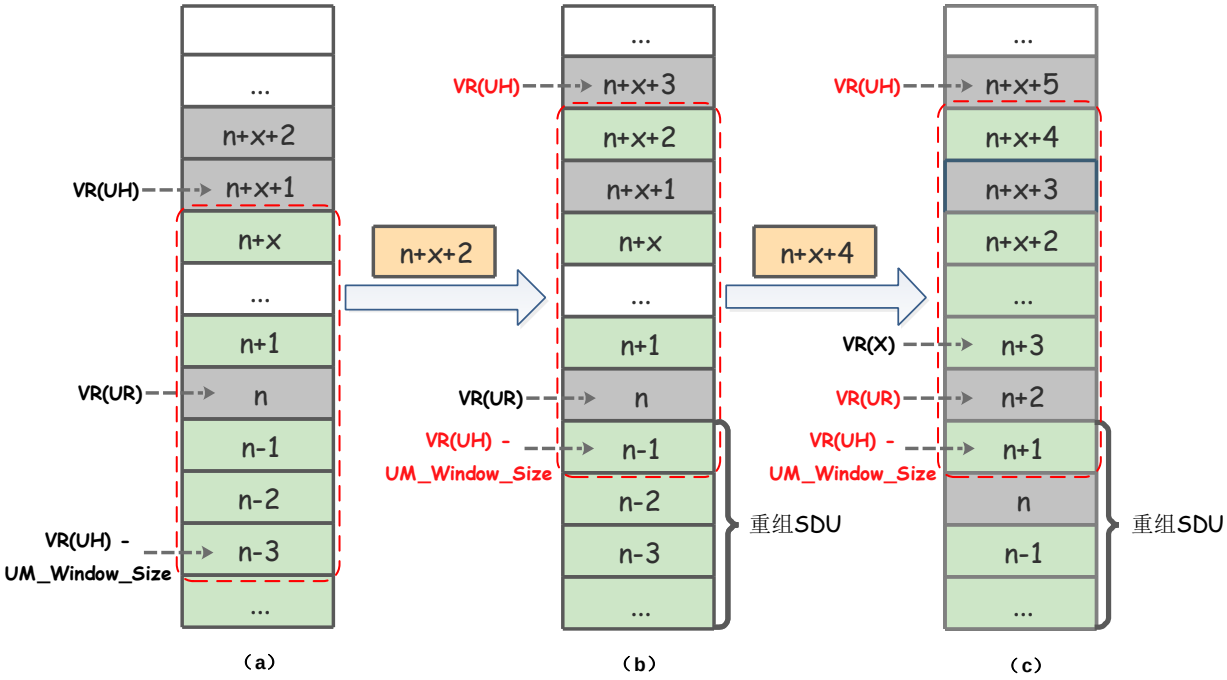


图 26-5：步骤 1 及步骤 2 举例

如图 26-5 所示，接收 buffer 的情况以及相关变量的取值如(a)所示。此时接收端收到一个位于重排序窗口之外的 SN = n+x+2 > VR(UH) = n+x+1 的 UMD PDU，因此 VR(UH)从 n+x+1 更新为 n+x+3，并导致 SN = n-3 和 SN = n-2 的 UMD PDU 移到了重排序窗口之外。接收端需要对 SN = n-3 和 SN = n-2 的 UMD PDU（可能还包括一些 SN < n-2 的 PDU）进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。此时 VR(UR)并未在重排序

窗口外，且在接收 buffer 中并不存在一个 SN = VR(UR)的 UMD PDU，所以无需更新 VR(UR)。此时的变量取值见图 26-5 的(b)。

接下来，接收端又收到一个位于重排序窗口之外的 SN = n+x+4 > VR(UH) = n+x+3 的 UMD PDU，此时 VR(UH)从 n+x+3 更新为 n+x+5，并导致 SN = n-1 和 SN = n 的 UMD PDU 移到了重排序窗口之外。在步骤 1 中，接收端需要对 SN = n-1 和 SN = n 的 UMD PDU（可能还包括一些 SN < n-1 的 PDU）进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。由于此时 VR(UR)被移到了重排序窗口外，所以将 VR(UR)设置成(VR(UH) – UM_Window_Size)，即 n+1。在步骤 2 中，接收端发现接收 buffer 中有一个 SN = VR(UR) = n+1 的 UMD PDU，因此将 VR(UR)更新成“SN > 当前 VR(UR) = n+1”，但还未接收到的第一个 UMD PDU 的 SN 值，即 n+2，并对 SN < n+2 的 UMD PDU 进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。此时变量的取值见图 26-5 的(c)。

步骤 3：如果 *t-Reordering* 正在运行，

- 如果 VR(UX) <= VR(UR)，或者
- 如果 VR(UX)落在了重排序窗口之外并且 VR(UX)不等于 VR(UH)
    - 停止并重置 *t-Reordering*。

t-Reordering 正在运行，意味着有 SN < VR(UX)的 UMD PDU 还未接收到。新收到 SN = x 的 UMD PDU 后，接收端需要重新判断是否需要继续运行 *t-Reordering*：

- 接收端认为小于 VR(UR)的 UMD PDU 都已接收到，这也意味着 VR(UX) <= VR(UR)时，小于 VR(UX)的 UMD PDU 都已接收到，所以此时没有运行 *t-Reordering* 的必要了。
- 由于 VR(UX)不可能大于 VR(UH)，所以当 VR(UX)落在了重排序窗口之外并且 VR(UX)不等于 VR(UH)时，VR(UX)必定落在了重排序窗口的下边界之外。而 UM 实体不会去接收“SN < (VR(UH) – UM_Window_Size)”的 UMD PDU，所以此时也没有运行 *t-Reordering* 的必要了。

步骤 4：如果 *t-Reordering* 没有运行（包括步骤 3 介绍的场景导致的停止运行），并且 VR(UH) > VR(UR)，则启动 *t-Reordering*，并将 VR(UX)设置成 VR(UH)。

VR(UH) > VR(UR)意味着此时至少有一个 SN < VR(UH)的 UMD PDU 还未接收到，所以此时应启动 *t-Reordering*，并在该定时器指定的时间内去接收还未收到的 PDU。

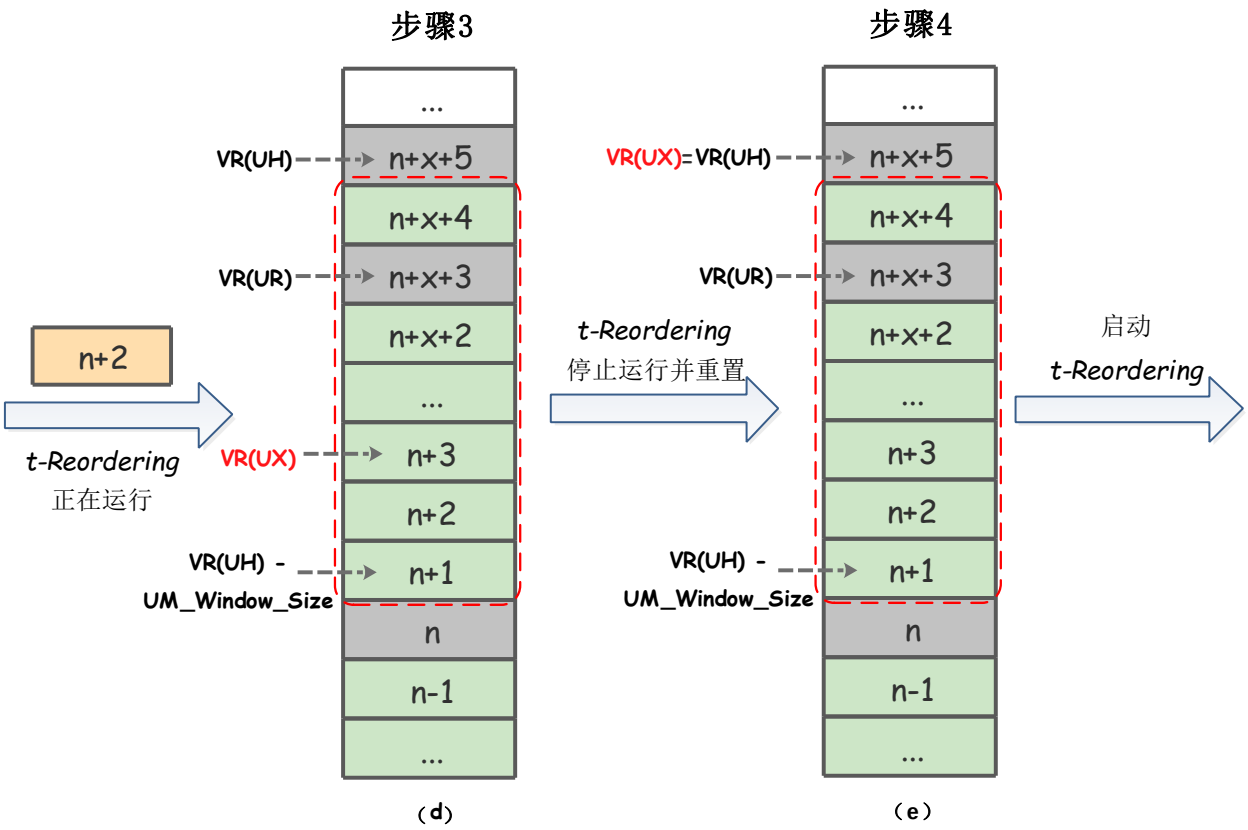


图 26-6：步骤 3 及步骤 4 举例

假设 VR(UX)以及其它变量的取值如图 26-6 的(c)所示。在图 26-6 中，*t-Reordering* 正在运行，接收端又收到了一个 SN = n+2 的 UMD PDU，此时 VR(UR)从 n+2 更新成 n+x+3，并导致“VR(UX) = n+3 <= VR(UR)”。接收端认为小于 VR(UX)的 UMD PDU 都已成功接收，因此会停止并重置 *t-Reordering*。此时变量的取值如图 26-6 的(d)所示。（VR(UX)位于重排序窗口之外的处理方式与此类似，这里就不再举例了）

由于步骤 3 的处理，*t-Reordering* 没有运行，并且 VR(UH) = n+x+5 > VR(UR) = n+x+3，意味着至少有一个 SN < VR(UH)（这里对应 SN = n+x+3）的 UMD PDU 还没有被接收到，因此会将 VR(UX)设置成 VR(UH)，并启动 *t-Reordering*。此时变量的取值如图 26-6 的(e)所示。

26.2.2.4 *t*-Reordering 超时处理

如果 *t-Reordering* 超时，则接收端会

- 将 VR(UR)更新成“SN > VR(UX)，但还未接收到”的第一个 UMD PDU 的 SN 值。
- 对“SN < 更新后的 VR(UR)”的 UMD PDU 进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。
- 如果此时 VR(UH) > VR(UR)，启动 *t-Reordering*，并将 VR(UX)设置成 VR(UH)。

t-Reordering 超时，说明 VR(UX) > VR(UR)，且“VR(UX)在重排序窗口之内，或 VR(UX)=VR(UH)”。也说明了“至少有一个 SN < VR(UX)的 UMD PDU 在该 *t-Reordering* 指定的时间内没有被接收到”，此时接收端不再去尝试接收 SN < VR(UX)的 PDU，并认为那些没有收到的 PDU 已经丢失了。

可以看出，UM 模式中，是通过 *t-Reordering* 超时来判断 SN < VR(UX)的某个 UMD PDU 是否丢失了的。

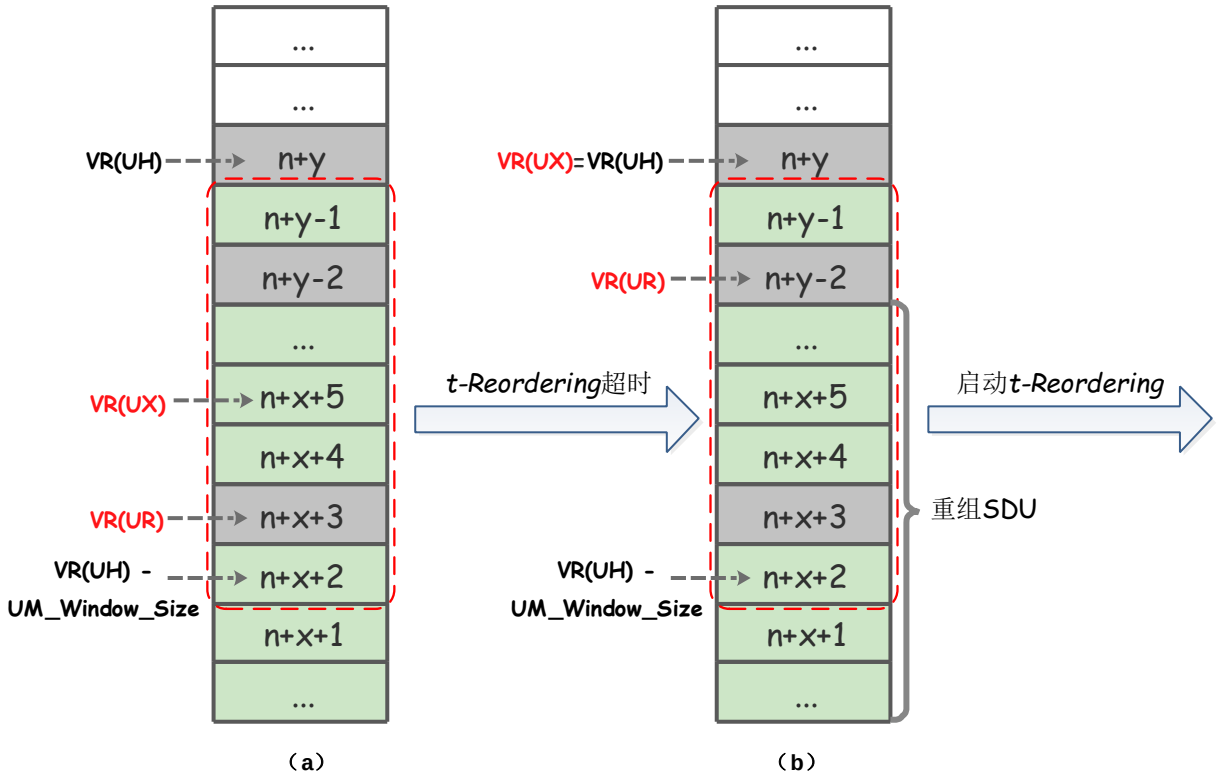


图 26-7：*t-Reordering* 超时处理举例

如图 26-7 的(a)所示，*t-Reordering* 超时的时候，还有一个 SN = VR(UR) = n+x+3 的 PDU 没有接收到。此时接收端不再去接收该 PDU，并将 VR(UR)更新成“SN > 当前 VR(UX) = n+x+5，但还未接收到”的第一个 UMD PDU 的 SN 值，这里对应 n+y-2。并对 SN < n+y-2 的 PDU 进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。由于此时 VR(UR) = n+y-2 < VR(UH) = n+y，因此要将 VR(UX)设置成 VR(UH)，并启动 *t-Reordering*。此时变量的取值如图 26-7 的(b)所示。

26.3 AM 模式

AM 模式通过错误检测和重传，提供了一种可靠的传输服务。AM 模式主要用于对错误敏感（error-sensitive）、允许一定延时（delay-tolerant）且可靠性要求较高的非实时性（non-realtime）业务。大多数的交互式或后台运行类业务属于这一范畴，如网页浏览、e-mail 和文件下载等。在控制面，大多数的 RRC 消息也使用 AM 模式，以利用 AM 模式的确认和重传机制来保证可靠性传输。

AM 模式通过逻辑信道 DL/UL DCCH 或 DL/UL DTCH 来接收/发送 RLC PDU。

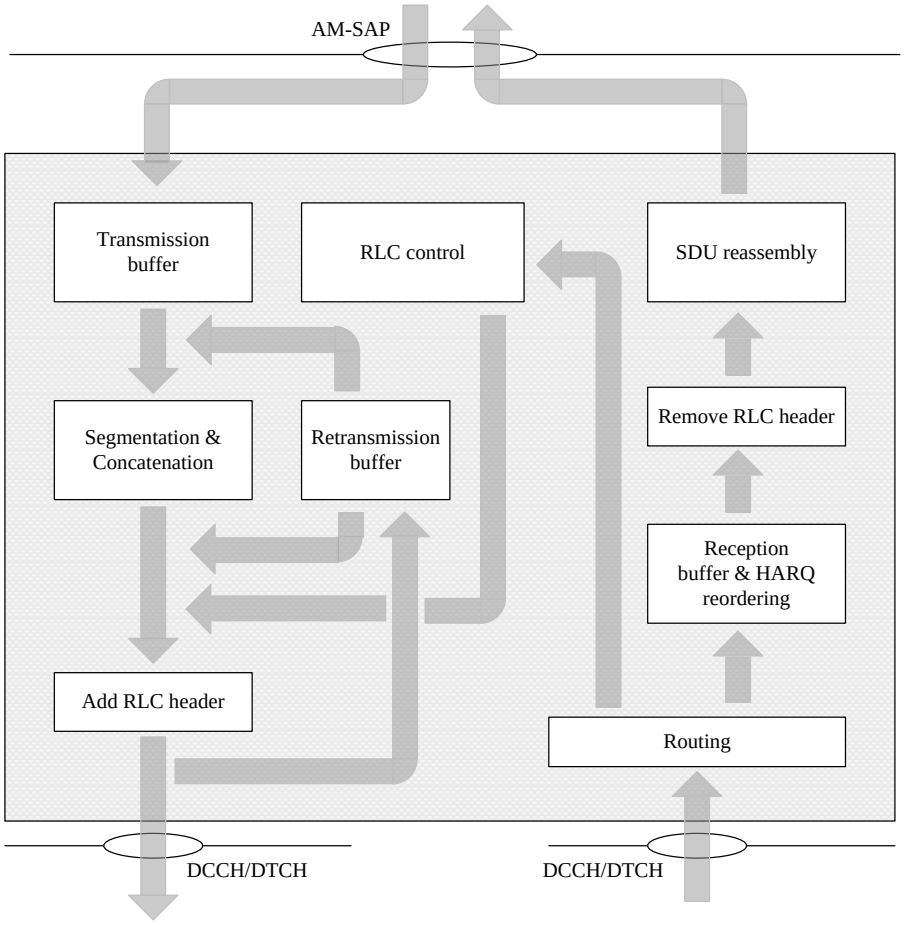


Figure 4.2.1.3.1-1: Model of an acknowledged mode enttiy

在 eNodeB 或 UE 侧，一个 AM 实体既包含接收侧，又包含发送侧，即能够同时收发数据。AM 实体提供了双向的数据传输服务。

除了重传相关的处理外，AM 模式与 UM 模式提供了类似的 RLC 层功能，如分段/重连、重排序、丢弃重复包以及重组 SDU 等等。

AM 实体发送/接收 2 种类型的 PDU：

- RLC data PDU：该 PDU 又分为 2 种类型：AMD PDU 和 AMD PDU segment。
- RLC control PDU：即 STATUS PDU。

26.3.1 发送端

结合 36.322 的 Figure 4.2.1.3.1-1 可以看出，AM 实体在发送端需要做以下几件事：

- 将来自上层（PDCP 层）的 RLC SDU 缓存在传输 buffer（transmission buffer）中；
- 在 MAC 层通知其发送 RLC PDU 时，分段/串联 RLC SDU 以生成 RLC PDU，并赋予合适的 SN 值，然后将生成的 RLC PDU 发给 MAC 层；
- 发给 MAC 层的 RLC PDU 同时会放在重传 buffer（retransmission buffer）中，以便支持可能存在的重传。如果收到了对应整个 RLC PDU 的 ACK，则对应的 RLC PDU 将从重传 buffer 中删除；如果收到了对应整个或部分 RLC PDU 的 NACK，则会重传整个或部分 PDU；
- 如果某个 RLC PDU 需要重传，且在一次传输机会中， MAC 层指定的大小不足以重传整个 RLC PDU，则需要对这个 RLC PDU 进行重分段。每个分段将以一个独立的 PDU 进行传输（但不同分段的 SN 都相同，并等于原始 RLC PDU 的 SN 值），并重新生成头部以指示重分段是如何进行的（不同分段会在不同的传输机会里传输）。如果 MAC 层指定的大小足以重传整个 RLC PDU，则重传整个 RLC PDU 而无需重分段。

26.3.1.1 发送端相关变量

在详细介绍 AM 模式发送端处理流程之前，我们需要先介绍发送端的相关变量。AM 实体在发送端会维护如下变量：

VT(A)：确认状态变量（Acknowledgement state variable）。该变量保存了按序要接收到一个肯定确认（positive acknowledgement）的下一个 AMD PDU 的 SN 值。

- 该变量对应传输窗口的下边界（包含 VT(A)）。
- 该变量初始化时为 0。
- 该变量在 AM 实体收到一个 SN = VT(A)的 AMD PDU 的肯定确认时予以更新，此时会将 VT(A)设置为还未收到肯定确认并拥有最小 SN 的 AMD PDU 的 SN 值，且该 SN 需满足 $VT(A) \le SN \le VT(S)$。注意：此时 SN = VT(S)的 AMD PDU 还未发送给 MAC 层。

简单地说，发送端认为 SN < VT(A)的所有 AMD PDU 都已被接收端成功接收，并且发送端已经收到这些 AMD PDU 的肯定确认。但 SN = VT(A)的 AMD PDU 还未收到肯定确认。

VT(MS)：最大发送状态变量（Maximum send state variable）。VT(MS) = VT(A) + AM_Window_Size，该变量对应传输窗口的上边界（不包含 VT(MS)）。即 VT(MS)保存了第一个超过传输窗口的 AMD PDU 的 SN 值。

AM_Window_Size 的值固定等于 512。并用于计算 VT(MS)和 VR(MR)。

VT(S)：发送状态变量（Send state variable）。该变量保存了下一个新生成的 AMD PDU 将被赋予的 SN 值。该变量初始化时为 0，并在 AM 实体发送一个 SN = VT(S)的 AMD PDU 给 MAC 层时更新。简单地说，“VT(S) – 1”是最近一个已发送的新 RLC data PDU（不是重传）的 SN 值。

AM 实体是按照 SN 递增的顺序逐个发送新的 AMD PDU 的。也就是说，当 AM 实体要发送一个新的 AMD PDU 给 MAC 层时，它会将这个 AMD PDU 的 SN 设置为 VT(S)，然后将 VT(S)加 1。

VT(A)是还未接收到肯定确认的最小 SN，“VT(S) – 1”是已经发送的 RLC data PDU 中的最大 SN。因此发送端期望接收到的确认信息中的 SN 范围应该为 $VT(A) \leq SN \leq VT(S)$。

Transmitting window：传输窗口。如果一个 SN 满足 $VT(A) \leq SN < VT(MS)$，则该 SN 位于传输窗口内，否则该 SN 位于传输窗口外。AM 实体的发送端不会给 MAC 层发送“SN 位于传输窗口外”的 RLC data PDU。这是因为

- 发送端认为 SN < VT(A)的所有 AMD PDU 都已被接收端成功接收，并且发送端已经收到这些 AMD PDU 的肯定确认。因此没必要重复发送这些 AMD PDU。
- 从未收到肯定确认的最小 SN，即 SN = VT(A)的 AMD PDU 算起，发送端只能发送最多 AM_Window_Size 个 AMD PDU，所以发送端不会发送 $SN \geq VT(MS)$的 AMD PDU。

POLL_SN：Poll 发送状态变量（Poll send state variable）。在发送完 P 字段设置为 1 的 AMD PDU，并更新了 VT(S)之后（如果需要更新的话），该变量会被设置为 VT(S) – 1。该变量初始值为 0。

图 26-3 是除 AM 发送端相关变量的一个例子。

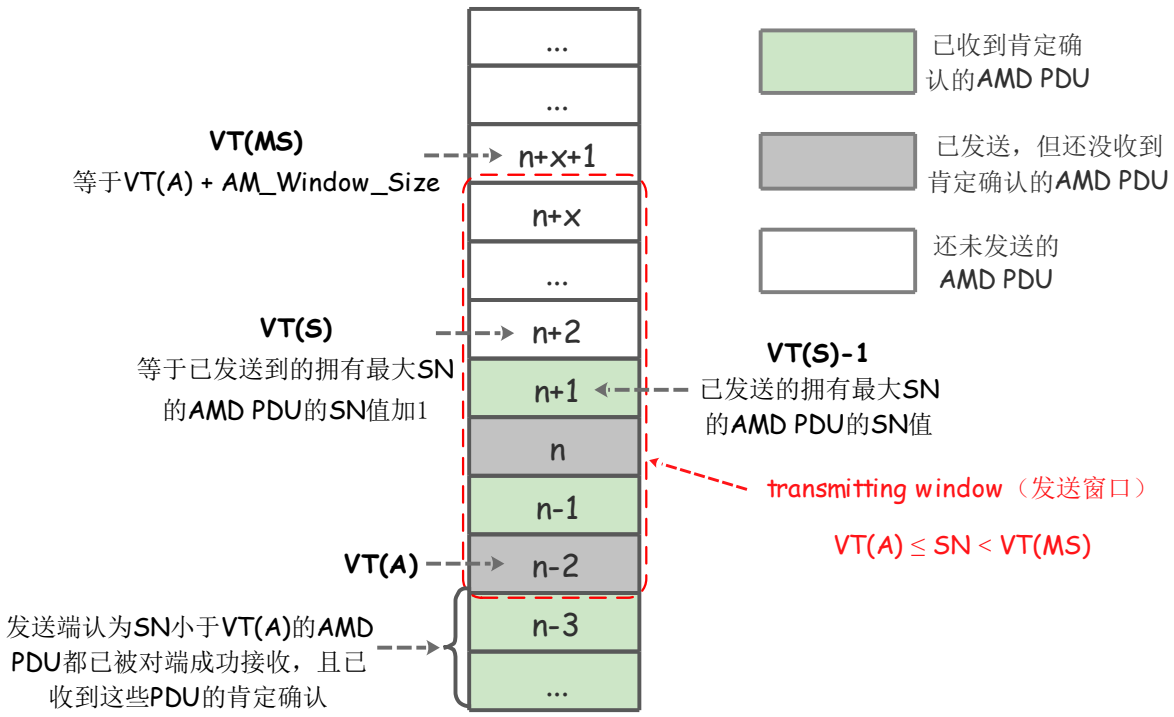


图 26-8：AM 发送端相关变量

26.3.1.2 分段、串联和重分段

与 UM 模式类似，AM 实体发送 RLC PDU 的前提条件是：MAC 层通知 AM 实体发送一个 RLC PDU，即通知 AM 实体有一个传输机会。MAC 层同时会告诉 AM 实体在这次传输机会中，可以传输的 RLC PDU 的总大小。

由 MAC 层指定的 RLC PDU 的大小通常并不等于 RLC SDU 的大小，所以在发送端需要分段/串联 RLC SDU 以便生成的 RLC PDU 匹配 MAC 层指定的大小。

AM 模式的分段、串联功能与 UM 模式类似，差别只在于二者的 header 有所不同。图 7 是 AM 模式的分段、串联功能举例。与图 1 相比，这里只有 header 部分有所不同，但其中 FI、E 和 SN 的含义和取值都是相同的。

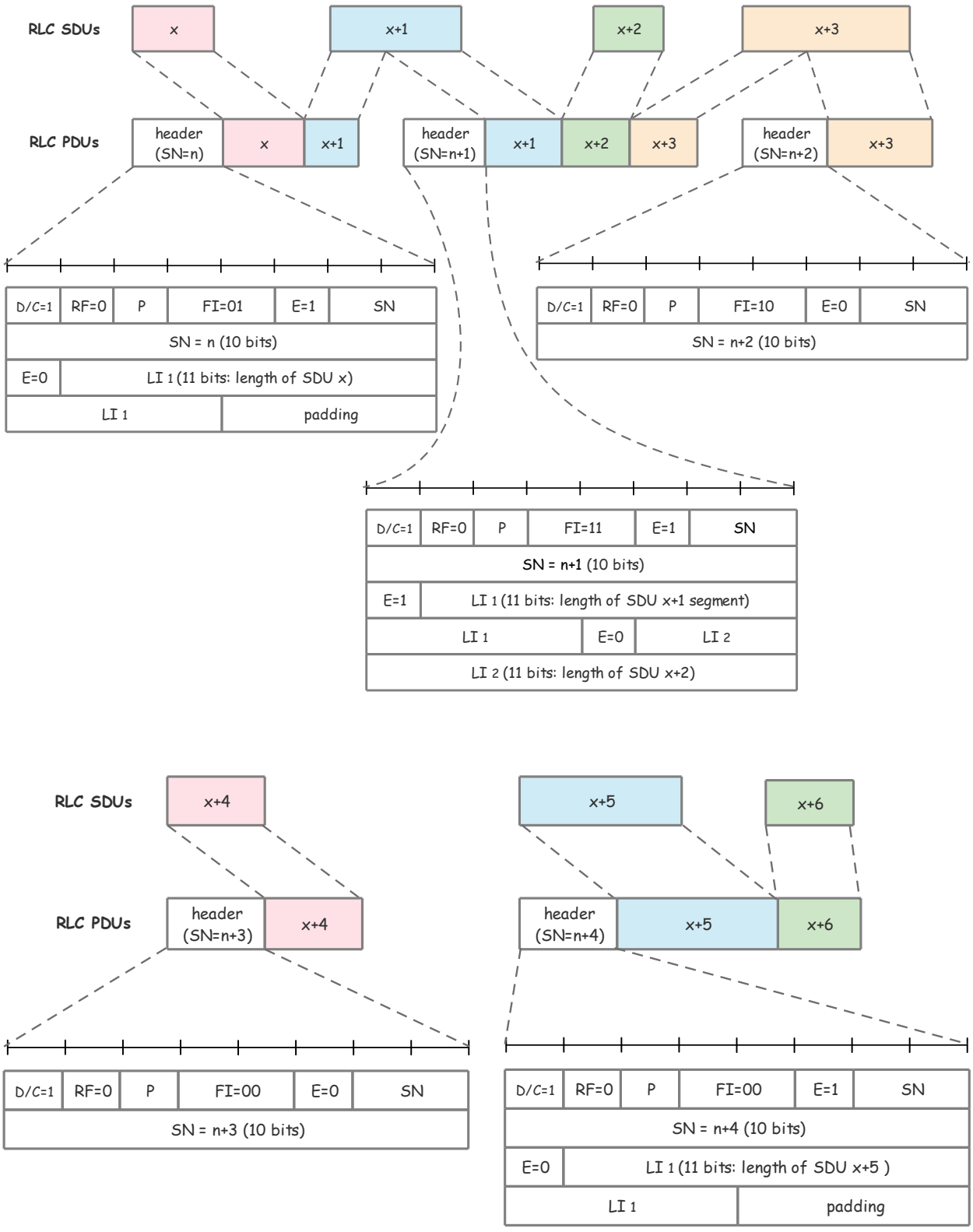


图 26-9：AM 分段和串联功能举例

与 UM 模式不同的是，AM 模式支持对 RLC data PDU 进行重传。当 MAC 层指定的大小不足以重传整个 RLC data PDU 时，AM 实体需要对 RLC data PDU 进行重分段，并生成 AMD PDU segment 发给 MAC 层。

只有原始 AMD PDU 的 Data 域（除 header 外的剩余数据，后续会介绍）会映射到新的 AMD PDU segment 的 Data 域。也就是说，重分段时，AMD PDU segment 只会重传原始 AMD PDU 的 Data 域，而不会重传原始 AMD PDU 的 header。此时需要为 AMD PDU segment 生成新的 header。

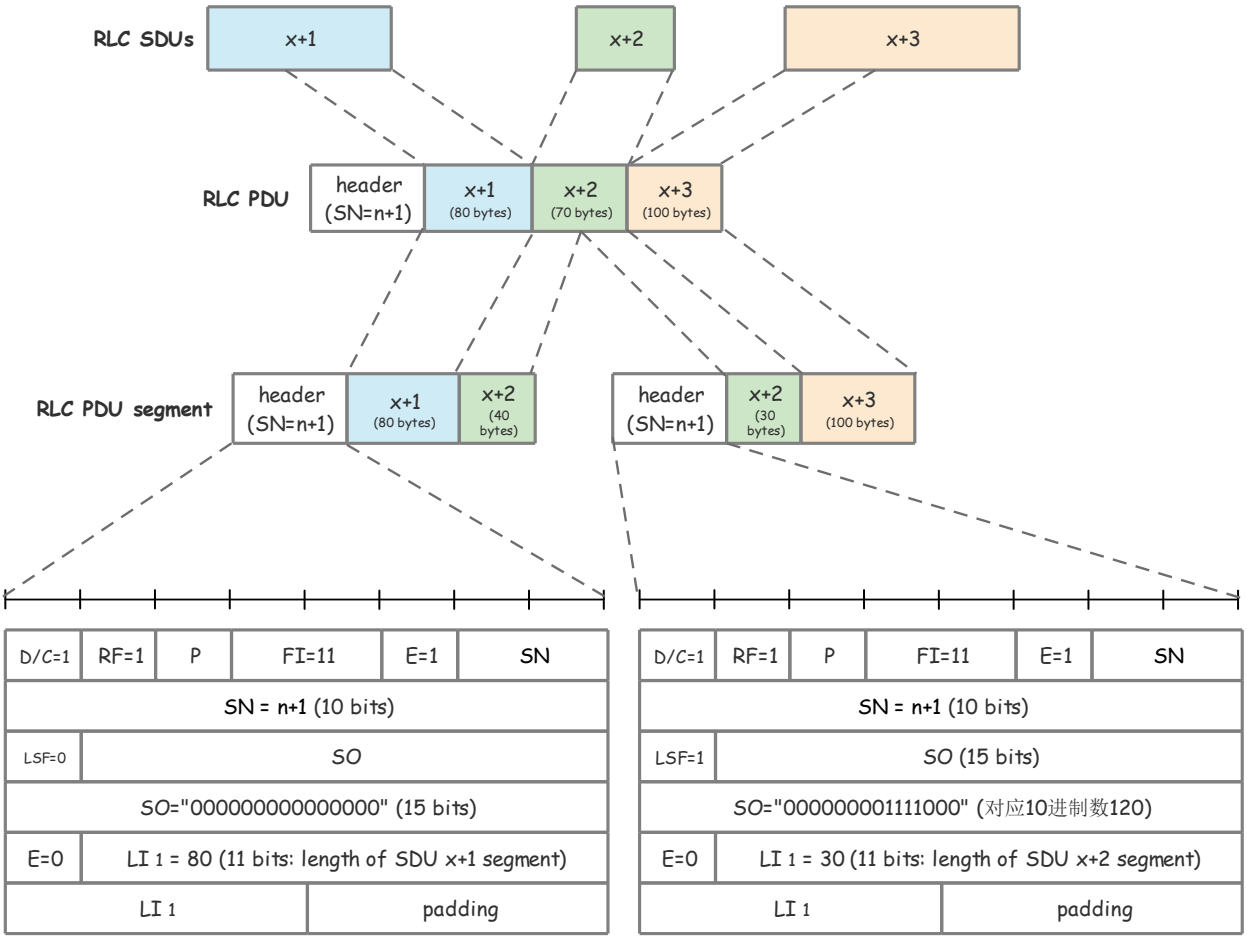


图 26-10：AM 的重分段功能举例

图 26-10 是重分段功能的一个例子。假设接收端没有收到图 26-9 中 SN = n+1 的 AMD PDU，且发送端需要对该 PDU 进行重传。我们同时假设在原始 AMD PDU 中，SDU x+1 分段的长度为 80 字节，SDU x+2 的长度为 70 字节，SDU x+3 分段的长度为 100 字节。

在接下来的第一次传输机会里，MAC 层指定的大小为 126 字节，不足以发送整个原始 AMD PDU，此时需要对原始 AMD PDU 进行重分段。除去 6 字节用于新生成的 AMD PDU segment 的 header，只能传输 120 字节的原始 AMD PDU 的 Data 域数据，即传输 80 字节的 SDU x+1 分段和 40 字节的 SDU x+2 分段。

在接下来的第二次传输机会里，MAC 层指定的大小为 136 字节时，除去 6 字节用于新生成的 AMD PDU segment 的 header，剩余的 130 字节正好可以传输原始 AMD PDU 的所有剩余 Data 域数据，即传输 30 字节的 SDU x+2 分段和 100 字节的 SDU x+3 分段。如果在第二次传输机会里，MAC 层指定的大小小于 136 字节，则需要继续对原始 AMD PDU 的剩余部分继续分段。对一个 AMD PDU 进行重分段的次数是没有限制的。

如果在第一次传输机会里，MAC 层指定的大小足以发送 SN = n+1 的 AMD PDU，则直接将该 PDU 发给 MAC 层，而不需要做重分段处理。

需要说明的是，分段是对 RLC SDU 进行的，UM/AM 模式下都存在。重分段是对 RLC PDU 进行的，只存在于 AM data PDU 的重传中。

26.3.1.3 AMD PDU

AMD PDU 用于 RLC data PDU 的第一次传输，或没有进行重分段的 AMD PDU 的重传。

结合图 26-9 的例子，我们将进一步介绍 AMD PDU 的结构及其相关字段的作用。

AMD PDU 由 2 部分组成：Data 域和 header（头部）。

Data 域由一个或多个“Data field element”组成。一个“Data field element”对应一个 RLC SDU 或一个 RLC SDU 分段。Data field element 是按照 RLC SDU 到达 RLC 实体的先后顺序映射到 Data 域的。图 26-9 中 SN = n 的 RLC PDU 的 Data 域包含了 2 个“Data field element”，分别对应 SDU x 和 SDU x+1 的分段。SN = n+1 的 RLC PDU 的 Data 域包含了 3 个“Data field element”，分别对应 SDU x+1 的分段、SDU x+2 和 SDU x+3 的分段。

header 由固定部分（每个 PDU 都有）和可能存在的扩展部分组成。

固定部分（fixed part，共 16 比特）由一个 D/C（1 比特）、一个 RF（1 比特）、一个 P（1 比特）、一个 FI（2 比特）、一个 E（1 比特）和一个 SN（10 比特）字段组成，其本身是字节对齐（byte-aligned）的。

扩展部分（extension part）由 1 个或多个“E（1 比特） + LI（11 比特）”组成。只有当 Data field element 的个数多于 1 个时，才存在扩展部分（extension part）。除了最后一个 Data field element 外，其它 Data field element 都有一个对应的“E + LI”。如果 LI 的个数为奇数，则最后一个 LI 之后需要添加 4 比特的 padding。可以看出，扩展部分中“E + LI”的个数等于 Data field element 的个数减去 1。

关于 AMD PDU 的结构，可参见 36.322 的 6.2.1.4 节。（或见下图。最左边的比特对应 MSB，最右边的比特对应 LSB）

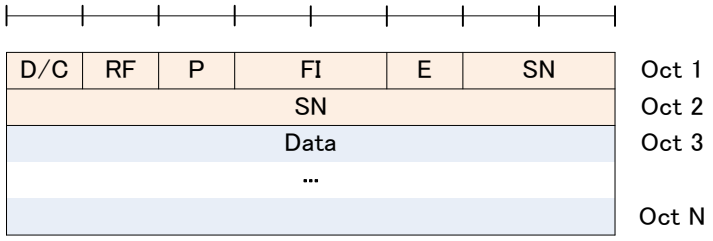


Figure 6.2.1.4-1: AMD PDU (No LI)

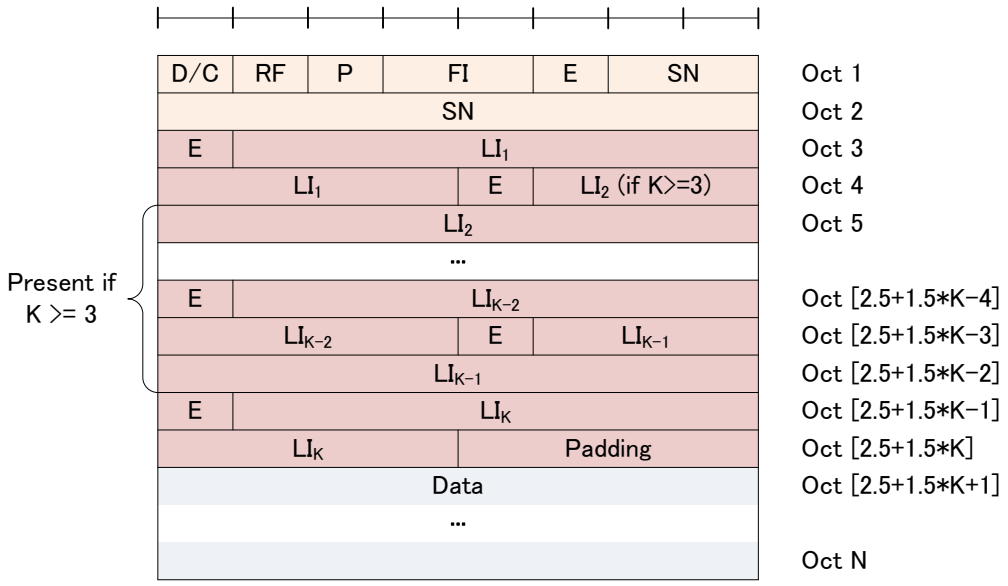


Figure 6.2.1.4-2: AMD PDU (Odd number of LIs, i.e. K = 1, 3, 5, …)

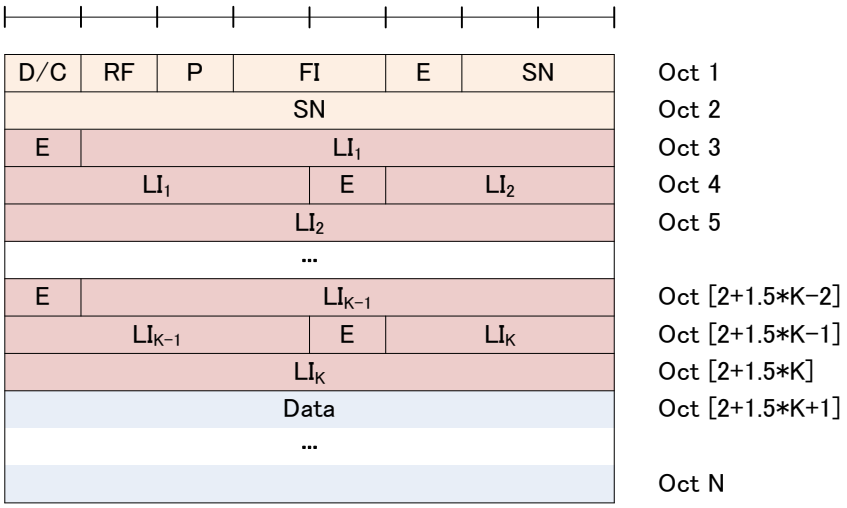


Figure 6.2.1.4-3: AMD PDU (Even number of LIs, i.e. K = 2, 4, 6, …)

D/C 是 Data/Control 的意思，该字段的值为 0 时，表示对应的 RLC PDU 是 RLC control PDU，即 STATUS PDU；值为 1 时，表示对应的 RLC PDU 是 RLC data PDU。

RF 是 Re-segmentation Flag 的意思，该字段的值为 0 时，表示对应的 RLC PDU 是 AMD PDU；值为 1 时，表示对应的 RLC PDU 是 AMD PDU segment（见下一节）。对于 RLC data PDU 的第一次传输，该字段为 0。

P 是 Polling bit 的意思，该值为 1 时，表示发送端请求对端发送一个 STATUS report；该值为 0 时，表示没有请求对端发送 STATUS report。

SN 唯一指定了一个 AMD PDU。在 AM 模式中，SN 的长度固定为 10 比特。

E、LI、FI 的含义与其在 UM PDU 中的含义一致，具体可见 26.2.1.2 节。

以图 26-9 为例，这里所有的 RLC PDU 都是 RLC data PDU，其均未进行重分段，所以 D/C 字段均设置为 1，RF 字段均设置为 0。P 字段的设置见 26.3.3.2 节的介绍。E 与 LI 字段的设置可参见 26.2.1.2 节的介绍，其含义与 UM PDU 中 E 与 LI 字段的含义相同。

26.3.1.4 AMD PDU segment

当重传的 AMD PDU 需要被重分段时，会使用 AMD PDU segment。

AMD PDU segment 由 2 部分组成：Data 域和 AMD PDU segment header（头部）。

Data 域由一个或多个“Data field element”组成。一个“Data field element”对应一个 RLC SDU 或一个 RLC SDU 分段。Data field element 是按照 RLC SDU 到达 RLC 实体的先后顺序映射到 Data 域的。图 26-10 中 SN = n+1 的 AMD PDU 被重分段成 2 个 AMD PDU segment。第一个 AMD PDU segment 的 Data 域包含了 2 个“Data field element”，分别对应 SDU x+1 的分段和 SDU x+2 的分段。第二个 AMD PDU segment 的 Data 域包含了 2 个“Data field element”，分别对应 SDU x+2 的分段和 SDU x+3 的分段。

只有原始 AMD PDU 的 Data 域会映射到新的 AMD PDU segment 的 Data 域。也就是说，重分段时，AMD PDU segment 只会重传原始 AMD PDU 的 Data 域，而不会重传原始 AMD PDU 的 header。

需要说明的是，协议中定义的 Data field element 是基于 SDU 而不是 PDU 来描述的。重分段相当于把原始 AMD PDU 中 Data 域的 RLC SDU 或 RLC SDU 的分段进行重新组合，并重新生成 header。

header 由固定部分（每个 PDU 都有）和可能存在的扩展部分组成。

固定部分（fixed part，共 32 比特）由一个 D/C（1 比特）、一个 RF（1 比特）、一个 P（1 比特）、一个 FI（2 比特）、一个 E（1 比特）、一个 SN（10 比特）、一个 LSF（1 比特）和 1 个 SO（15 比特）字段组成，其本身是字节对齐（byte-aligned）的。

扩展部分（extension part）由 1 个或多个“E（1 比特） + LI（11 比特）”组成。只有当 Data field element 的个数多于 1 个时，才存在扩展部分（extension part）。除了最后一个 Data field element 外，其它 Data field element 都有一个对应的“E + LI”。如果 LI 的个数为奇数，则最后一个 LI 之后需要添加 4 比特的 padding。可以看出，扩展部分中“E + LI”的个数等于 Data field element 的个数减去 1。

关于 AMD PDU segment 的结构，可参见 36.322 的 6.2.1.5 节。（或见下图。最左边的比特对应 MSB，最右边的比特对应 LSB）。

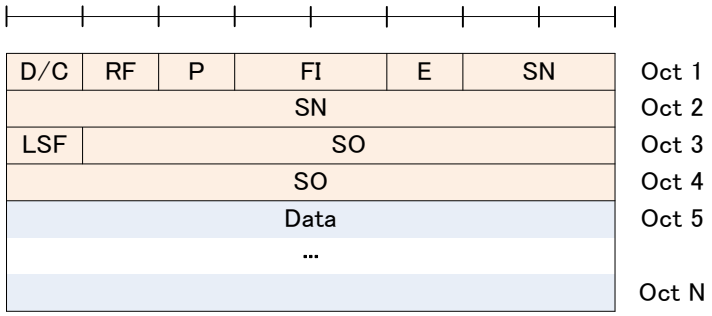


Figure 6.2.1.5-1: AMD PDU segment (No LI)

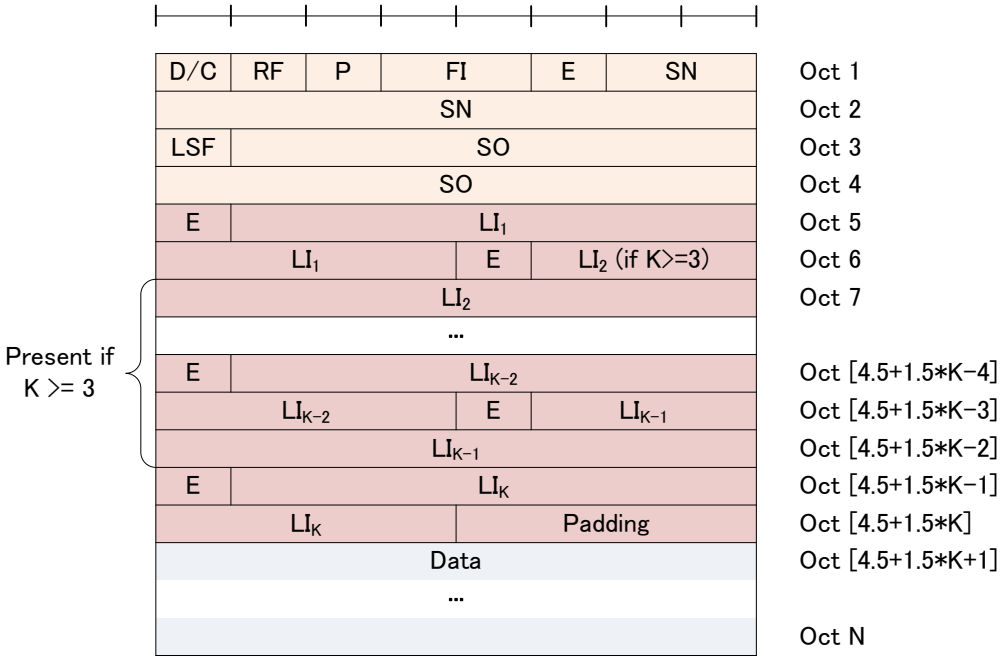


Figure 6.2.1.5-2: AMD PDU segment (Odd number of LIs, i.e. K = 1, 3, 5, …)

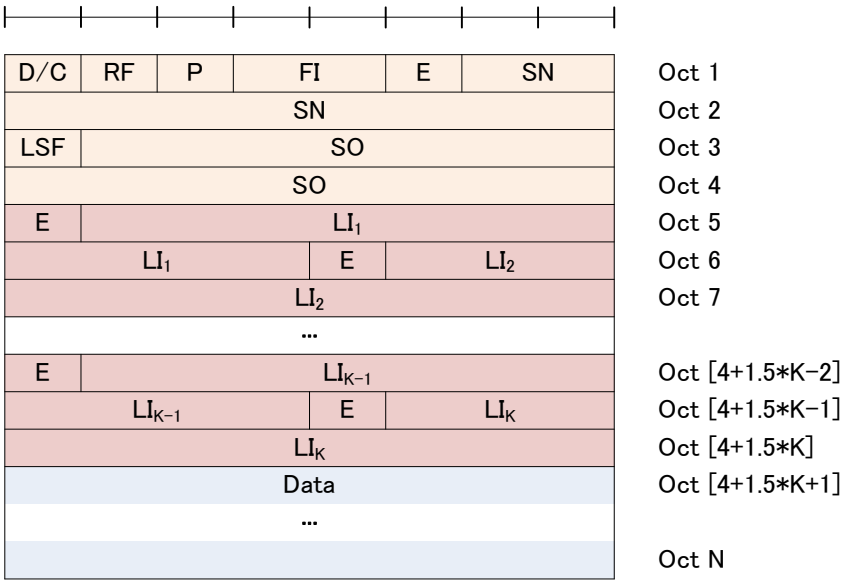


Figure 6.2.1.5-3: AMD PDU segment (Even number of LIs, i.e. K = 2, 4, 6, …)

SO 是 Segment Offset 的意思，该字段用于指示 AMD PDU segment 在原始 AMD PDU 中的位置（以字节为单位）。更确切地说，该字段指示了 AMD PDU segment 的 Data 域的第一个字节在原始 AMD PDU 的 Data 域中的位置。原始 AMD PDU 的 Data 域的第一个字节对应 SO 字段的值“000000000000000”（二进制数），即从 0 开始计数。

LSF 是 Last Segment Flag 的意思，该字段用于指示 AMD PDU segment 的最后一个字节是否对应一个 AMD PDU 的最后一个字节。该字段的值为 0 时，AMD PDU segment 的最后一个字节不是一个 AMD PDU 的最后一个字节；值为 1 时，AMD PDU segment 的最后一个字节是一个 AMD PDU 的最后一个字节。换句话说，LSF 指示了该 AMD PDU segment 是否是原始 AMD PDU 的最后一个分段。

以图 26-10 为例，SN = n+1 的 AMD PDU 被重分段成 2 个 AMD PDU segment。这 2 个 AMD PDU segment 都是 RLC data PDU，所以 D/C 字段均设置为 1。同时二者都是 AMD PDU segment，所以 RF 字段均设置为 1。

第一个 AMD PDU segment 由 2 个 Data field element（SDU x+1 的分段和 SDU x+2 的分段）组成，因此固定部分的 E 设置为 1，表示固定部分之后紧接着 1 个“E + LI”。扩展部分的第一个“E + LI”中的 E 设置为 0，表示其后紧接着 Data 域，LI（对应 LI_1）指示了 Data 域中 SDU x+1 的分段的

长度（80）。由于 Data 域的第一个字节并不对应 SDU x+1 的第一个字节，Data 域的最后一个字节也不对应 SDU x+2 的最后一个字节，所以 FI 的值设置为 11。该 AMD PDU segment 的最后一个字节并不是原始 AMD PDU 的最后一个字节，所以 LSF 的值设置为 0。该 AMD PDU segment 的 Data 域的第一个字节对应原始 AMD PDU 的 Data 域的第一个字节，所以 SO 字段被设置为“0000000000000000”，对应 10 进制数 0。

第二个 AMD PDU segment 由 2 个 Data field element（SDU x+2 的分段和 SDU x+3 的分段）组成，因此固定部分的 E 设置为 1，表示固定部分之后紧接着 1 个“E + LI”。扩展部分的第一个“E + LI”中的 E 设置为 0，表示其后紧接着 Data 域，LI（对应 LI_1）指示了 Data 域中 SDU x+2 的分段的长度（30）。由于 Data 域的第一个字节并不对应 SDU x+2 的第一个字节，Data 域的最后一个字节也不对应 SDU x+3 的最后一个字节，所以 FI 的值设置为 11。该 AMD PDU segment 的最后一个字节是原始 AMD PDU 的最后一个字节，所以 LSF 的值设置为 1。该 AMD PDU segment 的 Data 域的第一个字节对应原始 AMD PDU 的 Data 域的第 121 个字节，所以 SO 字段设置为“0000000001111000”，对应 10 进制数 120。

对应同一 AMD PDU 的所有 AMD PDU segment 的 SN 字段是相同的，并等于原始 AMD PDU 的 SN 值。所以这 2 个 AMD PDU segment 的 SN 字段均设置为 n+1。

与 UM PDU 类似，AMD PDU 和 AMD PDU segment 的 RLC header 并不指定 Data 域中最后一个 Data field element 的大小。其原因也与 UM PDU 相同：RLC PDU 的长度是由 MAC 指定的，该长度会在 MAC PDU 中对应该 RLC PDU 的 subheader 中的 L 字段中体现（见 36.321 的 6.1.2 节和 6.2.1 节）。“RLC PDU 的长度 - RLC PDU header 的长度 - 所有 LI 之和”即为最后一个 Data field element 的长度。

需要说明的是，本章中提到的“AMD PDU segment”专指 36.322 的 Figure 6.2.1.5-1 至 Figure 6.2.1.5-3 介绍的 RLC data PDU，即本节介绍的内容。而“AMD PDU 分段”专指原始 AMD PDU 的部分数据，即 AMD PDU 的 Data 域的部分内容。

26.3.1.5 发送过程

AM 实体在发送 RLC PDU 时，

- RLC control PDU 的传输优先级要高于 RLC data PDU（包括新传和重传的 RLC data PDU，以及 AMD PDU segment）
- RLC data PDU 重传的传输优先级要高于新传的 AMD PDU。

AM 实体不会发送 SN 位于传输窗口之外的 RLC data PDU 给 MAC 层。

当发送端要发送一个新的 AMD PDU 到 MAC 层时，它会将该 AMD PDU 的 SN 设置成 VT(S)，然后将 VT(S)加 1。

发送端发送了一个 RLC PDU 后，需要等待对端（接收端）回复是否成功接收了该 PDU。接收端是通过回复 STATUS PDU 来告诉接收端它是否成功接收了某个 PDU 的。如果接收端回复没有收到某个 PDU，则发送端需要重传该 PDU。重传处理对应 AM 模式的 ARQ 流程，这会在 26.3.3 节予以介绍。

发送端会去接收对端实体发送的 STATUS PDU，以确定对端是否成功接收了 RLC data PDU。如果发送端收到了对 SN = VT(A)的 AMD PDU 的肯定确认（positive acknowledgement），即确定接收端已经成功接收了 SN = VT(A)的 AMD PDU，则发送端会

- 将 VT(A)设置成“SN 位于 VT(A) ≤ SN ≤ VT(MS)内”且“还未收到肯定确认”且“拥有最小 SN”的 AMD PDU 的 SN 值。发送端认为 SN < VT(A)的 AMD PDU 都已被接收端成功接收，并已收到了这些 PDU 的肯定确认；
- 如果一个 RLC SDU 相关的所有 AMD PDU 都已收到对应的肯定确认，则说明该 SDU 已被对端成功接收，此时发送端会发送一个指示给上层（PDCP 层），告诉它该 SDU 已成功发送。

在 AM 模式中，只有收到 SN = VT(A)的 AMD PDU 的肯定确认，VT(A)才会更新，发送窗口才会向前移动。如果发送端一直没有收到 SN = VT(A)的 AMD PDU 的肯定确认，则发送端发完 SN = VT(MS)-1 的 AMD PDU 后，就无法继续发送新的 AMD PDU，而只能重传那些还未收到肯定确认的 AMD PDU，也即发送窗口被“阻塞”了。

26.3.2 接收端

结合 36.322 的 Figure 4.2.1.3.1-1 可以看出，AM 实体在接收端需要做以下几件事：

- 判断收到的 RLC data PDU 是否是重复包（duplication）。如果是的话，丢弃该重复包。
- 对 RLC data PDU 进行重排序（reordering）：MAC 层的 HARQ 操作可能导致到达 RLC 层的报文是乱序的，所以需要 RLC 层对数据进行重排序。
- 检测 MAC 层是否丢失了 RLC data PDU。如果丢失了，请求对端的 AM 实体重传该 PDU。

- 在已经重排序的 RLC data PDU 的基础上重组 RLC SDU，并按序递送 RLC SDU 给 PDCP 层。

在 AM 模式中，接收端收到的 RLC data PDU 可能是一个完整的 AMD PDU，也可能是一个 AMD PDU 的分段。只有当接收端收到了一个 AMD PDU 的所有分段时，我们才能称这个 PDU 被“成功接收”或“完全接收”了。

26.3.2.1 接收端相关变量

在详细介绍 AM 模式接收端处理流程之前，我们需要先介绍接收端的相关变量。AM 实体在接收端会维护如下变量：

VR(R)：接收状态变量（Receive state variable）。该变量保存了紧跟着最后一个按序完全接收的 AMD PDU 的 SN 值的下一个 SN。举个例子：假设 SN ≤ x 的 AMD PDU 都已经被完全接收，但 SN = x+1 的 AMD PDU 还没有被完全接收，则不管是否有 SN > x+1 的 AMD PDU 被成功接收，VR(R)的值都会被设置成 x+1。

- 该变量对应接收窗口的下边界（包含 VR(R)）。
- 该变量初始化时为 0。
- 该变量在 AM 实体收到一个 SN = VR(R)的 AMD PDU（所有 byte 分段都已成功接收）时予以更新，此时会将 VR(R)设置为 “SN > 当前 VR(R)”且“并非所有 byte 分段都被成功接收”的第一个 AMD PDU 的 SN。
- 接收端认为 SN 小于 VR(R)的 AMD PDU 都已被完全接收。

VR(MR)：最大可接受的接收状态变量（Maximum acceptable receive state variable）。VR(MR) = VR(R) + AM_Window_Size，该变量对应接收窗口的上边界（不包含 VR(MR)），即 VR(MR)保存了第一个超过接收窗口的 AMD PDU 的 SN 值。

VR(H)：最高接收状态变量（Highest received state variable）。该变量保存的 SN 值等于所有已经接收到的 RLC data PDU 中，拥有最高 SN 的那个 RLC data PDU 的 SN 值，再加 1。该变量初始化时为 0。简单地说，“VR(H) – 1”等于已接收的拥有最高 SN 的 AMD PDU（并不要求该 PDU 被完全接收）的 SN 值。

- 如果一个 SN = x 的 RLC data PDU 要被放入接收 buffer，且 x ≥ VR(H)，则 VR(H)会被设置成 x+1。

VR(MS)：最大 STATUS 传输状态变量（Maximum STATUS transmit state variable）。该变量保存的是当需要生成一个 STATUS PDU 时，“ACK_SN”字段可以配置的最高 SN 值。该变量初始化时为 0。接收端认为 SN < VR(MS)的那些还未成功接收的 AMD PDU 已经丢失，需要进行重传。注意：在生成 STATUS PDU 时，其 “ACK_SN”字段未必会设置成 VR(MS)，具体原因会在 26.3.3 节介绍。

VR(X)：*t-Reordering* 状态变量（*t-Reordering* state variable）。该变量保存的 SN 值紧随触发 *t-Reordering* 的 RLC data PDU 的 SN。当启动 *t-Reordering*（同时会更新 VR(X)）时，说明有小于 VR(X)的 AMD PDU 还未被完全接收（并且这些 PDU 还未确定是否丢失），此时需要等待这些 AMD PDU 或其分段以便进行重排序。

Receiving window：接收窗口。如果一个 SN 满足 VR(R) ≤ SN < VR(MR)，则该 SN 位于接收窗口内；否则该 SN 位于接收窗口外。

- 接收端认为 SN < VR(R)的所有 AMD PDU 都已成功接收，因此不会重复接收这些 AMD PDU。
- 从还未收到的最小 SN，即 SN = VR(R)的 AMD PDU 算起，接收端只能接收最多 AM_Window_Size 个 AMD PDU，所以接收端不会去接收 SN ≥ VR(MR)的 AMD PDU。

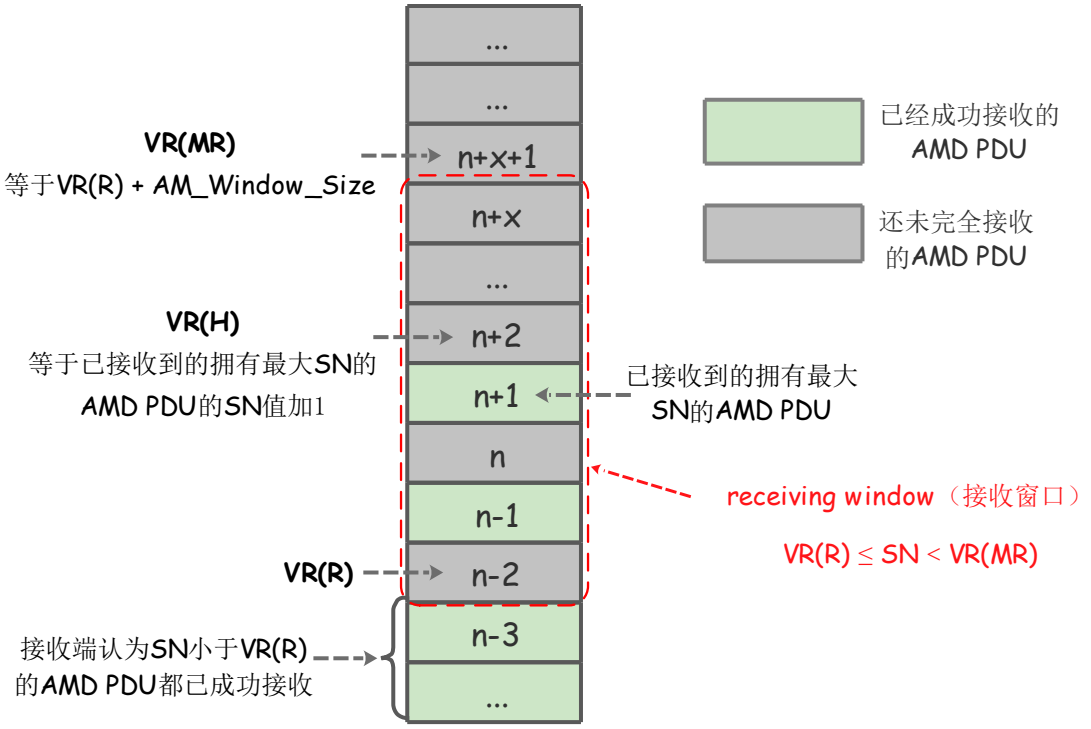


图 26-11：除 VR(X)、VR(MS)外的 AM 接收端相关变量

26.3.2.2 丢弃处理

当 AM 实体从 MAC 层收到一个 RLC data PDU 时，它会判断是丢弃该 PDU 还是将其放入接收 buffer 中。

当接收端收到一个 SN = x 的 RLC data PDU，且该 PDU 包含了 SN = x 的 AMD PDU 的编号从 y 到 z 的 byte 分段时（可能是一个完整的 AMD PDU，也可能是一个 AMD PDU 的部分分段），在以下 2 种情况下该 PDU 会被丢弃：

情况 1：接收端不会去接收位于接收窗口之外的 AMD PDU，所以 x < VR(R)和 x >= VR(MR)的 PDU 都将被丢弃。

情况 2：如果接收端之前已经收到过 SN = x 的 AMD PDU 的编号从 y 到 z 的 byte 分段，即该 PDU 为重复包，会予以丢弃。例如图 26-12 中包含 SN = n+3 的 AMD PDU 的分段 0~30 的 AMD PDU segment，以及 SN = n+5 的 AMD PDU，都将被丢弃。

而在其它情况下，接收端会将该 PDU 放入接收 buffer 中。例如图 26-12 中包含 SN = n+3 的 AMD PDU 的分段 0~80 的 AMD PDU segment，以及 SN = n+2 的 AMD PDU，都将被放入接收 buffer 中。

但如果一个 AMD PDU 的某些 byte 分段在之前已经被接收过，则会丢弃重复的 byte 分段。也就是说，假设接收端在收到 SN = x 的 PDU 的编号从 y 到 z 的 byte 分段之前，已经接收到 SN = x 的 AMD PDU 的编号从 y 到 m 的 byte 分段（其中 y <= m < z），则只有编号从 m+1 到 z 的 byte 分段会被放入到接收 buffer 中。例如图 26-12 中包含 SN = n+3 的 AMD PDU 的分段 0~80 的 AMD PDU segment，由于分段 0~50 已存在于接收 buffer 中，所以只有分段 51~80 会放入接收 buffer 中，其余的分段将被丢弃。

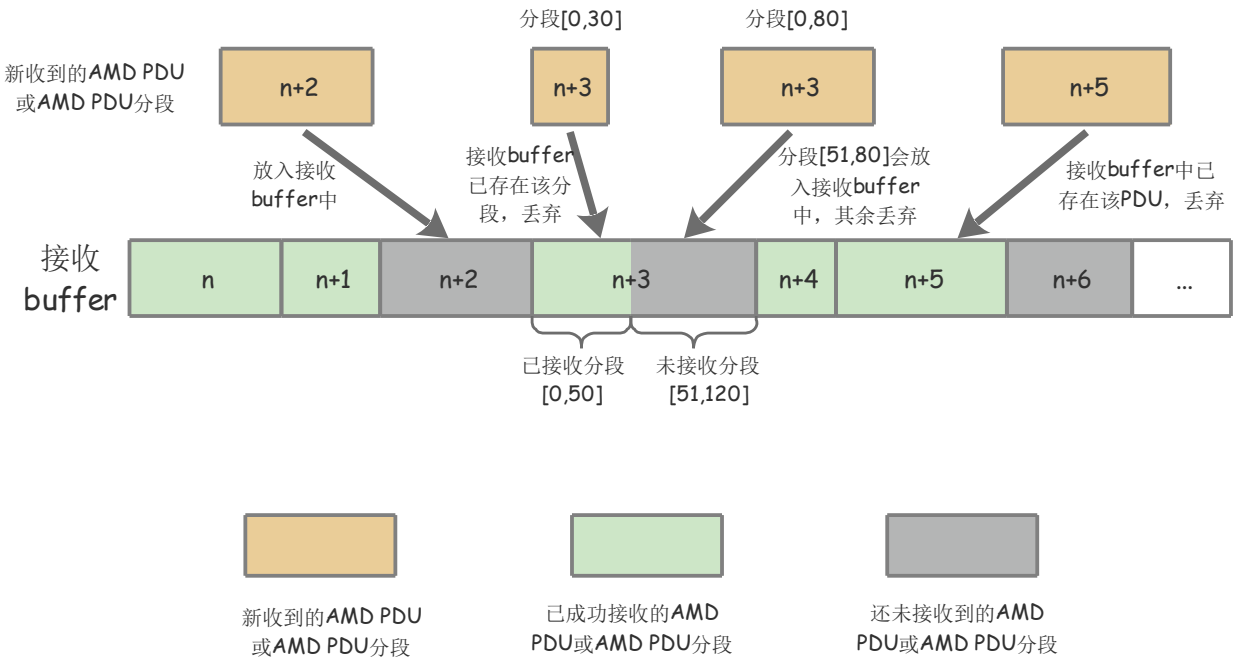


图 26-12：AM 模式的丢弃处理举例

简单地说，接收窗口之外的数据和已经接收过的数据（可能是整个 AMD PDU，也可能是 AMD PDU 的部分 byte 分段）都将被丢弃。同时可以看出，在 AM 模式中，放入接收 buffer 中的 AMD PDU 或 AMD PDU 分段的 SN 必定位于接收窗口之内。

注：byte 分段编号 0 对应 AMD PDU 的 Data 域的第一个字节；byte 分段编号 y 对应 AMD PDU 的 Data 域的第 y+1 个字节。

26.3.2.3 接收 buffer 中的 RLC data PDU 的处理

当一个 SN = x 的 RLC data PDU 放入接收 buffer 时，接收端会按如下步骤进行操作。

步骤 1：如果 x >= VR(H)（说明 SN = x 的 PDU 是已接收到的拥有最大 SN 的 PDU），此时会将 VR(H)设置成 x+1。

步骤 2：如果 SN = VR(MS)的 AMD PDU 的所有 byte 分段都已成功接收，则将 VR(MS)设置成“SN > 当前 VR(MS)”且“并非所有 byte 分段都已成功接收”的第一个 AMD PDU 的 SN 值。

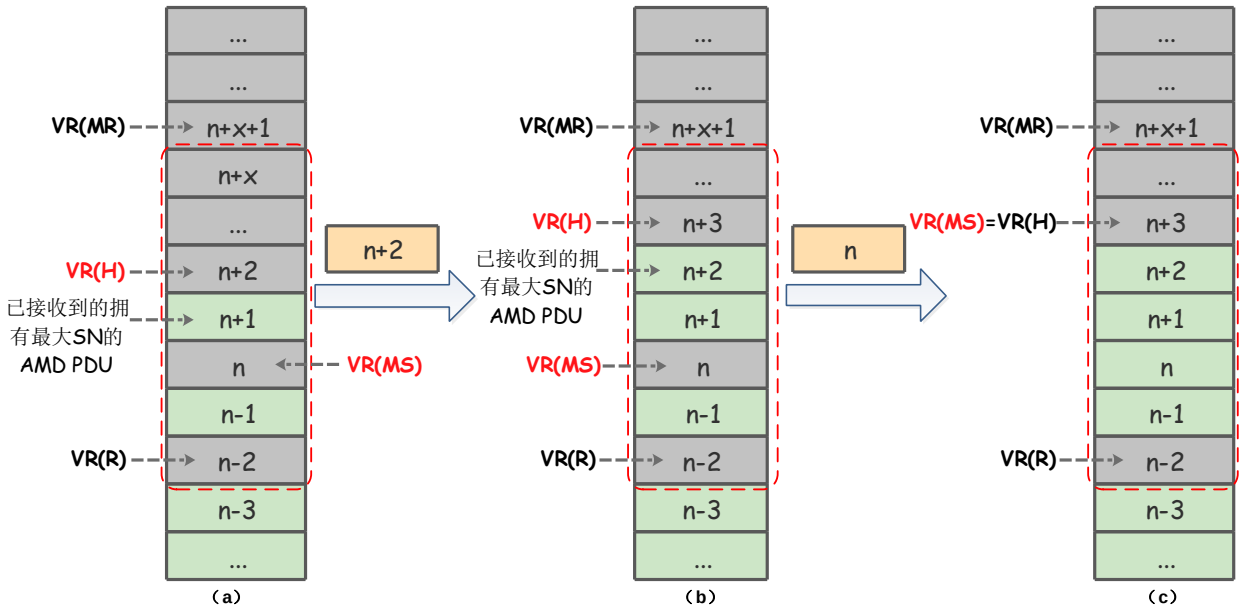


图 26-13：步骤 1 及步骤 2 举例

如图 26-13 所示，接收 buffer 的情况以及相关变量的取值如(a)所示。此时接收端收到一个 SN = n+2 >= VR(H)的 AMD PDU，则将 VR(H)从 n+2 更新为 n+3。此时的变量取值见图 26-13 的(b)。

接下来，接收端收到 SN = n 的 AMD PDU（或分段），导致 SN = VR(MS) = n 的 AMD PDU 被完全接收，此时会将 VR(MS)设置成“SN > 当前 VR(MS)”且“并非所有 byte 分段都已成功接收”的第一个 AMD PDU 的 SN 值，即 n+3。此时变量的取值见图 26-13 的(c)。

步骤 3：如果 x = VR(R)，

- 如果 SN = VR(R)的 AMD PDU 的所有 byte 分段都已成功接收，
  - 将 VR(R)更新成“SN > 当前 VR(R)”且“并非所有 byte 分段都已成功接收”的第一个 AMD PDU 的 SN 值。可以认为“SN < 更新后的 VR(R)”的 AMD PDU 都已成功接收；
  - 将 VR(MR)更新成 VR(R) + AM_Window_Size。
- 对“SN 位于接收窗口之外”的 AMD PDU 和“SN = VR(R)的 AMD PDU 的按序 byte 分段”进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。

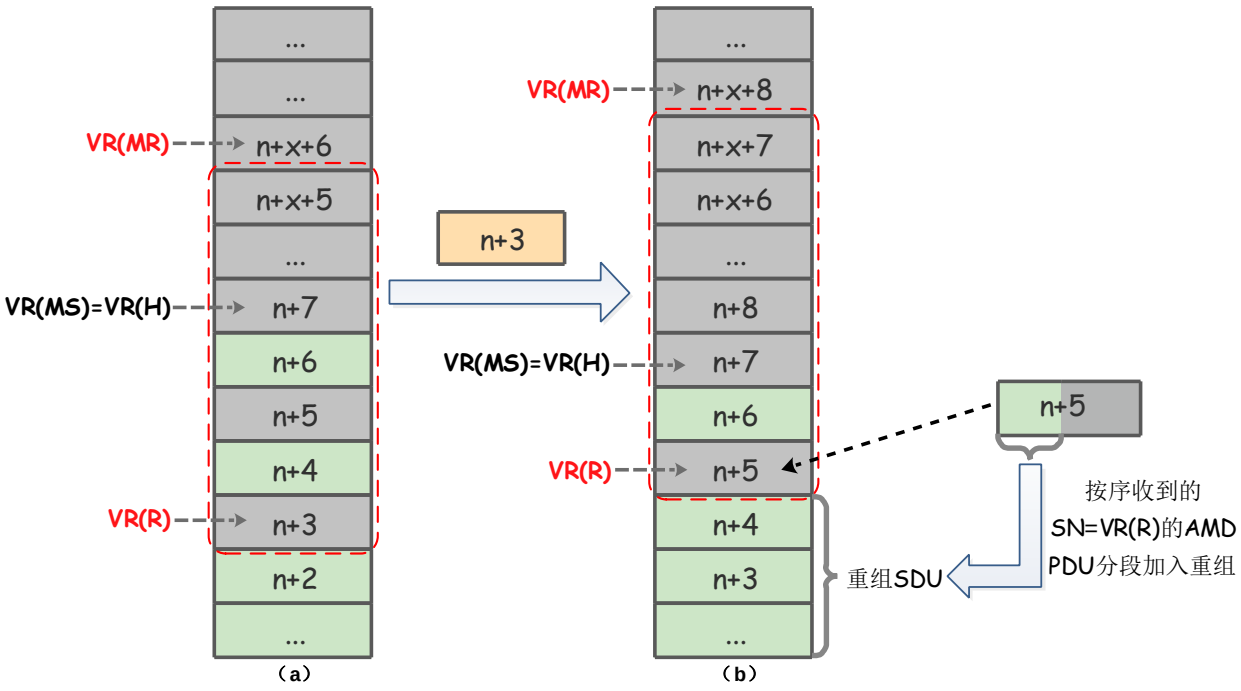


图 26-14：步骤 3 举例

如图 26-14 所示，接收 buffer 的情况以及相关变量的取值如(a)所示。此时接收端收到一个 SN = n+3 = VR(R)的 AMD PDU（或分段），导致 SN = n+3 的 AMD PDU 被完全接收，此时将 VR(R)会被设置成“SN > 当前 VR(R)”且“并非所有 byte 分段都已成功接收”的第一个 AMD PDU 的 SN 值，即 n+5。相应地，VR(MR) 更新成 VR(R) + AM_Window_Size。此时的变量取值见图 26-14 的(b)。

接收 buffer 中，SN = n+5 的 AMD PDU（还未完全接收）可能包含了一些分段，且该分段可能与 SN = n+4 的 AMD PDU 中的部分分段携带了同一 RLC SDU 的数据。所以这里需要将 SN < VR(R)的 AMD PDU 和“SN = VR(R)的 AMD PDU 的按序 byte 分段”进行重组，移除 RLC header，并将重组后的还未曾递送过的 RLC SDU 按 SN 递增的顺序递送给 PDCP 层。

步骤 4：如果 *t-Reordering* 正在运行，

- 如果 VR(X) = VR(R)，或者
- 如果 VR(X)落在了接收窗口之外并且 VR(X)不等于 VR(MR)
    - 停止并重置 *t-Reordering*。

t-Reordering 正在运行，意味着有 SN < VR(X)的 AMD PDU 还未成功接收。新收到 SN = x 的 AMD PDU 后，接收端需要重新判断是否需要继续运行 *t-Reordering*：

- 接收端认为小于 VR(R)的 AMD PDU 都已接收到，这也意味着 VR(X) = VR(R)时，小于 VR(X)的 AMD PDU 都已成功接收到，所以此时 *t-Reordering* 没有运行的必要了。
- 由于 VR(X)不可能大于 VR(MR)，所以当 VR(X)落在接收窗口之外且 VR(X)不等于 VR(MR)时，VR(X)必定落在了接收窗口的下边界之外。而 AM 实体不会去接收“SN < VR(R)”的 AMD PDU，所以此时也没有运行 *t-Reordering* 的必要了。

步骤 5：如果 *t-Reordering* 没有运行（包括步骤 4 介绍的场景下导致的停止运行），并且 VR(H) > VR(R)，则启动 *t-Reordering*，并将 VR(X)设置成 VR(H)。

VR(H) > VR(R)意味着此时至少有一个 SN < VR(H)的 AMD PDU 还未成功接收，所以此时应重启 *t-Reordering*，并在该定时器指定的时间内去接收还未成功收到的 AMD PDU。

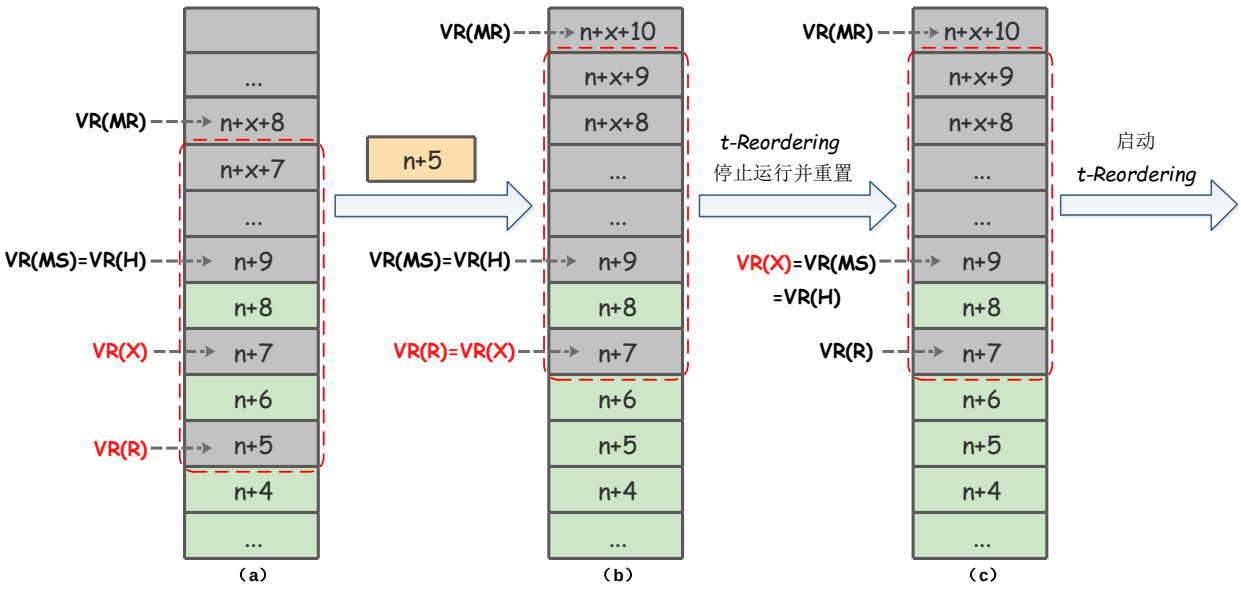


图 26-15：步骤 4 及步骤 5 举例

假设 VR(X)以及其它变量的取值如图 26-15 的(a)所示。*t-Reordering* 正在运行，接收端又收到了一个 SN = n+5 的 AMD PDU，此时 VR(R)从 n+5 更新成 n+7，并导致“VR(X) = VR(R)”。接收端认为小于 VR(X)的 AMD PDU 都已成功接收，因此会停止并重置 *t-Reordering*。此时的变量取值见图 26-15 的(b)。（VR(X)位于接收窗口之外的处理方式与此类似，这里就不再举例了）

步骤 4 处理完之后，*t-Reordering* 没有运行，此时 VR(H) = n+9 > VR(R) = n+7，意味着至少有一个 SN < VR(H)（这里对应 SN = n+7）的 AMD PDU 还没有被完全接收，因此将 VR(X)设置成 VR(H)，并启动 *t-Reordering*。此时的变量取值见图 26-15 的(c)。

UM 模式是基于接收窗口上边界的更新来同步更新下边界的，AM 模式是基于接收窗口下边界的更新来同步更新上边界。也就是说，在 AM 模式中，只有当 SN = VR(R)的 AMD PDU 的所有 byte 分段都已成功接收时，VR(R)才会更新，接收窗口才会向前移动。如果接收端一直没有成功收到 SN = VR(R)的 AMD PDU，则接收端收完 SN = VR(MR)-1 的 AMD PDU 后，就无法继续接收新的 AMD PDU，而只能接收那些位于接收窗口内且还未完全收到的 AMD PDU，也即接收窗口被“阻塞”了。

26.3.2.4 *t*-Reordering 超时处理

如果 *t-Reordering* 超时，则接收端会

- 将 VR(MS)更新成“SN >= VR(X)，但并非所有 byte 分段都已成功接收”的第一个 AMD PDU 的 SN 值。
- 如果此时 VR(H) > VR(MS)，则启动 *t-Reordering*，并将 VR(X)设置成 VR(H)。

t-Reordering 超时，说明“VR(X) > VR(R)”，且“VR(X)在接收窗口之内，或 VR(X)=VR(MR)”。也说明了“至少有一个 SN < VR(X)的 AMD PDU 在该 *t-Reordering* 指定的时间内没有被成功接收到”，此时接收端认为那些还未成功接收的 PDU 已经丢失了，会要求发送端重传这些 PDU。

可以看出，AM 模式中，是通过 *t-Reordering* 超时来判断 SN < VR(X)的某个 AMD PDU（或分段）是否丢失了的。只有确定为已经丢失了的 AMD PDU（或分段）才会要求对端进行重传。

VR(MS)更新后，可以确定 SN < VR(MS)的那些还未成功接收的 AMD PDU 已经丢失，需要进行重传。但对于 VR(MS) <= SN < VR(H)的那些还未成功接收的 AMD PDU（正在等待重排序），同样需要启动 *t-Reordering* 来判断这些 AMD PDU 是否丢失了。

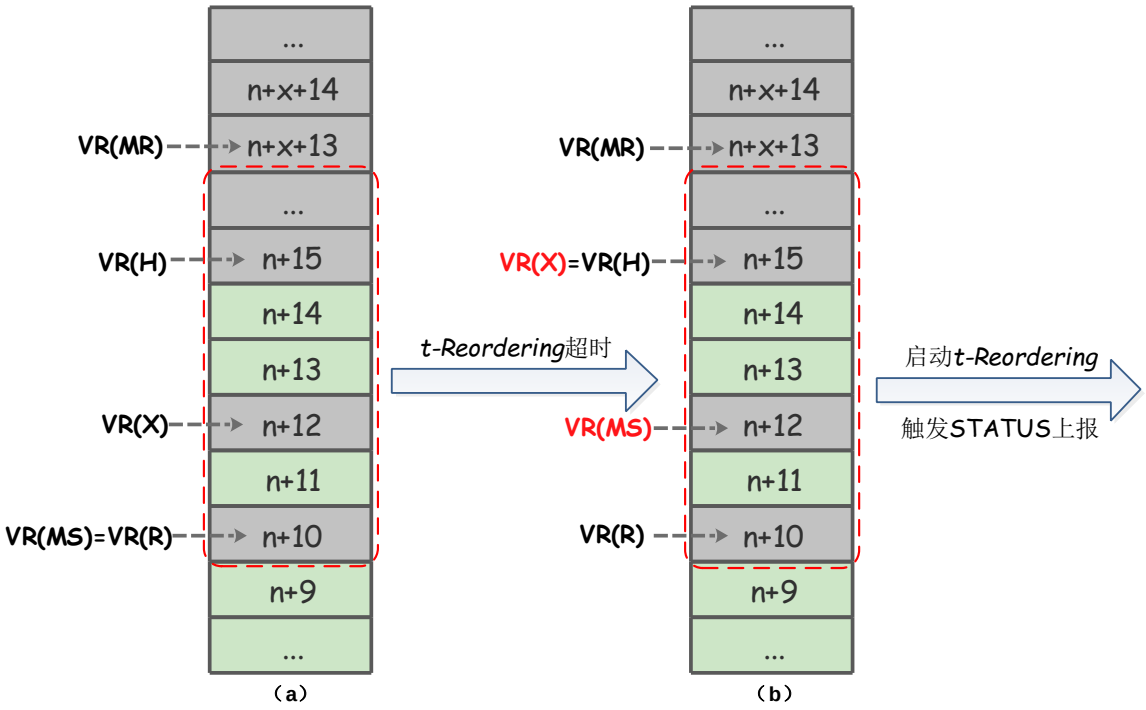


图 26-16：*t-Reordering* 超时处理举例

如图 26-16 的(a)所示，*t-Reordering* 超时的时候，还有 1 个 SN < VR(X)的 AMD PDU（SN = n+10）没有成功接收到。此时接收端认为这个 AMD PDU（或其分段）已经丢失，并将 VR(MS)更新成“SN >= VR(X) = n+12，但并非所有 byte 分段都已成功接收”的第一个 AMD PDU 的 SN 值，这里对应 n+12。由于此时 VR(MS) = n+12 < VR(H) = n+15，即需要等待 SN=n+12 的 AMD PDU 进行重排序，因此要将 VR(X)设置成 VR(H)，并启动 *t-Reordering*。同时会触发 STATUS reporting，要求对端重传丢失的 SN=n+10 的 AMD PDU（或其分段）。此时的变量取值见图 26-16 的(b)。

对比 UM 模式可知，UM 模式在 *t-Reordering* 超时时，就不再去尝试接收已经丢失了的 RLC PDU。而 AM 模式在 *t-Reordering* 超时时，会触发 STATUS reporting，要求发送端重传丢失了的 RLC PDU。

26.3.3 ARQ 流程

通过检测接收到的 RLC data PDU 的 SN，接收端可以知道丢失了哪些 PDU（或其分段），并要求发送端重传丢失的 PDU（或其分段）。

接收端会通过发送 STATUS PDU 告诉接收端，它成功接收了哪些 AMD PDU（肯定确认，positive acknowledgement），以及哪些 AMD PDU 或分段还没有成功接收到（否定确认，negative acknowledgement）。如果一个 AMD PDU 丢失了部分分段，则只有丢失的部分会被要求重传。

ARQ 只适用于 AM 模式。

26.3.3.1 重传

如果发送端收到了某个 AMD PDU 或 AMD PDU 分段的否定确认（说明接收端没能成功接收该 AMD PDU 或分段），并且对应的 AMD PDU 的 SN 落在 VT(A) <= SN < VT(S)的范围内，则认为该 AMD PDU 或 AMD PDU 分段需要重传。

- 发送端认为 SN < VT(A)的所有 AMD PDU 都已被接收端成功接收，并且发送端已经收到这些 AMD PDU 的肯定确认。因此没必要重复发送这些 AMD PDU 或分段，也就不关心这部分 AMD PDU 的确认信息。

- 发送端还未发送 SN >= VT(S)的 AMD PDU，因此无需关心这部分 AMD PDU 的确认信息。

为了避免无限制地重传 AMD PDU，RLC 层定义了一个最大重传次数 *maxRetxThreshold*。当发送端认为某个 AMD PDU 或 AMD PDU 分段需要重传时，它会

- 如果该 AMD PDU 是第一次重传，则将该 AMD PDU 相关联的 RETX_COUNT 设置为 0；
- 如果不是第一次重传，并且该 AMD PDU 或 AMD PDU 分段并非已经在等待重传，则将 RETX_COUNT 加 1；
- 如果 RETX_COUNT = *maxRetxThreshold*，即达到了最大重传次数，则需要告诉上层已经到了最大重传次数。

可以看出，每个需要重传的 AMD PDU 都有一个相关联的 RETX_COUNT。

当 MAC 层指示 AM 实体有一次传输机会，并且该实体要重传一个 AMD PDU 时，

- 如果 MAC 层指定的大小能够完全容纳该 AMD PDU，则直接将该 AMD PDU 发给 MAC 层（但 header 的 P 字段可能发生改变，具体见 26.3.3.2 节的介绍），即此时不做重分段处理；
- 如果 MAC 层指定的大小小于需要重传的 AMD PDU 的大小，则将该 AMD PDU 重分段，生成一个新的 AMD PDU segment 以匹配 MAC 层指定的大小，并将该 AMD PDU segment 发给 MAC 层，即此时要对 AMD PDU 做重分段处理。

当重传的是一个 AMD PDU 的部分分段时，如果 MAC 层指定的大小小于需要重传的 AMD PDU 分段，则会对该 AMD PDU 的分段继续做重分段处理，生成一个新的 AMD PDU segment 以匹配 MAC 层指定的大小，并将新的 AMD PDU segment 发给 MAC 层。也就是说，对一个 AMD PDU 进行重分段的次数是没有限制的。

只有原始 AMD PDU 的 Data 域会映射到新的 AMD PDU segment 的 Data 域。也就是说，重分段时，AMD PDU segment 只会重传原始 AMD PDU 的 Data 域，而不会重传原始 AMD PDU 的 header。并且对 AMD PDU segment 进行再分段处理时，也是基于原始 AMD PDU 的 Data 域进行的。

关于重分段的举例，可参见 26.3.1.2 节和 26.3.1.4 节的介绍。

26.3.3.2 Polling

接收端需要收到 STATUS PDU 后，才有可能更新 VT(A)，发送窗口才有可能向前移动。如果长时间没有收到对端发来的 STATUS PDU，就有可能导致发送窗口阻塞，并最终导致吞吐量的下降。所以在一定条件下，接收端需要主动要求对端发送 STATUS PDU，这就是 polling 的作用。

Polling 是发送端主动请求对端（即接收端）发送一个 STATUS PDU 的过程。以下几种场景下，发送端会主动要求对端发送一个 STATUS PDU：

场景 1：从最近一个发送的 P 字段设置为 1 的 AMD PDU 算起，发送的 AMD PDU 数超过 *pollPDU*，即 PDU_WITHOUT_POLL >= *pollPDU* 时。也就是说，最多间隔 *pollPDU* 个 AMD PDU 后，需要主动发起一次 polling。

场景 2：从最近一个发送的 P 字段设置为 1 的 AMD PDU 算起，发送的数据字节数超过 *pollByte*，即 BYTE_WITHOUT_POLL >= *pollByte* 时。也就是说，最多间隔 pollByte 字节数据后，需要主动发起一次 polling。

场景 3：组装了一个 AMD PDU 或 AMD PDU segment 后，除了那些已传输的且正在等待确认的 RLC data PDU 外，传输 buffer 和重传 buffer 均为空时。即 AM 实体认为数据已经全部发送完时，需要发起一次 polling。

场景 4：组装了一个 AMD PDU 或 AMD PDU segment 后，无法再继续发送一个新的 RLC data PDU 时。例如由于发送窗口阻塞，而导致无法发送新的 RLC data PDU 时，发送端会主动发起一次 polling，以便及时更新 VT(A)的值。

场景 5：定时器 *t-PollRetransmit* 超时。

在发送 polling 请求时，发送端会将发送的 RLC data PDU 的 P 字段设置为 1，并将 PDU_WITHOUT_POLL 和 BYTE_WITHOUT_POLL 设置为 0。

PDU_WITHOUT_POLL 初始为 0，它统计了从最近一次发送的 P 字段设置为 1 的 AMD PDU 算起，发送的 AMD PDU 数。

BYTE_WITHOUT_POLL 初始为 0，它统计了从最近一次发送的 P 字段设置为 1 的 AMD PDU 算起，发送的数据字节数。

每生成一个新的 AMD PDU，PDU_WITHOUT_POLL 会加 1，BYTE_WITHOUT_POLL 会加上每一个 Data field element 的新传字节数。

为了判断对端是否收到了 polling 请求，AM 实体定义了一个定时器 *t-PollRetransmit*。如果 *t-PollRetransmit* 超时，说明对端没有收到该 polling 请求，此时发送端会在接下来的一个传输机会里，重发 polling 请求。

将一个包含了 poll 的 RLC data PDU 发送给 MAC 层，并且在必要时增加了 VT(S)后，发送端会将 POLL_SN 设置成 VT(S) -1，并启动或重启 *t-PollRetransmit*。接收端会期待在 *t-PollRetransmit* 超时之前，收到 SN = POLL_SN 的 AMD PDU 的肯定或否定确认。

当发送端收到一个 STATUS 报告，且该报告包含了 SN = POLL_SN 的 RLC data PDU 的肯定或否定确认时（说明接收端已经收到了对应的 polling 请求），如果 *t-PollRetransmit* 正在运行，则发送端会停止并重置 *t-PollRetransmit*。

t-PollRetransmit 超时，说明对应 POLL_SN 的 polling 请求并没有被对端接收到，发送端需要重新发起 polling 请求。此时

- 如果除了那些已传输的且正在等待确认的 RLC data PDU 外，传输 buffer 和重传 buffer 均为空，即 AM 实体认为数据已经全部发送完了。或
- 如果无法再发送一个新的 RLC data PDU，例如由于发送窗口阻塞，而导致无法发送新的 RLC data PDU。
    - 则认为 SN = VT(S) -1 的 AMD PDU 和那些还未收到肯定确认的 AMD PDU 需要重传。

最后，发送端会将 RLC data PDU 的 P 字段设置为 1，重新进行 polling 处理。

26.3.3.3 STATUS PDU

STATUS PDU 由接收端发往发送端，以告诉发送端所接收到或丢失了的 RLC data PDU 信息。

我们先来介绍一下 STATUS PDU 的结构。

STATUS PDU 由一个 STATUS PDU payload 和一个 STATUS control PDU header 组成。

STATUS control PDU header 由一个 D/C 域和一个 CPT 域组成。

前面已经介绍过，D/C 是 Data/Control 的意思，该字段的值为 0 时，表示对应的 RLC PDU 是 RLC control PDU，即 STATUS PDU；值为 1 时，表示对应的 RLC PDU 是 RLC data PDU。

CPT 是 Control PDU Type 的意思，指示了 RLC control PDU 的类型。当前协议只定义了一种类型：STATUS PDU，此时 CPT 取值为“000”（二进制数）。CPT 的其它取值是预留的，并不使用。

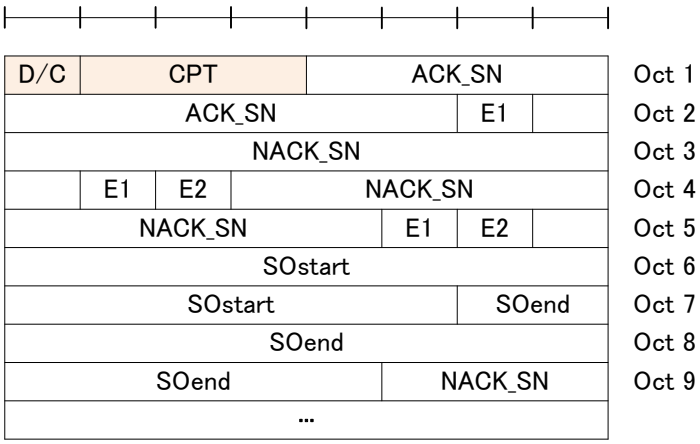


Figure 6.2.1.6-1: STATUS PDU

STATUS PDU payload 由一个“ACK_SN + E1”、0 个或多个“NACK_SN + E1 + E2”组合及其可能存在的 SOstart 和 SOend 组成。如果 STATUS PDU 不是字节对齐的，则需要在最后添加 1 至 7 比特的 padding 以实现字节对齐。

ACK_SN 对应没有在 STATUS PDU 中上报为丢失的，下一个还未接收到的 RLC Data PDU 的 SN 值。当发送端收到一个 STATUS PDU 时，除了那些由 NACK_SN 指示的 AMD PDU 和由 NACK_SN + SOstart + SOend 指示的 AMD PDU 分段，发送端认为 SN < ACK_SN 的 AMD PDU 都已被对端成功接收。

NACK_SN 对应那些被接收端认为丢失了的 AMD PDU 或 AMD PDU 分段的 SN 值。

NACK_SN 可能对应一个丢失了的 AMD PDU，也可能对应一个丢失了的 AMD PDU 分段。如果丢失的是一个 AMD PDU，则 STATUS PDU 中会有一个对应的“NACK_SN + E1 + E2”组合；如果丢失的是一个 AMD PDU 分段，则 STATUS PDU 中会有一个对应的“NACK_SN + E1 + E2 + SOstart + SOend”组合。

SOstart 和 SOend 一起指示了 SN = NACK_SN 的 AMD PDU 的哪一部分丢失了。其中 SOstart 对应丢失分段的第一个字节在原始 AMD PDU 的 Data 域中的位置，SOend 对应丢失分段的最后一个字节在原始 AMD PDU 的 Data 域中的位置。原始 AMD PDU 的 Data 域的第一个字节对应的值为“000000000000000”（二进制数），即从 0 开始计数。SOend 为“111111111111111”（二进制数）时，说明丢失了的 AMD PDU 分段包含了从 SOstart 开始，到原始 AMD PDU 的最后一个字节在内的所有字节。

E1 用于指示“ACK_SN + E1”、“NACK_SN + E1 + E2”或“NACK_SN + E1 + E2 + SOstart + SOend”组合之后是否紧跟着一个“NACK_SN + E1 + E2”组合。值为 0 时对应“否”，值为 1 时对应“是”。

E2 用于指示“NACK_SN + E1 + E2”组合之后是否紧跟着一个“SOstart + SOend”组合。值为 0 时对应“否”，值为 1 时对应“是”。

关于 STATUS PDU 的举例，可参见下一节的介绍。

26.3.3.4 Status reporting

有 2 种场景会触发 STATUS reporting：

场景 1：对端（发送端）发起了 Polling，即收到的 RLC data PDU（假设其 SN = x）的 P 字段设置成 1。如果该 PDU 是要被丢弃的，或“x < VR(MS)或 x >= VR(MR)”，则触发 STATUS report；否则直到“x < VR(MS)或 x >= VR(MR)”，才触发 STATUS report，这是为了保证 Status report 在 HARQ 重排序之后传输。

场景 2：检测到一个 RLC data PDU 接收失败，即定时器 *t-Reordering* 超时时，接收端会触发 STATUS report。

注意：当定时器 *t-Reordering* 超时时，会同时触发 VR(MS)的更新和 STATUS report，此时会先更新 VR(MS)，然后触发 STATUS report。

这 2 种场景下只是触发 STATUS reporting，但并不一定会立即上报。上报 STATUS PDU 还需要满足 2 个条件：（1）定时器 *t-StatusProhibit* 不在运行中，或已超时；（2）MAC 层指示了一个发送机会，并指定了 RLC PDU 的大小。

如果一个 STATUS report 发送时，接收端还未收到前一个 STATUS report 触发的重传，则可能导致同一个 AMD PDU 或 AMD PDU 分段被要求重传 2 次，从而导致了不必要的重传，并浪费了无

线资源。在 AM 模式下，这也是可能导致收到重复 PDU 的第二个原因。为了避免这种情况的发生，AM 模式中定义了一个 status prohibit 的功能，即禁止在定时器 *t-StatusProhibit* 运行期间发送 STATUS report。*t-StatusProhibit* 通常设置成稍大于 RLC round trip time 的值。可以看出，*t-StatusProhibit* 同时避免了接收端过多地发送 STATUS report。

当 STATUS reporting 已经被触发时，如果 *t-StatusProhibit* 没有运行，则在 MAC 层指示的第一个传输机会里，就构建 STATUS PDU，并发给 MAC 层；否则在 *t-StatusProhibit* 超时后的 MAC 层指示的第一个传输机会里，构建一个唯一的 STATUS PDU，并发给 MAC 层。即使在 *t-StatusProhibit* 运行期间，STATUS reporting 被触发了多次，此时也只会构建一个 STATUS PDU。

STATUS PDU 发给 MAC 层后，接收端会重新启动 *t-StatusProhibit*。

对于那些 VR(R) <= SN < VR(MS)且还未完全接收到的 AMD PDU，会按照 SN 递增，以及 PDU 内的 byte 分段递增的顺序，从 SN = VR(R)开始，设置 NACK_SN、SOstart 和 SOend，直到构建的 STATUS PDU 能够匹配 MAC 层指定的大小：

- 对于一个 byte 分段都没被接收到的 AMD PDU，将 STATUS PDU 中的对应的 NACK_SN 设置成该 AMD PDU 的 SN；
- 对于只接收到部分 byte 分段的 AMD PDU 中的那些还未被接收到的连续 byte 分段，会在 STATUS PDU 中添加一个对应的 NACK_SN，SOstart 和 SOend。

将 ACK_SN 设置成“下一个还未接收到的”且“并未在 STATUS PDU 中指示为丢失了的”RLC Data PDU 的 SN。

这里没有将 ACK_SN 直接设置成 VR(MS)的原因在于，当 MAC 层指定的大小不足以容纳所有 SN < VR(MS)且丢失了的 RLC data PDU 的 NACK 信息时，ACK_SN 就会被设置成小于 VR(MS)的值。

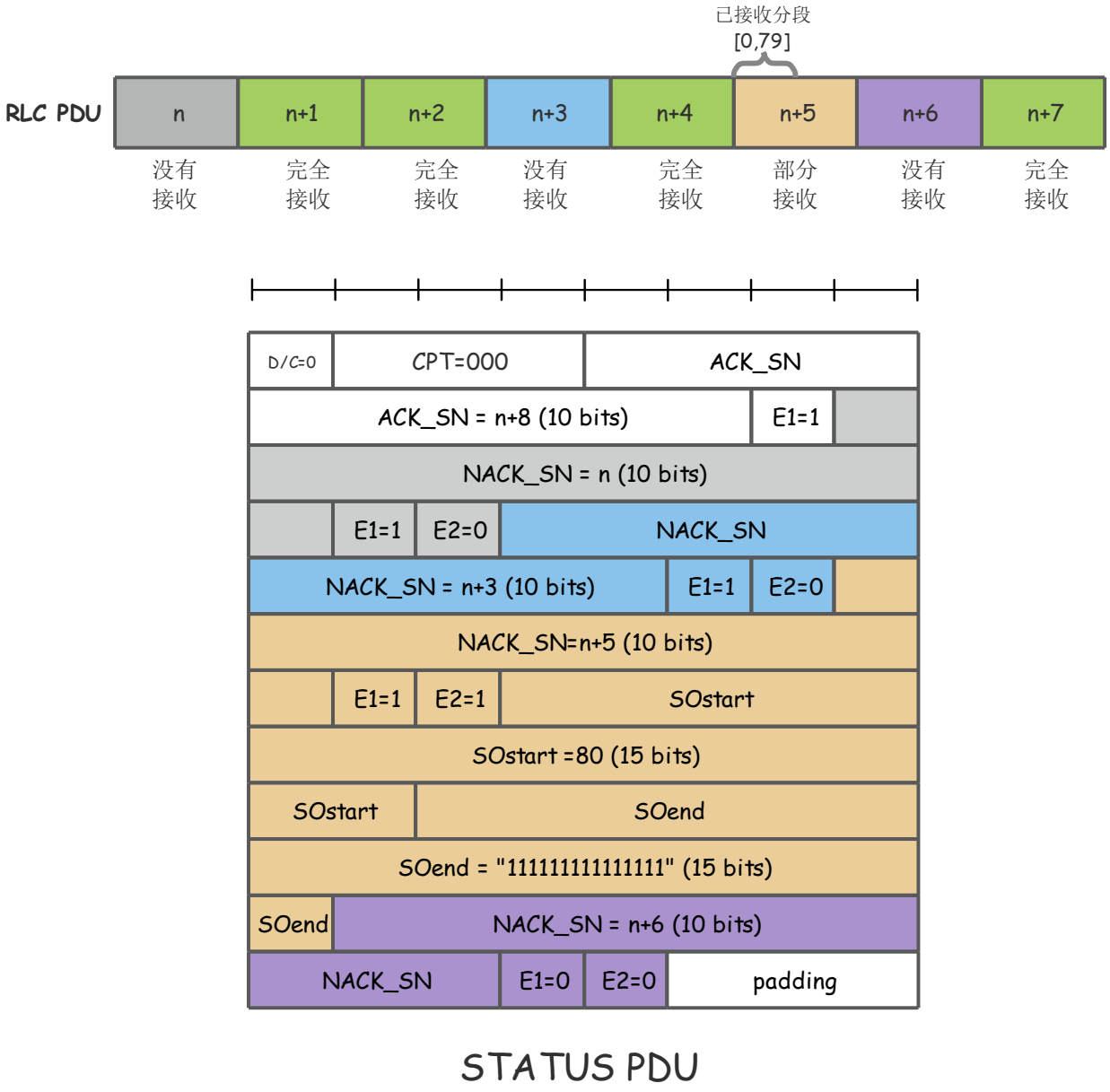


图 26-17：STATUS PDU 举例

图 26-17 是根据接收到的 RLC data PDU 的信息，生成对应的 STATUS PDU（大小为 12 字节）的一个例子。在该例子中，除了 NACK_SN = n、n+3 和 n+6 的 AMD PDU 完全丢失了，NACK_SN = n+5 的 AMD PDU 丢失了 80 到最后一个字节之间的分段外，接收端已经成功收到了 SN=n+8 之前（不包含 n+8）的所有 AMD PDU。此时 ACK_SN 设置为 n+8，说明除了 NACK_SN 指示的丢失了的 PDU（或分段）外，小于 n+8 的 AMD PDU 都已成功地接收了。

由于是 STATUS PDU，所以 D/C 字段设置为 0，CPT 设置为“000”（二进制数）。“ACK_SN + E1”的组合中，ACK_SN 设置为 n+8，E1 设置为 1，因为其后紧跟着 NACK_SN = n 的“NACK_SN + E1 + E2”组合。

NACK_SN = n 的“NACK_SN + E1 + E2”组合中的 E2 设置为 0，这是因为 SN = n 的整个 AMD PDU 都丢失了，而不是丢了某个分段，其后并不跟着“SOstart + SOend”。该组合的 E1 设置为 1，因为其后跟着 NACK_SN = n+3 的“NACK_SN + E1 + E2”组合。同样的理由，NACK_SN = n+3 的“NACK_SN + E1 + E2”组合中的 E1 设置为 1，E2 设置为 0。

紧接着的是对应 NACK_SN = n+5 的 AMD PDU 分段的“NACK_SN + E1 + E2 + SOstart + SOend”组合，其 E2 设置为 1，因为“NACK_SN + E1 + E2 ”之后跟着一个“SOstart + SOend”。这里 SOstart 的值为 80，SOend 的值为“11111111111111”（二进制数），表示丢失了原始 AMD PDU 的 Data 域中编号为 80 的字节到最后一个字节之间的分段。其 E1 设置为 1，因为其后跟着 NACK_SN = n+6 的“NACK_SN + E1 + E2”组合。

NACK_SN = n+6 的“NACK_SN + E1 + E2”组合之后既不跟着另一个“NACK_SN + E1 + E2”，也不跟着“SOstart + SOend”，所以其 E1 和 E2 均设置为 0。

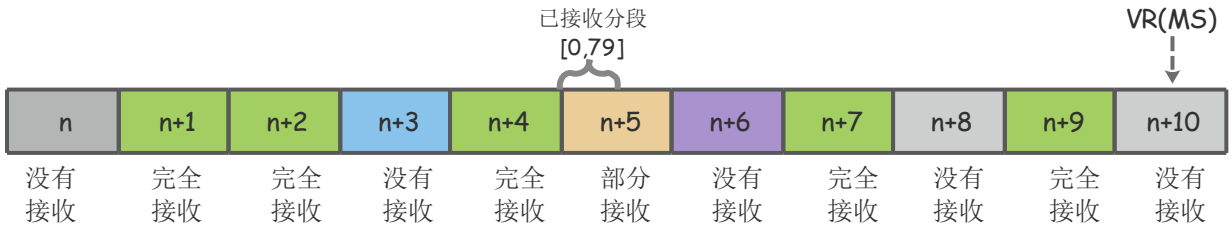


图 26-18：MAC 层指示的大小不足以让 STATUS PDU 容纳所有 NACK 信息举例

假设接收到的 RLC data PDU 情况如图 26-18 所示，且 VR(MS) = n+10。如果 MAC 层指定的大小正好为 12 字节，则生成的 STATUS PDU 只能容纳 SN < n+8 的丢失了的 AMD PDU（或分段）的信息，而不足以继续容纳 NACK_SN = n+8 的信息。此时生成的 STATUS PDU 与图 26-17 完全相同，ACK_SN = n+8，并不等于 VR(MS)。

【参考资料】

[1] TS 36.322

[2] 《4G LTE/LTE-Advanced for Mobile Broadband》的 12.2 节

[3] 《LTE - The UMTS Long Term Evolution, 2nd Edition》的 4.3 节

[4] http://www.sharetechnote.com/html/RLC_LTE.html

[5] http://www.masterltefaster.com/media/36322_860_RLC/36322_860_RLC.swf